

第三版前言

本书是为中国科学技术大学工科电子类本科生学习《微型计算机原理与系统》课程编写的教材。自1996年出版以来,被很多高校选为教材,受到了广大读者的欢迎,并提出了不少宝贵的意见和建议。为此,我们对原书进行了修订。

自20世纪70年代初第一代微型计算机问世以来,计算机技术以惊人的速度发展,尤其是在以Intel 8086/8088为CPU的16位IBM PC机诞生以后,又相继出现了以80386、80486为CPU的32位PC机。如今,以Pentium系列为CPU的高性能微型计算机已大量面市。但作为一类在世界上最流行的机种的代表,16位机的结构、组成原理、指令系统,编程方法和接口技术等,在后续的高档PC机设计中基本上都得到了体现,并具有向上兼容性。本书仍以8086/8088 CPU为基本出发点,详尽地论述有关微处理器及其指令系统的概念和程序设计方法,介绍构成微型计算机的存储器、各类可编程接口芯片、总线等各项技术。最后,对32位微型计算机的基本工作原理作了概要介绍。

全书共分13章,在内容安排上注重系统性、先进性和实用性,各章前后呼应,并加入了大量的程序和硬件设计实例,着眼于使读者能深入了解计算机的原理、结构和特点,以及如何运用这些知识来设计一个实用的微型计算机系统。第一章叙述微型计算机的发展、构成和数的表示方法,第二章阐述8086微型计算机系统的组成原理和体系结构,第三章对8086的指令系统作了详尽说明,第四章讨论8086汇编语言程序设计方法,并给出了许多程序设计的例子,第五章介绍存储器的分类及使用,第六章简述I/O接口和系统总线,第七章论述中断系统并介绍中断控制器8259A,第八章到第十二章详细介绍了I/O接口芯片的基本原理和它们的大量应用实例,包括定时器/计数器8253及8254、通用并行接口8255A、串行接口8251A、数/模和模/数转换器以及DMA控制器8237A等,并概述了IBM PC/XT机系统板的主要电路和工作原理。第十三章,概要性地介绍了32位微型计算机的基本工作原理,包括32位微处理器的结构和工作模式,寄存器组成,保护模式下的内存管理,32位机新增指令、编程实例及接口技术。

在本书的编写过程中,编者参考了国内外大量的文献资料,吸取各家之长,并结合多年来从事微型计算机课程教学和计算机应用研究方面的实际经验,对全书内容作了精心组织编排,文字上力求做到深入浅出、重点突出、通俗易懂,

并用大量图表和例子来帮助读者加深印象。每章所附的思考题与练习,将有助于读者巩固所学的知识。

本书第一、二、四、五、七章由吴秀清编写,第三、六、八、九、十、十一、十二、十三章由周荷琴编写。中国科学技术大学电子科技系的冯焕清教授对大部分书稿进行了审校,研究生潘剑峰、刘冰啸、刘勃、薛铮、刘莉、袁勋等在插图的绘制、例题的验证等方面做了许多工作,并对书中的内容提出了不少有益的建议。在此表示衷心的感谢。

编者还要感谢书末所列参考文献的所有国内外作者。

由于作者水平有限,错误和不当之处,敬请读者批评指正。

编 者
2004 年 12 月于合肥

目 录

第一章 绪论	1
1-1 微型计算机的发展概况	1
1-2 微型计算机系统	5
一、微型计算机 5	二、存储器 7
三、I/O 接口 10	四、总线 10
五、微型计算机的性能指标 13	
1-3 计算机数据格式	14
一、数制 14	二、计算机数据格式 16
习题	20
第二章 8086 系统结构	21
2-1 8086 CPU 结构	21
一、8086 CPU 的内部结构 22	二、寄存器结构 24
2-2 8086 CPU 的引脚及其功能	28
一、8086/8088 CPU 在最小模式中引脚定义 28	
二、8086/8088 CPU 在最大模式中引脚定义 32	
三、8088 与 8086 CPU 的不同之处 33	
2-3 8086 存储器组织	34
一、存储器地址的分段 34	二、8086 存储器的分体结构 36
三、堆栈的概念 38	
2-4 8086 系统配置	40
一、最小模式系统 41	二、最大模式系统 45
2-5 8086 CPU 时序	49
一、系统的复位和启动 49	二、最小模式下的总线操作 50
三、最大模式下的总线操作 53	四、最小模式下的总线保持 54
习题	55
第三章 8086 的寻址方式和指令系统	57
3-1 8086 的寻址方式	57
一、立即寻址方式 57	二、寄存器寻址方式 58
三、直接寻址方式 58	四、寄存器间接寻址方式 60
五、寄存器相对寻址方式 61	六、基址变址寻址方式 62
七、相对基址变址寻址方式 62	八、其它 64
3-2 指令的机器码表示方法	65
一、机器语言指令的编码目的和特点 65	二、机器语言指令代码的编制 66

3-3 8086 的指令系统	70
一、数据传送指令 70	
二、算术运算指令 78	
三、逻辑运算和移位指令 91	
四、字符串处理指令 96	
五、控制转移指令 100	
六、处理器控制指令 117	
七、指令的执行时间和软件延时 119	
习题	121
第四章 汇编语言程序设计	125
4-1 汇编语言程序格式	126
一、指令性语句 127	
二、伪指令语句 127	
三、数据项 127	
4-2 MASM 中的表达式	128
一、算术运算符 129	
二、逻辑运算符 130	
三、关系运算符 131	
四、数值返回运算符 131	
五、修改属性运算符 133	
六、其它运算符 135	
七、优先级 136	
4-3 伪指令语句	136
一、数据定义语句 137	
二、表达式赋值语句 139	
三、段定义语句 140	
四、过程定义语句 143	
五、程序开始和结束语句 145	
六、结构定义语句 147	
七、外部伪指令及对准伪指令 151	
八、高档微机增加的伪指令 153	
4-4 DOS 系统功能调用和 BIOS 中断调用	156
一、常用的软件中断 157	
二、DOS 系统功能调用 158	
三、BIOS 中断调用 165	
4-5 程序设计方法	170
一、顺序结构 170	
二、分支结构 171	
三、循环程序结构 176	
四、子程序结构 181	
五、综合举例 189	
4-6 宏汇编和条件汇编	197
一、宏汇编 197	
二、条件汇编 203	
习题	205
第五章 存储器	207
5-1 存储器分类	207
一、按用途分类 207	
二、按存储器性质分类 208	
5-2 随机存取存储器 RAM	209
一、静态随机存取存储器(SRAM) 209	
二、动态随机存取存储器(DRAM) 211	
三、存储器的工作时序 214	
四、高速缓冲存储器 216	
5-3 只读存储器	221
一、掩膜型 ROM 221	
二、可编程 ROM(PROM) 222	
三、可编程可擦除 ROM(EPROM) 222	
四、电可擦除可编程 ROM(EEPROM) 224	

5-4 CPU 与存储器的连接	225
一、存储器的地址选择 226	
二、存储器的数据线及控制线的连接 228	
5-5 存储器空间的分配和使用	230
一、IBM PC/XT 机中存储器空间分配 232	
二、IBM PC/AT 机中存储器空间分配 232	
三、PC 机中存储器的使用 233	
习题.....	237
第六章 I/O 接口和总线	239
6-1 I/O 接口	239
一、I/O 接口的功能 239	
二、简单的输入输出接口芯片 241	
三、I/O 端口及其寻址方式 244	
四、CPU 与外设间的数据传送方式 246	
五、PC 机的 I/O 地址分配 252	
6-2 总线	255
一、总线的概念 255	
二、IBM PC 总线 256	
三、AT 总线或 ISA 总线 258	
习题.....	262
第七章 微型计算机中断系统.....	263
7-1 概述	263
一、中断概念 263	
二、中断分类 264	
7-2 中断处理过程	266
一、CPU 响应中断过程 266	
二、中断向量表 267	
三、中断服务子程序 273	
四、中断响应时序 276	
7-3 中断优先级和中断嵌套	277
一、中断优先级 277	
二、中断嵌套 279	
7-4 可编程中断控制器 8259A	281
一、功能和引脚 281	
二、内部结构 282	
三、8259A 的中断管理方式 284	
四、8259A 的编程方法 289	
五、8259A 的中断级联 296	
习题.....	302
第八章 可编程计数器/定时器 8253 及其应用.....	304
8-1 8253 的工作原理	305
一、8253 的内部结构和引脚信号 305	
二、初始化编程步骤和门控信号的功能 309	
三、8253 的工作方式 310	
8-2 8253 的应用举例	315
一、8253 定时功能的应用例子 315	
二、8253 计数功能的应用例子 318	
三、8253 在 PC/XT 机中的应用 321	
习题.....	324
第九章 可编程外围接口芯片 8255A 及其应用	325
9-1 8255A 的工作原理	325

一、8255A 的结构和功能	325	二、8255A 的控制字	327
三、8255A 的工作方式和 C 口状态字	329		
9-2 8255A 的应用举例	337		
一、基本输入输出应用举例	337	二、键盘接口	339
三、8255A 在 PC/XT 机中的应用	342	四、PC/XT 机中的扬声器接口电路	346
五、并行打印机接口	348		
习题	354		
第十章 串行通信和可编程接口芯片 8251A	356		
10-1 串行通信的基本概念	356		
一、数据传送的方向	356		
二、串行传送的两种基本工作方式	357		
三、串行传送速率	358		
四、串行接口芯片 UART 和 USART	358		
五、调制解调器	360		
10-2 可编程串行通信接口芯片 8251A	361		
一、8251A 的内部结构和外部引脚	361	二、8251A 的编程	368
三、8251A 初始化编程举例	372		
10-3 EIA RS-232C 串行口和 8251A 应用举例	373		
一、EIA RS-232C 串行口	373	二、8251A 应用举例	375
10-4 串行同步数据通信协议	378		
一、二进制同步通信协议 BISYNC	378	二、高级数据链路控制协议 HDLC	380
习题	381		
第十一章 模数(A/D)和数模(D/A)转换	383		
11-1 概述	383		
一、一个实时控制系统	383	二、多路模拟开关	384
三、采样、量化和编码	386	四、采样保持器	389
11-2 D/A 转换器	391		
一、数/模转换器原理	391	二、数/模转换器的主要性能指标	393
三、几种数/模转换器	394		
11-3 A/D 转换	401		
一、模/数转换器原理	401	二、典型的模/数转换器	404
习题	414		
第十二章 8237A DMA 控制器及 PC/XT 机的系统板	416		
12-1 8237A 的组成和工作原理	416		
一、8237A 的内部结构	416	二、8237A 的引脚功能	418
三、8237A 的内部寄存器	420		
12-2 8237A 的时序	427		
一、外设和内存间的 DMA 数据传送时序	427		
二、空闲周期、有效周期和扩展写周期	428		
12-3 8237A 的编程和应用举例	430		
一、PC/XT 机中的 DMA 控制逻辑	430	二、8237A 的一般编程方法	432

三、PC/XT 机上的 DMA 控制器的使用	433
12-4 PC/XT 机的系统板	435
一、CPU 子系统	435
二、接口部件子系统	437
三、存储器子系统	438
习题	440
第十三章 32 位微机基本工作原理概述	442
13-1 32 位微处理器的结构与工作模式	443
一、32 位微处理器结构简介	443
二、32 位微处理器的工作模式	446
13-2 寄存器	449
一、用户级寄存器	450
二、系统级寄存器	453
三、程序调试寄存器	459
13-3 保护模式下的内存管理	460
一、段内存管理技术	461
二、分页内存管理技术	471
13-4 32 位机新增指令和程序设计简介	475
一、80386 的寻址方式	475
二、80386 的指令系统	477
三、程序设计实例	483
13-5 32 位机接口技术	490
一、主板的组成	490
二、Pentium II 主板	491
三、集成型主板	495
附录 A 8086/8088 指令系统一览表	499
附录 B 通用汇编程序伪指令	506
附录 C ASCII 码编码表	508
附录 D 中断向量地址表	509
附录 E 汇编程序的开发过程	510
一、源程序的编辑	510
二、源程序的汇编	510
三、链接	511
四、汇编和链接依次自动连续进行	511
五、运行	512
六、调试	512
附录 F DOS 功能调用	516
参考文献	521

第一章 绪 论

1-1 微型计算机的发展概况

自从1946年第一代电子计算机ENIAC(Electronic Numerical Integrator and Calculator)在美国研制成功以后,计算机的发展已经历了从电子管计算机、晶体管计算机、集成电路计算机、大规模集成电路计算机几代了。

第一代电子计算机使用了18800个电子管,重30吨,占地150平方米,耗电150千瓦,每秒完成5000次加法运算。第二代晶体管计算机在1958年推出,用晶体管代替了电子管,大大降低了计算机的成本和体积,运算速度成百倍的提高。1965年以中小规模集成电路为主体的计算机问世,使计算机的体积进一步缩小,配上各类操作系统,计算机性能极大地提高。1970年大规模集成电路(LSI)研制成功,计算机也发展到第四代,微型计算机正是第四代计算机的典型代表。1971年,在美国硅谷第一台微型计算机诞生了,从而开创了微型计算机的新时代。

微型计算机与大型机、中型机、小型机在系统结构和工作原理上相比没有本质的区别,但由于它采用了LSI器件,集成度高,使它具有独特的优点:体积小,重量轻,可靠性高,结构配置灵活,价格低廉,从而发展迅猛。1965年,摩尔(G. Moore)经统计发现,集成电路内芯片的晶体管数目,每隔18~24个月,其集成度就要翻一番,称为摩尔定律。微型计算机自1971年问世以来,几乎每隔二、三年就推出一代新的微处理器。下面看一下微处理器的发展情况:

1. 第一代微处理器(1971年开始)

4位和8位微处理器,典型产品为:

1971年 Intel 4004

1972年 Intel 8008

Intel 4004芯片采用PMOS工艺,集成度为2300只晶体管,时钟频率小于1MHz,平均指令执行时间为 $10\mu\text{s}\sim 15\mu\text{s}$,采用机器语言编程。这种微处理器为内嵌式处理器,又称灵巧型处理器(Smart),主要用在控制设备中,如现金计数器,交通灯控制等。

2. 第二代微处理器(1974年开始)

8位微处理器,典型产品为:

1974年 Intel 8080

1974年 Motorola MC6800

1975年 Zilog Z80;

1976年 Intel 8085

8080微处理器采用NMOS工艺,集成度达4500只晶体管,时钟频率2MHz,平均指令执行时间 $1\sim 2\mu\text{s}$,寻址64KB内存空间。用它构成的微型计算机在结构上已具有计算机的体系结构,有中断和DMA等功能,指令系统较为完善,软件上也配备了汇编语言、BASIC和FORTRAN语言,使用单用户操作系统。

3. 第三代微处理器(1978年开始)

16位微处理器,典型产品为:

1978 年 Intel 8086

1979 年 Zilog Z8000

1979 年 Motorola 68000

1982 年 Intel 80286, Motorola 68010

8086 微处理器采用 HMOS 工艺,集成度达 29000 万只晶体管,时钟频率有 5MHz、8MHz、10MHz,平均指令执行时间 $0.5\mu\text{s}$ 。具有丰富的指令系统,采用多级中断,多重寻址方式,有段寄存器结构,配有磁盘操作系统,数据库管理系统和多种高级语言,性能超过了 20 世纪 70 年代的中低档小型机水平。值得一提的是 8086 被用作 IBM PC 的 CPU,时钟频率为 5MHz,数据总线 16 位,地址总线 20 位,可寻址 1MB 内存空间,投放市场后迅速占领了市场,促进了个人计算机的应用与推广。

由于 8086 缺乏存储器管理,1982 年,Intel 公司启用 80286 CPU,研制了 IBM PC/AT 机。80286 与 8086 向上兼容,有 24 位地址总线,寻址能力达 16MB,时钟频率可达 25MHz。80286 在以下方面有显著的改进:80286 提出了实模式和保护模式两种存储器管理模式,使之突破了 8086 访问 1MB 存储空间的限制;引进了段描述符表的概念,可访问 1GB 的虚拟地址空间;IBM PC/AT 的运算速度比 IBM PC/XT 快 12 倍;支持虚拟存储器体系,满足了多用户和多任务的工作需要。80286 的封装是一种被称为 PGA 正方形包装,集成了 13.4 万只晶体管。

4. 第四代微处理器(1983 年开始)

32 位微处理器,典型产品为:

1983 年 Zilog Z80000

1984 年 Motorola 68020

1985 年 Intel 80386

1989 年 Intel 80486, Motorola 68040

1) 80386

80386 CPU 采用 CHMOS 工艺,集成度高达 27.5 万只晶体管,时钟频率 16MHz~33MHz,平均指令执行时间小于 $0.1\mu\text{s}$ 。数据总线和地址总线均为 32 位,寻址能力高达 4GB,采用段页式存储器管理机制,提供带有存储器保护的虚拟存储,可管理 64TB 的虚拟存储空间。它的运算模式除了具有实模式和保护模式外,还增加了一种“虚拟 86”的工作方式,可以通过同时模拟多个 8086 微处理器来提供多任务能力,运算速度超过 600 万条指令/秒。采用 6 级流水线,即取指令,译码,内存管理,执行指令和总线访问并行操作,有快速局部总线。80386 推出了数值协处理器 80387,加快了浮点操作速度,开发高速缓存解决内存速度瓶颈。80386 有丰富的外围配件支持,如 DMA 控制器 82258,中断控制器 8259A,磁盘控制器 8272,Cache 控制器 82385,硬盘控制器 82062 等,80386 使 32 位 CPU 成为 PC 工业的标准。

2) 80486

1989 年 Intel 公司推出了高性能 32 位微处理器 Intel 80486,集成度达 120 万只晶体管,主频有 25MHz,33MHz,50MHz(DX4 达到 66MHz,75MHz 和 100MHz)。80486 不仅将浮点运算部件集成进芯片之内,又增加了 8KB 的片内高速缓存(Cache),内部数据总线宽度为 64 位。80486 的整数处理部件采用了 RISC 技术(Reduced Instruction Set Computer,简化指令集计算机),可以在一个时钟周期内执行一条指令,使 80486 的处理速度极大提高。芯片内部其它方面保留 CISC(Complex Instruction Set Computer,复杂指令系统),用以处理复杂的指令,以保证兼容性。它还采用了突发总线方式,大大提高了与内存的数据交换速度。由于这些改进,80486 的性能比 80386DX 性能提高了 4 倍。80486 引进了时钟倍频技术,使主频超过 100MHz 成为可能。

5. 第五代微处理器(1993 年开始)

典型产品为:

1993 年 Pentium 586	1995 年 Pentium Pro
1996 年 Pentium MMX	1997 年 Pentium II
1999 年 Pentium III	2000 年 Pentium 4

随着人们对图形图像,实时视频处理,语音识别,CAD/CAM,大规模财务分析,大流量客户机/服务器应用等的需求日益迫切,第五代微处理器应运而生。

1) Pentium

1993 年 3 月,Intel 公司推出了最新微处理器 Pentium 586,主频为 60MHz(Pentium 586 系列主频有 66MHz,75MHz,90MHz,100MHz,120MHz,133MHz,150MHz,166MHz,200MHz),利用亚微米的 CMOS 技术设计 CPU,集成度高达 310 万只晶体管。Pentium 采用了全新的体系结构,运用超标量流水线设计。CPU 中有两条流水线并行工作,每个时钟周期执行两条整数指令;Pentium 片内有两片 8KB Cache,分别作指令 Cache 和数据 Cache,节省了 CPU 的存取时间;浮点运算单元重新设计,采用 8 级流水线和部分固化指令,提高了浮点运算速度;采用动态转移预测的全新概念,预测分支程序的指令流向,节省了 CPU 判别分支的时间。

2) Pentium Pro

Intel 公司 1995 年推出 Pentium Pro,时钟频率有 150MHz,166MHz,180MHz,200MHz。内部集成了 550 万只晶体管,除了有内部 16KB 的一级高速缓冲存储器(L1)外,它使二级高速缓存 L2(256KB 或 512KB)同 CPU 紧密耦合在一个封装内,两级缓存器芯片之间用高频宽的内部通信总线互连。它在微结构设计、CISC/RISC 的混合使用、动态执行、测试等方面都有新的特点。Pentium Pro 其内部数据总线与地址总线均为 64 位,采用 3 路超标量体系结构,14 级超级流水线,非顺序执行指令,进行分支指令预测和数据流分析,然后以优化顺序预测执行,从而将处理器停滞时间减到最小。

MMX 技术是 1996 年底发布的,集成了 450 万只晶体管,支持工作频率有 150MHz,166MHz,200MHz,233MHz,266MHz,300MHz,增加了 16KB 数据缓存和 16KB 指令缓存,4 路写缓存以及分支预测单元和返回堆栈技术。新增加 57 条 MMX 多媒体指令,专门用来处理音频、视频数据,使 CPU 拥有更强大的数据处理能力。奔腾 MMX 采用了双电压设计,其内核电压为 2.8V,系统 I/O 电压为 3.3V。

3) Pentium II

1997 年 5 月 Intel 发布了基于新体系结构的微处理器 Pentium II(P II),带来了 CPU 的一次飞跃,主频分为 233MHz,266MHz,300MHz,333MHz,350MHz,400MHz,450MHz,其内部集成了 750 万只晶体管,采用深 0.35 微米加工工艺,核心工作电压为 2.8V。P II 的主要特性是:双重独立总线结构(二级高速缓存总线及处理器到主内存的系统总线分别独立);内置 MMX 技术(MultiMedia Extentions 多媒体扩展);P II 的高速缓存 L1 从 16KB 增加到 32KB,外部高速缓存 L2 的容量为 512KB,并以 CPU 主频的一半速度运行;动态执行使在给定的时间内能处理更多的数据,提高了 CPU 的工作效率;采用单边接触插盒 SECC(Single Edge Contact)和 slot1 接口标准。1998 年,Intel 公司推出采用 0.25 微米工艺,插槽为 slot2 的高性能 P II CPU。

4) Pentium III

1999 年 2 月 Intel 公司发布了 Pentium III 微处理器,它采用 0.25 微米工艺制造,内部集成 950 万只晶体管。此外它还有以下新特点:主频为 450MHz,系统总线频率为 100MHz,

双重独立总线,一级缓存 L1 为 32KB(16KB 指令缓存和 16KB 数据缓存),二级缓存 L2 为 512KB,以 CPU 核心速度的一半运行。采用 SECC2 封装形式,增加了能够增强音频、视频和 3D 图形效果的 SSE(Streaming SIMD Extensions,数据流单数据多指令扩展)指令集,共 70 条新指令,工作电压 1.6V。能同时处理 4 个单精度浮点变量,提供全新的“处理器分离模式”。1999 年 10 月 Intel 正式发布的新一代 Pentium III 处理机,率先使用 0.18 微米工艺技术。CPU 主频达到 733MHz,1GHz,1.4GHz,芯片内部集成了 2800 万只晶体管,体积更小,性能更强,增加了 MMX 指令,使浮点运算和三维处理方面的能力明显增强。

5) Pentium 4

2000 年 Intel 发布了 Pentium 4,主频为 1.5GHz~3.6GHz,采用了 Intel 公司的 Net-burst 技术,与 PIII CPU 相比,体系结构的流水线深度增加了 1 倍,达到 20 级,从而极大地提高了 P4CPU 的性能和频率。PIII 有 70 条 SSE 指令集,P4 有 144 条 SSE2 新指令集,虽然 SSE2 比 SSE 指令集本身所能提供的性能提升只有 5%左右,但是,在对整数和浮点 SIMD (Single Instruction Multiple Data)操作的时候,性能将会有很大的提升。与以往的 L1 指令缓存不同,在 P4 当中使用了 12K 大小的处理指令跟踪缓存(Execution Trace Cache)。在微处理指令集解码之后将会被这部分 8 通道的缓存寄存起来,并且,这部分缓存同时也负责预测处理通道当中的数据。这样做的目的减少了长数据通道带来的坏处。P4 有着很独特的 L2 缓存,CPU 与 L2 缓存之间有 256bit 的内部传输通道,同时每个时钟周期都能实现从 L2 缓存当中交换数据,也就是说这个数据带宽的峰值是现有 CPU 和 L2 缓存之间的数据带宽当中最高的。

CPU 的发展已使微机在整体性能、处理速度、3D 图形图像处理、多媒体信息处理及通信等诸多方面达到甚至超过了小型机。

以 Intel 微处理器为代表,表 1-1 给出了微处理器性能的演进过程。

表 1-1 INTEL 微处理器性能演进表

性能 芯片	地址 总线	数据 总线	存储器 寻址空间	一级缓存	二级缓存	工作频率 Hz	集成度 (只/片)
8080	16	8	64KB			2M	4500
8088	20	8	1MB			5M	29000
8086	20	16	1MB			5M, 8M, 10M	29000
80286	24	16	16MB			12M, 20M, 25M	13.4 万
80386SX	24	16	16MB			16M, 25M, 33M	27.5 万
80386DX	32	32	4GB			16M, 33M, 40M	27.5 万
80486DX	32	32	4GB	8KB		25M~100M	120 万
Pentium	32	64	4GB	16KB		66M~200M	310 万
Pentium MMX	32 (36)	64	64GB	16KB		200M~300M	450 万
Pentium Pro	36	64	64GB	16KB	256KB	150M~200M	550 万
P II	36	64	64GB	32KB	512KB	233M~450M	750 万
P II Xeon	36	64	64GB	32KB	512KB	350M~450M	750 万
P III	36	64	64GB	32KB	512KB	450M~1.4G	950 万
P4	36	64	64GB	32KB	256KB~1MB	1.3G~2.8G	4200 万

1-2 微型计算机系统

从大型计算机到微型计算机,其基本结构属于冯·诺依曼型计算机,如图 1-1 所示。它包括运算器,控制器,存储器,输入设备和输出设备 5 个组成部分,基本工作原理是存储器存储程序控制的原理。原始的冯·诺依曼机结构上以运算器和控制器为中心,随着计算机系统的发展,演化为以存储器为中心的结构。

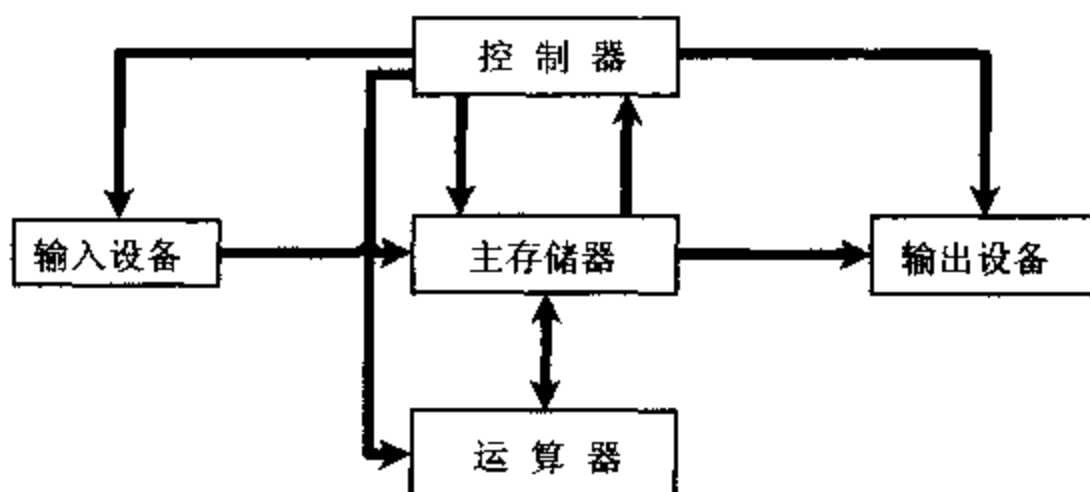


图 1-1 计算机基本结构

一、微型计算机

1. 微型计算机

微型计算机由微处理器,存储器,输入/输出接口电路和系统总线组成,图 1-2 给出了微型计算机的组成。

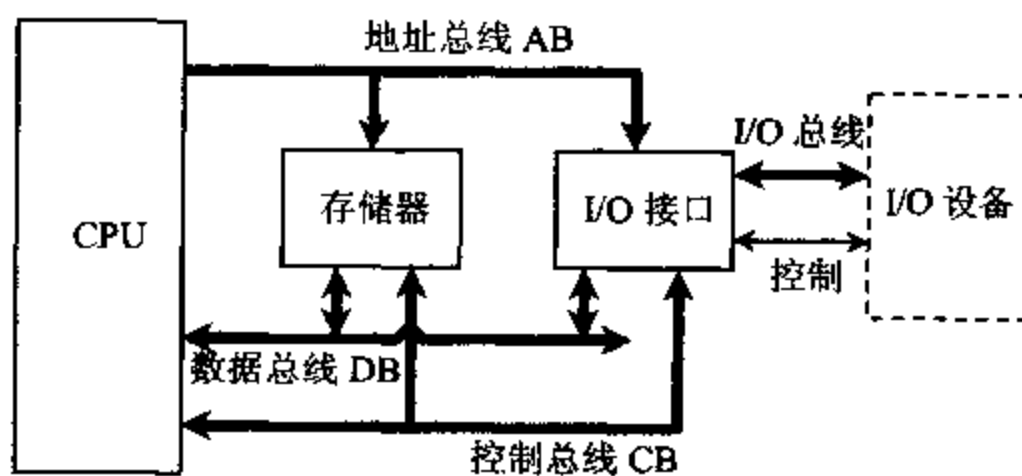


图 1-2 微型计算机的组成

微处理器(Microprocessor)是计算机系统的核心,也称 CPU(中央处理器)。它主要完成(1)从存储器中取指令,指令译码;(2)简单的算术逻辑运算;(3)在处理器和存储器或者 I/O 之间传送数据;(4)程序流向控制等。

存储器分为随机存储器 RAM 和只读存储器 ROM,存储器主要用来存放程序和数据。CPU 从存储器中读取指令,通过指令译码,执行相应的操作,必要时再从存储器或 I/O 设备

中取操作数,指令执行结果送入存储器或 I/O 设备。程序执行结束,任务完毕。

输入/输出接口电路用于将外部设备与 CPU(或存储器)相连接,它们之间进行信息传送时,使之在信息的格式、电平、速度上得到匹配。

总线将 CPU、存储器及 I/O 接口电路相连接,是负责在 CPU 与存储器和 I/O 之间传送地址,数据和控制信息的公共通道。有三种传送信息的总线:地址总线,数据总线和控制总线。

微型计算机已具有运算功能,能独立执行程序,但若没有输入/输出设备,数据及程序不能输入,运算结果无法显示或输出,仍不能正常工作,因此必须构成一个微型计算机系统才能提供使用。

2. 微型计算机系统

以微型计算机为主体,配上外部输入/输出设备及系统软件就构成了微型计算机系统。输入设备的作用是把信息送入计算机,使文字、图形、声音、图像等各种数据通过输入设备进入计算机。常用的输入设备有键盘、鼠标器、图形扫描仪、数字化仪、条形码读入器等。输出设备的作用将计算机对信息处理的结果输出给用户,输出设备有 CRT 显示器、打印机(针式打印机,激光打印机,喷墨打印机)、绘图仪等。

• 没有配置软件的计算机称为裸机,仍然什么工作也不能做,必须配置系统软件和应用软件。基本的系统软件如 DOS, Windows, Linux 等操作系统,以及各种语言的汇编,编译,解释程序,机器的调试程序,故障检测程序及程序库等,如 Basic, Fortran, C, 汇编等。应用软件主要指用户编写的应用程序。随着软件技术的发展,软件工具越来越多,会充分发挥微型计算机的能力,并给用户提供极大的方便。图 1-3 给出了微型计算机系统的组成。

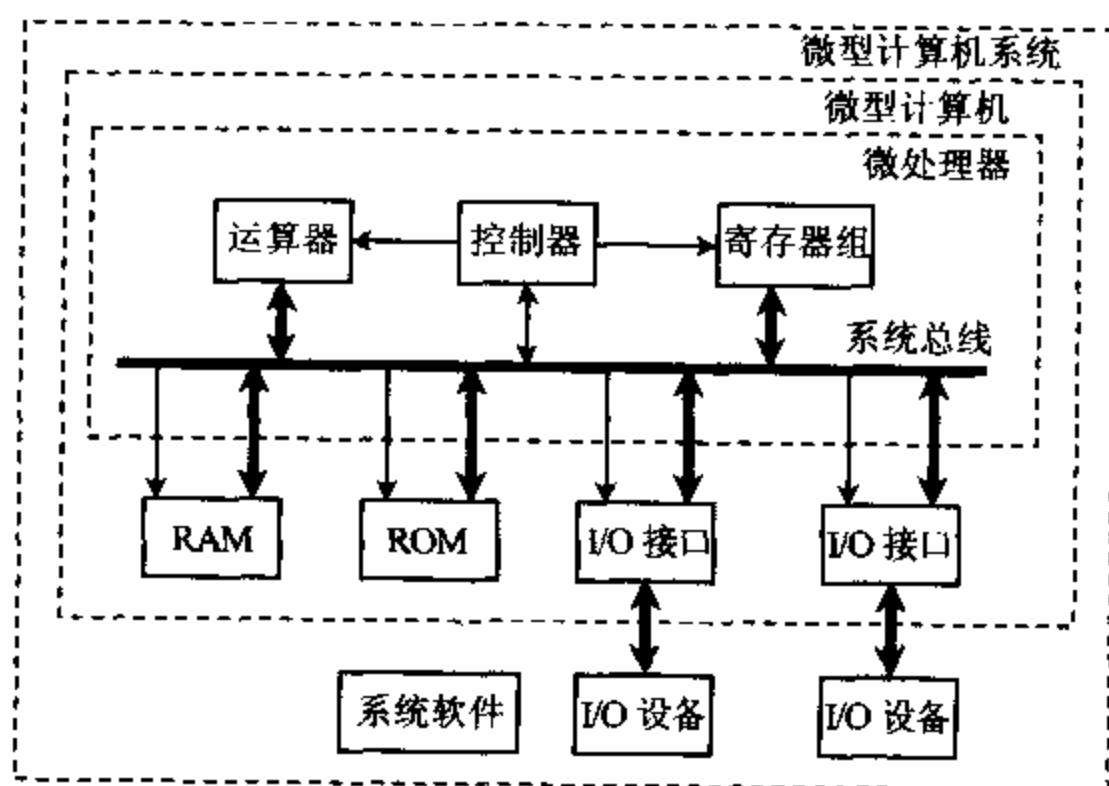


图 1-3 微型计算机系统的组成

3. 微处理器

微处理器是一个中央处理器 CPU,由算术逻辑部件 ALU,累加器和寄存器组,指令指针寄存器 IP(程序计数器),段寄存器,时序和控制逻辑部件,内部总线等组成。典型结构如图 1-4 所示。

其中算术逻辑部件 ALU 主要完成算术运算(+, -, ×, ÷ 等操作)及逻辑运算(与, 或, 非, 异或等操作)。数据寄存器和变址及指针寄存器用来存放参加运算的数据、中间结果或地址。指令指针寄存器 IP 指向要执行的下一条指令的偏移地址, 顺序执行指令时, 每取一

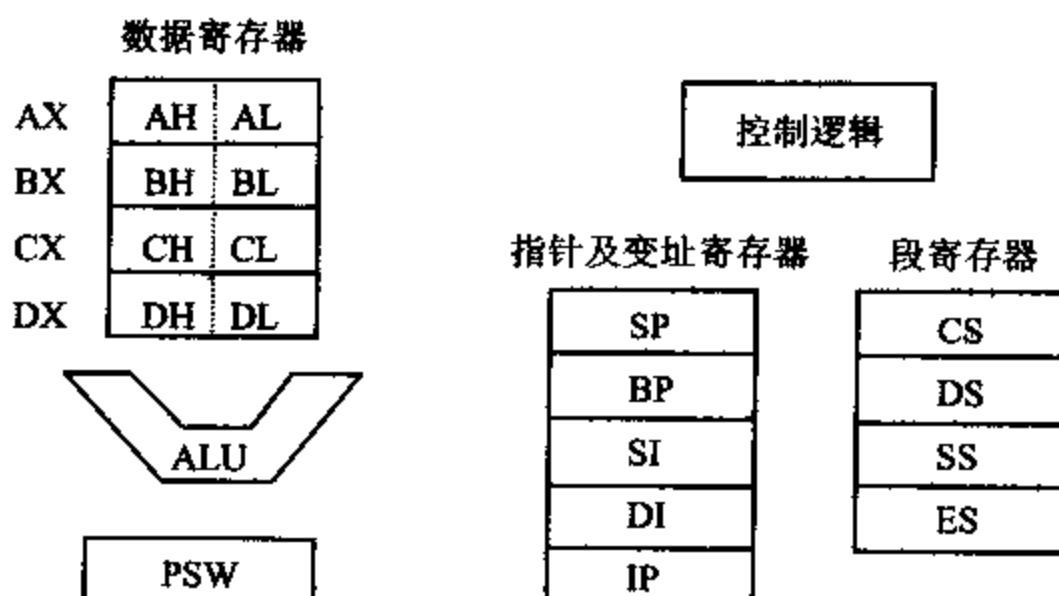


图 1-4 微处理器的结构

条指令增加相应计数。段寄存器给出存储单元的段地址, 与偏移地址组成 20 位物理地址对存储器寻址。标志位寄存器 PSW(或 flag)存放算术与逻辑运算结果的状态, 如溢出、符号、进位等, 这些状态位可作为转移指令的控制。

控制逻辑部件负责对整机的控制: 包括从存储器中取指令, 对指令进行译码和分析, 发出相应的控制信号和时序, 将控制信号和时序送到微型计算机的相应部件, 使 CPU 内部及外部协调工作。

微处理器不能构成独立工作的系统, 也不能独立执行程序, 必须配上存储器, 外部输入/输出接口构成一台微型计算机方能工作。

二、存储器

存储器用来存储程序和数据。存储器一般分为两大类, 内部存储器(内存或主存)和外部存储器(外存)。内存存放当前正在使用或经常使用的程序和数据, CPU 可以直接访问; 外存存放“海量”数据, 相对来说不经常使用, CPU 使用时要先调入内存。此外, 外存总是和外部设备相关的。存储器的体系如图 1-5 所示。

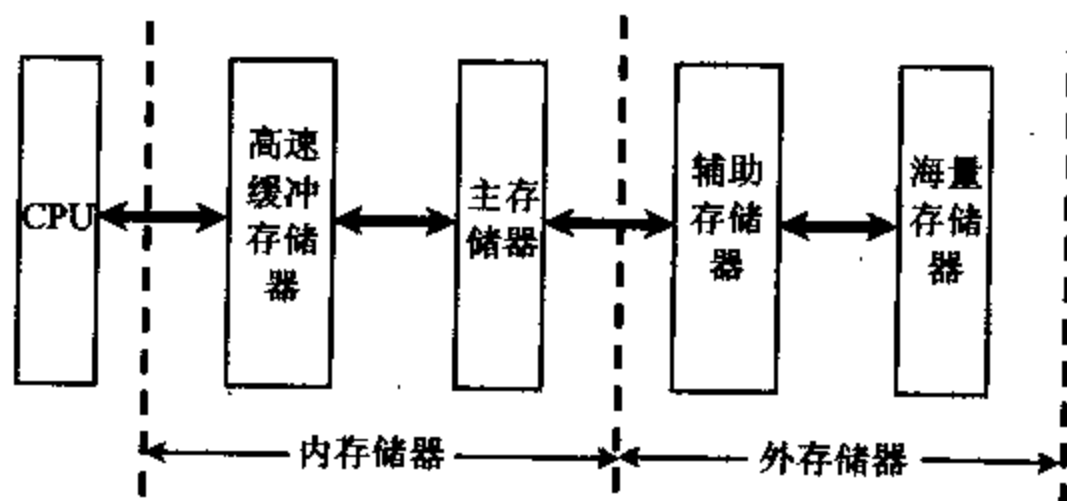


图 1-5 存储体系

1. 内部存储器

内部存储器主要是半导体存储器,主存储器分为随机存取存储器 RAM(Random Access Memory)和只读存储器 ROM(Read Only Memory)。

RAM 可以随机读写,断电后存储内容消失。RAM 又分为动态 RAM(Dynamic RAM)和静态 RAM(Static RAM)。DRAM 的特点是高密度,但存取速度慢。它用 MOS 电路和电容作为存储单元,由于电容放电要定时对其充电,称为刷新。SRAM 用双极型电路或 MOS 电路组成触发器作存储单元,不需要刷新,SRAM 的特点是高速,但存储容量小。微机中的内存条是由 DRAM 组成的,DRAM 集成度的提高也很迅速,差不多每 3 年提高 4 倍。随着 CPU 芯片主频的不断提高,DRAM 的存取速度也不断发展,存取时间从 200ns, 100ns, 80ns 提高到 70ns, 60ns。SRAM 的存取时间可小到 10ns。另有一种非易失性随机读写存储器(NVRAM),它在系统断电后不丢失数据。实际上,它把 SRAM 的实时读写功能与 EEPROM 的可靠非易失能力综合在一起。

ROM 只能读出已存储的内容,不能写入,已存储的内容由厂家或用户预先用设备写入,因此是非易失性的。ROM 又可分成可编程只读存储器 PROM(Programmable Read Only Memory)、可擦除可编程只读存储器 EPROM(Erasable Programmable Read Only Memory)和 EEPROM。PROM 是厂家根据用户需求将芯片内二极管烧断而存储其内容,一般是固化程序用。EPROM 或 EEPROM 用设备写入内容后,可由紫外光照或电擦除其内容,芯片可反复使用。

高速缓冲存储器 Cache 是存储空间较小、存取速度较高的一种存储器,它位于 CPU 和主存之间。CPU 对程序和数据的访问有局部性,Cache 中存放了处理机经常使用的程序和数据,使 CPU 可快速从 Cache 中读写所需的指令和数据,减少了访问主存的次数,提高了整个处理机的性能。

2. 外部存储器

外部存储器主要是磁记录存储器,典型的有软盘、硬盘。外部存储器需要有相应的硬件驱动器。软盘是涂有磁性材料的塑料片,目前 3.5 英寸软盘使用较多。每片有双面,每面有 80 个磁道,每个磁道包含 18 个扇区,每个扇区有 512 个字节。因而高密度双面软盘存储容量为 $80 \text{ 磁道/面} \times 2 \text{ 面} \times 18 \text{ 扇区/磁道} \times 512 \text{ 字节/扇区} = 1.44 \text{ MB}$ 。

硬盘整体装在密封的容器中,直径大多数为 3.5 英寸,2.5 英寸。硬盘容量逐年提高,目前微机中配置 120GB 以上,转速为 7200 转/分。硬盘接口是硬盘与主机系统的连接模块,接口的作用是将硬盘的数据缓存内的数据传输到主机内存或其它应用系统中。不同的接口类型有不同的最大接口带宽,影响硬盘传输数据的速率。典型的硬盘机接口有:

(1) IDE(Integrated Drive Electronics)标准接口,它是把控制器和盘体集成在一起的硬盘驱动器,也叫 ATA(Advanced Technology Attachment)接口。现在 PC 机使用的硬盘大多与 IDE 兼容,只需用一根电缆将它们与主板或接口卡相连。IDE 接口的优点是价格低廉,兼容性好,缺点是速度慢,内置使用,对接口电缆长度有限制。

(2) SCSI(Small Computer System Interface)标准接口,它是小型计算机系统接口,它比其它标准接口传输速率快。常用作高档电脑,工作站,服务器上的硬盘与其它储存装置的接口。SCSI 还广泛应用于光驱,扫描仪,打印机,光盘刻录机等设备上。SCSI 接口优点是适应面广,高性能,具有内置和外置两种,缺点是价格昂贵,安装复杂。

(3) ESDI 标准接口,它是增强型小型设备接口

(4) IPI 标准接口,它是目前正在发展的智能外设接口,用在高性能,大容量的硬盘机中。

除了磁记录存储器外,光盘存储器也被广泛使用。光盘是利用激光技术存储信息的装置,光盘存储器由光盘片和光盘驱动器组成。光盘类型有只读光盘 CD-ROM,由厂家将数据写入后,用户只能读出;一次写入光盘 CD-R(CD-Recordable)允许用户刻录自己内容的光盘,这种刻录模式要求数据连续写入而不被中断,直到整个数据全传送到 CD-R 的媒体为止;另外有可改写光盘 CD-RW(CD-Rewritable),可像磁盘一样多次读写,允许用户删除光盘上原有记录信息,并允许用户在光盘的相同物理区域上记录新信息。光盘具有大容量(容量为 650MB),标准化,重量轻,易保存,寿命长等特点。

3. 存储器组织

微型计算机中物理存储器系统随着微机系统而有不同配置。图 1-6 给出了 8086~80486 微处理器系列的物理存储系统。16 位微机系列配置两个存储体,分别连接数据总线 D7~D0 和 D15~D8,一次数据总线传送 16 位数据。32 位微机系列配置 4 个存储体,分别连接数据总线 D7~D0、D15~D8、D23~D16 及 D31~D24,一次传送 32 位数据。Pentium 以上的微机配置 8 个存储体,分别连接 64 位数据总线。

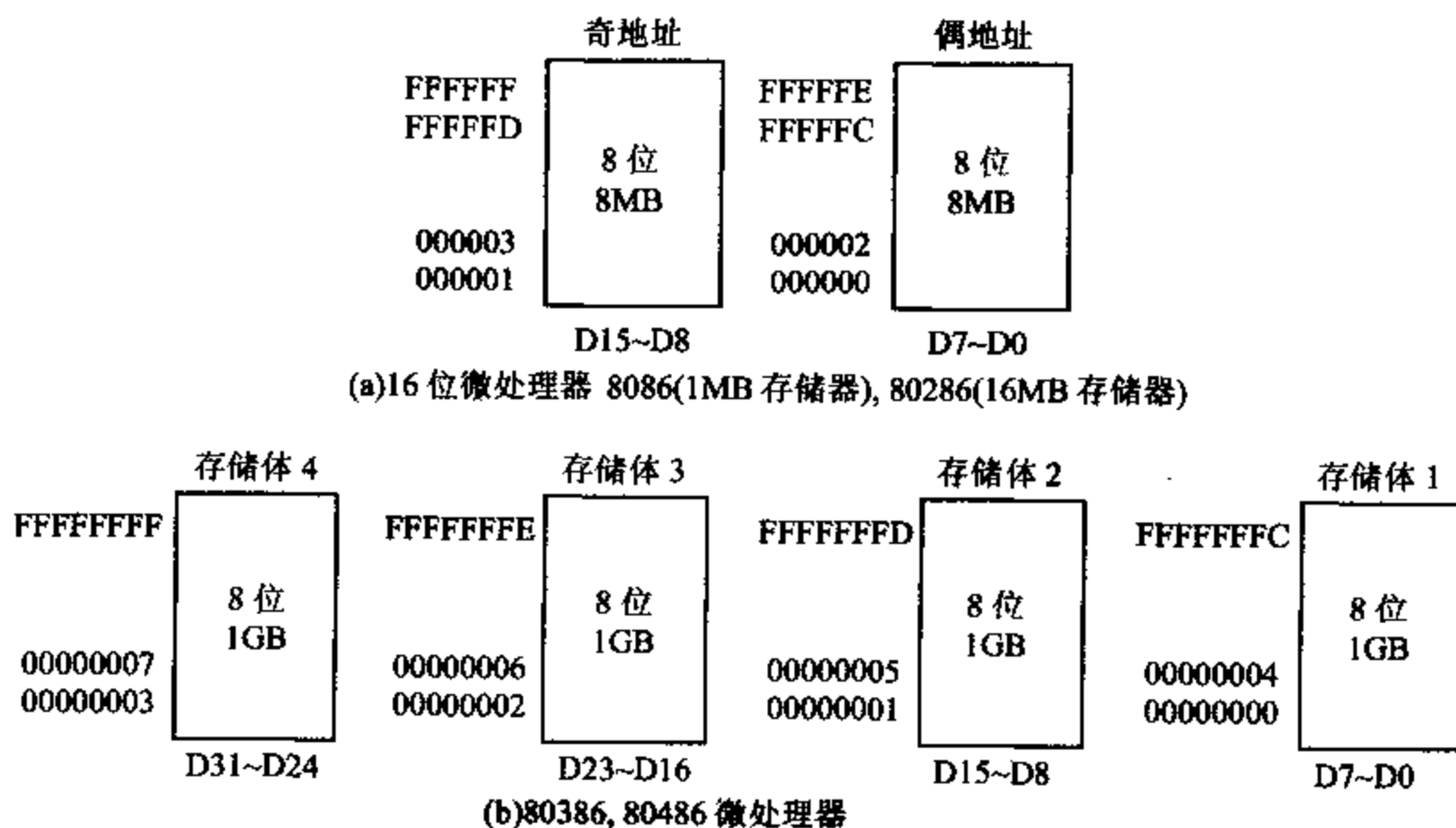


图 1-6 8086~80486 微处理器系列的物理存储系统

4. 存储器性能指标

存储器性能指标主要用存储容量,存取速度来衡量。

存储容量指存储器有多少个存储单元,基本存储单元为位(bit),一般以字节(Byte)或字(word)来计算,常用的单位为 KB(1024Byte), MB(1024KB), GB(1024MB)和 TB(1024GB),例如配置内存 DRAM 容量为 256MB,硬盘容量为 120GB 等。

存取速度是指从存储器中读出数据或数据写入存储器所需要的时间。包括 CPU 给出

存储器地址,存储器的选通信号和读/写信号到存储单元数据读出或写入一次,存储器恢复阶段等时间的总和。存储器芯片的型号,引脚,容量,存取速度都可以从手册中查到。

三、I/O 接口

输入/输出接口电路用于 CPU(或存储器)与外设之间的信息交换。由于外设种类繁多,这些设备与 CPU 之间的工作速度不同,信号电平不同,数据格式不同,因此要配备不同的 I/O 接口电路来辅助 CPU 工作,实现 CPU 与外设之间的速度匹配,信号电平匹配,信号格式匹配,时序控制,中断控制等。主要接口芯片有锁存器 74LS373,缓冲器 74LS245,可编程中断控制器 8259A,可编程计数/定时器 8254,可编程并行/串行接口 8255/8251A。高性能可编程 DMA 控制器 8237A,数/模和模/数转换芯片等。在现在的计算机系统中,这些芯片的功能被集成在大规模集成电路芯片中。

8259A:可编程中断控制器。外部设备请求与 CPU 传送数据时,由中断控制器向 CPU 发出中断请求,CPU 响应中断后,暂停正在服务的程序,转去服务外设的程序,中断服务结束后返回主程序。中断控制器负责对这一中断过程的控制。

8254:可编程计数/定时器。计算机中需要的实时时钟,对动态 RAM 的定时刷新,扬声器的定时发声,以及其它实时控制的定时信号等,均可由 8254 来实现计数/定时器功能。

8255:可编程并行接口芯片。可实现 CPU 与外设之间 8 位数据并行传送,用于打印机接口,CRT 控制接口等。

8251A:可编程串行接口。远距离传送数据时,一般采用串行通信方式,8251A 是一种通用同步/异步数据收发器,作为可编程通信接口芯片,能工作在全双工方式,也可选择同步或异步工作方式。

8237A:可编程 DMA 控制器。当外设有大批量数据传送时,可采用 DMA 工作方式,即外设直接与存储器交换数据。CPU 让出总线,由 DMA 控制器接管总线,产生地址、数据和控制信息。数据传送完毕,将总线返回 CPU。

A/D,D/A:模/数和数/模转换芯片。计算机处理数字量,而外部信号经常是模拟量,通过 A/D 和 D/A 转换器完成模拟量和数字量之间的信息转换。

四、总线

在计算机系统中,各个部件之间传送信息的公共通路叫总线,微型计算机是以总线结构来连接各个功能部件的。各个部件而向总线系统功能扩展时,只要符合总线标准,部件就可以加入到系统中去。

1. 总线标准的特性

总线标准有以下四个特性:

1) 物理特性:物理特性指的是总线物理连接的方式。包括总线的根数、总线的插头、插座是什么形状、引脚是如何排列等。例如 IBM PC/XT 机的总线共有 62 根线,分两排编号,当插件板插到槽中后,左面是 B 面,A 面是元件面。

2) 功能特性:功能特性描写的是这一组总线中每一根线的功能是什么。从功能上看,

总线分成3组:地址总线、数据总线和控制总线。地址总线的宽度指明了总线能够直接访问存储器的地址范围。数据总线的宽度指明了访问一次存储器或外部设备最多能够交换数据的位数。控制总线一般包括CPU与外界联系的各种控制命令。

3) 电器特性:电器特性定义每一根线上信号的传递方向及有效电平范围。一般规定送入CPU的信号叫IN(输入信号),从CPU送出的信号叫OUT(输出信号)。

4) 时间特性:时间特性定义了每根线上的信号在什么时间有效,

2. 总线分类

从总线的不同使用层次可以分为以下几类:

1) 内部总线

内部总线是微处理器内部各个部件之间传送信息的通路。由于制造芯片的面积和芯片引脚的限制,内部总线有的采用单总线结构,有利于集成度提高及成品率提高。有的微处理器内部采用双总线或三总线结构,有利于内部数据传送速度加快。内部总线是由微处理器芯片厂家生产设计的。

2) 元件级总线

元件级总线是连接计算机系统中两个主要部件的总线。元件级总线包括地址总线(Address Bus),数据总线(Data Bus)和控制总线(Control Bus)三种。

地址总线是CPU用来向存储器或I/O端口传送地址的,是三态单向总线。地址总线的位数决定了CPU可直接寻址的内存容量,8位微型机的地址总线是16位,最大寻址范围为64KB。16位微型机的地址总线是20位,最大寻址范围为1MB。32位微型机的地址总线是32位,可寻址空间达4GB。

数据总线是CPU与存储器及外设交换数据的通路,是三态双向总线。为8位,16位,32位,64位等。Intel 8088CPU内部字长为16位,外部数据总线为8位,称为准16位微处理器。

控制总线是用来传输控制信号的,传送方向就具体控制信号而定,如CPU向存储器或I/O接口电路输出读信号、写信号、地址有效信号,而I/O接口部件向CPU输入复位信号、中断请求信号和总线请求信号等。控制总线宽度是根据系统需要确定,一般为8位。

3) 系统总线

系统总线是微处理机机箱内的底板总线,用来连接构成微处理机的各个插件板。在80x86系列微机系统中,使用的系统总线主要有以下几种:

① ISA总线:工业标准体系结构(Industry Standard Architecture)总线。由IBM公司推出的16位标准总线,它由8位的PC总线扩展而来,数据传输率为8MB/s,主要用于IBM PC/XT,AT及兼容机上,也可用在80386/80486机上。

② EISA总线:扩展工业标准体系结构(Extended Industry Standard Architecture)总线。由COMPAQ,HP,AST等多家公司联合推出的32位标准总线,时钟速度8MHz,数据传输率为33MB/s,用于32位微机。

③ VESA总线:视频电子标准协会(Video Electronics Standards Association)联合多家公司推出的全开放通用局部总线。是32位标准总线,数据传输速率为133MB/s,时钟频率33MHz,它适用于80486微机。

④ PCI总线:外设互连(Peripheral Component Interconnect)局部总线。是Intel公司

推出的 32/64 位标准总线。数据传输速率为 132MB/s,适用于 Pentium 微型计算机。PCI 总线是同步且独立于微处理器的,具有即插即用的特性。PCI 总线允许任何微处理器通过桥接口连接到 PCI 总线上。

表 1-2 给出了几种计算机系统总线的特性。

表 1-2 几种计算机总线特性对照表

特性 \ 类型	ISA	EISA	VESA	PCI
总线宽度(位)	16	32	32,64	32,64
最大传输率(MB/s)	8	33	133,266	132,264
总线时钟	8	8.33	33.66	33.66
负载能力	8	6	6	10(PCI 桥)

4) 外部总线

外部总线用于微处理机系统与系统之间,系统与外部设备之间的信息通路。这种总线数据的传送方式有并行方式和串行方式。例如串行通信的 EIR-RS 232 总线,连接仪器仪表的 IEEE-488 总线等。

USB 总线:通用串行总线(Universal Serial Bus)。它是 1994 年底由 Compaq, IBM, Microsoft 等多家公司联合提出的。数据传输速度有 2 种:15MB/s 和 1.5MB/s。USB 连接方式十分灵活,支持热插拔,不需要单独的供电系统。它可以通过一条 4 线串行电缆访问 USB 设备。此接口用于键盘,声卡,简单的图像检索设备及调制解调器,优盘及移动硬盘均可。

3. 总线结构

随着微型计算机的发展,总线的结构从面向系统的单总线结构发展到面向存储器的双总线结构:

(1) 单总线结构

单总线结构如图 1-7 所示,系统的内部存储器和 I/O 接口均挂在单总线上。CPU 与主存,CPU 与 I/O 接口,存储器和 I/O 接口及各个 I/O 接口之间的信息传送都通过总线进行。单总线结构的优点是控制简单,易于扩充系统配置 I/O 设备,但由于系统所有的部件和设

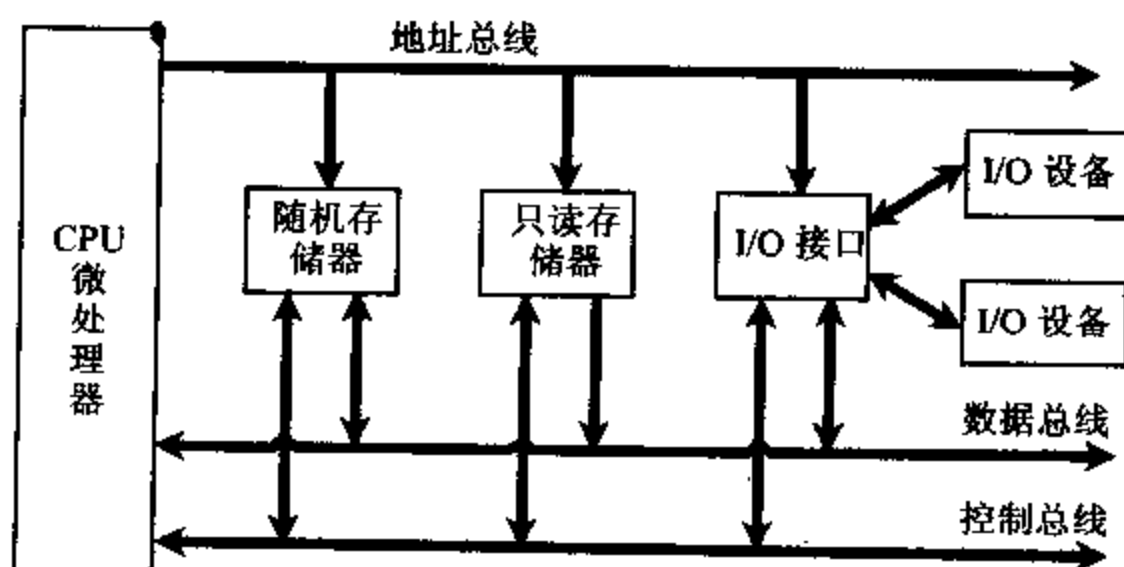


图 1-7 微机单总线结构

备都连在一组总线上,总线只能分时工作,使数据传输量受限。

(2) 面向 CPU 的双总线结构

图 1-8 给出了面向 CPU 的双总线结构,它是在 CPU 和主存储器之间,CPU 与 I/O 设备之间分别设置一组总线。双总线结构通过存储总线使 CPU 对主存储器进行读/写操作,而 CPU 与 I/O 设备之间的信息交换通过输入/输出总线,这样提高了微机系统的数据传送效率。双总线结构的缺点是外设与主存之间没有直接通路,要通过 CPU 进行信息交换,降低了 CPU 的工作效率。

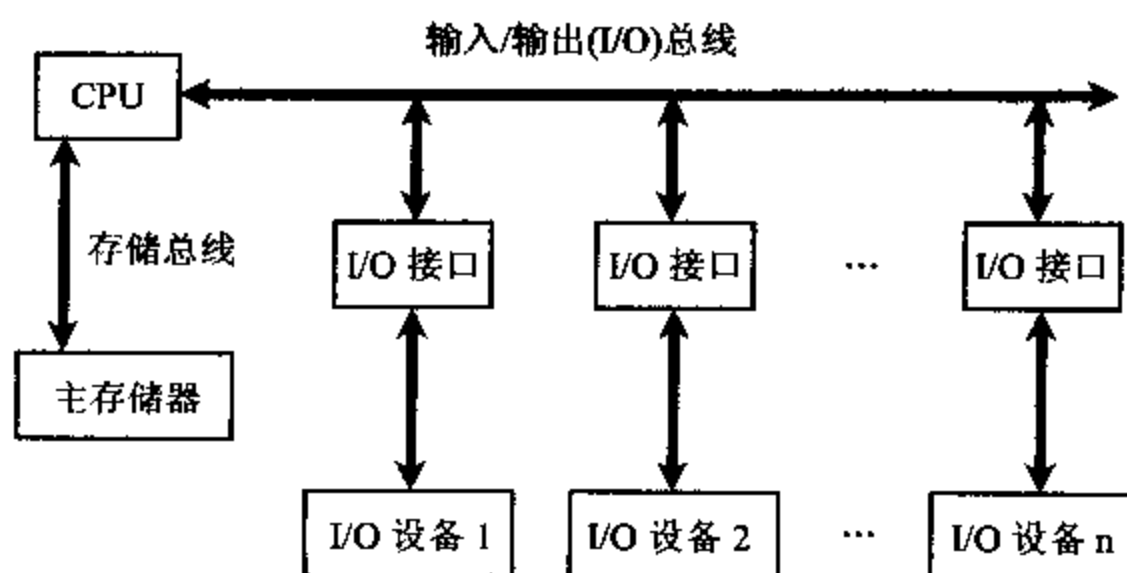


图 1-8 面向 CPU 的双总线结构

(3) 面向主存储器的双总线结构

面向主存储器的双总线结构如图 1-9 所示。它结合了以上二种总线结构的特点,所有的部件和设备均挂到总线上,可通过总线交换信息,同时又在 CPU 与主存储器之间增加了一组高速存储总线,使 CPU 与主存之间可直接高速交换信息。这种结构提高了信息传送效率,同时也不降低 CPU 的工作效率,通常在高档微机中采用面向主存储器的双总线结构。

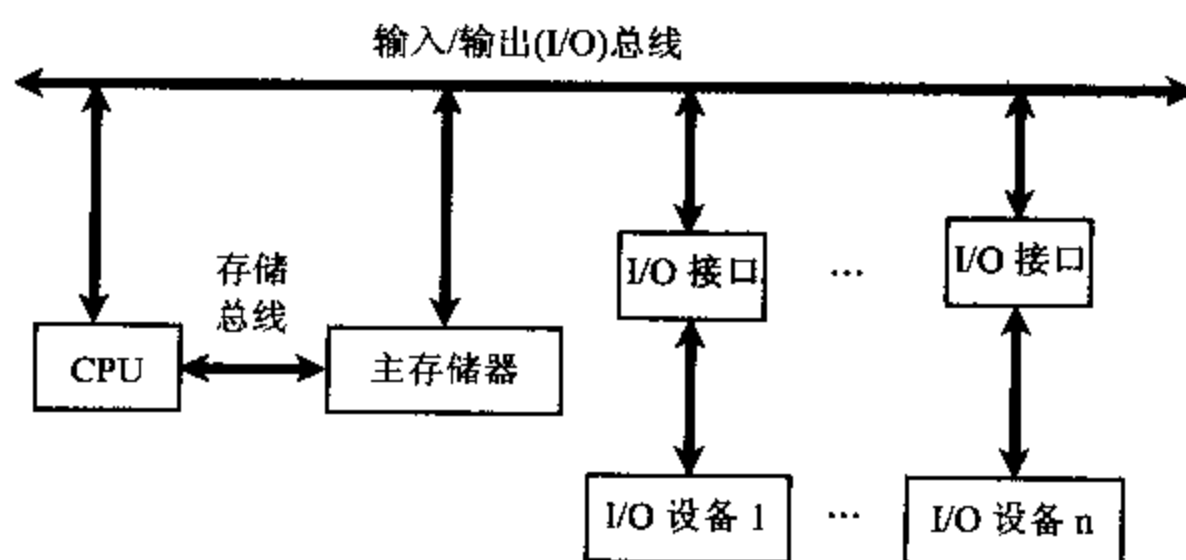


图 1-9 面向主存储器的双总线结构

五、微型计算机的性能指标

微型计算机的主要技术指标如下：

(1) 主频:主频是指微型计算机中 CPU 的时钟频率,微机运行的速度与主频有关。例如:

80486 主频 25MHz~66MHz, Pentium 主频 60MHz~200MHz, Pentium III 主频达到 450MHz~1.4GHz, Pentium 4 更高达 2.8GHz 以上。

(2) 字长:字长指微型计算机能够直接处理的二进制数的位数,字长越长运算精度越高,功能越强,目前微机字长以 32 位为主,但是 AMD 公司已经推出了 64 位处理器。

(3) 内存容量:内存容量指微机存储器能存储信息的字节数,内存容量越大,能存储信息越多,信息处理能力越强。目前微机内存容量一般配置为 256MB~512MB。

(4) 存取周期:存取周期是指主存储器完成一次读写所需的时间,存取时间越短,存取速度越快,整机的运算速度越高。存取周期与主存储器指标有关。

(5) 运算速度:运算速度指微机每秒所能执行的指令条数,单位用 MIPS (Million Instruction Per Second),即百万条指令/秒。执行不同类型的指令所需时间不同,因此使用各种指令的平均执行时间及相应运行指令的比例计算,作为衡量运算速度的标准。8088 是 0.75MIPS,高能奔腾的运算速度已超过了 1000MIPS。

1-3 计算机数据格式

一、数制

人们最常用的是十进制数,计算机中为了存储和计算方便,采用二进制数字系统,只包含 0,1 两个数。人们也使用十六进制,十进制或八进制来表示二进制数,下面介绍这几种数制及其转换。

1. 几种数制的表示

为了方便起见,使用后缀表明数的进制:

二进制——后缀 B 例 1101.1010B

八进制——后缀 Q 或 O 例 625.71Q

十进制——后缀 D 或省略 例 1796.34

十六进制——后缀 H 例 37C8.A2H

十进制数的基数为 10,十进制数制中,包含 0,1,2,3,4,5,6,7,8 和 9 十个符号数字。任何一个十进制数 X 可表示为:

$$X_n X_{n-1} \cdots X_1 X_0 X_{-1} \cdots X_{-m} = \sum_{i=-m}^n X_i \times 10^i$$

十进制数的每位数字都有相应的权与之对应,小数点左边的各位数字的权依次为 $10^0, 10^1, 10^2 \cdots$, 小数点右边各位数字的权依次为 $10^{-1}, 10^{-2} \cdots$ 。一个十进制数的值等于它的各位数字与对应的权值的乘积之和,如 267.85 可表示为:

$$267.85 = 2 \times 10^2 + 6 \times 10^1 + 7 \times 10^0 + 8 \times 10^{-1} + 5 \times 10^{-2}$$

二进制数的基数为 2,包含 0,1 两个符号数字。任何一个二进制数 X 可表示为:

$$(X)_2 = X_n X_{n-1} \cdots X_1 X_0 X_{-1} \cdots X_{-m} = \sum_{i=-m}^n X_i \times 2^i$$

二进制数的每位都有相应的权与之对应,一个二进制数的十进制值等于它的各位数字与对应数值的乘积之和。例如 1101.011B 可表示为:

$$1101.011B = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 0 \times 2^{-1} + 1 \times 2^{-2} + 1 \times 2^{-3} = 13.375$$

同理,八进制数的基数是 8,八进制数制中包含 0,1,2,3,4,5,6,7 八个符号数字。任何八进制数 X 可表示为:

$$(X)_8 = X_n X_{n-1} \cdots X_1 X_0 X_{-1} \cdots X_{-m} = \sum_{i=-m}^n X_i \times 8^i$$

一个八进制数的十进制值等于它的各位数字与对应权值的乘积之和。例如:

$$721.56Q = 7 \times 8^2 + 2 \times 8^1 + 1 \times 8^0 + 5 \times 8^{-1} + 6 \times 8^{-2} = 465.71875$$

十六进制数的基数为 16,十六进制包含 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F 十六个符号数字。任何一个十六进制数 X 可表示为

$$(X)_{16} = X_n X_{n-1} \cdots X_1 X_0 X_{-1} \cdots X_{-m} = \sum_{i=-m}^n X_i \times 16^i$$

一个十六进制数的十进制值等于它的各位数字与对应的权值的乘积之和。例如:

$$6C2.A1H = 6 \times 16^2 + C \times 16^1 + 2 \times 16^0 + A \times 16^{-1} + 1 \times 16^{-2} = 1730.6289$$

2. 其它数制转换到十进制数

任何基数的数制转换为十进制数时,该数每位上的数字与其对应的权值的乘积之和,便是其数制对应的十进制数

例 1-1 将 1110.101B 转换为十进制数。

数码	1	1	1	0	.	1	0	1
权值	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}
数值	$8 + 4 + 2 + 0 + 0.5 + 0 + 0.125 = 14.625$							

例 1 2 将 325.7Q 转换为十进制数。

数码	3	2	5	.	7
权值	8^2	8^1	8^0	.	8^{-1}
数值	$192 + 16 + 5 + 0.875 = 213.875$				

例 1-3 将 58.CAH 转换为十进制数。

数码	5	8	.	C	A
权值	16^1	16^0	.	16^{-1}	16^{-2}
数值	$80 + 8 + 0.75 + 0.039 = 88.789$				

3. 十进制转换成其它进制

十进制数转换成其它进制时,要分两部分计算。转换十进制整数部分时要用基数去除,转换小数部分时,要用基数去乘。

转换十进制整数部分的算法如下:

- 1) 用其它数制的基数除十进制数;
- 2) 保存余数(最先得到的余数为最低有效位);
- 3) 重复 1)和 2),直到商为 0。

例 1-4 将 18 分别转换为二进制,八进制,十六进制数。

$$\begin{array}{r}
 2 \overline{) 18} \quad \text{余数}=0 \\
 \underline{2 } 9 \quad \text{余数}=1 \\
 \underline{2 } 4 \quad \text{余数}=0 \\
 \underline{2 } 2 \quad \text{余数}=0 \\
 \underline{2 } 1 \quad \text{余数}=1 \\
 \underline{\phantom{2 }} 0
 \end{array}$$

结果 $18=10010B$

$$\begin{array}{r}
 8 \overline{) 18} \quad \text{余数}=2 \\
 \underline{8 } 2 \quad \text{余数}=2 \\
 \underline{\phantom{8 }} 0
 \end{array}$$

结果 $18=22Q$

$$\begin{array}{r}
 16 \overline{) 18} \quad \text{余数}=2 \\
 \underline{16 } 1 \quad \text{余数}=1 \\
 \underline{\phantom{16 }} 0
 \end{array}$$

结果 $18=12H$

转换十进制小数部分的算法如下:

- 1) 用其它数制的基数乘十进制数;
- 2) 保存结果的整数部分(最先得到的整数结果为最高位小数);
- 3) 重复步骤 1), 2), 直到小数部分为 0。

例 1-5 将 0.125 分别转换成二进制数, 八进制数, 十六进制数。

$$\begin{array}{r}
 0.125 \\
 \underline{2} \\
 0.25 \quad \text{整数}=0 \\
 \underline{2} \\
 0.5 \quad \text{余数}=0 \\
 \underline{2} \\
 1.0 \quad \text{余数}=1
 \end{array}$$

结果 $0.125=0.001B$

$$\begin{array}{r}
 0.125 \\
 \underline{8} \\
 1.0 \quad \text{整数}=1 \\
 \text{结果 } 0.125=0.1Q
 \end{array}$$

$$\begin{array}{r}
 0.125 \\
 \underline{16} \\
 2.0 \quad \text{整数}=2 \\
 \text{结果 } 0.125=0.2H
 \end{array}$$

4. 二进制编码的十六进制

二进制编码的十六进制(BCH)是用二进制编码表示的十六进制数据, 便于人们阅读。用二进制编码的 4 位表示 1 位十六进制数, 每个十六进制数分别转换成 BCH 码, 数位之间用空格分开。例如: $3AC6H = 0011 \ 1010 \ 1100 \ 0110 \ B$ 。

表 1-3 给出了二进制编码的十六进制 BCH 码。

表 1-3 二进制编码十六进制 BCH 码

十进制	二进制	十六进制	十进制	二进制	十六进制
0	0000	0	8	1000	8
1	0001	1	9	1001	9
2	0010	2	10	1010	A
3	0011	3	11	1011	B
4	0100	4	12	1100	C
5	0101	5	13	1101	D
6	0110	6	14	1110	E
7	0111	7	15	1111	F

二、计算机数据格式

1. 补码

计算机中的数用二进制表示, 数的符号也用二进制表示, 一般用最高有效位来表示数的

符号,0 表示正数,1 表示负数。将一个数与符号用数值化表示,这样的数称为机器数。机器数的字长由计算机字长决定,也就是决定了机器数的表示范围。8 位字长可以表示 256 个数,对无符号数,取数范围为 $0 \sim 255 (0 \sim FFH)$; 对有符号数,取数范围为 $-128 \sim +127 (80H \sim 7FH)$ 。16 位字长的微机,无符号数的取数范围为 $0 \sim 65535 (0 \sim FFFFH)$,有符号数的取数范围为 $-32768 \sim +32768 (8000H \sim 7FFFH)$ 。

机器数可以用原码,补码和反码表示,常用的是补码表示法。

补码表示法中正数 X 采用符号-绝对值表示,即数的最高有效位为 0,其余部分用数的绝对值。负数 X 用 $2^n - |X|$ 表示, n 为机器字长。求负数补码的简单方法是将此数对应的正数原码写出,再按位求反得到反码,反码加 1 得到补码。

例 1-6 对下列 8 位字长及 16 位字长数据,分别给出补码表示。

$$\begin{aligned} 37_{10} &= 00100101B = 25H \\ -37_{10} &= 11011011B = DBH \\ 7CA2H_{16} &= 01111100\ 10100010B \\ -7CA2H_{16} &= 10000011\ 01011110B \end{aligned}$$

用补码表示的数进行符号扩展时,正数的符号扩展应在前面补 0,负数的符号扩展应在前面补 1。例如机器字长 8 位,将数扩展到 16 位运算如下:

$$\begin{aligned} \text{例 1-7} \quad 54_{10} &= 00110110B \quad \text{扩展后 } 54_{10} = 00000000\ 00110110B = 0036H \\ -54_{10} &= 11001010B \quad \text{扩展后 } -54_{10} = 11111111\ 11001010B = FFCAH \end{aligned}$$

补码运算中的规则:

$$\begin{aligned} [x+y]_{\text{补}} &= x_{\text{补}} + y_{\text{补}} \\ [x-y]_{\text{补}} &= x_{\text{补}} + [-y]_{\text{补}} \end{aligned}$$

例 1-8 $x=28, y=-73$

$$\begin{aligned} [x+y]_{\text{补}} &= -45_{10} = 11010011B \\ x_{\text{补}} + y_{\text{补}} &= 28_{10} + (-73)_{10} = 00011100B + 10110111B = 11010011B \\ [x-y]_{\text{补}} &= 101_{10} = 01100101B \\ x_{\text{补}} - y_{\text{补}} &= 28_{10} + 73_{10} = 00011100B + 01001001B = 01100101B \end{aligned}$$

上面的例子说明采用补码来表示数,在计算机中的加、减法运算中,不必判断数的正负,只要符号位参加运算就能自动得到正确的结果。

2. BCD 码

二进制编码的十进制(BCD)是将十进制数的每一位以二进制数编码方式表示,十进制的 $0 \sim 9$ 分别用 BCD 数的 0000 到 1001 表示,而不是整个十进制数转换成二进制形式。BCD 码有压缩和非压缩两种形式。压缩 BCD 数据以每字节 2 个数字的形式存储,非压缩 BCD 数据以每字节 1 位数字的形式存储。压缩 BCD 数据常用于微处理机指令系统中的某些指令,例如 BCD 加法和减法。

例 1-9

十进制	压缩 BCD	非压缩 BCD	二进制
12	00010010	0000 0001 0000 0010	1100
623	00000110 00100011	0000 0110 0000 0010 0000 0011	0010 0110 1111

3. ASCII 码

计算机中的字符,字符串,在机器中必须用二进制数来表示,另外从键盘输入的信息(数

字和字符)或显示输出的信息都是以字符方式输入输出的,字符包括:

·字母:A~Z,a~z;

·数字:0~9

·专用字符:+ - * / ↑ SP(space 空格)

·非打印字符:BEL(响铃) LF(换行) CR(回车)

这些字符通常用美国信息交换代码 ASCII 码(American Standard Code for Information Interchange)来表示,ASCII 码用 8 位二进制码表示一个字符。其中低 7 位为字符的 ASCII 值,最高位一般用作校验位。详见附录 C。

3. 数据类型

计算机中存储的数据类型有字节(Byte),字(Word),双字(double word)等。一个字节为 8 位二进制数,字节数据以无符号和有符号的整数形式存储。一个字为 16 位二进制数,由两个字节组成,双字数据有 4 个字节,为 32 位二进制数。用汇编语言定义字节,字,双字数据时,分别用伪指令 DB,DW,DD 定义。在存储器中存放时,最低有效字节存放在低地址存储单元中,高有效位存放在高地址存储单元中。

例 1-10

DA1 DB 36H,7BH,57H ;定义 8 位数
DA2 DW 2345H, 678CH ;定义 16 位数
DA3 DD 3456AB9CH, 2C00H ;定义 32 位数

DA1,DA2 在存储器中存放形式如图 1-10。

DA1	36
	7B
	57
DA2	45
	23
	8C
	67
	...

图 1-10 例 1-10 数据存储形式

4. 浮点数

计算机中小数点的表示有定点和浮点两种:定点的小数点固定在最高位的最右边或最低位的最右边。图 1-11 给出了 32 位定点数的表示。图(a)所示小数点定在最低位的右边,表示此数是一个整数,图(b)所示小数点定在最高位的右边,表示此数为纯小数,S 表示符号位。

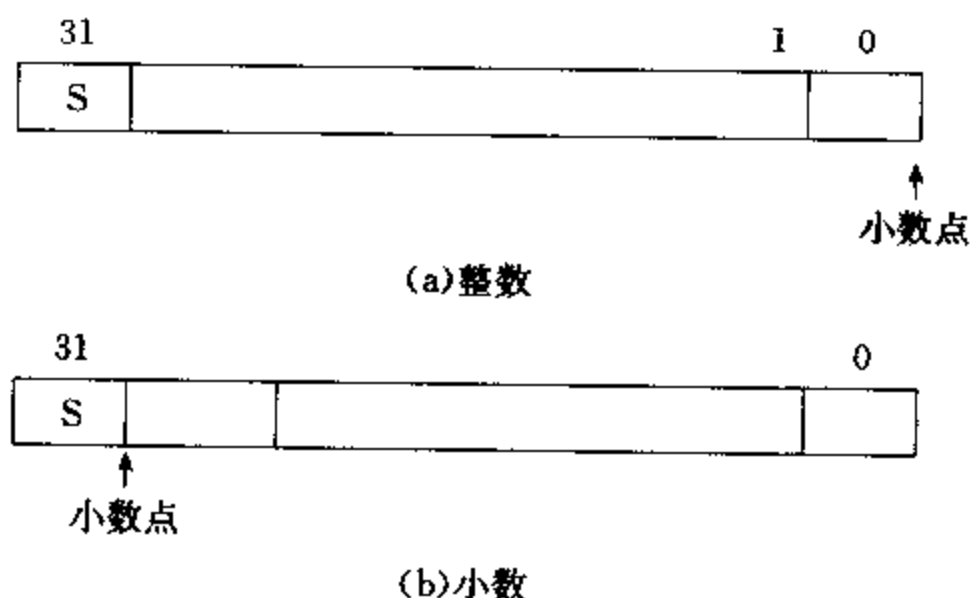


图 1-11 32 位定点数的表示

在 32 位字长的计算机中,若用定点补码表示数值,其范围为 $-2^{31} \sim 2^{31}-1$ 。因此要用浮点数来表示。浮点数包括符号位 S,尾数(有效小数)和指数(阶)。4 字节的浮点数为单

精度,8字节的浮点数为双精度。如图 1-12 所示:

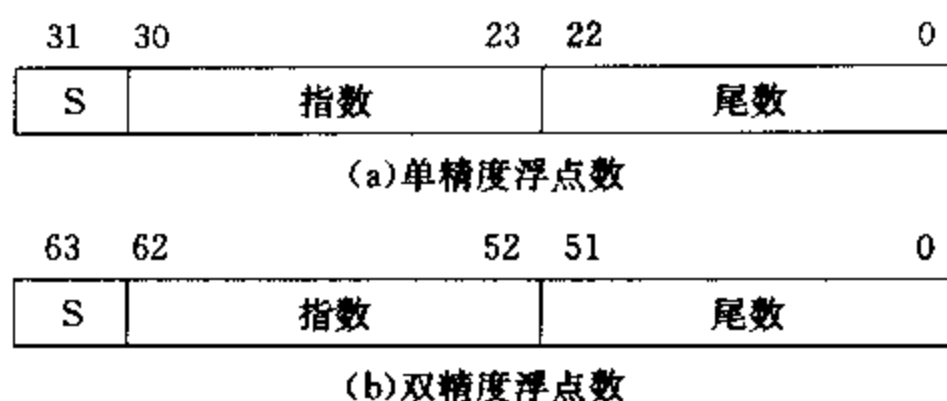


图 1-12 浮点数表示法

其中图(a)为单精度格式,包括一个符号位,8位阶(指数),24位小数。尾数可用原码,补码或反码表示,为了让数有统一的表示,规定尾数的最高位为1,称为规格化数。事实上24位尾数包括一个默认位(隐藏),存储23位就可以表示24位了,默认的1位是规格化实数的第1位。规格化一个数时,调整它使其值大于等于1,而小于2。例如12转换为二进制数为1100B,规格化后结果为 1.1×10^3 ,其中整数1不存储在23位尾数部分内,为默认位。阶(指数)以移码表示,对单精度浮点数偏移7FH(127),对双精度格式偏移量为3FFH(1023),存储浮点数阶码前,偏移量要先加到阶码上,对上例12来说,在单精度格式中移码后的阶表示为 $127+3$ 即130(82H)。表1-4给出了几个浮点数表示的实例。

表 1-4 单精度浮点数

十进制	二进制	非规格化	符号	移码阶	尾数
12	1100	1.1×2^3	0	10000010(82H)	1000000 00000000 00000000
-12	1100	-1.1×2^3	1	10000010(82H)	1000000 00000000 00000000
+100	1100100	1.1001×2^6	0	10000101(85H)	1001000 00000000 00000000
-1.75	1.11	-1.1×2^0	1	01111111(7FH)	1100000 00000000 00000000

表 1-4 中,十进制数 12 的二进制为 1100,非规格化数为 1.1×2^3 ,符号为 0;
 移码阶 $127+3=130(82H)=1000\ 0010$;
 尾数 1000000 00000000 00000000(小数点前的 1 为默认)。

例 1-11 将数 224 转换成单精度浮点数。

224 = 1110 0000 非规格化数为 1.11×2^7 ,符号为 0;

移码阶 $127+7=134(86H)=1000\ 0110$;

尾数为 1000000 00000000 00000000。

汇编程序中定义单精度浮点数和双精度浮点数分别用伪指令 DD, REAL4 及 DQ, REAL8。

例 1-12

NUM1	DD	1.234	;定义 1.234
NUM2	DD	-23.4	;定义 -23.4
NUM3	REAL4	4.2E2	;定义 420
NUM4	REAL4	1.234	;定义 4 字节实数

习 题

1. 什么是冯·诺依曼机?
2. 微处理器,微型计算机,微型计算机系统有什么联系与区别?
3. 微处理器有哪些主要部件组成? 其功能是什么?
4. 画一个计算机系统的方框图,简述各部分主要功能。
5. 列出计算机系统中的三种总线结构,画出面向存储器的双总线结构图。
6. 8086 微处理器可寻址多少字节存储器? Pentium II 微处理器可寻址多少字节存储器?
7. 什么是 PCI 局部总线? 什么是 USB?
8. 说明以下一些伪指令的作用。
(1)DB (2)DQ (3)DW (4)DD
9. 将下列二进制数转换为十进制数。
(1)1101.01B (2)111001.0011B
(3)101011.0101B (4)111.0001B
10. 将下列十六进制数转换为十进制数。
(1)A3.3H (2)129.CH
(3)AC.DCH (4)FAB.3H
11. 将下列十进制数转换为二进制、八进制、十六进制。
(1)23 (2)107
(3)1238 (4)92
12. 将下列十进制数转换为 8 位有符号二进制数。
(1)+32 (2)-12
(3)+100 (4)-92
13. 将下列十进制数转换为压缩和非压缩格式的 BCD 码。
(1)102 (2)44
(3)301 (4)1000
14. 将下列二进制数转换为有符号十进制数。
(1)10000000B (2)00110011B
(3)10010010B (4)10001001B
15. 将下列十进制数转换为单精度浮点数。
(1)+1.5 (2)-10.625
(3)+100.25 (4)-1200
16. 将下列单精度浮点数转换为十进制数。
(1)0 10000000 11000000000000000000000
(2)1 01111111 00000000000000000000000
(3)0 10000000 10010000000000000000000

第二章 8086 系统结构

由于制造工艺的原因,微处理器的结构受到几个方面的限制:

1. 引脚数限制:出于对工艺技术和生产成本的考虑,微处理器引脚一般在 40~64 脚(8086~80286)之间。80386 引脚有 132 脚,80486 引脚有 168 脚。
2. 芯片面积限制:由于工艺上的问题,增大芯片面积会使产品合格率下降,反而使成本昂贵,因此不能盲目增大芯片面积。
3. 器件速度限制:目前微处理器采用 MOS 工艺,可以提高集成度,降低功耗,但器件速度较慢负载能力较弱。

由于上述限制,使 16 位微处理器基本结构具有如下特点:

1. 引脚功能复用:由于引脚数限制,部分引脚设计为功能复用。例如,数据双向传输可由“读/写”信号来控制,决定数据处于输入还是输出状态。
2. 单总线、累加器结构:由于芯片面积限制,使微处理器内部寄存器的数目,数据通路的位数受到限制。因此绝大多数微处理器内部采用单总线、累加器为基础的结构。
3. 可控三态电路:微处理器外部总线同时连接多个部件,为避免总线冲突和信号串扰,采用可控三态电路与总线相连,不工作器件所连的三态电路处于高阻状态。
4. 总线分时复用:由于引脚不够,地址总线 and 数据总线使用了相同的引脚。采用总线分时复用技术解决了引脚不够的限制,但操作时间增加了。

Intel 8086 CPU 是 16 位微处理器。它采用 N—沟道,耗尽型负载的硅栅工艺(HMOS 工艺)制造,外型为双列直插式,有 40 个引脚。时钟频率有 3 种,8086 型微处理器为 5MHz,8086-2 型为 8MHz,8086-1 型为 10MHz。8086 CPU 有 16 根数据线和 20 根地址线,直接寻址空间为 2^{20} ,即为 1MB。

8086 CPU 有一组强有力的指令系统,内部有硬件乘除指令及串处理指令,可对多种数据类型进行处理。8086 CPU 与 8 位 CPU 8080 向上兼容,处理能力比 8080 高十倍以上,面相同任务程序代码长度可缩短 20%。8086 CPU 可与 8087 协处理器及 8089 输入/输出处理器构成多机系统,以提高数据处理及输入/输出能力。

8088 CPU 内部结构与 8086 基本相同,但对外数据总线只有 8 条,称为准 16 位微处理器。

2-1 8086 CPU 结构

微型计算机工作时,总是先从存储器中取指令,需要的话再取操作数,然后执行指令,送结果。通常 8 位机是串行执行的,而 16 位机可并行操作。8086 CPU 由总线接口部件 BIU 和指令执行部件 EU 组成,BIU 和 EU 的操作是并行的。总线接口部件 BIU 完成取指令,读操作数,送结果,所有与外部的操作由其完成。而指令执行部件 EU 从 BIU 的指令队列

中取出指令,并且执行指令,不必访问存储器或 I/O 端口。若需要访问存储器或 I/O 端口,也是由 EU 向 BIU 发出访问所需要的地址,在 BIU 中形成物理地址,然后访问存储器或 I/O 端口,取得操作数送到 EU,或送结果到指定的内存单元或 I/O 端口。这种并行工作方式,大大提高了系统工作效率。

图 2-1 给出了 8086 CPU 的内部结构框图。

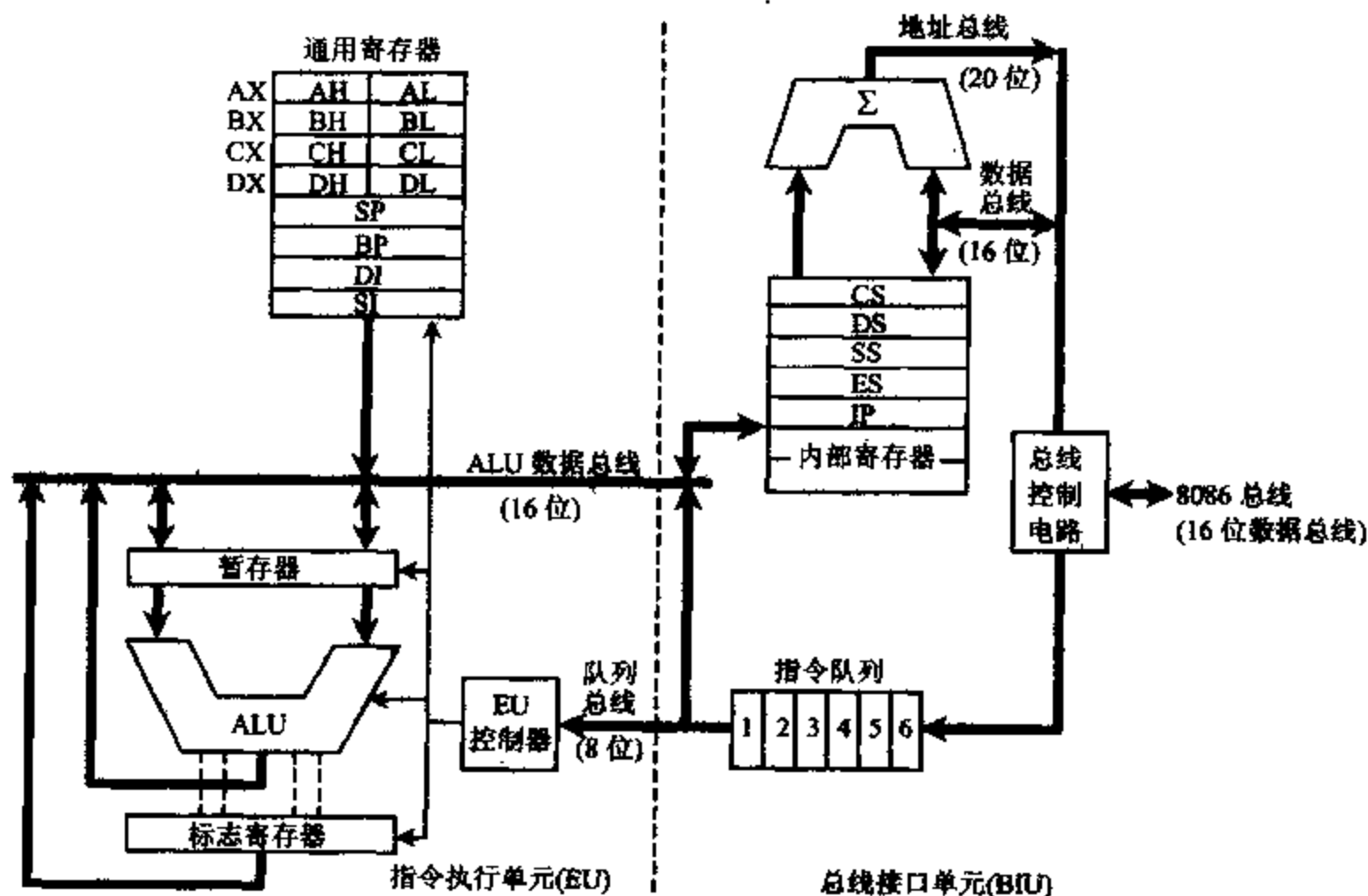


图 2-1 8086 CPU 内部结构框图

一、8086 CPU 的内部结构

1. 总线接口部件 BIU (Bus Interface Unit)

总线接口部件 BIU 是 8086 CPU 与外部 (存储器和 I/O 端口) 的接口,它提供了 16 位双向数据总线和 20 位地址总线,完成所有外部总线操作。

BIU 具有下列功能:地址形成、取指令、指令排队、读/写操作数和总线控制。它由下列各部分组成:

(1) 16 位段地址寄存器:

CS —— 代码段寄存器。

DS —— 数据段寄存器。

ES —— 附加段寄存器。

SS —— 堆栈段寄存器。

(2) 16 位指令指针寄存器 IP: 存放下一条要执行指令的偏移地址。

(3) 20 位物理地址加法器: 将 16 位逻辑地址变换成存储器读/写所需要的 20 位物理地址,实际上完成地址加法操作。

(4) 6 字节指令队列:预放 6 字节的指令代码。

(5) 总线控制逻辑:发出总线控制信号。

总线接口部件的工作过程如下:

首先由代码段寄存器 CS 中 16 位段基地址,在最低位后面补 4 个 0,加上指令指针寄存器 IP 中 16 位偏移地址,在地址加法器内形成 20 位物理地址,20 位地址直接送往地址总线,然后通过总线控制逻辑发出存储器读信号 $\overline{RD}$,启动存储器,按给定的地址从存储器中取出指令,送到指令队列中等待执行。

BIU 的指令队列可存储 6 字节指令代码,它是先进先出的队列寄存器,允许预取 6 字节指令代码。一般情况下,指令队列中填满指令,EU 可从指令队列中取出指令执行。指令代码装入指令队列输入端后,自动调整指令队列输入端指针。EU 从指令队列输出端取指令,且在 EU 取走一字节的指令代码后,自动调整指令队列输出端指针。当指令队列有 2 个或 2 个以上的字节空余时,BIU 自动将指令取到指令队列中。当指令队列已满,并且执行部件 EU 未向 BIU 申请读/写存储器操作数,则 BIU 不执行任何总线周期,处于空闲状态。

EU 从指令队列中取走指令,经指令译码后,向 BIU 申请从存储器或 I/O 端口读写操作数。只要收到 EU 送来的逻辑地址,BIU 又将通过地址加法器将现行数据段及送来的逻辑地址组成 20 位物理地址,在当前取指令总线周期完成后,在读/写总线周期访问存储器或 I/O 端口完成读/写操作。最后 EU 执行指令,由 BIU 将运算结果读出。

指令指针寄存器 IP 由 BIU 自动修改,指向下一条指令在现行代码段内的偏移地址。当进行 IP 入栈操作时,先是把 IP 入栈再自动调整 IP 到下一条要执行指令的地址。当 EU 执行转移指令时,则 BIU 清除指令队列,从转移指令的新地址取得指令,立即送给 EU 执行,然后从后续指令序列中取指令填满队列。

总线控制部件发出总线控制信号,实现存储器读/写控制和 I/O 读/写控制。它将 8086 CPU 的内部总线与外部总线相连,是 8086 CPU 与外部打交道不可缺少的路径。

2. 指令执行部件 EU(Execution Unit)

指令执行部件 EU 完成指令译码和执行指令的工作。

它由以下几个部分组成:

(1) 算术逻辑运算单元 ALU:完成 8 位或 16 位的二进制运算,16 位暂存器可暂存参加运算的操作数。

(2) 标志寄存器 PSW:存放 ALU 运算结果特征。

(3) 寄存器组:4 个通用 16 位寄存器 AX、BX、CX、DX,其中 AX 又称累加器。4 个专用 16 位寄存器:源变址寄存器 SI、目的变址寄存器 DI、堆栈指针寄存器 SP、基址指针寄存器 BP。

(4) EU 控制器:取指令控制和时序控制部件。

指令执行部件工作过程为:EU 从 BIU 的指令队列输出端取得指令,进行译码,若执行指令需要访问存储器或 I/O 端口去取操作数,则 EU 将操作数的偏移地址通过内部 16 位数据总线送给 BIU,与段地址一起,在 BIU 的地址加法器中形成 20 位物理地址,申请访问存储器或 I/O 端口,取得操作数送给 EU,EU 根据指令要求向 EU 内部各部件发出控制命令,完成执行指令的功能。

算术逻辑运算单元 ALU 完成各种算术运算及逻辑运算,运算的操作数可从存储器取

得,也可从寄存器组取来,16 位暂存器暂存参加运算的操作数。运算结果由内部总线送到 EU 的寄存器组或送到 BIU 的内部寄存器,由 BIU 写入存储器或 I/O 端口。运算后结果的特征改变了标志寄存器 PSW 的状态,供测试、判断及转移指令使用。EU 控制器负责从指令队列中取指令、指令译码及发各种控制命令以完成指令要求的功能。

一般情况下,指令顺序执行,EU 从指令队列中取指令而不是访问存储器取指令,所以取指令与执行指令可并行操作。但遇到转移指令、调用指令和返回指令,要将指令队列中的内容作废,由 BIU 重新取转移去的新地址中的指令代码,EU 才能继续执行指令,此时并行操作可能受到影响。但这种情况相对较少发生,因此,EU 与 BIU 之间相互配合又相互独立的非同步工作方式提高了 CPU 的工作效率。

二、寄存器结构

寄存器结构在计算机中起了重要的作用,它的存取速度比存储器快得多,这样可以相当于存储单元,用来存放运算过程中所需要的操作数地址、操作数及中间结果。8086 微处理器内部包含有 4 组 16 位寄存器,它们分别是通用寄存器组,指针和变址寄存器,段寄存器,指令指针及标志位寄存器。如图 2-2 所示。

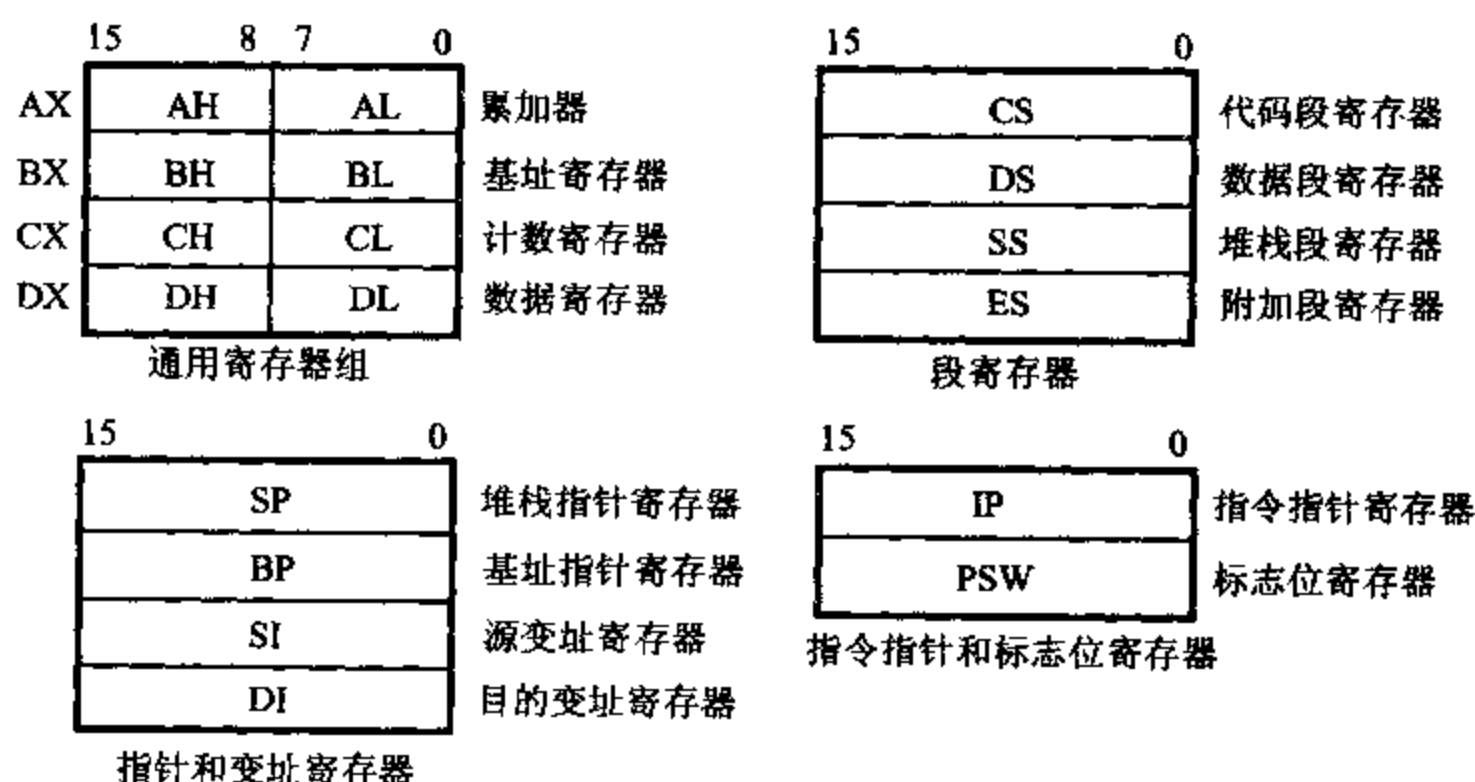


图 2-2 8086 CPU 寄存器组

1. 通用寄存器组

8086/8088 CPU 在指令执行部件 EU 中有 4 个 16 位通用寄存器,它们是 AX、BX、CX 和 DX,用以存放 16 位数据或地址。也可分为 8 个 8 位寄存器来使用,低 8 位是 AL、BL、CL 和 DL,高 8 位为 AH、BH、CH 和 DH,只能存放 8 位数据,不能存放地址。通用寄存器通用性强,对任何指令它们具有相同的功能。为了缩短指令代码的长度,在 8086 中,某些通用寄存器用作专门用途。例如,串指令中必须用 CX 寄存器作为计数寄存器,存放串的长度,这样在串操作指令中不必给定 CX 的寄存器号,缩短了串操作指令代码的长度,这种寻址方式也称为“隐含寻址”。同样,AX、BX、DX 寄存器又可分别称为累加器、基址寄存器及数据寄存器。

表 2-1 列出了 8086 通用寄存器的特殊用途。

表 2-1 寄存器的特殊用途

寄存器名	特 殊 用 途	隐含性质
AX, AL	在输入输出指令中作数据寄存器 在乘法指令中存放被乘数或乘积, 在除法指令中存放被除数或商	不能隐含 隐含
AH	在 LAHF 指令中, 作目标寄存器	隐含
AL	在十进制运算指令中作累加器 在 XLAT 指令中作累加器	隐含
BX	在间接寻址中作基址寄存器 在 XLAT 指令中作基址寄存器	不能隐含 隐含
CX	在串操作指令和 LOOP 指令中作计数器	隐含
CL	在移位/循环移位指令中作移位次数寄存器	不能隐含
DX	在字乘法/除法指令中存放乘积高位或被除数高位或余数 在间接寻址的输入输出指令中作地址寄存器	隐含 不能隐含
SI	在字符串运算指令中作源变址寄存器 在间接寻址中作变址寄存器	隐含 不能隐含
DI	在字符串运算指令中作目标变址寄存器 在间接寻址中作变址寄存器	隐含 不能隐含
BP	在间接寻址中作基址指针	不能隐含
SP	在堆栈操作中作堆栈指针	隐含

2. 指针和变址寄存器

8086/8088 CPU 中, 有一组 4 个 16 位寄存器, 它们是基址指针寄存器 BP, 堆栈指针寄存器 SP, 源变址寄存器 SI, 目的变址寄存器 DI。这组寄存器存放的内容是某一段内地址偏移量, 用来形成操作数地址, 主要在堆栈操作和变址运算中使用。

BP 和 SP 寄存器称为指针寄存器, 与 SS 联用, 为访问现行堆栈段提供方便。通常 BP 寄存器在间接寻址中使用, 操作数在堆栈段中, 由 SS 段寄存器与 BP 组合形成操作数地址, 即 BP 中存放现行堆栈段中一个数据区的“基址”的偏移量, 所以称 BP 寄存器为基址指针。

SP 寄存器在堆栈操作中使用, PUSH 和 POP 指令是从 SP 寄存器得到现行堆栈段的段内地址偏移量, 所以称 SP 寄存器为堆栈指针, SP 始终指向栈顶。

寄存器 SI 和 DI 称为变址寄存器, 通常与 DS 一起使用, 为访问现行数据段提供段内地址偏移量。在串指令中, 其中源操作数的偏移量存放在 SI 中, 目的操作数的偏移量存放在 DI 中, SI 与 DI 的作用不能互换, 否则传送地址相反。在串指令中, SI、DI 均为隐含寻址, 此时, SI 和 DS 联用, DI 和 ES 联用。

3. 段寄存器

8086/8088 CPU 可直接寻址 1MB 的存储器空间, 直接寻址需要 20 位地址码, 而所有内部寄存器都是 16 位的, 只能直接寻址 64KB, 因此采用分段技术来解决。将 1MB 的存储空间分成若干逻辑段, 每段最长 64KB, 这些逻辑段在整个存储空间中可浮动。

8086/8088 CPU 内部设置了 4 个 16 位段寄存器, 它们分别是代码段寄存器 CS、数据段

寄存器 DS、堆栈段寄存器 SS、附加段寄存器 ES,由它们给出相应逻辑段的首地址,称为“段基址”。段基址与段内偏移地址组合形成 20 位物理地址,段内偏移地址可以存放在寄存器中,也可以存放在存储器中。

例 2-1 代码段寄存器 CS 存放当前代码段基地址, IP 指令指针寄存器存放了下一条要执行指令的段内偏移地址, 其中 CS=2000H, IP=003AH。通过组合, 形成 20 位存储单元的寻址地址为 2003AH。

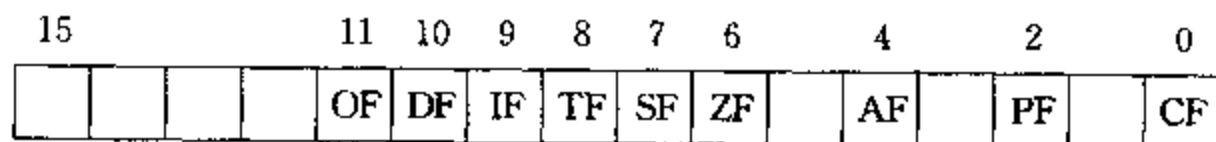
代码段内存放可执行的指令代码,数据段和附加段内存放操作的数据,通常操作数在现行数据段中,而在串指令中,目的操作数指明必须在现行附加段中。堆栈段开辟为程序执行中所要用的堆栈区,采用先进后出的方式访问它。各个段寄存器指明了一个规定的现行段,各段寄存器不可互换使用。程序较小时,代码段、数据段、堆栈段可放在一个段内,即包含在64KB之内,而当程序或数据量较大时,超过了64KB,那么可以定义多个代码段或数据段、堆栈段、附加段。现行段由段寄存器指明段地址,使用中可以修改段寄存器内容,指向其它段。有时为了明确起见,可在指令前加上段超越的前缀字节,以指定操作数所在段。

4. 指令指针寄存器

8086/8088 CPU 中设置了一个 16 位指令指针寄存器 IP, 用来存放将要执行的下一条指令在现行代码段中的偏移地址。程序运行中, 它由 BIU 自动将其修改, 使 IP 始终指向下一条将要执行的指令的地址, 因此它是用来控制指令序列的执行流程的, 是一个重要的寄存器。8086 程序不能直接访问 IP, 但可以通过某些指令修改 IP 的内容。例如, 当遇到中断指令或调用子程序指令时, 8086 自动调整 IP 的内容, 将 IP 中下一条将要执行的指令地址偏移量入栈保护, 待中断程序执行完毕或子程序返回时, 可将保护的内容从堆栈中弹出到 IP, 使主程序继续运行。在跳转指令时, 则将新的跳转目标地址送入 IP, 改变它的内容, 实现了程序的转移。

5. 标志寄存器 PSW

16 位标志寄存器 PSW 用来存放运算结果的特征,常用作后续条件转移指令的转移控制条件。其中 7 位没有用,9 个标志位分成两类:一类为状态标志,表示运算后结果的状态特征,它影响后面的操作。状态标志有 6 个:CF、PF、AF、ZF、SF 和 OF。另一类为控制标志,用来控制 CPU 操作,控制标志有 3 个:TF、IF 和 DF。PSW 寄存器的具体格式如下图所示:



(1) CF(Carry Flag)—进位标志位。本次运算中最高位有进位或借位时, $CF=1$ 。STC 指令使 CF 标志置位, CLC 指令使之复位, CMC 指令使之取反。

(2) PF(Parity Flag)—奇偶校验标志位。本次运算结果低 8 位中有偶数个“1”时, PF=1, 有奇数个“1”时, PF=0。

(3) AF(Auxiliary Carry Flag)—辅助进位标志位。本次运算结果,低 4 位向高 4 位有进位或借位时,AF=1。AF 一般用在 BCD 码运算中,判断是否需要十进制调整。

(4) ZF(Zero Flag)—全零标志位。本次运算结果为 0 时, ZF=1, 否则 ZF=0。

(5) SF(Sign Flag)—符号标志位。本次运算结果的最高位为1时, SF=1, 否则 SF=0。SF 反映了本次运算结果是正还是负。

(6) OF(Overflow Flag)—溢出标志位。本次运算过程中产生溢出时, OF=1。对带符号数, 字节运算结果的范围为-128~+127, 字运算结果的范围为-32768~+32767, 超过此范围为溢出。计算机进行加法运算时, 判断出低位向最高有效位产生进位, 而最高有效位向前无进位, 便置 OF=1; 或者相反, 当判断出低位向最高位无进位, 而最高位向前却有进位, 亦置 OF=1。在减法运算时, 当最高位需要借位, 而低位并不向最高位借位时, OF 置 1; 或者相反, 当最高位不需要借位, 而低位从最高位有借位时, OF 置 1。

例 2-2 将 5394H 与 -777FH 两数相加, 并说明其标志位状态:

$$\begin{array}{r} 0101 \ 0011 \ 1001 \ 0100 \\ + 1000 \ 1000 \ 1000 \ 0001 \\ \hline 1101 \ 1100 \ 0001 \ 0101 \end{array}$$

运算结果为-23EBH, 并置标志位为 CF=0、PF=0、AF=0、ZF=0、SF=1、OF=0。

(7) TF(Trap Flag)—单步标志位。调试程序时, 可设置单步工作方式, TF=1 时, CPU 每执行完一条指令, 就自动产生一次内部中断, 使用户能逐条跟踪程序进行调试。

(8) IF(Interrupt Flag)—中断标志位。IF=1 时, 允许 CPU 响应可屏蔽中断, 当 IF=0 时, 即使外部设备有中断申请, CPU 也不响应。由 STI 指令可使 IF 标志位置“1”。由 CLI 指令可使 IF 标志位置“0”。

(9) DF(Direction Flag)—方向标志位。控制串操作指令中地址指针变化方向, 若在串操作指令中, DF=0, 地址指针自动增量, 即由低地址向高地址进行串操作, 若 DF=1, 地址指针自动减量, 即由高地址向低地址进行串操作。由 STD 指令可使 DF 标志位置“1”, 由 CLD 指令可使 DF 标志位置“0”。

在调试程序 debug 中, 提供了测试标志位的方法, 它用符号来表示标志位的值。表 2-2 说明了各标志位在 debug 中的符号表示。(TF 在 debug 中不提供符号)

表 2-2 PSW 中标志位的符号表示

标志名		标志为 1	标志为 0
OF	溢出(是/否)	OV	NV
DF	方向(减/增量)	DN	UP
IF	中断(允许/关闭)	EI	DI
SF	符号(负/正)	NG	PL
ZF	零(是/否)	ZR	NZ
AF	辅助进位(是/否)	AC	NA
PF	奇偶(偶/奇)	PE	PO
CF	进位(是/否)	CY	NC

2-2 8086 CPU 的引脚及其功能

8086/8088 CPU 根据它的基本性能,应包括 20 条地址线,16 条数据线,加上控制信号,电源和地线,芯片的引脚比较多。但由于制造工艺的限制,8086/8088 CPU 芯片采用 40 条引脚的双列直插式封装,因此部分引脚采用了分时复用的方式。

另外 8086/8088 CPU 可以工作在两种工作模式(最小模式和最大模式),最小模式用于单机系统,系统所需要的控制信号全部由 8086 直接提供。最大模式用于多处理机系统,系统所需要的控制信号由总线控制器 8288 提供。这样,24 脚~31 脚的 8 条引脚在两种工作模式中具有不同的功能,下面简要地介绍 8086/8088 CPU 各引脚的功能。

一、8086/8088 CPU 在最小模式中引脚定义

图 2-3 给出了 8086 CPU 外部引脚图,图 2-4 给出了 8086 CPU 内部各功能部件连接的框图。

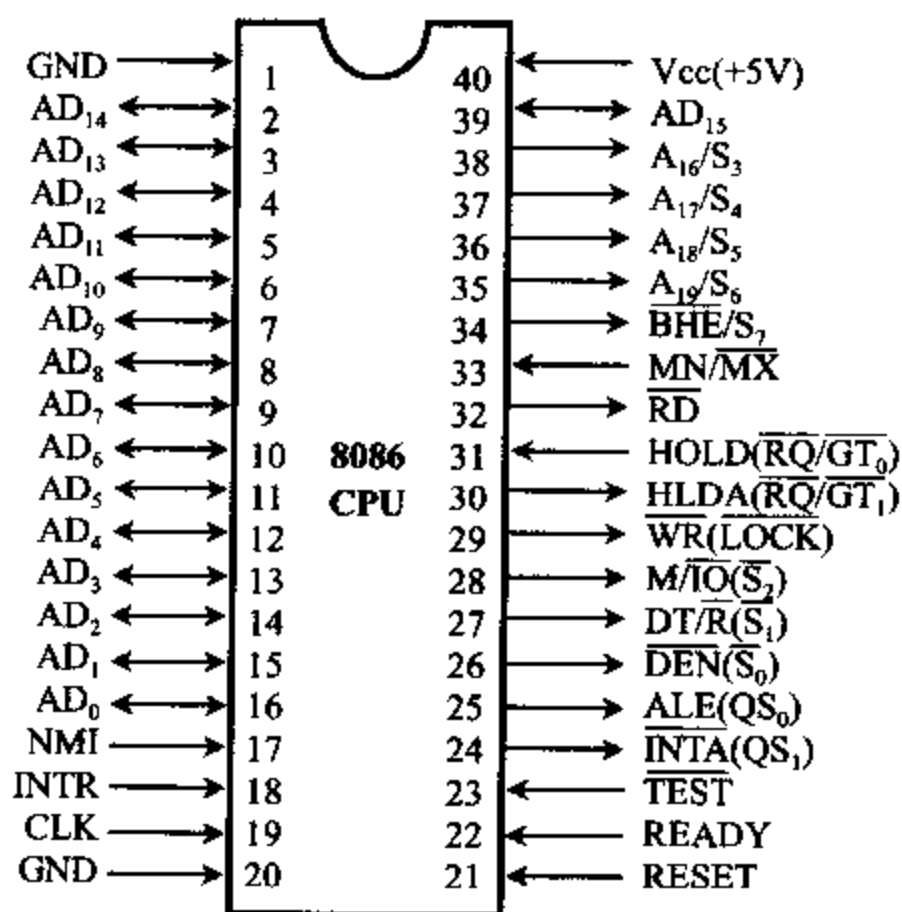


图 2-3 8086 CPU 外部引脚

1. $AD_{15} \sim AD_0$ (Address Data Bus)

16 条地址/数据总线,三态,分时复用。传送地址时输出,传送数据时双向输入/输出。在总线周期 T_1 状态,CPU 在这些引脚上输出存储器或 I/O 端口的地址,在 $T_2 \sim T_4$ 状态,用来传送数据。在中断响应及系统总线“保持响应”周期, $AD_{15} \sim AD_0$ 被置成高阻状态。

2. $A_{19}/S_6 \sim A_{16}/S_3$ (Address/Status)

地址/状态线,三态,输出,分时复用。在 T_1 状态作地址线用, $A_{19} \sim A_{16}$ 与 $A_{15} \sim A_0$ 一起构成 20 位物理地址,可访问存储器 1MB。当 CPU 访问 I/O 端口时, $A_{19} \sim A_{16}$ 为“0”。

在 $T_2 \sim T_4$ 状态作状态线用, $S_6 \sim S_3$ 输出状态信息, S_6 保持“0”,表明 8086 当前连在总线上, S_5 取中断允许标志的状态,若当前允许可屏蔽中断请求,则 S_5 置 1,若 $S_5 = 0$,则禁止一切可屏蔽中断。 S_4 、 S_3 用来指示当前正在使用哪一个段寄存器,其编码如表 2-3 所示。

当 $S_4 S_3 = 10$ 时,表示当前正在使用 CS 段寄存器对存储器寻址,或者当前正在对 I/O 端口或中断向量寻址,不需要使用段寄存器。当系统总线处于“保持响应”状态,这些引脚被置成高阻状态。

表 2-3 S_4S_3 状态编码含义

S_4	S_3	当前正在使用的段寄存器
0	0	ES
0	1	SS
1	0	CS, 或不需要使用段寄存器(I/O, INT)
1	1	DS

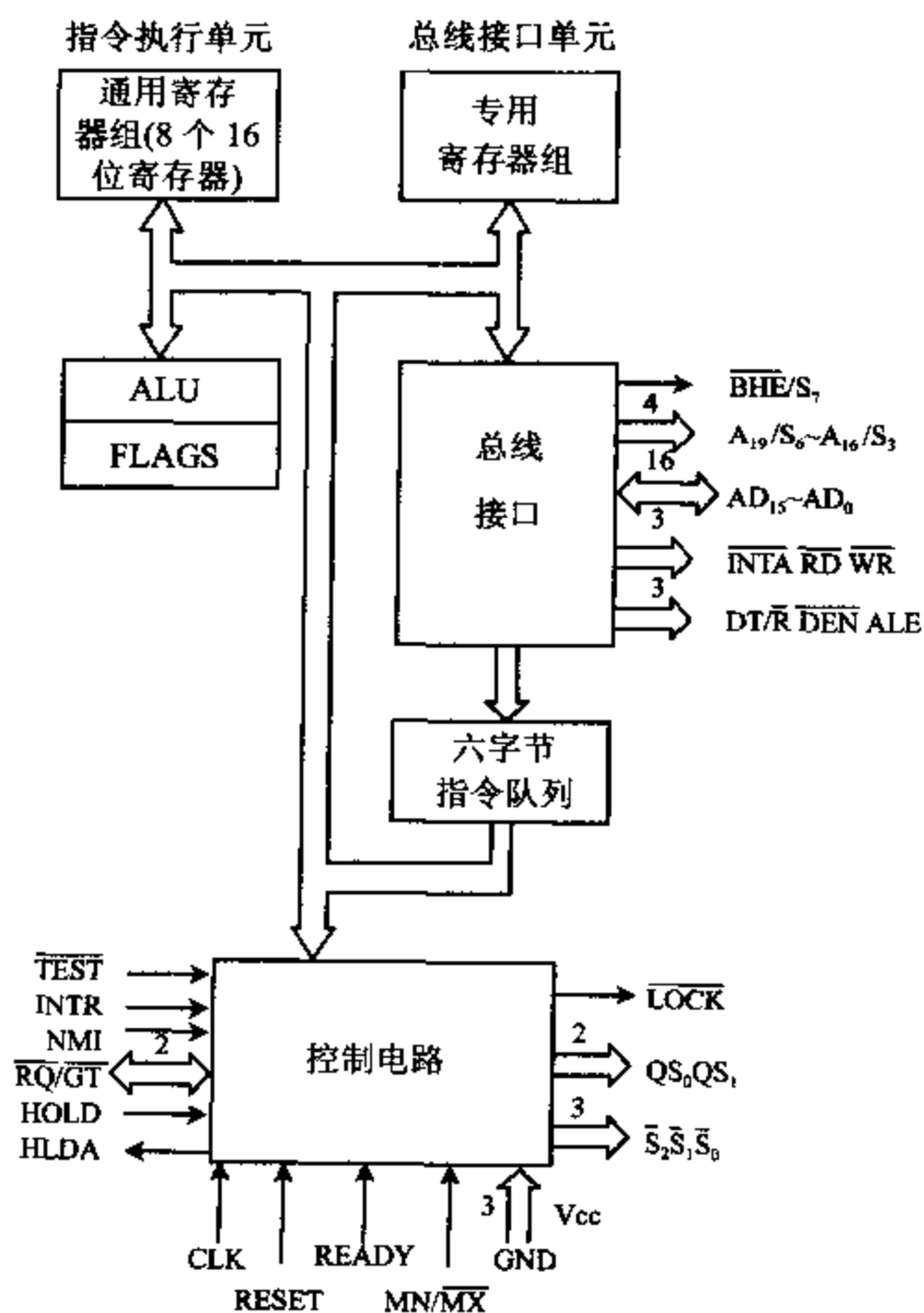


图 2-4 8086 CPU 内部功能块框图

3. $\overline{BHE}/S_7$ (Bus High Enable/Status)

高 8 位数据总线允许/状态信号，三态，输出， $\overline{BHE}$ 低电平有效。在存储器读/写，I/O 端口读/写及中断响应时，用 $\overline{BHE}$ 作高 8 位数据 $D_{15} \sim D_8$ 选通信号，即 16 位数据传送时，在 T_1 状态，用 $\overline{BHE}$ 指出高 8 位数据总线上数据有效，用 AD_0 地址线指出低 8 位数据线上数据有效。

在 $T_2 \sim T_4$ 状态， S_7 输出状态信息，（在 8086 芯片设计中， S_7 未赋予实际意义），在“保持响应”周期被置成高阻状态。

4. $\overline{MN}/\overline{MX}$ (Minimun/Maximun)

最小/最大工作模式选择信号,输入。当 $\overline{MN}/\overline{MX}$ 接 +5V 时,CPU 工作在最小模式,CPU 组成一个单处理器系统,由 CPU 提供所有总线控制信号。当 $\overline{MN}/\overline{MX}$ 接地时,CPU 工作在最大模式,CPU 的 $\overline{S_2} \sim \overline{S_0}$ 提供给总线控制器 8288,由 8288 产生总线控制信号,以支持构成多处理器系统。

5. $\overline{RD}$ (Read)

读选通信号,三态,输出,低电平有效。允许 CPU 读存储器或 I/O 端口(数据从存储器到 CPU)。由 $M/\overline{IO}$ 信号区分读存储器或读 I/O 端口,在读总线周期的 T_2 、 T_3 、 T_w 状态, $\overline{RD}$ 为低电平。在“保持响应”周期,被置成高阻状态。

6. $\overline{WR}$ (Write)

写选通信号,三态,输出,低电平有效。允许 CPU 写存储器或 I/O 端口(数据从 CPU 到存储器)。由 $M/\overline{IO}$ 信号区分写存储器或写 I/O 端口,在写总线周期的 T_2 、 T_3 、 T_w 状态, $\overline{WR}$ 为低电平,在 DMA 方式时,被置成高阻状态。

7. $M/\overline{IO}$ (Memory/Input and Output)

存储器或 I/O 端口控制信号,三态,输出。 $M/\overline{IO}$ 信号为高电平,表示 CPU 正在访问存储器, $M/\overline{IO}$ 信号为低电平,表示 CPU 正在访问 I/O 端口。一般在前一个总线周期的 T_1 状态, $M/\overline{IO}$ 有效,直到本周期的 T_1 状态为止。在 DMA 方式时, $M/\overline{IO}$ 置为高阻状态。

8. ALE(Address Latch Enable)

地址锁存允许信号,输出,高电平有效。作地址锁存器 8282/8283 的片选信号,在 T_1 状态,ALE 有效,表示地址/数据总线上传送的是地址信息,将它锁存到地址锁存器 8282/8283 中。这是由于地址/数据总线分时复用所需要的,ALE 信号不能浮空。

9. $\overline{DEN}$ (Data Enable)

数据允许信号,三态,输出,低电平有效。在最小模式系统中,有时利用数据收发器 8286/8287 来增加数据驱动能力, $\overline{DEN}$ 用作数据收发器 8286/8287 的输出允许信号,在 DMA 工作方式时,被置成高阻状态。

10. $DT/\overline{R}$ (Data Transmit/Receive)

数据发送/接收控制信号,三态,输出。 $DT/\overline{R}$ 用来控制数据收发器 8286/8287 的数据传送方向。当 $DT/\overline{R}=1$ 时,CPU 发送数据,完成写操作,当 $DT/\overline{R}=0$ 时,CPU 从外部接收数据,完成读操作。在 DMA 工作方式时, $DT/\overline{R}$ 被置成高阻状态。

11. READY(Ready)

准备就绪信号,输入,高电平有效。由存储器或 I/O 端口发来的响应信号,表示外部设备已准备好可进行数据传送了。CPU 在每个总线周期的 T_3 状态检测 READY 信号线,如果它是低电平,在 T_3 状态结束后 CPU 插入一个或几个 T_w 等待状态,直到 READY 信号有效后,才进入 T_4 状态,完成数据传送过程。

12. RESET(Reset)

复位信号,输入,高电平有效。CPU 接收到复位信号后,停止现行操作,并初始化段寄存器 DS,SS,ES,标志寄存器 PSW,指令指针 IP 和指令队列,而使 $CS=FFFFH$ 。RESET 信号至少保持 4 个时钟周期以上的高电平,当它变为低电平时,CPU 执行重新启动过程,8086/8088 将从地址 $FFFF0H$ 开始执行指令。通常在 $FFFF0H$ 单元开始的几个单元中存

放一条无条件转移指令,将入口转到引导和装配程序中,实现对系统的初始化,引导监控程序或操作系统程序。

13. INTR(Interrupt Request)

可屏蔽中断请求信号,输入,电平触发(或边沿触发),高电平有效。当外设接口向 CPU 发出中断申请时,INTR 信号变成高电平。CPU 在每条指令周期的最后一个时钟周期检测此信号,一旦检测到此信号有效,并且中断允许标志位 $IF=1$ 时,CPU 在当前指令执行完后,转入中断响应周期,读取外设接口的中断类型码,然后在存储器的中断向量表中找到中断服务程序的入口地址,转入执行中断服务程序。用 STI 指令,可使中断允许标志位 IF 置“1”,用 CLI 指令可使 IF 置“0”,从而可实现中断屏蔽。

14. $\overline{INTA}$ (Interrupt Acknowledge)

中断响应信号,输出,低电平有效。是 CPU 对外部发来的中断请求信号 INTR 的响应信号。在中断响应总线周期 T_2 、 T_3 、 T_w 状态,CPU 发出两个 $\overline{INTA}$ 负脉冲,第一个负脉冲通知外设接口已响应它的中断请求,外设接口收到第二个负脉冲信号后,向数据总线上放中断类型号。

15. NMI(Non — Maskable Interrupt Request)

不可屏蔽中断请求信号,输入,边沿触发,正跳变有效。此类中断请求不受中断允许标志位 IF 的影响,也不能用软件进行屏蔽。NMI 引脚一旦收到一个正沿触发信号,在当前指令执行完后,自动引起类型 2 中断,转入执行类型 2 中断处理程序。经常处理电源掉电等紧急情况。

16. $\overline{TEST}$ (Test)

测试信号,输入,低电平有效。在 CPU 执行 WAIT 指令期间,CPU 每隔 5 个时钟周期对 $\overline{TEST}$ 引脚进行一次测试,若测试到 $\overline{TEST}$ 为高电平,CPU 处于空转等待状态,当测试到 $\overline{TEST}$ 有效,空转等待状态结束,CPU 继续执行被暂停的指令。WAIT 指令是用来使处理器与外部硬件同步用的。

17. HOLD(Hold Request)

总线保持请求信号,输入,高电平有效。在最小模式系统中,表示其它共享总线的部件向 CPU 请求使用总线,要求直接与存储器传送数据。

18. HLDA(Hold Acknowledge)

总线保持响应信号,输出,高电平有效。CPU 一旦测试到 HLOD 总线请求信号有效,如果 CPU 允许让出总线,在当前总线周期结束时,于 T_4 状态发出 HLDA 信号,表示响应这一总线请求,并立即让出总线使用权,将三条总线置成高阻状态。总线请求部件获得总线控制权后,可进行 DMA 数据传送,总线使用完毕使 HOLD 无效。CPU 才将 HLDA 置成低电平。CPU 再次获得三条总线的使用权。

19. CLK(Clock)

时钟信号,输入。由 8284 时钟发生器产生,8086 CPU 使用的时钟频率,因芯片型号不同,时钟频率不同。8086 为 5MHz,8086-1 为 10MHz,8086-2 为 8MHz。

20. V_{CC} (+5V),GND(地)

CPU 所需电源 $V_{CC}=+5V$ 。GND 为地线。

二、8086/8088 CPU 在最大模式中引脚定义

8086 CPU 在最大模式中,24~31 引脚功能重新定义。

1. $\overline{S_2} \sim \overline{S_0}$ (Bus Cycle Status)

总线周期状态信号,三态,输出。在最大模式系统中,由 CPU 传送给总线控制器 8288,8288 译码后产生相应的控制信号代替 CPU 输出,译码状态如表 2-4 所示:

表 2-4 $\overline{S_2}$ 、 $\overline{S_1}$ 、 $\overline{S_0}$ 的编码作用

$\overline{S_2}$	$\overline{S_1}$	$\overline{S_0}$	作 用	$\overline{S_2}$	$\overline{S_1}$	$\overline{S_0}$	作 用
0	0	0	发中断响应信号	1	0	0	取指令
0	0	1	读 I/O 端口	1	0	1	读存储器
0	1	0	写 I/O 端口	1	1	0	写存储器
0	1	1	暂停	1	1	1	无源状态

无源状态:对 $\overline{S_2} \sim \overline{S_0}$ 来说,在前一个总线周期的 T_4 状态和本总线周期的 T_1 、 T_2 状态中,至少有一个信号为低电平,每种情况都对应了某种总线操作,称为有源状态。在总线周期的 T_3 、 T_w 状态,并且 READY 信号为高电平时, $\overline{S_2} \sim \overline{S_0}$ 全为高电平,此时一个总线操作过程要结束,而新的总线周期还未开始,称为无源状态。

2. $\overline{LOCK}$ (Lock)

总线封锁信号,三态,输出,低电平有效。 $\overline{LOCK}$ 有效时,CPU 不允许外部其它总线主控者获得对总线的控制权。 $\overline{LOCK}$ 信号可由指令前缀 LOCK 来设置,即在 LOCK 前缀后面的一条指令执行期间,保持 $\overline{LOCK}$ 有效,封锁其它主控者使用总线,此条指令执行完, $\overline{LOCK}$ 撤销。另外在 CPU 发出 2 个中断响应脉冲 $\overline{INTA}$ 之间, $\overline{LOCK}$ 信号也自动变为有效,以防止其它总线部件在此过程中占有总线,影响一个完整的中断响应过程。在 DMA 期间, $\overline{LOCK}$ 置于高阻状态。

3. $\overline{RQ}/\overline{GT_0}$ 、 $\overline{RQ}/\overline{GT_1}$ (Request/Grant)

总线请求信号输入/总线请求允许信号输出,双向,低电平有效。输入时表示其它主控者向 CPU 请求使用总线,输出时表示 CPU 对总线请求的响应信号,两个引脚可以同时与两个主控者相连。其中 $\overline{RQ}/\overline{GT_0}$ 比 $\overline{RQ}/\overline{GT_1}$ 有较高的优先权。

4. QS_1 、 QS_0 (Instruction Queue Status)

指令队列状态信号,输出,高电平有效。用来指示 CPU 中指令队列当前的状态,以便外部对 8086/8088 CPU 内部指令队列的动作跟踪。由 QS_1 、 QS_0 指示的指令队列含义如表 2-5 所示,亦可以让协处理器 8087 进行指令的扩展处理。

表 2-5 QS_1 、 QS_0 编码的功能

QS_1	QS_0	含 义
0	0	无操作
0	1	从指令队列中取走第一个字节
1	0	队列已空
1	1	从指令队列中取走后续字节

三、8088 与 8086 CPU 的不同之处

8088 CPU 的内部数据总线宽度是 16 位,外部数据总线宽度是 8 位,所以 8088 CPU 称为 16 位微处理器。图 2-5,图 2-6 分别指出了 8088 CPU 的外部引脚及内部结构。

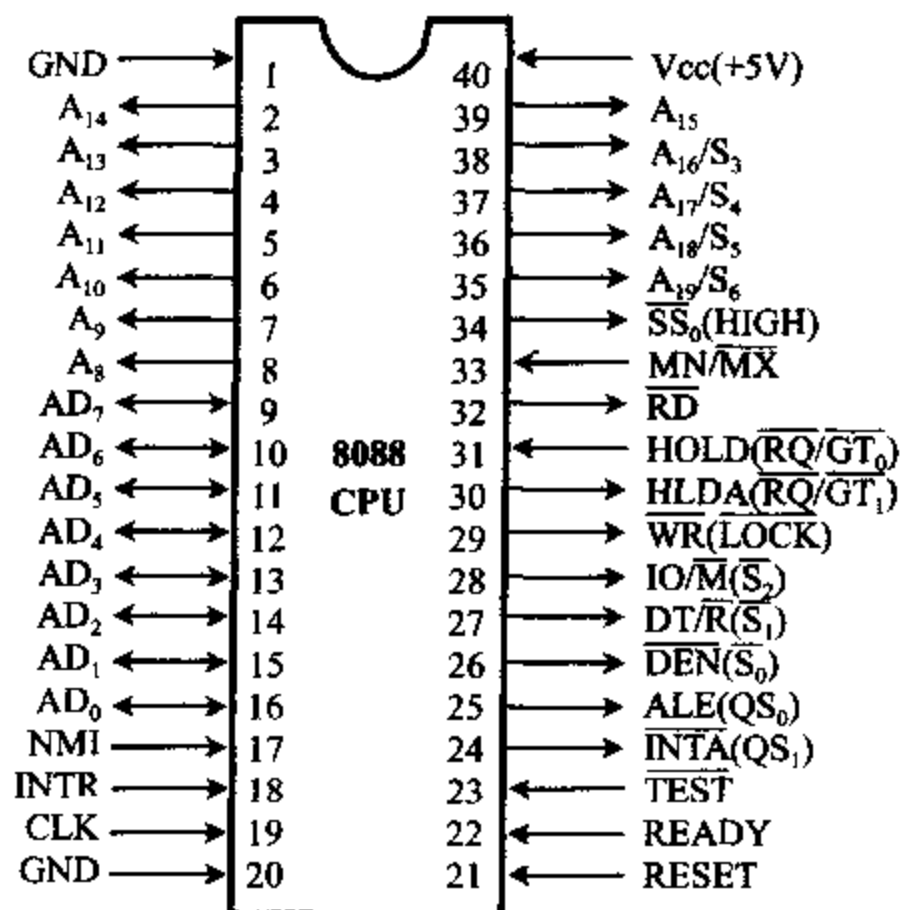


图 2-5 8088 CPU 外部引脚

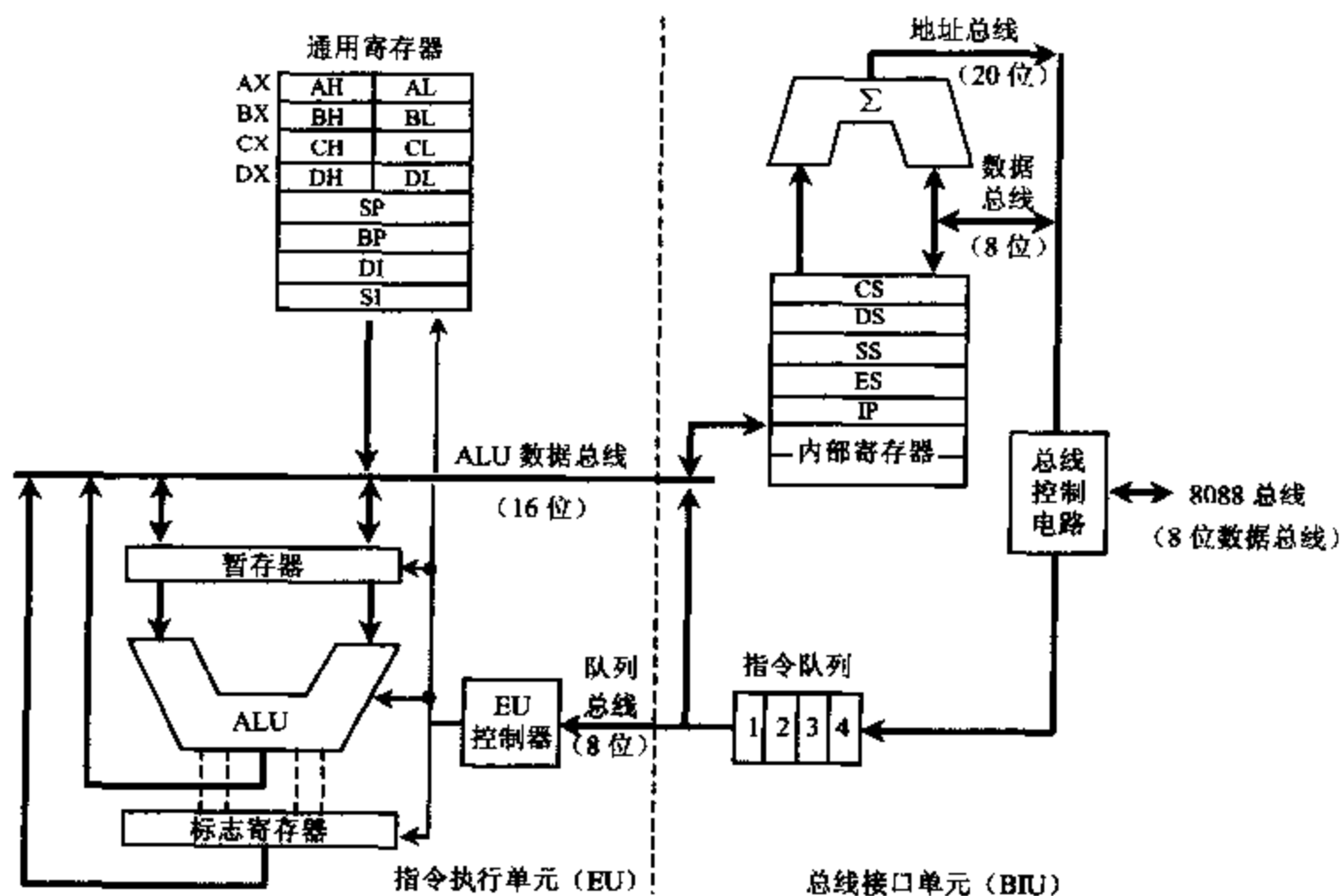


图 2-6 8088 CPU 内部结构框图

8088 CPU 的内部结构及外部引脚功能与 8086 CPU 大部分相同。主要不同之处如下:

1. 8088 的指令队列长度是 4 个字节,指令队列中只要出现一个空闲字节时,BIU 就会自动访问存储器,取指令来补充指令队列。(8086 要在指令队列中至少出现 2 个空闲字节时,才预取后续指令)。

2. 8088 CPU 中,BIU 的总线控制电路与外部交换数据的总线宽度是 8 位,总线控制电路与专用寄存器组之间的数据总线宽度也是 8 位,而 EU 的内部总线是 16 位,这样,对 16 位数的存储器读/写操作要两个读/写周期才可以完成。

3. 8088 外部数据总线只有 8 条,所以分时复用的地址/数据总线为 $AD_7 \sim AD_0$;而 $AD_{15} \sim AD_8$ 成为仅传递地址信息的 $A_{15} \sim A_8$ 。

4. 8088 中,用 $IO/\overline{M}$ 信号代替 $M/\overline{IO}$ 信号, $IO/\overline{M}$ 低电平时选通存储器,高电平时选通 I/O 接口,此举是为了与 8085 总线结构兼容。

5. 8088 中,只能进行 8 位数据传输, $\overline{BHE}$ 信号不需要了,改为 $\overline{SS_0}$,与 $DT/\overline{R}$ 、 $IO/\overline{M}$ 一起决定最小模式中的总线周期操作,表 2-6 指出了具体的组合关系。

表 2-6 8088 CPU 中 $IO/\overline{M}$ 、 $DT/\overline{R}$ 、 $\overline{SS_0}$ 组合关系

$IO/\overline{M}$	$DT/\overline{R}$	$\overline{SS_0}$	含 义
0	0	0	取指令
0	0	1	读存储器
0	1	0	写存储器
0	1	1	无源状态
1	0	0	发中断响应信号
1	0	1	读 I/O 端口
1	1	0	写 I/O 端口
1	1	1	暂停

2-3 8086 存储器组织

一、存储器地址的分段

1. 存储器地址的分段

在存储器中是以字节为单位存储信息的,每个存储单元有唯一的地址来确定。8086/8088 系统有 20 根地址线可寻址 1MB 字节的存储空间,即对存储器寻址要 20 位物理地址,而 8086 为 16 位机,CPU 内部寄存器只有 16 位,可寻址 64KB。因此 8086 系统把整个存储空间分成许多逻辑段,每段容量不超过 64KB。8086 系统对存储器的分段采用灵活的方法,允许各个逻辑段在整个存储空间中浮动,这样在程序设计时可使程序保持相对的完整性。段和段之间可以是连续的(整个存储空间分成 16 个逻辑段),也可以是分开的或重叠的。如图 2-7 所示:

任何一个存储单元的实际地址,都是由段地址及段内偏移地址两部分组成,从图 2-7 可以看出,任何一个存储单元,可以在一个段中定义,也可定义在两个重叠的逻辑段中,关键看段的首地址如何指定。IBM PC 机对段的首地址有限制,规定必须从每小段(paragraph)的首地址开始,每 16 字节为一小段,所以段起始地址必须能被 16 整除才行。

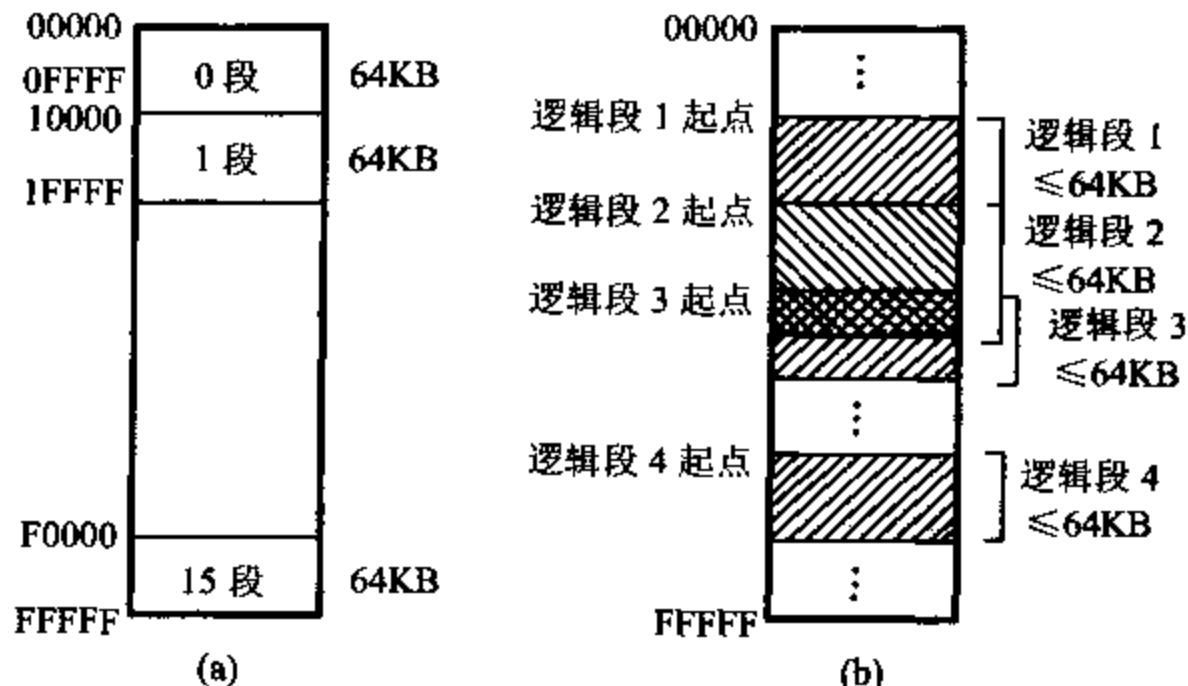


图 2-7 存储器分段示意图

2. 物理地址形成

8086 系统将段地址放在段寄存器中,称为“段基址”。有 4 个段寄存器,分别为代码段寄存器 CS,数据段寄存器 DS,附加段寄存器 ES 和堆栈段寄存器 SS。

段内“偏移地址”指出了从段地址开始的相对偏移位置。它可以放在指令指针寄存器 IP 中,或 16 位通用寄存器中,如何从 16 位段地址和 16 位偏移地址得到 20 位地址呢? 首先说明两个概念。

逻辑地址:存储器的任一个逻辑地址由段基址和偏移地址组成,都是无符号的 16 位二进制数,程序设计时采用逻辑地址。

物理地址:存储器的绝对地址,从 00000~FFFFFH,是 CPU 访问存储器的实际寻址地址,它由逻辑地址变换而来。物理地址计算如图 2-8 所示:

$$\text{物理地址} = \text{段基址} \times 16 + \text{偏移地址}$$

因为段基址指每段的起始地址,它必须是每小段的首地址,其低 4 位一定为 0,所以在实际工作时,是从段寄存器中取出段基址,将其左移 4 位,再与 16 位偏移地址相加,就得到了物理地址,此地址在 CPU 的总线接口部件 BIU 的地址加法器中形成。

3. 逻辑地址来源

逻辑地址来源如表 2-7 所示。由于访问存储器的操作类型不同,BIU 所使用的逻辑地址来源也不同,取指令时,自动选择 CS 寄存器值作段基址,偏移地址由 IP 来指定,计算出取指令的物理地址。当堆栈操作时,段基址自动选择 SS 寄存器值,偏移地址由 SP 来指定。当进行读/写存储器操作数或访问变量时,则自动选择 DS 或 ES 寄存器值作为段基址(必要时修改为 CS 或 SS),此时,偏移地址要由指令所给定的寻址方式来决定,可以是指令中包含的直接地址,可以是地址寄存器中的值,也可以是地址寄存器的值加上指令中的偏移量。注意的是当用 BP 作为基地址寻址时,段基址由堆栈寄存器 SS 提供,偏移地址从 BP 中取

得。图 2-9 指出了段寄存器与其它寄存器组合寻址存储单元的示意图。

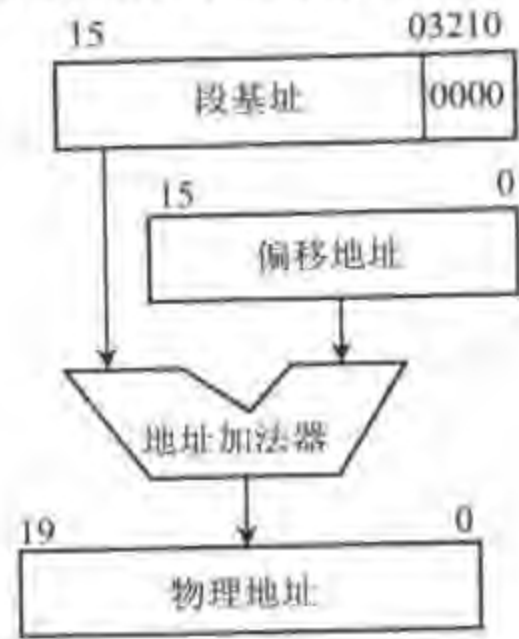


图 2-8 存储器物理地址计算

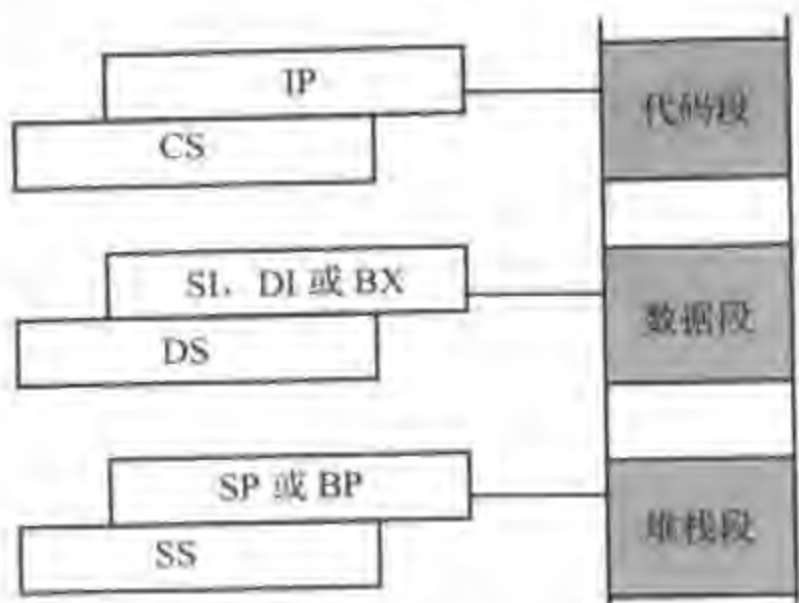


图 2-9 存储单元寻址示意图

在字符串寻址时,源操作数放在现行数据段中,段基址由 DS 提供,偏移地址由源变址寄存器 SI 取得,而目标操作数通常放在当前附加段中,段基址由 ES 寄存器提供,偏移地址从目标变址寄存器 DI 取得。

另外必须注意,在存储器地址的低端和高端,有一些专门用途的存储单元,用于中断和系统复位,用户不能占用,除非专门指定。一般情况下,各段在存储器中的分配由操作系统负责。

表 2-7 逻辑地址来源

操作类型	隐含段地址	替换段地址	偏移地址
取指令	CS	无	IP
堆栈操作	SS	无	SP
BP 为间址	SS	CS,DS,ES	有效地址 EA
存取变量	DS	CS,ES,SS	有效地址 EA
源字符串	DS	CS,ES,SS	SI
目标字符串	ES	无	DI

二、8086 存储器的分体结构

8086 系统中,1MB 的存储空间分成两个存储体:偶地址存储体和奇地址存储体,各为 512KB,示意图如图 2-10 所示:

当 $A_0=0$ 时,选择访问偶地址存储体,偶地址存储体与数据总线低 8 位相连,从低 8 位数据总线读/写一个字节。当 $\overline{BHE}=0$ 时,选择访问奇地址存储体,奇地址存储体与数据总线高 8 位相连,由高 8 位数据总线读/写一个字节。当 $A_0=0$ 、 $\overline{BHE}=0$ 时,访问两个存储体,读/写一个字。 A_0 、 $\overline{BHE}$ 功能组合如表 2-8。

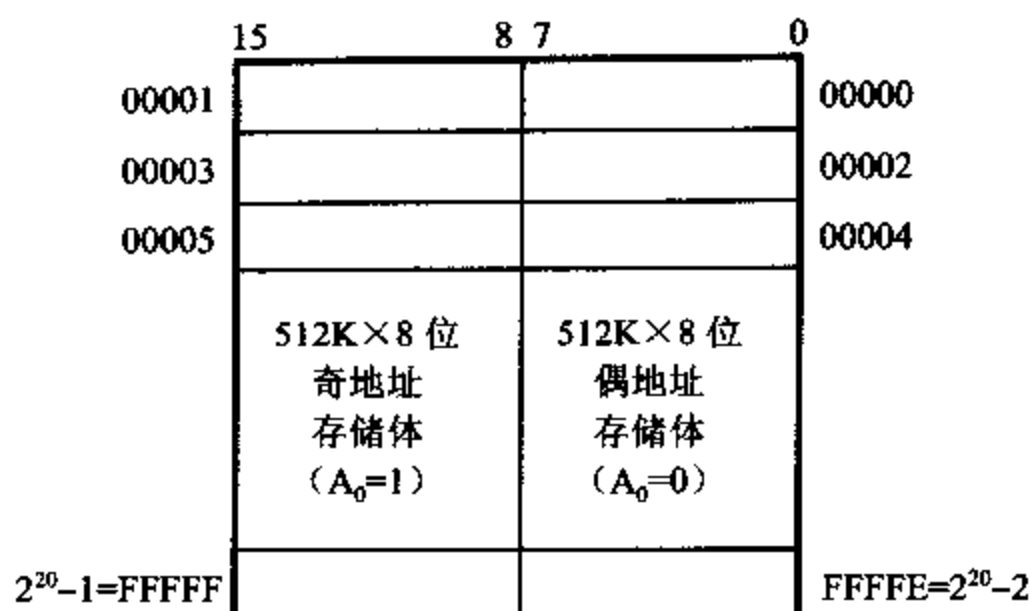
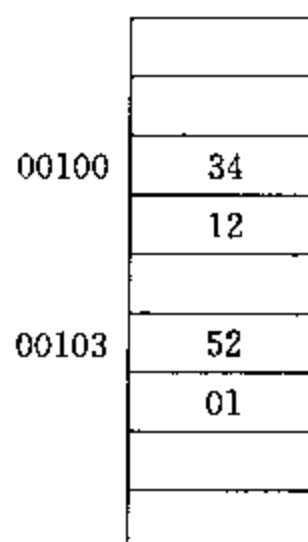


图 2-10 存储器分体结构示意图

表 2-8 $\overline{\text{BHE}}$ 、 A_0 编码含义

$\overline{\text{BHE}}$	A_0	操 作	总线使用情况
0	0	从偶地址开始读/写一个字	$AD_{15} \sim AD_0$
0	1	从奇地址单元读/写一个字节	$AD_{15} \sim AD_8$
1	0	从偶地址单元读/写一个字节	$AD_7 \sim AD_0$
1	1	无效	
0	1	从奇地址开始读/写一个字	$AD_{15} \sim AD_8$
1	0		$AD_7 \sim AD_0$

存储器中存放的信息称为存储单元的内容,例如存储单元 00100H 中的内容为 34H,表示为 (00100H) = 34H。一个字在存储器中按相邻两个字节存放,存入时以低位字节在低地址,高位字节在高地址的次序存放,字单元的地址以低位地址表示。例: (00100H) = 1234H, (00103H) = 0152H 在内存中放的位置如左图所示。从中看出,一个字可以从偶地址开始存放,也可以从奇地址开始存放,但 8086CPU 访问存储器时,都是以字为单位进行的,并从偶地址开始。若读/写一个字节,只启动某个存储体,只有相应的 8 位数据在数据总线上有效,即启动偶地址存储体,低 8 位数据线有效,或启动奇地址存储体,高 8 位数据线有效,另外 8 位数据被忽略了,图 2-11 (a)、(b)给出了示意图。



当 CPU 读/写一个字时,若字单元地址从偶地址开始,只需访问一次存储器,低位字节在偶地址单元,高位字节在奇地址单元。若字单元地址从奇地址开始,CPU 要两次访问存储器,第一次取奇地址上数据(偶地址 8 位数据被忽略),第二次取偶地址上数据(奇地址 8 位数据被忽略),图 2-11(c)、(d)给出了示意图。因此为了加快程序运行速度,编程时注意从存储器偶地址开始存放字数据,这种存放方式也称“对准存放”。

在 8088 CPU 系统中,外部数据线为 8 位,CPU 每次访问存储器只读/写一个字节,读/写一个字要两次访问存储器,整个存储器 1MB 看成一个存储体,由 $A_0 \sim A_{19}$ 直接寻址,无

需由 $\overline{BHE}$ 、 A_0 来选择高 8 位与低 8 位数据,整个系统的运行速度也慢些。

图 2-12 指出了 8086 系统和 8088 系统中存储器与总线的连接。

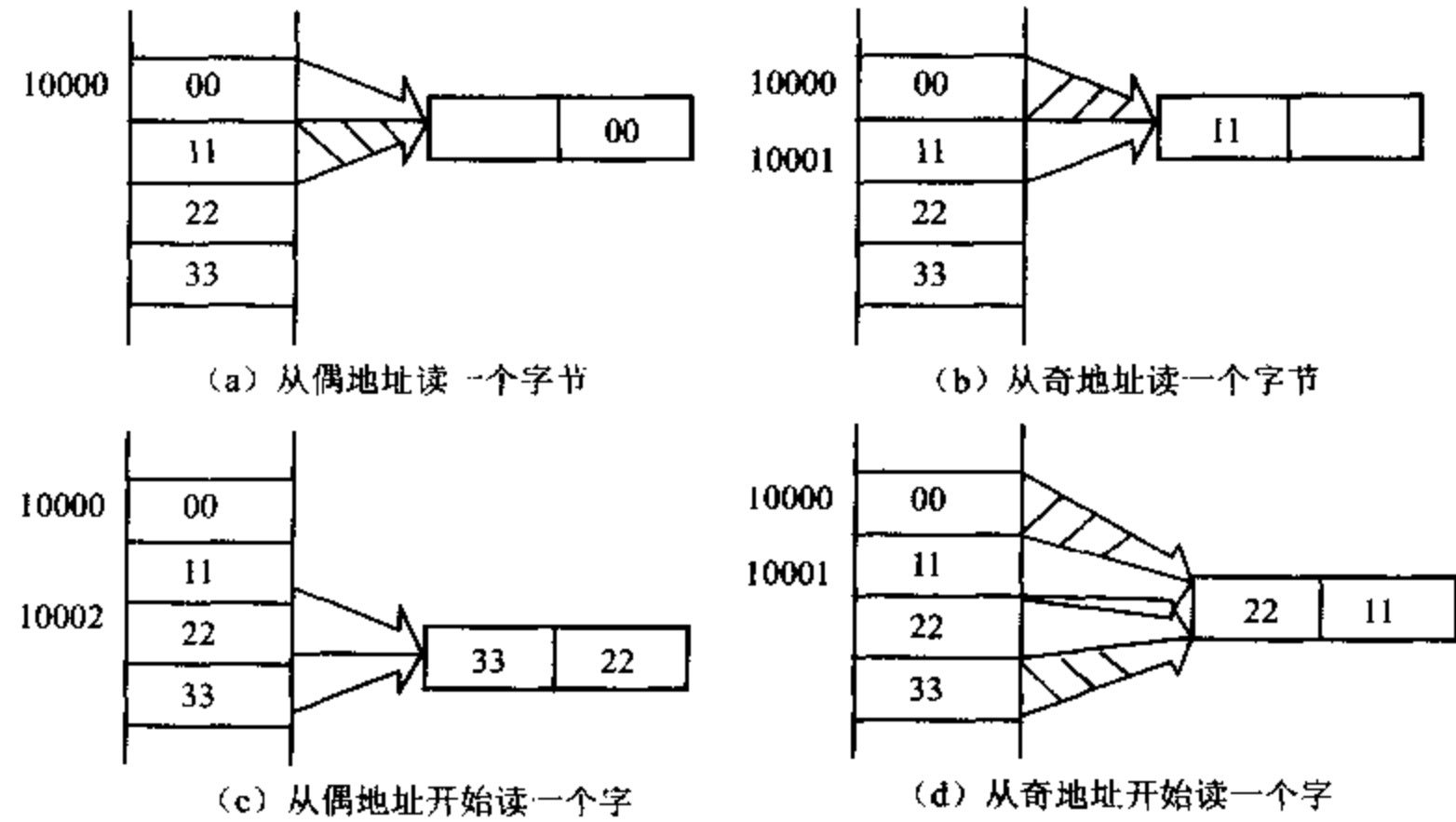


图 2-11 读存储器示意图

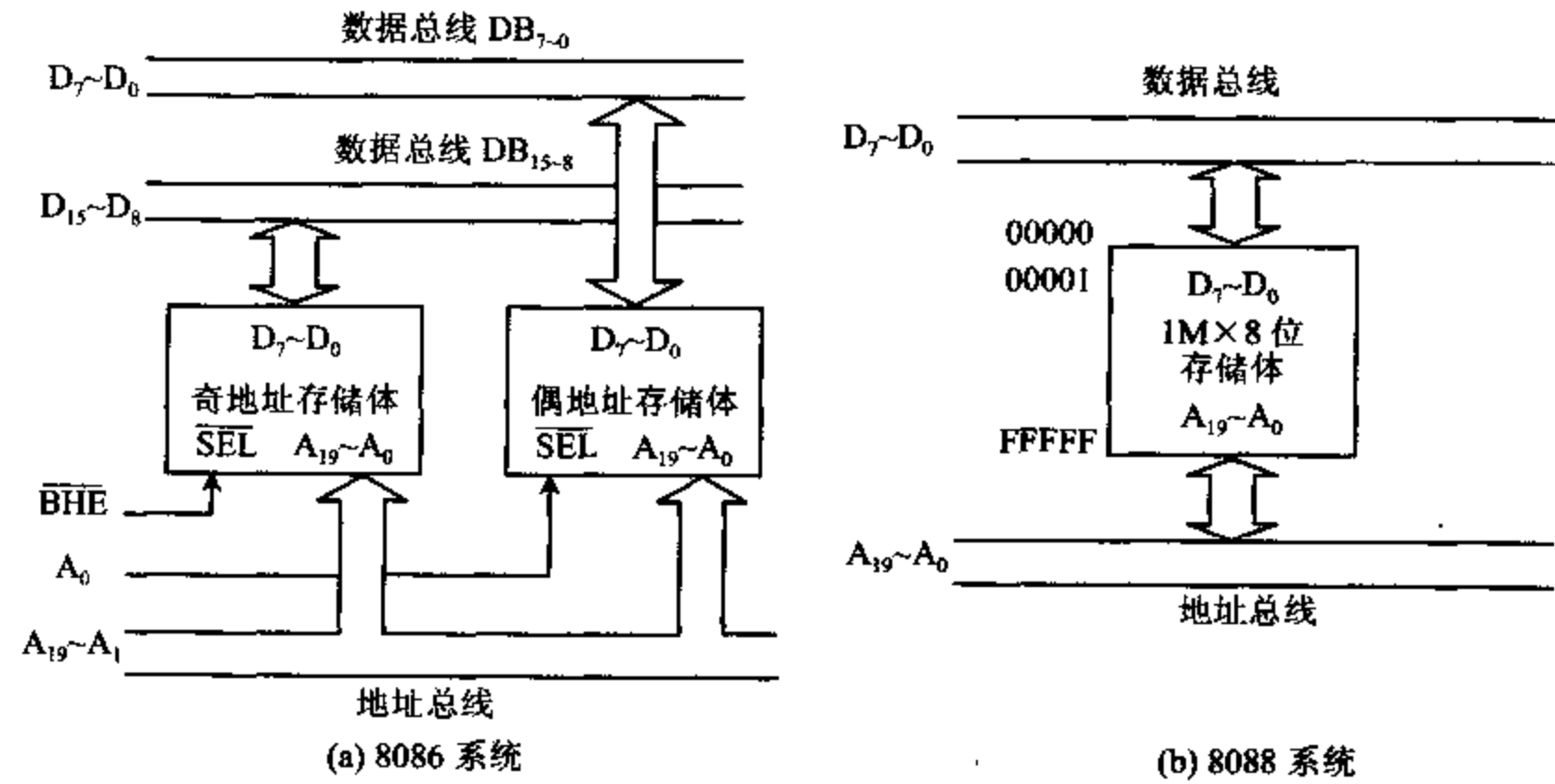


图 2-12 8086/8088 系统中存储器与总线的连接图

三、堆栈的概念

所谓堆栈是在存储器中开辟一个区域,用来存放需要暂时保存的数据。堆栈段是由段定义语句在存储器中定义的一个段,它可以在存储器 1MB 空间内任意浮动,堆栈段容量小于等于 64KB。段基址由堆栈寄存器 SS 指定,栈顶由堆栈指针 SP 指定,根据堆栈构成方式

不同,堆栈指针 SP 指向的可以是当前栈顶单元,也可以是栈顶上的一个“空”单元,一般采用 SP 指向当前栈顶单元。堆栈的地址增长方式一般是向上增长,栈底设在存储器的高地址区,堆栈地址由高向低增长。

例 2-3 假如当前 $SS=C000H$, 堆栈段 $<64KB$, $SP=1000H$, 指出当前栈顶在存储器中的位置。

当前栈顶在存储器中的地址为 $C1000H$ 。如图 2-13(a)所示:

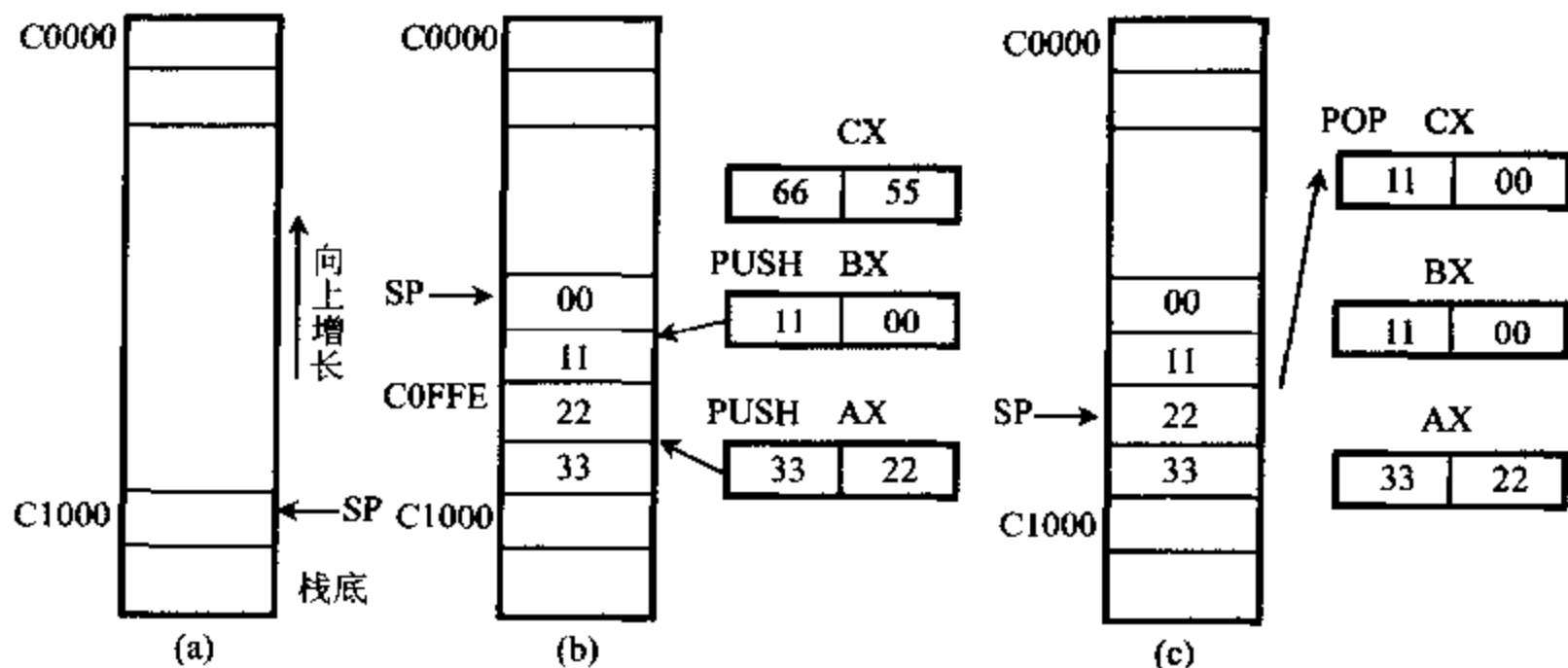


图 2-13 堆栈操作示意图

堆栈的工作方式是“先进后出”,用入栈指令 PUSH 和出栈指令 POP 可将数据压入堆栈或从堆栈中弹出数据,栈顶指针 SP 的变化由 CPU 自动管理。堆栈以字为单位进行操作,堆栈中的数据项以低字节在偶地址,高字节在奇地址的次序存放。当执行 PUSH 指令时, CPU 自动修改指针 $SP-2 \rightarrow SP$ 。使 SP 指向新栈顶,然后将低位数据压入 (SP) 单元,高位数据压入 $(SP+1)$ 单元。当执行 POP 指令时, CPU 先将当前栈顶 SP(低位数据)和 $SP+1$ (高位数据)中的内容弹出,然后再自动修改指针,使 $SP+2 \rightarrow SP$, SP 指向新栈顶。

例 2-4 上例中若 $AX=3322H$, $BX=1100H$, $CX=6655H$, 执行指令 PUSH AX, PUSH BX, 再执行指令 POP CX, 此时堆栈中内容发生什么变化, AX, BX, CX 中的内容是什么?

执行指令 PUSH AX, 则 $SP-2 \rightarrow SP$, 栈顶 SP 指向内存地址 $C0FFE H$, 数据 3322 分别压入堆栈单元 $C0FF F H$ 及 $C0FF E H$ 之中, 如图 2-13(b)所示。再执行指令 PUSH BX, 此时栈顶 SP 指向 $C0FF C H$, 数据 1100 分别压入堆栈单元 $C0FF D H$ 和 $C0FF C H$ 之中, 如图 2-13(b)所示。若再执行指令 POP CX, 数据 1100 弹到 CX 中, 栈顶指针指向 $C0FF E H$, 如图 2-13(c)所示。

堆栈主要用于中断及子程序调用,也可用于数据暂时保存。在进入中断服务子程序和子程序调用前,原来 CPU 中现行信息(指令指针 IP, 及寄存器中有关内容)都必须保存,在中断服务子程序和子程序调用结束返回主程序时,又必须恢复原来保存的信息,这些均由堆栈操作来完成。其中指令指针的入栈和出栈由 CPU 自动管理,而一些寄存器中内容的保存及返回,需要用户自己利用指令 PUSH、POP 来完成。由于堆栈操作的先进后出的特点,一定要注意两点:

1. 先进入的内容要后弹出,保证返回寄存器内容不发生错误。

例 2-5

```
PUSH    AX
PUSH    BX
PUSH    CX
POP     CX
POP     AX
POP     BX
```

可引起 AX、BX、CX 原来保存的内容改变,AX 与 BX 内容发生交换。

2. PUSH 和 POP 的指令要成对,若不匹配的话,会造成返回主程序的地址出错。

例 2-6

```
PUSH    AX
PUSH    BX
PUSH    CX
:
POP     CX
POP     BX
RET
```

由于少弹出一组数,会使 CPU 返回主程序时,返回地址取出的是原来 AX 中的内容,使整个程序执行出错。

2-4 8086 系统配置

根据使用目的不同,8086/8088 系统可以有最小模式和最大模式两种系统配置方式,两种方式的选择不是由程序进行控制的,而是由硬件设定的。当 CPU 的引脚 $\overline{MN}/\overline{MX}$ 端接高电平 +5V 时,构成最小模式,当 $\overline{MN}/\overline{MX}$ 接低电平时,构成最大模式。最小模式为单机系统,系统所需要的控制信号由 CPU 提供,实现和存储器及 I/O 接口电路的连接。最大模式可以构成多处理器/协处理器系统,即一个系统中存在两个以上微处理器,每个处理器执行自己的程序,常用的处理器有数值运算协处理器 8087,输入/输出处理器 8089。系统所需要的控制信号由总线控制器 8288 提供,8086 CPU 提供信号控制 8288,以实现全局资源分配及总线控制权传递。在两种模式中,8086/8088 CPU 的 24~31 引脚意义不同。表 2-9 说明了最小模式和最大模式特点,表 2-10 给出了 8086/8088 CPU 在两种模式下,引脚的不同意义。具体引脚说明在第二节中已叙述过。

表 2-9 最小模式和最大模式的特点

最小模式	最大模式
$\overline{MN}/\overline{MX}$ 接 +5V	$\overline{MN}/\overline{MX}$ 接地
构成单处理器系统	构成多处理器系统
系统控制信号由 CPU 提供	系统控制信号由总线控制器 8288 提供

表 2-10 8086/8088 CPU 在两种模式中的引脚名称

引脚编号	8086		8088	
	最小模式	最大模式	最小模式	最大模式
24	$\overline{\text{INTA}}$	QS_1	$\overline{\text{INTA}}$	QS_1
25	ALE	QS_0	ALE	QS_0
26	$\overline{\text{DEN}}$	$\overline{\text{S}}_0$	$\overline{\text{DEN}}$	$\overline{\text{S}}_0$
27	$\text{DT}/\overline{\text{R}}$	$\overline{\text{S}}_1$	$\text{DT}/\overline{\text{R}}$	$\overline{\text{S}}_1$
28	$\text{M}/\overline{\text{IO}}$	$\overline{\text{S}}_2$	$\text{IO}/\overline{\text{M}}$	$\overline{\text{S}}_2$
29	$\overline{\text{WR}}$	$\overline{\text{LOCK}}$	$\overline{\text{WR}}$	$\overline{\text{LOCK}}$
30	HLDA	$\overline{\text{RQ}}/\overline{\text{GT}}_1$	HLDA	$\overline{\text{RQ}}/\overline{\text{GT}}_1$
31	HOLD	$\overline{\text{RQ}}/\overline{\text{GT}}_0$	HOLD	$\overline{\text{RQ}}/\overline{\text{GT}}_0$
34	$\overline{\text{BHE}}/\text{S}_7$		SS_0	高阻

一、最小模式系统

8086 CPU 构成的最小模式系统的典型配置如图 2-14 所示：

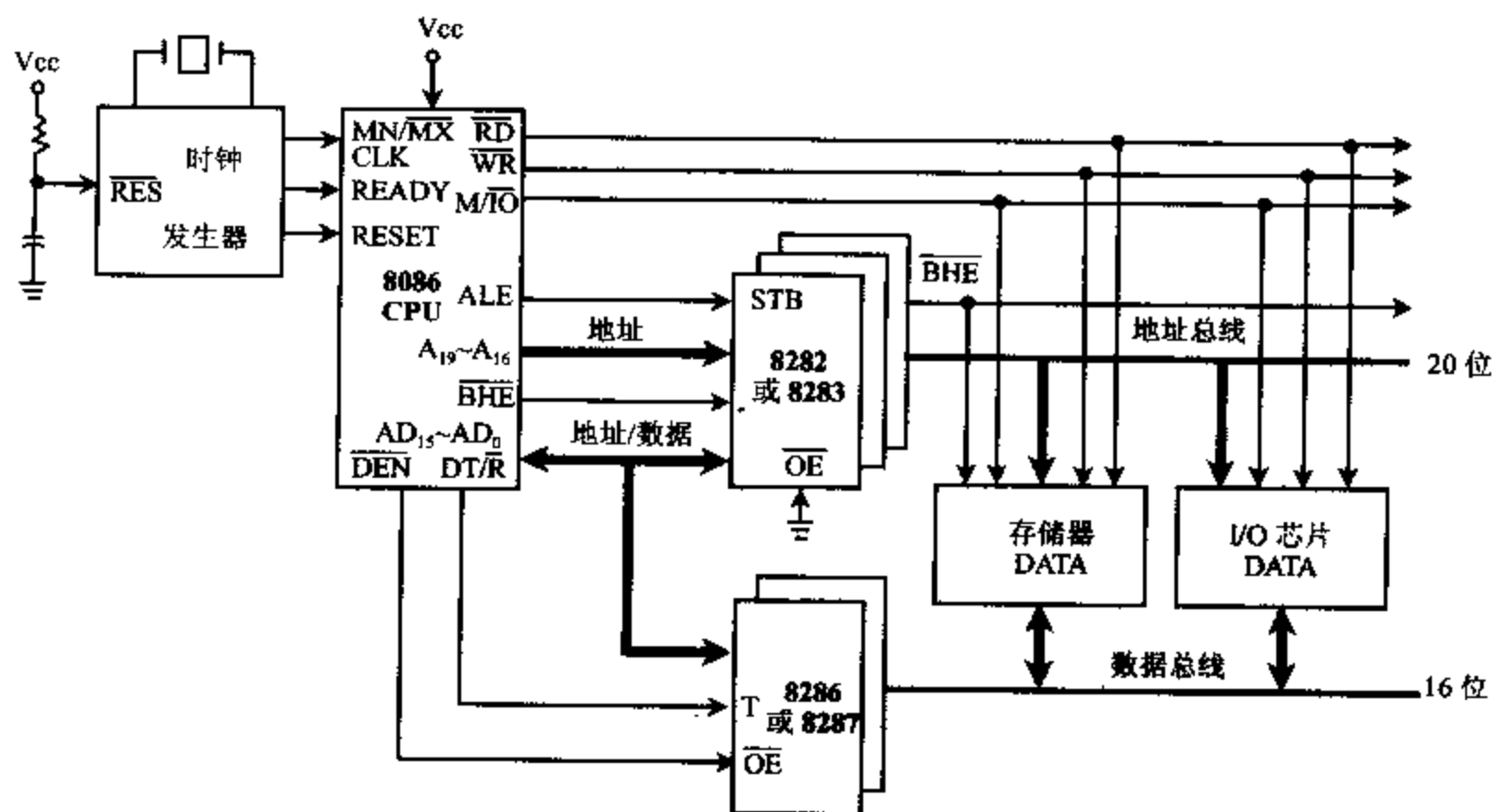


图 2-14 8086 最小模式系统配置

在最小模式系统中，除了 8086 CPU，存储器及 I/O 接口芯片外，还要加入

- 1 片 8284A，作为时钟发生器
- 3 片 8282/8283 或 74LS373，作为地址锁存器
- 2 片 8286/8287 或 74LS245，作为双向数据总线收发器

才能配置成一个系统。

1. 地址锁存器 8282/8283

CPU 与存储器（或 I/O 端口）进行数据交换时，CPU 首先要送出地址信号，然后再发出控制信号及传送数据。由于 8086 引脚限制，地址和数据分时复用一组总线，所以要加入地

址锁存器,先锁存地址,使在读/写总线周期内地址稳定。

8282/8283 是三态缓冲的 8 位数据锁存器,8282 的输入和输出信号是同相的,引脚及单元结构如图 2-15 所示,8283 的输入和输出信号反相。

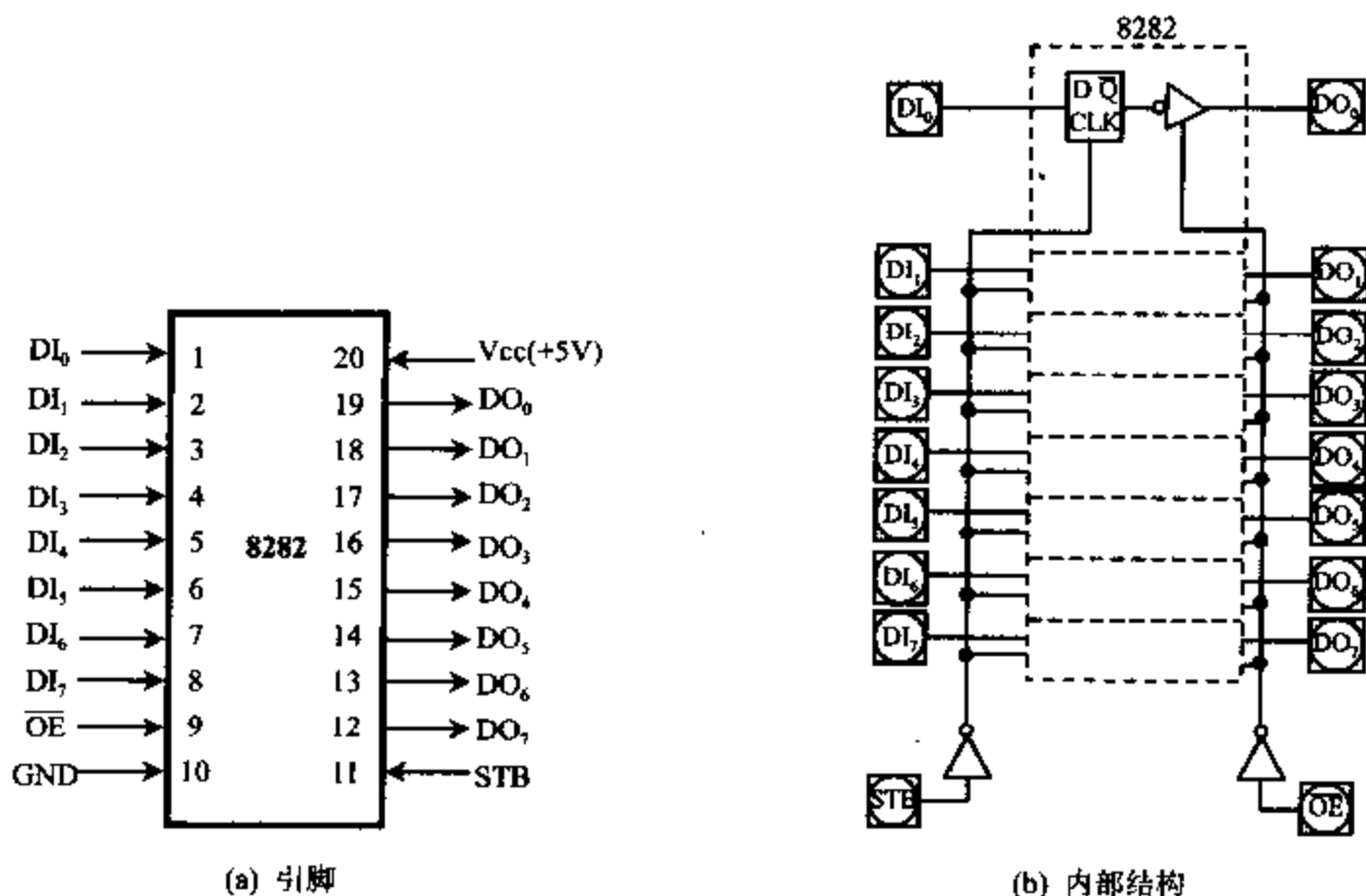


图 2-15 8282 引脚及内部结构图

$DI_7 \sim DI_0$: 8 位数据输入;

$DO_7 \sim DO_0$: 8 位数据输出;

STB: 选通信号。

$\overline{OE}$: 输出允许信号。

STB 是选通信号,与 CPU 的地址锁存允许信号 ALE 相连,当 STB 端选通信号出现,8 位输入数据锁存到 8 个 D 触发器中。 $\overline{OE}$ 是输出允许信号,由外部输入的控制信号,当 $\overline{OE}$ 为低电平时,锁存器中的 8 位数据输出送到数据总线上,当 $\overline{OE}$ 为高电平时,输出端呈高阻状态,在不带 DMA 控制器的 8086 单处理器系统中, $\overline{OE}$ 信号接地。

8282/8283 在最小模式系统中作地址锁存器用,20 位物理地址需要用 3 片。CPU 在读/写总线周期的 T_1 状态把 20 位地址和 $\overline{BHE}$ 信号送到总线上,在地址锁存允许信号 ALE 有效时,将地址和 $\overline{BHE}$ 锁存到 8282/8283 锁存器中,由于 $\overline{OE}$ 引脚接地,使 CPU 输出的地址码(锁存在 8282 中)和 $\overline{BHE}$ 信号稳定地输出到地址总线及控制总线上。74LS373 的功能与 8282 相同,在 IBM PC/XT 的系统板中作地址锁存器。

2. 双向数据总线收发器 8286/8287

8086 CPU 驱动数据的负载能力有限,当挂在数据总线上的部件增加时,可以利用双向数据总线收发器 8286/8287 来增加驱动能力。8286/8287 是三态 8 位双向数据收发器,8286 数据输入与输出同相,引脚及单元结构如图 2-16 所示,8287 数据输入与输出反相。

其中: $A_7 \sim A_0$: 输入/输出数据线;

$B_7 \sim B_0$: 输入/输出数据线;

$\overline{OE}$: 输出允许信号。

T:控制数据传送方向。

$\overline{OE}$ 是输出允许信号,控制数据收发器的开启,当 $\overline{OE}=0$ 时,允许数据通过 8286,当 $\overline{OE}=1$ 时,禁止数据通过 8286,输出呈高阻状态。在 8086/8088 系统中, $\overline{OE}$ 端与 CPU 的数据允许信号 $\overline{DEN}$ 端相连,控制 CPU 与存储器或 I/O 端口进行数据交换允许或禁止。

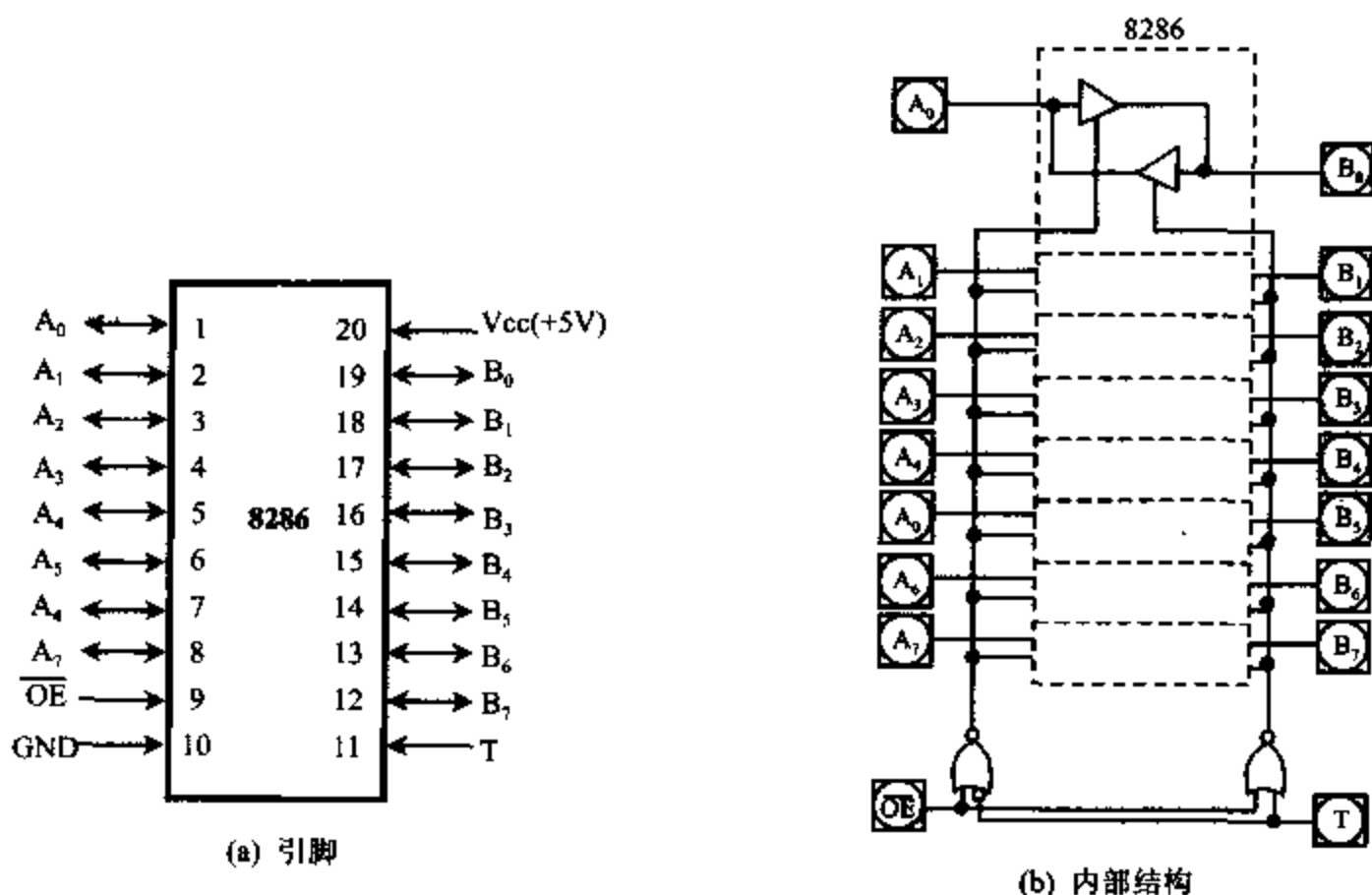


图 2-16 8286 引脚及内部结构图

T 信号控制数据传送方向,当 $T=1$ 时,8 位数据从 $A_7 \sim A_0$ 传送到 $B_7 \sim B_0$,当 $T=0$ 时,8 位数据反向传送,从 $B_7 \sim B_0$ 传送到 $A_7 \sim A_0$ 。T 端与 CPU 的数据发送/接收信号端 $DT/\overline{R}$ 相连,控制 8 位数据从 CPU 向存储器或 I/O 端口写入($DT/\overline{R}=1$),还是数据由存储器或 I/O 端口向 CPU 读出($DT/\overline{R}=0$)。 $\overline{OE}$ 与 T 的控制作用如表 2-11 所示:

表 2-11 $\overline{OE}$ 与 T 的功能

$\overline{OE}$	T	传送方向
0	1	$A_i \rightarrow B_i$ (CPU $\rightarrow$ 外部)
0	0	$B_i \rightarrow A_i$ (外部 $\rightarrow$ CPU)
1	1	高阻状态
1	0	高阻状态

3. 时钟发生器 8284

8086 CPU 的内部和外部的时间基准信号由时钟输入信号 CLK 提供,CLK 信号是由外部时钟发生器 8284 产生,并由驱动门电路进行功率放大。图 2-17 给出了时钟发生器 8284 的引脚及内部结构框图。

(1) 8284 引脚定义:

① $\overline{AEN}_1$ 、 $\overline{AEN}_2$:地址允许信号,输入,低电平有效。用来控制相应的总线准备好信号 RDY1 和 RDY2,在多总线管理时,使用这两个信号,在单总线管理方式时,这两个引脚接地。

② RDY1、RDY2:总线准备好信号,输入,高电平有效。由系统数据总线上的某个设备输入,指示数据已收到,或表示数据已准备好可供使用,分别受 $\overline{AEN}_1$ 、 $\overline{AEN}_2$ 控制。

③ READY:准备好信号,输出,高电平有效,由 RDY 同步后得到。

④ X1、X2:晶体连接端,输入。

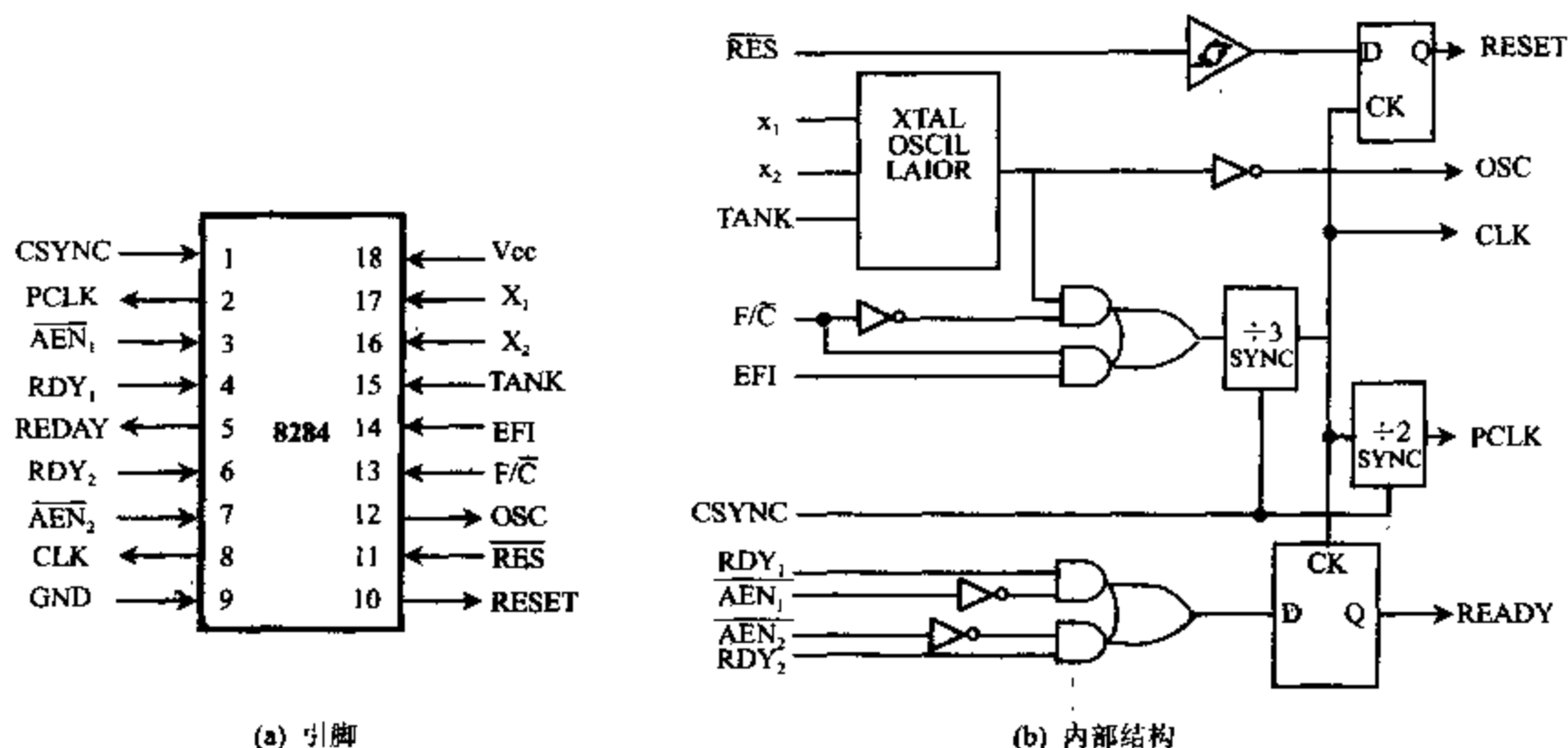


图 2-17 8284 引脚及内部结构框图

⑤ $F/\bar{C}$:频率/晶体选择端,输入,作工作方式选择用。当 $F/\bar{C}$ 为低电平时,CPU 所需时钟由晶体产生。当 $F/\bar{C}$ 为高电平时,CLK 由 EFI 输入产生。

⑥ EFI:外加频率输入端,输入。当 $F/\bar{C}=1$ 时,CLK 由输入的方波信号产生,输入信号的频率为 CLK 时钟频率的 3 倍。

⑦ CLK:时钟信号,输出。CLK 为提供给 CPU 或与总线相连的其它设备的时钟信号,CLK 的频率为晶体频率或 EFI 输入频率的 1/3,占空度为 1/3,输出电平与 MOS 电路的电平相兼容。

⑧ PCLK:供外设用的时钟,输出。输出频率为 CLK 频率的 1/2,占空度为 1/2,输出电平为 TTL 电平。

⑨ OSC:振荡器输出信号,输出。输出频率等于晶体频率,输出电平为 TTL 电平。

⑩ $\overline{RES}$:复位输入信号,输入,低电平有效。用它来产生复位信号 RESET。

⑪ RESET:复位输出信号,输出,高电平有效,用作 8086 系列 CPU 的复位信号。

⑫ CSYNC:时钟同步信号。输入,高电平有效。它使多个 8284 同步,当 CSYNC 为高电平时,内部计数器复位,当 CSYNC 为低电平时,内部计数器重新计数,在采用内部振荡器时,CSYNC 接地。

⑬ V_{cc} :电源接+5V、GND:接地端。

(2) 8284 时钟发生器的功能:

8284 时钟发生器包括 3 部分电路。

① 时钟信号发生器

$F/\bar{C}=0$ 时,时钟信号输入由 X_1 、 X_2 端接上晶体,由晶体振荡器产生时钟信号。

$F/\bar{C}=1$ 时,由 EFI 端接入外加振荡信号产生时钟信号。

时钟信号输出:IBM PC/XT 机中, $F/\bar{C}$ 接低电平, X_1 、 X_2 端接晶体,晶体振荡器工作频

率为 14.318MHz, 并产生三组时钟信号供 CPU 及外设使用。

OSC: 晶体振荡器工作频率 14.318MHz。

CLK: 3 分频 OSC 后的时钟, 输出频率 4.77MHz, 占空比为 1/3, 供 8088CPU 使用。

PCLK: 2 分频 CLK 后的时钟, 输出频率 2.385MHz, TTL 电平, 占空比为 1/2, 供 PC/XT 机的外设使用。OSC、CLK、PCLK 三者之间的关系如图 2-18 所示。

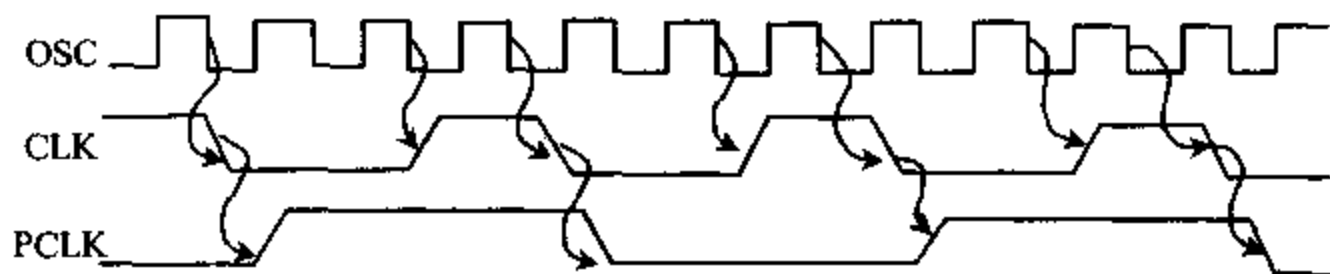


图 2-18 OSC、CLK、PCLK 三者之间的关系

② 复位生成电路

复位生成电路由一个施密特触发器和一个同步触发器组成, 由 $\overline{RES}$ 输入的信号来触发同步触发器, 由此产生复位信号 RESET, 送到 CPU 的 RESET 端, 复位信号由 CLK 的下降沿同步。在 PC/XT 机中, $\overline{RES}$ 端接“电源好”信号, 使系统上电自动复位。

③ 就绪控制电路

它由两个 D 触发器和一些门电路组成, 就绪控制信号有两组输入信号 RDY1、RDY2, 分别由地址允许信号 $\overline{AEN}_1$ 、 $\overline{AEN}_2$ 来控制, CSYNC 输入端规定了就绪信号同步操作的两种方式。外界的准备就绪信号 RDY 输入 8284, 经就绪控制电路同步, 输出准备好信号 READY, 在 CLK 下降沿处使 READY 信号有效。在 PC/XT 机中, CSYNC 接地, 同时, PC/XT 机只使用一组 $\overline{AEN}$ 、RDY, 另一组 $\overline{AEN}$ 接 +5V, RDY 接地。图 2-19 给出了 8284 和 8086 CPU 的连接图。

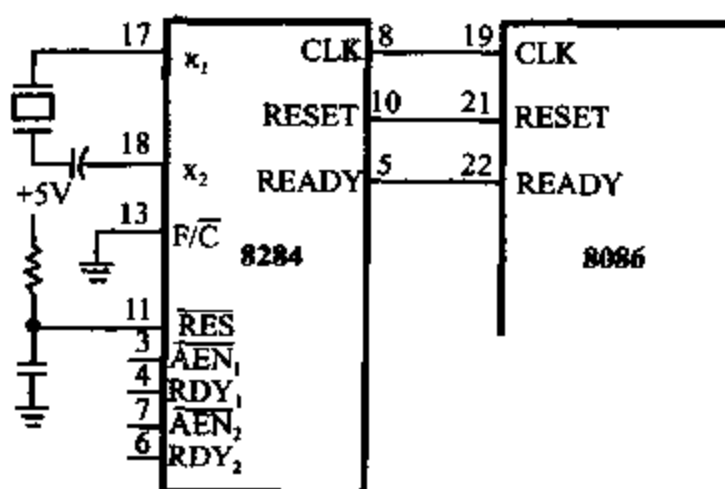


图 2-19 8284 时钟发生器与 8086 的连接

8086 最小模式系统允许接入其它要求共享总线的设备, 例 DMA 控制器 8237 芯片。8237 通过 HOLD 引脚向 CPU 发出总线请求信号, CPU 识别 HOLD 信号有效时, 在当前总线周期结束后, 发出总线响应信号, 使 HLDA 信号变成有效, 让出总线控制权由 DMA 控制器控制, 使外设与存储器之间直接进行数据传送。DMA 控制的数据传送结束, 释放系统总线, HOLD 信号变为低电平, CPU 重新获得总线使用权, 下一个总线周期继续操作。

二、最大模式系统

CPU 的引脚 $\overline{MN}/\overline{MX}$ 接地时, 8086 为最大模式系统, 在最大模式系统中需要增加总线控制器 8288 和总线裁决器 8289, 以完成 8086 CPU 为中心的多处理器系统的协调工作。此时 CPU 输出的状态信号 $\overline{S}_2 \sim \overline{S}_6$ 同时送给 8288 和 8289, 由 8288 输出原 CPU 所有的控制信

号;存储器读/写控制,I/O 端口读/写控制,中断响应信号等。由 8289 来裁决总线使用权赋给哪个处理器,以实现多主控者对总线资源的共享。图 2-20 给出了 8086 CPU 最大模式系统配置。

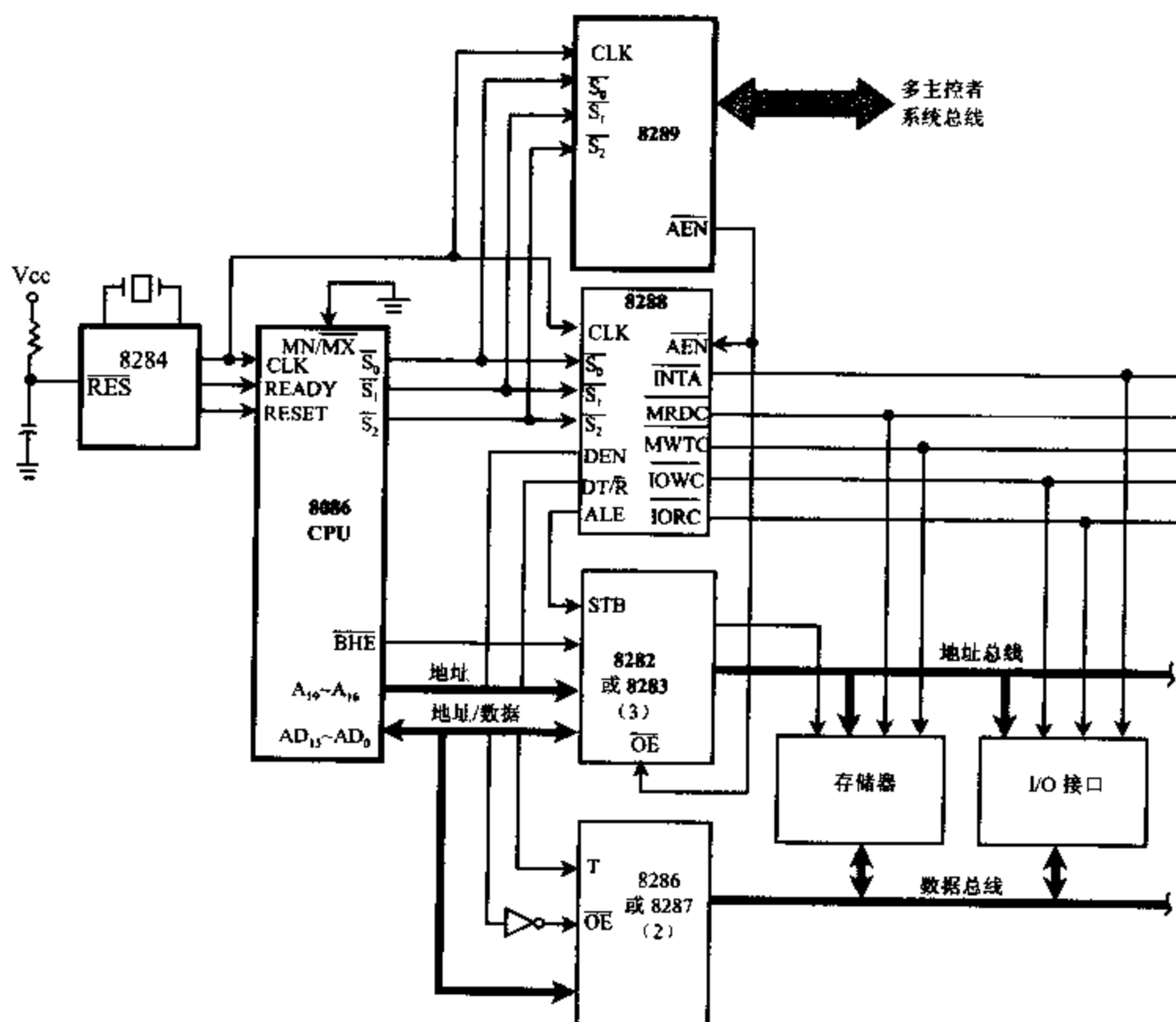


图 2-20 8086 最大模式系统配置

总线控制器 8288

总线控制器 8288 是 20 个引脚的双极型器件。8086 CPU 的总线状态信号 $\overline{S_2}$ 、 $\overline{S_1}$ 、 $\overline{S_0}$ 输入总线控制器 8288 后,由 8288 译码,并与输入控制信号 $\overline{AEN}$ 、 $\overline{CEN}$ 、 $\overline{IOB}$ 相配合,输出一系列的总线命令和控制信号。总线控制器的跨接线使它适应于多主机系统总线及局部总线。8288 的引脚和内部结构框图如图 2-21 所示:

各引脚功能说明如下:

(1) 总线状态信号

$\overline{S_2} \sim \overline{S_0}$:总线状态信号。由 CPU 输入,经内部状态译码器译码后,通过命令信号发生器产生总线命令信号。状态译码表如表 2-12 所示:其中 $\overline{S_0}$ 用来区分存储器传送还是 I/O 传送, $\overline{S_1}$ 用来区分执行的操作是输入还是输出。

(2) 控制输入信号

CLK:时钟信号,输入,由时钟发生器 8284 提供。

$\overline{AEN}$:地址允许信号,由总线裁决器 8289 输入,低电平有效。 $\overline{AEN}$ 是支持总线结构的控制信号,用作多总线之间同步控制。若 8288 处于局部总线工作方式, $\overline{AEN}$ 不影响 I/O 命令线。

CEN:命令允许信号,由外部输入,高电平有效。在多个 8288 工作时,相当于 8288 的片选信号。CEN 有效时,允许 8288 输出全部总线控制信号和命令信号,CEN 无效时,总线控制信号和命令信号呈高阻状态。系统任何时候只允许一个处理器主控,所以只有一片 8288 的 CEN 信号有效。

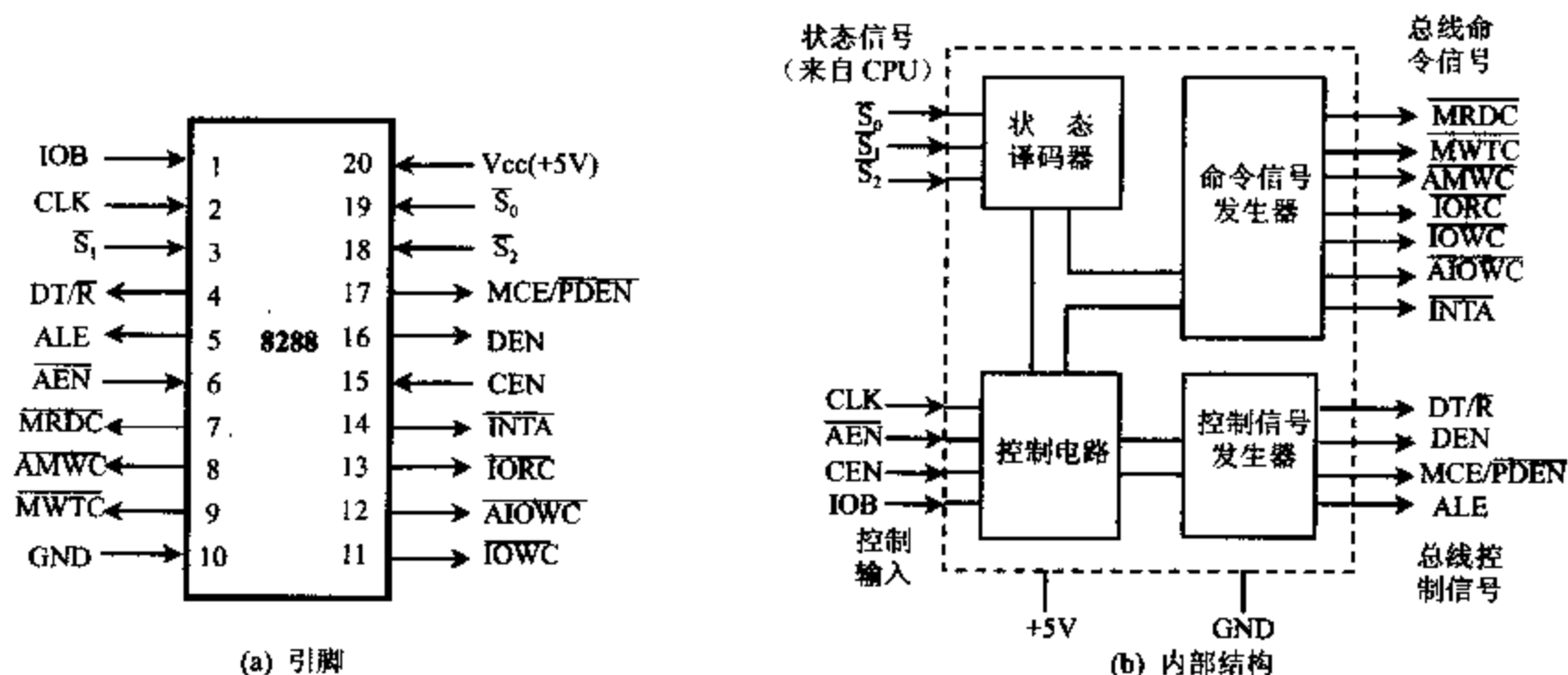


图 2-21 8288 的引脚及内部结构框图

表 2-12 $\overline{S_2} \sim \overline{S_0}$ 状态译码内容

总线状态信号			CPU 状态	8288 输出命令
$\overline{S_2}$	$\overline{S_1}$	$\overline{S_0}$		
0	0	0	中断响应	$\overline{INTA}$
0	0	1	读 I/O 端口	$\overline{IORC}$
0	1	0	写 I/O 端口	$\overline{IOWC}, \overline{AIOWC}$
0	1	1	暂停	无
1	0	0	取指令	$\overline{MRDC}$
1	0	1	读存储器	$\overline{MRDC}$
1	1	0	写存储器	$\overline{MWTC}, \overline{AMWC}$
1	1	1	无源状态	无

IOB:总线工作方式控制,输入,高电平有效。当 IOB 接高电平时,8288 处于局部总线工作方式,当 IOB 接低电平时,8288 处于系统总线工作方式。

(3) 总线命令信号

$\overline{INTA}$:中断响应信号,输出,低电平有效。此信号通知申请中断的设备,中断申请已被响应,由设备把“中断类型号”置于数据总线上。

$\overline{IORC}$:读 I/O 端口命令,输出,低电平有效。此信号相当于最小模式系统中 CPU 发出的 $\overline{RD}$ 和 $M/\overline{IO}$ 信号的组合, $\overline{RD}=0, M/\overline{IO}=0$ 允许 I/O 端口将数据置于数据总线上。

$\overline{IOWC}, \overline{AIOWC}$:写 I/O 端口命令,输出,低电平有效。相当于最小模式系统中 CPU 发

出的 $\overline{WR}$ 和 $M/\overline{IO}$ 信号的组合, $\overline{WR}=0$ 、 $M/\overline{IO}=0$ 允许I/O端口从数据总线上读入数据,其中 $\overline{AIOWC}$ 为超前写I/O端口命令,它比 $\overline{IOWC}$ 超前一个时钟周期出现,使一些较慢的外设可得到一个额外的时钟周期执行写入操作。

$\overline{MRDC}$:读存储器命令,输出,低电平有效。相当于最小模式系统中CPU发出的 $\overline{RD}$ 和 $M/\overline{IO}$ 的组合。 $\overline{RD}=0$ 、 $M/\overline{IO}=1$ 允许存储器将数据置于数据总线上。

$\overline{MWTC}$, $\overline{AMWC}$:写存储器命令,输出,低电平有效。相当于最小模式系统中CPU发出的 $\overline{WR}$ 和 $M/\overline{IO}$ 的组合。 $\overline{WR}=0$ 、 $M/\overline{IO}=1$ 允许存储器从数据总线上取出数据。其中 $\overline{AMWC}$ 为超前写存储器命令。它比 $\overline{MWTC}$ 提前一个时钟周期出现。

(4) 总线控制信号

ALE:地址锁存允许信号,输出,高电平有效。此信号用作8282的选通信号,将地址存入地址锁存器中。

DEN:数据传送允许信号,输出,高电平有效。此信号使数据收发器可以挂在局部数据总线上,也可以挂在系统数据总线上,此信号接数据收发器8286输出允许端 $\overline{EN}$ 。

$DT/\overline{R}$:数据收发控制信号,输出。此信号确定数据收发器的数据流通方向, $DT/\overline{R}$ =高电平时,表示发送(CPU写入数据到内存或I/O)。 $DT/\overline{R}$ =低电平时,表示接收(CPU从内存或I/O读出数据)。此信号接数据收发器8286控制端T。

$MCE/\overline{PDEN}$:主控级联允许/外设数据允许信号,输出。此端具有双重功能,当IOB接低电平时,8288工作在系统总线方式,此引脚作MCE,高电平有效。它在中断响应周期的 T_1 状态,可控制将主8259A向从8259A输出的级联地址 $CAS_2 \sim CAS_6$ 进行锁存。当IOB接高电平时,8288工作在局部总线方式,起 $\overline{PDEN}$ 的功能,低电平有效,用来控制外设通过局部总线传送数据。

最大模式一般用在多处理器系统,但在单处理器系统中有时也用8288来增加总线的驱动能力。如PC/XT机,一般情况下系统为单处理器工作方式,但使用了8288总线控制器,控制总线的驱动能力较强,当需要时,可方便地在扩展槽中插入协处理器板,构成多处理器系统,所以8288提供了两种工作方式,由引脚IOB来进行选择。

I/O总线方式:IOB引脚接高电平时,8288处于局部总线工作方式。在此方式中所有I/O命令线处于“允许状态”,即处理器进行I/O操作时,不需要总线裁决器发 $\overline{AEN}$ 信号,8288会立即发出相应的 $\overline{IORC}$ 、 $\overline{INTA}$ 或者 $\overline{IOWC}$ 、 $\overline{AIOWC}$ 信号,并通过 $\overline{PDEN}$ 和 $DT/\overline{R}$ 启动命令线,去控制总线收发器工作。此方式允许一个8288总线控制器管理两组外部总线。当CPU访问局部总线时不需要等待。在多处理器系统中,有些外设从属于某一处理器,采用局部总线方式是很有利的。

系统总线工作方式:IOB引脚接地,8288处于系统总线工作方式。在此工作方式下,总线裁决电路向8288的 $\overline{AEN}$ 端送一个低电平,表示总线可供使用(CEN 接高电平)。在多处理器系统中,如果I/O设备和存储器均被多个处理器共享。系统中必须使用总线裁决,所以8288必须工作在系统总线方式。在IBM PC/XT中,8288工作在系统总线方式,8288的输出信号受 $\overline{AEN}$ 和 CEN 控制,当 CEN 为高电平,并且 $\overline{AEN}$ 有效后,8288才能输出命令和控制信号。

2-5 8086 CPU 时序

计算机工作过程是执行指令的过程,8086 CPU 的操作是在时钟脉冲 CLK 的统一控制下进行的,8086 的时钟频率为 5MHz,时钟周期或 T 状态为 200ns。

指令周期(Instruction Cycle):执行一条指令所需的时间称为指令周期。不同指令的指令周期的长短是不同的,一个指令周期由几个总线周期组成。

总线周期(Bus Cycle):8086 CPU 中,BIU 完成一次访问存储器或 I/O 端口操作所需要的时间,称作一个总线周期。一个总线周期由几个 T 状态组成。

时钟周期(Clock Cycle):CPU 的时钟频率的倒数,也称 T 状态。

在 8086/8088 CPU 中,每个总线周期至少包含 4 个时钟周期($T_1 \sim T_4$),一般情况下,在总线周期的 T_1 状态传送地址, $T_2 \sim T_4$ 状态传送数据。8086 的主要操作时序有以下几种:

一、系统的复位和启动

8086/8088 CPU 通过 RESET 引脚上的触发信号来引起 8086 系统复位和启动,复位信号 RESET 至少维持 4 个时钟周期的高电平。

当 RESET 信号变成高电平时,8086/8088 CPU 结束现行操作,各个内部寄存器复位成初值,如表 2-13 所示:

其中代码段寄存器 CS 为 FFFFH,指令指针 IP 清 0,所以 8086/8088 在复位之后重新启动时,从内存的 FFFF0H 处开始执行指令。因此在 FFFF0 处存放了一条无条件转移指令,转移到系统引导程序的入口处,这样系统启动后就自动进入系统程序。

在复位时,由于标志寄存器被清 0,所有标志位均为 0,这样从 INTR 引脚进入的可屏蔽中断就被屏蔽了,因此在系统程序中要用开中断指令 STI 来设置中断允许标志。图 2-22 给出了 8086 复位操作时的时序。

表 2-13 复位时各内部寄存器的值

标志寄存器	清 零
指令指针 IP	0000H
CS 寄存器	FFFFH
DS 寄存器	0000H
SS 寄存器	0000H
ES 寄存器	0000H
指令队列	变空
其它寄存器	0000H

在 RESET 信号变成高电平后,经过一个时钟周期,所有的三态输出线被设置成高阻,并一直维持高阻状态(浮空),直到 RESET 信号回到低电平为止。但在高阻状态的前半个时钟周期,三态输出线被置成不作用状态,当时钟信号又变成高电平时,才置成高阻状态。置成高阻状态的三态输出线包括: $AD_{15} \sim AD_0$ 、 $A_{19}/S_6 \sim A_{16}/S_3$ 、 $\overline{BHE}/S_7$ 、 $M/\overline{IO}$ 、 $DT/\overline{R}$ 、 $\overline{DEN}$ 、 $\overline{WR}$ 、 $\overline{RD}$ 和 $\overline{INTA}$ 。另外有几条控制线在复位之后处于无效状态,但不浮空,它们是

ALE、H₁DA、 $\overline{\text{RQ}}/\overline{\text{GT}}_0$ 、 $\overline{\text{RQ}}/\overline{\text{GT}}_1$ 、QS₀、QS₁。

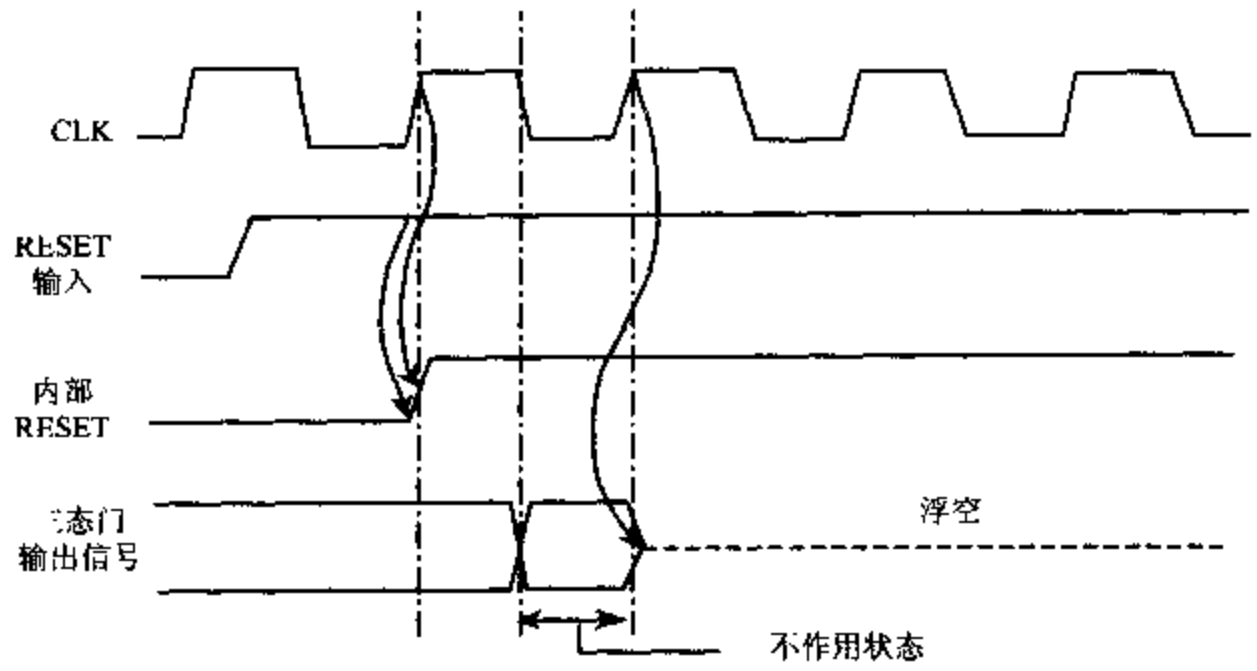


图 2-22 8086 复位操作的时序

二、最小模式下的总线操作

1. 读总线周期

图 2-23 表示 8086 CPU 从存储器或外设端口读取数据的时序。

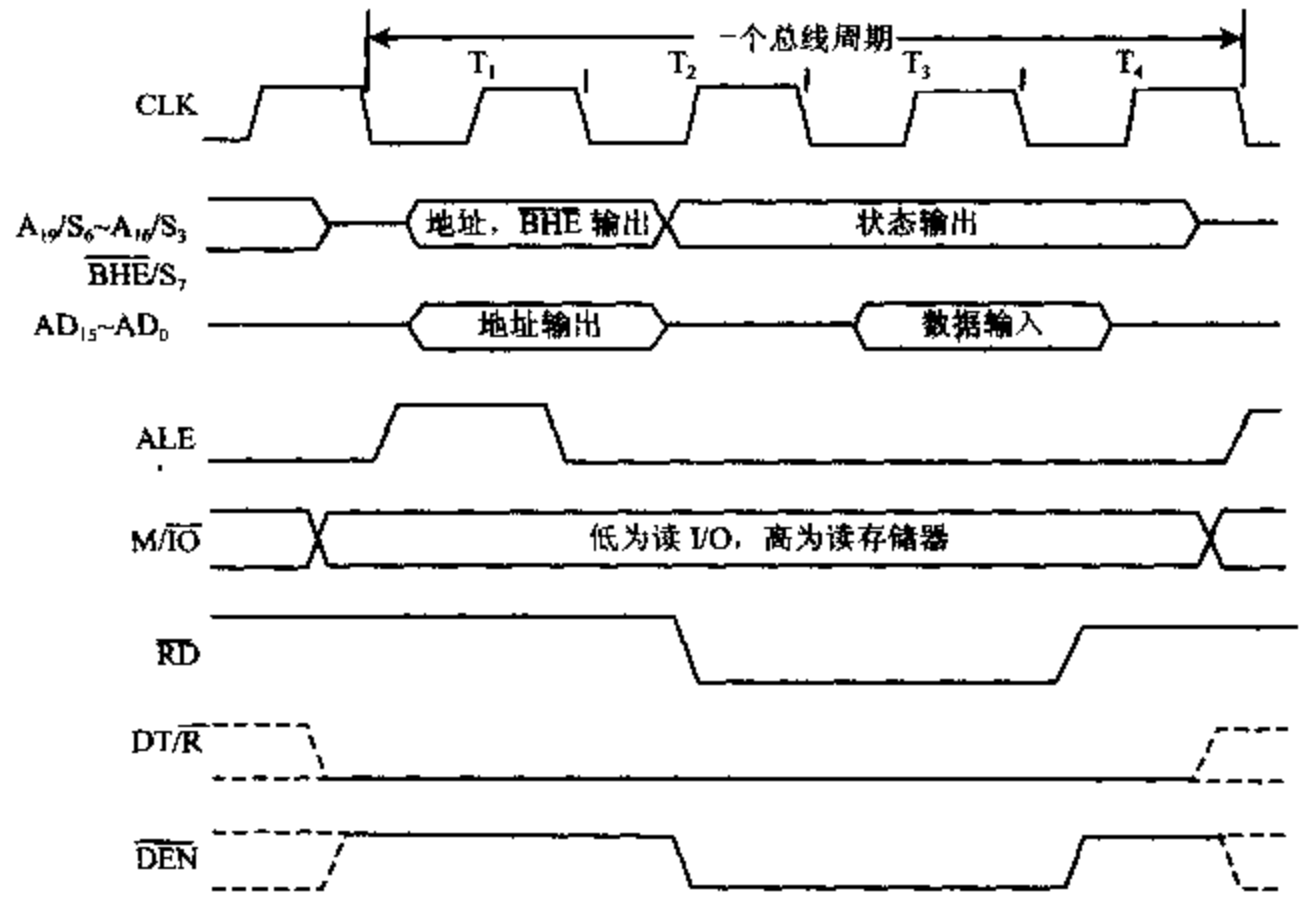


图 2-23 8086 读总线周期的时序

一个最基本的读总线周期包含 4 个 T 状态，即 T₁、T₂、T₃、T₄，在存储器和外设速度较慢时，在 T₃ 后可插入 1 个或几个等待状态 T_w。

T₁ 状态：

(1) M/ $\overline{\text{IO}}$ 信号在 T₁ 状态有效，指出 CPU 是从内存还是从 I/O 端口读取数据。M/ $\overline{\text{IO}}$

为高,从存储器读, $M/\overline{IO}$ 为低,从 I/O 端口读。 $M/\overline{IO}$ 信号的有效电平一直保持到总线周期结束的 T_4 状态。

(2) T_1 状态开始,20 位地址信号通过多路复用总线输出,指出要读取的存储器或 I/O 端口的地址。高 4 位地址从 $A_{19}/S_6 \sim A_{16}/S_3$ 地址/状态线送出,低 16 位从 $AD_{15} \sim AD_0$ 地址/数据线送出。

(3) ALE 引脚上输出一个正脉冲作地址锁存信号。在 T_1 状态结束时, $M/\overline{IO}$ 信号,地址信号均已有效,ALE 的下降沿用作锁存器 8282 的选通信号,使地址锁存,这样在总线周期的其它状态才可分时复用这些引脚传送数据或状态信息。

(4) $\overline{BHE}$ 信号有效,作为奇地址存储体的选体信号,配合地址信号可实现存储单元的寻址,它表示高 8 位数据线上的数据有效(偶地址存储体的选体信号为 A_0)。

(5) 系统中若接有数据总线收发器 8286 时,在 T_1 状态, $DT/\overline{R}$ 端输出低电平,表示本总线周期为读周期,用 $DT/\overline{R}$ 去控制 8286 接收数据。

T_2 状态:

(6) 地址信号消失, $A_{19}/S_6 \sim A_{16}/S_3$ 引脚上输出状态信息 $S_6 \sim S_3$,指出当前正在使用的段寄存器及中断允许情况。

(7) 低位地址线 $AD_{15} \sim AD_0$ 进入高阻状态,为读取数据做准备。

(8) $\overline{BHE}/S_7$ 变成高电平,输出状态信息 S_7 , S_7 在设计中未赋予实际意义。

(9) $\overline{RD}$ 信号有效,送到所有的存储器和 I/O 端口,但只选通地址有效的存储单元和 I/O 端口,使之能读出数据。

(10) 若系统中接有 8286, $\overline{DEN}$ 信号在 T_2 状态有效,作为 8286 的选通信号,使数据通过 8286 传送。

T_3 状态:

(11) T_3 状态一开始,CPU 采样 READY 信号,若此信号为低电平,表示系统中所连接的存储器或外设工作速度较慢,数据没有准备好,要求 CPU 在 T_3 和 T_4 状态之间再插入一个 T_w 状态。READY 是通过时钟发生器 8284 传递给 CPU 的。

(12) 当 READY 信号有效时,CPU 读取数据。在 $\overline{DEN}=0$ 、 $DT/\overline{R}=0$ 的控制下,内存单元或 I/O 端口的数据通过数据收发器 8286 送到数据总线 $AD_{15} \sim AD_0$ 上。CPU 在 T_3 周期结束时,读取数据。 $S_6 S_3$ 指出了当前访问哪个段寄存器,若 $S_6 S_3 = 10$,表示访问 CS 段,读取的是指令,CPU 将它送入指令队列中等待执行,否则读取的是数据,送入 ALU 进行运算。

T_w 状态:

(13) CPU 在每个 T_w 状态的前沿对 READY 信号采样,若为低电平继续插入 T_w 状态。当在 T_w 状态采样到 READY 信号为高电平时,在当前 T_w 状态执行完,进入 T_4 状态。在最后一个 T_w 状态,数据肯定已出现在数据总线上,此时 T_w 状态的动作与 T_3 状态一样。CPU 采样数据线 $AD_{15} \sim AD_0$ 。

T_4 状态:

CPU 在 T_3 与 T_4 状态的交界处采样数据。然后在 T_4 状态的后半周期,数据从数据总线上撤除,各个控制信号和状态信号线进入无效状态, $\overline{DEN}$ 无效,总线收发器不工作,一个

读总线周期结束。

2. 写总线周期

图 2-24 表示了 8086 CPU 写总线周期的时序。

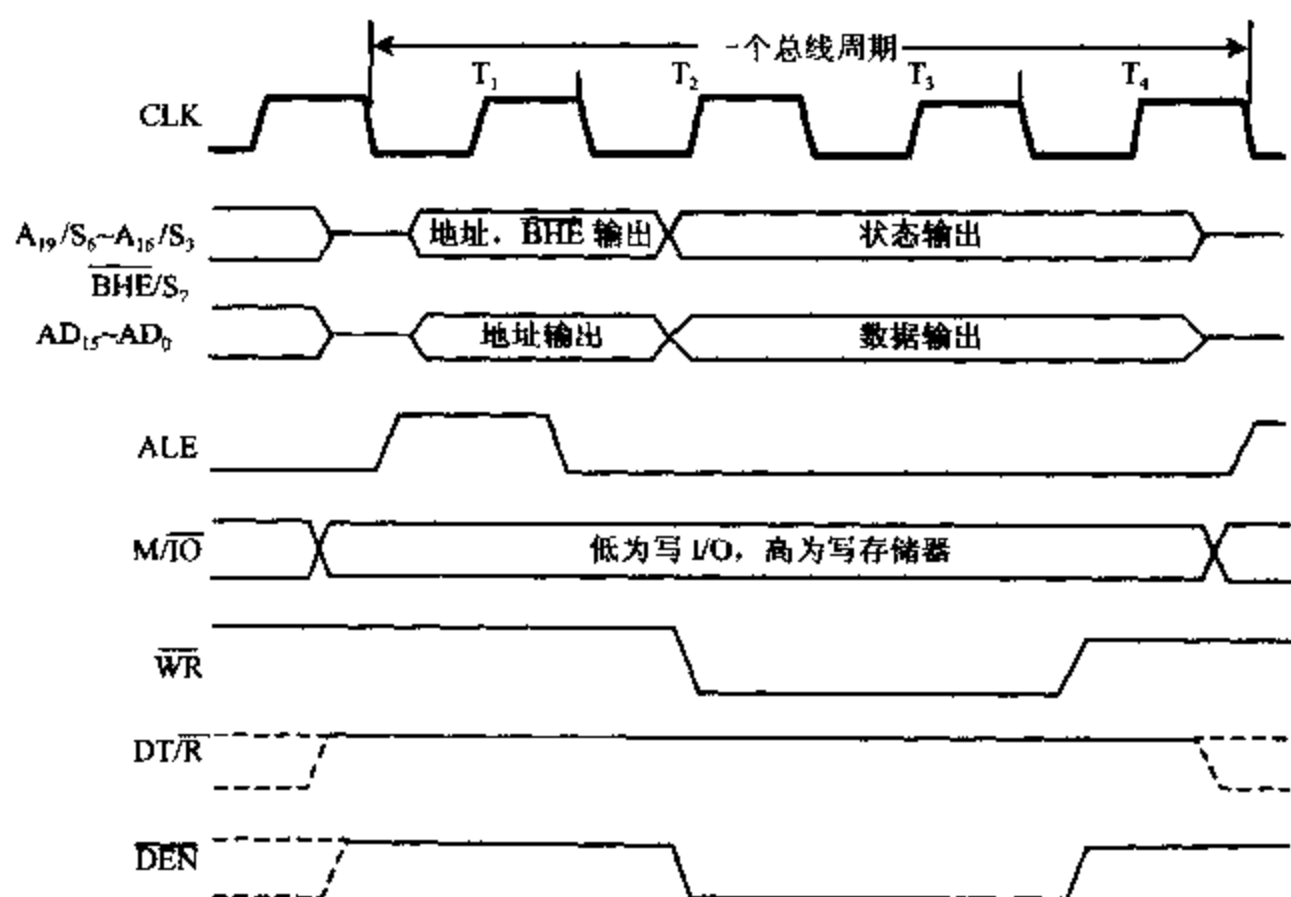


图 2-24 8086 写总线周期的时序

8086 CPU 写总线周期时序与读总线周期时序有许多相同之处。

在 T_1 状态, $M/\overline{IO}$ 信号有效, 指出 CPU 将数据写入内存还是 I/O 端口; CPU 给出写入存储单元或 I/O 端口的 20 位物理地址; 地址锁存信号 ALE 有效, 选存储体信号 $\overline{BHE}$, A_0 有效, $DT/\overline{R}$ 变高平, 表示本总线周期为写周期。

在 T_2 状态, 地址撤销, $S_6 \sim S_3$ 状态信号输出; 数据从 CPU 送到数据总线 $AD_{15} \sim AD_0$, $\overline{WR}$ 写信号有效; $\overline{DEN}$ 信号有效, 作为数据总线收发器 8286 的选通信号。

在 T_3 状态, CPU 采样 READY 线, 若 READY 信号无效, 插入一个到几个 T_w 状态, 直到 READY 信号有效, 存储器或 I/O 设备从数据总线上取走数据。

在 T_4 状态, 从数据总线上撤销数据, 各控制信号和状态信号线变成无效; $\overline{DEN}$ 信号变成高电平, 总线收发器不工作。

下面说明几点不同之处:

(1) 在 T_1 状态, $DT/\overline{R}$ 信号为高电平, 表示本总线周期为写周期, 即 CPU 将数据写入存储单元或 I/O 端口。

(2) 在 T_2 状态, 地址信号发出后, CPU 立即向地址/数据总线 $AD_{15} \sim AD_0$ 发出数据, 数据信号保持到 T_4 状态的中间, 使存储器或外设一旦准备好即可从数据总线取走数据。

(3) 写信号为 $\overline{WR}$ (代替 $\overline{RD}$), 在 T_2 状态有效, 维持到 T_4 状态, 选通存储器或 I/O 端口的写入。

3. 总线空操作

只有在 CPU 和存储器或 I/O 接口之间传输数据时, CPU 才执行总线周期, 当 CPU 不

执行总线周期时(指令队列 6 字节已装满, EU 未申请访问存储器), 总线接口部件不和总线打交道, 就进入了总线空闲周期 T_1 。此时状态信息 $S_6 \sim S_3$ 和前一个总线周期一样, 数据总线上信号不同, 若前一个总线周期是读周期, 则 $AD_{15} \sim AD_0$ 在 T_1 状态处于高阻状态, 若前一个总线周期是写周期, 则 $AD_{15} \sim AD_0$ 在 T_1 状态继续保持数据有效。

在空闲周期中, 虽然 CPU 对总线进行空操作, 但 CPU 内部操作仍然进行。例如 ALU 执行运算, 内部寄存器之间数据传输等, 即 EU 部件在工作。所以说, 总线空操作是总线接口部件 BIU 对总线执行部件 EU 的等待。

三、最大模式下的总线操作

8086 CPU 工作在最大模式时, 增加了总线控制器 8288, CPU 向 8288 输出状态信号 $\overline{S_2} \sim \overline{S_0}$, 根据 $\overline{S_2} \sim \overline{S_0}$ 的编码, 由 8288 输出总线控制信号, 如存储器读/写信号, I/O 端口读/写信号及中断信号, 因此在时序分析上要考虑 CPU 和总线控制器两者产生的信号。

1. 读总线周期

图 2-25 给出了 8086 最大模式下的读总线周期时序。

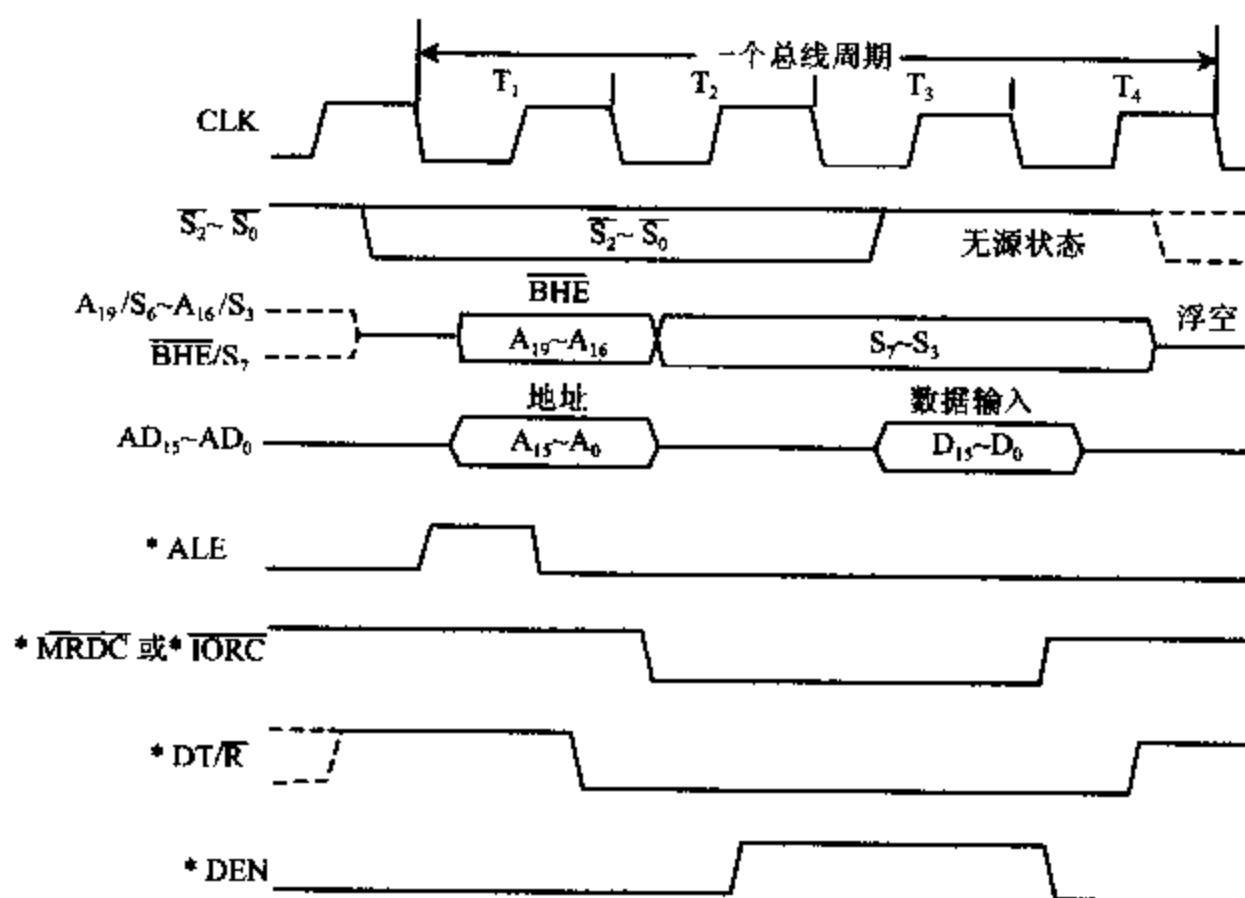


图 2-25 8086 最大模式下的读总线周期时序

下面仅说明与最小模式读操作不同之处。

(1) 在每个总线周期开始前, $\overline{S_2} \sim \overline{S_0}$ 全为高电平。总线控制器 8288 只要检测到 $\overline{S_2}$ 、 $\overline{S_1}$ 、 $\overline{S_0}$ 中任何一个或几个从高电平变到低电平, 便立即开始一个新的总线周期。

(2) 在最大模式下, $\overline{ALE}$ 、 $\overline{RD}$ 、 $\overline{DT/R}$ 、 $\overline{DEN}$ 信号由 8288 提供。图中分别表示为 $\overline{ALE}$ 、 $\overline{MRDC}$ 、 $\overline{IORC}$ 、 $\overline{DT/R}$ 、 $\overline{DEN}$ 。

(3) $\overline{MRDC}$ 和 $\overline{IORC}$ 指出了读存储器还是读 I/O 端口, 代替了读信号 $\overline{RD}$, 相当于最小模式中 $M/\overline{IO}$ 及 $\overline{RD}$ 的组合。

(4) 在 T_3 状态, 当 CPU 读取数据后, $\overline{S_2} \sim \overline{S_0}$ 全部进入高电平, 即无源状态, 一直维持到 T_4 。一旦进入无源状态, 意味着很快可以启动一个新的总线周期。

(5) 在 T_4 状态, 数据从总线上消失, $S_7 \sim S_3$ 进入高阻状态, 而 $\overline{S_2}$ 、 $\overline{S_1}$ 、 $\overline{S_0}$ 按照下一个总线周期的操作类型产生电平变化。

(6) T_w 状态的插入与最小模式相同。

2. 写总线周期

8086 CPU 在最大模式下的写总线周期时序如图 2-26 所示:

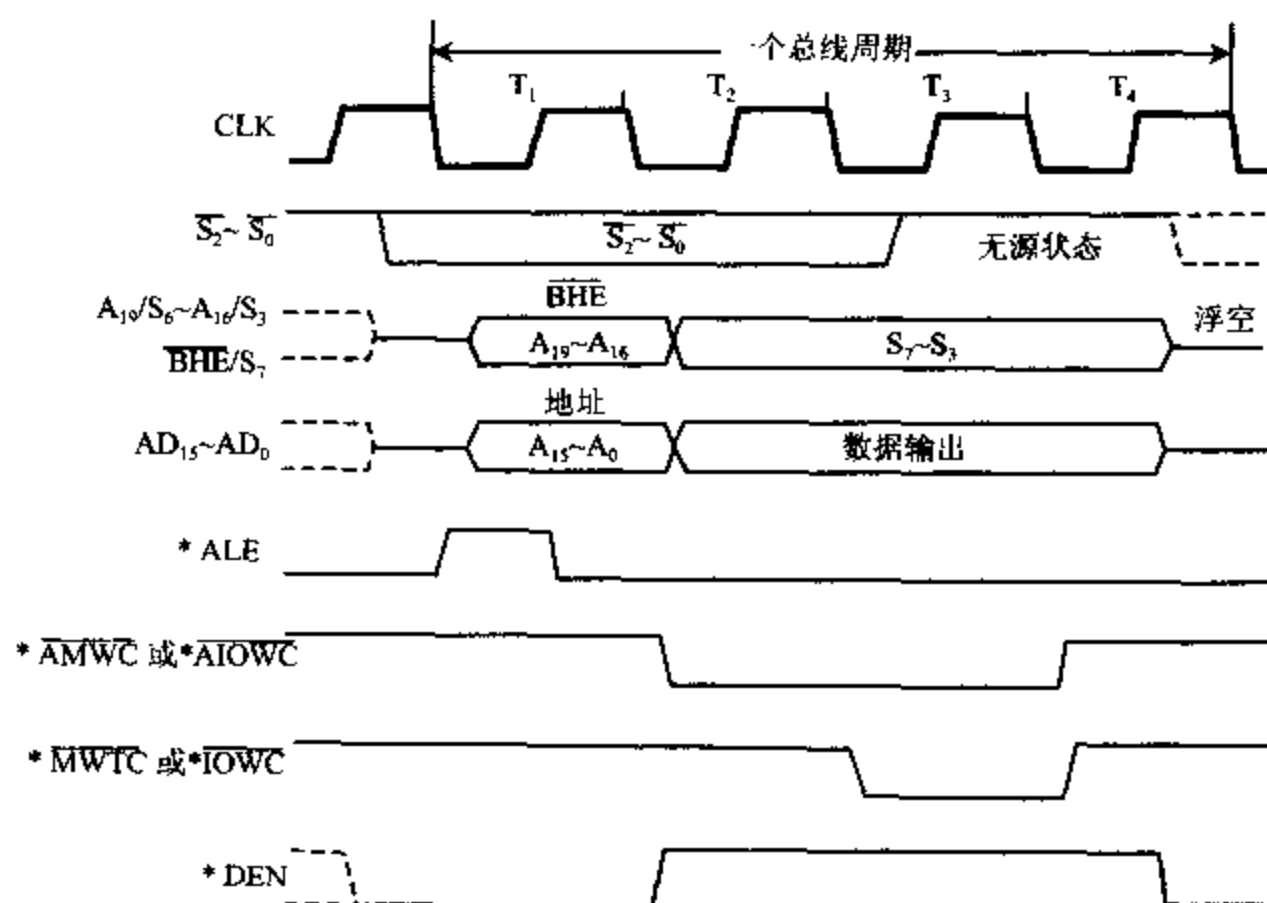


图 2-26 8086 最大模式下的写总线周期时序图

与最大模式读总线周期相比, 有几点要说明:

(1) $\overline{S_2} \sim \overline{S_0}$ 状态信息在各个 T 周期中的变化情况与最大模式下读周期中变化一样。

(2) 8288 提供了两组写存储器或写 I/O 端口的信号, 一组是普通的写信号 $\overline{*MWTC}$ 和 $\overline{*IOWC}$, (T_3 状态开始有效, 维持到 T_4 状态); 另一组是提前一个时钟周期的写信号 $\overline{*AWTC}$ 及 $\overline{*AIOWC}$, (T_2 状态开始有效, 维持到 T_4 状态)。 $\overline{*AWTC}$ 和 $\overline{*AIOWC}$ 信号可以使一些较慢的设备或存储器芯片得到一个额外的时钟周期进行写操作。

(3) CPU 在 T_2 状态就把数据送到数据总线 $AD_{15} \sim AD_0$ 。

四、最小模式下的总线保持

在一个系统中, CPU 以外的其它主模块要求获得控制总线的使用权时, 向 CPU 发出总线请求信号 $HOLD$ 。在每个时钟脉冲的上升沿, CPU 检测 $HOLD$ 引脚上的信号。如果检测到 $HOLD$ 为高电平, 并且允许让出总线, 那么在总线周期的 T_4 状态或空闲状态 T_1 之后的下一个时钟周期, CPU 发出总线响应信号 $HLDA$, 并且让出总线, 直到 $HOLD$ 信号无效, CPU 才收回总线控制权。

图 2-27 给出了总线请求和总线响应的时序。

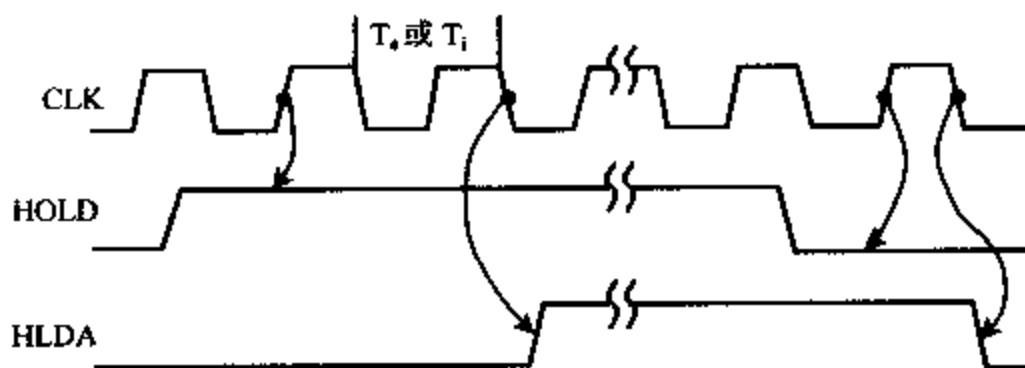


图 2-27 总线请求和总线响应的时序

(1) HOLD 信号变高电平后, CPU 要在下一个时钟周期的上升沿才检测到。然后用 T_h 或 T_i 状态的下降沿使 HLDA 变成高电平。若采样到 HOLD 信号时, 不在 T_h 或 T_i 状态, 可能会延迟几个时钟周期, 等到 T_h 或 T_i 状态才发 HLDA 信号。

(2) 8086 CPU 一旦让出总线控制权, 使地址线, 数据线及控制信号 $\overline{RD}$ 、 $\overline{WR}$ 、 $\overline{INTA}$ 、 $M/\overline{IO}$ 、 $\overline{DEN}$ 及 $DT/\overline{R}$ 处于浮空状态, 但 ALE 信号不浮空。

(3) HOLD 信号影响 8086 CPU 的总线接口部件 BIU 的工作(总线浮空), 但执行部件 EU 继续执行指令队列中的指令, 直到遇到需要使用总线的指令时, 执行部件 EU 才停下来。

(4) 当总线请求结束, HOLD 及 HLDA 信号变为低电平时, CPU 不立刻驱动三总线, 这些引脚继续浮空, 直到 CPU 执行一条总线操作, 才结束这些引脚的浮空状态。因此, 为了防止总线控制切换时, 因没有任何主模块的驱动而造成控制线电平飘移到最小电平以下, 在控制线和电源之间要连接一个提拉电阻。

8088 CPU 在最小模式/最大模式中的读/写总线周期时序图与 8086 CPU 的读/写总线周期基本上相同, 所不同的仅在于 8088 每次访问存储器或 I/O 端口, 只能读/写一个字节, 通过 $AD_7 \sim AD_0$ 传送, 不存在总线高位有效信号 $\overline{BHE}$ 。

习 题

- 8086 CPU 内部由哪两部分组成? 它们的主要功能是什么?
- 8086 CPU 中有哪些寄存器? 各有什么用途?
- 8086 CPU 与 8088 CPU 的主要区别是什么?
- 简要解释下列名词的意义: CPU、存储器、堆栈、IP、SP、BP、段寄存器、状态标志、控制标志、物理地址、逻辑地址、机器语言、汇编语言、指令、内部总线、系统总线。
- 要完成下述运算或控制, 用什么标志位判别? 其值是什么?
 - 比较两数是否相等。
 - 两数运算后结果是正数还是负数?
 - 两数相加后是否溢出?
 - 采用偶校验方式, 判定是否要补“1”?
 - 两数相减后比较大小。
 - 中断信号能否允许?
- 8086 系统中存储器采用什么结构? 用什么信号来选中存储体?
- 用伪指令 DB 在存储器中存储 ASCII 码字符串 ‘What time is it?’。并画出内存分

布图。

8. 用伪指令将下列 16 位十六进制数存储在存储器中,并画出内存分布图。

a) 1234H b) A122H c) B100H

9. 实模式下,段寄存器装入如下数据,写出每段的起始和结束地址。

a) 1000H b) 1234H c) 2300H d) E000H e) AB00H

10. 在实模式下对下列 CS:IP 的组合,求出要执行的下一条指令的存储器地址。

a) CS:IP=1000H;2000H b) CS:IP=2000H;1000H
c) CS:IP=1A00H;B000H d) CS:IP=3456H;AB09H

11. 实模式下,求下列寄存器组合所寻址的存储单元地址:

a) DS = 1000H, DI = 2000H b) SS = 2300H, BP = 3200H
c) DS = A000H, BX = 1000H d) SS = 2900H, SP = 3A00H

12. 若当前 SS=3500H,SP=0800H,说明堆栈段在存储器中的物理地址,若此时入栈 10 个字节,SP 内容是什么?若再出栈 6 个字节,SP 为什么值?

13. 某程序数据段中存放了两个字,1EE5H 和 2A8CH,已知 DS = 7850H,数据存放的偏移地址为 3121H 及 285AH。试画图说明它们在存储器中的存放情况。若要读取这两个字,需要对存储器进行几项操作?

14. 存储器中每段容量最多 64K 字节,若用 debug 调试程序中的 r 命令,在屏幕上有如下显示:

c:\>debug

—r

AX=0000 BX=0000 CX=0079 DX=0000 SP=FFEE BP=0000 SI=0000 DI=0000
DS=10E4 ES=10F4 SS=21F0 CS=31FF IP=0100 NV UP DI PL NZ NA PO NC

求:(1) 画出此时存储器分段示意图。(2) 写出状态标志 OF、SF、ZF、CF 的值。

15. 说明 8086 系统中“最小模式”和“最大模式”两种工作方式的主要区别是什么?

16. 8086 系统中为什么要用地址锁存器? 8282 地址锁存器与 CPU 如何连接?

17. 哪个标志位控制 CPU 的 INTR 引脚?

18. 什么叫总线周期? 在 CPU 读/写总线周期中,数据在哪个机器状态出现在数据总线上?

19. 8284 时钟发生器共给出哪几个时钟信号?

20. 8086 CPU 重新启动后,从何处开始执行指令?

21. 8086 CPU 的最小模式系统配置包括哪几部分?

第三章 8086 的寻址方式和指令系统

3-1 8086 的寻址方式

计算机的指令通常包含操作码和操作数两部分,前者指出操作的性质,后者给出操作的对象。寻址方式就是指令中说明操作数所在地址的方法。8086 访问操作数采用多种灵活的寻址方式,使指令系统可以方便地在 1M 存储空间内寻址。

指令有单操作数、双操作数和无操作数之分。如果是双操作数指令,要用逗号将两个操作数分开,逗号右边的操作数称为源操作数,左边的为目的操作数。例如,将寄存器 CX 中的内容送进寄存器 AX 的指令为 MOV AX, CX,其中,CX 为源操作数,AX 是目的操作数,而 MOV 则为操作码。操作数可以包含在寄存器、存储器或 I/O 端口地址中,它们也可以是立即数。操作数在寄存器中的指令执行速度最快,因为它们可以在 CPU 内部立即执行。立即数寻址指令可直接从指令队列中取数,所以它们的执行速度也较快。而操作数在存储器中的指令执行速度较慢,因为它要通过总线与 CPU 之间交换数据,当 CPU 进行读写存储器的操作时,必须先把一个偏移量送到 BIU,计算出 20 位物理地址,再执行总线周期去存取操作数,这种寻址方式最为复杂。

下面主要以 MOV 指令为例来说明 8086 指令的各种寻址方式。8088 的指令与 8086 完全兼容,本章介绍的各种寻址方式和 8086 的指令系统同样适用于 8088。

一、立即寻址方式 (Immediate Addressing)

在这种寻址方式下,操作数直接包含在指令中,它是一个 8 位或 16 位的常数,也叫立即数。这类指令翻译成机器码时,立即数作为指令的一部分,紧跟在操作码之后,存放在代码段内。如果立即数是 16 位数,则高字节存放在代码段的高地址单元中,低字节放在低地址单元中。立即寻址方式的指令常用来给寄存器赋初值。

例 3-1 MOV AL, 26H

该指令表示将一个 8 位立即数 26H 送到 AL 寄存器中。

例 3-2 MOV CX, 2A50H

它表示将立即数 2A50H 送到 CX 寄存器中。例 3-2 指令的机器码存放及执行过程如图 3-1 所示。

立即数不但可以送到寄存器中,还可以送到一个存储单元(8 位)中或两个连续的存储单元(16 位)中去。需要强调的是,在所有的指令中,立即数只能作源操作数,不能作目的操作数。另外要注意,以 A~F 打头的数字出现在指令中时,前面一定要加一个数字 0,以

免与其它符号相混淆。例如,将立即数 FF00H 送到 AX 的指令必须写成如下形式:

```
MOV AX, 0FF00H
```

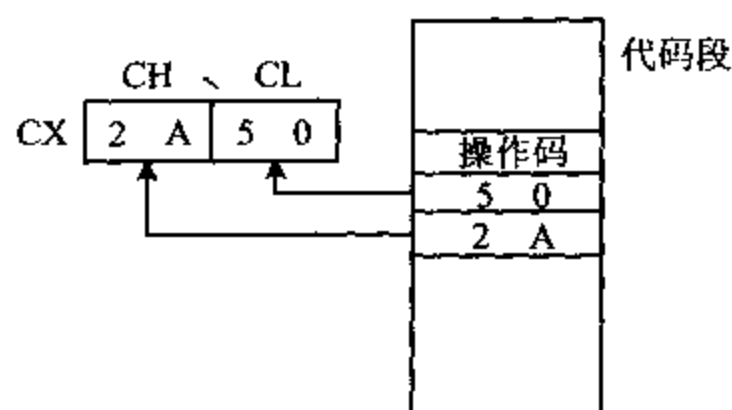


图 3-1 例 3-2 指令执行过程示意图

二、寄存器寻址方式 (Register Addressing)

在这种寻址方式下,操作数包含在寄存器中,由指令指定寄存器的名称。对于 16 位操作数,寄存器可以是 AX、BX、CX、DX、SI、DI、SP 和 BP 等。对于 8 位操作数,则用寄存器 AH、AL、BH、BL、CH、CL、DH 和 DL。

例 3-3 MOV DX, AX

假设该指令执行前 AX=3A68H, DX=18C7H, 则指令执行后, DX=3A68H, 而 AX 的内容保持不变。

例 3-4 MOV CL, AH

它表示将 AH 中的 8 位数据传送到 CL 寄存器。

注意,源操作数的长度必须与目的操作数一致,否则会出错。例如,你不能将 AH 寄存器的内容传送到 CX 中去,尽管 CX 寄存器放得下 AH 的内容,但汇编程序不知道将它放到 CH 还是 CL 中。

除上述两种寻址方式外,以下几种寻址方式下,指令的操作数都放在存储器中,需用不同的方法求出操作数的物理地址,来获得操作数。

三、直接寻址方式 (Direct Addressing)

1. 直接寻址方式

在 IBM PC 机中,把操作数的偏移地址称为有效地址 EA (Effective Address)。使用直接寻址方式的指令时,存储单元的有效地址直接由指令给出,在它们的机器码中,有效地址存放在代码段中指令的操作码之后。而该地址单元中的数据总是存放在存储器中,所以必须先求出操作数的物理地址,然后再访问存储器,才能取得操作数。要注意的是当采用直接寻址指令时,如果指令中没有用前缀指明操作数存放在哪一段,则默认为使用的段寄存器为数据段寄存器 DS, 因此,操作数的物理地址 $= 16 \times DS + EA$, 即 $= 10H \times DS + EA$ 。指令中有效地址上必须加一个方括号,以便与立即数相区别。

例 3-5 MOV AX, [2000H]

这条指令直接给出了操作数的有效地址 $EA=2000H$, 设 $DS=3000H$, 则源操作数的物理地址 $=16 \times 3000H + 2000H = 32000H$ 。由于目的操作数是 16 位寄存器 AX, 所以它将把该地址处的一个字送进 AX。若地址 $32000H$ 中的内容为 $34H$, $32001H$ 中的内容为 $12H$, 我们用 $(32000H) = 1234H$ 来表示这个字。则执行指令后, $AX=1234H$ 。指令执行过程如图 3-2 所示。

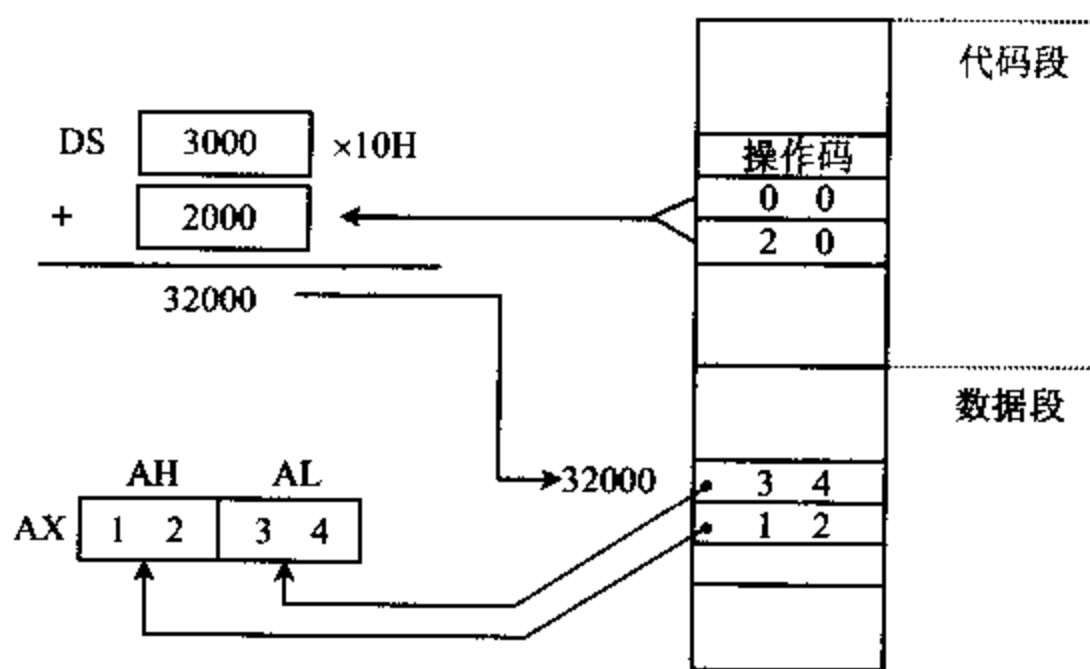


图 3-2 例 3-5 指令执行过程示意图

例 3-6 MOV AL, [2000H]

假设所有条件都和上例相同, 则该指令执行后将存储单元 $32000H$ 中的字节送到 AL 中, 结果为 $AL=34H$ 。

2. 段超越前缀

如果要对代码段、堆栈段或附加段寄存器所指出的存储区进行直接寻址, 应在指令中指定段超越前缀。例如, 数据若放在附加段中, 则应在有效地址前加“ES:”, 这里的冒号“:”称为修改属性运算符, 计算物理地址时要用 ES 作基地址, 而不再是默认值 DS。

例 3-7 MOV AX, ES: [500H]

该指令的源操作数的物理地址等于 $16 \times ES + 500H$ 。

3. 符号地址

在汇编语言中还允许用符号地址代替数值地址, 实际上就是给存储单元起一个名字, 这样, 如果要与这些单元打交道, 只要使用其名字即可, 不必记住具体数值是多少。

例 3-8 MOV AX, AREA1

这里的 AREA1 就是操作数的符号地址, 该指令执行后, 将从有效地址为 AREA1 的存储单元中取出一个字送到 AX 中去。

不过光从指令的形式上看, AREA1 不仅可代表符号地址, 也可以表示它是一个 16 位的立即数, 两者之间究竟如何来区别呢? 程序中还必须事先安排说明语句也叫做伪指令来加以说明。

例 3-9 AREA1 EQU 0867H

⋮
MOV AX, AREA1

这里,等值伪指令语句 EQU 用来给常数 0867H 定义一个符号名 AREA1,在此后的程序中,符号 AREA1 便代表一个立即数 0867H。执行 MOV AX, AREA1 指令后,AX=0867H。

例 3-10 AREA1 DW 0867H

⋮
MOV AX, AREA1

这里的 DW 伪指令语句用来定义变量,变量用作表示存储器中的数据,在程序运行过程中,变量是可以被修改的运算对象。本例中变量名为 AREA1,它表示内存中一个数据区的名字,也就是符号地址,该地址单元存放一个字数据 0867H。变量名可作为指令中的存储器操作数来引用。MOV AX, AREA1 指令执行后,表示将 AREA1 单元中的内容送到 AX 中,结果 AX=0867H。虽然,例 3-9 和例 3-10 的两条指令执行后结果相同,但符号 AREA1 的含义不同。

例 3-10 中的 MOV 指令也可写成

MOV AX, [AREA1]

符号地址也允许段超越,例如,下面两条指令是等价的,即

MOV AX, ES: AREA1
MOV AX, ES: [AREA1]

源操作数的物理地址均等于 $ES \times 16 + AREA1$ 。

有关汇编语言伪指令的概念,我们将在第四章详细讨论。

四、寄存器间接寻址方式 (Register Indirect Addressing)

指令中给出的寄存器中的值不是操作数本身,而是操作数的有效地址,这种寻址方式称为寄存器间接寻址。寄存器名称外面必须加方括号,以与寄存器寻址方式相区别。这类指令中使用的寄存器有基址寄存器 BX、BP 及变址寄存器 SI、DI。

如果指令中指定的寄存器是 BX、SI 或 DI,则默认操作数存放在数据段中,这时要用数据段寄存器 DS 的内容作为段地址,操作数的物理地址由 DS 左移 4 位后与 BX、SI 或 DI 相加形成。即,物理地址 = $16 \times DS + BX$

或 = $16 \times DS + SI$

或 = $16 \times DS + DI$ 。

例 3-11 MOV BX, [SI]

设: DS=1000H, SI=2000H, (12000H)=318BH

则: 物理地址 = $16 \times DS + SI$

= $10000H + 2000H$

=12000H

指令执行过程如图 3-3 所示,指令执行后,BX=318BH

如果指令中用寄存器 BP 进行间接寻址,则默认操作数在堆栈段中,操作数的段地址在寄存器 SS 中,操作数的物理地址= $16 \times SS + BP$ 。例如,指令 MOV AX,[BP]就是这种形式的指令。

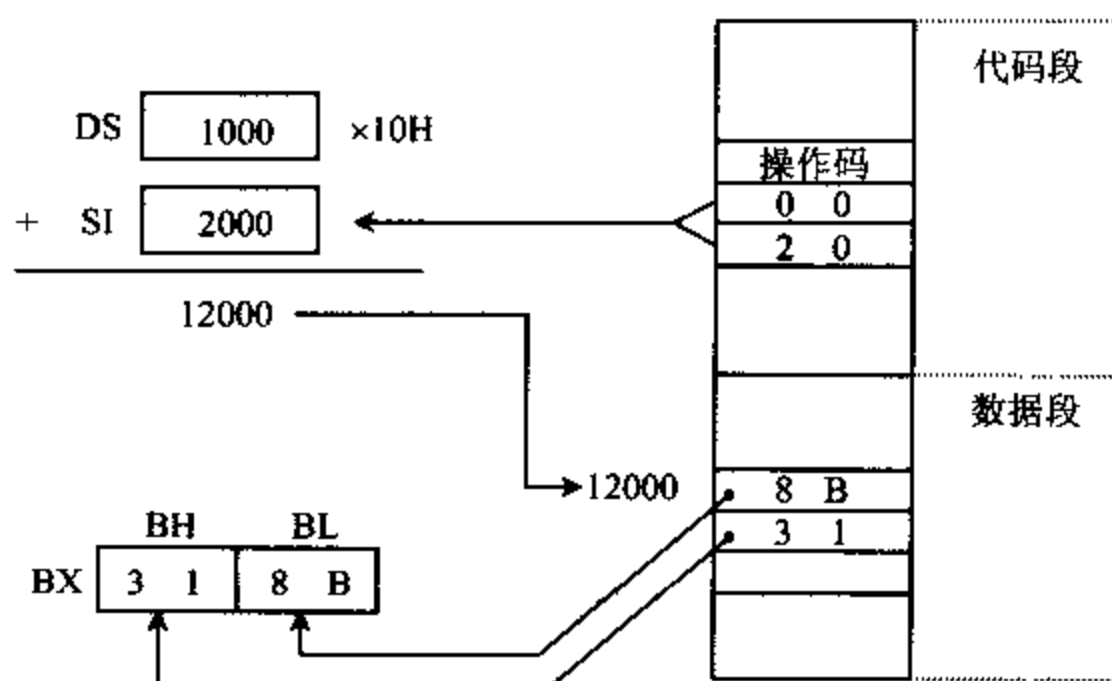


图 3-3 例 3-11 指令执行过程示意图

指令中也可以指定段超越前缀来从默认段以外的段中取得数据,例如:

```
MOV    BX, DS:[BP]
MOV    AX, ES:[SI]
```

前者的源操作数的物理地址为 $16 \times DS + BP$,后者为 $16 \times ES + SI$ 。

五、寄存器相对寻址方式 (Register Relative Addressing)

操作数的有效地址是一个基址或变址寄存器的内容与指令中指定的 8 位或 16 位位移量(Displacement)之和。这种寻址方式与寄存器间接寻址十分相似,主要区别是前者在有效地址上还要加一个位移量。同样,当指令中指定的寄存器是 BX、SI 或 DI 时,段寄存器使用 DS,当指定寄存器是 BP 时,段寄存器使用 SS。

例 3-12 MOV BX, COUNT[SI]

设:DS=3000H,SI=2000H,位移量 COUNT=4000H,(36000H)=5678H

则:物理地址= $16 \times DS + SI + COUNT$

=30000H+2000H+4000H

=36000H

指令执行过程如图 3-4 所示,执行结果为 BX=5678H。

上述指令也可用 MOV BX,[COUNT+SI]这种形式来表示。

这种寻址方式也允许使用段超越前缀,例如

```
MOV    DH, ES:ARRAY[SI]
```


则段地址为 ES, 物理地址 = $16 \times \text{ES} + \text{SI} + \text{ARRAY}$ 。

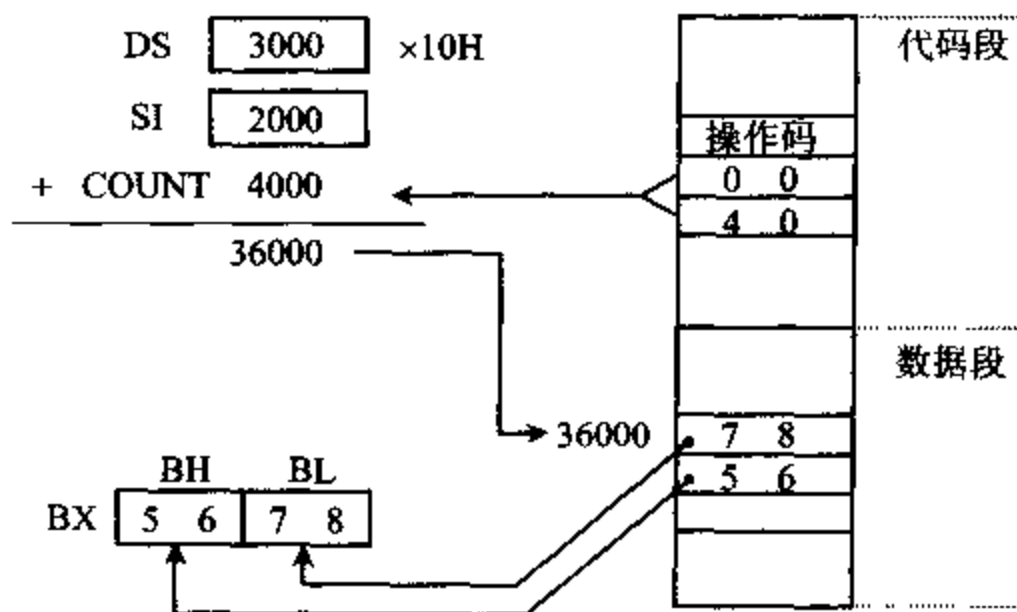


图 3-4 例 3-12 指令执行过程示意图

六、基址变址寻址方式 (Based Indexed Addressing)

操作数的有效地址是一个基址寄存器 (BX 或 BP) 和一个变址寄存器 (SI 或 DI) 的内容之和, 两个寄存器均由指令指定。

若: 基址寄存器为 BX 时, 段址寄存器用 DS

则: 物理地址 = $16 \times \text{DS} + \text{BX} + \text{SI}$

或 = $16 \times \text{DS} + \text{BX} + \text{DI}$

若: 基址寄存器为 BP 时, 段址寄存器应用 SS

则: 物理地址 = $16 \times \text{SS} + \text{BP} + \text{SI}$

或 = $16 \times \text{SS} + \text{BP} + \text{DI}$

例 3-13 MOV AX, [BX][SI]

设: DS = 3000H, BX = 1200H, SI = 0500H, (31700H) = ABCDH

则: 物理地址 = $16 \times \text{DS} + \text{BX} + \text{SI}$

= $30000\text{H} + 1200\text{H} + 0500\text{H}$

= 31700H

该指令的执行过程如图 3-5 所示, 执行结果, AX = ABCDH。指令中的方括号有相加的意思, 所以上述指令也可以写成:

MOV AX, [BX+SI]

七、相对基址变址寻址方式 (Relative Based Indexed Addressing)

操作数的有效地址是一个基址寄存器和一个变址寄存器的内容, 再加上指令中指定的 8 位或 16 位位移量之和。

若: 基址寄存器为 BX 时, 用 DS 作段寄存器

则: 物理地址 = $16 \times \text{DS} + \text{BX} + \text{SI} + 8 \text{ 位或 } 16 \text{ 位位移量}$

或 $= 16 \times DS + BX + DI + 8 \text{ 位或 } 16 \text{ 位位移量}$
 若:基址寄存器为 BP 时,应用 SS 作段寄存器
 则:物理地址 $= 16 \times SS + BP + SI + 8 \text{ 位或 } 16 \text{ 位位移量}$
 或 $= 16 \times SS + BP + DI + 8 \text{ 位或 } 16 \text{ 位位移量}$

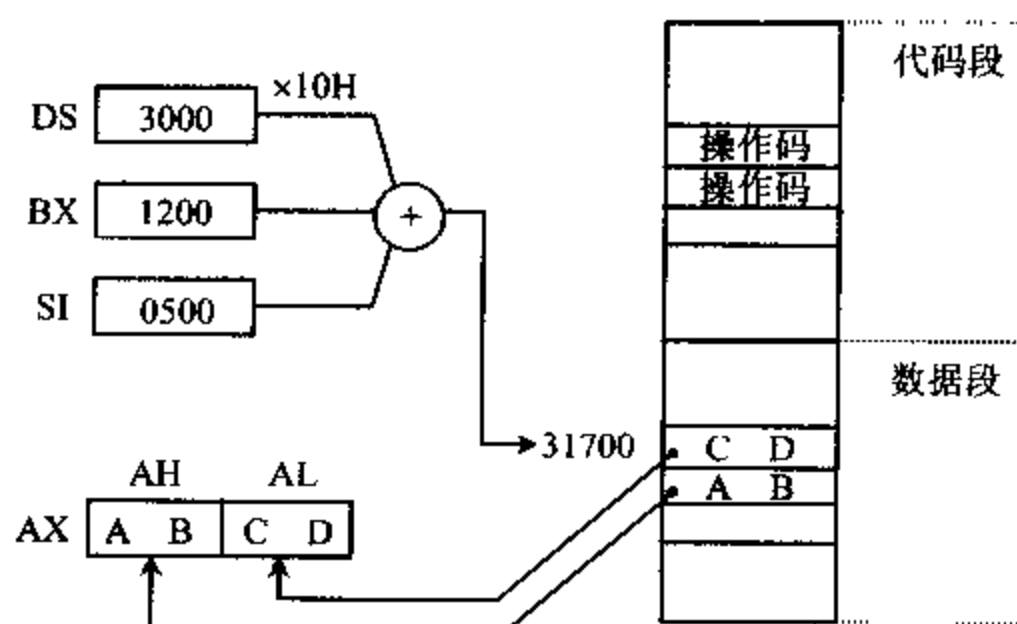


图 3-5 例 3-13 指令执行过程示意图

例 3-14 `MOV AX, MASK[BX][SI]`

设: $DS=2000H, BX=1500H, SI=0300H, MASK=0200H, (21A00H)=26BFH$

则:物理地址 $= 16 \times DS + BX + SI + MASK$

$$= 20000H + 1500H + 0300H + 0200H$$

$$= 21A00H$$

该指令的执行过程如图 3-6 所示,执行结果, $AX=26BFH$ 。

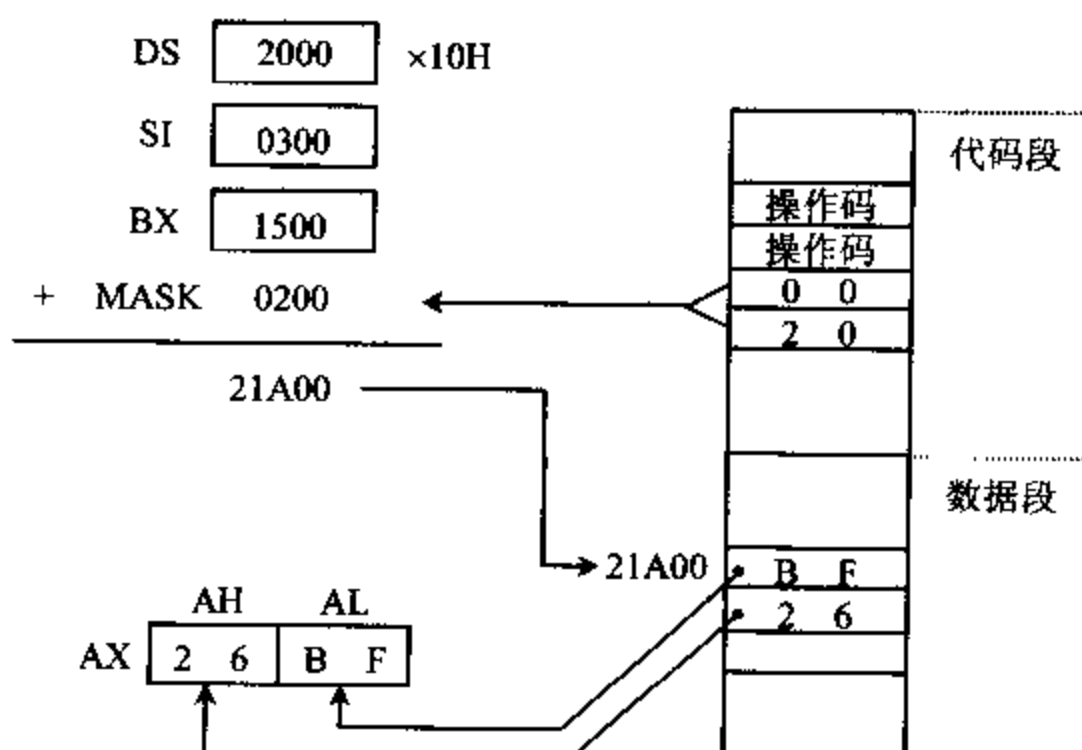


图 3-6 例 3-14 指令执行过程示意图

同样,上述指令也可写成如下几种形式:

`MOV AX, [MASK+BX+SI]`

```
MOV    AX, 200H[BX+SI]
MOV    AX, MASK[BX+SI]
```

从上面的讨论可以看到,在涉及到操作数的地址时,常常要在指令中使用方括号,有关带方括号的地址表达式必须遵循下列规则:

(1) 立即数可以出现在方括号内,表示直接地址,如[2000H]。

(2) 只有 BX、BP、SI、DI 这四个寄存器可以出现在[]内,它们可以单独出现,也可以由几个寄存器组合起来(只能相加),或以寄存器与常数相加的形式出现,但 BX 和 BP 寄存器不允许出现在同一个[]内,SI 和 DI 也不能同时出现。

(3) 由于方括号有相加的含义,下面几种写法都是等价的:

```
6[BX][SI]
[BX+6][SI]
[BX+SI+6]
```

(4) 若方括号内包含 BP,则隐含使用 SS 来提供基地址,它们的物理地址的计算方法为:

$$\text{物理地址} = 16 \times \text{SS} + \text{EA}$$

包含 BP 的操作数有下面三种形式:

```
DISP[BX+SI]      ;EA=BP+SI+DISP
DISP[BX+DI]      ;EA=BP+DI+DISP
DISP[BX]          ;EA=BP+DISP
```

其中,DISP 表示 8 位或 16 位位移量,也可以为 0。

这种情况下也允许用段超越前缀将 SS 修改为 CS、DS 或 ES 中的一个,在计算物理地址时,应将上式中的 SS 改为相应的段寄存器。

其余情况均隐含使用 DS 来提供基地址,它们的物理地址的计算方法为:

$$\text{物理地址} = 16 \times \text{DS} + \text{EA}$$

这类操作数可以有以下几种形式:

```
[DISP]           ;EA=DISP
DISP[BX+SI]      ;EA=BX+SI+DISP
DISP[BX+DI]      ;EA=BX+DI+DISP
DISP[BX]         ;EA=BX+DISP
DISP[SI]         ;EA=SI+DISP
DISP[DI]         ;EA=DI+DISP
```

同样,也可用段超越前缀将上式中的 DS 修改为 CS、ES 或 SS 中的一个。

八、其它

除了上面介绍的立即数寻址方式、寄存器寻址方式和各种存储器寻址方式外,还有以下一些寻址方式。

1. 隐含寻址

指令中不指明操作数,但有隐含规定的寻址方式。例如指令 DAA, 它的含义是对寄存

器 AL 中的数据进行十进制数调整,结果仍保留在 AL 中。

2. I/O 端口寻址

8086 有直接端口和间接端口两种寻址方式。在直接端口寻址方式中,端口地址由指令直接提供,它是一个 8 位立即数。由于一个 8 位二进制数的最大值为 $2^8-1=255$,所以在这种寻址方式中,能访问的端口号为 00~FFH,即 256 个端口。

例 3-15 IN AL, 63H

表示将端口 63H 中的内容送进 AL 寄存器。

在间接寻址方式中,被寻址的端口号由寄存器 DX 提供,这种寻址方式能访问多达 $2^{16}=64K$ 个 I/O 端口,端口号为 0000~FFFFH。

例 3-16

```
MOV    DX, 213H      ;DX=口地址号 213H
IN      AL, DX        ;AL←端口 213H 中的内容
```

3. 一条指令有几种寻址方式

上面介绍的各种寻址方式都是针对源操作数的,目的操作数均用寄存器来表示。实际上,目的操作数也可以用除立即寻址方式以外的所有寻址方式指定,许多指令还具有各自的隐含规则,所以一条指令可能包含几种寻址方式。

对于 MOV 指令来讲,当源操作数用存储器方式寻址时,不仅要求出源操作数的地址,还要将该地址中的内容取出来,作为源操作数,再送到目的操作数中去。而当目的操作数用存储器方式寻址时,只要求出目的操作数的物理地址,就可将源操作数送到这个地址中去。对于寄存器作源操作数时,或目的操作数时,情况也是类似的。

例 3-17 MOV [BX], AL

设:BX=3600H,DS=1000H,AL=05H

则:目的操作数的物理地址 = $16 \times DS + BX$
= 10000H + 3600H
= 13600H

指令执行结果为 (13600H) = 05H。

4. 转移类指令寻址

有关这类指令的寻址方式,将在本章后面讨论控制转移指令时再作介绍。

3-2 指令的机器码表示方法

一、机器语言指令的编码目的和特点

1. 机器语言指令

用汇编语言(即主要由指令系统组成的语言)编写的程序称为汇编语言源程序,若直接将它送到计算机,机器并不认识那些构成程序的指令和符号的含义,还必须由汇编程序将源

程序翻译成计算机能认识的二进制机器语言指令(机器码)后,才能被计算机识别和执行,得到运算结果。

通常,计算机用户采用汇编语言编写程序时,一般可不必了解每条指令的机器码。不过,若要透彻了解计算机的工作原理,以及能看懂包含机器码的程序清单,对程序进行正确的调试、排错等,就需要熟悉机器语言。所以我们要简单介绍一下机器语言指令的基本概念和编码方式。

2. 机器语言指令的编码特点

对于 Z80、8085 等 8 位微处理器,进行指令编码是很容易的事,只要有一张指令编码表,汇编语言源程序与机器码之间的对应关系就一目了然,很容易通过查表求出每条指令的机器码。但对于 8086 系统,情况就不那么简单了。

例如,在指令“MOV 目的操作数, BX”中,可用作目的操作数的 16 位寄存器就有 8 个,若用存储器寻址方式指定目的操作数,又可以有多达 24 种编码方式,见表 3-2。这样,目的操作数的编码就有 32 种。如将上述指令改为“MOV BX, 源操作数”,即用 BX 作目的操作数,则源操作数的编码方式又可以有 32 种。结果,仅一条含有 BX 作操作数的 MOV 指令就有 64 种编码格式。同理,用 BL 作目的操作数或源操作数的 MOV 指令,也可以有 64 种不同的编码。

可见,8086 指令的二进制编码是非常多的,很难列出一张 8086 指令与机器语言的对照表。但我们可以为每种基本指令类型给出一个编码格式,对照格式填上不同的数字来表示不同的寻址方式、数据类型等,就能求得每条指令的机器码。指令通常由操作码和操作数等两部分组成,每条指令的操作码很容易从指令编码表中查到。操作数尽管千变万化,看起来十分复杂,但不外乎由寄存器、存储器、立即数和端口地址等这几部分组成。对于寄存器和存储器寻址方式可列表给出编码方式,立即数和 I/O 地址可直接填入指令的编码格式表中。这样也就能方便地求得 8086 指令的机器码。

8086 指令系统采用变长指令,指令的长度可由 1~6 字节组成。最简单的指令是一字节指令,指令中只包含 8 位操作码,没有操作数。如清进位位指令 CLC 的机器码为 1111 1000,可直接从指令编码表中查到。对于大部分指令来说,除了操作码(不一定是 8 位)外,还包含操作数部分,所以要由几个字节组成。不同的指令,其操作码和寻址方式都是不一样的,故指令的长度也不一样。

二、机器语言指令代码的编制

1. 编码格式说明

我们用寄存器之间或寄存器与存储器之间交换数据的 MOV 指令,来说明指令的编码格式,具体格式如图 3-7 所示。

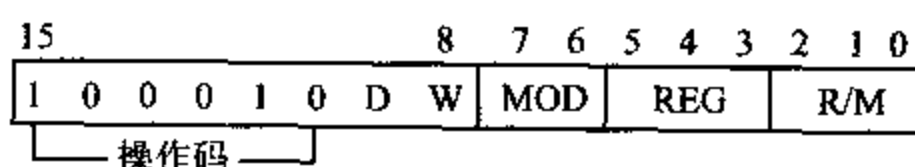


图 3-7 典型的 MOV 指令的编码格式

其中,第一个字节的高6位是操作码100010,W位说明传递数据的类型是字还是字节,W=0,为字节;W=1,是字。D位标明数据传送的方向,D=0,数据从寄存器传出;D=1,数据传至寄存器。寄存器号由第二字节的REG字段说明,用3位编码可寻址8种不同的寄存器,再根据第一字节中W=1还是0,选择8位或16位寄存器。例如,当REG=010,W=1时表示寻址DX寄存器;REG=010,W=0时寻址DL寄存器。8086寄存器字段的编码如表3-1所示(对使用段寄存器的指令,REG字段占2位,其编码也列在表3-1中)。

表 3-1 8086 寄存器编码表

REG	W=1(字)	W=0(字节)	REG	段寄存器
000	AX	AL	01	CS
011	BX	BL	11	DS
001	CX	CL	00	ES
010	DX	DL	10	SS
100	SP	AH		
111	DI	BH		
101	BP	CH		
110	SI	DH		

在这类 MOV 指令中有两个操作数,其中有一个必为寄存器,我们已知道,其编号由 REG 字段决定,另一个操作数可能是寄存器,也可能是存储器单元,由指令代码的第二个字节中的 MOD 和 R/M 字段来指定。表 3-2 给出 MOD 和 R/M 的编码格式,其中 D8 表示 8 位位移量,D16 为 16 位位移量。

表 3-2 MOD 和 R/M 的编码

MOD R/M	00	01	10	11	
				W=0	W=1
000	[BX]+[SI]	[BX]+[SI]+D8	[BX]+[SI]+D16	AL	AX
001	[BX]+[DI]	[BX]+[DI]+D8	[BX]+[DI]+D16	CL	CX
010	[BP]+[SI]	[BP]+[SI]+D8	[BP]+[SI]+D16	DL	DX
011	[BP]+[DI]	[BP]+[DI]+D8	[BP]+[DI]+D16	BL	BX
100	[SI]	[SI]+D8	[SI]+D16	AH	SP
101	[DI]	[DI]+D8	[DI]+D16	CH	BP
110	D16(直接地址)	[BP]+D8	[BP]+D16	DH	SI
111	[BX]	[BX]+D8	[BX]+D16	BH	DI

如果另一个操作数也是寄存器,则 MOD=11,这时可根据寄存器的名称及 W 的状态从表 3-2 中查出 R/M 的编码;W=0 时,3 位 R/M 字段可组成 8 个 8 位寄存器,W=1 时,3 位 R/M 字段组成 8 个 16 位寄存器。如另一个操作数是存储单元,则 MOD≠11,在这种情况下,只要通过简单的计算就能确定有效地址 EA。EA 可包含在寄存器内,也可能是一个或两个寄存器与 8 位(D8)或 16 位(D16)位移量之和。MOD 字段的 3 种编码和 R/M 的 8 种编码,一共可组成 24 种不同的编码格式,即涉及到存储器操作的寻址方式可以有 24 种不同的表示方法。对指令进行编码时,要是指令中包含 8 位位移量,需再增加一个字节存放位移量 disp-L;若包含 16 位的位移量,则要增加 2 个字节存放位移量。第 3 个字节存放位移量

的低字节 disp-L,第 4 个字节存放位移量高字节 disp-H。

下面是对 MOV 指令进行编码的几个示例。

2. 寄存器间传送指令的编码

例 3-18 求指令 MOV SP, BX 的机器码。

这条指令的功能是将 BX 寄存器的内容送到 SP 寄存器中。从附录 B 可知,该指令的操作码为 100010; 因为传送的是字数据,所以 $W=1$; 在指令中有两个寄存器,若选择将 SP 寄存器的编码 100 送到 REG 字段,则 D 必须取 1,表示数据传至所选的寄存器 SP; 由于指令中另一个操作数 BX 也是寄存器,因此 $MOD=11$; 再根据 $W=1$ 及寄存器名称为 BX,从表 3-2 可知 $R/M=011$ 。这样,就可求得图 3-8 所示的指令编码。

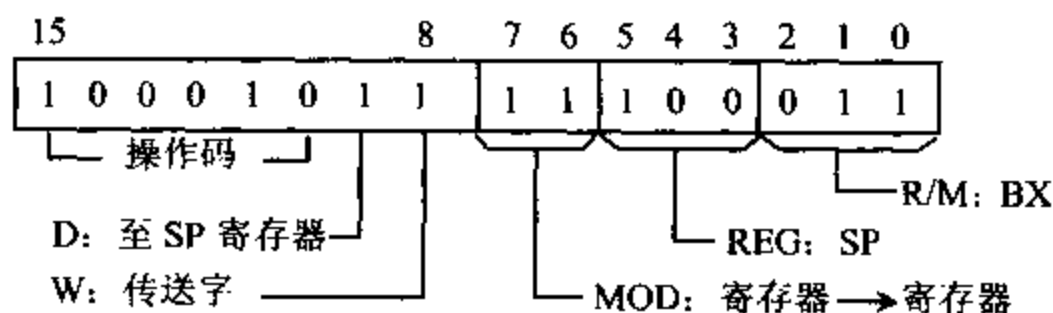


图 3-8 指令 MOV SP, BX 的编码

如果我们将选择 BX 的编码 011 送到 REG 字段,则 $D=0$,表示数据从 BX 传出, R/M 字段填入第二个寄存器 SP 的编码 100,其余同上,这样 MOV SP, BX 指令又可有另一种编码格式,如图 3-9 中所示。

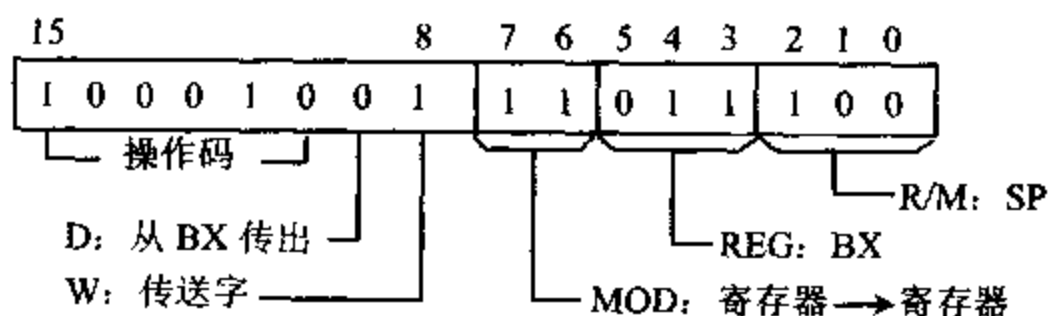


图 3-9 指令 MOV SP, BX 的另一种编码

3. 寄存器与存储器间传送指令的编码

例 3-19 求指令 MOV CL, [BX+1234H] 的机器码。

这条指令的功能是将有效地址为 $(BX+1234H)$ 存储单元中的数据字节传送到 CL 寄存器中,指令的编码如图 3-10 中所示。

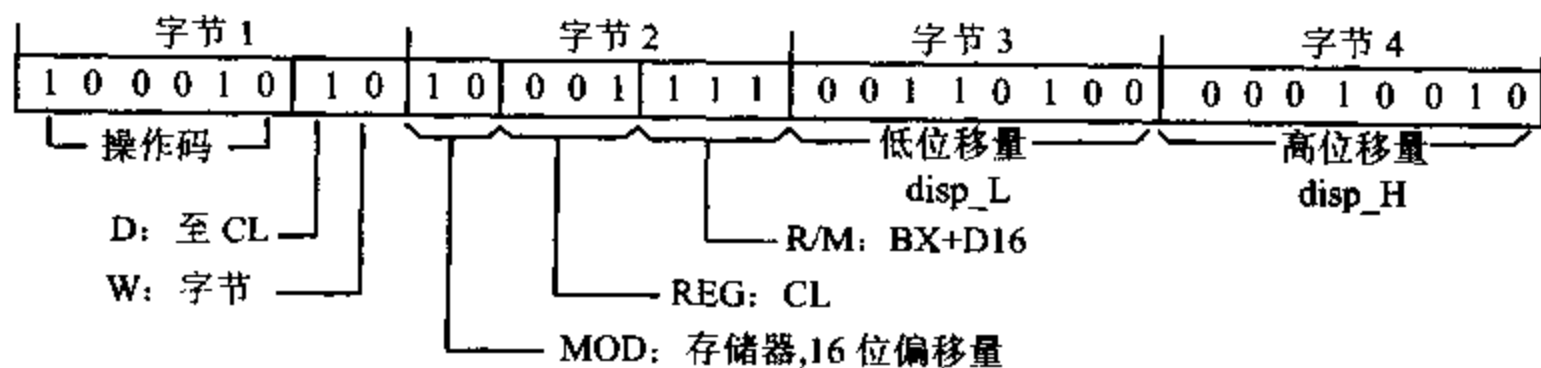


图 3-10 指令 MOV CL, [BX+1234H] 的编码

求该指令编码的第 1、2 字节的方法,与例 3-18 类似,可通过查表得到;第 3 字节存放 16 位位移量的低字节 34H,第 4 字节存放高字节 12H。所以该指令的编码为 8A 8F 34 12H。

4. 立即数寻址指令的编码

对于立即数寻址的指令,除了操作码外,还要有一到两个字节用于存放立即数据。

例 3-20 求指令 MOV DX, 5678H 的机器码。

该指令的功能是将立即数 5678H 送到 DX 寄存器中。从附录 B 可知,这条指令的编码为 3 字节。第 1 字节为“1011 W REG”,其中 1011 为操作码,由于传送的是字数据,所以 W=1,由表 3-2 知,DX 寄存器的编码为 010,即 REG 字段填 010,因此该指令的第一字节编码为 1011 1010。指令的第 2 字节存放立即数的低字节 78H,第 3 字节为高字节 56H。所以这条指令的 16 进制编码为 BA 78 56H。

例 3-21 求指令 MOV [BX+2100H], 0FA50H 的机器码。

该指令的功能是将 16 位立即数 FA50H 送到有效地址为 (BX+2100H) 的字存储单元中,其中低字节 50H 送到 (BX+2100H) 单元,高字节 FAH 送到 (BX+2101H) 单元。它是一个 6 字节指令,指令中不但有 16 位立即数,还有 16 位位移量。指令编码如图 3-11 中所示。

字节 1	字节 2	字节 3	字节 4	字节 5	字节 6
11000111	10000111	00000000	00100001	01010000	11111010
└─操作码─┐ W	MOD	R/M	位移量低字节	位移量高字节	立即数低字节 立即数高字节
└─传送字─┐	└─BX+D16─┐				

图 3-11 指令 MOV [BX+2100H], 0FA50H 的编码

该指令的操作码含两个字节。第一个字节为 1100011W,第二个字节为 MOD 000 R/M,由于传送的是 16 位立即数,所以 W 位=1,指令中所用的存储器寻址方式的编码为 [BX]+D16,由表 3-2 知,MOD=10,R/M=111,第二字节中还有 3 位为 000。这样,可求得该指令的前两个字节为 1100 0111 1000 0111,即 C787H。指令的第 3 和第 4 字节分别为 16 位位移量的低字节 (disp-L) 00H 和高字节 (disp-H) 21H。第 5 和第 6 字节存放立即数低字节 (data-L) 50H 和高字节 (data-H) FAH。因此,该指令的 6 字节编码为 C7 87 00 21 50 FA,在内存中按从低地址到高地址的次序存放。

若指令中的立即数只有 8 位,如指令 MOV [BX+3200H], 86H,则可省去第 6 字节 data-H。对于立即数和位移量均为 8 位的指令,如 MOV [BX+10H], 20H,指令代码的 1、2 字节可查表求得,第 3 字节为 8 位位移量 disp-L,第 4 字节为 8 位立即数 data-L。

5. 包含段寄存器的指令的编码

对于含有段寄存器的指令,寄存器字段 REG 不是占有 3 位而是 2 位,从表 3-1 可查得相应的编码为:CS=01,DS=11,ES=00,SS=10。

例 3-22 求指令 MOV DS, AX 的机器码。

该指令将 AX 寄存器的内容传送到数据段寄存器 DS。该指令的编码格式为:

10001110 MOD 0 REG R/M

这里,段寄存器 DS 的编码为 11,所以 REG 字段为 11,因另一个操作数也是寄存器,所以 MOD=11,而 R/M 字段应填上 AX 的三位代码 000。这样,就可以得到该指令的编码为 1000 1110 1101 1000,即 8E D8H。

6. 段超越前缀指令的编码

对带有段超越前缀的指令进行编码时,要在指令代码前加一个 8 位的段超越前缀代码,

代码的格式为 001××110,其中××位表明段超越寄存器,编码与上面列出的相同。指令的其它代码仍按前面的方法求得。

例 3-23 若指令 MOV [BX],DL 的编码为 88 17H,试求指令 MOV CS:[BX],DL 的代码。

如上所述,该指令的编码是在不带段超越前缀的指令代码 88 17 前,加上一个字节 001××110。由于段寄存器 CS 的代码为 01,所以指令的第 1 个字节的编码为 00101110,即 2EH。由此可得到该指令的机器码为 2E 88 17H。

3-3 8086 的指令系统

按功能分类,8086 的指令共有六大类,它们是:数据传送指令、算术运算指令、逻辑运算和移位指令、字符串处理指令、控制转移指令以及处理器控制指令。8086 的所有指令也都适用于 8088。

下面分别介绍各类指令的格式、功能和应用实例,最后说明一下指令的执行时间问题。

一、数据传送指令

数据传送指令共 14 条,列于表 3-3 中,它们可完成寄存器和寄存器之间、寄存器与存储器之间、累加器 AX 或 AL 与 I/O 端口之间的字或字节的传送,堆栈操作指令也归入这一类,此外还包括一组标志传送指令和一组地址传送指令。这类指令中,除 SAHF 和 POPF 指令外,对标志位均没有影响。

表 3-3 数据传送指令

通用数据传送指令	
MOV	字节或字的传送
PUSH	入栈指令
POP	出栈指令
XCHG	交换字或字节
XLAT	表转换
输入输出指令	
IN	输入
OUT	输出
地址目标传送指令	
LEA	装入有效地址
LDS	装入数据段寄存器
LES	装入附加段寄存器
标志传送指令	
LAHF	标志寄存器低字节装入 AH
SAHF	AH 内容装入标志寄存器低字节
PUSHF	标志寄存器入栈指令
POPF	出栈,并送入标志寄存器

1. 通用数据传送指令 (General Purpose Data Transfer)

(1) MOV 传送指令(Move)

指令格式: MOV 目的, 源

指令功能: 将源操作数(一个字节或一个字)传送到目的操作数。

指令中至少要有一项明确说明传送的是字节还是字, MOV 指令允许数据传送的途径如图 3-12 所示。

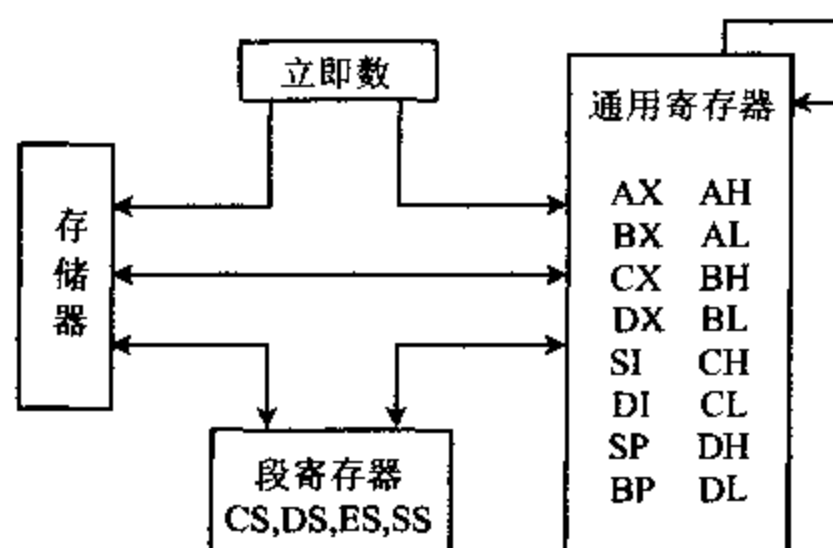


图 3-12 MOV 指令允许传送数据的途径

从图中可知, MOV 指令允许在 CPU 的寄存器之间、存储器和寄存器之间传送字节和字数据, 也可将立即数送到寄存器或存储器中。但是要注意, IP 寄存器不能用作源操作数或目的操作数, 目的操作数也不允许用立即数和 CS 寄存器。另外, 除了源操作数为立即数的情况外, 两个操作数中必有一个是寄存器, 但不能都是段寄存器。这就是说, MOV 指令不能在两个存储单元之间直接传送数据, 也不能在两个段寄存器之间直接传送数据。有关 MOV 指令的用法, 前两节中已举了不少例子, 下面再举一些例子来进一步说明它的功能与用法。

例 3-24 MOV AL, 'B'

这条 MOV 指令把字符 B 的 ASCII 码(42H)传送到 AL 寄存器中。

例 3-25 MOV AX, DATA

MOV DS, AX

设 DATA 为数据段的段地址值, 则这两条指令执行后将数据段的段地址值 DATA 送入 DS 寄存器。由于 DATA 是一个 16 位立即数, 不能被直接送进 DS, 需要先送进另一个数据寄存器(本例中为 AX), 然后从这个数据寄存器传到 DS 中。在程序开头放进这两条指令后, DS 便指向当前数据段, 这样, 对数据段中所有数据进行存取时, 就不用再考虑这些数据所在位置处的段地址了。

下面的一些例子, 将要进一步涉及到数据段的基地址和数据段中各变量的偏移地址的概念, 为此, 我们在这里对这些概念做些初步的介绍, 更详细的讨论将在下一章中进行。

在汇编语言程序中, 数据通常存放在数据段中。例如, 下面是某个程序的数据段:

```
DATA    SEGMENT           ;数据段开始
AREA1   DB  14H, 3BH
```

```

AREA2  DB  3  DUP(0)
ARRAY  DW  3100H, 01A6H
STRING DB  'GOOD'
DATA   ENDS
;数据段结束

```

AREA1	1	4
	3	B
AREA2	0	0
	0	0
	0	0
ARRAY	0	0
	3	1
	A	6
STRING	0	1
	'G'	
	'O'	
	'O'	
	'D'	

图 3-13 数据段占用存储空间的情况

这里,数据段以段说明符 SEGMENT 开始,ENDS 结束,DATA 是数据段的段名。DB 伪操作符用来定义字节变量,说明其后的每一个操作数都占一个字节。DW 伪操作符则定义字变量,说明其后的每一个操作数都占一个字,并且低字节数据放在低地址单元中,高字节数据放在高地址单元中。DUP 是复制操作符,它前面的数字“3”用来说明在存储器中保留 3 个字节单元,它们的初值均为 0。

经过汇编后,DATA 将被赋予一个具体的段地址,而各变量将自偏移地址 0000H 开始依次存放,各符号地址也被赋予确定的值,等于它们在数据段中的偏移量。例如,上述的数据段经汇编之后占用存储空间的情况如图 3-13 所示。由图可见,AREA1 的偏移地址为 0000H, AREA2 的偏移地址为 0002H, ARRAY 的偏移地址为 0005H 等。

例 3-26 MOV DX, OFFSET ARRAY

将 ARRAY 的偏移地址送到 DX 寄存器中,其中 OFFSET 为属性操作符,表示应把跟在后面的符号地址的值(而不是内容)作为操作数。如 ARRAY 的值如图 3-13 所示,则指令执行后,符号地址 ARRAY 的偏移量 0005H 被送到了 DX 寄存器中。

```

例 3-27  MOV  AL, AREA1      ;AL←AREA1 中的内容 14H
          MOV  AREA2, AL      ;0002H 单元←14H

```

设 AREA1 和 AREA2 的值仍如图 3-13 所示,则第一条 MOV 指令将 AREA1 存储单元中的内容(14H)送进 AL,再由第二条 MOV 指令传送到 AREA2 存储单元中。

例 3-28 MOV AX, TABLE[BP][DI]

将地址为 $16 \times SS + BP + DI + \text{TABLE}$ 的字存储单元中的内容送进 AX。

(2) PUSH 进栈指令(Push Word onto Stack)

指令格式: PUSH 源

指令功能: 将源操作数推入堆栈。

源操作数可以是 16 位通用寄存器、段寄存器或存储器中的数据字,但不能是立即数。堆栈是以“先进后出”的方式工作的一个存储区,栈区的段址由 SS 寄存器的内容确定。堆栈的最大容量可为 64K,即一个段的最大容量。堆栈指针 SP 始终指向栈顶,其值可以从 FFFEh(偶地址)开始,向低地址方向发展,最小为 0。

每次执行 PUSH 操作时,先修改 SP 的值,使 $SP \leftarrow SP - 2$ 后,然后把源操作数(字)压入堆栈中 SP 指示的位置上,低位字节放在较低地址单元(真正的栈顶单元),高位字节放在较

高地址单元。由于堆栈操作都是以字为单位进行的,所以 SP 总是指向偶地址单元。SS 和 SP 的值可由指令设定。

(3) POP 出栈指令 (Pop Word off Stack)

指令格式: POP 目的

指令功能: 把当前 SP 所指向的堆栈顶部的一个字送到指定的目的操作数中。

目的操作数可以是 16 位通用寄存器、段寄存器或存储单元,但 CS 不能作目的操作数。每执行一次出栈操作, $SP \leftarrow SP + 2$, 即 SP 向高地址方向移动, 指向新的栈顶。

下面举例说明堆栈指令的操作过程。

例 3-29 设 $SS=2000H$, $SP=40H$, $BX=3120H$, $AX=25FEH$, 依次执行下列指令:

PUSH BX

PUSH AX

POP BX

堆栈中的数据和 SP 的变化情况如图 3-14 所示。

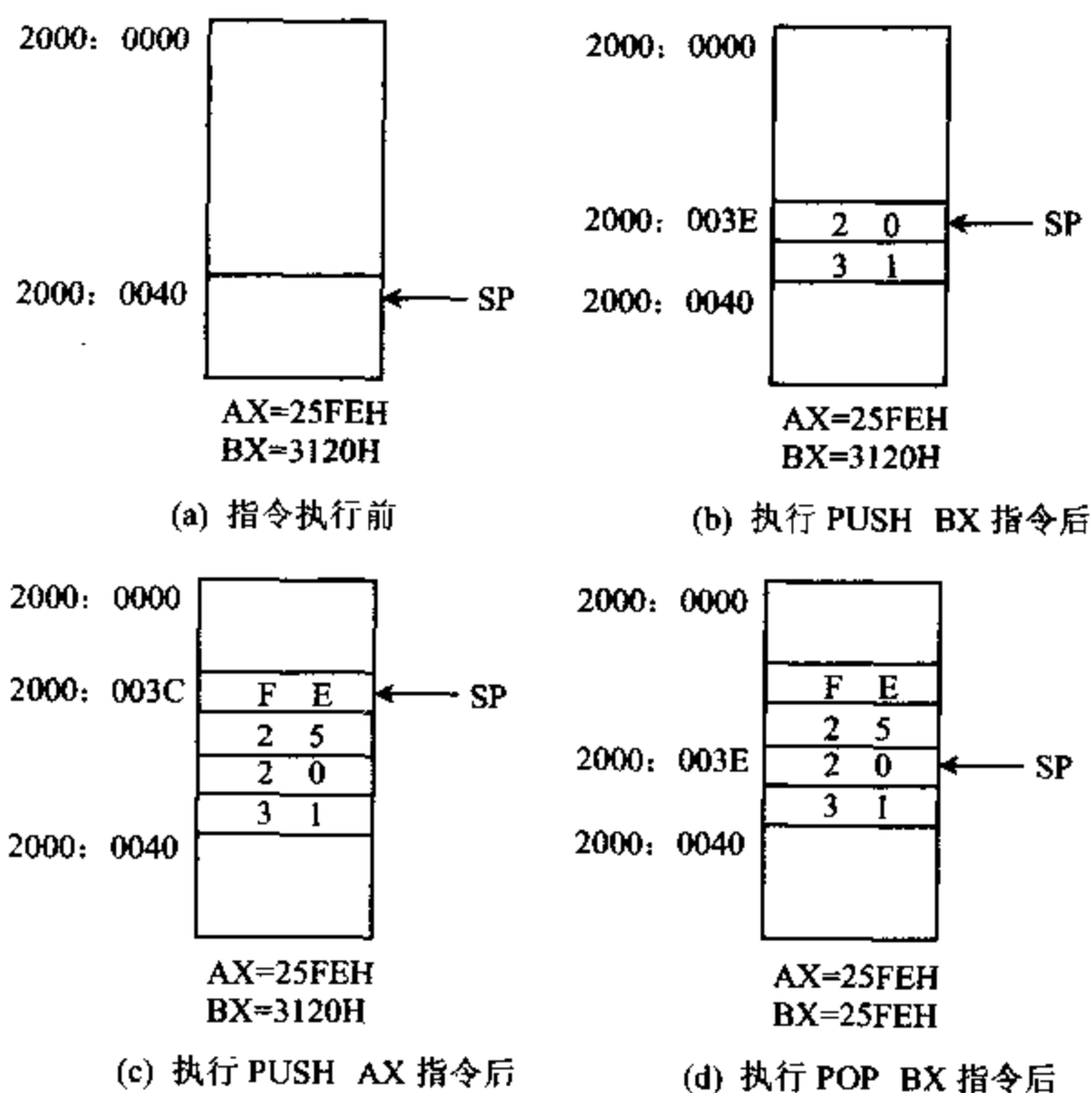


图 3-14 PUSH 和 POP 指令执行过程

(4) XCHG 交换指令 (Exchange)

指令格式: XCHG 目的, 源

指令功能: 把一个字或字节的源操作数和目的操作数相交换。

交换可以在寄存器之间、寄存器与存储器之间进行,但段寄存器不能作为操作数,也不

能直接交换两个存储单元中的内容。

例 3-30 设 $AX = 2000H$, $DS = 3000H$, $BX = 1800H$, $(31A00H) = 1995H$, 执行下面指令:

XCHG $AX, [BX+200H]$

它把内存中的一个字与 AX 中的内容进行交换, 源操作数的物理地址 $= 3000 \times 10H + 1800H + 200H = 31A00H$, 该地址处存放的字数据为 $1995H$ 。因此, 指令执行后,

$AX = 1995H$, $(31A00H) = 2000H$

(5) XLAT 表转换指令 (Table Lookup-Translation)

指令格式: XLAT 转换表

或 XLAT

指令功能: 将一个字节从一种代码转换成另一种代码。

我们经常要用查表的方式实现代码转换。例如, 在控制 LED 显示器时, 需要根据所要显示的数字, 查得它的七段码; 在进行声响报警时, 则要根据当前要报警的内容, 选定送给定时电路的时间常数等。XLAT 指令能够简化这种查表操作。

使用 XLAT 指令之前必须先建立一个表格, 并将转换表的起始地址装入 BX 寄存器中。 AL 中事先也要送一个初值, 这个值等于表头地址与所要查找的某一项之间的位移量。表格中的内容则是所需要转换的代码, 表格最多包含 256 个字节。执行 XLAT 指令后, 根据位移量可以从表中查到转换后的代码值, 并自动送入 AL 寄存器中, 得到所需结果。

XLAT 指令有两种格式, 其中第一种格式中的“转换表”为表格的首地址, 一般用符号表示, 以提高程序的可读性, 但它也可以省略, 即用第二种格式。

例 3-31 若十进制数字 0~9 的 LED 七段码对照表如表 3-4 所示, 试用 XLAT 指令求数字 5 的七段码值。

表 3-4 十进制数的七段显示码表

十进制数字	七段显示码	十进制数字	七段显示码
0	40H	5	12H
1	79H	6	02H
2	24H	7	78H
3	30H	8	00H
4	19H	9	18H

首先用 DB 伪指令在内存中建立一个表格, 用于存放数字 0~9 的七段码值, 若表格起始地址为 TABLE, 则数字 0~9 的七段码分别存放在相对于 TABLE 的位移量为 0~9 的单元中。

执行 XLAT 指令前, 应先把表格首地址 TABLE 送入 BX , 再将数字 5 的七段码在表格中的位移量送 AL 中, 然后执行 XLAT 指令, 这样就可可在 AL 中得到数字 5 的七段代码值 12H。程序如下:

TABEL DB 40H, 79H, 24H, 30H, 19H ;七段码表格,

```

        DB 12H, 02H, 78H, 00H, 18H
        :
        MOV AL, 5                ;AL←数字 5 的位移量
        MOV BX, OFFSET TABLE   ;BX←表格首地址
        XLAT TABLE              ;查表得 AL=12H

```

2. 输入输出指令 (Input and Output)

输入输出指令用来完成 I/O 端口与累加器之间的数据传送,指令中给出 I/O 端口的地址值。当执行输入指令时,把指定端口中的数据读入累加器中,执行输出指令时,则把累加器中的数据写入指定的端口中。

(1) IN 输入指令 (Input)

指令格式:

- ① IN AL, 端口地址
- 或 IN AX, 端口地址
- ② IN AL, DX ;端口地址存放在 DX 寄存器中
- 或 IN AX, DX

指令功能:从 8 位端口读入一个字节到 AL 寄存器,或从 16 位端口读一个字到 AX 寄存器。16 位端口由两个地址连续的 8 位端口组成,从 16 位端口输入时,先将给定端口中的字节送进 AL,再把端口地址加 1,然后将该端口中的字节读入 AH。

IN 指令有两种格式。第一种格式,端口地址(00~FFH)直接包含在 IN 指令里,共允许寻址 256 个端口。由于 8086 CPU 可以直接访问地址为 0000~FFFFH 的 64K 个 I/O 端口,当端口地址号大于 FFH 时,必须用第二种寻址方式,即先将端口号送入 DX 寄存器,再执行输入操作。

例 3-32 下面是用 IN 指令从输入端口读取数据的几个具体例子:

```

IN AL, 0F1H        ;AL←从 F1H 端口读入一个字节
IN AX, 80H          ;AL←80H 口的内容
                   ;AH←81H 口的内容
MOV DX, 310H        ;端口地址 310H 先送入 DX 中
IN AL, DX           ;AL←310H 端口的内容

```

例 3-33 IN 指令中也可使用符号来表示地址,例如下面指令从一个模/数(A/D)转换器读入一个字节的数字量到 AL 中:

```

ATOD EQU 54H        ;A/D 转换器口地址为 54H
IN AL, ATOD          ;将 54H 口的内容读入 AL 中

```

(2) OUT 输出指令 (Output)

指令格式:

- ① OUT 端口地址, AL
- 或 OUT 端口地址, AX
- ② OUT DX, AL ;DX=端口地址

或 OUT DX, AX

指令功能:将 AL 中的一个字节写到一个 8 位端口,或把 AX 中的一个字写到一个 16 位端口。同样,对 16 位端口进行输出操作时,也是对两个连续的 8 位端口进行输出操作。

例 3-34 下面是几个用 OUT 指令对输出端口进行操作的例子:

OUT	85H, AL	;85H 端口←AL 内容
MOV	DX, 0FF4H	
OUT	DX, AL	;FF4H 端口←AL 内容
MOV	DX, 300H	;DX 指向 300H
OUT	DX, AX	;300H 端口←AL 内容
		;301H 端口←AH 内容

3. 地址目标传送指令 (Address Object Transfers)

这是一类专用于传送地址码的指令,它可以用来传送操作数的段地址和偏移地址,共包含以下三条指令:

(1) LEA 取有效地址指令 (Load Effective Address)

指令格式:LEA 目的, 源

指令功能:取源操作数地址的偏移量,并把它传送到目的操作数所在单元。

LEA 指令要求源操作数必须是存储单元,而且目的操作数必须是一个除段寄存器之外的 16 位寄存器。使用时要注意它与 MOV 指令的区别,MOV 指令传送的一般是源操作数中的内容而不是地址。

例 3-35 假设 SI=1000H, DS=5000H, (51000H)=1234H

执行指令 LEA BX, [SI]后, BX=1000H

执行指令 MOV BX, [SI]后, BX=1234H

有时,LEA 指令也可以用取偏移地址的 MOV 指令代替。

例 3-36 下面两条指令就是等价的,它们都取 TABLE 的偏移地址,然后送到 BX 中,即

LEA	BX, TABLE
MOV	BX, OFFSET TABLE

但有些时候,必须使用 LEA 指令来完成某些功能,不能用 MOV 指令取代 LEA。

例 3-37 某数组含 20 个元素,每个元素占一个字节,序号为 0~19。设 DI 指向数组开头处,如要把序号为 6 的元素的偏移地址送到 BX 中,不能直接用 MOV 指令来实现,必须使用下面指令:

LEA BX, 6[DI]

(2) LDS 将双字指针送到寄存器和 DS 指令 (Load Pointer using DS)

指令格式:LDS 目的, 源

指令功能:从源操作数指定的存储单元中,取出一个变量的 4 字节地址指针,送进一对目的寄存器。其中前两个字节(表示变量的偏移地址)送到指令中指定的目的寄存器中,后

两个字节(表示变量的段地址)送入 DS 寄存器。

指令中源操作数必须是存储单元,从该单元开始的连续 4 个字节单元中,存放着一个变量的地址指针。目的操作数必须是 16 位寄存器,常使用 SI 寄存器,但不能用段寄存器。

例 3-38 设 DS=1200H, (12450H)=F346H, (12452H)=0A90H

执行指令 LDS SI, [450H]后, SI=F346H, DS=0A90H

(3) LES 将双字指针送到寄存器和 ES 指令 (Load Pointer using ES)

指令格式: LES 目的, 源

指令功能: 这条指令与 LDS 指令的操作基本相同, 所不同的是要将源操作数所指向的地址指针中的段地址部分送到 ES 寄存器中, 而不是 DS 寄存器, 目的操作数常用 DI 寄存器。

例 3-39 设 DS=0100H, BX=0020H, (01020H)=0300H, (01022H)=0500H

执行指令 LES DI, [BX]后, DI=0300H, ES=0500H

4. 标志传送指令 (Flag Transfers)

可完成标志位传送的指令共有 4 条。

(1) LAHF 标志送到 AH 指令 (Load AH from Flags)

指令格式: LAHF

指令功能: 把标志寄存器 SF、ZF、AF、PF 和 CF 分别传送到 AH 寄存器的位 7、6、4、2 和 0, 位 5、3、1 的内容未定义, 可以是任意值。执行这条指令后, 标志位本身并不受影响。这 5 个标志送进 AH 后, AH 便相当于 8080/8085 的标志寄存器, 从而能对 8080/8085 程序进行转换, 使它们能运行在 8086/8088 系统上。图 3-15 是它的操作示意图。

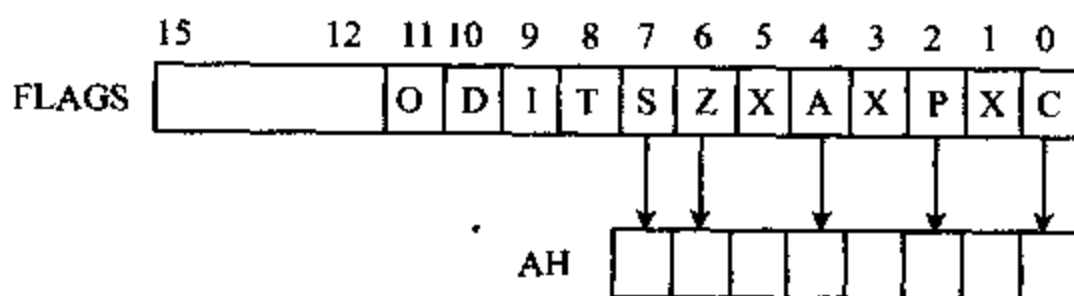


图 3-15 LAHF 指令传送标志的操作

(2) SAHF AH 送标志寄存器 (Store AH into Flags)

指令格式: SAHF

指令功能: 把 AH 内容存入标志寄存器。这条指令与 LAHF 的操作相反, 它把寄存器 AH 中的 7、6、4、2、0 位传送到标志寄存器的 SF、ZF、AF、PF 和 CF 位, 高位标志 OF、DF、IF 和 TF 不受影响。同样, 这条指令也是为 8080/8085 提供兼容性。

(3) PUSHF 标志入栈指令 (Push Flags onto Stack)

指令格式: PUSHF

指令功能: 把整个标志寄存器的内容推入堆栈, 同时修改堆栈指针, 使 $SP \leftarrow SP - 2$, 这条指令执行后对标志位无影响。

(4) POPF 标志出栈指令 (Pop Flags off Stack)

指令格式: POPF

指令功能:把当前堆栈指针 SP 所指的一个字,传送给标志寄存器 PSW,并修改堆栈指针,使 $SP \leftarrow SP + 2$ 。

成对地使用 PUSHF 和 POPF 这两条指令,可对标志寄存器进行保存和恢复,常用在过程(子程序)调用和中断服务程序的开头与结尾处,对进行过程调用或发生中断时主程序的状态(即标志位)进行保护。

另外,这两条指令也可用来改变追踪标志 TF。在 8086 指令系统中没有直接改变 TF 的指令,若要改变 TF,可先用 PUSHF 指令将标志推入堆栈,然后设法改变栈顶字存储单元的 D₈ 位,再用 POPF 指令把堆栈中修改过的内容传送回标志寄存器。这样,只有 TF 标志按需要改变了,而其余标志未受影响。

二、算术运算指令

算术运算指令可处理 4 种类型的数:无符号二进制整数、带符号二进制整数、无符号压缩十进制整数(Packed Decimal)和无符号非压缩十进制整数(Unpacked Decimal)。二进制数可以是 8 位或 16 位长,若是带符号数,则用补码表示。压缩十进制数可在一个字节中存放两个 BCD 码十进制数。若只在一个字节的低半字节存放一个十进制数,而高半字节为零,这种数称为非压缩十进制数。例如,对于十进制数字 58,用压缩十进制数表示时,只需一个字节,即 0101 1000B;当表示成非压缩十进制数时,需要两个字节,即 0000 0101B 和 0000 1000B。

上述 4 种类型的数的表示方法概括于表 3-5 中。从表中可见,对于一个 8 位二进制数,把它看成 4 种不同类型的数时,所表示的数值是不同的。例如,对 8 位二进制数 1000 1001B,当你把它看成无符号二进制数时,表示十进制数的 137;作为带符号二进制数时,表示十进制数的一119;当它作为非压缩十进制数时,是无效的,因为它的高 4 位不是全零;把它看作为压缩十进制数时,表示 89。而对于另一个二进制数 1100 0101B,由于其高 4 位为二进制数 1100,不能用来表示一位 BCD 码,所以作为非压缩和压缩十进制数均是无效的。算术运算指令处理的数都必须是有有效的,否则会导致错误的结果。

表 3-5 4 种类型数的表示方法

二进制码 (B)	十六进制 (H)	无符号二 进制(D)	带符号二 进制(D)	非压缩 十进制	压缩 十进制
0000 0111	07	7	+7	7	07
1000 1001	89	137	-119	无效	89
1100 0101	C5	197	-59	无效	无效

8086/8088 指令系统提供了加、减、乘、除四种基本运算指令,可处理无符号或带符号的 8 位或 16 位二进制数的算术运算,还提供了各种调整操作指令,故可进行压缩的或非压缩的十进制数的算术运算。绝大部分算术运算指令都影响状态标志位。对于加法和减法运算指令,带符号数和无符号数的加法和减法运算的操作过程是一样的,故可以用同一条加法或减法指令来完成。而对于乘法和除法运算,带符号数和无符号数的运算过程完全不同,必须分别设置无符号数的乘除法指令。表 3-6 列出了各种算术运算指令的符号和意义,下面将

对它们一一进行介绍。

表 3-6 算术逻辑指令

加 法	
ADD	加法
ADC	带进位的加法
INC	增量
AAA	加法的 ASCII 调整
DAA	加法的十进制调整
减 法	
SUB	减法
SBB	带借位的减法
DEC	减量
NEG	取负
CMP	比较
AAS	减法的 ASCII 调整
DAS	减法的十进制调整
乘 法	
MUL	无符号数乘法
IMUL	整数乘法
AAM	乘法的 ASCII 调整
除 法	
DIV	无符号数除法
IDIV	整数除法
AAD	除法的 ASCII 调整
CBW	把字节转换成字
CWD	把字转换成双字

1. 加法指令 (Addition)

(1) ADD 加法指令 (Addition)

指令格式: ADD 目的, 源

指令功能: 将源和目的操作数相加, 结果送到目的操作数中, 即

目的 $\leftarrow$ 源 + 目的

(2) ADC 带进位的加法指令 (Addition with Carry)

指令格式: ADC 目的, 源

指令功能: 功能与 ADD 类似, 只是在两个操作数相加的同时, 还要把进位标志 CF 的当前值加进去作为和, 再把结果送到目的操作数中, 即

目的 $\leftarrow$ 源 + 目的 + CF

这两条指令的源操作数可以是寄存器、存储器或立即数, 目的操作数只能用寄存器和存储单元, 存储单元可以由表 3-2 中所示的 24 种表示方法。但要注意源和目的操作数不能同时为存储器, 而且它们的类型必须一致, 即都是字节或字。

例 3-40 下面列举这两种加法指令的实例, 来说明它们的用法:

```
ADD    AL, 18H          ;AL←AL+18H
ADC    BL, CL           ;BL←BL+CL+CF
```

ADC	AX, DX	;AX←AX+DX+CF
ADD	AL, COST[BX]	;将 AL 内容和物理地址=DS:(COST+BX)的 ;存储字节相加,结果送到 AL 中
ADD	COST[BX], BL	;将 BL 与物理地址=DS:(COST+BX)的存储 ;字节相加,结果留在该存储单元中

这两条指令影响的标志位为: CF、OF、PF、SF、ZF 和 AF。

例 3-41 试用加法指令对两个 8 位 16 进制数 5EH 和 3CH 求和,并分析加法运算指令执行后对标志位的影响。

我们用 AL 和 BL 寄存器来存放这两个数,然后用 ADD 指令求和,结果将留在 AL 之中。即

MOV	AL, 5EH	;AL=5EH (94)
MOV	BL, 3CH	;BL=3CH (60)
ADD	AL, BL	;结果 AL=9AH

为讨论 ADD 指令执行时对标志位的影响情况,我们将上述两个数的相加过程,用算式表示:

$$\begin{array}{r} 0101 \ 1110 \\ + \ 0011 \ 1100 \\ \hline 1001 \ 1010 \end{array}$$

运算后标志位: ZF=0, AF=1, CF=0, SF=1, PF=1, OF=1。

这是因为运算结果非 0,故零标志 ZF=0;低 4 位向高 4 位有进位,所以半进位标志 AF=1;D₇ 位没有产生进位,进位标志 CF=0;因 D₇=1,符号标志 SF=1;结果中有偶数个 1,使奇偶标志 PF=1;对于溢出标志 OF,计算机根据两个数以及它们的结果的符号来决定,当两个加数的符号相同,而结果的符号与之相反时,OF=1。

如何对这些标志进行解释,取决于你编写的程序,或者说是人为决定的。

例如,若编程人员将上述两个加数都看成无符号数,则 D₇ 位也代表数据位,不是符号位,运算结果为 9AH,即十进制数 154。如果两数之和超过 FFH,进位标志 CF 将置 1。在这种情况下,SF 标志和 OF 标志都没有意义。因此对于无符号数,编程人员通常关心的只是 ZF 和 CF 标志。在进行 BCD 码运算或需要进行奇偶校验时,才考虑 AF 或 PF 标志。

要是将这两个加数都当成带符号数,符号标志 SF 和溢出标志 OF 就很重要了,而进位标志 CF 却没有意义。本例中,表示两个正数 94 和 60 相加,其和为 154,由于 154 超过了带符号数能表示的范围-128~+127,即产生了溢出,故 OF 标志置 1。这时虽然符号标志 SF=1,但由于 OF=1,运算出错,结果无效。如果没有产生溢出(即 OF=0),SF=1 时,表示结果为负数。

对运算结果的标志位进行判断,并作相应的处理,需要使用条件转移和过程调用指令,我们将在后面的章节里进一步讨论。

(3) INC 增量指令 (Increment)

指令格式: INC 目的

指令功能:对目的操作数加 1,结果送回目的操作数。即

目的 $\leftarrow$ 目的 + 1

目的操作数可以在通用寄存器或内存中。这条指令主要用在循环程序中,对地址指针和循环计数器等进行修改。指令执行后影响 AF、OF、PF、SF 和 ZF,但进位标志 CF 不受影响。

例 3-42 下面是 INC 指令的两个应用例子:

```
INC    BL                ;BL 寄存器中内容增 1
INC    CX                ;CX 寄存器中内容增 1
```

由于该指令只有一个操作数,如果要使内存单元的内容增 1,则程序中必须有说明该存储单元是字还是字节的符号或说明语句。

例 3-43

```
INC    BYTE PTR[BX]      ;内存字节单元内容增 1
INC    WORD PTR[BX]      ;内存字单元内容增 1
```

这里 PTR 为类型说明符,前面加 BYTE 说明操作数类型为字节,加 WORD 则说明操作数类型为字。对于操作数类型不清楚的指令,可用 PTR 操作符进行说明。

(4) AAA 加法的 ASCII 调整指令 (ASCII Adjust for Addition)

指令格式: AAA

指令功能:在用 ADD 或 ADC 指令对两个非压缩十进制数或 ASCII 码表示的十进制数作加法后,运算结果已存在 AL 的情况下,用此指令将 AL 寄存器中的运算结果调整为 1 位非压缩十进制数,仍保留在 AL 中,如果 AF=1,表示向高位有进位,则进到 AH 寄存器中。

如前所述,非压缩十进制数的高 4 位为全 0,低 4 位为十进制数字 0~9。例如,将 9 表示成 0000 1001,而 5 为 0000 0101 等。

AAA 指令执行时,将对 AL 中的运算结果进行如下调整:

若 AL 低 4 位 > 9 或半进位标志 AF=1

则① $AL \leftarrow AL + 6$

② 用与操作 (AND) 将 AL 高 4 位清 0

③ AF 置 1,CF 置 1,AH $\leftarrow$ AH + 1

否则,仅将 AL 寄存器的高 4 位清 0。

例 3-44 若 AL=BCD 9, BL=BCD 5,求两数之和。设 AH=0,则运算过程如下:

```
ADD  AL,BL      ;      0000 1001 ... 9
                ; + 0000 0101 ... 5
                ; -----
AAA             ;      0000 1110 ... 低 4 位 > 9
                ; + 0000 0110 ... 加 6 调整
                ; -----
                ;      0001 0100
                ; AND 0000 1111 ... 清高 4 位
                ; -----
                ;      0000 0100 ... AL=4
                ;      CF=1, AF=1, AH=1
                ; 结果为 AX=0104H, 表示非压缩十进制数 14
```

用 ASCII 码表示的十进制数,高半字节均为 3,在运算中是多余的,常常需先用 AND 指令将它屏蔽掉(即清为 0)后再进行运算。使用 AAA 指令,可以不必屏蔽高半字节,只要在相加后立即执行 AAA 指令,便能在 AX 中得到一个正确的非压缩十进制数。

例 3-45 求 ASCII 码表示的数 9(39H)与 5(35H)之和。设 AH=0,则运算过程如下:

MOV AL,'9'	;	AL=39H
MOV BL,'5'	;	BL=35H
ADD AL,BL	;	0011 1001 ... '9'
	;	+ 0011 0101 ... '5'
	;	-----
AAA	;	0110 1110 ... 低 4 位 > 9
	;	+ 0000 0110 ... 加 6 调整
	;	-----
	;	0111 0100
	;	AND 0000 1111 ... 清高 4 位
	;	-----
	;	0000 0100 ... AL=4
	;	CF=1, AF=1, AH=1
	;	结果为 AX=0104H, 即非压缩十进制数 14

若想把 AX 中的结果“1”和“4”表示成 ASCII 码,只要用在 AAA 指令后加上一条“或”操作指令 OR AX,3030H,便使 AX 中的结果变成了 ASCII 码 3134H。

(5) DAA 加法的十进制调整指令 (Decimal Adjust for Addition)

指令格式: DAA

指令功能: 将两个压缩 BCD 数相加后的结果调整为正确的压缩 BCD 数。相加后的结果必须在 AL 中,才能使用 DAA 指令。

DAA 指令执行时,调整过程为:

若做加法后 AL 中的低半字节 > 9 或 AF=1

则 $AL \leftarrow AL + 6$, 对低半字节进行调整

若此时 AL 中高半字节结果 > 9 或 CF=1

则 $AL \leftarrow AL + 60H$, 对高半字节进行调整,并使 CF 置 1,否则 CF 置 0。

例 3-46 若 AL=BCD 38, BL=BCD 15, 求两数之和。运算过程如下:

ADD AL,BL	;	0011 1000 ... 38
	;	+ 0001 0101 ... 15
	;	-----
DAA	;	0100 1101 ... 低 4 位 > 9
	;	+ 0000 0110 ... 加 6 调整
	;	-----
	;	0101 0011 ... 结果为 AL=BCD 53, CF=0

例 3-47 若 AL=BCD 88, BL=BCD 49,求两数之和。运算过程为:

ADD	AL,BL	;	1000 1000 ... 88
		;	+ 0100 1001 ... 49
		;	
DAA		;	1101 0001 ... AF=1
		;	+ 0000 0110 ... 加 6 调整
		;	
		;	1101 0111 ... 调整后高半字节>9
		;	+ 0110 0000 ... 加 60H 调整
		;	
		;	0011 0111 ... 结果为 AL=BCD 37, CF=1

2. 减法指令 (Subtraction)

(1) SUB 减法指令 (Subtraction)

指令格式: SUB 目的, 源

指令功能: 将目的操作数减去源操作数, 结果送回目的操作数。即

目的 $\leftarrow$ 目的 - 源

例 3-48

SUB	AX, BX	;	AX $\leftarrow$ AX - BX
SUB	DX, 1850H	;	DX $\leftarrow$ DX - 1850H
SUB	BL, [BX]	;	BL 中内容减去物理地址 = DS:(BX) 处的字节, 结果存入 BL。

(2) SBB 带借位的减法指令 (Subtract with Borrow)

指令格式: SBB 目的, 源

指令功能: 与 SUB 类似, 只是在两个操作数相减后, 还要减去进位/借位标志 CF 的当前值。即

目的 $\leftarrow$ 目的 - 源 - CF

例 3-49

SBB	AL, CL	;	AL $\leftarrow$ AL - CL - CF
-----	--------	---	------------------------------

SBB 主要用于多字节减法中。

(3) DEC 减量指令 (Decrement)

指令格式: DEC 目的

指令功能: 对指定的目的操作数减 1, 结果送回此操作数。即

目的 $\leftarrow$ 目的 - 1

例 3-50

DEC	BX	;	BX $\leftarrow$ BX - 1
DEC	WORD PTR [BP]	;	堆栈段中位于 [BP] 偏置处的字减 1

(4) NEG 取负指令 (Negate)

指令格式: NEG 目的

指令功能: 对目的操作数取负, 即用零减去操作数, 再把结果送回目的操作数。即

目的 $\leftarrow$ 0 - 目的

例 3-51

NEG AX ;将 AX 中的数取负(正数变负数,负数变正数)
 NEG BYTE PTR[BX] ;对数据段中位于[BX]偏置处的字节取负

(5) CMP 比较指令 (Compare)

指令格式: CMP 目的, 源

指令功能: 将目的操作数减去源操作数,但结果不回送到目的操作数中,仅将结果反映在标志位上,接着可用条件跳转指令决定程序的去向。即

目的 - 源

例 3-52

CMP AL, 80H ;AL 与 80H 作比较
 CMP BX, DATA1 ;BX 与数据段中偏移量为 DATA1 处的字比较

比较指令主要用在希望比较两个数的大小,而又不破坏原操作数的场合。

以上五种指令实际上都做减法运算,而且都可以进行字或字节运算。对于 SUB、SBB 和 CMP 这类双操作数指令,源操作数可以是寄存器、存储器或立即数;目的操作数可以是寄存器或存储器,但不能为立即数,而且要求两个操作数不能同时为存储器。对于单操作数指令,目的操作数可以是寄存器或存储器,但不能为立即数,如果是存储器操作数,还必须说明其类型是字节还是字。运算之后,除 DEC 指令不影响 CF 标志外,它们均影响 OF、SF、ZF、AF、PF 和 CF 标志。在减法操作后,如果源操作数大于目的操作数,需要借位时,进位/借位标志 CF 将被置 1。

例 3-53 设 AL=1011 0001B, DL=0100 1010B,若要求 AL-DL,只要执行指令 SUB AL, DL,与加法操作那样,对结果的解释也取决于参与运算的数的性质。运算过程如下:

二进制减法	当成无符号数	当成带符号数
1011 0001	177	- 79
- 0100 1010	- 74	-) + 74
--- 0110 0111	103	+103

运算后标志位 ZF=0, AF=1, CF=0, SF=0, PF=0, OF=1。

运算结果非 0,故零标志 ZF=0;低 4 位向高 4 位有借位,所以半进位/借位标志 AF=1;D₇ 位没有产生进位,进位标志 CF=0;因 D₇=0,符号标志 SF=0;结果中有奇数个 1,使奇偶标志 PF=0;对于溢出标志 OF,若两个数的符号相反,而结果的符号与减数相同,则 OF=1。

若把两数当成无符号数,CF=0 表示没有借位,结果 103 是 177 与 74 的差。这时,标志位 SF=0 和 OF=1 并无意义。

要是把两数当成带符号数,表示的操作为一个负数(-79)减去一个正数(74),结果为正数(103),显然结果不正确。正确的结果应为: -79-(+74)=-153。由于一个字节能表示的带符号数的范围为-128~+127,-153 超过了这个范围,产生了溢出,所以溢出标志 OF 被置 1,表示运算结果错误。

(6) AAS 减法的 ASCII 调整指令 (ASCII Adjust for Subtraction)

指令格式: AAS

指令功能：在用 SUB 或 SBB 指令对两个非压缩十进制数或以 ASCII 码表示的十进制数进行相减后，对 AL 中所得结果进行调整，在 AL 中得到一个正确的非压缩十进制数之差。如果有借位，则 CF 置 1。AAS 指令必须紧跟在 SUB 或 SBB 指令之后。

· AAS 指令执行时，调整过程为：

若 AL 寄存器的低 4 位 > 9 或 $AF=1$ ，

则① $AL \leftarrow AL - 6$ ，AF 置 1

② 将 AL 寄存器高 4 位清零

③ $AH \leftarrow AH - 1$ ，CF 置 1

否则，不需要调整

例 3-54 设 $AL = \text{BCD } 3$ ， $CL = \text{BCD } 8$ ，求两数之差。显然，结果为 BCD 5，但要向高位借位。运算过程如下：

SUB	AL,CL	;	0000	0011	BCD 3
		;	-	0000	1000 BCD 8
		;			
AAS		;	1111	1011	低 4 位 > 9
		;	-	0000	0110 减 6 调整
		;			
		;	1111	0101	
		;	A	0000	1111 高 4 位清 0
		;			
		;	0000	0101	AL=5
		;	结果为 5，CF=1，表示有借位		

(7) DAS 减法的十进制调整指令 (Decimal Adjust for Subtraction)

指令格式：DAS

指令功能：在两个压缩十进制数用 SUB 或 SBB 相减后，结果已存在 AL 中的情况下，对所得结果进行调整，在 AL 中得到正确的压缩十进制数。同样，它也要对 AL 中高半字节和低半字节分别进行调整。

DAS 指令执行时，调整过程为：

如果 AL 寄存器的低 4 位 > 9 或 $AF=1$

则 $AL \leftarrow AL - 6$ ，AF 置 1

如果此时 AL 高半字节 > 9 或标志位 $CF=1$

则 $AL \leftarrow AL - 60H$ ，CF 置 1

例 3-55 设 $AL = \text{BCD } 56$ ， $CL = \text{BCD } 98$ ，求两数之差。运算过程如下：

SUB	AL,CL	;	0101	0110 ...	BCD 56
		;	-	1001	1000 ... BCD 98
		;			
		;	1011	1110 ...	低 4 位 > 9 ，CF=AF=1
DAS		;	-	0000	0110 ... 减 6 调整
		;			
		;	1011	1000 ...	高 4 位 > 9


```

;    — 0110 0000 ... 减 60H 调整
;
;    0101 1000 ... BCD 58
;    结果为 AL=BCD 58, CF=1, 表示有借位

```

3. 乘法指令 (Multiply)

(1) MUL 无符号数乘法指令 (Multiply)

指令格式: MUL 源

指令功能: 把源操作数和累加器中的数都当成无符号数, 然后将两数相乘, 源操作数可以是字节或字。

如果源操作数是一个字节, 它与累加器 AL 中的内容相乘, 乘积为双倍长的 16 位数, 高 8 位送到 AH, 低 8 位送 AL。即

$$AX \leftarrow AL * \text{源}$$

如果源操作数是一个字, 则它与累加器 AX 的内容相乘, 结果为 32 位数, 高位字放在 DX 寄存器中, 低位字放在 AX 寄存器中。即

$$(DX, AX) \leftarrow AX * \text{源}$$

乘法指令中, 源操作数可以是寄存器, 也可以是存储单元, 但不能是立即数。当源操作数是存储单元时, 必须在操作数前加 B 或 W 说明是字节还是字。

例 3-56

```

MUL    DL           ; AX ← AL * DL
MUL    CX           ; (DX, AX) ← AX * CX
MUL    BYTE[SI]     ; AX ← AL * (内存中某字节), B 说明字节乘法
MUL    WORD[BX]     ; (DX, AX) ← AX * (内存中某字), W 说明字乘法

```

MUL 指令执行后影响 CF 和 OF 标志, 如果结果的高半部分(字节操作为 AH、字操作为 DX)不为零, 表明其内容是结果的有效位, 则 CF 和 OF 均置 1。否则, CF 和 OF 均清 0。通过测试这两个标志, 可检测并去除结果中的无效前导零。乘法指令使 AF、PF、SF 和 ZF 的状态不定。

例 3-57 设 AL=55H, BL=14H, 计算它们的积。只要执行下面这条指令:

```
MUL    BL
```

结果, AX=06A4H。由于 AH=06H≠0, 高位部分有效, 所以置 CF=1, OF=1。

如果要做带符号数的乘法, 是否也能使用 MUL 指令呢? 不能! 让我们来看一个例子。

例 3-58 试计算 FFH×FFH。用二进制表示成如下形式:

$$\begin{array}{r}
 1111 \ 1111 \\
 \times 1111 \ 1111 \\
 \hline
 1111 \ 1110 \ 0000 \ 0001
 \end{array}$$

若把它们当成无符号数, 相当于进行 $255 \times 255 = 65025$ 的运算, 结果正确。若把它们看作带符号数, 上面的计算表示 $(-1) \times (-1) = -511$, 显然结果不正确。

由此可见, 如果用 MUL 指令作带符号数的乘法, 会得到错误的结果, 所以必须用下面

介绍的 IMUL 指令,才能使 $(-1) \times (-1)$ 得到正确的结果 0000 0000 0000 0001。

(2) IMUL 整数乘法指令 (Integer Multiply)

指令格式: IMUL 源

指令功能: 把源操作数和累加器中的数都作为带符号数,进行相乘。

存放结果的方式与 MUL 相同。如果源操作数为字节,则与 AL 相乘,双倍长结果送到 AX 中。如源操作数为字,则与 AX 相乘,双倍长结果送到 DX 和 AX 中,最后给乘积赋予正确的符号。

执行 IMUL 指令后,如果乘积的高半部分不是低半部分的符号扩展(不是全零或全 1),则视高位部分为有效位,表示它是积的一部分,于是置 $CF=1, OF=1$ 。若结果的高半部分为全零或全 1,表明它仅包含了符号位,那么使 $CF=0, OF=0$ 。利用这两个标志状态可决定是否需要保存积的高位字节或高位字。IMUL 指令执行后,AF、PF、SF 和 ZF 不定。

例 3-59 设 $AL=-28H, BL=59H$, 试计算它们的乘积。这时,可使用下面指令:

```
IMUL BL
```

结果, $AX=F98CH=-1652, CF=1, OF=1$

至于 IMUL 指令采用什么算法来实现上述功能可以有几种方案,如可以直接采用补码乘法运算的算法,也可将参加运算的操作数恢复成原码,数位当成无符号数相乘,然后给乘积赋予正确的符号,这些工作由计算机自动完成。

(3) AAM 乘法的 ASCII 调整指令 (ASCII Adjust for Multiply)

指令格式: AAM

指令功能: 对已存在 AL 中的两个非压缩十进制数相乘的乘积进行十进制数的调整,使得在 AX 中得到正确的非压缩十进制数的乘积,高位放在 AH 中,低位在 AL 中。两个 ASCII 码数相乘之前,必须先屏蔽掉每个数字的高半字节,从而使每个字节包含一个非压缩十进制数 (BCD 数),再用 MUL 指令相乘,乘积放到 AL 寄存器中,然后用 AAM 指令进行调整。

调整过程为: 把 AL 寄存器内容除以 10,商放在 AH 中,余数在 AL 中。即

$AH \leftarrow AL/10$ 所得的商

$AL \leftarrow AL/10$ 所得的余数

指令执行后,将影响 ZF、SF 和 PF,但 AF、CF 和 OF 无定义。

例 3-60 求两个非压缩十进制数 09 和 06 之乘积,可用如下指令实现:

```
MOV    AL, 09H          ;置初值
MOV    BL, 06H
MUL    BL               ;AL←09 与 06 之乘积 36H
AAM                    ;调整得 AH=05H(十位),AL=04H(个位)
```

最后可在 AX 中得到正确结果 $AX=0504H$,即 BCD 数 54。

如果 AL 和 BL 中分别存放 9 和 6 的 ASCII 码,求两数之积时要用以下指令实现:

```
AND    AL, 0FH          ;屏蔽高半字节
AND    BL, 0FH
MUL    BL               ;相乘
AAM                    ;调整
```

如要将结果转换成 ASCII 码,可再用指令 OR AX,3030H 实现,使 AX=3534H。

由于 8086/8088 指令系统中,十进制乘法运算不允许采用压缩十进制数,所以乘法的调整指令仅此一条。

4. 除法指令 (Division)

(1) DIV 无符号数除法指令 (Division, unsigned)

指令格式: DIV 源

指令功能: 对两个无符号二进制数进行除法操作。源操作数可以是字或字节。

如果源操作数为字节,16 位被除数必须放在 AX 中,8 位除数为源操作数,它可以是寄存器或存储单元。相除之后,8 位商在 AL 中,余数在 AH 中。即

$AL \leftarrow AX / \text{源(字节)的商}$

$AH \leftarrow AX / \text{源(字节)的余数}$

要是被除数只有 8 位,必须把它放在 AL 中,并将 AH 清 0,然后相除。

如果源操作数为字,32 位被除数在 DX,AX 中,其中,DX 为高位字,16 位除数作源操作数,它可以是寄存器或存储单元。相除之后,AX 中存 16 位商,DX 存 16 位余数。即

$AX \leftarrow (DX, AX) / \text{源(字)的商}$

$DX \leftarrow (DX, AX) / \text{源(字)的余数}$

要是被除数只有 16 位,除数也是 16 位,则必须将 16 位被除数送到 AX 中,再将 DX 寄存器清 0,然后相除。

与被除数和除数一样,商和余数也都为无符号数。DIV 指令执行后,所有标志均无定义。

(2) IDIV 整数除法指令 (Integer Division)

指令格式: IDIV 源

指令功能: 该指令执行的操作与 DIV 相同,但操作数都必须是带符号数,商和余数也都是带符号数,而且规定余数的符号和被除数的相同,因此 IDIV 指令也称为带符号数除法指令。指令执行后,所有标志位均无定义。

进行除法操作时,无论对无符号数相除(DIV)还是带符号数相除(IDIV),都要注意一个问题: 由于除法指令字节操作时商为 8 位,字操作时商为 16 位,如果字节操作时,被除数的高 8 位绝对值大于除数的绝对值,或在字操作时,被除数的高 16 位绝对值大于除数的绝对值,就会产生溢出,也就是说商数超过了目标寄存器 AL 或 AX 所能存放数的范围。这时计算机自动产生一个中断类型为 0 的除法错中断,相当于执行了除数为 0 的运算,所得的商和余数都不确定。

对于无符号数,字节操作时,允许最大商为 FFH,字操作时最大商为 FFFFH,若超过这个范围就会溢出。对于带符号数,字节操作时商的范围为 $-127 \sim +127$,或 $-81H \sim +7FH$; 字操作时商的范围为 $-32767 \sim +32767$,或 $-8001H \sim 7FFFH$ 。

例 3-61 两个无符号数 7A86H 和 04H 相除的商应为 1EA1H,若用 DIV 指令进行计算,即

```
MOV    AX, 7A86H
MOV    BL, 04H
DIV    BL
```

这时,由于 BL 中的除数 04H 为字节,被除数为字,商 1EA1H 大于 AL 中能存放的最大无符号数 FFH,结果将产生除法错误中断。

对于带符号数除法指令,字节操作时要求被除数为 16 位,字操作时要求被除数为 32 位。如果被除数不满足这个条件,不能简单地将高位置 0,而应该先用下面的符号扩展指令 (Sign Extension) 将被除数转换成除法指令所要求的格式,再执行除法指令。

(3) CBW 把字节转换为字指令 (Convert Byte to Word)

指令格式: CBW

指令功能: 把寄存器 AL 中字节的符号位扩充到 AH 的所有位,这时 AH 被称为是 AL 的符号扩充。

若 AL 中的 $D_7=0$,就将这个 0 扩展到 AH 中去,使 $AH=00H$,即

	D_7	D_0		D_7	D_0	
AH	0 0		AL	0		AL=正数

若 AL 中的 $D_7=1$,则将这个 1 扩展到 AH 中去,使 $AH=FFH$,即

	D_7	D_0		D_7	D_0	
AH	1 1		AL	1		AL=负数

CBW 指令执行后,不影响标志位。

(4) CWD 把字转换成双字指令 (Convert Word to Double Word)

指令格式: CWD

指令功能: 把 AX 中字的符号位扩充到 DX 寄存器的所有位中去。

若 AX 的 $D_{15}=0$,则 $DX \leftarrow 0000H$,即

	D_{15}	D_0		D_{15}	D_0	
DX	0 0		AX	0		AX=正数

若 AX 的 $D_{15}=1$,则 $DX \leftarrow FFFFH$,即

	D_{15}	D_0		D_{15}	D_0	
DX	1 1		AX	1		AX=负数

CWD 指令执行后,也不影响标志位。

例 3-62 编程求 $-38/3$ 的商和余数

```

MOV    AL, 11011010B    ;被除数-38
MOV    CH, 00000011B    ;除数+3
CBW                                ;将 AL 符号扩展到 AH 中
                                ;使 AX=11111111 11011010B
IDIV   CH                ;AX/CH
                                ;AL=11110100B=-12(商),AH=11111110B=-2(余数)

```

(5) AAD 除法的 ASCII 调整指令 (ASCII Adjust for Division)

指令格式: AAD

指令功能: 在做除法前,把 BCD 码转换成二进制数。

前面介绍的 ASCII 调整指令都是在用加法、减法和乘法指令对两个非压缩的 BCD 码运算之后,紧跟着用一条 AAA、AAS 或 AAM 指令,对运算结果进行调整。而除法的 ASCII 调整指令则不同,它是在除法之前进行的。

在把 AX 中的两位非压缩格式的 BCD 数除以一个非压缩的 BCD 数前,要先用 AAD 指令把 AX 中的被除数调整成二进制数,并存到 AL 中,才能用 DIV 指令进行运算。调整的过程为:

$AL \leftarrow AH \times 10 + AL$

$AH \leftarrow 00$

本指令根据 AL 寄存器的结果影响 SF、ZF 和 PF,对 OF、CF 和 AF 无定义。

例 3-63 设 AX 中存有两个非压缩 BCD 数 0307H,即十进制数的 37,BL 中存有一个非压缩 BCD 数 05H,若要完成 AX/BL 的运算,可用以下指令:

AAD

DIV BL

第一条指令先将 AX 中的两个 BCD 数转换成二进制数, $03 \times 10 + 7 = 37 = 25H$,并将 $25H \rightarrow AL$,显然经调整后的被除数 25H 才真正代表 37,再用 DIV 指令做除法,可得到正确的结果:

$AL = 7$ (商)

$AH = 2$ (余数)

由于 8086 只提供了非压缩十进制数的乘、除法调整指令,如果要进行压缩十进制数的乘、除法运算,应先将操作数转换成非压缩十进制数,再按非压缩十进制数进行运算,分几步完成操作。

例 3-64 编写程序,计算 $75 \div 6 = 12 \dots 3$

该除法运算过程表示如下:

	$\begin{array}{r} \text{第一个商为 1} \\ \downarrow \\ 1 \ 2 \leftarrow \text{第二个商为 2} \\ \hline 6 \overline{) 7 \ 5} \\ \underline{- \ 6} \\ 1 \ 5 \\ \underline{- \ 1 \ 2} \\ 3 \leftarrow \text{第二个余数为 3} \end{array}$
第一个余数为 1 $\rightarrow$	

程序如下:

FIRST	DB 06H		;除数 6
SECOND	DB 75H		;被除数 75(BCD 数)
THIRD	DB 2 DUP(0)		;存商
FOUR	DB ?		;存余数
	:		
	MOV AH, 00H		;第一个被除数高位 AH 清 0
	MOV AL, SECOND		;AL←被除数 75
	AND AL, 0F0H		;截取高 4 位
	MOV CL, 04H		

ROL	AL, CL	;移至低 4 位
DIV	FIRST	;AX/06,即 0007/06
		;得结果:AL←商为 1,AH←余数 1
MOV	THIRD+1, AL	;结果单元←第一个商 1
MOV	AL, SECOND	;AL←被除数 75
AND	AL, 0FH	;AL←截低 4 位,故 AX=0105H
AAD		;将 AX 中内容 0105H 调整为 0FH
DIV	FIRST	;0FH/6,结果:AL←商为 2,AH←余数为 3
MOV	THIRD, AL	;THIRD 单元←第二个商 2
MOV	FOUR, AH	;FOUR 单元←第二个余数 3

图 3-16 表示上述除法程序执行过程中,数据在内存中的存放格式。

FIRST	0 6	除数
SECOND	7 5	被除数
THIRD	0 2	商
THIRD+1	0 1	
FOUR	0 3	余数

图 3-16 例 3-64 程序的数据存放格式

三、逻辑运算和移位指令

逻辑运算和移位指令对字节或字操作数进行按位操作,这类运算可分成逻辑运算、算术逻辑移位和循环移位三类,见表 3-7。

表 3-7 逻辑运算和移位指令

逻辑运算	
NOT	取反
AND	逻辑乘(与)
OR	逻辑加(或)
XOR	异或
TEST	测试
算术逻辑移位	
SHL/SAL	逻辑/算术左移
SHR	逻辑右移
SAR	算术右移
循环移位	
ROL	循环左移
ROR	循环右移
RCL	通过进位的循环左移
RCR	通过进位的循环右移

1. 逻辑运算指令 (Logical Operations)

(1) NOT 取反指令 (Logical Not)

指令格式: NOT 目的

指令功能: 将目的操作数求反, 结果送回目的操作数, 即

目的 $\leftarrow$ $\overline{\text{目的}}$

目的操作数可以是 8 位或 16 位寄存器或存储器。对于存储器操作数, 要说明其类型是字节还是字。指令执行后, 对标志位无影响。

例 3-65 NOT 指令的几种用法:

NOT	AX	; AX $\leftarrow$ AX 取反
NOT	BL	; BL $\leftarrow$ BL 取反
NOT	BYTE PTR[BX]	; 对存储单元内容取反后送回该单元

NOT 指令只有一个操作数。以下几种逻辑运算指令均为双操作数指令, 对操作数的规定与算术运算指令一样, 即源操作数可以是 8 位或 16 位立即数、寄存器或存储器, 目的操作数只能是寄存器或存储器, 两个操作数不能同时为存储器。指令执行后, 均将 CF 和 OF 清零, ZF、SF 和 PF 反映操作结果, AF 未定义, 源操作数不变。

(2) AND 逻辑与指令 (Logical AND)

指令格式: AND 目的, 源

指令功能: 对两个操作数进行按位逻辑与操作, 结果送回目的操作数, 即

目的 $\leftarrow$ 目的 $\wedge$ 源。

它主要用于使操作数的某些位保留(和“1”相与), 而使某些位清除(和“0”相与)。

例 3-66 假设 AX 中存有数字 5 和 8 的 ASCII 码, 即 AX=3538H, 要将它们转换成 BCD 码, 并把结果仍放回 AX, 可用如下指令实现:

AND	AX, 0F0FH	; AX $\leftarrow$ 0508H
-----	-----------	-------------------------

它将 AH 和 AL 中的高 4 位用全 0 屏蔽掉, 截取低 4 位, 最后在 AX 中得到 5 和 8 的 BCD 码 0508H。

(3) OR 逻辑或指令 (Logical OR)

指令格式: OR 目的, 源

指令功能: 对两个操作数进行按位逻辑或操作, 结果送回目的操作数, 即

目的 $\leftarrow$ 目的 $\vee$ 源。

它主要用于使操作数的某些位保留(和“0”相或), 而使某些位置 1(和“1”相或)。

例 3-67 假设 AX 中存有两个 BCD 数 0508H, 要将它们分别转换成 ASCII 码, 结果仍在 AX 中, 则可用如下指令实现:

OR	AX, 3030H	; AX $\leftarrow$ 3538H
----	-----------	-------------------------

(4) XOR 异或操作指令 (Exclusive OR)

指令格式: XOR 目的, 源

指令功能: 对两个操作数进行按位逻辑异或运算, 结果送回目的操作数, 即

目的 $\leftarrow$ 目的 $\vee$ 源。

它主要用于使操作数的某些位保留(和“0”相异或),而使某些位取反(和“1”相异或)。

例 3-68 若 AL 中存有某外设端口的状态信息,其中 D_1 位控制扬声器发声,要求该位在 0、1 之间来回变化,原来是 1 变成 0,原来是 0 变成 1,其余各位保留不变,可以用以下指令实现:

```
XOR    AL, 00000010B
```

(5) TEST 测试指令 (Test)

指令格式: TEST 目的, 源

指令功能: 对两个操作数进行逻辑与操作,并修改标志位,但不回送结果,即指令执行后,两个操作数都不变,即

目的 $\wedge$ 源。

它常用在要检测某些条件是否满足,但又不希望改变原有操作数的情况下。紧跟在这条指令后面的往往是一条条件转移指令,根据测试结果产生分支,转向不同的处理程序。

例 3-69 设 AL 寄存器中存有报警标志。若 $D_7=1$,表示温度报警,程序要转到温度报警处理程序 T-ALARM; $D_8=1$,则转压力报警程序 P-ALARM。为此,可按下而方法使用 TEST 指令来实现这种功能:

```
TEST    AL, 80H           ;查 AL 的  $D_7=1$ ?
JNZ     T-ALARM           ;是 1(非零),则转温度报警程序
TEST    AL, 40H           ; $D_7=0, D_8=1$ ?
JNZ     P-ALARM           ;是 1,转压力报警
```

其中 JNZ 为条件转移指令,表示结果非 0 ($ZF=1$) 则转移。

2. 算术逻辑移位指令 (Shift Arithmetic and Shift Logical)

可对寄存器或存储器中的字或字节的各位进行算术移位或逻辑移位,移动的次数由指令中的计数值决定。图 3-17 是移位指令的操作示意图。

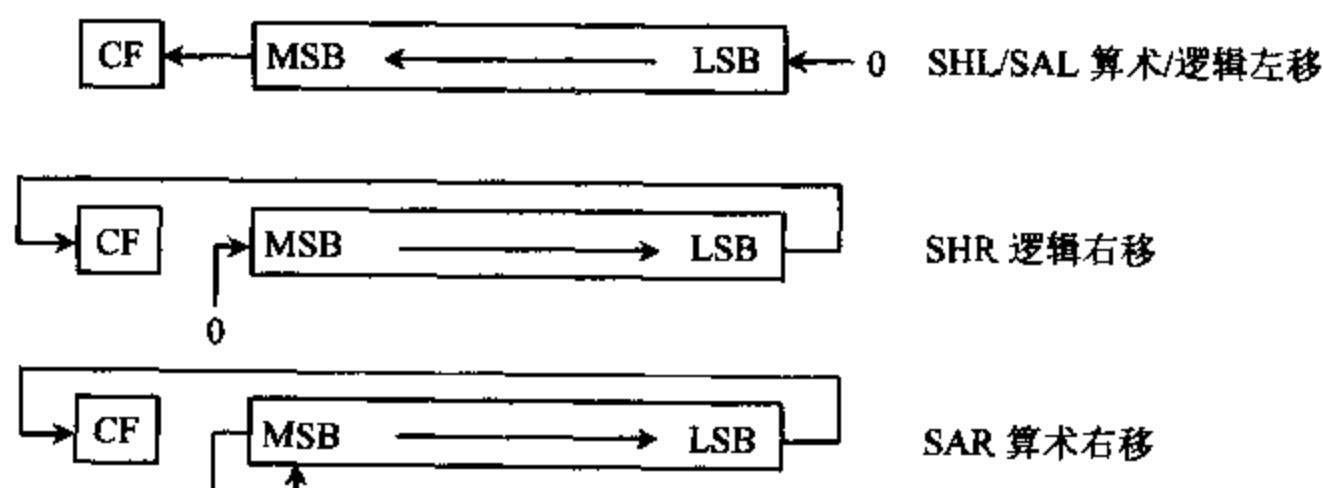


图 3-17 算术逻辑移位指令操作示意图

(1) SAL 算术左移指令 (Shift Arithmetic Left)

指令格式: SAL 目的, 计数值

(2) SHL 逻辑左移指令 (Shift Logic Left)

指令格式: SHL 目的, 计数值

指令功能: 以上两条指令的功能完全相同, 均将寄存器或存储器中的目的操作数的各

位左移,每移一次,最低有效位 LSB 补 0,而最高有效位 MSB 进入标志位 CF。移动一次,相当于将目的操作数乘以 2。指令中的计数值决定所要移位的次数。若只需要移位一次,可直接将指令中的计数值置 1。要是移位次数大于 1,应先将移位次数送进 CL 寄存器,再把 CL 放在指令的计数值位置上。

在移位次数为 1 的情况下,如果移位后的最高位(符号位)的值被改变,则 OF 标志置 1,否则 OF 清 0。但在多次移位的情况下,OF 的值不确定。不论移一次或多次,CF 总是等于目的操作数最后被移出去的那一位的值。SF 和 ZF 将根据指令执行后目的操作数的状态来决定,PF 只有当目的操作数在 AL 中时才有效,AF 不定。

例 3-70

```
MOV    AH,    06H           ;AH=06H
SAL     AH,    1           ;将 AH 内容左移一位后,AH=0CH
MOV     CL,    03H
SHL     DI,    CL           ;将 DI 内容左移 3 次
SAL     BYTE PTR[BX], 1    ;将内存单元的字节左移 1 位
```

(3) SHR 逻辑右移指令 (Shift Logic Right)

指令格式: SHR 目的, 计数值

指令功能: 对目的操作数中各位进行右移,每执行一次移位操作,操作数右移一位,最低位进入 CF,最高位补 0。右移次数由计数值决定,同 SAL/SHL 指令一样。

若目的操作数为无符号数,每右移一次,使目的操作数除以 2。例如,右移 2 次相当于除以 4,右移 3 次相当于除以 8 等等。但是,用这种方法做除法时,余数将被丢掉。

例 3-71 用右移的方法作除法 $133/8=16\cdots5$,即

```
MOV     AL, 10000101B       ;AL=133
MOV     CL, 03H             ;CL=移位次数
SHR     AL, CL              ;右移 3 次
```

指令执行后,AL=10H=16,余数 5 被丢失。标志位 CF=1,ZF=0,SF=0,PF=0,OF 和 AF 不定。

(4) SAR 算术右移指令 (Shift Arithmetic Right)

指令格式: SAR 目的, 计数值

指令功能: 它的功能与 SHR 很相似,移位次数也由计数值决定。每移位一次,目的操作数各位右移一位,最低位进入 CF,但最高位(即符号位)保持不变,而不是补 0。每移一次,相当于对带符号数进行除 2 操作。

例 3-72 用 SAR 指令计算 $-128/8=-16$ 的程序段如下:

```
MOV     AL, 10000000B       ;AL=-128
MOV     CL, 03H             ;右移次数为 3
SAR     AL, CL              ;算术右移 3 次后,AL=0F0H=-16
```

这里要说明一下,由于移位次数即计数值可以是 1,也可以放在 CL 中,这样,对 8086 的移位指令,计数值的范围可以是 1~255。但事实上,当计数值很大时已无实际意义,反而明显降低了指令的执行速度。所以 286、386 等高档机对此进行了改进,规定放在 CL 中

的移位次数最多只能是 31(即 00011111B),若超过此值,只截取最低的 5 位数值。

3. 循环移位指令 (Rotate)

上述的算术逻辑移位指令,移出操作数的数位均被丢失,而循环移位指令把操作数从一端移到操作数的另一端,这样从操作数中移走的位就不会丢失了。

循环移位指令共四条:

(1) ROL 循环左移指令 (Rotate Left)

指令格式: ROL 目的, 计数值

(2) ROR 循环右移指令 (Rotate Right)

指令格式: ROR 目的, 计数值

(3) RCL 通过进位位循环左移 (Rotate through Carry Left)

指令格式: RCL 目的, 计数值

(4) RCR 通过进位位循环右移 (Rotate through Carry Right)

指令格式: RCR 目的, 计数值

循环移位指令的操作示意图如图 3-18 所示。

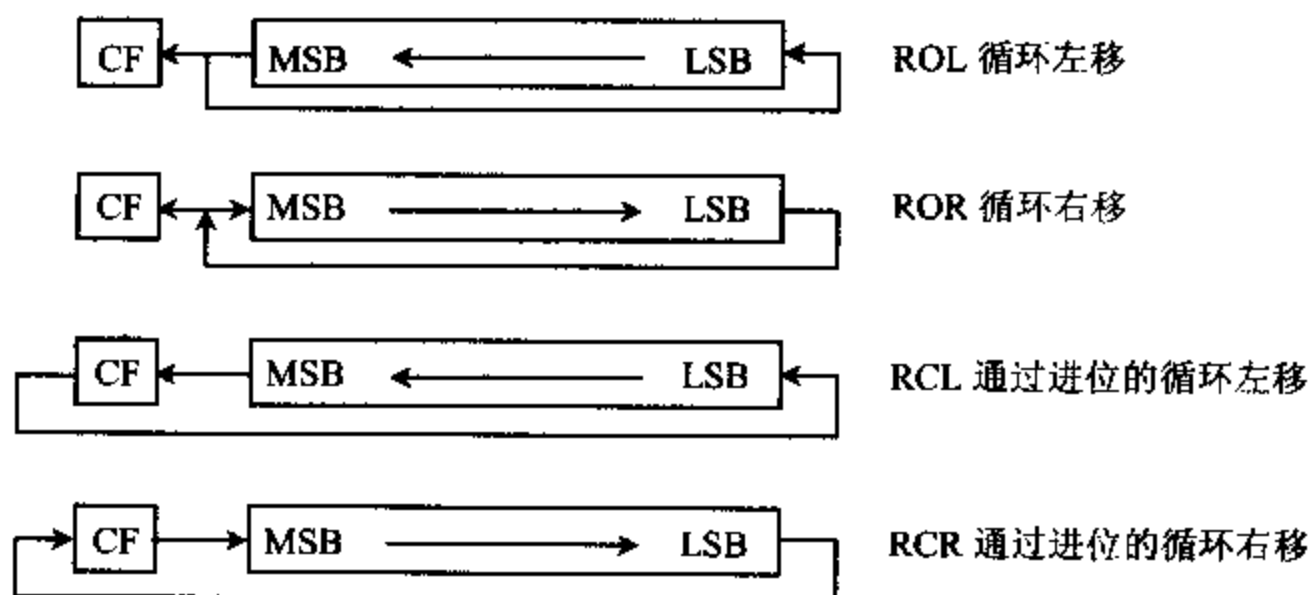


图 3-18 循环移位操作示意图

4 条指令都按指令中计数值规定的移位次数进行循环移位,移位后的结果仍送回目的操作数。与算术逻辑移位指令一样,目的操作数可以是 8/16 位的寄存器操作数或内存操作数,循环移位的次数可以是 1,也可以由 CL 寄存器的值指定。

这 4 条指令中,ROL 和 ROR 指令没有把进位标志 CF 包含在循环中,而 RCL 和 RCR 指令把 CF 作为整个循环的一部分,一起参加循环移位。OF 位只有在移位次数为 1 的时候才有效,在移位后当前最高有效位(符号位)发生变化(由 1 变 0 或由 0 变 1)时,则 OF 标志置 1,否则 OF 置 0。在多位循环移位时,OF 的值是不确定的。CF 的值总是由最后一次被移出的值决定。

例 3-73

ROL BX, CL ;将 BX 中的内容不带进位位左移 CL 中规定的次数
ROR WORD PTR[SI], 1 ;将物理地址为 DS×16+SI 单元的字不带进位右移 1 次

例 3-74

设 CF=1, AL=1011 0100B

若执行指令 ROL AL, 1, 则 AL=0110 1001B, CF=1, OF=1
 若执行指令 ROR AL, 1, 则 AL=0101 1010B, CF=0, OF=1
 若执行指令 RCR AL, 1, 则 AL=1101 1010B, CF=0, OF=0
 若执行指令 MOV CL, 3 和 RCL AL, CL, 则 AL=1010 0110B, CF=1, OF 不确定

四、字符串处理指令

这里所谓的字符串是指一系列存放在存储器中的字或字节数据，不管它们是不是 ASCII 码。字符串长度可达 64K 字节，组成字符串的字节或字称为字符串元素，每种字符串指令对字符串中的元素只进行同一种操作。

8086 提供 5 条 1 字节的字符串操作指令，专门对存储器中的字节串和字串数据进行传送、比较、扫描、存储及装入等 5 种操作。

使用字符串操作指令时，可以有两种方法告诉汇编程序是进行字节操作还是字操作。一种方法是用指令中的源串和目的串名（即操作数）来表明是字节还是字，另一种方法是在指令助记符后加 B 说明是字节，加 W 说明是字操作。这样每种指令就都有 3 种格式。各种字符串操作指令的类型和格式如表 3-8 所示。

表 3-8 字符串操作指令的类型和格式

指令名称	字节/字操作		字节操作	字操作
字符串传送	MOVS	目的串, 源串	MOVSB	MOVSW
字符串比较	CMPS	目的串, 源串	CMPSB	CMPSW
字符串扫描	SCAS	目的串	SCASB	SCASW
字符串装入	LODS	源串	LODSB	LODSW
字符串存储	STOS	目的串	STOSB	STOSW

字符串指令执行时，必须遵守以下的隐含约定：

(1) 源串位于当前数据段中，由 DS 寻址，源串的元素由 SI 作指针，即源串字符的起始地址（或末地址）为 DS: SI。源串允许使用段超越前缀来修改段地址。

(2) 目的串必须位于当前附加段中，由 ES 寻址，目的串元素由 DI 作指针，即目的串字符的起始地址（或末地址）为 ES: DI，但目的串不允许使用段超越前缀修改 ES。如果要在同一段内进行串运算，必须使 DS 和 ES 指向同一段。

(3) 每执行一次字符串指令，指针 SI 和 DI 会自动进行修改，以便指向下一个待操作的单元。

(4) DF 标志控制字符串处理的方向。DF=0 为递增方向，这时，DS: SI 指向源串首地址，每执行一次串操作，使 SI 和 DI 增加，字节串操作时，SI、DI 分别增 1，字串操作时，SI 和 DI 分别增 2；DF=1 为递减方向，这时，DS: SI 指向源串末地址，每执行一次串操作，使 SI 和 DI 分别减量，字节串操作时减 1，字串操作时减 2。可用标志操作指令 STD 和 CLD 来改变 DF 的值，STD 使 DF 置 1，CLD 将 DF 清 0。

(5) 要处理的字符串长度（字节或字数）放在 CX 寄存器中。

为了加快串运算指令的执行速度,可在基本指令前加重复前缀,使数据串指令重复执行。每重复执行一次,地址指针 SI 和 DI 都根据方向标志自动进行修改,CX 的值则自动减 1。能与基本指令配合使用的重复前缀有:

REP	无条件重复	(Repeat)
REPE/REPZ	相等/结果为零则重复	(Repeat while Equal/Zero)
REPNE/REPNZ	不相等/结果非零则重复	(Repeat while Not Equal/Not Zero)

无条件重复前缀指令 REP 常与串传送指令(MOVS)连用,连续进行字符串传送操作,直到整个字符串传完毕,CX=0 为止。重复前缀 REPE 和 REPZ 具有相同的含义,它们常与串比较指令(CMPS)连用,连续进行字符串比较操作。当两个字符串相等(ZF=1)和 CX≠0 时,则重复进行比较,直到 ZF=0 或 CX=0 为止。重复前缀 REPNE 和 REPNZ 也具有相同的意义,它们常与串扫描指令(SCAS)连用,当结果非 0(ZF=0)和 CX≠0 时,重复进行扫描,直到 ZF=1 或 CX=0 为止。

带有重复前缀的串运算执行时间可能很长,在指令执行过程中允许有中断进入,因此在处理每个元素前都在查询是否有中断请求,一旦外部有中断进入,CPU 将暂停执行当前的串操作指令,转去执行相应的中断服务程序,使中断服务完成后,再返回去继续执行被中断的串操作指令。

下面分别介绍这 5 条字符串操作指令。

1. MOVS 字符串传送指令 (Move String)

指令格式: MOVS 目的串, 源串

指令功能: 把由 SI 作指针的源串中的一个字节或字,传送到由 DI 作指针的目的串中,且自动修改指针 SI 和 DI。

实际应用中,经常需要在存储单元之间传送数据。然而,MOV 指令不能直接在存储单元间进行数据传送,为了实现这种操作,必须以某一通用寄存器为桥梁,先把一个存储单元中的数据送到指定的通用寄存器中,再将寄存器中的数据传送到另一个存储单元中。每进行一次传送操作,还必须修改地址指针。如果改用 MOVS 指令,便能很方便地实现这种功能,它不但能将数据从内存的某一地址(源地址)传送到另一个地址(目的地址),还能自动修改源和目的地址。若使用重复前缀,还可以利用一条指令传送一批数据。

例 3-75 要求把数据段中以 SRC_MESS 为偏移地址的一串字符“HELLO!”,传送到附加段中以 NEW_LOC 开始的单元中。实现该操作的程序如下:

```
DATA        SEGMENT                ;数据段
SRC_MESS DB 'HELLO!'                ;源串
DATA        ENDS
;
EXTRA       SEGMENT                ;附加段
NEW_LOC DB 6 DUP(?)                ;存放目的串
EXTRA       ENDS
;
CODE        SEGMENT                ;代码段
ASSUME CS:CODE, DS:DATA, ES:EXTRA
```

```

START:  MOV     AX, DATA
        MOV     DS, AX           ;DS=数据段段址
        MOV     AX, EXTRA
        MOV     ES, AX           ;ES=附加段段址
        LEA     SI, SRC- MESS     ;SI 指向源串偏移地址
        LEA     DI, NEW- LOC      ;DI 指向目的串偏移地址
        MOV     CX, 6             ;CX 作串长度计数器
        CLD                       ;清方向标志,地址增量
        REP     MOVSB             ;重复传送串中的各字节,直到 CX=0 为止
CODE    ENDS
        END     START

```

上例中的 REP MOVSB 指令也可用以下几条指令代替:

```

AGAIN:  MOVS     NEW- LOC, SRC- MESS
        DEC     CX
        JNZ     AGAIN

```

比较这两种方法,显然可以发现,使用有重复前缀 REP 的 MOVSB 指令,程序更简洁。

2. CMPS 字符串比较指令 (Compare String)

指令格式: CMPS 目的串, 源串

指令功能: 从 SI 作指针的源串中减去由 DI 作指针的目的串数据,相减后的结果反映在标志位上,但不改变两个数据串的原始值。同时,操作后源串和目的串指针会自动修改,指向下一对待比较的串。

常用这条指令来比较两个串是否相同,并由加在 CMPS 指令后的一条条件转移指令,根据 CMPS 执行后的标志位值决定程序的转向。

在 CMPS 指令前可以加重复前缀,即

```

REPE CMPS
或 REPZ CMPS

```

这两条指令功能相同,若比较结果为 $CX \neq 0$ (指定的长度还未比较完) 和 $ZF = 1$ (两串相等),则重复比较,直至 $CX = 0$ (比完了) 或 $ZF = 0$ (两串不相等) 时才停止操作。

也可以改用重复前缀 REPNE 或 REPNZ,它们表示:若 $CX \neq 0$ (串没有结束) 和串不等 ($ZF = 0$) 则重复比较,直至 $CX = 0$ 或 $ZF = 1$ 时才停止比较。

例 3-76 比较两个字符串,一个是你在程序中设定的口令串 PASSWORD,另一个是从键盘输入的字符串 IN- WORD,若输入串与口令串相同,程序将开始执行。否则,程序驱动 PC 机的扬声器发声,警告用户口令不符,拒绝往下执行。这可以用 CMPS 指令来实现,有关程序段如下:

```

DATA    SEGMENT                               ;数据段
PASSWORD DB  '750430LI'                       ;口令串
IN- WORD DB  '750424LE'                       ;从键盘输入的串
COUNT  EQU  8                                ;串长度
DATA    ENDS

```

```

...
CODE      SEGMENT                ;代码段
          ASSUME DS:DATA,ES:DATA
...
          LEA SI, PASSWORD        ;源串指针
          LEA DI, IN-WORD         ;目的串指针
          MOV CX, COUNT           ;串长度
          CLD                     ;地址增量
          REPZ CMPSB               ;CX≠0 且串相等时重复比较
          JNE SOUND               ;若不相等,转发声程序
OK:        ...                    ;比完且相等,往下执行
          ...
SOUND:     ...                    ;使 PC 机扬声器发声
          ...                    ;并退出
CODE      ENDS

```

3. SCAS 字符串扫描指令 (Scan String)

指令格式: SCAS 目的串

指令功能: 从 AL(字节操作)或 AX(字操作)寄存器的内容减去附加段中以 DI 为指针的目的串元素,结果反映在标志位上,但不改变源操作数。同时,操作后目的串指针会自动修改,指向下一个待搜索的串元素。

利用 SCAS 指令,可在内存中搜索所需要的数据。这个被搜索的数据也称为关键字。指令执行前,必须事先将它存在 AL(字节)或 AX(字)中,才能用 SCAS 指令进行搜索。SCAS 指令前也可以加重复前缀。

例 3-77 在某一字符串中搜寻是否有字符 A,若有,则把搜索次数记下来,送到 BX 寄存器中,若没有查到,则将 BX 寄存器清 0。设字符串起始地址 STRING 的偏移地址为 0,字符串长度为 CX。程序段如下:

```

          MOV     DI, OFFSET STRING    ;DI=字符串偏移地址
          MOV     CX, COUNT            ;CX=字符串长度
          MOV     AL, 'A'              ;AL=关键字 A 的 ASCII 码
          CLD                          ;清方向标志
          REPNE   SCASB                ;CX≠0(没查完)和 ZF=0(不相等)时重复
          JZ      FIND                ;若 ZF=1,表示已搜到,转出
          MOV     DI, 0                ;若 ZF=0,表示没搜到,DI←0
FIND:     MOV     BX, DI               ;BX←搜索次数
          HLT                          ;停机

```

上述程序中,DI 初值存起始地址偏移量 0,搜索一次后,DI 自动加 1,使 DI 的值等于 1,以后,每执行一次搜索操作,DI 自动增 1。所以,正好可用 DI 的值来表示搜索次数。

4. LODS 数据串装入指令 (Load String)

指令格式: LODS 源串

指令功能: 把数据段中以 SI 作为指针的串元素,传送到 AL(字节操作)或 AX(字操作)

中,同时修改 SI,使它指向串中的下一个元素,SI 的修改量由方向标志 DF 和源串的类型确定,即遵守上述隐含约定(4)。

为该指令加重复前缀没有意义,这是因为每重复传送一次数据,累加器中的内容就被改写,执行重复传送操作后,只能保留最后写入的那个数据。

5. STOS 数据串存储指令 (Store String)

指令格式: STOS 目的串

指令功能: 将累加器 AL 或 AX 中的一个字节或字,传送到附加段中以 DI 为目标指针的目的串中,同时修改 DI,以指向串中的下一个单元。

STOS 指令与 REP 重复前缀连用,即执行指令 REP STOS,能方便地用累加器中的一个常数,对一个数据串进行初始化,例如初始化为全 0 的串。

例 3-78 若在数据段中有一个数据块,起始地址为 BLOCK,数据块中的数为 8 位带符号数,要求将其中所含的正、负数分开,然后把正数送到附加段中始址为 PLUS_DATA 的缓冲区,负数则送到附加段中始址为 MINUS_DATA 的缓冲区。

我们可以将这块数据看成一个数据串,用 SI 作源串指针,DI 和 BX 分别作正、负数目的缓冲区的指针,CX 用于控制循环次数,于是可写出如下程序段:

```
START:  MOV     SI,    OFFSET BLOCK           ;SI 为源串指针
        MOV     DI,    OFFSET PLUS_DATA      ;DI 为正数目的区指针
        MOV     BX,    OFFSET MINUS_DATA     ;BX 为负数目的区指针
        MOV     CX,    COUNT                 ;CX 放循环次数
        CLD
GOON:   LODS     BLOCK                       ;AL←取源串的一个字节
        TEST    AL,    80H                   ;是负数?
        JNZ     MINUS                         ;是,转 MINUS
        STOSB                                     ;非负数,将字节送正数区
        JMP     AGAIN                         ;处理下一个字节
MINUS:  XCHG     BX,    DI                     ;交换正负数指针
        STOSB                                     ;负数送入负数区
        XCHG     BX,    DI                     ;恢复正负数指针
AGAIN:  DEC      CX                           ;次数减 1
        JNZ     GOON                         ;未处理完,继续传送
        HLT                                     ;停机
```

程序中,正负数的存储均使用 STOSB 指令,该指令必须以 SI 为源指针,DI 为目的指针,但存储负数时,负数区的目的指针在 BX 中,因此要用 XCHG 指令将 BX 内容送进 DI,让 DI 指向负数区,同时也把 DI 中的正数区目的指针保护了起来。在执行 STOSB 指令后,再用 XCHG 指令交换回来,以便下次转回 GOON 标号后,LODS 指令仍能正确执行。

五、控制转移指令

通常,程序中的指令都是顺序地逐条执行的,在 8086 中,指令的执行顺序由 CS 和 IP 决定,每取出一条指令,指令指针 IP 自动进行调整,一条指令执行完后,就从该指令之后的

下一个存储单元中取出新的指令来执行。利用控制转移指令可以改变 CS 和 IP 的值,从而改变指令的执行顺序。为满足程序转移的不同要求,8086 提供了无条件转移和过程调用、条件转移、循环控制以及中断等几类指令,如表 3-9 所示。

表 3-9 控制转移指令

无条件转移和过程调用指令	
JMP	无条件转移
CALL	过程调用
RET	过程返回
条 件 转 移	
JZ/JE 等 10 条指令	直接标志转移
JA/JNBE 等 8 条指令	间接标志转移
条 件 循 环 控 制	
LOOP	CX $\neq$ 0 则循环
LOOPE/LOOPZ	CX $\neq$ 0 和 ZF=1 则循环
LOOPNE/LOOPNZ	CX $\neq$ 0 和 ZF=0 则循环
JCXZ	CX=0 则转移
中 断	
INT	中断
INTO	溢出中断
IRET	中断返回

1. 无条件转移和过程调用指令 (Unconditional Transfer and Call)

(1) JMP 无条件转移指令 (Jump)

指令格式: JMP 目的

指令功能: 使程序无条件地转移到指令中指定的目的地址去执行。

这类指令又分成两种类型:

第一种类型: 段内转移或近 (NEAR) 转移, 转移指令的目的地址和 JMP 指令在同一代码段中, 转移时仅改变 IP 寄存器的内容, 段地址 CS 的值不变。

第二种类型: 段间转移, 又称为远 (FAR) 转移, 转移指令的目的地址和 JMP 指令不在同一段中, 发生转移时, CS 和 IP 的值都要改变, 也就是说程序要转到另一代码段去执行。

不论是段内还是段间转移, 就转移地址提供的方式而言, 又可分为两种方式:

第一种方式: 直接转移, 在指令码中直接给出转移的目的地址, 目的操作数用一个标号来表示, 它又可分为段内直接转移和段间直接转移。

第二种方式: 间接转移, 目的地址包含在某个 16 位寄存器或存储单元中, CPU 必须根据寄存器或存储器寻址方式, 间接地求出转移地址。同样, 这种转移类型又可分为段内间接转移和段间间接转移。

所以无条件转移指令可分成段内直接转移、段内间接转移、段间直接转移和段间间接转移这四种不同类型和方式, 如表 3-10 所示。

表 3-10 无条件转移指令的类型和方式

类 型	方式	寻址目标	指令举例
段内 转移	直接	立即短转移(8 位)	JMP SHORT PROG-S
	直接	立即近转移(16 位)	JMP NEAR PTR PROG-N
	间接	寄存器(16 位)	JMP BX
	间接	存储器(16 位)	JMP WORD PTR 5[BX]
段间 转移	直接	立即转移(32 位)	JMP FAR PTR PROG-F
	间接	存储器(32 位)	JMP DWORD PTR [DI]

下面参照表 3-10 中列举的例子,再对无条件转移指令作进一步的说明。

① 段内直接转移指令

指令格式为: JMP SHORT 标号

JMP NEAR PTR 标号 (或 JMP 标号)

这是一种段内相对转移指令,目的操作数均用标号表示,程序转向的有效地址等于当前 IP 寄存器的内容加上 8 位或 16 位位移量(DISP)。若位移量位是 16 位,表示近转移,说明目的地址与当前 IP 的距离在 $-32768 \sim +32767$ 个字节之间。如果转移的范围在 $-128 \sim +127$ 个字节之内,则称为短转移,指令中只需用 8 位位移量,它是近转移指令的一个特例。

在机器语言指令中,8 位或 16 位位移量用带符号数表示,正的位移量表示向高地址方向转移,负的位移量表示向低地址方向转移,负位移量必须用补码表示。段内近—短转移指令的机器码及其操作功能如图 3-19 所示,其中第一个字节为操作码,后面的字节是位移量。注意,由于 IP 为 16 位长,当它与 8 位的位移量相加时,实际上是用符号扩展法将 8 位位移量扩展成 16 位数后才相加的。

段内近转移指令

E9	DISP-L	DISP-H
----	--------	--------

执行操作: $IP \leftarrow IP + 16 \text{ 位位移量}$

段内短转移指令

EB	DISP-L
----	--------

执行操作: $IP \leftarrow IP + 8 \text{ 位位移量}$

图 3-19 段内短-近转移指令机器码及操作

在汇编指令语句中,目的操作数用符号地址也就是标号表示。对于位移量为 16 位的近转移,则在标号前加说明符 NEAR PTR,该说明符也可以省略不写。对于位移量为 8 位的短转移,则在标号前需加说明符 SHORT。例如:

```
JMP SHORT PROG-S ;段内短转移
JMP NEAR PTR PROG-N ;段内近转移(或写为 JMP PROG-N)
```

下面是一个含有一条无条件转移指令的简单程序的列表文件,它是由汇编语言源程序经汇编程序翻译后产生的。即

行号	偏移量	机器码	程 序
1	0000	CODE	SEGMENT
2			ASSUME CS: CODE
3	0000	0405	PROG-S: ADD AL, 05H
4	0002	90	NOP
5	0003	EBFB	JMP SHORT PROG-S
6	0005	90	NOP
7	0006	CODE	ENDS
8			END

程序共 8 行,最左边一列是程序的行号。整个程序仅包含一个代码段,段名为 CODE,它以 CODE SEGMENT 语句开头,以 CODE ENDS 语句结束。整个程序以 END 语句结束。行号右面的一列数字表示每条指令的首字节距离代码段基地址的偏移量(Offset),偏移量右面的数字代表每条指令的机器码。标号为 PROG-S 的加法指令的偏移量为 0,该指令占 2 个字节,所以下一条空操作指令(NOP)首字节的偏移量为 2,NOP 指令除占用一个字节空间和用去 3 个时钟周期外,不做任何事情。

JMP SHORT PROG S 指令的首字节偏移量为 3,当 8086 取出这条 2 字节的 JMP 指令后,首先增量地址指针,使 IP=5,指向 CPU 原打算执行的第二条 NOP 指令。然而,JMP 指令将改变程序执行的次序,它要转向的目的地址是 PROG-S,而 PROG-S 的偏移量为 0,所以 JMP 指令中的位移量 DISP 为:

$DISP = \text{目的地址偏移量} - IP \text{ 的当前值} = 0 - 5 = -5$,其补码为 FBH,经符号扩展后成为 FFFBH,这样,JMP 指令将把 IP 修改为:

$$IP = IP + DISP = 0005 + FFFB = 0$$

于是,程序转到偏移量为 0 的位置,即标号为 PROG S 处执行。

对于段内近转移指令,除了位移量可以是 16 位外,和段内短转移指令一样,也采取相对寻址,在汇编格式中也使用符号地址(标号)。不过段内近转移指令占有 3 个字节,由于计算位移量时,总是从紧跟 JMP 指令之后的下条地址开始计算,所以取出这类 JMP 指令之后,IP 要先加 3,再与目标地址的偏移量相加。对于位移量是 16 位的 JMP 指令,它可以转移到段内任何地址去执行指令。

② 段内间接转移指令

这类指令转向的 16 位有效地址存放在一个 16 位寄存器或字存储器单元中。

用寄存器间接寻址的段内转移指令,要转向的有效地址存放在寄存器中,执行的操作为:

$$IP \leftarrow \text{寄存器内容}$$

例 3-79

JMP BX

若该指令执行前 BX=4500H,则指令执行时,将当前 IP 修改成 4500H,程序转到代码段内偏移地址为 4500H 处执行。

对于用存储器间接寻址的段内转移指令,要转向的有效地址存放在存储单元中,所以

JMP 指令先要计算出存储单元的物理地址,再从该地址处取一个字送到 IP。即:

$IP \leftarrow \text{字存储单元内容}$

例 3-80

JMP WORD PTR 5[BX]

设指令执行前,DS=2000H, BX=100H, (20105H)=4F0H

则指令执行后, $IP = (20000H + 100H + 5H) = (20105H) = 4F0H$, 即转到代码段内偏移地址为 4F0H 处执行。

这种指令的目的操作数前要加 WORD PTR, 表示进行的是字操作。

③ 段间直接(远)转移指令

指令中用远标号直接给出了转向的段地址和偏移量, 所以只要用指令中的偏移地址取代 IP 寄存器的内容, 用指令中指定的段地址取代 CS 寄存器的内容, 就可使程序从一个代码段转到另一个代码段去执行。

例 3-81

JMP FAR PTR PROG-F

指令中用说明符 FAR PTR 说明 PROG-F 为远标号, 指令执行的操作为:

$IP \leftarrow \text{PROG-F 的段内偏移量}$

$CS \leftarrow \text{PROG-F 所在段的段地址}$

设标号 PROG-F 所在段的基地址=3500H, 偏移地址=080AH

则指令执行后, $IP=080AH$, $CS=3500H$, 程序转到 3500:080AH 处执行。

④ 段间间接转移指令

将目的地址的段地址和偏移量事先放在存储器中的 4 个连续地址单元中, 其中前两个字节为偏移量, 后两个字节为段地址, 转移指令中给出存放目标地址的存储单元的首字节地址值。这种指令的目的操作数前要加说明符 DWORD PTR, 表示转向地址需取双字。

例 3-82

JMP DWORD PTR [SI+0125H]

设指令执行前, $CS=1200H$, $IP=05H$, $DS=2500H$, $SI=1300H$, 内存单元(26425H)=4500H, (26427H)=32F0H。而指令中的位移量 $DISP=0125H$, 其中高位部分为 $DISP-H=01H$, 低位部分为 $DISP-L=25H$ 。

由附录 B 可知, 该指令占 4 个字节, 第一个字节是操作码, 编码为 FFH, 第二个字节的编码为: MOD 101 R/M, 因该指令属于 $[SI]+D16$ 这种形式, 查表 3-2 可得到 MOD=10, R/M=100, 第 3 和第 4 字节分别为位移量低字节和高字节, 于是, 该指令的编码格式为:

OP	MOD 101 R/M	DISP-L	DISP-H
1111 1111	10 101 100	0010 0101	0000 0001

即机器码为 FF AC 25 01H, 它被存放在当前代码段中。

指令要转向的地址存放在内存中,内存单元的物理地址为 $DS:(SI+DISP)$,即

$$\begin{aligned}\text{目的操作数地址} &= DS \times 16 + SI + DISP \\ &= 2500H \times 16 + 1300H + 0125H \\ &= 26425H\end{aligned}$$

从该单元中取出 JMP 指令的转移地址,并赋予 IP 和 CS,即 $IP=4500H$, $CS=32F0H$ 。所以程序转到 $32F0:4500H$ 处执行。

这条指令的执行过程如图 3-20 所示。

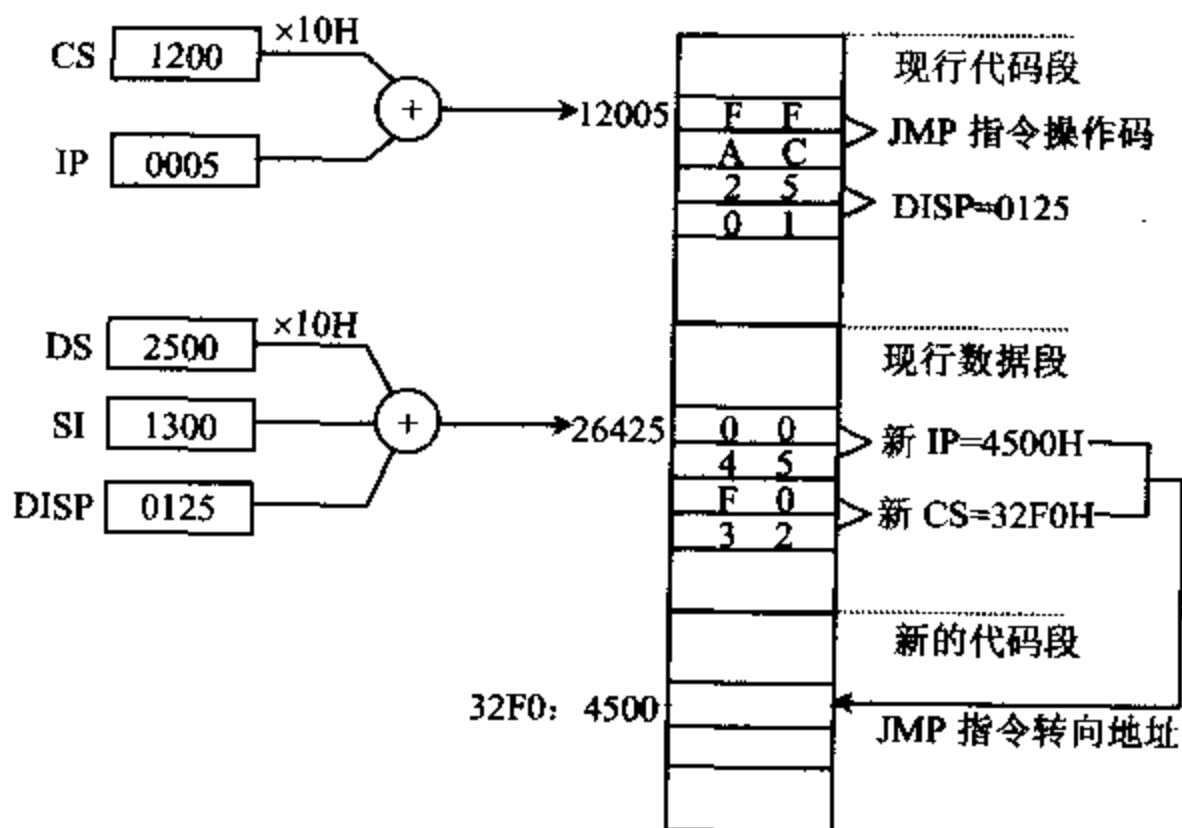


图 3-20 例 3-82 指令的执行过程

(2) 过程调用和返回指令 (Call and Return)

在编写程序时,往往把某些能完成特定功能而又经常要用到的程序段,编写成独立的模块,并把它称为过程 (Procedure),习惯上也称作子程序 (Subroutine),然后在程序中用 CALL 语句调用这些过程,调用过程的程序称为主程序。过程以语句 PROC 开头,用语句 ENDP 结束。在 ENDP 之前要放一条过程返回指令 RET,它与 CALL 指令相呼应,使过程执行完毕后,能正确返回主程序中紧跟在 CALL 指令后面的那条指令继续运行。

若在过程运行中又去调用另一个过程,称为过程嵌套。在模块化程序设计中,过程调用已成为一种必不可少的手段,它使程序结构清晰,可读性强,同时也能节省内存。

过程调用和返回指令的格式如下:

CALL 过程名

RET

过程调用有近调用和远调用两种类型。近过程调用指调用指令 CALL 和被调用的过程在同一代码段中,远过程调用则是指两者不在同一代码段中。CALL 指令执行时分两步进行:

第一步是将返回地址,也就是 CALL 指令下面那条指令的地址推入堆栈。对于近调用来说,执行的操作是:

$SP \leftarrow SP - 2$, IP 入栈

对于远调用来说,执行的操作是:

$SP \leftarrow SP - 2$, CS 入栈

$SP \leftarrow SP - 2$, IP 入栈

第二步是转到子程序的入口地址去执行相应的子程序。入口地址由 CALL 指令的目的操作数提供,寻找入口地址的方法与 JMP 指令的寻址方法基本上是一样的,也有四种方式:段内直接调用,段内间接调用,段间直接调用和段间间接调用,但没有段内短调用指令。

执行过程中的 RET 指令后,从栈中弹出返回地址,使程序返回主程序继续执行。这里也分为两种情况:

如果从近过程返回,则从栈中弹出一个字 $\rightarrow IP$,并且使 $SP \leftarrow SP + 2$ 。

如果从远过程返回,则先从栈中弹出一个字 $\rightarrow IP$,并且使 $SP \leftarrow SP + 2$;再从栈中弹出一个字 $\rightarrow CS$,并使 $SP \leftarrow SP + 2$ 。

下面举例说明 CALL 和 RET 指令的四种寻址方式。

① 段内直接调用和返回

例 3-83

CALL PROG-N ; PROG-N 是一个近标号

根据附录 B 可知,该指令含 3 字节,编码格式为:

E8	DISP-L	DISP-H
----	--------	--------

设调用前: $CS = 2000H$, $IP = 1050H$, $SS = 5000H$, $SP = 0100H$, PROG-N 与 CALL 指令之间的字节距离等于 $1234H$ (即 $DISP = 1234H$)

则执行 CALL 指令的过程为:

- $SP \leftarrow SP - 2$, 即新的 $SP = 0100H - 2 = 00FEH$
- 返回地址的 IP 入栈

由于存放 CALL 指令的内存首地址为 $CS : IP = 2000 : 1050H$,该指令占 3 字节,所以返回地址为 $2000 : 1053H$,即 $IP = 1053H$ 。于是 $1053H$ 被推入堆栈。

- 根据当前 IP 值和位移量 DISP 计算出新的 IP 值,作为子程序的入口地址,即

$$\begin{aligned} IP &= IP + DISP \\ &= 1053H + 1234H \\ &= 2287H \end{aligned}$$

- 程序转到本代码段中偏移地址为 $2287H$ 处执行。指令 CALL PROG-N 的执行过程如图 3-21(a)所示。

RET 指令的寻址方式与 CALL 指令的寻址方式一致,在本例中是段内直接调用,所以过程 PROG-N 中的 RET 指令将执行如下操作:

- $IP \leftarrow (SP \text{ 和 } SP + 1) \text{ 单元内容}$,即 $IP = 1053H$
- $SP \leftarrow SP + 2$,即新的 $SP = 00FEH + 2 = 0100H$

这样,控制就回到主程序中 CALL PROG-N 指令下面的那条指令;也就是 $2000 : 1053$

处继续执行程序。这时堆栈指针为 SP=0100H。RET 指令的执行过程如图 3-21(b)所示。

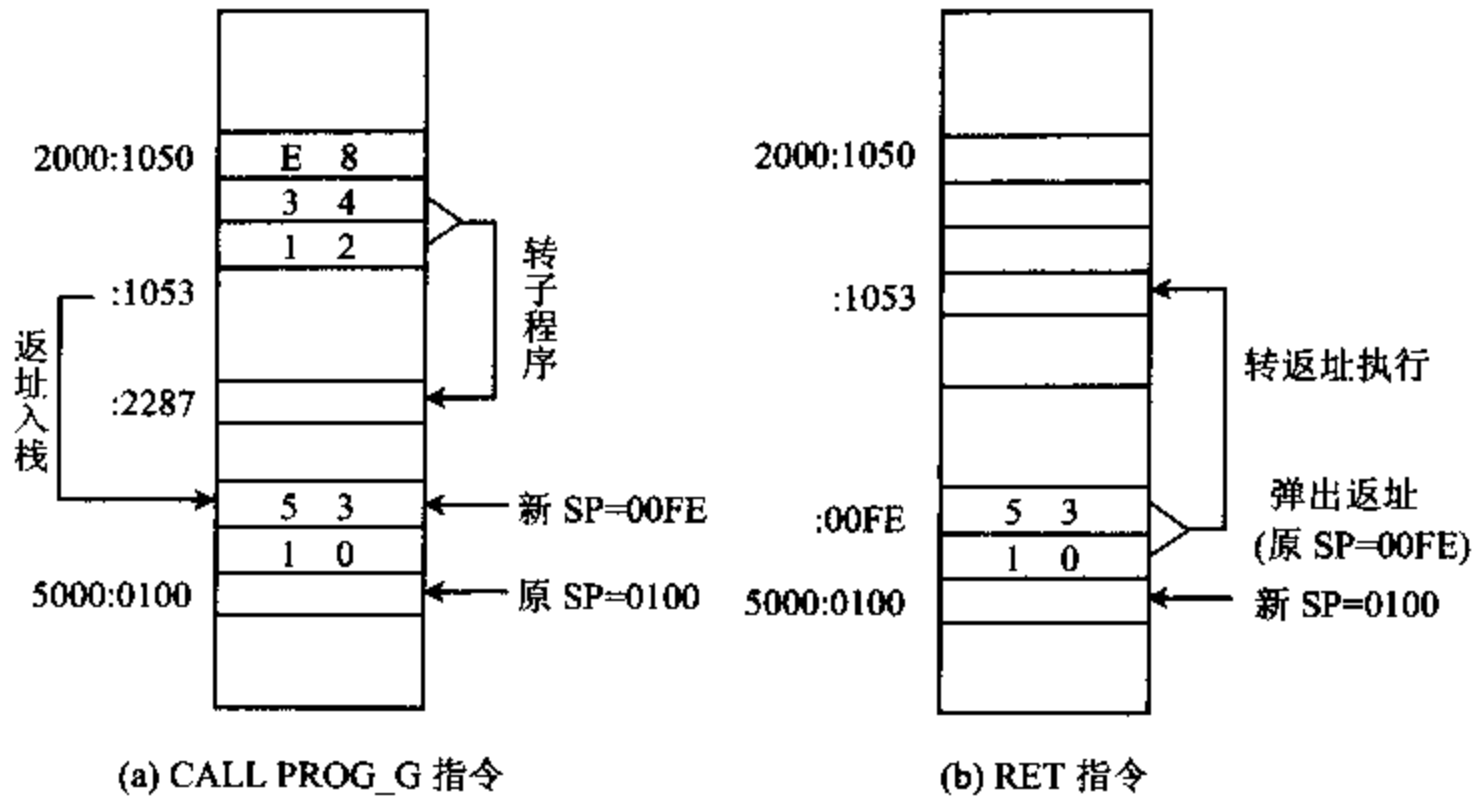


图 3-21 段内直接调用和返回指令执行情况

② 段内间接调用和返回

例 3-84 下面是两条段内间接调用指令的例子：

```
CALL BX
CALL WORD PTR [BX+SI]
```

它们执行的操作分三步进行，具体为：

```
SP←SP-2
IP 入栈
IP←EA
```

这类指令与段内直接调用指令的前两步操作过程完全一样，但第三步求子程序入口地址的方法不一样，它要根据从目的操作数计算出来的有效地址 EA 来转移。

设：DS=1000H，BX=200H，SI=300H (10500H)=3210H

第一条指令的操作数是 BX 寄存器，它包含了过程的 16 位偏移地址 EA=0200H，执行调用指令后，IP←0200H，也就是转到段内偏移地址为 0200H 处执行程序。

第二条指令的子程序入口地址在内存字单元中，其值为

$$(16 \times DS + BX + SI) = (10000H + 0200H + 0300H) = (10500H) = 3210H$$

即 EA=3210H，调用指令执行后，IP←3210H，也就是转到段内偏移地址为 3210H 处执行程序。

对应的 RET 指令执行的操作与段内直接过程的返回指令相类同。

③ 段间直接调用

例 3-85

```
CALL FAR PTR PROG-F ;PROG-F 是一个远标号
```

该指令含 5 字节, 编码格式为:

9A	DISP-L	DISP-H	SEG-L	SEG-H
----	--------	--------	-------	-------

设调用前: CS=1000H, IP=205AH, SS=2500H, SP=0050H, 标号 PROG-F 所在单元的地址指针 CS=3000H, IP=0500H。

存放 CALL 指令的内存首地址为 1000:205AH, 由于该指令长度为 5 个字节, 所以返回地址应为 1000:205FH。

执行远调用 CALL 指令的过程如图 3-22 所示, 具体为:

- $SP \leftarrow SP - 2$ 即 $SP = 0050H - 2 = 004EH$
- CS 入栈 即 CS=1000H 入栈
- $SP \leftarrow SP - 2$ 即 $SP \leftarrow 004CH$
- IP 入栈 即 IP=205FH 入栈
- 转子程序入口 将 PROG-F 的段地址和偏移地址送 CS:IP,
即 $CS \leftarrow 3000H, IP \leftarrow 0500H$
- 执行子程序

过程 PROG-F 中的 RET 指令的寻址方式也是段间直接调用, 返回时执行的操作为:

- $SP \leftarrow SP + 2$ 即 $SP \leftarrow 004CH + 2 = 004EH$
- $IP \leftarrow$ 栈中内容 $IP \leftarrow 205FH$
- $SP \leftarrow SP + 2$ $SP \leftarrow 004EH + 2 = 0050H$
- $CS \leftarrow$ 栈中内容 $CS \leftarrow 1000H$

所以程序转返回地址 CS:IP=1000:205FH 处执行。

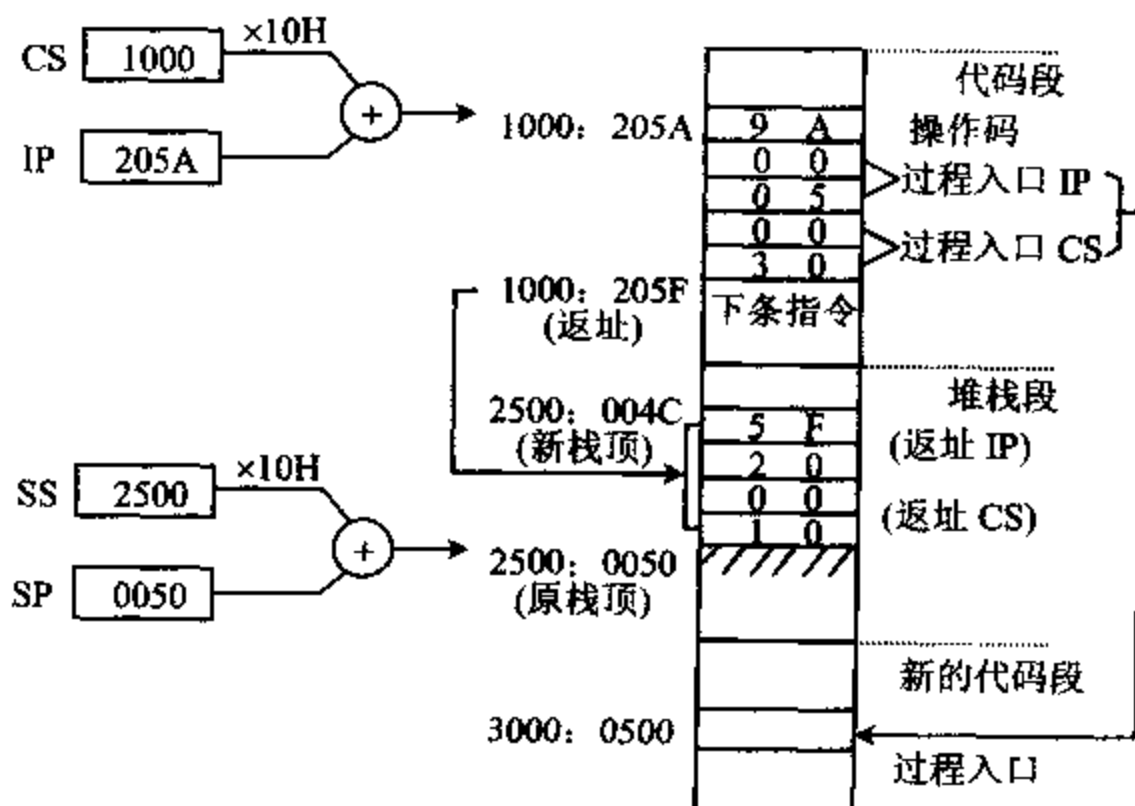


图 3-22 段间调用指令执行过程

④ 段间间接调用

这类调用指令的操作数必须是存储单元, 从该单元开始存放的双字表示过程的入口地

址,其中前2个字节是偏移量,后两个字节为代码段基地址。指令中用 DWORD PTR 说明是对存储单元进行双字操作。

例 3-86

CALL DWORD PTR [BX]

设调用前,DS=1000H,BX=200H,(10200H)=31F4H,(10202)=5200H。

执行指令时,先将返回地址的偏移量和段地址都推入堆栈,再转向过程入口。指令中操作数的物理地址=DS×16+BX=10000H+200H=10200H,从该单元开始取得的双字就是过程的入口地址。所以,入口地址的 CS:IP 分别为:

IP←(10200H) 即IP=31F4H

CS←(10202H) CS=5200H

8086 还有另一种带参数的返回指令,形式为:

RET n

常将 n 称为弹出值,它让 CPU 在弹出返回地址后,再从堆栈中弹出 n 个字节的数据,也就是让 SP 再加上 n。n 用表达式表示,其值可以是 0000~FFFFH 范围内的任何一个偶数。

例如,指令 RET 8 表示从堆栈中弹出地址后,再使 SP 的值加上 8。

在 RET 指令上加弹出值可以让调用过程的主程序通过堆栈向过程传递参数。这些参数必须在调用过程前推入堆栈,过程在运行中可以通过堆栈指针找到它们。当过程返回时,这些参数已没有用处,应该把它们从栈中弹出。使用带弹出值的 RET 指令后,可以在过程返回主程序时,使 SP 自动增量,从而不需要用 POP 指令就能把这些参数从堆栈中弹出。

2. 条件转移指令 (Conditional Transfer)

条件转移指令是根据上一条指令执行后,CPU 设置的状态标志作为判别测试条件来决定是否转移。每一种条件转移指令都有它的测试条件,当条件成立,便控制程序转向指令中给出的目的地址,去执行那里的指令,否则,程序仍顺序执行。

所有的条件转移均为段内短转移,也就是说,转移指令与目的地址必须在同一代码段中。目的地址由当前 IP 值与指令中给出的 8 位相对位移量相加而成,它与转移指令之后的那条指令间的距离,允许为-128~+127 字节。8 位偏移量是用符号扩展法扩展到 16 位后才与 IP 相加的。

条件转移指令通常用在比较指令或算术逻辑运算指令之后,根据比较或运算结果,转向不同的目的地址。在指令中,目的地址均用标号表示,因此指令的格式为:

条件操作符 标号

条件转移指令共有 18 条,可以归类成以下两大类:

(1) 直接标志转移指令

这类转移指令在指令助记符中直接给出标志状态的测试条件,它们以 CF,ZF,SF,OF 和 PF 等 5 个标志的 10 种状态为判断的条件,共形成 10 条指令。有些指令有两种不同的助记符,如“结果为 0”和“相等则转移”,都用 ZF=1 作为测试条件,可用助记符 JZ 或 JE 表示。我们将这类情况记作 JZ/JE,表示这两个助记符代表同样的指令。表 3-11 列出了所有的直接标志转移指令。

表 3-11 直接标志条件转移指令

指令助记符	测试条件	指令功能	
JC	CF=1	有进位	转移
JNC	CF=0	无进位	转移
JZ/JE	ZF=1	结果为零/相等	转移
JNZ/JNE	ZF=0	不为零/相等	转移
JS	SF=1	符号为负	转移
JNS	SF=0	符号为正	转移
JO	OF=1	溢出	转移
JNO	OF=0	无溢出	转移
JP/JPE	PF=1	奇偶位为 1/为偶	转移
JNP/JPO	PF=0	奇偶位为 0/为奇	转移

例 3-87 求 AL 和 BL 寄存器中的两数之和,若有进位,则 AH 置 1,否则 AH 清 0。可用如下程序段来实现该操作:

```

        ADD    AL, BL        ;两数相加
        JC     NEXT         ;若有进位,转 NEXT
        MOV    AH, 0        ;无进位,AH 清 0
        JMP    EXIT         ;往下执行
NEXT:    MOV    AH, 1        ;有进位,AH 置 1
EXIT:    ...                ;程序继续进行

```

(2) 间接标志转移

这类指令的助记符中不直接给出标志状态位的测试条件,但仍以某一个标志的状态或几个标志的状态组合,作为测试的条件,若条件成立则转移,否则顺序往下执行。间接标志转移指令共有 8 条,列于表 3-12 中。每条指令都有两种不同的助记符,中间用“/”隔开。

表 3-12 间接标志条件转移指令

类别	指令助记符	测试条件	指令功能
无符号数 比较测试	JA/JNBE	CF ∨ ZF=0	高于/不低于等于 转移
	JAЕ/JNB	CF=0	高于等于/不低于 转移
	JB/JNAE	CF=1	低于/不高于等于 转移
	JBE/JNA	CF ∨ ZF=1	低于等于/不高于 转移
带符号数 比较测试	JG/JNLE	(SF ∨ OF) ∨ ZF=0	大于/不小于等于 转移
	JGE/JNL	SF ∨ OF=0	大于等于/不小于 转移
	JL/JNGE	SF ∨ OF=1	小于/不大于等于 转移
	JLE/JNG	(SF ∨ OF) ∨ ZF=1	小于等于/不大于 转移

这些指令通常放在比较指令 CMP 之后,通过测试状态位来比较两个数的大小。对两个无符号数进行比较后,一定要用无符号数比较测试指令决定程序走向;对带符号数比较后,则要用带符号数比较测试指令。使用时要严格加以区分,否则会得到错误的结果。

在无符号数比较测试指令中,指令助记符中的“A”是英文 Above 的缩写,表示“高于”之意,“B”是英文 Below 的缩写,表示“低于”之意。

例 3-88

设 AL=F0H, BL=35H, 执行指令

```
CMP AL, BL      ;AL-BL
JAE NEXT
```

执行指令后,若 AL 中的无符号数大于或等于 BL 中的数,则转到标号为 NEXT 的指令去执行,否则执行下一条指令。JAE/JNB 是根据 CF 标志是否为 0 决定转移的,若 CF=0,即无进位,则转移,这与直接标志转移指令中的 JNC 功能完全一样。同样,JB/JNAE 与 JC 指令的功能相同。

对于无符号数进行比较的转移指令比较容易理解,对于带符号数进行比较,情况比较复杂,不能仅根据 SF 或 OF 标志来决定两个数的大小,而要将它们组合起来考虑。指令助记符中,“G”(Great than)表示大于,“L”(Less than)表示小于。有了这几条指令,对比较带符号数的大小提供了很大的方便。

下面举例说明条件转移等指令的用法。

例 3-89 设某个学生的英语成绩已存放在 AL 寄存器中,若低于 60 分,则打印 F (FAIL);若高于或等于 85 分,则打印 G (GOOD);当在 60 分和 84 分之间时,打印 P (PASS)。可用下面程序实现要求的操作:

```
                CMP    AL, 60                ;与 60 分比较
                JB     FAIL                  ;<60,转 FAIL
                CMP    AL, 85                ;≥60,与 85 分比较
                JAE    GOOD                  ;≥85,转 GOOD
                MOV     AL, 'P'              ;其它,将 AL←'P'
                JMP     PRINT                ;转打印程序
FAIL:           MOV     AL, 'F'              ;AL←'F'
                JMP     PRINT                ;转打印程序
GOOD:           MOV     AL, 'G'              ;AL←'G'
PRINT:          ...                        ;打印存在 AL 中的字符
```

例 3-90 我们再来看一个温度控制的例子。假设某温度控制系统中,从温度传感器输入一个 8 位二进制的摄氏温度值。当系统中温度低于 100 度时,则打开加热器;当温度上升到 100 度或 100 度以上时,关闭加热器,进行下一步处理。设温度传感器的端口号为 320H,同时假设控制加热器的输出信号连到端口 321H 的最低有效位,当将这一位置 1 时,加热器便打开,清 0 时则关闭加热器。实现上述温度控制的程序为:

```
GET_TEMP:      MOV     DX, 320H              ;DX 指向温度输入端口
                IN      AL, DX                ;读取温度值
                CMP     AL, 100               ;与 100 度比较
                JB      HEAT_ON               ;<100 度,加热
                JMP     HEAT_OFF              ;≥100 度,停止加热
HEAT_ON:        MOV     AL, 01H               ;D0 位置 1,加热
                MOV     DX, 321H              ;加热器口地址
                OUT     DX, AL                ;打开加热器
```

	JMP	GET-TEMP	;继续检测温度
HEAT-OFF;	MOV	AL, 00	;D ₆ 位置 0, 停止加热
	MOV	DX, 321H	
	OUT	DX, AL	;关闭加热器
	:		;进行其它处理

例 3-91 在以首地址为 TABLE 的 10 个内存字节单元中存放了 10 个带符号数, 要求统计其中正数、负数和零的个数, 并将结果分别存入 PLUS、NEGT 和 ZERO 单元。

程序如下:

TABLE	DB	01H, 80H, 0F5H, 32H, 86H	
	DB	74H, 49H, 0AFH, 25H, 40H	
PLUS	DB	0	;存正数的个数
NEGT	DB	0	;存负数的个数
ZERO	DB	0	;存零的个数
	:		
	MOV	CX, 10	;数据总数
	MOV	BX, 0	;BX 清零
AGAIN;	CMP	TABLE[BX], 0	;取一个数与 0 比
	JGE	GRET-EQ	;≥0, 转 GRET-EQ
	INC	NEGT	; < 0, 负数个数加 1
	JMP	NEXT	;往下执行
GRET-EQ;	JG	P-INC	; > 0, 转 P-INC
	INC	ZERO	; = 0, 零个数加 1
	JMP	NEXT	;往下执行
P-INC;	INC	PLUS	;正数个数加 1
NEXT;	INC	BX	;数据地址指针加 1
	DEC	CX	;数据计数器减 1
	JNZ	AGAIN	;未完, 继续统计
	:		

3. 循环控制指令 (Iteration Control)

循环控制指令是一组增强型的条件转移指令, 用来控制一个程序段的重复执行, 重复次数由 CX 寄存器中的内容决定。这类指令的字节数均为 2, 第一字节是操作码, 第二字节是 8 位偏移量, 转移的目标都是短标号。它们的操作过程与条件转移类似, 转移地址等于当前 IP 加上 8 位偏移量, 8 位偏移量与 IP 相加时, 先按符号扩展法扩展到 16 位后再相加, 循环指令中的偏移量都是负值。循环控制指令均不影响任何标志, 这类指令共有 4 条。

(1) LOOP 循环指令 (Loop)

指令格式: LOOP 短标号

指令功能: 这条指令用于控制重复执行一系列指令。指令执行前必须事先将重复次数放在 CX 寄存器中, 每执行一次 LOOP 指令, CX 自动减 1。如果减 1 后 CX ≠ 0, 则转移到指令中所给定的标号处继续循环; 若自动减 1 后 CX = 0, 则结束循环, 转去执行 LOOP 指令之

后的那条指令。一条 LOOP 指令相当于执行以下两条指令的功能：

```
DEC CX
JNZ 标号
```

因此,我们可以把例 3-91 程序中的最后两行,即根据 CX 减 1 后的值是否为 0,来判断统计有没有结束,改为使用下面这条指令来实现:

```
LOOP AGAIN
```

例 3-92 设商店里有 8 种商品,它们的价格分别为 83 元,76 元,65 元,84 元,71 元,49 元,62 元和 58 元,现要将每种商品提价 7 元,编程计算每种商品提价后的价格。

这是一个简单的加法问题,我们将商品的原价按 BCD 码的形式,依次存放在标号为 OLD 开始的 8 个存储单元中,而新的价格存放在 NEW 开始的 8 个单元,然后用 LOOP 指令来实现 8 次循环。即

```
OLD      DB  83H, 76H, 65H, 84H
          DB  71H, 49H, 62H, 58H
NEW      DB  8 DUP(?)
          :
          MOV CX, 08H                ;共 8 种商品
          MOV BX, 00H                ;BX 作指针,初值为 0
NEXT:     MOV AL, OLD[BX]             ;读入一个商品的原价
          ADD AL, 7                   ;加上提价因子
          DAA                         ;调整为十进制数
          MOV NEW[BX], AL            ;存放结果
          INC BX                      ;地址指针加 1
          LOOP NEXT                  ;如还未加满 8 次,继续循环
          :                           ;已加完 8 次
```

从 LOOP 指令到它所指向的标号之间的指令也称为循环体,在循环体内的所有指令将被重复执行,执行次数由 CX 寄存器的初值决定。循环体中也可以只含一条指令,即 LOOP 指令自身,这样的程序段常用来实现延时。例如:

```
          MOV CX, 10                  ;循环次数为 10
DELAY:    LOOP DELAY                 ;本指令重复执行 10 次
```

例 3-93 这是一个用循环和跳转指令来控制 PC 机的扬声器发声的程序。在 PC 机中,61H 端口的 D₁ 和 D₀ 位接到扬声器接口电路上,在 D₀=0 的情况下:当 D₁=1 时,扬声器被接通,D₁=0 则断开,通过控制这一位的值,就能产生一个由 1 和 0 构成的二进制序列,使扬声器发声。61H 端口中的其它位则用来控制 PC 机的内部开关状态、奇偶校验及键盘状态等,这些内容将在第十章中讨论 8255 芯片时进行详细的介绍,在这里,我们先将这些状态保存起来。控制扬声器发声的程序如下:

```
          IN AL, 61H                  ;AL←从 61H 端口读取数据
          AND AL, 0FCH                ;保护 D7~D2 位,D0 位清零
MORE:     XOR AL, 02                  ;触发 D1 位,使之在 0 和 1 间变化
```

	OUT 61H, AL	;控制扬声器开关通断
	MOV CX, 260	;CX=循环次数
DELAY:	LOOP DELAY	;循环延时
	JMP MORE	;再次触发

本例中,LOOP 指令重复执行 260 遍,起延时作用,使开关的接通和断开维持一定的时间。这是非常必要的,否则开关动作太快,发出的声音频率太高,人耳听不出来。

(2) LOOPE/LOOPZ 相等或结果为零时循环 (Loop If Equal/Zero)

指令格式: LOOPE 标号

或 LOOPZ 标号

指令功能: LOOPE 是相等时循环,LOOPZ 是结果为零时循环,这是两条能完成相同功能,而具有不同助记符的指令,它们用于控制重复执行一组指令。指令执行前,先将重复次数送到 CX 中,每执行一次指令,CX 自动减 1,若减 1 后 $CX \neq 0$ 和 $ZF=1$,则转到指令所指定的标号处重复执行;若 $CX=0$ 或 $ZF=0$,便退出循环,执行 LOOPZ/LOOPE 之后的那条指令。

例 3-94 设有一个由 50 个字节组成的数组存放在 ARRAY 开始的内存单元中,现要对该数组中的元素进行测试,若元素为 0,而且不是最后一个元素,便继续进行下一个元素的测试,直到找到第一个非零元素或查完了为止。

程序如下:

ARRAY	DB	XX,XX,...	;含 50 个元素的数组
	:		
	MOV	BX, OFFSET ARRAY	;BX 指向数组开始单元
	DEC	BX	;指针减 1
	MOV	CX, 50	;CX=元素个数
NEXT:	INC	BX	;指向数组的下个元素
	CMP	[BX], 00H	;数组元素与 0 比较
	LOOPE	NEXT	;若元素为 0 和 $CX \neq 0$,循环
	:		;否则,结束查找

(3) LOOPNE/LOOPNZ 不相等或结果不为零循环 (Loop If Not Equal/Not Zero)

指令格式: LOOPNE 标号

或 LOOPNZ 标号

指令功能: LOOPNE 是不相等时循环,而 LOOPNZ 是结果不为零循环,它们也是一对功能相同但形式不一样的指令。指令执行前,应将重复次数送入 CX,每执行一次,CX 自动减 1,若减 1 后 $CX \neq 0$ 和 $ZF=0$,则转移到标号所指定的地方重复执行;若 $CX=0$ 或 $ZF=1$,则退出循环,顺序执行下一条指令。

例 3-95 设一个由 17 个字符组成的字符串存放在 STRING 开始的内存中,现要查找该字符串中是否包含空格符。若没有找到空格符和尚未查完,则继续查找,直到找到第一个空格符或查完了才退出循环。

下面是实现上述操作的程序:

STRING	DB	'Personal Computer'	;字符串
--------	----	---------------------	------

```

        :
        MOV    BX, OFFSET STRING      ;BX 指向字符串的开始
        DEC    BX                     ;BX-1
        MOV    CX, 17                 ;CX=字符串长度
NEXT:    INC    BX                     ;指向下一个字符
        CMP    [BX], 20H              ;字符串元素与空格比较
        LOOPNE NEXT                  ;若不是空格和 CX≠0,循环
        :                             ;找到空格或 CX 已为 0

```

(4) JCXZ 若 CX 为 0 跳转 (Jump If CX Zero)

指令格式: JCXZ 标号

指令功能: 若 CX 寄存器为零, 则转移到指令中标号所指定的地址处, 否则将往下顺序执行, 它不对 CX 寄存器进行自动减 1 的操作。

这条指令主要用在循环程序开始处。为了要使程序跳过循环, 只要事先把 CX 寄存器清零。

4. 中断指令 (Interrupt)

(1) 中断概念

所谓中断是指计算机在执行正常的程序的过程中, 由于某些事件发生, 需要暂时中止当前程序的运行, 转到中断服务程序去为临时发生的事件服务, 中断服务程序执行完毕后, 又返回正常程序继续运行, 这个过程称为中断。

引起中断的原因称为中断源, 8086 的中断源有两种:

第一种是外部中断, 也称为硬件中断, 它们从 8086 的不可屏蔽中断引脚 NMI 或可屏蔽中断引脚 INTR 引入。NMI 引脚上的中断请求, 情况比较紧急, 如存储器校验错、电源掉电等发生时, CPU 必须马上响应。从 INTR 脚上来的请求信号, CPU 可以响应, 也可以不响应。如果 CPU 内部标志寄存器中的 IF 置 1, 则允许响应这类中断; 若 IF 标志为 0, 则不予响应。外部的可屏蔽中断是通过 8259A 可编程中断控制器连到 CPU 上的, 主要用来管理 I/O 设备与 CPU 之间的通信。

第二种中断是内部中断, 这种中断是为了解决 CPU 在运行过程中发生一些意外情况, 如除法运算出错和溢出错误等, 或者是为便于对程序进行调试而设置的。此外, 也可以在程序中安排一条中断指令 INT n, 利用这条指令直接产生 8086 的内部中断, 指令中的 n 为中断类型号。

CPU 每响应一次中断, 首先, 不但要像过程调用指令那样, 把 CS 和 IP 寄存器的值也即断点送到堆栈保护起来, 而且还要将标志寄存器的值入栈保护, 以便在中断服务程序执行完后, 能正确恢复 CPU 的状态。然后, 找到中断服务程序的入口地址, 转相应的中断服务程序。中断服务程序结束时, 通过执行中断返回指令 IRET, 从堆栈中恢复中断前 CPU 的状态和断点, 返回正常程序继续执行。

中断服务程序的入口地址通常被称为中断向量 (Interrupt Vector) 或中断矢量。8086 可处理 256 类中断, 类型号为 0~255。每类中断有一个入口地址, 需用 4 个字节存储 CS 和 IP, 256 类中断的入口地址要占用 1K 字节, 它们位于内存 00000~003FFH 的区域中。存储这些地址的连续空间称为中断向量表或中断矢量表。

00000H	类型 0 的中断服务程序入口地址
00004H	类型 1 的中断服务程序入口地址
00008H	类型 2 的中断服务程序入口地址
0000CH	⋮
003FCH	类型 255 的中断服务程序入口地址

图 3-23 8086 的中断向量表示意图

为专用中断。它们分别是：

① 除法错中断（类型 0）

在执行除法指令时，若发现除数为 0 或者所得的商超过了寄存器能容纳的范围，则处理器自动产生一个类型为 0 的中断。

② 单步中断（类型 1）

如果 CPU 的单步标志 TF 置 1，那么每执行完一条指令后，会自动产生类型为 1 的中断，CPU 响应后，暂停执行下条指令，而转到单步中断服务程序去执行，其结果是能将 CPU 的内部寄存器和有关存储器的内容显示在 CRT 上，便于跟踪程序的执行过程，实现动态排错。

③ 不可屏蔽中断（类型 2）

当 8086 的 NMI 引脚上接收到由低变高的电平变化时，将自动产生类型为 2 的不可屏蔽中断。

④ 断点中断（类型 3）

断点中断也是为调试程序而设置的，它的类型号为 3。如果在程序中的关键地方设置了断点，每当程序执行到断点时便产生中断，这时也可以像单步中断一样，查看各寄存器和有关存储单元的内容。断点可以设在程序中的任何地方，设置的方法是插入一条 INT 3 指令。利用断点中断，可以加快程序的调试速度。

⑤ 溢出中断（类型 4）

如果溢出标志 OF 置 1，则可由溢出中断指令 INTO 产生类型为 4 的中断。

除了单步中断外，所有的内部中断都不受中断允许标志 IF 的控制，因此都不能被屏蔽。关于中断的详细内容，将在第八章中进一步作深入讨论，这里仅介绍与中断有关的 3 条指令。

(2) 中断指令

① INT n 软件中断指令 (Interrupt)

软件中断指令，也称为软中断指令，其中 n 为中断类型号，其值必须在 0~255 的范围内。它可以在编程时安排在程序中的任何位置上，因此也被称为陷阱中断。

CPU 执行 INT n 指令时，先把标志寄存器的内容推入堆栈，再把当前断点的段基地址 CS 和偏移地址 IP 入栈保护，并清除中断标志 IF 和单步标志 TF。然后将中断类型号 n 乘

图 3-23 给出了 8086 的中断向量表示意图，每种类型的中断向量中，高 2 字节存放中断服务程序入口地址的段地址 (CS)，低 2 字节存放相应于该地址段的偏移量 (IP)。由于每个中断向量要占 4 个字节单元，所以必须将中断指令中给定的类型号乘以 4 才能得到规定类型的中断向量。例如，对于类型号为 5 的中断，它的中断矢量放在 $5 \times 4 = 00014H$ 开始的 4 个连续的内存单元中。

在 8086 的 256 类中断中，有些类型的中断是专为硬件提供的，有些为 DOS 或基本输入输出系统 BIOS (Basic Input Output System) 等所用，最低的 5 个类型的中断被定义

以 4, 找到中断服务程序的入口地址表的表头地址, 从中断矢量表中获得中断服务程序的入口地址, 将其置入 CS 和 IP 寄存器, CPU 就自动转到相应的中断服务程序去执行。

原则上讲, 利用 INT n 指令能以软件的方法调用所有 256 个中断的服务程序, 尽管其中有些中断实际上是由硬件触发的。这样, 除了在程序中需要调用特定的中断服务程序时插入 INT n 指令调用大量为外设服务的子程序外, 还可以利用这条指令来调试各种中断服务程序。例如, 可用 INT 0 指令让 CPU 执行除法出错中断服务程序, 而不必运行除法程序。又如, 可用 INT 2 指令执行 NMI 中断服务程序, 从而可以不必在 NMI 引脚上加外部信号, 便能对 NMI 子程序进行调试。

② INTO 溢出中断指令 (Interrupt On Overflow)

当带符号数进行算术运算时, 如果溢出标志 OF 置 1, 则可由溢出中断指令 INTO 产生类型为 4 的中断, 若 OF 清零, 则 INTO 指令不产生中断, CPU 继续执行后续程序。

为此, 在带符号数进行加减法运算之后, 必须安排一条 INTO 指令, 一旦溢出就能及时向 CPU 提出中断请求, CPU 响应后可做出相应的处理, 如显示出错信息, 使运算结果无效等。由于运算结果出了错误, 溢出中断处理完之后, CPU 将不返回原程序继续运行, 而是把控制权交给操作系统。

如果程序中不加 INTO 指令, 即使运算后产生了溢出错误, 也不会向 CPU 发中断请求, 从而会导致错误的运算结果。

③ IRET (Interrupt Return)

中断返回指令 IRET。它总是被安排在中断服务程序的出口处, 当它执行后, 首先从堆栈中依次弹出程序断点, 送到 IP 和 CS 寄存器中, 接着弹出标志寄存器的内容, 送回标志寄存器, 然后按 CS: IP 的值使 CPU 返回断点, 继续执行原来被中断的程序。

六、处理器控制指令

1. 标志操作指令

除了有些指令执行后会影响到标志外, 8086 还提供了一组标志操作指令, 它们可直接对 CF、DF 和 IF 标志位进行设置或清除等操作, 但不包含 TF 标志。指令执行后不影响其它标志, 只影响本指令指定的标志, 这些指令的名称和功能如表 3-13 所示。

表 3-13 标志操作指令

指令助记符	操 作	指 令 名 称
CLC	CF←0	进位标志清 0 (Clear Carry)
CMC	CF← $\overline{\text{CF}}$	进位标志求反 (Complement Carry)
STC	CF←1	进位标志置 1 (Set Carry)
CLD	DF←0	方向标志位清 0 (Clear Direction)
STD	DF←1	方向标志位置 1 (Set Direction)
CLI	IF←0	中断标志位清 0 (Clear Interrupt)
STI	IF←1	中断标志位置 1 (Set Interrupt)

(1) CLC、CMC 和 STC

进位标志 CF 常用在多字节或多字运算中, 用来说明低位向高位的进位情况。在这种

运算中,经常要对 CF 进行处理。利用 CLC 指令,可使进位标志 CF 清 0,利用 CMC 指令可使 CF 取反,而利用 STC 指令则使 CF 置 1。

(2) CLD 和 STD

方向标志 DF 在执行字符串操作指令时用来决定地址的修改方向,当串操作由低地址向高地址方向进行时,DF 应清 0,反之 DF 应置 1。CLD 指令使 DF 清 0,而 STD 指令则使 DF 置 1。

(3) CLI 和 STI

中断允许标志 IF 是 1 还是 0 用于决定 CPU 是否可以响应外部的可屏蔽中断请求。指令 CLI 使 IF 清 0,将禁止 CPU 响应中断,而指令 STI 使 IF 置 1,允许 CPU 响应这类中断。

2. 外部同步指令

8086 CPU 具有多处理机的特征,为了充分发挥硬件的功能,设置了 3 条使 CPU 与其它协处理器同步工作的指令,以便共享系统资源。这几条指令执行后均不影响标志位。

(1) ESC 换码指令 (Escape)

指令格式: ESC 外部操作码, 源操作数

指令功能: 换码指令用来实现 8086 对 8087 协处理器的控制。

ESC 指令的编码格式为:

15	11	10	8	7	6	5	3	2	0
1	1	0	1	1	x	x	x	MOD	y y y R/M

前 5 位编码 11011 是 ESC 指令的操作码。xxx 和 yyy 字段的 6 位代码是 6 位操作码,是协处理器从 8086 的程序中获得的一个操作码。其中的 xxx 为协处理器号,最多可接 8 个协处理器;yyy 用来表示要求处理器完成的功能,最多可以完成 8 种功能。MOD 和 R/M 字段用来指定操作数。

由于 8086 和 8087 的系统总线是互连的,所以两个处理器一直共同监视着从存储器中取出的每一条指令。8087 只处理与自己有关的 ESC 指令,而不理睬 8086 的其它指令,8086 也只处理属于它自己的指令。一旦取到一条 ESC 指令,8087 的忙碌(BUSY)引脚就变成高电平,并将它送到与之相连的 8086 的 $\overline{\text{TEST}}$ 脚上,8087 协处理器便可以开始工作。

8087 能执行的 ESC 指令有 3 类,它们是不需要访问存储器的指令、需要访问存储器取得操作数的指令和需要将操作数写入存储器的指令。若取到的是不要访问存储器的 ESC 指令,操作比较简单,一般直接由 8087 本身完成。对于要访问存储器的指令,则按下面方式进行处理:若 MOD $\neq$ 11,则源操作数是存储器,需由 8086 计算出指令中的操作数地址,再从中取出数据,将它作为 8087 的源操作数;如果 MOD=11,则指令中的源操作数就是 8087 的寄存器,8087 将根据 8086 计算出来的目的操作数,与存储器交换数据。操作结束后,通过 $\overline{\text{TEST}}$ 状态线,向 8086 CPU 送出低电平,使 CPU 开始执行后续指令。

由此可见,通过 ESC 指令,可使外部的协处理器从 8086 指令流中获得一条协处理器指令和(或)一个存储器操作数。也就是说,协处理器的指令系统只是 8086 指令系统的一个子集,它们的主操作码都是 ESC,协处理器是根据附加操作码来完成规定的操作的。

(2) WAIT 等待指令 (Wait)

等待指令(WAIT)通常跟在 ESC 指令之后,CPU 执行 ESC 指令后,表示 8086 CPU 正

处于等待状态,它不断检测 8086 的测试引脚 $\overline{\text{TEST}}$,每隔 5 个时钟周期检测一次,若此脚为高电平,则重复执行 WAIT 指令,处理器处于等待状态。一旦 $\overline{\text{TEST}}$ 脚上的信号变为低电平,便退出等待状态,执行下条指令。

(3) LOCK 封锁总线指令 (Lock Bus)

它是一种前缀,可加在任何指令的前端,用来维持 8086 的总线封锁信号 $\overline{\text{LOCK}}$ 有效,凡带有 LOCK 前缀的指令在执行过程中,将禁止其它处理器使用总线。

3. 停机指令和空操作指令

(1) HLT 停机指令 (Halt)

它使 CPU 进入暂停状态,不进行任何操作,只有当下列情况之一发生时,CPU 才脱离暂停状态:

- 在 RESET 线上加复位信号;
- 在 NMI 引脚上出现中断请求信号;
- 在允许中断的情况下,在 INTR 引脚上出现中断请求信号;

在程序中,通常用 HLT 指令来等待中断的出现。

(2) NOP 空操作或无操作指令 (No Operation)

这是一条单字节指令,执行时需耗费 3 个时钟周期的时间,但不完成任何操作。它不影响标志位。NOP 指令通常被插在其它指令之间,在循环等操作中增加延时。还常在调试程序时使用空操作指令。例如用 3 条 NOP 指令代替一条调试时暂不执行的 3 字节指令,调试好后再用这条指令代换 NOP 指令。

七、指令的执行时间和软件延时

8086 指令的执行速率是由晶振控制产生的时钟决定的,每条指令执行时都需要几个时钟周期。例如,寄存器与寄存器之间的数据传送指令,对寄存器的各种移位指令执行时各需要 2 个时钟周期,DAS 指令需要 4 个时钟周期,JNZ 指令执行时如引起转移,则需要 16 个时钟周期,如不发生转移时仅需要 4 个时钟周期。附录 A 中列出了每条指令执行时所需要的时钟周期数。若已知 CPU 工作时的时钟频率,就可求出每个时钟周期的时间是多少,从而算出执行每条指令或一系列指令所需的时间。

例如,若已知 8086 CPU 的工作时钟为 5MHz,则 1 个时钟周期为

$$T = \frac{1s}{5 \times 10^6} = 0.2\mu s$$

如果某一条指令执时需要 4 个时钟周期,则需花的时间为 $0.2\mu s \times 4 = 0.8\mu s$ 。

表 3-14 列出一些指令执行时所需要的时钟数和时间值。

表 3-14 一些指令执行时需要的时钟数和时间值

指 令	时钟周期数	执行时间
MOV 寄存器,寄存器	2	$0.2\mu s \times 2 = 0.4\mu s$
CALL 段内直接调用	19	$0.2\mu s \times 19 = 3.8\mu s$
CALL 段间间接调用	$37 + EA$	$0.2\mu s \times (37 + EA)\mu s$

表 3-14 里第 3 条指令中的 EA 项表示,该指令执行时还要考虑计算有效地址所需要的时间,寻址方式不同,计算有效地址所需的时间也不一样,即 EA 的值不一样,具体数值如表 3-15 所示。

表 3-15 计算 EA 所需的时间

寻址方式	时钟周期数
直接寻址	6
寄存器间接寻址	5
寄存器相对寻址	9
基址变址寻址	7~8
相对基址变址寻址	11~12

在程序中需要一定的延时或进行定时时,可以选用适当的指令(如 NOP 等)和循环控制指令构成延时程序。这种方法常被称为软件延时或定时。

例 3-96 设 CPU 的时钟频率为 5MHz,试编写一个延时 1ms 的程序。

我们用下面的程序来实现 1ms 延时,为了计算所需要的延时常数 N,在每条指令后面列出了它们的时钟周期数。

```

                                ;时钟周期×执行次数
DEL_1MS: MOV CX, N           ;4×1
NEXT:    NOP                  ;3×N
          NOP                  ;3×N
          LOOP NEXT           ;循环时为 17,不循环为 5

```

由于 CPU 的时钟为 5MHz,所以一个时钟周期为:

$$T = \frac{1s}{5 \times 10^6} = 0.2\mu s$$

这样,延时 $t=1ms$ 所需要的总的时钟周期数 C_T 为:

$$C_T = \frac{t}{T} = \frac{1ms}{0.2\mu s} = 5000$$

第一条指令只执行一次,两条 NOP 指令各执行 N 次,最后的 LOOP 指令,共循环执行 N-1 次,最后一次不循环。因此,总的时钟周期数 C_T 又可以用循环常数 N 来表示:

$$\begin{aligned}
 C_T &= 4 + 3N + 3N + 17(N-1) + 5 \\
 &= 23N - 8 \\
 &= 5000
 \end{aligned}$$

所以 $N = (5000 + 8) / 23 = 218 = 0DAH$,将此值代进第一条指令,就能使上面这段程序实现 1ms 的精确延时。

例 3-97 若希望获得时间更长的延时,可以采用嵌套的多重循环程序来实现。这时程序中往往有两个定时常数,通常先指定其中的一个,再计算出另一个来。观察下面的程序:

```

                                ;时钟×次数
          MOV BX, N1            ;4×1
CNT1:    MOV CX, N2            ;4×N1

```

```

CNT2: LOOP CNT2          ;(17×N2-12)×N1
      DEC  BX             ;2×N1
      JNZ  CNT1           ;16×N1-12

```

这样,上面程序执行所需要的总的时钟周期数 C_T 为:

$$\begin{aligned}
 C_T &= 4 + (4 \times N1) + ((17 \times N2 - 12) \times N1) + (2 \times N1) + (16 \times N1 - 12) \\
 &= 17 \times N1 \times N2 + 10 \times N1 - 8
 \end{aligned}$$

假设 CPU 的时钟频率仍为 5MHz,可根据所需的延时,计算出总的时钟周期数。然后,先选定常数 $N2$,例如取它的最大可能值 FFFFH,再根据上式算出 $N1$ 来。

习 题

1. 分别说明下列指令的源操作数和目的操作数各采用什么寻址方式。

- | | |
|------------------------|------------------------|
| (1) MOV AX, 2408H | (2) MOV CL, 0FFH |
| (3) MOV BX, [SI] | (4) MOV 5[BX], BL |
| (5) MOV [BP+100H], AX | (6) MOV [BX+DI], '\$' |
| (7) MOV DX, ES:[BX+SI] | (8) MOV VAL[BX+DI], DX |
| (9) IN AL, 05H | (10) MOV DS, AX |

2. 已知: $DS=1000H$, $BX=0200H$, $SI=02H$, 内存 $10200H \sim 10205H$ 单元的内容分别为 10H, 2AH, 3CH, 46H, 59H, 6BH。下列每条指令执行完后 AX 寄存器的内容各是什么?

- (1) MOV AX, 0200H
- (2) MOV AX, [200H]
- (3) MOV AX, BX
- (4) MOV AX, 3[BX]
- (5) MOV AX, [BX+SI]
- (6) MOV AX, 2[BX+SI]

3. 设 $DS=1000H$, $ES=2000H$, $SS=3500H$, $SI=00A0H$, $DI=0024H$, $BX=0100H$, $BP=0200H$, 数据段中变量名为 VAL 的偏移地址值为 0030H, 试说明下列源操作数字段的寻址方式是什么? 物理地址值是多少?

- | | |
|-------------------------|-------------------------|
| (1) MOV AX, [100H] | (2) MOV AX, VAL |
| (3) MOV AX, [BX] | (4) MOV AX, ES:[BX] |
| (5) MOV AX, [SI] | (6) MOV AX, [BX+10H] |
| (7) MOV AX, [BP] | (8) MOV AX, VAL[BP][SI] |
| (9) MOV AX, VAL[BX][DI] | (10) MOV AX, [BP][DI] |

4. 写出下列指令的机器码

- | | |
|--------------------------|----------------|
| (1) MOV AL, CL | (2) MOV DX, CX |
| (3) MOV [BX+100H], 3150H | |

5. 已知程序的数据段为:

```
DATA SEGMENT
A      DB '$', 10H
B      DB 'COMPUTER'
C      DW 1234H, 0FFH
D      DB 5 DUP(?)
E      DD 1200459AH
DATA ENDS
```

求下列程序段执行后的结果是什么。

```
MOV AL, A
MOV DX, C
XCHG DL, A
MOV BX, OFFSET B
MOV CX, 3[BX]
LEA BX, D
LDS SI, E
LES DI, E
```

6. 指出下列指令中哪些是错误的,错在什么地方。

- | | |
|---------------------------|---------------------------|
| (1) MOV DL, AX | (2) MOV 8650H, AX |
| (3) MOV DS, 0200H | (4) MOV [BX], [1200H] |
| (5) MOV IP, 0FFH | (6) MOV [BX+SI+3], IP |
| (7) MOV AX, [BX][BP] | (8) MOV AL, ES:[BP] |
| (9) MOV DL, [SI][DI] | (10) MOV AX, OFFSET 0A20H |
| (11) MOV AL, OFFSET TABLE | (12) XCHG AL, 50H |
| (13) IN BL, 05H | (14) OUT AL, 0FFEH |

7. 已知当前数据段中有一个十进制数字 0~9 的 7 段代码表,其数值依次为 40H, 79H, 24H, 30H, 19H, 12H, 02H, 78H, 00H, 18H。要求用 XLAT 指令将十进制数 57 转换成相应的 7 段代码值,存到 BX 寄存器中,试写出相应的程序段。

8. 已知当前 SS=1050H, SP=0100H, AX=4860H, BX=1287H, 试用示意图表示执行下列指令过程中,堆栈中的内容和堆栈指针 SP 是怎样变化的。

```
PUSH AX
PUSH BX
POP BX
POP AX
```

9. 下列指令完成什么功能?

- | | |
|-------------------|----------------|
| (1) ADD AL, DH | (2) ADC BX, CX |
| (3) SUB AX, 2710H | (4) DEC BX |
| (5) NEG CX | (6) INC BL |
| (7) MUL BX | (8) DIV CL |

10. 已知变量 X1 和 X2 的定义如下:

```
X1    DW  1024H
      DW  2476H
X2    DW  3280H
      DW  9351H
```

请按下列要求进行运算或操作后,将运算结果存到 RESULT 单元中,低位在前,高位在后。试分别写出指令序列。

- (1) 将 X1 和 X2 两个字数据相加。
- (2) 将 X1 和 X2 两个双字数据相加。
- (3) 将 X1 和 X2 两个字数据相减。
- (4) 将 X1 和 X2 两个字数据交换位置。

11. 在数据段中有两个相邻的字节数据 2BH 和 F5H,编程实现下列操作,并说明各标志位的状态。

- (1) 把它们当成带符号数相加后,结果存到内存单元中。
- (2) 把它们当成无符号数相减(2BH-F5H)后,结果存到内存单元中。
- (3) 把它们当成无符号数相加后,结果存到内存单元中。
- (4) 把它们当成无符号数相乘后,结果存到内存单元中。

12. 某班有 7 个同学的英语成绩低于 80 分,分数存在 ARRAY 数组中,试编程完成以下工作:

- (1) 给每人加 5 分,结果存到 NEW 数组中。
- (2) 把总分存到 SUM 单元中。
- (3) 把平均分存到 AVERAGE 单元中。

13. 已知 AX=2508H, BX=0F36H, CX=0004H, DX=1864H, 求下列每条指令执行后的结果是什么? 标志位 CF 等于什么?

- | | |
|------------------|---------------------|
| (1) AND AH, CL | (2) OR BL, 30H |
| (3) NOT AX | (4) XOR CX, 0FFF0H |
| (5) TEST DH, 0FH | (6) CMP CX, 00H |
| (7) SHR DX, CL | (8) SAR AL, 1 |
| (9) SHL BH, CL | (10) SAL AX, 1 |
| (11) RCL BX, 1 | (12) ROR DX, CL |

14. 假设数据段定义如下:

```
DATA    SEGMENT
STRING  DB  'The Personal Computer & TV'
DATA    ENDS
```

试用字符串操作等指令编程完成以下功能:

- (1) 把该字符串传送到附加段中偏移量为 GET_CHAR 开始的内存单元中。
- (2) 比较该字符串是否与“The computer”相同,若相同则将 AL 寄存器的内容置 1, 否则置 0。并要求将比较次数送到 BL 寄存器中。

(3) 检查该字符串是否有“&”符,若有则用空格符将其替换。

(4) 把字符串大写字母传送到附加段中以 CAPS 开始的单元中,其余字符传到以 CHART 开始的单元中。然后将数据段中存储上述字符串的单元清 0。

15. 编程将 AX 寄存器中的内容以相反的次序传送到 DX 寄存器中,并要求 AX 中的内容不被破坏,然后统计 DX 寄存中 1 的个数是多少。

16. 设 CS=1200H, IP=0100H, SS=5000H, SP=0400H, DS=2000H, SI=3000H, BX=0300H, (20300)=4800H, (20302H)=00FFH, TABLE=0500H, PROG_N 标号的地址为 1200:0278H, PROG_F 标号的地址为 3400:0ABCH。说明下列每条指令执行完后,程序将分别转移到何处执行?

- (1) JMP PROG_N
- (2) JMP BX
- (3) JMP [BX]
- (4) JMP FAR PROG_F
- (5) JMP DWORD PTR[BX]

如将上述指令中的操作码 JMP 改成 CALL,则每条指令执行完后,程序转何处执行?并请画图说明堆栈中的内容和堆栈指针如何变化。

17. 如在下列程序段的括号中分别填入以下指令:

- (1) LOOP NEXT
- (2) LOOPE NEXT
- (3) LOOPNE NEXT

试说明在这三种情况下,程序段执行完后,AX,BX,CX,DX 寄存器的内容分别是什么。

```
START: MOV AX, 01H
        MOV BX, 02H
        MOV DX, 03H
        MOV CX, 04H
NEXT:   INC AX
        ADD BX, AX
        SHR DX, 1
        (    )
```

18. 中断向量表的作用是什么?它放在内存的什么区域内?中断向量表中的什么地址用于类型 3 的中断?

19. 设类型 2 的中断服务程序的起始地址为 0485:0016H,它在中断向量表中如何存放?

20. 若中断向量表中地址为 0040H 中存放 240BH,0042H 单元里存放的是 D169H,试问:

- (1) 这些单元对应的中断类型是什么?
- (2) 该中断服务程序的起始地址是什么?

21. 简要说明 8086 响应类型 0~4 中断的条件是什么?

22. 设 8086 CPU 的时钟频率为 5MHz,请编写延时 5ms 的子程序。

第四章 汇编语言程序设计

汇编语言(Assembly Language)是利用指令的助记符、符号地址、标号来编写的语言,它是机器语言的符号表示,是较低级的语言。利用汇编语言编写的程序称为源程序,指令系统中的每条指令都是构成源程序的基本语句。但机器不能识别源程序,要通过汇编程序翻译成二进制代码的浮动目标程序,然后由连接程序将目标文件与库文件相连,最后得到可执行的程序,才可在机器上直接运行。

汇编语言是面向机器的语言,是和机器的硬件密切相关的,不同 CPU 的机器有不同的汇编语言。与高级语言相比,用高级语言编写的源程序可读性好,编程简单,用户容易掌握,一般非计算机技术人员均采用它编程。但是汇编语言充分利用了机器的硬件结构特点,为用户提供了直接控制目标代码的手段,可以对输入/输出设备(如 CRT、打印机、硬磁盘等)进行操作,实时性能好,而且用汇编语言编写的程序效率高,节省内存,运行速度快,可以直接与操作系统进行接口。所以计算机高级技术人员大量使用汇编语言来编写计算机系统程序,实时通信程序和实时控制程序等。

8086 系统中常用的汇编程序是 MASM6. X 版本,因此除了指令系统外,还要了解 MASM 中的标号、表达式、伪指令,必须按 MASM 中规定的格式来编写源程序,才能正确汇编成可执行程序。下面先看一个完整的用汇编语言编写程序的格式。

例 4-1 在屏幕上显示并打印字符串“This is a sample program.”

```
DATA    SEGMENT                                ;数据段
        DA1    DB  'This is a sample program.'
        DB  0DH,0AH,'$'

DATA    ENDS
STACK   SEGMENT
        ST1    DB  100 DUB(?)

STACK   ENDS
CODE    SEGMENT                                ;代码段
MAIN    PROC  FAR
        ASSUME CS:CODE, DS:DATA, SS:STACK
START:  MOV     AX, STACK                        ;送堆栈段地址
        MOV     SS, AX
        POSH    DS                             ;返回 DOS
        MOV     AX, 0
        POSH    AX
        MOV     AX, DATA                      ;送数据段地址
        MOV     DS, AX
        MOV     AH, 9                          ;DOS 9 号功能调用,显示字符串
        MOV     DX, OFFSET DA1
        INT     21H
```



```

                RET
MAIN            ENDP
CODE            ENDS
                END      START

```

从上面的例子可以看到,一个完整的汇编程序编写格式要包括以下几部分:段定义、段分配、设置段地址、返回 DOS 语句及程序结束,需要时加上过程调用。

(1) 汇编语言编写的原程序是分段的,要定义代码段、数据段、堆栈段,每段由段定义伪指令 SEGMENT 开始,ENDS 结束,并赋予段名区分不同段。段定义的基本格式如下:

```

段名    SEGMENT
        .....
段名    ENDS

```

原程序中至少有一个代码段,此时数据可放在代码段中;堆栈段如果不定义,由计算机自动分配。段名可以自己定义,用字母和数字组成。计算机识别不同的段由段分配伪指令 ASSUME 来完成。段分配的格式为:

```
ASSUME    CS:段名, DS:段名, SS:段名, ES:段名
```

(2) 本例中把程序作为过程,由 DOS 调用过程,过程调用由伪指令 PROC ... ENDP 实现,过程的调用格式如下(也可省略):

```

过程名    PROC  FAR(NEAR)          ; FAR 表示远调用,NEAR 表示近调用可缺省
        .....
过程名    ENDP

```

(3) 若程序已经分别定义了数据段、堆栈段和附加段,主程序的开始要设置这些段的地址。代码段的地址不能人为设置,由计算机分配。堆栈段和数据段设置的具体语句为:

```

MOV      AX, STACK                ; 送堆栈段地址
MOV      SS, AX
MOV      AX, DATA                ; 送数据段地址
MOV      DS, AX

```

(4) 程序执行完毕要返回 DOS 操作系统,有两种方式实现。一种是在程序的开始部分编写如下语句:

```

PUSH     DS
MOV      AX, 0
PUSH     AX

```

将 DS 的内容及 0 作为段地址和偏移地址入栈,在程序结束时返回 DOS。以上三句语句必须写在堆栈段设置后面,否则堆栈段的设置使一些指令不起作用了。第二种方法是在程序结束前使用 DOS 功能调用指令,如下所示:

```

MOV      AX, 4C00H
INT      21H

```

(5) 全部源程序用 END 语句结尾,END 后面可以加上程序执行起始的名称 START,汇编程序遇见 END 语句就结束。

4-1 汇编语言程序格式

汇编语言源程序用语句书写,MASM 中可使用的语句分成两类,它们是指令性语句和

伪指令语句。

一、指令性语句

指令性语句与机器指令相对应,汇编程序可将它翻译成目标代码(机器指令代码)。语句格式为:

标号: 指令助记符 操作数,操作数 ;注释

标号表示本指令语句的符号地址,标号后面必须紧跟冒号“:”。标号可使用的字符为字母(A~Z,a~z)、数字(0~9)或某些特殊字符(@、_、?)等。第一个字符必须为字母或某些特殊字符,最大有效字符长度为 31 个字符。标号可以省略,它经常作为转移指令的一个操作数,用以表示转移的地址。

指令助记符是该语句的指令名称的代表符号,它指出指令的操作类型,汇编程序将其翻译成机器指令。它是语句中的关键字,因此不可省略。

操作数表示参加本指令运算的数据,根据指令要求可以有一个或多个操作数,有的指令不需要操作数,多个操作数之间用逗号“,”隔开,操作数与指令助记符之间用空格隔开。操作数可以是常数、变量、标号、寄存器名或表达式。

注释用来说明一条指令或一段程序的功能,它可以省略。注释前必须加上分号“;”,汇编程序对“;”后面的内容不汇编,加注释使程序容易读懂。

二、伪指令语句

伪指令语句没有对应的机器指令,汇编程序汇编源程序时对伪指令进行处理,它可完成数据定义,存储区分配,段定义,段分配,指示程序结束等功能。伪指令语句的格式为:

名字 伪指令指示符 操作数,操作数 ;注释

名字是给伪指令取的名称,它用符号地址表示,名字后不允许带冒号“:”,名字可以省略。伪指令中的名字通常是变量名、段名、过程名、符号名等。

伪指令指示符是汇编程序 MASM 规定的符号,常用的有变量定义语句(DB、DW),符号定义语句(EQU、=),段定义语句(SEGMENT...ENDS),段分配语句(ASSUME),结构定义语句(STURC...ENDS),过程定义语句(PROC...ENDP)等类型,后面将详细说明。

操作数是由伪指令具体要求的,有的伪指令不允许带操作数,有的伪指令要求带多个操作数,多个操作数之间必须用逗号分开。操作数可以是常数、变量、字符串、表达式等。

注释的功能和使用与指令性语句相同。

三、数据项

汇编语言中使用的操作数,可以是常数、寄存器、存储器、变量、标号或表达式,其中常数、变量和标号是三种基本数据项。

1. 常数

常数必须是固定值,没有属性,是确定的数据。用二进制表示时,以字母“B”结尾,例 00110100B。用八进制表示时,以字母“Q”或“O”结尾,例 1037O、2370Q。用十进制表示时,以字母“D”结尾或省略,也可用科学表示法,例 1234D、5678、2.735E-2。用十六进制表示时,以字母“H”结尾,字母“A~E”开头时,前面必须加 0,例 56H、0A7F2H。

用字符串表示时,将可打印的 ASCII 码字符串用单引号‘ ’括起来,机内存放的是各字符的 ASCII 码。例‘ABC’,‘This is a sample’,‘12 * 45 \$@’。

2. 变量

变量通常指存放在存储单元中的值,在程序运行中是可以修改的。所有的变量都具有三个属性。

(1) 段值(SEGMENT):指变量所在段的段基址。

(2) 段内偏移地址(OFFSET):指变量地址与所在段首地址之间的地址偏移字节数。

(3) 类型(TYPE):变量的类型属性指变量中每个元素所包含的字节数,类型有:字节变量(BYTE)、字变量(WORD)及双字变量(DWORD)等。

3. 标号

标号是可执行指令语句的地址的符号表示,它可作为转移指令的目标操作数,以确定程序转向的目标地址,它具有三个属性。

(1) 段值(SEGMENT):标号所在段的段基址。

(2) 段内偏移地址(OFFSET):标号地址与所在段的段首址之间的偏移地址字节数。

(3) 类型(TYPE):标号的类型属性指在转移指令中标号可转移的距离,也称距离属性。类型 NEAR,表示此标号为近标号,只能实现本代码段内转移或调用,类型 FAR,表示此标号为远标号,可以作为其它代码段中的目标地址,实现段间转移或调用。若标号后面紧跟冒号,表示隐含此标号距离属性为 NEAR,也可用伪指令将此属性改为 FAR。

4-2 MASM 中的表达式

表达式由运算对象及运算符组成,在汇编时由汇编程序对它进行运算,运算结果作为一个语句中的操作数去使用。运算对象可以是常数、变量或标号,运算结果可以是一个常数,也可以是一个存储器的地址,在此地址中存放了数据(称为变量)或指令(称为标号)。

MASM 中使用了 6 类运算符,即:

- 算术运算符(Arithmetic Operators)
- 逻辑运算符(Logical Operators)
- 关系运算符(Relational Operators)
- 数值返回运算符(Value-Returning Operators)
- 修改属性运算符(Modifying attribute Operators)
- 其它运算符(Other Operators)

表 4-1 给出了 MASM 中可采用的运算符号。

表 4-1 MASM 的表达式中运算符

类 型	符 号	名 称	运算结果
算术运算符	+ - * / MOD SHL SHR	加法 减法 乘法 除法 模除 左移 右移	和 差 乘积 商 余数 左移后二进制数 右移后二进制数
逻辑运算符	AND OR XOR NOT	与运算 或运算 异或运算 非运算	逻辑与结果 逻辑或结果 逻辑异或结果 逻辑非结果
关系运算符	EQ NE LT LE GT GE	相等 不等 小于 小于等于 大于 大于等于	结果为真输出全“1” 结果为假输出全“0”
数值返回	OFFSET SEG TYPE LENGTH SIZE	返回偏移地址 返回段基址 返回元素字节数 返回变量单元数 返回变量总字节数	偏移地址 段基址 字节数 单元数 总字节数
修改属性	段寄存器名 PTR THIS HIGH LOW SHORT	段前缀 修改类型属性 指定类型/距离属性 分离高字节 分离低字节 短转移说明	修改段 修改后类型 指定后类型 高字节 低字节 -128~127 字节间转移
其它运算符	() [] . < > MASK WIDTH	圆括号 方括号 点运算符 尖括号 记录位图 记录宽度	改变运算符优先级 下标或间接寻址 连接结构与变量 修改变量 位图形 记录/字段位数

一、算术运算符

算术运算符包括+(加)、-(减)、*(乘)、/(除,只取除法运算结果之商)、MOD(模,只取除法运算结果之余数)、SHL(左移,左移1位相当于乘2)、SHR(右移,右移1位相当于除2)7种。

所有的算术运算符均可以对数据进行运算,运算对象与运算结果都是整数。若对地址运算,通常是在标号上加/减某一个数字量,例 DA1+2、K2-3 各表示一个存储单元的地址,对地址乘是没有意义的。

例 4-2 数组 ARRAY 定义如下:

```

ARRAY    DB  1,2,3,4,5,6,7,8
TRY      DB  20
          MOV   AX, 30 * 5
          MOV   CX, (TRY-ARRAY)    ;数组长度存入 CX

```

汇编时,计算表达式形成指令为:

```

          MOV   AX, 150
          MOV   CX, 8

```

例 4-3 源程序指令格式如下:

```

DA       EQU   300
          MOV   AX, DA-80
          MOV   BX, DA MOD 100
          MOV   CX, DA/100
          MOV   DH, 01100100B SHR 2

```

汇编时,计算表达式形成指令为

```

DA       EQU   300
          MOV   AX, 220
          MOV   BX, 0
          MOV   CX, 3
          MOV   DH, 19H

```

二、逻辑运算符

逻辑运算符包括 AND(与)、OR(或)、NOT(非)、XOR(异或)4 种,逻辑运算符是按位运算的,只能对常数进行运算,得到结果也是常数。

例 4-4

```

MOV   AL, NOT 0FFH
MOV   BL, 8CH AND 73H
MOV   AH, 8CH OR 73H
MOV   CH, 8CH XOR 73H

```

汇编时,计算表达式形成指令为

```

MOV   AL, 0
MOV   BL, 0
MOV   AH, 0FFH
MOV   CH, 0FFH

```

逻辑运算符与 8086 指令系统中的指令助记符 AND、OR、NOT、XOR 符号完全相同,但二者是不会混淆的。作为 MASM 的运算符是在汇编过程中进行计算的,而指令助记符是在程序执行时进行运算的。

例 4-5

```

IN     AL, PORT          ;PORT 为输入端口号
AND    DX, PORT AND 0FEH
OUT    DX, AX            ;DX 为输出端口号

```

后一个 AND(运算符),汇编时计算表达式,得到一个端口号,如原输入端口号 PORT

为 80H,则表达式 PORT AND OFEH 的值为 80H,若原输入端口号 PORT 为 81H,则计算表达式值也得到 80H。前一个 AND(指令助记符),在运行程序时将 DX 内容与计算出的表达式值相‘与’,结果送到 DX 中,DX 为输出端口号。

三、关系运算符

关系运算符包括 EQ(相等)、NE(不等)、LT(小于)、GT(大于)、LE(小于或等于)、GE(大于或等于)6 种。

关系运算符的两个操作数必须是数据,或是同一段内的两个存储单元的地址。进行关系运算的比较操作后,结果是一个数值,若结果为真,输出全是 1,即 0FFH 或 0FFFFH。若结果为假,输出全是 0。关系运算符一般与逻辑运算符组合起来使用。

例 4-6 MOV AX,10H GT 16
 ADD BL,6 EQ 0110B
 MOV CX,((PORT LT 5) AND 100) OR ((PORT GE 5) AND 200)

汇编时形成指令为

```
MOV    AX,0
ADD    BL,0FFH
MOV    CX,100                                ;当 port<5 时
```

本例最后一行表示当端口地址 PORT 小于 5 时,汇编结果相当于指令

```
MOV    CX,100
```

若 PORT 大于等于 5 时,汇编结果相当于指令

```
MOV    CX,200
```

四、数值返回运算符

数值返回运算符也经常称作分析运算符(Analytic operators)。

数值返回运算符包括 OFFSET、SEG、TYPE、LENGTH、SIZE 5 种,它们加在变量或标号前,返回运算对象的某个参数值,例如偏移地址值、段地址值、类型属性、变量包含的单元数等。

1. OFFSET

格式:OFFSET 变量或标号

OFFSET 返回标号或变量的偏移地址值,为程序设计中常用的运算符。

例 4-7 若 DA1 为数据段中一个变量名:

```
MOV    BX, OFFSET DA1
```

汇编程序将变量 DA1 的偏移地址送到 BX 中,相当于指令

```
LEA    BX, DA1
```

2. SEG

格式: SEG 变量或标号

SEG 用来取变量或标号的段基值。

例 4-8 MOV AX, SEG M1
 MOV DS, AX

M1 是段名为 DATA 的数据段中的一个变量名, 假如 DATA 段从 0500H 开始, 上述指令把 0500H 作为立即数在连接时插入指令, 汇编成 MOV AX, 0500H (因为段地址是由连接程序分配的), 执行程序时, AX 的内容变成 0500H, 并存放到 DS 中。

3. TYPE

格式: TYPE 变量或标号

TYPE 加在变量前, 返回变量的类型属性, TYPE 加在标号前, 返回标号的距离属性。

表 4-2 给出了 TYPE 运算符的返回数值对照表。

例 4-9 A1 DB 20H, 30H
 A2 DW 0438H
 A3 DD ?
L1: MOV AH, TYPE A1
 MOV BH, TYPE A2
 ADD AL, TYPE A3
 MOV BL, TYPE L1

汇编时形成指令

MOV AH, 1
MOV BH, 2
ADD AL, 4
MOV BL, 0FFH

表 4-2 TYPE 运算符返回值

	类 型	返回值
变 量	DB	1
	DW	2
	DD	4
	DQ	8
标 号	NEAR	-1 [FFH]
	FAR	-2 [FEH]

4. LENGTH

格式: LENGTH 变量

当变量中使用 DUP 时, LENGTH 返回此变量所包含的单元数, 对其它变量则返回 1。

例 4-10

M1 DW 100 DUP (?)
M2 DW 1, 2, 3
M3 DB 'A B C D'
 MOV CX, LENGTH M1
 MOV BL, LENGTH M2
 MOV AL, LENGTH M3

汇编时形成指令

MOV CX, 100 ; 返回此变量包含 100 个字单元
MOV BL, 1
MOV AL, 1

5. SIZE

格式: SIZE 变量

SIZE 运算符加在变量前, 返回该变量包含的总字节数。

$SIZE = LENGTH * TYPE$

例 4-11 对例 4-10 定义的 M1, M2, M3

```
MOV    CX, SIZE M1
MOV    BL, SIZE M2
MOV    AL, SIZE M3
```

汇编时形成指令

```
MOV    CX, 200           ;返回此变量包含 200 个字节单元
MOV    BL, 2
MOV    AL, 1
```

五、修改属性运算符

修改属性运算符也经常称作综合运算符(Synthetic operators)。

修改属性运算符有段操作符、PTR、THIS、HIGH、LOW、SHORT 6 种, 可以在程序运行过程中, 通过修改属性运算符来修改变量或标号的属性, 包括段属性、偏移地址属性、类型属性等。

1. 段操作符

格式: 段前缀: 变量或地址表达式

段前缀有段寄存器 CS、DS、ES、SS 后跟冒号“:”, 用来表示某个变量或地址被修改到哪个段寄存器提供的段基址中。

例 4-12 MOV AX, ES:[BX] ;段超越到 ES 段

2. PTR

格式: 类型/距离 PTR 变量或标号

其功能是将 PTR 左边的类型属性赋给右边的变量或标号。PTR 本身并不分配存储单元, 仅给已分配的存储单元赋予新的属性, 以保证运算时操作数类型的匹配, 常与类型 BYTE、WORD、NEAR、FAR 等连用。

例 4-13 N1 DB 15H, 36H
 N2 DW 1122H, 3344H

```
LO:  MOV    AX, WORD PTR N1      ;使 N1 类型转换成字与 AX 类型匹配
      MOV    BL, BYTE PTR N2     ;使 N2 类型转换成字节与 BL 类型匹配
```

例 4-14 MOV [BX], 10H

此指令用于将立即数送入 BX 间址指定的存储单元中, 可以通过 PTR 指明是存入字节单元还是字单元。


```
MOV    BYTE PTR [BX], 10H    [BX]←10H
MOV    WORD PTR [BX], 10H    [BX], [BX+1]←0010H
```

也可用 PTR 来改变距离属性,

```
JMP    FAR PTR LO
```

在 JMP 语句中将标号 LO 改为 FAR,使 JMP 指令安排在其它代码段中也可以使用,以实现段间转移。

3. THIS

格式:变量/标号 EQU THIS 类型/距离

THIS 的功能是将 EQU THIS 右边的类型/距离属性,赋给左边的变量/标号,该变量或标号的段地址和偏移地址与下一个存储单元的地址相同。

例 4-15 FIRST EQU THIS BYTE
 TABLE DW 200 DUP(?)

FIRST 的偏移地址值与 TABLE 的偏移地址值相同,区别在于 FIRST 变量为字节类型, TABLE 为字类型。

例 4-16

```
SP      EQU THIS FAR
MOV     AX, 100
```

此时 MOV AX,100 前有标号 SP,并赋予 FAR 属性,允许其它段的 JMP 指令跳到本段 SP 标号地址上。

4. SHORT

格式:SHORT 标号

SHORT 用来说明转移类指令中转向地址的属性,指出转向的目标地址与本指令之间的距离在 -128~+127 之间,即限制在短转移范围内。

例 4-17

```
L1:     JMP     SHORT L2
        :
L2:     MOV     AX, 0
```

5. HIGH 和 LOW

格式:HIGH/LOW 变量或标号

HIGH 和 LOW 称为字节分离运算符,对一个数或地址表达式,HIGH 从中分离出高位字节,LOW 分离出低位字节。

例 4-18

```
K1      EQU     0ABCDH
K2      EQU     1234H
MOV     AH, HIGH K1
MOV     BL, LOW K2
```

汇编时形成指令

```
MOV    AH, 0ABH
MOV    BL, 34H
```

六、其它运算符

其它运算符有(), [], < >, ·, MASK 和 WIDTH 等 6 种。

1. 圆括号()

圆括号用来改变运算符的优先级别,()中的运算符具有最高优先权。

2. 方括号[]

方括号主要用来表示地址表达式或多重变量的下标值。

(1) 用[]表示地址表达式

例 4-19

```
M1      DB  10H, 20H, 30H, 40H
M2      DW  1234H, 5678H, 9ABCH
M3      DW  5    DUP(?)

MOV     BX, OFFSET M1      ;变量 M1 的偏移地址→BX
MOV     CL, [BX]           ;M1 变量中第一个单元的值 10H→CL
MOV     BX, OFFSET M2
MOV     DX, [BX+2]         ;将 M2 变量中第三个单元的值 5678H→DX
```

(2) 用[]来表示多重变量的下标值。

例 4-20 M1, M2, M3 同例 4-19 的定义。

```
MOV     AL, M1[3]          ;M1 变量中第三个元素 40H→AL
MOV     CX, M2[0]          ;M2 变量中第 0 个元素 1234H→CX
MOV     M3[1], CX          ;1234H→M3 的第二个单元
```

3. 尖括号< >,及圆点·

< >运算符在结构中专用,表示结构中的变量在预置结构付本时是否修改,修改成什么数值。

·运算符在结构中专用,表示结构付本名与变量名连接在一起,作为预置的结构付本中的各个变量。< >和·的具体说明在结构中解释。

4. MASK 和 WIDTH

MASK 和 WIDTH 运算符在记录中专用。

WIDTH 为记录名/字段名,运算后返回数值表示指定记录或字段的位的长度。

MASK 为字段名,返回数值为 8 位/16 位二进制数,对应指定字段的各位置“1”,其它位置“0”。

七、优先级

表达式是常数,变量,标号和运算符的组合,在计算表达式值时,应按优先级高低进行计算,同时遵循同级运算符从左到右的原则计算,圆括号()可改变优先级次序,表 4-3 给出了运算符的优先级别。实际上表达式的值由汇编程序计算,程序编写时要正确掌握次序,以免程序出错。表 4-3 中,优先级 1 为最高级,优先级 10 为最低级。

表 4-3 运算符优先级次序

优先级	运 算 符
1	(), [], < >, ·, LENGTH, WIDTH, SIZE, MASK
2	PTR, OFFSET, SEG, TYPE, THIS, CS:, DS:, ES:, SS:
3	HIGH, LOW
4	*, /, MOD, SHL, SHR
5	+, -
6	EQ, NE, LT, LE, GT, GE
7	NOT
8	AND
9	OR, XOR
10	SHORT

4-3 伪指令语句

伪指令语句没有对应的机器代码,并不像指令语句那样由 CPU 来执行,它是由 MASM 汇编程序对源程序汇编期间进行处理的。主要完成变量定义,存储器分配,指示程序开始和结束,段定义,段分配等。伪指令语句有如下几种类型:

- 数据定义语句 DB, DW, DD 等
- 标号赋值语句 EQU, =
- 段定义语句 SEGMENT...ENDS
- 段分配语句 ASSUME
- 过程定义语句 PROC...ENDP
- 程序开始结束语句 ORG, END

这些语句经常使用,没有这些伪指令,汇编程序不能得到正确的汇编结果。此外还有:

- 群定义语句 GROUP
- 结构定义语句 STRUC...ENDS
- 记录定义语句 RECODE

这一类伪指令帮助用户灵活简捷地使用汇编语言编程。高档微机增加的伪指令在后面

加以介绍。

一、数据定义语句

格式 1: 变量名 助记符 操作数, 操作数... ; 注释

格式 2: 变量名 助记符 n DUP(操作数, 操作数...) ; 注释

功能: 将操作数存入变量名指定的存储单元中, 或者只分配存储空间不存入数据。

变量名——它用符号表示, 可以省略, 作用与指令语句中的标号相同, 但后面不跟冒号“:”。汇编程序汇编时将此变量的助记符后的第一个字节的偏移地址作为它的符号地址。

助记符——所用伪指令助记符主要有:

DB: 用来定义字节, 表示每个操作数占用一个字节。

DW: 用来定义字, 表示每个操作数占用一个字。

DD: 用来定义双字, 表示每个操作数占用两个字。

DQ: 用来定义四个字, 表示每个操作数占用四个字。

DT: 用来定义十个字节, 表示每个操作数占用十个字节。

操作数——操作数可以是常数, 字符串, 变量, 标号, 表达式等, 多个操作数之间必须用逗号分开。

在格式 2 中, 用 n DUP() 表示时, n 必须是正整数, 表示括号中的操作数的重复次数, DUP 后面必须带括号。

注释——说明伪指令的功能, 可以省略, 注释前必须带分号“;”, 汇编程序对此不汇编。

例 4-21 操作数是常数或表达式

```
DA1    DB  10H, 52H          ; 变量 DA1 中装入 10H, 52H
DA2    DW  1122H, 34H        ; 变量 DA2 中装入 22H, 11H, 34H, 00
DA3    DD  5 * 20H, 0FFEEH   ; 变量 DA3 中装入 A0H, 00H, EEH, FFH
```

汇编后数据在存储器中存放格式如图 4-1 所示。

例 4-22 操作数是字符串

```
FIRST   DB  'HELLO'          ; 将字符串 'HELLO' 的 ASCII 码装入 FIRST 单元开始的存储单元
SECOND  DW  'OK'              ; 将字符串 'OK' 的 ASCII 码装入 SECOND 开始的存储单元
```

注意: 用 DW 定义字符串时, 只允许包括两个字符, 多于两个字符时, 只能用 DB 定义, 图 4-2(a) 指出了汇编后字符串在存储器中存放的格式, 图 4-2(b), (c) 指出了 DB 和 DW 定义 'OK' 时不同的存放格式。

例 4-23 操作数用? 定义不确定值的变量, 用作保留存储空间, 以便存放运算结果。

```
M1      DB  ?                ; 定义变量 M1 为不确定字节, 保留 1 字节空间
M2      DW  0D55H, ?          ; 定义变量 M2 第二个字为不确定, 保留两个字节空间
```

存储器中存放格式如图 4-3 所示。

例 4-24 操作数用 DUP 来定义重复变量

ONE	DB 5 DUP (0)	;重复 5 个 0 存入 ONE 起始的存储单元
TWO	DW 10 DUP (?)	;重复 10 次,保留 10 个字的存储单元空间
THREE	DB 4 DUP (1,2 DUP (20H))	;DUP 嵌套

图 4-4 指出了例 4-24 在存储器中存放格式。

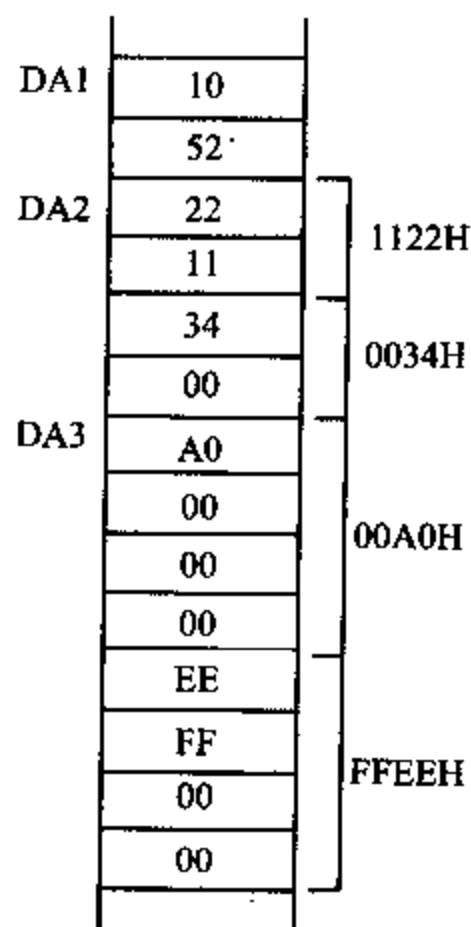


图 4-1 例 4-21 数据存储格式

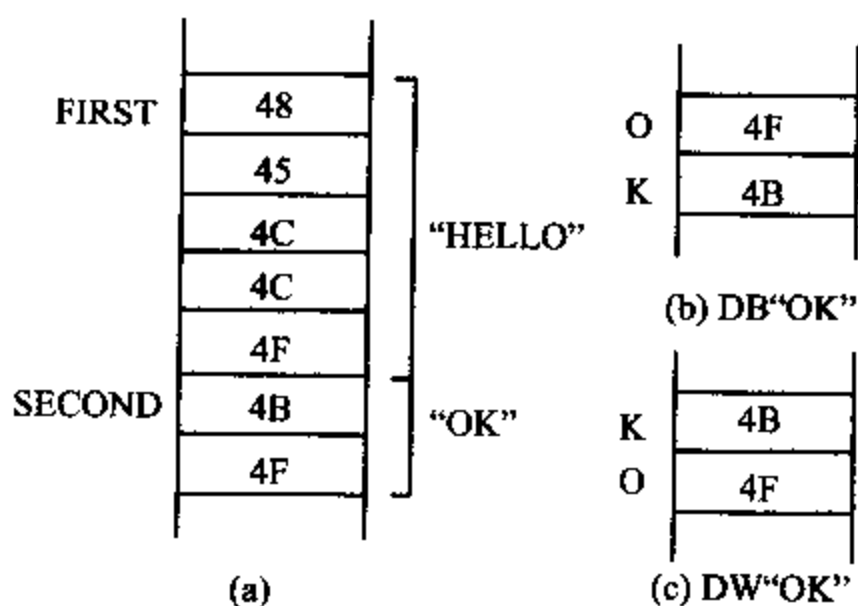


图 4-2 例 4-22 字符串存储格式

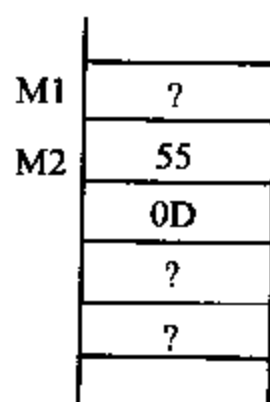


图 4-3 例 4-23 不确定值存储格式

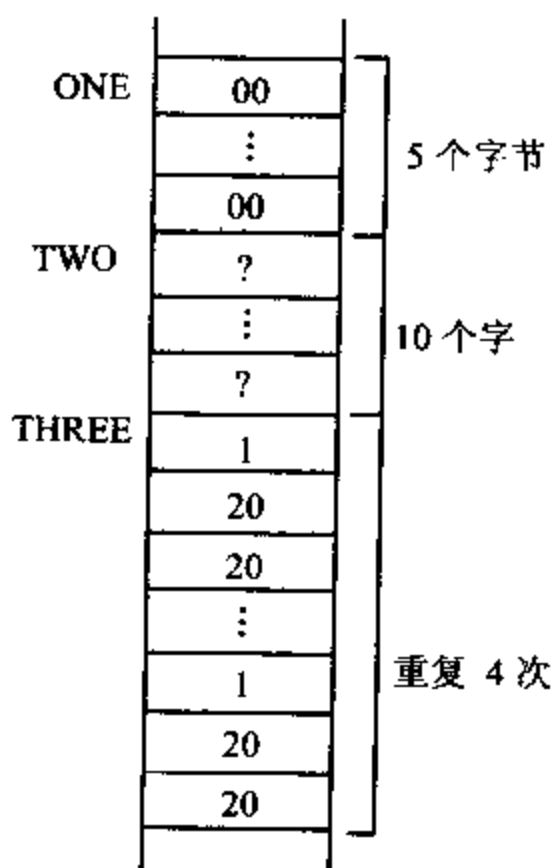


图 4-4 例 4-24 的存储格式

用伪指令 DW 和 DD 可以将变量或标号的偏移地址存入存储器,当用 DD 来定义时,原变量或标号的偏移地址存入低位字中,原变量或标号的段基址存入高位字中。

例 4-25 用地址表达式定义变量。

如果当前已有变量 PAR1, 标号 ADR2 和 ADR3, 可以用它们定义新变量如下:

```
ONE    DW  PAR1      ;将变量 PAR1 的偏移地址赋给字变量 ONE
TWO    DW  ADR2      ;将标号 ADR2 的偏移地址赋给字变量 TWO
THREE  DD  ADR3      ;将标号 ADR3 的偏移地址和段基址赋给双字变量 THREE
```

在例 4-25 中, 假如 PAR1, ADR2, ADR3 在同一代码段 CS=2000H 中, 其偏移地址分别为 0100H, 0200H, 0300H, 则汇编后存储器中变量存放格式如图 4-5 所示。

例 4-26 变量的类型属性可以通过属性操作符 PTR 来指定。

```
OPE1  DB  1,2
OPE2  DW  2233H,5566H
      MOV  AX,OPE1+1
      MOV  AL,OPE2
```

汇编程序在汇编这一段程序时, 会发现指令中两个操作数的类型属性不匹配, 因而汇编程序提示出错, 若修改成如下指令, 汇编时就不会出错。

```
MOV  AX, WORD PTR OPE1+1 ;AX=3302H
MOV  AL, BYTE PTR OPE2   ;AL=33H
```

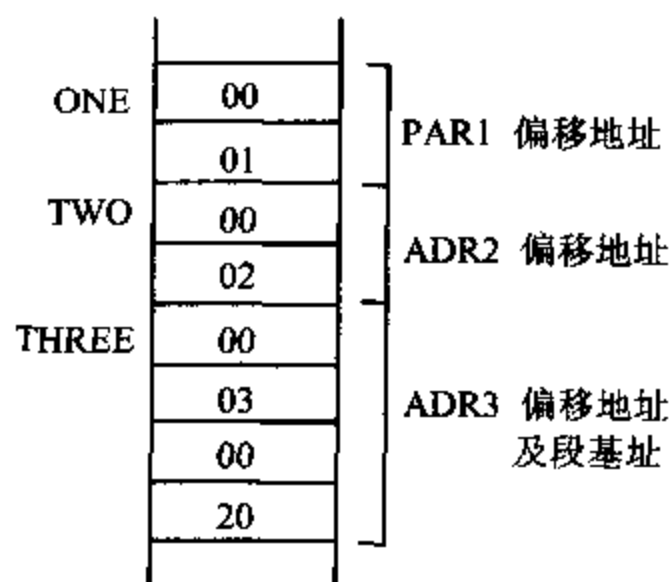


图 4-5 例 4-25 的存储格式

二、表达式赋值语句

表达式赋值语句有两种, 赋值语句 EQU 和等号语句 =, 它们均不占用内存。

1. 赋值语句 EQU

格式: 符号名 EQU 表达式

功能: 用来给变量, 标号, 常数, 指令, 表达式等定义一个符号名, 程序中用到 EQU 左边的变量、标号时可用右边的常数值或表达式代替, 但一经定义在同一个程序模块中不能重新定义。

例 4-27

```
COUNT EQU 100      ;常数值赋给符号名 COUNT
DATA   EQU COUNT+2 ;表达式值赋给符号名 DATA
A1     EQU [BX+SI]  ;变址寻址存储单元内容赋给符号名 A1
B1     EQU OFFSET A1;偏移地址值赋给符号名 B1
C1     EQU ADD      ;加法指令赋给符号名 C1
```

在 EQU 语句右边表达式中有变量或标号的表达式, 必须先给他们定义, 如例 4-27 中 DATA EQU COUNT+2 必须先定义 COUNT, 否则汇编程序将指示出错。

PURGE 语句可以解除对某一个标号的赋值,使它在后面可以重新定义。

```
PURGE C1 ;C1 不再代替 ADD
```

2. 等号语句=

等号语句“=”与 EQU 语句具有相同功能,区别仅在于 EQU 中左边的标号不允许重新定义,而用“=”定义的语句允许重复定义。

例 4-28

```
COUNT=100
COUNT=COUNT+10
A1=BX+SI
      MOV AX, [A1] ;[BX+SI]单元中内容→AX
B1=ADD
A1=BX
      MOV CX, [A1] ;[BX]单元中内容→CX
```

例 4-28 中 A1 第一次使用时,A1 为[BX+SI]单元中的内容,第二次使用时,A1 已修改为[BX]单元中的内容。

三、段定义语句

存储器的物理地址由段基址和偏移地址组合而成,任何一个逻辑段,无论是代码段,数据段,堆栈段,附加段都必须进行段定义,以便连接程序把不同段和模块连成一个可执行程序。此外还必须明确段和段寄存器之间的关系,这可使用段分配语句来完成。

1. 段定义语句 SEGMENT...ENDS

格式:段名 SEGMENT 定位类型 组合类型 ‘分类名’

• 逻辑段内容

段名 ENDS

功能:将一个逻辑段定义成一个整体。

段名——是逻辑段的标识符,不可省略,它确定了逻辑段在存储器中的地址,SEGMENT 和 ENDS 前的段名必须相同。

SEGMENT...ENDS——是段定义的伪指令助记符,任何一个逻辑段必须以 SEGMENT 开始,ENDS 结束,不可省略,并且必须成对出现,两者之间是本逻辑段的内容。SEGMENT 后面可以带有三个参数,定位类型,组合类型,‘分类名’,三个参数必须按格式中规定的次序排列,分类名必须用单引号‘ ’括起来。三个参数用来增加类型及属性说明,一般可以省略,如果需要用连接程序把本程序与其它程序相连时,需要用到这些参数。

(1) 定位类型(Align Type)

定位类型参数是对该段起始地址定位。通常段名确定了该段的首地址,整个逻辑段存放在首地址开始的一片连续存储单元中。一般情况下各个逻辑段的首地址在‘节’的整数边

界上(MASM把1M字节的存储空间从0开始,每16个存储单元叫一节),即每个逻辑段的起始地址是16的整数倍(末4位为0)。实际使用时,可由定位类型参数来定位各逻辑段的首地址。定位类型参数主要有下面4种:

① PARA——指定定位段的起始地址必须在节的整数边界,当定位类型参数缺省时,就当成PARA。

② BYTE——指定该段起始地址定位在存储单元的任何字节地址。

③ WORD——指定该段起始地址定位在字的边界,即段的首地址必须是偶数。

④ PAGE——指定该段起始地址定位在页的边界,即段的首地址必须是256的整数倍。

(2) 组合类型(Combine Type)

组合类型参数主要提出了各个逻辑段之间的组合方式,例各段独立,各段覆盖或顺序组合等。主要参数有以下6种:

① NONE——该段与其它同名段不进行连接,各段独立存在于存储器中,NONE可作为缺省参数。

② PUBLIC——该段与其它模块中的同名段连接时,由低地址到高地址连接起来,组成一个逻辑段,连接次序由连接命令指定,连接时满足定位类型要求。

③ COMMON——该段在连接时与其它模块中的同名段有相同的起始地址,采用覆盖的方式在存储器中存放,连接长度为各分段中最大长度。

④ AT表达式——定位该段的起始地址在表达式所指定的节(16的整数倍)边界上。一般情况下各个逻辑段在存储器中的位置由系统自动分配,当用户要求某个逻辑段在指定节的边界上时,就要用AT参数来实现,AT不能用来指定代码段。

⑤ STACK——指定该段为堆栈段,此参数在堆栈段中不可省略,多个模块只需设置一个堆栈段,各个模块中的堆栈段采用覆盖方式组合,容量为各个模块中所设置的最大堆栈段容量。

⑥ MEMORY——定位该段与其它模块中的同名段有相同的首地址,采用覆盖方式在存储器中组合连接,其功能与COMMON参数类似,区别是第一个带MEMORY参数的逻辑段覆盖在其它同名段的最上层,其它带此参数的同名段按照COMMON方式处理。

(3) ‘分类名’(Class Name)

段定义语句的第三个参数为分类名,必须用单引号‘ ’括起来,分类名可选择不超过40个字符的名称,主要作用是汇编程序连接时将所有分类名相同的逻辑段组成一个段组。

段定义语句中的参数设置,可以增强伪指令语句的功能。段定义语句允许嵌套设置,即一个逻辑段内再设置其它逻辑段,但不允许各个逻辑段相互交叉设置。

2. 段分配语句(ASSUME)

在8086/8088系统中存储器采用分段结构,各段容量 $\leq 64\text{KB}$,用户可以设置多个逻辑段,但只允许4个逻辑段同时有效,段分配语句用来完成将逻辑段分别定义成代码段,数据段,堆栈段及附加段。

格式: ASSUME CS:段名,DS:段名,SS:段名,ES:段名

功能: 定义 4 个逻辑段,指明段和段寄存器的关系。

ASSUME —— 为伪指令助记符,放在代码段的开始,不可省略。提供给汇编程序,说明当前代码段,数据段,堆栈段和附加段 4 个段如何定义。

ASSUME 后面有各段寄存器的名 CS:,DS:,SS:,ES:,用来存放当前有效的逻辑段的段基址,后面紧跟冒号“:”及段名。各段寄存器之间由逗号“,”分开,段名必须是用段定义语句 SEGMENT...ENDS 定义过的名字。可以用 ASSUME NOTHING 取消前面由 ASSUME 所指定的段寄存器。

例如 ASSUME ES:NOTHING

4 个逻辑段不一定全部要定义,通常代码段和数据段是必须的,附加段可以省略。但当代码段中使用了串指令,必须设置附加段作目标串基址用,附加段也可用来存放数据,增大数据段容量。

由于 ASSUME 伪指令只指定某个段分配给哪个段寄存器,并将代码段的段基址自动装入 CS 寄存器中,而不能自动把其它段基址装入相应的段寄存器中,所以在代码段的开始要有一段初始化程序完成这一工作。对堆栈段来说除了将段基址送入 SS 寄存器外,还可以将栈顶偏移地址置入堆栈指示器 SP 中。

例 4-29 两个 16 位无符号二进制数相乘。

```
DATA    SEGMENT                ;数据段
        D1      DW 1234H
        D2      DW 5678H
        P1      DD ?
        P2      DD ?
DATA    ENDS
STACK   SEGMENT    STACK 'STACK'
        DW      100 DUP(?)
STACK   ENDS
CODE    SEGMENT
        ASSUME   CS:CODE, DS:DATA, SS:STACK
MAIN    PROC     FAR
START:  MOV      AX, STACK      ;初始化 SS, SP
        MOV      SS, AX
        PUSH     DS
        SUB      AX, AX        ;返回 DOS 用
        PUSH     AX
        MOV      AX, DATA     ;初始化 DS
        MOV      DS, AX
L1:     MOV      AX, D1          ;D1 * D2, 积的高位在 DX, 低位在 AX
        MUL      D2
        MOV      BX, OFFSET P1 ;积→P2 和 P1 所指向的存储单元
```

```

                MOV     [BX], AX
                MOV     [BX+2], DX
                RET
MAIN          ENDP
CCODE        ENDS
                END     START

```

四、过程定义语句

过程也称作子程序。在主程序中,经常要用到一些程序段,程序段的功能和结构相同,仅有一些变量赋值不同,此时可以将这些程序段独立编写,用过程定义语句进行定义,然后在主程序中对它进行过程调用。这样既节省了内存空间,也便于进行模块化程序设计,使编程清晰,使用灵活。

格式:过程名 PROC 属性
 ;过程内容
 RET N

过程名 ENDP

功能:定义一个过程,主程序可以用 CALL 指令调用它。

过程名——是给所定义的过程取的名字,不可缺省。它是主程序调用(CALL 指令)的目标操作数,即子程序入口的符号地址。像标号一样过程名具有三种属性:

段属性:为该过程所在段的段基址。

偏移地址属性:指该过程第一个字节与段首址之间距离字节。

距离属性:为 NEAR 或 FAR。格式中的属性就指距离属性,定义 NEAR 允许过程在段内调用,定义 FAR 允许过程在段间调用,NEAR 为缺省使用。

PROC...ENDP ——过程定义伪指令助记符,成对出现,不可缺省。二者前面有相同的过称名,整个过程内容包括在 PROC...ENDP 之内。

RET N ——为过程内部的返回指令。过程内部至少有一条 RET 指令,它可以在过程的任何位置上,使过程返回到主程序调用它的 CALL 指令之下一条指令。RET 后面跟的 N 为弹出值,可以缺省,N 表示从过程返回以后,堆栈中应有 N 个字节的值作废(以栈顶开始),N 必须为正偶数。过程内部可以有多个 RET,表示此过程具有多个返回出口(在不同条件下,从不同出口返回)。

在汇编语言源程序中,使用 CALL 指令调用过程,过程调用允许嵌套和递归调用。嵌套调用指在一个被调用的过程中,又调用了另一个过程;递归调用是指在一个被调用的过程中,又调用了本身的过程。嵌套与递归的深度由堆栈段的容量决定,因为过程调用时必须将当前的地址压入堆栈保护起来,使调用返回时能返回到正确的指令地址。另外在子程序入口也有许多参数要保护,以免影响主程序原运行状态。

例 4-30 用过程调用的方法,将内存中 N 个 BCD 码相加。

```
DATA SEGMENT
```

```

        ONE      DB  22,33,44,55
        TWO      DB  55,66,77,88
        SUM      DB  20  DUP (?)
DATA    ENDS
STACK   SEGMENT  STACK
        STT      DB  100  DUP (?)
        TOP      EQU LENGTH STT
STACK   ENDS
CODE    SEGMENT
MAIN    PROC     FAR
        ASSUME   CS:CODE, DS:DATA, SS:STACK
START:  MOV      AX, STACK
        MOV      SS, AX
        MOV      SP, TOP
        PUSH     DS
        SUB      AX, AX
        PUSH     AX
        MOV      AX, DATA
        MOV      DS, AX
        MOV      SI, OFFSET ONE      ;SI 指向第一个加数
        MOV      BX, OFFSET TWO
        MOV      DI, OFFSET SUM
        CLD                          ;清方向标志
        CLC                          ;清进位标志
        MOV      CX, 4
LL:     CALL     ABC
        LOOP     LL
        RET
MAIN    ENDP
ABC     PROC     NEAR                ;完成单字节数据加法运算
        LODSB                    [SI]→AL, SI+1→SI
L1:     ADC      AL, [BX]          ;相加
        DAA                          ;十进调整
        STOSB                     ;AL→[DI], DI+1→DI
        INC      BX                ;指针改变
        RET
ABC     ENDP
CODE    ENDS
        END      START

```

例 4-31 远过程定义及调用格式

```

MCODE SEGMENT
MAIN  .
      .
SPD   PROC    FAR           ;过程定义,远过程属性
      . . . . .
      RET
SPD   ENDP
      . . . . .
      CALL    SPD           ;同一段内调用
      . . . . .
MCODE ENDS
NCODE SEGMENT              ;另一段 NCODE
      . . . . .
      CALL    SPD           ;远过程调用
      . . . . .
NCODE ENDS
      END      MAIN

```

例 4-32 过程嵌套调用格式

```

MSUB  PROC    FAR
      . . . . .
      CALL    SBU1
      . . . . .
      RET

SUB1  PROC    NEAR
      . . . . .
      RET
SUB1  ENDP
MSUB  ENDP

```

五、程序开始和结束语句

1. NAME

格式:NAME 程序名

功能:为源程序目标模块赋名字。

NAME ——为伪指令助记符,放在程序开始,在输出汇编语言源程序的列表文件时,将在每一页的开头打印出该程序名。若源程序中省略 NAME 伪指令,汇编程序将源文件名作目标模块的名字。

2. ORG

格式: ORG 表达式

功能: 给汇编程序设置位置指针, 指定下面语句的起始偏移地址。

ORG ——是伪指令助记符, 不可缺省。

表达式——给定的偏移地址值, 表达式的计算结果必须是正整数。一般情况下段定义语句(SEGMENT)指出了段的起点, 偏移地址为 0, 段内各个语句或数据的地址由段地址开始依次类推可确定。当用户要求指定某条指令或数据为某个指定地址时, 可用 ORG 语句来改变, ORG 语句可以放在程序的任何位置。

例 4-33 用 ORG 指定代码段地址。

```
CODE    SEGMENT
        ORG      100H                ;此代码段起始地址偏移 100H
        ASSUME   CS:CODE, DS:DATA,...
START:  MOV      AX, DATA
        :
CODE    ENDS
```

例 4-34 用 ORG 改变数据段地址。

```
DATA    SEGMENT
        ORG      100H
        A1       DB  10H, 20H, 30H    ;A1 偏移地址为 100H
        ORG      200H
        A2       DW  3031H, 3233H    ;A2 偏移地址为 200H
DATA    ENDS
```

3. END

格式: END 标号名

功能: 标记汇编源程序结束。

END ——是伪指令助记符, 不可缺省, 放在源程序的最后一行, 每个模块只有一个 END, 汇编程序到 END 语句停止汇编。

标号名——是该程序中第一条可执行语句的标号名, 可以缺省, 若一个程序包含多个模块, END 后面带的标号为主程序模块中的标号名称。

例 4-35

```
CODE    SEGMENT
START:
        :
SUB1    PROC  NEAR
        :
SUB1    ENDP
CODE    ENDS
        END      START
```

以上几类伪指令语句是经常用到的, 下面所介绍的伪指令语句对于灵活的编程较为

有用。

六、结构定义语句

一般数据语句用 DB, DW, DD, DQ 等就可以定义了, 但有些数据结构比较复杂, 例如学籍管理, 每项是一组变量, 每个组中各变量的长度又不一致(例学生项, 此项内有姓名, 年龄, 性别, 学科, 成绩, 爱好……各个变量长度不同), 汇编程序提供了自定义数据结构的伪指令, 就是结构性数据语句: 结构定义语句及记录定义语句。

结构定义语句的使用要有三个步骤: 结构语句定义, 结构付本预置及结构的使用。

1. 结构定义(STRUC... ENDS)

格式: 结构名 STRUC

(用 DB, DW, DD 等语句定义结构中数据变量)

结构名 ENDS

功能: 结构定义语句可以把各种不同类型的数据放在同一个数据结构中, 便于某些数据处理的需要。

结构名——结构定义的名称, 不可缺省, 在 STRUC 与 ENDS 前的结构名要相同, 不允许超前引用。

STRUC... ENDS——结构定义伪指令助记符, 不可缺省, 必须成对出现。

结构中数据变量用 DB, DW, DD 等变量定义语句进行定义, 定义后的结构未分配存储空间, 结构中各个变量具有各自的局部偏移量, 它是指各变量的第一字节与结构起始地址之间的字节距离。它们的类型属性取决于所采用的变量定义语句。只有在结构被预置后, 才具有确定的存储单元位置。

结构中的变量可分成几种类型。

(1) 简单变量: 变量中只包含一个元素, 当结构被引用时, 可以被修改成不同内容, 但变量类型仍为简单变量。

(2) 多重变量: 变量中包含多个元素, 当结构被引用时, 不允许对它们进行修改, 保持结构定义初值不变。

(3) 字符串变量: 变量为字符串, 当结构被引用时, 允许用同样长度的字符串来进行修改。

(4) 多重结构: 变量本身又是另一个结构。

例 4-36 定义一个数据表格 TAB 的结构

```
TAB    STRUC
        T1    DB 'ABCD'           ;字符串
        T2    DW ?                 ;简单变量
        T3    DW SEG L1            ;简单变量
        T4    DW 2 DUP(0)          ;多重变量
        T5    DW 1122H, 3344H      ;多重变量
TAB    ENDS
```

2. 结构付本预置

结构定义后,在汇编过程中不产生目标代码,也不分配存储空间,必须先预置结构付本,汇编程序给每个结构付本分配存储空间,此时结构中的变量才与存储单元发生关联。结构中允许修改的变量在不同的结构付本中可被修改成不同的数值。

预置结构付本的格式有如下两种。

格式 1:结构付本名 结构名 <元素值,元素值...> ;注释

格式 2:结构付本名 结构名 N DUP (<元素值,元素值...>) ;注释

结构付本名——是给所预置的结构付本起的名字,可以缺省。

结构名——与结构定义时的结构名相同,不可缺省。

尖括号——为专用运算符,表示在预置付本时,结构中的变量改成什么值,也可以不进行修改。

尖括号中的元素值,可以缺省,表示此结构付本对原结构中的所有变量不进行修改。若有多个元素值,它们之间用逗号分开,对不修改的变量用逗号表示,修改的元素值用修改量代替,其后序变量保持不变时,可不再写元素值。

若用第二种格式预置结构付本,N DUP (<元素值,元素值...>)中 N 表示需要预置相同的结构付本的个数。

例 4-37 结构付本中元素值的表示

<>	;结构付本对结构定义中的所有变量不修改。
<20H>	;第一个元素值改为 20H,其它后序变量不变。
< ,352AH>	;第一个变量不变,第二个简单字变量改为 352AH,后序变量保持不变。
<'OK',,,0DH>	;第一个字符串变量改为字符串'OK',第二,第三个变量保持不变, ;第四个简单变量改为 0DH,后序变量保持不变。

例 4-38 对例 4-36 的 TAB 结构预置 4 个结构付本

```
ONE      TAB  <>
TWO      TAB  <'STOP'>
THREE    TAB  < ,0FH,SEG 1,2>
FOUR     TAB  5  DUP(<'EFGH',55H>)
```

经过预置的结构付本中的各个变量,要用运算符“·”将结构付本名和变量名联系起来,即用结构付本名·变量名来表示,这样指出了它在不同的结构付本中的变量元素值。例如在 TWO 结构付本中 T1 变量表示为:TWO·T1。结构付本预置在数据段中完成,经汇编后它们在存储器中都有具体位置。

若已知当前数据段基址为 1000H,结构付本 ONE 的偏移地址为 2000H,则例 4-38 预置的 4 个结构付本在存储器中地址分配如图 4-6 所示。

在图 4-6 中,FOUR TAB 5 DUP (<'EFGH',55>)连续预置了 5 个相同的结构付本,为了区分各个相同结构付本中的变量,在变量后面再加一个下标,用方括号[]括起来,下标值为此结构付本与第一个相同的结构付本之间的字节的距离,例 FOUR·T1[16]表示 FOUR·T1[16]与 FOUR·T1[0]相差 16 个字节。

AX 的内容送到内存单元(12014H),(12015H)中。

例 4-40 将变量 FOUR · T5[0]的值取出送到 BX 寄存器

```
MOV    BX, FOUR · T5[0]
```

汇编程序由变量名 FOUR · T5[0]计算它的物理地址为 1203CH(段基址 DS=1000H, 偏移地址=2030H+0CH),执行上述指令后,将内存单元(1203CH)和(1203DH)中的内容 1122H 送到 BX 寄存器。

对结构付本中的变量也可以通过间接方式来访问,下面两条指令就实现间接传送,完成例 4-40 同样功能。

```
MOV    SI, OFFSET FOUR    ;FOUR 的起始偏移地址 2030H→SI
MOV    BX, [SI] · T5[0]    ;由变量名完成表达式运算,得到物理
                           ;地址 1203CH,并将其内容送到 BX
```

例 4-41 定义描述学生情况的结构 STUDENT,预置 10 个学生的结构付本,并比较其中两个同学的学习成绩。

定义结构 STUDENT

```
STUDENT STRUC
        NUMBER DD 0206078
        NAME   DB '×× ×× ××'
        SEX     DB '女'    ;字符串变量
        AGE     DB 20      ;简单变量
        SPECIA DB '通信'
        POLITI DB '团员'
        CREDIT  DW 100
STUDENT ENDS
```

预置 STUDENT 的结构付本。

```
STUDENT0 STUDENT <0206070, '陆红梅', '女', 20, '通信', '团员', 95>
STUDENT1 STUDNET <0206071, '王 强', '男', 21, '微波', '团员', 96>
STUDENT2 STUDENT <0206072, '李 进', '男', 20, '图像', '团员', 96>
      :
STUDENT8 STUDENT <0206078, '张 红', '女', 19, '通信', '团员', 90>
STUDENT9 STUDENT <0206079, '钱伟国', '男', 19, '声学', '团员', 92>
```

比较王强与张红二个同学的成绩,可用下列语句完成

```
MOV    AX, STUDENT1 · CREDIT
CMP    AX, STUDENT8 · CREDIT
```

或者改用下列语句

```
MOV    BX, OFFSET STUDENT1
MOV    AX, [BX] · CREDIT
CMP    AX, STUDENT8 · CREDIT
```

七、外部伪指令及对准伪指令

1. 外部伪指令

程序中包含多个模块时,有些程序或数据在各个模块间要相互共享,可用外部伪指令 PUBLIC 和 EXTRN 来实现此功能。其中 PUBLIC 用来定义共享模块,EXTRN 用来调用共享模块。

格式:

PUBLIC 名称,名称,... ;注释

EXTRN 名称:类型,名称:类型,... ;注释

PUBLIC ——伪指令助记符,不可缺省。

名称——本语句的操作数,它是本模块中已经定义过的变量,标号或常数,可供其它模块共享。多个名称之间用逗号分开,不可缺省。

EXTRN ——伪指令助记符,不可缺省。

名称——其它模块中用 PUBLIC 语句定义过的变量,标号或常数,供本模块引用,不可缺省。名称后面紧跟冒号“;”。

类型属性——是指该名称应具有的属性,若所定义的名称是变量,则类型为 BYTE 或 WORD;若名称是标号,则类型为 NEAR 或 FAR;若名称是常数,则类型为 ABS。类型属性应与在其它模块中被定义时的属性相同。多个名称之间用逗号,将它们分开。

EXTRN 语句的引用,必须与已用 PUBLIC 语句定义过的名称相呼应。

例 4-42

```
DATA    SEGMENT
        A1      DB 30H, 31H
        A2      DW 4  DUP (0)
        A3      EQU 0011H
        A4      DB 2  DUP (?)
DATA    ENDS
CODE    SEGMENT
        ASSUME  CS, CODE, DS, DATA
START:  MOV     AX, DATA
        :
        TMF     LABEL FAR
        :
        PUBLIC  A2, A3, TMF      ; A2, A3, TMF 可供其它模块共享
        :
CODE    ENDS
PDATA  SEGMENT
        P1      DB 0AH, 0BH
        P2      DB 2  DUP (?)
PDATA  ENDS
```

```

PCODE    SEGMENT
EXTRN    A2;WORD, A3;ABS, TMF;FAR
MAIN:    MOV     AX, PDATA
        :
        MOV     AX, OFFSET A2
        MOV     CX, A3
        :
        JMP     TMF
        :
PCODE    ENDS
        END     MAIN

```

在 PCODE 模块中,可引用 CODE 模块中已定义的 A2,A3 和 TMF,实现了数据或程序共享。

2. 对准伪指令

格式: EVEN

功能: EVEN 伪指令使下一语句的地址调整为偶地址。

EVEN 直接放在某一语句前,汇编程序汇编时就会完成将地址调整在偶地址上。

例 4-43

```

DATA     SEGMENT
        X1      DB  0DH
        EVEN
        X2      DW  100 DUP (?)
DATA     ENDS

```

1000	01
	00
	02
	00
1004	08
	10
	0A
	00
100A	0D
	10

假若 X1 偏移地址从 100H 开始,在上述数据段中,若没有加入 EVEN 伪指令,X2 的 100 个字将从偏移地址 101H 开始,而加上 EVEN 伪指令后,X2 数据调整到 102H 开始,每个字从偶地址开始存放,提高了存储器存取速度。

另外在汇编语言源程序中经常可使用地址计数器的值‘\$’来保存当前正在汇编的指令的地址。

例 4-44

```

ORG      $+6           ;表示从当前地址跳过 6 个字节
ABC      DW  1,2,$+4,0AH,0DH,$+3

```

表示若 ABC 的偏移地址为 1000H 时,相当于

```

ABC      DW  1,2,1008H,0AH,0DH,100DH

```

其中:ABC 中 \$+4 中 \$ 的当前值为 1004H, \$+3 中 \$ 的当前值为 100AH。

ABC 在内存中存放结果如图 4-7 所示。

图 4-7 例 4-44 的存储格式

3. LABEL

LABEL 伪指令给已定义的变量或标号取另一个名字,并可重新定义它的类型属性,使同一变量或标号在不同地方被引用时,可采用不同的名字,具有不同的类型属性,这样提高了程序的灵活性。

格式:名称 LABEL 类型属性

名称——为 LABEL 语句下一行所使用的语句中的变量或标号取的别名。

LABEL ——伪指令助记符,不可缺省。

类型属性——规定了所起别名的变量或标号的类型,此别名与原变量标号具有相同的段基址及偏移地址。

(1) LABEL 与变量连用

LABEL 与变量连用时,给下一个变量起一个别名,类型属性可修改成 BYTE, WORD 等。

例 4-45

DATB	LABEL BYTE	;DATB 为 DATW 的别名,类型为字节
DATW	DW 3031H, 3233H	;DATW 变量类型为字
	MOV AL, DATB[0]	;31H→AL
	MOV BX, DATW[1]	;3330H→BX

例 4-46 堆栈段中经常使用 LABEL。

```
STACK    SEGMENT STACK 'STACK'
          DW      100 DUP (?)
          TOP      LABEL WORD
STACK    ENDS
```

此处定义 100 个字的堆栈, TOP 为栈底的名,类型为字。

(2) LABEL 与标号连用

LABEL 与标号连用时,给下一语句定义的标号取一个别名,并可改变距离属性为 FAR 或 NEAR。

例 4-47

```
DISF     LABEL FAR
DISN:    MOV     AX, [SI]
```

DISF 与 DISN 指向同一条指令, DISF 是 DISN 的别名,但距离属性改为 FAR, 当其它代码段对它调用时,可以使用。

八、高档微机增加的伪指令

1. 高档微机新增伪指令

80286 以上微机增加了一些伪指令,这些指令主要用来选择 80X86 指令系统、存储器的工作模式、编程模型等,见表 4-4。

表 4-4 高档微机增加的伪指令

伪指令	功 能
.286	选择 80286 指令系统
.286P	选择 80286 保护模式指令系统
.386	选择 80386 指令系统
.386P	选择 80386 保护模式指令系统
.486	选择 80486 指令系统
.486P	选择 80486 保护模式指令系统
.586	选择 Pentium 指令系统
.586P	选择 Pentium 保护模式指令系统
.287	选择 80287 数字协处理器
.387	选择 80387 数字协处理器
.EXIT	用来使程序设计模型退回到 DOS
.MODEL	选择编程模型
.STARTUP	在编程模型中指示程序的开始
ALIGN2	按字或双字分界的段中数据的开始
USES	MASM 6. X 版本指示自动保存过程使用的寄存器
USE16	指导汇编程序 80386~Pentium 以上微处理器使用 16 位的指令模式和数据长度
USE32	指导汇编程序 80386~Pentium 以上微处理器使用 32 位的指令模式和数据长度

在默认情况下,汇编程序只接受 8086/8088 的伪指令,除非将 .386 或 .386P 伪指令,或者微处理器选择的其它开关中的某一个放在程序前面。。386 伪指令通知汇编程序按实模式或使用 386 指令系统,而 .386P 通知汇编程序使用 386 保护模式指令系统。多数软件都是为 80386 或更新的微处理器设计的,因此常常使用 .386 开关。。486,,.486P,,.586,,.586P 等伪指令所含功能与 .386,,.386P 类似。。EXIT,,.MODEL,,.STARTUP 等伪指令在使用模型的格式中用到,下面另作讨论。

如果使用 Microsoft 的 MASM6. x 版汇编程序,PROC 伪指令规定并且自动保存过程中使用的任何寄存器。USES 语句预先指示过程使用的寄存器,使汇编程序在过程开始前就能自动保存它们,而在过程中 RET 指令结束前恢复它们。例如:

DISP PROC USES AX BX CX ; 寄存器之间用空格隔开

上述语句在过程开始前自动将 AX,BX,CX 压入堆栈,而在过程末尾 RET 指令执行前从堆栈中弹出它们。注意,所列出的寄存器之间用空格分开。

2. 模型方式编程格式

汇编程序使用两种基本格式开发软件,一种使用完整的段定义,另一种使用模型。前面已介绍了完整的汇编程序段定义,它是多数汇编程序通用的,包括 Intel 的汇编程序。它能较好的控制汇编语言任务,在软件开发时是经常使用的,特别是在复杂程序中。下面介绍另一种使用模型的格式,可以在短程序中使用。

MASM 汇编程序可以使用多种模型,主要的存储模型如表 4-5。

为了标注模型,使用 .MODEL 语句,其后是存储器系统长度。表 4-5 中 TINY 模型对许多小的程序有效,它要求全部的软件和数据都要安排在 64KB 存储器段内,生成的是 .COM 文件,而不是 .EXE 文件。SMALL 模型要求只用一个数据段和一个代码段,总计

128KB 的存储器,其它模型可以大到 HUGE 模型。

表 4-5 汇编语言的存储模型

模 型	说 明
TINY(微)	所有数据及代码装入同一个代码段内。此模型的程序按 .COM 文件格式编写,要求程序从地址 0100H 处开始
SMALL(小)	这种模型包含两个段:一个 64KB 的数据段和一个 64KB 的代码段
MEDIUM(中)	这种模型包含一个 64KB 的数据段和任意多个代码段,以供大程序使用
COMPACT(压缩)	包含一个代码段和任意多个数据段
LARGE(大)	LARGE 模型允许任意多个代码段和数据段
HUGE(巨型)	允许数据段大于 64KB,其它与 LARGE 模型相同
FLAT(平展)	仅限于 MASM6. X 版本。FLAT 模型使用一个 512KB 的段来存储所有的代码和数据,应注意的是该模型主要用于 Windows NT 中

模型方式的基本格式如下:

```

.MODEL SMALL           ;存储格式伪指令
.STACK                 ;堆栈段定义
.DATA
...                    ;数据定义
.CODE
.STARTUP               ;程序开始伪指令
...                    ;程序代码
.EXIT 0                ;程序结束伪指令
...                    ;子程序代码
END                    ;汇编结束伪指令

```

例 4-48 将一个存储块 LIST1 中 100 个字节的内容复制到存储块 LIST2 中。

```

.MODEL SMALL
.386
.STACK 100H           ;定义堆栈段
.DATA                 ;定义数据段
LIST1 DB 100 DUP(?)
LIST2 DB 100 DUP(?)
.CODE                 ;定义代码段
.STARTUP              ;说明程序的起点,设置 DS,SS
CLD
MOV SI, OFFSET LIST1 ;移动数据
MOV DI, OFFSET LIST2
MOV CX, 100
REP MOVSB
.EXIT 0                ;程序结束,形成返回 DOS 的指令
END

```

.CODE, .DATA 和 .STACK 分别定义代码段、数据段和堆栈段;

. STARTUP 伪指令的使用时, MOV AX, DATA, MOV DS, AX 等送段地址的语句可以取消。同时. STARTUP 伪指令也可以删除 END 语句后面的起始地址;

. EXIT 伪指令返回 DOS 并带有错误参数 0, 0 表示没有错误, 若. EXIT 后面没有参数, 则返回 DOS 后不定义错误代码, 此外 MOV AH, 4CH, INT 21H 语句可以省略。

4-4 DOS 系统功能调用和 BIOS 中断调用

和所有的计算机一样, 微型计算机的硬件环境必须在操作系统的管理下, 才能进行工作。缺少操作系统的计算机, 即所谓裸机, 是一个无生命的壳体。微机上所配的磁盘操作系统(Disk Operating System)简称 DOS 或 MS-DOS。DOS 向用户提供了许多命令及系统功能, 其中命令有内部命令, 如 DIR、TYPE、CD 等, 用户可以在 DOS 提示符下键入这些命令来使用。另外有外部命令, 如 PRINT、XCOPY、FORMAT 等, 用户亦可以键入它们的名称由磁盘调入内存执行。

此外, DOS 还具有对 I/O 设备管理及磁盘与文件管理的功能, 它们一部分被固化在系统的 ROM 中, 可作为 ROM BIOS 模块。另一部分存放在系统磁盘上, 在系统启动时被装入内存, 用户的应用程序及 MS-DOS 的大部分命令都将通过软件中断来调用它们。表 4-6 列出了 DOS 常用的软中断指令, 主要有 INT 20H~INT 2FH。调用这些软中断时, 只要给定入口参数, 接着写一条中断指令 INT n 就可以了。

表 4-6 DOS 常用的软中断命令

软中断指令	功 能	入口参数	出口参数
INT 20H	程序正常退出	无	无
INT 21H	系统功能调用	AH=功能号, 相应入口号	相应出口号
INT 22H	结束退出		
INT 23H	Ctrl-Break 处理		
INT 24H	出错退出		
INT 25H	读磁盘	AL=驱动器号 CX=读入扇区数 DX=起始逻辑扇区号 DS:BX=内存缓冲区地址	CF=0 成功 CF=1 出错
INT 26H	写磁盘	AL=驱动器号 CX=写入扇区数 DX=起始逻辑扇区号 DS:BX=内存缓冲区地址	CF=0 成功 CF=1 出错
INT 27H	驻留退出	DS:DX=程序长度	

所有这些 DOS 软件中断中, 功能最强的是 INT 21H, 它提供了一系列的 DOS 功能调用。DOS 版本越高, 所给出的 DOS 功能调用越多, DOS 6.2 包含了 100 多个功能调用, 这些子程序分别实现外部设备管理、文件读/写、文件管理、目录管理和内存分配等功能。每

个子程序对应一个功能号,给定入口/出口参数后,用 INT 21H 来调用。可以说 INT 21H 的中断调用几乎包括了整个系统的功能,用户不需要了解 I/O 设备的特性及接口要求就可以利用它们编程,对用户来说非常有用。

一、常用的软件中断

1. 读/写磁盘扇区的软件中断

INT 25H 和 INT 26H 软件中断指令,分别用来实现对磁盘指定扇区进行读/写,这两条指令执行时,会分别转去执行 BIOS 中的读/写磁盘扇区子程序。使用这两条指令前,必须按表 4-6 中入口参数的要求,对指定的寄存器分别设置读/写驱动器号,读/写扇区数,起始逻辑扇区号和读/写内存的缓冲区首址,然后才执行相应的中断命令。

例 4-49 读出当前盘(双面盘,9 扇区/道)上的目录。

MOV	AL, 0	;A 盘为当前盘
MOV	CX, 7	;读 7 个扇区中的目录
MOV	DX, 5	;目录从 0 面 0 道 6 扇区开始,对应区号为 5
MOV	BX, 1000H	;读到内存 DS:1000H 区域中
INT	25H	;读磁盘
JMP	0	;返回控制台命令接收状态

2. 退出程序的软件中断

用户程序中可以安排指令退出程序,返回控制台命令接收状态。

用 INT 20H 退出程序时,不需要任何入口参数,INT 20H 中止当前进程,关闭所有打开的文件,清磁盘缓冲区。用 JMP 0 指令和用 INT 20H 指令是一样的,因为在数据段的 0 单元开始是程序段前缀 PSP+0, PSP+1,这两个单元中存放了 INT 20H 指令。区别是 JMP 0 返回方式只能用于扩展名为 COM 的文件中,因为 COM 文件小于 64K,运行时 DS, CS, ES, SS 在同一段中,所以 JMP 0 指令真正能转移到程序段前缀首部,而扩展名为 EXE 的文件不具备这个特点。

用 INT 27H 退出程序时,MS-DOS 会把此用户程序看成是系统的一个组成部分而驻留内存,因此在其它程序装配运行时,这部分程序不会受到覆盖。通常,用户对自己编写的中断处理程序进行装配以后,常用这种方式返回控制台命令接收状态,其它用户程序可以用软中断方式调用这部分程序。必须注意 DX 中要设置驻留程序的长度,否则返回后程序不能驻留。

INT 22H, INT 23H 和 INT 24H 不是真正的中断,它们是在特定条件下通过 DOS 发送的。INT 22H 是当程序结束时,将控制传送到程序段前缀的偏移量为 0AH 的地址上,在程序段建立时,该地址从中断 22H 的向量地址中被复制到程序段前缀 PSP 中。

INT 23H 是当用户在键盘输入或在屏幕输出时按下 Ctrl-Break 键,则控制传送到中断向量表中的 INT 23H 向量,在程序段建立时,中断 23H 中的向量地址被复制到程序段前缀。

INT 24H 是在磁盘 I/O 功能调用时,出现一个致命的磁盘错误,则控制送到中断向量表的 INT 24H 向量,在程序段建立时,INT 24H 向量的地址被复制到程序段前缀。DOS

2.0以上高版本用 4CH 及 31H 比使用 20H,27H 更好。

二、DOS 系统功能调用

DOS 系统功能调用分别实现设备管理、文件读/写、文件管理和目录管理等功能。每个子程序对应一个功能号,所有的系统功能调用的格式是一致的,按下面 4 步进行:

- (1) 系统功能号送到 AH 寄存器中;
- (2) 入口参数送到指定寄存器中;
- (3) 用 INT 21H 指令执行功能调用;
- (4) 根据出口参数分析功能调用执行情况。

有些系统功能调用比较简单,不需要设置入口参数或者没有出口参数。DOS 系统功能调用的功能及入口/出口参数表,详细见附录 F。

- (1) 设备管理包括:键盘输入、显示输出、设置磁盘缓冲器、选择当前盘等功能调用。
- (2) 目录管理包括:查找目录项、更改目录项、建立子目录、删除子目录等功能调用。
- (3) 文件管理包括:建立文件、打开文件、读/写文件、删除文件等功能调用。

例 4-50 2 号功能调用,结果为在屏幕上显示‘A’。

```
MOV    DL, 'A'
MOV    AH, 2
INT    21H
```

下面给出一些常用的 DOS 功能调用的说明。

1. DOS 键盘功能调用

键盘提供了字符键(数字 0~9,字母 A~Z,a~z,%,\$,#),功能键(Home,End,Del,Ins,PgUp,PgDown...等)和控制键(Ctrl,Alt,Shift)。每个键都有对应的键值,即标准 ASCII 码值,通过 DOS 功能调用可读入键值到 AL 寄存器或存储器中,表 4-7 列出了 DOS 键盘功能调用的有关命令。

表 4-7 DOS 键盘功能调用

AH	功 能	入口参数	出口参数
1	从键盘输入一个字符,并在屏幕上显示,检查 Ctrl-Break 键	DL=0FFH	AL=字符
8	键盘输入一个字符,无回显		AL=字符
6	直接键盘输入/输出字符,不检查 Ctrl-Break 键		AL=字符
7	直接键盘输入/出字符,无回显,不检查 Ctrl-Break 键		AL=字符
0AH	输入字符串到内存缓冲区	DS:DX=缓冲区首址	AL=FFH 有键入 AL=0 无键入
0BH	检查键盘输入状态		
0CH	清键盘缓冲区,调用键盘输入功能	AL=键盘功能号(1,6,7,8,A)	

(1) 键入单字符

DOS 功能调用中 1,8,6,7 号功能调用都能完成从键盘输入一个字符到 AL 寄存器,差别在 1 号和 6 号功能调用键入同时在屏幕上显示字符,8 号和 7 号功能调用不回显。

① 1 号功能调用:从键盘输入字符并显示,调用命令为:

```
MOV    AH, 1
INT     21H
```

执行上述命令后,系统扫描键盘等待有键按下,若有键按下,就将键值(ASCII 码)读入,先检查是否为 Ctrl-Break 键,若是就自动调用中断 INT 23H,执行退出命令,否则将键值送 AL 寄存器并在屏幕上显示此字符。

例 4-51 交互式程序中用户按下数字键 1,2,3,程序转入相应的服务子程序,若按下其它键就继续等待。

```
KEY:    MOV    AH, 1           ;读入键值→AL
        INT     21H
        CMP     AL, '1'       ;键值为'1'否?
        JE      ONE
        CMP     AL, '2'       ;键值为'2'否?
        JE      TWO
        CMP     AL, '3'       ;键值为'3'否?
        JE      THREE
        JMP     KEY           ;否则返回等待键入
ONE:     ⋮
TWO:     ⋮
```

② 8 号功能调用:从键盘输入字符但不回显,8 号功能调用命令为:

```
MOV     AH, 8
INT      21H
```

它与 1 号功能类同,检查键入是否为 Ctrl-Break 键,但屏幕无显示。

③ 6 号功能调用:直接控制台输入/输出,6 号功能调用命令为

```
MOV     DL, 0FFH
MOV     AH, 6
INT      21H
```

它可以从键盘输入字符,也可以向屏幕输出字符,并且不检查是否为 Ctrl-Break 键。

当 DL=0FFH 时,表示从键盘输入,若标志位 ZF=0,AL 中为键值,若 ZF=1,表示无键按下,AL 中不是键值。DL≠0FFH 时,表示屏幕输出。

④ 7 号功能调用:直接控制台输入/输出但无回显,命令格式为

```
MOV     AH, 7
INT      21H
```

7号功能与6号功能调用相同,但屏幕不显示,并且不检查是否为Ctrl-Break键。

(2) 输入字符串

0AH功能调用:能从键盘接收字符串到内存的输入缓冲区,要求预先定义一个输入缓冲区,缓冲区的第一个字节指出能容纳的最大字符个数,由用户给出;第二个字节存放实际输入的字符个数,由系统最后填入;从第三个字节开始存放从键盘接收的字符,直到ENTRE键结束。

若实际键入的字符数大于给定的最大字符数,就会发出‘嘟嘟’声,并且光标不再向右移动,后面输入的字符丢失。若键入的字符数小于给定的最大字符数,缓冲区其余部分填0。0AH功能调用时,要求将DS:DX指向缓冲区第一个字节。

例 4-52 开辟一个缓冲区,从键盘输入一个字符串,将输入的字符数→CL寄存器,并将指针指向字符串的第一个字符。

```

    BUFF    DB 100                ;用户定义存放 100 字节的缓冲区
           DB ?                    ;系统填入实际输入字符字节数
           DB 100 DUP(?)          ;存放输入字符的 ASCII 码值
    MOV     AX,DATA
    MOV     DS,AX
    MOV     DX,OFFSET BUFF
    MOV     AH,0AH
    INT     21H
    MOV     BX,DX
    MOV     CL,[BX+1]             ;取输入字符数→CL
    ADD     DX,2                  ;使指针指向第一个字符
    :
```

(3) 检验键盘状态

0BH功能调用:检验是否有键按下,若有键按下,AL=0FFH,若没有键按下,AL=0,无论检测到是否有键按下,程序继续执行下一条指令。

例 4-53 检测键盘工作,若用户未按键,程序循环执行,若用户按下任何键,程序转去调用发声子程序。

```

LOOP:  MOV   AH, 0BH              ;检测键盘状态,0BH 功能调用
       INT   21H
       INC   AL                   ;若 AL=0FFH,+1 后为 0
       JNZ   LOOP                ;无键按下,循环执行
       CALL  SOUND                ;有键按下,调子程序
```

(4) 清除键盘缓冲区

0CH功能调用:先清除键盘缓冲区,然后执行AL中指定的功能,AL中可以指定1,6,7,8或0AH功能号,使程序在输入字符前将以前键入的字符清掉。

例 4-54 MOV AH, 0CH ;清键盘缓冲区

```

MOV    AL, 7                ;执行 7 号功能调用
INT     21H

```

2. DOS 显示功能调用

DOS 显示功能调用能够显示单字符或字符串, 这些功能都自动向前移动光标, 表 4-8 给出了 DOS 显示功能调用的有关命令。

表 4-8 DOS 显示功能调用

AH	功 能	入口参数	说 明
2	显示一个字符, 检验 Ctrl-Break 键	DL=字符	光标跟随字符移动
6	显示一个字符, 不检验 Ctrl-Break 键	DL=字符	光标跟随字符移动
9	显示字符串	DS:DX=串地址	串以 '\$' 结束, 光标随串移动

(1) 单字符显示

① 2 号功能调用: 2 号功能调用实现将字符送到屏幕显示出来。它要求将要显示字符的 ASCII 码值送入 DL 寄存器。

```

MOV     DL, ':'
MOV     AH, 2
INT     21H

```

调用结果屏幕上在光标位置处显示 ':'。

② 6 号功能调用: 是直控制台输入/输出调用, 除前面谈到的键盘输入功能外, 在 DL 不等于 0FFH 时, 表示向屏幕输出, DL 中存放输出字符的 ASCII 码值。

```

MOV     DL, 24H              ; '$' 的 ASCII 码值为 24H
MOV     AH, 6
INT     21H

```

调用结果, 屏幕在光标处显示 '\$' 字符。

(2) 字符串输出

9 号功能调用: 显示字符串, 要求 DS:DX 指向串地址首址, 并且字符串必须以 '\$' 字符为结束符。若要求显示字符串后光标自动回车换行, 则在 '\$' 字符前再加上 0DH(回车), 0AH(换行)字符。

例 4-55 在屏幕上显示 'HOW DO YOU DO?' 字符串, 且光标换行。

```

CR      EQU 0DH
LF      EQU 0AH
MES     DB  'HOW DO YOU DO?', CR, LF, '$'
MAIN:   MOV     AX, DATA
        MOV     DS, AX
        MOV     DX, OFFSET MES      ;DS:DX 指向字符串 MES
        MOV     AH, 9               ;9 号功能调用
        INT     21H

```

3. DOS 打印功能调用

(1) DOS 打印功能

5 号功能调用完成将 DL 寄存器中的字符送到打印机,若需要回车换行,也同样将回车换行的字符码送到 DL 寄存器。打印机的标准控制字符见右表。

字符码	功 能
08H	空格
09H	水平 TAB(横表)
0AH	换行
0BH	垂直 TAB(纵表)
0CH	换页
0DH	回车

例 4-56 完成一串字符打印,遇到 '\$' 结束。打印开始换页,打印结束换行。

```

TEXT DB 0CH, 'Good morning!', 0DH, 0AH, '$'
MOV BX, 0
MOV AH, 5
NEXT: MOV DL, TEXT[BX]
      CMP DL, '$'
      JE STOP
      INT 21H
      INC BX
      JMP NEXT

```

STOP;

(2) 特殊的打印命令

特殊的打印命令如下表所示,ESC 键(1BH)和一些命令连用能改变打印方式。

字符码	功 能
0FH	设置紧缩方式
0EH	设置扩展方式
12H	取消紧缩方式
14H	取消扩展方式
1BH 30H	设置每英寸 8 行
1BH 32H	设置每英寸 6 行
1BH 45H	设置加重打印方式
1BH 46H	取消加重打印方式

例 4-57 打印一句标题,设置成紧缩方式,每寸 8 行,打印并回车换行。

```

TEXT DB 0FH, 1BH, 30H, 'TITLE...', 0DH, 0AH
MOV BX, 0
MOV CX, 20
MOV AH, 5
NEXT: MOV DL, TEXT[BX]
      INT 21H
      INC BX
      LOOP NEXT

```

4. 日期与时间设置

利用 DOS 系统功能调用,能很方便地取得或设置日期和时间,表 4-9 给出了有关的功能调用命令。

表 4-9 日期和时间设置功能调用

功能号	功 能	入口参数	出口参数
2BH	设置日期	CX:年号 DH:月号 DL:日号	AL=0 成功 AL=0FFH 无效
2AH	取得日期		CX:年号 DH:月号 DL:日号
2DH	设置时间	CH:小时 CL:分 DH:秒 DL:百分之一秒	AL=0 成功 AL=0FFH 无效
2CH	取得时间		CH:小时 CL:分 DH:秒 DL:百分之一秒

(1) 设置日期

2BH 功能调用能设置有效日期,预先要在 CX 中存放年号,DH 中存放月份(1~12),DL 中存放日号(1~31),然后用 2BH 功能调用。若设置成功,日期有效,AL=0,否则 AL=0FFH。

例 4-58 设置日期为 1995 年 8 月 20 日。

```
MOV    CX, 1995
MOV    DH, 8
MOV    DL, 20
MOV    AH, 2BH
INT    21H
```

(2) 取得日期

2AH 功能调用能取得日期,将当前有效日期存放在 CX,DX 寄存器中,存放格式与设置日期相同,使用格式为:

```
MOV    AH, 2AH
INT    21H
```

执行结果 CX 中得到当前年号,DH 及 DL 中为月号和日号,为二进制数,本调用不需要入口参数。

(3) 设置时间

2DH 功能调用用来设置时间,调用时预先在 CH 中存放时(0~23),CL 中存放分(0~59),DH 中存放秒(0~59),DL 中存放百分之一秒(0~99),然后用 2DH 功能调用。若设置成功,此时间有效 AL=0,否则 AL=0FFH。

例 4-59 设置时间为 9 点 20 分 15.05 秒。

```
MOV    CH, 09
MOV    CL, 20
MOV    DH, 15
MOV    DL, 05
MOV    AH, 2DH
INT     21H
```

执行结果修改了当前时间为设置值。

(4) 取得时间

2CH 功能调用用来取得当前时间,时间存放格式同设置时间格式,不需要入口参数,使用格式为:

```
MOV    AH, 2CH
INT     21H
```

执行结果 CX:DX 中得到当前时间的二进制值。

5. 异步通讯

表 4-10 给出了 DOS 异步通讯功能调用命令。

表 4-10 DOS 异步通讯功能调用

功能号	功 能	入口参数	出口参数
3	异步通讯口输入		AL=输入的 8 位数据
4	异步通讯口输出	DL=输出的 8 位数据	

(1) 异步通讯输入

3 号功能调用实现从标准异步通讯端口(串行口 COM1)输入一个字符,并将字符置入 AL 寄存器,它没有入口参数,在 PC 系统中,启动时 DOS 将异步通讯端口初始化为 2400 波特,8 位字长,1 位停止位,不设奇偶校验位。其它机器上的 DOS 实现可能有不同的初始化。

```
MOV    AH, 3
INT     21H
```

(2) 异步通讯输出

4 号功能调用实现将 DL 寄存器中的数据输出到异步通讯端口。

```
MOV    DL, 'A'
MOV    AH, 4
INT     21H
```

若输出设备忙,4 号功能调用等待,直到设备准备好接收字符。

需要注意在多数 DOS 系统中,串行口没有缓冲和中断,如果串行口传送数据比程序处理速度快,在通讯中数据可能会丢失。且 DOS 没有提供端口读状态及检测通讯错误的功能,因此可以采用 ROM 中的 BIOS INT 14H 功能调用或直接对端口地址(依赖硬件)编程,可得到较好的通讯效果。

6. 返回操作系统

4CH 功能调用能够结束当前正在执行的程序,返回操作系统,屏幕显示操作提示符。

```
MOV    AH, 4CH
INT     21H
```

此功能调用无入口参数。

DOS 功能调用共有 100 多种,其它因较繁琐,不再一一说明,使用时请参看附录。

三、BIOS 中断调用

驻留在 ROM 中的 BIOS 提供了系统加电自检,引导装入 I/O 设备的处理程序及接口控制等功能模块来处理所有的系统中断。与 DOS 功能调用相同,用户可以直接用指令设置参数,然后中断调用 BIOS 中的程序,给用户编程带来极大的方便。表 4-11 列出了 IBM PC 机主要的 BIOS 中断类型。

表 4-11 BIOS 中断类型

CPU 中断类型	
0 除法错	4 溢出
1 单步	5 打印屏幕
2 非屏蔽中断	6 保留
3 断点	7 保留
8259 中断类型	
8 8254 系统定时器	0CH 保留(通讯)
9 键盘	0DH 保留(Alt 打印机)
0AH 保留	0EH 软盘
0BH 保留(通讯)	0FH 打印机
BIOS 中断类型	
10H 显示器	16H 键盘
11H 设备检验	17H 打印机
12H 内存大小	18H 驻留 BASIC
13H 磁盘	19H 引导
14H 通讯	1AH 时钟
15H I/O 系统扩充	40H 软盘
用户应用程序	
1BH 键盘 Break	1CH 定时器
4AH 报警	
数据表指针	
1DH 显示器参量	41H 1# 硬盘参量
1EH 软盘参量	46H 2# 硬盘参量
1FH 图形字符扩充	

1. 键盘中断调用

INT 16H 中断调用提供了基本键盘操作,如表 4-12 所示:

表 4-12 BIOS 键盘中断(INT 16H)

AH	功 能	返回参数
0	从键盘读一个字符	AL=字符, AH=扫描码
1	读键盘缓冲区字符	ZF=0 时, AL=字符 ZF=1 时, 缓冲区空
2	取特殊功能键状态	AL=特殊功能键状态

利用 INT 16H 功能调用时, 在 AH 中放功能号, 然后使用 INT 16H 指令。将键入的 ASCII 码送到 AL。

```
MOV    AH, 0
INT     16H
```

在 INT 16H 功能调用中, 功能号 AH=2 时, AL 中返回表示特殊功能键的状态变换情况。其中, 位 7 是插入键(Ins), 位 6 是大小字母键(Caps Lock), 位 5 是数字键(Num Lock), 位 4 是滚动键(Scroll Lock), 位 3 是交替键(Alt), 位 2 是控制键(Ctrl), 位 1 是左移键(Left), 位 0 是右移键(Right)。

2. 显示中断调用

INT 10H 功能调用可以进行屏幕设置。使用 INT 10H 调用时, AH 中存放功能号, 并在指定寄存器中存放入口参数。如表 4-13 所示:

表 4-13 显示中断调用(INT 10H)

AH	功 能	入口参数	出口参数
0	对 CRT 初始化	AL=CRT 工作方式	
1	置光标类型	CX=光标开始, 结束行	
2	置光标位置	DX=行、列, BH=页号	
3	读光标位置	BH=页号	CX=光标开始、结束行 DX=行、列
4	读光笔位置		
5	选择显示页	AL=页号	
6	屏幕显示向上滚动或清屏	AL=上滚行数	
7	屏幕显示向下滚动或清屏	AL=下滚行数	
8	读光标处字符/属性	BH=页号	AH=属性, AL=字符
9	在光标处写字符/属性	AL=字符, BL=属性, BH=页号	
0AH	在光标处写字符	AL=字符, BH=页号	
0BH	设置屏幕彩色背景	BX=彩色标识和彩色值	
0CH	在指定坐标处写点	DX=行号, CX=列号	
0DH	在指定坐标处读点	DX=行号, CX=列号	AL=像素值
0EH	写字符	AL=字符	
0FH	取当前屏幕状态		AH=字符列数

例 4-60 在屏幕中间建立一个 20 列宽, 9 行高的窗口, 然后将键盘键入的字符在屏幕窗口上显示出来, 当一行(20 个字符)填满时, 此行自动向上滚动。

```
CODE    SEGMENT
        ASSUME CS, CODE
```

```

        PUSH    DS
        SUB     AX, AX
        PUSH    AX
CLEAR:  MOV     AH, 6           ;清屏
        MOV     AL, 0         ;空白屏幕的代码
        MOV     CX, 0         ;左上角行、列
        MOV     DH, 24        ;右下角行、列
        MOV     DL, 79
        MOV     BH, 7         ;空白行属性
        INT     10H
POS-CU: MOV     AH, 2         ;光标定位在窗口的左下角
        MOV     DH, 16        ;行号
        MOV     DL, 30        ;列号
        MOV     BH, 0         ;当前页号
        INT     10H
        MOV     CX, 14H       ;从键盘输入字符
GET:    MOV     AH, 1
        INT     21H
        CMP     AL, 3         ;输入字符是否 Ctrl-C
        JZ      EXIT
        LOOP    GET          ;取下 1 个字符
SCRO:   MOV     AH, 6         ;滚行
        MOV     AL, 1         ;行数
        MOV     CH, 8         ;开窗口
        MOV     CL, 30
        MOV     DH, 16
        MOV     DL, 50
        MOV     BH, 7         ;属性码为 7 表示普通行
        INT     10H
        JMP     POS-CU       ;光标复位
EXIT:   RET
CODE    ENDS
        END

```

3. 打印中断调用

INT 17H 打印中断调用有 3 个功能, AH 中存放功能号, DX 中存放打印机号(最多允许连接三台打印机, 机号分别为 0, 1 和 2)。表 4-14 是有关打印机 BIOS 的中断操作。

表 4-14 打印机 BIOS 中断(INT 17H)

AH	功 能	入口参数	出口参数
0	指定字符在打印机上打印出来	DX=打印机号 AL=要打印字符的 ASCII 码	AH=打印机状态信息
1	初始化打印机	DX=打印机号 AL=初始化命令	AH=打印机状态信息
2	读取打印状态信息	DX=打印机号	AH=打印机状态信息

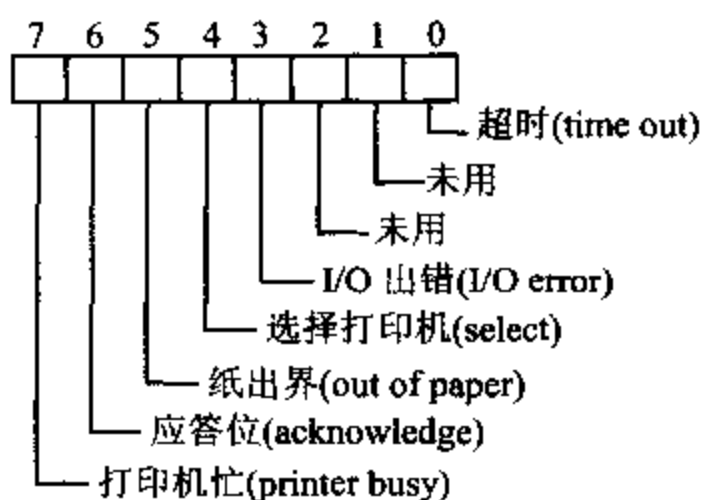


图 4-8

BIOS 17H 的功能 2 是把打印机的状态字读入 AH 中,打印机的状态字节如图 4-8 所示:

例 4-61 初始化打印机,键入字符并打印输出。

```

MOV  AH, 1      ;初始化打印机
MOV  DX, 0      ;0 号打印机
INT  17H
MOV  AH, 1      ;键入字符
INT  21H
MOV  AH, 0      ;打印字符
INT  17H

```

4. 时间设置和读取

INT 1AH 可以实现对时间的设置和读取,调用此功能时, AH 中存放功能号,如表4-15 所示。

表 4-15 时间的设置和读取(INT 1AH)

AH	功 能	入口参数	出口参数
0	读取时间		CH:CL=时:分 DH:DL=秒:1/100 秒
1	设置时间	CH:CL=时:分 DH:DL=秒:1/100 秒	
2	读时钟 (AT 机)		CH:CL=时:分(BCD) DH:DL=秒:1/100 秒(BCD)
6	置报警时间 (AT 机)	CH:CL=时:分(BCD) DH:DL=秒:1/100 秒(BCD)	
7	清除报警		

时间计数器每 55ms 自动加 1,所以将 CX 和 DX 中数除以 65520 得小时数,余数除以 1092 得分数,余数除以 18.2 得秒数。

例 4-62 计算某段程序的执行时间。

```

STI
MOV  CX, 0      ;设时间计数器初值为 0
MOV  DX, 0
MOV  AH, 1
INT  1AH
CALL PROG      ;执行某段程序
MOV  AH, 0      ;读时间计数器值
INT  1AH

```

5. 串行通讯功能调用

INT 14H 调用了 ROM 串行通讯口程序,如表 4-16 所示。

表 4-16 串行通讯口 BIOS 功能调用(INT 14H)

AH	功 能	调用参数	返回参数
0	初始化串行通讯口	AL=初始化参数 DX=通讯口号 COM1=0, COM2=1	AH=通讯口状态 AL=调制解调器状态
1	向串行通讯口写字符	AL=所写字符 DX=通讯口号 COM1=0, COM2=1	写字符成功: AH ₇ =0, AL 不变 写字符失败: AH ₇ =1 AH ₀₋₆ =通讯口状态
2	从串行通讯口读字符	DX=通讯口号 COM1=0, COM2=1	读成功: AH ₇ =0, AL 不变 读失败: AH ₇ =1 AH ₀₋₆ =通讯口状态
3	取通讯口状态	DX=通讯口号 COM1=0, COM2=1	AH=通讯口状态 AL=调制解调器状态

AH=0 时,将通讯口初始化,初始化参数在 AL 中,AL 各位的含义如图 4-9 所示。返回参数中通讯口状态字节在 AH 中,AH 中各位为 1 的含义如图 4-10 所示。

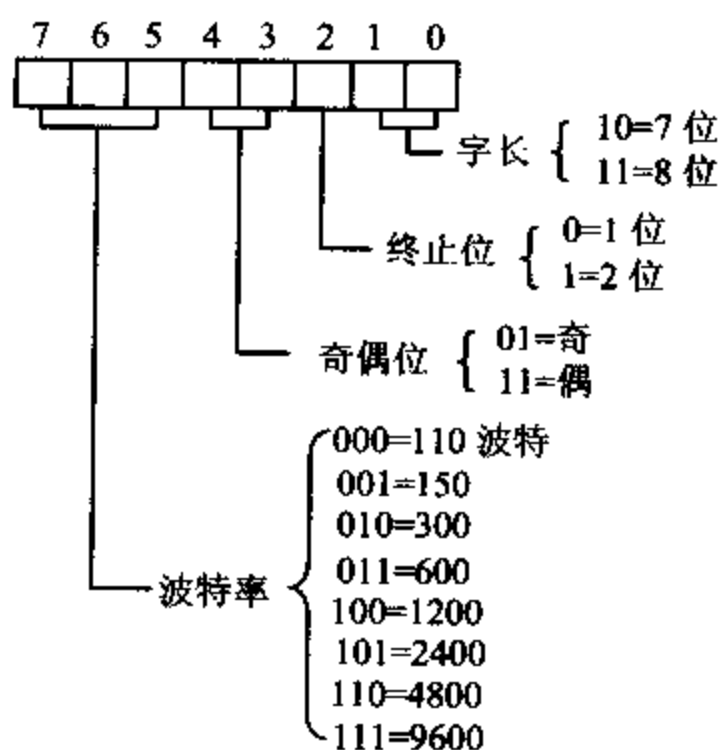


图 4-9 串行通讯口初始化参数

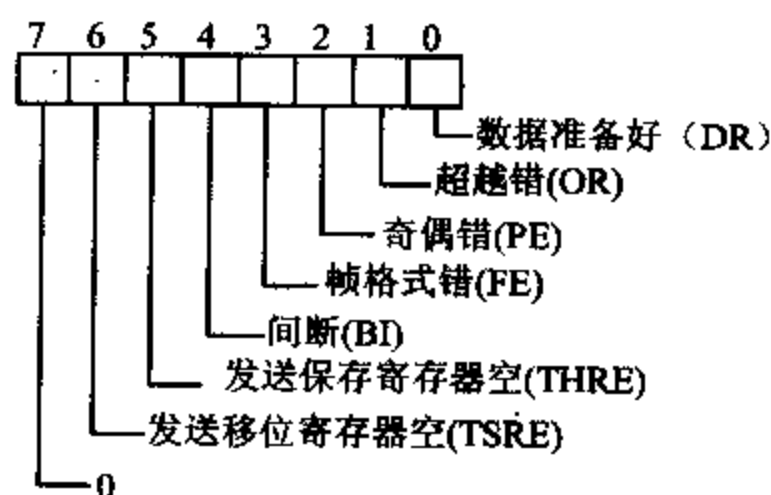


图 4-10 串行通讯口状态字节

例 4-63 从通讯口 0 读入字符并显示出来,如果字符没有准备好则等待,如果传送有错则显示出错信息“?”。

```

CHECK:  MOV    AH, 3           ;读通讯口状态
        MOV    DX, 0          ;COM1
        INT    14H
        AND    AH, 1           ;字符准备好了吗?
        JZ     CHECK
        MOV    AH, 2           ;字符准备好,读通讯口
        MOV    DX, 0
        INT    14H
        TEST   AH, 80H         ;读入字符正确吗?

```

	JNZ	ERR	;不正确,显示出错信息
	AND	AL, 7FH	
	MOV	BX, 0	
	MOV	AH, 0EH	;显示字符
	INT	10H	
	JMP	CHECK	;检查下一个
ERR:	MOV	AL, '?'	
	MOV	BX, 0	
	MOV	AH, 0EH	
	INT	10H	

4-5 程序设计方法

前面几章已讨论了指令系统和汇编语言程序设计基础,而设计出一个好的程序不仅要能正常运行,完成要求的功能,还应该具有下列特点:

- (1) 程序结构模块化,程序易读,易调试及维护。
- (2) 执行速度快。
- (3) 占用内存空间小。

结构化设计在程序复杂的情况下尤为重要。一般来说设计汇编语言源程序的基本步骤如下:

- (1) 分析问题,抽象出描述问题的数学模型,并确定实现数学模型的算法。
- (2) 绘制程序流程图,通常先画粗框图,在结构模块中再画细框图。框图一般有起始框,执行框,判断框和终止框,如图 4-11 所示。

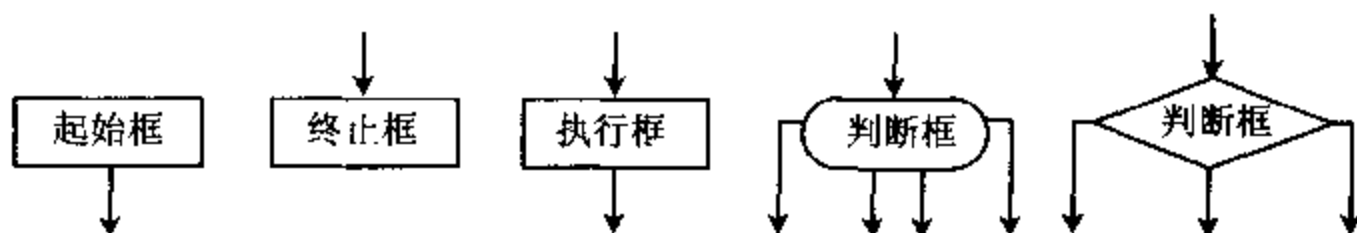


图 4-11 程序框图示意图

(3) 分配存储空间及工作单元。分配数据段,堆栈段,程序段各在内存什么位置,各个寄存器主要做什么用。

- (4) 按流程图设计编写程序。
- (5) 静态检查,上机调试。(一般分段调试较好)
- (6) 程序运行,结果分析。

在进行汇编语言源程序设计时,通常用到四种程序结构:顺序结构,分支结构,循环结构,子程序结构。下面分别加以说明。

一、顺序结构

顺序结构的程序一般是简单程序,程序顺序执行,无分支,无循环,也无转移,图中没有

· 判断框。

例 4-64 内存中 TABLE 开始存放 0~9 的平方值,通过人机对话,当任给定一个数 X (0~9),查表得 X 的平方值,放在 AL 中。

```
.MODEL SMALL
.386
.STACK 100H
.DATA
TABLE DB 0,1,4,9,16,25,36,49,64,81
BUF DB 'Please input one number (0~9):', 0DH, 0AH, '$'
.CODE
.STARTUP ;说明程序起点,设置 DS-SS
MOV DX, OFFSET BUF ;9 号功能调用,提示输入一个数
MOV AH, 9
INT 21H
MOV AH, 1 ;1 号功能调用,键入数送入 AL
INT 21H
AND AL, 0FH
MOV BX, OFFSET TABLE
MOV AH, 0 ;查表得输入数得平方值
ADD BX, AX
MOV AL, [BX] ;保存查表结果到 AL
.EXIT 0
END
```

二、分支结构

1. 分支结构

一般情况下,程序顺序执行,但经常要求程序根据不同条件选择不同的处理方法,这就需要用到分支结构,分支结构如图 4-12 所示。

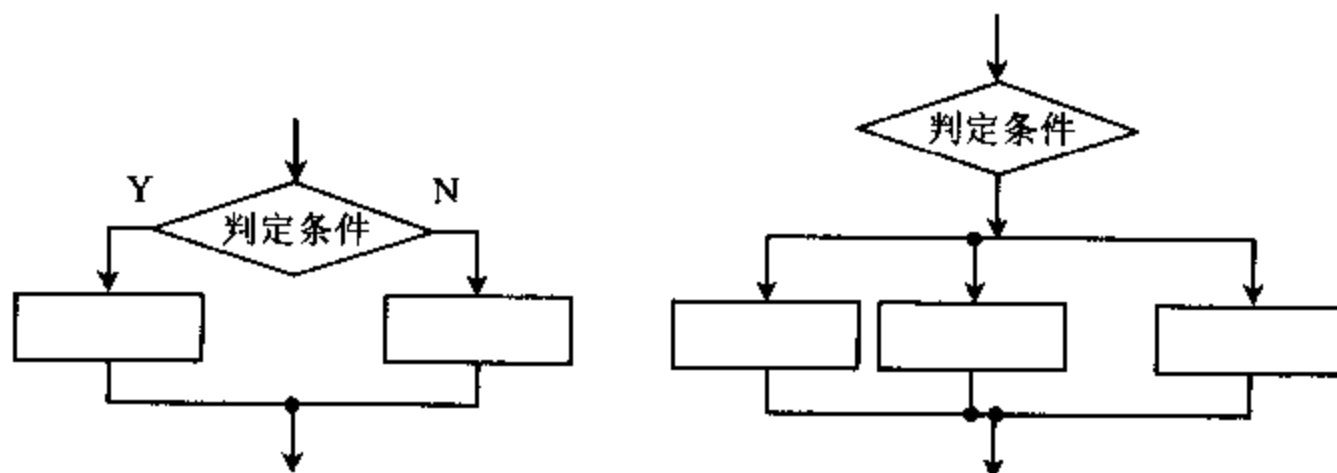


图 4-12 分支结构框图形式

例 4-65 存储器中有一串字符串首址为 BUF,字符串长度 N 小于 256,要求分别计算出其中数字‘0’~‘9’,字母‘A’~‘Z’和其它字符的个数,并分别将它们的个数存放到此字

字符串的下面三个单元中。

```

.MODEL SMALL
.386
.STACK 100H
.DATA
BUF DB N
      DB 01H,38H,47H,90H,33H,09H,76H
NUM DB 3 DUP(?)
.CODE
.STARTUP
      MOV     CH,BUF                ;数组个数 N-->CH
      MOV     BX,1
      MOV     DX,0                  ;DH 计数字的个数,DL 计字母的个数
LP:    MOV     AH,BUF[BX]
      CMP     AH,30H
      JL      NEXT                  ;小于'0'转
      CMP     AH,39H
      JG      ABC                   ;大于'9'转
      INC     DH                    ;数字个数加1
      JMP     NEXT
ABC:   CMP     AH,41H
      JL      NEXT                  ;小于'A'转
      CMP     AH,5AH
      JG      NEXT                  ;大于'Z'转
      INC     DL                    ;字母个数加1
NEXT:  INC     BX                    ;数组地址加1
      DEC     CH                    ;计数减1
      JNZ     LP
      MOV     NUM,DH                ;数字的个数送入内存单元
      MOV     NUM+1,DL              ;字母的个数送入内存单元
      MOV     AH,BUF
      SUB     AH,DH                  ;N-DH-DL=其它字符的个数
      SUB     AH,DL
      MOV     NUM+2,AH              ;其它字符个数送入内存单元
.EXIT 0
END

```

2. 多分支

有的分支结构为多分支,可以利用多个条件转移指令来实现,依次测试条件是否满足,若满足转入相应分支入口,若不满足继续向下测试,直到全部测试完。这种方法编程简单,直观,但运行速度慢,要依次检查才能进入要求的入口。

例 4-66 有 8 个加工子程序,入口地址分别为 P1,P2,...P8。编程实现检测键盘输入命令,使系统分别转向 8 个加工子程序。


```

MOV    AH, 1
INT     21H                      ;1 号功能调用,键盘接收
CMP     AL, '1'                  ;键值为 1,转 1 号加工子程序
JE      P1
CMP     AL, '2'                  ;键值为 2,转 2 号加工子程序
JE      P2
:
CMP     AL, '8'
JE      P8                      ;键值非 1~8,转向停止
JMP     ST
P1: ...                          ;1 号加工子程序
:
P8: ...
ST:     HLT

```

3. 跳转表实现多分支

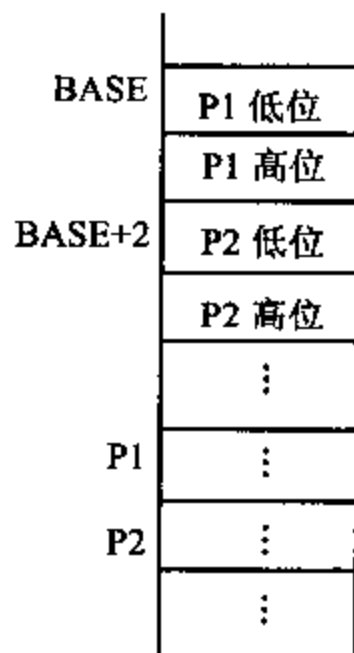
利用跳转表实现多分支,就克服了上面方法的缺点,可以直接找到相应入口。利用这种方法要在存储器中先建立一个跳转表,表中包括每个分支的入口地址,跳转指令或关键字,利用此表就可实现分支结构。

(1) 根据表内地址分支

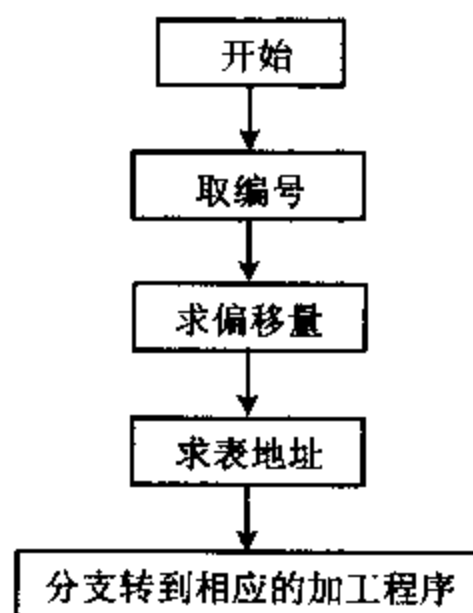
跳转表中存放了每个分支程序的入口地址,只要找到表地址,再将其中内容取出,即可得到每个分支的入口地址。

表地址 = 跳转表首地址 + 偏移地址

图 4-13(a)给出了跳转表在内存中的存放方法,图(b)给出了按表地址分支的流程图。



(a) 跳转表在内存中的存放方法



(b) 跳转表分支流程图

图 4-13 跳转表分支流程图

例 4-67 将例 4-66 中程序改成用跳转表来实现:

```

BASE    DW  P1,P2,P3,P4          ;定义跳转表
          DW  P5,P6,P7,P8

```

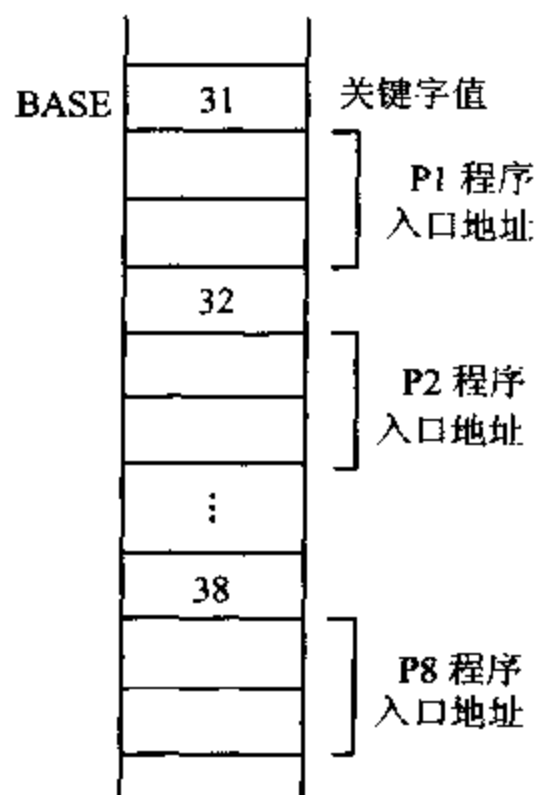

KEY	DB ?	
MOV	AH, 1	;键值在 AL 中
INT	21H	
AND	AL, 0FH	
MOV	BX, OFFSET BASE	;取首地址
MOV	AH, 0	
ADD	AL, AL	
ADD	BX, AX	;求表地址
JMP	WORD PTR [BX]	;转入相应入口地址
:		

(2) 根据表内指令分支

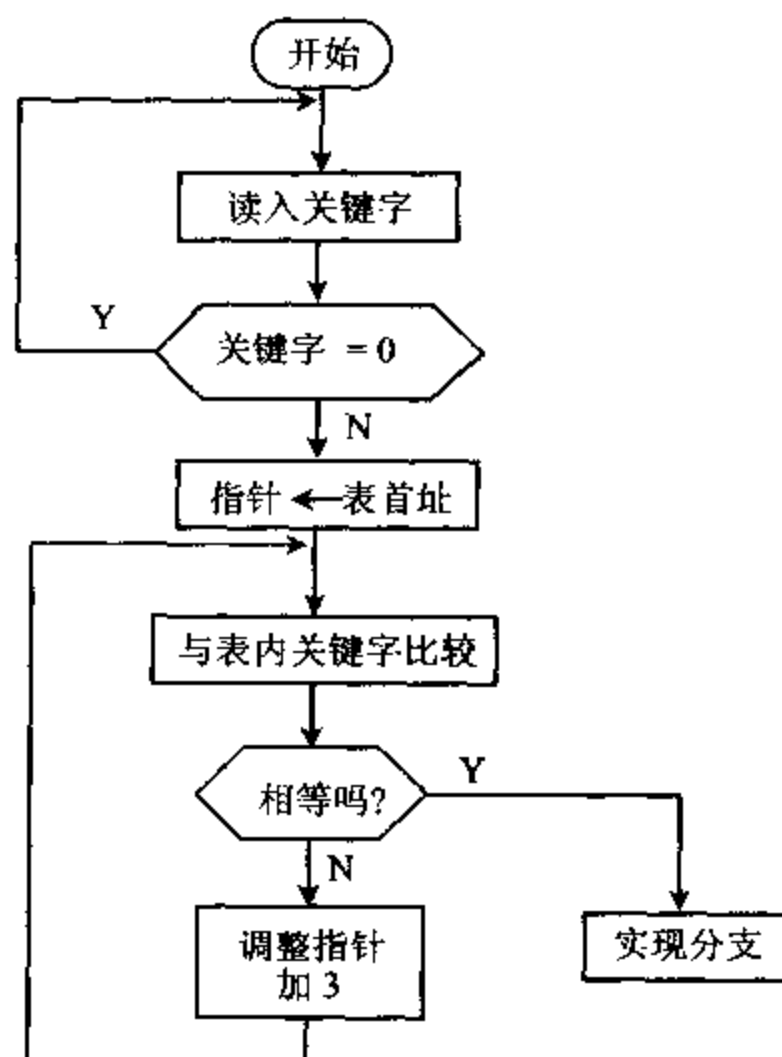
跳转表内存放着转移指令,查表后程序将转到相应子程序去。因为表中存放的是转移指令,求得表地址后,直接转到表地址,就可执行此地址中的转移指令,实现多条件分支。

(3) 根据关键字分支

跳转表中存放关键字,及相应分支地址,图 4-14(a)给出了关键字跳转表的格式,图(b)给出了关键字分支流程图。



(a) 关键字跳转表



(b) 关键字分支流程图

图 4-14 关键字分支流程图

例 4-68 设有首地址为 BUFFER 的数组,已按升序排好,字组的长度为 $N(=10)$,在该数组中查找数 $M(=80)$,若找到它则从数组中删掉,若找不到将它插入正确的排序位置,DX 中记录数组最后的长度。

```

DATA    SEGMENT
        BUFFER DW 5,10,32,47,53,77,89,106,115,124
        N      DW 10
        M      EQU 80
DATA    ENDS
CODE    SEGMENT
        ASSUME CS:CODE, DS:DATA, ES:DATA
MAIN    PROC FAR
START   PUSH    DS
        SUB     AX, AX
        PUSH    AX
        MOV     AX, DATA
        MOV     DS, AX
        MOV     ES, AX
        MOV     AX, M           ;待查数→AX
        MOV     CX, N           ;计数→CX
        MOV     DX, N
        MOV     DI, OFFSET BUFFER
        CJD
REPNE    SCASW                   ;串扫描查找
        JE      DEL             ;查到 ZF=1
        DEC     DX               ;未查到,此数插入正确位置
        MOV     SI, DX           ;关键字与最后一个数比较
        ADD     SI, DX
L1:      CMP     AX, BUFFER[SI]
        JL      L2               ;关键字比数组中某个字小
        MOV     BUFFER[SI+2], AX ;否则插在后面
        JMP     L3
L2:      MOV     BX, BUFFER[SI]   ;数组下移一位
        MOV     BUFFER[SI+2], BX
        SUB     SI, 2
        JMP     L1
L3:      ADD     DX, 2             ;修改长度
        JMP     NEXT1
DEL:     JCXZ    NEXT             ;找到,删此元素
DEL1:    MOV     BX, [DI]         ;其后元素依次前移
        MOV     [DI-2], BX
        ADD     DI, 2
        LOOP    DEL1
NEXT:    DEC     DX               ;改变数组长度
NEXT1:   RET
MAIN    ENDP
CODE    ENDS
        END     START

```

三、循环程序结构

1. 循环程序结构形式

循环程序有两种结构形式：一种是“先执行，后判断”结构，另一种为“先判断，后执行”结构。图 4-15 给出了两种循环程序结构图。

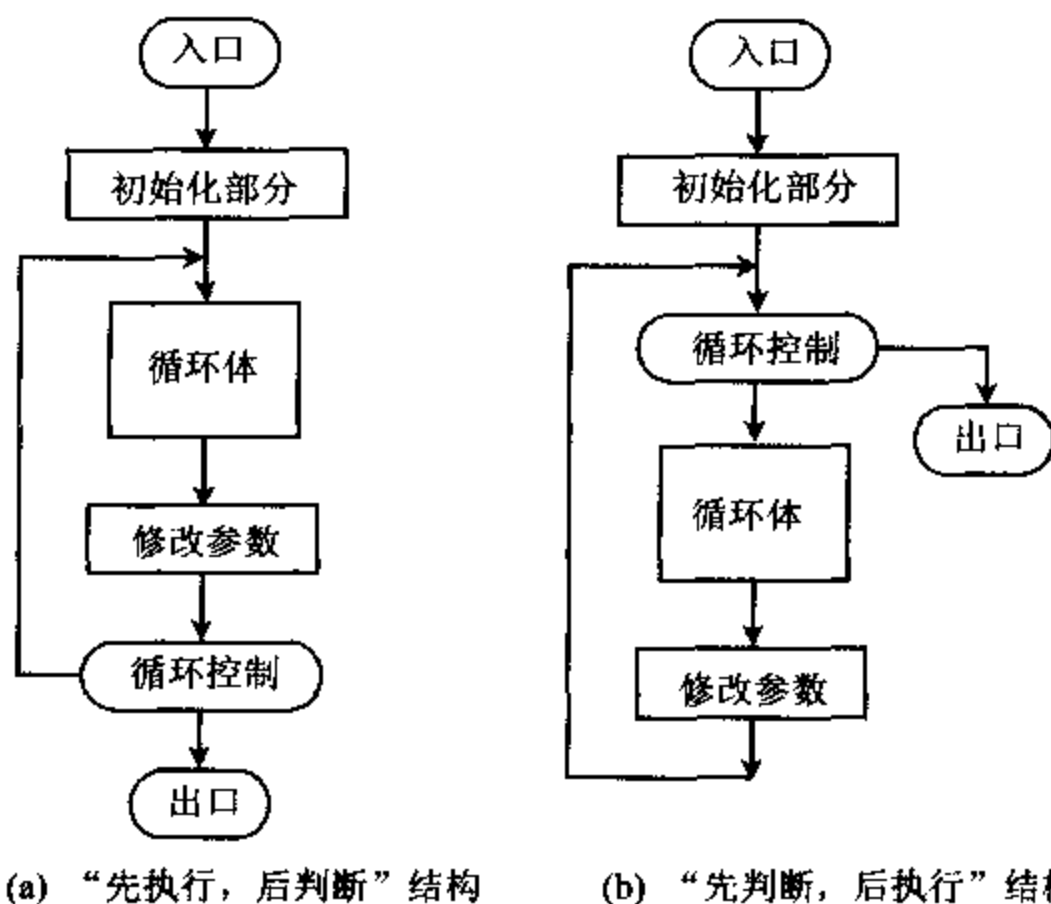


图 4-15 循环程序结构图

无论哪种循环结构都包括四部分：

- (1) 初始化：为循环做准备，设置循环计数值，设置变量初值。
- (2) 循环体：循环部分的核心，包括循环的全部执行指令。
- (3) 修改参数：修改操作数地址，为下次循环做准备。
- (4) 循环控制：修改计数器值，判断循环控制条件，决定是否跳出循环。

对“先执行，后判断”结构，进入循环后至少要执行一次循环体，再判断循环是否结束。对“先判断，后执行”结构，进入循环后，先判断循环结束条件，再决定是否执行循环体，可能循环体一次也不执行，即循环次数为 0。一般来说，前者适合于循环次数固定的程序，后者适合于循环次数不固定的程序。循环程序中循环计数判断可以用 LOOP/LOOPE 语句，也可用条件跳转语句。

例 4-69 将 BX 中的 16 进制数转换为 ASCII 码，存放到 BUF 开始的内存单元中去，并在屏幕显示出数值。

	MOV	SI, OFFSET BUF	;设置内存地址
	MOV	CH, 4	;计数初值=4
NEXT:	MOV	CL, 4	
	ROL	BX, CL	;最高位移到右边
	MOV	AL, BL	;一位数转换成 ASCII 码

```

        AND    AL, 0FH
        ADD    AL, 30H
        CMP    AL, 3AH           ;字符为 A~F 吗?
        JL     STORE
        ADD    AL, 7
STORE:  MOV    [SI], AL           ;字符存入内存
        MOV    AH, 2             ;调用屏幕显示
        MOV    DL, AL
        INT    21H
        INC    SI                 ;修改计数并判断
        DEC    CH
        JNZ    NEXT
        HLT                      ;循环结束

```

本程序编写采用“先执行,后判断”的结构。流程图如图 4-16 所示。

例 4-70 AX 寄存器中有一个 16 位二进制数,编程统计其中 1 的个数,结果放到 CL 寄存器。

```

        MOV    CL, 0             ;初始化
L1:     AND    AX, AX             ;控制循环
        JZ     STOP
        SAL    AX, 1             ;循环体
        JNC    L2
        INC    CL
L2:     JMP     L1
STOP:   HLT

```

此程序编写采用“先判断,后执行”的结构,先判断若 $AX=0$,则不必循环了。计 1 的个数采用移位方法,16 次移位结束, $AX=0$,自动结束,省去中间判断循环次数。

2. 用逻辑尺的方法控制循环

循环控制条件是循环程序设计的关键,必须结合对算法的分析来选择控制条件。有时程序要求按不同次序处理两种函数操作,可以采用逻辑尺的方法控制循环。

例 4-71 某个采样系统第 1,2,5,7,10 次采样时,采用 FUN1 计算公式,第 3,4,6,8,9 次采样时采用 FUN2 计算公式。可用一个位串来控制,某位为 0 采用 FUN1 计算公式,某位为 1 采用 FUN2 计算公式。每次循环后使位串左移 1 位,用 CF 值来控制转到不同分支,实现此控制的位串称为逻辑尺。本例要求的位串为 0011 0101 1000 0000,可将其放在某一寄存器或存储单元中,有关程序段如下:

```

FUN1=X+5
FUN2=X-3
LOGRUL EQU 0011010110000000B           ;逻辑尺
COUNT EQU 10                           ;循环次数
BUF     DB 20 DUP(?)                     ;采集数据
BLOCK   DB 20 DUP(?)                     ;处理后数据

```

```

MOV     DX, LOGRUL
MOV     CX, COUNT
MOV     SI, OFFSET BUF
MOV     DI, OFFSET BLOCK
NEXT:   MOV     AX, WORD PTR [SI]
        ROL     DX, 1
        JC      FUN2
FUN1:   ADD     AX, 5
        JMP     NEXT1

```

```

;逻辑尺→DX
;设循环次数
;设地址指针
;左移一位
;进位为 1 转 FUN2
;FUN1=X+5

```

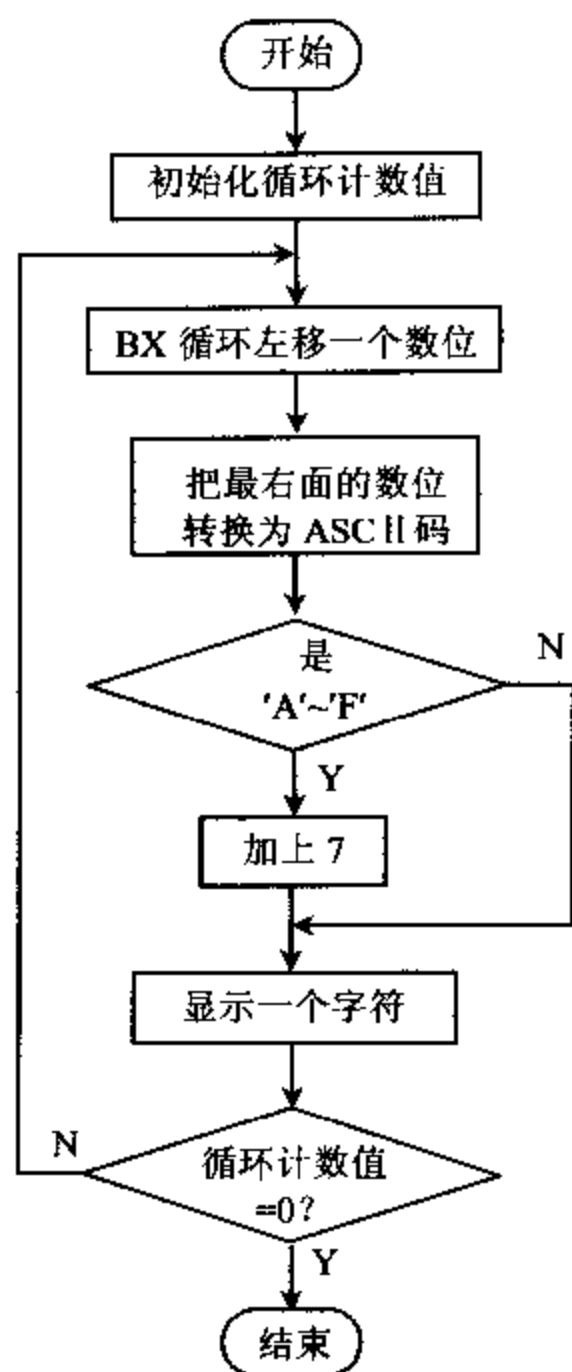


图 4-16 16 进制数转化成 ASCII 码流程图

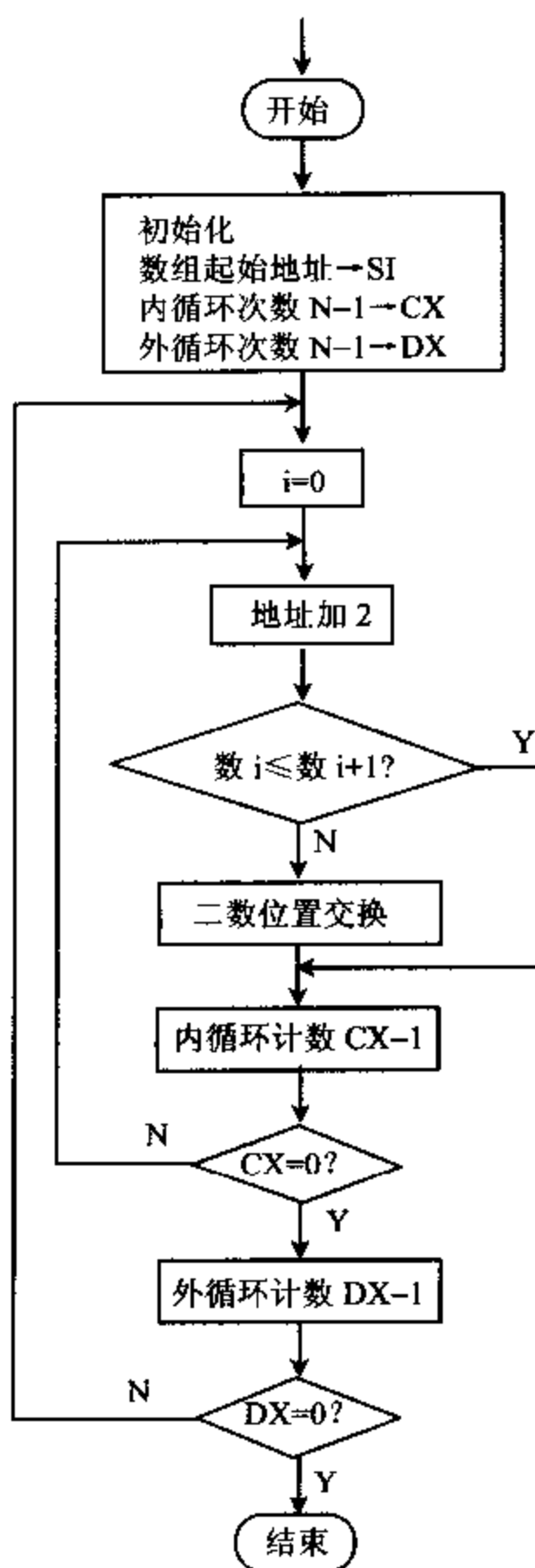


图 4-17 数组排序冒泡算法之一流程图

```

FUN2:  SUB      AX, 3                      ;FUN2=X-3
NEXT1: MOV      WORD PTR [DI], AX          ;送结果
      INC      SI                        ;修改地址指针
      INC      SI
      INC      DI
      INC      DI
      LOOP     NEXT
      MOV      AH, 4CH                    ;返回 DOS
      INT      21H

```

使用逻辑尺的方法有时很方便,例如在矩阵运算中利用这种方法,为0的元素运算可以跳过,节省计算时间。

3. 多重循环

有些循环结构比较复杂,需要用多重循环完成。多重循环设计与单重循环设计方法相同,但应注意:

- (1) 各重循环的初始控制条件及程序实现。
- (2) 内循环可以嵌套在外循环中,也可几个内循环并列在外循环中,但各层循环之间不能交叉,可以从内循环跳到外循环,不可以从外循环中直接跳进内层循环。
- (3) 防止出现死循环,即不能让循环回到初始条件,引起死循环。

例 4-72 存储器数据段从 BUF 开始存放一个数组,数组中第一个字中存放该数组的长度 N,编制一个程序使此数组中的数据按照从小到大的次序排列。

采用冒泡排序算法。从第一个数据开始相邻的数进行比较,若次序不对,两数交换位置。第一遍比较(N-1)次后,最大的数已到了数组尾,第二遍仅需比较(N-2)次就够了,共比较(N-1)遍就完成了排序,这样共有两重循环。图 4-17 给出了程序流程图。

```

DATA    SEGMENT
        BUF      DW      N,15,37,8600,A768H,3412H,1256H,76H
DATA    ENDS
STACK   SEGMENT  STACK  'STACK'
        SA        DB  100 DUP (?)
        TOP       LABEL WORD
STACK   ENDS
CODE    SEGMENT
        ASSUME    CS:CODE, DS:DATA, SS:STACK
MAIN    PROC      FAR
START:  MOV       AX, STACK
        MOV       SS, AX
        MOV       SP, OFFSET TOP
        PUSH      DS
        SUB       AX, AX
        PUSH      AX
        MOV       AX, DATA
        MOV       DS, AX

```

```

        MOV     BX, 0
        MOV     CX, BUF[BX]           ;设计数器 CX,内循环次数
        DEC     CX
L1:      MOV     DX, CX                 ;设计数器 DX,外循环次数
L2:      ADD     BX, 2
        MOV     AX, BUF[BX]           ;取 BUF[I]与 BUF[I+2]
        CMP     AX, BUF[BX+2]         ;若 BUF[I]≤BUF[I+2]转
        JBE     CONTI
        XCHG    AX, BUF[BX+2]         ;否则两数交换
        MOV     BUF[BX], AX
CONTI:   LOOP    L2                     ;内循环
        MOV     CX, DX                 ;外循环次数→CX
        MOV     BX, 0                 ;地址返回第一个数据
        LOOP    L1                     ;外循环
        RET
MAIN     ENDP
CODE     ENDS
        END     START

```

例 4-73 排序可采用另一种算法,用设置标志的方法排序。当排序已完成,即没有数进行交换时,可以结束外循环,不必循环 $N-1$ 遍,节省了操作时间。设立一个 FLAG 位作标志,进入外循环时标志为 0,在内循环中每进行一次交换,标志置 1。内循环结束测试标志,若为 1 再一次进入外循环;若为 0 表示内循环没有进行数据交换,可以结束外循环了。下面给出了排序算法 2 的有关程序。

```

START:   MOV     DI, OFFSET BUF       ;数组起始地址
        MOV     CX, [DI]              ;设计数器 CX,内循环用
        DEC     CX                     ;N-1,内循环次数
L1:      MOV     DX, CX                 ;设计数器 DX,外循环用
        MOV     BH, 0                 ;BH 为 FLAG
L2:      ADD     DI, 2
        MOV     AX, [DI]              ;若 BUF[I]>BUF[I+1],两数交换,FLAG=1
        CMP     AX, [DI+2]
        JBE     CONTI
        XCHG    AX, [DI+2]
        MOV     [DI], AX
        MOV     BH, 1
CONTI:   LOOP    L2
        CMP     BH, 0                 ;FLAG=0,内循环未交换数,跳出外循环,
                                       ;否则 DX-1,又一次内循环,继续比较
        JE      STOP
        MOV     CX, DX
        MOV     DI, OFFSET BUF
        LOOP    L1

```



```

MOV    AH,4CH
INT     21H
STOP;   RET

```

四、子程序结构

1. 子程序使用

汇编语言中多次使用的程序段可写成一个相对独立的程序段,将它定义为“过程”或称子程序,需要执行这段程序时,就进行“过程”调用,执行完毕后再返回原来调用它的程序。采用子程序结构编程,使程序结构模块化,程序清晰,修改容易。每一个子程序包括在过程定义语句 PROC...ENDP 中间。过程定义有属性 NEAR 或 FAR,调用程序和过程在同一代码段中,则用 NEAR,调用程序和过程若不在同一代码段中,使用 FAR。一般来说,主过程应定义为 FAR 属性,因为可以把程序的主过程看作 DOS 调用的一个子过程,而 DOS 对主过程的调用和返回都是 FAR 属性。过程调用的指令为 CALL,过程返回的指令为 RET。通常编写子程序时,写一个子程序说明,能使模块结构一目了然。子程序说明包括:

- (1) 功能描述:子程序的名称,功能及性能
- (2) 子程序中用到的寄存器和存储单元
- (3) 子程序的入口参数,出口参数
- (4) 子程序中调用其它子程序的名称

例 4-74 有一个子程序说明如下:

```

;名称:BCD2BIN
;功能:将一个字节的 BCD 码转换成二进制数
;所用寄存器: CX
;入口参数: AL 存放两位 BCD 码
;出口参数: AL 存放二进制数
;调其它子程序:无

```

子程序形式如下:

```

BCD2BIN    PROC    NEAR(或 FAR)
            PUSH    CX
            MOV     CH, AL
            AND     CH, 0FH                ;存低 8 位
            MOV     CL, 4
            SHR     AL, CL                ;高 8 位右移 4 位后乘 10
            MOV     CL, 10
            MUL     CL
            ADD     AL, CH                ;高 8 位加低 8 位
            POP     CX
            RET

```


过程调用时,主程序使用 CALL 指令,调用中要处理好三个问题:

(1) 保护调用程序的返址,这一点由 CALL 指令本身完成,CPU 执行 CALL 指令时会自动将当前断点的偏移地址值 IP 入栈。若是段间调用,将段基址 CS 及偏移地址值 IP 入栈。当子程序返回时,遇到子程序中 RET 指令,则自动将当前栈顶值弹出到 IP 及 CS 寄存器中,因此要特别注意堆栈的使用,防止弹出地址值错误。

(2) 保护某些寄存器内容,子程序中要用到某些寄存器,为了不破坏寄存器中原有的信息,要将需要保护的寄存器内容入栈,一般安排在子程序开头,用一组 PUSH 指令,在子程序结尾处,相应安排一组 POP 指令,将所保护的寄存器原来的内容恢复。堆栈工作方式为先进后出,所以要注意 PUSH 和 POP 指令组的次序。

(3) 主程序与子程序相互之间参数的传递,由于相互之间可以传递参数才使子程序更灵活,更具有通用性,参数传递的方法有 3 种。

① 用寄存器传递参数:适合于参数较少的情况,传递速度较快。

② 用存储器传递参数:适合参数较多的情况,需要事先在存储器中建立一个参数表。

③ 用堆栈传递参数:适合参数较多的情况,尤其是在子程序嵌套与递归调用的情况下,比较不容易出错。

下面举例说明参数传递的方式。

例 4-75 数据段定义两个数组,编程序实现数组段分别求和(不计溢出)。

```
DATA    SEGMENT
        ARY1    DW    100  DUP (?)      ;定义数组 1
        SUM1    DW    ?
        ARY2    DW    100  DUP (?)      ;定义数组 2
        SUM2    DW    ?
DATA    ENDS
STACK   SEGMENT  STACK
        SA      DW    50  DUP (?)
        TOP     EQU  LENGTH SA
STACK   ENDS
CODE    SEGMENT
        ASSUME  CS:CODE, DS:DATA, SS:STACK
MAIN    PROC    FAR
START:  MOV     AX, DATA
        MOV     DS, AX
        MOV     AX, STACK
        MOV     SS, AX
        MOV     SP, TOP
        LEA     SI, ARY1                ;数组 1 首地址,入口参数
        MOV     CX, LENGTH ARY1        ;数组 1 长度,入口参数
        CALL    SUM                    ;调用求和子程序
        LEA     SI, ARY2                ;数组 2 首地址,入口参数
```

```

        MOV     CX, LENGTH ARY2      ;数组 2 长度,入口参数
        CALL    SUM                  ;调用求和子程序
        MOV     AH, 4CH
        INT     21H
        RET
MAIN     ENDP
SUM      PROC     NEAR                ;子程序
        XOR     AX, AX                ;AX 清 0
L1:      ADD     AX, WORD PTR[SI]      ;加数组元素
        INC     SI
        INC     SI
        LOOP    L1
        MOV     WORD PTR[SI], AX      ;数组和送入 SUM
        RET
SUM      ENDP                        ;子程序返回
CODE     ENDS
        END     START

```

本例是通过存储器来传递参数的,需要传递的数组的数保留在存储器中,调用前只需将数组偏移地址放入 SI 寄存器,在过程中通过寄存器间址就可取得存储器中的操作数,运算结果直接由过程写回到存储器中,回送给调用程序。

例 4-76 通过堆栈传递参数,实现十进制数数组求和,要求主程序和过程不在同一个代码段中,要进行段间调用。源程序如下:

```

MDATA    SEGMENT
        ARY1    DB 20  DUP (?)        ;定义数组 1
        SUM1    DW ?
        ARY2    DB 100 DUP (?)        ;定义数组 2
        SUM2    DW ?
MDATA    ENDS
MSTACK   SEGMENT  STACK
        SB      DW 100  DUP (?)
        TOP     LABEL WORD
MSTACK   ENDS
MCODE    SEGMENT                ;主程序段
        ASSUME  CS;MCODE, DS;MDATA, SS;MSTACK
MAIN     PROC     FAR
START:   MOV     AX, MSTACK
        MOV     SS, AX
        MOV     SP, OFFSET TOP
        PUSH    DS                ;初始化 DS, SS, SP
        MOV     AX, 0
        PUSH    AX
        MOV     AX, MDATA

```

```

MOV     DS, AX
MOV     AX, OFFSET ARY1      ;PADD 过程入口参数进栈
PUSH    AX
MOV     AX, SIZE ARY1
PUSH    AX
CALL    FAR PTR PADD
MOV     AX, OFFSET ARY2
PUSH    AX
MOV     AX, SIZE ARY2
PUSH    AX
CALL    FAR PTR PADD
RET
MAIN    ENDP
MCODE   ENDS
PCODE   SEGMENT              ;过程段
        ASSUME CS:PCODE, DS:MDATA, SS:MSTACK
PADD    PROC    FAR
        PUSH    BX            ;寄存器保护
        PUSH    CX
        PUSH    BP
        MOV     BP, SP
        PUSHF
        MOV     CX, [BP+10]    ;数组长度→CX
        MOV     BX, [BP+12]    ;数组 ARY 起始地址→BX
        MOV     AX, 0
NEXT:   ADD     AL, [BX]        ;数组相加
        DAA
        MOV     DL, AL
        MOV     AL, 0
        ADC     AL, AH
        DAA
        MOV     AH, AL
        MOV     AL, DL
        INC     BX
        LOOP    NEXT
        MOV     [BX], AX      ;送数组和→SUM
        POPF
        POP     BP
        POP     CX
        POP     BX
        RET     4             ;返回作废参数
PADD    ENDP
PCODE   ENDS
END      START

```

这是一个用堆栈传递参数,完成段间过程调用的实例,这里要注意几点:

(1) 使用堆栈传递参数,调用前要给过程传递两个参数,主程序中将数组的偏移地址值及数组长度压入堆栈,然后调用过程,过程完成数组求和运算。

(2) 程序设计要求段间过程调用,因此进入过程时要重新定义 CS 段,使之指向当前有效代码段 PCODE,在过程运行中可直接调用堆栈中参数,完成累加运算,并将结果送回指定的存储单元。

(3) 过程返回时用返回指令 RET 4,要将堆栈中由 CALL 指令之前传递来的 4 个字节作废,然后才能返回主程序,以保证下面过程调用时参数传递正确。

(4) 在整个程序运行中,堆栈中数据变化如图 4-18 所示:

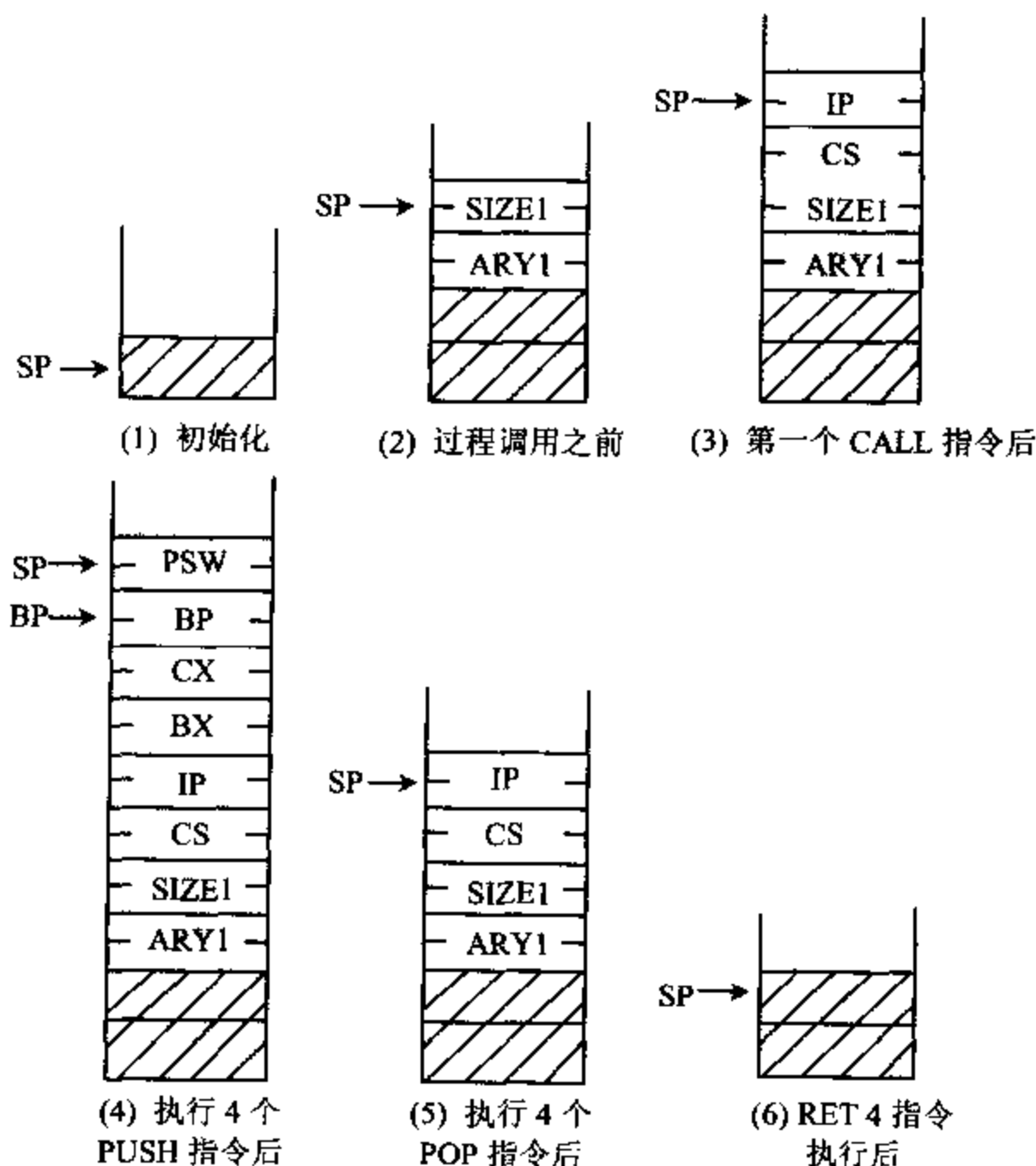


图 4-18 堆栈数据变化示意图

- ① 初始化后,SP 指向堆栈底,堆栈为空;
- ② 主程序调用过程前,二个参数(数组偏移地址及数组长度)已入栈;
- ③ 进行过程调用,执行 CALL 指令时,原主程序段地址及调用后下一条指令的偏移地址自动入栈;

④ 在过程段中由 PUSH 指令将 BX,CX,BP 及标志寄存器相继入栈。

继续执行过程段 PCODE 程序,可以利用基址指示器 BP 给出数组偏移地址 ARY1 在堆栈中位置[BP+12]处得到,数组长度 SIZE1 在堆栈中位置[BP+10]处得到。

⑤ 过程运行到 RET 4 之前,由 POP 指令将 PSW,BP,CX 及 BX 相继弹出。

⑥ 执行 RET 指令自动将保留的断点地址顺序弹出送回 IP 和 CS 中,但堆栈中还有 4 个字节的参数 SIZE1 和 ARY1 会影响下次过程调用参数的正确传递,所以用 RET 4 将此 4 个字节的参数弹出作废。到此过程结束,返回主程序,第二个 CALL 指令重复上述过程。

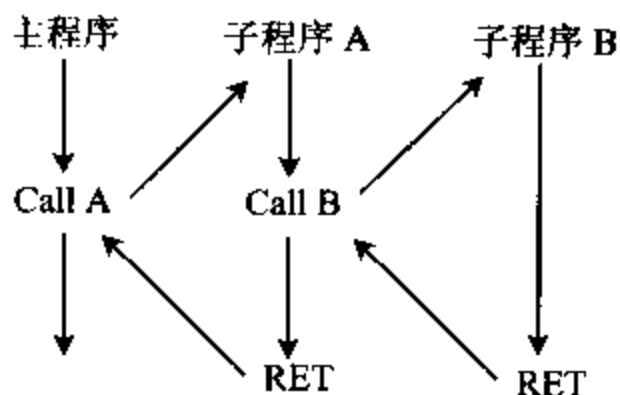


图 4-19 子程序嵌套示意图

2. 子程序嵌套与递归调用

子程序本身又可调用其它子程序,称为子程序嵌套,嵌套的层数不限,只要堆栈空间足够就可以。但要注意寄存器的保护及恢复,避免各层子程序之间寄存器使用冲突,造成程序出错。图 4-19 给出了子程序嵌套示意图。

子程序调用子程序本身,称为子程序递归调用。下面举例说明程序递归调用:

例 4-77 要求计算 $N!$ ($N \geq 0$)

这是一个递归调用的计算方法, $N! = N * (N-1) * (N-2) \cdots * 1$

$$N! = 1, \quad N=0$$

$$N! = N * (N-1)!, \quad N > 0$$

由上面公式知道求 $N!$ 即为计算 $N * (N-1)!$,而 $(N-1)!$ 要调用 $N!$ 子程序,只要将调用参数修改即可。可以通过堆栈将调用参数、寄存器内容等保护起来,每调用一次将 $(N-1)$ 有关信息入栈,直到 $N=0$ 为止,然后开始返回,返回时将 N 乘以 $(N-1)!$,直到 N 为设置值为止。

$N!$ 子程序说明

```

;N! 程序:AL=N,DX=N!
.MODEL SMALL
.386
.STACK
.DATA
    P1      DB  'Input the number:(0~6)', '$' ;提示输入信息
    P2      DB  'The result is:', '$'         ;提示输入信息
    CR      DB  0DH, 0AH, '$'                ;回车换行
.CODE
.STARTUP
    MOV     AH, 09
    LEA     DX, P1
    INT     21H                               ;显示 P1,提示输入 N
    MOV     AH, 1
    INT     21H                               ;输入数字到 AL
    AND     AL, 0FH                           ;AL 存放 N
    MOV     BL, AL
    LEA     DX, CR
    MOV     AH, 09
    INT     21H                               ;回车换行
    MOV     DX, 0                             ;DX 置零,用于保存结果

```

```

MOV     AL, BL
;调用过程 FACT;输入:AL=N;输出:DX=N!
CALL    FACT
MOV     AX, DX
;调用过程 B2TOBCD;输入:AX=16 位二进制数;输出:AX=4 位压缩型 BCD 码
CALL    B2TOBCD
MOV     BX, AX
MOV     AH, 09
LEA     DX, P2
INT     21H
;调用过程 DISP;输入:BX=4 位压缩型 BCD 码;输出:在屏幕上显示
CALL    DISP
.EXIT 0

;输入参数:AL=N
;输出参数:DX=16 进制结果
FACT PROC
    CMP     AL, 0                      ; N! 子程序
    JNZ     CHN
    MOV     DL, 1                      ; N=0, N! =1
    RET
CHN:  PUSH  AX                      ; N 入栈
      DEC   AL                      ; N-1
      CALL  FACT                    ; 递归调用 FACT 子程序
      POP   AX                      ; N 弹出
      MUL   DL                      ; N 逐层返回相乘
      MOV   DX, AX                  ; 送结果到内存
      RET
FACT ENDP

;输入参数:AX=16 位二进制数
;输出参数:CF=0,则 AX=4 位压缩型 BCD 码。CF=1,则要转换的数大于 9999,AX 不变
;使用寄存器:CX;存放除数;DX:存放中间结果
B2TOBCD PROC
    CMP     AX, 9999                  ; AX>9999,则 CF 置 1
    JBE     TRAN
    STC
    JMP     EXIT
TRAN:  PUSH  CX
      PUSH  DX
      SUB   DX, DX                    ; DX 置 0
      MOV   CX, 1000                 ; 计算千位数
      DIV   CX
      XCHG  AX, DX                    ; 商在 DX 中,余数在 AX 中
      MOV   CL, 4
      SHL   DX, CL                    ; DX 左移 4 位

```

MOV	CL, 100	;计算百位数
DIV	CL	
ADD	DL, AL	;百位数加到 DL 中,DX 左移 4 位
MOV	CL, 4	
SHL	DX, CL	
XCHG	AL, AH	;余数保留在 AL 中
SUB	AH, AH	
MOV	CL, 10	;计算十位数
DIV	CL	
ADD	DL, AL	
MOV	CL, 4	
SHL	DX, CL	
ADD	DL, AH	
MOV	AX, DX	
POP	DX	
POP	CX	
EXIT; RET		
B2TOBCD ENDP		
;输入参数:BX=4 位压缩型 BCD 码		
;输出:在屏幕上显示结果		
;使用寄存器:CX,AX		
DISP	PROC	
	PUSH AX	
	PUSH CX	
	MOV CH, 4	;显示的数字个数
	MOV CL, 4	;左移位数:4
LZ:	ROL BX, CL	;循环左移
	MOV DL, BL	
	AND DL, 0FH	
	CMP DL, 0	
	JNE LNZ	;最高位是 0,则不输出,转到下一位
	DEC CH	
	JNZ LZ	
LL:	ROL BX, CL	;循环左移
	MOV DL, BL	
	AND DL, 0FH	
LNZ:	ADD DL, 30H	
	MOV AH, 2	
	INT 21H	
	DEC CH	
	JNZ LL	
	POP CX	
	POP AX	
	RET	


```
DISP ENDP
END
```

五、综合举例

1. 代码转换

例 4-78 将十六位二进制数转换成 4 位压缩型 BCD 码算法:将 AX 中的二进制数先后除以 1000、100 和 10,每次除法所得的商,即是 BCD 数的千位、百位和十位数,余数是个位数。

子程序名: B2TOBCD

输入参数: AX=十六位二进制数。

输出参数: CF=0,则 AX=4 位压缩型 BCD 码。CF=1,则要转换的数大于 9999,AX 不变。

使用寄存器: CX:存放除数,DX:存放中间结果。

```
B2TOBCD PROC FAR
    CMP     AX, 9999          ;AX>9999,则 CF 置 1
    JBE     TRAN
    STC
    JMP     EXIT
TRAN:  PUSH  CX
    PUSH  DX
    SUB    DX, DX            ;DX 清 0
    MOV    CX, 1000          ;计算千位数
    DIV    CX
    XCHG   AX, DX            ;商在 DX 中,余数在 AX 中
    MOV    CL, 4
    SHL    DX, CL            ;DX 左移 4 位
    MOV    CL, 100           ;计算百位数
    DIV    CL
    ADD    DL, AL             ;百位数加到 DL 中,DX 左移 4 位
    MOV    CL, 4
    SHL    DX, CL
    XCHG   AL, AH            ;余数保留在 AL 中
    SUB    AH, AH
    MOV    CL, 10            ;计算十位数
    DIV    CL
    ADD    DL, AL            ;十位数加到 DL 中,DX 左移 4 位
    MOV    CL, 4
    SHL    DX, CL
    ADD    DL, AH            ;加个位数
    MOV    AX, DX            ;结果送到 AX 中
    POP    DX
    POP    CX
EXIT:  RET
B2TOBCD ENDP
```


例 4-79 将十进制数的 ASCII 字符串转换成有符号二进制数算法:首先检测字符串为正还是为负,若是为负,将标识符 MINU 置 1,然后进行字符串转换,转换结束判别标识符 MINU,若 MINU 是 1,把 AX 中的数取补。字符串转换方法采用累加和(DX)乘 10,然后读取下一个字符并转换成二进制数加到累加和中。

子程序名:AS2BIN

输入参数:DX=字符串的偏移量,CX=字符串的字符个数。

输出参数:CF=0,则 AX=二进制数。CF=1,则字符串非法,AX=0。

使用寄存器:SI,BX

```
AS2BIN    PROC    FAR
          PUSH    CX
          PUSH    DX
          PUSH    SI
          MOV     SI, DX           ;DX 存到 SI 中
          CMP     CX, 6
          JA      ERR
          CLD
          MOV     AL, [SI]        ;查符号
          CMP     AL, '-'
          JNE     CHK
          MOV     MINU, 1
          JMP     DECS
CHK:       CMP     AL, '+'
          JNE     CLRD
DECS:     DEC     CX
          INC     SI
CLRD:     SUB     DX, DX          ;DX 清 0,放结果
NEXT:     CALL    CHE            ;乘 10
          JC      ERR
          LODSB                    ;读下一个字符到 AL
          CMP     AL, '0'         ;字符不在 0~9 之间,出错
          JB      ERR
          CMP     AL, '9'
          JA      ERR
          AND     AX, 000FH       ;转换成二进制数
          ADD     DX, AX
          JC      ERR            ;累加和超过 65535 出错
          LOOP    NEXT
          MOV     AX, DX          ;结果送入 AX
          CMP     MINU, 1        ;负数求补
          JNE     EXIT
          NEG     AX
          JMP     EXIT
ERR:      STC                    ;字符串为非法
          MOV     AX, 0
```

```

EXIT;    POP    SI
         POP    DX
         POP    CX
         RET
CHE      PROC    NEAR          ;乘 10 子程序
         PUSH   BX
         MOV    BX, DX
         SHL    DX, 1
         SHL    DX, 1
         ADD    DX, BX
         SHL    DX, 1
         POP    BX
         RET
CHE      ENDP
AS2BIN   ENDP

```

2. 算术运算

例 4-80 完成两个多位十进制数相乘,并将结果送屏幕显示。

该程序在代码段中设置了三个过程,MMUL 过程完成两个多位十进制数相乘子程序并输出结果。MULS 过程完成两个一位十进制数相乘,结果以非压缩型 BCD 码形式保留在 AX 中。DOUT 过程完成累加部分积,采用过程嵌套调用的方法。

```

NAME     MULT
DATA     SEGMENT                                ;数据段
X        DB  1,5,6,7                          ;定义被乘数
N1       EQU  $-X
Y        DB  1,2,3,4                          ;定义乘数
N2       EQU  $-Y
Z        DB  30H  DUP(?)
P        DB  '4567×1234='
Q        DB  30H  DUP(?)
DATA     ENDS
STACK    SEGMENT  STACK                        ;堆栈段
DW       50H  DUP(?)
TOP      LABEL WORD
STACK    ENDS
CODE     SEGMENT                                ;代码段
ASSUME   CS, CODE,  DS, DATA,  SS, STACK
MAIN     PROC    FAR
START:   MOV     AX, DATA
         MOV     DS, AX
         MOV     AX, STACK
         MOV     SS, AX
         MOV     SP, OFFSET TOP
         MOV     CL, 4

```

	MOV	SI, 0	
	MOV	BX, N2	
LOOP1:	MOV	CH, Y[BX-1]	
	CALL	MMUL	
	INC	SI	
	DEC	BX	
	JNZ	LOOP1	
	MOV	SI, DX	
	MOV	BX, 0	
ASC:	MOV	AL, Z[SI]	
	AND	AL, 0FH	
	OR	AL, 30H	
	MOV	Q[BX], AL	
	INC	BX	
	DEC	SI	
	CMP	SI, 0	
	JNL	ASC	
	MOV	Q[BX], '\$'	
DISP:	MOV	AH, 9	
	MOV	DX, OFFSET P	
	INT	21H	
	MOV	AH, 4CH	
	INT	21H	; 主程序结束
	RET		
MAIN	ENDP		
MMUL	PROC	NEAR	; 两个多位十进制数相乘, 并输出结果
	PUSH	SI	
	PUSH	BX	
	MOV	BX, N1	
LOOP2:	MOV	AL, X[BX-1]	
	CALL	MULS	
	CALL	DOUT	
	INC	SI	
	DEC	BX	
	JNZ	LOOP2	
	POP	BX	
	POP	SI	
	RET		
MMUL	ENDP		
MULS	PROC	NEAR	; 两个一位十进制数相乘, 结果以
	PUSH	BX	; 非压缩型 BCD 码形式保留在 AX 中
	MOV	BL, CH	
	AND	AL, 0FH	
	AND	BL, 0FH	
	MUL	BL	

```

                AAM
                POP     BX
                RET
MULS           ENDP
DOUT           PROC     NEAR                ;累加部分积
                PUSH    SI
NEXT:          ADD     AL, Z[SI]
                AAA
                MOV     Z[SI], AL
                MOV     AL, AH
                XOR     AH, AH
                INC     SI
                MOV     DX, SI
                CMP     AL, 0
                JNZ     NEXT
                POP     SI
                RET
DOUT           ENDP
CODE           ENDS
                END     START

```

3. DOS 和 BIOS 功能调用

例 4-81 画图程序,实现在键盘上按↑↓→←键可以不同方向在屏幕上画线,并通过 F1 和 F2 能变换图的前景色和背景色,按 ESC 键返回 DOS。

```

.MODEL  SMALL
.386
.STACK
.DATA
    TOPROW  EQU  0000H    ;行的范围是 0000H~00C8H
    RGTCOL  EQU  0280H
    LETCOL  EQU  0000H    ;列的范围是 0000H~0140H
    BOTROW  EQU  01E0H
    COLORF  DB  ?        ;前景色
    COLORB  DB  ?        ;背景色
.CODE
.STARTUP
    MOV     AH, 0FH
    MOV     BH, 0
    INT     10H           ;得到当前显示类型
    PUSH    AX            ;将当前显示方式入栈保存
    MOV     AH, 0
    MOV     AL, 12H
    INT     10H           ;设置显示方式为彩色方式 13H: 640 * 480, 16 色
    MOV     COLORB, 00H   ;初始化

```

	MOV	COLORF, 05H	
	MOV	BH, 0	
	MOV	AH, 0BH	
	MOV	BL, COLORB	
	INT	10H	
	MOV	DX, 200	;列
	MOV	CX, 200	;行
	MOV	AH, 0CH	
	MOV	AL, COLORF	
	INT	10H	
READY:	MOV	AH, 10H	
	INT	16H	;等待键盘输入
	CMP	AH, 01H	;01H 是 'ESC' 的键盘扫描码
	JE	STOP	
	CMP	AH, 3BH	;背景色
	JE	BSET	
	CMP	AH, 3CH	;前景色
	JE	FSET	
	CMP	AH, 4DH	;向右画图
	JE	DRAW1	
	CMP	AH, 4BH	;向左画图
	JE	DRAW2	
	CMP	AH, 48H	;向上画图
	JE	DRAW3	
	CMP	AH, 50H	;向下画图
	JE	DRAW4	
	JMP	READY	;输入其它键时重新扫描键盘
BSET:	INC	COLORB	;修改背景色
	MOV	AH, 0BH	
	MOV	BL, COLORB	
	MOV	BH, 0	
	INT	10H	
	JMP	READY	
FSET:	INC	COLORF	;修改前景色
	JMP	READY	
DRAW1:	INC	CX	
	CMP	CX, RGTCOL	;判断是否到达右边界
	JG	SDR1	
	JMP	DISP	
SDR1:	DEC	CX	;到达右边界时,光标不动
	JMP	READY	
DRAW2:	DEC	CX	
	CMP	CX, LETCOL	;判断是否到达左边界
	JL	SDR2	
	JMP	DISP	

```

SDR2:   INC     CX           ;到达左边界时,光标不动
        JMP     READY
DRAW3:  DEC     DX
        CMP     DX, TOPROW
        JL      SDR3
        JMP     DISP
SDR3:   INC     DX
        JMP     READY
DRAW4:  INC     DX
        CMP     DX, BOTROW
        JG      SDR4
        JMP     DISP
SDR4:   DEC     DX
        JMP     READY
DISP:   MOV     BL, COLORF
        MOV     AH, 0CH
        MOV     AL, COLORF
        INT     10H
        JMP     READY
STOP:   POP     AX           ;原始显示方式出栈
        MOV     AH, 0
        INT     10H         ;恢复原始显示方式
        .EXIT 0
END

```

例 4-82 音乐程序。

本程序利用 8254 定时器来驱动扬声器,当 8255A 并行接口芯片输出端口 61H 控制扬声器为接通状态就可以发出一定频率的声音,实现在扬声器里播放一段设定的乐曲。具体可以分为四个步骤:

- 1) 为演奏的乐曲定义一个频率表和一个节拍时间表;
- 2) 分别将两个表的偏移地址放入 SI 和 BP;
- 3) 从表中取出音符的频率放入 DI,取出音符的持续时间(10Ms 的倍数)放入 BX;
- 4) 使用 8254 的 2 号定时器来驱动扬声器发出相应频率的声音。

```

STACK      SEGMENT
           STR  DW  50 DUP(?)
STACK      ENDS
DATA       SEGMENT
FREQUENCY  DW      330,330,349,392,392,349,330,294           ;乐曲的频率表
           DW      262,262,294,330,330,294,294
           DW      330,330,349,392,392,349,330,294
           DW      262,262,294,330,294,262,262
           DW      294,294,330,262,294,330,349,330,262
           DW      294,330,349,330,294,262,294,196
           DW      330,330,349,392,392,349,330,349,294

```

```

        DW      262,262,294,330,294,262,262,0

TIME    DW      6000,6000,6000,6000,6000,6000,6000,6000      ;乐曲节拍时间表
        DW      6000,6000,6000,6000,12000,3000,9000
        DW      6000,6000,6000,6000,6000,6000,6000,6000
        DW      6000,6000,6000,6000,12000,3000,9000
        DW      6000,6000,6000,6000,6000,3000,3000,6000,6000
        DW      6000,3000,3000,6000,6000,6000,6000,6000
        DW      12000,6000,6000,6000,6000,6000,6000,3000,3000
        DW      6000,6000,6000,6000,12000,3000,9000

DATA    ENDS

CODE    SEGMENT
ASSUME  CS: CODE, DS: DATA, SS: STACK
START:  MOV     AX, STACK
        MOV     SS, AX
        MOV     AX, DATA
        MOV     DS, AX
        MOV     SI, OFFSET FREQUENCY      ;获得存储频率的偏移地址
        MOV     BP, OFFSET TIME           ;获得存储节拍时间的偏移地址
        CALL    PLAY                      ;音乐子程序调用
        MOV     AH, 4CH                   ;程序结束
        INT     21H

PLAY    PROC                                     ;播放音乐的子程序
FREQ:   MOV     DI, [SI]                   ;获得需要发声音符的频率放入 DI,
                                           ;0 表示音乐结束

        CMP     DI, 0
        JE      ENDPLAY
        MOV     BX, DS:[BP]               ;获得音符的持续时间放入 BX
        CALL    SOUND                     ;发声子程序调用
        ADD     SI, 2                     ;使 SI 指向下一个要播放的频率
        ADD     BP, 2                     ;使 BP 指向下一个要播放的频率
        JMP     FREQ

ENDPLAY: RET
PLAY    ENDP

SOUND   PROC                                     ;使扬声器发声的子程序
        MOV     AL, 0B4H
        OUT     43H, AL                   ;初始化 8254 的 2 号定时器
        MOV     DX, 14H
        MOV     AX, 4F38H                 ;533H * 896
        DIV     DI                         ;获得定时常数放在 AX 中
        OUT     42H, AL                   ;写入二号定时器的低 8 位
        MOV     AL, AH

```


	OUT	42H, AL	;写入二号定时器的高8位
	IN	AL, 61H	;从61号端口读取数据
	MOV	AH, AL	
	OR	AL, 3	;打开扬声器
	OUT	61H, AL	
WATING;	MOV	CX, 0CFFFH	;延时长度
DELAY;	LOOP	DELAY	;每个音符之间的延时
	DEC	BX	
	JNZ	WATING	
	MOV	AL, AH	
	OUT	61H, AL	
	RET		
SOUND	ENDP		
CODE	ENDS		
END	START		

4-6 宏汇编和条件汇编

一、宏汇编

在汇编语言程序设计中,有的程序段要多次使用,除了以前谈到的过程调用的方法外,还可以用宏汇编的方法实现,尤其在子程序段本身较短,而传递的参数较多的情况下,使用宏汇编更加有效。宏是源程序中一段独立的程序段,首先对它进行定义,然后就可用宏指令语句多次调用它了。

1. 宏定义

指令使用前必须先进行宏定义,宏定义格式为:

```
宏指令名      MACRO 形式参数,形式参数,...
                <宏体>
                ENDM
```

宏指令名:宏定义的名字,不可缺省,宏调用时要使用它,第一个符号必须是字母,其后可以是字母或数字。

MACRO...ENDM:宏定义伪指令助记符,不可缺省。它们成对出现,表示宏定义的开始和结束,ENDM 前不带宏指令名。

宏体:一段有独立功能的程序代码段。

形式参数:又称哑元,各个哑元之间用逗号隔开,可以缺省。

2. 宏调用

经宏定义后的宏指令可以在源程序中调用,宏调用格式为:

```
宏指令名      实参,实参...
```

宏调用只需要有宏指令名,若宏定义中有形参,那么宏调用时必须带有实际参数来替代

形参,实际参数的个数,顺序,类型与形参一一对应,各个实参之间用逗号分开。原则上实参的个数与形参的个数相等,但汇编程序不要求它们必须相等,若实参个数大于形参个数,则多余的实参不予考虑,若实参个数小于形参个数,则多余的形参作“空”处理。

3. 宏展开

汇编程序在对源程序汇编时,对每个宏调用作宏展开,即用宏定义中的宏体取代宏指令名,并用实参一一对应代替形参(即实元取代哑元),每条插入的宏体指令前带上加号“+”。下面举例说明宏定义,宏调用及宏展开。

例 4-83 不带参数的宏定义,用宏指令来实现将 AL 中内容右移 4 位。

宏定义:

```
SHIFT    MACRO
          MOV    CL, 4
          SAR     AL, CL
          ENDM
```

宏调用: SHIFT

宏展开:将下段程序插入宏调用语句位置上。

```
+MOV    CL, 4
+SAR     AL, CL
```

4. 宏调用中参数传递

宏定义中的参数可以有多个,实参可以是数字,寄存器或操作码。

例 4-84 宏定义带一个参数,用宏指令实现将 AL 中内容右移任意次(<256)。

宏定义:

```
SHIFT    MACRO N
          MOV    CL, N
          SAR     AL, CL
          ENDM
```

宏调用 1: SHIFT 4

宏调用 2: SHIFT 7

宏展开 1: +MOV CL, 4 ;AL 中内容算术右移 4 次
 +SAR AL, CL

宏展开 2: +MOV CL, 7 ;AL 中内容算术右移 7 次
 +SAR AL, CL

例 4-85 宏定义带 2 个参数,用宏指令实现将任意寄存器内容,右移位任意次(小于 256)。

宏定义:

```
SHIFT    MACRO N, M
          MOV    CL, N
          SAR     M, CL
          ENDM
```

宏调用 1: SHIFT 4, AL

宏调用 2: SHIFT 6, DI

宏展开 1:	+MOV CL, 4	;AL 中内容算术右移 4 次
	+SAR AL, CL	
宏展开 2:	+MOV CL, 6	;DI 中内容算术右移 6 次
	+SAR DI, CL	

例 4-86 宏定义带 3 个参数,参数可为操作码,用宏指令实现对寄存器的内容左移或右移任意次。

宏定义:

```
SHIFT      MACRO  N, M, P
            MOV     CL, N
            S&P     M, CL
            ENDM
```

宏调用 1:	SHIFT 3, AX, HR	
宏调用 2:	SHIFT 5, BH, AL	
宏展开 1:	+MOV CL, 3	;AX 中内容逻辑右移 3 位
	+SHR AX, CL	
宏展开 2:	+MOV CL, 5	;BH 中内容算术左移 5 位
	+SAL BH, CL	

宏定义可用部分操作码作参数,但在宏定义体中必须用“&”作分隔符,& 是一个操作符,它在宏定义体中可作为哑元的前缀,宏展开时,可以把 & 前后两个符号合并成一个符号。

例 4-87

宏定义:

```
SJP      MACRO  X, Y, Z, W
            MOV     AX, X
            C&W     AX, Y
            J&Z     NEXT
            ENDM
```

宏调用:	SJP BX, SI, NZ, MP
宏展开:	+MOV AX, BX
	+CMP AX, SI
	+JNZ NEXT

例 4-88

宏定义:

```
SLL      MACRO  P
            JMP     WA&P
            ENDM
```

宏调用:	SLL Q
宏展开:	+JMP WAQ

例 4-89 若没有 & 连接,宏展开成+JMP WAP,汇编程序将 WAP 看成一个标号,而不是将 P 看成一个哑元,这样程序跳转位置出错。

5. 宏定义嵌套

在宏定义中允许使用宏调用,但必须先定义后调用。

例 4-89

宏定义:

```
DBF      MACRO  P, Q
          MOV    BX, P
          ADD    AX, Q
          ENDM
```

```
DBFS     MACRO  X1, X2, X3
          PUSH   AX
          PUSH   BX
```

```
DBF      X1, X2
          MOV    X3, AX
          POP    BX
          POP    AX
          ENDM
```

宏调用: DBFS 3AC0H, BX, [3AC4H]

宏展开:

```
+PUSH AX
+PUSH BX
+DBF X1, X2           ;此语句不占内存
+MOV BX, 3AC0H        ;DBF 宏定义展开
+ADD AX, BX
+MOV [3AC4H], AX
+POP BX
+POP AX
```

宏定义允许嵌套,即宏定义体内也可以包含宏定义。

例 4-90

宏定义:

```
DEFM     MACRO  MACN, OPER
MACN     MACRO  A, B, C
          PUSH   AX
          MOV    AX, A
          OPER   AX, B
          MOV    C, AX
          POP    AX
          ENDM
        ENDM
```

其中 MACN 为内层宏定义名,又是外层宏定义的元素,当调用 DEFM 时就形成一个宏定义。

宏调用: DEFM ADDIT, ADD

宏展开:

```
+ADDIT MACRO A, B, C ;形成加法宏定义
PUSH AX
MOV AX, A
```

```

ADD    AX, B
MOV    C, AX
POP    AX
ENDM

```

若宏调用为: `DEFM SUBT, SUB`

则宏展开: `+SUBT MACRO A, B, C` ;形成减法宏定义

```

PUSH    AX
MOV    AX, A
SUB    AX, B
MOV    C, AX
POP    AX
ENDM

```

形成内层宏定义后,再使用宏调用。

宏调用: `ADDIT 1000, BX, CX`

宏展开: `+PUSH AX`
`+MOV AX, 1000`
`+ADD AX, BX`
`+MOV CX, AX`
`+POP AX`

6. 其它宏指令

(1) 取消宏定义语句

格式为: `PURGE 宏指令名, 宏指令名...`

`PURGE`:伪指令助记符,不可缺省,因为经过定义的宏指令名,不允许重新定义,必须用 `PURGE` 语句将其取消后,才能重新定义,此语句一次可以取消多个宏指令名。

宏指令名:需要取消的宏指令名,有多个宏指令名时,用逗号“,”将它们分开。

例 4-91

宏定义:

```

SUB    MACRO S1, S2, S3
      ADD    AX, S1
      XOR    AX, S2
      MOV    S3, AX
      ENDM

```

宏调用: `SUB AX, BX, DI`
`PURGE SUB`

上例中 `SUB` 宏指令名与指令助记符相同,因宏指令优先,使同名的指令或伪操作失效。宏调用后,用 `PURGE` 取消宏定义,恢复 `SUB` 的指令含义。

(2) 重复执行宏指令语句

汇编语言需要连续重复完成相同的操作,可以使用重复执行伪指令。格式为:

```

REPT    <表达式>
      <宏体>

```

ENDM

REPT...ENDM: 伪指令助记符, 必须成对出现, 不可省略。

宏体: 需要重复的指令体。

表达式: 重复次数。

例 4-92 将 1 到 10 分配给连续的 10 个存储单元。

```
                X=0
REPT            10
                X=X+1
                DB  X
            ENDM
汇编后产生:    +DB  1
                +DB  2
                ⋮
                +DB 10
```

(3) 带参数的重复执行宏指令

格式为:

```
IRP  形式参数  <参数表>
      <宏体>
      ENDM
```

IRP...ENDM: 伪指令助记符, 必须成对出现, 不可省略。

宏体: 要重复的指令体, 重复次数由参数个数决定。

参数表: 每次重复时所取的参数, 参数表用尖括号< >括起来。每次重复, 依次取参数表中一项, 代入宏体中哑元。

例 4-93

```
IRP            REG    <AX, BX, CX, DX>
                PUSH  REG
            ENDM
汇编后:        +PUSH  AX
                +PUSH  BX
                +PUSH  CX
                +PUSH  DX
```

(4) 带字符串重复执行宏指令

格式为:

```
IRPC 形式参数  <字符串>
      <宏体>
      ENDM
```

IRPC...ENDM: 伪指令助记符, 必须成对出现, 不可省略。

宏体: 重复执行的指令体, 每次重复时依次用字符串中字符代替形式参数, 重复次数取决于字符串中字符的个数。

字符串;可用尖括号也可不用尖括号括起来。

例 4-94

```
IRPC      X, <HELLO>
          DB      X
          ENDM
汇编后:   +DB      48H
          +DB      45H
          +DB      4CH
          +DB      4CH
          +DB      4FH
```

展开后将 HELLO 的 ASCII 字符放入存储器当前地址的 5 个连续单元中。

7. 宏指令与子程序的区别

宏汇编是用一条宏指令来代替一个程序段,可有效地缩短源程序的书写长度,且格式清晰,调用方便。在某种意义上,过程调用也有类似的功能,但二者之间有明显的区别,主要区别在以下几个方面:

(1) 过程调用使用 CALL 语句,由 CPU 执行,宏指令调用由宏汇编程序 MASM 中宏处理程序来识别。

(2) 过程调用时,每调用一次都要保留程序的断点和保护现场,返回时要恢复现场和恢复断点,增加了操作时间,执行速度慢。而宏指令调用时,不需要这些入栈及出栈操作,执行速度较快。

(3) 过程调用的子程序与主程序分开独立存在,经汇编后在存储器中只占有一个子程序段的空间,主程序转入此处运行,因此目标代码长度短,节省内存空间。而宏调用是在汇编过程中展开,宏调用多少次,就插入多少次,因此目标代码长度长,占内存空间多;

(4) 一个子程序设计,一般完成某一个功能,多次调用完成相同操作,仅入口参数可以改变,而宏指令可以带哑元,调用时可以用实元取代,使不同的调用完成不同的操作,增加使用的灵活性。

如果多次调用的程序较长,速度要求不高,适宜用过程调用方法,如果多次调用的程序较短,而操作又希望修改,适宜用宏指令语句。

二、条件汇编

条件汇编是对给定的条件进行测试,汇编程序根据测试结果,将一段程序嵌入源程序汇编或不进行汇编,它的一般格式为:

```
IF      条件 (表达式)
      (指令体 1)                ;条件为真汇编指令体 1
ELSE    (指令体 2)                ;条件为假汇编指令体 2
ENDIF
```

IF...ENDIF:条件汇编伪指令助记符,必须成对出现,不可省略。

条件:测试条件,条件为真汇编 IF...ENDIF 中指令体,否则不汇编。

ELSE:选择命令,条件为真汇编指令体 1,条件为假汇编指令体 2,可以省略。

条件汇编的形式有下列几种:

语句	条 件	说 明
IF	表达式	表达式的值不等于 0,条件满足
IFE	表达式	表达式的值等于 0,条件满足
IFDEF	符号	符号已定义或被说明为外部符号,条件满足
IFNDEF	符号	符号未定义或未通过外部说明,条件满足
IFB	参数	参数为空,条件满足
IFNB	参数	参数不为空,条件满足
IFIDN	字符串 1,字符串 2	字符串 1 和字符串 2 相同,条件满足
IFDIF	字符串 1,字符串 2	字符串 1 和字符串 2 不相同,条件满足

条件伪指令可以在宏定义体内,也可以在宏定义体外,允许嵌套任意次。

例 4-95 可以根据不同情况,产生无条件转移指令,或条件转移指令。

宏定义:

```

GOTO    MACRO  L, X, REL, Y
        IFB    <REL>
        JMP    L
ELSE
        MOV    AX, X
        CMP    AX, Y
        J&REL  L
        ENDIF
        ENDM
宏调用:  GOTO    LOOP, SUM, NZ, 15
        :
        GOTO    EXIT
宏展开:  +MOV    AX, SUM
        +CMP    AX, 15
        +JNZ    LOOP
        :
        +JMP    EXIT

```

例 4-96 用条件汇编来结束宏定义中递归调用,将 AX 左移 N 次,用 COUNT 计算递归次数,当 COUNT 与 N 相等时就结束递归调用。

宏定义:

```

MUER    MACRO  X, N
        SAL    X, 1
        INC    COUNT
        IF     COUNT=N
        MUER   X, N
        ENDIF
        ENDM

```


宏调用:	MUER	AX, 3	;COUNT 初始值为 0
宏展开:	+SAL	AX, 1	
	+INC	COUNT	
	+SAL	AX, 1	
	+INC	COUNT	
	+SAL	AX, 1	

习 题

1. 下列变量各占多少字节?

```

A1    DW    23H, 5876H
A2    DB    3  DUP (?), 0AH, 0DH, '$'
A3    DD    5  DUP (1234H, 567890H)
A4    DB    4  DUP (3 DUP (1, 2, 'ABC'))

```

2. 下列指令完成什么功能?

```

MOV  AX, 00FFH AND 1122H+3344H
MOV  AL, 15 GE 1111B
MOV  AX, 00FFH LE 255+6/5
AND  AL, 50 MOD 4
OR   AX, 0F00FH AND 1234 OR 00FFH

```

3. 有符号定义语句如下:

```

BUF  DB    3, 4, 5, '123'
ABUF DB    0
L     EQU  ABUF-BUF

```

求 L 的值为多少?

4. 假设程序中的数据定义如下:

```

PAR      DW    ?
PNAME    DB    16 DUP (?)
COUNT   DD    ?
PLENTH   EQU   $-PAR

```

求 PLENTH 的值为多少? 表示什么意义?

5. 对于下面的数据定义,各条 MOV 指令执行后,有关寄存器的内容是什么?

```

DA1  DB    ?
DA2  DW    10  DUP (?)
DA3  DB    'ABCD'
      MOV  AX, TYPE DA1
      MOV  BX, SIZE DA2
      MOV  CX, LENGTH DA3

```


6. 下段程序完成后, AH 等于什么?

```
IN    AL, 5FH
TEST  AL, 80H
JZ    L1
MOV   AH, 0
JMP   STOP
L1:   MOV AH, 0FFH
STOP: HALT
```

7. 编程序完成下列功能:

(1) 利用中断调用产生 5 秒延时;

(2) 利用中断调用, 在屏幕上显示 1~9 之间随机数。

8. 编两个通用过程完成将 AX 中存放的二进制数转换成压缩型 BCD 码以及将 BCD 码转换成二进制数。

9. 编写两个通用过程, 一个完成 ASCII 码转换成二进制数功能, 另一个完成 ASCII 字符打印输出功能。

10. 编制两个通用过程, 完成十六进制数转换成 ASCII 码并将 ASCII 码字符显示。

11. 某程序可从键盘接收命令(0~5), 分别转向 6 个子程序, 子程序入口地址分别为 P0~P5, 编制程序, 用跳转表实现分支结构。

12. 在首地址为 TABLE 的数组中按递增次序存放着 100 个 16 位补码数, 编写一个程序, 把出现次数最多的数及其出现次数分别存放于 AX 和 BL 中。

13. 将键盘上输入的十六进制数转换成十进制数, 在屏幕上显示。

14. 将 AX 中的无符号二进制数转换成 ASCII 字符串表示的十进制数。

15. 从键盘输入 20 个有符号数, 将它们排序并在屏幕上显示。

16. 编写多字节有符号数的加法程序, 从键盘接收两个加数, 在屏幕上显示结果。

17. 编写 2 位非压缩型 BCD 码相乘的程序。

18. 编写完整的程序求 N!, 求 N 大于 6 时的运算结果, 并在屏幕上显示结果。

19. 在附加段中有一个数组, 首地址为 BUFF, 数组中第一个字节存放了数组的长度。编一个程序在数组中查找 0, 找到后把它从数组中删去, 后续项向前压缩, 其余部分补 0。

20. 编程完成将第二个字符串插入到第一个字符串的指定位置上。

21. 将学生的班级、姓名、学号、课程名、成绩定义为一个结构, 用结构预置语句, 产生 5 个学生的成绩登记表, 编程序将成绩小于 60 分的学生姓名, 成绩显示出来。

22. 编程序统计学生的数学成绩, 分别归类 90~99 分, 80~89 分, 70~79 分, 60~69 分及 60 分以下, 并将各段的人数送入内存单元中。

23. 编制宏定义, 将存储器区中一个用 '\$' 结尾的字符串传送到另一个存储器区中, 要求源地址、目的地址、串结尾符号可变。

24. 定义宏指令名 FINSUM: 它完成比较两个数 X 和 Y, 若 $X > Y$, 执行 $X + 2 * Y$ 结果送到 SUM, 若 $X \leq Y$, 执行 $2 * X + Y$ 结果送到 SUM。

25. DOS 功能调用需要在 AH 寄存器中存放不同的功能码, 试将这种功能调用定义成宏指令 DOS, 再定义宏指令 DISP, 完成显示字符的功能, 并展开宏调用 DISP, '*'。

26. 编一段程序产生乐曲。

第五章 存储器

5-1 存储器分类

存储器是计算机的主要组成部分之一,是用来存储程序和数据部件,存储器表征了计算机的“记忆”功能,存储器的容量越大,计算机的性能也就越好。

存储器可以按不同方法进行分类。

一、按用途分类

按存储器用途分类,可以分成内部存储器和外部存储器。

1. 内部存储器

内部存储器也称为内存,是主存储器,位于计算机主机的内部,用来存放当前正在使用的或经常使用的程序和数据,CPU 可以直接对它进行访问。内存的存取速度较快,一般是用半导体存储器件构成。

内存的容量大小受到地址总线位数的限制,对 8086 系统,20 条地址总线,可以寻址内存空间为 1MB。80386 系统,地址总线为 32 条,可以寻址 4GB,PⅡ 以上微机地址总线为 36 位,可寻址 64GB。在实际使用中,由于内存芯片价格较贵,目前微型计算机系统中配置 256MB~512MB 的内存容量,这样许多程序和数据要存放在磁盘外存中,使用时再调到内存。正是由于内存的快速存取和容量较小的特点,它用来存放系统软件,如系统引导程序、监控程序或者操作系统中的 ROM BIOS,以及当前要运行的应用软件。

2. 外部存储器

外部存储器也称为外存,是辅助存储器。外存的特点是大容量,所存储的信息既可以修改,也可以保存,存取速度较慢,要由专用的设备来管理。

由于内存容量的限制,一部分系统软件必须常驻内存,另外一些系统软件和应用软件则在用到时,再由外存传送到内存,此外还有一些程序或数据需要长期保存。构成内存的器件是不能实现这个功能的,因此设计出各种外部存储器,它的容量不受限制,也称为海量存储器。一般外部存储器由磁表面存储器件构成,存取速度较慢。在微型计算机中,常见的外存有软盘、硬盘、光盘、微缩胶片等。但外存要配置专门的驱动设备才能完成对它的访问功能,比如要配备软盘驱动器、硬盘驱动器、光盘驱动器等驱动设备。

计算机工作时,一般由内存 ROM 中的引导程序启动系统,再从外存中读取系统程序 and 应用程序,送到内存的 RAM 中,程序运行的中间结果放在 RAM 中,(内存不够时也放在外存中),程序结束时将最后结果存入外部存储器。

二、按存储器性质分类

内存按存储器性质分类通常分为随机存取存储器(RAM)和只读存储器(ROM)。

1. RAM 随机存取存储器(Random Access Memory)

CPU 能根据 RAM 的地址将数据随机地写入或读出,电源切断后,所存数据全部丢失。通常我们所说的计算机内存容量有多少字节,均是指 RAM 存储器的容量。按照集成电路内部结构的不同,RAM 又分为两种:

(1) SRAM 静态 RAM(Static RAM)

静态 RAM 速度非常快,只要电源存在内容就不会自动消失。但它的基本存储电路为 6 个 MOS 管组成 1 位,因此集成度相对来说较低,功耗也较大。一般,高速缓冲存储器(Cache memory)用它组成。

(2) DRAM 动态 RAM(Dynamic RAM)

DRAM 的内容在 10^{-3} 或 10^{-6} 秒之后自动消失,因此必须周期性的在内容消失之前进行刷新(Refresh)。由于它的基本存储电路由一个晶体管及一个电容组成,因此它的集成度高,成本较低,另外耗电也少,但它需要一个额外的刷新电路。DRAM 运行速度较慢,SRAM 比 DRAM 要快 2~5 倍,一般,PC 机的标准存储器都采用 DRAM 组成。

2. ROM 只读存储器(Read Only Memory)

ROM 存储器是将程序及数据固化在芯片中,数据只能读出,不能写入,电源关掉,数据也不会丢失,ROM 中通常存储操作系统的程序(BIOS)或用户固化的程序。

ROM 按集成电路内部结构的不同,可分为下面三种:

(1) PROM 可编程 ROM(Programmable ROM)

将设计的程序固化进去后,ROM 内容不可更改。

(2) EPROM 可擦除、可编程 ROM(Erasable PROM)

可编程固化程序,且在程序固化后可通过紫外光照擦除,以便重新固化新数据。

(3) EEPROM 电可擦除可编程 ROM(Electrically Erasable PROM)

可编程固化程序,并可利用电来擦除芯片内容,以重新编程固化新数据。

图 5-1 给出了存储器的分类,其中光盘不是磁表面存储器件。

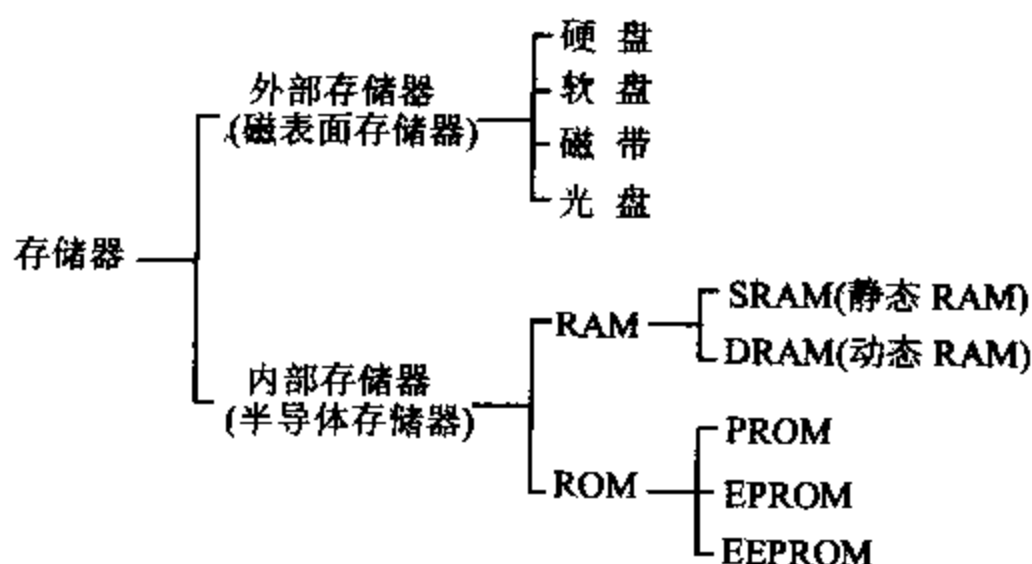


图 5-1 存储器的分类

不同存储器芯片,存取速度不相同,因此在选择存储器芯片时要考虑几个方面:

- (1) 只读存储器还是随机存储器。
- (2) 芯片位容量,它是表示存储功能的指标。
- (3) 存取时间,即访问存储器的时间。
- (4) 功耗,CMOS 器件功耗低,但速度慢,HMOS 的存储器件在速度、功耗、容量方面进行了折衷。
- (5) 价格

5-2 随机存取存储器 RAM

随机存取存储器(RAM)可以随时在任意位置上存取信息,根据存储器芯片内部基本单元电路的结构,RAM 分为静态 RAM 及动态 RAM。

一、静态随机存取存储器(SRAM)

1. 静态 RAM 的构成

静态 RAM 存储一位信息的单元电路可以用双极型器件构成,也可用 MOS 器件构成。双极型器件构成的电路存取速度快,但工艺复杂,集成度低,功耗大,一般较少使用这种电路,而采用 MOS 器件构成的电路。静态 RAM 的单元电路通常是由 6 个 MOS 管子组成的双稳态触发器电路,可以用来存储信息“0”或者“1”,只要不掉电,“0”或“1”状态能一直保持,除非重新通过写操作写入新的数据。同样对存储器单元信息的读出过程也是非破坏性的,读出操作后,所保存的信息不变。使用静态 RAM 的优点是访问速度快,访问周期达 20ns~40ns。静态 RAM 工作稳定,不需要进行刷新,外部电路简单,但基本存储单元所包含的管子数目较多,且功耗也较大,它适合在小容量存储器中使用。

静态 RAM 通常由地址译码器,存储矩阵,控制逻辑和三态数据缓冲器组成,存储器芯片内部结构框图如图 5-2 所示。

(1) 存储矩阵

一个基本存储单元存放一位二进制信息,一块存储器芯片中的基本存储单元电路按字结构成位结构的方式排列成矩阵。按字结构方式排列时,读/写一个字节的 8 位制作在一块芯片上,若选中则 8 位信息从一个芯片中同时读出,但芯片封装时引线较多。例如 1K 的存储器芯片由 128×8 组成,访问它要 7 根地址线和 8 根数据线。

位结构是 1 个芯片内的基本单元作不同字的同一位,片内按矩阵排列,8 位由 8 块芯片组成。优点是芯片封装时引线较少,例如 1K 存储器芯片由 1024×1 组成,访问它要 10 根地址线和 1 根数据线,但使用芯片为 8 块。封装引线数减少,成品合格率就会提高。

(2) 地址译码器

CPU 读/写一个存储单元时,先将地址送到地址总线,高位地址经译码后产生片选信号选中芯片,低位地址送到存储器。由地址译码器译码选中所需要的片内存储单元,最后在读/写信号控制下将存储单元内容读出或写入。

地址译码器完成存储单元的选择,通常有线性译码和复合译码两种方式,一般采用复合译码。如 1024×1 的位结构芯片排列成 32×32 矩阵, $A_0 \sim A_4$ 送到 X 译码器(行译码), $A_5 \sim A_9$ 送到 Y 译码器(列译码)。如图 5-2 所示, X 和 Y 译码器各输出 32 根线,由 X 和 Y 方向同时选中的单元为所访问的存储单元。若采用线性译码器,10 根地址线输入到地址译码器后,有 1024 根输出线来选择存储单元,结构复杂化了。

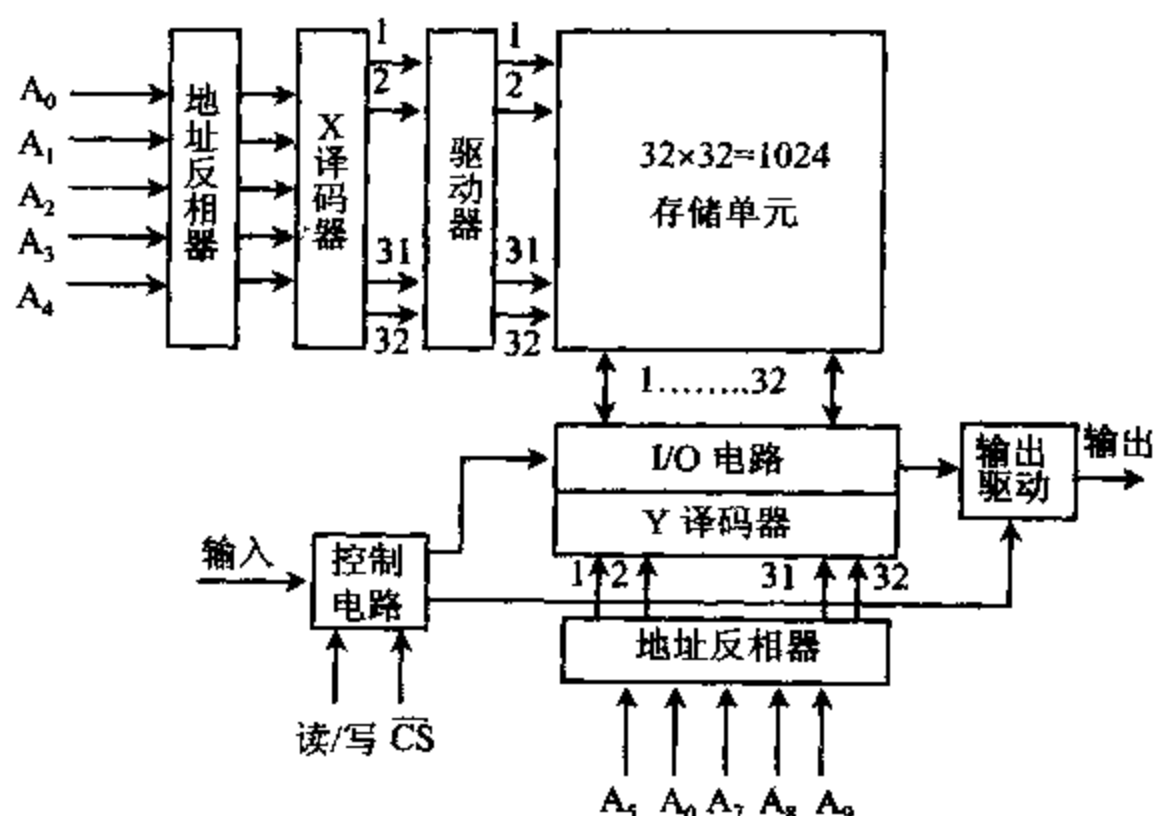


图 5-2 存储器芯片内部结构框图

(3) 控制逻辑与三态数据缓冲器

存储器读/写操作由 CPU 控制, CPU 送出的高位地址经译码后, 送到逻辑控制器的 $\overline{CS}$ 端。 $\overline{CS}$ 信号为片选信号, $\overline{CS}$ 有效, 存储器芯片选中, 允许对其进行读/写操作, 当读写控制信号 $\overline{RD}$ 、 $\overline{WR}$ 送到存储器芯片的 $R/\overline{W}$ 端时, 存储器中的数据经三态数据缓冲器的 $D_0 \sim D_7$ 端送到数据总线上或将数据写入存储器。

2. 静态 RAM 的例子

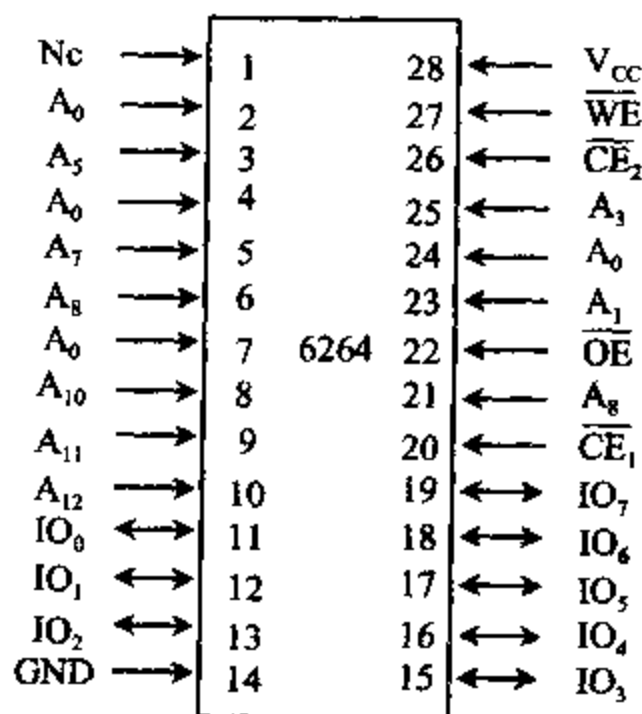


图 5-3 $8K \times 8$ SRAM 引脚图

典型的静态 RAM 芯片如:

2114 ($1K \times 4$ 位); 6116 ($2K \times 8$ 位)
6264 ($8K \times 8$ 位); 62128 ($16K \times 8$ 位)
62256 ($32K \times 8$ 位)

图 5-3 给出了 6264 ($8K \times 8$ 位) 芯片的引脚图。

其中: $A_{12} \sim A_0$: 地址线

$IO_7 \sim IO_0$: 数据线

$\overline{WE}$: 写允许信号, 低电平有效

$\overline{OE}$: 读允许信号, 低电平有效

$\overline{CE}_1, \overline{CE}_2$: 片选

V_{CC} : +5V; V_{SS} : 地

存储器的型号 6264 ($8K \times 8$) 给出了存储器件的存储单元数及每个单元的位数。 $8K \times 8$ 表示 RAM

芯片有 8K 个存储单元,每个单元存储 8 位数。

6264 芯片的地址线引脚 $A_{12} \sim A_0$ 用来选择存储器件中的一个存储单元,13 根线 $A_{12} \sim A_0$ 可选择 8K 个存储单元($2^{13}=8192$)。存储器的地址由 CPU 输入,8 位数据输出时,它与地址总线 $A_{12} \sim A_0$ 相连接;16 位数据输出时,它与地址总线 $A_{13} \sim A_1$ 相连。 A_0 用来选择偶地址存储体,奇地址存储体用 $\overline{BHE}$ 选择。

6264 芯片的数据线引脚为双向输入/输出线,对于 8 位宽的数据,经常写成 $D_7 \sim D_0$,本例中写成 $IO_7 \sim IO_0$,表示每个存储单元存储 8 位数据,它与数据总线低 8 位 $D_7 \sim D_0$ 相连。若要求 16 位数据输出,要用 2 片 6264,芯片的引脚分别与数据总线 $D_{15} \sim D_0$ 和 $D_7 \sim D_0$ 相连。

片选线引脚用来选择存储器芯片,芯片被选中才允许存储器执行读/写操作。每个存储器件,有一个或两个片选信号。6264 SRAM 有 $\overline{CE}_1$ 和 $\overline{CE}_2$ 两个片选信号,要求两个信号都是低电平芯片才激活。通常片选信号连到地址译码器。

控制线引脚是用作读/写控制,有 1 个或 2 个控制线。本例中有 $\overline{WE}$ 写允许和 $\overline{OE}$ 读允许 2 个控制,低电平有效,分别连接控制总线的 $\overline{WR}$ 和 $\overline{RD}$ 。若存储器件只有一个控制线 $\overline{WE}$,则低电平时为写控制,高电平时为读控制。图 5-4 给出了 6264 芯片与 CPU 的连接。

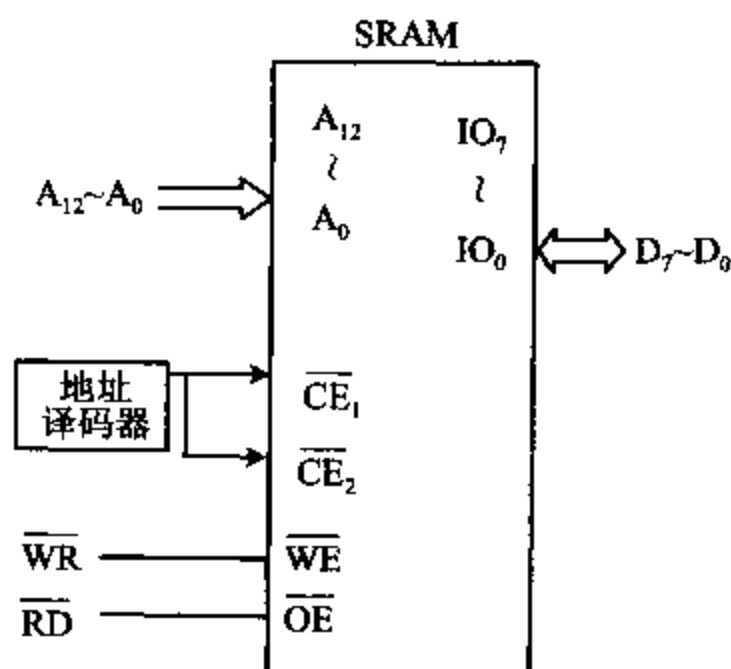


图 5-4 SRAM(8K×8)与 CPU 的连接图

二、动态随机存取存储器(DRAM)

1. 动态 RAM 的构成

动态 RAM 与静态 RAM 一样,由许多基本存储单元按行和列排列组成矩阵。最简单的动态 RAM 的基本存储单元是一个晶体管和一个电容,因而集成度高,成本低,耗电少,但它是利用电容存储电荷来保存信息的,电容通过 MOS 管的栅极和源极会缓慢放电而丢失信息,必须定时对电容充电,也称作刷新。另外,为了提高集成度,减少引脚的封装数,DRAM 的地址线分成行地址和列地址两部分,因此,在对存储器进行访问时,总是先由行地址选通信号 $\overline{RAS}$ 把行地址送入内部设置的行地址锁存器,再由列地址选通信号 $\overline{CAS}$ 把列地址送入列地址锁存器,并由读/写信号控制数据的读出或写入。所以刷新和地址两次打入是

DRAM 芯片的主要特点。

动态 RAM 基本单元主要有 4 管动态 RAM、3 管动态 RAM 及单管动态 RAM 组成,它们各有特点。4 管动态 RAM 使用管子多,使芯片容量小,但器件的读出过程就是刷新过程,不用为刷新而外部另加逻辑电路。3 管动态 RAM 所用管子少一点,但读/写数据线分开,读/写选择线也分开,并要另加刷新电路。单管动态 RAM 所用器件最少,但读出信号弱,要采用灵敏度高的读出放大器来完成读出功能。

动态 RAM 依靠电容存储电荷来决定存放信息是“1”或“0”。图 5-5 以单管动态 RAM 为例说明其工作原理,读操作时先由行地址译码,某行选择信号为高电平时,此行上管子 Q 导通,由刷新放大器读取电容 C 上的电压值折合为“0”或“1”,再由列地址译码,使某列选通。行和列均选通的基本存储单元允许驱动,并读出数据,读出信息后由刷新放大器对其进行重写,以保存信息。

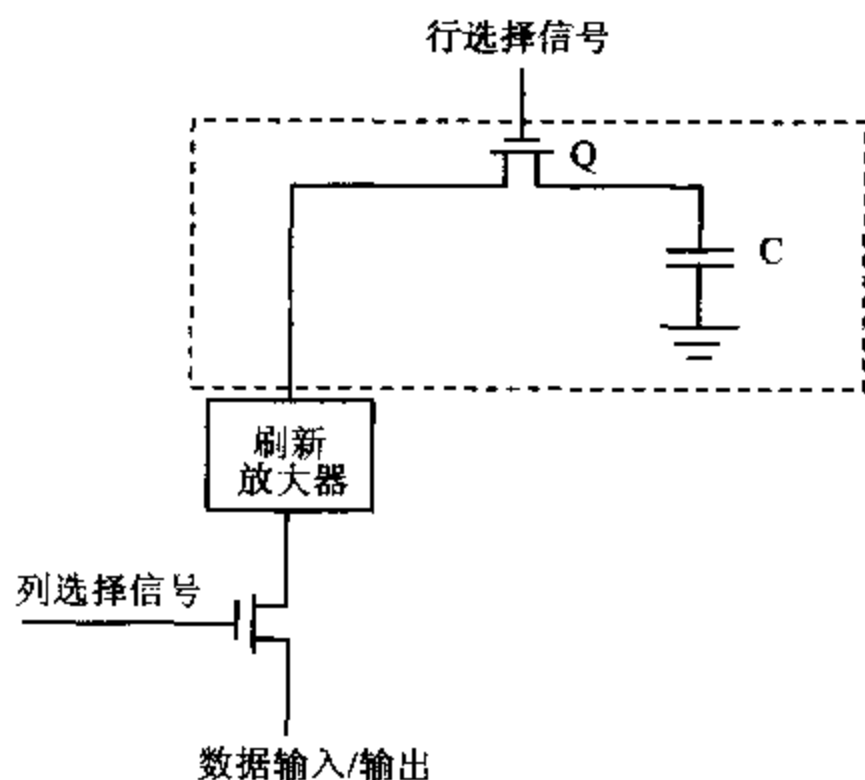


图 5-5 单管动态 RAM 基本存储单元

写操作时,行和列的选择信号为“1”,基本存储单元被选中,数据输入/输出线送来的信息通过刷新放大器和 Q 管送到电容 C,数据写入存储单元。

动态 RAM 需要配置刷新逻辑电路,在刷新周期中,存储器不能执行读/写操作,但由于它的单片上的高位密度(单管可组成)和低功耗(每个存储单元功耗为 0.05mw,而静态 RAM 为 0.2mw),及价格低廉等优点,使之在组成大容量存储器时作为主要使用器件。

2. 动态 RAM 的刷新

动态 RAM 都是利用电容存储电荷的原理来保存信息的,由于 MOS 管输入阻抗很高,存储的信息可以保存一段时间,但时间较长时电容会逐渐放电使信息丢失,所以动态 RAM 需要在预定的时间内不断进行刷新。所谓刷新,即把写入到存储单元的数据进行读出,经过读放大器放大之后再写入以保存电荷上的信息。两次刷新的时间间隔与温度有关,在 0~55°范围内为 1ms~3ms,典型的刷新时间间隔为 2ms。虽然每进行一次读写操作,实际上也进行了刷新,但读/写操作是随机的,不能保证内存中所有的 RAM 单元在 2ms 中可以由读/写操作来刷新,因此要安排存储器刷新周期及刷新控制电路来系统地完成对动态 RAM 的刷新。动态存储器的刷新是一行一行进行的,每刷新一行的时间称为刷新周期。刷新方式

有集中刷新方式和分散刷新方式两种。

DRAM 控制器是 CPU 和 DRAM 之间的接口电路,由它把 CPU 的信号转换成适合 DRAM 芯片的信号,解决 DRAM 芯片地址两次打入和刷新控制等问题。DRAM 控制器的逻辑框图如图 5-6 所示,包括下列功能电路:

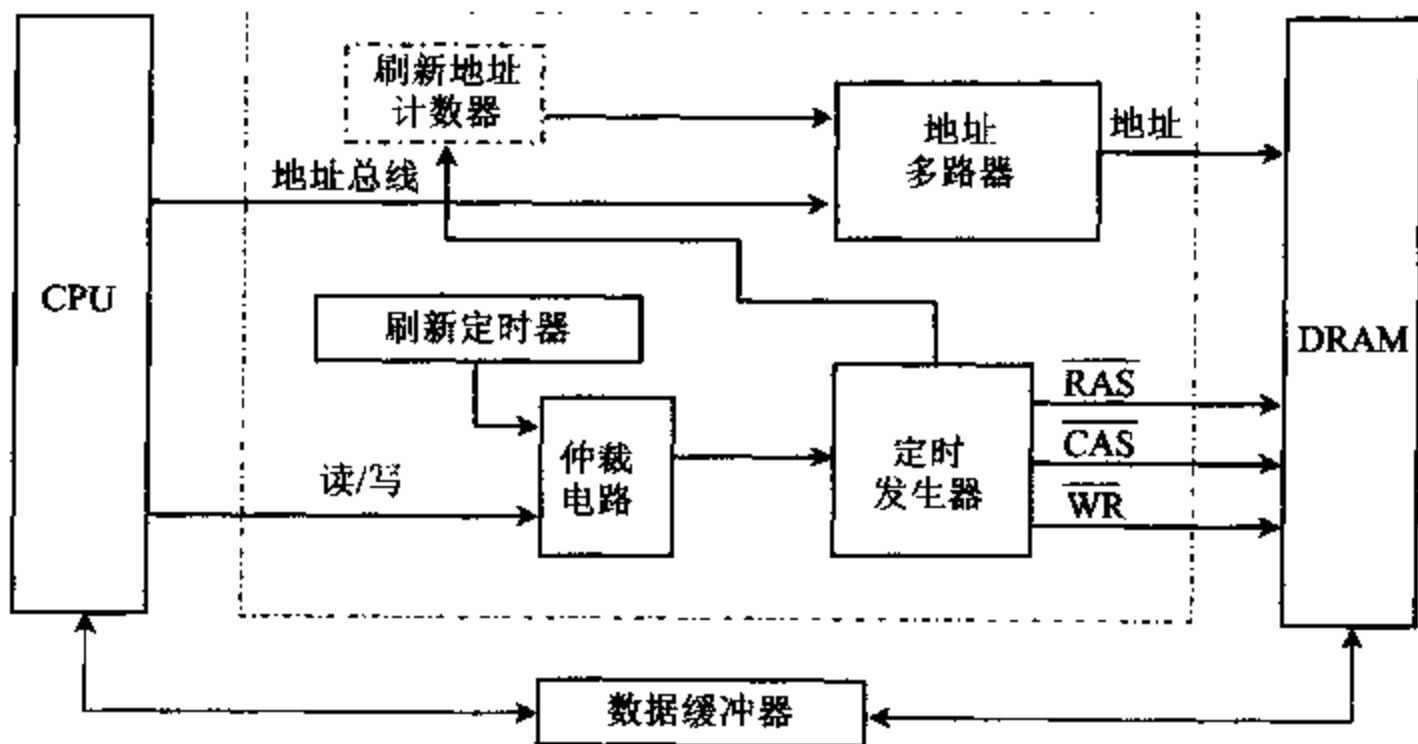


图 5-6 DRAM 控制器逻辑框图

(1) 地址多路器:把来自 CPU 的地址转换成行地址和列地址,分两次送到 DRAM 芯片,实现 DRAM 芯片地址的两次打入。

(2) 刷新定时器:完成对 DRAM 芯片进行定时刷新的功能,1M 位 DRAM 芯片,要求 8ms 内刷新 512 次。

(3) 刷新地址计数器:只用 $\overline{\text{RAS}}$ 的刷新操作,需要提供刷新地址计数器。对于 1M 位的芯片,需要 512 个地址,因此刷新地址计数器要由 9 位构成。但是,256K 位以上的芯片,多数内部具有这种刷新地址计数器,可以采用 $\overline{\text{CAS}}$ 在 $\overline{\text{RAS}}$ 之前的刷新方式。

(4) 仲裁电路:来自 CPU 的访问存储器的请求和来自刷新定时电路的刷新请求同时产生时,由仲裁电路对两者的优先权进行裁定。

(5) 定时发生器:提供行地址选通信号 $\overline{\text{RAS}}$ 、列地址选通信号 $\overline{\text{CAS}}$ 和写信号 $\overline{\text{WE}}$,供 DRAM 芯片使用。

3. 动态 RAM 例子

Intel 2164 是 64K×1 的 DRAM 芯片,它的内部有 4 个 128×128 基本存储电路矩阵,图 5-7(a)给出了 2164 DRAM 的引脚图。

其中: $A_0 \sim A_7$:地址线

$\overline{\text{WE}}$:读/写控制线, $\overline{\text{WE}}=1$ 为读出, $\overline{\text{WE}}=0$ 为写入;

$\overline{\text{RAS}}$:行选通信号, $\overline{\text{CAS}}$:列选通信号;

D_{IN} :数据输入, D_{OUT} :数据输出;

V_{CC} :+5V, GND:地;

2164 片内有 64K 个地址单元,需要 16 条地址线寻址。采用行和列两部分地址,地址线只需 8 条。内部有地址锁存器,利用外接多路开关,先由 $\overline{\text{RAS}}$ 信号选通 8 位行地址并锁存。

再由 $\overline{\text{CAS}}$ 信号选通 8 位列地址并锁存,16 位地址选中 64K 存储单元之中一个。64K 存储体有 4 个 128×128 的存储矩阵,每个 128×128 的存储矩阵,由 7 条行地址和 7 条列地址进行选择,再由 1/4 I/O 门选中一个单元进行读写。刷新时由一个行地址同时对 4 个存储矩阵的同一行,即 $4 \times 128 = 512$ 个单元进行刷新。由 $\overline{\text{WE}}$ 控制数据的读或写,2164 芯片无专门的片选信号,行选通信号可认为是片选信号。

图 5-7(b)给出了 41256($256\text{K} \times 1$)的 DRAM 引脚图。

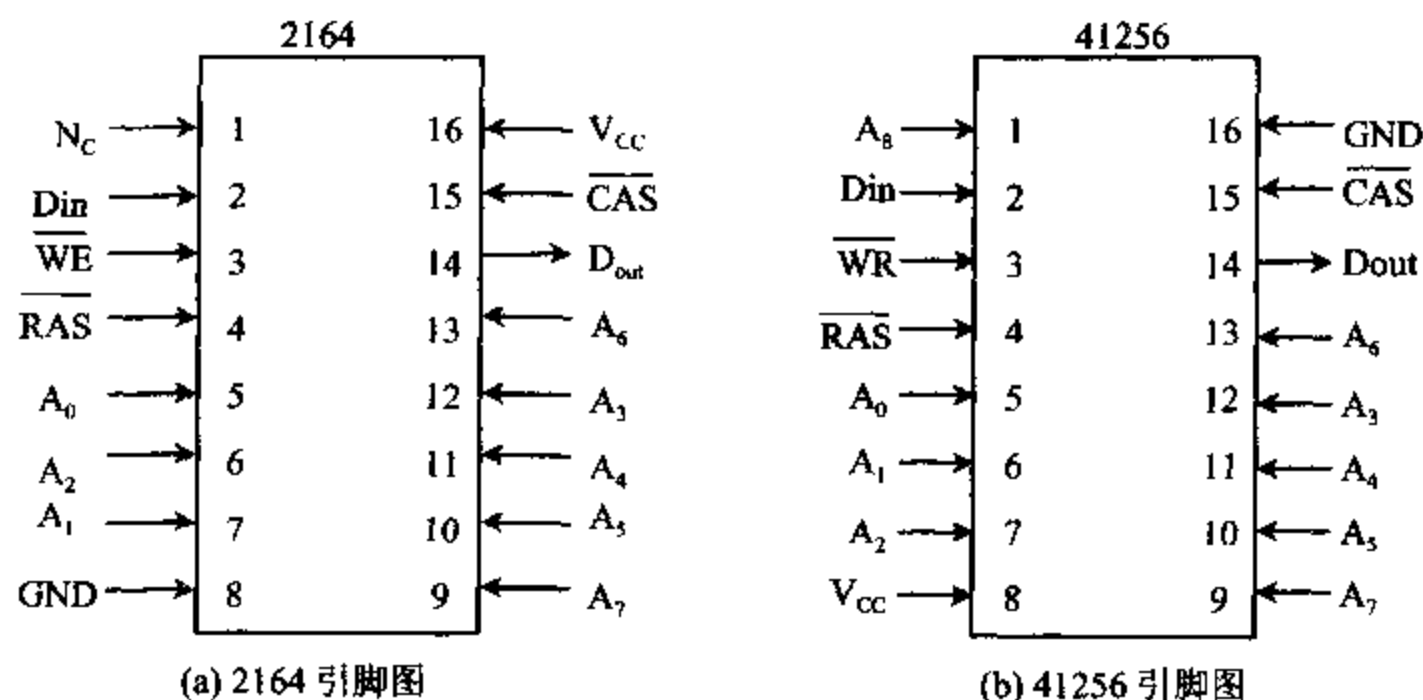


图 5-7 DRAM 引脚图

计算机中的内存由 DRAM 组成,更大容量的 DRAM 如 $1\text{M} \times 1$, $4\text{M} \times 1$, $16\text{M} \times 1$, $64\text{M} \times 1$ 和 $256\text{M} \times 1$ 等。早期的 DRAM 插在称为 SIMM(Single In-line Memory Modules 单列直插式存储器模块)的小电路板上,它们有 30 引脚或 72 引脚。30 引脚的 SIMM 常被组织成 $1\text{M} \times 8$, $1\text{M} \times 9$, $4\text{M} \times 8$, $4\text{M} \times 9$, 第 9 位为奇偶校验位。72 脚的 SIMM 组织成 $1\text{M} \times 32$, $1\text{M} \times 36$ (带奇偶校验位),其它还有 $2\text{M} \times 32$, $4\text{M} \times 32$, $8\text{M} \times 32$, $16\text{M} \times 32$ 等,它们也有奇偶校验位,例如 $4\text{M} \times 36$ 。Pentium 微处理有 64 位数据总线,不使用 8 位的 SIMM,72 脚的 SIMM 用起来也很麻烦,因为必须成对使用以获得 64 位宽的数据线。它们使用的 64 位 DIMM(Dual In-line Memory Modules,双列直插式存储器模块),DIMM 正变为大多数系统的标准。这些模块中的存储器被组织成 64 位宽,通常容量是 $16\text{MB}(2\text{M} \times 64)$, $32\text{MB}(4\text{M} \times 64)$, $64\text{MB}(8\text{M} \times 64)$, $128\text{MB}(16\text{M} \times 64)$, $256\text{MB}(32\text{M} \times 64)$ 等。

目前,PC 机上广泛使用的内存条为 SDRAM(同步动态随机存取存储器)和 DDR-SDRAM(双数据率同步动态 RAM),其中 DDR-SDAM 为当今存储器的主流产品。在需要获得更高性能的 PC 机中,可采用 RDRAM(Rambus 动态 RAM),当然价格也更高了。

三、存储器的工作时序

1. 静态 RAM 器件对存储器读周期和写周期时序的要求

为了使存储器与 CPU 很好地配合构成一个微型计算机系统,存储器芯片的工作时序应和 CPU 的读/写时序密切配合,因此有必要分析一下存储器的工作时序。选择存储器时最重要的参数是存取时间,在存储器读周期中,具体是指读取时间,在存储器写周期中,就是

指写入时间。访问存储器所需要的时间是指存储器接收到稳定的地址输入到读/写操作所需时间,访问时间的长短与存储器制造工艺有关,例如用双极型技术制造的器件速度快,但功耗大,价格贵,用互补金属氧化物半导体技术制造的器件功耗低,但速度慢。

图 5-8 给出了静态 RAM 存储器对读/写周期的时序要求。

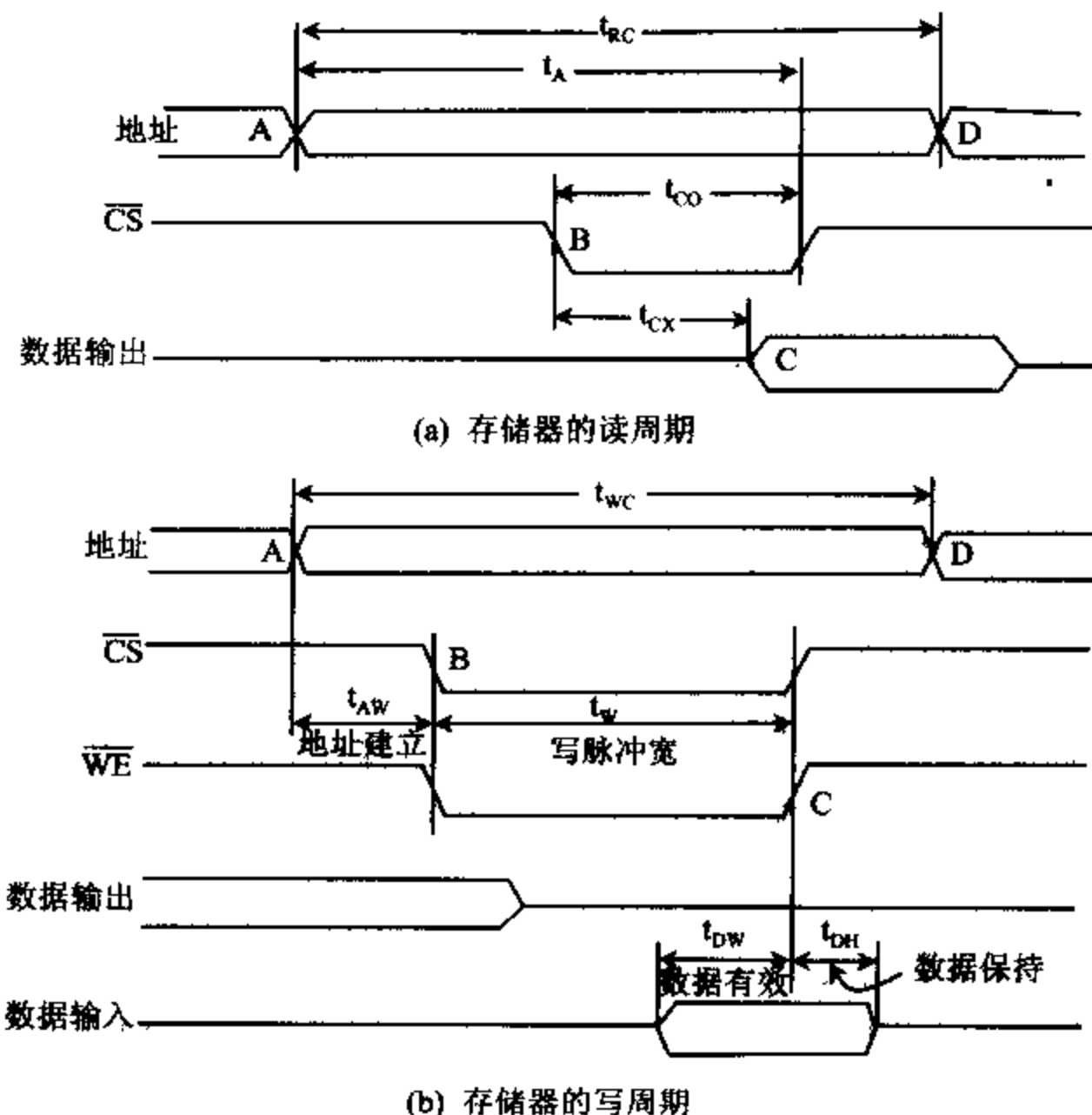


图 5-8 存储器对读/写周期的时序要求

存储器对读周期时序要求如图 5-8(a)所示。

t_A : 读取时间,地址有效到数据读出有效之间的时间,MOS 器件在 50ns~500ns 之间。

t_{CO} : 片选到稳定输出,从 $\overline{CS}$ 片选信号有效到数据输出稳定的时间,一般 $t_A > t_{CO}$ 。

t_{CX} : 片选到输出有效,从 $\overline{CS}$ 片选信号有效到数据输出有效的时间。

t_{AR} : 读恢复时间,输出数据有效之后,存储器不能立即输入新的地址来启动下一次读操作,因为存储器在输出数据后要有一定的时间来内部操作,这段时间称恢复时间。

存储器的读出周期是指启动一个读操作到启动下一次内存操作(读或写)之间所需要的时间,读出周期 $t_{RC} = \text{读取周期 } t_A + \text{读恢复周期 } t_{AR}$ 。

存储器对读周期的时序要求是:

(1) CPU 送出存储单元地址(图中 A 点),读周期开始,读周期比读取时间长。为了保证 t_A 时间后,读出数据在数据线上稳定,要求在地址信号有效后,不超过 $t_A - t_{CO}$ 的时间段中,片选信号 $\overline{CS}$ 有效。若 $\overline{CS}$ 不能及时到达,则 t_A 之后可能数据仅出现在内部数据总线上,

而不能将数据送到系统总线上。

(2) 输出数据有效后(图中 C 点),只要地址信号和输出允许信号没有撤销,输出数据一直保持有效。

(3) 在整个读周期,要求 $R/\overline{W}$ 应保持高电平。

在存储器芯片和 CPU 连接时,必须保证下面时间要求:

(1) 从地址信号有效到 CPU 要求的数据稳定之间的时间间隔必须大于 t_A 。

(2) 从片选信号有效到 CPU 要求的数据稳定之间的时间间隔必须大于 t_{CX} ,否则外部电路必须产生 $\overline{WAIT}$ 信号,迫使 CPU 插入 T_w 周期来满足上面的时间要求。

存储器对写周期时序要求,如图 5-8(b)所示。

t_{WC} :写周期时间。

t_{AW} :地址建立时间,地址出现到稳定的时间。

t_w :写脉冲宽,读/写控制线维持低电平的时间。

t_{DW} :数据有效时间。

t_{DH} :数据保持时间。

t_{WR} :写操作恢复时间,存储器完成内部操作所需时间。

(1) 写周期开始,要求有一段地址建立时间(图中 A 点到 B 点),此时 $\overline{WE}$ 必须为高电平,否则在地址变化期间可能会有误写入,使存储单元内容出错。所以 $\overline{WE}$ 有效前,地址就已经稳定。同样在 $\overline{WE}$ 变高电平后要经过写操作恢复时间,地址信号才能改变。

(2) 写周期期间 $\overline{CS}$ 、 $\overline{WE}$ 为低电平,要求 t_w 写脉冲宽度必须大于规定的值,以保证可靠的写入。

(3) 为了保证可靠的写入,要写入的数据必须在 $\overline{CS}$ 和 $\overline{WE}$ 有效前已稳定地出现在数据总线上,并在 $\overline{CS}$ 和 $\overline{WE}$ 变高电平之前保持稳定。

(4) 写周期时间(图中 A 点到 D 点)为地址建立时间、写脉冲宽度和写操作恢复时间三者之和。

以上讨论的读周期和写周期都是指存储器件本身能达到的最小时间要求,当将存储系统作为一个整体考虑时,涉及到系统总线驱动电路和存储器接口电路的延迟,实际的读/写周期要长得多。

四、高速缓冲存储器

1. 高速缓冲存储器的使用

DRAM 芯片存取时间在 $100\text{ns} \sim 200\text{ns}$ 之内,随着 CPU 速度的不断提高,DRAM 的速度难以满足 CPU 的要求,一般情况下,CPU 访问存储器时要插入等待周期,对高速 CPU 来说这是一种极大的浪费。SRAM 的访问周期可达 $20\text{ns} \sim 40\text{ns}$,目前已有 10ns 的器件。CPU 工作在 16MHz 时,使用 40ns 的 SRAM 足以在两个时钟周期内完成访问存储器的操作,也就是说总线访问可以在零等待的情况下完成。

系统设计时,为了使 CPU 全速运行,可采用 CACHE 技术,将经常访问的代码和数据保存到 SRAM 组成的高速缓冲器中,把不常访问的数据保存到 DRAM 组成的大容量存储器中,这样使存储器系统的价格降低,同时又提供了接近零等待的性能。通常 Cache 的容量

为 32K~256K,由 8K×8 位或 32K×8 位的 SRAM 芯片组成。

Cache 技术是为了把主存储器看成是高速存储器而设置的小容量局部存储器,它有效地利用了某些程序访问存储器在时间上和空间上有局部区域的特性,如子程序的反复调用,变量的重复使用,都是使用某个特定区域。这样如果针对某个特定的时间片,用连接在局部总线上的由 SRAM 组成的高速缓冲存储器(Cache)代替低速大容量的主存储器,作为 CPU 集中重复访问的区域,系统的性能就会取得明显的改善。

如图 5-9 所示,Cache RAM 是位于 CPU 和主存储器之间容量小而速度快的存储器,通常由 SRAM 组成。可以把 Cache 看作是主存储器中面向 CPU 的一组高速暂存寄存器,它保存有一份主存储器的“内容拷贝”,该“内容拷贝”是最近曾被 CPU 使用过的。

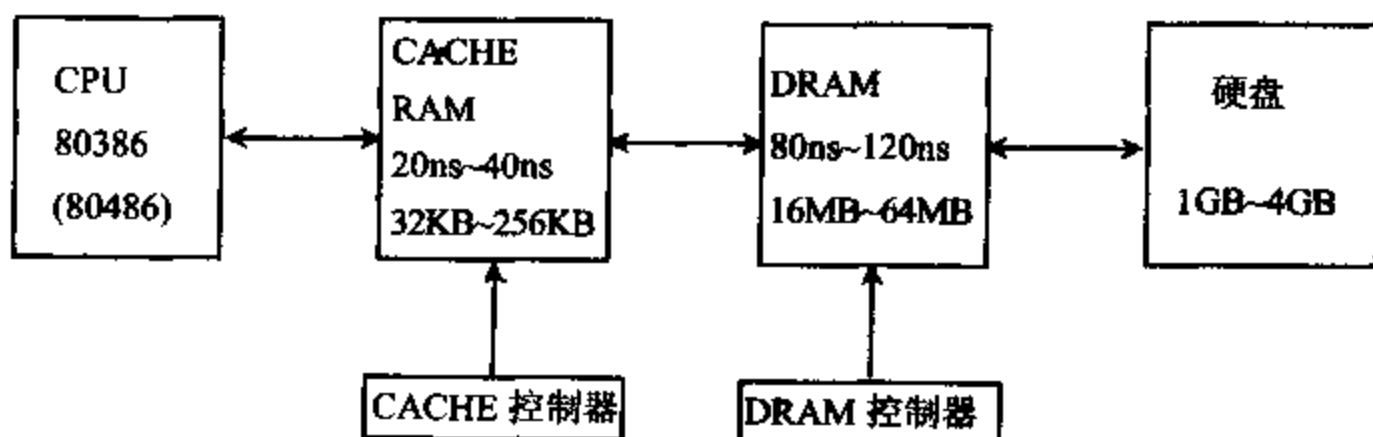


图 5-9 CACHE 在系统存储器中的位置

平时,系统程序、应用程序以及用户数据是存放在硬盘中的。正在执行中的程序或需要常驻的程序由操作系统装入到存储器,而在主存储器中经常被 CPU 使用的一部分内容,要“拷贝”到 Cache 存储器中。所以,开机时 Cache 中无任何内容,当 CPU 送出一组地址去读取主存储器时,读取的存储器的内容被同时“拷贝”到 Cache 之中。此后,每次 CPU 读取存储器时,Cache 控制器要检查 CPU 送出的地址,判别 CPU 要读取的数据是否在 Cache 存储器中。若是存在于 Cache 之中,则称为 Cache 命中,CPU 可以用极快的速度从 Cache 中读取数据。装有 32K 缓存时,命中率为 86%。装有 64K 缓存时,命中率为 92%。

若 CPU 要读取的数据不是存在于 Cache 之中,则称为 Cache 未命中,这时就需要从主存储器中读取数据。未命中时从主存储器读取数据,可能比访问无 Cache 的主存储器要插入更多的等待周期,因而降低了系统的效率。程序中的调用和跳转等指令,会造成非区域性操作,使命中率降低。所以,提高命中率是 Cache 设计的主要目标。

为了提高命中率,高速缓存控制器将主存储器划分成块,通常以 4、8、16、32 字节为一块。当 CPU 访问的存储器单元内容不在缓存中,高速缓存控制器就将主存储器中该单元及临近单元的内容调入高速缓存,若高速缓冲存储器已装满,则按某种调度算法,更新高速缓冲存储器,并要写入主存储器。

2. Cache 的结构

Cache 一般由两部分组成,一部分存放由主存储器来的数据,另一部分存放该数据在主存储器中的地址。(此部分称地址标记存储器,记为 Tag)。由关联性,高速缓冲存储器结构可分为:全相联 Cache(Fully Associative Cache);直接映象 Cache(Direct Mapped Cache)和成组相联 Cache(Set Associative Cache)。

全相联 Cache:多数程序运行时需要访问位于主存储器不同位置的子程序,数据,堆栈

及缓冲区等。因此 Cache 中存储的块与块之间,存储块的地址之间或存储的顺序没有直接关系。全相联 Cache 不但保存许多互不相关的数据块,同时要存储每一个数据块在主存储器中的地址(存放在 Cache 的地址标记存储器 Tag 中),当 CPU 访问主存储器时,Cache 控制器将访问主存储器的地址与 Tag 中存放的所有地址去逐一比较,匹配成功才命中。

直接映象 Cache:对直接映象 Cache 结构,地址仅需要比较一次就能确定是否命中。主要思想是将 Cache 中的每块分配一个索引字段,以确定块内寻址,用 Tag 字段区分存放在 Cache 中的主存储器位置。也就是说将主存储器分成若干页,主存储器中的每一页与 Cache 存储器大小相同,Cache 中的 Tag 存储器保存主存储器的页号(页地址),Cache 中的索引字段保存主存储器在此页中的偏移地址,即主存储器的偏移量直接映象为 Cache 的偏移量。直接映象 Cache 在程序频繁访问不同页号主存储块时,Cache 控制器要做频繁的转换,降低了系统性能,但由于它成本低,性能能够满足要求,亦被广泛使用。

成组相联 Cache:成组相联 Cache 结构结合了全相联 Cache 和直接映象 Cache 的特点,在 Cache 中具有多组直接映象的 Cache,它们并行地起着高速缓存的作用。也就是说每组 Cache 中采用直接映象结构,组之间采用全相联结构。CPU 访问存储器时,Cache 控制器要进行两次比较,才决定是否命中。目前计算机中常采用双路组相联或四路组相联高速缓冲存储器。图 5-10 给出了 Cache 的三种结构框图。

3. Cache 的数据更新

在高速缓冲存储器系统中,主存储器与 Cache 中的数据可能由于一个被修改了,而另一个未修改而不一致。所以必须有一个数据更新系统来保持两个存储器内容的一致性。要保证 Cache 中更新的内容写入与之相关的主存储块中,主存储器的修改方式有两种:通写式和回写式。

通写式:Cache 中的数据块一经修改。立即将其写入主存储器相关存储块中。使主存储器始终保持 Cache 中的最新内容。此方法的优点是简单,不必担心更新内容的丢失,但缺点是每次对 Cache 的修改同时要写入主存储器,使总线操作频繁,影响系统性能。可以改进成缓冲通写法,用一个缓冲器存放写入主存的数据,以减少 CPU 等待时间。

回写法:当 Cache 中的数据块被其他数据替换时,才将 Cache 中的数据写回到主存储器。与通写方式相比,回写式减少了写存储器的次数,一个更新后的 Cache 缓存块,只要它未与其它主存储块相关联,就不必写回到主存储器,当然它的 Cache 控制器要复杂一些。

4. 影响 CACHE 性能的因素

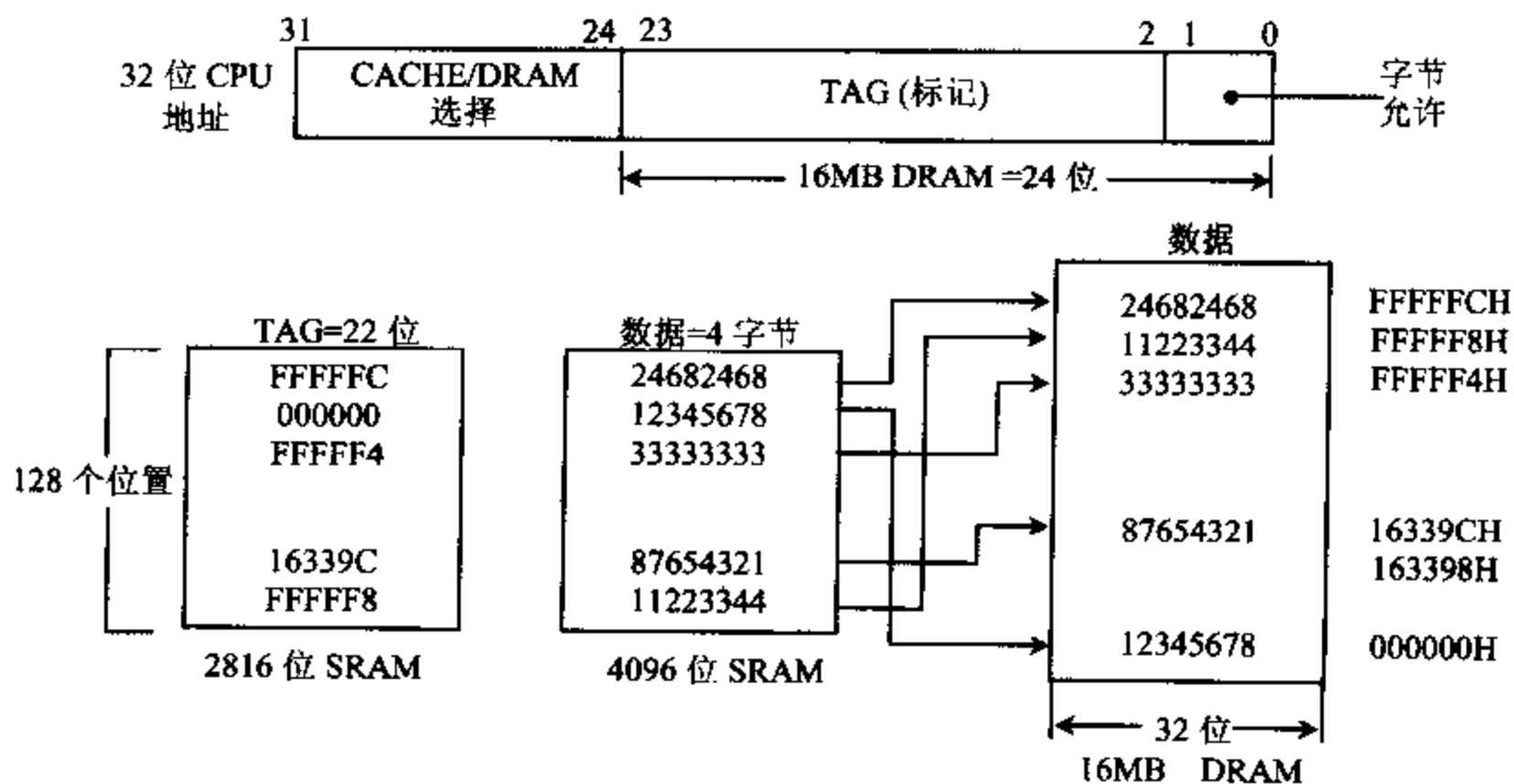
影响 Cache 性能的因素有:

(1) Cache 规模大小:Cache 规模越大,其内存保存的信息也越多,命中率就越高,但相应成本也越高。

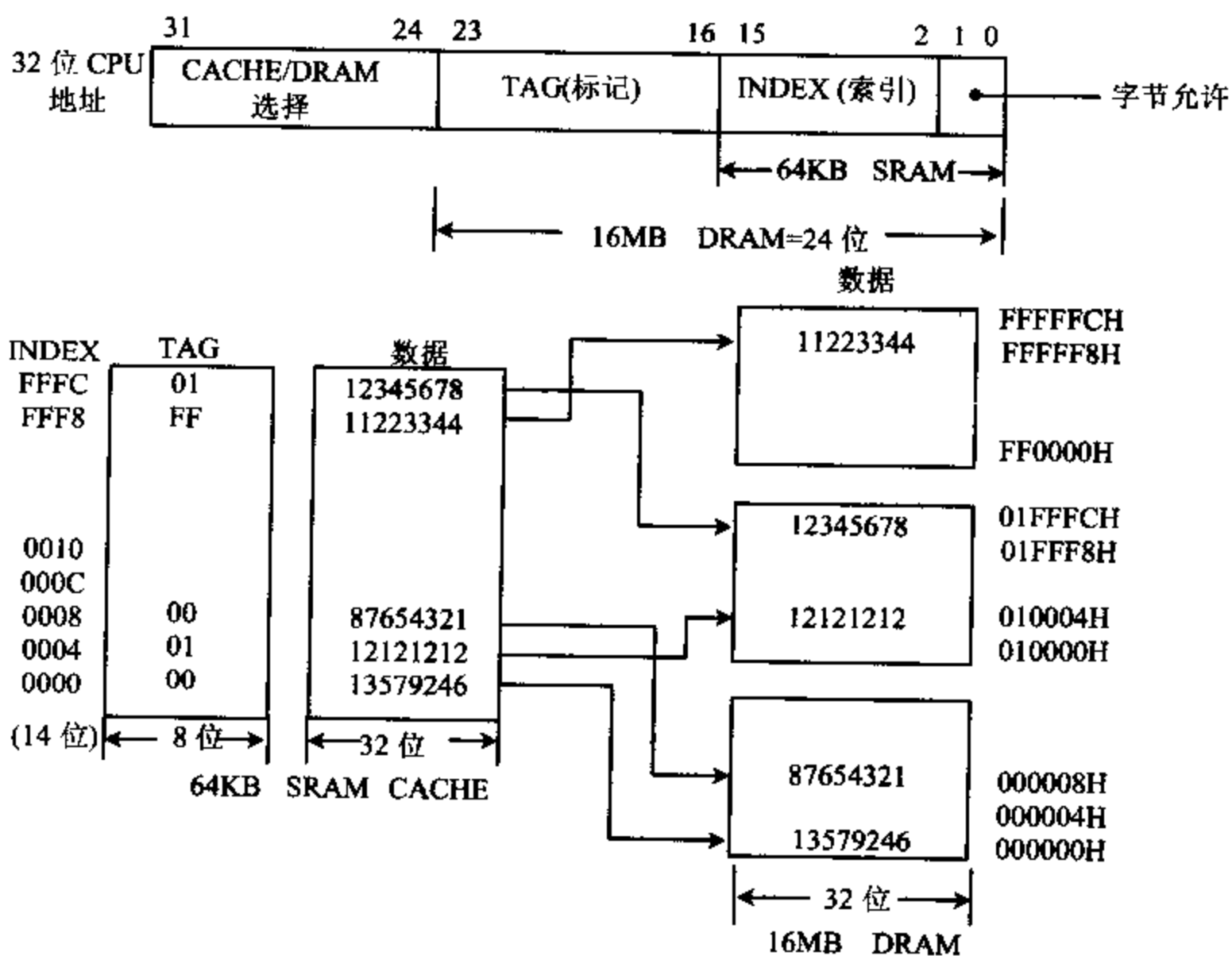
(2) 关联方式:根据统计四路相联的 Cache 比二路相联的 Cache 未命中率减少 25%,比直接映象 Cache 未命中率减少 40%。因此增加 Cache 相联的路数可以增加 Cache 的命中率,但系统复杂度和系统成本增加,操作速度有所下降。

(3) Cache 行大小:当每次 Cache 行不命中而进行 Cache 数据更新时,要由 Cache 从主存储器读取数据。Cache 行越大,填充 Cache 行的数据越多,所需时间也越长。但另一方面,可用的有效数据越多,其命中率也越高。

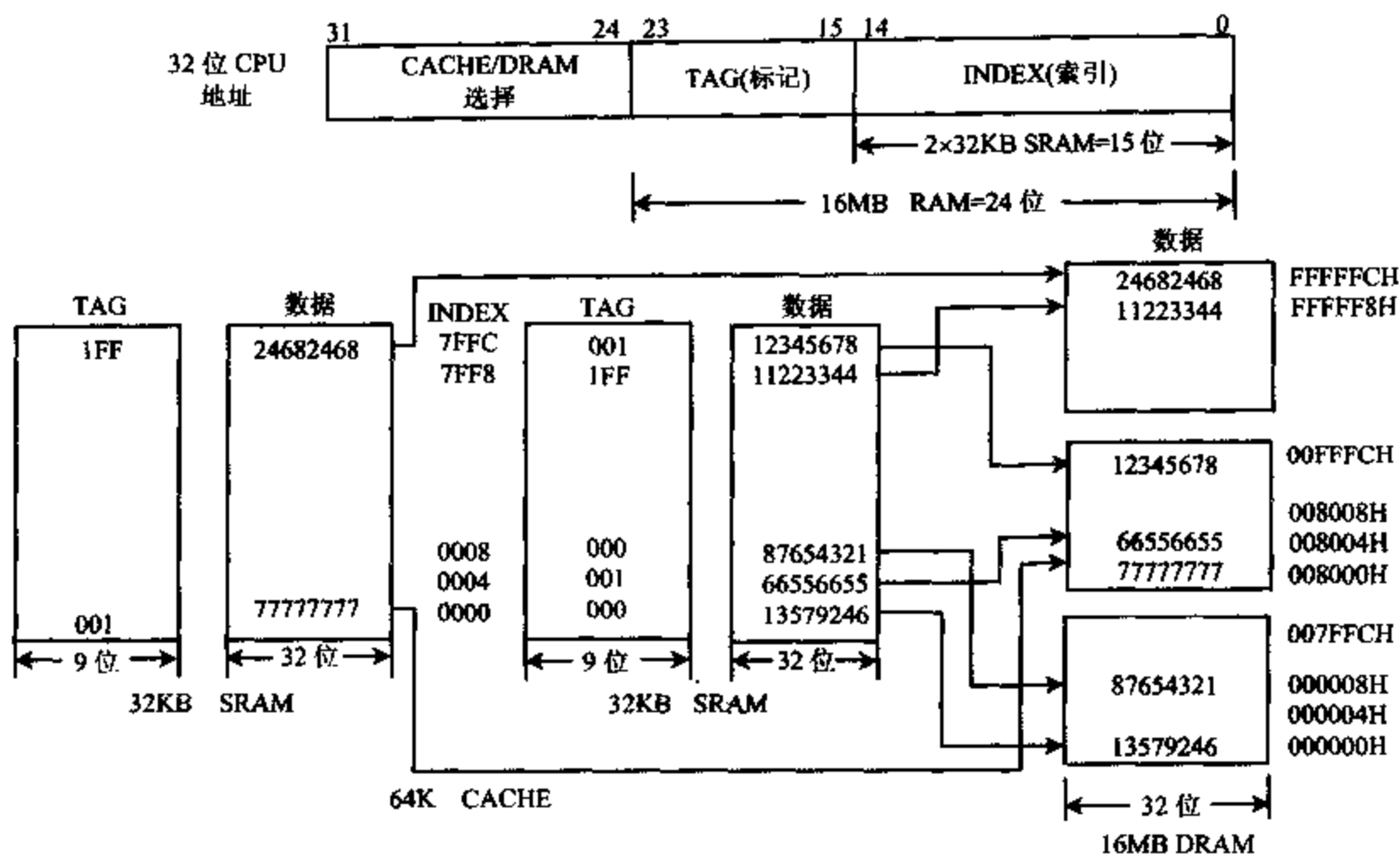
(4) 速度:Cache 存储器和 Cache 控制逻辑本身的存取速度不同,配置速度高的 Cache,



(a) 全相联 CACHE 结构框图



(b) 直接映象 CACHE 结构框图



(c) 二路成组相联 CACHE 结构框

图 5-10 CAHE 结构框图

使整机性能提高。

(5) 配置：给处理机系统增加诸如写保护，总线监视以及多处理协调等配置，可以加快 Cache 操作速度，但也增加了系统成本及复杂性。

表 5-1 给出了一级 Cache 在不同配置下的命中率，表中数字是统计得出的最低命中率，命中率还和软件程序有关，仅供参考。

表 5-1 一级 Cache 命中率

规模大小	关联方式	Cache 行大小	命中率%
8KB	直接映象	4B	73%
16KB	直接映象	4B	81%
32KB	直接映象	4B	86%
32KB	二路相联	4B	87%
32KB	直接映象	8B	91%
64KB	直接映象	4B	88%
64KB	二路相联	4B	89%
64KB	四路相联	4B	89%
64KB	直接映象	8B	92%
64KB	二路相联	8B	93%
128KB	直接映象	4B	89%
128KB	二路相联	4B	89%
128KB	直接映象	8B	93%

Pentium 片内 Cache 总容量为 16KB,其中 8KB 为数据 Cache,8KB 为指令 Cache,这两个 Cache 对应用软件来说是透明的,均采用二路相联映象技术。8KB 指令 Cache 和数据 Cache 又被进一步分成 128 个 Cache 子块,每个子块包括两个 Cache 行,每个 Cache 行大小为 32 位。数据 Cache 支持 MESI(Modified/Exclusive/Share/Invalid 修改/独占/共享/无效几种状态)写回 Cache 一致性协议,指令 Cache 内具有内在的写保护功能,能防止代码被破坏。

Pentium Pro CPU 芯片还配置一个 256KB 的二级 Cache 芯片,当芯片内不命中时,二级 Cache 提供所需的指令和数据,二级 Cache 是指令和数据统一为一体的 Cache,一般大小有 256KB,512KB,1MB 等,它们采用四路相联的体系结构。

5-3 只读存储器

除了随机存取存储器外,另一类为只读存储器(ROM)。ROM 存储的内容一般不会改变,掉电时也不会丢失,使用时可随时将内容读出。ROM 器件具有结构简单,位密度比读写存储器高,非易失性和可靠性高等特点,一般用来存放系统启动程序,常驻内存的监控程序,参数表,字库等,用户设计的单片机或单板机系统中也可用它来存放用户程序。

根据 ROM 信息写入的方式,ROM 分为 4 种。

1. 掩膜型 ROM

ROM 中信息是在芯片制造时由厂家写入的,用户对这类芯片无法进行任何修改。

2. 可编程只读存储器 PROM

这种 ROM 出厂时,里面没有信息,用户采用一些设备可以将内容写入 PROM。但 PROM 中内容一旦写入,就不能再改变了。

3. 可擦除可编程只读存储器 EPROM

用户可以用特定设备将内容写入,之后可用紫外光照将内容擦除,再重新写入。

4. 电可擦除的可编程只读存储器 EEPROM

用户可以用特定设备对芯片编程,用一定的通电方式将其内容擦除,再重新写入。

一、掩膜型 ROM

掩膜型 ROM 中信息是厂家根据用户给定的程序或数据对芯片图形掩膜进行两次光刻而决定的。这类 ROM 可由二极管、双极型晶体管和 MOS 型晶体管构成,每个存储单元只用一个耦合元件,集成度高,MOS 型 ROM 功耗小,但速度比较慢,微型计算机系统中用的 ROM 主要是这种类型。双极型 ROM 速度比 MOS 型快,但功耗大,只用在速度要求较高的系统中。

在数量较少时,掩膜型 ROM 造价很贵,如果进行批量生产,就相当便宜了。适用于计算机系统开发完成后,大批量使用。

二、可编程 ROM(PROM)

可编程只读存储器的内容可以由用户编写,但只允许编程一次。PROM 由二极管矩阵组成,用可熔金属丝连接存储单元发射极,出厂时所有管子的熔丝都是连着的。写入时,利用外部引脚输入地址,对其中的二极管进行选择,使某些二极管上通以足够大的电流,把所选定回路的熔丝烧断,这些被烧断的二极管代表“0”,未烧断的二极管代表“1”,从而实现了一次性信息存储,不能再进行修改。PROM 价格与生产批量无关,但造价比较贵,非批量使用时可用 PROM,批量使用时用掩膜型 ROM 比较便宜。

三、可编程可擦除 ROM(EPROM)

1. EPROM 工作原理

掩膜型 ROM 和 PROM 中的内容一旦写入,就无法改变,而 EPROM 却允许用户根据需要对它编程,且可以多次进行擦除和重写,因而 EPROM 得到了广泛的应用。

实现 EPROM 的技术是浮栅雪崩注入式技术,信息存储由电荷分布决定,MOS 管的栅极被 SiO_2 包围,称为浮置栅,控制栅连到字线。平时浮置栅上没有电荷,若控制栅上加正向电压使管子导通,则 ROM 存储信息为“1”。EPROM 的存储单元电路原理图如图 5-11(a)所示。

编程写入时若在漏极和衬底、漏级和源级间加上 +12V 电压,使内部 PN 结反向击穿,形成较大的电流,部分电荷会在浮置栅上捕获注入。当电压移去后由于绝缘层的包围,注入的电荷无法泄漏,相当于管子开启电压提高,控制栅上加上正向电压(+5V)后,管子仍截止,ROM 存储信息为“0”。

平时浮置栅上的电荷由于没有放电通路,在 125°条件下经过 10 年后仍能保持 70% 的电荷,因此可以作为只读存储器长期保存信息。EPROM 芯片上有一个石英窗口,当紫外光源照到石英窗口上,电路浮置栅上的电荷就会形成光电流泄漏走,使电路恢复起始状态,从而把写入的信息擦去,这样又可对 EPROM 重新编程。但 EPROM 写入过程很慢,所以它仍然作为只读存储器使用。

一块 EPROM 在初始状态下,所有的位均为“1”,写入时只能将“1”改变为“0”,用紫外光照后才能将“0”变为“1”,CMOS 器件则相反。一般光照 15 分钟左右,视具体器件型号而定,光照时间过长,也会影响器件使用寿命。

2. EPROM 例子

Intel 2764 是 $8\text{K} \times 8$ 的 EPROM,图 5-11(b)给出了它的引脚。

$A_{12} \sim A_0$:地址线 13 根,输入,连地址总线。

$D_7 \sim D_0$:数据线 8 根,编程时作数据输入,读出时为数据输出,连数据总线。

$\overline{\text{CE}}$:芯片允许端,输入,低电平有效,连地址译码器输出。

$\overline{\text{OE}}$:输出允许,输入,低电平有效,连 $\overline{\text{RD}}$ 信号。

$\overline{\text{PGM}}$:编程脉冲控制端,输入,连编程控制信号。

V_{PP} :编程时电压输入。

V_{CC} :电源电压, +5 伏。

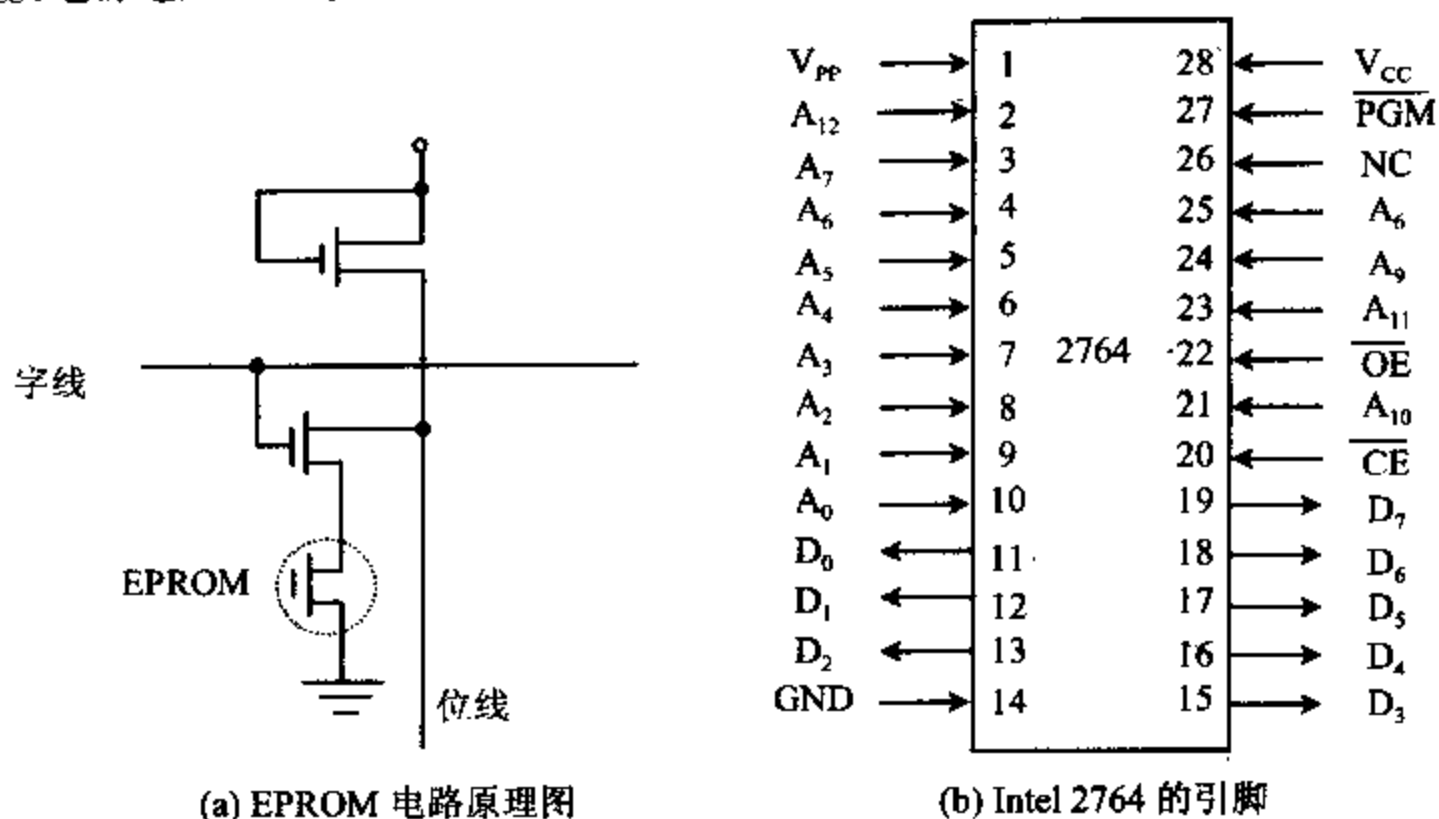


图 5-11

EPROM 有 4 种工作方式, 即读方式、编程方式、检验方式和备用方式, 如表 5-2 所示。

表 5-2 Intel 2764 的工作方式

信号端	V_{CC}	V_{PP}	$\overline{CE}$	$\overline{OE}$	$\overline{PGM}$	$D_7 \sim D_0$
读方式	+5V	+5V	低	低	低	输出
编程方式	+5V	+12V	高	高	正脉冲	输入
检验方式	+5V	+12V	低	低	低	输出
备用方式	+5V	+5V	无关	无关	高	高阻
未选中	+5V	+5V	高	无关	无关	高阻

在读方式下, V_{CC} 和 V_{PP} 接 +5V 电压, 从地址线 $A_{12} \sim A_0$ 输入所选单元的地址, 当 $\overline{CE}$ 和 $\overline{PGM}$ 端为低电平时, 数据线上出现所寻址单元的数据。图 5-12(a) 是读方式的时序图。注意芯片允许信号 $\overline{CE}$ 必须在地址稳定后有效, 才能保证读出所寻址单元的数据。

在编程方式下, V_{CC} 加 +5V 电压, V_{PP} 要加 +12V 电压 (有的芯片加 +25V 电压), $\overline{CE}$ 端为高电平, 从 $A_{12} \sim A_0$ 端输入要编程单元的地址, 在 $D_7 \sim D_0$ 端输入编程数据。在 $\overline{PGM}$ 端加上编程脉冲, 宽度为 50ms 的 TTL 高电平脉冲, 即可实现写入。注意必须在地址和数据稳定之后, 才能加上编程脉冲。图 5-12(b) 给出了 2764 的编程时序。

在检验方式下, V_{CC} 加 +5V, V_{PP} 加 +12V 电压, $\overline{CE}$ 接低电平, $\overline{PGM}$ 为低电平。检验总是和编程方式配合使用, 每次写入 1 个字节数据后, 紧接着将写入的数据读出, 去检查写入的信息是否正确。图 5-12(b) 给出了检验时序。

在备用方式下, 也就是使 EPROM 工作在功率下降方式, 此时与芯片未选中类似, 但功耗仅为读方式下的 25%。备用方式时, 只要在 $\overline{PGM}$ 端输入一个 TTL 高电平即可, 此时数据输出呈高阻状态。因为在读方式下, $\overline{CE}$ 和 $\overline{PGM}$ 两端连在一起, 若某芯片未被选中, $\overline{CE}$ 和 $\overline{PGM}$ 处于高电平状态, 那么此芯片就处于备用方式了, 大大减少了功耗。

目前市场上已提供了多种编程器,它们与微型计算机系统相连,通过编程软件选择某种型号 EPROM,自动提供所需的+12V 编程电压及编程脉冲对其进行编程,使用十分方便。

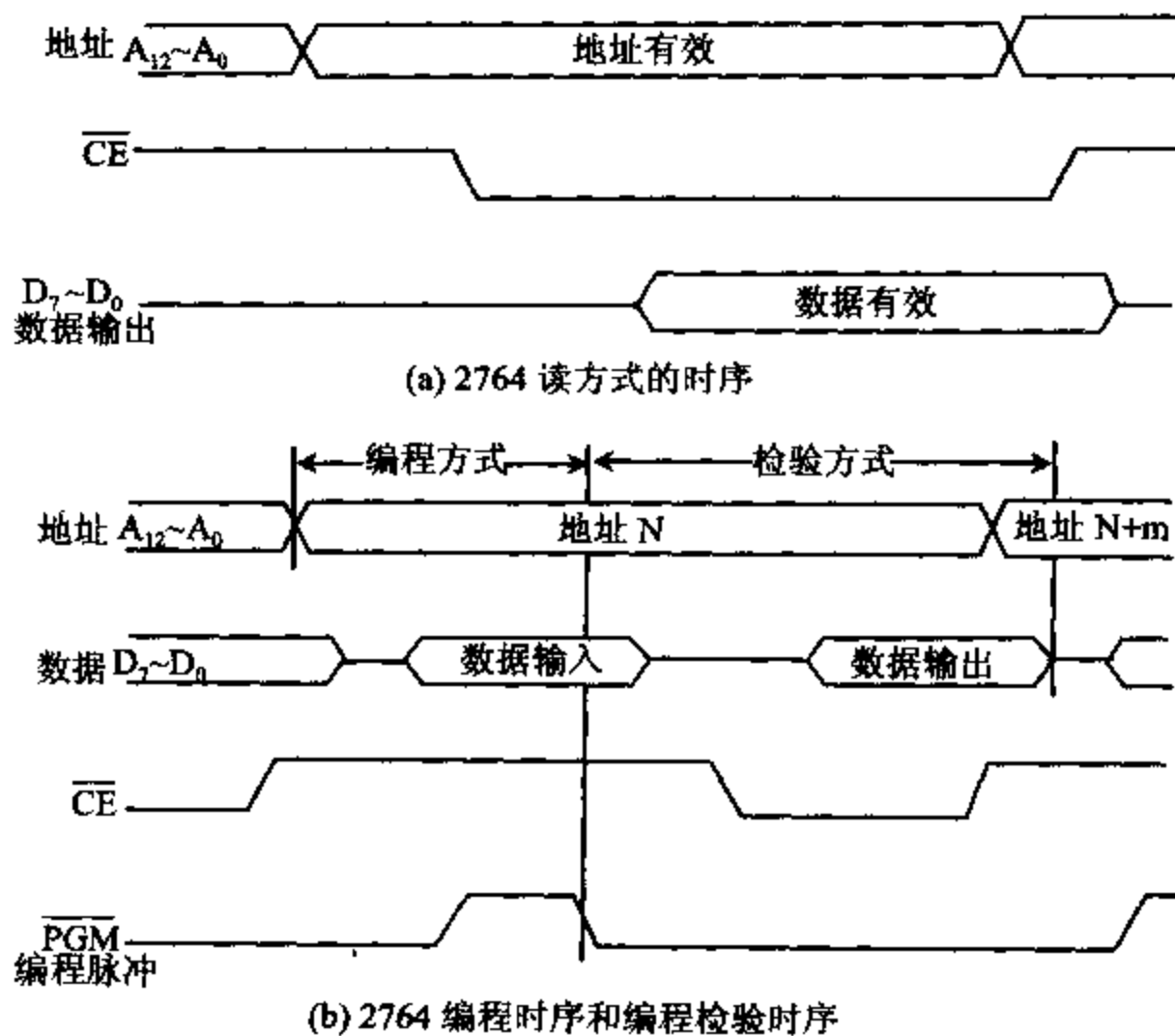


图 5-12 2764 的时序

EPROM2764 与 CPU 相连接时,与 RAM 类似,地址线引脚 $A_{12} \sim A_0$ 连接地址总线 $A_{12} \sim A_0$,8 位数据输出时,数据线引脚 $D_7 \sim D_0$ 连接数据总线 $D_7 \sim D_0$,注意方向是输出(只读)。16 位数据输出时,要用二块 EPROM 芯片,分别连接数据总线的 $D_{15} \sim D_8$ 和 $D_7 \sim D_0$ 。片选信号 $\overline{CE}$ 连地址译码器,控制信号 $\overline{OE}$ 为输出允许,连控制总线的 $\overline{RD}$ 。图 5-13 给出了 EPROM2764 与 CPU 的连接。

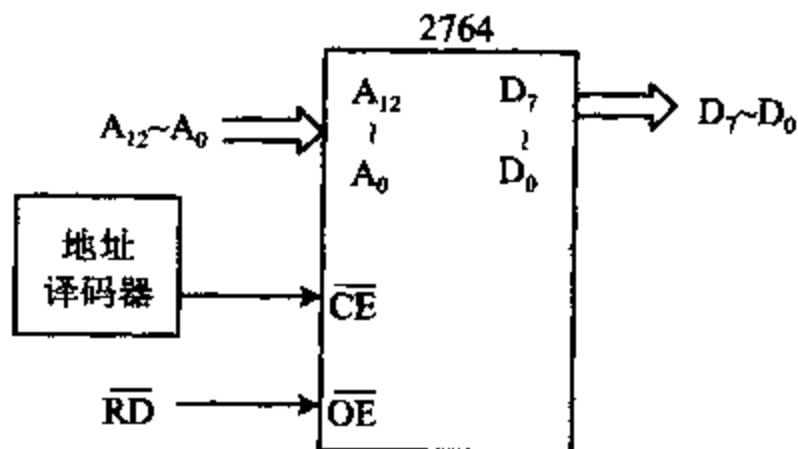


图 5-13 2764 EPROM 与 CPU 的连接

四、电可擦除可编程 ROM(EEPROM)

EPROM 尽管可以擦除后重新进行编程,但擦除时需用紫外线光源,使用起来仍然不太

方便。电可擦除的可编程 ROM,简称 EEPROM(E²PROM),它的外形管脚与 EPROM 相似仅擦除过程不需要用紫外线光源。

EEPROM 有 4 种工作方式,即读方式、写方式、字节擦除方式和整体擦除方式。表 5-3 列出了 Intel 2815 EEPROM 的工作方式。

在读方式时,从地址端输入所要读取的存储单元的地址, $\overline{CE}$ 和 $\overline{OE}$ 为低电平, V_{PP} 加+4V~+6V 电压,输出端便会出现读得的数据。这是常用的工作方式。

在写方式时,从地址端输入要写入的地址,数据输入端为要写入的数据, $\overline{CE}$ 和 $\overline{OE}$ 端为高电平, V_{PP} 加+21V 电压,此时可编程写入。

表 5-3 Intel 2815 的工作方式

信号端	V_{PP}	$\overline{CE}$	$\overline{OE}$	$D_7 \sim D_0$
读方式	+5V	低电平	低电平	输出
写方式	+21V	高电平	TTL 高电平	输入
字节擦除方式	+21V	低电平	TTL 高电平	TTL 高电平
整体擦除方式	+21V	低电平	+9~+15V	TTL 高电平

在字节擦除方式下,由地址端输入要擦除的字节的地址, $\overline{CE}$ 为低电平, $\overline{OE}$ 为高电平, V_{PP} 加上+21V 电压,数据端则要加上 TTL 高电平,可对指定字节进行擦除。

在整体擦除方式下,可使整片 EEPROM 回到初始状态。在此方式下, $\overline{OE}$ 端要加上+9V~+15V 高电平, V_{PP} 端加+21V 电压,数据端加上 TTL 高电平。

5-4 CPU 与存储器的连接

在 CPU 对存储器进行读写操作时,首先在地址总线上给出地址信号,然后发出相应的读或写控制信号,最后才能在数据总线上进行数据交换,所以 CPU 与存储器的连接包括地址线、数据线和控制线的连接等三个部分。在连接时要考虑以下几个问题:

(1) CPU 总线的负载能力

一般来说,CPU 总线的直流负载能力可带一个 TTL 负载,目前存储器基本上是 MOS 电路,直流负载很小,主要负载是电容负载。因此在小型系统中,CPU 可以直接和存储器芯片相连,在较大的系统中,考虑到 CPU 的驱动能力,必要时应加上数据缓冲器(例如 74LS245)或总线驱动器来驱动存储器负载。

(2) CPU 的时序和存储器存取速度之间的配合

CPU 在取指令和读/写操作数时,有它自己固定的时序,应考虑选择何种存储器来与 CPU 时序相配合。若存储器芯片已经确定,应考虑如何实现 T_w 周期的插入。

(3) 存储器的地址分配和片选

内存分为 ROM 区和 RAM 区,RAM 又分为系统区和用户区,每个芯片的片内地址,由 CPU 的低位地址来选择。一个存储器系统有多片芯片组成,片选信号由 CPU 的高位地址译码后取得。应考虑采用何种译码方式,实现存储器的芯片选择。

(4) 控制信号的连接

8086 CPU 与存储器交换信息时,提供了以下几个控制信号: $\overline{M}/\overline{IO}$ 、 $\overline{RD}$ 、 $\overline{WR}$ 、 ALE 、 $\overline{READY}$ 、 $\overline{WAIT}$ 、 $DT/\overline{R}$ 和 $\overline{DEN}$,这些信号与存储器要求的控制信号如何连接才能实现所需要的控制功能。

一、存储器的地址选择

一个存储器系统通常由许多存储器芯片组成,对存储器的寻址必须有两个部分,通常是将低位地址线连到所有存储器芯片,实现片内寻址,将高位地址线通过译码器或线性组合后输出作为芯片的片选信号,实现片间寻址。由地址线的连接决定了存储器的地址分配,下面分别叙述三种存储器地址选择的方法。

1. 线性选择方式

无论 ROM 或 RAM 芯片,芯片引脚都包括地址线,数据线,读/写控制线和片选 $\overline{CS}$ 线,只有片选信号 $\overline{CS}$ 有效时,才可能对该芯片进行操作。

例 5-1 RAM 芯片 Intel 6264 容量为 $8K \times 8$ 位,用 2 片静态 RAM 芯片 6264,组成 $16K \times 8$ 位的存储器系统。地址选择的方式是将地址总线低 13 位 ($A_{12} \sim A_0$) 并行地与存储器芯片的地址线相连,而 $\overline{CS}$ 端与高位地址线相连。

为了区分 6264 两组不同的芯片,可用 $A_{13} \sim A_{19}$ 中任一根地址线来控制,如图 5-14 所示,用 A_{13} 来控制。 A_{13} 为“0”选中 1# 芯片, A_{13} 为“1”选中 2# 芯片,此时 1# 芯片的段内地

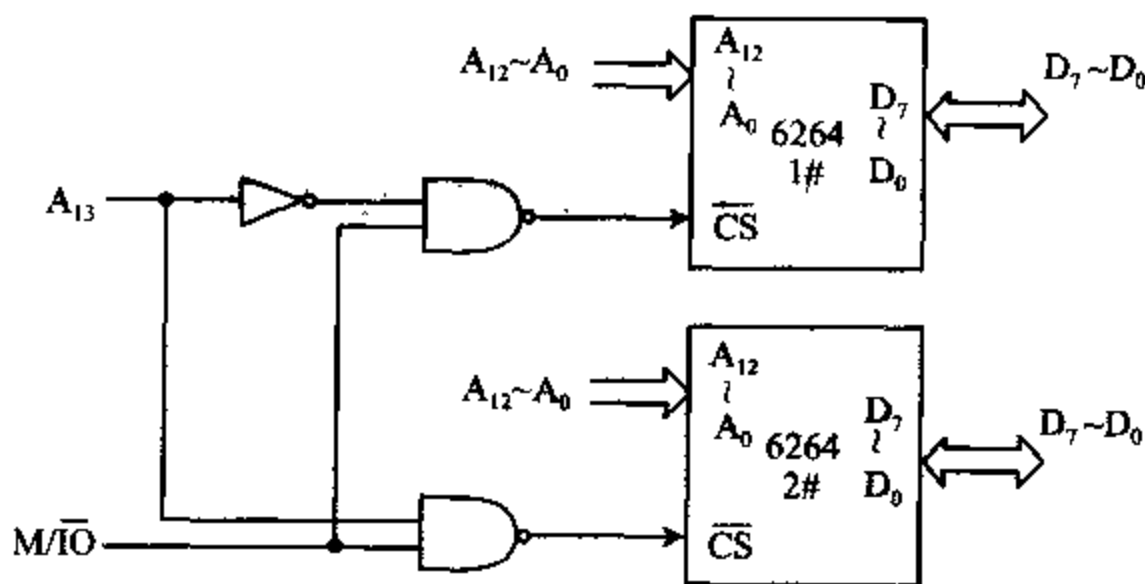


图 5-14 线性选择方式

址为 $0000 \sim 1FFFH$, 2# 芯片的地址为 $2000H \sim 3FFFH$ 。但是只要 $A_{13} = 0$, $A_{14} \sim A_{19}$ 为任意值都选中 1# 芯片,而只要 $A_{13} = 1$, $A_{14} \sim A_{19}$ 为任意值都选中 2# 芯片,所以它们的地址是重叠的。在一个段 64KB 中,地址重叠区有 4 个,即有 4 组地址可用于 1# 号芯片寻址:

$0000 \sim 1FFFH$, $4000 \sim 5FFFH$, $8000 \sim 9FFFH$, $C000 \sim DFFFH$

4 组地址可用于 2# 芯片寻址:

$2000 \sim 3FFFH$, $6000 \sim 7FFFH$, $A000 \sim BFFFH$, $E000 \sim FFFFH$

至于不同的段内重叠区就更多了,这种寻址方式称为线选方式。采用线性控制方式时,不仅地址重叠,而且用不同的地址线作选片控制,它们的地址分配也是不同的,若上例用 A_{14} 作控制线,则它们的基本地址是:

1# 芯片: $0000 \sim 1FFFH$

2# 芯片: 4000~5FFFH

此时内存地址不连续。若用 A_{15} 作控制线, 把 64K 内存地址分成上下 2 个区, 每区 32K, 前 32K 地址都选中第一组, 后 32K 地址选中第二组。

总之线性选择方式简单, 节省译码电路, 但地址分配重叠, 且地址空间不连续, 在存储容量较小且不要求扩充的系统中, 线性选择法是一种简单经济的方法。

2. 全译码选择方式

全译码选择地址的方式是对全部地址总线进行译码, 当有 16 根地址线时, 可直接寻址 64KB 单元。

例 5-2 假设一个微机系统的 RAM 容量为 4KB, 采用 $1K \times 8$ 的 RAM 芯片, 安排在 64K 空间的最低 4K 位置, $A_9 \sim A_0$ 作为片内寻址, $A_{15} \sim A_{10}$ 译码后作为芯片寻址, 则 4K 芯片占用的地址空间分别为:

第一组: 地址范围为 0000~03FFH

第二组: 地址范围为 0400~07FFH

第三组: 地址范围为 0800~0BFFH

第四组: 地址范围为 0C00~0FFFH

其中 $A_9 \sim A_0$ 并行接入内存芯片地址线, 用作片内地址选择, $A_{15} \sim A_{10}$ 6 根地址线通过 6:64 译码器译码后得到各芯片唯一的片选信号, 连到内存芯片 $\overline{CS}$ 端, 用作片间选择信号。图 5-15 给出了全译码选择的例子。采用全译码方法选择地址, 译码电路比较复杂, 但所得的地址是唯一的连续的, 并且便于内存扩充。

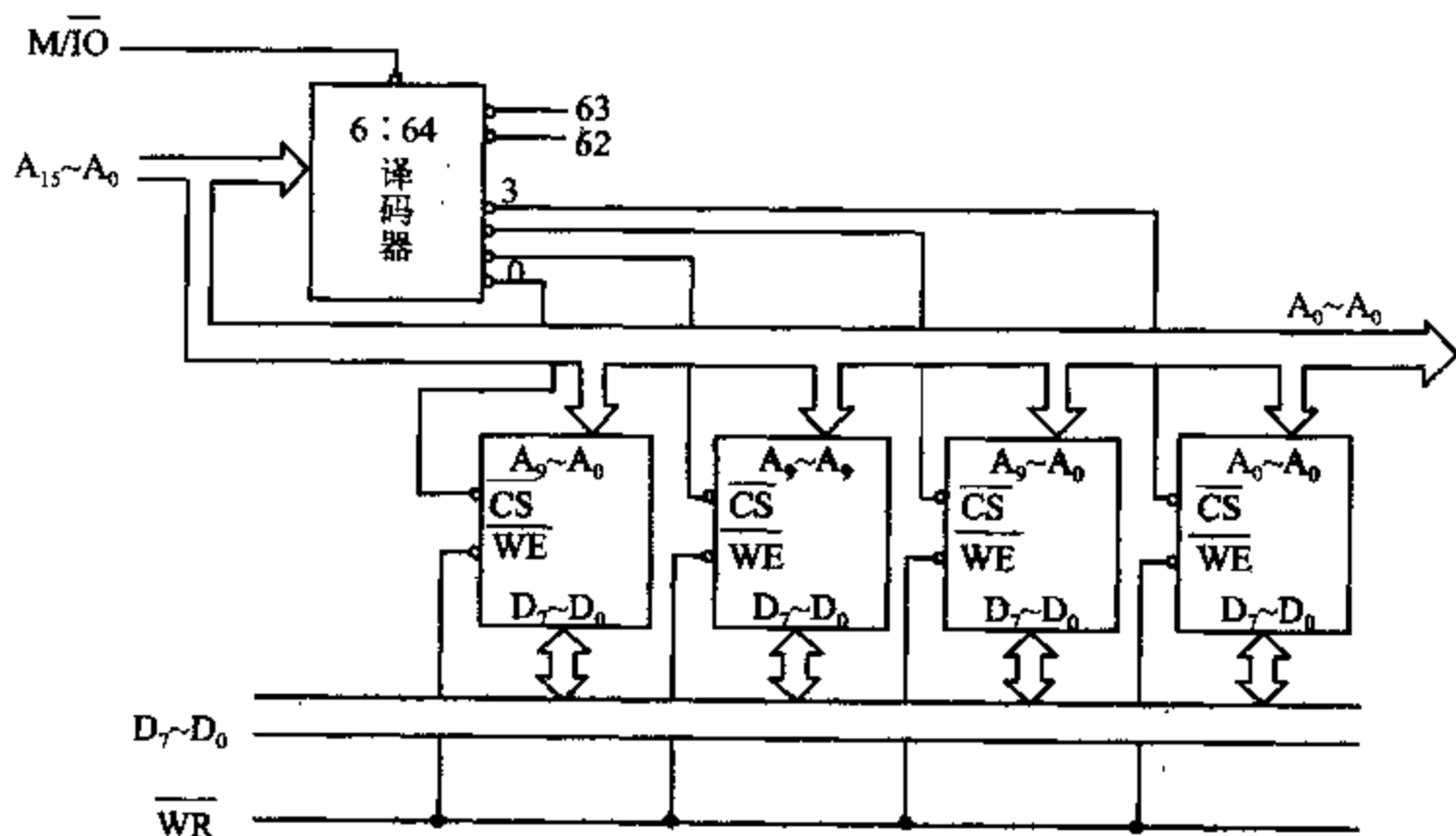


图 5-15 全译码地址选择方式

3. 部分译码选择方式

部分译码选择方式是将高位地址线中的几位经过译码后作为片选控制, 它是线性选择法与全译码选择法的混合方式, 通常译码器采用 3:8 译码器 74LS138。

74LS138 译码器管脚及译码输出真值表如图 5-16 所示。

74LS138 的 $G_1, \overline{G_{2B}}, \overline{G_{2A}}$ 为控制端,组合为 100 时输出才有效(输出为低电平有效),当输入端 C B A 三位相应组合为 000~111 时,每次译码一个输出端为 0,其余输出端为 1,具体组合如图 5-16 真值表所给出。

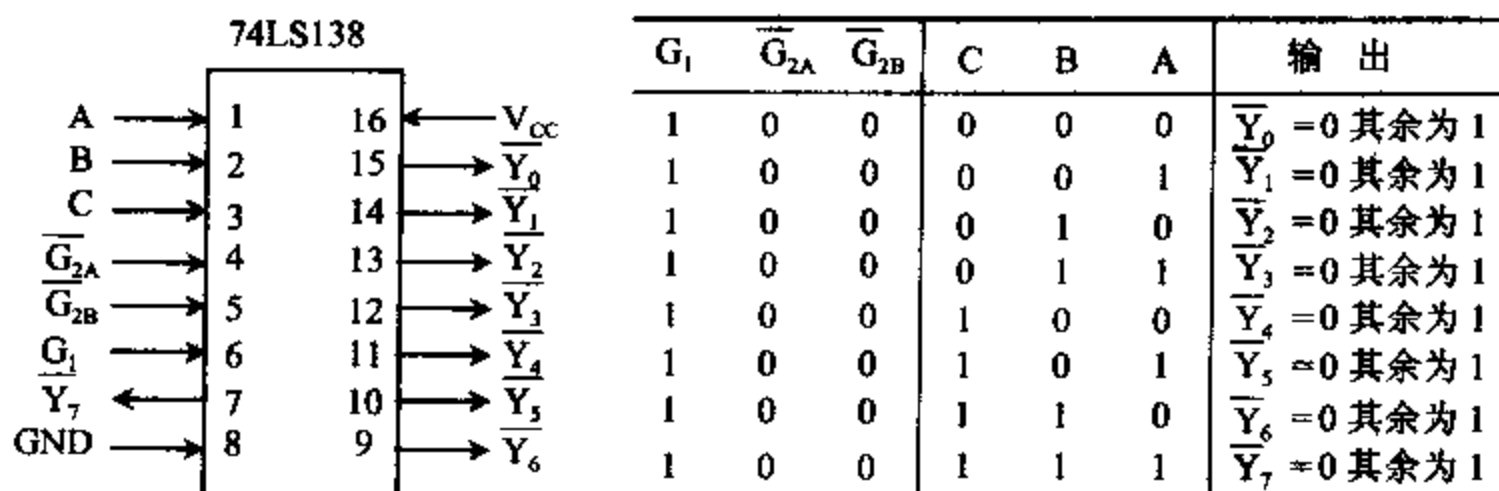


图 5-16 74LS138 译码器管脚及译码输出真值表

例 5-3 如果要设计一个 $8K \times 8$ 的存储器系统,采用 $2K \times 8$ 的 RAM 芯片 4 片,选用 $A_{10} \sim A_0$ 作为片内寻址,用 $A_{13} \sim A_{11}$ 作为 74LS138 的译码输入,利用输出端 $\overline{Y_0} \sim \overline{Y_3}$ 作为片选信号,则其地址分配为:

第一片:0000~07FFH

第二片:0800~0FFFH

第三片:1000~17FFH

第四片:1800~1FFFH

在存储器的一段 64KB 内, A_{14} 和 A_{15} 可以任意选择,所以地址仍有重叠区。若采用 $\overline{Y_4} \sim \overline{Y_7}$ 作为片选信号,4 片 RAM 芯片的地址分配又不同,分别为:

第一片:2000~27FFH

第二片:2800~2FFFH

第三片:3000~37FFH

第四片:3800~3FFFH

部分译码方式的可寻址空间比线性选择范围大,比全译码选择方式的地址空间要小。部分译码方式的译码器比较简单,但地址扩展受到一定的限制,并且出现地址重叠区。使用不同信号作片选控制信号时,它们的地址分配也将不同,此方式经常应用在设计较小的微型计算机系统中。

总之,CPU 与存储器相连时,将低位地址线连到存储器所有芯片的地址线上,实现片内选址。将高位地址线单独选用(线选法)或经过译码器(部分译码或全译码)译码输出控制芯片的选片端,以实现芯片间寻址。连接时要注意地址分布及重叠区。

二、存储器的数据线及控制线的连接

8086 CPU 有 20 位地址线,可寻址 1MB 的存储空间。8086 CPU 数据线有 16 位,可以读/写一个字节,也可读/写一个字。与 8086 CPU 相连的存储器是用 2 个 512KB 的存储体组成的,它们分别称为低位(偶地址)存储体和高位(奇地址)存储体,用 A_0 和 $\overline{BHE}$ 信号分别

来选择两个存储体,用 $A_{19} \sim A_1$ 来选择存储体体内的地址。若 $A_0=0$ 选中偶地址存储体,它的数据线连到数据总线低 8 位 $D_7 \sim D_0$,若 $\overline{BHE}=0$ 选中奇地址存储体,它的数据线连到数据总线高 8 位 $D_{15} \sim D_8$ 。若读写一个字, A_0 和 $\overline{BHE}$ 均为 0,两个存储体全选中。

8086 CPU 与存储器芯片连接的控制信号主要有地址锁存信号 ALE ,读选通信号 $\overline{RD}$,写选通信号 $\overline{WR}$,存储器或 I/O 选择信号 $M/\overline{IO}$,数据允许输出信号 $\overline{DEN}$,数据收发控制信号 $DT/\overline{R}$,准备好信号 $READY$ 。在最小模式系统配置中,数据线和地址线经过地址锁存器 8282 及数据收发器 8286 输出。下面举一个 ROM, RAM 与 CPU 连接的例子。

例 5-4 要求用 $4K \times 8$ 的 EPROM 芯片 2732, $8K \times 8$ 的 RAM 芯片 6264, 译码器 74LS138 构成 8K 字 ROM 和 8K 字 RAM 的存储器系统,系统配置为最小模式。图 5-17 给出了系统连接图。

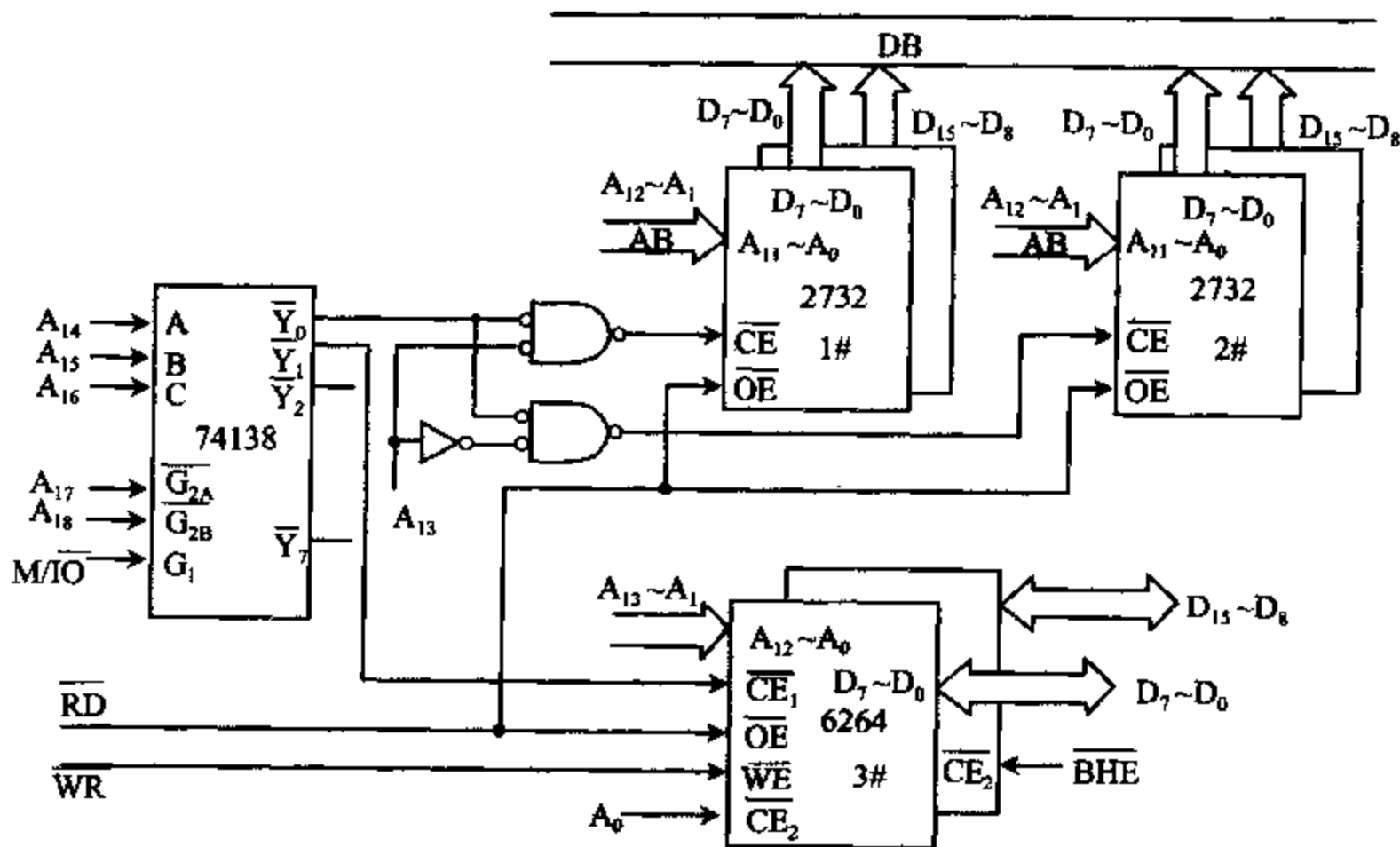


图 5-17 一个存储子系统例子

注:图中未画出对 ROM 芯片数据线的低 8 位与高 8 位的选择,可将 A_0 及 $\overline{BHE}$ 分别与 $\overline{Y_0}$ 信号相“与”来实现此选择。

ROM 芯片,8K 字用 4 片 2732 芯片组成,片内用 12 根地址线 $A_1 \sim A_{12}$ 寻址。

RAM 芯片,8K 字用 2 片 6264 芯片组成,片内用 13 根地址线 $A_1 \sim A_{13}$ 寻址。

芯片选择由 74LS138 译码器输出 $\overline{Y_0}$ 、 $\overline{Y_1}$ 完成。

ROM 芯片由 $\overline{RD}$ 信号(连 $\overline{OE}$ 端)来完成数据读出。RAM 芯片由 $\overline{RD}$ (连 $\overline{OE}$ 端)和 $\overline{WR}$ (连 $\overline{WE}$ 端)来完成数据读/写, A_0 、 $\overline{BHE}$ 用来区分数据线的低 8 位及高 8 位。

由于 ROM 芯片容量为 $4K \times 8$ 位, RAM 芯片容量为 $8K \times 8$ 位,用 A_{13} 和 $\overline{Y_0}$ 输出进行二次译码,来选择两组 ROM 芯片,这样可以保证存储器系统地址连续。

74LS138 译码器的输入端 C, B, A 分别连地址线 $A_{16} \sim A_{14}$, 控制端 G_1 、 $\overline{G_{2A}}$ 和 $\overline{G_{2B}}$ 分别连 $M/\overline{IO}$ 和 A_{17} 、 A_{18} , 计算得到存储器的地址范围为:

1# 芯片: $00000 \sim 01FFFH$

2# 芯片:02000~03FFFH

3# 芯片:04000~07FFFH

实际使用时,8086 为存储器产生独立的写选通信号,如图 5-18 所示。这里,74LS32 或门组合 A_0 与 $\overline{WR}$,产生低位存储体选择信号 $\overline{LWR}$, $\overline{BHE}$ 和 $\overline{WR}$ 组合产生高位存储体选择信号($\overline{HWR}$)。80286 写选通信号的产生使用 $\overline{MWTC}$ 信号代替 $\overline{WR}$ 。

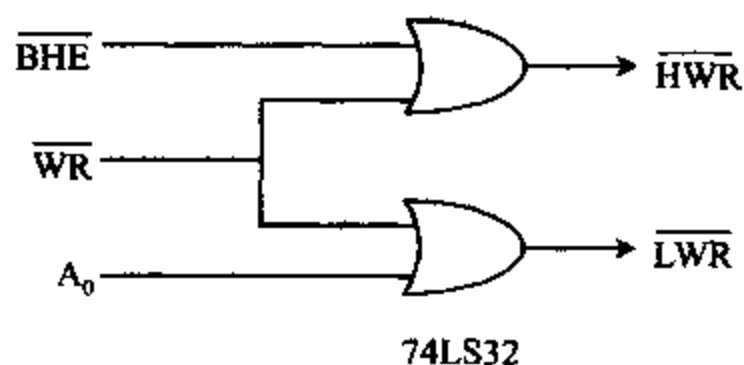


图 5-18 存储体写选择输入信号

80386DX 和 80486 微处理器中包含 4 个 8 位存储体,存储体的选择由选择信号 $\overline{BH3}$, $\overline{BH2}$, $\overline{BH1}$, $\overline{BH0}$ 实现。若传送 32 位数,4 个存储体都被选中。如果传送 16 位数,则 2 个存储体(高 16 位数据或低 16 位数据)被选中。如果传送一个 8 位数据,则一个存储体被选中。同 8086 一样,80386DX 和 80486 对每个存储体需要独立的写选通信号。写选通信号的产生如图 5-19 所示。

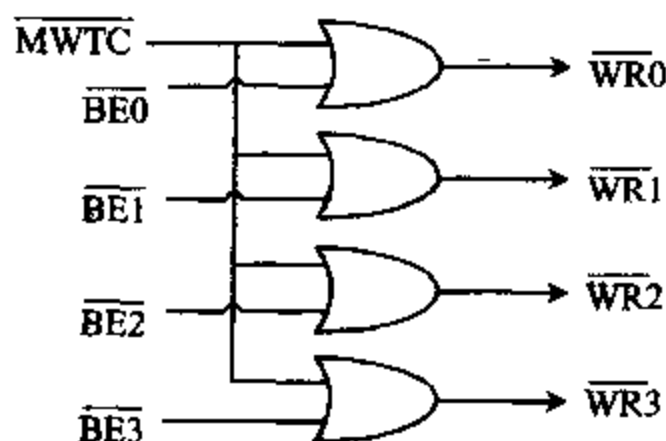


图 5-19 80386DX 和 80486 微处理器的存储体写信号

同样 Pentium 以上的微处理器,具有 64 位数据总线,需要 8 个独立的写信号。

顺便指出,32 位微处理器包含 32 位地址线,需要用可编程逻辑器件 PLD 作译码器,有 3 种 PLD 器件:PLA(可编程逻辑阵列),PAL(可编程阵列逻辑)和 GAL(门阵列逻辑)。在最新的存储器接口中 PAL 替代了 PROM 地址译码器,并且通过使用软包 PAL 编程程序来编程 PAL。

5-5 存储器空间的分配和使用

以前 16 位或 32 位微型计算机,大都以 MS-DOS 作为操作系统,而最早设计时,由于存储器芯片价格昂贵,以及软件对内存要求不高,设计主存储器为 640KB。随着大型软件系统出现,多道程序要求允许不同程序同时存取,需要大量存储空间,640KB 的限制成了计算

机的致命伤。随着计算机技术的发展,CPU 的寻址空间不断扩大,80386 CPU 可寻址 4GB,目前软硬件技术已十分成熟,可以解决 CPU 的寻址空间问题。计算机通过地址总线对存储器寻址,地址总线的宽度决定了计算机的寻址能力,各种类型的计算机的总线宽度及寻址能力如表 5-4 所示。

表 5-4 86 系列计算机的寻址能力

CPU	数据总线	地址总线	寻址能力	支持操作系统模式
8088	8 位	20 位	1MB	实模式
8086	16 位	20 位	1MB	实模式
80286	16 位	24 位	16MB	实模式、保护模式、
80386SX	16 位	32 位	4GB	实模式、保护模式、V86 模式
80386DX	32 位	32 位	4GB	实模式、保护模式、V86 模式
80486	32 位	32 位	4GB	实模式、保护模式、V86 模式

实模式(Real Mode):实模式就是 8086/8088 CPU 所采用的工作模式,20 条地址线能寻址 1MB 存储空间。

它的寻址方式为: 段地址:偏移地址

存储器的实际地址为: 段地址 * 16 + 偏移地址

保护模式(Protect Mode):80286 CPU 有 24 条地址线能寻址 16MB 存储空间,80386 CPU 有 32 条地址线能寻址 4GB 存储空间,利用保护模式的寻址方式,能够访问整个存储器地址空间。

它的寻址方式为: 选择器:偏移地址

存储器的实际地址换算如图 5-20 所示。它由段管理部件把 16 位选择器变成了 32 位段起始地址(基地址),再由 32 位起始地址同逻辑地址中指定的偏移量相加,就得到实际的物理地址。

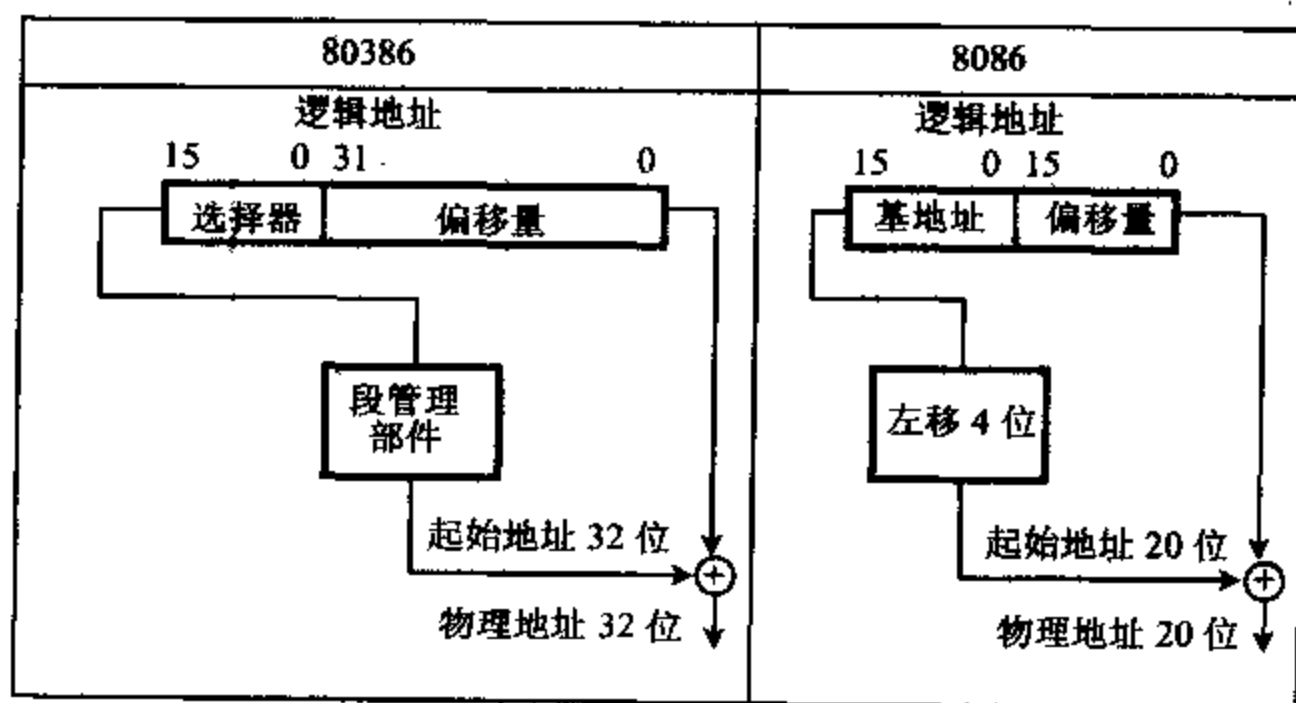


图 5-20 存储器的地址转换

V86 模式:V86 模式也称虚拟 86 模式,是 80386 以上 CPU 中才有的模式,是保护模式的一种子模式。V86 模式可同时提供多个 8086 实模式的存储空间,又有保护功能,存储器

的运行和控制是在进入保护模式后由程序来切换的。

一、IBM PC/XT 机中存储器空间分配

IBM PC/XT 机采用 8088 CPU,可以寻址 1MB 存储空间,但 MS-DOS 只能管理 640KB 内存,剩下的 384KB 做什么用了? 图 5-21 给出了 PC/XT 机的存储空间分配。

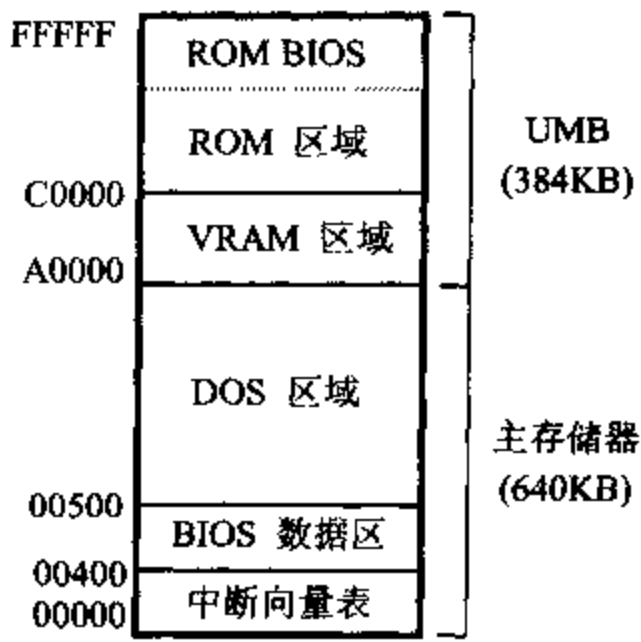


图 5-21 IBM PC/XT 机的存储空间分配

其中,从 00000H~9FFFFH 的 640KB 为主存储器,由 DOS 进行管理。A0000H~BFFFFH 的 128KB 为 VRAM 区,分配给视频适配器上的存储器使用。C0000H~FFFFFH 为 ROM 区,供 BIOS 和其它程序使用。VRAM 区和 ROM 区合起来称为上位存储器 (Upper Memory, UM), 总共 384KB。这些 ROM 空间用户程序无法存取,此部分存储器在系统主机板上,或在接口板上(例如在 VGA 卡上的 ROM),用户不感觉到它的存在。

在 PC/XT 机的 ROM 区,ROM BIOS 占 8KB,从 FE000H~FFFFFH,ROM BASIC 使用了从 F6000H 开始的 32KB 区域,硬盘使用了从 C8000H 开始的 8KB 区域。在 VRAM 区,MDA 使用了从 B0000H 开始的 4KB 区域,CGA 使用了从 B8000H 开始的 16KB 区域。

二、IBM PC/AT 机中存储器空间分配

IBM PC/AT 机中使用 80286 CPU 以上的芯片,80286 CPU 有两种工作模式:实模式和保护模式,在实模式下它的工作和 8086 CPU 相同。但 80286 CPU 有 24 根地址线,可访问 16MB 地址空间。在 PC/AT 机中可以专门对 80286 CPU 的 A_{20} 地址线进行控制,当 $A_{20}=1$ 时,允许访问 1MB 以上的 64KB 地址空间,这个地址空间称作高位存储区(HMA),这是在实模式下可以访问的保护模式的存储器区域。

在保护模式下 80286 CPU 可以访问 16MB 地址空间,1MB 以上的地址空间称作扩展存储区(XMS),图 5-22 给出了 IBM PC/AT 机的存储空间分配。

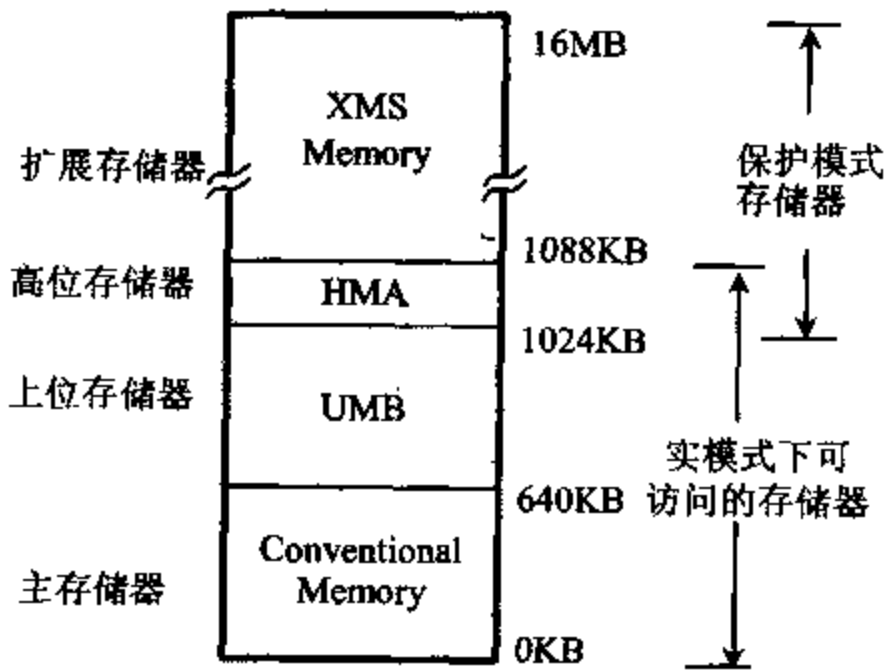


图 5-22 IBM PC/AT 机的存储空间分配

三、PC 机中存储器的使用

下面以 PC/AT 机为例,来讨论 PC 机中各类存储器的定义、特点和它们之间的关系。

1. 主存储器(Conventional Memory)

主存储器是指地址范围为 0~640KB 的 RAM,也称为基本存储器或常规存储器。

图 5-23 给出了主存储器 640KB 的分配。

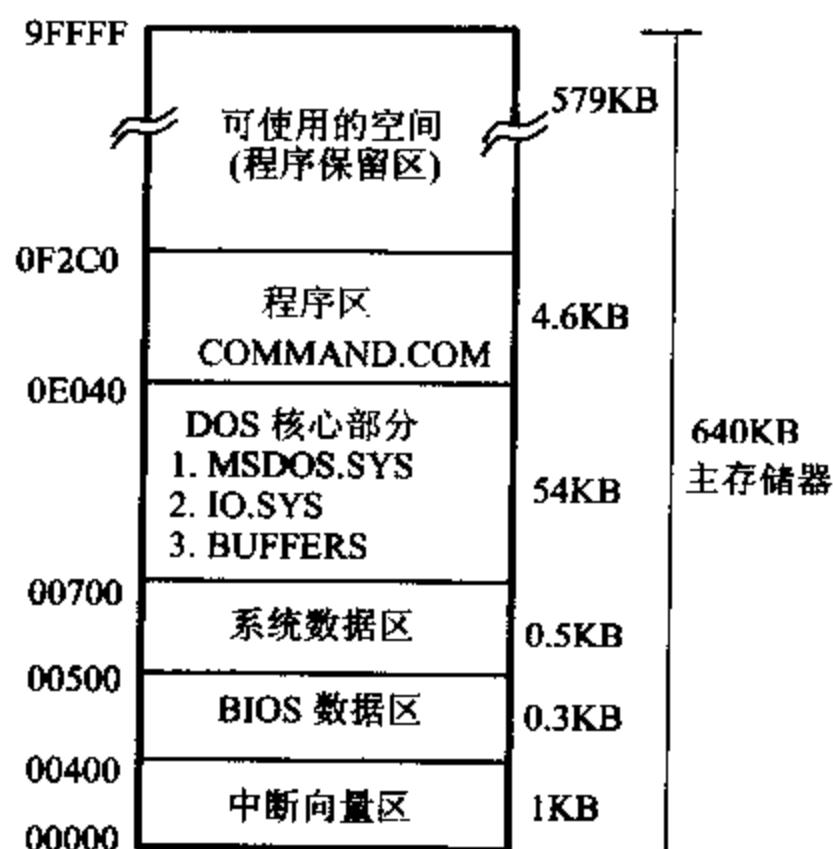


图 5-23 主存储器 640KB 的分配

2. 上位存储器 UM(Upper memory)

从 640KB 到 1024KB 的内存保留区称为上位存储器(UM),留给系统配置使用,内存保留区空间分配如图 5-24 所示。

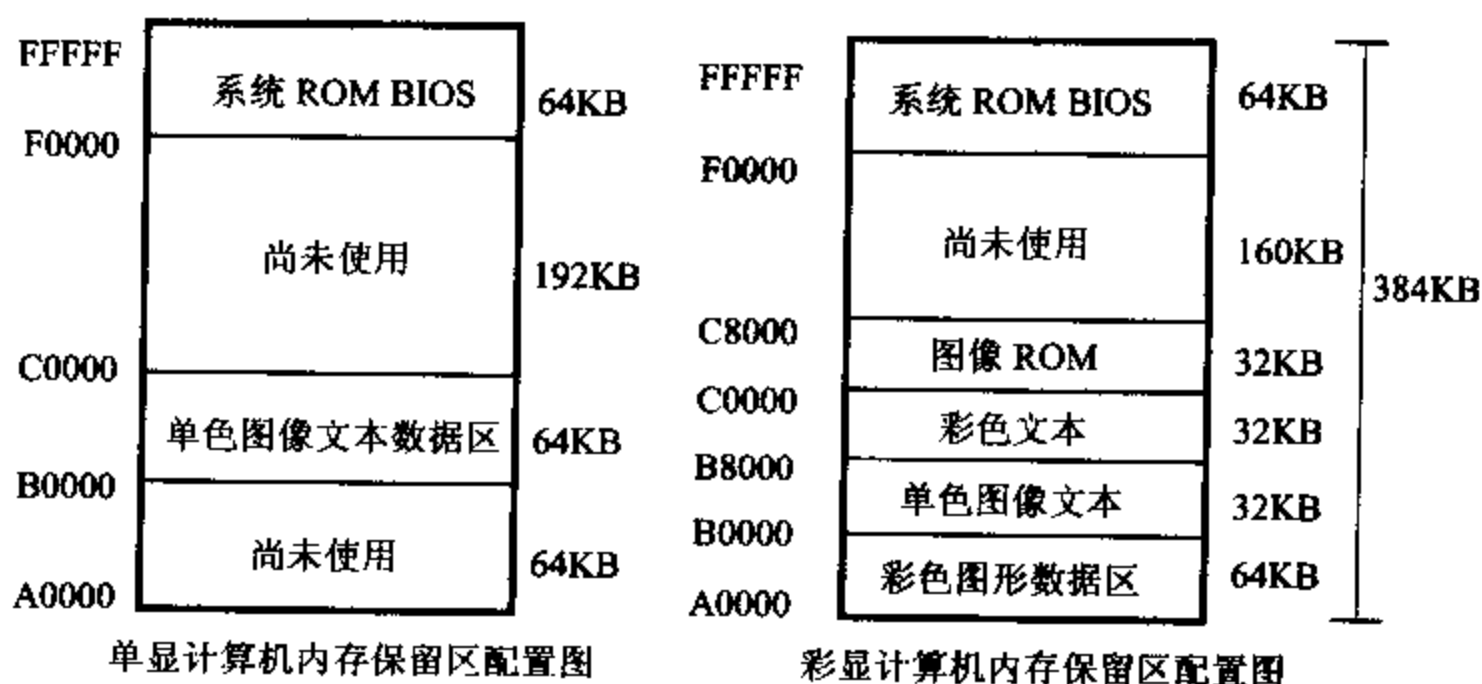


图 5-24 内存保留区配置

从该配置图中可以看到,内存保留区的 384KB 里面有不少区域未使用,单色显示器有 256KB 未用,彩色显示器系统上有 160KB 未用。但 384KB 的内存保留区中未用的区域不能直接用来存取数据,要利用一些软件(如 EMM386. EXE)使这个区域“再生”,将未用的那些区域变成所谓的上位存储块(Upper Memory Block, UMB),可以在这一区域执行一些常驻程序或驱动程序,如 DOS 的 APPEND、DOSKEY 或 MOUSE. SYS 等,这样使主存储器中存放 DOS 的区域减少,供用户使用的区域加大。图 5-25 说明了这种情况。

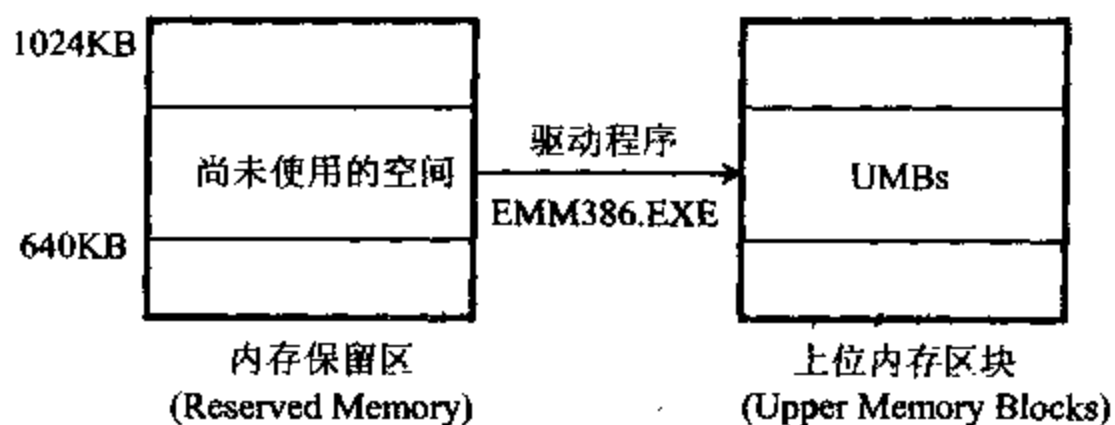


图 5-25 上位存储器“再生”

3. 扩展存储器(Extended Memory)

扩展存储器是 640KB 以上的存储器,不包括 384KB 的内存保留区的区域。注意,DOS 的寻址能力为 1MB 存储空间,是指 640KB 主存储器和 384KB 的内存保留区,而人们所说的 1MB 内存容量是指 PC 机主板上实际拥有的 1024KB RAM,这两个概念要分清楚。如图 5-26 所示,1MB RAM 的实际地址是 0KB~640KB 和 1024KB~1408KB,即 1024KB 以上的 384KB 为扩展存储器。如果有 16MB 的存储器,扣除 640KB 主存储器,剩下的就是扩展存储器(内存保留区不包括在内)。

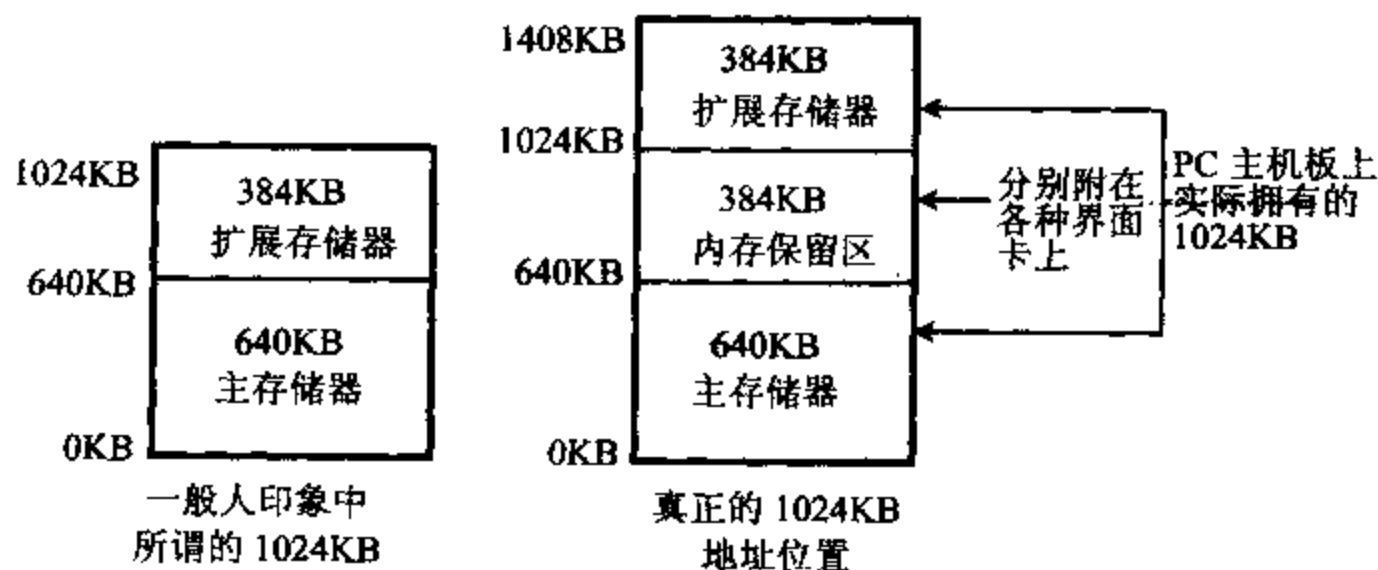


图 5-26 扩展存储器配置

扩展存储器管理程序 HIMEM. SYS 按照扩展存储器规范(XMS)来管理扩展存储器,人们也将按此规范管理的扩展存储器称为 XMS。

4. 高位存储器(High Memory Area)

扩展存储器中位于 1024KB~1088KB 的 64KB 存储区,被称为高位内存区(HMA)。

当在 CONFIG.SYS 中设置了 HIMEM.SYS 后,可将 DOS 核心载入高位内存区,节省主存储器空间。图 5-27 给出了高位内存区使用的示意图。

设置方法为:DEVICE=C:\DOS\HIMEM.SYS

DOS=HIGH

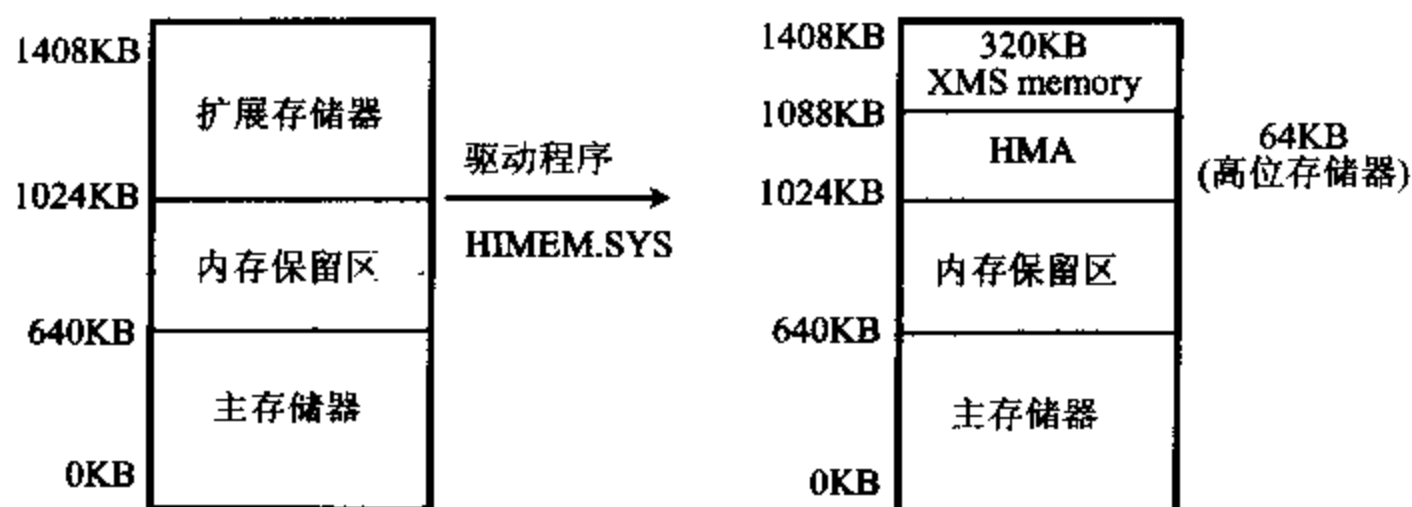


图 5-27 高位内存区的使用

5. 扩充存储器 (Expanded Memory)

8086/8088 只能寻址 1MB,为补充内存的不足,人们设计了内存扩充卡,按照扩充存储器规范(EMS),用扩充存储器管理程序(EMM)来管理。人们也将按此规范管理的扩充存储器称为 EMS。图 5-28 给出了扩充存储器的使用原理。

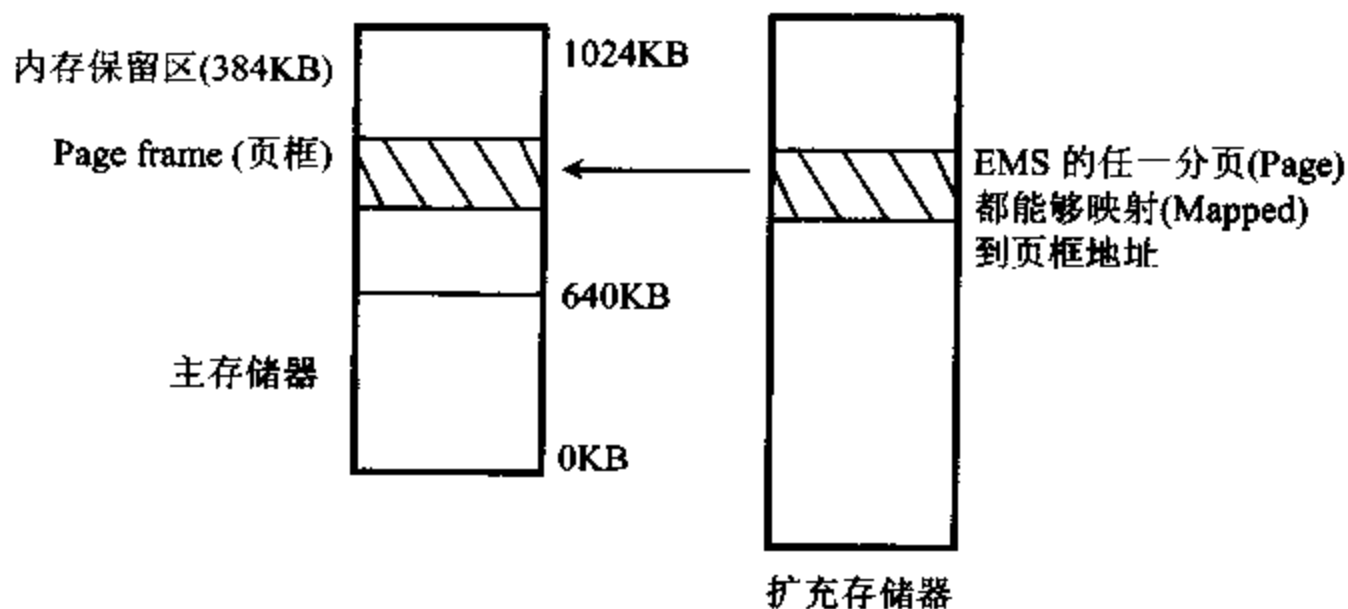


图 5-28 扩充存储器使用的原理

使用扩充存储器的基本原理是将扩充存储器分成“区段”,每一个区段叫做分页,每个分页容量为 16KB,4 个分页组合成 1 个页框,用来交换此地址中的内容。当主存储器内的程序要使用扩充存储器时,扩充存储器管理程序(EMM)会先把适当的页数拷贝到一个称作页框的区域中,页框放在内存保留区内,程序将从页框中取得所要的数据。

在上位存储器中,将未使用的 64KB 作为页框,用于读写数据。通过管理程序 EMM,自然会将每一页所存放的数据记录到扩充存储器内,并知道何时需要取出哪几页的数据到页框中,何时需要更换页框内的数据供程序使用。

由于扩充存储器管理采用了这种分页方法,在一个时间内只能存取 64KB 信息,故扩充存储器 EMS 的执行速度慢一些。可以利用扩充存储器管理程序 EMM386. EXE,使用扩展存储器来模拟扩充存储器。

6. 五种存储器的关系

五种存储器的关系在图 5-29 表示。

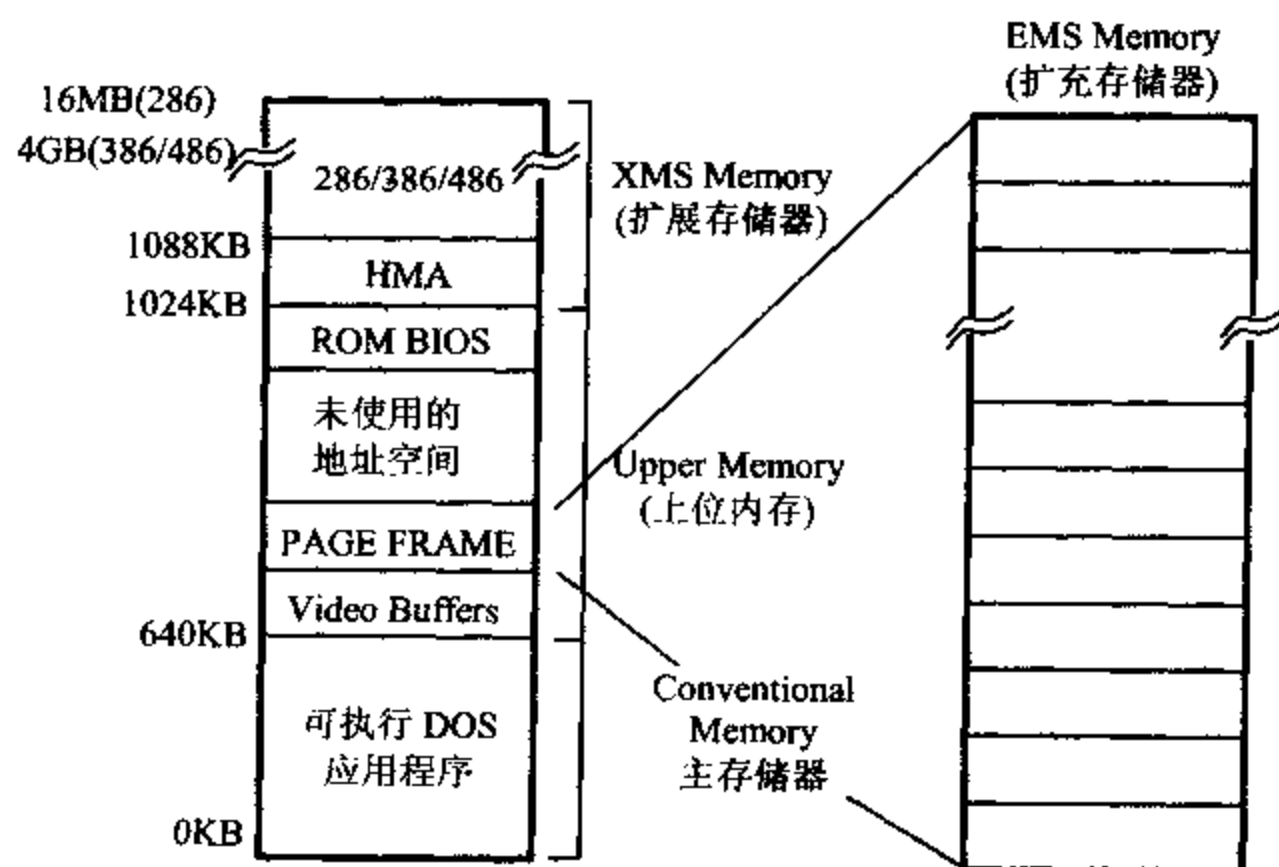


图 5-29 五种存储器类型和关系图

(1) 主存储器指一开机 DOS 进入的 640KB。通常所有的程序、数据都必须在这个空间中进行处理。

(2) 高位存储器 (HMA) 和扩充存储器 (EMS), 都是通过驱动程序对扩展存储器 (XMS)“模拟”来的。

(3) 在 CONFIG. SYS 中第一行加入下列命令就可以设置高位内存区,将 DOS 核心载入。即

DEVICE=HIMEM. SYS

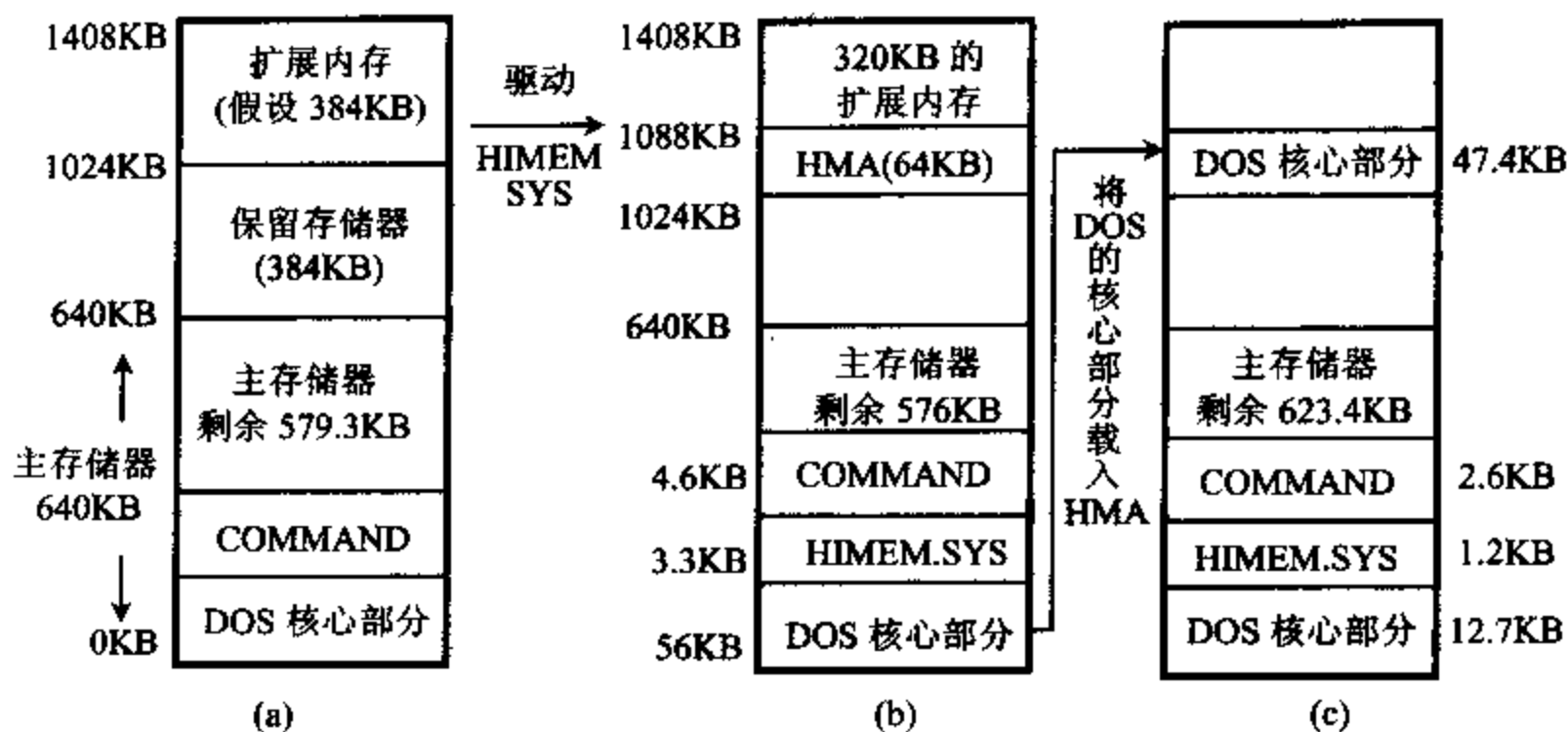
DOS=HIGH

(4) 在 CONFIG. SYS 中第二行加入下列命令就可以设置上位内存块,提供给小型常驻程序使用。并将扩展存储器“模拟”成扩充存储器,提供给支持 EMS 的软件(例如 Lotos 123, AutoCAD 等)使用。即

DEVICE=EMM386. EXE FRAME=C800 RAM ;彩显适用,单显改 C000

(5) 扩展存储器管理程序 HIMEM. SYS 既管理了扩展存储器的存取方式,又管理了 HMA 和 UMB。扩充存储器管理程序 EMM386. EXE 负责将 XMS 模拟成扩充存储器。

图 5-30 给出了 HIMEM. SYS 和 DOS=HIGH 的使用情况说明。



- (a) 原始硬盘配置, 开机后, DOS 占用 56KB, COMMAND 部分占 4.6KB, 剩余 579.3KB。
 (b) 驱动高位内存区 HMA, 在 CONFIG.SYS 中加入命令: DEVICE=HIMEM.SYS, 重新开机后, 产生 HMA, 主存储器剩余 576KB。
 (c) 将 DOS 核心放入 HMA, 在 CONFIG.SYS 中再加入命令: DOS=HIGH, 重新开机后, DOS 核心部分已放入 HMA, 可节省主存储器 47.4KB。

图 5-30 HIMEM.SYS 和 DOS=HIGH 使用说明

习 题

1. 静态 RAM 与动态 RAM 有何区别?
2. ROM、PROM、EPROM、EEPROM 在功能上各有何特点?
3. DRAM 的 $\overline{\text{CAS}}$ 和 $\overline{\text{RAS}}$ 输入的用途是什么?
4. 什么是 Cache? 作用是什么? 处在微处理机中的什么位置?
5. 直接映象 Cache 和成组相联 Cache 的组成结构有什么不同?
6. 为什么要保持 Cache 内容与主存储器内容的一致性? 为了保持 Cache 与主存储器内容的一致性应采取什么方法?
7. 用 1024×1 位的 RAM 芯片组成 $16K \times 8$ 位的存储器, 需要多少芯片? 在地址线中有多少位参与片内寻址? 多少位组合成片选择信号? (设地址总线为 16 位)
8. 现有一存储体芯片容量为 512×4 位, 若要用它组成 4KB 的存储器, 需要多少这样的芯片? 每块芯片需要多少寻址线? 整个存储系统最少需要多少寻址线?
9. 利用 1024×8 位的 RAM 芯片组成 $4K \times 8$ 位的存储器系统, 试用 $A_{15} \sim A_{12}$ 地址线用线性选择法产生片选信号, 存储器的地址分配有什么问题, 并指明各芯片的地址分配。
10. 当从存储器偶地址单元读一个字节数据时, 写出存储器的控制信号和它们的有效逻辑电平信号。(8086 工作在最小模式)
11. 当要将一个字写入到存储器奇地址开始的单元中去, 列出存储器的控制信号和它

们的有效逻辑电平信号。(8086 工作在最小模式)

12. 设计一个 $64\text{K} \times 8$ 存储器系统,采用 74LS138 和 EPROM 2764 器件,使其寻址存储器的地址范围为 $40000\text{H} \sim 4\text{FFFFH}$ 。

13. 用 $8\text{K} \times 8$ 位的 EPROM 2764, $8\text{K} \times 8$ 位的 RAM 6264 和译码器 74LS138 构成一个 16K 字 ROM, 16K 字 RAM 的存储器子系统。8086 工作在最小模式,系统带有地址锁存器 8282,数据收发器 8286。画出存储器系统与 CPU 的连接图,写出各块芯片的地址分配。

14. 上题中若从 74LS138 的 Y_2 开始选择 ROM 和 RAM 芯片,写出各块芯片的地址分配。

15. 8086CPU 组成的计算机系统中, 1MB 存储空间从使用上分成哪几部分?

16. 何谓上位存储器? 有什么用途? 何谓高位内存区?

17. 扩展存储器与扩充存储器有什么区别? 如何使用扩充存储器?

第六章 I/O 接口和总线

6-1 I/O 接口

一、I/O 接口的功能

1. 采用 I/O 接口的必要性

计算机有各种用途,但不论用于何种场合,都离不开信息处理。所处理的信息,甚至包括完成信息处理的程序本身,均要由输入设备提供;而处理后的结果数据,则要送给输出设备,以各种形式报告给用户。例如,键盘、鼠标器、磁盘和扫描仪等是大家熟悉的输入设备,而磁盘、CRT 显示器、打印机、X-Y 绘图仪等则是最常见的输出设备。所有这些设备统称为计算机的外部设备(Peripherals),简称外设或 I/O 设备。为了让这些外部设备按计算机的要求有次序地输入或接收数据,计算机的 CPU 还要能控制输入输出设备启动或停止,以及了解它们的当前工作状态,并据此送出相应的控制命令。通常,我们把计算机与外设间的这种交换数据、状态和控制命令的过程统称为通信(Communication)。

CPU 与外部设备交换信息的过程,和它与存储器交换数据那样,也是在控制信号的作用下通过数据总线来完成的。但后者要简单得多,因为存储器芯片的存取速度与微处理器的时钟频率在同一数量级,而且存储器本身又具有数据缓冲的能力,因此,CPU 可以通过数据总线很方便地与存储器进行数据交换。而外部设备种类繁多,从工作原理来讲,可分为机械式、电动式、电子式和其它形式等几类,它们对所传输的信息的要求也各不相同,这就给计算机和外设之间的信息交换带来以下一些问题:

(1) 速度不匹配

CPU 的速度很高,而外设的速度要低得多,而且不同的外设速度差异甚大,它们之中既有每秒钟能传送兆位数量级的硬磁盘,也有每秒钟只能打印百位字符的串行打印机或速度更慢的键盘。

(2) 信号电平不匹配

CPU 所使用的信号都是 TTL 电平,而外设大多是复杂的机电设备,往往不能用 TTL 电平所驱动,必须有自己的电源系统和信号电平。

(3) 信号格式不匹配

CPU 系统总线上传送的通常是 8 位、16 位或 32 位的并行数据,而各种外设使用的信息格式各不相同。有些设备上用的是模拟量,而有些是数字量或开关量;有些设备上的信息是电流量,而有些却是电压量;有些设备采用串行方式传送数据,而有些则用并行方式。

(4) 时序不匹配

各种外设都有自己的定时和控制逻辑,与计算机的 CPU 时序不一致。因此输入输出设备不能直接与 CPU 的系统总线相连,必须在 CPU 与外设之间设置专门的接口(Interface)电路来解决这些问题。

2. 接口的功能

接口电路是专门为解决 CPU 与外设之间的不匹配、不能协调工作而设置的,它处在总线和外设之间,一般应具有以下基本功能:

(1) 设置数据缓冲以解决两者速度差异所带来的不协调问题

CPU 和外设间速度不协调的问题可以通过设置数据缓冲来解决,也就是事先把要传送的数据准备在那里,在需要的时刻完成传送。经常使用锁存器和缓冲器,并配以适当的联络信号来实现这种功能。

例如,当快速的 CPU 要将数据传送到慢速的外设时,事先可把数据送到锁存器中锁住,等外设做好接收数据的准备工作后再把数据取走。反之,若外设要把数据送到 CPU 去,也可先把数据送进输入寄存器(它也是一种锁存器),再发联络信号通知 CPU 来取走数据。在输入数据时,多个外设不容许同时把数据送到数据总线上,以免引起总线竞争而毁坏总线。为此,必须在输入寄存器和数据总线之间放一个缓冲器,只有 CPU 发出的选通命令到达时,特定的输入缓冲器被选通,外设送来的数据才抵达数据总线。有些接口电路中还用 RAM 芯片作缓冲器,如键盘或打印机接口,这样可存储一批数据,为主机和外设间进行批量数据交换创造了条件。

(2) 设置信号电平转换电路

外设和 CPU 之间信号电平的不一致问题,可通过在接口电路中设置电平转换电路来解决。典型的例子是计算机和外设间的串行通信,可采用 MC1488、MC1489、MAX232 和 MAX233 等芯片来实现电平转换,对此我们将在第十一章中详细讨论。

(3) 设置信息转换逻辑以满足对各自格式的要求

由于外设传送的信息可以是模拟量,也可以是数字量或开关量,而计算机只能处理数字信号。因此,模拟量必须经模/数转换(A/D)变换成数字量后,才能送到计算机去处理。而计算机送出的数字信号也必须经数/模转换(D/A)变成模拟信号后,才能驱动某些外设工作。于是,就要用包含 A/D 转换器和 D/A 转换器的模拟接口电路来完成这些功能。至于开关量,可以有两种状态,如开关的闭合和断开,阀门的打开和关闭等,也要被转换成用 0 或 1 表示的一位数字量后,才能被计算机识别或接受它的控制。

虽然大多数外设使用的都是数字量,当它们与计算机通信时,仍然存在信号的转换问题。因为计算机的数据总线传送的通常是 8 位或 16 位的并行数据,而有些外设采用串行方式传送数据,所以必须将 CPU 送出的并行数据,经并变串电路转换成串行信息后,才能送给串行外设。反之,串行设备的数据,也必须经串变并的转换后才能送给 CPU。即使是使用并行数据的外设,其数据长度和数据格式也可能与主机的不同,因而也需要进行数据格式的转换。这些工作均可由专门的接口电路来完成。

(4) 设置时序控制电路来同步 CPU 和外设的工作

接口电路接收 CPU 送来的命令或控制信号、定时信号,实施对外设的控制与管理,外设的工作状态和应答信号也通过接口及时返回给 CPU,以握手联络(handshaking)信号来保证主机和外部 I/O 操作实现同步。

(5) 提供地址译码电路

CPU 要与多个外设打交道,一个外设又往往要与 CPU 交换几种信息,因而一个外设接口中通常包含若干个端口,而在同一时刻,CPU 只能与某一个端口交换信息。外设端口不能长期与 CPU 相连,只有被 CPU 选中的设备才能接收数据总线上的数据,或将外部信息送到数据总线上。这就需要有外设地址译码电路,使 CPU 在同一时刻只能选中某一个 I/O 端口。

此外,在接口电路中还有输入输出控制、读/写控制及中断控制等逻辑。当然,并不是所有接口都具备上述全部功能,所控制的外设不同,接口电路的功能可能不完全一样。

由此可见,I/O 接口电路是外设和计算机之间传送信息的交接部件,也有人称它为界面,它使两者之间能很好地协调工作,每一个外设都要通过接口电路才能和主机相连。随着大规模集成电路技术的发展,出现了许多通用的可编程接口芯片,可用它们来方便地构成接口电路。这里先介绍几种简单的输入输出接口芯片的工作原理和使用方法,后面几章将重点讨论几种常用的可编程 I/O 接口芯片的工作原理、编程方法以及这些芯片如何与 CPU 和外设相连等问题。

二、简单的输入输出接口芯片

最常用的简单输入输出接口芯片主要有缓冲器(Buffer)和锁存器(Latch)。

1. 缓冲器 74LS244 和 74LS245

连接在总线上的缓冲器都具有三态输出能力,在 CPU 或 I/O 接口电路需要输入输出数据时,在它的使能控制端 EN(或 G)作用一个低电平脉冲,使它内部的各缓冲单元接通,即处在输出 0 或 1 的透明状态,数据被送上总线。当使能脉冲撤除后,它处在高阻态。这

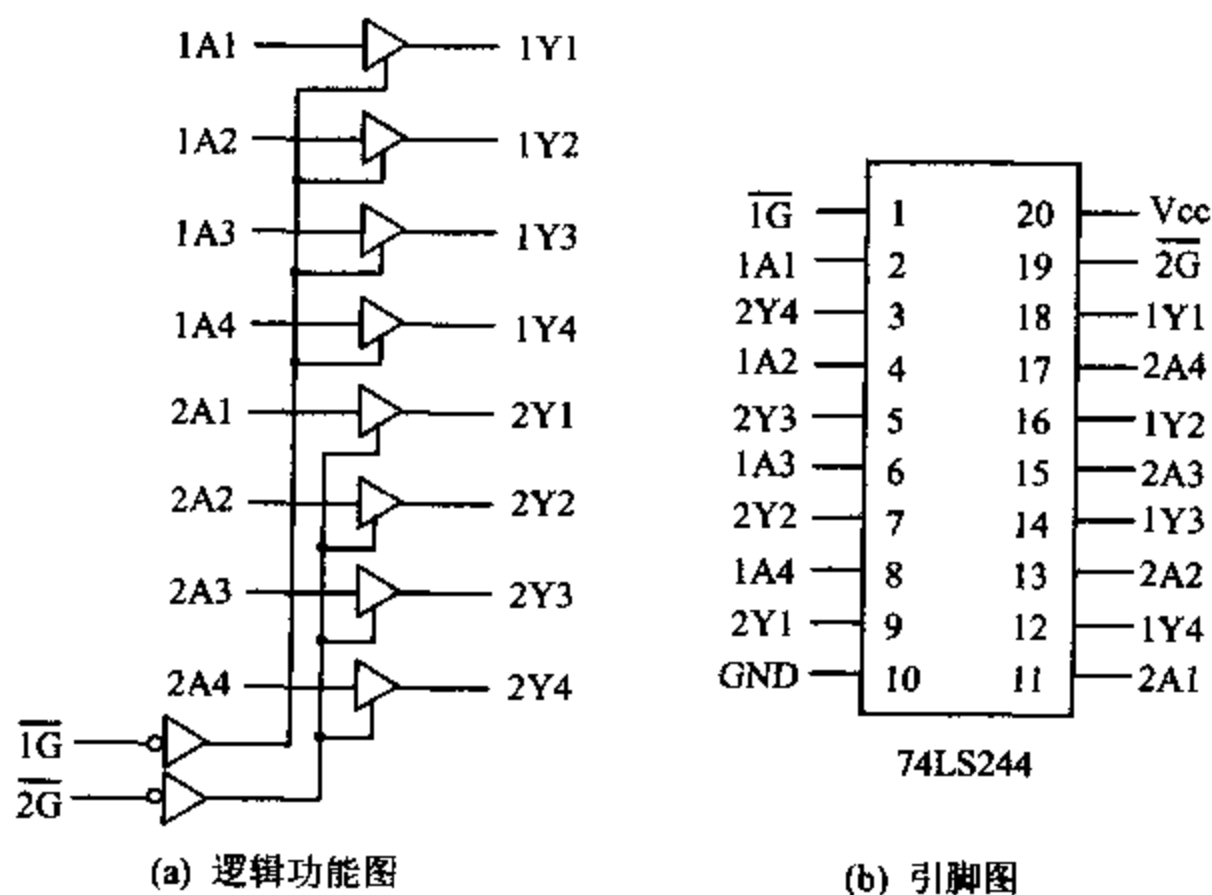


图 6-1 74LS244 逻辑功能和引脚图

时,各缓冲单元像一个断开的开关,等于将它所连接的电路从总线上脱开。74LS244 和 74LS245 就是最常用的数据缓冲器。除缓冲作用外,它们还能提高总线的驱动能力。

(1) 74LS244

74LS244 是一种 8 路数据缓冲器,其逻辑功能和引脚如图 6-1 所示。

由图可见,该缓冲器内部包含 8 个三态缓冲单元,它们被分为两组,每组 4 个单元,分别由门控信号 $\overline{1G}$ 和 $\overline{2G}$ 控制。当 $\overline{1G}$ 为低电平时,A 输入端 1A1~1A4 的高电平或低电平将被传送到 Y 输出端 1Y1~1Y4,当 $\overline{2G}$ 为低电平时,2A1~2A4 的信号被传送到 2Y1~2Y4,当 $\overline{1G}$ 和 $\overline{2G}$ 为高电平时,输出呈高阻态。把它用于 8 位数据总线时,可将 $\overline{1G}$ 和 $\overline{2G}$ 端连在一起,由一个片选信号来控制。74LS244 常用来构成外设输入数据端口,这时它的输入端 A 与外设数据线相连,而输出端 Y 并接在 CPU 的数据总线上。74LS244 是一种单向数据缓冲器,数据只能从 A 端传送到 Y 端,若要进行双向数据传送,可选用双向数据总线缓冲器 74LS245。

(2) 74LS245

74LS245 的逻辑功能和引脚如图 6-2 所示。它内部包含 8 个双向、三态缓冲器。控制信号中,除了一个低电平有效的门控信号输入端 $\overline{G}$ 之外,它还有一个方向控制端 DIR。只有当 $\overline{G}$ 为低电平时,数据才能从 A 传送到 B 或从 B 传送到 A;当 DIR 为高电平时,数据从 A 端传向 B 端,而低电平则让数据从 B 端流向 A 端。

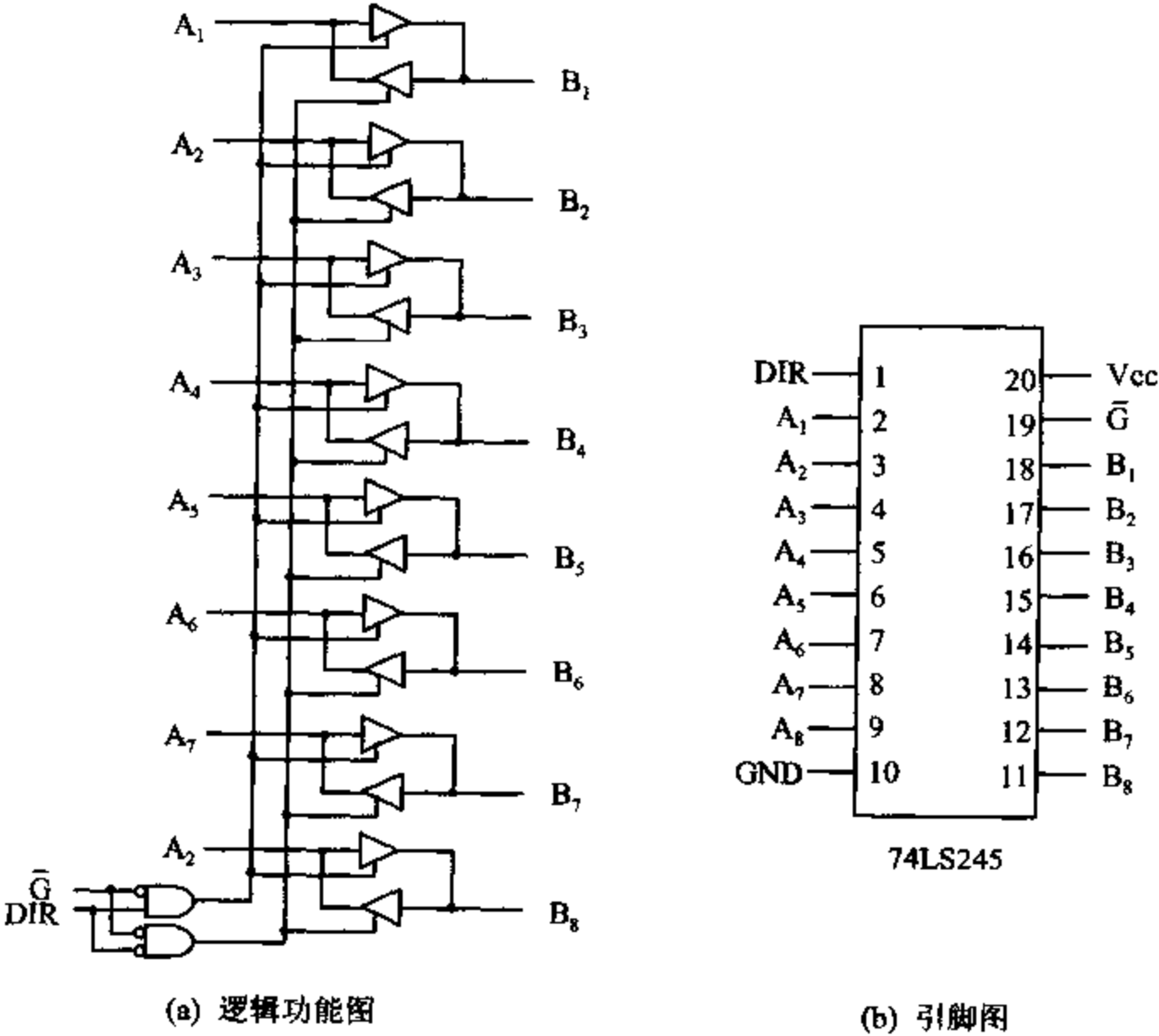


图 6-2 74LS245 逻辑功能和引脚图

2. 锁存器 74LS373

锁存器具有暂存数据的能力,能在数据传输过程中将数据锁住,然后在此后的任何时刻,在输出控制信号的作用下将数据传送出去。

74LS373 是一种常用的 8D 锁存器,它可以直接挂到总线上,并具有三态总线驱动能力。图 6-3 是它的逻辑图,括号中的数字为引脚号,而表 6-1 是它的真值表。

表 6-1 74LS373 真值表

$\overline{OE}$	G	D	O
低	高	高	高
低	高	低	低
低	低	×	锁存
高	×	×	高阻态

从图 6-3 可见,74LS373 由一个 8 位寄存器和一个 8 位三态缓冲器构成,寄存器的每个单元则是一个具有记忆功能的 D 触发器。它有两个控制输入端,即输入使能端 G 和允许输出端 $\overline{OE}$ 。当 G 为高电平时,加在各触发器的 D 输入端的 0 或 1 电平被打到它的 Q 端,且记忆在那里。 $\overline{Q}$ 端电平与 Q 端的相反。此后,若在 $\overline{OE}$ 端作用一个低电平脉冲,记忆在 $\overline{Q}$ 端的电平将经三态门再反相后传送到输出端 O。可见,如果这两个控制脉冲同时作用,锁存器的输出 O 将随输入 D 而变,呈透明态。若将 G 端的高电平撤除使之变成低电平, $\overline{OE}$ 保持低电平,输出端 O 将是前面锁存的数据,这时 D 端的任何变化都不影响输出。如果 $\overline{OE}$ 端为高电平,则不论 G 的电平如何,输出将呈高阻态,与总线断开。 $\overline{OE}$ 、G、D 和 O 信号之间的关系如表 6-1 所示。

在实际应用中,可根据需要设置 74LS373 的控制信号。例如,希望先输入数据,然后在以后的适当时刻再输出,可对 G 和 $\overline{OE}$ 分别进行控制;如果只要使用它的记忆功能,不需要三态缓冲,可直接把 $\overline{OE}$ 端接地,仅控制 G。

像 74LS373 这样具有三态输出缓冲功能的锁存器还有 74LS374,它们的内部结构几乎相同,唯一的不同是后者必须用 G 信号的上升沿来触发。此外,74LS273 是不含三态门的 8D 锁存器,相当于一个 8 位寄存器,可用在那些仅需要锁存数据,不要三态缓冲的场合。不少中、大规模集成接口芯片的内部,也都具有锁存器和缓冲器逻辑。锁存器的用途较广,既可用于构成输出端口,也能用于输入端口的设计。

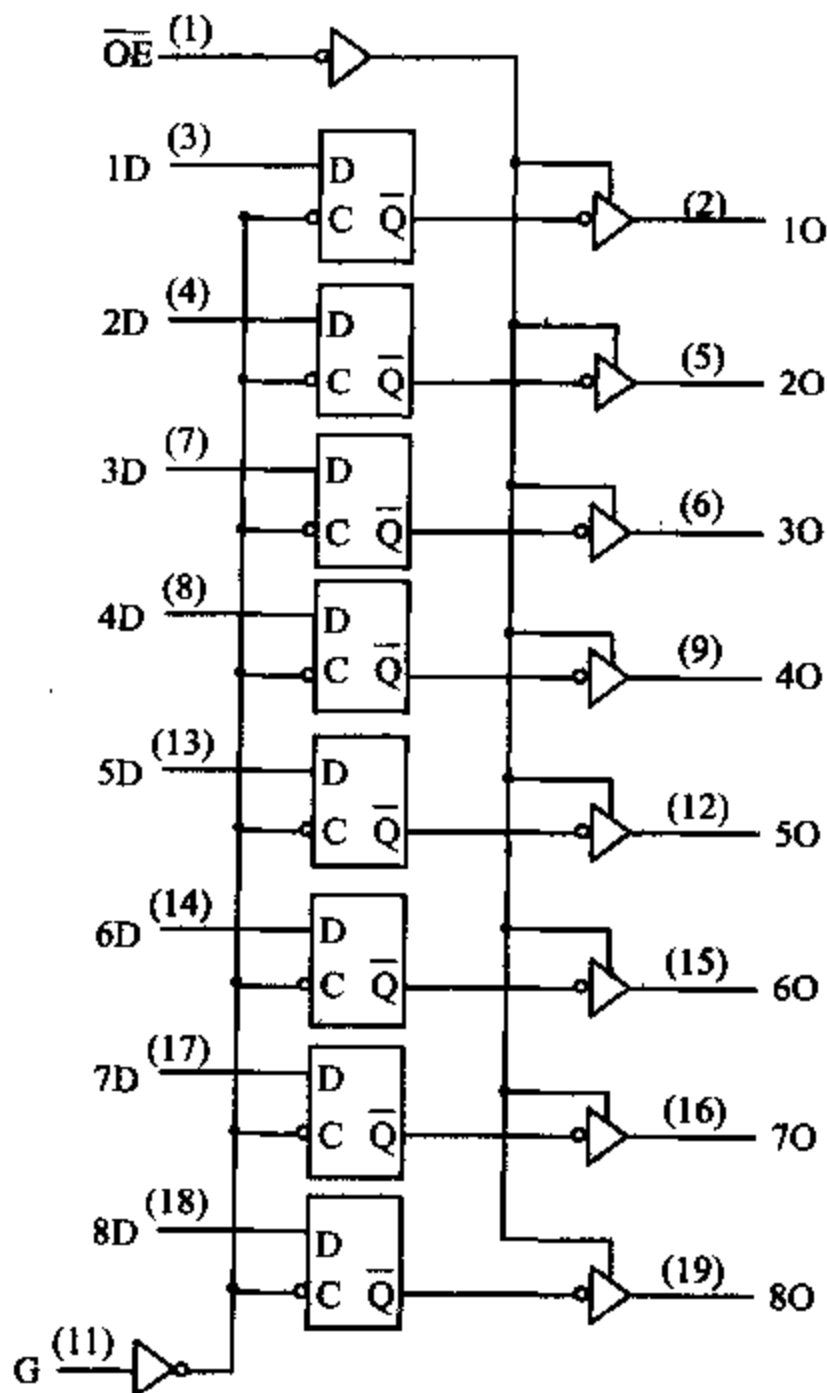


图 6-3 74LS373 逻辑图

三、I/O 端口及其寻址方式

1. I/O 端口

CPU 与外设通信时,传送的信息主要包括数据信息、状态信息和控制信息。在接口电路中,这些信息分别进入不同的寄存器,通常将这些寄存器和它们的控制逻辑统称为 I/O 端口(Port),CPU 可对端口中的信息直接进行读写。在一般的接口电路中都要设置以下几种端口:

(1) 数据端口

数据端口(Data Port)用来存放外设送往 CPU 的数据以及 CPU 要输出到外设去的数据。这些数据是主机和外设之间交换的最基本的信息,长度一般为 1~2 字节。数据端口主要起数据缓冲的作用。

(2) 状态端口

状态端口(Status Port)主要用来指示外设的当前状态。每种状态用 1 位表示,每个外设可以有几个状态位,它们可由 CPU 读取,以测试或检查外设的状态,决定程序的流程。需要指出的是,除了状态端口中的内容外,接口电路中往往还会有若干状态线,它们用电平的高低来指示外设当前状态,实现 CPU 与外设之间的通信联络,它们的名称和电平与状态位之间不一定完全对应。

接口电路状态端口中最常用的状态位有:

① 准备就绪位(Ready)。如果是输入端口,该位为 1,表明端口的数据寄存器已准备好数据,等待 CPU 来读取;当数据被取走后,该位清为 0。若是输出端口,这一位为 1,表明端口中的输出数据寄存器已空,即上一个数据已被外设取走,可以接收 CPU 的下一个数据了;当新数据到达后,这一位便被清 0。不同的器件,这个状态位所用的名称可能不一样,例如,也可能用输入缓冲器满,输出缓冲器满,发送缓冲器空等等,但它们的含义和用法均与准备就绪位类同。

② 忙碌位(Busy)。用来表明输出设备是否能接受数据。若该位为 1,表示外设正在进行输出数据传送操作,暂时不允许 CPU 送新的数据过来。本次数据传送完毕,该位清 0,表示外设已处于空闲状态,又允许 CPU 将下一个数据送到输出端口。

③ 错误位(Error)。如果在数据传送过程中发现产生了某种错误,可将错误状态位置 1。CPU 查到出错状态后便进行相应的处理,例如重新传送或中止操作。系统中可以设置若干个错误状态位,用来表明不同性质的错误,如奇偶校验错、溢出错等。

除了上述几个状态位外,状态寄存器各位的设置可随设备种类的要求而定,无统一规定。

(3) 命令端口

命令端口(Command Port)也称为控制端口(Control Port),它用来存放 CPU 向接口发出的各种命令和控制字,以便控制接口或设备的动作。常见的命令信息位有启动位、停止位、允许中断位等。接口芯片不同,控制字的格式和内容是各不相同的,常见的控制字有方式选择控制字,操作命令字等。

通常,CPU 与外设交换的数据是以字节为单位进行的,因此一个外设的数据端口含有

8 位。而状态口和命令口可以只包含一位或几位信息,所以不同外设的状态口允许共用一个端口,命令口也可共用。在自己设计的 I/O 接口中,常用 D 触发器和三态缓冲器来构成这两种端口。

由上面的介绍可以看到,数据信息、状态信息和控制信息的含义各不相同,按理这些信息应分别传送。但在微型计算机系统中,CPU 通过接口和外设交换数据时,只有输入(IN)和输出(OUT)两种指令,所以只能把状态信息和命令信息也都当作数据信息来传送,并且将状态信息作为输入数据,控制信息作为输出数据,于是三种信息都可以通过数据总线来传送了。但要注意,这三种信息被送入三种不同端口的寄存器,因而能实施不同的功能。

2. I/O 端口的寻址方法

CPU 对外设的访问实质上是对 I/O 接口电路中相应的端口进行访问,因此和存储器那样,也需要由译码电路来形成 I/O 端口地址。I/O 端口的编址方式有两种,分别称为存储器映象寻址方式和 I/O 指令寻址方式。

(1) 存储器映象寻址方式

若把系统中的每一个 I/O 端口都看作一个存储单元,并与存储单元一样统一编址,这样访问存储器的所有指令均可用来访问 I/O 端口,不用设置专门的 I/O 指令,这种寻址方式称为存储器映象的 I/O 寻址方式(Memory Mapped I/O)。这种方式实际上是把 I/O 地址映射到存储空间,作为整个存储空间的一小部分。换句话说就是,系统把存储空间的一小部分划出来供外设使用,大部分仍归存储单元所有,这就是存储器映象寻址方式名称的由来。如 Motorola 的 MC6800、MC68000 及 68HC05 等微处理器便采用这种方法访问 I/O 设备。

这种寻址方式的优点是微处理器的指令集中不必包含 I/O 操作指令,简化了指令系统的设计;能用类型多、功能强的访问存储器指令,对 I/O 设备进行方便、灵活的操作;I/O 地址空间可大可小,能根据实际系统上的外设数目来调整。缺点主要是 I/O 端口占用了存储单元的地址空间,为了尽可能减小所占的内存空间,必须用全译码方式来形成 I/O 地址,使 I/O 译码电路变得较复杂。此外,访问存储器的指令一般要比 I/O 指令长一、两个字节,这会延长输入输出操作的时间。

(2) I/O 单独编址方式

若对系统中的输入输出端口地址单独编址,构成一个 I/O 空间,它们不占用存储空间,而是用专门的 IN 指令和 OUT 指令来访问这种具有独立地址空间的端口,这种寻址方式称为 I/O 单独编址方式。8088 和 8086 等微处理器都采用这种寻址方式来访问外设。在 8086 中,用地址总线的低 16 位($A_{15} \sim A_0$)来寻址 I/O 端口,最多可以访问 $2^{16} = 65536$ 个输入或输出端口。实际应用中,输入端口和输出端口可用相同的地址,因此系统能寻址的总端口数还将扩大一倍,不过没有哪个系统上使用了这么多的 I/O 端口。CPU 中的 $M/\overline{IO}$ 控制信号用来区分是 I/O 寻址还是存储器寻址,当它为高时,表示 CPU 执行的是存储器操作,为低则是访问 I/O 端口。

这种寻址方式的优点是将输入输出指令和访问存储器的指令明显区分开,使程序清晰,可读性好;而且 I/O 指令长度短,执行的速度快,也不占用内存空间;I/O 地址译码电路较简单。不足之处是 CPU 指令系统中必须有专门的 IN 和 OUT 指令,这些指令的功能没有访问存储器指令强;另外,CPU 要能提供区分存储器读/写和 I/O 读/写的控制信号,例如 Z80 CPU 的 $\overline{MREQ}$ 和 $\overline{IORQ}$ 、8086 的 $M/\overline{IO}$ 和 8088 的 $IO/\overline{M}$ 信号。

两种寻址方式各有利弊,一般要根据所用的 CPU 类型来确定 I/O 寻址方式。例如,对于 Motorola 公司的 CPU,通常没有专门的 IN 和 OUT 指令,因此都采用存储器寻址方式编址,而对于 8086/8088 系统,习惯上都采用 I/O 寻址方式。

四、CPU 与外设间的数据传送方式

随着计算机技术的飞速发展,计算机系统中输入输出设备的种类越来越多,速度差异十分悬殊,对这些设备的控制也变得越来越复杂,CPU 与外设之间的数据传输必须采用多种控制方式,才能满足各类外设的要求。在微型机系统中,可采用的输入输出方式主要有程序控制方式、中断方式和 DMA 方式等三种。前两种方式主要由软件实现,DMA 方式主要由硬件实现。下面对这几种数据传送方式做一些简要的介绍。

1. 程序控制方式

程序控制传送方式是指 CPU 与外设之间的数据传送是在程序控制下完成的,它又可以分成无条件传送和条件传送两种方式。

(1) 无条件传送方式

无条件传送方式也称为同步传送方式,主要用于对简单外设进行操作,或者外设的定时是固定的或已知的场合。也就是说,对于这类外设,在任何时刻均已准备好数据或处于接收数据状态,或者在某些固定时刻,它们处在数据就绪或准备接收状态,因此程序可以不必检查外设的状态,而在需要进行输入或输出操作时,直接执行输入输出指令。当 I/O 指令执行后,数据传送便立即进行。这是一种最简单的传送方式,所需要的硬件和软件都较少。

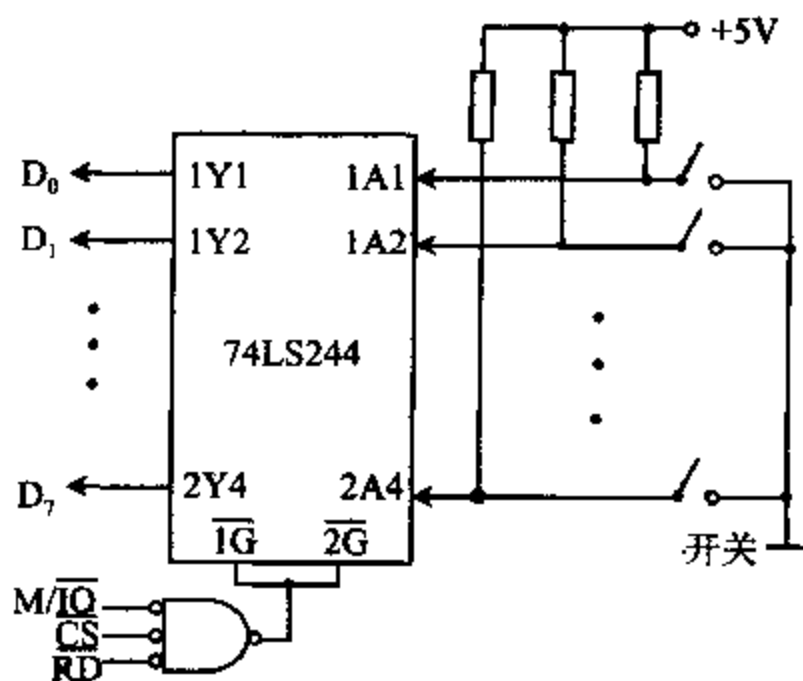


图 6-4 简单输入端口

例如,要将几个按键开关的状态输入 CPU 时,可如图 6-4 那样,将这些开关连接到一个三态缓冲器,缓冲器的输出端接到 CPU 的数据总线,构成一个最简单的输入端口。如果开关断开,上拉电阻保证缓冲器的每个输入端为高电平;当某个开关合上时,相应的输入端变为低电平。即任何时刻,开关总有一个固定的状态(开或关)。在需要了解它们的状态时,可随时执行输入指令,它使 $M/\overline{IO}$ 、 $\overline{RD}$ 和选中此端口的片选信号 $\overline{CS}$ 同时变成有效的低电平,它们相与后的低电平开启缓冲器的三态门,使各开关的当前状态以二进制值的形式出现在数据总线上,并被读入 CPU,检查这个字节各位的内容,便能了解各开关的当前状态。在其它时刻,三态门呈高阻态,将开关和数据总线隔离。

另一个无条件传送的典型例子是用程序来控制 LED 显示器的点燃和熄灭。LED 又称发光二极管,当它被加上 2V 左右的正偏电压时便发光,因此能被 TTL 电平所驱动。常用它们指示计算机或仪器的某些状态,例如 PC 机面板上指示硬盘正在工作的 HDD 指示灯就是一个最常见的 LED 应用例子。通常用一个由锁存器构成的输出端口来把 LED 接到计算

机的数据总线上,并串接一个限流电阻,如图 6-5 所示。一个 8 位输出最多能驱动 8 个 LED。图中,各 LED 的阴极接地,称为共阴连接。这样,需要点燃某个 LED 时,只要用输出指令向此端口输出一个字节,该字节中相应于点燃的 LED 的位是 1,其余各位为 0。输出指令使 $\overline{M/\overline{IO}}$ 、 $\overline{WR}$ 和片选信号 $\overline{CS}$ 同时变低,它们相与后的低电平信号经反相后触发锁存器,将输出指令送到数据总线上的值锁存在输出端,使指定的 LED 发光。由于锁存器的作用,此值能一直保存到下一条输出指令到达为止,因此在这段时间里,LED 的状态也将保持不变。显然,在这个例子中,LED 总是处于可用状态,随时都可以向这个端口输出数据,控制各 LED 的点亮或熄灭。

此外,某些设备与 CPU 之间的信息交换可能不很频繁,从而能保证每次用输出指令输出数据时,I/O 接口电路中的数据输出缓冲器总是空的,或者外设总是处于空闲状态,而在用输入指令输入数据时,接口电路中的输入缓冲器内总准备好了数据。遇到这类情况时也可采用无条件传送方式。不过,这类情况比较少见,电路上也容易实现。因此,除了上面讨论的简单输入输出端口外,通常都采用下面几种方式传送数据。

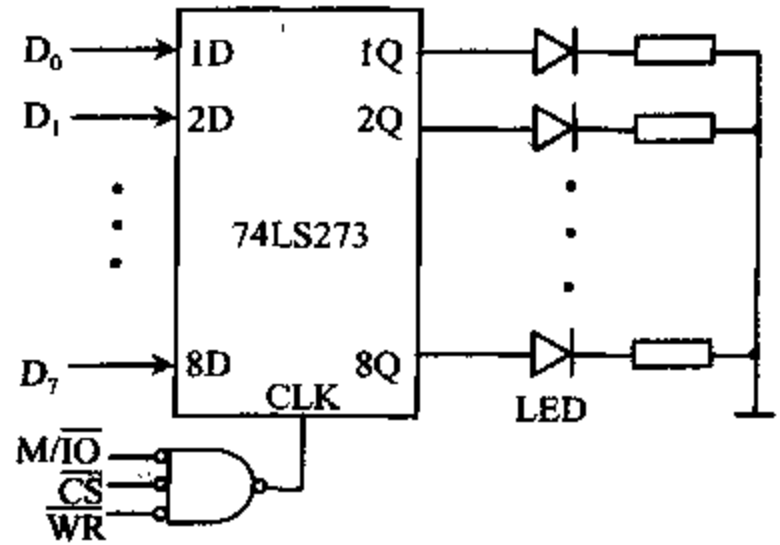


图 6-5 简单输出端口

(2) 条件传送

条件传送方式也称为查询式传送方式。一般情况下,当 CPU 用输入或输出指令与外设交换数据时,很难保证输入设备总是准备好了数据,或者输出设备已经处在可以接收数据的状态。为此,在开始传送前,必须先确认外设已处于准备传送数据的状态,才能进行传送,于是就提出了查询式传送方式。

采用这种方式传送数据前,CPU 要先执行一条输入指令,从外设的状态口读取它的当前状态。如果外设未准备好数据或处于忙碌状态,则程序要转回去反复执行读状态指令,不断检测外设状态;如果该外设的输入数据已准备好,CPU 便可执行输入指令,从外设读入数据。

查询式输入方式的接口电路和工作流程分别如图 6-6 和图 6-7 所示。

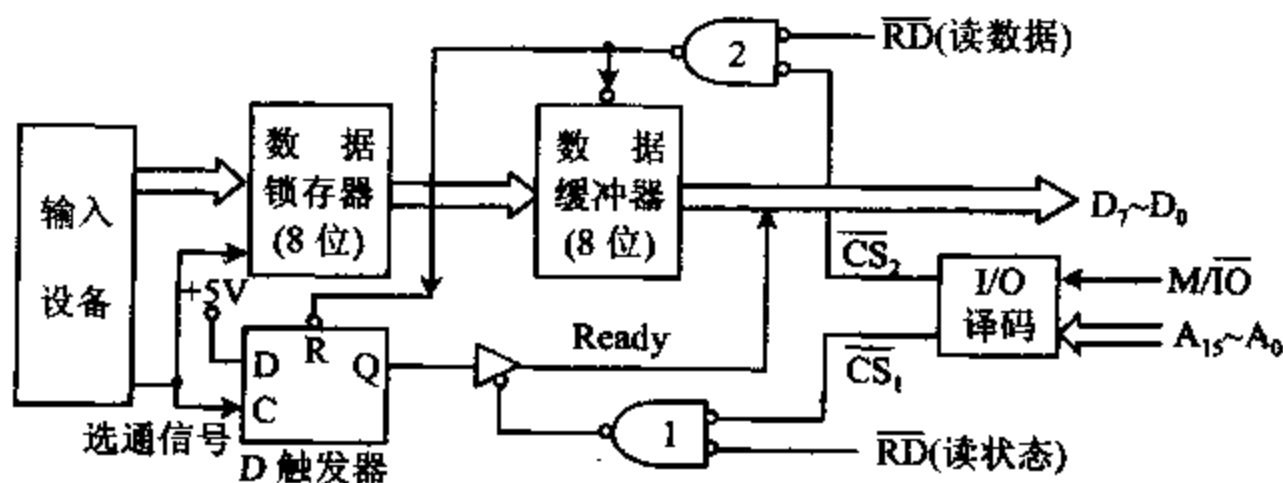


图 6-6 查询式输入接口电路

由图可见,接口电路包含状态口和输入数据口两部分,分别由 I/O 端口译码器的两个

片选信号和 $\overline{RD}$ 信号控制。状态口由一个 D 触发器和一个三态门(通常是三态缓冲器中的一路)构成。输入数据口由一个 8 位锁存器和一个 8 位缓冲器构成,它们可以被分别选通。

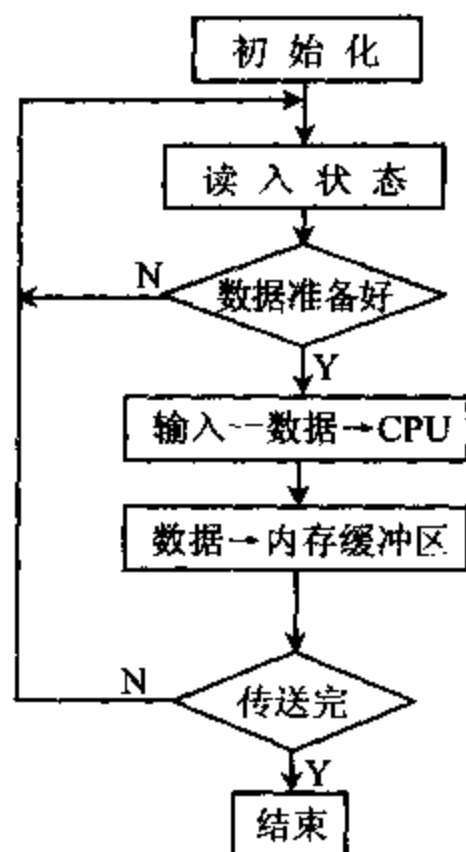


图 6-7 查询式输入流程图

当输入设备准备好数据后,就向 I/O 接口电路发一个选通信号。此信号有两个作用:一方面将外设的数据打入接口的数据锁存器中,另一方面使接口中的 D 触发器的 Q 端置 1。CPU 首先执行 IN 指令读取状态口的信息,这时 $M/\overline{IO}$ 和 $\overline{RD}$ 信号均变低。 $M/\overline{IO}$ 为低,使 I/O 译码器输出低电平的状态口片选信号 $\overline{CS1}$ 。 $\overline{CS1}$ 和 $\overline{RD}$ 经门 1 相与后的低电平输出,使三态缓冲器开启,于是 Q 端的高电平经缓冲器(1 位)传送到数据线上的 READY (D_0) 位,并被读入累加器。

程序检测到 READY 位为 1 后,便执行 IN 指令读数据口。这时 $M/\overline{IO}$ 和 $\overline{RD}$ 信号再次有效,先形成数据口片选信号 $\overline{CS2}$, $\overline{CS2}$ 和 $\overline{RD}$ 经门 2 输出低电平。它一方面开启数据缓冲器,将外设送到锁存器中的数据经 8 位数据缓冲器送到数据总线上后进入累加器,另一方面将 D 触发器清 0,一次数据传送完毕。接着就可以开始下一个数据的传送。当规定数目的数据传送完毕后,传送程序

结束,程序将开始处理数据或进行别的操作。

设:状态口的地址为 PORT-S1,输入数据口的地址为 PORT-IN,传送数据的总字节数为 COUNT-1,则查询式输入数据的程序段为:

MOV	BX, 0	;初始化地址指针 BX
MOV	CX, COUNT-1	;字节数
READ-S1:	IN AL, PORT-S1	;读入状态位
	TEST AL, 01H	;数据准备好否?
	JZ READ-S1	;否,循环检测
	IN AL, PORT-IN	;已准备好,读入数据
	MOV [BX], AL	;存到内存缓冲区中
	INC BX	;修改地址指针
	LOOP READ-S1	;未传送完,继续传送
	:	;已传送完

查询式输出方式的接口电路和工作流程分别如图 6-8 和图 6-9 所示。

与输入接口相类似,输出接口电路也包含两个端口:状态口和数据输出口。状态口也由一个 D 触发器和一个三态门构成,而数据输出口只含一个 8 位数据锁存器。

当 CPU 准备向外设输出数据时,它先执行 IN 指令读取状态口的信息。这时,低电平的 $M/\overline{IO}$ 和有效的端口地址信号使 I/O 译码器的状态口片选信号 $\overline{CS1}$ 变低, $\overline{CS1}$ 再和有效的 $\overline{RD}$ 信号经门 1 相与后输出低电平,它使状态口的三态门开启,从数据总线的 D_1 位读入 BUSY 状态。若 $BUSY=1$,表示外设处在接收上一个数据的忙碌状态。只有在 $BUSY=0$ 时,CPU 才能向外设输出新的数据。

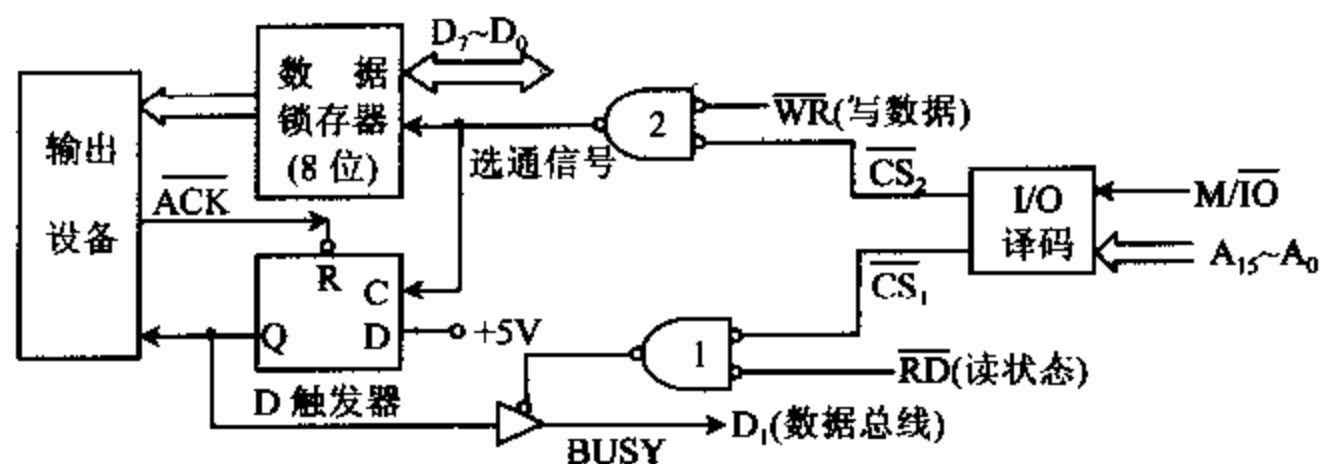


图 6-8 查询式输出接口电路

当 CPU 检查到 $BUSY=0$ 时,便执行 OUT 指令将数据送向数据输出口。这时低电平的 $M/\overline{IO}$ 使 I/O 译码器的状态口片选信号 $\overline{CS_2}$ 变低, $\overline{CS_2}$ 再和 $\overline{WR}$ 信号经门 2 相与后输出低电平的选通信号,它用来选通数据锁存器,将数据送向外设。同时,选通信号的后沿还使 D 触发器翻转,置 Q 为高电平,即把状态口的 BUSY 位置成 1,表示忙碌。

当输出设备从接口中取走数据后,就送回一个应答信号 $\overline{ACK}$,它将 D 触发器清 0,即置 $BUSY=0$,允许 CPU 送出下一个数据。

设:状态口的地址为 PORT-S2,输出数据口的地址为 PORT-OUT,传送数据的总字节数为 COUNT-2,则查询式输出数据的程序段为:

MOV	CX, COUNT-2	;传送的字节数
READ-S2: IN	AL, PORT-S2	;读入状态位
TEST	AL, 02H	;忙否?
JNZ	READ-S2	;忙,循环检测
MOV	AL, 输出数据	;不忙
OUT	PORT-OUT, AL	;输出数据
LOOP	READ-S2	;未传送完,循环
:		;已送完

2. 中断方式

用查询方式使 CPU 与外设交换数据时,CPU 要不断读取状态位,检查输入设备是否已准备好数据,输出设备是否忙碌或输出缓冲器是否已空。若外设没有准备就绪,CPU 就必须反复查询,进入等待循环状态。由于许多外设的速度很低,这种等待过程会占去 CPU 的绝大部分时间,而真正用于传输数据的时间却很少,使 CPU 的利用率变得很低。

例如,若一个操作员每秒钟可从键盘输入 5 个字符,平均每个字符占 $200000\mu s$ 的时间,实际上计算机只要用 $10\mu s$ 就能从键盘读入一个字符,这样就有 $999950\mu s$ 的时间花在检测键盘状态和等待上,也就是说 99.99% 的时间因等待而白白浪费掉了。如果有多个设备工

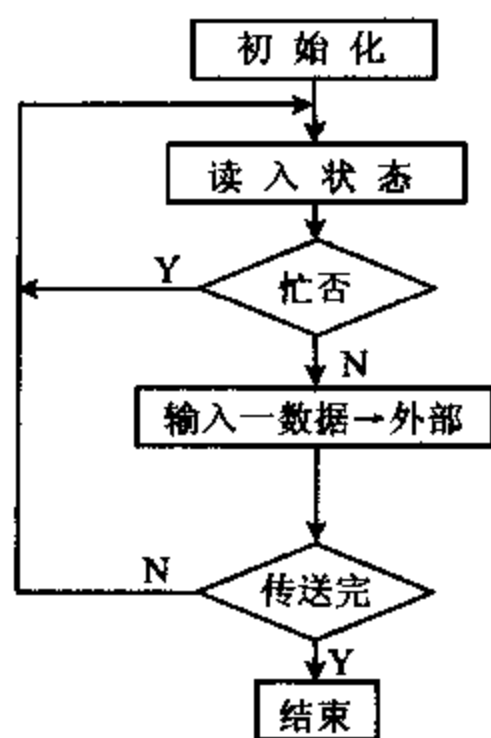


图 6-9 查询式输出流程图

作,还要轮流查询,这些设备的工作速度又往往各不相同,这不仅极大地浪费了 CPU 的时间,而且还会因为程序进入等待某些慢速外设数据的循环而造成快速外设数据的大量流失,这在许多系统中是不允许的,尤其不能用于要求实时数据处理的场合。为提高 CPU 的利用率和进行实时数据处理,CPU 常采用中断方式与外设交换数据。

采用中断方式后,CPU 平时可以执行主程序,只有当输入设备将数据准备好了,或者输出端口的数据缓冲器已空时,才向 CPU 发中断请求。CPU 响应中断后,暂停执行当前的程序,转去执行管理外设的中断服务程序。在中断服务程序中,用输入或输出指令在 CPU 和外设之间进行一次数据交换。等输入或输出操作完成之后,CPU 又回去执行原来的程序。

有关中断的概念,将在第七章中详细介绍,这里不再赘述。

3. DMA 方式

(1) DMA 方式的提出

利用中断方式进行数据传送,可以大大提高 CPU 的利用率,但在中断方式下,仍必须通过 CPU 执行程序来完成数据传送。每进行一次数据传送,CPU 都要执行一次中断服务程序。这时,CPU 都要保护和恢复断点,通常还要执行一系列保护和恢复寄存器的指令,即保护现场,以便完成中断处理后能正确返回主程序。显然,这些操作与数据传送没有直接关系,但会花费掉 CPU 的不少时间。再说,8086 CPU 一旦进入中断后,指令队列就要清除,执行部件(EU)要等总线接口部件(BIU)将中断处理子程序中的指令取到队列后才执行;恢复断点时,指令队列也要被清除,执行部件也必须等总线接口部件把断点处的指令装入后才开始执行。所以,在这段时间内,执行部件和总线接口部件就不能并行工作,这也会造成数据传送效率的降低。当 CPU 与高速 I/O 设备交换数据,或与外设进行成组数据交换时,中断方式仍显得太慢。

例如,当磁盘和内存成批地交换信息时,磁头的读/写速度可超过 200000 字节/秒,因此只有在 $5\mu\text{s}$ 内完成一个字节的传送,才能充分发挥磁盘的大容量的性能优势。如果采用中断方式进行磁盘和内存间的成批数据传送,只能逐字节地进行。例如读盘时,要先把从磁盘读出的数据送进 CPU 的寄存器,再从寄存器搬入内存,然后修改地址指针和字节计数器。这些操作均要用指令来实现,显然不可能在 $5\mu\text{s}$ 之内完成。为了解决这个问题,可采用一种称为 DMA(Direct Memory Access)的传送方式,也就是直接存储器存取方式。

DMA 方式也要利用系统的数据总线、地址总线和控制总线来传送数据。原先,这些总线是由 CPU 管理的,但当外设需要利用 DMA 方式进行数据传送时,接口电路可以向 CPU 提出请求,要求 CPU 让出对总线的控制权,用一种称为 DMA 控制器的专用硬件接口电路来取代 CPU,临时接管总线,控制外设和存储器之间直接进行高速的数据传送,而不要 CPU 进行干预。这种控制器能给出访问内存所需要的地址信息,并能自动修改地址指针,也能设定和修改传送的字节数,还能向存储器和外设发出相应的读/写控制信号。在 DMA 传送结束后,它能释放总线,把对总线的控制权又交还给 CPU。可见,用 DMA 方式传输数据时,不需要进行保护和恢复断点及现场之类的额外操作,一旦进入 DMA 操作,就可直接在硬件的控制下快速完成一批数据的交换任务,数据传送的速度基本上取决于外设和存储器的存取速度。

有些 DMA 控制器,除了可在外设和内存间直接交换数据外,还能控制内存与内存之间的直接数据交换,如 Intel 8237A DMA 控制器就能完成这种功能。下面以磁盘和内存间的数据的 DMA 传送为例,来简单介绍一下利用 8237A DMA 控制器传送数据的大致过程,有关它的工作原理和使用方法等,将在第十二章作专门介绍。

(2) DMA 控制器的连线和操作

DMA 控制器经常与磁盘驱动器、数据流记录仪、磁带机和数据采集卡等配合使用,来实现这些外设和内存间的高速数据交换。Intel 8237A 是一种较常用的 DMA 控制器,被用在 PC/XT 和 PC/AT 机中,以提高系统存取磁盘驱动器的速度。图 6-10 是这种应用的连接示意图。在采用 DMA 控制器的系统中,CPU 与 DMA 控制器分时地使用地址总线、数据总线和控制总线,图中用 3 个开关来表示这种总线的切换。

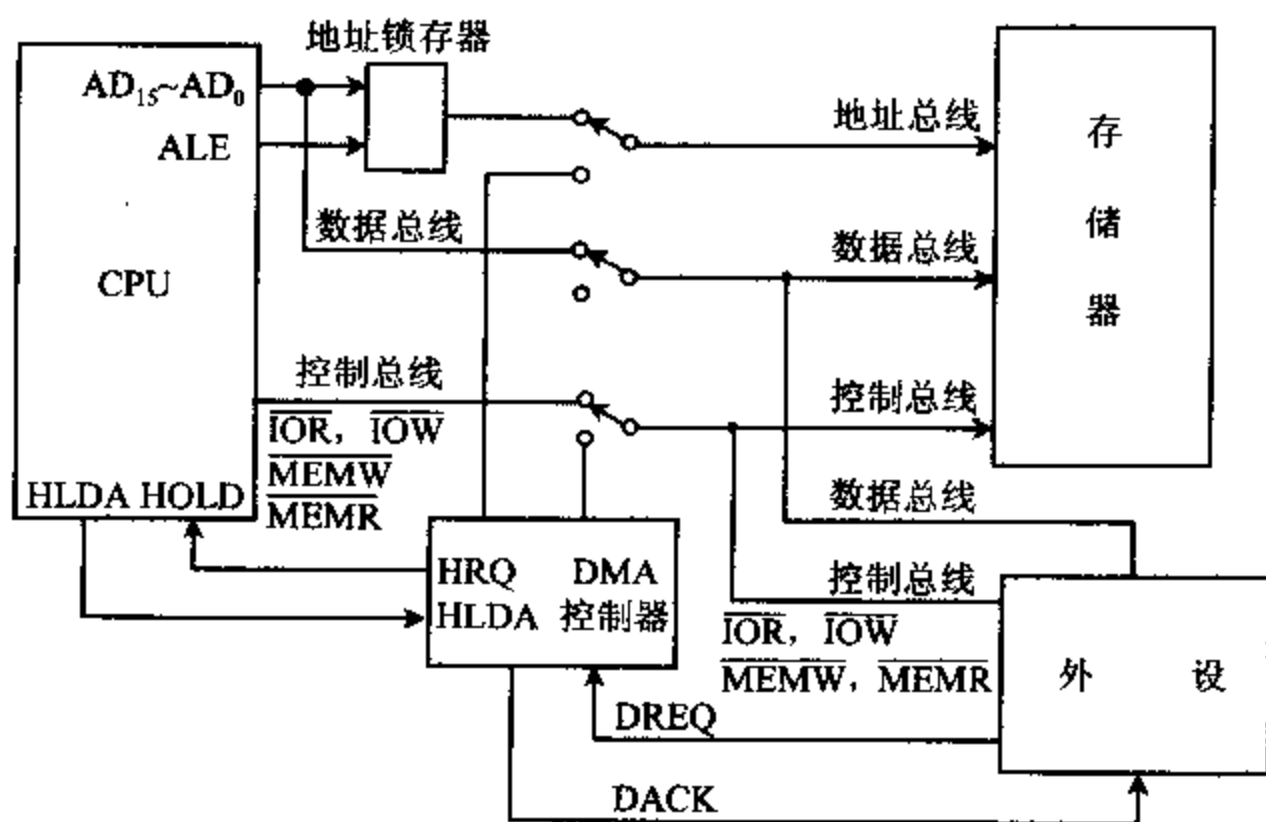


图 6-10 微机系统中 DMA 控制器操作框图

用 DMA 方式读磁盘的过程分以下几步进行:

① 系统启动时,3 个开关打向上端,总线与 CPU、存储器和外设相连,并由 CPU 来控制。进行一次 DMA 传输前,首先要对 8237A 进行初始化编程,包括设定要传送数据的字节计数器初值,指定参与传送的内存块的起始地址,以及选定 DMA 通道(8237A 内部有 4 个 DMA 通道)和所用的传送方式等。

② 然后,CPU 向磁盘控制器发出读盘命令,由磁盘控制器找到要读取的数据位置,并开始读出数据。

③ 当磁盘控制器准备好了第一个字节的数据后,就向 DMA 控制器发送一个 DMA 请求信号 DREQ。如果 DMA 控制器的输入通道没有被屏蔽,DMA 控制器就送一个保持请求信号 HRQ 到 CPU 的 HOLD 输入端。CPU 从 HOLD 端收到 HRQ 信号,并完成当前的总线操作后,就中止当前程序的运行,将它的总线浮空,并发回一个保持响应信号 HLDA 给 DMA 控制器,作为收到 HRQ 信号的应答。DMA 控制器收到 HLDA 信号后,便发送一个控制信号,使 3 个总线开关置向下方,让总线和 DMA 控制器相连,而与 CPU 脱开。这样,

DMA 控制器就接管了总线。

④ DMA 控制器获得总线控制权后,便通过地址总线向存储器发送地址信号,指示要被写入内存的第一个数据的地址。随后,DMA 控制器向磁盘控制器发出 DMA 确认信号 DACK,通知磁盘控制器准备好要输出的数据字节。

⑤ 接着,DMA 控制器使控制总线上的 I/O 读信号 $\overline{IOR}$ 和存储器写信号 $\overline{MEMW}$ 有效。 $\overline{IOR}$ 有效使磁盘控制器能从磁盘向数据总线输出数据字节; $\overline{MEMW}$ 有效使所寻址的存储单元能够接受从数据总线上写入的数据。

⑥ 每完成一个字节数据的传送,DMA 控制器就自动修改内部地址寄存器的内容,使它指向下一个字节的地址,并将统计总字节数的字计数器减 1,再重复上述传送过程。若字计数器减为 0,并由 0 减为 FFFFH 时,表示这批数据已传送完毕,DMA 过程结束。

⑦ DMA 传送结束后,DMA 控制器便撤销它对 CPU 发出的保持信号 HRQ,并释放总线。图中的 3 个开关又拨回到上方,使总线与 CPU 相连,CPU 恢复对总线的控制权,并从中止处开始继续执行后续程序。

CPU 在每一个非锁定时钟周期(Lock 为高)结束后,都要检测一下 HOLD 引脚,看是否有 DMA 请求信号。若有,便暂时中止正在执行的程序,进入上述的 DMA 周期。

五、PC 机的 I/O 地址分配

当微型机系统中采用 I/O 单独编址方案来控制外部设备时,常用 74LS138 译码器和必要的逻辑门电路来设计 I/O 译码电路。这时,可将要参与编码的地址信号和指示 I/O 操作的控制信号接到译码器的输入端。当 I/O 指令执行时,译码器的输出端便能产生低电平 I/O 端口选择信号,即片选(Chip Select, $\overline{CS}$)信号。这些片选信号被送到各 I/O 接口的控制端或片选($\overline{CS}$)端,就能选中相应的端口,对它进行 I/O 读或 I/O 写操作。下面给出 PC/XT 和 PC/AT 机的 I/O 端口地址分配表,并介绍 I/O 端口译码电路图。

1. PC/XT 机的 I/O 端口分配

在 IBM PC/XT 机中,中断控制、DMA 控制、动态 RAM 刷新、系统配置识别、键盘代码读取及扬声器发声等都是由可编程 I/O 接口芯片控制的。这些接口芯片包括:8259A 中断控制器、8237A-5 DMA 控制器、8255A-5 并行接口芯片、8253-5 计数器/定时器等。它们都安装在 PC/XT 机的系统板上,每块接口芯片都要使用 I/O 端口地址。在系统板上还有 8 个 I/O 扩展槽,也称为 I/O 通道。在每个扩展槽中,可插入 I/O 适配器(Adapter)插件,这些插件提供磁盘驱动器、打印机、CRT 显示器、异步通信设备等外设的接口,这些接口电路也需用 I/O 端口地址。系统对这些端口的地址有一个统一的安排。

在 8086/8088 系统中,可使用 16 根地址线($A_{15} \sim A_0$)对输入输出端口进行寻址,形成 64K 的 I/O 端口地址范围。但在 PC/XT 机系统中,只使用低 10 位有效地址($A_9 \sim A_0$)进行寻址,因此 I/O 端口地址空间只有 1K。其中 A_9 位具有特殊意义,当 $A_9=0$ 时,寻址系统板上的 512 个端口;当 $A_9=1$ 时,寻址 I/O 通道上的 512 个端口。系统板和 I/O 通道上的 I/O 端口地址分配如表 6-2 所示。表中,系统板地址分成两个部分,前者为译码电路产生的地址,后者用括号括起来,它是 I/O 接口芯片实际使用的地址。

表 6-2 PC/XT 机的 I/O 端口分配表

分类	地址范围(H)	I/O 设备(端口)
系 统 板	000~01F(00~0F)	8237A-5 DMA 控制器
	020~03F(20~21)	8259A 中断控制器
	040~05F(40~43)	8253-5 计数器/定时器
	060~07F(60~63)	8255A-5 并行接口
	080~09F(80~83)	DMA 页寄存器
	0A0~0BF(A0)	NMI 屏蔽寄存器
	0C0~0DF	保留
	0E0~0FF	保留
I/O 通 道	200~20F	游戏 I/O
	2F8~2FF	异步通信 2(COM 2)
	300~31F	实验卡(原型卡)
	320~32F	硬盘适配器
	378~37F	并行打印机接口
	380~38F	同步通信控制器
	3B0~3BF	单显/打印机适配器
	3F0~3F7	软磁盘适配器
	3F8~3FF	异步通信 1(COM 1)

I/O 通道上各设备的端口地址译码由各插件板自行完成,系统板上的 I/O 地址采用固定地址法译码逻辑,译码电路如图 6-11 所示。

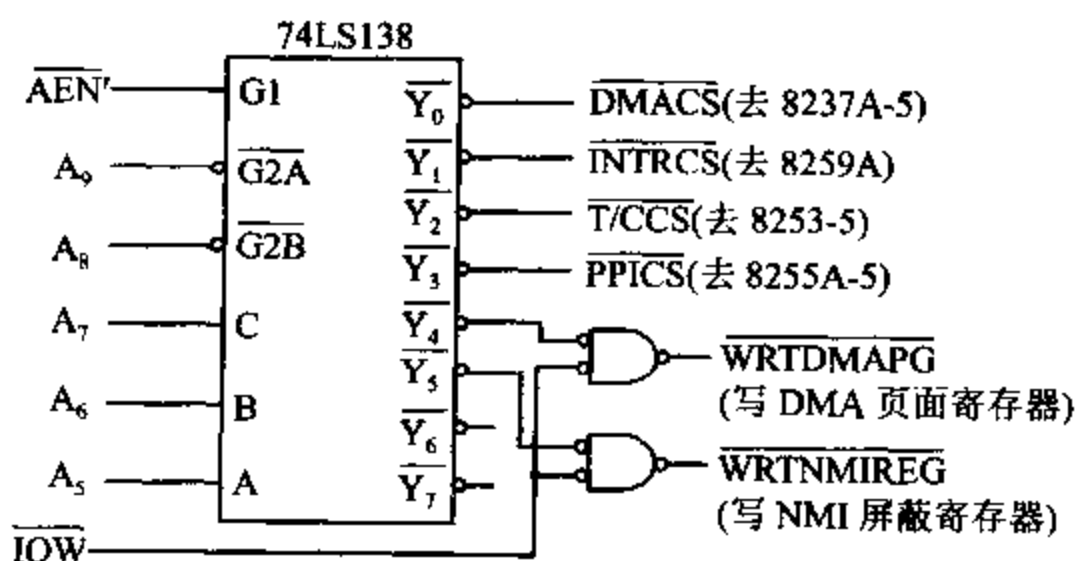


图 6-11 系统板上 I/O 端口译码电路

在图 6-11 中,各接口芯片的片选信号由 74LS138 译码电路产生,在 CPU 控制系统总线时, $\overline{AEN'}=1$,这时若 $A_9A_8=00$,则译码器工作,它根据输入信号 $A_7A_6A_5$ 进行译码,在 $\overline{Y_0} \sim \overline{Y_7}$ 中产生一个低电平输出信号,作为对应接口控制电路的选中信号,接到相应接口芯片的 $\overline{CS}$ 端或控制端。该电路在 I/O 读写命令控制下进行工作。I/O 地址的低 4 位 $A_3 \sim A_0$ 用作控制芯片内部寄存器的选择信号,这样每一个译码输出端都包含 $2^4=16$ 个端口地址。有些接口芯片内部有 16 个寄存器,如 8237A-5 DMA 控制器,它就使用 00~0FH 共 16 个端

口地址。但多数接口芯片内部没有 16 个寄存器,如 8259A-5 只有 2 个寄存器,8253-5 和 8255A-5 各有 4 个寄存器。这时,较高地址位可以不用,仅用 A_1 和 A_0 等低地址位来选择端口。例如 8259A 中断控制器占用的端口地址范围为 020~03FH,但实际只用到最低两个端口地址 20H 和 21H。而 NMI 屏蔽寄存器仅用了 1 个 I/O 地址 A0H。这里顺便提一下该端口的功能:在开机上电时,从不可屏蔽中断引脚 NMI 进入 8088 的中断是不可屏蔽的,即允许 NMI,但此后可用软件将此屏蔽位置位或复位,具体方法为:

将 80H 写入 A0H 端口时,则设置屏蔽,表示允许 NMI。

将 00H 写入 A0H 端口时,则清除屏蔽,表示禁止 NMI。

2. PC/AT 机的 I/O 端口地址表

以 80286 为 CPU 的 PC/AT 机中,也只使用低 10 位地址信号进行译码形成 I/O 端口地址,地址范围为 000~3FFH,但使用 2 片 8237A-5 DMA 控制器,2 片 8259A 中断控制器(分为主片和从片),定时器使用 8254-2。PC/AT 及其兼容机的 I/O 端口地址分配情况如表 6-3 所示,系统板上使用的口地址为 000~0FFH,其余部分分配在 I/O 扩展板上。

表 6-3 PC/AT 及兼容机的 I/O 端口地址分配表

分类	地址范围(H)	I/O 设备(端口)
系 统 板	000~01F	DMA 控制器 1,8237A-5
	020~03F	中断控制器 1,8259A(主片)
	040~05F	定时器,8254-2
	060~06F	键盘接口处理器,8042
	070~07F	实时时钟,NMI 屏蔽寄存器
	080~09F	DMA 页寄存器,74LS612
	0A0~0BF	中断控制器 2,8259A(从片)
	0C0~0DF	DMA 控制器 2,8237A-5
	0F0	清除协处理器忙信号
	0F1	复位协处理器
	0F8~0FF	协处理器
I/O 通 道	1F0~1F8	硬磁盘
	200~207	游戏 I/O 口
	278~27F	并行口 2(LPT2)
	2F8~2FF	串行口 2(COM2)
	300~31F	实验卡(原型卡)
	360~36F	保留
	378~37F	并行打印机口 1(LPT1)
	380~38F	SDLC,双同步通信口 2
	3A0~3AF	双同步通信口 1
	3B0~3BF	单色显示器/打印机适配器
	3C0~3CF	保留
	3D0~3DF	彩色/图形监视器适配器
	3F0~3F7	软磁盘控制器
	3F8~3FF	串行口 1(COM1)

6-2 总 线

一、总线的概念

为使系统灵活、简单和便于扩展功能,多数微型计算机系统都采用模块化结构。一台微型计算机由多个模块组成,每个模块都具有独立的功能。一个模块往往就是一块电路板,它可以做成主板或插件板(卡)的形式。如 PC/XT 机系统就是由 CPU 主板、显示器适配器卡、打印机接口板、软盘/硬盘接口卡、存储器插件板等,再加上所需的外部设备组成的。其中主板上有多 I/O 扩展槽,各种插件板插入扩展槽中。微型计算机与其它仪器、仪表或控制系统等组合在一起,又可构成专用系统。显然在系统与系统之间,插件与插件之间以及同一插件上的各芯片之间都有互连和通信的问题,也就是说各块插件板或主板必须能与别的板通信,同一插件板上的各种片子间也要能互相传送信息,这就需要一种能实现这种通信的公共通路。在微型计算机系统中,将这种用于各部件之间传送信息的公共通路称为总线(Bus)。

1. 总线的分类

根据总线中信息传送的类型可分为地址总线、数据总线和控制总线,此外还有电源线和地线,对此我们已在前面做过介绍。若按总线的规模、用途和应用场合,则可以分成以下三类:

(1) 片级总线

片级总线也叫做元件级总线,是由芯片内部通过引脚引出的总线,用于芯片一级的互连线。它实现 CPU 主板或其它插件板上的各种芯片间的互连。例如,CPU 与存储器、I/O 接口电路、译码电路间的连线就属于这类总线。由于 8086/8088 CPU 受引脚的限制,芯片引脚上的地址总线和数据总线是复用的,需要用地址锁存器等将它们分离后引到总线上。另外,当有较多的存储器或 I/O 芯片加到系统上时,为了增加系统的驱动能力,还要用驱动缓冲器等芯片进行驱动后,再将信号引到总线上。在有关章节中,我们已对这类总线做了一些介绍。在后续章节里介绍各种典型接口芯片时,我们还将作进一步的讨论。

(2) 系统总线

系统总线也叫内总线或板级总线,它用于微型计算机中各插件板之间的连线,也就是通常所说的微机总线。

(3) 外部总线

外部总线也称为通信总线,它用于微型计算机系统之间,或微型计算机系统与其它电子仪器或设备之间的通信。这种总线不是微型计算机特有的,一般是借用电子工业其它领域的总线。

本章重点讨论系统总线。在使用这两类总线时,用户和制造厂商都希望总线具有通用性,也就是说,希望不同厂家生产的插件板可以互换,不同系统之间可以互连和通信。因此,

只要按统一的总线标准进行设计或连接就可以。这样,对于制造厂家来说,只要按总线接口规范设计 CPU 主板、I/O 接口板或存储器插件板,然后将插件板插入主机的总线扩展槽中,就可构成系统,很适合于大批量生产、组装和调试,也便于更新和扩充系统。对于用户来说,可根据自身需要,灵活地选购接口板或存储器插件,来组装成适合自己的应用需要的系统或更新原有系统。例如一般用户选购计算机时都购置硬盘和软盘驱动器接口板、打印机接口板、VGA 彩色显示器适配卡等;有些需要处理模拟信号的用户可再选购 A/D、D/A 接口板;需要通过电话网来传送信息的可选购调制解调器(MODEM)板;当需要构成一个多媒体系统时,用户可选购声音卡、图像解压缩卡、CD-ROM 驱动器等;必要时用户还可根据总线标准的要求,自行设计接口电路板。目前,使用 486 等高档机的用户可直接购置一块多功能卡,上面包含硬盘驱动器、软盘驱动器、并行打印机和串行通信口等各种接口电路,这样就不需要用多块插件板了。

2. 总线标准

总线标准指在计算机界承认或推荐的系统中互连各个模块的标准,它通常对总线所用插座的尺寸、引线数目、各引线信号的含义和时序等都做了明确的统一规定。

常用的标准系统总线有:

- IBM PC 机的 62 芯 PC 总线。
- PC/AT 机的 AT 总线或 ISA 总线。
- 高性能 PC 机的 EISA 总线。
- PCI(Peripheral component Interconnect)总线,即外围部件互连局部总线。

常用的标准外部总线包括:

- IEEE-488 总线。
- EIA RS-232 总线。

下面介绍 PC 机中所用的 PC 总线和 ISA 总线。EIA RS-232 总线将在第十章中介绍。在需要了解其它总线时,可参阅有关的资料。下面,对这几种总线信号的引脚、名称和意义分别进行介绍。

二、IBM PC 总线

IBM PC/XT 机的主板上有 8 个 62 芯的 I/O 扩展槽,序号为 J₁~J₈。连接扩展槽的是 IBM PC 总线,共有 62 根引线。这 62 根线中包含分离的 20 根地址总线和 8 根数据总线,其余为控制总线、电源线和地线。62 芯 PC 总线的引脚排列如图 6-12 所示,有上划线的信号表示该信号为低电平时有效。信号名称后括号内的 I,表示该信号是从扩展槽输入到系统板的,而 O 则反之。

1. A₁₉~A₀ (O)

地址总线。用于传送存储器和 I/O 的地址,当传送 I/O 地址时,A₁₉~A₁₆无效。地址信号可由 CPU 或 DMA 控制器产生。

2. D₇~D₀ (I/O)

数据总线。它们为 CPU、存储器或 I/O 设备提供传输数据信息的通路。由于 PC/XT 机以 8088 为 CPU,它虽具有处理 16 位数据的能力,但它只有 8 根外部数据线,每次只能传

送一个字节。所以也将 PC 总线称为 8 位 PC 总线。

3. ALE (O)

地址锁存允许信号。它由总线控制器 8288 产生,当它有效后产生由高电平到低电平的下降沿时,将 CPU 送出的地址信号进行锁存。

4. MEMR (O)

存储器读命令。当 CPU 执行存储器读命令时,该信号有效,将所选中的存储单元中的数据读到数据总线上。DMA 控制器也可使该信号有效。

5. MEMW (O)

存储器写命令。当 CPU 执行存储器写命令时,该信号有效,将数据总线上的数据写入所选中的存储单元中。DMA 控制器也可使该信号有效。

6. IOR (O)

I/O 读命令。当 CPU 执行输入指令时,该信号有效,把所选中的 I/O 端口中的数据读到数据总线上,DMA 控制器也可使该信号有效。

7. IOW (O)

I/O 写命令。当 CPU 执行输出指令时,该信号有效,把数据总线上的数据写到所选中的 I/O 端口中,DMA 控制器也可使该信号有效。

8. IRQ₂~IRQ₇ (I)

6 级中断请求信号,要求由低到高的上升沿有效。8259A 中断控制器共有 8 个中断请求输入端 IR₀~IR₇,其中 IR₀ 和 IR₁ 被系统板占用,分别用于时钟和键盘中断,其余 6 个中断请求输入端 IR₂~IR₇ 引到 62 芯总线上,分别对应于 IRQ₂~IRQ₇。这些信号都由 I/O 设备送到 8259A,通过 8259A 向 CPU 提出中断请求,其中 IRQ₂ 优先级最高,IRQ₇ 最低。

9. DRQ₁~DRQ₃ (I)

DMA 请求信号。这些信号由 DMA 控制器 8237A-5 产生,由于 8237A-5 有 4 个 DMA 通道,它们能产生 4 路 DMA 请求信号 DREQ₀~DREQ₃,其中 DREQ₀ 为系统板所用,用来对动态 RAM 进行刷新,其余 3 个信号 DREQ₁~DREQ₃ 引到 62 芯总线上,分别对应于 DRQ₁~DRQ₃,用来响应外设的 DMA 请求或对扩展槽中的动态 RAM 进行刷新。

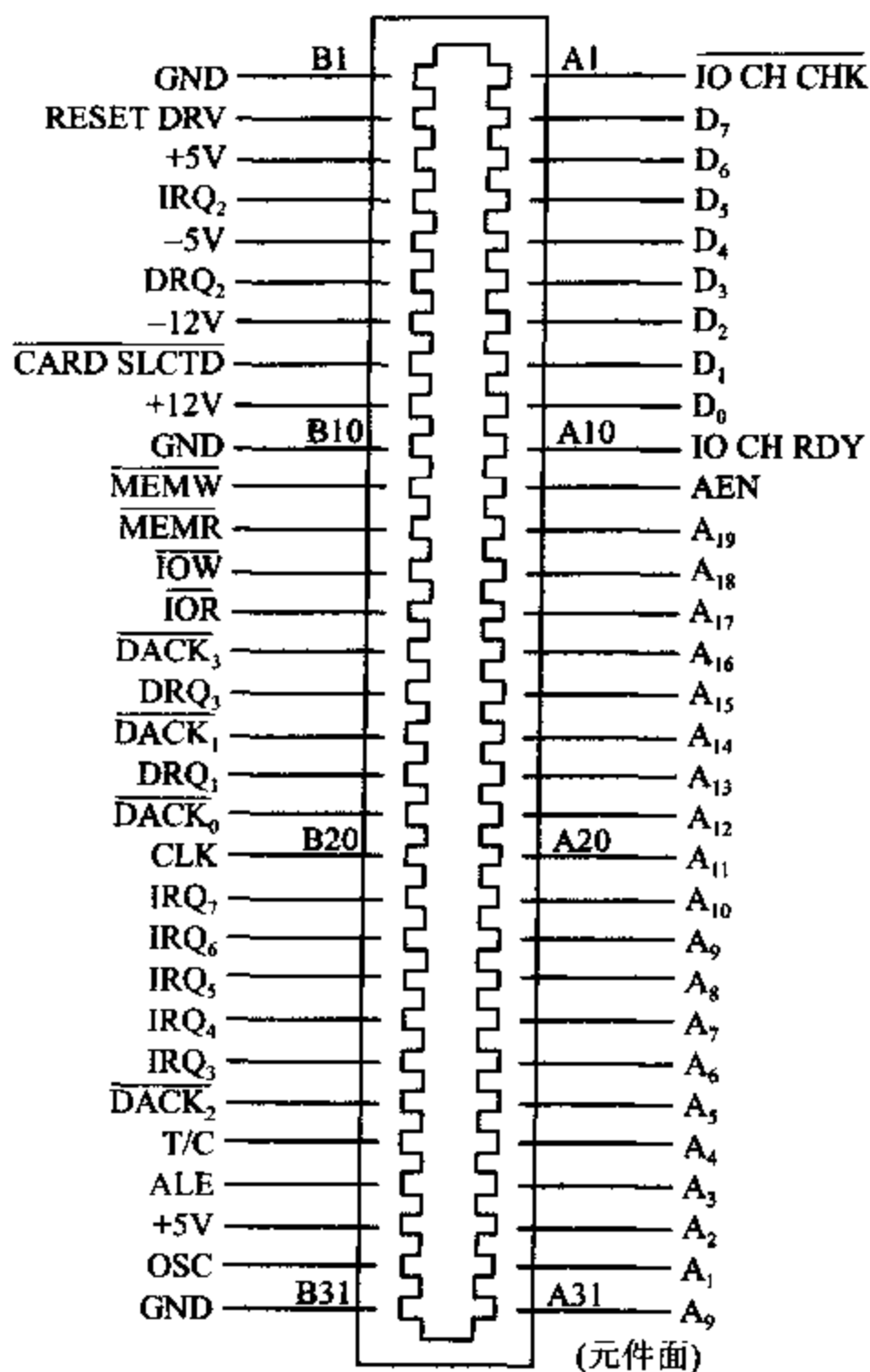


图 6-12 IBM PC 总线引脚图

10. AEN (O)

地址允许信号。它由 8237A-5 输出,当它有效时,迫使 CPU 让出对总线的控制权,而由 DMA 控制器来控制地址总线、数据总线和控制总线。

11. T/C (O)

计数结束信号。当 DMA 控制器的通道计数达到终点时,T/C 线上产生有效的高电平脉冲,通知外设,DMA 传送已经结束。

12. RESET DRV (O)

系统总清信号。该信号有效时,使系统各部件复位。

13. $\overline{\text{IO CH CK}}$ (I)

I/O 通道奇偶校验信号。当它为低电平时,表示 I/O 通道上的扩展存储器的奇偶校验有错,使 CPU 进入不可屏蔽中断(NMI)。

14. I/O CH RDY (I)

I/O 通道准备好信号,平常为高电平。一些慢速的存储器或 I/O 设备可通过将该信号变为低电平来使 CPU 或 DMA 控制器插入等待周期,从而延长存储器周期或 I/O 周期。此信号为低电平的时间不得超过 10 个时钟周期。

15. OSC (O)

晶体振荡信号。它的频率为 14.31818MHz,周期为 70ns,占空比为 1/2。

16. CLK (O)

系统时钟信号。该信号由 OSC 信号经 8284A 时钟产生器三分频后得到,频率为 4.77MHz,周期为 210ns,占空比为 1/3,其中高电平占 1/3,低电平占 2/3。

17. $\overline{\text{CARD SLCTD}}$

插件板选中信号,接插件板的 B_8 引脚。在 62 芯 I/O 通道中, J_8 槽与 $J_1 \sim J_7$ 槽略有些区别,在 $J_1 \sim J_7$ 的 I/O 通道中, B_8 引脚是备用线,它们被连在一起,但系统不使用它。在 J_8 槽中, B_8 为插件板选中信号($\overline{\text{CARD SLCTD}}$),当该信号为低电平时, J_8 被选中,CPU 可读取 J_8 槽上的适配器。

18. +5V -5V +12V -12V

电源线,其中+5V 使用 2 个引脚,其余均用一个引脚。

19. GND

地线,使用 1 个引脚。

三、AT 总线或 ISA 总线

AT 总线是以 80286 为 CPU 的 PC/AT 机及其兼容机所用的总线,也可用在 80386/80486 机上。由于 80286 CPU 的外部数据总线的宽度为 16 位,为了充分发挥这个优势,对 PC 总线进行了修改,将数据总线的宽度由 8 位增加到 16 位。后来 AT 总线也被称为 ISA 总线,即工业标准结构总线(Industry Standard Architecture)。为保证 ISA 总线与 PC 总线兼容,以便使许多具有 8 位数据宽度的功能扩展卡仍能在 PC/AT 机上使用,ISA 总线的插座在原来 62 引脚的 PC/XT 插座基础上,又增加了一个 36 引脚的插座。这样,既能插入只有 62 引脚的 PC/XT 兼容 8 位扩展卡,又可以利用整个插座插入 16 位扩展卡。

ISA 总线的前 62 个引脚($A_1 \sim A_{31}$ 、 $B_1 \sim B_{31}$)信号的排列和意义与 62 芯 PC 总线基本相同,仅有两处做了改动。一处是原来 B_{19} 引线为通道 0 的 DMA 响应信号 $\overline{DACK}$,现因 AT 机的动态 RAM 刷新不再通过伪 DMA 传输来完成,而是直接由系统板上 RAM 刷新电路产生 $\overline{REFRESH}$ 来替代原信号,也可由 I/O 扩展卡上的其它微处理器驱动刷新信号。另一处改动是原 J_8 槽的 B_9 处引入一个零等待 (OWS) 信号,它表示扩展槽中的信号无需处理器插入任何等待状态,即可完成当前的总线周期。

ISA 总线的后 36 引脚($C_1 \sim C_{18}$ 、 $D_1 \sim D_{18}$)设置了高 8 位数据线、高 7 位地址线及新增加的一些控制信号,还有电源线及地线。

ISA 总线的引脚排列如图 6-13 所示,各引脚功能说明如下:

1. $SA_{19} \sim SA_0$ (I/O)

地址总线。用来传送存储器和 I/O 地址,利用这 20 条地址线再加上 $LA_{23} \sim LA_{17}$,就可对多达 16MB 的存储空间进行存取操作。当 $BALE$ 处于高电平时, $SA_{19} \sim SA_0$ 就挂到系统总线上;在 $BALE$ 的下降沿,它们被锁存。这些信号由 CPU 或 DMA 控制器产生,也可由装在 I/O 通道(即 I/O 扩展板)上的 CPU 或 DMA 控制器来驱动。

2. $LA_{23} \sim LA_{17}$ (I/O)

非锁定地址总线。这些信号也用于对系统中的存储器和 I/O 设备进行寻址,它们与 $SA_{19} \sim SA_0$ 配合,使允许的寻址空间达 16MB。当 $BALE$ 为高电平时,这些信号才有效。在微处理器周期期间, $LA_{23} \sim$

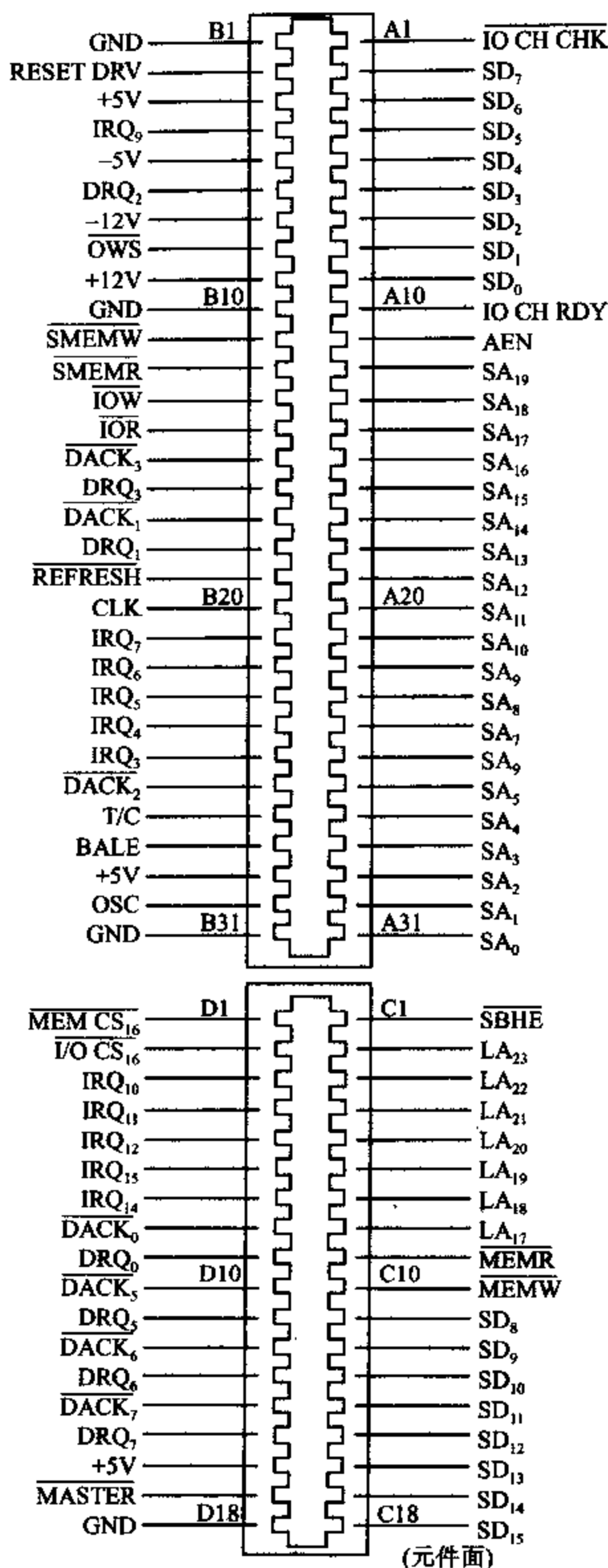


图 6-13 ISA 总线引脚图

LA₁₇不被锁存,所以在整个周期内不保持有效。它们的用途是为一个等待状态存储周期生成存储器译码信号,这些译码信号应由 I/O 适配器锁定在 BALE 的下降沿上。这些信号也可由装在 I/O 通道上的 CPU 或 DMA 控制器驱动。

3. SD₁₅~SD₀ (I/O)

数据总线。它们为 CPU、存储器和 I/O 设备提供 16 位的数据总线。在 I/O 通道上,所有的 8 位设备都使用 SD₇~SD₀ 和 CPU 通信,16 位设备则使用 SD₁₅~SD₀。为了支持 8 位设备,在进行 8 位数据传送时,D₁₅~D₈ 上的数据将作为 D₇~D₀ 来选通。在 16 位 CPU 向 8 位设备传送数据时,要把 16 位数据转换成 2 个 8 位数据来传送,其中 SD₁₅~SD₈ 上的数据要经过变换后再送到 SD₇~SD₀ 上去。

4. BALE (O)

地址锁存允许信号。该信号由 82288 总线控制器产生,在系统板上,它用来锁存 CPU 送来的有效地址和存储器地址译码信号,在 I/O 通道上与 AEN 一起使用时,可作为一个有效处理器或 DMA 地址的指示信号。

5. I/O CH CK (I)

I/O 通道校验。这是 I/O 通道上的存储器或 I/O 设备向系统板提供的奇偶校验信号,当该信号有效时,表示出现了无法更正的系统错误。

6. I/O CH RDY (I)

I/O 通道就绪。它受存储器或 I/O 设备控制,当它为低电平时,表示存储器或 I/O 设备没有准备好,这时就要插入等待周期,直到外设或存储器准备就绪为止。机器周期的延长时间必须为时钟周期(167ns)的整数倍,而且不得超过 2.5μs。

7. IRQ₃~7、IRQ₈~12、IRQ₁₄、IRQ₁₅ (I)

中断请求信号。AT 系统板上有两块 8259A 中断控制器芯片,通过级联可以产生 15 个中断输入信号,除了在系统板上使用的外,还有 11 个中断信号没有使用,它们与 AT 总线上的上述 11 个 IRQ 引脚相连,可管理 11 级中断。

两块芯片级联时,从片接在主片的 IR₂ 引脚上,对这两块芯片进行初始化编程时,都置为一般全嵌套方式,边沿触发,一般结束中断(EOI)。因此,当 IRQ 线由低电平变为高电平时,即产生中断请求,此高电平一直保持到微处理器请求中断,执行中断服务程序为止。这 11 级中断中,优先级从高到低的次序为 IRQ₉~IRQ₁₂、IRQ₁₄、IRQ₁₅、IRQ₃~IRQ₇。

8. IOR、IOW (I/O)

这两个信号的含义与 PC 总线的基本相同,但也可以由 I/O 通道中的其它 CPU 或 DMA 控制器驱动。

9. SMEMR (O)、MEMR (I/O)

存储器读信号。该信号有效时将选中的存储单元中的内容送到数据总线上。SMEMR 信号仅当存储器译码信号选中低于 1MB 存储空间时才有效,而 MEMR 在所有的存储器读周期都有效。MEMR 可由系统中的任何微处理器或 DMA 控制器驱动;SMEMR 则由 MEMR 和低于 1MB 的存储器译码信号选中。当 I/O 通道上的微处理器要驱动 MEMR 时,必须在使 MEMR 有效之前,先使总线上的地址保持一个有效的时钟周期。

10. SMEMW (O)、MEMW (I/O)

存储器写信号。该信号有效时把当前数据总线上的数据写到所选中的存储单元中,

$\overline{\text{SMEMW}}$ 仅当存储器译码信号选中低于 1MB 的存储空间时有效,而 $\overline{\text{MEMW}}$ 适用于所有的存储器写周期。 $\overline{\text{MEMW}}$ 可由系统中的任何微处理器或 DMA 控制器驱动, $\overline{\text{SMEMW}}$ 则由 $\overline{\text{MEMW}}$ 和低于 1MB 的存储器译码信号选中。当 I/O 通道上的微处理器要驱动 $\overline{\text{MEMW}}$ 时,必须在使 $\overline{\text{MEMW}}$ 有效之前,先使总线上的地址保持一个有效的时钟周期。

11. $\text{DRQ}_{0\sim3}, \text{DRQ}_{5\sim7}$ (I)

DMA 请求信号。这是由外设和 I/O 通道上的微处理器所驱动的异步通道请求信号。由于 AT 机使用两片 8237A DMA 控制器芯片,经级联可产生 7 个 DMA 通道,分别接受这 7 个 DMA 请求。它们的优先级次序为: DRQ_0 的优先级最高, DRQ_7 的优先级最低。 DRQ 变为有效的高电平后,就产生 DMA 请求,这个高电平一直要保持到 DMA 响应信号 DACK 有效为止。在这 7 个 DMA 请求信号中, $\text{DRQ}_0 \sim \text{DRQ}_3$ 用于 8 位 DMA 传送,而 $\text{DRQ}_5 \sim \text{DRQ}_7$ 则用于 16 位传送。

12. $\overline{\text{DACK}}_{0\sim3}, \overline{\text{DACK}}_{5\sim7}$ (O)

DMA 响应信号。这是对 7 路 DMA 请求信号 $\text{DRQ}_{0\sim3}, \text{DRQ}_{5\sim7}$ 的响应信号。

13. AEN (O)

地址允许信号。当它有效时,指示系统板上的 CPU 与总线脱开,由 DMA 控制器来控制系统总线。

14. $\overline{\text{REFRESH}}$ (I/O)

刷新信号。用来指示一个存储器刷新周期。当它作为输出信号输出一个低电平时,它启动外部 RAM 的刷新周期;当它作为输入信号输入一个低电平时,将迫使从外设驱动刷新周期。

15. T/C (O)

此信号的含义与 PC 总线的相同,当任何一个 DMA 通道的计数器计到终点时, T/C 上产生一个有效的高电平。

16. $\overline{\text{SBHE}}$ (I/O)

总线高字节允许信号。当它有效时,表示数据总线上传输的是高位字节($\text{SD}_{15} \sim \text{SD}_8$)。16 位设备用 $\overline{\text{SBHE}}$ 信号控制数据总线缓冲器接到高 8 位数据线 $\text{SD}_{15} \sim \text{SD}_8$ 上。

17. $\overline{\text{MASTER}}$ (I)

主设备号。该信号和 DRQ 信号一起使用,对系统进行控制。I/O 通道上的微处理器或 DMA 控制器可按级联方式将 DRQ 信号发送到 DMA 通道和接收 DACK 信号。当接到有效的 $\overline{\text{DACK}}$ 信号时,I/O 通道上的微处理器可以使 $\overline{\text{MASTER}}$ 成为低电平,允许 I/O 通道上的微处理器控制系统地址总线、数据总线和控制总线。 $\overline{\text{MASTER}}$ 变成低电平后,I/O 通道上的微处理器在驱动地址和数据总线之前,需要等待一个系统时钟周期,在发出读和写命令前,要等待两个系统时钟周期。如果此信号保持低电平的时间超过 $15\mu\text{s}$,因这段时间内没有进行刷新,系统存储器的信息可能会丢失。

18. $\overline{\text{MEM CS16}}$ (I)

存储器 16 位芯片选择。若当前的数据传送是有一个等待状态的 16 位存储周期,就发一个有效的 $\overline{\text{MEM CS16}}$ 信号给系统板。该信号是对 $\text{LA}_{23} \sim \text{LA}_{17}$ 译码后形成的,它必须采用能吸收 20mA 电流的集电极开路门或三态驱动器来驱动。

19. $\overline{\text{I/O CS16}}$ (I)

I/O 16 位芯片选择。当该信号有效时,则通知系统板,当前的数据传送是有一个等待

状态的 16 位 I/O 周期。此信号由地址译码器驱动,驱动电路应采用能吸收 20mA 电流的集电极开路门或三态驱动器。

20. OSC (O)

晶体振荡信号。该信号的频率为 14.31818MHz,周期为 70ns,占空比为 1/2;它提供给其它的功能部件使用。

21. $\overline{OVS}$ (I)

零等待状态信号。该信号有效时告诉 CPU,扩展槽中的设备不要插入任何等待状态就可完成当前的总线周期。它也要由能吸收 20mA 电流的集电极开路门或三态驱动器来驱动。

22. CLK (O)

系统时钟。它是一个频率为 6MHz 的系统时钟信号,周期为 167ns,占空比为 1/2,与微处理器的周期时钟同步,该信号仅用作同步信号。

23. RESET DRV (O)

复位驱动。该信号在系统上电或从过低电压恢复时,复位或初始化系统逻辑。

绝大部分的 286 和 386 PC/AT 机上具有 6 个 16 位扩展槽和 2 个 8 位扩展槽。由于 ISA 总线只有 16 根数据线和 24 根地址线,而 386 和 486 机具有 32 根数据线和 32 根地址线,故在高档机上使用 ISA 总线不能充分发挥这些 CPU 的作用,明显地降低了数据传到总线上的速度。大多数基于 ISA 总线的 386 机,把多达 16M 字节、32 位宽度的内存,以及一个高速缓存直接集成到主机板上。由于该存储器可以不通过外设总线而被直接访问,所以它可以 386 的速度全速运行,因而能部分地解决 ISA 总线传输速度慢的问题。在这些系统中,ISA 总线只用来同硬盘控制卡、CRT 控制卡、网络接口卡等外设通信,因这些卡大多一次仅传送 8 位或 16 位数据,所以 ISA 总线的速度限制不会对单用户系统的性能有明显的影响。

习 题

1. CPU 与外设交换数据时,为什么要通过 I/O 接口进行? I/O 接口电路有哪些主要功能?
2. 在微机系统中,缓冲器和锁存器各起什么作用?
3. 什么叫 I/O 端口? 一般的接口电路中可以设置哪些端口? 计算机对 I/O 端口编址时采用哪两种方法? 在 8086/8088 CPU 中一般采用哪种编址方法?
4. CPU 与外设间传送数据主要有哪几种方式?
5. 说明查询式输入和输出接口电路的工作原理。
6. 简述在微机系统中,DMA 控制器从外设提出请求到外设直接将数据传送到存储器的工作过程。
7. 某一个微机系统中,有 8 块 I/O 接口芯片,每个芯片占有 8 个端口地址,若起始地址为 9000H,8 块芯片的地址连续分布,用 74LS138 作译码器,试画出端口译码电路,并说明每块芯片的端口地址范围。
8. 什么叫总线? 总线分哪几类? 在微型计算机中采用总线结构有什么好处?
9. PC 总线和 ISA 总线各用于何种类型的微型计算机中? 它们的数据总线各有多少根?

第七章 微型计算机中断系统

7-1 概 述

一、中断概念

CPU 与外部设备之间的数据交换,可以采取无条件传送,查询传送,也可采用中断方式。当 CPU 正常运行程序时,由于微处理器内部事件或外设请求,引起 CPU 中断正在运行的程序,转去执行请求中断的外设(或内部事件)的中断服务子程序,中断服务程序执行完毕,再返回被中止的程序,这一过程称为中断。利用中断可以避免不断检测外部设备状态,提高 CPU 的效率。

1. 中断源

引起程序中断的事件称为中断源。中断源有外部中断和内部中断,内部中断由程序预先安排的中断指令(INT n)引起,或由于 CPU 运算中产生的某些错误(如除法出错、运算溢出)引起。外部中断是外部设备或协处理器向 CPU 发出中断申请引起的。

2. 中断响应

中断请求何时发生是随机的。CPU 在每条指令的最后一个 T 周期去检测 INTR 引脚,CPU 一旦检测到有中断请求,在满足中断响应的条件下(IF=1),CPU 响应中断,向外设发出 $\overline{\text{INTA}}$ 中断响应信号。并保护断点(当前 CS,IP 和 PSW 值入栈),然后转向中断服务程序。中断服务程序执行完毕,CPU 返回原执行程序的中断处,继续向下执行,称为中断返回。

3. 中断向量表

CPU 响应中断后,必须由中断源提供地址信息,引导程序进入中断服务子程序,这些中断服务程序的入口地址存放在中断向量表中。内存中专门开辟了一个区域,存放中断向量表(也称中断矢量表)。

4. 中断优先级

当有多个中断源请求中断时,中断系统判别中断申请的优先级,CPU 响应优先级高的中断,挂起优先级低的中断。当 CPU 在运行中断服务子程序时,又有新的更高优先级的中断申请进入,CPU 要挂起原中断进入更高级的中断服务子程序,实现中断嵌套功能。

5. 中断屏蔽

当中断源申请中断时,CPU 可以由软件设置,使之不能响应,称为中断屏蔽。

对于各种计算机系统,中断系统的构成差别很大,但都具有基本功能:

(1) 能实现中断响应、中断服务、中断返回、中断屏蔽;

(2) 能实现中断优先级排队;

(3) 能实现中断嵌套。

本章讨论与 8086/8088 CPU 相配合的中断系统及中断接口芯片 8259A。

二、中断分类

8086/8088 有一个强有力的中断系统,可以处理 256 种不同的中断。8086/8088 系统上的中断源如图 7-1 所示。以产生中断的方法来分类,256 种中断可以分为两大类:外部中断和内部中断。

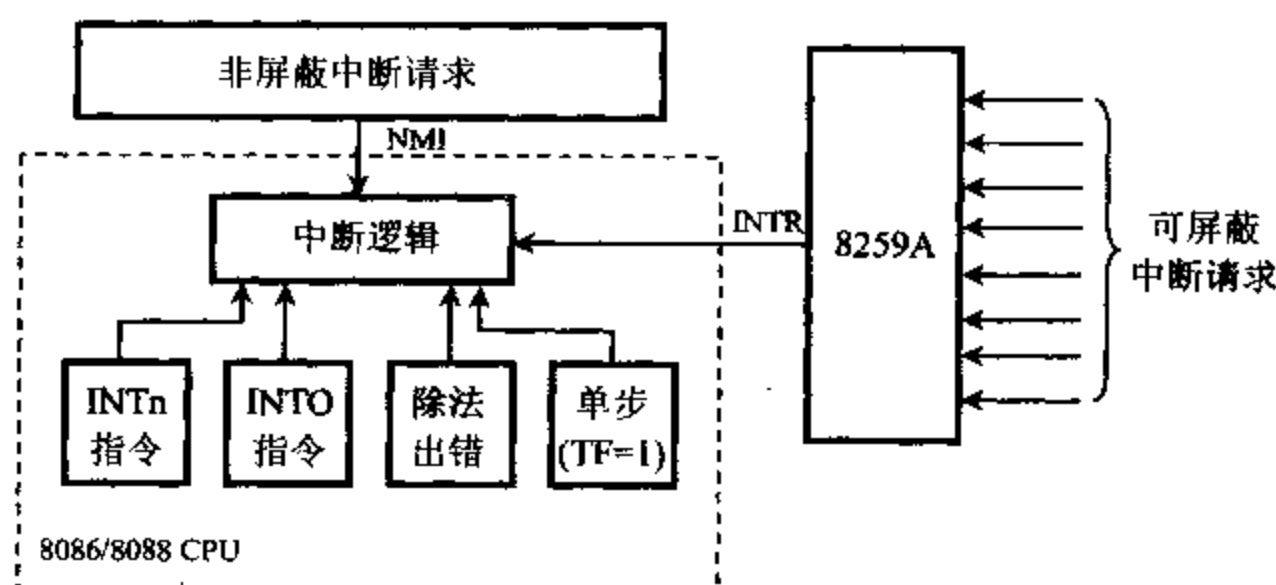


图 7-1 8086/8088 中断源

外部中断也称为硬件中断,是由外部的硬件产生的,硬件中断又分成不可屏蔽中断和可屏蔽中断。下面分别加以说明。

1. 外部中断

8086/8088 CPU 有两条外部中断请求线:不可屏蔽中断请求线 NMI,及可屏蔽中断请求线 INTR。

外部产生的不可屏蔽中断请求,由 CPU 的引脚 NMI 引入,采用边沿触发,上升沿之后维持两个时钟周期高电平有效。对于不可屏蔽中断用户是不能用软件来屏蔽的,一旦有不可屏蔽中断请求,如电源掉电等紧急情况,CPU 必须予以响应。在 IBM PC/XT 系统中,不可屏蔽中断可以有三种原因引起,一是系统板上的动态 RAM 有奇偶校验错误,二是 I/O 通道的扩展板出现奇偶校验错误,三是协处理器 8087 有中断请求。CPU 在指令周期的最后一个机器周期的最后一个 T 状态采样 NMI 线,若有中断请求,就转入 NMI 中断处理。不可屏蔽中断请求的中断类型号为 2,即中断处理程序的入口地址在 0 段的 0008H~000BH 的 4 个单元中。

外部设备提出的可屏蔽中断请求,由 CPU 的引脚 INTR 引入,采用电平触发,高电平有效,INTR 信号的高电平必须维持到 CPU 响应中断才结束。可屏蔽中断是用户可以通过软件设置来屏蔽的外部中断,即使外部设备有中断请求,CPU 可以不予响应。由外部设备引起的中断请求要得到响应有两个条件:一个是外设中断请求是否被屏蔽,一个是 CPU 是否允许响应中断。

在 8086 CPU 系统中,外部设备的中断请求信号接入可编程中断控制器 8259A 的 IR_i 端,而 8259A 的中断输出 INT 连到 CPU 的 INTR 引脚上。8259A 中设有中断屏蔽寄存器,它的 8 位对应控制 8 个外设,通过设置这个寄存器的某位为 0 或为 1,可以允许或禁止某个外部设备的中断请求,中断屏蔽寄存器某位为 1 就屏蔽了相对应的外设的中断请求,可以通过编程来实现对 8259A 中各个寄存器的设置。一块 8259A 可管理 8 个中断,当外设超过 8 个时,可以使用多个 8259A 进行级联,扩大到 64 级中断。外部设备与 8259A 的连接是由用户来设计的,硬件连线决定了中断类型号和中断优先级次序。

CPU 是否允许响应中断,与标志寄存器的中断允许位 IF 有关。当外设中断申请时,在当前指令执行完后,CPU 首先查询 IF 位,若 IF=0,CPU 就禁止响应任何外设中断;若 IF=1,CPU 就允许响应外设的中断请求。IF 的状态由指令来设置,指令 STI 设置中断允许位,使 IF=1,称为开中断;指令 CLI 禁止中断允许位,使 IF=0,称为关中断。

2. 内部中断

内部中断又称为软件中断。软件中断通常有三种情况引起:由中断指令 INT 引起的中断;由 CPU 的某些运算错误引起的中断;由调试程序 debug 设置的中断。

(1) 由中断指令 INT 引起的中断

CPU 执行一条 INT n 指令后会立即产生中断,并且调用系统中相应的中断处理程序去完成中断功能,指令中 n 指出了中断类型号。

例 7-1 测试存储器容量

```
INT    12H
```

CPU 执行这条指令时,立即产生一个中断。并从中断向量表的 0:12H*4 开始的单元中取出四个字节,其内容为中断服务程序的段地址及偏移地址,然后转到此入口去执行中断服务程序,完成对存储器的测试。返回参数是存储器的大小,放在 AX 中。

(2) 由 CPU 的某些运算错误引起的中断

CPU 在运行程序时,会发现一些运算中出现的错误,此时 CPU 就会中断程序,让用户去处理这些错误。主要有:

① 除法错中断:除法错中断类型号为 0,在除法运算中,若除数为 0 或商超过了寄存器所能表达的范围,就产生一个类型为 0 的中断,转入类型 0 中断处理。

② 溢出中断:溢出中断类型号为 4,专用指令为 INTO。

在运算中,若溢出标志位 OF 置 1,下面紧跟溢出中断指令 INTO,立刻会产生一个类型 4 的中断,若 OF 为 0,INTO 指令不起作用。因此在加、减法运算指令后应安排一条 INTO 指令,否则运算产生溢出后无法向 CPU 发出溢出中断请求。

例 7-2 测试加法的溢出

```
ADD    AX, VALUE
INTO
```

(3) 由调试程序 debug 设置的中断

在调试程序时,为了检查中间结果或寻找程序中的错误,在程序中可设置断点或进行单步跟踪,调试程序 debug 有此功能,它也是由中断来实现的。

① 单步中断:单步是每次只执行一条指令,然后屏幕显示出当前各寄存器和有关存储

单元的内容,以及下条要执行的指令。这样逐条运行指令,来跟踪程序的流程,以检查出程序中的错误。

单步中断是在标志位 $TF=1$ 时,每条指令执行后,CPU 自动产生的类型 1 中断。产生单步中断时,CPU 同样自动地将 PSW、CS 和 IP 内容入栈,然后清除 TF、IF,进入单步中断处理程序。单步处理程序结束时,原来的 PSW 从堆栈中取出,CPU 重新置成单步方式。

② 断点中断:断点中断为中断类型 3,用 debug 调试程序时,可用 G 命令设置断点。当 CPU 执行到断点时便产生中断,同时显示当前各寄存器和有关存储器的内容及下条要执行的指令,供用户检查。设置断点实际上是把一条断点指令 INT 3 插入到断点设置处,CPU 执行到 INT 3 指令便产生类型 3 中断。断点可以设在程序的任何位置,并可以设置多个。

上面谈到的三种类型的软件中断,都是不可屏蔽中断。

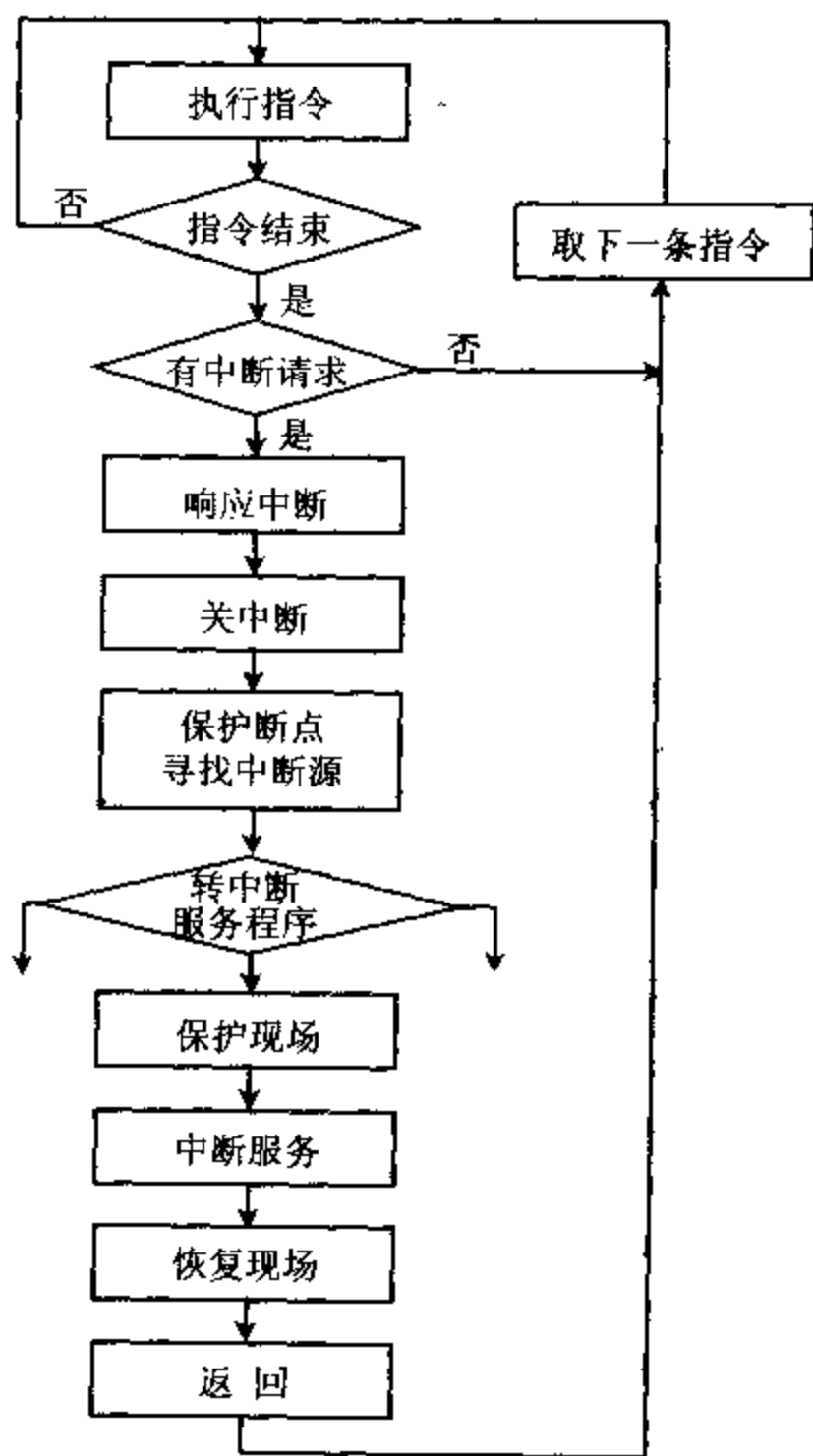


图 7-2 可屏蔽中断处理流程图

7-2 中断处理过程

可屏蔽中断处理的过程一般分成几步:中断请求;中断响应;保护现场;转入执行中断服务子程序;恢复现场和中断返回。其流程如图 7-2 所示。

一、CPU 响应中断过程

CPU 响应中断要有三个条件:

- 外设提出中断申请
- 本中断位未被屏蔽
- 中断允许

当中断接口电路中的中断屏蔽触发器未被屏蔽时,外设可通过中断接口发出中断申请。外设向 CPU 发出中断请求的时间是随机的,而 CPU 在每条指令的最后一个机器周期的最后一个 T 状态去采样中断请求输入线 INTR,当 CPU 在 INTR 引脚上接收到一个有效的中断请求信号,而 CPU 内部中断允许触发器是开放的(开中断可用指令 STI 来实现),则在当前指令执行完后 CPU 响应中断。

CPU 响应中断后,对外设接口发出两个中断响应信号 $\overline{INTA}$,外设收到第二个 $\overline{INTA}$ 以

后,立即往数据线上给 CPU 送中断类型号。CPU 在响应外部中断,并转入相应中断服务子程序的过程中,自动依次做以下工作:

- (1) 从数据总线上读取中断类型号,将其存入内部暂存器。
- (2) 将标志寄存器 PSW 的值入栈。
- (3) 将 PSW 中的中断允许标志 IF 和单步标志 TF 清 0,以屏蔽外部其它中断请求,及避免 CPU 以单步方式执行中断处理子程序。
- (4) 保护断点,将当前指令下面一条指令的段地址 CS 和指令指针 IP 的值入栈,使中断处理完毕后,能正确返回到主程序继续执行。
- (5) 根据中断类型号到中断向量表中找到中断向量,转入相应中断服务子程序。
- (6) 中断处理程序结束以后,从堆栈中依次弹出 IP、CS 和 PSW,然后返回主程序断点处,继续执行原来的程序。

在有些情况下,即使中断允许标志位 IF 为 1,CPU 也不能立即响应外部的可屏蔽中断请求,而是要再执行完下一条指令才响应外部中断。例如,发出中断请求时,CPU 正在执行封锁指令。如果执行向段寄存器传送数据的指令(MOV 和 POP 指令),也要等下一条指令执行完后,才允许中断。当遇到等待指令或串操作指令时,允许在指令执行过程中进入中断,但在一个基本操作完成后响应中断。

对不可屏蔽中断请求,不必判断 IF 是否为 1,也不是由外设接口给出中断类型号,从 NMI 引脚进入的中断请求规定为类型 2。在运行中断子程序过程中,若 NMI 引脚上有不可屏蔽中断请求进入,CPU 仍能响应。

软件中断由程序设定,没有随机性,它不受中断允许标志位 IF 的影响,中断类型号由指令 INT n 中 n 决定。正在执行软件中断时,如果有不可屏蔽中断请求,就会在当前指令执行完后立即予以响应。如果有可屏蔽中断请求,并且 IF=1,也会在当前指令执行完后予以响应。

图 7-3 给出了 8086/8088 的中断响应过程。

二、中断向量表

寻找中断源可以用查询中断及矢量中断两种方法。

查询中断是采用软件查询方法,中断响应后启动中断查询程序,依次查询哪个设备的中断请求触发器为 1,检测到后,转向此设备预先设置的中断服务程序入口地址。此方法较简单,但花费时间多,并且后面的设备服务机会少,在 8086 系统中一般采用矢量中断方法。

矢量中断是将每个设备的中断服务程序的入口地址(矢量地址)集中,依次放在中断向量表中。当 CPU 响应中断后,控制逻辑根据外设提供的中断类型号查找中断向量表,然后将中断服务程序的入口地址送到段寄存器和指令指针寄存器,CPU 转入中断服务子程序。这样大大加快了中断处理的速度。

1. 中断向量表

中断向量表又称中断服务程序入口地址表。8086/8088 系统允许处理 256 种类型的中断,对应类型号为 0~FFH。在存储器的 00000H~003FFH,占用 1K 字节空间,用作存放中断向量。每个类型号占 4 个字节,高 2 个字节存放中断入口地址的段地址,低 2 个字节存

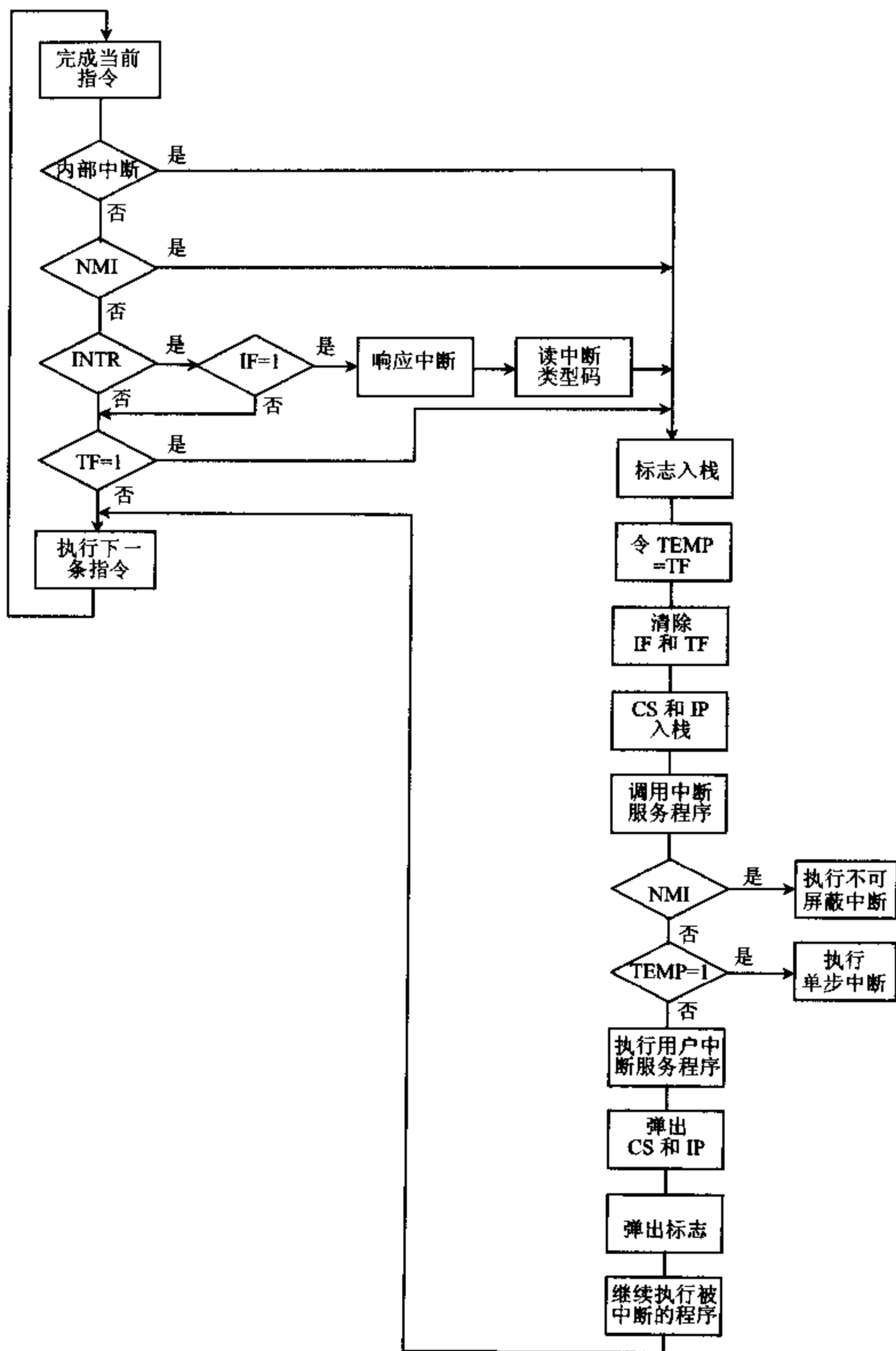


图 7-3 8086/8088 的中断响应过程

放段内偏移地址,如图 7-4 所示:

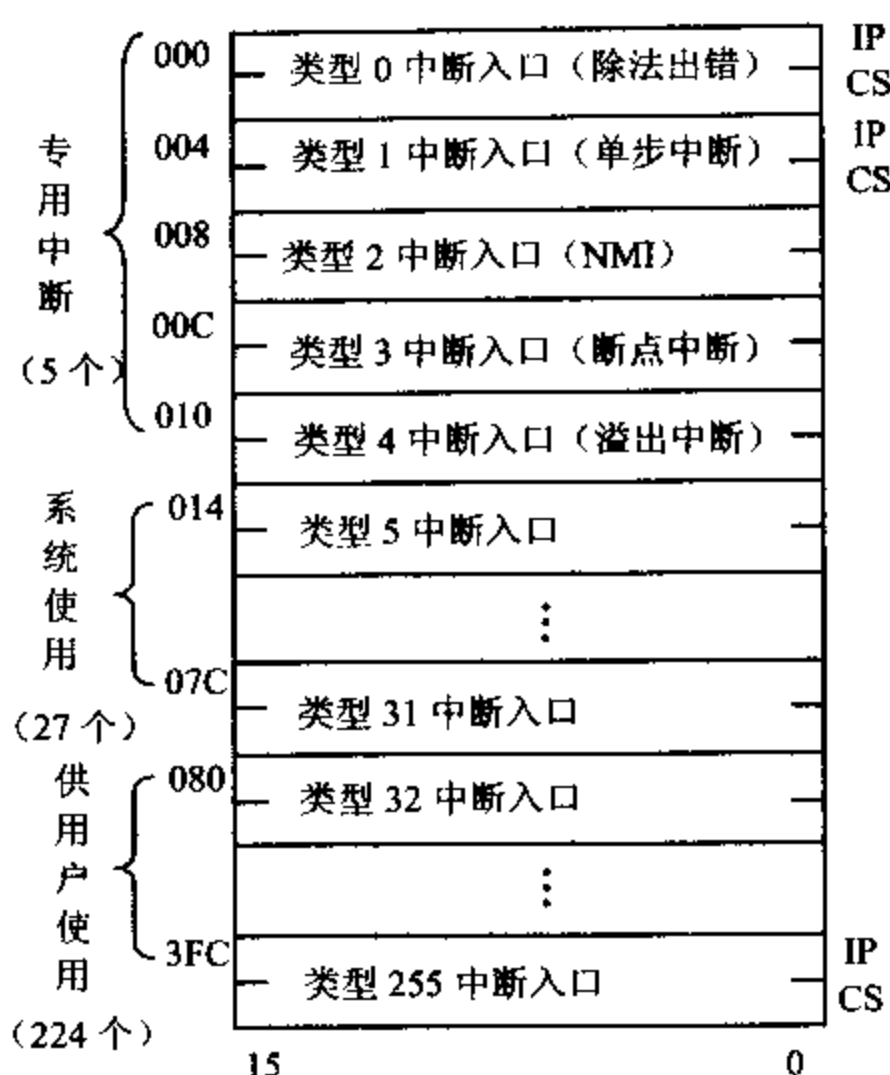


图 7-4 8086/8088 中断向量表

各个中断处理程序的段地址和段内偏移地址按中断类型号顺序存放在中断向量表中。因此由中断类型号 $n \times 4$ 即可得到相应中断向量的地址,取 $4n$ 和 $4n+1$ 单元中的内容(中断入口段内偏移地址)装入指令指针寄存器 IP,取 $4n+2$ 和 $4n+3$ 单元中内容(中断入口段地址)装入代码段寄存器 CS,即可转入中断处理程序。

例 7-3 某中断的中断类型号为 68H,图 7-5 图示了中断操作过程:

- (1) 取中断类型号 68H
- (2) 计算中断向量地址 $68H \times 4 = 1A0H$
- (3) 取中断入口地址的偏移地址送入 IP, $IP = 2050H$,段地址送入 CS, $CS = A000H$
- (4) 转向中断服务程序
- (5) 中断返回到 INT 68H 指令的下一条指令

2. 中断向量(中断入口地址)的设置

IBM PC 对 256 种中断类型已进行了地址分配,附录 D 中给出了中断向量地址分配表。其中类型 0~4 为专用中断,中断入口地址已由系统定义,用户不能修改。类型 0 为除法出错中断,类型 1 为单步中断,类型 2 为 NMI 端引入的不可屏蔽中断,类型 3 为断点中断,类型 4 为溢出中断。类型 5~类型 31 为系统使用中断,不允许用户修改,类型 08H~0FH 为 8259A 中断向量,类型 10H~1FH 为 BIOS 中断向量。

其余的中断类型号原则上可以由用户定义,但是,实际上,有些中断类型目前已有用途,例 INT 21H 为系统功能调用,中断类型 20H~3FH 为 DOS 中断调用。

需要说明的是 80286 以上的微处理器前 5 个中断向量与 8086 系统是相同的,其它的中

断向量不向下兼容 8086 系统,例如类型 5 为边界中断,类型 7 为协处理器不存在中断等等。另外注意保护模式下的中断,其中断向量表使用一组存储在中断描述符表中的 256 个中断描述符取代中断向量。

供用户使用的中断类型号,它可由用户定义为软中断,由 INT n 指令引用;也可通过 INTR 端直接接入,或通过中断控制器 8259A 引入可屏蔽硬件中断。使用时用户要自己将中断服务程序入口地址置入相应的中断向量表内。有两种方法可为中断类型号 n 设置中断向量,即将中断服务程序的入口地址置入中断类型号 n 所对应的中断向量表中。一种方法用指令来设置,另一种方法利用 DOS 功能调用来设置。

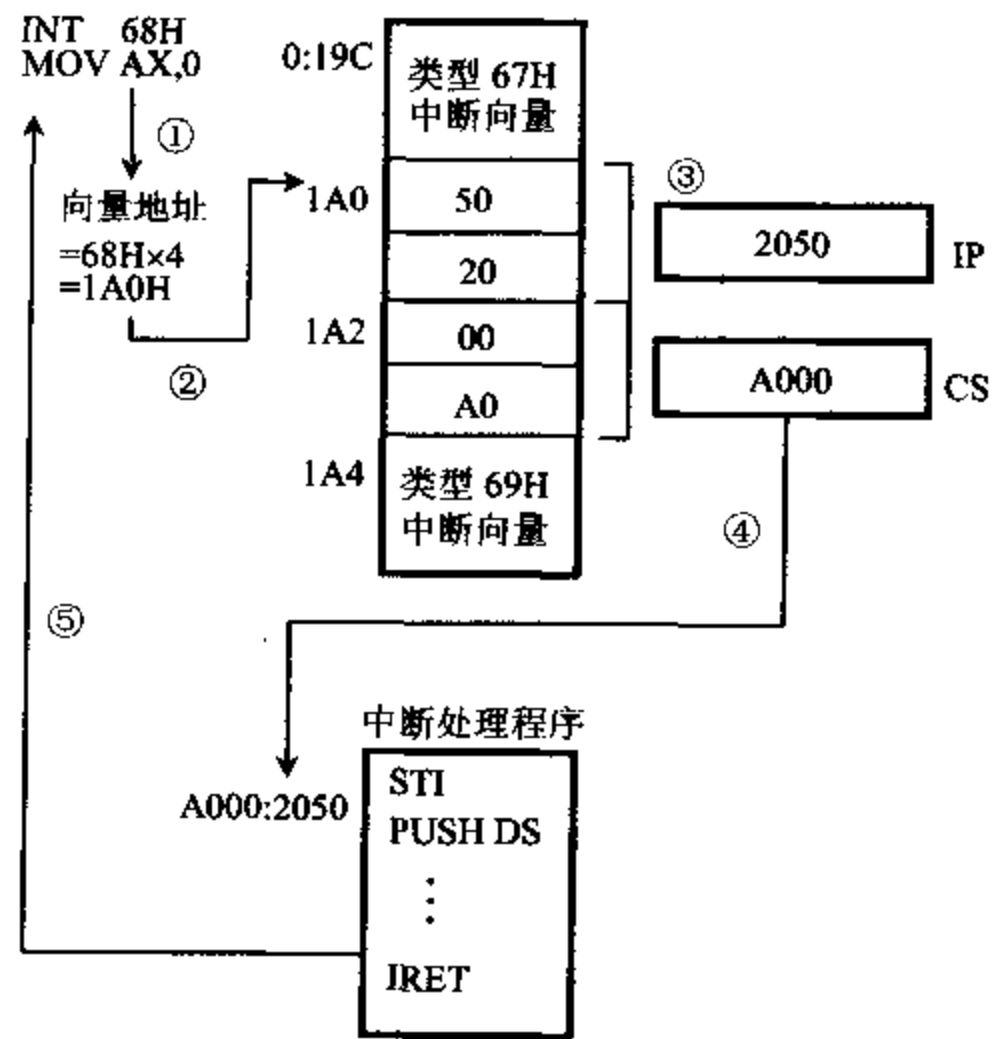


图 7-5 中断操作过程例子

例 7-4 用指令来设置中断服务程序的入口地址到中断类型号 n 所对应的中断向量表中。

```

MOV    AX, 0                ;主程序中设置
MOV    ES, AX
MOV    DI, N * 4            ;中断类型号 * 4
MOV    AX, OFFSET INTRAD    ;送中断子程序的偏移地址→AX。
CLD
STOSW                          ;偏移地址送到[4n],[4n+1]单元
MOV    AX, CS
STOSW                          ;段地址送到[4n+2],[4n+3]单元
STI                                ;开中断
:
INTRAD:                          ;中断服务子程序
PUSH    AX
STI

```

```

:
POP    AX
IRET

```

例 7-5 用指令设置中断向量方法之二。

```

MOV    AX, 0
MOV    ES, AX
MOV    BX, N * 4
MOV    AX, OFFSET INTRAD      ;置入偏移地址
MOV    ES;WORD PTR[BX], AX
MOV    AX, SEG INTRAD         ;置入段地址
MOV    ES;WORD PTR[BX+2], AX
STI
:
INTRAD:                                ;中断服务子程序
:
IRET

```

实际上,在设置或检查任何中断向量时,总是避免直接使用中断向量的绝对地址,而是利用 DOS 功能调用 INT 21H 设置中断向量和取出中断向量。此外要注意,在设置自己的中断向量时,应先保存原中断向量,再设置新的中断向量,在程序结束前恢复原中断向量。

设置中断向量:把由 AL 中指定中断类型号的中断向量 DS:DX,放置在中断向量表中。

预置:AL=中断类型号

DS:DX=中断服务程序入口地址

AH=25H

执行:INT 21H

取中断向量:把由 AL 中指定中断类型号的中断向量,从中断向量表中取到 ES:BX 中。

预置:AL=中断类型号

AH=35H

执行:INT 21H

返回:ES:BX=中断服务程序入口地址

例 7-6 利用 DOS 功能调用设置中断向量和取中断向量。

```

MOV    AL, N                                ;取中断向量到 ES:BX 中
MOV    AH, 35H
INT     21H
PUSH    ES                                ;存原中断向量
PUSH    BX
PUSH    DS
MOV     AX, SEG INTRAD                    ;设置中断向量段地址在 DS
MOV     DS, AX
MOV     DX, OFFSET INTRAD
MOV     AL, N                            ;中断类型号 n

```

```

MOV    AH, 25H                ;设置中断向量
INT     21H
POP     DS
:
POP     DX                    ;恢复原中断向量
POP     DS
MOV     AL, N
MOV     AH, 25H
INT     21H
RET
INTRAD:                        ;中断服务子程序
:
IRET

```

3. 中断类型号的获取

矢量中断中,中断入口地址与中断类型号有关,那么中断类型号如何获取呢?

(1) 对于除法出错,单步中断,不可屏蔽中断 NMI,断点中断和溢出中断,CPU 分别自动提供中断类型号 0~4。

(2) 对于用户自己确定的软件中断 INT n,类型号由 n 决定。

(3) 对外部可屏蔽中断 INTR,可用硬件电路(例如通用并行接口芯片 8212)设计产生中断类型号。

(4) 对外部可屏蔽中断 INTR,可以用可编程中断控制器 8259A 获得中断类型号。

如图 7-1 所示,8 个中断请求信号接到 8259A。当外设申请中断时,8259A 响应优先权高的中断源,将中断请求信号送到 CPU 的 INTR 端。8259A 收到 CPU 发出的第二个中断响应信号 $\overline{INTA}$ 时,将对应中断源的中断类型号送给 CPU,CPU 获取中断类型号后,自动转入相应的中断服务程序。

IBM PC 机内装有一片 8259A,它的中断入口分配如表 7-1 所示:

表 7-1 8259A 的中断源分配

8259A 输入端	中断类型号	外 设
IR ₀	08H	定时器(通道 0)
IR ₁	09H	键盘
IR ₂	0AH	彩色图像接口
IR ₃	0BH	未用
IR ₄	0CH	串行(RS-232 接口)
IR ₅	0DH	未用
IR ₆	0EH	软盘
IR ₇	0FH	打印机

8259A 中有中断屏蔽寄存器,它的端口地址为 21H,中断屏蔽寄存器的位 7~0 对应 IR₇~IR₀。可以通过设置寄存器的各位为 0 或 1 去控制每一个中断源的中断允许或屏蔽,此位为 0,允许中断,此位为 1,禁止中断。在 8259A 初始化中应给出此设置。

例 7-7 若某系统中允许定时及键盘中断,则中断屏蔽控制字为:

```
MOV    AL, 11111100B
OUT    21H, AL
```

在中断服务程序的结束处,应发出中断结束命令(EOI)给中断命令寄存器,中断命令寄存器的端口地址为 20H。

例 7-8 IBM PC 机中断结束命令的程序为:

```
MOV    AL, 20H
OUT    20H, AL
```

综合上面的介绍,我们对中断有了一定的了解,下面将有关中断的主程序编写方法归纳如下:

1. 主程序中的初始化

- (1) 设置中断向量。
- (2) 设置 8259A 的中断屏蔽寄存器的中断屏蔽位。
- (3) 设置 CPU 中断允许位标志 IF(开中断 STI)。

2. 硬件(外设接口)和 CPU 自动完成

- (1) 外设接口向 CPU INTR 端发中断请求。
- (2) 当前指令执行完后,CPU 发两个中断响应信号 $\overline{INTA}$ 给外设接口。
- (3) CPU 取中断类型号 n 。
- (4) CPU 自动将当前 PSW、CS、IP 内容入栈保护。
- (5) 清除 IF、TF,禁止外部中断和单步中断。
- (6) 从中断向量表中取 $(4n)$ 地址中内容 $\rightarrow$ IP,取 $(4n+2)$ 地址中内容 $\rightarrow$ CS。
- (7) 转向中断服务子程序。

这里一定要注意三点:

(1) 对重复前缀的指令(如 REP MOVSB)作为一条指令处理。执行一次重复前缀和串指令即可响应中断,而不是把串操作全部执行完。

(2) 遇到开中断指令 STI 和中断返回指令 IRET,要在这两条指令执行完后,再执行一条指令才能响应中断。

(3) CPU 自动清除 IF 及 TF 位,使 CPU 进入中断服务程序后,不允许再产生新的中断,如果在中断服务程序中还允许外部中断进入,则在中断服务程序中必须再开中断。

三、中断服务子程序

中断服务子程序的功能各有不同,但所有的中断服务子程序都有相同的结构形式。

(1) 程序开始必须保护中断时的现场,可以通过一系列 PUSH 指令将 CPU 各寄存器的值入栈保护。

(2) 若允许中断嵌套,则用 STI 指令来设置开中断,使中断允许标志 IF=1。

(3) 执行中断处理程序。

(4) 用 CLI 指令来设置关中断,使中断允许标志 IF=0,禁止其他中断请求进入。

(5) 给中断命令寄存器送中断结束命令 EOI,使当前正在处理的中断请求标志位被清

除,否则同级中断或低级中断的请求仍会被屏蔽掉。

(6) 恢复中断时的现场,通过一系列 POP 指令将 CPU 各寄存器的值恢复。

(7) 用中断返回指令 IRET 返回主程序,此时堆栈中保存的断点值和标志值分别装入 IP、CS 和 PSW。

进入中断服务程序时,TF 和 IF 被清除,不再响应其他外设的中断请求,所以要设置开中断,以允许中断进入,实现中断嵌套。恢复寄存器内容时,为了防止有中断进入破坏其内容,要执行关中断,然后在中断返回时,原来的 PSW 返回,使 IF=1,又再开中断,这样返回主程序后,中断请求能得到允许。中断结束命令 EOI 一般在中断处理结束前发出,使一次中断处理的过程是完整的。

例 7-9 编写中断处理程序,要求主程序运行时,每 10 秒响铃一次,同时屏幕上显示信息“The bell is ring!”

可以利用中断类型 1CH 进行处理,因为系统定时器(中断类型 8)的中断处理程序中,时钟中断一次(约 18.2 次/秒)要调用一次 INT 1CH。在 ROM BIOS 中,1CH 的处理程序只有一条 IRET 指令,仅为用户提供一个中断类型号。这样可以利用系统定时器的中断间隔,将用户设计的程序来代替原有的 INT 1CH 程序。在编写程序时,有两个部分的工作:

(1) 在主程序初始化部分,先保存当前中断向量表内容,再置新的中断向量。

(2) 在主程序结束部分恢复保存的 1CH 向量。

```
DATA    SEGMENT
        COUNT    DW    1
        MESS     DB    'The bell is ring!',0AH,0DH,'$'
DATA    ENDS
STACK   SEGMENT
        DB    100    DUP(?)
STACK   ENDS
CODE    SEGMENT
MAIN    PROC      FAR
        ASSUME    CS:CODE, DS:DATA, ES:DATA, SS:STACK
START:  MOV        AX, STACK
        MOV        SS, AX
        PUSH       DS
        SUB        AX, AX
        PUSH       AX
        MOV        AX, DATA
        MOV        DS, AX
        MOV        AL, 1CH                ;得到原中断向量
        MOV        AH, 35H
        INT        21H
        PUSH       ES                    ;存储原中断向量
        PUSH       BX
        PUSH       DS
        MOV        DX, OFFSET RING        ;RING 的偏移地址和段地址
```

```

        MOV     AX, SEG RING
        MOV     DS, AX
        MOV     AL, 1CH          ;设置中断向量
        MOV     AH, 25H
        INT     21H
        POP     DS
        IN      AL, 21H          ;设置中断屏蔽位
        AND     AL, 0FEH
        OUT     21H, AL
        STI
        MOV     DI, 2000        ;延迟
DELAY:   MOV     SI, 3000
DELAY1:  DEC     SI
        JNZ     DELAY1
        DEC     DI
        JNZ     DELAY
        POP     DX              ;取原中断向量
        POP     DS
        MOV     AL, 1CH
        MOV     AH, 25H
        INT     21H
        RET
MAIN    ENDP
RING    PROC     NEAR
        PUSH    DS
        PUSH    AX
        PUSH    CX
        PUSH    DX
        MOV     AX, DATA
        MOV     DS, AX
        STI
        DEC     COUNT          ;10 秒计数
        JNZ     EXIT
        MOV     DX, OFFSET MESS
        MOV     AH, 09H        ;显示信息
        INT     21H
        MOV     DX, 100
        IN      AL, 61H        ;响铃
        AND     AL, 0FCH
SOUND:  XOR     AL, 02H
        OUT     61H, AL
        MOV     CX, 140H
WAIT:   LOOP    WAIT

```

```

        DEC     DX
        JNE     SOUND
        MOV     COUNT,182           ;10 秒的值
EXIT:   CLI
        POP     DX
        POP     CX
        POP     AX
        POP     DS
        IRET
RING    ENDP
CODE    ENDS
        END     START

```

四、中断响应时序

CPU 对可屏蔽中断请求的响应过程要执行两个连续的中断响应 $\overline{INTA}$ 总线周期，每个总线周期包括 4 个时钟周期 $T_1 \sim T_4$ 。第一个中断响应总线周期，CPU 通知外设准备响应中断，外设应该准备好中断类型号，第二个中断响应总线周期，CPU 接收外设接口发来的中断类型号。

在第一个 $\overline{INTA}$ 周期中，CPU 将地址/数据总线置于浮动状态，在 $T_2 \sim T_4$ 期间发出中断响应信号 $\overline{INTA}$ 给 8259A，表示 CPU 响应此中断请求，禁止来自其它总线控制器的总线请求。在最大模式时，CPU 启动 $\overline{LOCK}$ 信号，通知系统中总线仲裁器 8289，使系统中其它处理器不能访问总线。

在第二个 $\overline{INTA}$ 周期中，CPU 向 8259A 发出第二个 $\overline{INTA}$ 信号，8259A 响应第二个 $\overline{INTA}$ ，在 T_2 和 T_3 周期将一个字节的中断类型号 N 送到数据总线低 8 位。CPU 读取中断类型号 N ，乘以 4，得到中断向量表的地址继而查得中断服务程序入口地址。然后 CPU 保护 PSW，清标志位 IF 和 TF，将断点返回地址 CS 和 IP 入栈，转向中断服务程序入口。图 7-6 给出了 8086/8088 中断响应时序。8086 执行中断响应时，在两个中断响应周期之间要插入

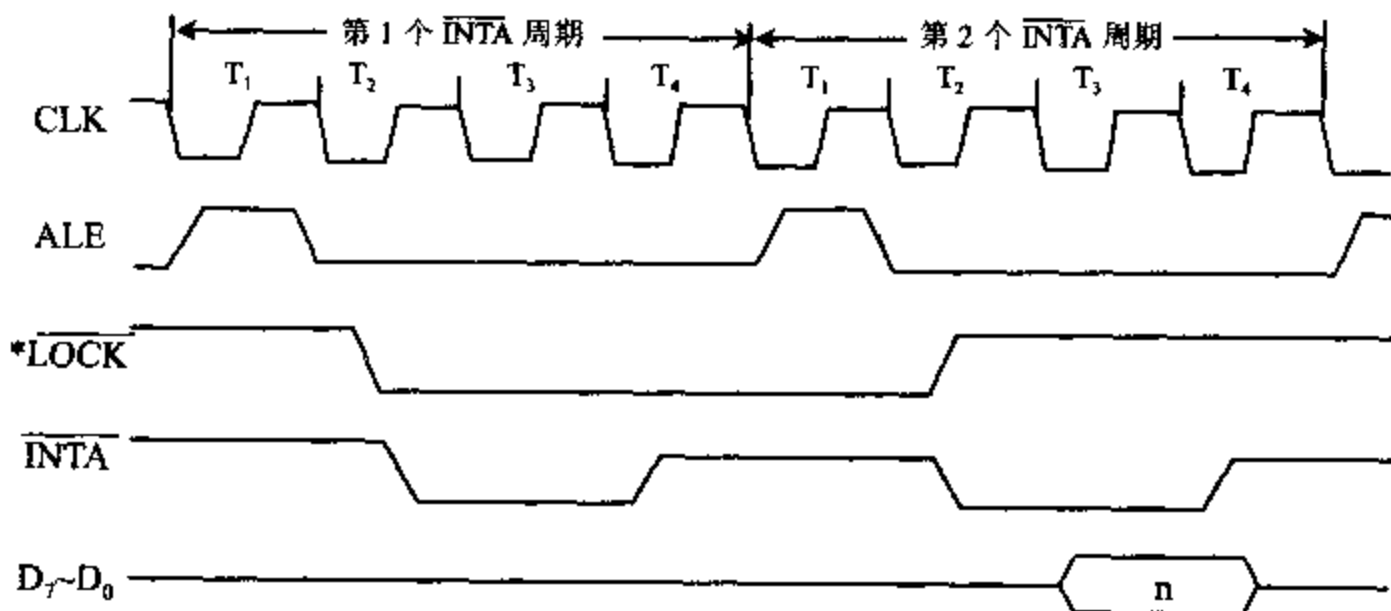


图 7-6 8086/8088 中断响应总线周期时序图

2~3 个空闲状态,8088 系统并没有在两个中断响应总线周期中插入空闲状态。

下面作几点说明:

(1) 8086 要求中断请求信号 INTR 是一个电平信号,必须维持 2 个时钟周期的高电平。否则 CPU 执行完一条指令后,如果总线接口部件正在执行总线周期,则会使中断请求得不到响应而执行其它的总线周期。

(2) 8086 工作在最大模式时,不从 $\overline{\text{INTA}}$ 引脚上发中断响应脉冲,而是由 $\overline{\text{S}}_2\overline{\text{S}}_1\overline{\text{S}}_0$ 组合为 000,通过总线控制器 8288 发出 $\overline{\text{INTA}}$ 中断响应信号。

(3) 8086 不允许在两个 $\overline{\text{INTA}}$ 周期之间响应总线保持请求(通过 HOLD 或 $\overline{\text{RQ}}/\overline{\text{GT}}$ 线请求),但如果同时出现中断请求和总线保持请求,则 CPU 先对总线保持请求服务。

(4) 外设的中断类型号一般通过 16 位数据总线的低 8 位传送给 8086,所以提供中断向量的外设接口应该接在数据总线的低 8 位上。

(5) 在两个中断响应总线周期中,地址/数据/状态总线是浮空的,但是 $\text{M}/\overline{\text{IO}}$ 为低电平,而 ALE 在每个总线周期的 T_1 状态输出一个高电平脉冲,作为地址锁存信号。

(6) 软件中断和非屏蔽中断不按照这种时序来响应中断。

7-3 中断优先级和中断嵌套

在实际系统中,经常有多个中断源同时向 CPU 请求中断,CPU 响应哪个中断源的中断请求,由中断优先级排队决定,CPU 先响应优先级高的中断请求。当 CPU 正在处理中断时,有更高优先级别的中断请求,并且 $\text{IF}=1$,CPU 能响应更高级别的中断请求,而屏蔽掉低级的中断请求,形成了中断嵌套,或称为多重中断。多个中断源的中断流程如图 7-7 所示,与单级中断流程图 7-2 相比,流程图中增加了屏蔽本级与较低级中断请求;中断服务前要开中断,以允许中断嵌套;恢复现场前要关中断,使恢复现场不受干扰;中断返回后自动开中断,以允许其它中断能被 CPU 响应。

一、中断优先级

IBM PC 机中规定优先级从高到低的次序为:

内中断(除法错,INT0,INT n)

不可屏蔽中断(NMI)

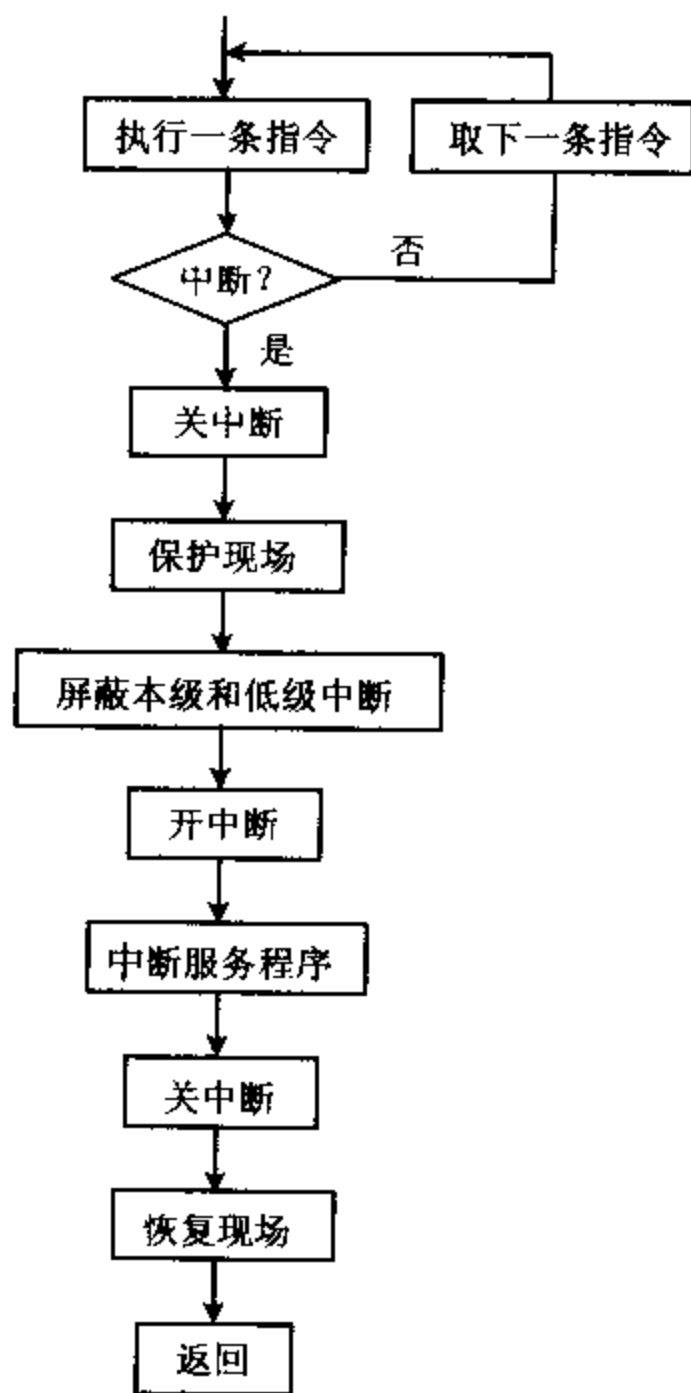


图 7-7 多个中断源中断流程图

可屏蔽中断(INTR)

单步中断

对可屏蔽中断的优先级设定有三种方法:

1. 软件查询中断优先级

软件查询中断方式,是将各个外设的中断请求信号通过或门相“或”后,送到 CPU 的 INTR 端,同时把几个外设的中断请求状态位组成一个端口,赋以端口号。任一外设有中断请求, CPU 响应中断后进入中断处理子程序,用软件读取端口内容,逐位查询端口的每位状态,查到哪个外设有请求中断,就转入哪个外设的中断服务程序。查询程序的次序,决定了外设优先级别的高低,先测试的中断源优先级别最高。当然在软件查询程序中也可用移位或屏蔽法来改变端口各位的测试次序,但查询时间较长,对中断源较多的情况不合适。

2. 硬件查询优先方式——菊花链法

菊花链法是采用硬件查询优先的方式,它是在每个外设的对应接口上连接一个逻辑电路构成一个链,控制了中断响应信号的通路,图 7-8 给出了它的原理图。

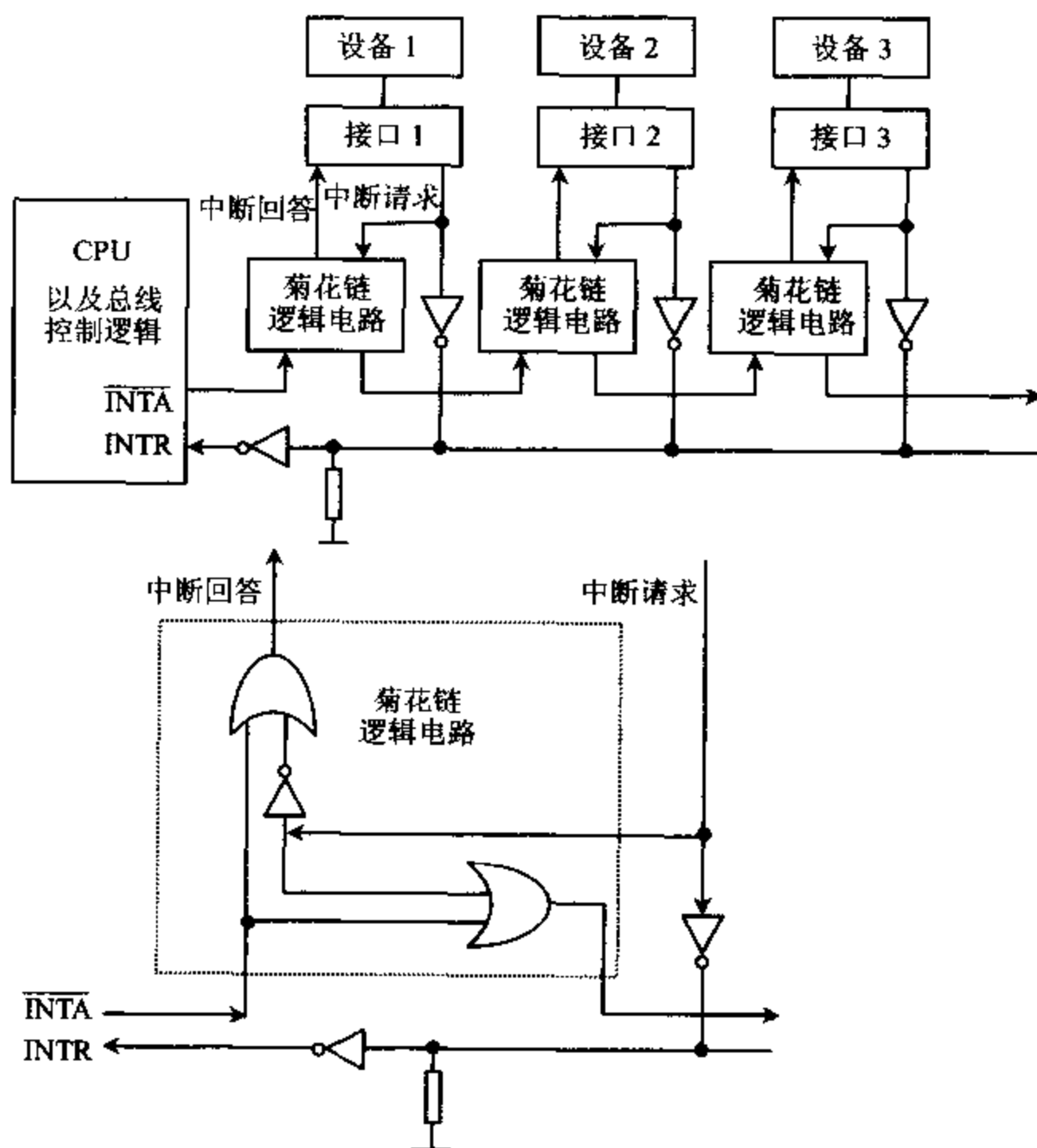


图 7-8 菊花链优先查询法

当任一外部设备申请中断后,中断请求信号送到 CPU 的 INTR 端, CPU 发出 $\overline{INTA}$ 中

断响应信号。当前一组的外设没有发出中断申请时, $\overline{INTA}$ 信号会沿着菊花链线路向后传递, 传送到发出中断请求的接口。当某一级的外设发出了中断申请, 此级的逻辑电路就阻塞了 $\overline{INTA}$ 的通路, 后面的外设接口不能接收到 $\overline{INTA}$ 信号。此级接口收到 $\overline{INTA}$ 信号后撤销中断请求信号, 向总线发送中断类型号, 从而 CPU 可以转入中断处理。

当多个外设接口同时申请中断时, 显然最接近 CPU 的接口优先得到中断响应。菊花链的排列, 使外设接口不会竞争中断响应信号 $\overline{INTA}$, 从硬件线路上就决定了越靠近 CPU 的外设接口, 优先级越高, 首先响应中断。

3. 矢量中断优先级

矢量中断优先级的设置是采用中断优先级控制器。图 7-9 给出了它的典型设计原理框图。

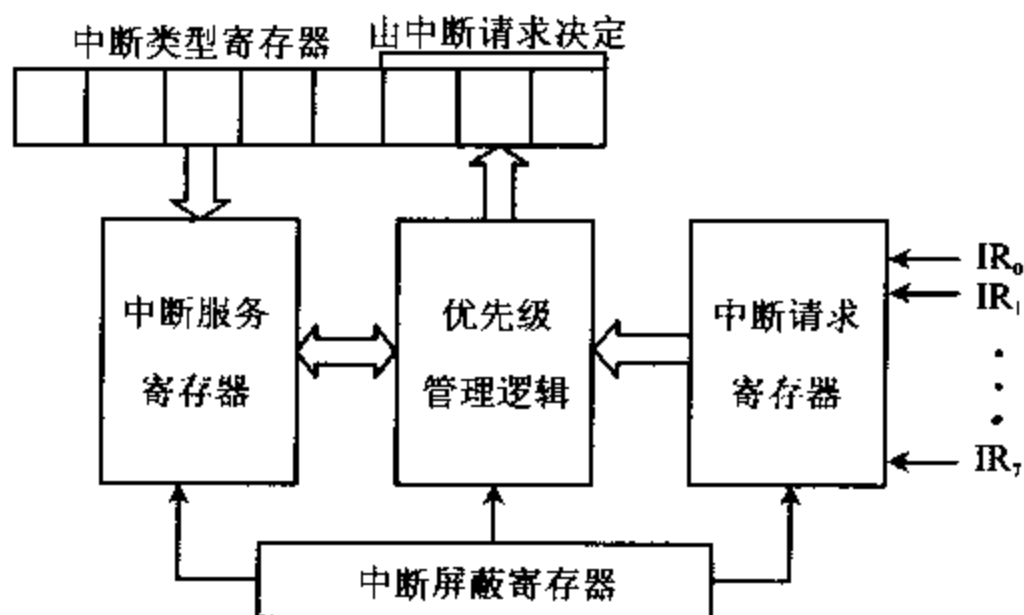


图 7-9 矢量中断优先级控制器的原理图

外设可以有 8 个中断请求 $IR_0 \sim IR_7$ 送入中断请求寄存器, 中断屏蔽寄存器可由用户设置屏蔽某几位的中断请求。中断优先级管理逻辑电路判别出最高优先级中断请求, 将其中断级转换成 3 位码, 送到中断类型寄存器的低 3 位及当前中断服务寄存器。此后, 中断优先级控制器向 CPU 发出中断请求信号, 当 CPU 开中断时, CPU 发出中断响应信号, 如上所述开始一个中断处理过程。中断处理结束引起中断服务寄存器对应位清 0, 级别较低的中断请求才能得到响应。

可编程中断控制器 8259A 就是这样一种结构的芯片, 通过软件的设置可以有多种优先权设置方式, 下节将详细说明。

二、中断嵌套

IBM PC 机没有规定中断嵌套的深度, 但使用中受到堆栈容量的限制, 必须要有足够的堆栈单元来保存多重中断的断点及寄存器。8259A 在完全嵌套优先级工作方式下, 中断优先次序为 $IR_0, IR_1 \dots IR_7$, 图 7-10 图示说明了中断嵌套序列的例子。

主程序执行过程中 IR_2, IR_1 中断请求到达, 优先执行 IR_2 处理程序。正在执行中, IR_1 中断请求又进入, IR_1 优先级高, CPU 中断 IR_2 处理程序转去执行 IR_1 处理程序。 IR_1 处理程序结束, 发出 EOI 结束命令, 并返回 IR_2 处理程序。因 IR_2 处理程序中提前发出了 EOI

结束命令, IR_2 中断服务寄存器对应位清 0, 允许级别较低的 IR_4 处理程序执行。在 IR_4 处理程序中, IR_3 中断请求到达。在开中断后 IR_3 优先级高, 转入 IR_3 中断处理程序, IR_3 中断处理完毕, 发出 EOI 结束命令及 IRET 命令, 返回到 IR_4 程序, 同样 IR_3 、 IR_2 逐级返回主程序。

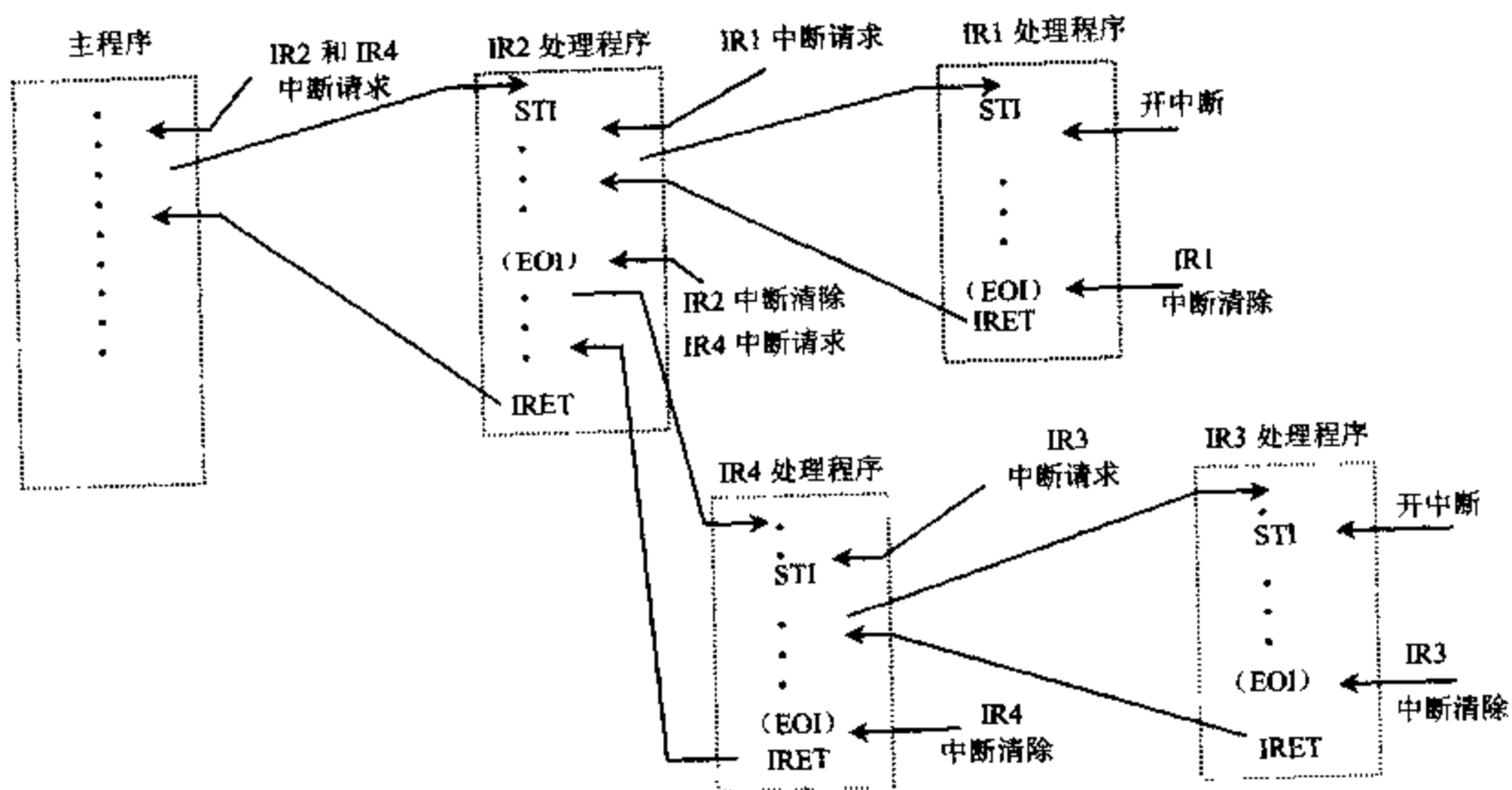


图 7-10 中断嵌套序列

从图 7-10 可以看到:

1. 主程序必须有开中断指令, 使 $IF=1$ 才能响应中断。进入中断处理程序时, 系统自动关中断, 在中断服务程序中必须有 STI 开中断指令, 这样才可以允许其它中断进入实现中断嵌套。
2. 中断结束返回前要有 EOI 中断结束命令, 用来清除中断服务寄存器中的对应位, 允许低级中断进入。最后有中断返回指令 IRET, 使程序返回到被中断的程序的断点处。
3. 中断处理程序中如果没有 STI 指令, 中断处理中不会受其它中断影响, 在执行 IRET 指令后, 因为自动返回中断断点及中断标志寄存器 PSW 的内容, 当 IF 的值为 1, 系统便能开放中断。
4. 一个正在执行的中断处理程序, 中断服务寄存器相应位置“1”, 在开中断 ($IF=1$) 的情况下, 能够被优先级高于它的中断源中断。但如果中断处理中提前发出了 EOI 命令, 则清除了正在执行的中断服务, 中断服务寄存器置“1”位被清 0, 允许响应同级或低级的中断申请。从图 7-10 中可以看到在 IR_2 处理程序中, 由于发出了 EOI 命令, 清除了 IR_2 的中断申请。所以较低优先级的 IR_3 请求到达后, 转去处理 IR_4 中断请求。但这种情况要尽量避免, 防止重复嵌套, 使优先级高的中断不能及时服务, 因此一般 EOI 结束命令放在中断返回指令 IRET 前面。

7-4 可编程中断控制器 8259A

一、功能和引脚

8259A 是 8086/8088 系列的可编程中断控制器,它的主要功能是:

- (1) 具有 8 级优先级控制,通过级联可以扩展到 64 级优先级控制。
- (2) 每一级中断可由程序单独屏蔽或允许。
- (3) 可提供中断类型号传送给 CPU。
- (4) 可以通过编程选择多种不同工作方式。

8259A 为 28 个引脚的双列直插式芯片,其引脚图和内部结构方框图如图 7-11 所示。

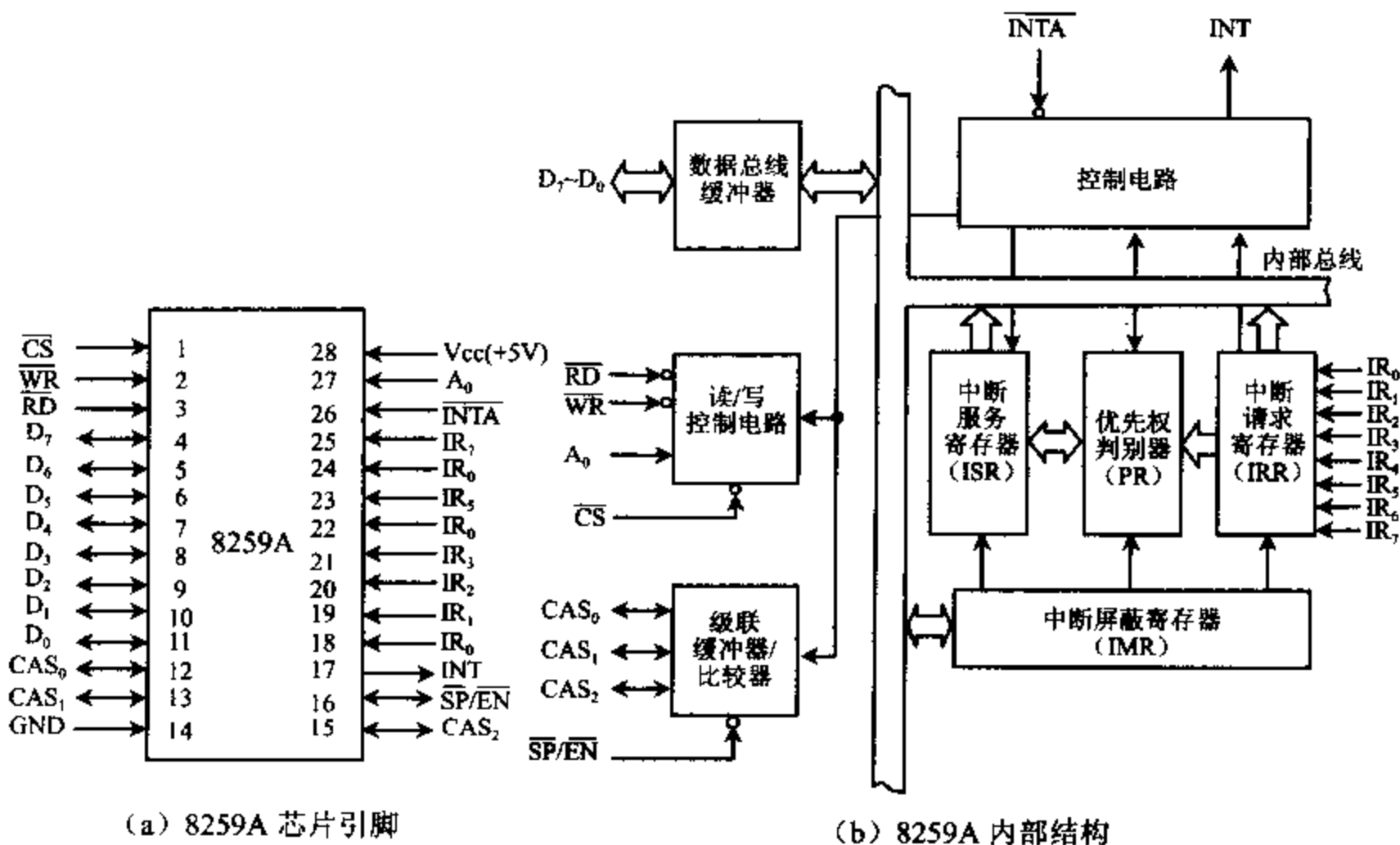


图 7-11 8259A 引脚和内部结构方框图

$D_7 \sim D_0$: 双向数据线,三态,它直接或通过总线驱动器与系统的数据总线相连。

$IR_7 \sim IR_0$: 外设的中断请求信号输入端,输入,中断请求信号可以是电平触发或边沿触发。中断级联时,连接 8259A 从片 INT 端。

$\overline{RD}$: 读命令信号,输入,低电平有效,用来控制数据由 8259A 读到 CPU。

$\overline{WR}$: 写命令信号,输入,低电平有效,用来控制数据由 CPU 写到 8259A。

$\overline{CS}$: 片选信号,输入,通过译码电路与高位地址总线相连。

A_0 : 选择 8259A 的两个端口,输入,连低位地址线。

INT: 向 CPU 发出的中断请求信号,输出,与 CPU 的 INTR 端相连。

$\overline{INTA}$: CPU 给 8259A 的中断响应信号, 输入。8259A 要求两个负脉冲的中断响应信号, 第一个是 CPU 响应中断的信号, 第二个 $\overline{INTA}$ 结束后, CPU 读取 8259A 送去的中断类型号。

$CAS_2 \sim CAS_0$: 双向级联信号线。8259A 作主片时, 为输出线, 作从片时, 为输入线。与 $\overline{SP}/\overline{EN}$ 配合实现 8259A 级联。

$\overline{SP}/\overline{EN}$: 编程/双向使能缓冲。作为输入使用时, 用来决定本片 8259A 是主片还是从片; 若 $\overline{SP}/\overline{EN} = 1$, 则为主片; 若 $\overline{SP}/\overline{EN} = 0$, 则为从片。作为输出使用时, 启动 8259A 到 CPU 之间的数据总线驱动器。 $\overline{SP}/\overline{EN}$ 作为输入还是输出, 决定于 8259A 是否采用缓冲方式工作, 若采用缓冲方式工作, 则 $\overline{SP}/\overline{EN}$ 作为输出, 若采用非缓冲方式, $\overline{SP}/\overline{EN}$ 作为输入。

二、内部结构

1. 数据总线缓冲器

数据总线缓冲器是 8 位双向三态缓冲器, 是 8259A 与系统数据总线接口, 通常连接低 8 位数据总线 $D_7 \sim D_0$ 。CPU 编程控制字写入 8259A、8259A 的状态信息读出、及中断响应时 8259A 送出的中断类型号, 都经过它传送。

2. 读写控制电路

读写控制电路接收 CPU 送来的读/写命令 $\overline{RD}$ 、 $\overline{WR}$, 片选信号 $\overline{CS}$ 及端口选择信号 A_0 。高位地址译码后送 $\overline{CS}$ 作片选信号。 A_0 连地址总线 A_0 或 A_1 , 用来选择 8259A 的两个 I/O 端口, 一个为奇地址, 另一个偶地址。读写操作由这 4 个信号控制来实现的, 使 8259A 接收 CPU 送来的初始化命令字 (ICW) 和操作命令字 (OCW), 或将内部状态信息送给 CPU。 $\overline{RD}$ 、 $\overline{WR}$ 、 $\overline{CS}$ 、 A_0 的控制作用见表 7-2。表 7-2 中 D_4 、 D_3 代表控制字的第 4 位和第 3 位。

在 IBM PC/XT 机中用 $A_9 \sim A_1$ 译码来产生 $\overline{CS}$ 信号, 组合为 $00001 \times \times \times \times$, 产生 I/O 端口地址为 $20H \sim 3FH$, 共 32 个。而 8259A 只需要两个 I/O 端口地址, IBM PC/XT 取 $20H (A_0 = 0)$ 、 $21H (A_0 = 1)$ 两个地址在编程时使用。但其它 30 个地址为影像地址, 不可能再分配给其它 I/O 设备使用。

表 7-2 8259A 的读写功能

$\overline{CS}$	$\overline{RD}$	$\overline{WR}$	A_0	D_4	D_3	读写操作	指令
0	1	0	0	1	$\times$	CPU $\rightarrow$ ICW ₁	OUT
0	1	0	1	$\times$	$\times$	CPU $\rightarrow$ ICW ₂ 、ICW ₃ 、ICW ₄ 、OCW ₁	
0	1	0	0	0	0	CPU $\rightarrow$ OCW ₂	
0	1	0	0	0	1	CPU $\rightarrow$ OCW ₃	
0	0	1	0			IRR/ISR $\rightarrow$ CPU	IN
0	0	1	1			IMR $\rightarrow$ CPU	
1	$\times$	$\times$	$\times$			高阻	
$\times$	1	1	$\times$			高阻	

8088 系统中数据线为 8 位, 8259A 数据线为 8 位, 所以地址总线的 A_0 连 8259A 的 A_0 , 可以分配给 8259A 两个端口地址, 一个奇地址, 一个偶地址, 从而满足 8259A 的编程要求。

在 8086 系统中数据总线为 16 位, CPU 传送数据时, 低 8 位数据总线传送到偶地址端口, 高 8 位数据总线传送到奇地址端口。当 8 位 I/O 接口芯片与 8086 CPU 16 位数据总线相连接时, 既可以连到低 8 位数据总线, 也可以连到高 8 位数据总线, 实际设计时, 如果 8259A 的 $D_7 \sim D_0$ 与 CPU 数据总线低 8 位相连, 为了保证 CPU 与 8259A 用低 8 位传输数据, CPU 的 A_1 连 8259A 的 A_0 。这样对 CPU 来说 $A_0=0$, A_1 可以为 1 或为 0, CPU 读写始终是用偶地址。对 8259A 来说 A_1 可以为 1 或为 0, 给 8259A 的端口分配了两个地址, 一个奇地址, 一个偶地址, 符合了 8259A 的编程要求。

3. 级联缓冲/比较器

8259A 与系统总线相连有两种方式:

(1) 缓冲方式: 在多片 8259A 级联的系统中, 8259A 通过总线驱动器和数据总线相连, 这就是缓冲方式。在缓冲方式下, 8259A 的 $\overline{SP}/\overline{EN}$ 端与总线驱动器允许端相连, 控制总线驱动器启动, $\overline{SP}/\overline{EN}$ 作为输出端。当 $\overline{EN}=0$, 8259A 控制数据从 8259A 送到 CPU, 当 $\overline{EN}=1$ 时, 控制数据从 CPU 送到 8259A。

(2) 非缓冲方式: 单片 8259A 或少量 8259A 级联时, 可以将 8259A 直接与数据总线相连, 称为非缓冲方式。非缓冲方式下, 8259A 的 $\overline{SP}/\overline{EN}$ 端作输入端, 控制 8259A 作为主片还是从片, $\overline{SP}=1$, 表示此 8259A 为主片, $\overline{SP}=0$, 表示此 8259A 为从片。单片 8259A 时, $\overline{SP}/\overline{EN}$ 接高电平。

由初始化命令字 ICW_1 来设置缓冲方式或非缓冲方式。

4. 中断请求寄存器 IRR

中断请求寄存器是一个 8 位寄存器, 存放外部输入的中断请求信号 $IR_7 \sim IR_0$ 。当某个 IR 端有中断请求时, IRR 相应的某位置“1”。可以允许 8 个中断请求信号同时进入, 此时 IRR 寄存器被置成全“1”。当中断请求被响应时, IRR 的相应位复位。

5. 中断屏蔽寄存器 IMR

中断屏蔽寄存器是一个 8 位寄存器, 用来存放对各级中断请求的屏蔽信息。当用软件编程使 IMR 寄存器中某一位置“0”时, 允许 IRR 寄存器中相应位的中断请求进入中断优先级判别器。若 IMR 中某位为“1”, 则此位中断请求被屏蔽。各个中断屏蔽位是独立的, 屏蔽了优先级高的中断, 不影响其它较低优先级的中断允许。

6. 优先级判别器 PR

优先级判别器对保存在 IRR 寄存器中的中断请求进行优先级识别, 送出最高优先级的中断请求到中断服务寄存器 ISR 中去。当出现多重中断时, PR 判定是否允许所出现的中断去打断正在处理的中断, 让优先级更高的中断优先处理。

7. 中断服务寄存器 ISR

中断服务寄存器是一个 8 位寄存器, 保存正在处理中的中断请求信号, 某个 IR 端的中断请求被 CPU 响应后, 当 CPU 发出第一个 $\overline{INTA}$ 信号时, ISR 寄存器中的相应位置“1”, 一直保持到该级中断处理结束为止。允许多重中断时, ISR 多位同时被置成“1”。

8. 控制电路

控制电路是 8259A 的内部控制器。根据中断请求寄存器 IRR 的置位情况和中断屏蔽寄存器 IMR 设置的情况, 通过优先级判别器 PR 判定优先级, 向 8259A 内部及其它部件发出控制信号。并向 CPU 发出中断请求信号 INT 和接收 CPU 的中断响应信号 $\overline{INTA}$, 使中

断服务寄存器 ISR 相应位置“1”，并使中断请求寄存器 IRR 相应位置“0”。当 CPU 第二个 $\overline{\text{INTA}}$ 信号到来，控制 8259A 送出中断类型号，使 CPU 转入中断服务子程序。如果方式控制字 ICW₄ 的中断自动结束位为“1”，则在第二个 $\overline{\text{INTA}}$ 脉冲结束时，将 8259A 中断服务寄存器 ISR 的相应位清“0”。

三、8259A 的中断管理方式

8259A 有多种工作方式，这些工作方式都是通过编程方法来设置的，使用十分灵活。首先我们来看一下 8259A 的编程结构，然后再介绍 8259A 的中断工作方式。

1. 8259A 的编程结构

从图 7-12 8259A 编程结构中可以看到，中断管理方式是通过 8259A 初始化时写入初始化命令字和操作命令字来设置的。初始化命令字写入寄存器 ICW₁~ICW₄，它是由初始化程序设置的，初始化命令字一经设定，在系统工作过程中就不再改变。操作命令字写入寄存器 OCW₁~OCW₃，它是由应用程序设定的，用来对中断处理过程进行控制，在系统运行过程中，操作命令字可以重新设置。

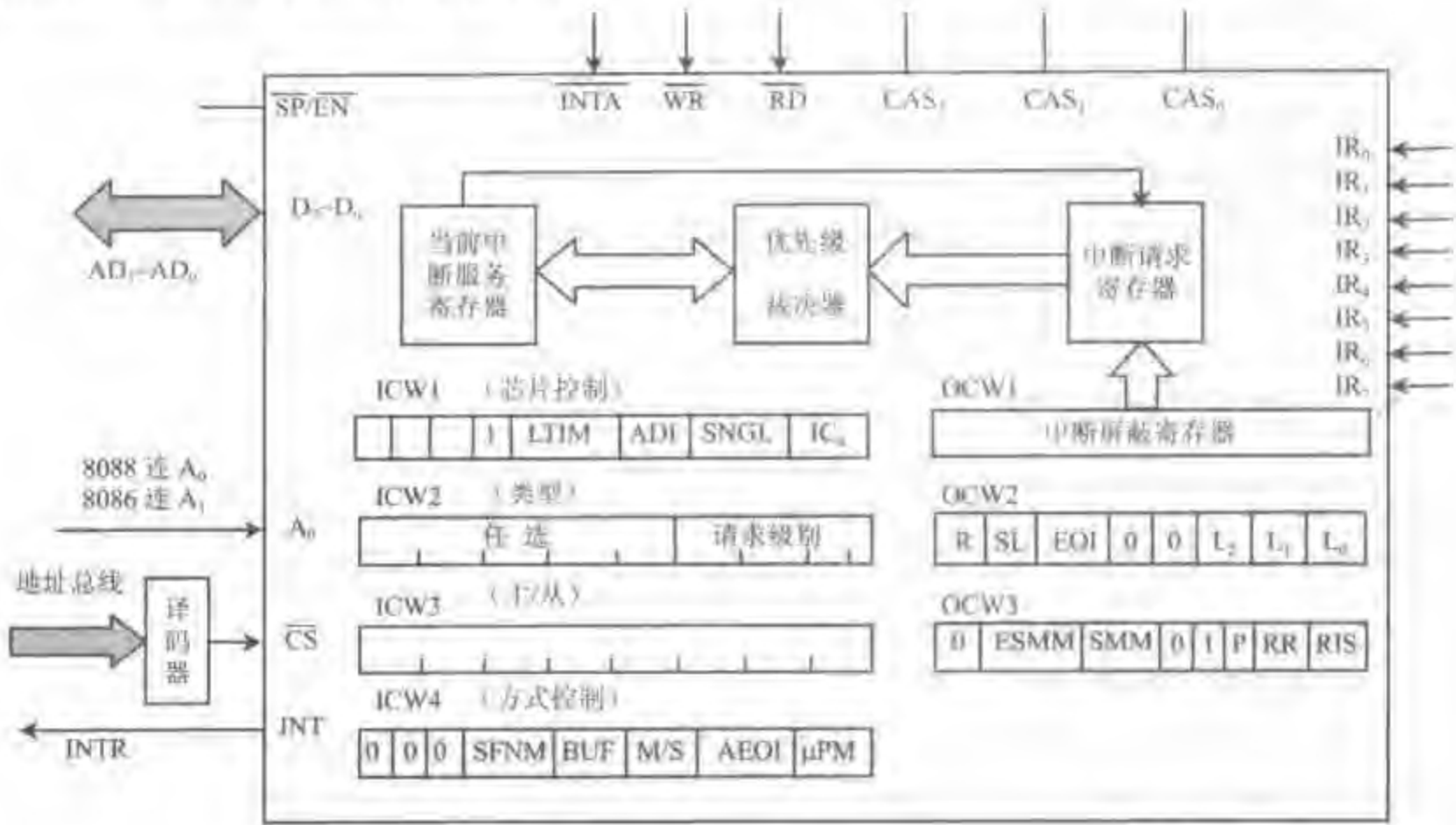


图 7-12 8259A 的编程结构

8259A 的中断优先级的管理采用多种方式，优先级既可以固定设置，又可以循环设置，给用户极大的方便。中断优先级设定后，允许中断嵌套，通常允许高级中断打断低级中断，不允许低级或同级中断打断高级中断。特殊情况下与中断结束方式有关，也可以低级中断打断高级中断，称为重复中断。

2. 优先级设置方式

(1) 完全嵌套方式

若 8259A 初始化后没有设置其它优先级的方式，就自动进入完全嵌套方式。在这种方

式下,中断优先级分配固定级别 0~7 级,IR₀ 具有最高优先级,IR₇ 优先级最低。也可用初始化命令字 ICW₄ 中 SFNM=0,将 8259A 置成完全嵌套优先级方式。

在完全嵌套工作模式下,当一个中断请求被响应后,中断服务寄存器 ISR 中的对应位置“1”,中断类型号被放到数据总线上,CPU 转入中断服务程序。一般情况下(除自动中断结束方式外),在 CPU 发出中断结束命令 EOI 前,ISR 寄存器中此对应位一直保持“1”。当新的中断请求进入时,中断优先级裁决器将新的中断请求和当前 ISR 寄存器中置“1”位比较,判断哪一个优先级更高。允许打断正在处理的中断,优先处理更高级的中断,实现中断嵌套,但禁止同级与低级中断请求进入。中断嵌套时,ISR 寄存器中内容发生变化,又有一个对应位置“1”,当实现 8 级中断嵌套时,ISR 寄存器内容为 0FFH。

在完全嵌套方式中有两种中断结束方式:普通 EOI 结束方式和自动 AEOI 结束方式。

(2) 特殊全嵌套工作方式

在级联时,还有一种特殊全嵌套工作方式,它与全嵌套工作方式基本相同。区别在于当处理某级中断时,有同级中断请求进入,8259A 也会响应,从而实现了对同级中断请求的特殊嵌套。

在级联方式中,主片编程为特殊全嵌套工作方式,从片为其它优先级方式(全嵌套方式或优先级循环方式)。当从片上有中断请求进入并正在处理时,同一从片上又进入更高优先级的中断请求,从片能响应更高优先级中断请求,并向主片申请中断,但对主片来说是同级中断请求。当主片处于特殊全嵌套工作方式时,主片就能允许对相同级别的中断请求开放。当然,和普通全嵌套一样,对来自主片上其它引脚的优先级较高的中断请求是开放的。所以特殊全嵌套工作方式是专门为多片 8259A 系统提供的,可以用来确认从片内部优先级的工作方式。

特殊全嵌套工作方式的设置是主片初始化时 ICW₄ 中的 SFNM=1,同时应将主片 ICW₄ 中 AEOI 位置“0”,设成非自动结束方式,通常用特殊 EOI 结束方式。

(3) 优先级自动循环方式

在优先级自动循环方式中,优先级别可以改变。初始优先级次序规定为 IR₀、IR₁、…、IR₇,当任何一级中断被处理完后,它的优先级别变为最低,将最高优先级赋给原来比它低一级的中断请求,其它依次类推。例当前 IR₃ 中断请求,则处理 IR₃,处理完 IR₃ 后,IR₄ 变成最高优先级,优先级依次为 IR₄、IR₅、IR₆、…、IR₂、IR₃。所以,优先级自动循环方式适合用在多个中断源优先级相等的场合。

用操作命令字 OCW₂ 中 R、SL=10 就可设置优先级自动循环方式。

根据结束方式不同,有两种自动循环方式:普通 EOI 循环方式和自动 EOI 循环方式。

(4) 优先级特殊循环方式

优先级特殊循环方式和优先级自动循环方式相比,不同之处在于优先级特殊循环方式中,初始时最低优先级由程序规定,最高优先级也就确定了。例如初始时指定 IR₁ 为最低优先级,则 IR₂ 为最高优先级,其它依次类推。而优先级自动循环方式初始时最高优先级一定是 IR₀。

用操作命令字 OCW₂ 中 R、SL=11 就可设置优先级特殊循环方式,根据结束方式不同,通常用特殊 EOI 循环方式。

3. 中断结束方式

在固定优先级方式中,对中断结束的处理有自动 AEOI 结束方式和非自动结束方式,非自动结束方式又分普通 EOI 结束方式和特殊 SEOI 结束方式。

中断结束处理实际上就是对中断服务寄存器 ISR 中对应位的处理。当一个中断得到响应时,8259A 使 ISR 寄存器中对应位置“1”,表明此对应外设正在服务,并为中断优先判别器提供判别依据。中断结束时,必须使 ISR 寄存器中对应位置“0”,否则中断优先权判别会不正常。什么时刻使 ISR 中对应位置“0”,就产生不同的中断结束方式。

(1) 普通 EOI 结束方式

在完全嵌套工作方式下,任何一级中断处理结束返回上一级程序前,CPU 向 8259A 传送 EOI 结束命令字,8259A 收到 EOI 结束命令后,自动将 ISR 寄存器中级别最高的置“1”位清“0”(此位对应当前正在处理的中断)。EOI 结束命令字必须放在返回指令 IRET 前,没有 EOI 结束命令,ISR 寄存器中对应位仍为“1”,继续屏蔽同级或低级的中断请求。若 EOI 结束命令字放在中断服务程序中其它位置,会引起同级或低级中断在本级未处理完前进入,容易产生错误。普通 EOI 结束命令字是设置 OCW₂ 中 EOI 位为 1,即 OCW₂ 中 R、SL、EOI 组合为 001。对 IBM PC/XT 机,发 EOI 结束命令字指令为:

```
MOV    AL,20H
OUT    20H,AL           ;8259A 端口为 20H,21H
```

(2) 特殊 EOI 结束方式

在非全嵌套工作方式下,中断服务寄存器无法确定哪一级中断为最后响应和处理的,这时要采用特殊 SEOI 结束方式。CPU 向 8259A 发特殊 EOI 结束命令字,命令字中将当前要清除的中断级别也传给 8259A。此时,8259 将 ISR 寄存器中指定级别的对应位清“0”,它在任何情况下均可使用。

特殊 EOI 结束命令字是将 OCW₂ 中 R、SL、EOI 设置成 011,而 I₂~I₀ 三位指明了中断结束的对应位。

(3) 自动 EOI 结束方式

在自动 AEOI 方式中,任何一级中断被响应后,ISR 寄存器对应位置“1”,但在 CPU 进入中断响应周期,发第二个 INTA 脉冲后,8259A 自动将 ISR 寄存器中对应位清“0”。此时,尽管对某个外设正在进行中断服务,但对 8259A 来说,ISR 寄存器中没有指示,好像已结束了中断处理一样。这种方式虽然简单,但因为 ISR 寄存器中没有标志,低级中断申请时,可以打断高级中断,产生重复嵌套,嵌套深度也无法控制,容易产生错误,使用时要特别小心。通常在只有一片 8259A,多个中断不会嵌套情况下使用。

自动 AEOI 方式设置是在 8259A 初始化时,用初始化命令字 ICW₄ 中 AEOI=1 的方法实现的。

在级联方式下,一般用非自动结束方式,无论用普通 EOI 结束方式,还是用特殊 EOI 结束方式,中断处理结束时,要发两次中断结束命令,一次是对主片发的,另一次是对从片发的。

4. 循环优先级的循环方法

在循环优先级方式中,与中断结束方式有关,有三种循环方式。

(1) 普通 EOI 循环方式

在主程序或中断服务程序中设置操作命令字, 当任何一级中断被处理完后, 使 CPU 给 8259A 回送普通 EOI 循环命令, 8259A 收到 EOI 循环命令后, 将 ISR 寄存器中, 最高优先级的 IR_i 置“1”位清“0”, 并赋给它最低优先级, 将最高优先级赋给它的下一级 IR_{i+1} , 其它依次类推。表 7-3 是普通 EOI 循环的例子。

例 7-10 某中断系统 IR_0 为最高优先级, IR_7 为最低优先级。有 IR_2 、 IR_5 两个中断请求。设置为普通 EOI 循环方式, 要求给出 IR_2 及 IR_5 中断处理完后中断优先级的变化情况。

表 7-3 普通 EOI 循环方式

	ISR	ISR_7	ISR_6	ISR_5	ISR_4	ISR_3	ISR_2	ISR_1	ISR_0
原始状态	内容	0	0	1	0	0	1	0	0
	优先级	7	6	5	4	3	2	1	0
处理完 IR_2	ISR 内容	0	0	1	0	0	0	0	0
	优先级	4	3	2	1	0	7	6	5
处理完 IR_5	ISR 内容	0	0	0	0	0	0	0	0
	优先级	1	0	7	6	5	4	3	2

普通 EOI 循环方式命令字是设置 OCW_2 。在非自动结束方式中, OCW_2 中 R、SL、EOI 设置成 101, $L_2 \sim L_0$ 不起作用。

(2) 特殊 EOI 循环方式

特殊 EOI 循环方式即指定最低级循环方式, 最低优先级由编程确定, 最高优先级也相应而定, 例指定 IR_5 为最低优先级, 则 IR_6 就为最高优先级, 其它各级依次类推。这样在当前中断服务程序结束前, 使 CPU 给 8259A 回送特殊 EOI 结束命令, 8259A 收到此命令字后, 指定最低优先级, 并重新排列优先级级别。

例 7-11 某一时刻 8259A 中 IR_2 、 IR_6 有中断嵌套服务。在 IR_2 中断服务程序中安排了最低优先权赋给 IR_3 , 指令执行后, 中断优先级变化情况如表 7-4。

表 7-4 特殊 EOI 循环方式

	ISR	ISR_7	ISR_6	ISR_5	ISR_4	ISR_3	ISR_2	ISR_1	ISR_0
初始状态	内容	0	1	0	0	0	1	0	0
	优先级	7	6	5	4	3	2	1	0
执行置位优先权指令后	ISR 内容	0	1	0	0	0	1	0	0
	优先级	3	2	1	0	7	6	5	4

设定特殊 EOI 循环方式时, 设置 OCW_2 中 R、SL、EOI=111, $L_2 \sim L_0$ 指定了一个最低优先级。

(3) 自动 EOI 循环方式

在自动 EOI 循环方式中, 任何一级中断被响应后, 中断响应总线周期中第二个 $\overline{INTA}$ 信号的后沿自动将 ISR 寄存器中相应位清 0, 并立即改变各级中断的优先级, 改变方式与普通 EOI 循环循环方式相同。使用这种方式要小心, 防止重复嵌套产生。

自动 EOI 循环方式设置是 OCW_2 中 R、SL、EOI=100。

5. 中断源屏蔽方式

CPU 由 CLI 指令禁止所有可屏蔽中断进入, 中断优先级管理也可以对中断请求单独

屏蔽,通过对中断屏蔽寄存器的操作可以实现对某几位的屏蔽。有两种屏蔽方式:

(1) 普通屏蔽方式

将中断屏蔽寄存器 IMR 中某一位或某几位置“1”,即可将对应位的中断请求屏蔽掉。普通屏蔽方式的设置通过设置操作命令字 OCW₁ 来实现。

例 7-12 屏蔽第 2、3、5、6 位进入的中断请求,假设 8259A 的端口地址为 20H,21H。

```
MOV    AL, 01101100B
OUT    21H, AL
```

对 OCW₁ 的设置可以在主程序中,也可放在中断服务程序中,具体根据中断处理要求而定。

(2) 特殊屏蔽方式

某些场合,希望一个中断服务程序能动态地改变系统的优先级结构。例如当 CPU 正在处理中断程序的某一部分时,希望禁止低级中断请求,但在执行中断处理程序的另一部分时,希望开放较低级中断请求,此时可采用特殊屏蔽方式。此方式能对本级中断进行屏蔽,而允许优先级比它高或低的中断进入,特殊屏蔽方式总是在中断处理程序中使用,特殊屏蔽方式的设置是通过设置操作命令字 OCW₃ 中的 ESMM, SMM=11 来实现的,例如当前正在执行 IR₃ 的中断服务程序,设置了特殊屏蔽方式后,再用 OCW₁ 对中断屏蔽寄存器中第 3 位置“1”时,就会同时使当前中断服务寄存器中对应位自动清“0”,这样可以既屏蔽了当前正在处理的中断,又开放了较低级别的中断。待中断服务程序结束时,应将 IMR 寄存器的第 3 位复位,并将 SMM 位复位,标志退出特殊屏蔽方式。

6. 中断请求引入方式

中断请求引入有三种方式:

(1) 边沿触发方式

在边沿触发方式下,8259A 将中断请求输入端出现的上升沿作为中断请求信号。中断请求输入端出现上升沿触发信号后,可以一直保持高电平。

(2) 电平触发方式

在电平触发方式下,8259A 将中断请求输入端出现的高电平作为中断请求信号。但当中断得到响应后,中断输入端必须及时撤出高电平,否则在 CPU 进入中断处理过程,并且开中断的情况下,原输入端的高电平会引起第二次中断的错误。

初始化命令字 ICW₁ 中的 LTIM 位可用来设置这两种触发方式,LTIM=1,设置为电平触发方式,LTIM=0 设置为边沿触发方式。

(3) 中断查询方式

在中断查询方式下,外部设备向 8259A 发中断请求信号,中断请求可以是边沿触发,也可以是电平触发。但 8259A 不通过 INT 信号向 CPU 发中断请求信号,因为 CPU 内部的中断允许触发器复位,所以禁止了 8259A 对 CPU 的中断请求。CPU 要使用软件查询来确定中断源,才能实现对外设的中断服务。因此,中断查询方式既有中断的特点,又有查询的特点。

CPU 执行的查询软件中必须有查询命令,才能实现查询功能。CPU 通过操作命令字 OCW₃ 的设置来发出查询命令的。若外设发出中断请求,8259A 的中断服务寄存器相应位

置“1”,CPU 就可以在查询命令之后的下一个读操作,读取中断服务寄存器中的优先级。所以,CPU 所执行的查询程序应包括如下过程:

- ① 系统关中断。
- ② 用 OUT 指令使 CPU 向 8259A 端口(偶地址端口)送 OCW₃ 命令字。
- ③ 若外设已发出过中断请求,8259A 在当前中断服务寄存器中使对应位置“1”,且立即组成查询字。

④ CPU 用 IN 指令从端口(偶地址)读取 8259A 的查询字。

OCW₃ 命令字构成的查询命令格式为:

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
×	0	0	0	1	1	0	0

其中 D₂ 位为 1,使 OCW₃ 具有查询性质。8259A 得到查询命令后,立即组成查询字,等待 CPU 读取。CPU 从 8259A 中读取的查询字格式为:

D ₇	D ₆	D ₅	D ₄	D ₃	D ₂	D ₁	D ₀
IR					W ₂	W ₁	W ₀

其中 IR=1,表示有设备请求中断服务,IR=0,表示没有设备请求中断服务。

W₂、W₁、W₀ 组成的代码表示当前中断请求的最高优先级。

中断查询方式一般用在多于 64 级中断的场合,或者用在一个中断服务程序中几个模块分别为几个中断设备服务的情况下。一般除了 8259A 以外,还要有一些附加电路来帮助完成查询功能。

四、8259A 的编程方法

对 8259A 的编程有两类命令字:初始化命令字 ICW 和操作命令字 OCW。系统复位后,初始化程序对 8259A 置入初始化命令字。初始化后可通过发出操作命令字 OCW 来定义 8259A 的操作方式,实现对 8259A 的状态、中断方式和优先级管理的控制。初始化命令字只发一次,操作命令字允许重置,以动态改变 8259A 的操作与控制方式。

1. 初始化命令字

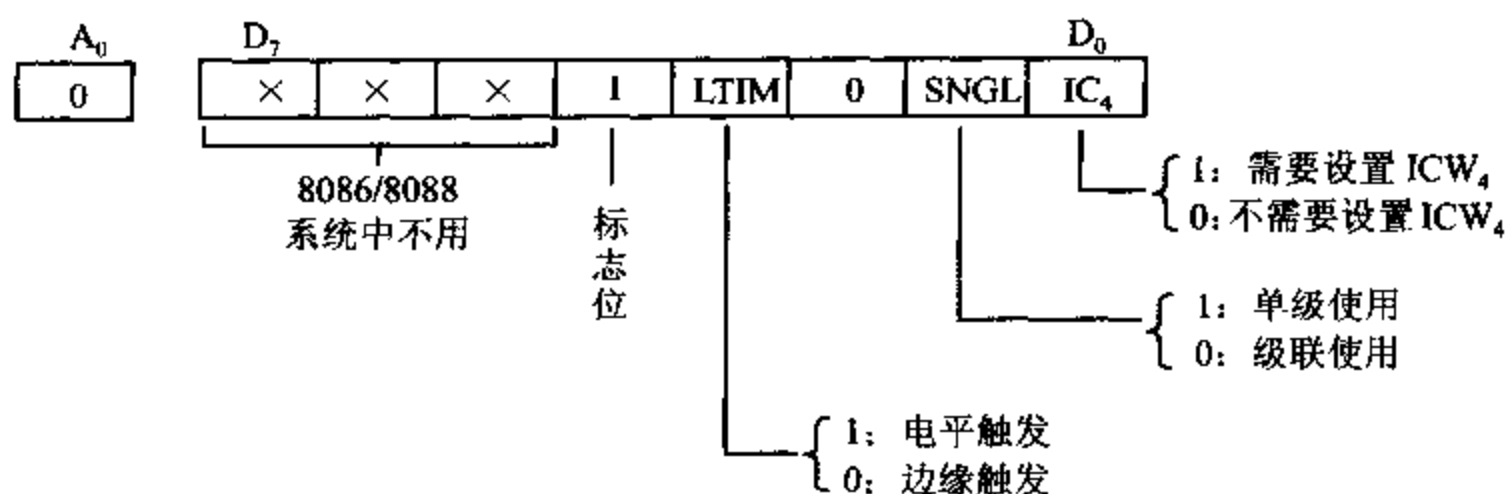
初始化命令字完成的功能:

- (1) 设定中断请求信号触发形式,高电平触发或上升沿触发。
- (2) 设定 8259A 工作方式,单片或级联。
- (3) 设定 8259A 中断类型号基值,即 IR₀ 对应的中断类型号。
- (4) 设定优先级设置方式。
- (5) 设定中断处理结束时的结束操作方式。

对 8259A 编程初始化命令字,共预置 4 个命令字:ICW₁、ICW₂、ICW₃、ICW₄。初始化命令字必须顺序填写,但并不是任何情况下都要预置 4 个命令字,用户根据具体情况而定。8259A 有两个端口地址,一个为偶地址,一个为奇地址。

(1) ICW₁ —— 芯片控制初始化命令字。

格式:



A_0 : 写入命令字的端口地址, $A_0 = 0$, ICW_1 必须写入 8259A 的偶地址端口中, 对 IBM PC/XT 机而言, 地址为 20H。

标志位: ICW_1 的位 4 等于 1, 是标志位, 以区别 OCW_2 和 OCW_3 控制字的设置。

IC_4 : 说明是否要设置 ICW_4 命令, 在 8086/8088 系统中设为 1, 表示要求设置命令字 ICW_4 。

SNGL: 说明级联使用情况。SNGL=1, 表示使用单片 8259A, SNGL=0, 表示使用多片 8259A 级联。

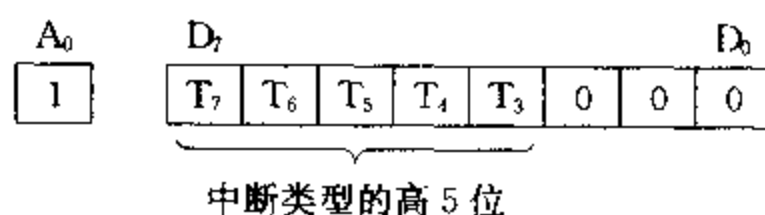
LTIM: 定义中断请求信号触发方式, LTIM=1, 表示用高电平触发方式, LTIM=0, 表示用上升沿触发方式。

例 7-13 IBM PC/XT 系统初始化中, 设 $ICW_1 = 13H$, 表示系统中 8259A 为单片方式, 上升沿触发, 要求设置 ICW_4 。指令为:

```
MOV    AL, 13H
OUT    20H, AL
```

(2) ICW_2 —— 设置中断类型号初始化命令字。

格式:



A_0 : $A_0 = 1$, ICW_2 必须写到 8259A 的奇地址端口中。

8259A 中 IR_0 端对应的中断类型号为中断类型号基值, 它是可以被 8 整除的正整数, ICW_2 用来设置这个中断类型号基值, 由此提供外部中断的中断类型号。

ICW_2 低 3 位为 0, 高 5 位由用户设定。当 8259A 收到 CPU 发来的第二个 $\overline{INTA}$ 信号, 它向 CPU 发送中断类型号, 其中高 5 位为 ICW_2 的高 5 位, 低 3 位根据 $IR_0 \sim IR_7$ 中响应哪级中断(对应 000~111)来确定。

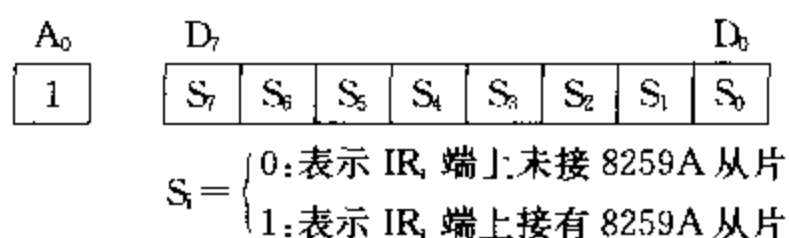
例 7-14 在 IBM PC/XT 系统中, $T_7 \sim T_3 = 00001$, 所以对应 8 个中断的类型号为 08H~0FH。 $A_0 = 1$, I/O 端口地址为 21H。设置 ICW_2 的指令为:

```
MOV    AL, 8
OUT    21H, AL
```

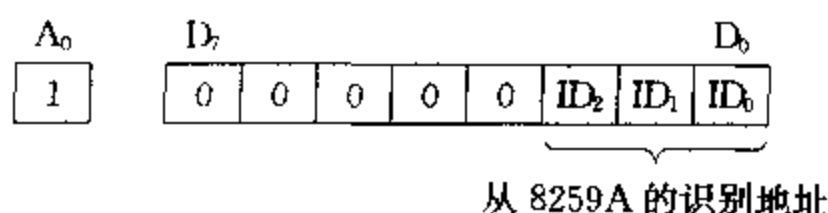
(3) ICW₃ ——标识主片/从片初始化命令字。

ICW₃ 命令字在级联时(即 ICW₁ 中 SNGL=0)才设置。

8259A 主片格式:



8259A 从片格式:



A₀: A₀=1, ICW₃ 必须写到 8259A 的奇地址端口。

对于 8259A 主片,某位为 1,表示对应 IR_i 端上接有 8259A 从片。某位为 0,表示对应位未连 8259A 从片。

对于 8259A 从片, ID₂~ID₀=000~111 表示从片接在主片的哪个中断请求输入端上,例 ID₂~ID₀=010,表示从片接在主片 8259A 的 IR₂ 端。

在多片 8259A 级联情况下,主片与从片的 CAS₂~CAS₀ 相连,主片的 CAS₂~CAS₀ 为输出,从片的 CAS₂~CAS₀ 作为输入。当 CPU 发第一个中断响应信号 $\overline{\text{INTA}}$ 时,主片通过 CAS₂~CAS₀ 发一个编码 ID₂~ID₀,从片的 CAS₂~CAS₀ 收到主片发来的编码与本身 ICW₃ 中 ID₂~ID₀ 相比较,如果相等,则在第二个 $\overline{\text{INTA}}$ 信号到来后,将自己的中断类型号送到数据总线上。

例 7-15 某 8259A 主片的 IR₃, IR₇ 端连接两个 8259A 从片,编写初始化命令字。

主片:

```
MOV    AL, 88H
OUT     21H, AL           ;主片端口地址 20H,21H
```

从片 1:

```
MOV     AL, 03H
OUT     A1H, AL           ;从片 1 端口地址 A0H,A1H
```

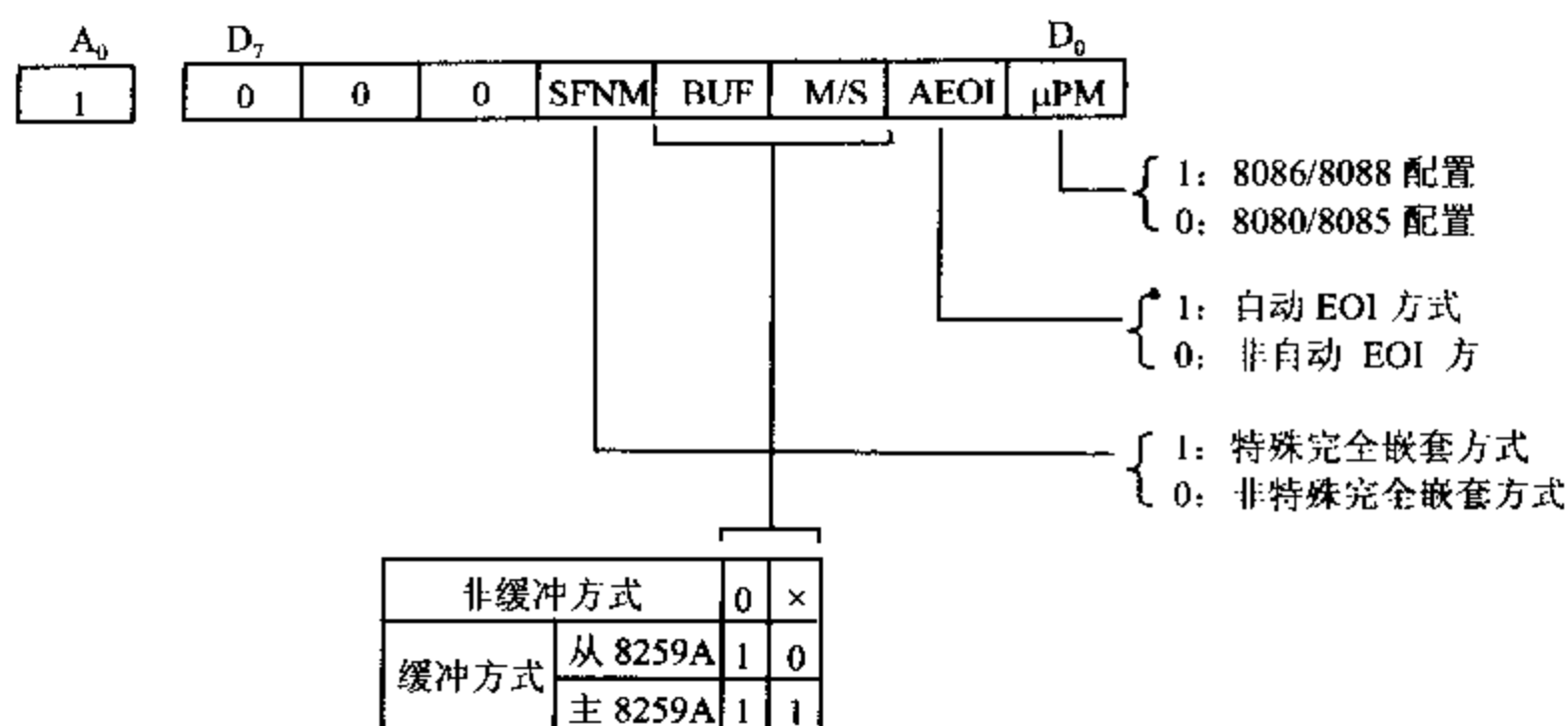
从片 2:

```
MOV     AL, 07H
OUT     B1H, AL           ;从片 2 端口地址 B0H,B1H
```

(4) ICW₄ ——方式控制初始化命令字。

ICW₁ 中 IC₄ 为 1 时,要求预置 ICW₄ 命令字,对 8086/8088 系统必须预置 ICW₄。

格式:



A_0 : $A_0=1$, ICW_1 必须写入 8259A 的奇地址端口。

μPM : $\mu PM=1$, 表示 8259A 与 8086/8088 系统配合工作, $\mu PM=0$, 表示 8259A 与 8080/8085 系统配合工作。

AEOI: 规定中断结束方式。

AEOI=1, 中断自动结束方式, CPU 响应中断请求过程中, 向 8259A 发第二个 $\overline{INTA}$ 脉冲时, 清除中断服务寄存器中本级对应位, 这样在中断服务子程序结束返回时, 不需要其它任何操作, 称为中断自动结束, 一般不常采用。

AEOI=0, 为非自动结束方式, 必须在中断服务子程序中安排输出指令, 向 8259A 发操作命令字 OCW_3 , 清除相应中断服务标志位, 才标志中断结束。在 IBM PC/XT 系统中该位为 0。

M/S BUF: 表示 8259A 是否采用缓冲方式。

BUF=1, 采用缓冲方式, 8259A 通过总线驱动器与数据总线相连, $\overline{SP}/\overline{EN}$ 作输出端, 控制数据总线驱动器启动, 此时 $\overline{SP}/\overline{EN}$ 线中 $\overline{EN}$ 有效, $\overline{EN}=0$ 允许缓冲器输出, $\overline{EN}=1$ 允许缓冲器输入。此时, M/S=1, 表示该片是 8259A 主片, M/S=0, 表示该片是 8259A 从片。

BUF=0, 采用非缓冲方式, $\overline{SP}/\overline{EN}$ 线中 $\overline{SP}$ 有效, $\overline{SP}=0$, 该片是 8259A 从片, $\overline{SP}=1$, 该片是 8259A 主片, 此时, M/S 信号无效。

IBM PC/XT 系统中 BUF 设定为 1, $\overline{SP}/\overline{EN}$ 端加 +5V, M/S 设定为 0。

BUF, M/S 与 $\overline{SP}/\overline{EN}$ 之间关系可用表 7-5 来表示。

表 7-5 BUF、M/S、 $\overline{SP}/\overline{EN}$ 之间关系

BUF 位	M/S 位	$\overline{SP}/\overline{EN}$ 端		
		$\overline{SP}$ 有效 (输入信号)	$\overline{SP}=1$ $\overline{SP}=0$	主 8259A 从 8259A
1 缓冲方式	1 主 8259A	$\overline{EN}$ 有效 (输出信号)	$\overline{EN}=1$	CPU→8259A
	0 从 8259A		$\overline{EN}=0$	8259A→CPU

SFNM: 定义级联方式下的嵌套方式。

SFNM=1,工作在特殊全嵌套方式,SFNM=0,工作在完全嵌套方式,IBM PC/XT 系统设置为 0。

例 7-16 设置 IBM PC/XT 机的 ICW₄ 命令字:

```
MOV    AL, 09H
OUT    21H, AL
```

初始化命令字的设置有固定次序,端口地址也有明确规定,并不会因为只有两个端口输出命令而混淆,设置次序如图 7-13 所示。初始化命令字必须从 ICW₁ 开始设置,依下顺序进行设置,并分别根据 ICW₁ 中的 SNGL 位和 IC₄ 位决定是否设置 ICW₃ 和 ICW₄。级联时要设置 ICW₃,并且主片与从片的 ICW₃ 设置不同。

在初始化命令字设置完成前,对 A₀=1 的输出指令不可能是操作命令字。而初始化命令序列设置完成后,在操作命令字的设置中,不可能第二次初始化,因此不会混淆。

例 7-17 在 IBM PC/XT 系统中,ROM BIOS 中的 8259A 初始化程序为:

```
MOV    AL, 13H           ;单片,上升沿触发
OUT    20H, AL
MOV    AL, 08H           ;中断类型号基值为 08H
OUT    21H, AL
MOV    AL, 09H           ;缓冲、非自动结束
OUT    21H, AL
```

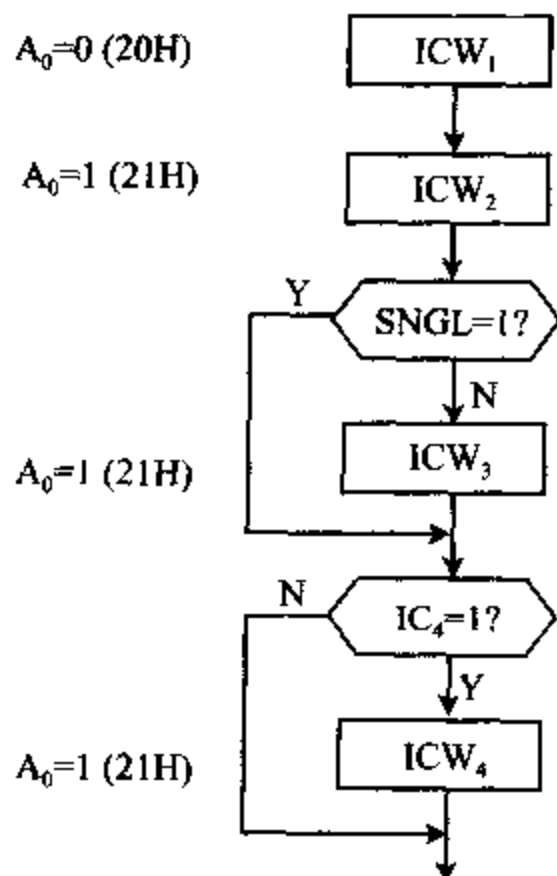


图 7-13 初始化命令字设置次序

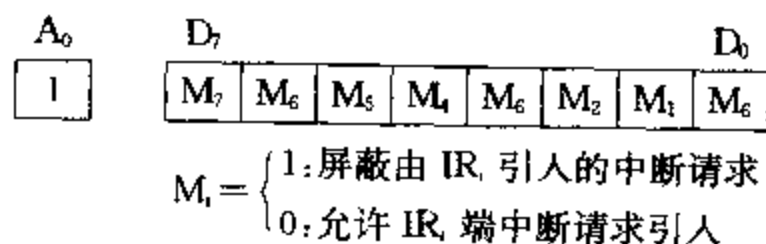
8259A 经初始化命令字 ICW 预置后已进入初始化状态,可接收来自 IR_i 端的中断请求,自动进入操作命令状态,接受 CPU 写入 8259A 的操作命令字 OCW。

2. 操作命令字

操作命令字决定中断屏蔽,中断优先级次序,中断结束方式等。中断管理较复杂,包括有:完全嵌套优先方式、特殊嵌套优先方式、自动循环优先方式、特殊循环优先方式、特殊屏蔽方式、查询方式等。它是由操作命令字的设置来实现的,设置时,次序上没有严格要求,但端口地址有严格规定,OCW₁ 必须写入奇地址端口,OCW₂ 和 OCW₃ 必须写入偶地址端口。

(1) OCW₁ ——中断屏蔽操作命令字

格式:



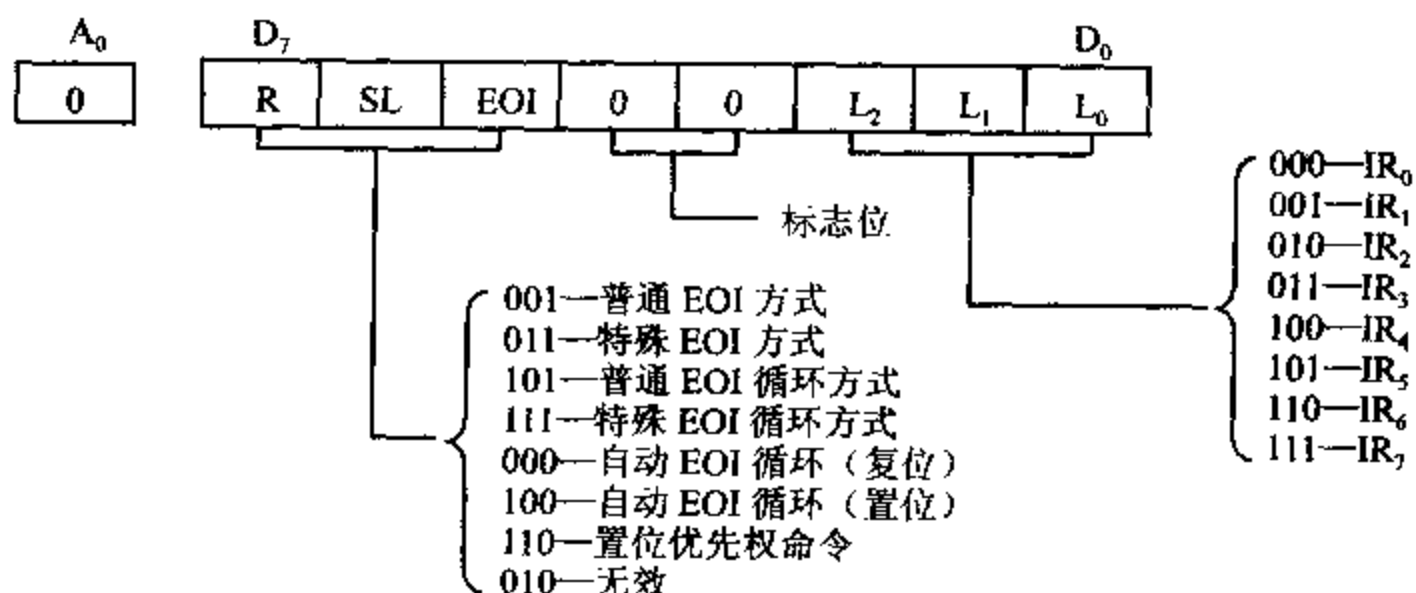
$A_0: A_0=1$, OCW_1 命令字必须写入 8259A 奇地址端口。

OCW_1 命令字的各位直接对应中断屏蔽寄存器 IMR 的各位, 当 OCW_1 中某位 M_i 为 1, 对应位的中断请求受到屏蔽, 某位为 0, 对应位的中断请求得到允许。

例 7-18 设某中断系统要求屏蔽 IR_3, IR_5 , 8259A 编程指令为:

```
MOV    AL, 00101000B
OUT    21H, AL
```

(2) OCW_2 —— 优先权循环方式和中断结束方式操作字
格式:



$A_0: A_0=0$, OCW_2 命令字必须写入 8259A 偶地址端口

标志位: OCW_2 的 D_4, D_3 位等于 00, 是标志位。以区别 ICW_1 和 OCW_3 控制字的设置。

$L_2 \sim L_0$: $SL=1$ 时, $L_2 \sim L_0$ 有效。 $L_2 \sim L_0$ 有两个用途, 一个是当 OCW_2 设置为特殊 EOI 结束命令时, $L_2 \sim L_0$ 指出清除中断服务寄存器中的哪 1 位, 第二个当 OCW_2 设置为特殊优先级循环方式时, $L_2 \sim L_0$ 指出循环开始时设置的最低优先级。

R, SL, EOI : 这三位组合起来才能指明优先级设置方式和中断结束控制方式, 但每位也有自己的意义。

$R(ROTATE)$: $R=1$, 中断优先级是按循环方式设置的, 即每个中断级轮流成为最高优先级。当前最高优先级服务后就变成最低级, 它相邻的下一级变成最高级, 其它依次类推。 $R=0$, 设置为固定优先级, 0 级最高, 7 级最低。

$SL(SPECIFIC LEVEL)$: 指明 $L_2 \sim L_0$ 是否有效。 $SL=1$, OCW_2 中 $L_2 \sim L_0$ 有效, $SL=0$, $L_2 \sim L_2$ 无意义。

$EOI(END OF INTERRUPT)$: 指定中断结束。

$EOI=1$, 用作中断结束命令, 使中断服务寄存器中对应位复 0, 在非自动结束方式使用。 $EOI=0$, 不执行结束操作命令。如果初始化时, ICW_1 的 $AEIOI=1$, 设置为自动结束方式, 此时 OCW_2 中的 EOI 位应为 0。

OCW_2 的功能包括两个方面, 一个是决定 8259A 是否采用优先级循环方式, 另一个是中断结束采用普通的还是特殊的 EOI 结束方式。表 7-6 给出了 R, SL, EOI 三位的组合功能。

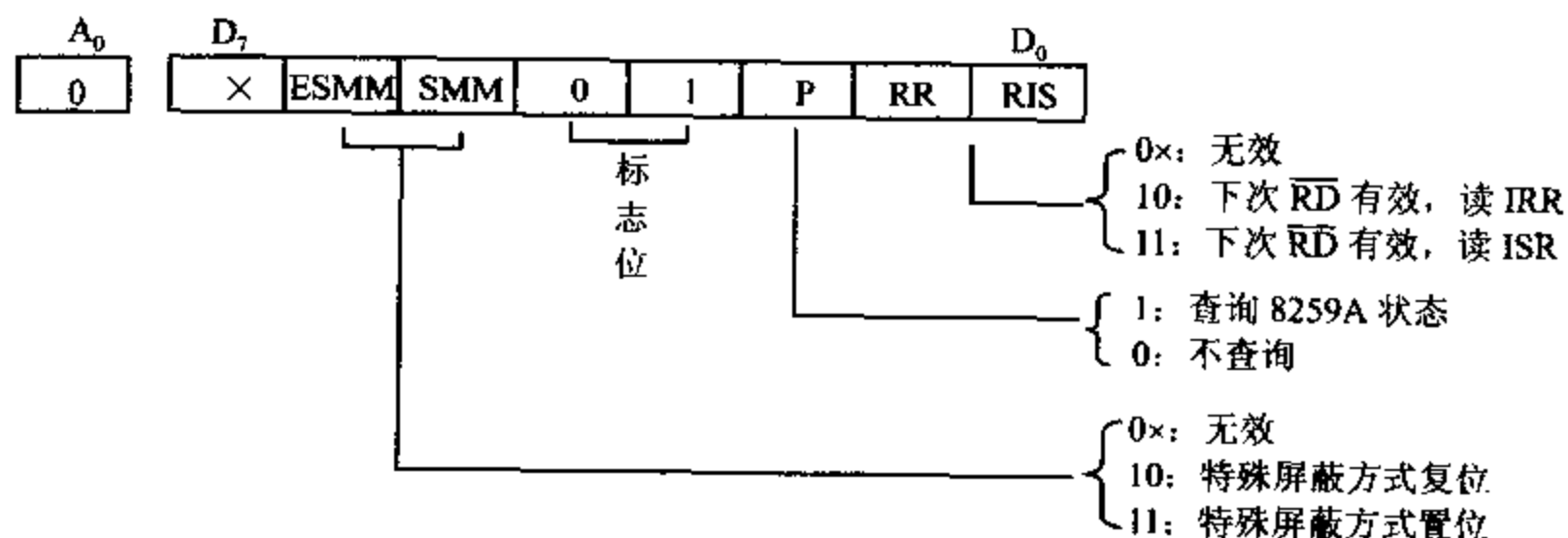
表 7-6 R、SL、EOI 组合功能

R	L	EOI	
1	0	0	设置自动 EOI 循环方式 在中断响应周期的第二个 $\overline{\text{INTA}}$ 信号结束时, 将 ISR 寄存器中正在服务的相应位置 0, 本级赋予最低优先级, 最高优先级赋给它的下一级, 其它中断优先级依次循环赋给。
0	0	0	取消自动 EOI 循环
1	0	1	设置普通 EOI 循环 一旦中断结束, 8259A 将中断服务寄存器 ISR 中, 当前级别最高的置 1 位清 0, 此级赋予最低优先级, 最高优先级赋给它的下一级, 其它中断优先级依次循环赋给。
1	1	1	设置特殊 EOI 循环 一旦中断结束, 将中断服务寄存器 ISR 中, 由 $L_2 \sim L_0$ 字段给定级别的相应位清 0, 此级赋予最低优先级, 最高优先级赋给它的下一级, 其它中断优先级依次循环赋给。
1	1	0	置位优先级循环 8259A 按 $L_2 \sim L_0$ 字段确定一个最低优先级, 最高优先级赋给它的下一级, 其它中断优先级依次循环赋给, 系统工作在优先级特殊循环方式。
0	0	1	普通 EOI 结束方式 一旦中断处理结束, CPU 向 8259A 发出 EOI 结束命令, 将中断服务寄存器 ISR 中当前级别最高的置 1 位清 0。一般用在完全嵌套(包括特殊全嵌套)工作方式。
0	1	1	特殊 EOI 结束方式 一旦中断处理结束, CPU 向 8259A 发出 EOI 结束命令, 8259A 将中断服务寄存器 ISR 中, 由 $L_2 \sim L_0$ 字段指定的中断级别的相应位清 0。
0	1	0	OCW ₂ 没有意义

(3) OCW₃ ——特殊屏蔽方式和查询方式操作字

OCW₃ 功能有三个: 设定特殊屏蔽方式, 设置对 8259A 寄存器的读出及设置中断查询工作方式。

格式:



$A_0, A_0=0$, OCW₃ 必须写入 8259A 偶地址端口。

标志位: D_4 及 D_3 位组合为 01 时, 表示为 OCW₃, 以区别 OCW₂ (OCW₂ 中此 2 位组合为 00)。而 $D_4=1$ 时, 此操作字为 ICW₁。

RR, RIS: RR 为读寄存器状态命令, RR=1, 允许读寄存器状态, RIS 为指定读取对象。

RR, RIS=10, 用输入指令(IN 指令), 在下一个 $\overline{RD}$ 脉冲到来后, 将中断请求寄存器 IRR 的内容读到数据总线上。

RR, RIS=11, 用输入指令, 在下一个 $\overline{RD}$ 脉冲到来后, 将中断服务寄存器 ISR 的内容读到数据总线上。顺便指出, 8259A 中断屏蔽寄存器 IMR 的值, 随时可通过输入指令从奇地址端口读取。读同一寄存器的命令只需要发送一次, 不必每次重写 OCW₃。

P: 查询方式位

P=1, 设置 8259A 为中断查询工作方式。在查询工作方式下, CPU 不是靠接收中断请求信号来进入中断处理过程, 而是靠发送查询命令, 读取查询字来获得外部设备的中断请求信息。CPU 先送操作命令 OCW₃ (P=1) 给 8259A, 再送一条输入指令将一个 $\overline{RD}$ 信号送给 8259A, 8259A 收到后将中断服务寄存器的相应位置 1, 并将查询字送到数据总线, 查询字反映了当前外设有无中断请求及中断请求的最高优先级是哪个, 查询字为:

A ₀	D ₇	D ₆			D ₀		
0	IR	×	×	×	×	W ₂	W ₁ W ₀

例 7-19 P=1 时, 优先级次序为 IR₃、IR₄、…、IR₂。当前在 IR₄ 和 IR₁ 引脚上有中断请求。CPU 再执行一条输入指令, 得到查询字为 1××××100。说明当前级别最高的中断请求为 IR₄, CPU 转入 IR₄ 中断处理程序。

中断查询方法的实际使用场合往往在多于 64 级中断的 8259A 级联系统中。

ESMM, SMM: 置位和复位特殊屏蔽方式。

ESMM, SMM=11, 设置 8259A 采用特殊屏蔽方式, 只屏蔽本级中断请求, 允许高级中断或低级中断进入。

ESMM, SMM=10, 取消特殊屏蔽方式, 恢复原来优先级的控制。

ESMM=0, 设置无效。此时 SMM 位不起作用, ESMM 可称为 SMM 位的允许位。

操作控制字 OCW₁~OCW₃ 的设置, 安排在初始化命令字之后, 用户根据需要可在程序的任何位置去设置。尽管 8259A 只有两个端口地址, 但不会混淆命令字及控制字的, 因为 ICW₂、ICW₃、ICW₄ 和 OCW₁ 写入 8259A 奇地址端口。初始化时 ICW₁ 后面紧跟 ICW₂、ICW₃、ICW₄, 而 OCW₁ 是单独写入的, 不会紧跟在 ICW₁ 后面。ICW₁、OCW₂、OCW₃ 写入 8259A 偶地址端口, 但一方面 ICW₁ 在初始化时写入, 另一方面可用 D₄ 位区分, D₄=1 为 ICW₁, D₄=0 为 OCW; 再用 D₃ 位区分, D₃=0 为 OCW₂, D₃=1 为 OCW₃。所以对 8259A 编程时写入的字是不会混淆的。

五、8259A 的中断级联

一片 8259A 管理 8 级中断, 当申请中断的外设多于 8 级时, 可以将 8259A 级联使用, 图 7-14 给出了两级级联的例子, 第一级为 8259A 主片, 第二级为 8259A 从片, 主片可接 1~8 片 8259A 从片, 这样最多可管理 64 级中断源。

级联使用时, 主片 8259A 的 $\overline{SP}/\overline{EN}$ 端接 V_{cc}, 从片的 $\overline{SP}/\overline{EN}$ 端接地。若系统中连接数据总线驱动器, 主片的 $\overline{SP}/\overline{EN}$ 端与数据总线驱动器的输出允许端 $\overline{OE}$ 相连。从片的 INT 脚接主片的 IR_i 端, 主片的 IR_i 端若未接从片, 可直接连中断源。主片的 CAS₂~CAS₀ 作为输

出端,从片的 $CAS_2 \sim CAS_0$ 作输入端,二者相连。

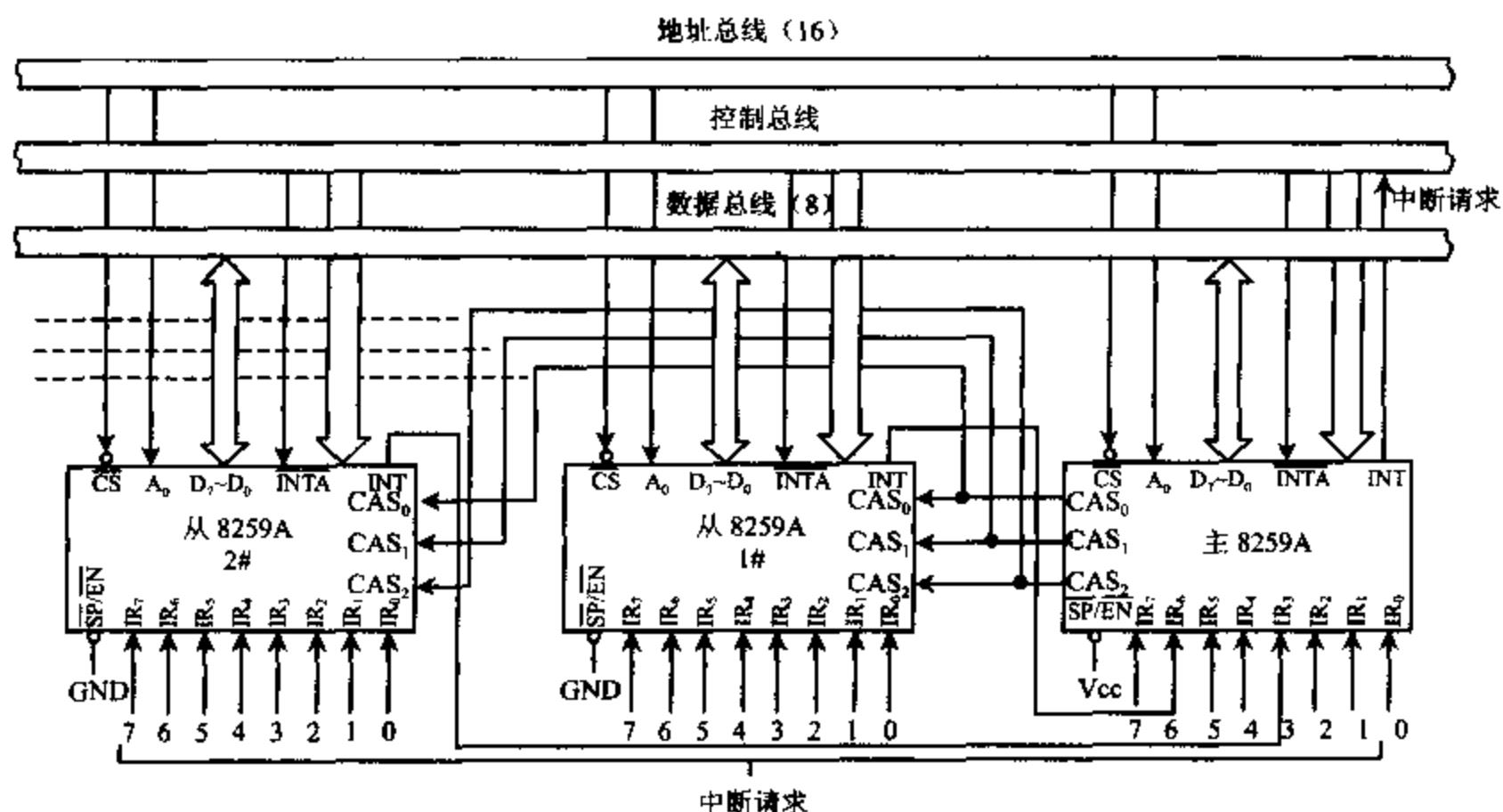


图 7-14 8259A 级联方式连接图

在级联系统中,主片和从片都要设置初始化命令字进行初始化,设置主片初始化命令字与无级联单片 8259A 初始化时不同处有几点:

- (1) 级联时, ICW_1 中 $SNGL=0$, 单片时 $SNGL=1$ 。
- (2) 级联时,要求设置 ICW_3 , 若某个 IR_i 引脚上连有从片,主片 ICW_3 的对应位设为 1, 未连从片的对应位设为 0。单片不要设置 ICW_3 。
- (3) 级联时,可设置为特殊全嵌套工作方式,此时, ICW_4 中 $SFNM=1$, 通常应定义在特殊完全嵌套工作方式。

设置从片初始化命令字时,要注意以下两点:

- (1) 从片的 ICW_1 中, $SNGL=0$ 。
- (2) 从片必须设置 ICW_3 , 由 ICW_3 中三个最低有效位 $ID_2 \sim ID_0$ 的组合来标记此从片连到主片哪个 IR_i 引脚上。

在完全嵌套工作方式下,8259A 在级联使用时,某从片 8259A 的 IR_i 端收到一个或多个中断请求信号,经从片优先权判别器判优后,确定本片当前最高优先级。从片发出一个中断请求信号 INT 到主片,再经过主片优先权判别器判优后,确定当前最高优先级。通过主片 INT 输出端发中断请求送到 CPU,若 $IF=1$,CPU 响应中断回送两个 $\overline{INTA}$ 信号。

主片收到第一个 $\overline{INTA}$ 信号,置主片中断服务寄存器 ISR 相应为“1”,清中断请求寄存器 IRR 相应位为“0”。检测 ICW_3 决定中断请求是否来自从片,若是,则将从片的级联地址从 $CAS_2 \sim CAS_0$ 三条线上输出到所有 8259A 从片,只有级联地址与 $CAS_2 \sim CAS_0$ 相同的从片才能选通。从片收到第一个 $\overline{INTA}$ 信号后,将从片中 ISR 寄存器中相应位置“1”,将 IRR 寄存器中相应位清“0”。

第二个 $\overline{INTA}$ 信号到达后,选中的从片将中断类型号送到数据总线,以后操作与单片 8259A 工作情况相同。

级联时中断优先级判别是由从片判别本片内最高优先级后,向 8259A 主片再申请中断,然后由主片判别当前最高优先级,向 CPU 发中断请求信号 INT。注意当 8259A 主片设置为特殊全嵌套工作方式时,允许相同级别的中断请求通过。

8086/8088 中使用一片 8259A 芯片支持 8 个外部中断,在 80286/80386 中使用主从两片 8259A 芯片级联,主片 8259A 的 IR_2 输入作为级联中断输入,用于传送从片 8259A 的中断请求输入 INT,这样,可管理 15 个外部中断。图 7-15 标出了每个中断请求输入的功能及其分配的中断类型号,其中,主片 8259A 地址为:020H~021H,从片 8259A 地址为:0A0H~0A1H。

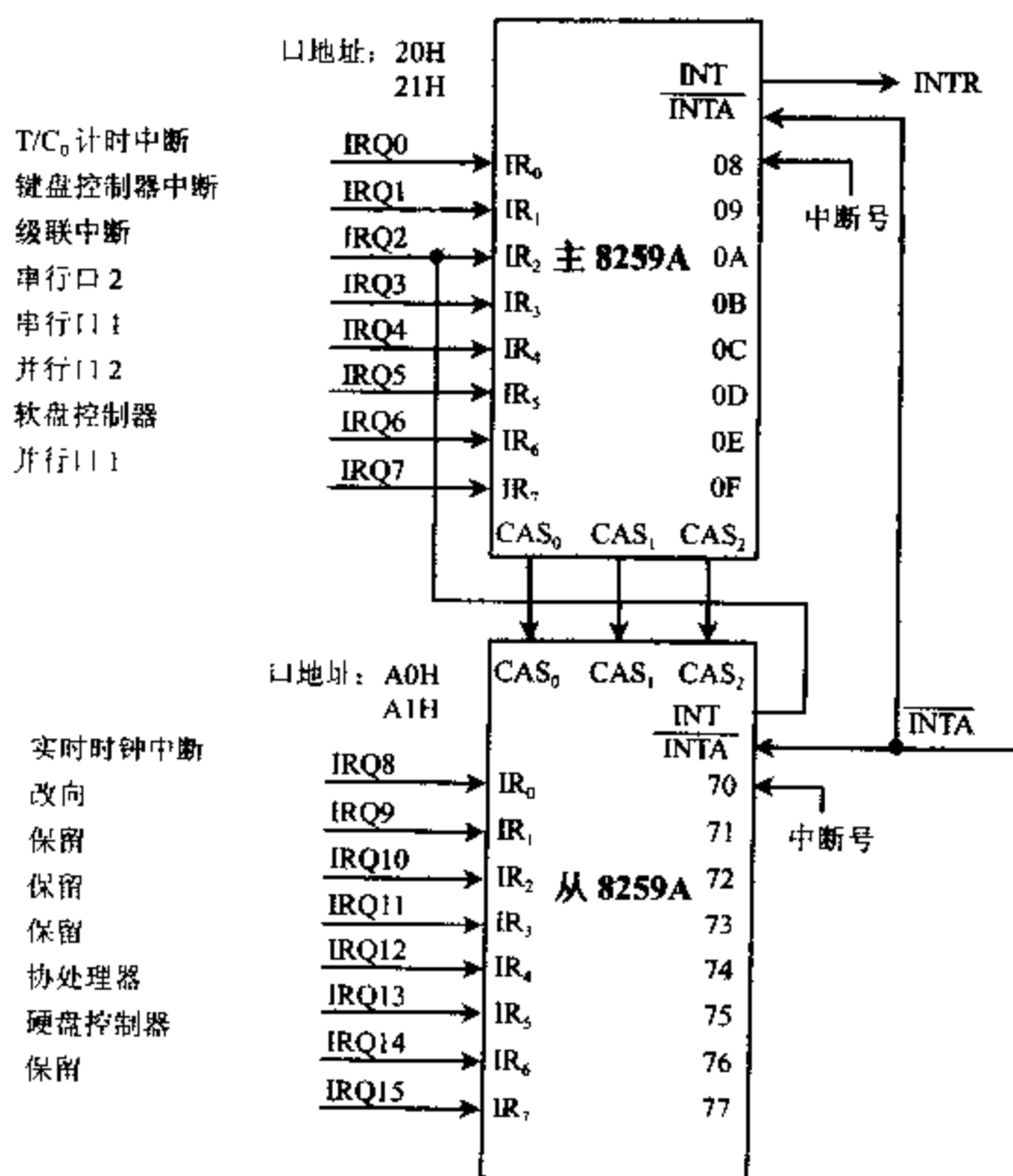


图 7-15 PC/AT 机中的 8259A 中断级联

例 7-20 一个 8259A 主片,连接 2 片 8259A 从片,从片分别经主片的 IR_3 及 IR_6 引脚接入,则系统中优先级排列次序为:

主片 IR_0 、 IR_1 、 IR_2

从片 IR_0 、 IR_1 、 $\dots$ 、 IR_7

主片 IR_4 、 IR_5

从片 IR_0 、 IR_1 、 $\dots$ 、 IR_7

主片 IR_7 。

中断处理结束时,CPU 应发出两个 EOI 结束命令,一个送给主片 8259A,另一个送给从片 8259A,使 8259A 主片和从片的 ISR 寄存器相应位清 0,这样一次中断处理过程结束。

例 7-21 特殊全嵌套工作方式:某系统 8086CPU 工作在最小模式,中断系统有 1 片 8259A 主片,2 片 8259A 从片构成级联工作方式。从片 1# 从 8259A 主片 IR_6 端引入,从片 2# 从 8259A 主片 IR_5 端引入,8259A 主片的 IR_0 及 IR_1 引入 2 个中断请求信号。如图 7-14 所示。

初始化时,设置为特殊全嵌套工作方式,当 8259A 从片 1# 由 IR_6 端收到一个中断请求,此时从片 1# 中断请求寄存器 IRR 置成状态为:

IRR	D_7						D_0	(8259A 从片 1#)
	0	1	0	0	0	0	0	

经过优先级判定,当前 8259A 从片 1# 为最高优先级,将从片中断服务寄存器 ISR 置成状态为:

ISR	D_7						D_0	(8259A 从片 1#)
	0	1	0	0	0	0	0	

且从片 IRR 寄存器清“0”,通过本片 INT 向 8259A 主片的 IR_6 发中断请求,将 8259A 主片的 IRR 寄存器置成:

IRR	D_7						D_0	(8259A 主片)
	0	1	0	0	0	0	0	

由 8259A 主片判定为当前最高优先级后,ISR 寄存器状态为:

ISR	D_7						D_0	(8259A 主片)
	0	1	0	0	0	0	0	

将主片 IRR 寄存器清“0”,并通过 8259A 主片的 INT 向 CPU 发中断请求信号,当 CPU 采样到 INTR 有效,并且响应中断请求,发两个 $\overline{INTA}$ 信号,获得中断类型号,转到执行相应的中断服务程序。

在 CPU 中断服务过程中,8259A 从片 1# 收到 IR_1 来的中断请求,同上面过程,从片 1# 的 IRR 寄存器状态为:

IRR	D_7						D_0	(8259A 从片 1#)
	0	0	0	0	0	0	1	0

因 IR_1 比 IR_6 优先级高,从片 1# 的 ISR 寄存器的状态为:

ISR	D_7						D_0	(8259A 从片 1#)
	0	1	0	0	0	0	1	0

此时从片 1# 的 IRR 寄存器清“0”,并由 INT 向 8259A 主片的 IR_6 端再发中断请求,由于采用特殊完全嵌套方式,8259A 主片允许同级中断参加优先级判别,确定为当前最高优先级,主片 ISR 寄存器状态不变。

ISR	D_7						D_0	(8259A 主片)
	0	1	0	0	0	0	0	

并再次向 CPU 发中断请求,CPU 暂停执行原来的中断服务程序转去执行更高级的中

断服务程序,实现中断级联方式的特殊完全嵌套过程。

3. 8259A 级联使用例子

例 7-22 某系统中两片 8259A 采用中断级联方式组成中断系统,从片的 INT 端连 8259A 主片的 IR_3 端。若当前 8259A 主片从 IR_1 、 IR_5 端引入两个中断请求,中断类型为 31H、35H。中断服务程序的段基址为 1000H,偏移地址分别为 2000H 及 3000H。8259A 从片由 IR_4 、 IR_5 端引入两个中断请求,中断类型为 44H 和 45H,中断服务程序段基址为 2000H,偏移地址为 3600H 及 4500H,图 7-16 给出了级联连接图,图 7-17 为此例中断入口地址表内容。

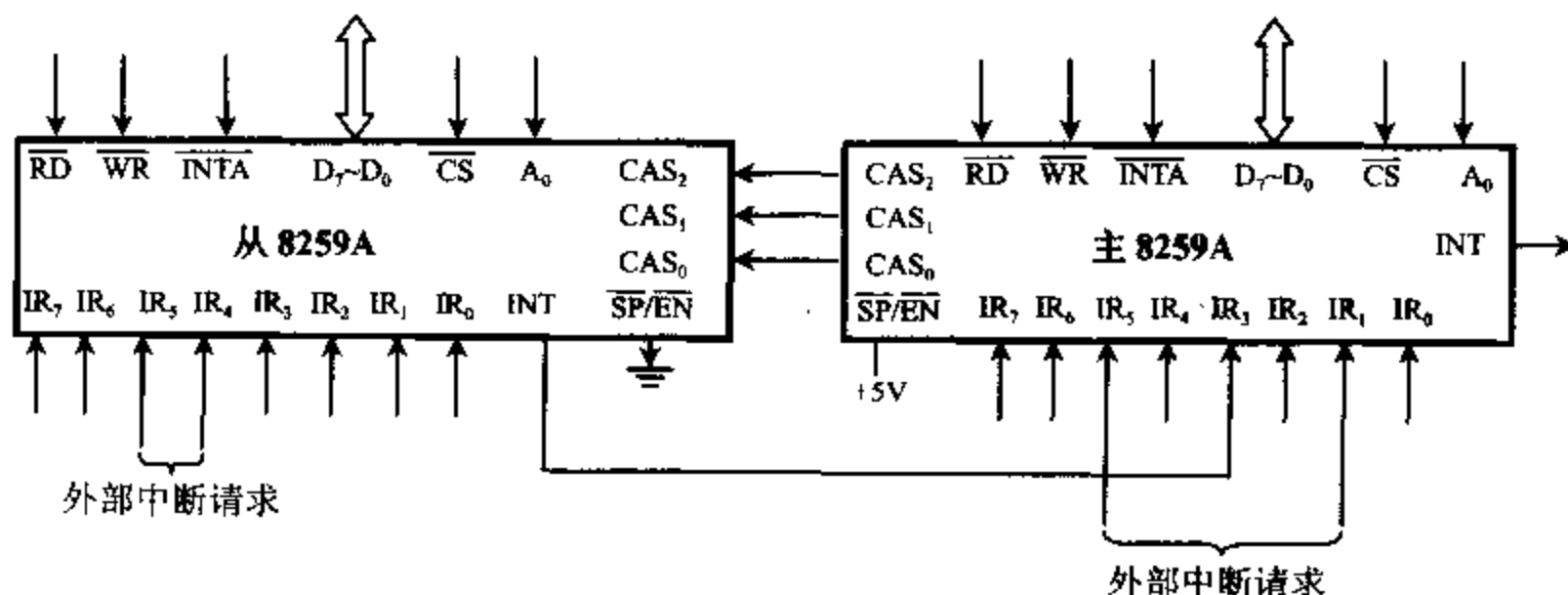


图 7-16 8259A 级联使用实例

000C4	00 20	IP	中断类型号 31H 入口地址	主 8259A 引入 的中断请求
000C6	00 10	CS		
000D4	00 30	IP	中断类型号 35H 入口地址	
000D6	00 10	CS		
	⋮			
00110	00 36	IP	中断类型号 44H 入口地址	从 8259A 引入 的中断请求
00112	00 20	CS		
00114	00 45	IP	中断类型号 45H 入口地址	
00116	00 20	CS		

图 7-17 中断入口地址表内容

(1) 中断向量形成:将 4 个中断入口地址写入中断向量表。

```

MOV    AX, 1000H           ;送入段地址
MOV    DS, AX
MOV    DX, 2000H           ;送入偏移地址
MOV    AL, 31H             ;中断类型号 31H
MOV    AH, 25H
INT     21H
MOV    DX, 3000H           ;中断类型号 35H
MOV    AL, 35H
INT     21H
MOV    AX, 2000H
MOV    DS, AX
MOV    DX, 3600H
MOV    AL, 44H             ;中断类型号 44H
MOV    AH, 25H
INT     21H
MOV    DX, 4500H           ;中断类型号 45H
MOV    AL, 45H
INT     21H

```

(2) 主片 8259A 初始化编程:8259A 主片端口地址为 FFC8H 和 FFC9H。

```

MOV    AL, 11H             ;定义 ICW1,主片 8259A 级联使用,边沿触发
MOV    DX, 0FFC8H
OUT     DX, AL
MOV    AL, 30H             ;定义 ICW2,中断类型号 30H~37H
MOV    DX, 0FFC9H
OUT     DX, AL
MOV    AL, 08H             ;定义 ICW3,IR3 端接从片 8259A 的 INT 端
OUT     DX, AL
MOV    AL, 11H             ;定义 ICW4,特殊全嵌套方式,非缓冲方式,
OUT     DX, AL             ;非自动 EOI 结束方式
MOV    AL, 0D5H            ;定义 OCW1,允许 IR1、IR3、IR5 中断,其余端
OUT     DX, AL             ;口中断请求屏蔽

```

(3) 8259A 从片初始化编程:从片 8259A 的端口地址为 FFCAH 和 FF CBH。

```

MOV    AL, 11H             ;定义 ICW1,级联使用边沿触发,要设置 ICW4
MOV    DX, 0FFCAH
OUT     DX, AL
MOV    AL, 40H             ;定义 ICW2,引入中断类型号为 40H~47H
MOV    DX, 0FFCBH
OUT     DX, AL
MOV    AL, 03H             ;定义 ICW3,从片接在主片的 IR3 端
OUT     DX, AL

```


MOV	AL,	01H	;定义 ICW ₄ ,完全嵌套方式,非缓冲方式,
OUT	DX,	AL	;非自动 EOI 结束方式
MOV	AL,	0CFH	;定义 OCW ₁ ,允许 IR ₄ ,IR ₅ 中断引入,
OUT	DX,	AL	;其余端口中断请求屏蔽

无论对主片 8259A 或从片 8259A,操作命令字可根据需要在操作过程中设置,OCW₂ 命令字定义中断结束方式时,通常放在中断服务子程序中。上例主片的中断结束命令为:

MOV	AL,	20H	;定义 OCW ₂ ,普通 EOI 结束方式
MOV	DX,	0FEC8H	
OUT	DX,	AL	

从片的中断结束命令为:

MOV	DX,	0FFCAH	;定义 OCW ₂ ,普通 EOI 结束方式
MOV	AL,	20H	
OUT	DX,	AL	

习 题

1. 什么叫中断? 什么叫可屏蔽中断和不可屏蔽中断?
2. 列出微处理器上的中断引脚和与中断有关的指令。
3. 8086/8088 系统中可以引入哪些中断?
4. CPU 响应中断的条件是什么? 简述中断处理过程。
5. 中断服务子程序中中断指令 STI 放在不同位置会产生什么不同结果? 中断嵌套时, STI 指令应如何设置?
6. 中断结束命令 EOI 放在程序不同位置处会产生什么不同结果?
7. 中断向量表的功能是什么?
8. 假定中断类型号 15 的中断处理程序的首地址为 ROUT15, 编写主程序中为建立一个中断向量的程序。
9. 8086/8088CPU 如何获得中断类型号?
10. 给定 SP=0100H、SS=0500H、PSW=0240H, 在存储单元中已有内容为(00024)=0060H、(00026H)=1000H, 在段地址为 0800H 及偏移地址为 00A0H 的单元中, 有一条中断指令 INT9。试问, 执行 INT9 指令后, SS、SP、IP、PSW 的内容是什么? 栈顶的三个字是什么?
11. 8259A 优先权管理方式有哪几种? 中断结束方式又有几种?
12. 单片 8259A 在完全嵌套中断工作方式下, 要写哪些初始化命令字及操作命令字?
13. 系统中新增加一个中断源, 在软件上应增加哪些内容, 此中断系统才能正常工作。
14. 系统中有 3 个中断源, 从 8259A 的 IR₀、IR₂、IR₄ 端引入中断, 以脉冲触发。中断类型号分别为 50H、52H、54H, 中断入口地址分别为 5020H、6100H、3250H, 段地址为 1000H。使用完全嵌套工作方式, 普通 EOI 结束, 试编写初始化程序, 使 CPU 响应任何一

级中断时,能正确工作。并编写一段中断服务子程序,保证中断嵌套的实现及正确返回。

15. 假如外设 A_1 、 A_2 、 A_3 、 A_4 、 A_5 按优先级排列,外设 A_1 优先级最高,按下列提问,说明中断处理的运行次序,(中断服务程序中有 STI 指令)

(1) 外设 A_3 、 A_4 同时发中断请求;

(2) 外设 A_3 中断处理中,外设 A_1 发中断请求;

(3) 外设 A_1 中断处理未完成前,发出 EOI 结束命令,外设 A_5 发中断请求。

16. 某系统中有 3 片 8259A 级联使用,1 片为 8259A 主片,2 片为 8259A 从片,从片接入 8259A 主片的 IR_2 和 IR_5 端,并且当前 8259A 主片的 IR_3 及两片 8259A 从片的 IR_4 各接有一个外部中断源。中断类型基号分别为 80H、90H、A0H,中断入口段基址在 2000H,偏移地址分别为 1800H、2800H、3800H,主片 8259A 的端口地址为 CCF8H、CCFAH。一片 8259A 从片的端口地址为 FEE8H、FEEAH,另一片为 FEECH、FEEEH。中断采用电平触发,完全嵌套工作方式,普通 EOI 结束。

(1) 画出硬件连接图;

(2) 编写初始化程序。

第八章 可编程计数器/定时器 8253 及其应用

在微型计算机系统中,常需要用到定时功能。例如,在 IBM PC 机中,需要有一个实时时钟以实现计时功能,还要求按一定的时间间隔对动态 RAM 进行刷新,此外,扬声器的发声也是由定时信号来驱动的。在计算机实时控制和处理系统中,则应按一定的采样周期对处理对象进行采样,或定时检测某些参数等等,都需要定时信号。此外,在许多微机应用系统中,还会用到计数功能,需对外部事件进行计数。

主要有三种方法来实现定时功能,即软件定时、不可编程的硬件定时和可编程的硬件定时。

软件定时是最简单的定时方法,它不需要硬件支持,只要让机器循环执行某一条或一系列指令,这些指令本身并没有具体的执行目的,但由于执行每条指令都需要一定的时间,重复执行这些指令就会占用一段固定的时间。因此,习惯上将这种定时方法称为软件延时。通过正确选取指令和合适的循环次数,便很容易实现定时功能。利用这种方法定时,完全由软件编程来控制 and 改变定时时间,灵活方便,而且节省费用。我们已在第三章中介绍过这种方法。这种方法的明显缺点是 CPU 的利用率太低,在定时循环期间,CPU 不能再去做任何其它有用的工作,而仅仅是在反复循环,等待预定的定时时间的到来,这在许多情况下是不允许的。比如,对动态存储器的定时刷新操作,只要处于开机状态,就需要一直不停地进行下去,显然不能采用软件定时。为了提高 CPU 的利用率,常采用可编程和不可编程的硬件电路来实现定时。

555 芯片是一种常用的不可编程器件,加上外接电阻和电容就能构成定时电路。这种定时电路结构简单,价格便宜,通过改变电阻或电容值,可以在一定的定时范围内改变定时时间。但这种电路在硬件已连接好的情况下,定时时间和范围就不能由程序来控制 and 改变,而且定时精度也不高。

可编程定时器/计数器电路利用硬件电路和中断方法控制定时,定时时间和范围完全由软件来确定 and 改变,并由微处理器的时钟信号提供时间基准,因这种时钟信号由晶体振荡器产生,故计时精确稳定。但该时钟信号频率太高,所以要把它送到专门的计数器/定时器芯片进行分频后,才能产生所需的各种定时信号。用可编程定时器/计数器电路进行定时时,先要根据预定的定时时间,用指令对计数器/定时器芯片设定计数初值,然后启动芯片进行工作。计数器一旦开始工作后,CPU 就可以去做别的工作了,等计数器计到预定的时间,便自动形成一个输出信号,该信号可用来向 CPU 提出中断请求,通知 CPU 定时时间已到,使 CPU 作相应的处理。或者直接利用输出信号去启动设备工作。这种方法不但显著提高了 CPU 的利用率,而且定时时间由软件设置,使用起来十分灵活方便,加上定时时间又很精确,所以获得了广泛的应用。

如果系统中有产生代表外部事件的脉冲信号源,也可以利用计数器/定时器芯片对外部事件进行计数。

Intel 8253 就是一种能完成上述功能的计数器/定时器芯片,被称为可编程间隔定时器(Programmable Interval Timer, PIT)。8253 内部具有 3 个独立的 16 位计数器通道,通过对它进行编程,每个计数器通道均可按 6 种不同的方式工作,并且都可以按 2 进制或 10 进制格式进行计数,最高计数频率能达到 2MHz。8253 还适用于许多其它的情况,如用作可编程方波频率产生器、分频器、程控单脉冲发生器等等。

Intel 8254 是 8253 的增强型产品,它与 8253 的引脚兼容,功能几乎完全相同,不同之处仅在于以下两点:

(1) 8253 的最大输入时钟频率为 2MHz,而 8254 的最大输入时钟频率可高达 5MHz,8254-2 则为 10MHz。

(2) 8254 有读回(read-back)功能,可以同时锁存 1~3 个计数器的计数值及状态值,供 CPU 读取,而 8253 每次只能锁存和读取一个通道的计数器,且不能读取状态值。

下面主要以 Intel 8253 为例,介绍计数器/定时器芯片的基本工作原理和使用方法。在讨论计数器/定时器的应用实例时,将举例说明 8254 的读回功能。

8-1 8253 的工作原理

一、8253 的内部结构和引脚信号

8253 的内部结构和引脚信号分别如图 8-1 和图 8-2 所示。

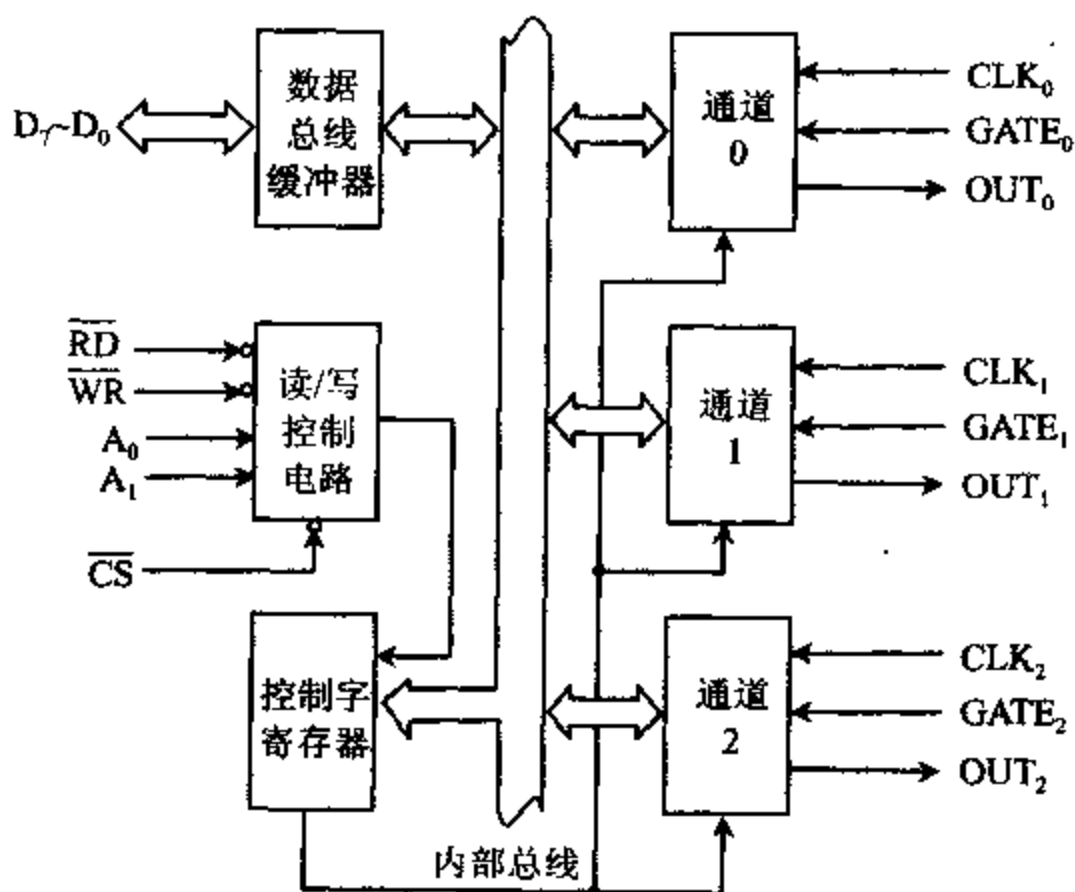


图 8-1 8253 的内部结构

从图 8-1 可见,8253 内部包含数据总线缓冲器、读/写控制逻辑、控制字寄存器和 3 个结构完全相同的计数器,这 3 个计数器分别称为计数器 0,计数器 1 和计数器 2。各部分的功能和有关引脚的意义分述如下:

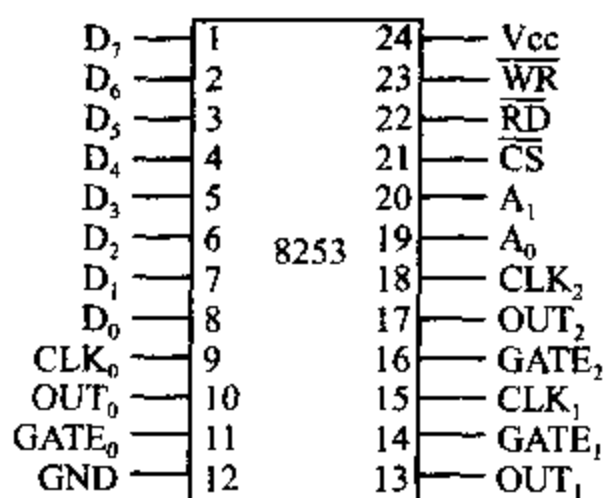


图 8-2 8253 的引脚

1. 数据总线缓冲器

数据总线缓冲器是 8253 与系统数据总线相连接时用的接口电路,它由 8 位双向三态缓冲器构成,CPU 用输入、输出指令对 8253 进行读/写操作的信息,都经 8 位数据总线 D₇~D₀ 传送,这些信息包括:

- (1) CPU 在对 8253 进行初始化编程时,向它写入的控制字。
- (2) CPU 向某一计数器写入的计数初值。
- (3) 从计数器读出的计数值。

2. 读/写控制逻辑

读/写控制逻辑接收系统控制总线送来的输入信号,经组合后形成控制信号,对各部分操作进行控制。可接收的信号有:

(1) $\overline{CS}$ 片选信号,低电平有效,由地址总线经 I/O 端口译码电路产生。只有当 $\overline{CS}$ 为低电平时,CPU 才能对 8253 进行读写操作。

(2) $\overline{RD}$ 读信号,低电平有效。当 $\overline{RD}$ 为低电平时,表示 CPU 正在读取所选定的计数器通道中的内容。

(3) $\overline{WR}$ 写信号,低电平有效。当 $\overline{WR}$ 为低电平时,表示 CPU 正在将计数初值写入所选中的计数通道中或者将控制字写入控制字寄存器中。

(4) A₁A₀ 端口选择信号。在 8253 内部有 3 个计数器通道(0~2)和一个控制字寄存器端口。当 A₁A₀=00 时,选中通道 0;A₁A₀=01 时,选中通道 1;A₁A₀=10 时,选中通道 2;A₁A₀=11 时,选中控制字寄存器端口。

如果 8253 与 8 位数据总线的微机相连,只要将 A₁A₀ 分别与地址总线的最低两位 A₁A₀ 相连即可。比如,在以 8088 为 CPU 的 PC/XT 机中,地址总线高位部分(A₉~A₁)用于 I/O 端口译码,形成选择各 I/O 芯片的片选信号,低位部分(A₃~A₀)用于各芯片内部端口的寻址。若 8253 的端口基地址为 40H,则通道 0,1,2 和控制字寄存器端口的地址分别为 40H,41H,42H 和 43H。

如果系统采用的是 8086 CPU,则数据总线为 16 位。CPU 在传送数据时,总是将低 8 位数据送往偶地址端口,将高 8 位数据送到奇地址端口。反之,偶地址端口的数据总是通过低 8 位数据总线送到 CPU,奇地址端口的数据总是通过高 8 位数据总线送到 CPU。当仅具有 8 位数据总线的存储器或 I/O 接口芯片与 8086 的 16 位数据总线相连时,既可以连到高 8 位数据总线,也可以接在低 8 位数据总线上。在实际设计系统时,为了方便起见,常将这些芯片的数据线 D₇~D₀ 接到系统数据总线的低 8 位,这样,CPU 就要求芯片内部的各个端口都使用偶地址。

假设一片 8253 被用于 8086 系统中,为了保证各端口均为偶地址,CPU 访问这些端口时,必须将地址总线的 A₀ 置为 0。因此,我们就不能像在 8088 系统中那样,用地址线 A₀ 来选择 8253 中的各个端口。而改用地地址总线中的 A₂A₁ 实现端口选择,即将 A₂ 连到 8253 的

A_1 引脚,而将 A_1 与 8253 的 A_0 引脚相连。若 8253 的基地址为 $F0H(11110000B)$,因 $A_2A_1=00$,所以它也就是通道 0 的地址; $A_2A_1=01$ 选择通道 1,所以通道 1 的地址为 $F2H(11110010B)$; $A_2A_1=10$,选择通道 2,即口地址为 $F4H(11110100B)$; $A_2A_1=11$ 选中控制字寄存器,即口地址为 $F6H(11110110B)$ 。

各输入信号经组合形成的控制功能如表 8-1 所示。

表 8-1 8253 输入信号组合的功能表

$\overline{CS}$	$\overline{RD}$	$\overline{WR}$	A_1A_0	功 能
0	1	0	0 0	写入计数器 0
0	1	0	0 1	写入计数器 1
0	1	0	1 0	写入计数器 2
0	1	0	1 1	写入控制字寄存器
0	0	1	0 0	读计数器 0
0	0	1	0 1	读计数器 1
0	0	1	1 0	读计数器 2
0	0	1	1 1	无操作
1	×	×	×	禁止使用
0	1	1	×	无操作

3. 计数器 0~2

8253 内部包含 3 个完全相同的计数器/定时器通道,对 3 个通道的操作完全是独立的。每个通道都包含一个 8 位的控制字寄存器、一个 16 位的计数初值寄存器、一个计数器执行部件(实际的计数器)和一个输出锁存器。执行部件实际上是一个 16 位的减法计数器,它的起始值就是初值寄存器的值,该值可由程序设置。输出锁存器用来锁存计数器执行部件的值,必要时 CPU 可对它执行读操作,以了解某个时刻计数器的瞬时值。计数初值寄存器、计数器执行部件和输出锁存器都是 16 位寄存器,它们均可被分成高 8 位和低 8 位两个部分,因此也可作为 8 位寄存器来使用。

每个通道工作时,都是对输入到 CLK 引脚上的脉冲按 2 进制或 10 进制(BCD 码)格式进行计数。计数采用倒计数法,先对计数器预置一个初值,再把初值装入实际的计数器。然后,开始递减计数。即每输入一个时钟脉冲,计数器的值减 1,当计数器的值减为 0 时,便从 OUT 引脚输出一个脉冲信号。输出信号的波形主要由工作方式决定,同时还受到从外部加到 GATE 引脚上的门控信号控制,它决定是否允许计数。

当用 8253 作外部事件计数器时,在 CLK 脚上所加的计数脉冲是由外部事件产生的,这些脉冲的间隔可以是不相等的。如果要用它作定时器,则 CLK 引脚上应输入精确的时钟脉冲。这时,8253 所能实现的定时时间,决定于计数脉冲的频率和计数器的初值,即

定时时间 = 时钟脉冲周期 $t_c \times$ 预置的计数初值 n

例如,在某系统中,8253 所使用的计数脉冲频率为 $0.5MHz$,即脉冲周期 $t_c=2\mu s$,如果给 8253 的计数器预置的初值 $n=500$,则当计数器计到数值为 0 时,定时时间 $T=2\mu s \times 500=1ms$ 。

对 8253 来讲,外部输入到 CLK 引脚上的时钟脉冲频率不能大于 $2MHz$ 。如果大于

2MHz,则必需经分频后才能送到 CLK 端,使用时要注意。

8253 的 3 个计数器都各有 3 个引脚,它们是:

- (1) $CLK_0 \sim CLK_2$ 计数器 0~2 的输入时钟脉冲从这里输入。
- (2) $OUT_0 \sim OUT_2$ 计数器 0~2 的输出端。
- (3) $GATE_0 \sim GATE_2$ 计数器 0~2 的门控脉冲输入端。

4. 控制字寄存器

控制字寄存器是一种只写寄存器,在对 8253 进行编程时,由 CPU 用输出指令向它写入控制字,来选定计数器通道,规定各计数器通道的工作方式,读写格式和数制。控制字的格式如图 8-3 所示。

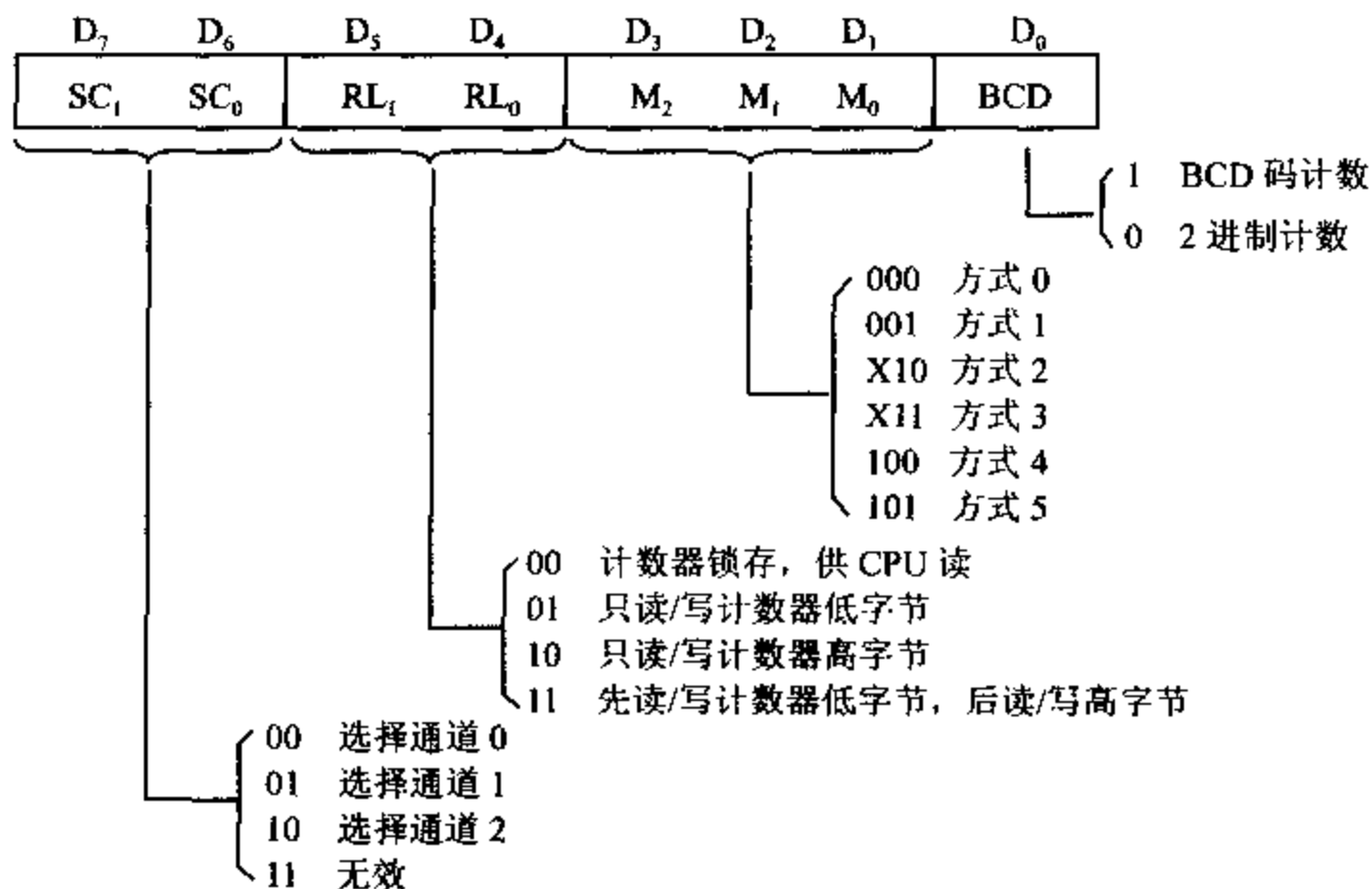


图 8-3 8253 控制字格式

SC₁SC₀ ——通道选择位。由于 8253 内部有 3 个计数通道,需要有 3 个控制字寄存器分别规定相应通道的工作方式,但这 3 个控制字寄存器只能使用同一个端口地址,在对 8253 进行初始化编程,设置控制字时,需由这两位来决定在向哪一个通道写入控制字。选择 SC₁SC₀=00,01,10 分别表示向 8253 的计数器通道 0~2 写入控制字。SC₁SC₀=11 时无效。

RL₁RL₀ ——读/写操作位,用来定义对选中通道中的计数器的读/写操作方式。当 CPU 向 8253 的某个 16 位计数器装入计数初值,或从 8253 的 16 位计数器读入数据时,可以只读写它的低 8 位字节或高 8 位字节。RL₁RL₀ 组成 4 种编码,表示 4 种不同的读/写操作方式,即:

RL₁RL₀=01,表示只读/写低 8 位字节数据,只写入低 8 位时,高 8 位自动置为 0;

RL₁RL₀=10,表示只读/写高 8 位字节数据,只写入高 8 位时,低 8 位自动置为 0;

RL₁RL₀=11,允许读/写 16 位数据。由于 8253 的数据线只有 8 位(D₇~D₀),一次只能传送 8 位数据,故读/写 16 位数据时必须分两次进行,先读/写计数器的低 8 位字节,后

读/写高 8 位字节；

$RL_1 RI_0 = 00$, 把通道中当前数据寄存器的值送到 16 位锁存器中, 供 CPU 读取该值。

BCD——计数方式选择位。当该位为 1 时, 采用 BCD 码计数, 写入计数器的初值用 BCD 码表示, 初值范围为 $0000 \sim 9999H$, 其中 0000 表示最大值 10000 , 即 10^4 。例如, 当我们预置的初值 $n = 1200H$ 时, 就表示预置了一个十进制数 1200 。当 BCD 位为 0 时, 则采用二进制格式计数, 写入计数器中的初值用二进制数表示。在程序中, 二进制数可以写成 16 进制数的形式, 所以初值范围为 $0000 \sim FFFFH$, 其中 0000 表示最大值 65536 , 即 2^{16} 。这时, 如果我们仍预置了一个初值 $n = 1200H$, 就表示预置了一个十进制数 4608 。

$M_2 M_1 M_0$ ——工作方式选择位。8253 的每个通道都有 6 种不同的工作方式, 即方式 $0 \sim 5$, 当前工作于哪种方式, 由这 3 位来选择。每种工作方式的特点、计数器的输出与输入及门控信号之间的关系等问题, 将在后面作进一步介绍。

二、初始化编程步骤和门控信号的功能

1. 8253 的初始化编程步骤

刚接通电源时, 诸如 8253 之类的可编程外围接口芯片通常都处于未定义状态, 在使用之前, 必须用程序把它们初始化为所需的特定模式, 这个过程称为初始化编程。对 8253 芯片进行初始化编程时, 需按下列步骤进行:

(1) 写入控制字

用输出指令向控制字寄存器写入一个控制字, 以选定计数器通道, 规定该计数器的工作方式和计数格式。写入控制字还起到复位作用, 使输出端 OUT 变为规定的初始状态, 并使计数器清 0。

(2) 写入计数初值

用输出指令向选中的计数器端口地址中写入一个计数初值, 初值设置时要符合控制字中有关格式的规定。初值可以是 8 位数据, 也可以是 16 位数据。若是 8 位数, 只要用一条输出指令就可完成初值的设置。如果是 16 位数, 则必须用两条输出指令来完成, 而且规定先送低 8 位数据, 后送高 8 位数据。注意, 计数初值为 0 时, 也要分成两次写入, 因为在二进制计数时, 它表示 65536 , BCD 计数时, 它表示 10000 。

由于 3 个计数器分别具有独立的编程地址, 而控制字寄存器本身的内容又确定了所控制的寄存器的序号, 因此对 3 个计数器通道的编程没有先后顺序的规定, 可任意选择某一个计数器通道进行初始化编程, 只要符合先写入控制字, 后写入计数初值的规定即可。

例如, 在某微机系统中, 8253 的 3 个计数器的端口地址分别为 $3F0H$ 、 $3F2H$ 和 $3F4H$, 控制字寄存器的端口地址为 $3F6H$, 要求 8253 的通道 0 工作于方式 3, 并已知对它写入的计数初值 $n = 1234H$, 则初始化程序为:

MOV	AL,	00110111B	;控制字:选择通道 0,先读/写低字节,后高字节,方式 3,BCD 计数
MOV	DX,	3F6H	;指向控制口
OUT	DX,	AL	;送控制字
MOV	AL,	34H	;计数值低字节
MOV	DX,	3F0H	;指向计数器 0 端口


```

OUT    DX, AL      ;先写入低字节
MOV    AL, 12H     ;计数值高字节
OUT    DX, AL      ;后写入高字节

```

在计数初值写入 8253 后,还要经过一个时钟脉冲的上升沿和下降沿,才能将计数初值装入实际的计数器,然后在门控信号 GATE 的控制下,对从 CLK 引脚输入的脉冲进行递减计数。

2. 门控信号控制功能

门控信号 GATE 在各种工作方式中的控制功能如表 8-2 所示,其中符号“—”表示无影响。

表 8-2 门控信号 GATE 的控制功能

工作方式	GATE 为低电平或下降沿	GATE 为上升沿	GATE 为高电平
方式 0	禁止计数	—	允许计数
方式 1	—	从初始值开始计数,下一个时钟后输出变为低电平	—
方式 2	禁止计数,使输出变高	从初值开始计数	允许计数
方式 3	禁止计数,使输出变高	从初值开始计数	允许计数
方式 4	禁止计数	—	允许计数
方式 5	—	从初值开始计数	—

从表 8-2 可以看到,可以用门控信号的上升沿、低电平或下降沿来控制 8253 进行计数。对于方式 0 和方式 4,当 GATE 为高电平时,允许计数,GATE 为低电平或下降沿时,禁止计数。对于方式 1 和方式 5,只有当门控信号产生从低电平到高电平的正跳变时,才允许 8253 从初始值开始计数。但两者对输出电平的影响是有区别的,在方式 1 时,GATE 信号触发 8253 开始计数后,就使输出端 OUT 变成低电平,而方式 5 的 GATE 触发信号不影响 OUT 端的电平。对方式 2 和方式 3,GATE 为高电平时允许计数,低电平或下降沿时禁止计数,若 GATE 变低后又产生从低到高的正跳变时,将会再次触发 8253 从初值开始计数。

三、8253 的工作方式

8253 的每个通道都有 6 种不同的工作方式,下面分别进行介绍。

1. 方式 0 ——计数结束中断方式(Interrupt on Terminal Count)

此方式的定时波形如图 8-4 所示,工作过程如下:

当对 8253 的任一通道写入控制字,并选定工作于方式 0 时,该通道的输出端 OUT 立即变为低电平。要使 8253 能够进行计数,门控信号 GATE 必须为高电平。如图 8-4 所示,设 GATE 为高电平。若 CPU 利用输出指令向计数通道写入初值 $n(=4)$ 时, $\overline{WR}_n$ 变成低电平。在 $\overline{WR}_n$ 的上升沿时, n 被写入 8253 内部的计数器初值寄存器。在 $\overline{WR}_n$ 上升沿后的下一个时钟脉冲的下降沿时,才把 n 装入通道内的实际计数器中,开始进行减 1 计数。也就是说,从写入计数器初值到开始减 1 计数之间,有一个时钟脉冲的延迟。此后,每从 CLK 引脚输入一个脉冲,计数器就减 1。总共经过 $n+1$ 个脉冲后,计数器减为 0,表示计数计到

终点,计数过程结束,这时 OUT 引脚由低电平变成高电平。这个由低到高的正跳变信号,可以接到 8259A 的中断请求输入端,利用它向 CPU 发中断请求信号。OUT 引脚上的高电平信号,一直保持到对该计数器装入新的计数值,或设置新的工作方式为止。

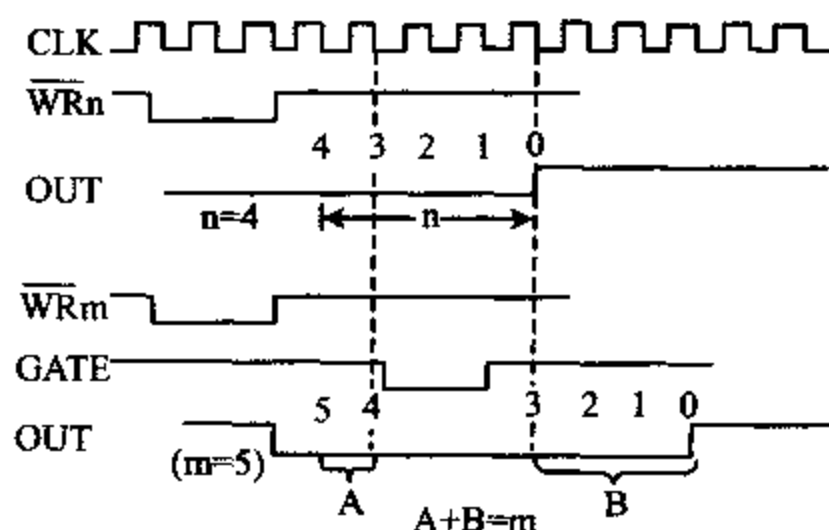


图 8-4 方式 0 波形图

在计数的过程中,如果 GATE 变为低电平,则暂停减 1 计数,计数器保持 GATE 有效时的值不变,OUT 仍为低电平。待 GATE 回到高电平后,又继续往下计数。我们用图 8-4 中下半部分的波形来说明这种情况,这时,计数初值取 $m=5$ 。

按方式 0 进行计数时,计数器只计一遍。当计数器计到 0 时,不会再装入初值重新开始计数,其输出将保持高电平。若重新写入一个新的计数初值,OUT 立即变成低电平,计数器将按照新的计数值开始计数。

2. 方式 1 ——可编程单稳态输出方式(Programmable One-shot)

8253 工作于方式 1 时的波形如图 8-5 所示。当 CPU 用控制字设定某计数器工作于方式 1 时,该计数器的输出 OUT 立即变为高电平。在这种方式下,在 CPU 装入计数值 n 后,无论 GATE 是高电平还是低电平,都不进行减 1 计数,必须等到 GATE 由低电平向高电平跳变,形成一个上升沿后,才能在下-一个时钟脉冲的下降沿,将 n 装入计数器的执行部件,同时,输出端 OUT 由高电平向低电平跳变。以后,每来一个时钟脉冲,计数器就开始减 1 操

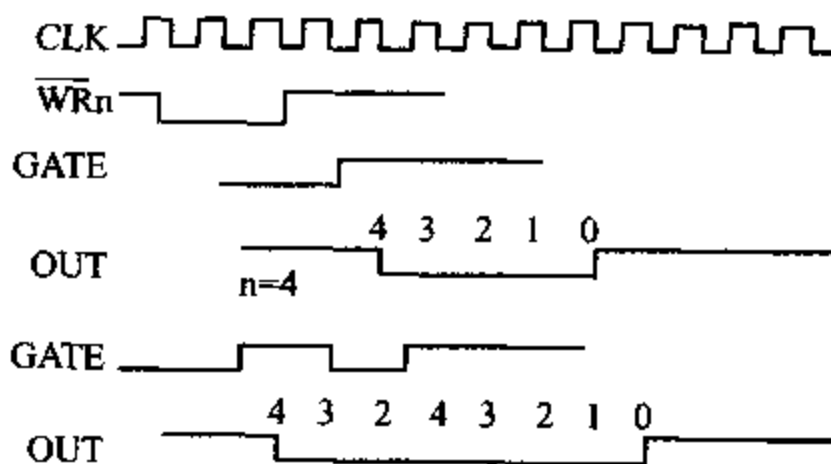


图 8-5 方式 1 波形图

作。当计数器的值减为零时,输出端 OUT 产生由低到高的正跳变。这样,就可在 OUT 引脚上得到一个负的单脉冲,单脉冲的宽度可由程序来控制,它等于时钟脉冲的宽度乘以计数值 n 。

在计数过程中,若 GATE 产生负跳变,不会影响计数过程的进行。但若在计数器回零

前, GATE 又产生从低到高的正跳变, 则 8253 又将初值 n 装入计数器执行部件, 重新开始计数, 其结果会使输出的单脉冲宽度加宽。因此, 只要计数器没有回零, 利用 GATE 的上升沿可以多次触发计数器从 n 开始重新计数, 直到计数器减为 0 时, OUT 才回到高电平。

在方式 1 时, 门控信号的上升沿作为触发信号, 使输出变低, 当计数值变为 0 时, 又使输出自动回到高电平。所以, 这时的 8253 实际上处于一种单稳态工作方式。单稳态输出脉冲的宽度主要取决于计数初值的大小, 但也受门控信号的影响, 在单稳态受触发后输出未回稳态(高电平)时, 若又受到触发, 会使单稳态输出的负脉冲变宽。

3. 方式 2 ——比率发生器(Rate Generator)

方式 2 的定时波形如图 8-6 所示。

当对某一计数通道写入控制字, 选定工作方式 2 时, OUT 端输出高电平。如果 GATE 为高电平, 则在写入计数值后的下一个时钟脉冲时, 将计数值装入执行部件。此后, 计数器随着时钟脉冲的输入而递减计数。当计数值减为 1 时, OUT 端由高电平变为低电平, 待计数器的值减为 0 时, OUT 引脚又回到高电平, 即低电平的持续时间等于一个输入时钟周期。与此同时, 还将计数初值重新装入计数器, 开始一个新的计数过程, 并由此周而复始地循环计数。如果装入计数器的初值为 n , 那么在 OUT 引脚上, 每隔 n 个时钟脉冲就产生一个负脉冲, 其宽度与时钟脉冲的周期相同, 频率为输入时钟脉冲频率的 n 分之一。所以, 这实际上是一种分频工作方式。

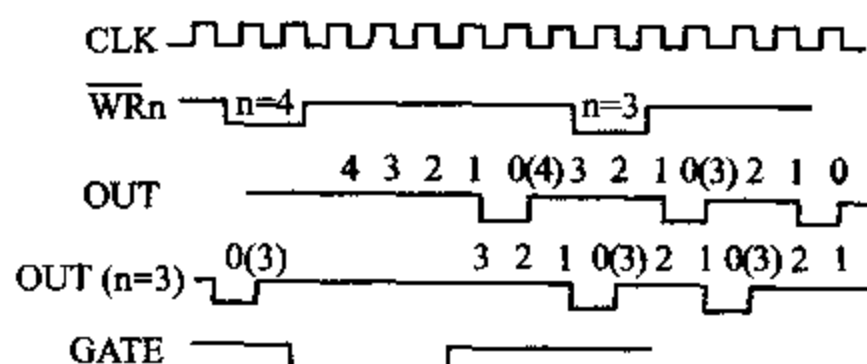


图 8-6 方式 2 波形图

在操作过程中, 任何时候都可由 CPU 重新写入新的计数值, 它不会影响当前计数过程的进行。如图 8-6 所示, 原来的计数值 $n=4$, 在计数过程中计数值回零前, 又写入新的计数值 $n=3$, 8253 仍按 $n=4$ 进行计数。当计数值减为 0 时, 一个计数周期结束, 8253 将按新写入的计数值 $n=3$ 进行计数。

在计数过程中, 当 GATE 变为低电平时, 将迫使 OUT 变为高电平, 并禁止计数; 当 GATE 从低电平变为高电平, 也就是 GATE 端产生上升沿时, 则在下一个时钟脉冲时, 又把预置的计数初值装入计数器, 从初值开始递减计数, 并循环进行。

当需要产生连续的负脉冲序列信号时, 可使 8253 工作于方式 2。

4. 方式 3 ——方波发生器(Square Wave Generator)

方式 3 和方式 2 的工作相类似, 但从输出端得到的不是序列负脉冲, 而是对称的方波或基本对称的矩形波。当 GATE 为高电平时的输出波形如图 8-7 所示, 工作过程如下:

当输入控制字后, OUT 端输出变为高电平。如果 GATE 为高电平, 则在写入计数值后的下一个时钟脉冲时, 将计数值装入执行部件, 并开始计数。

如果写入计数器的初值为偶数, 则当 8253 进行计数时, 每输入一个时钟脉冲, 均使计

数值减 2。计数值减为 0 时,OUT 输出引脚由高电平变成低电平,同时自动重新装入计数初值,继续进行计数。当计数值减为 0 时,OUT 引脚又回到高电平,同时再一次将计数初值装入计数器,开始新一轮循环计数;如果写入计数器的初值为奇数,则当输出端 OUT 为高电平时,第一个时钟脉冲使计数器减 1,以后每来一个时钟脉冲,都使计数器减 2,当计数值减为 0 时,输出端 OUT 由高电平变为低电平,同时自动重新装入计数初值继续进行计数。这时第一个时钟脉冲使计数器减 3,以后每个时钟脉冲都使计数器减 2,计数值减为 0 时,OUT 端又回到高电平,并重新装入计数初值后,开始新一轮循环计数。这两种情况下,从 OUT 端输出的方波频率都等于时钟脉冲的频率除以计数初值。但要注意,当写入的计数初值为偶数时,输出完全对称的方波,写入初值为奇数时,其输出波形的高电平宽度比低电平多一个时钟周期。

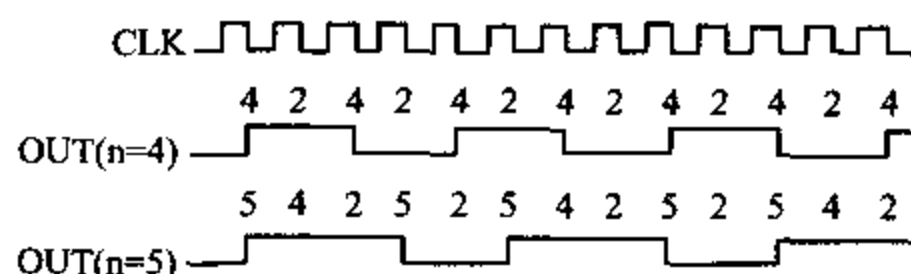


图 8-7 方式 3 波形图

在计数过程中,若 GATE 变成低电平时,就迫使 OUT 变为高电平,并禁止计数,当 GATE 回到高电平时,重新从初值 n 开始进行计数。

如果希望改变输出方波的速率,CPU 可在任何时候重新装入新的计数初值,在下一个计数周期就可按新的计数值计数,从而改变方波的速率。

5. 方式 4 ——软件触发选通 (Software Triggered Strobe)

方式 4 的波形图如图 8-8 所示。

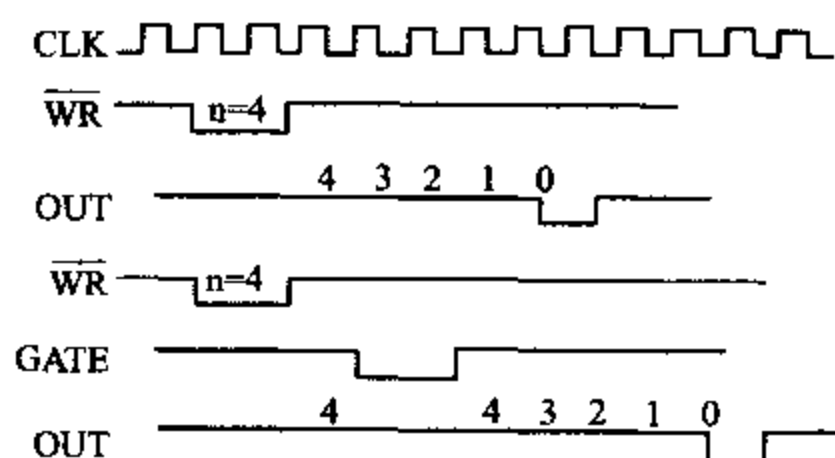


图 8-8 方式 4 波形图

当对 8253 写入控制字,进入工作方式 4 后,OUT 端输出变为高电平。如果 GATE 为高电平,那么,写入计数初值后,在下一个时钟脉冲后沿将自动把计数初值装入执行部件,并开始计数。当计数值减为 0 时,OUT 端输出变低,经过一个时钟周期后,又回到高电平,形成一个负脉冲。方式 4 之所以称为软件触发选通方式,这是因为计数过程是由软件把计数初值装入计数寄存器来触发的。用这种方法装入的计数初值 n 仅一次有效,若要进行计数,必须重新装入计数初值。若在计数过程中写入一个新的计数值,将按新的计数初值进行计数,同样也只计一次。

如果在计数的过程中 GATE 变为低电平,则停止计数,当 GATE 变为高电平后,又重新将初值装入计数器,从初值开始计数,直至计数器的值减为 0 时,从 OUT 端输出一个负脉冲。

6. 方式 5 ——硬件触发选通(Hardware Triggered Strobe)

方式 5 也称为硬件触发选通方式,波形时序如图 8-9 所示。

编程进入工作方式 5 后,OUT 端输出高电平。当装入计数值 n 后,不管 GATE 是高电平还是低电平,减 1 计数器都不会工作。一定要等到从 GATE 引脚上输入一个从低到高的正跳变信号时,才能在下一个时钟脉冲后沿把计数初值装入执行部件,并开始减 1 计数。当计数器的值减为 0 时,输出端 OUT 产生一个宽度为一个时钟周期的负脉冲,然后 OUT 又回到高电平。计数器回 0 后,8253 又自动将计数值 n 装入执行部件,但并不开始计数,要等到 GATE 端输入正跳变后,才又开始减 1 计数。由于从 OUT 端输出的负脉冲,是通过硬件电路产生的门控信号上升沿触发减 1 计数而形成的,所以这种工作方式称为硬件触发选通方式。

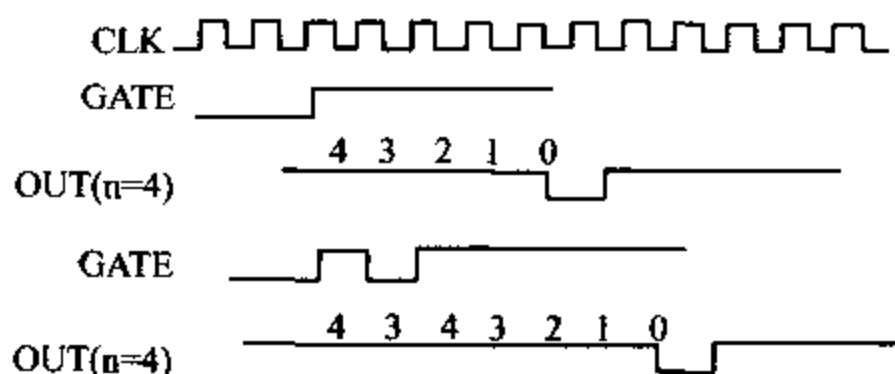


图 8-9 方式 5 波形图

计数器在计数过程中,不受门控信号 GATE 电平的影响,但只要计数器未回 0,GATE 的上升沿却能多次触发计数器,使它重新从计数初值 n 开始计数,直到计数值减为 0 时,才输出一个负脉冲。

如果在计数过程中写入新的计数值,但没有触发脉冲,则计数过程不受影响。当计数器的值减为 0 后,GATE 端又输入正跳变触发脉冲时,将按新写入的初值进行计数。

由上而的讨论可知,6 种工作方式各有特点,因而适用的场合也不一样。现将各种方式的主要特点概括如下:

对于方式 0,在写入控制字后,输出端即变低,计数结束后,输出端由低变高,常用该输出信号作为中断源。其余 5 种方式写入控制字后,输出均变高。方式 0 可用来实现定时或对外部事件进行计数。

方式 1 用来产生单脉冲。

方式 2 用来产生序列负脉冲,每个负脉冲的宽度与 CLK 脉冲的周期相同。

方式 3 用于产生连续的方波。方式 2 和方式 3 都实现对时钟脉冲进行 n 分频。

方式 4 和方式 5 的波形相同,都在计数器回 0 后,从 OUT 端输出一个负脉冲,其宽度等于一个时钟周期。但方式 4 由软件(设置计数值)触发计数,而方式 5 由硬件(门控信号 GATE)触发计数。

这 6 种工作方式中,方式 0、1 和 4,计数初值装进计数器后,仅一次有效。如果要通道再次按此方式工作,必须重新装入计数值。对于方式 2、3 和 5,在减 1 计数到 0 值后,8253 会

自动将计数值重装进计数器。

8-2 8253 的应用举例

8253 可以用在微型机系统中,构成各种计数器、定时器电路或脉冲发生器等。使用 8253 时,先要根据实际需要设计硬件电路,然后用输出指令向有关通道写入相应的控制字和计数初值,对 8253 进行初始化编程,这样 8253 就可以工作了。由于 8253 的 3 个计数通道是完全独立的,因此可以分别对它们进行硬件设计和软件编程,使三个通道工作于相同或不同的工作方式。为了清楚起见,下面分成定时功能和计数功能两个方面来介绍 8253 的应用,然后给出 8253 在 PC/XT 机中的应用实例。

一、8253 定时功能的应用例子

1. 用 8253 产生各种定时波形

在某个以 8086 为 CPU 的系统中使用了一块 8253 芯片,通道的基地址为 310H,所用的时钟脉冲频率为 1MHz。要求 3 个计数通道分别完成以下功能:

- (1) 通道 0 工作于方式 3,输出频率为 2kHz 的方波;
- (2) 通道 1 产生宽度为 480 μ s 的单脉冲;
- (3) 通道 2 用硬件方式触发,输出单脉冲,时间常数为 26。

据此设计的硬件电路如图 8-10 所示。由图可见,8253 芯片的片选信号 $\overline{CS}$ 由 74LS138

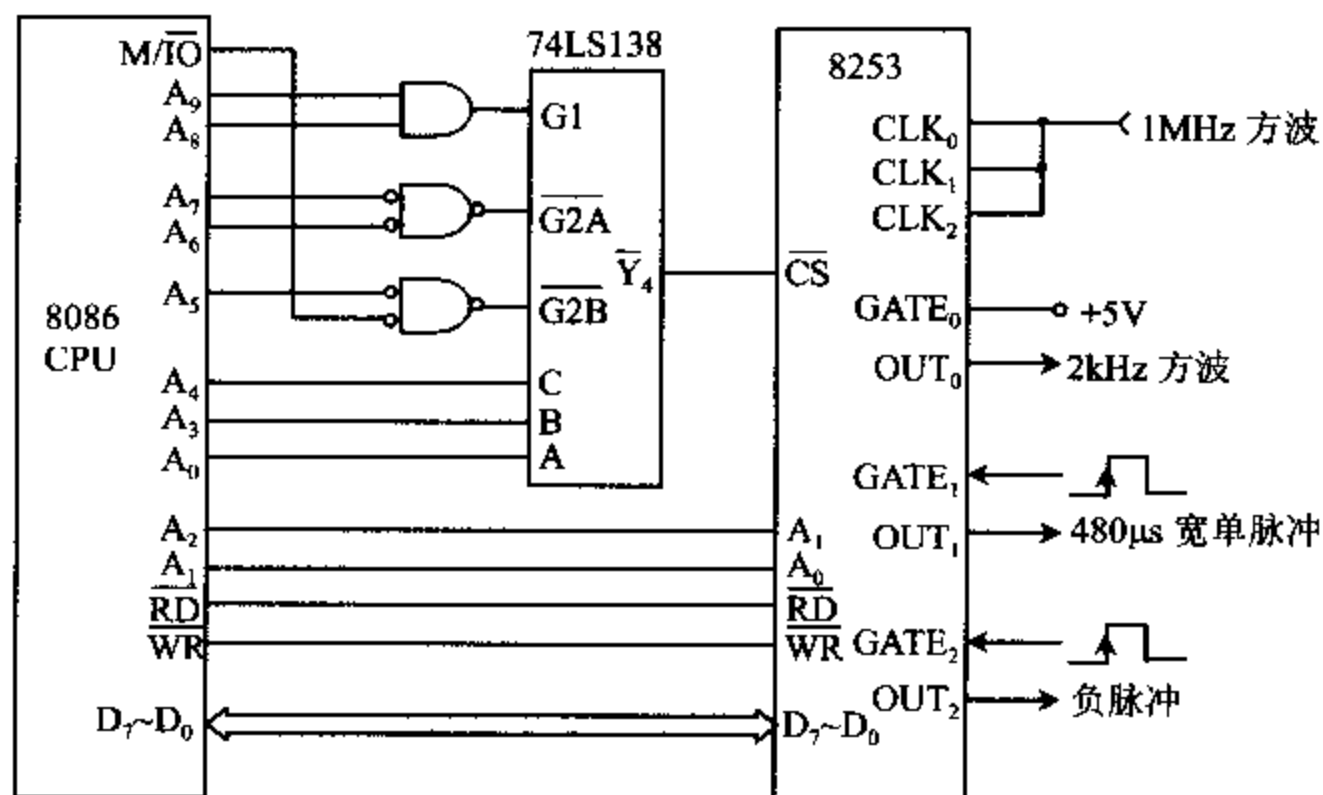


图 8-10 8253 定时波形产生电路

构成的地址译码电路产生,只有当执行 I/O 操作(即 $M/\overline{IO}$ 为低)时以及 $A_9 A_8 A_7 A_6 A_5 = 11000$ 时,译码器才能工作。当 $A_4 A_3 A_0 = 100$ 时, $\overline{Y_4} = 0$,使 8253 的片选信号 $\overline{CS}$ 有效,选中偶地址端口,端口基地址值为 310H。CPU 的 $A_2 A_1$ 分别与 8253 的 $A_1 A_0$ 相连,用于 8253

芯片内部寻址,使 8253 的 4 个端口地址分别为 310H、312H、314H 和 316H。8253 的 8 根数据线 $D_7 \sim D_0$ 必须与 CPU 的低 8 位数据总线 $D_7 \sim D_0$ 相连。另外,8253 的 $\overline{RD}$ 、 $\overline{WR}$ 脚分别与 CPU 的相应引脚相连。3 个通道的 CLK 引脚连在一起,均由频率为 1MHz(周期为 $1\mu s$)的时钟脉冲驱动。

通道 0 工作于方式 3,即构成一个方波发生器,它的控制端 $GATE_0$ 须接 +5V,为了输出 2kHz 的连续方波,应使时间常数 $N0=1MHz/2kHz=500$ 。

通道 1 工作于方式 1,即构成一个单稳态电路,由 $GATE_1$ 的正跳变触发,输出一个宽度由时间常数决定的负脉冲。此功能一次有效,需要再形成一个脉冲时,不但 $GATE_1$ 脚上要有触发,通道也需重新初始化。需输出宽度为 $480\mu s$ 的单脉冲时,应取时间常数 $N1=480\mu s/1\mu s=480$ 。

通道 2 工作于方式 5,即由 $GATE_2$ 的正跳变触发减 1 计数,在计到 0 时形成一个宽度与时钟周期相同的负脉冲。此后,若 $GATE_2$ 脚上再次出现正跳变,又能产生一个负脉冲。这里假设预置的时间常数为 26。

对 3 个通道的初始化程序如下:

```

;通道 0 初始化程序
MOV  DX, 316H           ;控制口地址
MOV  AL, 00110111B      ;通道 0 控制字,先读写低字节,后高字节,方式 3,BCD 计数
OUT  DX, AL             ;写入方式字
MOV  DX, 310H           ;通道 0 口地址
MOV  AL, 00H            ;低字节
OUT  DX, AL             ;先写入低字节
MOV  AL, 05H            ;高字节
OUT  DX, AL             ;后写入高字节
;通道 1 初始化程序
MOV  DX, 316H           ;控制口地址
MOV  AL, 01110011B      ;通道 1 方式字,先读写低字节,后高字节,方式 1,BCD 计数
OUT  DX, AL             ;写入方式字
MOV  DX, 312H           ;通道 1 口地址
MOV  AL, 80H            ;低字节
OUT  DX, AL             ;先写入低字节
MOV  AL, 04H            ;高字节
OUT  DX, AL             ;后写入高字节
;通道 2 初始化程序
MOV  DX, 316H           ;控制口地址
MOV  AL, 10011011B      ;通道 2 控制字,只读写低字节,方式 5,BCD 计数
OUT  DX, AL             ;写入方式字
MOV  DX, 314H           ;通道 2 口地址
MOV  AL, 26H            ;低字节
OUT  DX, AL             ;只写入低字节

```

2. 控制 LED 的点亮或熄灭

8253 的计数和定时功能,可以应用到自动控制、智能仪器仪表、科学实验、交通管理等

许多场合。例如,工业控制现场数据的巡回检测,A/D 转换器采样率的控制,步进马达转动的控制,交通灯开启和关闭的定时,医疗监护仪器中参数超限报警器音调的控制等等。下面是一个用 8253 来控制一个 LED 发光二极管的点亮和熄灭的例子,要求点亮 10 秒钟后再让它熄灭 10 秒钟,并重复上述过程。加上适当的驱动电路后,便可以用在交通红绿灯控制和灯塔等场合。

假设这是一个 8086 系统,8253 的各端口地址为 81H、83H、85H 和 87H。图 8-11 是其硬件电路。8253 的 8 根数据线 $D_7 \sim D_0$ 与 CPU 的高 8 位数据线 $D_{15} \sim D_8$ 相连,这样才能选中奇地址端口。通道 1 的 OUT_1 与 LED 相连,当它为高电平时,LED 点亮,低电平时,LED 熄灭。只要对 8253 编程,使 OUT_1 输出周期为 20 秒,占空比为 1:1 的方波,就能使 LED 交替地点亮和熄灭 10 秒钟。若将频率为 2MHz(周期为 $0.5\mu s$)的时钟直接加到 CLK_1 端,则 OUT_1 输出的脉冲周期最大只有 $0.5\mu s \times 65536 = 32768\mu s = 32.768ms$,达不到 20 秒的要求。为此,需用几个通道级连的方案来解决这个问题。

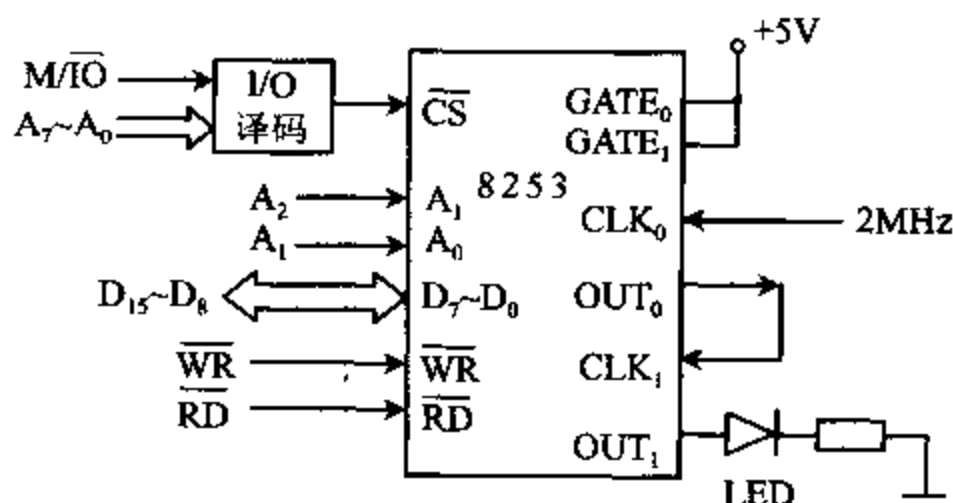


图 8-11 用 8253 控制 LED 点亮或熄灭

如图所示,将频率为 2MHz 的时钟信号加在 CLK_0 输入端,并让通道 0 工作于方式 2。若选择计数初值 $N0 = 5000$,则从 OUT_0 端可得到序列负脉冲,其频率为 $2MHz/5000 = 400Hz$,周期为 2.5ms。再把该信号连到 CLK_1 输入端,并使通道 1 工作于方式 3。为了使 OUT_1 输出周期为 20 秒(频率为 $1/20 = 0.05Hz$)的方波,应取时间常数 $N1 = 400Hz/0.05Hz = 8000$ 。

初始化程序如下:

```
MOV AL, 00110101B      ;通道 0 控制字,先读写低字节,后高字节,方式 2,BCD 计数
OUT 87H, AL
MOV AL, 00H             ;计数初值低字节
OUT 81H, AL
MOV AL, 50H             ;计数初值高字节
OUT 81H, AL

;
MOV AL, 01110111B      ;通道 1 控制字,先读写低字节,后高字节,方式 3,BCD 计数
OUT 87H, AL
MOV AL, 00H             ;计数初值低字节
OUT 83H, AL
```



```
MOV AL, 80H
OUT 83H, AL
```

;计数初值高字节

二、8253 计数功能的应用例子

8253 可以用于各种需要进行计数的场合。下面，我们用一个具体的例子来说明它在这方面的应用。假设一个自动化工厂需要统计在流水线上所生产的某种产品的数量，可采用 8086 微处理器和 8253 等芯片来设计实现这种自动计数的系统。下面介绍这种自动计数系统的电路和控制软件的设计方法。

1. 硬件电路设计

这个自动计数系统由 8086 CPU 控制，用 8253 作计数器。此外，还要用到一片 8259A 中断控制器芯片和若干其它电路。图 8-12 仅给出了计数器部分的电路图，8086 和 8259A 未画在图上。

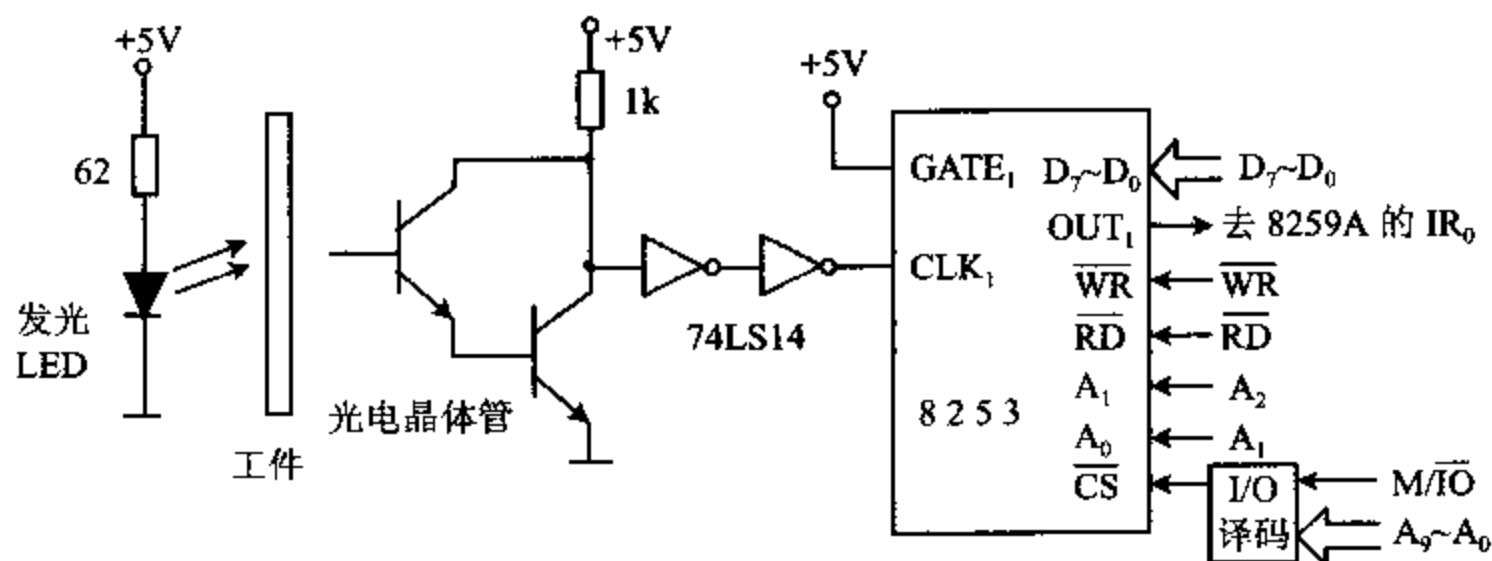


图 8-12 对工件进行计数的电路

电路由一个红外 LED 发光管、一个复合型光电晶体管、两个施密特触发器 74LS14 及一片 8253 芯片等构成。用 8253 的通道 1 来进行计数，工作过程如下：

当 LED 发光管与光电管之间无工件通过时，LED 发出的光能照到光电管上，使光电晶体管导通，集电极变为低电平。此信号经施密特触发器驱动整形后，送到 8253 的 CLK₁，使 8253 的 CLK₁ 输入端也变成低电平。当 LED 与光电管之间有工件通过时，LED 发出的光被它挡住，照不到光电管上，使光电管截止，其集电极输出高电平，从而使 CLK₁ 端也变成高电平。待工件通过后，CLK₁ 端又回到低电平。这样，每通过一个工件，就从 CLK₁ 端输入一个正脉冲，利用 8253 的计数功能对此脉冲计数，就可以统计出工件的个数来。两个施密特触发反相器 74LS14 的作用，是将光电晶体管集电极上的缓慢上升信号，变换成满足计数电路要求的 TTL 电平信号。

8253 的片选输入 $\overline{CS}$ 端接到 I/O 端口地址译码器的一个输出端， $\overline{RD}$ 和 $\overline{WR}$ 端分别与 CPU 的 $\overline{RD}$ 和 $\overline{WR}$ 信号相连。8253 的数据线 D₇~D₀ 与 CPU 的低 8 位地址线相连，如前所述，这时 I/O 端口地址必须是偶地址，所以把 A₁ 和 A₀ 分别与 CPU 地址总线的 A₂ 和 A₁ 相连。8253 通道 1 的门控输入端 GATE₁ 接 +5V 高电平，即始终允许计数器工作。通道 1 的输出端 OUT₁ 接到 8259A 的一个中断请求输入端 IR₀。

2. 初始化编程

硬件电路设计好后,还必须对 8253 进行初始化编程,计数电路才能工作。编程时,可选择计数器 1 工作于方式 0,按 BCD 码计数,先读/写低字节,后读/写高字节,根据图 8-3 可得到控制字为 01110001B。如选取计数初值 $n=499$,则经过 $n+1$ 个脉冲,也就是 500 个脉冲,OUT₁ 端输出一个正跳变。它作用于 8259A 的 IR₀ 端,通过 8259A 的控制,向 CPU 发出一次中断请求,表示计满了 500 个数,在中断服务程序中使工件总数加上 500。中断服务程序执行完后,返回主程序,这时需要由程序把计数初值 499 再次装入计数器 1,才能继续进行计数。

设 8253 的 4 个端口地址分别为 F0H, F2H, F4H 和 F6H,则初始化程序为:

```
MOV AL, 01110001B    ;控制字
OUT 0F6H, AL
MOV AL, 99H
OUT 0F2H, AL          ;计数值低字节送计数器 1
MOV AL, 04H
OUT 0F2H, AL          ;计数值高字节送计数器 1
```

这种计数方案也可用于其它需要计数的地方,如统计在高速公路上行驶的车辆数、统计进入工厂的人数等场合。

3. 计数值的读取

在许多用到 8253 的计数功能的场合,常常需要读取计数器的现行计数值。例如,还是上面提到的自动化工厂里的生产流水线上,要对生产的工件进行自动装箱。若每个包装箱能装 1000 个工件,在装满之后,就移走箱子,并通知控制系统开始对下一个包装箱装箱。这时,可用 8253 计数器对进入包装箱的工件进行计数。计数器从初值 $n=999$ 开始计数,每通过一个工件,计数器就减 1。当计数器减为 0 时,向 CPU 发中断请求,通知控制系统自动移走箱子。

上述系统只有在计数器计满 1000 后,才会转到中断服务程序中去累计工件数。如果在箱子尚未装满时,想了解箱子中已装了多少个工件,可通过读取计数器的现行值来实现。这时,可先从计数器中读取现行的计数值,再用 1000 减去现行值,就可求得当前装入箱中的工件数。

在读计数器现行值时,计数过程仍在进行,而且不受 CPU 的控制。因此,在 CPU 读取计数器的输出值时,可能计数器的输出正在发生改变,即数值不稳定,可能导致错误的读数。为了防止这种情况发生,必须在读数前设法终止计数或将计数器输出端的现行值锁存。这可以采用下面两种方法:

一种方法是在读数前用外部硬件切断计数脉冲信号,或者使门控信号变为低电平,迫使 8253 停止计数。这种方法的缺点是需要硬件电路配合。此外,由于外部事件源被切断或正常的计数过程被禁止,干扰了实际的计数过程。因此,这不是一种好的方法,在我们这个例子里,就不宜采用这种读数方法。

另一种方法是先用计数器锁存命令锁存现行计数值,然后将它读出。如前所述,在每个

计数通道中都有一个 16 位的输出锁存器,可以在任何时刻将计数器的现行值锁住。当需要读取计数器的现行值时,先向 8253 送一个控制字,并使控制字中的 $RL_1 RL_0 = 00$,这表示向 8253 发了一个锁存命令,现行计数值立即被锁存起来。接下来,就可从相应的计数器通道中读取计数值。该控制字中的 $SC_1 SC_0$ 用来确定要锁存的是 3 个计数器中的哪一个。控制字的低 4 位对锁存命令无影响,可以将它们置为 0。读取计数值的方法由对 8253 进行初始化编程时所写入的控制字中的 $RL_1 RL_0$ 位来确定,当 $RL_1 RL_0 = 01$ 时,只读取计数器的低字节, $RL_1 RL_0 = 10$ 时,只读取计数器的高字节, $RL_1 RL_0 = 11$ 时,先读写计数器低字节,后读写高字节。

比较起来,第二种方法完全由软件实现,并可随时读取计数值,而且不会干扰正常的计数过程和引起错误,是常用的方法。上例中,在要读取箱子中的现行工件数时,可执行下面的程序段:

```
MOV    AL, 01000000B    ;锁存计数器 1 命令
MOV    DX, 0F6H         ;控制口
OUT    DX, AL           ;发锁存命令
MOV    DX, 0F2H         ;计数器 1
IN     AL, DX            ;读取计数器 1 的低 8 位数
MOV    AH, AL           ;保存低 8 位数
IN     AL, DX            ;读取计数器 1 的高 8 位数
XCHG   AH, AL           ;将计数值置于 AX 中
```

由于在上述程序执行前,对 8253 进行初始化编程时,已将计数器 1 置为先读/写低 8 位数,后读/写高 8 位数,所以,程序可以根据这样的次序连续读取 2 个字节的数据。如对计数器初始化为只读/写低 8 位或高 8 位数,则只允许读取一个字节。

在计数器的锁存命令发出后,锁存的计数值将保持不变,直至被读出为止。计数值从锁存器读出后,数值锁存状态即被自动解除,输出锁存器的值又将随计数器的值而变化。

利用这种方法读取 8253 的计数器值时,每执行一次锁存命令,只能锁存一个通道的计数值。如果想读取 8253 的 3 个计数器的值,就要向 8253 送 3 个锁存命令字。同样,用这种方法也可以读取 8254 的内部计数器的数值。但对于 8254 来说,还有另外一种读回功能,一次可以锁存多个计数器的值,从而可连续读取 1~3 个计数器的值。

4. 8254 的读回功能

当利用 8254 的读回(Read Back)命令功能,向 8254 的控制字寄存器写入一个读回命令字时,每次可锁存 1~3 个通道的计数值。此外,利用 8254 的读回功能,还可锁存 1~3 个计数通道的状态字,供 CPU 读取。通过读取状态字,可以核对向 8254 写入的控制字是否正确,还能了解当前输出引脚的电平状态,以及计数值是否已写入执行单元等。

8254 的读回命令的格式如图 8-13 所示。其中 $D_7 D_6$ 位为标志位,必须等于 11(8253 无此功能),用来表明这是读回命令字。 D_6 位为 0。 D_5 位和 D_4 位分别用来决定是否要锁存计数器的数值及状态位的信息。 $D_3 \sim D_0$ 位用来选择计数通道号。

当 D_5 位置 0 时,将锁存计数器的计数值。至于是锁存哪一个计数器的值,由 $D_3 \sim D_1$ 位来决定。 D_3 位等于 1,表示锁存计数器 2 的值, D_3 或 D_1 等于 1,则表示锁存计数器 1 或计

数器 0 的值。这样,通过 $D_3 \sim D_1$ 位不同组合,可同时锁存一个、两个或三个计数器的值。计数器的值被锁存后,就能用前面介绍的读取 8253 计数值的类似方法来读取 8254 的计数值。该值被读出后,锁存器的输出又随计数的输出而变了。

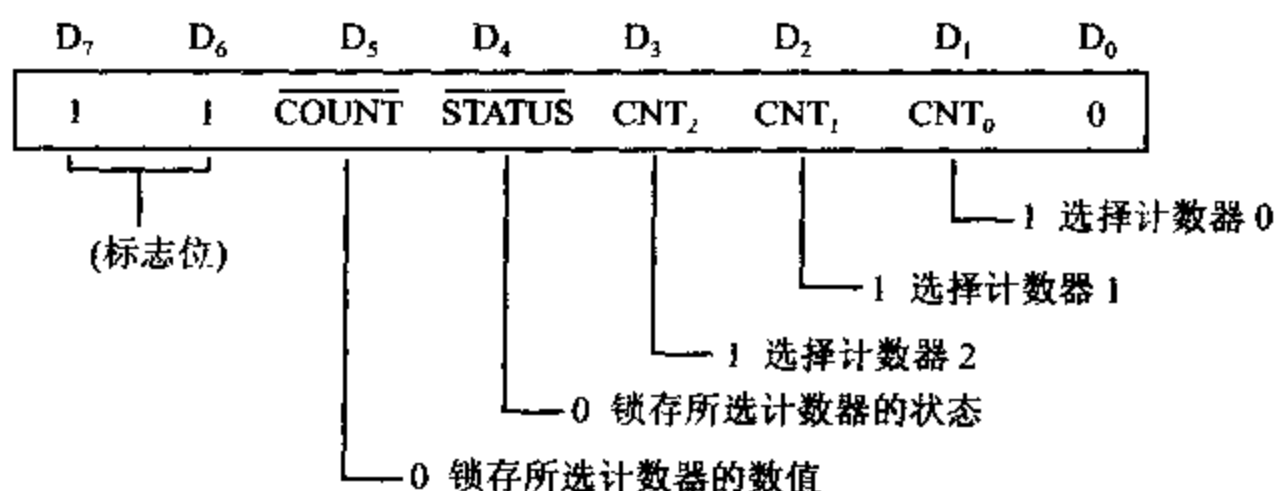


图 8-13 8254 的读回命令字格式

若 D_4 位置 0,将锁存计数器的状态信息。状态信息被锁存后,也可以由 CPU 用输入指令读回。用户通过读取状态信息,可核查所选中通道的计数值、工作方式、输出引脚 OUT 的现行状态及计数器是否已写入计数通道等信息。状态字的格式如图 8-14 所示。其中 $D_7 \sim D_0$ 位即为写入该通道的控制字的相应部分, RW_1RW_0 相当于 8253 的 RL_1RL_0 位。具体意义如下:

RW_1RW_0 ——读/写操作位,反映对该通道的计数器所设置的读/写操作方式。

BCD ——反映通道所设置的计数方式。

$M_2M_1M_0$ ——反映通道所设置的工作方式。

D_7 ——通道输出状态位。当 $D_7=1$ 时,表示输出高电平, $D_7=0$ 时,输出为低电平。

D_6 ——无效计数位(NULL COUNT),反映计数值是否已写入计数器执行单元。当向通道写入控制字和计数值后, $D_6=1$;当计数值写入计数器执行单元后, $D_6=0$ 。

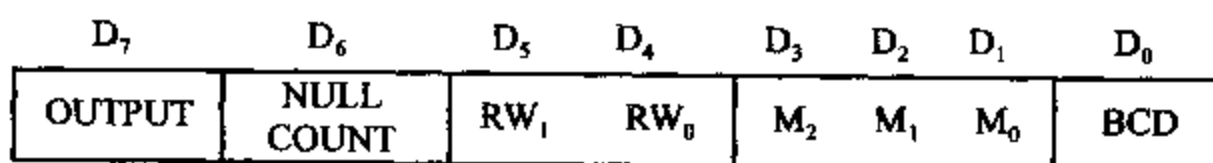


图 8-14 8254 的计数状态字

三、8253 在 PC/XT 机中的应用

在 PC/XT 机中,使用 8253-5 作计数器/定时器电路,8253-5 与 8253 的引脚和功能完全一致,仅在有些性能指标方面略高于 8253。图 8-15 是 8253-5 在 IBM PC/XT 机中的连线图。

从图 8-15 可以看到,8253-5 的 $\overline{RD}$ 、 $\overline{WR}$ 信号与系统中相应的控制信号相连, A_1A_0 与地址总线的对应端相连, $D_7 \sim D_0$ 与系统的 8 位数据总线相连,片选信号 $\overline{CS}$ 与 I/O 译码器的输出信号 $\overline{T/C} \overline{CS}$ 相连,地址在 40H~5FH 范围内均有效($A_9 \sim A_5 = 00010$)。ROM BIOS 访问 8253-5 时,内部 3 个计数器的端口地址为 40H、41H、42H,控制字寄存器的端口地址为

43H。外部时钟信号 PCLK 由 8284A 时钟发生器产生,其频率为 2.38636MHz,经 U21 二分频后,形成频率为 $f=1.19318\text{MHz}$ 的脉冲信号,作为 3 路计数器的输入时钟。8253-5 的 3 个计数器都有专门的用途,下面分别介绍它们的使用情况。

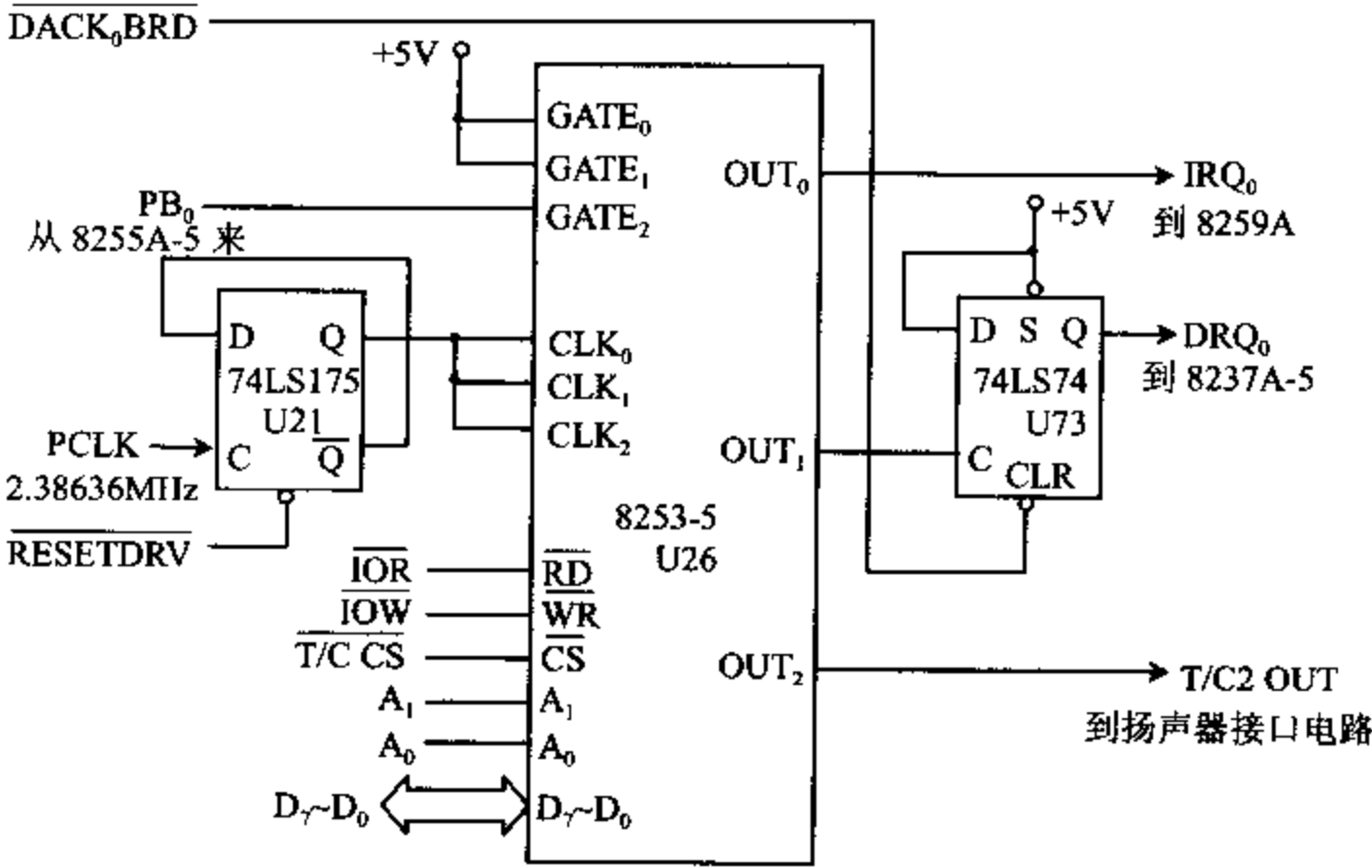


图 8-15 8253-5 在 PC/XT 机中的连线图

1. 计数器 0 —— 实时时钟

计数器 0 用作定时器, $GATE_0$ 接 +5V, 使计数器 0 处于常开状态, 开机初始化后, 它就一直处于计数工作状态, 为系统提供时间基准。在对计数器 0 进行始化编程时, 选用方式 3 (方波发生器), 二进制计数。对计数器预置的初值 $n=0$, 相当于 $2^{16}=65536$, 这样在输出端 OUT_0 可以得到序列方波, 其频率为 $f/n=1.19318\text{MHz}/65536=18.2\text{Hz}$ 。它经系统板上的总线 IRQ_0 被直接送到 8259A 中断控制器的中断请求端 IR_0 , 使计算机每秒钟产生 18.2 次中断, 也就是每隔 55 毫秒请求一次中断。CPU 可以此作为时间基准, 在中断服务程序中对中断次数进行计数, 就可形成实时时钟。例如中断 100 次, 时间间隔即为 5.5 秒。这对于时间精度要求不是非常高的场合是很有用的。

如果用一个 16 位的计数器对中断次数进行计数, 每中断一次, 计数器加 1, 当 16 位的计数器计满后产生进位时, 表示产生了 65536 次中断, 所经过的时间为 $65536/18.2=3600\text{秒}=1\text{小时}$ 。

对 8253 的计数器 0 进行初始化编程的程序为:

MOV AL, 00110110B	;控制字:通道 0,先写低字节,后高字节,方式 3,2 进制计数
OUT 43H, AL	;写入控制字
MOV AX, 0000H	;预置计数值 $n=65536$
OUT 40H, AL	;先写低字节
MOV AL, AH	
OUT 40H, AL	;后写高字节

2. 计数器 1 —— 动态 RAM 刷新定时器

计数器 1 的 $GATE_1$ 也接 +5V, 使计数器 1 也处于常开状态, 它定时向 DMA 控制器提供动态 RAM 刷新请求信号。初始化编程时, 设置成方式 2 (比率发生器), 计数器预置的初值为 18。这样, 从 OUT_1 端可输出负脉冲序列, 其频率为 $1.19318\text{MHz}/18 = 66.2878\text{kHz}$, 周期为 $15.09\mu\text{s}$ 。 OUT_1 输出的负脉冲的上升沿使 D 触发器 U_{73} 置 1, 从 Q 端输出 DRQ_0 信号, 它被送到 DMA 控制器 8237A-5 的 $DREQ_0$ 端, 作为通道 0 的 DMA 请求信号。在通道 0 执行 DMA 操作时对动态 RAM 进行刷新, 8237A-5 的回答信号 $\overline{DACK_0BRD}$ 使 D 触发器 U_{73} 清 0, 这样每隔 $15.09\mu\text{s}$ 向 8237A-5 DMA 控制器提出一次 DMA 请求, 由 DMA 控制器实施对动态 RAM 的刷新操作。

初始化计数器 1 的程序为:

```
MOV AL, 01010101B      ;控制字:计数器 1,只写低字节,方式 2,BCD 计数
OUT 43H, AL             ;写入控制字
MOV AL, 18H             ;预置初值 BCD 数 18
OUT 41H, AL             ;送入低字节
```

3. 计数器 2 —— 扬声器音调控制

计数器 2 工作于方式 3, 对计数器预置的初值为 $n = 533\text{H} = 1331$, 故从 OUT_2 输出的方波频率为 $1.19318\text{MHz}/1331 = 896\text{Hz}$ 。但该计数器的 $GATE_2$ 不是接 +5V, 而是受并行接口芯片 8255A-5 的 PB_0 端控制, 因此它不是处于常开状态。当 PB_0 端送来高电平时, 允许计数器 2 计数, 使 OUT_2 端输出方波。该方波与 8255A-5 的 PB_1 信号相与后, 送到扬声器驱动电路, 驱动扬声器发声。发声的频率由预置的初值 n 决定, 发声时间的长短受 PB_1 控制, 当 $PB_1 = 1$ 时, 允许发声, 当 $PB_1 = 0$ 时, 禁止发声。通过控制 PB_1 与 PB_0 的电平, 就可以发出各种不同音调的声音。由于 8255A-5 还控制其它设备, 所以在控制扬声器发声的程序中, 还必须保护 PB 端口原来的状态, 这样就不会影响其它设备的工作。

初始化计数器 2 的程序为:

```
MOV AL, 10110110B      ;控制字:计数器 2,先写低字节,后高字节,方式 3,2 进制计数
OUT 43H, AL             ;写入控制字
MOV AX, 533H            ;预置初值 n=533H
OUT 42H, AL             ;先送出低字节
MOV AL, AH              ;后送出高字节
OUT 42H, AL             ;后送出高字节
IN AL, PORT B           ;取 8255A B 口的当前值(口地址为 61H)
MOV AH, AL              ;保存该端口的值
OR AL, 03H              ;使  $PB_1$  和  $PB_0$  均置 1
OUT PORT-B, AL          ;接通扬声器
```

以上程序使扬声器发出单一频率(896Hz)的声音。

习 题

1. 8253 芯片有哪几个计数通道? 每个计数通道可工作于哪几种工作方式? 这些操作方式的主要特点是什么?

2. 8253 的最高工作频率是多少? 8254 与 8253 的主要区别是什么?

3. 对 8253 进行初始化编程分哪几步进行?

4. 设 8253 的通道 0~2 和控制端口的地址分别为 300H、302H、304H 和 306H, 定义通道 0 工作在方式 3, $CLK_0 = 2MHz$ 。试编写初始化程序, 并画出硬件连线图。要求通道 0 输出 1.5kHz 的方波, 通道 1 用通道 0 的输出作计数脉冲, 输出频率为 300Hz 的序列负脉冲, 通道 2 每秒钟向 CPU 发 50 次中断请求。

5. 8253 芯片用于某计数系统中, 设计数初值为 500, 每当计数值减为 0 时, 通过 8259A 向 CPU 发中断请求, 中断类型号为 0AH, 当计数计到 2650 时停止计数。试编程求出总的计数值, 并把它送到 AX 寄存器中。

6. 某微机系统中, 8253 的端口地址为 40H~43H, 时钟频率为 5MHz。要求通道 0 输出方波, 使计算机每秒钟产生 18.2 次中断; 通道 1 每隔 $15\mu s$ 向 8237A 提出一次 DMA 请求; 通道 2 输出频率为 2000Hz 的方波。试编写 8253 的初始化程序, 并画出有关的硬件连线图。

7. 设某系统中 8254 芯片的基地址为 F0H, 在对 3 个计数通道进行初始化编程时, 都设为先读写低 8 位, 后读写高 8 位, 试编程完成下列工作:

(1) 对通道 0~2 的计数值进行锁存并读出来。

(2) 对通道 2 的状态值进行锁存并读出来。

第九章 可编程外围接口芯片 8255A 及其应用

8255A 是一种通用的可编程并行 I/O 接口芯片 (Programmable Peripheral Interface, PPI), 它是为 Intel 系列微处理器设计的配套电路, 也可用于其它微处理器系统中。通过对它进行编程, 芯片可工作于不同的工作方式。在微型计算机系统中, 用 8255A 作接口时, 通常不需要附加外部逻辑电路就可直接为 CPU 与外设之间提供数据通道, 因此它得到了极为广泛的应用。本章先介绍 8255A 的基本工作原理, 然后通过实例详细说明 8255A 在开关电路、键盘、七段 LED 显示器、打印机等接口电路及 PC/XT 中的应用。我们可以利用它们来解决工作中遇到的实际问题。此外 8255A 还可用于软盘接口电路、CRT 控制接口电路、A/D 和 D/A 接口电路等许多场合。

9-1 8255A 的工作原理

一、8255A 的结构和功能

8255A 的外部引脚和内部结构分别如图 9-1 和图 9-2 所示。

由图 9-2 可见, 8255A 由以下几个部分组成: 数据端口 A、B、C (其中 C 口被分成 C 口上半部分和 C 口下半部分两个部分), A 组和 B 组控制逻辑, 数据总线缓冲器和读/写控制逻辑。各组成部分及有关引脚的功能分述如下:

1. 数据端口 A、B 和 C

8255A 内部包含 3 个 8 位的输入输出端口 A、B 和 C, 通过外部的 24 根输入输出线与外设交换数据或进行通信联络。端口 A 和端口 B 都可以用作一个 8 位的输入口或 8 位的输出口, C 口既可以作为一个 8 位的输入口或输出口用, 又可作为两个 4 位的输入输出口 (C 口上半部分和 C 口下半部分) 使用, 还常常用来配合 A 口和 B 口工

作, 分别用来产生 A 口和 B 口的输出控制信号和输入 A 口和 B 口的端口状态信号。

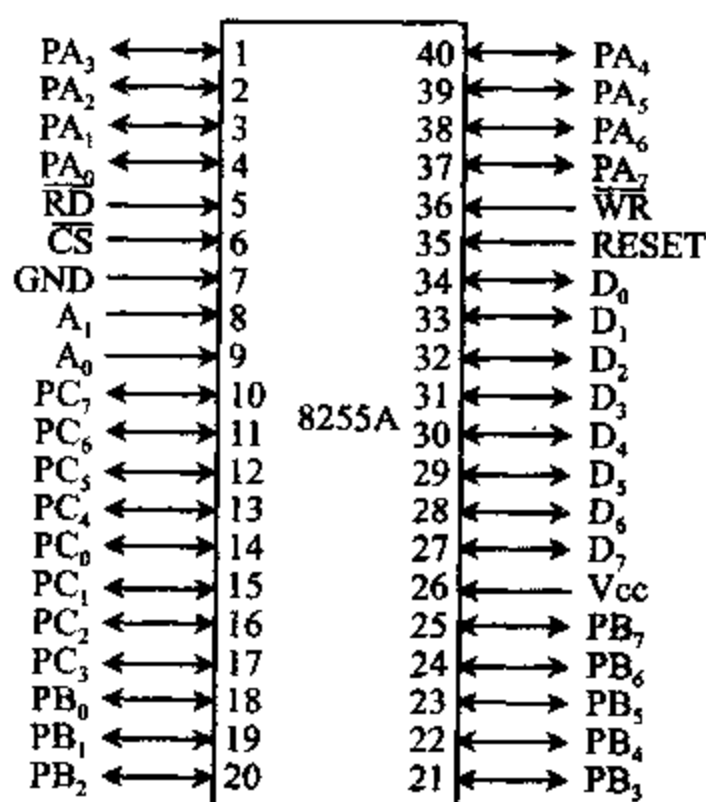


图 9-1 8255A 的引脚

各端口在结构和功能上有不同的特点:

端口 A 包含一个 8 位的数据输出锁存器/缓冲器, 一个 8 位的数据输入锁存器, 因此 A 口作输入或输出时数据均能锁存。

端口 B 包含一个 8 位的数据输入/输出锁存器/缓冲器, 一个 8 位的数据输入缓冲器。

端口 C 包含一个 8 位的数据输出锁存器/缓冲器, 一个 8 位的数据输入缓冲器, 无输入锁存功能, 当它被分成两个 4 位端口时, 每个端口有一个 4 位的输出锁存器。

与 3 个端口相连的 24 根输入输出引线分别是 $PA_7 \sim PA_0$ 、 $PB_7 \sim PB_0$ 和 $PC_7 \sim PC_0$, 这些线都与外部设备相连, 具体作用与端口的工作方式有关。

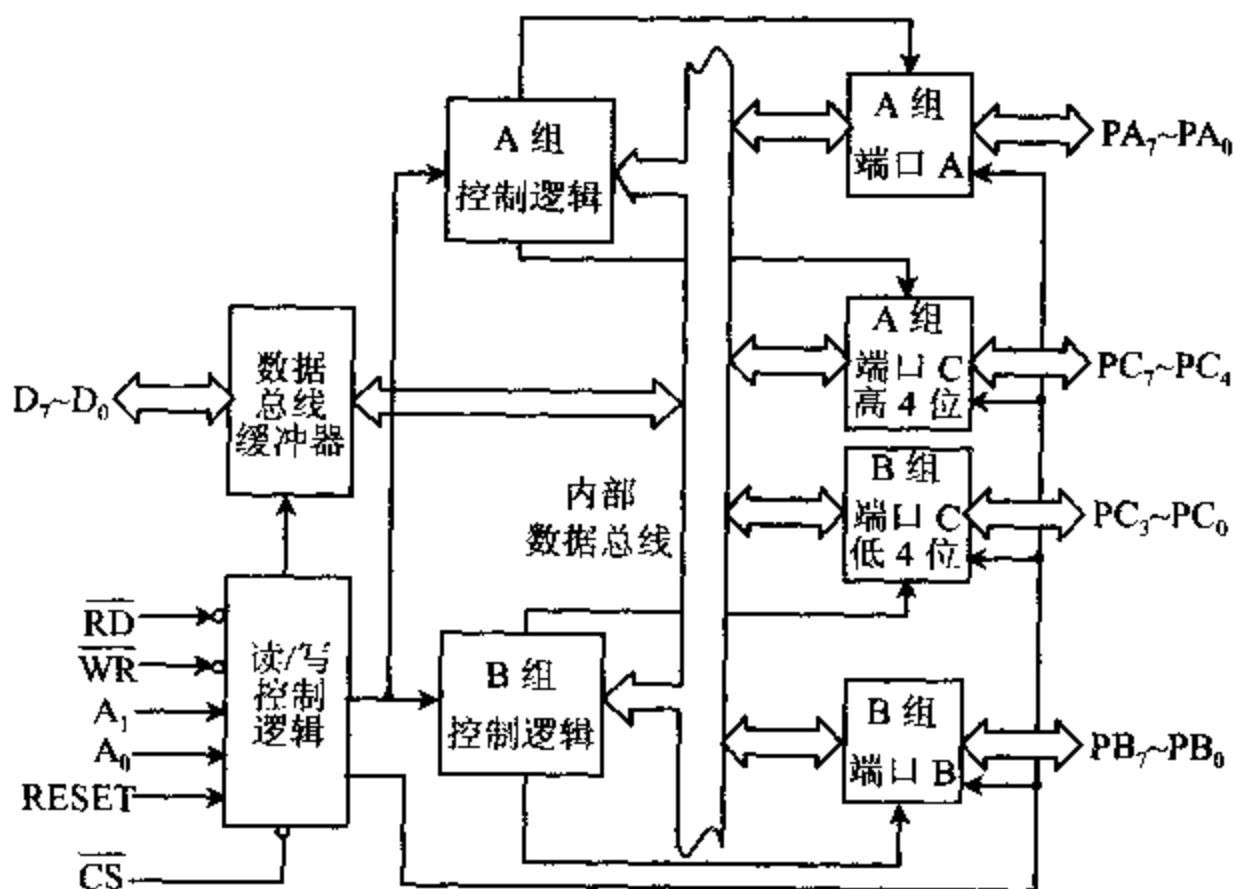


图 9-2 8255A 的内部结构

2. A 组和 B 组控制逻辑

这是两组根据 CPU 的编程命令控制 8255A 工作的电路。它们内部有控制寄存器, 用来接收 CPU 送来的命令字, 然后分别决定 A 组和 B 组的工作方式, 或对端口 C 的每一位执行置位/复位等操作。

8255A 的端口 A 和端口 C 的上半部分 ($PC_7 \sim PC_4$) 由 A 组控制逻辑管理, 端口 B 和端口 C 的下半部分 ($PC_3 \sim PC_0$) 由 B 组控制逻辑管理。这两组控制逻辑都从读/写控制逻辑接受命令信号, 从内部数据总线接收控制字, 然后向各有关端口发出相应的控制命令。

3. 数据总线缓冲器

这是一个双向三态的 8 位缓冲器, 用作 8255A 和系统数据总线之间的接口。通过这个缓冲器和与之相连的 8 位数据总线 $D_7 \sim D_0$, 接收 CPU 送来的数据或控制字, 外设传送给 CPU 的数据或状态信息, 也要通过这个数据总线缓冲器送给 CPU。

4. 读/写控制逻辑

这部分电路用来管理所有的内部或外部数据信息、控制字或状态字的传送过程。它接收从 CPU 的地址总线和控制总线来的信号, 并产生对 A 组和 B 组控制逻辑进行操作的控

制信号。

系统送到读/写控制逻辑的信号包括：

RESET 复位信号，高电平有效。该信号有效时，将 8255A 控制寄存器内容都清零，并将所有的端口(A、B 和 C)都置成输入方式。

$\overline{CS}$ 片选信号，低电平有效，由地址总线经 I/O 端口译码电路产生。只有当该信号有效时，CPU 与 8255A 之间才能进行通信，也就是 CPU 可对 8255A 进行读/写等操作。

$\overline{RD}$ 读信号，低电平有效。当 $\overline{RD}$ 为低时，CPU 可从 8255A 读取数据或状态信息。

$\overline{WR}$ 写信号，低电平有效。当 $\overline{WR}$ 有效时，CPU 可向 8255A 写入数据或控制字。

$A_1 A_0$ 端口选择信号。在 8255A 内部有 3 个数据端口(A、B、C)和一个控制字寄存器端口。当 $A_1 A_0 = 00$ 时，选中端口 A； $A_1 A_0 = 01$ 时，选中端口 B； $A_1 A_0 = 10$ 时，选中端口 C； $A_1 A_0 = 11$ 时，选中控制字寄存器端口。

8255A 的 $A_1 A_0$ 和 $\overline{RD}$ 、 $\overline{WR}$ 、 $\overline{CS}$ 组合起来实现的各种基本操作如表 9-1 所示。

表 9-1 8255A 的基本操作

A_1	A_0	$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	操 作
0	0	0	1	0	端口 A→数据总线
0	1	0	1	0	端口 B→数据总线
1	0	0	1	0	端口 C→数据总线
0	0	1	0	0	数据总线→端口 A
0	1	1	0	0	数据总线→端口 B
1	0	1	0	0	数据总线→端口 C
1	1	1	0	0	数据总线→控制字寄存器
×	×	×	×	1	数据总线三态
1	1	0	1	0	非法状态
×	×	1	1	0	数据总线三态

与 8253 那样，当 8255A 用在 8 位数据总线的微处理器系统中时，端口选择信号输入端 $A_1 A_0$ 分别与地址总线的 $A_1 A_0$ 相连即可；而在 16 位数据总线的系统中，通常将地址总线的 $A_2 A_1$ 连到 8255A 的 $A_1 A_0$ 端。若它的数据线 $D_7 \sim D_0$ 接在 CPU 数据总线的低 8 位上，则要用偶端口地址来寻址 8255A；而当 $D_7 \sim D_0$ 接在数据总线的高 8 位上时，要用奇地址口。

二、8255A 的控制字

8255A 有两类控制字。一类控制字用于定义各端口的工作方式，称为方式选择控制字；另一类控制字用于对 C 端口的任一位进行置位或复位操作，称为置位复位控制字。对 8255A 进行编程时，这两种控制字都被写入控制字寄存器中。但方式选择控制字的 D_7 位总是 1，而置位复位控制字的 D_7 位总是 0。8255A 正是利用这一位来区分这两个写入同一端口的不同控制字的， D_7 位也称为这两个控制字的标志位。下面介绍这两个控制字的具体格式。

1. 方式选择控制字

8255A 具有 3 种基本的工作方式,在对 8255A 进行初始化编程时,应向控制字寄存器写入方式选择控制字,用来规定 8255A 各端口的工作方式。这 3 种基本工作方式是:

方式 0 —— 基本输入输出方式

方式 1 —— 选通输入输出方式

方式 2 —— 双向总线 I/O 方式

当系统复位时,8255A 的 RESET 输入端为高电平,使 8255A 复位,所有的数据端口都被置成输入方式;当复位信号撤除后,8255A 继续保持复位时预置的输入方式。如果希望它以这种方式工作,就不用另外再进行初始化。

通过用输出指令对 8255A 的控制字寄存器编程,写入设定工作方式的控制字,可以让 3 个数据口以不同的方式工作。其中,端口 A 可工作于 3 种方式中的任一种;端口 B 只能工作于方式 0 和方式 1,而不能工作于方式 2;端口 C 常被分成两个 4 位的端口,除了用作输入输出端口外,还能用来配合 A 口和 B 口工作,为这两个端口的输入输出操作提供联络信号。

方式选择控制字的格式如图 9-3 所示。

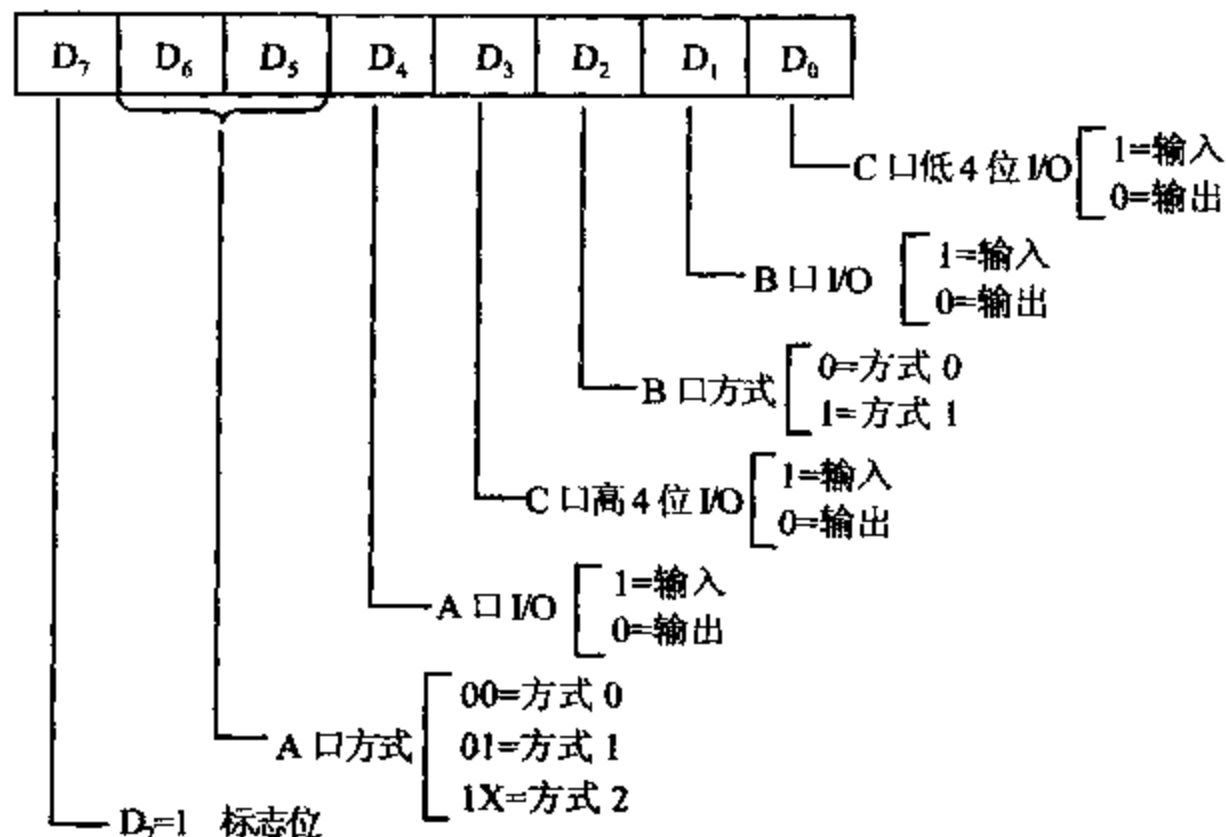


图 9-3 方式选择控制字格式

其中,D₇ 位为标志位,它必须等于 1;D₆D₅ 位用于选择 A 口的工作方式;D₄ 位用于选择 B 口的工作方式;其余 4 位分别用于选择 A 口、B 口、C 口高 4 位和 C 口低 4 位的输入输出功能,置 1 时表示输入,置 0 时表示输出。

2. 置位/复位控制字

端口 C 的数位常用作控制或应答信号,通过对 8255A 的控制口写入置位/复位控制字,可使端口 C 的任意一个引脚的输出单独置 1 或置 0,或者为应答式数据传送发出中断请求信号。在基于控制的应用中,经常希望在某一位上产生一个 TTL 电平的控制信号,利用端口 C 的这个特点,只需要用简单的程序就能形成这样的信号,从而简化了编程。

置位/复位控制字的格式如图 9-4 所示。

D_7 位为置位/复位控制字标志位,它必须等于 0; $D_3 \sim D_1$ 位用于选择对端口 C 中某一位进行操作; D_0 位指出对选中位是置 1 还是清 0。 $D_0 = 1$ 时,使选中位置 1; $D_0 = 0$ 时,使选中位清 0。

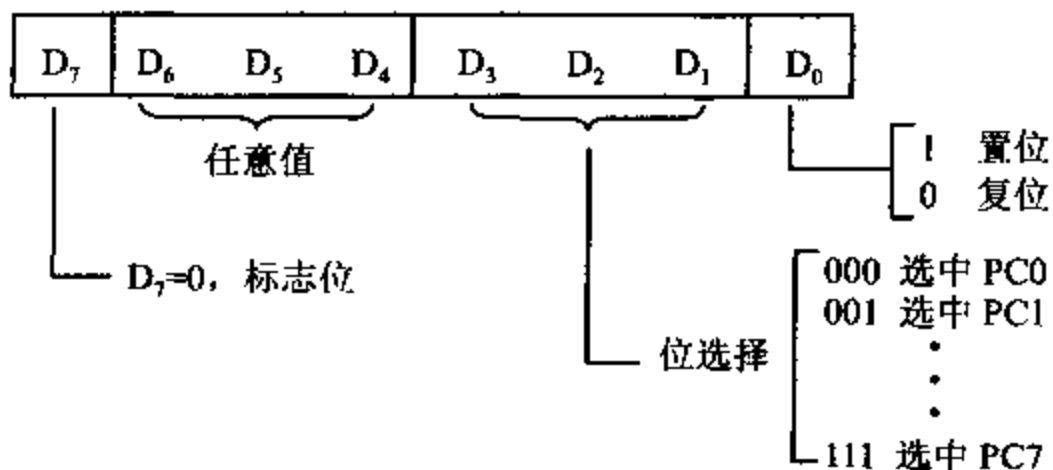


图 9-4 置位/复位控制字格式

例如,设一片 8255A 的口地址为 60H~63H, PC_5 平时为低电平,要求从 PC_5 的引脚输出一个正脉冲。可以用程序先将 PC_5 置 1,输出一个高电平,再把 PC_5 清 0,输出一个低电平,结果, PC_5 引脚上便输出一个正脉冲。实现这个功能的程序段如下:

```
MOV    AL, 00001011B
OUT    63H, AL           ;置  $PC_5$  为高电平
MOV    AL, 00001010B
OUT    63H, AL           ;置  $PC_5$  为低电平
```

三、8255A 的工作方式和 C 口状态字

8255A 具有 3 种工作方式,通过向 8255A 的控制字寄存器写入方式选择字,就可以规定各端口的工作方式。当 8255A 工作于方式 1 和方式 2 时,C 口可用作 A 口或 B 口的联络信号,用输入指令可以读取 C 口的状态。下面具体介绍这 3 种不同的工作方式和 C 口状态字格式。

1. 方式 0

方式 0 称为基本输入输出(Basic Input/Output)方式,它适用于不需要用应答信号的简单输入输出场合。在这种方式下,A 口和 B 口可作为 8 位的端口,C 口的高 4 位和低 4 位可作为两个 4 位的端口。这 4 个端口中的任何一个既可作输入也可作输出,从而构成 16 种不同的输入输出组态。在实际应用时,C 口的两半部分也可以合在一起,构成一个 8 位的端口。这样 8255A 可构成 3 个 8 位的 I/O 端口,或两个 8 位、两个 4 位的 I/O 端口,以适应各种不同的应用场合。

CPU 与这些端口交换数据时,可以直接用输入指令从指定端口读取数据,或用输出指令将数据写入指定的端口,不需要任何其它用于应答的联络信号。对于方式 0,还规定输出信号可以被锁存,输入不能锁存,使用时要加以注意。

如果要使各端口都工作于方式 0, 则方式选择字的格式如图 9-5 所示。

其中, $D_6D_5=00$, 选择 A 口工作于方式 0; $D_6=0$, 选择 B 口工作于方式 0; $D_7=1$ 为标志位; 余下的 D_4D_3 和 D_1D_0 这四位可以任意取 0 或取 1, 由此构成 4 个端口的 16 种不同组态。

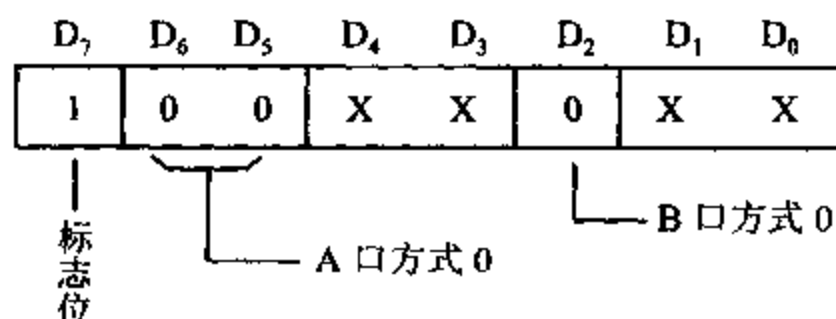


图 9-5 各端口均工作于方式 0 时的控制字

例如, 设 8255A 的控制字寄存器的端口地址为 63H, 若要求 A 口和 B 口工作于方式 0, A 口、B 口和 C 口的上半部分(高 4 位)作输入, C 口的下半部分(低 4 位)为输出, 可用下列指令来设置这种方式:

```
MOV    AL, 10011010B
OUT    63H, AL
```

2. 方式 1

方式 1 也称为选通输入/输出(Strobe Input/Output)方式。在这种方式下, A 口和 B 口作为数据口, 均可工作于输入或输出方式。而且, 这两个 8 位数据口的输入、输出数据都能锁存, 但它们必须在联络(handshaking)信号控制下才能完成 I/O 操作。端口 C 的 6 根线用来产生或接受这些联络信号。

选通输入/输出方式又可分以下几种情况:

(1) 选通输入方式

如果 A 口和 B 口都工作于选通输入方式, 则它们的端口状态、联络信号和控制字如图 9-6 所示。

当 A 口工作于方式 1, 并作输入端口时, 端口 C 的 PC_4 、 PC_5 和 PC_3 用作端口 A 的状态和控制线; 当 B 口工作于方式 1, 并作输入端口时, 端口 C 的 PC_2 、 PC_1 和 PC_0 作端口 B 的状态和控制线。端口 C 还余下两位 PC_6 和 PC_7 , 它们仍可用作输入或输出, 由方式选择控制字中的 D_3 位来定义 PC_6 和 PC_7 的传送方向。 $D_6=1$ 时, PC_6 和 PC_7 作输入; $D_3=0$ 时, PC_6 和 PC_7 作输出。

各控制联络信号的意义分述如下:

$\overline{STB}$ (Strobe) 选通信号, 低电平有效, 由外部输入。

当该信号有效时, 8255A 将外部设备通过端口数据线 $PA_7 \sim PA_0$ (对于 A 口) 或 $PB_7 \sim PB_0$ (对于 B 口) 输入的数据送到所选端口的输入缓冲器中。端口 A 的选通信号 $\overline{STB}_A$ 从 PC_4 引入, 端口 B 的选通信号 $\overline{STB}_B$ 由 PC_2 引入。

IBF(Input Buffer Full) 输入缓冲器满信号, 高电平有效。

这是 8255A 送给外设的状态信号, 当它有效时, 表示输入设备送来的数据已传送到 8255A 的输入缓冲器中, 即缓冲器已满, 8255A 不能再接收别的数据。此信号一般供 CPU

查询用。IBF 由 $\overline{STB}$ 信号所置位, 而由读信号的后沿(也就是上升沿)将其复位, 复位后表示输入缓冲器已空, 又允许外设将一个新的数据送到 8255A。PC₅ 作端口 A 的输入缓冲器满信号 IBF_A, PC₁ 作 B 口的输入缓冲器满信号 IBF_B。

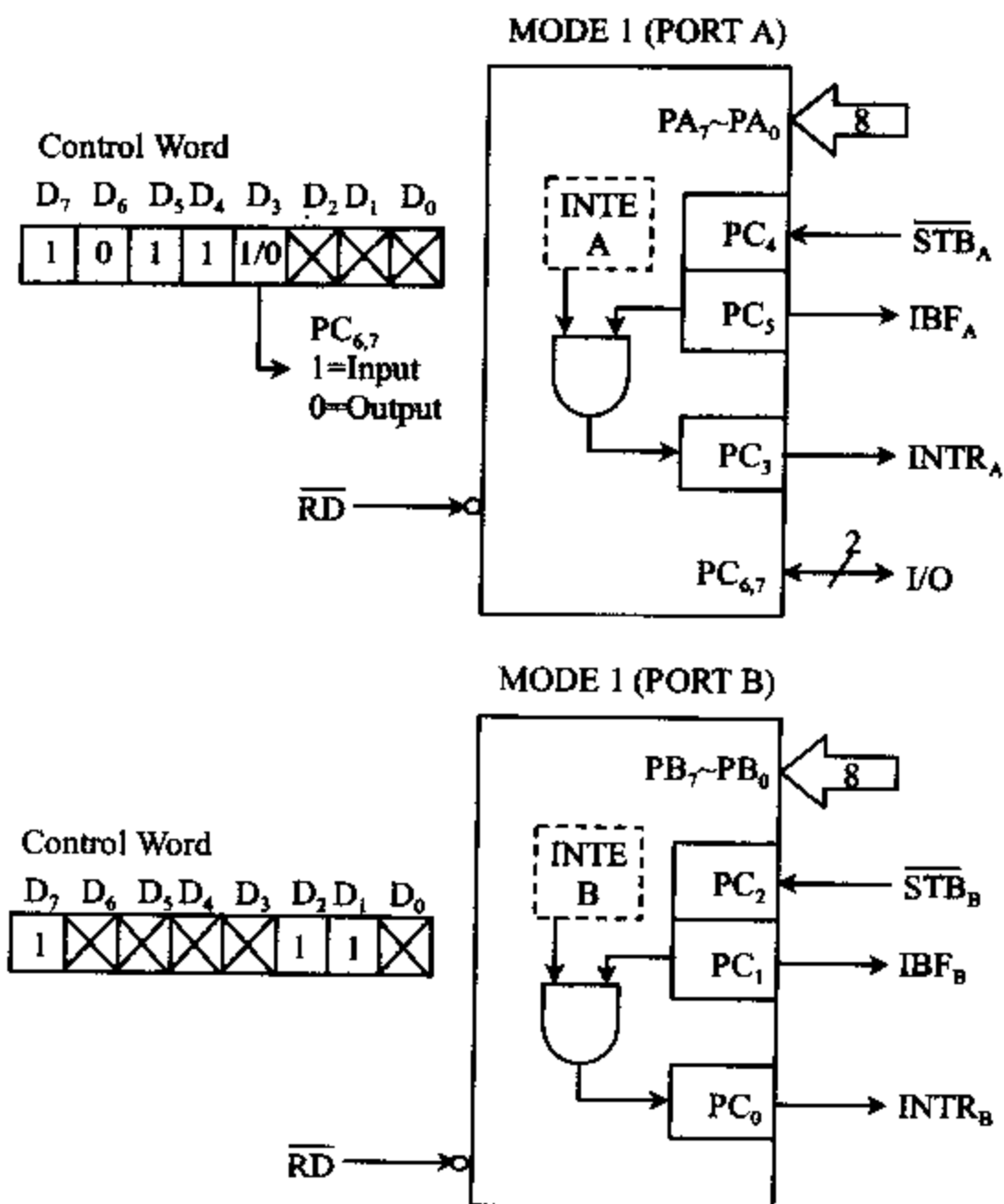


图 9-6 选通输入方式

INTE(Interrupt Enable) 中断允许信号。

这是一个控制 8255A 是否能向 CPU 发中断请求的信号, 它没有外部引出脚。在 A 组和 B 组的控制电路中, 分别设有中断请求触发器 INTE A 和 INTE B, 只有用软件才能使这两个触发器置 1 或清 0。其中 INTE A 由置位/复位控制字中的 PC₄ 位控制, INTE B 由 PC₂ 位控制。当我们对 8255A 写入置位复位控制字使 PC₄ 位置 1 时, INTE A 被置 1, 表示允许 A 口中断; 若使 PC₄ 位清 0, 则禁止 A 口发中断请求, 也就是使 A 口处于中断屏蔽状态。同样, 可以通过编程 PC₂ 位来控制 INTE B, 允许或禁止 B 口中断。特别要注意的是, 由于这两个触发器无外部引出脚, 因此 PC₄ 或 PC₂ 脚上出现高电平或低电平信号时, 并不会改变中断允许触发器的状态。

INTR(Interrupt Request) 中断请求信号。

它是 8255A 向 CPU 发出的中断请求信号, 高电平有效。只有当 $\overline{STB}$ 、IBF 和 INTE 三者都高时, INTR 才能被置为高电平。也就是说, 当选通信号结束, 已将输入设备提供的一

个数据送到输入缓冲器中,输入缓冲器满信号 IBF 已变成高电平,并且中断是允许的情况下,8255A 才能向 CPU 发出中断请求信号 INTR。CPU 响应中断后,可用 IN 指令读取数据,读信号 $\overline{\text{RD}}$ 的下降沿将 INTR 复位为低电平。INTR 通常和 8259A 的一个中断请求输入端 IR 相连,通过 8259A 的输出端 INT 向 CPU 发中断请求。A 口的中断请求信号 INTR_A 由 PC_3 引脚输出,B 口的中断请求信号 INTR_B 由 PC_0 引脚输出。

方式 1 选通输入时序如图 9-7 所示。根据时序图,我们来分析一下这种方式的工作过程:

①当外设把一个数据送到端口数据线 $\text{PA}_7 \sim \text{PA}_0$ (对于 A 口)或 $\text{PB}_7 \sim \text{PB}_0$ (对于 B 口)后,就向 8255A 发出负脉冲选通信号 $\overline{\text{STB}}$,外设的输入数据锁存到 8255A 的输入锁存器中。②选通信号发出后,经 t_{SIB} 时间,IBF 有效,它作为对输入设备的回答信号,用于通知外设输入缓冲器已满,不要再送新的数据过来。③选通信号结束后,经 t_{SIT} 时间,若 $\overline{\text{STB}}$ 、IBF 和 INTE 三者同时为高电平,使 INTR 有效。这个信号可向 CPU 发中断请求,CPU 响应中断后,通过执行中断服务程序中的 IN 指令,使读信号 $\overline{\text{RD}}$ 有效(低电平)。④读信号有效后,经 t_{RIT} 时间后,使 INTR 变低,清除中断。⑤读信号结束后,数据已读入累加器,经 t_{RIB} 时间,IBF 变低,表示缓冲器已空,一次数据输入的过程结束,通知外设可以再送一个新的数据来。

对于 8255A,选通信号的宽度 t_{ST} 最小为 500ns, t_{SIB} 、 t_{SIT} 、 t_{RIB} 最大为 300ns, t_{RIT} 最大为 400ns。

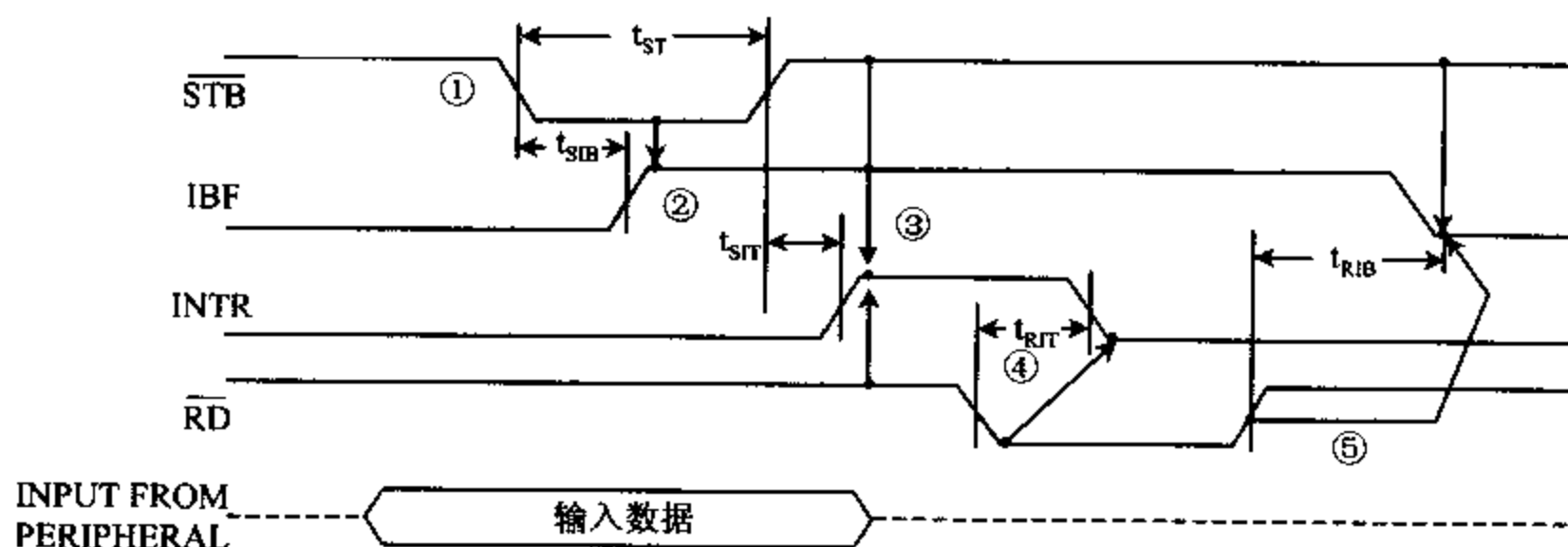


图 9-7 方式 1 选通输入时序

(2) 选通输出方式

如果 A 口和 B 口都工作于选通输出方式,它们的联络控制信号和控制字的格式如图 9-8 所示。

在这种方式下, A 口和 B 口都作输出口,端口 C 的 PC_3 、 PC_6 和 PC_7 作 A 口的联络控制信号, PC_0 、 PC_1 和 PC_2 作 B 口的联络控制信号,端口 C 余下的两位 PC_4 和 PC_5 可作输入或输出,当方式选择字的 $D_3=1$ 时, PC_4 和 PC_2 作输入, $D_3=0$ 时, PC_4 和 PC_6 作输出。

这时,各控制信号的意义如下:

$\overline{\text{OBF}}$ (Output Buffer Full) 输出缓冲器满信号,输出,低电平有效。

当它为低电平时,表示 CPU 已将数据写到 8255A 的指定输出端口,即数据已被输出锁存器锁存,并出现在端口数据线 $\text{PA}_7 \sim \text{PA}_0$ 和 $\text{PB}_7 \sim \text{PB}_0$ 上,通知外设将数据取走。实际上,它是由 8255A 送给外设的选通信号。 $\overline{\text{OBF}}$ 由输出命令 $\overline{\text{WR}}$ 的上升沿置成低电平,而外

设回答信号 $\overline{ACK}$ 将其恢复成高电平。 PC_7 被指定作 A 口的输出缓冲器满信号 $\overline{OBF}_A$, PC_1 作 B 口的缓冲器满信号 $\overline{OBF}_B$ 。

$\overline{ACK}$ (Acknowledge) 外设的回答信号,低电平有效,由外设送给 8255A。

当它为低电平时,表示 CPU 输出到 8255A 的 A 口或 B 口的数据已被外设接受。 PC_6 被指定用作 A 口的回答信号 $\overline{ACK}_A$, PC_2 为 B 口的回答信号 $\overline{ACK}_B$ 。

INTE(Interrupt Enable) 中断允许信号。

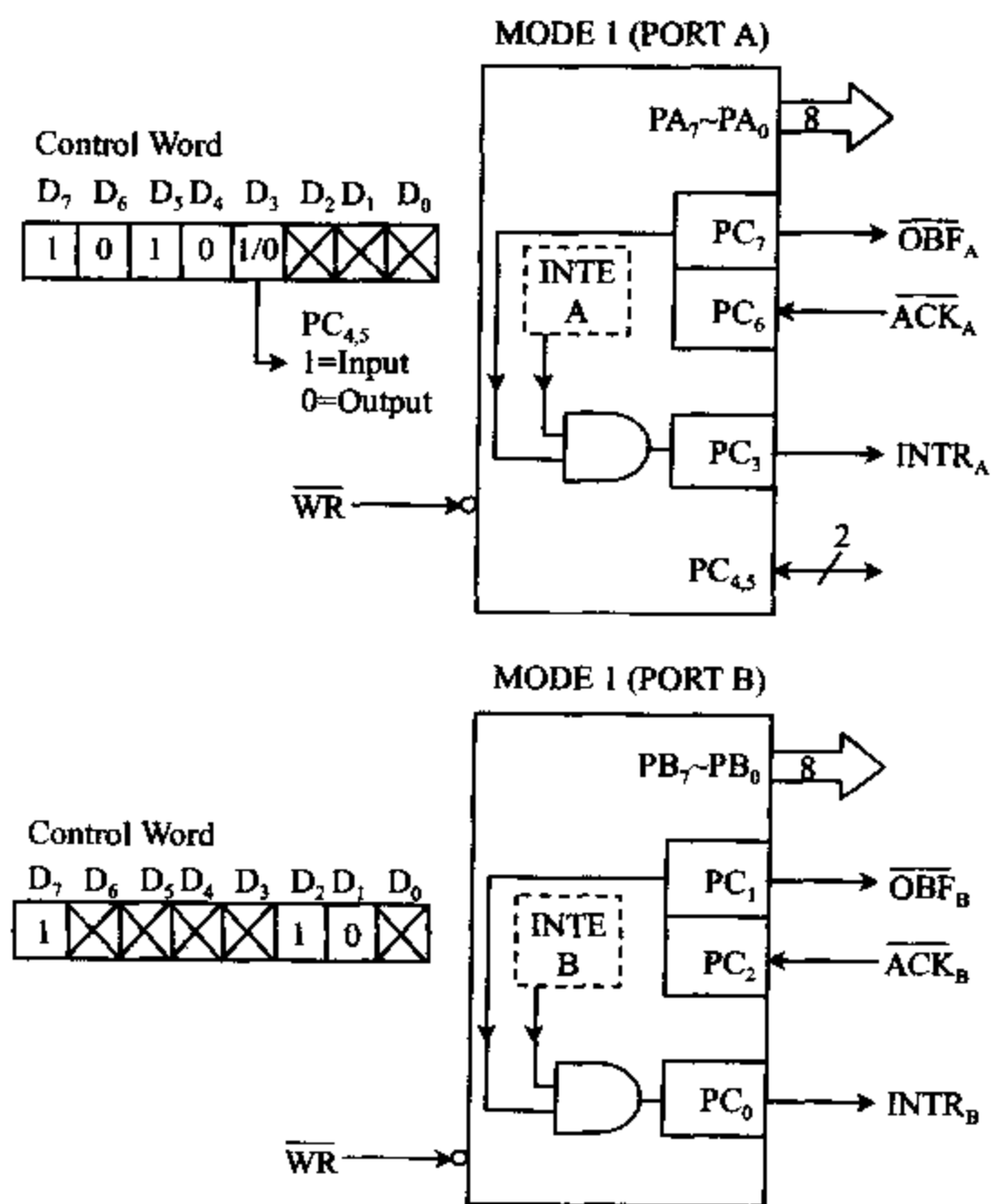


图 9-8 方式 1 输出端口状态和联络信号

其意义与 A 口、B 口均工作于选通输入方式时的 INTE 信号一样。INTE 为 1 时,端口处于中断允许状态;INTE 为 0 时,端口处于中断屏蔽状态。A 口的中断允许信号 INTE A 由 PC_6 控制,B 口的中断允许信号 INTE B 则由 PC_2 控制,它们均由置位/复位控制字将其置为 1 或清为 0,以决定中断是允许还是被屏蔽。

INTR(Interrupt Request) 中断请求信号,高电平有效。

在中断是允许的情况下,当输出设备已收到 CPU 输出的数据之后,该信号变高,可用于向 CPU 提出中断请求,要求 CPU 再输出一个数据给外设。只有当 $\overline{ACK}$ 、 $\overline{OBF}$ 和 INTE 都为 1 时,才能使 INTR 置 1。写信号将 INTR 复位为低电平。INTR 通常与 8259A 的某一个中断输入引脚 IR 相连,通过 8259A 向 CPU 发中断请求。 PC_6 引脚被指定用作 A 口的中

断请求信号线 INTR_A , PC_0 为 B 口的中断请求信号线 INTR_B 。

方式 1 选通输出时序如图 9-9 所示。输出设备在中断方式下与 CPU 交换数据的过程,大致是这样的:

①当 8255A 的输出缓冲器空,且中断是开放的情况下,可向 CPU 发中断请求。CPU 响应中断后,转入中断服务程序,用 OUT 指令将 CPU 中的数据输出到 8255A 的输出缓冲器中,这时 $\overline{\text{WR}}$ 信号变低。②经 t_{WTT} 时间后清除中断请求信号 INTR 。③此外, $\overline{\text{WR}}$ 信号的后沿使 $\overline{\text{OBF}}$ 有效,通知外设从 8255A 输出缓冲器中取走数据。④外设收到这个数据后,发回应答信号 $\overline{\text{ACK}}$ 。⑤ $\overline{\text{ACK}}$ 有效之后,再经 t_{AOB} 时间, $\overline{\text{OBF}}$ 无效,表示缓冲器已空。⑥ $\overline{\text{ACK}}$ 回到高电平后,经 t_{AIT} 时间, INTR 变高,向 CPU 发出中断请求,要求 CPU 送新的数据过来。数据传送的过程又将按上面的顺序重复进行。

t_{WTT} 、 t_{AOB} 、 t_{AIT} 的最大时间分别为 850ns、350ns、350ns。

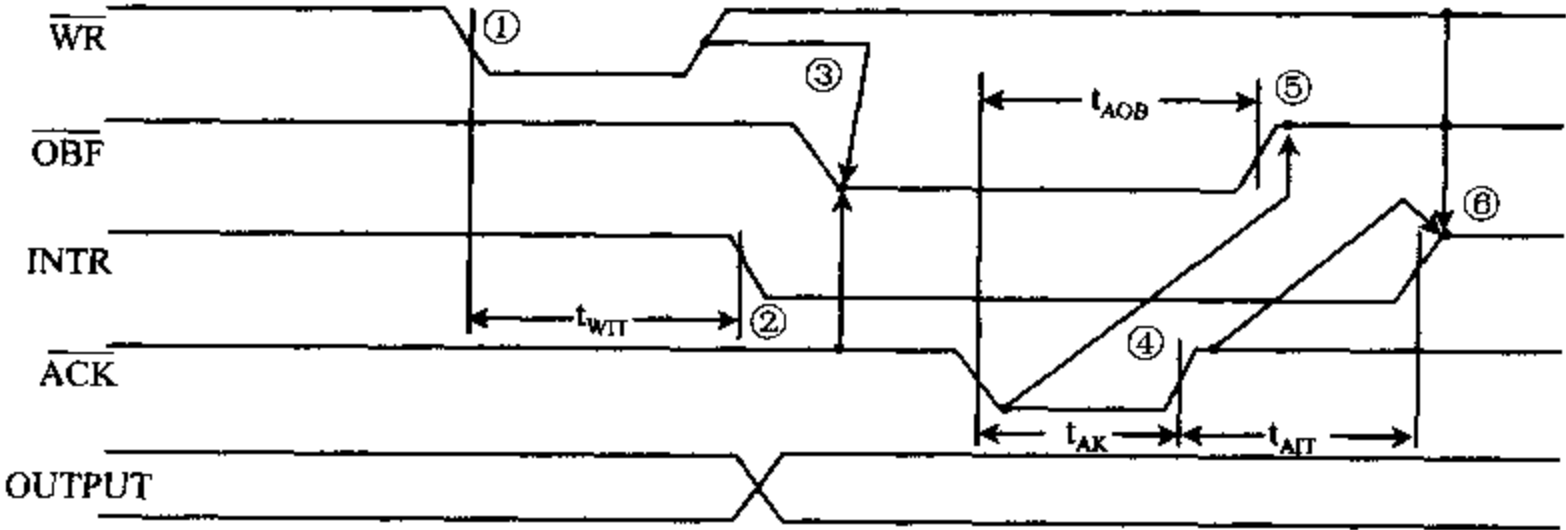


图 9-9 方式 1 选通输出时序

(3) 选通输入/输出方式组合

8255A 工作于方式 1 时,还允许对 A 口和 B 口分别进行定义,一个端口作输入,另一个端口作输出。如果将 A 口定义为方式 1 输入口,而将 B 口定义为方式 1 输出口,则其控制字格式和联络控制信号如图 9-10(a)所示。在这种情况下,端口 C 的 $\text{PC}_0 \sim \text{PC}_5$ 作状态和控

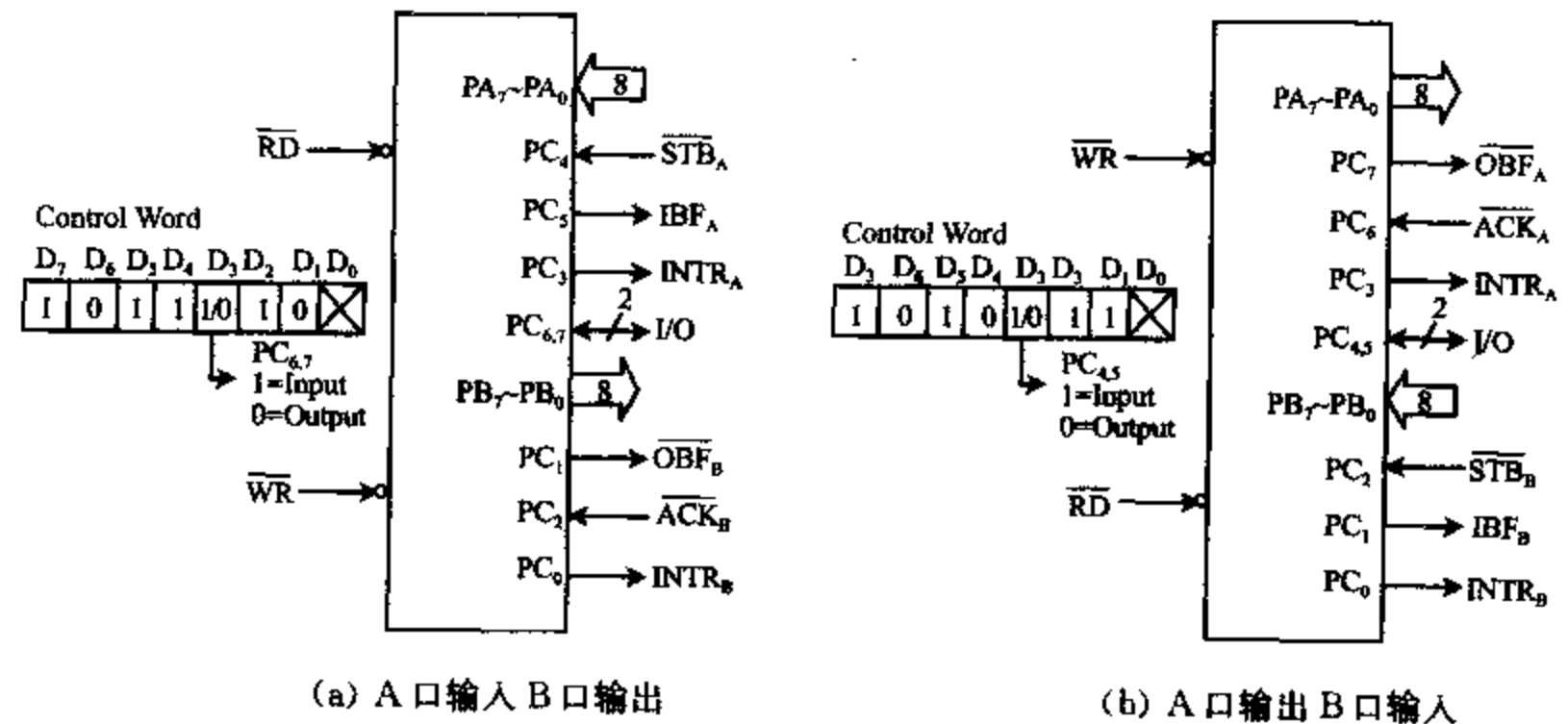


图 9-10 方式 1 组合端口状态和控制字

制线, C 口余下的两位 PC_6 和 PC_7 可作数据输入/输出用。当控制字的 $D_3=1$ 时, PC_6 和 PC_7 作输入; $D_3=0$ 时, PC_6 和 PC_7 作输出。

当 A 口定义为方式 1 输出口、B 口为方式 1 输入口时, 其方式控制字格式和联络控制信号如图 9-10(b) 所示。这时, 由 PC_6 、 PC_7 和 $PC_6 \sim PC_3$ 作控制信号, PC_4 和 PC_5 作输入或输出。当控制字的 $D_3=1$ 时, PC_4 、 PC_5 为输入; 当 $D_3=0$ 时, PC_4 、 PC_5 为输出。

由图可见, 在选通输入/输出方式下, 端口 C 的低 4 位总是作控制用, 而高 4 位总有两位仍可用于输入或输出。因此在控制字中, 用于决定 C 口高半部分是输入还是输出的 D_3 位可以取 1 或 0, 而决定 C 口低 4 位为输入或输出的 D_0 位可以是任意值。

对于选通方式 1, 还允许将 A 口或 B 口中的一个端口定义为方式 0, 另一个端口定义为方式 1。这种组态所需控制信号较少, 情况也比较简单, 读者可自行分析。

3. 方式 2

方式 2 称为双向总线方式 (Bidirectional Bus)。只有 A 口可以工作于这种方式。在这种方式下, CPU 与外设交换数据时, 可在单一的 8 位端口数据线 $PA_7 \sim PA_0$ 上进行, 既可以通过 A 口把数据传送到外设, 又可以从 A 口接收从外设送过来的数据, 而且输入和输出数据均能锁存, 但输入和输出过程不能同时进行。在主机和软盘驱动器交换数据时就采用这种方式。

端口 A 工作于方式 2 时, 端口 C 的 5 位 ($PC_3 \sim PC_7$) 作 A 口的联络控制信号, 对应关系如图 9-11 所示, 图中也给出了方式控制字的格式。

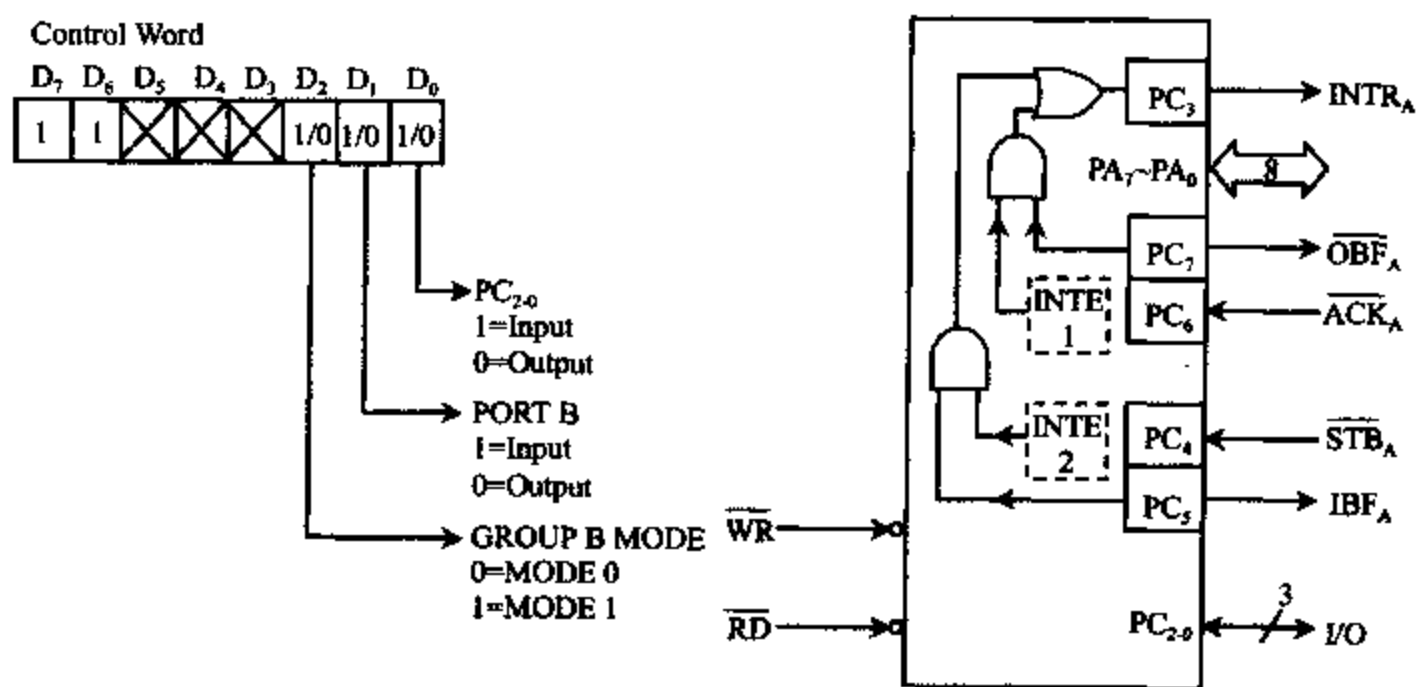


图 9-11 A 口工作于方式 2 时的端口状态和控制字

各控制信号的意义与方式 1 相似。其中, $\overline{OBF}_A$ 和 $\overline{ACK}_A$ 为输出操作时的联络控制信号, $\overline{STB}_A$ 和 IBF_A 为输入操作时的联络控制信号, 而 $INTR_A$ 在输入或输出时均用作联络控制信号。各信号意义如下:

$INTR_A$ 中断请求信号, 高电平有效。

$INTR_A$ 变成有效的条件与方式 1 相同。由于输入或输出操作引起的中断请求信号都通过同一引脚输出, 因此 CPU 响应中断时, 必须通过查询 $\overline{OBF}_A$ 和 IBF_A 的状态, 才能确定是输入过程还是输出过程引起的中断。

$\overline{OBF}_A$ 输出缓冲器满信号, 低电平有效。

当它有效时,表示 CPU 已将等待输出给外设的数据写到 8255A 的端口 A, 通知外设将数据取走。

$\overline{\text{ACK}}_A$ 外设对 $\overline{\text{OBF}}_A$ 的应答信号,低电平有效。

当 CPU 将数据写入端口 A, $\overline{\text{OBF}}_A$ 变为有效后, 输出数据并不能出现在端口数据线 $\text{PA}_7 \sim \text{PA}_0$ 上。只有当外设发出有效的 $\overline{\text{ACK}}_A$ 信号后, 才能使端口 A 的三态缓冲器开启, 输出锁存器中的数据被送到 $\text{PA}_7 \sim \text{PA}_0$ 上。当 $\overline{\text{ACK}}_A$ 无效时, 输出缓冲器处于高阻态。

$\overline{\text{STB}}_A$ 选通输入信号,低电平有效。

当它有效时,将外设送到 8255A 的数据置入输入锁存器。

IBF_A 输入缓冲器满信号,高电平有效。

当它有效时,表示外设的数据已送到输入锁存器中,等待 CPU 取走。

$\text{INTE } 1$ 和 $\text{INTE } 2$ 分别为端口 A 的输出或输入中断允许信号。

它们都必须用软件方法进行设置,设置方法与方式 1 相同。 $\text{INTE } 1$ 由 PC_6 位控制置位复位, $\text{INTE } 2$ 由 PC_4 位控制置位复位。

方式 2 的时序如图 9-12 所示,在理解该时序图时,可以将方式 2 看成是方式 1 输出和方式 1 输入的结合。

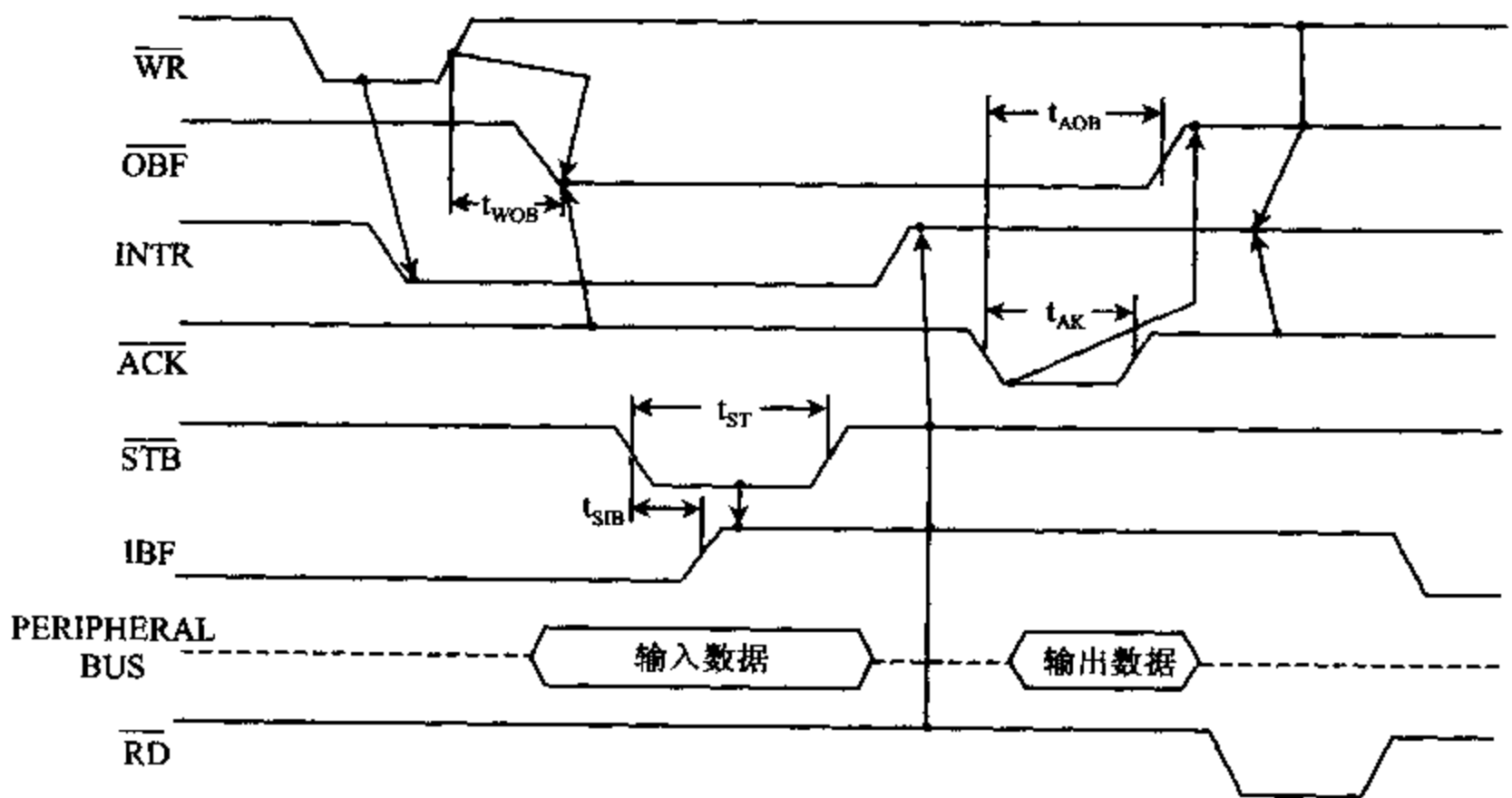


图 9-12 方式 2 的时序

根据此时序图,我们对方式 2 的工作过程做些简单说明。图中画了一个数据输出和一个数据输入过程的时序,实际上,当端口 A 工作在方式 2 时,输入过程和输出过程的顺序是任意的,输入或输出数据的次数也是任意的。

对于输出过程, CPU 响应中断,用输出指令向 8255A 的端口 A 写入一个数据时,写信号 $\overline{\text{WR}}$ 有效。它一方面使中断请求信号 INTR_A 变低,撤销中断请求。另一方面, $\overline{\text{WR}}$ 的后沿经 t_{WOB} 时间后,使输出缓冲器满信号 $\overline{\text{OBF}}_A$ 变低, $\overline{\text{OBF}}_A$ 送往外设。外设收到这个信号后,发回应答信号 $\overline{\text{ACK}}_A$, 它开启 8255A 的输出锁存器,使输出数据出现在 $\text{PA}_7 \sim \text{PA}_0$ 线上。 $\overline{\text{ACK}}_A$ 信号还使输出缓冲器满信号 $\overline{\text{OBF}}_A$ 变为无效,从而可以开始下一个数据传送过程。

对于输入过程,当外设把数据送往 8255A 时,选通信号 $\overline{STB_A}$ 也一起到来,将外部的输入数据锁存到 8255A 的输入锁存器中,从而使输入缓冲器满信号 IBF_A 成为高电平。选通信号结束时,使中断请求信号变高。CPU 响应中断执行 IN 指令时, $\overline{RD}$ 信号有效,将数据读入累加器,随后 IBF_A 变低,输入过程结束。

当 A 口工作于方式 2 时,B 口可工作于方式 0 或方式 1。而且既可以是输入也可以是输出。如果 B 口工作于方式 0,不需要联络控制信号,C 口余下的 3 位 $PC_2 \sim PC_0$ 仍可作输入或输出用;如果 B 口工作于方式 1, $PC_2 \sim PC_0$ 作 B 口的联络信号,这时 C 口的 8 位数据都配合 A 口和 B 口工作。B 口为输出时, PC_1 为 $\overline{OBF_B}$ 、 PC_2 为 $\overline{ACK_B}$ 、 PC_0 为 $INTR_B$;B 口作输入时, PC_2 为 $\overline{STB_B}$ 、 PC_1 为 IBF_B 、 PC_0 为 $INTR_B$ 。

4. C 口状态字

当 8255A 工作于方式 0 时,C 口各位作输入输出用。当它工作于方式 1 和方式 2 时,C 口产生或接收与外设间的联络信号,这时,读取 C 口的内容可使编程人员测试或检查外设的状态,用输入指令对 C 口进行读操作就可读取 C 口的状态。C 口的状态字有以下几种格式:

(1) 方式 1 状态字

输入状态字

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
I/O	I/O	IBF_A	INTE A	$INTR_A$	INTE B	IBF_B	$INTR_B$

其中 $D_7 \sim D_3$ 位为 A 组状态字, $D_3 \sim D_0$ 位为 B 组状态字。

输出状态字

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
$\overline{OBF_A}$	INTE A	I/O	I/O	$INTR_A$	INTE B	$\overline{OBF_B}$	$INTR_B$

同样, $D_7 \sim D_3$ 位为 A 组状态字, $D_3 \sim D_0$ 位为 B 组状态字。

(2) 方式 2 状态字

D_7	D_6	D_5	D_4	D_3	D_2	D_1	D_0
$\overline{OBF_A}$	INTE 1	IBF_A	INTE 2	$INTR_A$	X	X	X

其中 $D_7 \sim D_3$ 位为 A 组状态字, $D_3 \sim D_0$ 位为 B 组所用,当 B 口工作于方式 1 时,这几位作 B 口状态字,B 口工作于方式 0 时,这几位不是状态位,而是作输入输出用。

9-2 8255A 的应用举例

一、基本输入输出应用举例

在工业控制等实际应用中,经常需要检测某些开关量的状态。例如,在某一系统中,有

8 个开关 $K_7 \sim K_0$, 要求不断检测它们的通断状态, 并随时在发光二极管 $LED_7 \sim LED_0$ 上显示出来。开关断开, 相应的 LED 点亮; 开关合上, LED 熄灭。我们选用 8086 CPU, 8255A 和 74LS138 译码器等芯片, 构成如图 9-13 所示的硬件电路, 来实现上述功能。

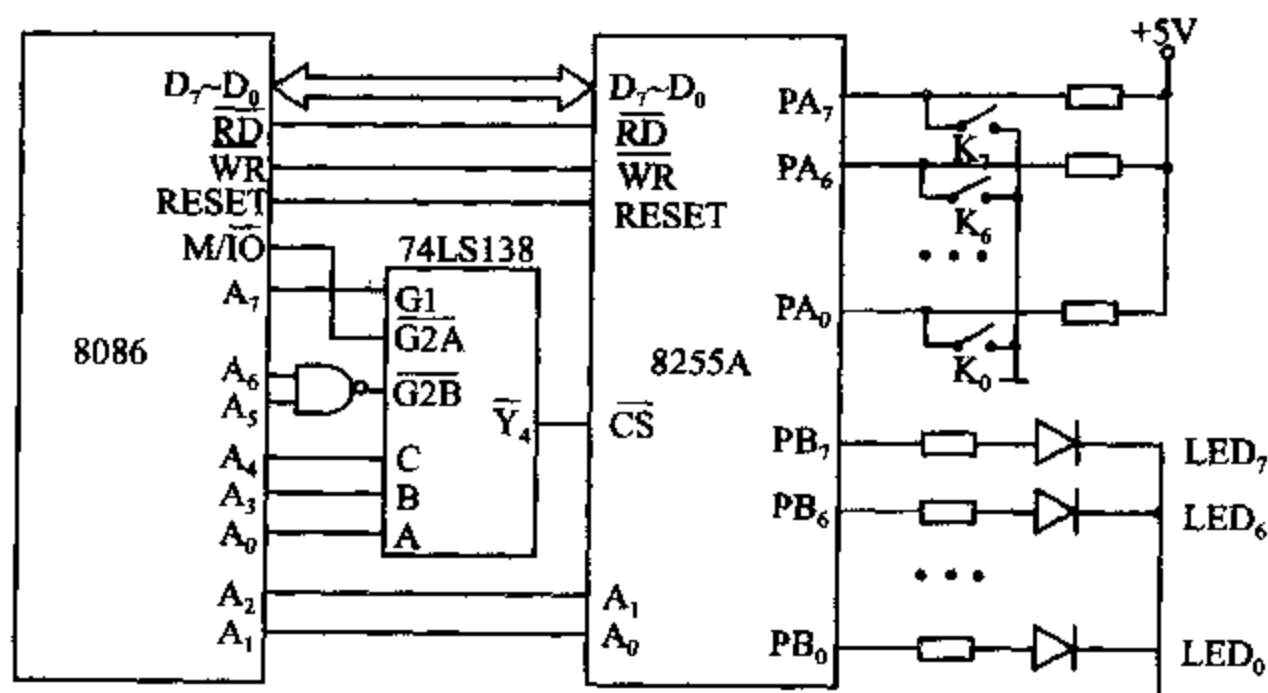


图 9-13 读开关状态连线图

由图可见, 8255A 的 A 口作输入口, 8 个开关 $K_7 \sim K_0$ 分别接 $PA_7 \sim PA_0$ 。B 口为输出口, $PB_7 \sim PB_0$ 分别接显示器 $LED_7 \sim LED_0$ 。8255A 的 $\overline{RD}$ 、 $\overline{WR}$ 和 RESET 引脚分别与 CPU 的相应输出相连。8255A 的数据线 $D_7 \sim D_0$ 与 8086 的低 8 位数据总线 $D_7 \sim D_0$ 相连, 从上一章的讨论可知, 这时 8255A 的 4 个口地址都应为偶地址, A_0 必须总等于 0, 用地址线的 A_2 、 A_1 来选择片内的 4 个端口。图中, 地址线 A_7 接译码器的 G_1 , $M/\overline{IO}$ 与 $\overline{G2A}$ 相连, A_6 、 A_5 接与非门输入端, 与非门输出与 $\overline{G2B}$ 相连。当 $A_7 A_6 A_5 = 111$, $A_4 A_3 A_0 = 100$ 时, $\overline{Y_4} = 0$, 选中 8255A。这样, 4 个端口地址分别为 F0H、F2H、F4H 和 F6H, 对应于 8255A 的 A 口、B 口、C 口和控制字寄存器。

编程时先要确定方式选择控制字。由于 A 口工作于方式 0 输入, B 口为方式 0 输出, C 口未用, 控制字中与 C 口对应的位可以被置为 0, 这样, 写入控制端口 F6H 的控制字为 10010000。完成初始化后, 即可将 A 口的开关状态读入寄存器 AL。若开关合上, AL 中的相应位为 0, 断开则为 1。当把 AL 中的内容从 B 口输出时, 相应于 0 的位上的 LED 熄灭, 表示对应的开关是合上的; 否则 LED 点亮, 指示开关断开。具体程序如下:

MOV	DX, 0F6H	; 控制字寄存器	
MOV	AL, 10010000B	; 控制字	
OUT	DX, AL	; 写入控制字	
TEST-IT:	MOV	DX, 0F0H	; 指向 A 口
	IN	AL, DX	; 从 A 口读入开关状态
	MOV	DX, 0F2H	; 指向 B 口
	OUT	DX, AL	; B 口控制 LED, 指示开关状态
	JMP	TEST-IT	; 循环检测

二、键盘接口

在微型机系统中,键盘是一种最常用的外设,它由多个开关组合而成。可以用来制造键盘的按键开关有好多种,最常用的有机械式、薄膜式、电容式和霍尔效应式等4种。机械式开关较便宜,但压键时会产生触点抖动,即在触点可靠地接通前会通断多次,而且长期使用后可靠性会降低。高质量机械式开关的寿命约100万次。薄膜式开关可做成很薄的密封单元,不易受外界潮气或环境污染,常用于微波炉、医疗仪器或电子秤等设备的按键。不同薄膜开关的寿命差别很大。电容式开关没有抖动问题,但需要特制电路来测电容的变化,平均寿命约2000万次。霍尔效应按键是另一种无机机械触点的开关,具有很好的密封性,平均寿命高达1亿次甚至更高,但开关机制复杂,价格很贵。计算机上用的键盘一般都用机械式开关。

当你在键盘上压一个键时,等于在压下一个开关。因此,从原理上讲,键盘可以采用图9-13那样的结构。不过,这会带来一个很大的缺点,那就是每个键要用一条线,每8个开关还要占用一个8位的并行端口。一个具有64个键的键盘需要64条连线,与8个8位的并行端口相连。所以,这种结构只能用在仅有几个键的小键盘中。对于大多数的键盘,按键被排成行和列的矩阵。下面以机械式开关构成的16个键的键盘为例,来讨论键盘接口的工作原理,这种原理对采用其它类型的开关的键盘也是适用的。

设16个键分别为16进制数字0~9和A~F,键盘排列、连线及接口电路如图9-14所示。16个键排成4行×4列的矩阵,接到微型机的一对端口上。端口由8255A构成,其中端口A作输出,端口B作输入。矩阵的4条行线接到输出端口A的 $PA_3 \sim PA_0$,用程序能

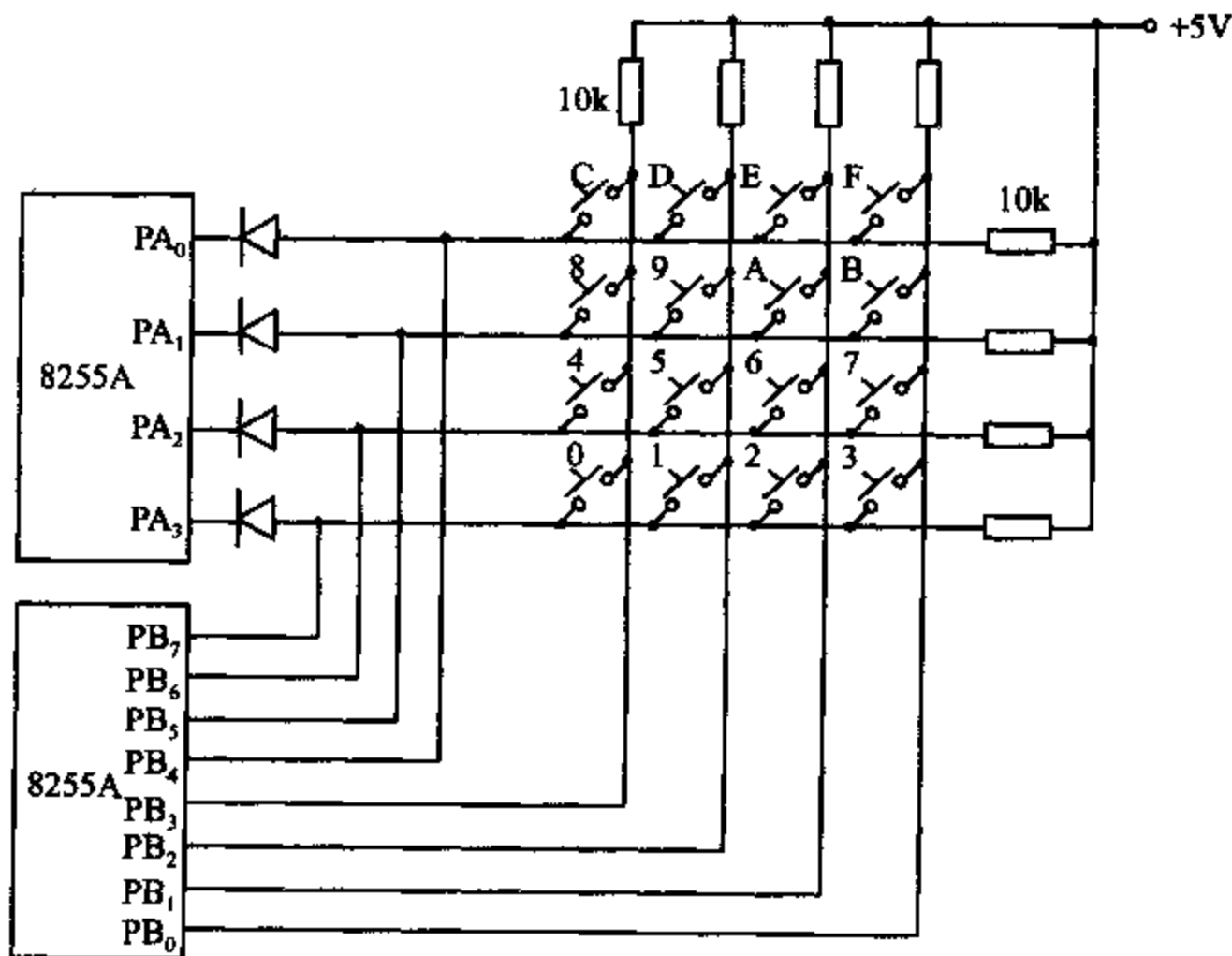


图 9-14 键盘接口电路

改变这 4 条行线上的电平。4 条列线连到输入端口 B 的 $PB_3 \sim PB_0$, 4 条行线还同时接到输入端口 B 的 $PB_7 \sim PB_4$ 上。这样, 用输入指令读取 B 口状态时, 可同时读取键盘的行列信号。

在无键压下时, 由于接到 +5V 上的上拉电阻的作用, 列线被置成高电平。压下某一键后, 该键所在的行线和列线接通。这时, 如果向被压下键所在的行线上输出一个低电平信号, 则对应的列线也呈现低电平。当从 B 口读取列线信号时, 便能检测到该列线上的低电平。读取 B 口的状态时, 还能读到行线上的低电平信号。这样, 根据读入的行和列状态中低电平的位置, 便能确定哪个键被压下了。

识别键盘上哪个键被压下的过程称为键盘扫描, 上述键盘的扫描包含以下几步:

(1) 检测是否所有键都松开了, 若没有则反复检测。

(2) 当所有键都松开了, 再检测是否有键压下, 若无键压下则反复检测。

(3) 若有键压下, 要消除键抖动, 确认有键压下。

(4) 对压下的键进行编码, 将该键的行列信号转换成 16 进制码, 由此确定哪个键被压下了。如出现多键重按的情况, 只有在其它键均释放后, 仅剩一个键闭合时, 才把此键当作本次压下的键。

(5) 该键释放后, 再回到(2)。

检测矩阵中是否有键压下的一种简单方法是, 自输出口 A 向所有行线输出 0 电平, 再通过 B 口的低 4 位读取列值, 若其中有 0 值, 便是有键压下了。

在开始一次扫描时, 先应确认上一次压下的键是否已松开。即先向所有行线输出低电平, 再读入各列线值, 只有当所有的行线和列线均为高电平, 表示以前压下的键都已释放了, 才开始检测是否有键压下。

当检测到有键压下后, 必须消除键抖动 (Debounce)。消除键抖动的常用方法是在检测到有键压下后, 延长一定时间 (通常为 20ms), 再检查该键是否仍被压着。若是, 才认定该键确实被按下了, 而不是干扰。

确认有键压下后, 再确定被压下键所在的行列号。为获取行列信息, 先从 A 口输出一个低电平到一行线上, 再从 B 口读入各列的值, 若没有一列为低电平, 说明压下的键不在此行。于是, 再向下一行输出一个低电平, 再检测各列线上是否有低电平。依次对每一行重复这个过程, 直至查到某一系列线上出现低电平为止。被置成低电平的行和读到低电平的列, 便是被压下键所在的行列值。

已知被压下的键所在的行号 (0~3) 和列号 (0~3) 后, 就能得到该键的扫描码。例如, 对于数字 0, 它位于 3 行、3 列, 压下“0”键时, 从 B 口可读得 D_7 位和 D_3 位为 0, 其余位为 1, 所以数字 0 的编码为 01110111B, 即 77H; 对于数字 6, 处于 2 行、1 列, 压下“6”键时, D_3 位和 D_1 位为 0, 其余位为 1, 所以数字 6 的编码为 10111101B=BDH。类似地, 其余各键的编码也可一一求得。将这些编码值列成表, 放在数据段中, 用查表程序来查对, 便能确定压下的是什么键。

下面是键盘检测、去抖动、键值编码和确定键名的汇编语言程序。程序运行后, 若返回值 AH=0, 表示已读到有效的键值, 并在 AL 中存有 0~F 键的 16 进制代码; 若 AH=1, 则表示出错。

; 端口地址

PORT_A EQU 0FF9H ; 8255 A 口地址


```

PORT-B      EQU    0FFBH          ;8255 B口地址
PORT-CTL     EQU    0FFFH          ;8255 控制口地址
;数据段,键盘扫描码表
DATA         SEGMENT
;              0      1      2      3      4      5      6      7
TABLE        DB  77H, 7BH, 7DH, 7EH, 0B7H,0BBH,0BDH,0BEH
;              8      9      A      B      C      D      E      F
              DB  0D7H,0DBH,0DDH,0DEH,0E7H,0EBH,0EDH,0EEH
DATA         ENDS
;堆栈段
STACK        SEGMENT      STACK
              DW  50      DUP (0)
TOP-STACK    LABEL      WORD
STACK        ENDS
;代码段
CODE         SEGMENT
              ASSUME      CS:CODE, DS:DATA, SS:STACK
START:        MOV  AX,     STACK
              MOV  SS,     AX
              LEA  SP,     TOP-STACK
              MOV  AX,     DATA
              MOV  DS,     AX
;初始化 8255A,方式 0,A 口作输出,B 口和 C 口为输入
              MOV  DX,     PORT-CTL      ;指向控制口
              MOV  AL,     10001011B     ;控制字
              OUT  DX,     AL             ;写入控制字
;向所有行送 0
              MOV  DX,     PORT-A        ;A 口
              MOV  AL,     00H
              OUT  DX,     AL             ;向 A 口各位输出 0
;读列,查看是否所有键均松开
              MOV  DX,     PORT-B
WAIT-OPEN:    IN   AL,     DX             ;键盘状态读入 B 口
              AND  AL,     0FH           ;只查低 4 位(列值)
              CMP  AL,     0FH           ;是否都为 1(各键均松开)?
              JNE  WAIT-OPEN            ;否,继续查
;各键均已松开,再查列是否有 0,即是否有键压下
WAIT-PRES:    IN   AL,     DX             ;读 B 口
              AND  AL,     0FH           ;只查低 4 位
              CMP  AL,     0FH           ;是否有键压下
              JE   WAIT-PRES            ;无,等待
;有键压下,延时 20ms,消抖动
              MOV  CX,     16EAH

```


DELAY:	LOOP DELAY	;延时 20ms
;再查列,看键是否仍被压着		
	IN AL, DX	
	AND AL, 0FH	
	CMP AL, 0FH	
	JE WAIT-PRES	;已松开,转出等待压键
;键仍被压着,确定哪一个键被压下		
	MOV AL, 0FEH	;先使 D ₆ =0
	MOV CL, AL	;CL=1111 1110B
NEXT-ROW:	MOV DX, PORT-A	;A口
	OUT DX, AL	;向一行输出低电平
	MOV DX, PORT-B	;B口
	IN AL, DX	;读入 B 口状态
	AND AL, 0FH	;只截取列值
	CMP AL, 0FH	;是否均为 1?
	JNE ENCODE	;否,表示有键压下,转去编码
	ROL CL, 01	;均为 1,使下行输出 0
	MOV AL, CL	
	JMP NEXT-ROW	;查看下行
;已找到有一列为低电平,对压键的行列值编码		
ENCODE:	MOV BX, 000FH	;建立地址指针,先指向 F 键对应的地址
	IN AL, DX	;从 B 口读入行列号
NEXT-TRY:	CMP AL, TABLE[BX]	;读入的行列值与表中查得的相等吗?
	JE DONE	;相等,转出
	DEC BX	;不等,指向下一个(键值较小者)地址
	JNS NEXT-TRY	;若地址尚未减为负值,继续查
	MOV AH, 01	;若减为负值,置出错码 01→AH 中
	JMP EXIT	;退出
DONE:	MOV AL, BL	;BL 中存有键的 16 进制代码
	MOV AH, 00	;AH=0,读到有效键值
EXIT:	HLT	
CODE	ENDS	
	END	

三、8255A 在 PC/XT 机中的应用

在 PC/XT 机中,以 8255A-5 为接口芯片,来读取键盘输入的扫描码和系统配置 DIP 开关的设置状态,同时还可以控制扬声器发声及奇偶校验电路的工作。8255A-5 与 8255A 的引脚、功能及编程方法完全一样,仅电气特性指标比 8255A 略高一点。下面从硬件连接和软件编程这两方面来说明该芯片在 PC/XT 系统中的应用。

1. 硬件连接

图 9-15 是 8255A-5 在 PC/XT 机系统中的硬件连接图。图中,8255A-5 的 D₇~D₀ 与系

统数据总线 $D_7 \sim D_0$ 相连, $\overline{RD}$ 、 $\overline{WR}$ 端分别接 $\overline{IOR}$ 和 $\overline{IOW}$, 接受系统 I/O 总线周期的读写控制命令。8255A-5 的片选端 $\overline{CS}$ 和 I/O 接口译码电路的 $\overline{PPICS}$ 相连, 地址范围为 60H~7FH。在 ROM BIOS 中的 8255A-5 驱动程序里, 使用的地址范围是 60H~63H。 $A_1 A_0$ 端接地址总线的 $A_1 A_0$, 以选择 8255A-5 内部的 4 个端口, A 口、B 口、C 口及控制字寄存器的端口地址分别为 60H、61H、62H 和 63H。系统中, A 口、B 口和 C 口均工作于方式 0。

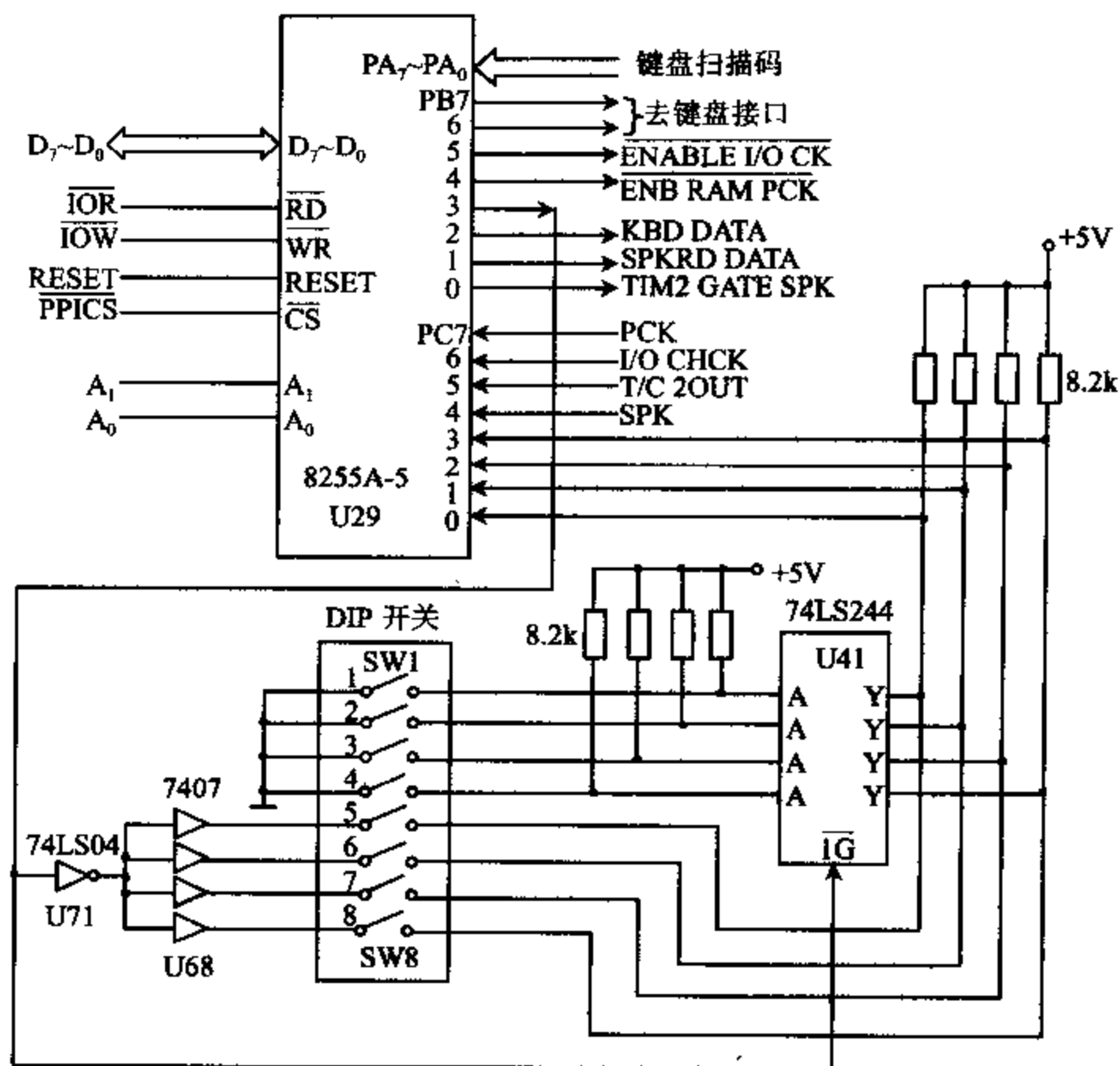


图 9-15 PC/XT 机中 8255A-5 的连接

下面具体介绍各端口的功能。

(1) 端口 A

机器刚上电自检时, 系统处于自检方式, 端口 A 处于输出方式, 用来输出当前检测部件的标志, 确定故障的来源。自检完成之后, 系统进入正常工作方式, 端口 A 工作于输入方式, 用来读取键盘的 8 位扫描码。

(2) 端口 B

B 口工作于输出方式, 用来输出若干控制信号, 即:

PB_0 输出 TIM2 GATE SPK 信号, 送到 8253-5 芯片的 $GATE_2$ 端。当 $PB_0 = 1$ 时, 允许计数器 2 工作, 产生控制扬声器发声的音调信号; 当 $PB_0 = 0$ 时, 计数器 2 不工作。

PB_1 输出 SPKDATA 信号, 控制扬声器发声。 $PB_1 = 1$ 时, 允许 8253-5 通道 2 产生的音调信号加到扬声器驱动电路, 否则禁止扬声器发声。

PB₂ 保留使用的输出信号。它可用来控制键盘接口,输出键盘检测数据 KBD DATA,接入键盘接口电路供测试键盘用。

PB₂ 控制系统配置 DIP 开关状态的读入。当 PB₃=1 时,U41(74LS244)被封锁,DIP 开关的高 4 位状态(SW8~SW5)被读入端口 C 的低 4 位(PC₃~PC₀);当 PB₃=0 时,U41 被选通,将 DIP 开关的低 4 状态(SW4~SW1)读入 PC₃~PC₀。开关合上(ON)时,读入的状态电平为 0;断开(OFF)时,读入的状态电平为 1。

PB₄ 输出 $\overline{\text{ENB RAM PCK}}$ 信号。当 PB₄=0 时,允许系统板上的 RAM 奇偶校验电路工作;否则,禁止校验电路工作。

PB₅ 输出 $\overline{\text{ENABLE I/O CK}}$ 信号。其功能与 $\overline{\text{ENB RAM PCK}}$ 类似,但该信号用来控制 I/O 通道上扩展 RAM 奇偶校验电路的工作。

PB₆ 和 PB₇ 输出到键盘接口电路。

(3) 端口 C

C 口工作于输入方式,用来读取系统内部的状态。这些状态包括:

PC₃~PC₀ 系统配置开关 DIP 的设置状态。

PC₁ 扬声器电路的发声数据 SPK,也即加到扬声器上的驱动信号。

PC₅ 扬声器的音调信号状态 T/C2OUT,也就是从 8253-5 的 OUT2 输出的信号。

PC₆ I/O 通道奇偶校验结果 I/O CHCK。若结果为高,则产生 NMI 中断请求信号。是否允许送出 I/O CHCK 信号,受 PB₅ 的控制。

PC₇ 系统板上奇偶校验电路的结果 PCK。若结果为高,则产生 NMI 中断请求信号。是否允许发出 PCK 信号,受 PB₄ 的控制。

在弄清了 8255A-5 的 3 个端口的功能后,我们再来讨论系统配置开关 DIP 的功能和读取开关状态的过程。

在 PC/XT 机系统板上有一个 8 位的 DIP 开关,它由 8 个小开关组成,结构为双列直插式。当小开关 1~8 打到 ON 一边时,开关接通,电位为零,开关状态用 0 表示;当打到 OFF 一边时,开关断开,电位为高电平,用 1 表示。各小开关的功能如表 9-2 所示。

表 9-2 DIP 开关的功能

开关	状态	功 能
1	1/0	正常操作/循环执行加电自检
2	1/0	无协处理器/有协处理器
4,3	00	64K 系统板 RAM
	01	128K 系统板 RAM
	10	192K 系统板 RAM
	11	256K 系统板 RAM
6,5	00	无显示器
	01	40×25 彩显
	10	80×25 彩显
	11	80×25 单显
8,7	00	1 个软驱
	01	2 个软驱
	10	3 个软驱
	11	4 个软驱

8 个 DIP 小开关的通断状态从 8255A-5 的 $PC_0 \sim PC_3$ 读入 CPU, 工作过程如下:

当 $PB_3 = 0$ 时, U41(74LS244)的控制端 $\overline{IG}$ 为低电平, 使 U41 内部的 4 路三态缓冲器选通, 将 DIP 开关的低 4 位状态送到 $PC_0 \sim PC_3$ 。由于这 4 个开关的左端接地, 因此合上时为 0, 打开为 1。 PB_3 还同时接到反相器 U71(74LS04)的输入端, 反相后的高电平信号送到集电极开路门驱动器 U68(7407)的 4 个输入端。 U68 是同相门, 输出仍为高电平, 因它是集电极开路门, 即使与它相连的开关合上, 其高电平输出也不会影响 $PC_0 \sim PC_3$ 上的信号状态。这样, CPU 可读取开关低 4 位的状态。

当 $PB_3 = 1$ 时, U41 的 $\overline{IG}$ 端为高电平, 使 U41 中的 4 个三态门呈高阻态, 将低 4 位开关的状态与 $PC_0 \sim PC_3$ 断开。 PB_3 的高电平经 U71 反相后变成低电平, 经 U68 驱动后输出低电平, 等于将高 4 位开关的左端接地。结果, 这 4 个开关的通断状态便送到 $PC_0 \sim PC_3$, 被 CPU 读入。

2. 软件编程

从前面 8255A-5 的硬件电路介绍中已知道, 在刚加电时, 8255A-5 的 3 个端口均工作于方式 0, 而且端口 A 和端口 B 为输出, 端口 C 为输入, 这时, 方式选择字应置成 10001001B。在正常操作时, 端口 A 为输入, 其余端口的工作方式同上, 这时, 方式选择字为 10011001B。两种情况下可分别用下列指令完成设置方式的工作:

```
MOV    AL, 10001001B
OUT    63H, AL           ;输出自检方式时的控制字

MOV    AL, 10011001B
OUT    63H, AL           ;输出正常方式时的方式字
```

对端口 B, 通过编程可实现不同的控制功能。例如, 希望禁止系统板和 I/O 扩展板的 RAM 奇偶校验, 可用如下程序段:

```
IN      AL, 61H           ;读入 B 口状态
OR      AL, 00110000B      ;使  $PB_4$ 、 $PB_5$  置 1, 禁止奇偶校验
OUT     61H, AL
```

这里, 我们先用 IN 指令读取 B 口的状态, 再用 OR 指令使 PB_4 、 PB_5 置 1, 即禁止奇偶校验, 然后将新的状态字送回端口 B, 这样做并不影响 B 口其它各位的控制功能。既然已将 B 口设定为输出方式, 为什么又能用 IN 指令读取 B 口的状态呢? 这是由 B 口的特殊结构所决定的。回忆本章开头所介绍的 8255A-5 的端口结构, 除了一个用作输入的 8 位缓冲器外, B 口还有一个输入/输出锁存器/缓冲器。因此, 当从 B 口输出时, 数据不但能被锁存, 还能随时用 IN 指令读回锁存在那里的输出状态。这时, 由于 B 口被编程为输出方式, 因而读到的不是从外设输入到 B 口的数据。

端口 C 工作于输入方式时, 可用程序读取各位的数值, 以了解系统的内部状态。例如, 在需要检查 8 位 DIP 开关的状态时, 可用以下程序段:

```
IN      AL, 61H           ;读 B 口状态
AND     AL, 11110111B      ; $PB_3$  置 0, 其余位不变
```

OUT	61H, AL	;送回 B 口
IN	AL, 62H	;读 C 口状态
AND	AL, 0FH	;取低 4 位开关状态
MOV	AH, AL	;存入 AH
IN	AL, 61H	;读 B 口状态
OR	AL, 00001000B	;PB ₃ 置 1, 其余位不变
OUT	61H, AL	;送回 B 口
IN	AL, 62H	;读高 4 位开关状态进 AL 低 4 位
MOV	CL, 4	
ROL	AL, CL	;左移 4 次后送到 D ₇ ~D ₄ 位
AND	AL, 0F0H	;截取高 4 位开关量
OR	AL, AH	;8 位开关状态组合在 AL 中

四、PC/XT 机中的扬声器接口电路

扬声器是计算机的一个简单输出设备,用来产生一定音调的声响,进行必要的报警和提示,也可以通过编程,控制加到扬声器上的信号的频率,奏出乐曲来。在 PC/XT 机中,扬声器接口电路由 8255A-5、8253-5、驱动器和低通滤波器等构成,如图 9-16 所示。图中,8253-5 是音频信号源,8255A-5 作控制器,驱动器用来增大 8253-5 输出的 TTL 电平信号的驱动能力,低通滤波器将脉冲信号转换成接近正弦波的音频信号,去驱动扬声器发声。扬声器发声过程的控制方法如下:

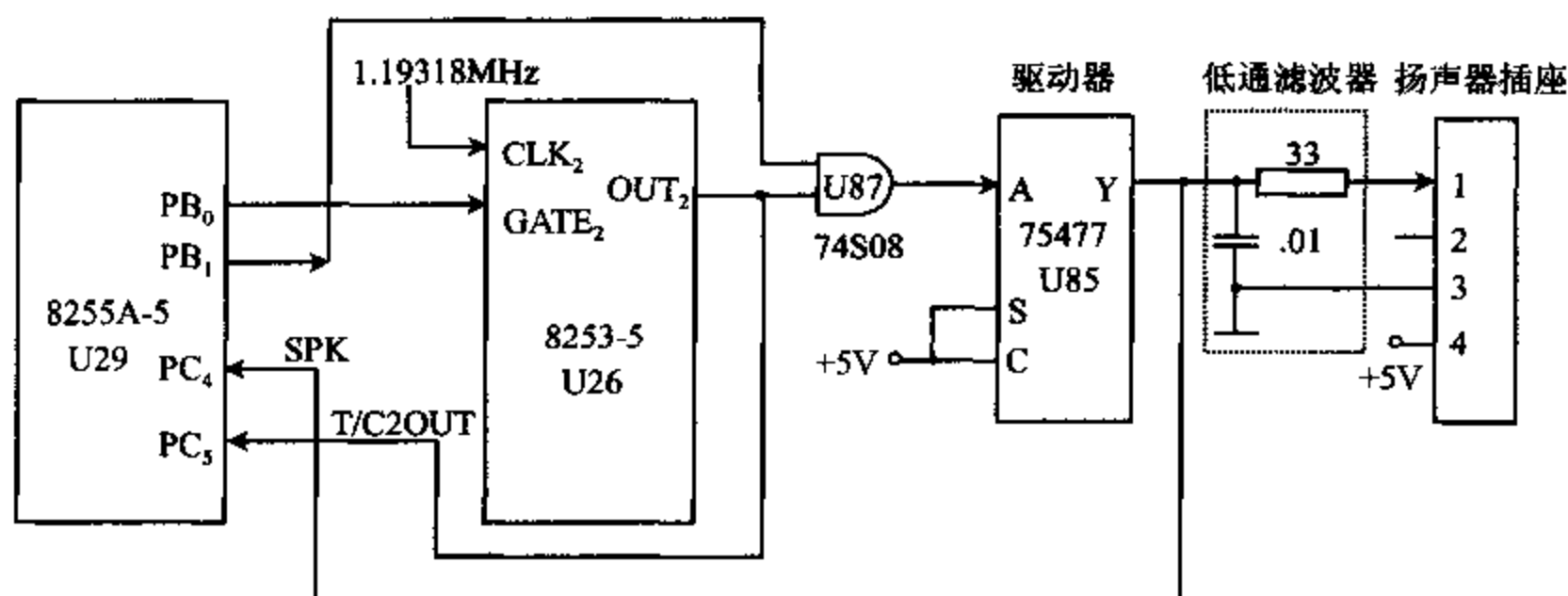


图 9-16 扬声器接口电路

8253-5 的计数器 2 的 CLK2 端所加的时钟脉冲频率为 1.19318MHz。可根据这个频率和所要产生的声音频率,计算出定时常数,经编程让计数器 2 输出指定频率的波形。8255A-5 的 PB₀ 接 8253-5 的 GATE₂,作为计数器的门控信号,允许或禁止 8253-5 计数。8255A-5 的 PB₁ 接与门 U87 的一个输入端,用来对计数器 2 的 OUT₂ 端输出的波形作进一步的控制。当 PB₁=1 时,8253-5 从 OUT₂ 输出的波形才能通过与门 U87 送到驱动器

75477 的 A 端,产生 1/2W 的驱动功率,再通过阻容低通滤波器,滤除高次谐波后送到扬声器插座,使之发声。当 8255A-5 的 $PB_1=0$ 时,OUT₂ 输出的波形不能通过与门 U87,扬声器不会发声。这样,通过对 PB_1 电平的设置可控制扬声器发声时间的长短。当 $PB_1PB_0=11$ 时,扬声器能连续发声。

接口电路中的 T/C2OUT 接 8255A-5 的 PC₅,CPU 可通过读取 PC₅ 的状态来了解计数器的输出状态。驱动器 U85 的输出端 Y 接到 8255A-5 的 PC₄,CPU 可从这里读取驱动器输出到扬声器的信号的状态。

根据上面介绍的扬声器接口电路,我们可以来编写一个产生指定频率为 f 的音频信号的通用发声程序。首先把 8253-5 的计数器 2 编程为工作方式 3。由于 $f_{CLK2}=1.19318MHz=1193180Hz$,为使输出端 OUT₂ 得到频率为 f 的方波,必须向计数器 2 写入初值 n ,其值为:

$$n=f_{CLK2}/f=1193180/f=1234DEH/f$$

这里, f 应为人耳能听到的音频频率,其范围约为 20Hz~20000Hz,可以用 16 进制数的形式事先存入 DI 寄存器。在程序中,为了求得 n ,需要用字除法进行运算。即先将被除数的高字节(12H)送到 DX 中,低字节(34DEH)送到 AX 中,再除以 DI 中的数,在 AX 中得到的商就是初值 n 。

下面是能产生频率为 f 的通用发声程序:

MOV	AL, 10110110B	;8253 控制字:通道 2,先写低字节,后写高字节
		;方式 3,二进制计数
OUT	43H, AL	;写入控制字
MOV	DX, 0012H	;被除数高位
MOV	AX, 34DEH	;被除数低位
DIV	DI	;求计数初值 n ,结果在 AX 中
OUT	42H, AL	;送出低 8 位
MOV	AL, AH	
OUT	42H, AL	;送出高 8 位
IN	AL, 61H	;读入 8255A 端口 B 的内容
MOV	AH, AL	;保护 B 口的原状态
OR	AL, 03H	;使 B 口后两位置 1,其余位保留
OUT	61H, AL	;接通扬声器,使它发声

从上面的讨论可知,只要能从与门 U87 的输出端送出一定频率的方波,就能使扬声器发声。上面介绍的方法是使 PB_1 置 1,OUT₂ 端送出方波来驱动扬声器发声的。我们也可以用另一种方法使扬声器发声,那就是使 OUT₂ 输出为 1, PB_1 端输出方波,同样能使 U87 送出一定频率的方波。具体做法是先使 PB_0 置 0,再编程 PB_1 ,使它输出的电平在 1、0 之间来回变化。这是因为 PB_0 接到 8253-5 计数器 2 的门控输入端 GATE₂,当 8253-5 工作于方式 3,门控信号为低电平时,输出端 OUT₂ 恒为高电平,这样就允许 PB_1 产生的方波信号通过 U87 输出。实现这种发声方案的程序如下:

IN	AL, 61H	;读入 B 口状态
----	---------	-----------

```

        AND    AL, 11111100B      ;使 PB0 置 0, OUT2 输出高电平
BEEP:  XOR    AL, 02H             ;PB1 由 1→0 或由 0→1
        OUT    61H, AL
        MOV    CX, 320
HERE:   LOOP   HERE              ;延时
        JMP    BEEP

```

读者可以根据上面讨论的方法,通过编程来控制 8253-5 的定时时间,使扬声器发出不同频率的声音,并设法控制它发声的时间长短,以形成各种调子的声响。

五、并行打印机接口

与键盘和显示器那样,打印机也是 PC 机的一个最基本的外设。打印机的种类很多,常见的有针式打印机、喷墨打印机和激光打印机等。早期的打印机,由于打印速度较低,普遍采用 RS-232 标准的串行接口与计算机连接。近十几年来,随着 PC 机性能的不断提高,打印机技术也在迅速发展,针式打印机速度已达到每秒 100 个汉字,激光打印机则可在一分钟内完成 10 页 A4 纸的打印。因此,目前打印机都采用传送速率较高的并行接口与计算机相接。

在并行接口上,计算机送出的待打印数据和打印格式控制符等,都是以 ASCII 字符的形式,经 8 根并行的数据线传送给打印机的。打印机接收到这些数据后,将它们存进机内的 RAM 缓冲器,并由机内微处理器对它们进行分析,区分出被打印信息和控制命令。当它检测到回车符('0DH')时,便将一行字符打印出来;而在接收到换行('0AH')符时,便实现换行操作。由于计算机向打印机发送字符的速度比打印机打印这些字符的速度快得多,所以计算机必须用应答方式向打印机传送 ASCII 字符。若打印机缓冲器已满,打印机必须通知计算机,自己正处于忙碌状态,暂时不要再送任何新数据过来。当打印机缓冲器中出现一定的空间时,它再撤销忙碌标志,允许接受计算机的数据。

下面介绍一种采用 8255A 设计的并行打印机端口,说明怎样利用 8255A 工作于应答方式来实现并行接口的功能。

1. 打印机接口信号

并行打印机接口常采用 Centronics 标准,它的传输距离仅为 1.5 米。在 PC 机一侧采用标准的 25 针 D 型插座,与 RS-232 串行口的 DB25 插座外形相同;而在打印机一侧采用 36 芯的 AMP CHAPM36 双排插座。除 8 位数据线外,接口中至少还有选通(STROBE)、忙碌(BUSY)、应答(ACKNLG)等信号,有的还有出错、缺纸、总清等几个信号,其中有打印机输出的状态信息,也有计算机发出的控制信号。早期的 PC 机有专门的打印机适配卡,上面有一个打印机并行接口和一个并行插座,可被插入扩展槽,提供打印机接口。目前,在高档 PC 机上,并行接口包含在多功能卡上。MS-DOS 操作系统赋予并行口的逻辑设备名为 LPT,若系统上有一个以上的并行口,操作系统默认 LPT1 为打印机口。要是你将打印机接到第二个并行口上,必须通过软件设置,告诉操作系统打印机接在 LPT2 并行口上。

典型的打印机并行接口信号如表 9-3 所示。

表 9-3 打印机并行接口信号

信 号	名 称	方 向	说 明
DATA1~DATA8	数据	输入	8 位并行数据
$\overline{\text{STROBE}}$	选通脉冲	输入	低电平时将 8 位并行数据送到打印机的输入缓冲器中,脉宽 $>0.5\mu\text{s}$
$\overline{\text{SLCT IN}}$	选择输入	输入	只有当该信号为低电平时,打印机才能接收数据(出厂时一般将其置为低电平)
$\overline{\text{AUTO FEED XT}}$	自动走纸	输入	当该信号为低电平时,打印之后(遇回车符)自动走纸一行
$\overline{\text{INIT}}$	打印机初始化	输入	该信号为低电平时,打印机控制器复位到初始状态,并清除打印缓冲器,平时为高电平
$\overline{\text{ACKNLG}}$	应答	输出	负脉冲, $5\mu\text{s}$ 宽,表示数据已被接受,打印机准备接受下一个数据。
BUSY	忙碌	输出	高电平表示打印机不能接受数据,下列情况下变为高电平:1)数据输入期间;2)打印机操作期间;3)脱机状态;4)打印机出错状态
PE	缺纸	输出	高电平表示无打印纸
SLCT	选择状态	输出	高电平表示联机状态,低电平为脱机状态
$\overline{\text{ERROR}}$	出错	输出	当打印机处于出错、脱机或缺纸状态时,该信号变为低电平
GND	地		

注:表中,信号的方向是从打印机一端来观察的。

由上表可知,除地线外,打印机的信号主要分成 3 类:

- (1) 数据信号 DATA 1~DATA 8 为计算机送来的可打印字符或打印控制码的 ASCII 码。
 - (2) 打印机接收到的控制信号,包括 $\overline{\text{STROBE}}$ 、 $\overline{\text{SLCT IN}}$ 、 $\overline{\text{AUTO FEED XT}}$ 、 $\overline{\text{INIT}}$, 这是计算机送往打印机的信号,用于控制打印机的动作。
 - (3) 从打印机输出的状态信号,如 $\overline{\text{ACKNLG}}$ 、BUSY 等,用来说明打印机的工作状态。
- 图 9-17 给出了打印机传送字符时的波形图,具体工作过程如下:

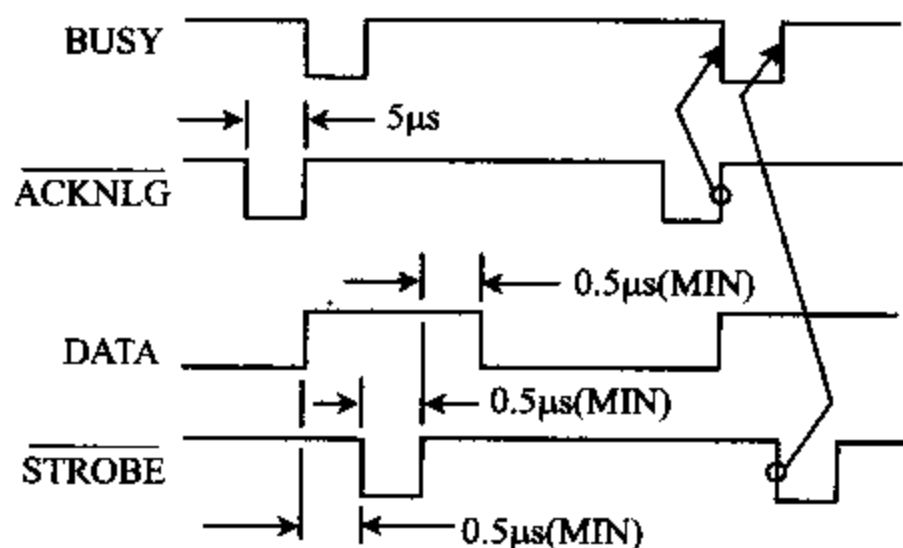


图 9-17 打印机传送字符时序图

打印机接通电源后,先在打印机处理器控制下完成初始化,此后在需要时,也可由计算机发出控制命令对它进行初始化。接着,计算机通过检测忙碌信号 BUSY,确定打印机是否已准备好接收数据。如果 BUSY 为低电平,说明打印机已准备好(不忙),便可通过 8 根并行数据线向打印机发送一个 ASCII 字符。经 $0.5\mu\text{s}$ 之后,选通信号 $\overline{\text{STROBE}}$ 变为有效的低电平,并维持 $0.5\mu\text{s}$,该信号用来通知打印机准备接收字符。当 $\overline{\text{STROBE}}$ 由低变高后,数据还必须在数据线上至少保持 $0.5\mu\text{s}$ 的时间。当数据被打印机接收之后,应答信号 $\overline{\text{ACKNLG}}$ 变低。应答信号的上升沿使 BUSY 变低,说明打印机又可接收下一个字符了。

2. 打印机接口电路

用 8255A 作接口芯片的打印机并行接口电路如图 9-18 所示。

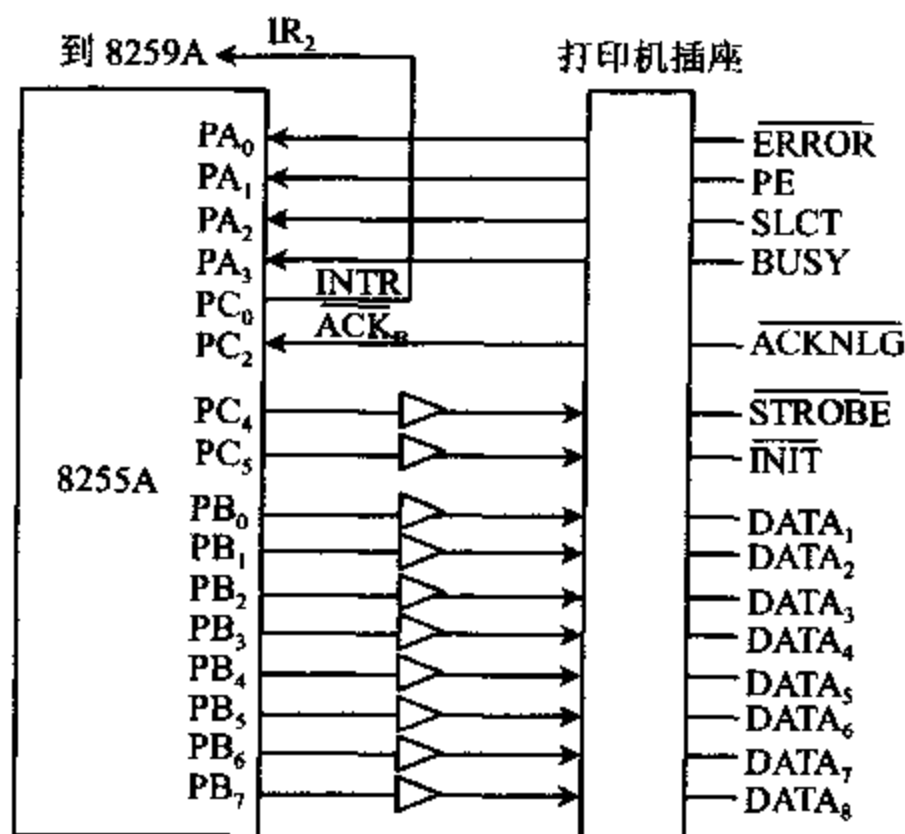


图 9-18 用 8255A 设计的打印机并行接口

图中,8255A 的端口 B 工作于选通输出方式, $\text{PB}_0 \sim \text{PB}_7$ 作数据输出线。 PC_0 位用来输出中断请求信号 INTR,它连到 8259A 的中断输入引脚 IR_2 。从打印机送来的应答信号 $\overline{\text{ACKNLG}}$ 连到 PC_2 脚。在选通输出方式下,常用 PC_1 位上的 $\overline{\text{OBF}}$ 信号作为选通信号,但在这个应用中,使用 $\overline{\text{OBF}}$ 不方便,所以将 PC_1 悬空,而用 PC_4 来形成 $\overline{\text{STROBE}}$,通过对这一位的置位/复位操作,产生所需的选通信号。

端口 A 工作于方式 0 输入,计算机可从 $\text{PA}_0 \sim \text{PA}_3$ 读入打印机的 4 个状态信号 $\overline{\text{ERROR}}$ 、PE、SLCT 和 BUSY,以确定打印机的状态。另外,8255A 的 PC_5 位接打印机的初始化命令输入端 $\overline{\text{INIT}}$,以便由程序来控制打印机的初始化。

在搞清了打印机接口的结构基础上,便很容易确定 8255A 的控制字。从上面的讨论可知,B 口工作于方式 1 输出,A 口工作于方式 0 输入,C 口的 PC_4 、 PC_5 为输出,故 C 口的高 4 位应初始化为输出。这样,方式选择控制字应为 10010100。此外,采用的是应答方式传送数据,在中断允许的前提下,当 B 口的输出缓冲器空,且 PC_2 脚上的打印机应答信号 $\overline{\text{ACKNLG}}$ 已由低变为高电平,即 BUSY 信号已为低电平时,8255A 便会从 PC_0 脚向 8259A 中断控制器的 IR_2 脚发出中断请求信号,由中断服务程序输出一个字符。为允许 B 口中断,

必须使 INTE B 置 1, 所以要用置位/复位控制字使 PC₂ 位置 1, 即写入置位/复位字 00000101。对 8255A 进行初始化时, 这两个控制字都必须写入控制字寄存器中。

3. 打印机驱动程序

在计算机中, 用来管理打印机的程序称为打印机驱动程序。下面以图 9-18 的硬件电路为基础, 介绍采用中断方式管理打印机的驱动程序的编写方法。

设该系统中, 8255A 的 A 口、B 口、C 口和控制字寄存器的端口地址分别为 2F8H、2FAH、2FCH 和 2FEH。中断控制器 8259A 的偶地址端口地址为 2F0H, 奇地址端口地址为 2F2H。程序需要设置一些内存单元, 用来存放向打印机传送数据时所需要的参数, 包括要打印的字符信息、字符串长度、地址指针及标志单元等。此外, 还要编写打印主程序和驱动打印机动作的中断服务子程序。

主程序先进行初始化操作, 包括初使化 8259A、8255A 和初使化打印机等。接着进入向打印机传送字符串的操作。开始传送 ASCII 字符前, 应先读入打印机状态, 当打印机处于正常的待命状态, 也即不忙 (PA₃ = 0)、联机 (PA₂ = 1)、有纸 (PA₁ = 0) 和无错 (PA₀ = 1) 时, 便将字符串的起始地址装入指针存储单元, 将字符串长度装入字符计数器, 并使 INTE B = 1, 允许 8255A 中断。然后, 程序处于等待状态, 等待打印机接口发出中断请求。如上所述, 当 8255A 的输出缓冲器空和打印机送来的 ACKNLG 信号为高电平时, 8255A 就产生中断请求, CPU 响应后, 使程序转入中断服务过程。

中断服务程序负责将一个字符送到打印机接口。它先把那些需要用到的寄存器的内容压入堆栈保护起来, 再从字符缓冲区中读入一个待打印的 ASCII 字符, 接着在 PC₄ 脚上形成一个低电平选通脉冲, 由它将字符打入打印机输入缓冲器。每产生一次中断, 须使字符串地址指针加 1, 字符计数器减 1。若字符计数器没有减为 0, 说明待打印的字符尚未传送完毕, 就将中断结束命令码 (EOI) 送往 8259A, 再从堆栈中弹出寄存器内容, 等待下次中断。当字符计数器减为 0 时, 字符缓冲区中的所有字符都已传送完, 就向 8255A 送一个清 PC₂ 位的置位/复位控制字, 禁止 8255A 再次产生中断, 然后返回断点。

如果要在打印机上打印字符串 "This is the Test!", 可用下列程序实现。

```

;=====
;                                PRINTER TEST
;=====
;
;端口地址分配表
PORT-A    EQU    2F8H           ;8255 端口 A
PORT-B    EQU    2FAH           ;8255 端口 B
PORT-C    EQU    2FCH           ;8255 端口 C
PORT-CTL   EQU    2FEH           ;8255 控制口
PORT-0     EQU    2F0H           ;8259 偶地址
PORT-1     EQU    2F2H           ;8259 奇地址
;数据段
DATA      SEGMENT
MESS-1    DB    "This is the Test !" ;打印信息
          DB    0DH, 0AH             ;回车换行

```

```

MESS-LEN    EQU    $-MESS-1                ;字符串长度
PRNT-DONE    DB     0                        ;打印完标志
POINTER      DW     0                        ;存储指向 MESS-1 的指针
COUNT       DB     0                        ;计数器
PRNT-ERR     DB     0                        ;出错标志
DATA         ENDS

;堆栈段
STACK        SEGMENT STACK
              DW     50 DUP (0)
              LABEL WORD
STACK        ENDS

;=====
;打印主程序
CODE          SEGMENT
              ASSUME CS:CODE, DS:DATA, SS:STACK
MAIN          PROC    FAR
              MOV     AX, STACK
              MOV     SS, AX
              LEA     SP, TOP
              MOV     AX, CS
              MOV     DS, AX                ;DS 指向代码段
              MOV     DX, OFFSET PRNT-INT  ;打印驱动过程入口地址偏移量送 DX
              MOV     AH, 25H              ;AH=系统调用功能号 25H
              MOV     AL, 0AH              ;AL=中断类型号 0AH
              INT     21H                  ;中断入口地址在 DS:DX 中

;初始化 8259A,使 IR2 中断允许
              MOV     DX, PORT-0          ;指向 8259A 偶地址
              MOV     AL, 00010011B        ;ICW1,边沿触发,单级,8086
              OUT     DX, AL               ;送出 ICW1
              MOV     DX, PORT-1          ;指向 8259A 奇地址端口
              MOV     AL, 00001000B        ;设置中断类型码
              OUT     DX, AL               ;送 ICW2
              MOV     AL, 00000001B        ;ICW4: 8086 模式,非自动 EOI
              OUT     DX, AL
              MOV     AL, 11111001B
              OUT     DX, AL                ;OCW1:只允许 IR2 和键盘中断

;初始化 8255A,B 口方式 1 输出,A 口方式 0 输入,C 口高 4 位为输出
              MOV     DX, PORT-CTL        ;8255A 控制字寄存器
              MOV     AL, 10010100B        ;控制字,格式如上所述
              OUT     DX, AL
              STI                             ;允许 8086 INTR 中断

;通过 PC4 向打印机送高电平选通信号
              MOV     AL, 00001001B        ;选通信号(PC4)置为高电平

```

```

                                OUT     DX, AL
;初始化打印机,从 PC5 引脚送出 50μs 宽的 INIT 负脉冲
                                MOV     AL, 00001011B
                                OUT     DX, AL                      ;置 INIT(PC5)为高电平
                                MOV     AL, 00001010B
                                OUT     DX, AL                      ;置 INIT 为低电平
                                MOV     CX, 17H
PAUSE-1:  LOOP     PAUSE 1                      ;延时 50μs
                                MOV     AL, 00001011B
                                OUT     DX, AL                      ;使 INIT 再度变为高电平
;从端口 A 读取打印机状态,已准备好状态应为 AL=XXXX0101
                                MOV     PRNT-ERR, 0                ;清除出错标志单元
                                MOV     DX, PORT- A                ;A 口地址
                                IN      AL, DX                      ;读取状态信息
                                AND     AL, 0FH                    ;取低 4 位
                                CMP     AL, 00000101B              ;打印机已准备好否?
                                JZ      SEND- IT                    ;已准备好,则将字符送出
                                MOV     CX, 16EAH                  ;否,延时
PAUSE-2:  LOOP     PAUSE-2                      ;等待 20ms 再读一次
                                IN      AL, DX                      ;再读
                                AND     AL, 0FH
                                CMP     AL, 00000101B
                                JZ      SEND- IT                    ;已准备好,转送出字符
                                MOV     PRNT-ERR, 1                ;仍未就绪,置出错标志
                                JMP     FIN                          ;终止程序
;已准备好,建立指向信息存储区的指针,已打印完标志清 0,表示未打印完
SEND- IT:  MOV     AX, OFFSET MESS-1
                                MOV     POINTER, AX                ;建立存储器信息指针
                                MOV     PRNT-DONE, 0                ;已打印完标志清 0
                                MOV     COUNT, MESS-LENG           ;置字符串长度
;置位 PC2,使 8255A 的 INTE B 置 1,允许中断
                                MOV     DX, PORT-CTL
                                MOV     AL, 00000101B
                                OUT     DX, AL                      ;置位 PC2,INTE 置 1
;等待打印机中断
WAIT-INT:  JMP     WAIT-INT
FIN:      NOP
                                MOV     AH, 4CH
                                INT     21H
MAIN      ENDP
;=====
;打印驱动中断服务子程序
PRNT-INT  PUSH     AX                      ;保护寄存器

```

```

        PUSH    BX
        PUSH    DX
        STI                                ;允许 8259A 更高级中断
        MOV     DX, PORT_B                ;8255 B 口地址
        MOV     BX, POINTER               ;装入指向的信息
        MOV     AL, [BX]                  ;取一个字符
        OUT     DX, AL                     ;向打印机送该字符
;通过 PC4 向打印机发选通负脉冲
        MOV     DX, PORT_CTL              ;控制口
        MOV     AL, 00001000B
        OUT     DX, AL                     ;PC4 送出低电平
        MOV     AL, 00001001B
        OUT     DX, AL                     ;再送出高电平
;增量地址指针,减量计数器
        INC     POINTER
        DEC     COUNT
        JNZ     NEXT                       ;字符都送完了吗? 未完转 NEXT
;字符已打印完,复位 PC2,禁止 8255 PC0 上的中断请求
        MOV     AL, 00000100              ;已送完,PC2 置 0
        OUT     DX, AL                     ;INTE B 置 0,禁止 8255A 中断
        MOV     PRNT-DONE, 1              ;将送完字符标志置 1
NEXT:    MOV     AL, 00100000B              ;非特殊 EOI 的 OCW2
        MOV     DX, PORT-0                ;指向 8259A 控制地址
        OUT     DX, AL                     ;结束中断
        POP     DX                         ;恢复寄存器
        POP     BX
        POP     AX
        IRET                                ;返回
CODE     ENDS
        END

```

习 题

1. 8255A 的 3 个端口在功能上各有什么不同的特点? 8255A 内部的 A 组和 B 组控制部件各管理哪些端口?
2. 8255A 有哪几种工作方式? 各用于什么场合? 端口 A、端口 B 和端口 C 各可以工作于哪几种工作方式?
3. 8255A 的方式选择字和置位复位字都写入什么端口? 用什么方式区分它们?
4. 若 8255A 的系统基地址为 2F9H,且各端口都是奇地址,则 8255A 的三个端口和控制寄存器的地址各是多少?

已知 CPU 的系统总线为 $A_9 \sim A_0, D_{15} \sim D_0, \overline{M/\overline{IO}}, \overline{IOR}, \overline{IOW}, \overline{RESET}$, 试画出 8255A 的地址译码电路及它与 CPU 系统总线的连线图。

5. 设 8255A 的 A 口, B 口, C 口和控制字寄存器的端口地址分别 80H, 82H, 84H 和 86H。要求 A 口工作在方式 0 输出, B 口工作在方式 0 输入, C 口高 4 位输入, 低 4 位输出, 试编写 8255A 的初始化程序。

6. 8255A 的端口地址同第 5 题, 要求 PC4 输出高电平, PC5 输出低电平, PC6 输出一个正脉冲, 试写出完成这些功能的指令序列。

7. 8255A 的端口地址同第 5 题, 若 A 口工作在方式 0 输入, B 口工作在方式 1 输出, C 口各位的作用是什么? 控制字是什么? 若 B 口工作在方式 0 输出, A 口工作在方式 1 输入, C 口各位作用是什么? 控制字是什么?

8. 若 A 口工作在方式 2, B 口工作在方式 1 输入, C 口各位作用是什么? 若 A 口工作在方式 2, B 口工作在方式 0 输出, C 口各位的作用是什么?

9. 8255A 的口地址为 80H~83H, 8253 的口地址为 84H~87H。

(1) 若 A 口接 8 个开关 $K_7 \sim K_0$, B 口接 8 个指示灯 $LED_7 \sim LED_0$, 当开关合上时相应的指示灯点亮, 断开时灯灭, 要求每隔 0.5 秒检测一次开关状态, 并在开关上显示出来, 试画出硬件连线图, 编写实现这种功能的程序。

(2) 若把接在端口 A 上的开关去掉, 要求接在端口 B 上的指示灯轮流熄灭, 每只灯熄灭 1 秒钟, 请编程实现这种功能。

10. 设 8255A 的口地址为 300H~303H, A 口接 4 个开关 $K_3 \sim K_0$, B 口接一个七段 LED 显示器, 用来显示 4 个开关所拨通的 16 进制数字 0~F, 开关都合上时, 显示 0, 都断开时显示 F, 每隔 2 秒钟检测一次, 试画出硬件连线图, 并编写实现这种功能的程序。

第十章 串行通信和可编程 接口芯片 8251A

10-1 串行通信的基本概念

计算机与外部的信息交换称为通信(Communication),基本的通信方式有两种,一种是并行通信,另一种是串行通信。

并行通信时,数据各位同时传送。例如,CPU 通过 8255A 与外设交换数据时,就采用并行通信方式。这种方式传输数据的速度快,但使用的通信线多,如果要并行传送 8 位数据,需要用 8 根数据线,另外还要加上一些控制信号线。随着传输距离的增加,通信线成本的增加将成为突出的问题,而且传输的可靠性随着距离的增加而下降。因此并行通信适用于近距离传送数据的场合。

在远距离通信时,一般都采用串行通信方式。它具有需要的通信线少和传送距离远等优点。串行通信时,要传送的数据或信息必须按一定的格式编码,然后在单根线上,按一位接一位的先后顺序进行传送。发送完一个字符后,再发送第二个。接收数据时,每次从单根线上一位接一位地接收信息,再把它们拼成一个字符,送给 CPU 作进一步处理。当微机与远程终端或远距离的中央处理机交换数据时,都采用串行通信方式。采用串行通信的另一个出发点是,有些外设,如调制解调器(MODEM)、鼠标器等,本身需要用串行方式通信,因为这些设备是以串行方式存取数据的。此外,有些外设,如打印机、绘图仪等,除了使用并行方式外,也可采用串行方式与计算机通信。不过,随着打印机的打印速度的提高,大多数打印机上已不再设置串行接口了。

下面介绍串行通信中涉及到的一些基本概念。

一、数据传送的方向

串行通信时,数据在两个站(或设备)A 与 B 之间传送,按传送方向可分成单工、半双工和全双工等三种不同的方式,如图 10-1 所示。

1. 单工 (Simplex)

单工数据线仅能在一个方向上传输数据,两个站之间进行通信时,一边只能发送数据,另一边只能接收数据,如图 10-1(a)所示。现在把这种通信方式称为单向通信。

2. 半双工 (Half Duplex)

在半双工方式中,数据可在两个设备之间向任一个方向传输,但两个设备之间只有一根传输线,故同一时间内只能在一个方向上传输数据,不能同时收发,如图 10-1(b)所示。无线电对讲机就是半双工传输的一个例子,一个人在讲话的时候,另一个人只能听着,因为一端在发送信息时,接收端的电路是断开的。

3. 全双工(Full Duplex)

如果在一个数据通信系统中,对数据的两个传输方向采用不同的通路,这样的系统就可以工作在全双工方式,如图 10-1(c)所示。采用全双工的系统可以同时发送和接收数据,电话系统就是全双工传送数据的一个例子。计算机的主机和显示终端(它由带键盘的 CRT 显示器构成)进行通信时,通常也采用全双工方式。一方面,键盘上敲入的字符可以送到主机内存,另一方面主机内存的信息可以送到显示终端。在键盘上敲入一个字符后,并不立即显示出来,而是等计算机收到该字符后,再回送给终端,由终端将该字符显示出来。这样,对主机而言,前一个字符的回送过程和后一个字符的输入过程是同时进行的,并通过不同的线路进行传送,即系统工作于全双工方式。

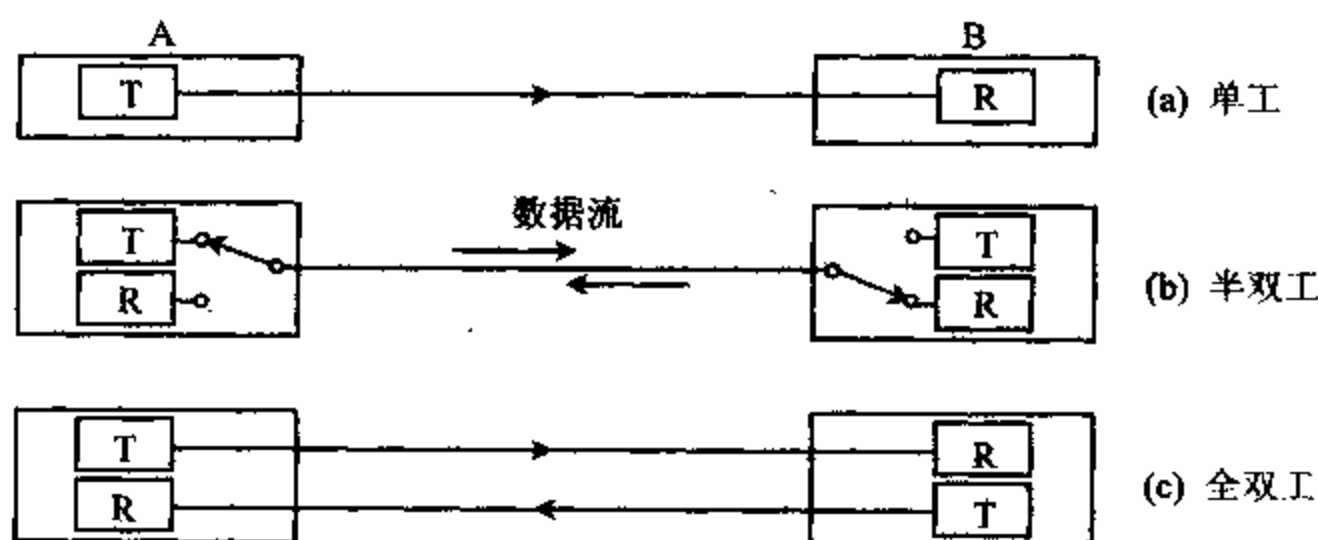


图 10-1 单工、半双工和全双工传送方式

二、串行传送的两种基本工作方式

串行通信有两种基本工作方式:异步方式和同步方式。

1. 异步方式 (Asynchronous)

采用异步方式时,数据发送的格式如图 10-2 所示。

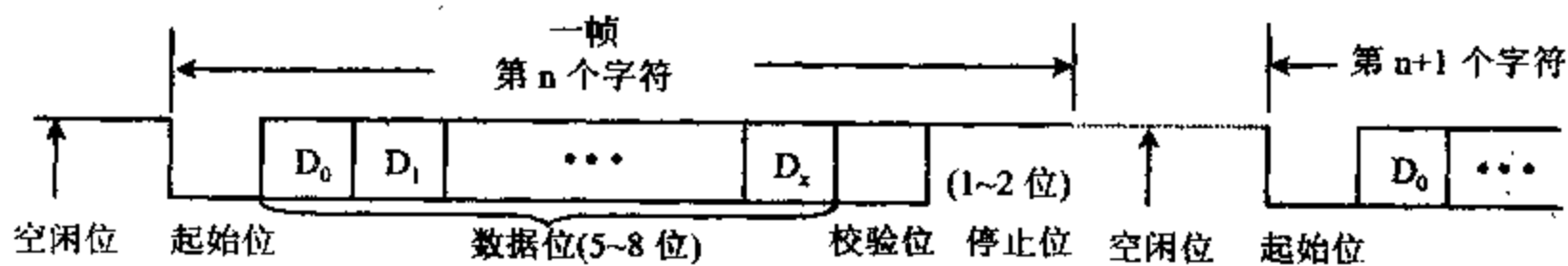


图 10-2 异步串行数据发送格式

不发送数据时,数据信号线总是呈现高电平,处于 MARK 状态,也称为空闲状态。当有数据要发送时,数据信号线变成低电平,并持续一位的时间,用于表示字符的开始,这一位称为起始位,这种低电平状态也称为 SPACE 状态。起始位之后,在信号线上依次出现待发送的每一位字符数据,最低有效位 D_0 最先出现,因此它被最早发送出去。采用不同的编码方案,待发送的每个字符的位数就不同,它可以由 5 位、6 位、7 位或 8 位构成。当字符用 ASCII 码表示时,数据位占 7 位($D_0 \sim D_6$)。在数据位的后面有一个奇偶校验位,加上这一

位后,使字符中“1”的位数为奇数(进行奇校验时)或偶数(进行偶校验时),系统中也可以不用奇偶校验位。在奇偶校验位的后面至少应有一位高电平表示停止位,用于指示字符的结束。停止位可以是一位,也可以是一位半或2位。如果传输完一个字符后立即传输下一个字符,那么后一个字符的起始位就紧跟在前一个字符的停止位之后,否则停止位后又进入空闲状态。

可见,用异步方式发送一个7位的ASCII字符时,实际需发送10位、10.5位或11位信息。如果用10位来发送的话,就意味着发送过程中将会浪费百分之三十的传输时间。为了提高串行数据传输的速率,可以采用同步传送方式。

2. 同步方式 (Synchronous)

串行同步字符的格式如图10-3所示。没有数据发送时,传输线处于MARK状态。为了表示数据传输的开始,发送方先发送一个或两个特殊字符,该字符称为同步字符。当发送方和接收方达到同步后,就可以一个字符接一个字符地发送一大块数据,而不再需要用起始位和停止位了,这样就可以明显地提高数据的传输速率。采用同步方式传送数据时,在发送过程中,收发双方还必须用同一个时钟进行协调,用于确定串行传输中每一位的位置。接收数据时,接收方可利用同步字符将内部时钟与发送方保持同步,然后将同步字符后面的数据逐位移入,并转换成并行格式,供CPU读取,直至收到结束符为止。

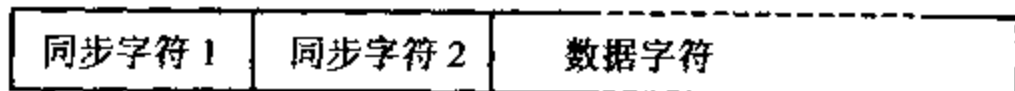


图 10-3 同步串行数据发送格式

上述同步传送方式中使用的数据格式是面向字符型的,同步字符也可以用一串特殊的位信息来代替,该信息称为标志字节。不过,这类面向位型的同步传送方式需要较复杂的控制协议的支持,我们将在本章最后进行讨论。

三、串行传送速率

在串行通信中,常用波特率(Baud Rate)来表示数据传送的速率。在计算机中,每秒钟内所传送数据的位数称为波特率,单位为波特(Bd),实际上它是传送每一位信息所用时间的倒数。如果一个串行字符由1个起始位,7个数据位,1个奇偶校验位和1个停止位等10个数位构成,每秒钟传送120个字符,则数据传送的波特率为:

$$10 \text{ 位/字符} \times 120 \text{ 字符/秒} = 1200 \text{ 位/秒} = 1200 \text{ 波特}$$

传送每位信息所占用的时间为:

$$1 \text{ 秒} / 1200 = 0.833 \text{ 毫秒}$$

常用的波特率为110、300、600、1200、2400、4800、9600和19200波特,它也是国际上规定的标准波特率。同步传送的波特率高于异步传送方式,可达到64000波特。

四、串行接口芯片 UART 和 USART

由于计算机是按并行方式传送数据的,当它采用串行方式与外部通信时,必须进行串并

行变换。发送数据时,需通过并行输入、串行输出移位寄存器将 CPU 送来的并行数据转换成串行数据后,再从串行数据线上发送出去;接收数据时,则需经串行输入、并行输出移位寄存器,将接收到的串行数据转换成并行数据后送到 CPU 去。另外,在传送数据的过程中,需要一些握手联络信号,确保发送方和接收方以相同的速度工作,同时还要检测传送过程中可能出现的一些错误等等,这就需要有专门的可编程串行通信接口芯片来实现这些功能。通过对这些接口芯片进行编程,可以设定不同的工作方式、选择不同的字符格式和波特率等。

常用的通用串行接口芯片有两类,一种是仅用于异步通信的接口芯片,称为通用异步收发器 UART(Universal Asynchronous Receiver-Transmitter),如 National INS 8250 就是这种器件,IBM PC 机中用 INS 8250 作串行接口芯片。另一种芯片既可以工作于异步方式,又可工作于同步方式,称为通用同步异步收发器 USART(Universal Synchronous-Asynchronous Receiver-Transmitter),如 Intel 8251A 就是这种器件。本章重点讨论 8251A 芯片的工作原理及使用方法。为使大家对 UART 的工作原理有所了解,下面简单介绍一种硬件 UART 的电路和工作过程。

一种通用的硬件 UART 的电路如图 10-4 所示。它由三部分组成:接收器、发送器和控制器。接收器把串行码转换为并行码,发送器把并行码转换成串行码,而控制器则用来接收 CPU 发来的控制信号,执行 CPU 所要求的操作,并输出状态信息和控制信息。

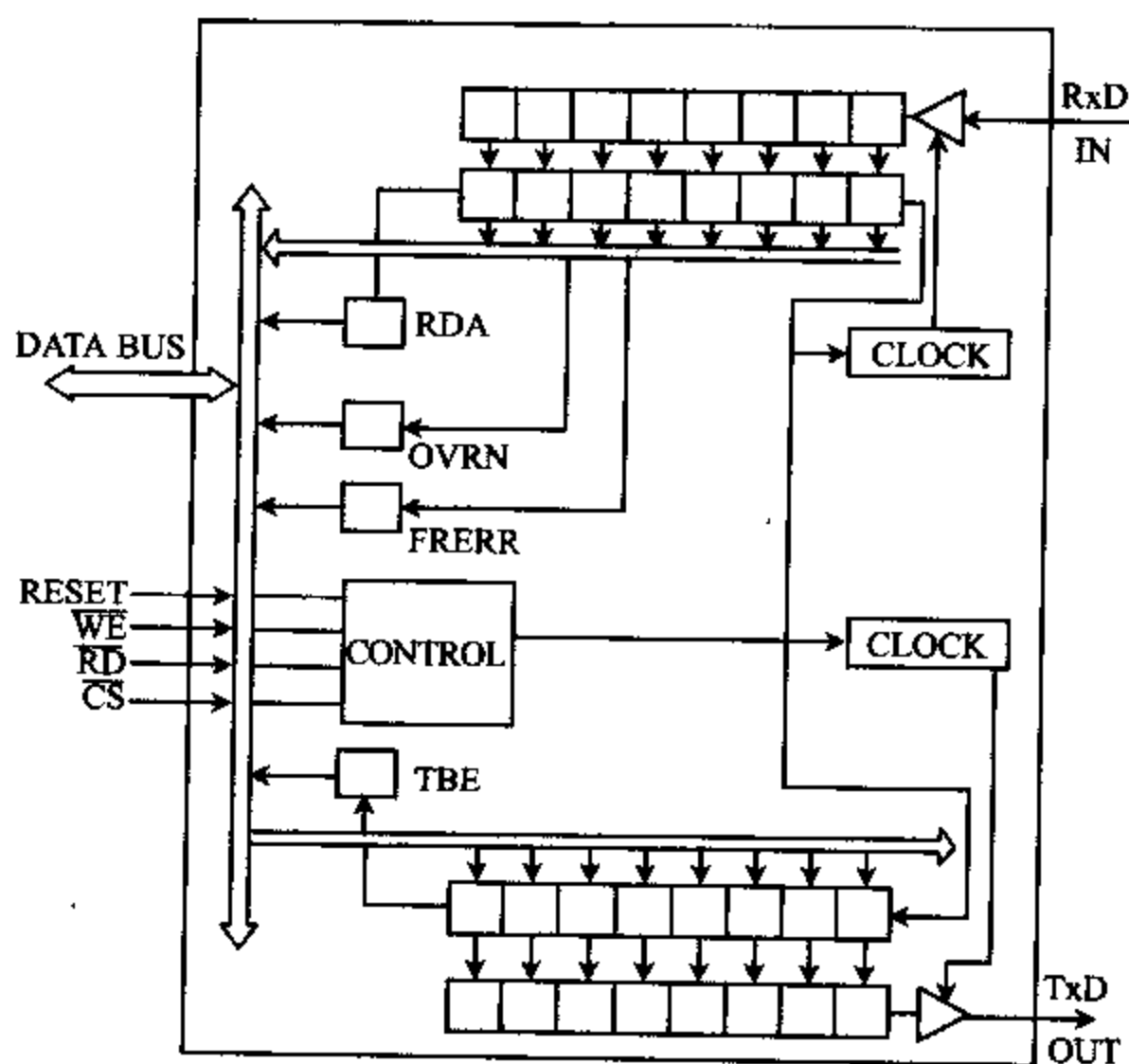


图 10-4 硬件 UART 电路图

UART 工作于接收方式时,其接收电路始终监视着串行输入端 RxD,当发现数据线上出现一个起始位时,就开始一个字符的接收过程。在时钟脉冲 CLOCK 的控制下,先逐位把

数据移入移位寄存器,再按相应的格式将串行数据转换成并行数据,送入并行寄存器中,等待 CPU 来读取。如果设置了奇偶校验位,在传送的过程中还能进行奇偶校验,如奇偶校验错,则置奇偶校验出错标志(图中未标出)。在接收的过程中,还能自动检测每个字符的停止位,若无停止位,则置帧出错标志 FRERR。如果传送的过程中,前一个字符还未取走,又送一个新的字符过来,便置溢出(丢失)标志 OVRN。

UART 工作于发送方式时,发送缓冲器把来自 CPU 的并行数据加上相应的控制信息,如起始位、停止位和奇偶校验位等,再在时钟脉冲的控制下,经并/串变换电路转换成串行数据后,从 TxD 引脚一位一位地发送出去。

为让 CPU 正确控制数据的接收与发送,电路中还设有接收数据就绪(RDA)和发送缓冲器空(TBE)等状态信息。

五、调制解调器

使用 UART 或 USART 接口芯片设计的串行接口,数据传输距离局限在数千英尺之内,不适宜于长距离传送。为了能长距离发送串行数据,常常利用标准电话线进行传送,因其传输线路及连接设备均已具备了。但电话线只能传送带宽为 300Hz~3000Hz 的音频信号,它不能直接传送频带很宽的数字信号。解决这个问题的方法是,在发送数据时,先把数字信号转换成音频信号后,再利用电话线进行传输,接收数据时又将音频信号恢复成数字信号。能将数字信号转换成音频信号及将音频信号恢复成数字信号的器件称为调制解调器,即 MODEM(Modulator-Demodulator)。

调制的主要形式有幅度(Amplitude)调制、频率键移 FSK(Frequency-Shift Keying)、相位键移 PSK(Phase-Shift Keying)和多路载波(Multiple Carrier)等几种,下面扼要介绍前两种调制方法。

1. 幅度调制

它用改变信号幅度的方法来表示数字信号 0 和 1。一种常用的调幅方法为:当接通频率为 387Hz 的正弦波时表示数字 1,断开正弦波时表示数字 0,调幅波形如图 10-5 所示。

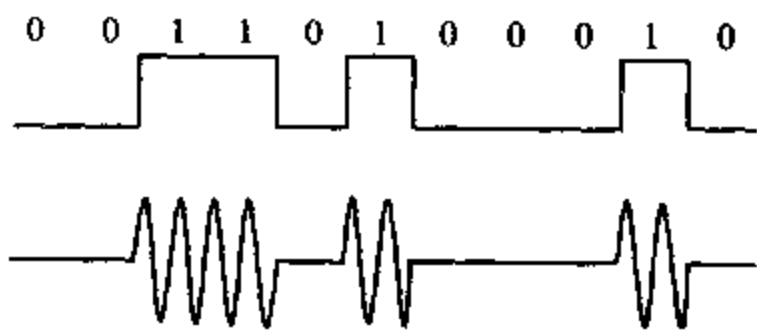


图 10-5 用调幅正弦波表示数字 1 和 0

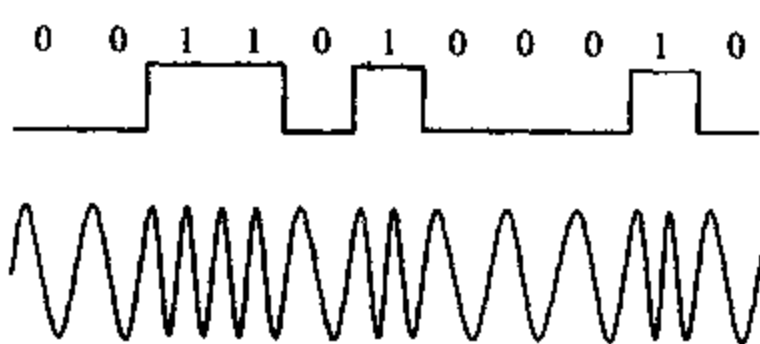


图 10-6 用两种不同频率(FSK)表示数字 1 和 0

幅度调制的另一种方法是,调幅时总是有正弦波输出,一种输出幅度表示数字 0,另一种幅度表示数字 1。幅度调制方式仅用于非常低速的反向通道传输,或与其它类型调制方法(如相位调制)协同使用。

2. 频率键移调制(FSK)

频率键移调制(FSK)是一种常用的调制方法,它用一种频率信号表示数字 0,另一种频

率信号表示数字 1, 波形如图 10-6 所示。

为了实现全双工通信, 常使用四种不同的频率来表示不同方向上的两种不同数字。例如, 对于 Bell 103A, 300Bd FSK MODEM 标准, 规定在一个方向上用 2025Hz 的频率表示 0, 用 2225Hz 表示 1; 在另一个方向上用 1070Hz 表示 0, 用 1270Hz 表示 1。该标准是基于贝尔电话公司的 DeFacto 标准。

当 MODEM 将数字 0 和 1 调制成音频信号时, 音频信号的频率必须落在电话线的带宽范围内, 而且音频的最大调制率为传输带宽的一半。比如, 假设一个两芯电话线的最大带宽为 2400Hz (不能超过 3000Hz), 则在该电话线上, 半双工信号的最大调制速率只有 1200Bd; 对于全双工通信而言, 每个方向传输要用一半带宽, 则在双芯电话线上每个方向上的最大调制速率只有 600Bd。如果使用四芯电话线, 每个方向有自己的线路, 因而每个方向的最大调制速率可达 1200Bd。所以, 采用这种简单的 FSK 调制方法, 只能局限于以 1200Bd 的速率在两芯电话线上进行半双工通信, 或以 1200Bd 的速率在四芯电话线上进行全双工通信。

如果要用更高的速率传输数据, 则要用相位键移和多路载波调制, 这两种方法较为复杂, 这里不做介绍了。

10-2 可编程串行通信接口芯片 8251A

8251A 是 Intel 公司生产的一种通用同步/异步数据收发器 (USART), 被广泛应用于以 Intel 8080、8085、8088、8086 和 8048 等为 CPU 的微型计算机中。它作为可编程通信接口器件, 能工作于全双工方式, 而且既可工作于同步方式, 又可工作于异步方式。

工作于同步方式时, 通过对 8251A 进行编程, 可选择每个字符的数据位数为 5~8 位, 数据传送的波特率为 DC (直流)~64K 位/秒, 还可选择内同步或外同步字符。

工作于异步传送方式时, 通过编程可选择每个字符的数据位数 (5~8 位)、波特率系数 (时钟速率与传输速率之比, 可为 1、16 和 64)、停止位位数 (1、1.5 或 2 位), 能检查假启动位, 并能自动产生、检测和处理中止符等。异步传送的波特率为 DC~19.2K 位/秒。

无论工作于同步方式还是异步方式, 均具有检测奇偶校验错、溢出错和帧错误的功能。下面介绍 8251A 的工作原理和使用方法。

一、8251A 的内部结构和外部引脚

8251A 的内部结构方框图和外部引脚图分别如图 10-7 和 10-8 所示。它内部由数据总线缓冲器、接收缓冲器、接收控制电路、发送缓冲器、发送控制电路、读/写控制逻辑和调制解调器等电路组成, 内部总线实现各部件相互间的通信。

8251A 内部各部件及外部各有关引脚的功能分述如下:

1. 数据总线缓冲器

它用作 8251A 与系统数据总线之间的接口, 内部包含 3 个三态、双向、8 位缓冲器, 它们是状态缓冲器、接收数据缓冲器和发送数据/命令缓冲器。前两个缓冲器分别用于存放 8251A 的状态信息和它所接收的数据, CPU 可用 IN 指令从这两个缓冲器中分别读取状态

信息和数据。发送数据/命令缓冲器用来存放 CPU 用 OUT 指令向 8251A 写入的数据或命令（控制）字。

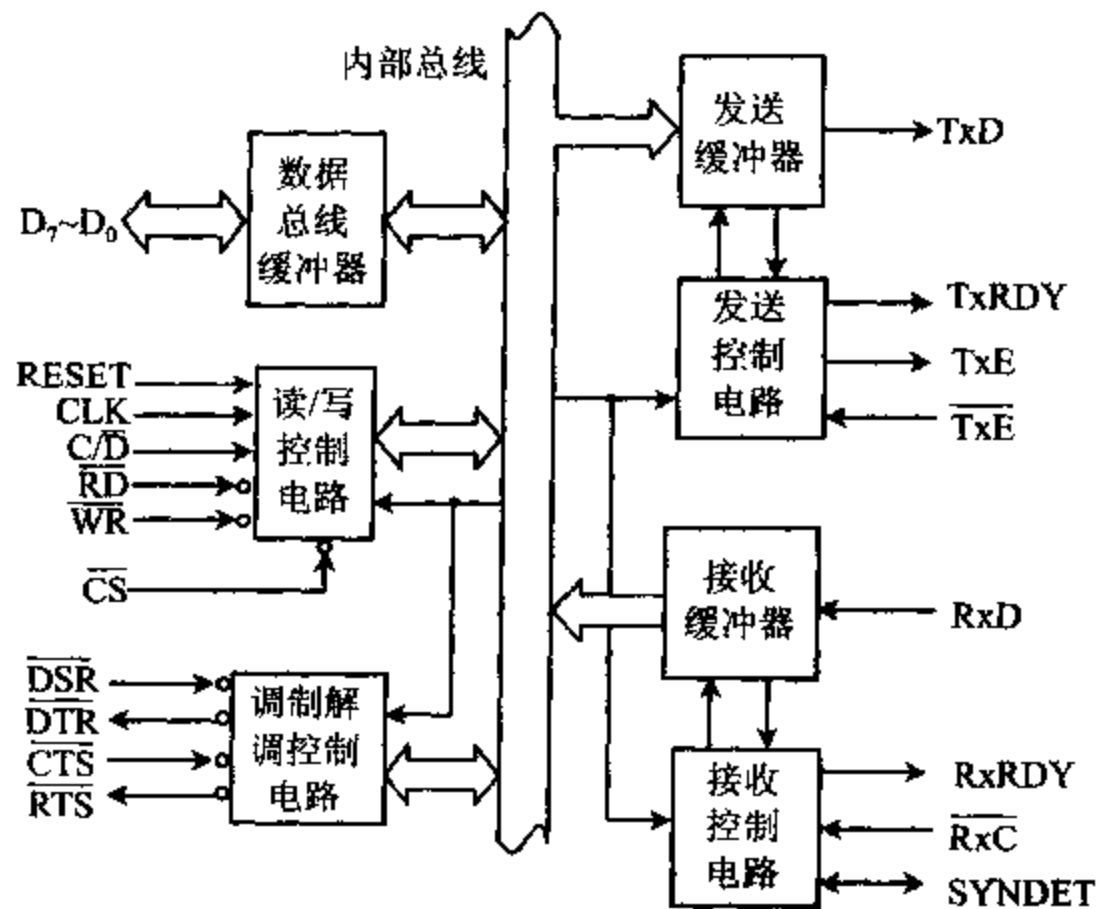


图 10-7 8251A 内部结构方框图

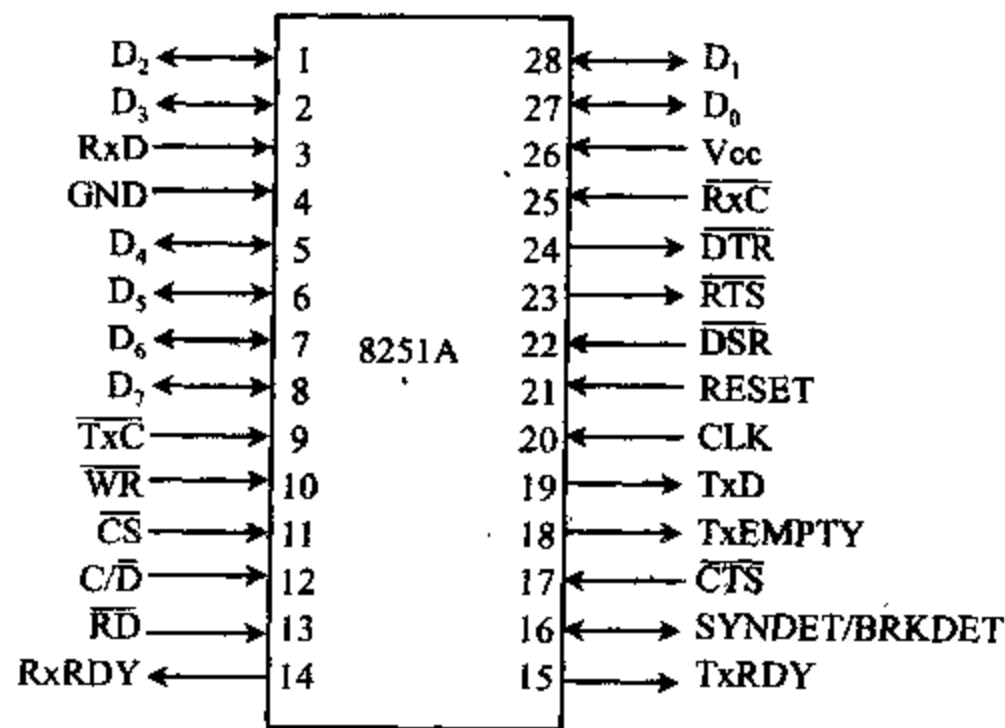


图 10-8 8251A 引脚图

与数据总线缓冲器有关的引脚有 $D_7 \sim D_0$ 数据线，它们与系统的数据总线相连，除用来在 8251A 和 CPU 间传送数据外，还传送 CPU 对 8251A 的编程命令和 8251A 送往 CPU 的状态信息。

2. 接收缓冲器和接收控制电路

接收缓冲器由接收移位寄存器、串/并变换电路和同步字符寄存器等构成，在时钟脉冲的控制下，它逐个接收从 Rx D 引脚上输入的串行数据，并将它们送入移位寄存器，待接收到一个字符数据后，通过串/并变换电路，将移位寄存器中的的数据变成并行数据，再通过内部

总线送到接收数据缓冲器中。接收数据的速率取决于送到接收时钟端 $\overline{\text{RxC}}$ 的时钟频率。

在异步方式下,接收时钟 $\overline{\text{RxC}}$ 的频率可以是波特率的 1、16 或 64 倍,或者说波特率系数为 1、16 或 64。使用比波特率高的时钟频率,能使接收移位寄存器在位信号的中间同步,而不是在信号起始边沿同步,这样将减少信号噪声在信号起始处引起读数错误的机会。

当 CPU 发出允许接收数据的命令时,接收缓冲器就一直监视着数据接收引脚 $\text{Rx}\overline{\text{D}}$ 上的信号电平。无信号时, $\text{Rx}\overline{\text{D}}$ 为高电平,一旦检测到 $\text{Rx}\overline{\text{D}}$ 为低电平,就启动接收控制器中的内部计数器,对时钟频率进行计数。若接收时钟频率为波特率的 16 倍,则计数器计到半个数位传输时间,也就是计到 8 时,如检测到 $\text{Rx}\overline{\text{D}}$ 引脚上仍是低电平,就确认收到了一个有效的起始位,而不是干扰信号。在这之后,8251A 每隔一个数位传送时间,也就是 16 个时钟周期,就对 $\text{Rx}\overline{\text{D}}$ 进行一次采样,8251A 对数据的采样过程如图 10-9 所示。

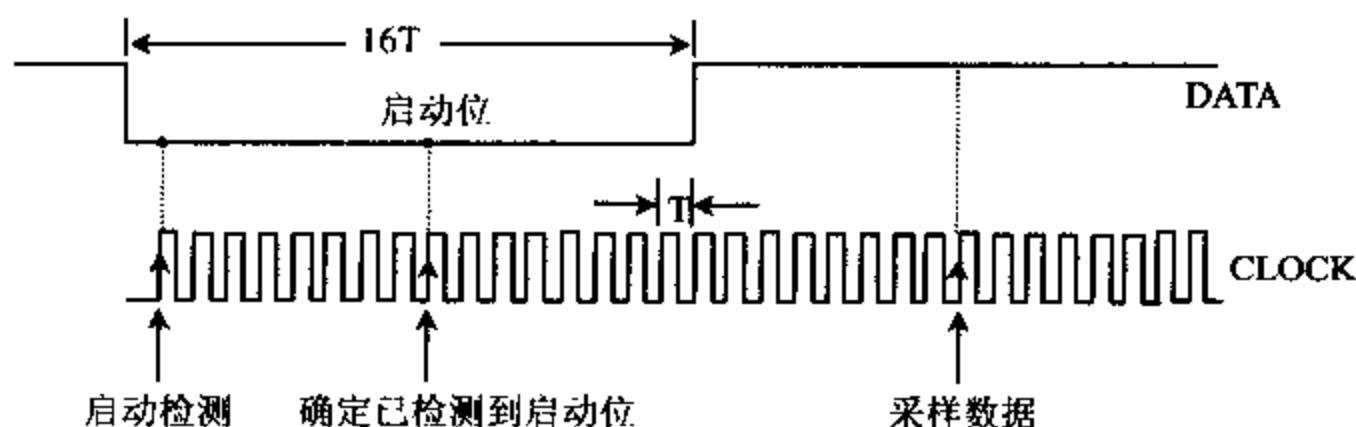


图 10-9 8251A 对数据的采样过程

采集到的数据被送到输入移位寄存器中,并被移位和进行奇偶校验(如果设置奇偶校验的话)等操作,然后删除起始位和停止位,得到并行数据,再经内部总线送到数据总线缓冲器中的接收数据缓冲器。这时使 RxRDY 引脚输出高电平,用来通知 CPU,8251A 已从外部接收一个字符,等待送到 CPU 去。当 RxRDY 引脚上产生高电平时,芯片内部状态寄存器中的 RxRDY 位也被置成高电平。

对于同步传送方式,又分内同步和外同步两种情形。

工作于内同步方式时,CPU 发出允许接收和进入搜索命令后,就一直监测 $\text{Rx}\overline{\text{D}}$ 引脚,把接收到的每一位数据送入移位寄存器,并与同步字符寄存器的内容进行比较。若两者不相同,则继续接收数据和进行移位比较等操作;若两者相同,则 8251A 将 SYNDET 引脚置为高电平,表示已实现同步过程。如果对 8251A 编程为采用双同步字符方式工作,则需搜索到两个同步字符后,才认为已实现同步。

若采用外同步方式工作,则由外部电路来检测同步字符。外部检测到同步字符后,就从同步输入端 SYNDET 输入一个高电平,通知 8251A,当前已检测到同步字符,8251A 就会立即脱离对同步字符的搜索过程。只要 SYNDET 上的高电平能维持一个 $\overline{\text{RxC}}$ 时钟周期,8251A 便认为已达到了同步。实现同步之后,接收器才能接收同步数据。首先,接收器利用时钟信号对 $\text{Rx}\overline{\text{D}}$ 线进行采样,然后把采得的数据送到移位寄存器中,每当接收到的数据位数达到一个字符所规定的数位时,就将移位寄存器里的内容经内总线送到输入缓冲器中,同时使 RxRDY 引脚上输出高电平,表示已收到一个可用字符。

与接收端有关的信号有:

(1) $\text{Rx}\overline{\text{D}}$ (Receiver Data) 接收数据,输入

外部串行数据从 RxD 引脚逐位移入接收移位寄存器中,经串并变换,变成并行数据后,便进入接收数据缓冲器,等待输入到 CPU 去。

(2) RxRDY(Receiver Ready) 接收数据准备好,输出,高电平有效

此信号有效时,表示接收数据缓冲器中已收到一个字符数据,可将其输入到 CPU 去。当 8251A 与 CPU 之间采用中断方式传送数据时,RxRDY 可作为中断请求信号,由中断服务程序用 IN 指令读入数据;在查询方式时,此信号可作为一个状态信号,当程序查到此信号为高电平时,由 IN 指令读取数据。每当 CPU 从 8251A 的接收数据缓冲器中读取一个字符后,RxRDY 就复位为低电平,等到下次接收到一个新字符后,才又变为有效的高电平。

(3) SYNDET/BRKDET(Sync Detect/Break Detet) 同步检测/断点检测,输入或输出

8251A 工作于同步方式和异步方式时,该引脚有不同的用处。

当 8251A 工作于同步方式时,它用于同步检测,系统复位时,此引脚变成低电平。对于内同步方式,它为输出信号。如 8251A 检测到了同步字符(在双同步字符情况下,需检测到第二个同步字符)后,SYNDET 输出高电平,表明 8251A 已达到了同步状态。CPU 执行一次读状态操作后,SYNDET 被自动复位。对于外同步方式,SYNDET 为输入信号,该引脚由低电平变为高电平时,使 8251A 在下一个 $\overline{RxC}$ 的上升沿开始接收字符,一旦达到同步,SYNDET 端的高电平可以去除。对 8251A 编程为外同步检测时,内同步检测是被禁止的。

当 8251A 工作于异步方式时,该引脚为断点检测端,BRKDET 是输出信号。每当 8251A 从 RxD 端连续收到两个由全 0 数位组成的字符(包括起始、停止和奇偶校验位)时,该引脚输出高电平,表示当前线路上无数据可读,只有当 RxD 端收到一个“1”信号或 8251A 复位时,BRKDET 才复位,变成低电平。断点检测信号可作为状态位,由 CPU 读出。

(4) $\overline{RxC}$ (Receiver Clock) 接收时钟,由外部输入

$\overline{RxC}$ 决定 8251A 接收数据的速率,当 8251A 工作于同步方式时, $\overline{RxC}$ 端输入的时钟频率应等于接收数据的波特率。如采用异步工作方式,它的频率可以是波特率的 1 倍、16 倍或 64 倍。接收时钟应与对方的发送时钟相同。

3. 发送缓冲器和控制电路

当 CPU 要向外部发送数据时,先用 OUT 指令把要发送的数据经 8251A 的发送数据缓冲器并行输入并锁存到发送缓冲器中,再由发送缓冲器中的移位寄存器将并行数据转换成串行数据后,经 TxD 引脚串行发送出去。

对于异步传送方式,发送控制器能按程序规定的字符格式,给发送数据加上起始位、奇偶校验位和停止位,然后从起始位开始,经移位寄存器移位后,逐位将数据从数据输出线 TxD 上发送出去。发送速率取决于 $\overline{TxC}$ 引脚上接的发送时钟频率, $\overline{TxC}$ 的频率可以是发送波特率的 1 倍、16 倍或 64 倍。

对于同步传送方式,发送器在发送数据字符之前,先送出 1 个或 2 个同步字符,然后逐位输出串行数据。在同步发送时,字符之间是不允许存在空隙的,若由于某种原因(如出现更高优先级的中断)迫使 CPU 在发送过程中停止发送字符,8251A 将不断地插入同步字符,直到 CPU 送来新的字符后,再重新输出数据。同步传送时,数据传输率等于 $\overline{TxC}$ 的时钟频率。

与发送端有关的信号有:

(1) TxD(Transmitter Data) 发送数据,输出

8251A 把 CPU 送来的并行数据转换成串行格式后,逐位从 TxD 引脚发送到外部。

(2) TxRDY(Transmitter Ready) 发送器准备好,输出,高电平有效

当允许 8251A 发送数据,而且数据总线缓冲器中的发送数据/命令缓冲器为空时,TxRDY 有效,它表示发送缓冲器已准备好从 CPU 接收一个数据。对于中断传送方式,TxRDY 有效时请求中断,由中断服务程序用 OUT 指令从 CPU 输出一个数据到 8251A。在查询方式下,TxRDY 可作为状态信号,当 CPU 检测到该信号有效时,才向 8251A 输出一个数据。当 CPU 向 8251A 输出一个并行数据后,TxRDY 被清为低电平。

(3) TxE(Transmitter Empty) 发送器空,输出,高电平有效

当发送器空信号有效时,表示 8251A 发送器中的并行到串行转换器空,也即已完成一次发送操作,缓冲器中已无数据可向外部发送。在异步传送方式,由 TxD 引脚向外部输出空闲位;在同步传送方式,由于不允许在字符间留有空隙,所以就由 TxD 引脚向外部输出同步字符。当 8251A 从 CPU 接收到一个数据后,TxE 便成为低电平。

(4) $\overline{\text{TxC}}$ (Transmitter Clock) 发送器时钟,输入

$\overline{\text{TxC}}$ 确定 8251A 的发送速率。对于同步方式, $\overline{\text{TxC}}$ 端输入的时钟频率应等于发送数据的波特率;对于异步方式,可由软件定义发送的时钟是波特率的 1 倍、16 倍或 64 倍。

4. 读/写控制电路

读/写控制电路用来接收 CPU 的控制信号和控制命令字,决定 8251A 的工作状态,并向 8251A 内部其余功能部件发相应的控制信号。从 CPU 送到读/写控制电路的控制信号有:

(1) RESET 复位信号,输入,高电平有效

RESET 信号有效时,8251A 进入空闲(Idle) 状态,等待对芯片进行初始化编程。在用指令对 8251A 写入复位命令字后,也能使它进入空闲状态。

(2) CLK 时钟,输入

时钟信号用来产生 8251A 内部的定时信号。对于同步方式,CLK 的频率必须比 $\overline{\text{TxC}}$ 和 $\overline{\text{RxC}}$ 大 30 倍,对于异步方式,CLK 的频率应比 $\overline{\text{TxC}}$ 和 $\overline{\text{RxC}}$ 大 4.5 倍。

(3) $\overline{\text{WR}}$ 写,低电平有效

当 $\overline{\text{WR}}$ 为低电平时,表示 CPU 正在把数据或控制字写入 8251A。

(4) $\overline{\text{RD}}$ 读,低电平有效

当 $\overline{\text{RD}}$ 为低电平时,表示 CPU 正从 8251A 读出数据或状态信息。

(5) $\overline{\text{CS}}$ 片选信号,低电平有效

$\overline{\text{CS}}$ 为低时,8251A 芯片被选中,可以对它进行读写操作。当 $\overline{\text{CS}}$ 为高时,数据总线处于浮空状态,不能对该芯片进行读写操作。 $\overline{\text{CS}}$ 由地址译码电路产生。

(6) C/ $\overline{\text{D}}$ (Control/Data) 控制/数据信号,输入

C/ $\overline{\text{D}}$ =1 时,表示当前通过数据总线传送的是控制信息或状态字,当 C/ $\overline{\text{D}}$ =0 时,传送的是数据信息。C/ $\overline{\text{D}}$ 、 $\overline{\text{RD}}$ 、 $\overline{\text{WR}}$ 和 $\overline{\text{CS}}$ 这几个信号组合起来组成的读写操作如表 10-1 所示。

表 10-1 8251A 读写操作表

C/D	$\overline{RD}$	$\overline{WR}$	$\overline{CS}$	操 作
0	0	1	0	CPU 从 8251A 读数据
0	1	0	0	CPU 向 8251A 写数据
1	0	1	0	CPU 读取 8251A 的状态字
1	1	0	0	CPU 向 8251A 写入控制字
×	1	1	0	数据总线浮空
×	×	×	1	数据总线浮空

由上表可知,对 8251A 读写数据时, $C/\overline{D}=0$,选择数据端口;向 8251A 写入控制字或从 8251A 读取状态字时, $C/\overline{D}=1$,使用控制端口。

5. 调制解调器控制电路

当终端和远程计算机或计算机与远程中央处理机之间进行通信时,可用 8251A 作远距离通信接口芯片,并需与调制解调器相连,经标准电话线传输数据。8251A 用 4 条信号线: $\overline{DTR}$ 、 $\overline{DSR}$ 、 $\overline{RTS}$ 和 $\overline{CTS}$ 来实现与 MODEM 之间的通信联络,联络方式分为同步方式和异步方式两种。在异步方式时, $\overline{RxC}$ 和 $\overline{TxC}$ 信号的频率可以是波特率的 1 倍、16 倍或 64 倍,它们由波特率产生器提供,也可用时钟信号 CLK 经 8253 分频后形成。8251A 与异步 MODEM 的连接如图 10-10 所示。8251A 与同步 MODEM 的连接图与之十分相似,不同之处仅在于同步方式时 $\overline{RxC}$ 和 $\overline{TxC}$ 信号直接由调制解调器提供,其频率与波特率的数值相等。

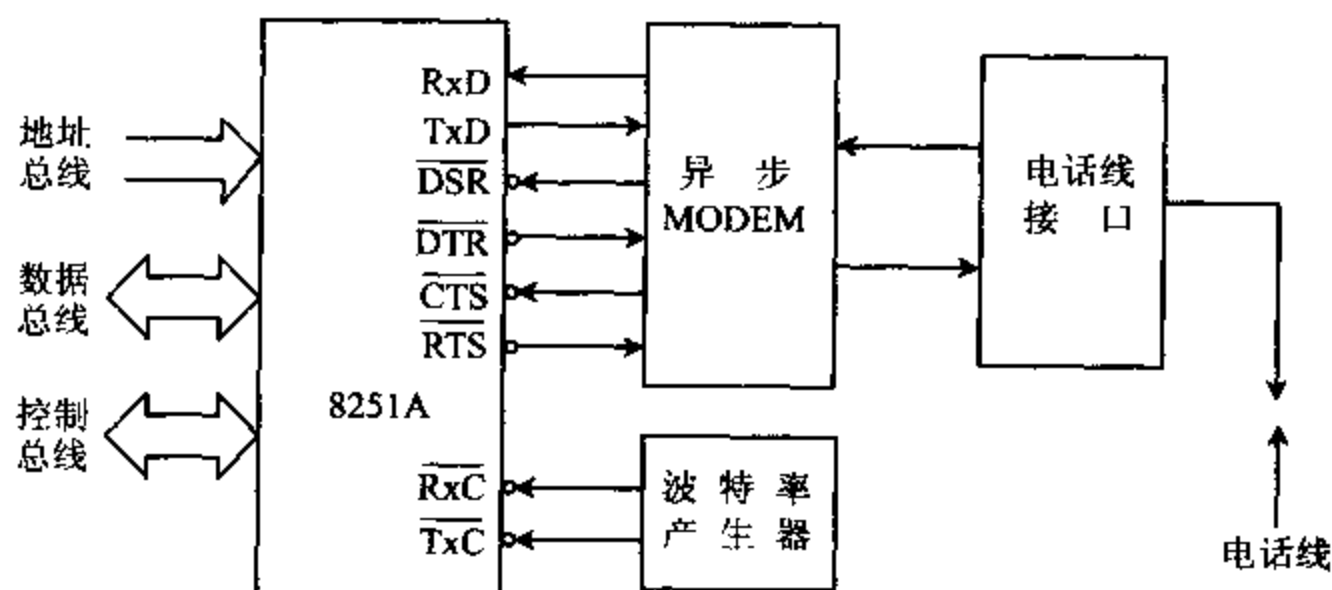


图 10-10 8251A 与异步 MODEM 连接图

通常,将 MODEM 和其它用于远距离发送串行数据的设备称为数据通信设备 DCE (Data Communication Equipment),用于收发数据的终端和计算机称为数据终端设备 DTE (Data Terminal Equipment)。

8251A 与 MODEM 接口的 4 条信号线的功能分述如下:

(1) $\overline{DTR}$ (Data Terminal Ready) 数据终端准备好,输出,低电平有效

当终端电源接通,做好接收数据的准备工作后,就可向 MODEM 发出有效的 $\overline{DTR}$ 信号,告诉 MODEM 数据终端已准备好。它可用软件定义,只要使控制字中的 DTR 位置 1,就能使 $\overline{DTR}$ 引脚上产生有效的低电平。

(2) $\overline{\text{DSR}}$ (Data Set Ready) 数据装置准备好,输入,低电平有效

$\overline{\text{DSR}}$ 有效时,表示 MODEM 已准备好数据,实际上它是对 $\overline{\text{DTR}}$ 的回答信号,CPU 可用 IN 指令读入 8251A 状态寄存器中 DSR 位的内容,来检测 $\overline{\text{DSR}}$ 的状态,当 $\text{DSR}=1$ 时,表示 $\overline{\text{DSR}}$ 引脚上产生了有效的低电平。

(3) $\overline{\text{RTS}}$ (Request To Send) 请求发送信号,输出,低电平有效

$\overline{\text{RTS}}$ 有效时,表示计算机或终端已准备好数据,需要发送,该信号向 MODEM 发出请求发送信号。它可用软件定义,使命令字中的 RTS 位置 1,则 $\overline{\text{RTS}}$ 引脚上将产生有效的低电平信号。

(4) $\overline{\text{CTS}}$ (Clear To Send) 清除发送信号,输入,低电平有效

当 MODEM 收到 $\overline{\text{RTS}}$ 命令,完全做好了发送串行数据的准备之后,就向终端回送 $\overline{\text{CTS}}$ 低电平信号。这时,如果控制字中 TxEN 位=1,表示 8251A 的发送缓冲器中已收到 CPU 的一个数据,发送器就可以发送串行数据。所以实际上 $\overline{\text{CTS}}$ 是对 $\overline{\text{RTS}}$ 的回答信号。当终端发送完所有字符后,使 $\overline{\text{CTS}}$ 信号变高,发送过程结束。要是在数据的发送过程中清除 $\overline{\text{CTS}}$,也就是使 $\overline{\text{CTS}}$ 无效或者使 TxEN=0,那么发送器将正在发送的字符发送完后就停止继续发送。

在实际使用时,上述四个控制信号是通用的,如果需要的话,它们可以被赋予不同的物理意义,用于调制解调器以外的功能。

6. 8251A 与 CPU 及外设的连接

假设我们用 8251A 构成的串行接口与 CRT 显示器或鼠标器等外设相连,并工作于异步方式,这时,就不需要用到上述那些控制 MODEM 的信号。图 10-11 是 8251A 与 CPU 及某个具有串行接口的外设的连线示意图。

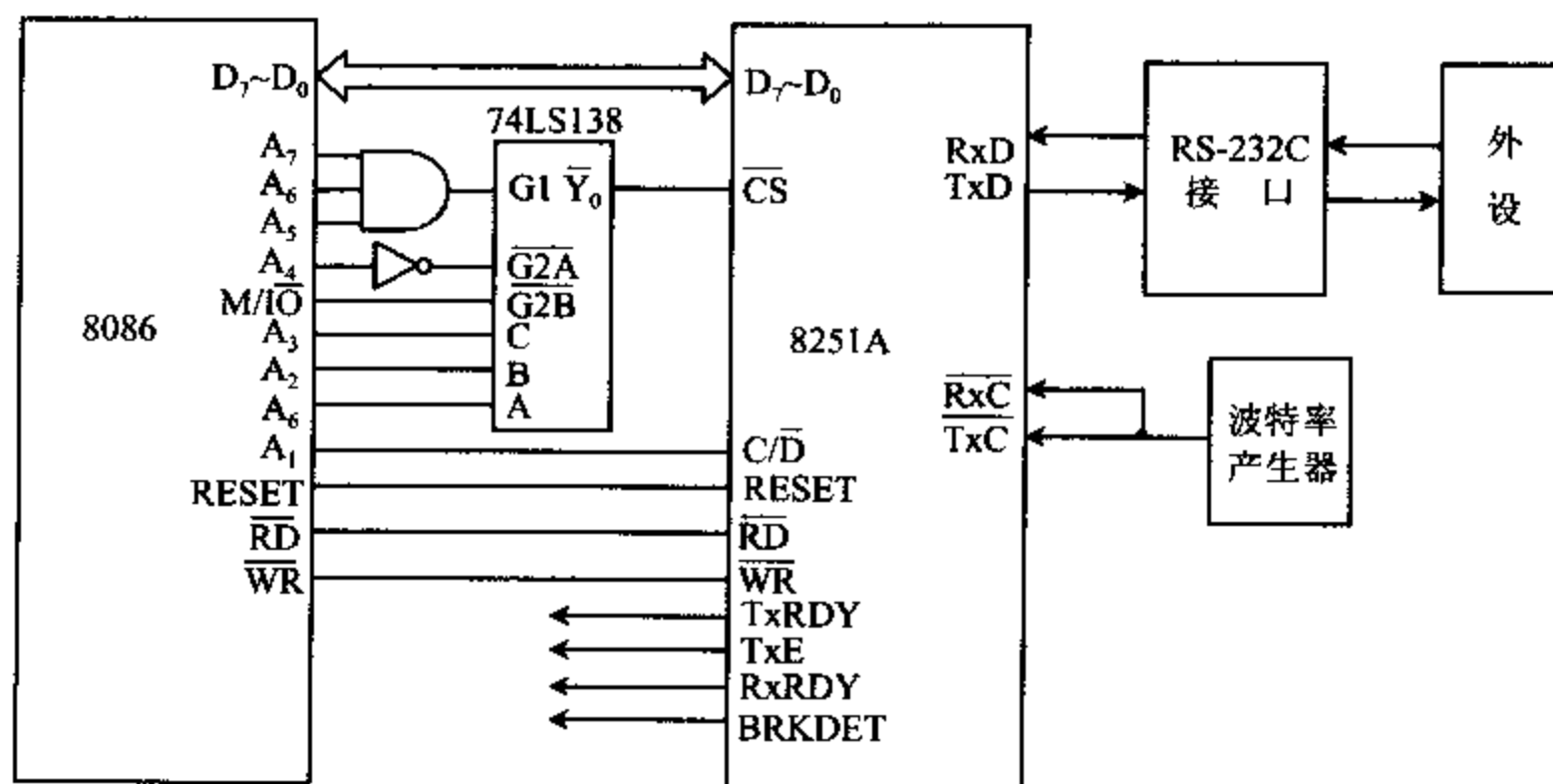


图 10-11 8251A 与 CPU 及外设的连线图

从图中可以看到,8251A 的信号可分成两组,一组是 8251A 与 CPU 之间的接口信号,另一组是它与外设之间的接口信号。

(1) 8251A 与 CPU 之间的连线

如图,8251A的 $\overline{RD}$ 、 $\overline{WR}$ 、CLK和RESET信号可直接与CPU的相应引脚相连。而8251A的数据线 $D_7 \sim D_0$ 与CPU的低8位数据总线 $D_7 \sim D_0$ 相连, $C/\overline{D}$ 与系统地址总线 A_1 相连。地址总线的其余位经译码后产生片选信号,与8251A的 $\overline{CS}$ 相连。

$TxRDY$ 、 TxE 、 $RxRDY$ 和BRKDET都是8251A的输出信号,它们是CPU与8251A之间的收发联络信号。在查询方式时,它们被用作状态信号;在中断方式时, $TxRDY$ 和 $RxRDY$ 可作为向CPU请求发送或接收数据的中断请求信号。

(2) 8251A与外设之间的连线

在这组引线中, RxD 接收外设送来的串行数据, TxD 向外设发送串行数据。为了符合RS-232C串行接口标准对信号电平的要求,在接口电路中具有专门的电平变换电路,发送数据时,将TTL电平的 TxD 信号转换为RS-232C电平,接收数据时,则把RS-232C电平的 RxD 信号转换为8251A能接受的TTL电平。

发送时钟输入端 $\overline{TxC}$ 和接收时钟输入端 $\overline{RxC}$ 连在一起,由波特率产生器为它们提供所需要的时钟脉冲信号。

(3) 端口地址译码电路

我们已经知道8251A是具有8位数据总线的接口芯片, $C/\overline{D}$ 端为1时,选中控制口,传送控制信号和状态信号; $C/\overline{D}=0$ 时,选中数据口,传送数据信息。如果8251A与8位数据总线的微机相连,通常只要将 $C/\overline{D}$ 与最低位地址线 A_0 相连,高位地址线用于端口地址译码,产生片选信号。这样,当 $A_0=0$ 时,选中数据口; $A_0=1$ 时,选中控制口。例如:数据口为F0H,控制口为F1H。

如果8251A与16位数据总线的CPU(如8086)相连,则它既可以与数据总线的高8位相连,也可与低8位相连,但是必须遵守以下约定:低8位数据总线总是与偶地址单元或端口相连,而高8位数据总线总是与奇地址单元或端口相连。若要使8位接口芯片仅与低8位数据总线相连,同时又要能区分8251A的控制口和数据口,只要让地址总线的最低位 A_0 参加I/O端口译码,只有在 $A_0=0$ 时,才选中8251A芯片。这样可保证CPU对8251A进行读写操作时,8251A的口地址必为偶地址,符合低8位数据总线总是与偶地址端口相连的约定。另一方面,将地址总线的次低位 A_1 与8251A的 $C/\overline{D}$ 端相连,用于选择8251A的控制口或数据口,当 $A_1=0$ 时,选中数据口; $A_1=1$ 时,选中控制口。

从图10-11的译码电路可以看到, $A_7A_6A_5A_4=1111$ 和 $M/\overline{IO}=0$ 时,使74LS138的 $G1=1$, $\overline{G2A}=\overline{G2B}=0$,译码器74LS138的使能信号有效。CPU为了选中偶地址端口, A_0 必须为0,若 $A_3A_2A_0=000$,则74LS138的 $\overline{Y}_0$ 有效,可选中8251A。由于 A_1 未参加译码,它可以是1或者0,于是8251A的口地址可以为F0H和F2H。当 $A_1=0$ 时,8251A的口地址为F0H,选中数据口;当 $A_1=1$ 时,8251A的口地址为F2H,选中控制口。

二、8251A的编程

8251A是一种多功能的串行接口芯片,使用前必须向它写入方式字及命令字等,对它进行初始化编程后,才能收发数据,使用中可以利用状态字来了解它的工作状态。方式字用来确定8251A的工作方式,如规定它工作于同步还是异步方式,传送的波特率及字符长度各是多少,是否允许奇偶校验等。命令字控制8251A按方式字所规定的方式进行工作,如允

许或禁止 8251A 收发数据,启动搜索同步字符,迫使 8251A 内部复位等。

下面先给出对 8251A 进行初始化编程的流程图,然后对方式字、命令字及状态字的格式分别进行介绍。

1. 8251A 的编程流程图

当系统上电后用硬件电路使 8251A 复位,或通过软件编程使它复位后,就可对 8251A 进行初始化编程了。对 8251A 进行初始化编程的流程图如 10-12 所示。

由图可见,系统复位后首先应将方式字写入控制口,它用来确定 8251A 的工作方式。若将 8251A 置成同步工作方式,则在方式字后,需往控制口写入 1 个或 2 个同步字符,同步字符的个数由方式字的有关位决定。在同步字符之后,往控制口写入命令字。若置为异步工作方式,则在输出方式字后,紧跟着就往控制口写入命令字。写入命令字后就使 8251A 处于规定的工作状态,准备发送或接收数据了。

在传送数据的过程中,若需要改变传送方式,则必须写入使 8251A 内部复位的命令字,然后再重新写入新的方式字和命令字。8251A 在工作过程中,还允许用 IN 指令读取 8251A 的状态字,用于了解 8251A 的当前工作状态,以便控制 CPU 与 8251A 之间的数据交换。

2. 方式字

8251A 的方式字的格式如图 10-13 所示。

方式字的最低两位(D_1D_0)用来定义 8251A 的工作方式,当它们不等于全 0 时,8251A 工作于异步方式。异步方式字格式如图 10-13(a)所示。

B_2B_1 的三种不同取值用来确定波特率系数,也就是 TxC 、 RxC 信号与波特率之间的系数,它们之间有如下关系:

$$\text{收发时钟频率} = \text{收发波特率} \times \text{波特率系数}$$

若收发时钟频率为 19200,波特率系数为 $\times 16$,则收发波特率为 $19200/16=1200$ 。

L_2L_1 位用来定义数据字符的长度,可以是 5、6、7 或 8 位。 PEN 和 EP 位决定是否有校验位及是奇校验还是偶校验。 S_2S_1 位用于决定停止位的个数。

当方式字的最低两位 $D_1D_0=00$ 时,8251A 工作于同步方式,同步方式字的格式如图 10-13(b)所示。 ESD 位为外同步检测位,当它为 1 时,8251A 工作于外同步方式, $SYNDET$ 为输入;为 0 时,工作于内同步方式, $SYNDET$ 为输出。 SCS 位为单字符同步位, $SCS=1$ 时,8251A 使用单同步字符; $SCS=0$ 时,采用双同步字符。 L_2L_1 及 EP 、 PEN 位的意义与异

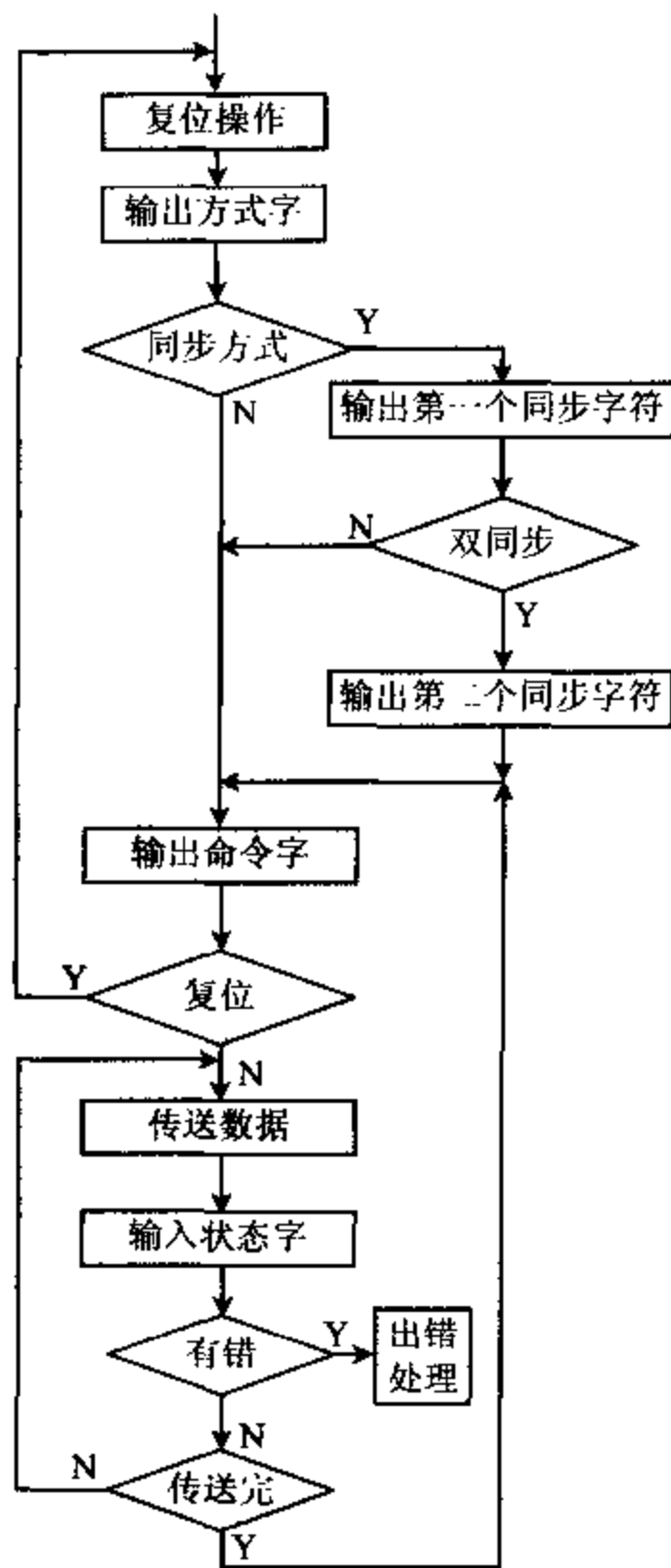


图 10-12 8251A 编程流程图

步方式字相同。

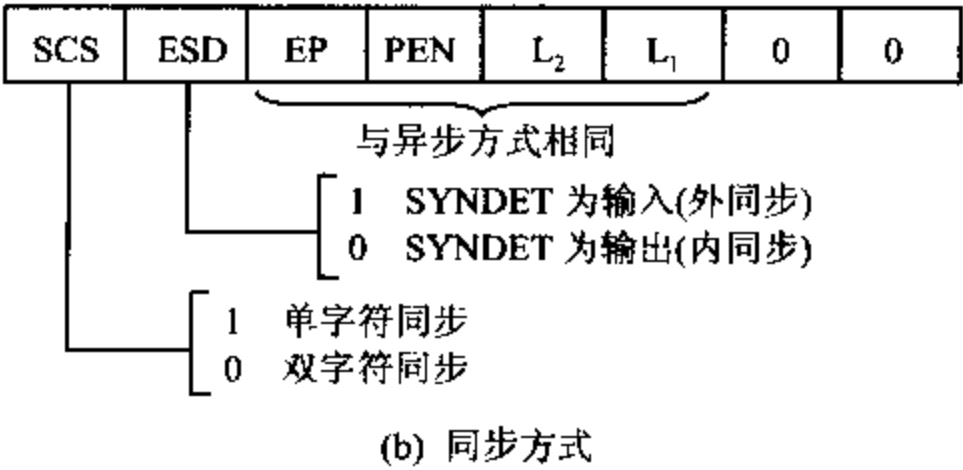
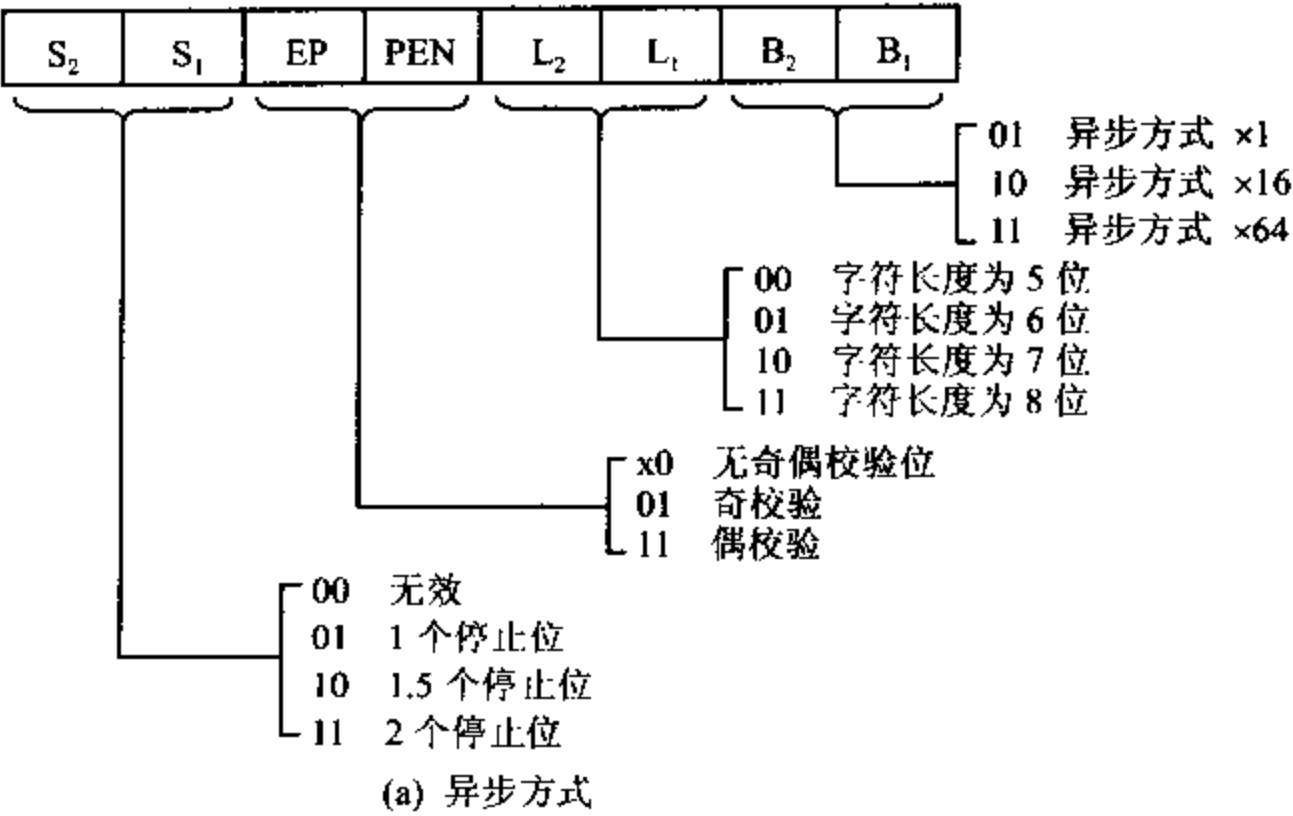
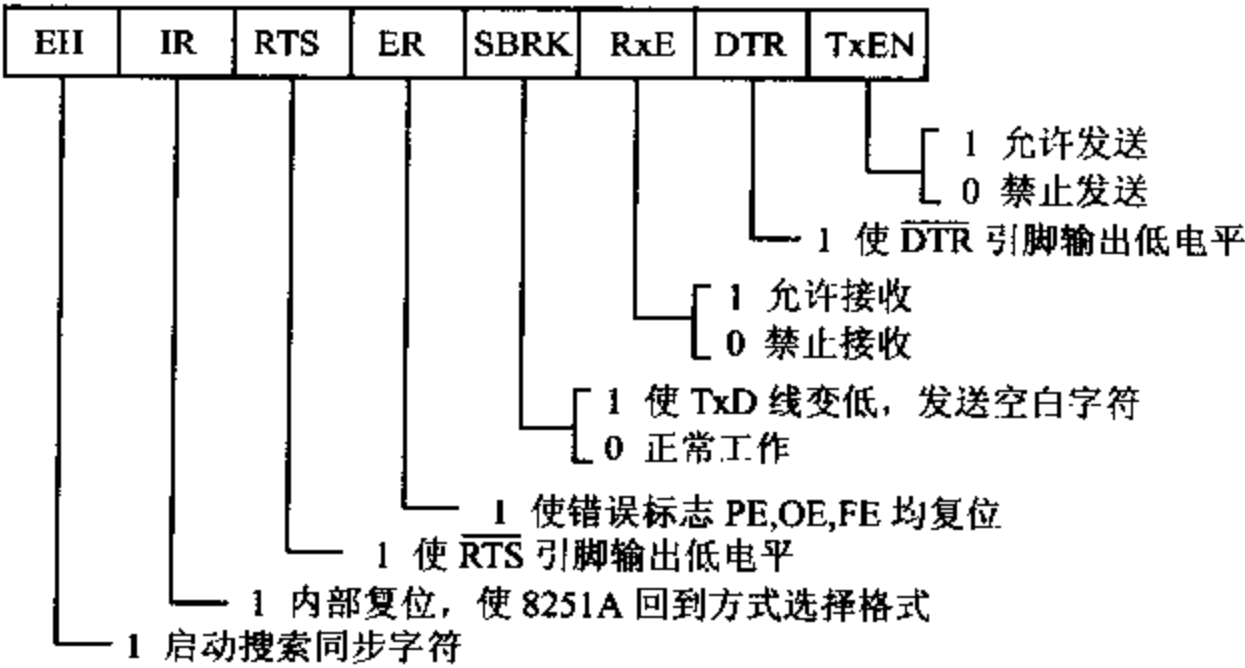


图 10-13 8251A 方式字格式

3. 命令字

8251A 的命令字格式如图 10-14 所示。



TxEN 位是允许发送位。只有当 TxEN=1 时,才允许发送器通过 TxD 引脚向外发送数据。

RxE 位是允许接收位。当 RxE=1 时,接收器才能通过 RxD 线接收从外部发送过来的串行数据。

DTR 是数据终端准备好位。当 DTR 位置 1 时,就迫使 DTR 引脚输出有效的低电平,用以通知 MODEM,数据终端已做好了接收数据的准备。

RTS 位是请求发送位,当 RTS 位置 1 时,就迫使 RTS 引脚输出有效的低电平,表示计算机已准备好了数据,用该信号向 MODEM 或外设请求发送数据。

SBRK 位是发送空白字符位(Send Break Character)。正常工作时,SBRK 位应保持为 0,当它为 1 时,就迫使 TxD 变为低电平,也就是一直在发送空白字符(全 0)。

ER 位是清除错误标志。8251A 允许设置三个出错标志,它们是奇偶校验错标志 PE、溢出标志 OE 和帧校验错标志 FE。当 ER 位等于 1 时,将 PE、OE 和 FE 三个标志位同时清 0。这三个标志的意义在下面讨论状态字时再作进一步说明。

IR 位为内部复位信号。该位置 1 时使 8251A 内部复位,迫使 8251A 回到接收方式字的状态。在这种状态下,只有再向 8251A 的控制口写入一个新的方式字,重新对芯片进行初始化编程后,8251A 才能正常工作。

EH 位为外部搜索方式位,它只对内同步方式有效。该位置 1 时,8251A 会从 RxD 引脚输入的信息流中搜索特定的同步字符,若找到了同步字符(双同步时要搜索到两个同步字符),就使 SYNDET/BRKDET 引脚输出高电平。

4. 状态字

在数据通信系统中,常常要了解 8251A 的工作状态,如检查传送中是否产生了错误,TxRDY 是否有效等,以便控制 CPU 与 8251A 之间的数据交换。8251A 内部设有状态寄存器,CPU 可随时用 IN 指令读取状态寄存器的内容,在 CPU 读状态时,8251A 将自动禁止改变状态。状态字的格式如图 10-15 所示。

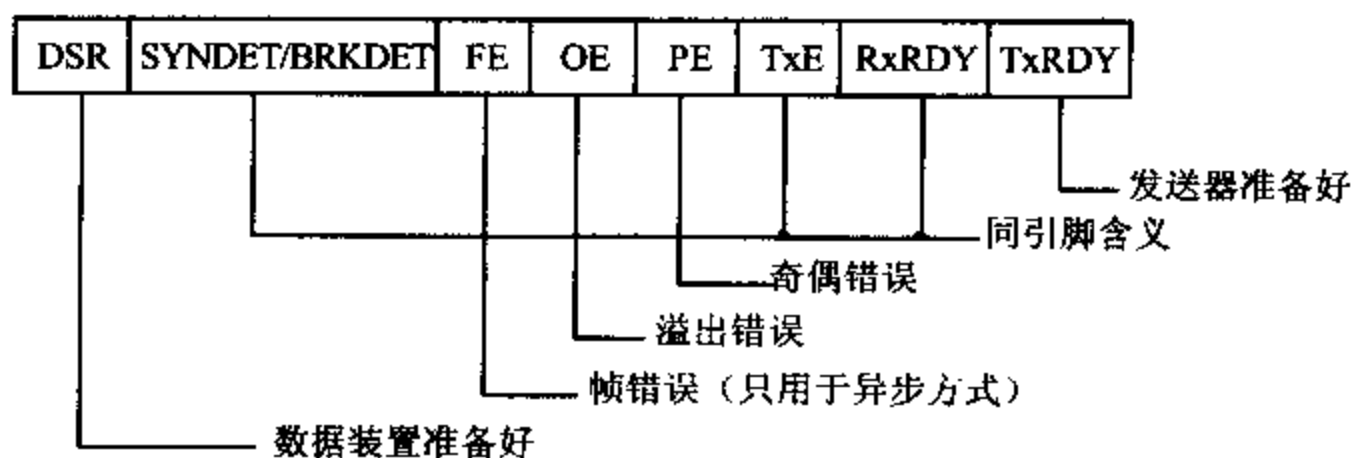


图 10-15 8251A 状态字格式

其中 RxRDY、TxE、SYNDET/BRKDET 位的意义与同名引脚的功能完全相同。

TxRDY 是发送器准备好状态位,它与引脚信号有些区别。对于状态寄存器中的 TxRDY 位,只要发送数据缓冲器空就被置 1;而引脚 TxRDY 置 1 的条件是,发送数据缓冲器空、 $\overline{\text{CTS}}=0$ 和 TxEN=1 必须同时成立。

PE 位是奇偶校验错标志位(Parity Error)。PE=1 表示当前产生了奇偶校验错误,它

并不中止 8251A 的工作。

OE 位是溢出(丢失)错误标志位(Overrun Error)。若 CPU 还没把输入缓冲器中的前一个字符取走,新的字符又被送入缓冲器,OE 标志位便被置 1,表示产生了溢出。该标志位不禁止 8251A 工作,但发生溢出时,前一个字已经丢失。

FE 为帧错误标志(Frame Error),只用于异步方式。一帧数据必须以起始位开始,停止位结束,中间是字符位和奇偶校验位(若允许校验的话)。若任一个字符的结束处没有检测到有效的停止位,则 FE 标志置 1。该标志不禁止 8251A 工作。

当向 8251A 输出命令字并使 ER 位置 1 时,则 PE、OE 和 FE 这三个标志被复位。

DSR 位是数据装置准备好位。当 DSR=1 时,表示调制解调器已准备好发送数据,这时输入引脚 $\overline{\text{DSR}}$ 产生有效的低电平。

三、8251A 初始化编程举例

1. 异步方式初始化程序

在接通电源时,8251A 能通过硬件电路自动进入复位状态,但不能保证总是正确地复位。为了确保送方式字和命令字之前 8251A 已正确复位,应先向 8251A 的控制口连续写入 3 个全 0,然后再向该端口送入一个使 D_0 位等于 1 的复位控制字(40H),用软件命令使 8251A 可靠复位。它被复位后,就可向它写入方式字和命令字,这两个字都被写入控制口。8251A 是通过写入次序来区分这两个字的,先写入的是方式字,在方式字后送入控制口的是命令字。

另外要注意,对 8251A 的控制口进行一次写入操作后,需要有写恢复时间。若 CLK 引脚上输入时钟信号的周期为 t ,须经过 16 个时钟周期(16 t)后才能再写入第二个字。即两次写操作之间必须延时 16 个时钟周期才能保证可靠写入。最简单的做法是在两次写操作之间插入几条指令,再加上 OUT 指令本身要 8 个时钟周期,使延时时间足以超过 16 个时钟周期。下面给出能实现这种延时功能的程序段,为便于多次调用,程序段以宏指令的形式给出。

```
REVTIME MACRO
    MOV    CX, 02      ;4 个时钟周期
D0:      LOOP  DX      ;17 个或 5 个时钟周期
ENDM
```

但在向 8251A 写入数据字符时,不必考虑这种恢复时间,这是因为 8251A 必须等前面一个字符移出后,才能写入新字符,移位所需的时间远大于恢复时间。

若要求 8251A 工作于异步方式,波特率系数为 16,具有 7 个数据位,一个停止位,有偶校验,控制口地址为 3F2H,写恢复时间程序为 REVTIME,则对 8251A 进行初始化的程序为:

```
MOV  DX, 3F2H      ;控制口
MOV  AL, 00H
OUT  DX, AL        ;向控制口写入“0”
REVTIME            ;延时,等待写操作完成
```


OUT DX, AL	;向控制口写入第二个“0”
REVTIME	;延时
OUT DX, AL	;向控制口写入第三个“0”
REVTIME	;延时
MOV AL, 40H	;复位字
OUT DX, AL	;写入复位字
REVTIME	;延时
MOV AL, 01111010B	;方式字:波特率系数为 16,7 个数据位,一个停止位,偶校验
OUT DX, AL	;写入方式字
REVTIME	;延时
MOV AL, 00010101B	;命令字:允许接收发送数据,清错误标志
OUT DX, AL	;写入命令字

2. 同步方式初始化程序

如果 8251A 工作于同步方式,则初始化 8251A 时,先和异步方式一样,向控制口写入 3 个 0 和一个软件复位命令字(40H),接着向控制口写入方式字,然后往控制口送同步字符。若方式字中规定为双同步字符,则需对控制口再写入第二个同步字符。常用 ASCII 字符集中的 16H 作为收发双方同意的一个同步字符。写入同步字符后,再对 8251A 的控制口写入一个命令字,选通发送器和接收器,允许芯片对从 RxD 引脚上送来的数据位搜索同步字符。

现在仍假设 8251A 的控制口地址为 3F2H,写恢复延时程序为 REVTIME,如要求 8251A 工作于同步方式,采用双同步字符、奇校验、数据位为 7 位,则对 8251A 写入复位字以后的初始化程序为:

:	;先向控制口写入 3 个 0,再送复位字 40H
MOV DX, 3F2H	;控制口
MOV AL, 00011000B	;方式字:双同步,内同步,奇校验,7 个数据位
OUT DX, AL	;送方式字
REVTIME	;延时
MOV AL, 16H	
OUT DX, AL	;送入第一个同步字符
REVTIME	
OUT DX, AL	;送入第二个同步字符
REVTIME	
MOV AL, 10010101B	;命令字:启动搜索同步字符,错误标志复位,允许收发
OUT DX, AL	

10-3 EIA RS-232C 串行口和 8251A 应用举例

一、EIA RS-232C 串行口

在 20 世纪 60 年代,随着串行通信技术在计算机领域中的广泛使用,电子工业协会 EIA

(Electronic Industry Association)开发了一个 EIA RS-232C 串行接口标准,这里 RS 意为推荐标准(Recommend Standard)。这个标准对串行接口电路中所用的插头插座的规格、各引脚的名称和功能、信号电平等均做了统一的规定,各厂家必须按此标准来配置串行接口,以便于互连。

RS-232C 标准具体规定如下:

1. 信号电平

逻辑高电平(或 MARK):有负载时为 $-3V \sim -15V$,无负载时为 $-25V$ 。

逻辑低电平(或 SPACE):有负载时为 $+3V \sim +15V$,无负载时为 $+25V$ 。

通常使用 $\pm 12V$ 作为 RS-232C 电平。但由于如今广泛使用的计算机本身及 I/O 接口芯片多采用 TTL 电平,即 $0 \sim 0.8V$ 为逻辑 0, $+2.0V \sim +5V$ 为逻辑 1,显然,它与 RS-232C 电平不匹配。为此,必须设计专门的电路来进行电平转换。

典型的电平转换电路是 MC1488 和 MC1489。发送数据时用 MC1488,它将 TTL 电平转变为 RS-232C 电平。MC1488 工作时,需用 $\pm 12V$ 两种电源供电,接收数据时用 MC1489,它将 RS-232C 电平转换成 TTL 电平。1489 只需用单一的 $+5V$ 电源供电。

近年来又出现了一类新型的电平转换器,如美国 MAXIM 公司推出的 MAX232, MAX233,见图 10-16 所示,它们仅需 $+5V$ 电源供电,使用十分方便。这两种电平转换器均

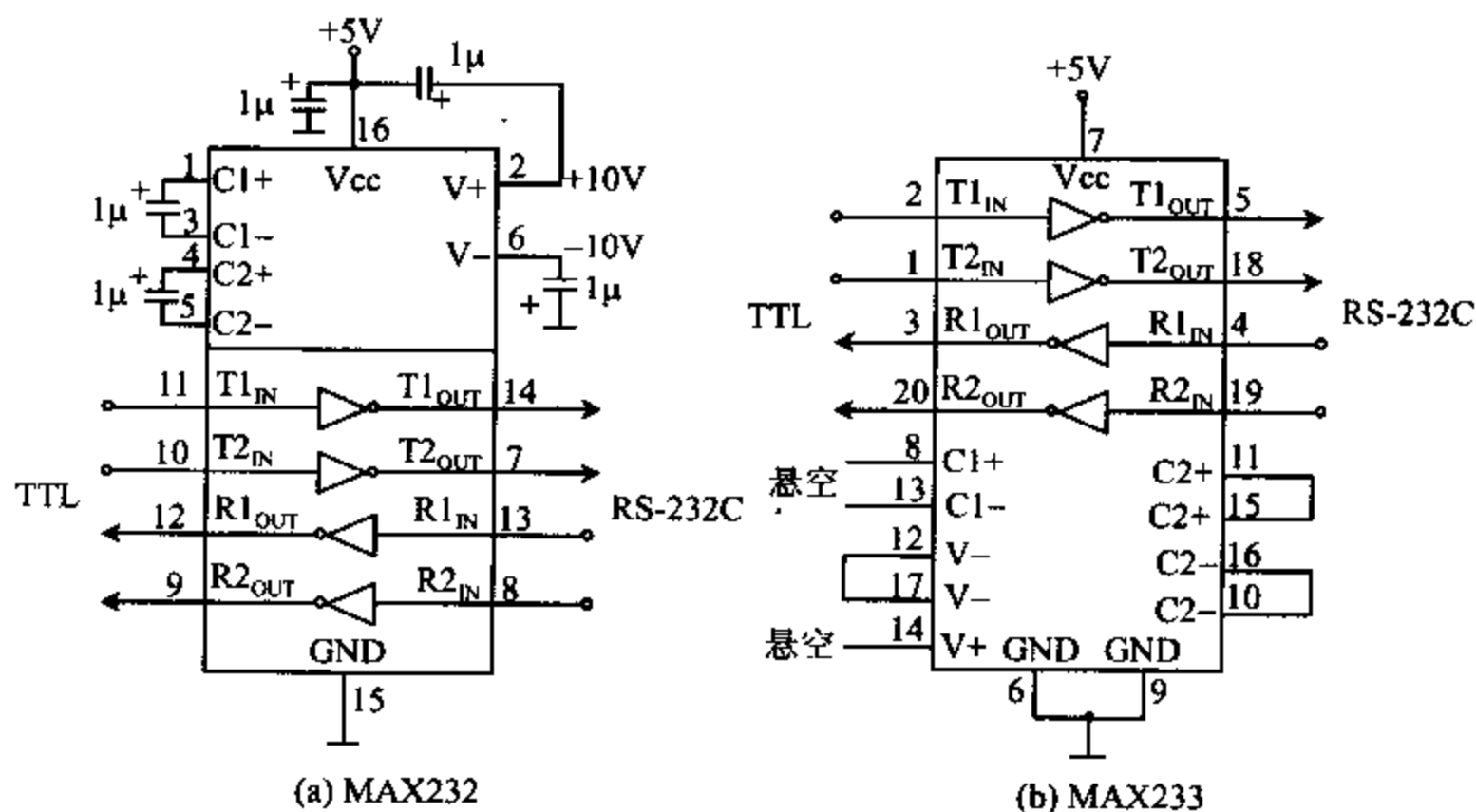


图 10-16 两种 RS-232C 串行口电平转换器

可将 2 路 TTL 电平转换成 RS-232C 电平,也可将 2 路 RS-232C 电平转换成 TTL 电平。使用时要注意的是 MAX232 需外接 5 个 1μ 的电容,而 MAX233 不需外接电容,使用起来更加方便,但 MAX233 的价格要略高一点。

2. 接插件规格

RS-232C 串行接口规定使用 25 芯的 D 型插头插座进行连接,其引脚形状和引脚号如图 10-17(a)所示。对不需要用 25 芯引脚的系统上,常采用 9 芯 D 型接插件,其形状和引脚号如图 10-17(b)所示。图中给出的都是凸型接插件,此外还有凹型接插件,使用这些插件

时,要注意在插头座上所标的引脚序号。

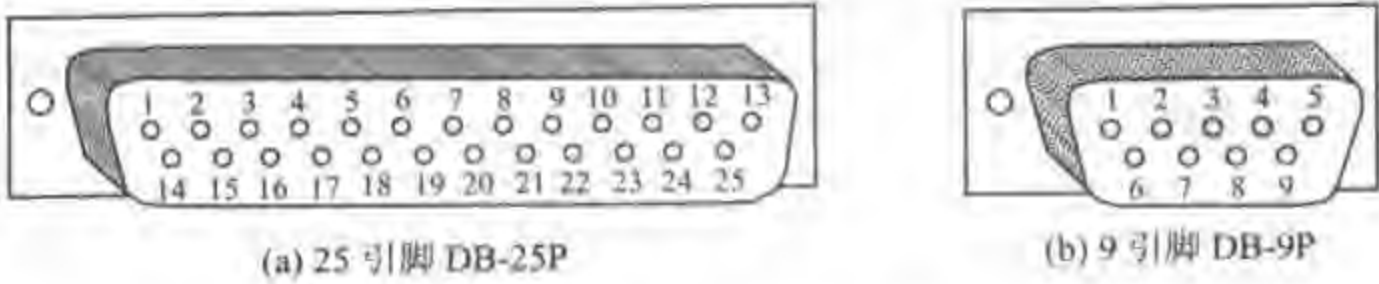


图 10-17 RS-232C 的接插件(凸型)

3. 信号定义

RS-232C 标准对 25 芯插件的每一个引脚的信号名称、功能等都做了具体规定,还有几个引脚未定义或予以保留,以备今后扩充时用。在多数应用情况下,仅用到其中的少数几个引脚信号,表 10-2 给出了最基本引脚的名称和功能,9 芯插件的主要信号也在表中给出。

表 10-2 RS-232C 最基本引脚的名称和功能

9 芯引脚号	25 芯引脚号	名 称	功 能
	1		保护地
3	2	TxD	发送数据
2	3	RxD	接收数据
7	4	RTS	请求发送
8	5	CTS	清除发送
6	6	DSR	数据装置准备好
5	7	GND	信号地
1	8	CD	载波信号检测
4	20	DTR	数据终端准备好
	9,10	—	保留
	11,18,25	—	未定义

在上述信号中,有保护地(1 脚)和信号地(7 脚)两个地信号,其中信号地是所有信号的公共地。为了防止在信号地上感应大的交流地电流,必须在终端或计算机的电源处把这两个地信号连在一起。

对于 TxD 和 RxD 这两根数据信号线,EIA 的逻辑 1 表示数字位的 1 或 MARK(实际的负电压),EIA 的逻辑 0 表示数字位的 0 或 SPACE(实际的正电压),使用电平转换器时都加了反相器。对于 RTS、CTS、DSR 等控制状态信号线来说,EIA 的逻辑 0 为信号的有效状态,即开关的接通状态(ON),此时电平值为 +3V~+15V。

二、8251A 应用举例

利用 RS-232C 串行口进行较近距离串行通信时,CPU 和大多数外设相连或 CPU 与 CPU 之间进行通信时,不需要使用 MODEM。最常用的方法是采用三线传输的最小方式进行通信,即只使用发送数据线 TxD、接收数据线 RxD 及地线这三根信号线进行通信,其中地线可与 25 芯插件的 1,7 脚相连。

下面,我们举一个实际例子来进一步说明 8251A 和 RS-232C 的使用方法。假如有两台以 8086 为 CPU 的微机之间需进行通信,它们用 8251A 作接口芯片,通过 RS-232C 串行接口实现通信,硬件电路如图 10-18 所示。图中仅画出一台微机的接口电路,另一台微机的接口部分完全是类似的。

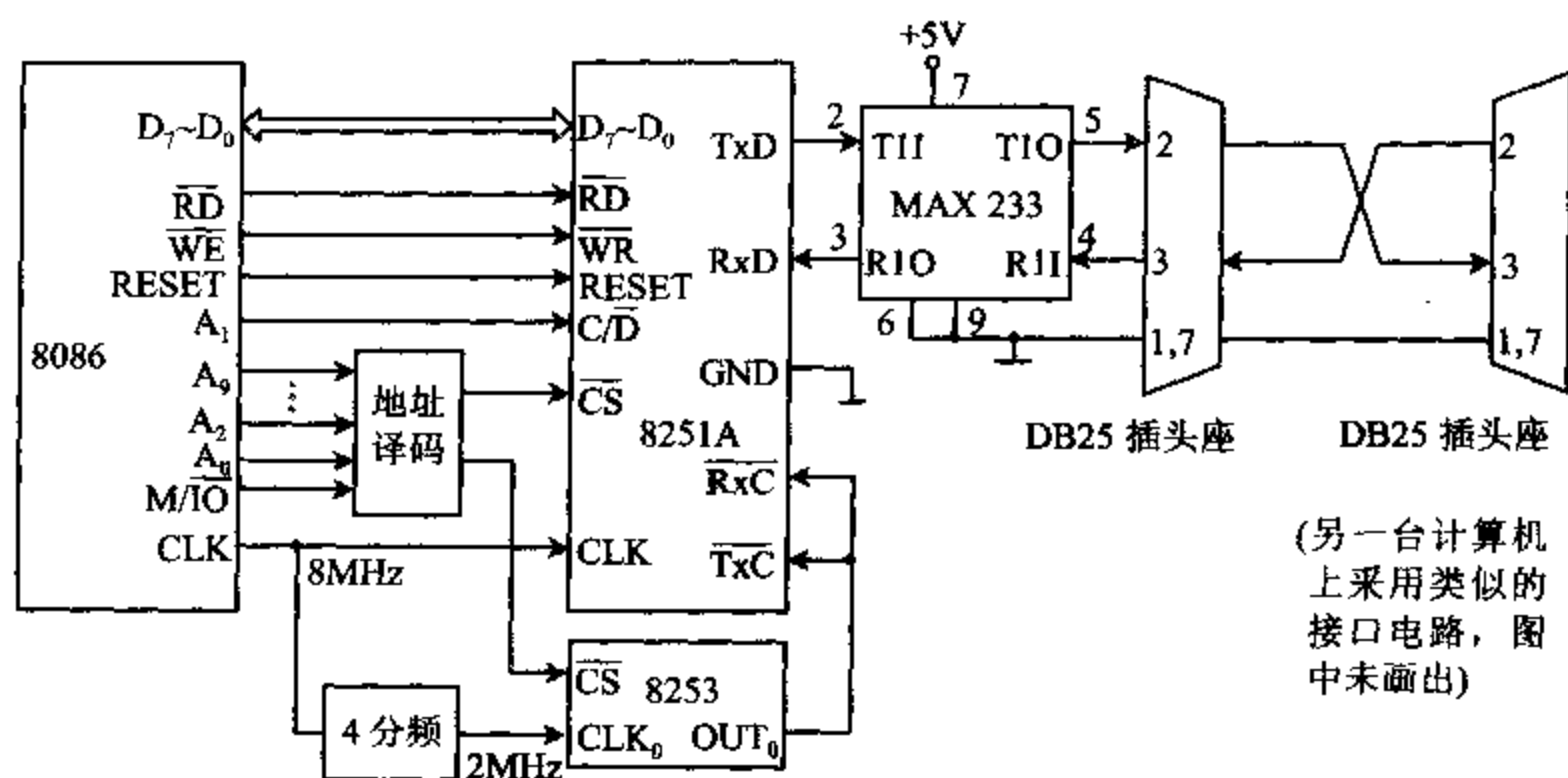


图 10-18 双机通信接口电路图

由图可见,8251A 的 $\overline{RD}$ 、 $\overline{WR}$ 、RESET 等信号与 CPU 的相应端相连,8251A 的 $D_7 \sim D_0$ 和 8086 的低 8 位数据线 $D_7 \sim D_0$ 相连,8251A 的片选信号由地址译码电路提供, $C/\overline{D}$ 与地址总线的 A_1 相连,用于选择数据口和控制口。这些信号的连接方法前面已介绍过了。

从图 10-18 中我们还可以看到,8251A 的主时钟和 CPU 使用同一个时钟 CLK,这里假设主时钟 CLK 的频率为 8MHz。CLK 还经分频电路分频后形成 2MHz 的信号,送到 8253 的 CLK_0 输入端,再经 8253 分频后,送到 8251A 的 $\overline{RxC}$ 和 $\overline{TxC}$ 端,作 8251A 的接收时钟和发送时钟。

若 8253 工作于方式 3,串行数据传送的波特率为 9600Bd,波特率系数为 16,则 $\overline{RxC}$ 和 $\overline{TxC}$ 的频率应是:

$$9600 \times 16 = 153600 \text{ Hz} = 0.1536 \text{ MHz}$$

8253 的通道 0 的分频系数为:

$$n = 2 \text{ MHz} / 0.1536 \text{ MHz} = 13$$

这样,系统工作时,便可从 OUT_0 端得到频率为 0.1536MHz 的方波信号,作为 8251A 的接收时钟信号 $\overline{RxC}$ 和发送时钟信号 $\overline{TxC}$ 。

由于 8251A 的输入输出信号均是 TTL 电平,与 RS-232C 的电平不一致,因此输出信号 TxD 要经 1488 转换成 RS-232C 电平后才能将数据发送出去;反之,另一台计算机送来的 RxD 信号是 RS-232C 电平,也要经 1489 转换成 TTL 电平后才能送给 8251A。另外,在使用 RS-232C 标准的 25 芯接插件时要注意,不能直接将双方的输出信号线(2 脚)接在一起,也不能直接将输入信号线(3 脚)接在一起,这样将无法工作,而必须按图 10-18 中所示,采用交叉连接的方法。也就是说将第一台机器的发送端 TxD(2 脚)与第二台机器的接收端

RxD(3 脚)相连,将第二台机器的发送端与第一台机器的接收端相连,这样才能使一方发送数据,另一方接收数据。

假如第一台计算机所用的 8251A 的数据口和控制口地址分别为 1F0H 和 1F2H,两台机器之间采用查询方法、异步传送、半双工通信。发送数据时,发送端 CPU 不断查询 TxRDY 的状态是否为有效的高电平,若为高,表示发送缓冲器空,可用 OUT 指令向 8251A 输出一个数据字节。接收数据时,CPU 不断检测 RxRDY 是否为有效的高电平,若为高则表示接收数据已准备好,CPU 可用 IN 指令从 8251A 输入一个数据。设第一台计算机要求发送的数据存放在以 BUFF-T 为始址(偏移量)的内存单元中,发送数据个数为 COUNT-T,接收数据存放到以 BUFF-R 为始址的内存单元中,接收数据个数为 COUNT-R,则对于第一台计算机来说,发送一批数据的初始化程序和控制数据传送的程序为:

```

...                                ;先向控制口写 3 个 0,再向控制口写入 40H,
                                ;使系统复位
BEG-T:  MOV    DX, 1F2H            ;控制口
        MOV    AL, 7AH            ;方式字:异步方式,7 个数据位,1 个停止位
                                ;偶校验、波特率系数为 16
        OUT    DX, AL
        MOV    CX, 02H            ;延时
D1:     LOOP   D1
        MOV    AL, 11H
        OUT    DX, AL            ;清除错误标志,允许发送
        MOV    CX, 02H            ;延时
D2:     LOOP   D2
        MOV    DI, BUFF-T         ;发送缓冲区始址
        MOV    CX, COUNT-T        ;发送数据个数
NEXT-T: IN     AL, DX              ;读入状态
        TEST   AL, 01H            ;TxRDY 有效吗?
        JZ     NEXT-T             ;否,则等待
        MOV    DX, 1F0H           ;是,数据口地址送 DX
        MOV    AL, [DI]           ;从缓冲区取一个数据
        OUT    DX, AL            ;向 8251A 输出一个数据
        INC    DI                 ;修改缓冲区指针
        LOOP   NEXT-T            ;没送完则继续
...                                ;送完

```

同一台机器上接收一批数据的初始化程序和控制数据传送的程序为:

```

...                                ;使系统复位
BEG-R:  MOV    DX, 1F2H            ;控制口
        MOV    AL, 7AH
        OUT    DX, AL            ;送出方式字,同发送部分
        MOV    CX, 02H            ;延时
D3:     LOOP   D3
        MOV    AL, 14H

```

	OUT	DX, AL	;输出命令字;清错误标志,允许接收
	MOV	CX, 02H	;延时
D4:	LOOP	D4	
	MOV	DI, BUFF-R	;接收数据缓冲区始址
	MOV	CX, COUNT-R	;接收数据个数
NEXT-R:	IN	AL, DX	;读入状态字
	TEST	AL, 02H	;RxRDY 有效吗?
	JZ	NEXT-R	;否,循环等待
	TEST	AL, 38H	;是,查是否有错
	JNZ	ERROR	;有错,则转出错处理程序
	MOV	DX, 1F0H	;无错
	IN	AL, DX	;读入一个数据
	MOV	[DI], AL	;输入数据 → 缓冲区
	INC	DI	;修改缓冲区指针
	LOOP	NEXT-R	;数据传送没完成则继续
	...		;完成
ERROR:	...		;出错处理

同样,在第二台计算机上,也需要编写类似的初始化程序和数据收发程序。两台计算机之间进行通信时,双方的波特率必须一致。

事实上,一台计算机不仅可以与另一台计算机进行通信,还可与 CRT 终端、单片机开发系统或其它具有串行接口的外设通信。发送时钟 $\overline{\text{TxC}}$ 可以用 8253 一类的定时器/计数器电路对主时钟分频后产生,也可由专门的波特率产生器提供。数据传送也可采用全双工方式,这时双方可以同时收发数据。

10-4 串行同步数据通信协议

为了加快数据的传送速率,当使用高速 MODEM、数字通信通道及电话线进行远程通信时,都使用串行同步传送方式,这时需要一系列的联络信号,除了前面已介绍的握手信号外,发送方和接收方之间还需要更高一级的协议来保证数据的正确传输。协议是双方订立的一些规则,实际上是一整套关于信息传送顺序、信息格式和信息内容等的约定。串行数据协议有许多种,最常用的两种是:

- (1) IBM 二进制同步通信协议 BISYNC(Binary Synchronous Communication Protocol)
- (2) 高级数据链路控制协议 HDLC(High-Level Data Link Control Protocol)

下面对它们分别进行介绍。

一、二进制同步通信协议 BISYNC

1. BISYNC 协议格式

BISYNC 是一种字节控制协议。协议规定,在报文开始时发送方和接收方之间用一些

ASCII 或 EBCDIC 字符作握手信号,在同一时刻只允许在一个方向上传输数据,即使在全双工系统中也如此。普通报文(收发的数据信息)的格式如图 10-19 所示。



图 10-19 BISYNC 普通报文格式

我们先假定接收方已做好了接收数据的准备。如果发送方没有报文发送过来,线路将保持高电平,处于空闲(MARK)状态。发送报文时,先送出一个或两个同步字符 SYNC,同步字符用特殊的 ASCII 码或 EBCDIC(Extended Binary-Coded Decimal Interchange Code)码表示,如 ASCII 码的 16H 常用作双方同意的一个同步字符。接收方利用这个同步字符将其时钟与发送方的时钟同步,接着可发送报头。报头以特定字符 SOH(Start of Header)开始,它的 ASCII 码为 01H。报头内容一般根据特定的系统来定义,可能包括类型、奇偶校验及报文目的地址等。

报头之后的信息为正文,正文以特定的正文开始字符 STX(Start of Text)开始,它的 ASCII 码为 02H,正文之后必须发正文结束符 ETX(End of Text)或块结束符 ETB(End of Block),即 ASCII 字符 03H 和 17H。正文部分可能含有 128 个或 256 个字符,不同的系统使用不同长度的正文。紧随 ETX 或 ETB 之后发送的是一个或两个块校验字符 BCC(Block Check Character)。对使用 ASCII 码的系统来讲,BCC 为一个字节,表示的是为正文而计算的复杂奇偶校验信息。对使用 EBCDIC 码的系统来说,BCC 为两个字节,它是正文循环冗余校验 CRC(Cyclic Redundancy Check)的 16 位结果。接收系统对接收到的数据也进行校验计算,并将结果与发送方发来的 BCC 值进行比较,如果两者不相等,则接收方就向发送方发一个报文,请求对方重新发送报文。下面讨论收发双方如何利用报文进行数据交换。

2. BISYNC 协议数据传送过程

假设有一个远程终端与远方的计算机进行通信,利用 8251A 作接口芯片,采用半双工连接方式,并假设远方计算机处于接收方式,终端处于发送方式,而且终端内也有 CPU 控制收发数据。收发过程如下:

当终端中的程序决定向计算机发送一组数据时,先发送一个报文,报文的正文只有一个询问(Enquiry)字符 ENQ,它的 ASCII 码为 05H。然后转入接收方式等待计算机回答。计算机收到 ENQ 报文后,若没有准备好接收数据,就回一个应答报文,报文的正文部分只有一个否定应答字符 NAK(Negative Acknowledge),其 ASCII 码为 15H。若已做好了接收数据的准备,则回发一个肯定应答字符 ACK(Affirmative Acknowledge),它的 ASCII 码为 06H。在这之后,计算机就转入接收状态,等待终端发下一个报文过来。

如果终端从计算机那儿收到的是 NAK 字符,则放弃发送或等一会再试。如收到的是 ACK,则向计算机发数据报文,整个报文以 BCC 结束。终端发送完数据后就转入接收方式,等待计算机的回答,以确认报文是否已被计算机正确接收。计算机对收到的数据块进行 BCC 计算,并将计算结果与报文发来的 BCC 值进行比较,如两者不相等,表示收到的报文

不正确,计算机就向终端发一个 NAK 报文,请求终端重新发送报文过来。如果两个 BCC 相等,表示报文正确,计算机就向终端发一个 ACK 报文,通知终端可以发下一个报文。

8251A 工作时,先要对它进行初始化编程,使之工作于同步方式,并对它写入 1~2 个同步字符,这些内容在前面已做了较详细的介绍。根据前面的讨论,送入同步字符后便可发送数据了,但那种收发方式可靠性不够高。遵照 BISYNC 协议进行串行同步通信时,发送数据不仅仅发送同步字符和正文两部分,而是要以如图 10-19 的格式,从 8251A 的 TxD 引脚上依次发送同步字符、SOH、报头、STX、正文、ETX 和 BCC 等字符。此外,每次发送一组数据都必须停下来,等待接收方的应答信号,这样可保证可靠收发数据。

利用二进制同步协议(BISYNC)收发数据虽有较高的可靠性,但每收发一组数据都必须先发查询信号,再停下来等待对方发应答信号过来。由于需要等待及线路切换等占用了一定的时间,使实际的传送速率降低了将近一半。为解决这个问题,可采用下面要介绍的另一种协议——HDLC 协议,它可显著提高数据传送的速率。

二、高级数据链路控制协议 HDLC

高级数据链路控制协议与 IBM 后来开发的同步数据链路协议 SDLC(Synchronous Data Link Control Protocol)十分相似,它们不像 BISYNC 那样用字节 SOH、STX、和 EXT 等来标记报文的不同部分或用作报文,而改用位串,所以是一种面向位型协议。这两种协议不仅速度快,而且可以适应处理多点共享一条数据链路的问题。我们主要讨论 HDLC 协议。

1. HDLC 协议格式

HDLC 协议规定,数据以帧(Frame)为单位传送,每帧信息即为一个报文,它由一组位串(Bit String)构成,格式如图 10-20 所示。共有三类帧,它们是信息帧 I、监控帧 S 和无编号帧 U,帧的每一部分称为域(Field),三者的格式相同,但控制域各位的含义不同。

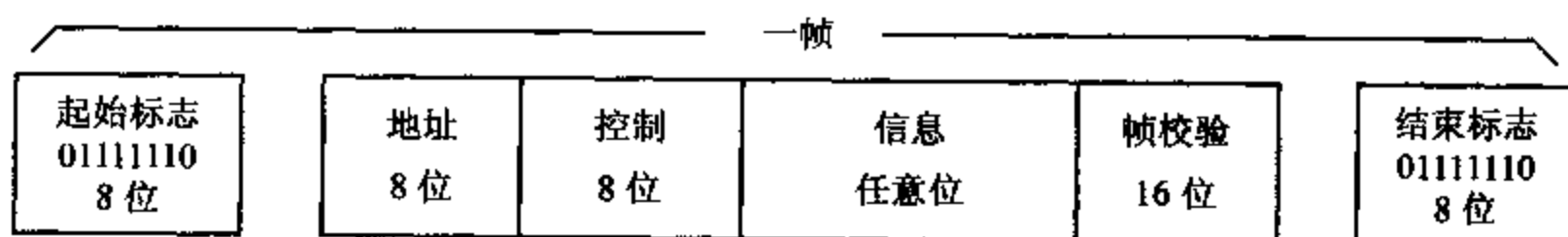


图 10-20 HDLC 帧格式

每帧都以特定的位模式 01111110 开始和结束,这种位模式称为标志或标志域。

标志域后跟 8 位地址,对于发送方的控制帧和信息帧来说,存放的是目的站地址;而对于响应的接收方来说,控制帧和信息帧的地址域存放的是源站地址。

在地址之后根据需要设置一个字节的控制域,对于三类帧来说,控制域各位的含义不尽相同。

信息域只出现在信息帧里,位于控制域之后。对 HDLC 协议来说,信息域可以为任意位数,而对 SDLC 来说,信息域必须为 8 的倍数。在某些系统中,每帧可发送 1000 或 2000 位信息。如果在数据流中出现 01111110,就会误以为是标志域,因此要设法进行调整。调整操作必须由专门的硬件电路完成。具体方法为:一旦数据流中出现 5 个连续的 1,就在其

后自动填一个 0,接收方再将填充的 0 去掉。

信息域后的下一个域是 16 位帧校验序列 FCS(Frame Check Sequence),它是一个循环冗余字,它是根据帧起始标志和结束标志之间的所有位计算出来的,但不包括为防止假标志字节而填充的 0。接收方也计算该 CRC 值,以便检测是否有错。

最后,一帧以另一个标志字节作为结束标志,前一帧的结束标志可能是后一帧的起始标志。当通信线路上无数据传送时,用全 1 或持续的标志位来表示空闲位。

在微型机中实现 HDLC 协议时,不能使用标准的 USART,这是因为协议要求填充技术,即上面提到的为防止出现假标志字节而填充和移除 0 的操作。这种操作需要用特殊的器件来完成,例如 Intel 8273 HDLC/SDLC 协议控制器,它能自动填 0 和去除 0,产生和校验 FCS 字,为 RS-232C 产生接口信号,它也能直接与微型机系统总线接口。

2. HDLC 数据传送过程

在 HDLC 数据传输过程中,当 HDLC 用于一条数据链路上的两点或多点系统之间通信时,其中一个站或系统作为链路控制器,该站被称为主站,链路上的其它站则被称为从站。假设主站(如中心计算机)要向从站(如一台微机或终端)发送几帧信息,整个传送过程如下:

主站首先发送一个 S 帧,其中包含目的从站地址和查询接收方是否准备好的控制字。然后从站发送一个 S 帧,其中含有主站地址和表明从站状态的控制字。如果从站接收器处于准备好状态,主站就会发送一系列的信息帧。信息帧包含从站地址、一个控制字、一组信息及帧校验字序列。控制字中给出该帧是否为最后一帧的标志,及该帧在序列中的帧号。

当从站接收每个信息帧时,把数据读入存储器中,并计算该帧的帧校验序列。每正确接收一帧信息,就将表示帧号 N_r 的内部计数器加 1。当主站发送到最后一帧时,使控制字中表示最后一帧的标志 P/F 位置 1,这是主站给从站的一个信号,表示主站想了解从站正确接收了多少帧信息。从站收到这个信息后,就向主站发一个 S 帧来响应,它用控制字中的 3 位编码($D_7 \sim D_5$)来给出正确接收的帧号 N_r+1 ,实际给出的是希望接收的下一帧的编号。如从站已正确接收了两帧信息,则编号为 3。主站下次将发送帧号为 3 的那帧信息。这样,如果主站发送的信息有错,就可以重发,而且根据编号可以知道从哪一帧开始重发。实际上,HDLC 工作于全双工方式,每发送一帧都可以轮询从站是否已正确收到前一帧信息。当从站向主站发送,或从站向另一个从站发送时,其过程与主站向从站发送是类似的。

利用 HDLC 协议传送数据时,一帧可以发送许多位数据,构成帧的非信息位占的比例较低,因而数据传送的效率很高。另外,发送方不必像 BISYNC 协议那样每发送一个短报文之后,都必须停下来等待应答,所以传送速度很快。虽说产生错误后必须重发数帧信息,但在低错误率系统中,这是无关紧要的。因此,HDLC 常与高速 MODEM 一起使用,还常与其它一些高级协议一起用于网络通信中。

习 题

1. 串行通信与并行通信的主要区别是什么? 各有什么优缺点?
2. 在串行通信中,什么叫单工、半双工、全双工工作方式?
3. 什么叫同步工作方式? 什么叫异步工作方式? 哪种工作方式的效率更高? 为什么?

4. 用图表示异步串行通信数据的位格式,标出起始位、停止位和奇偶校验位,在数字位上标出数字各位发送的顺序。

5. 什么叫波特率? 什么叫波特率因子? 常用的波特率有哪些?

6. 若某一终端以 2400 波特的速率发送异步串行数据,发送 1 位需要多少时间? 假设一个字符包含 7 个数据位、1 个奇偶校验位、1 个停止位,发送 1 个字符需要多少时间?

7. 什么叫 UART? 什么叫 USART? 列举典型芯片的例子。

8. 什么叫 MODEM? 用标准电话线发送数字数据为什么要用 MODEM? 调制的形式主要有哪几种?

9. 若 8251A 以 9600 波特的速率发送数据,波特率因子为 16,发送时钟 $\overline{\text{TxC}}$ 频率为多少?

10. 8251A 的 SYND $\overline{\text{ET}}$ /BRKDET 引脚有哪些功能?

11. 如果系统中无 MODEM,8251A 与 CPU 之间有哪些连接信号?

12. 在一个以 8086 为 CPU 的系统中,若 8251A 的数据端口地址为 84H,控制口和状态口的地址为 86H,试画出地址译码电路、数据总线和控制总线的连线图。

13. 设 8251A 的端口地址如题 12,要求 8251A 工作于内同步方式,同步字符为 2 个,用偶校验,7 个数据位,试对 8251A 进行初始化编程。

14. 若 8251A 的端口地址为 FF0H,FF2H,要求 8251A 工作于异步工作方式,波特率因子为 16,有 7 个数据位,1 个奇校验位,1 个停止位,试对 8251A 进行初始化编程。

15. RS-232C 的逻辑高电平与逻辑低电平的范围是什么? 怎样与 TTL 电平的器件相连? 规定用什么样的接插件? 最少用哪几根信号线进行通信?

16. 某微机系统用串行方式接收外设送来的数据,再把数据送到 CRT 去显示,若波特率为 1200,波特率因子为 16,用 8253 产生收发时钟,系统时钟频率为 5MHz,收发数据个数为 COUNT,数据存放到数据段中以 BUFFER 为始址的内存单元中。8253 和 8251A 的基地址分别为 300H 和 304H。

(1) 画出系统硬件连线图。

(2) 编写 8253 和 8251A 的初始化程序。

(3) 编写接收数据和发送数据的程序。

17. 如果 8251A 工作于串行同步传送方式,采用 BISYNC 串行数据协议使计算机与串行终端之间进行通信,试问:

(1) 普通报文用什么格式表示?

(2) 若串行终端要向计算机发送一组数据,两者之间依次要发送哪些字符?

(3) BISYNC 链路接收站怎样表示已检测到数据中的错误?

18. 回答下列问题:

(1) 面向位型的协议 HDLC 怎样表示一帧报文的开始?

(2) HDLC 系统怎样标志位模式出现在报文的数据段中?

(3) HDLC 接收方怎样告诉发送方,在已接收的数据中发现了错误?

第十一章 模数(A/D)和数模(D/A)转换

11-1 概 述

当用计算机来构成数据采集或过程控制等系统时,所要采集的外部信号或被控制对象的参数,往往是温度、压力、流量、声音和位移等连续变化的模拟量。但是,计算机只能处理不连续的数字量,即离散的有限值。因此,必须用模数转换器即 A/D 转换器 (Analog to Digital Converter, ADC),将模拟信号变成数字量后,才能送入计算机进行处理。计算机处理后的结果,也要经过数模转换器即 D/A 转换器 (Digital to Analog Converter, DAC),转换成模拟量后,在示波器上显示结果波形和在记录仪上描记下来,或者驱动执行部件,达到控制的目的。

一、一个实时控制系统

一个包含 A/D 和 D/A 转换器的实时闭环控制系统的框图如图 11-1 所示。

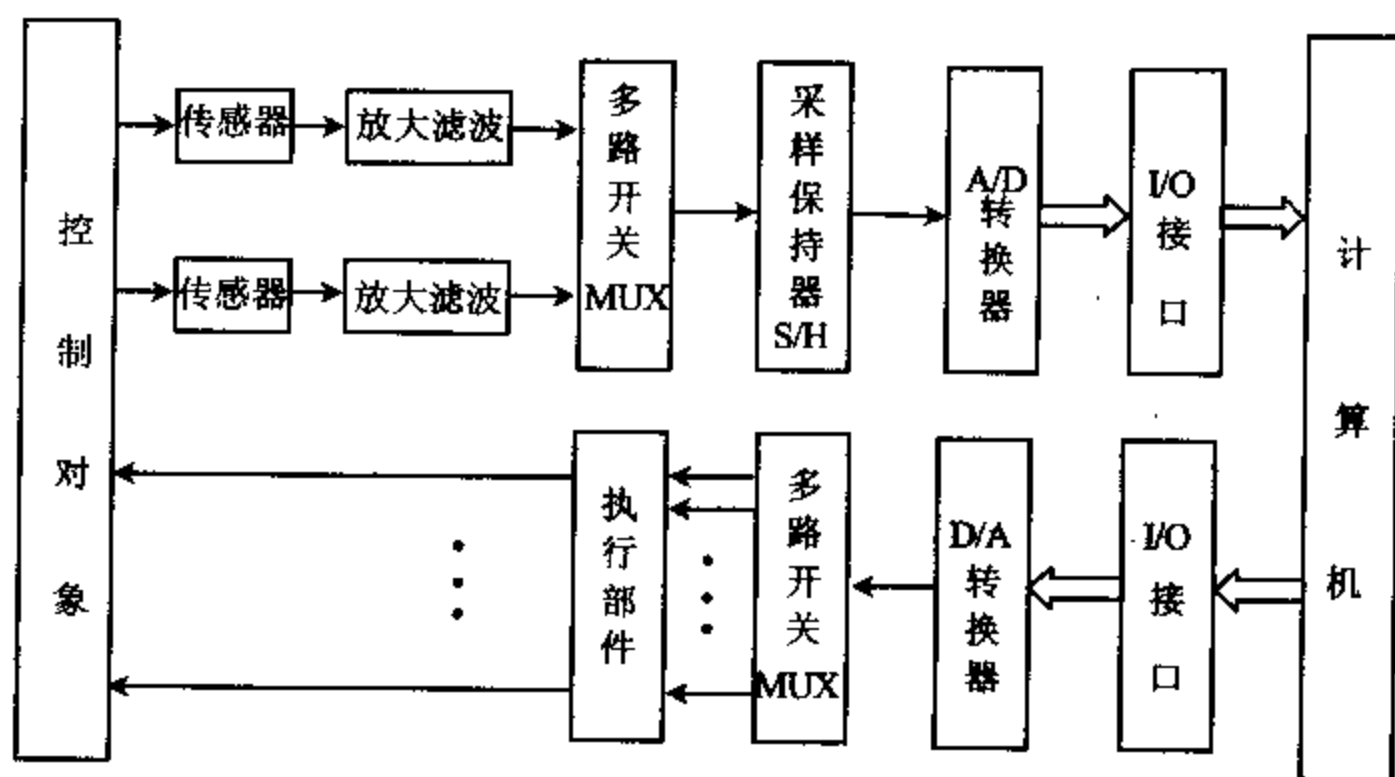


图 11-1 包含 A/D 和 D/A 的实时控制系统

在图 11-1 中, A/D 和 D/A 转换器分别是模拟量输入和模拟量输出通路中的核心部件, 后面两节将对它们作较详细的讨论。如果将图 11-1 所示的实时闭环系统中的 D/A 转换通路去掉, 则成了一个将现场模拟信号变为数字信号, 并送计算机进行处理的数据采集系统。反之, 若系统中只包含 D/A 转换通路, 就构成了一个程序控制系统。

在实际应用中,自外界输入的各种非电模拟信号,一般都要像图中那样,先由传感器把它们转换成模拟电流或电压信号后,才能被进一步处理和加到 A/D 转换器去转换成数字量。

传感器的种类很多。同一种物理量可以用几种不同的传感器来测量,同一种传感器,根据它们的特性又可分成不同的型号。例如,室温和体温常用热敏电阻测量;如果测量的是工业窑炉的炉温,则可选用各种热电偶;压力可以用压阻式、压电式、振动式等压力传感器来测量;若测量人体血压,则应采用专门的血压传感器。此外,还有流量传感器、位移传感器、光电传感器等各种不同功能的传感器。

大多数传感器产生的信号都很微弱,通常只有 μV 或 mV 量级,必须用高输入阻抗的运算放大器对它们进行放大,使它们达到一定的幅度(通常为几伏量级)。必要时还要进行滤波,选取信号中一定频率范围内的成分,去掉各种干扰和噪声。若信号的大小与 A/D 转换器的输入范围不一致,还需进行电平转换。所有这些在数字化之前的处理称为信号的预处理。

在实时控制或多路数据采集系统中,常常要同时测量几路甚至几十路信号,若每路使用一个 A/D 转换器,由于它们的价格较高,会显著增加成本。为此,常采用多路开关对被测信号进行切换,使各路信号共用一个 A/D 转换器,这样还能减小系统的体积和功耗。

多路切换的方法有两种:一种是如图 11-1 那样,外加多路模拟开关 MUX(Multiplexer)实现多路信号的切换;另一种是选用内部带多路转换开关的 A/D 转换器。例如,后面将要介绍的 ADC0809,就是带有 8 路模拟开关的 A/D 转换器。D/A 通道中也可以使用多路开关,不过由于 D/A 转换器比较便宜,实际系统中较少使用多路开关。

若模拟信号变化比较缓慢,它可以直接加到 A/D 转换器的输入端;如果信号变化较快,为了保证模数转换的正确性,还需要使用采样保持器。

下面对多路开关和采样保持器的原理作进一步的讨论。

二、多路模拟开关

多路模拟开关用来切换模拟信号。对于 A/D 通道来说,需要用多路输入、一路输出的模拟开关,使输入的多路模拟信号轮流与 A/D 转换器接通。对于 D/A 通道,则要在 D/A 转换器之后加一个一路输入、多路输出的模拟开关,使输出的模拟信号轮流分配到各模拟通路。

这两种电路都已有集成电路产品,如 AD7501,AD7503 等都是多路输入、一路输出的多路模拟开关,CD4051,CD4052,CD4097 都是可进行双向切换的多路开关,它们既可作多路输入、一路输出的模拟开关,也可作一路输入、多路输出的模拟开关。下面以 8 选 1 双向多路开关 CD4051 为例,来介绍多路模拟开关的结构、功能和使用方法。

图 11-2 是 CD4051 的引脚图,图 11-3 则是它的逻辑功能图。

如图 11-3 所示,CD4051 由逻辑电平转换电路、地址译码电路和 CMOS 开关等三部分组成。其中 $\bar{S}$ 引脚为选通端,只有当 $\bar{S}$ 为低电平时,才能选中某一通道,使开关接通。 $A_2 \sim A_0$ 是开关通道号输入端,当 $A_2 \sim A_0$ 输入 000~111 时,分别对应 0~7 通道上的开关处于闭合状态。通常, $\bar{S}$ 和 $A_2 \sim A_0$ 信号由接在 CPU 数据总线上的一个锁存器提供,这样就可

以用输出指令实现通道选择。 $\bar{S}$ 端和 $A_2 \sim A_0$ 引脚均要求输入 TTL 电平信号,而各个 CMOS 开关则要求用 CMOS 电平控制,逻辑电平转换电路完成从 TTL 电平到 CMOS 电平的转换。8 个 I/O 引脚 $I/O_0 \sim I/O_7$ 可以作为输入端,这时 O/I 引脚便作为输出端,开关实现 8 到 1 的选择功能。由于 CMOS 开关可以双向工作,即信号也允许从 O/I 引脚输入,根据需要,从 8 个 I/O 引脚中的某一个输出,实现 1~8 的分配功能。该片子有 3 个电源引脚,其中, V_{SS} 通常与系统模拟地相连, V_{DD} 接正电压(如 12V), V_{EE} 接负电压(如 -12V)或地。

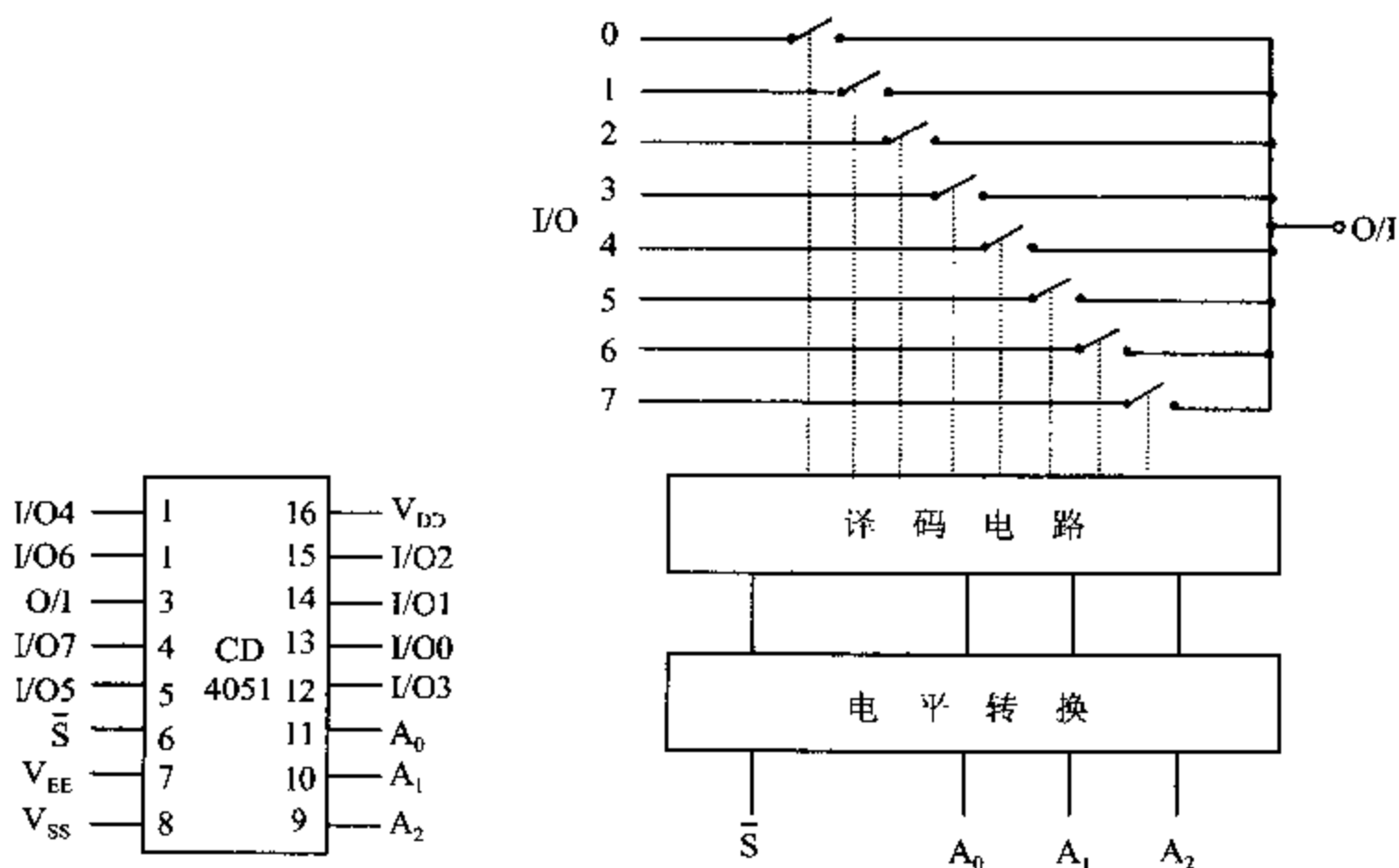


图 11-2 CD4051 的引脚图

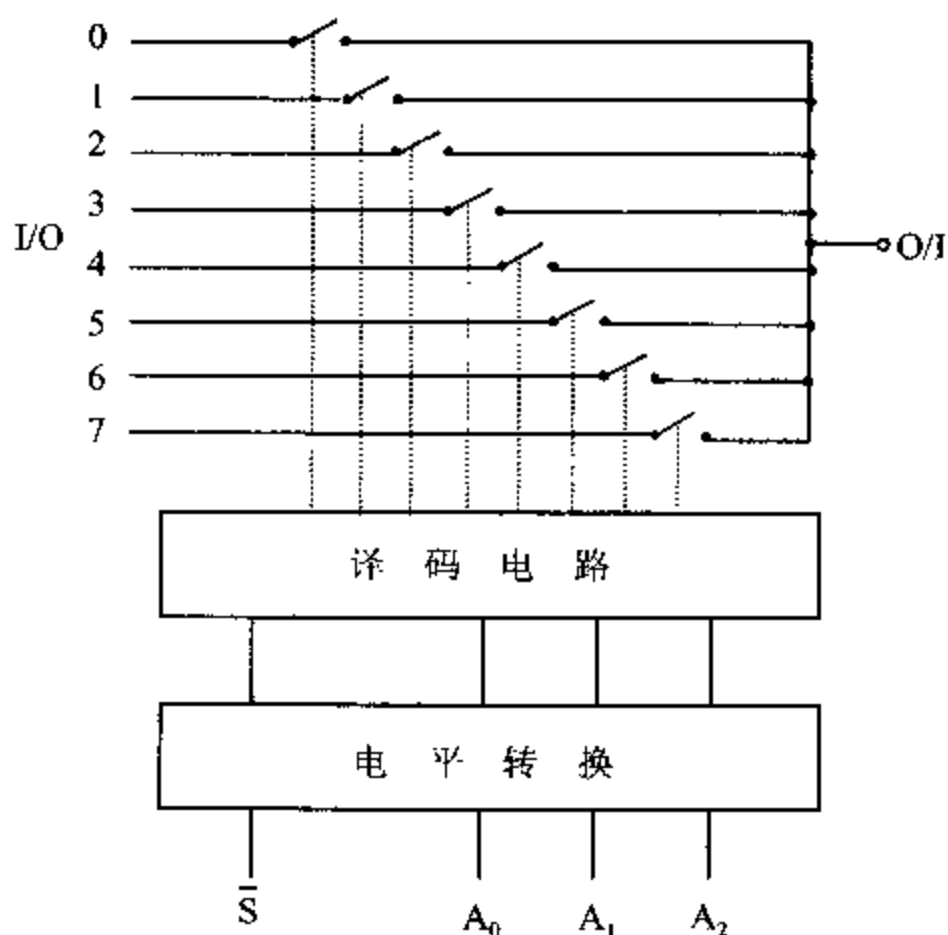


图 11-3 CD4051 的逻辑功能图

多路模拟开关的典型应用是与采样保持器和 A/D 转换器配合,构成多路数据采集通道。图 11-4 是用 CD4051 作多路开关,构成具有 8 个输入通道的数据采集电路的例子。

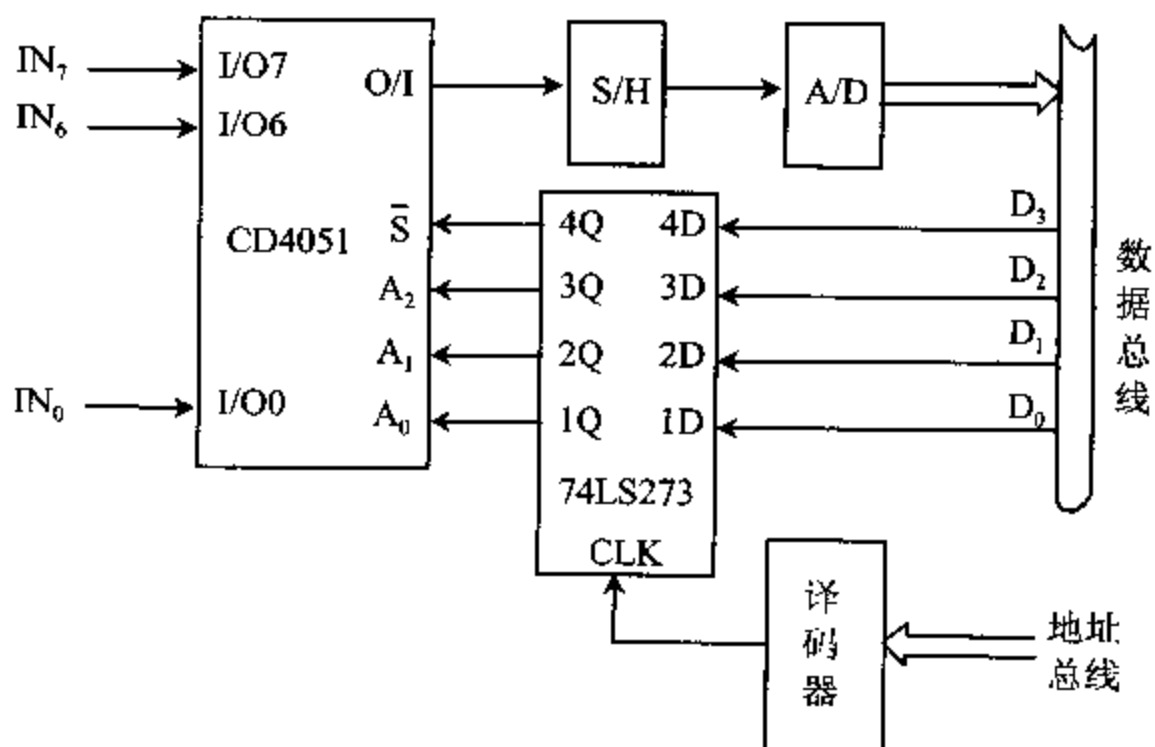


图 11-4 用多路开关构成的 8 通道数据采集电路

经放大、滤波等预处理之后的 8 路模拟信号 $IN_0 \sim IN_7$ 分别送到 CD4051 的 8 个 I/O 引脚, CPU 数据总线的 $D_3 \sim D_0$ 接到锁存器 74LS273 的输入端。锁存后的 D_3 位 4Q 与选通端 $\bar{S}$ 相连, 当 $D_3 = 0$ 时, $\bar{S} = 0$, 使 CD4051 选通; $D_3 \sim D_0$ 位经锁存后分别与 CD4051 的通道号输入端 $A_2 \sim A_0$ 相连, 实现通道选择。对于 CPU 来说, 锁存器相当于一个输出端口, 由 I/O 地址译码器中的一个片选信号选通, 因此可用 OUT 指令将希望的编码经 $D_3 \sim D_0$ 打入锁存器后送到多路开关的 $A_2 \sim A_0$ 和 $\bar{S}$ 端。

假设锁存器的 I/O 地址为 ADPORT, 若要将接在 IN_6 端的模拟信号切换到 A/D 转换器去, 可用如下指令实现:

```
MOV    AL, 00000110B      ;  $D_3$  必须为 0,  $D_3 \sim D_0 = 110$ , 表示通道 6
OUT    ADPORT, AL
```

三、采样、量化和编码

模拟信号经预处理后从多路开关输出时, 信号幅度已达到几伏的数量级, 它还必须经过采样、量化和编码的过程才能成为数字量。

1. 采样和量化

采样就是按相等的时间间隔 t 从电压信号上截取一个个离散的电压瞬时值, 这些值都有精确的大小。严格地讲, 必须用无穷多位小数才能真正表示出它们的电压值来, 但实际上只能把这些采集下来的电压瞬时值表示到一定的精度。

如图 11-5 所示, 有一个被采样的信号电压的幅度范围为 $0 \sim 7$ 伏, 若把它们分为 8 层, 包括电平 $0V$ 在内, 每个分层为 1 伏, 然后看每个采样处于哪个分层之中, 该分层的起始电平就是这个采样的数字量。

例如, t_0 时刻采样的实际值为 3.7 伏, 它处于 $3V \sim 4V$ 分层中, 因此它的数字量就是 3; 其余依次类推。这个过程称为量化。每个分层所包含的最大电压值与最小电压值之差为 1 伏, 它被称为量化单位, 用 q 表示。显然, 量化单位越小, 一定范围内的电压值被分的层数就越多, 采样值的数字量与实际电平之间的误差也越小, 精度也就越高。

A/D 转换器输出的数字量, 可采用若干种代码表示。图 11-5 中采用二进制编码, 即用 $000 \sim 111$ 分别表示数字量 $0 \sim 7$ 。有关编码的问题, 下面再作进一步讨论。

从量化过程可以看到, t 越小, 或者说采样率 f_s 越高, 每秒内采集的点数就越多, 采得的值便越接近于原信号。但若 f_s 太大, 模数转换实现起来就比较困难, 成本也会显著提高。为了能保证从数字量恢复出来的原信号质量较好, 要求采样率必须满足采样定理所规定的指标, 即 $f_s \geq 2f_m$, 其中 f_m 是信号所包含的最高频率分量, 可通过对信号进行频谱分析后确定。实际应用中, 常取 $f_s = (3 \sim 4)f_m$ 。

在对模拟信号时行采样时, 为了与计算机表示数的方法一致, 分层数必须是 2^n 。例如, 图 11-5 中的分层数为 $2^3 = 8$, 或 $n = 3$ 。实际的 A/D 转换器, n 通常为 8, 10, 12, 16 等。为简单起见, 常用位数 n 来表示 A/D 转换器的分辨率 (Resolution), 它表明 A/D 转换器能分辨最小量化信号的能力。

每个 A/D 转换器都有一个允许的最大输入电压范围, 通常还有一个参考电压 V_R 输入

引脚,在输入电压值范围内改变 V_R 的值,可以设定 A/D 转换器能够转换的电压范围,也即指定它的量程。

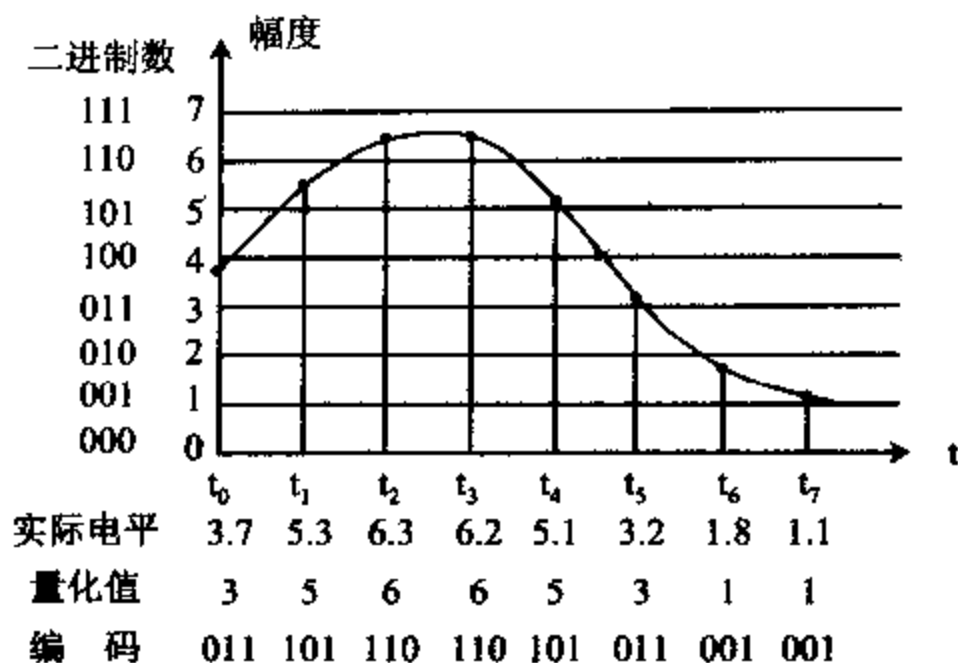


图 11-5 模拟电压信号被量化的例子

例如,一个 8 位的 A/D 转换器,它将把输入电压信号分成 $2^8 = 256$ 层,若它的量程为 0 ~ 5V,那么,它能分辨的最小的量化信号电平,即量化单位为:

$$q = \frac{\text{电压量程范围}}{2^n} = \frac{5.0V}{256} \approx 0.019V = 19mV$$

q 正好是 A/D 输出的数字量中最低位 $LSB=1$ 时所对应的电压值,因而也称为 1LSB。

2. 编码

经采样和量化后,模拟量转化成了数字量,数字量可以用不同的代码来表示,这就是数字量的编码。

由于模数转换器的输出要与计算机接口,采用何种形式表示数字量要考虑计算机处理是否方便等因素,而数模转换器接受计算机输出的数字量,实现数字量到模拟量的逆变换,也涉及到数字量的编码问题。编码的形式有好几种,如二进制码,BCD 码,ASCII 码等。各器件采用的编码方式在制造过程就固定好了,有些器件可通过外部连线,选择几种编码方式。下面介绍两种最常用的编码形式:自然二进制码和双极性二进制编码。

(1) 自然二进制码

前面已经讲到,量化过程是先将一个由参考电压 V_R 设定的满量程(FSR)电压值分成 2^n 等分,然后将采样所得的模拟量与这些分层进行比较,看落在哪个分层范围内,便量化成相应的数字量。输入模拟量与满量程相比得到的是一个分数,若以满量程为 1,则这个比值总是小于 1 的小数。为此,数据转换中常用二进制小数形式表示数字量,这就是自然二进制码,或称为自然二进制小数码。

n 位自然二进制小数码表示一个小数 N 的形式是:

$$N = d_1 2^{-1} + d_2 2^{-2} + \dots + d_n 2^{-n}$$

式中,系数 d_i 的取值只有 0 或 1 两种可能,它表示二进制小数中第 i 位上的数码, 2^{-n} 表示了小数中各位上的加权。第 1 位加权最大,当 $d_1=1$ 时,它对 N 的贡献(加权)最大,等于 $1/2$,它被称为最高有效位 MSB; 最右边的第 n 位的加权最小,等于 $1/2^n$,被称为最小有效

位 LSB,实际上就等于量化单位 q 。

采用自然二进制编码时,小数点不必表示出来。例如,二进制小数 0.110101 被记作 110101。根据上面的式子,它可转换成十进制小数:

$$N = 1 \times 0.5 + 1 \times 0.25 + 0 \times 0.125 + 1 \times 0.0625 + 0 \times 0.03125 + 1 \times 0.015625 \\ = 0.828125 = 82.8125\%$$

也就是说,二进制码 110101 所表示的模拟量是满量程的 82.8125%。如果用 V_R 表示参考电压,即满量程值,用 V_X 表示实际模拟电压值,则

$$V_X = V_R \times N$$

当 $V_R = +10V$ 时,数字量 110101 表示模拟电压为: $V_X = 10V \times 0.828125 = 8.28125V$ 。而当 $V_R = +5V$ 时,它表示的模拟电压为: $V_X = 5V \times 0.828125 = 4.140625V$ 。

可见,A/D 转换器的量程不同,同一个数字量所表示的模拟量大小也不一样。如前所述,我们可以通过改变 A/D 和 D/A 转换器的参考电压,来改变它们的量程。

若已知参考电压 V_R 和模拟电压 V_X ,也可以计算出 V_X 的数字量 N 来。例如,当 $V_R = +5V$, $V_X = 1.0V$ 时, V_X 的数字量为:

$$N = V_X / V_R = 1.0V / 5.0V = 0.2$$

将它转换成二进制小数,结果为 $0.2 = 0.00110010$,由于在机器中小数点不表示出来,所以用 00110010 或 32H 来表示 V_X 的数字量。

表 11-1 列出了在满量程 FSR 分别为 5V 和 10V 的情况下,4 位二进制小数码的表示方法。

表 11-1 4 位($n=4$)二进制小数码的表示

输入模拟量	二进制小数	输出数码	对应电压	
			FSR=+5V	FSR=+10V
0	0.0000	0000	0.000	0.000
1/16FSR	0.0001	0001	0.312	0.625
2/16FSR	0.0010	0010	0.625	1.250
3/16FSR	0.0011	0011	0.937	1.875
⋮				
15/16FSR	0.1111	1111	4.687	9.375

从表 11-1 可知,数字量的最大值(全 1 或满码)并不等于满量程电压,它等于 $FSR \times (1-2^{-n})$,也就是说,它比满量程小 1LSB。例如,在 $FSR = +5V$ 时,满码 1111 表示 $5V \times (1-2^{-4}) = 4.6875V$ 。如果用的是 12 位 A/D 转换器,满码为 1111 1111 1111,它对应的电压值为 $5V \times (1-2^{-12}) = 4.9988V$ 。

(2) 双极性二进制编码

要是 A/D 转换器允许输入双极性信号,如 $\pm 5V$,为了表示 V_X 的极性,还可用双极性码来表示数字量。常用的双极二制编码主要有符号-数值二进制码和偏移二进制码两种编码方式,它们的 4 位二进制编码如表 11-2 所示。

表 11-2 常用的双极性二进制编码表

数	输入量	符号-数值	偏移二进制	对应电压(V), FSR=±5V
+7	+7/8FSR	0111	1111	+4.375
+6	+6/8FSR	0110	1110	+3.750
+5	+5/8FSR	0101	1101	+3.125
+4	+4/8FSR	0100	1100	+2.500
+3	+3/8FSR	0011	1011	+1.875
+2	+2/8FSR	0010	1010	+1.250
+1	+1/8FSR	0001	1001	+0.625
0	0+	0000	1000	0.000
	0-	1000		
-1	-1/8FSR	1001	0111	-0.625
-2	-2/8FSR	1010	0110	-1.250
-3	-3/8FSR	1011	0101	-1.875
-4	-4/8FSR	1100	0100	-2.500
-5	-5/8FSR	1101	0011	-3.125
-6	-6/8FSR	1110	0010	-3.750
-7	-7/8FSR	1111	0001	-4.375
-8	-8/8FSR	—	0000	-5.000

D/A 转换是 A/D 转换的逆过程, 量化和编码的原理对 D/A 也是适用的。

四、采样保持器

1. 采样过程

将模拟信号转换成离散信号的采样过程可以用图 11-6 形象地表示。

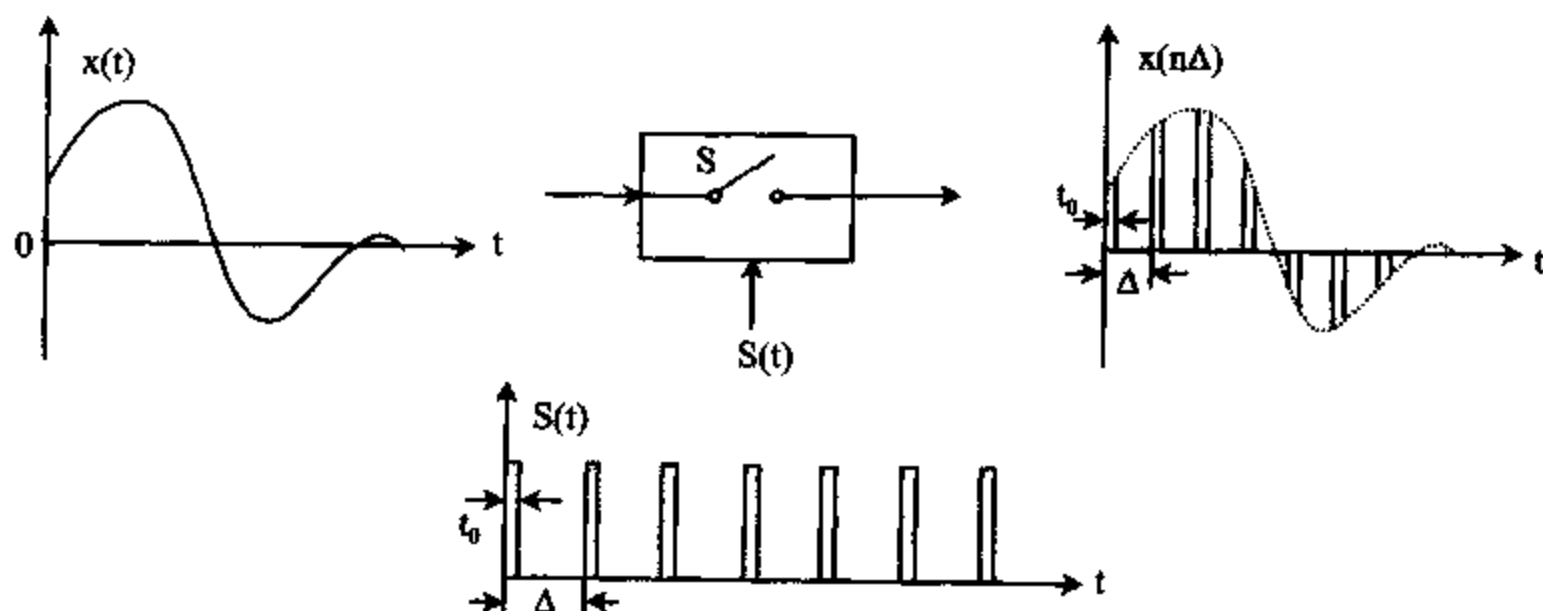


图 11-6 采样过程

图中,连续的模拟信号 $x(t)$ 加到采样器的输入端,采样器的输出受采样脉冲 $S(t)$ 的控制,采样脉冲是一种周期为 Δ ,宽度为 t_0 的矩形脉冲序列。每当采样脉冲出现时,开关 S 接通 t_0 秒,输入模拟信号可以通过采样器到达输出端;采样脉冲消失时, S 断开,采样器无输出。这样,从采样器输入的连续模拟信号在采样脉冲的调制下,在输出端得到宽度为 t_0 ,周期为 Δ 的脉冲序列 $x(n\Delta)$,脉冲序列的幅度被 $x(t)$ 所调制,这个过程就是采样。 $x(n\Delta)$ 序列就是采样所得的离散模拟量,这些离散量出现的重复频率就是采样脉冲的频率 f_s , f_s 称为采样率,其值为 $1/\Delta$ 。

一个模数转换器完成一次模数转换,要进行量化、编码等操作,每种操作均需花费一定的时间,这段时间称为模数转换时间 t_c 。在转换时间 t_c 内,若输入模拟信号比较平坦,即 $x(t)$ 的变化量 Δx 很小,可认为 $\Delta x \approx 0$ 。这样,由采样过程引入的误差可以忽略不计。在这种情况下,模数转换器可直接与采样器的输出端相连。一般将采样开关制作在 A/D 转换器内部,使采样器和转换器合为一体。

当 $x(t)$ 变化速率较高时,在转换过程中,输入模拟量有一个可观的 Δx ,结果将会引入较大的误差。也就是说,在 A/D 转换过程中,加在转换器上的电平在波动,这样,就很难说输出的数字量表示 t_c 期间输入信号上哪一点的电压值,在这种情况下就要用采样保持器来解决这个问题。

2. 采样保持器

使用采样保持器可以将采样和保持这两个过程分开,即在采样开关与模数转换器之间加接一个电压保持器,采样保持电路和它的输入输出波形如图 11-7 所示。

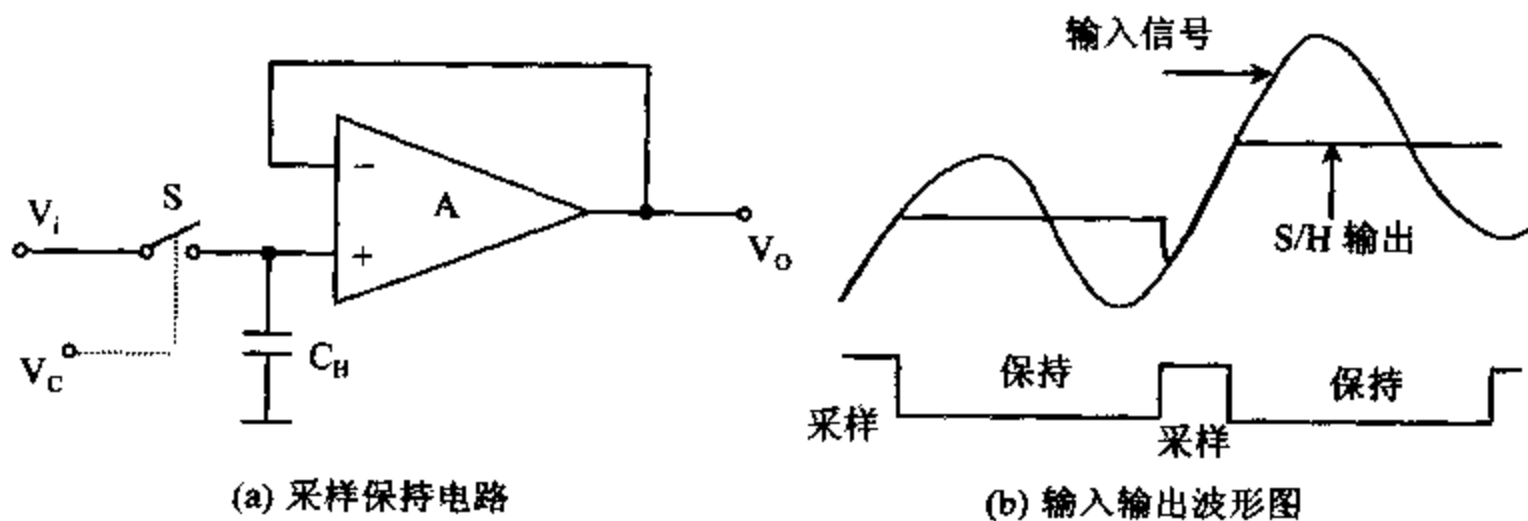


图 11-7 采样保持电路和输入输出波形

最基本的采样保持器电路如图 11-7(a)所示,它由模拟开关 S ,保持电容 C_H 和缓冲放大器 A 组成。假设控制信号 V_c 中的高电平为采样命令,在它作用期间, S 合上,输入模拟信号 V_i 通过开关 S 向保持电容 C_H 充电。由于缓冲放大器的跟随特性,这期间输出电压 V_o 跟随输入电压 V_i 而变化。当采样脉冲命令结束后,开关 S 断开,电容 C_H 上的电压能在一段时间内保持基本不变,缓冲放大器的输出电压 V_o 便被保持在开关断开前瞬间的值,从而实现了采样和保持的功能。采样保持器的输入输出波形如图 11-7(b)所示。

采样保持器有三个重要的指标:

(1) 孔径时间 T_{AP}

采样脉冲结束瞬间,便是保持命令的开始。但是,从保持命令发出到开关完全断开需要

一定的时间,这段时间称为孔径时间 T_{AP} (Aperture Time),它是模拟开关从闭合状态到断开状态的过渡时间。在 T_{AP} 时间内,输出信号 V_O 仍跟随 V_i 而变。由于它的存在,致使希望的采样时间展宽了,因而使实际的电压保持值与希望的保持值之间产生误差,影响转换精度。

(2) 捕捉时间 T_{AC}

捕捉时间 T_{AC} 也称为采集时间 (Aquisition Time),它是指控制信号 V_C 从保持电平转化为采样电平后,采样保持器的输出由保持值过渡到重新跟踪输入信号 V_i 的变化所需要的时间。 T_{AC} 主要由模拟开关 S 的导通延时和建立跟踪关系所需要的稳定过渡引起的。控制信号 V_C 中的采样脉冲宽度一定要大于 T_{AC} ,才能保证采样阶段中可靠地采集到输入信号 V_i 。

(3) 保持电压衰减速率

在保持状态下,保持电容的漏电流及其它杂散电流会引起保持电压衰减,其变化率为:

$$\frac{dV_O}{dt} = \frac{I_D}{C_H}$$

式中, I_D 是保持期间流入或流出保持电容 C_H 的总泄漏电流。增大 C_H 可减小保持电压的衰减率,但会引起捕捉时间 T_{AC} 增大。所以,要挑选低泄漏的电容器作保持电容,以减小保持电压的衰减速率。

采样保持器一般由输入缓冲器,输出缓冲器,采样保持开关和控制逻辑,保持电容等几部分组成。目前,采样保持器已集成在一块芯片内,常用的采样保持电路有以下三类:

(1) 通用型。如 AD582, AD583, LF198, LF398 等,它们的捕捉时间 T_{AC} 在几 μs 量级,孔径时间 T_{AP} 约为几十 ns 到 100 多 ns。

(2) 高速型。如 THS-0025, THS-0060, THC-0030, THC-1500 等,它们的 T_{AC} 和 T_{AP} 均很小,只有 20ns~30ns。

(3) 高分辨率型。如 SHA1144,它是专门为 14 位高分辨率型的 A/D 转换器设计的。

11-2 D/A 转换器

前面介绍了 A/D、D/A 通道的基本概念及多路开关、采样保持器等基本部件。下面讨论模数和数模转换中的核心部件 A/D 转换器和 D/A 转换器。由于 D/A 转换器的原理比较简单,而且大部分 A/D 转换器内部还包含 D/A 转换电路,因此先介绍 D/A 转换器的工作原理和使用方法。

一、数/模转换器原理

D/A 转换器是把输入的数字量转换为与输入量成比例的模拟信号的器件。多数 D/A 转换器把数字量(如二进制编码)变成模拟电流,如要将其转换成模拟电压还要使用电流/电压转换器(I/V)来实现。少数 D/A 转换器内部有 I/V 变换电路,可直接输出模拟电压值。I/V 转换电路由运算放大器构成。

为了了解 D/A 转换器的工作原理,先分析一下图 11-8 所示的 4 路输入加法器电路,即

权电阻网络 D/A 转换器。

图中, $d_1 \sim d_4$ 为 4 位输入数字量, $R, 2R, 4R$, 和 $8R$ 为加权电阻, $S_1 \sim S_4$ 是电子模拟开关。当某位 $d_i = 1$ 时, 相应开关闭合, 接通权电阻; $d_i = 0$ 时, 开关断开。运算放大器的同相输入端(+)接地, 由于运放的输入阻抗非常高, 流入反相输入端(-)的电流几乎为 0, 同相端与反相端之间的电流也非常小, 所以反相输入端的电压也为 0, 与同相端的电位相等, Σ 点被称为虚地点。参考电压 V_R 为权电阻支路提供权电流, 各支路中电流的大小与权电阻成反比关系。

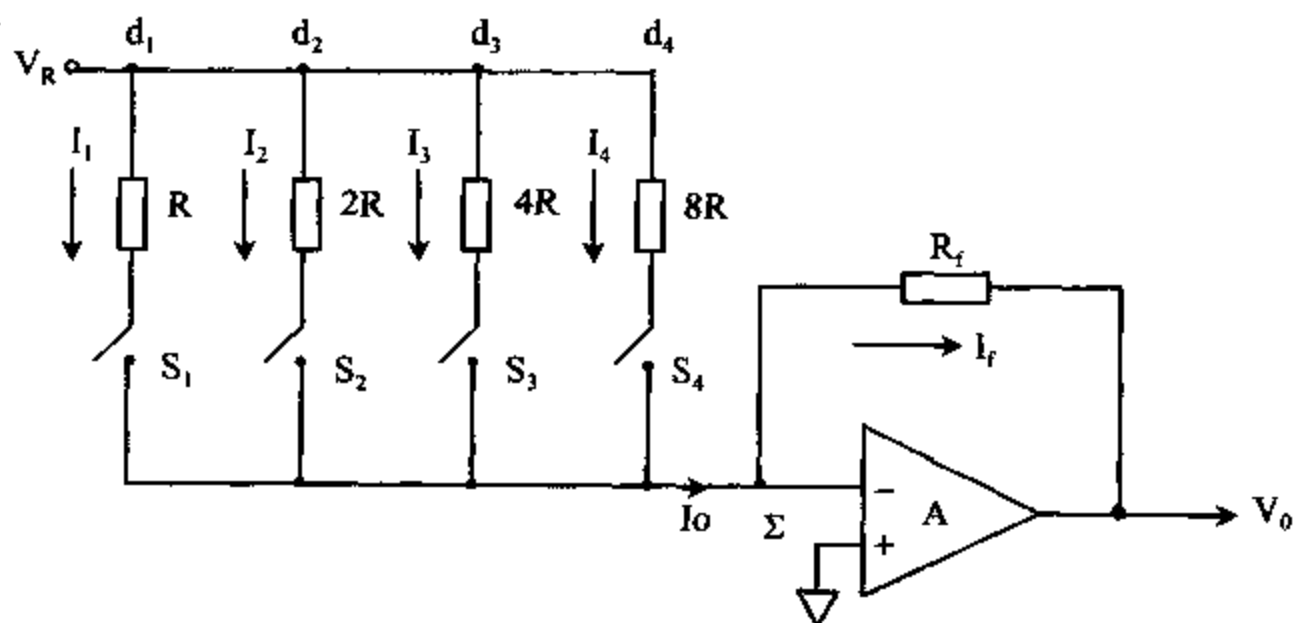


图 11-8 权电阻网络 DAC

流入相加点 Σ 的总电流为:

$$\begin{aligned} I_O &= d_1 I_1 + d_2 I_2 + d_3 I_3 + d_4 I_4 \\ &= d_1 \frac{V_R}{R} + d_2 \frac{V_R}{2R} + d_3 \frac{V_R}{4R} + d_4 \frac{V_R}{8R} \\ &= \frac{2V_R}{R} (d_1 2^{-1} + d_2 2^{-2} + d_3 2^{-3} + d_4 2^{-4}) \end{aligned}$$

由于流入运放的电流为 0, 所以 $I_f = I_O$ 。又因 Σ 点虚地, 故运放的输出电压 $V_O = -I_f \times R_f$ 。这样, 对于同样的输入数据, 当参考电压 V_R 改变时, I_O 和 V_O 均随之而改变。若固定 V_R , 则输出电压 V_O 与输入数字量成正比关系。

如果 $R_f = R/2$, 输入数字量 $d_1 d_2 d_3 d_4 = 1000$, $V_R = +5V$, 则输出电压

$$\begin{aligned} V_O &= -I_O \times R_f \\ &= -\frac{2V_R}{R} \times \left(1 \times \frac{1}{2} + 0 \times \frac{1}{4} + 0 \times \frac{1}{8} + 0 \times \frac{1}{16} \right) \times \frac{R}{2} \\ &= -\frac{1}{2} V_R = -2.5V \end{aligned}$$

这样, 用二进制数字控制开关的通断, 就可产生相应的输出电压信号。但是, 与模数转换器只能包含有限数目的分层数那样, 数模转换器中的开关和权电阻的数目也是有限的, 因此, D/A 转换器输出的电压仅是某些固定的值。例如, 对于上面的例子, 输入数字量的范围限制在 0000~1111B 的范围内, 相应的输出电压值也只有 16 种, 它们的大小落在 0V 到 $V_R(1 - 2^{-4})V$ 范围内。在用 D/A 转换器形成一个波形时, 就要每隔一定的时间 Δt , 由程序将一个数字量送给 DAC, 形成一个电压值。结果, 当你用示波器观察这个波形时, 会发现波形上

有许多台阶,它们就是这些不连续的电压值形成的。 Δt 越小,这些台阶就越窄;D/A 转换器中包含的开关和权电阻数越多,也就是 D/A 的位数越多,任意两个相邻的数字量形成的电压台阶之间的高度差也就越小,即输出波形与真实的模拟信号越接近。

从上面的例子可以看出,D/A 转换器的核心部分是一组按输入二进制数字控制开关产生二进制加权电流的部件。实现 D/A 转换的方案还有好几种,如 R-2R 梯形电阻网络 DAC, $2^n R$ 电阻分压式 DAC 等,它们都做在集成电路芯片内部。限于篇幅,在此就不一一介绍了。

二、数/模转换器的主要性能指标

1. 输入数字量

它包括输入数字量的码制、数据格式和它们的逻辑电平等,手册上均有说明。多数 D/A 转换器只接受自然二进制编码,少数产品采用双极性二进制编码或 BCD 编码等。输入数据的格式一般都是并行码,后面将举例说明。输入数据的逻辑电平一般为 TTL 电平,少数产品还可能接受 CMOS 或 PMOS 电平的数字量。

2. 输出模拟量

多数 D/A 转换器是电流输出型,手册上总是给出在规定的参考电压 V_R (或 V_{ref}) 下,输入为满码时的输出电流表达式。有些器件有一对电流输出引脚 I_{O1} 和 I_{O2} ,也记为 I_{OUT1} 或 I_{OUT2} ,例如,8 位 D/A 转换器 DAC0832 的输出电流为:

$$I_{O1} = \frac{V_R}{15k} \times \frac{\text{输入数字量}}{256}$$

$$I_{O2} = \frac{V_R}{15k} \times \frac{255 - \text{输入数字量}}{256}$$

输入为满码时的输出电流为:

$$I_{O1} = \frac{V_R}{15k} \times \frac{255}{256}$$

$$I_{O2} = 0$$

式中,数值 15k 表示 DAC 内部 R-2R 电阻网络中的电阻阻值。为了将输出电流转换成输出电压,可在 DAC 的输出端加一个运算放大器 A 和反馈电阻 R_f 构成的 I/V 转换电路。通常, R_f 做在 D/A 转换器的内部,在 D/A 片子上有专门的引脚,并让 R_f 的大小与权电阻相同。这样,只需用一个运放就能构成 I/V 转换电路,如图 11-9 所示。

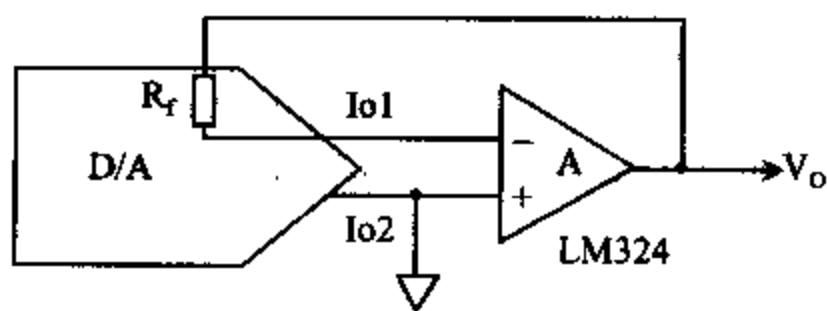


图 11-9 I/V 转换电路

从图中的 I/V 转换电路可得到如下的输出电压表达式:

$$V_O = -I_{O1} \cdot R_f$$

$$= -\frac{V_R}{15k} \cdot \frac{\text{输入数字量}}{256} \cdot 15k$$

$$= -V_R \cdot \frac{\text{输入数字量}}{256}$$

由此可知,输出电压仅与参考电压和输入数字量有关。

3. 分辨率(Resolution)

分辨率的概念在前面已经提到,它是指输入数据发生 1LSB 的变化时所对应的输出模拟量的变化,分辨率 Δ 与输入数字量的位数 n 之间有如下关系:

$$\Delta = \frac{\text{FSR}}{2^n}$$

式中,FSR 是 D/A 转换器的满量程,它近似等于输入为满码时的输入电压值。通常也用百分数来表示分辨率。例如:

对于 8 位 D/A 转换器, $2^n = 256$, 其分辨率为 $1/256 \times \text{FSR} = 0.39\% \text{FSR}$

对于 12 位 D/A 转换器, $2^n = 4096$, 其分辨率为 $1/4096 \times \text{FSR} = 0.0244\% \text{FSR}$

由于分辨率与转换器的位数之间具有固定的对应关系,一般简单地用它们的位数来表示分辨率,如 D/A 转换器的分辨率可以是 8,10,12,16 等等。

4. 精度(Accuracy)

由于转换器内部电路的误差等原因,当送一个确定的数字量给 DAC 后,它的实际输出值与该数值应产生的理想输出值之间会有一定的误差,它就是 D/A 转换器的精度,通常用此差值与满量程输出电压或电流的百分比来表示。例如,某一转换器的电压满量程为 10 伏,其精度为 0.02%,则输出电压的最大误差为 $10.00\text{V} \times 0.02\% = 20\text{mV}$ 。一般 D/A 转换器的误差应不大于 1/2LSB。

5. 线性误差(Linearity Error)

D/A 转换器的输入数字量都是连续的数值,每两个相邻的数据之间的差值为 1。若将这些连续的数据送给 DAC,应该输出一个线性变化的模拟电压。但实际的输出并不是理想线性的,通常用偏离理想转换特性的最大偏差与满量程之间的百分比来表示线性误差。一般要求线性误差不大于 1/2LSB。如 8 位 DAC,线性误差应小于 0.2%;12 位 DAC,则要求小于 0.01%。

6. 建立时间(Setting Time)

建立时间也称为稳定时间,用 t_s 表示,它是指从数字量输入到建立稳定的输出电流的时间。超高速的 DAC, $t_s < 100\text{ns}$; 较高速的 DAC 的 $t_s = 1\mu\text{s} - 100\text{ns}$; 高速 DAC, $t_s = 10\mu\text{s} - 1\mu\text{s}$; 低速 DAC 的 $t_s > 100\mu\text{s}$ 。例如,12 位 D/A 转换器 DAC1210 的 $t_s = 1\mu\text{s}$ 。

三、几种数/模转换器

1. AD7524

AD7524 是美国 AD 公司(Analog Device)生产的 8 位电流输出型 D/A 转换器,采用 CMOS 工艺制造,功耗只有 10mW,精度为 1/8 LSB。它被设计成单电源供电,而且电源电压的范围很宽,从 +5V 到 +15V,它可以直接与使用 +5V 电源的微机总线相连。图 11-10 是它的一个实用电路。

CPU 把 AD7524 当成一个由 8 位数据锁存器构成的输出端口,由地址总线经译码后形

成的 I/O 片选信号与 AD7524 的 $\overline{CS}$ 端相连, 赋予它一个端口地址。系统总线的 I/O 写信号 $\overline{IOW}$ 接到它的写使能端 $\overline{WR}$ 。当用输出指令将待转换的 8 位数据从数据总线 $D_7 \sim D_0$ 送到 DAC 时, $\overline{WR}$ 和 $\overline{CS}$ 有效, 转换立即开始, 经 t_s 时间后, 转换结束。实际上转换进行得很快, 一条 OUT 指令执行完后, 转换就结束了。这就是说, 只要 CPU 选中 D/A 转换器的端口, 执行一条 OUT 指令, 就可实现一次 D/A 转换。

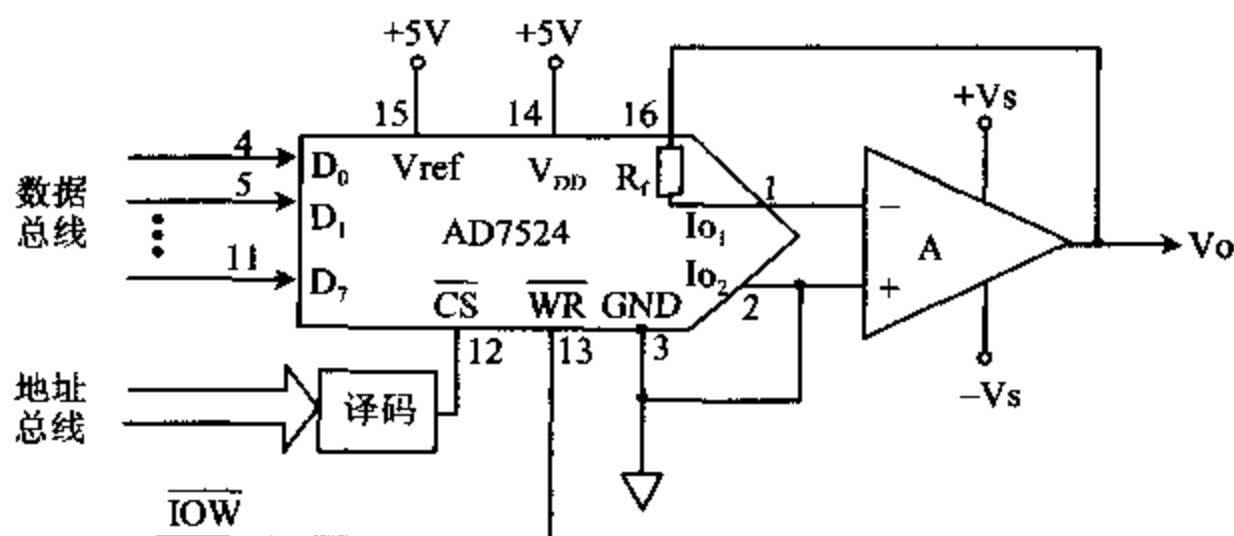


图 11-10 AD7524 的一个实用电路

转换结束后, 在互补的电流输出端 I_{O1} 和 I_{O2} 端可得到输出电流, 经 I/V 转换电路后, 在 V_O 端形成模拟电压。 V_O 的大小由输入数字量和参考电压 V_{ref} 决定, V_{ref} 允许的范围为 $\pm 10V$, 本例中 $V_{ref} = +5V$, 其输出电压为:

$$V_O = -V_R \frac{\text{输入数字量}}{256}$$

通常再在输出端加一级反相型电压跟随器, 使 $V_O' = V_O$ 。所以当输入数字量为 0 时, $V_O' = 0V$; 当数字量为 FFH 时, $V_O = 4.98V$ 。如果把输出信号 V_O 与示波器的 Y 轴相连, 再把两者的地连在一起, 就可在示波器上观察 D/A 转换后的输出波形。

通过编程来改变送给 DAC 的数据和控制向 DAC 发送数据的时间, 就能用它来产生各种波形。下面是用图 11-10 的电路形成特定波形的两个编程例子。

例 11-1 设图 11-10 中 DAC 的口地址为 80H, 要求输出从 0V 向 4.98V 线性增长的周期性锯齿波。程序如下:

```
START:  MOV    AL, 0FFH           ;初值为 FFH
AGAIN:  INC     AL                ;AL 增 1
        OUT    80H, AL           ;D/A 转换
        CALL   DELAY             ;延时
        JMP    AGAIN
```

输出波形如图 11-11 所示。实际上, 波形是由 255 个阶梯波组成的, 阶梯的宽度由程序中 DELAY 子程序的延时时间决定。

例 11-2 要求用图 11-10 的电路, 形成如图 11-12 所示的正向和反向三角波, 波形下限的电压为 0.5V, 上限的电压为 2.5V。

由于 $1\text{LSB} = 5V/256 = 0.019V$, 所以下限电压对应的数据为:

$$0.5V/0.019V = 26 = 1AH$$

上限电压对应的数据为:

$$2.5V/0.019V=128=80H$$

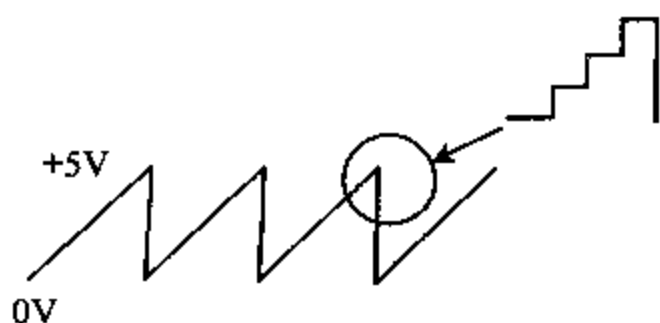


图 11-11 输出锯齿波



图 11-12 输出三角波

程序段如下:

```

BEGIN:  MOV    AL, 1AH      ;下限值
UP:     OUT     80H, AL     ;D/A 转换
        INC     AL         ;数值增 1
        CMP     AL, 81H    ;超过上限了吗?
        JNZ     UP         ;没有,继续转换
        DEC     AL         ;超过了,数值减量
DOWN:   OUT     80H, AL     ;D/A 转换
        DEC     AL         ;数值减 1
        CMP     AL, 19H    ;低于下限了吗?
        JNZ     DOWN       ;没有
        JMP     BEGIN      ;低于,转下一个周期
    
```

用 8 位 DAC 还可产生方波、梯形波和正弦波等。形成方波最容易,只要先输出一个下限电平,再用延时程序将其保持 t_1 时间,再输出一个高电平,并将它保持 t_2 时间,然后重复上述过程。梯形波则是由方波和三角波结合而成的。若要形成其它一些复杂的周期性波形,只要将一个周期的波形数据存在内存中,由程序将它们依次取出来送给 DAC,并重复这个过程。必要时,还能用 A/D 转换器从实际的波形上采集并整理后,获得所需的周期数据。

2. DAC0832

(1) 性能指标

NSC 公司(National Semiconductor Corporation)生产的 DAC0832,是一种内部带有数据输入寄存器的 8 位 D/A 转换器,采用先进的 CMOS 工艺制成,芯片内有 R-2R 梯形电阻网络,用于对参考电压产生的电流进行分流,完成模数转换,转换结果以一组差动电流 I_{OUT1} 和 I_{OUT2} 输出。

主要参数为:

分辨率	8 位
转换时间	$1\mu s$
满量程误差	$\pm 1LSB$
参考电压	$\pm 10V$
单电源	$+5V \sim +15V$

它可直接与 8085, 8048, Z80, 8088 和 8086 等微处理器总线相连。同类产品有 DAC0830 和 DAC0831,它们之间只是某些电气参数和特性略有不同。

(2) 内部结构和引脚功能

DAC0832 是一种具有 20 引脚的双列直插式器件,内部结构和外部引脚分别如图 11-13 和图 11-14 所示。

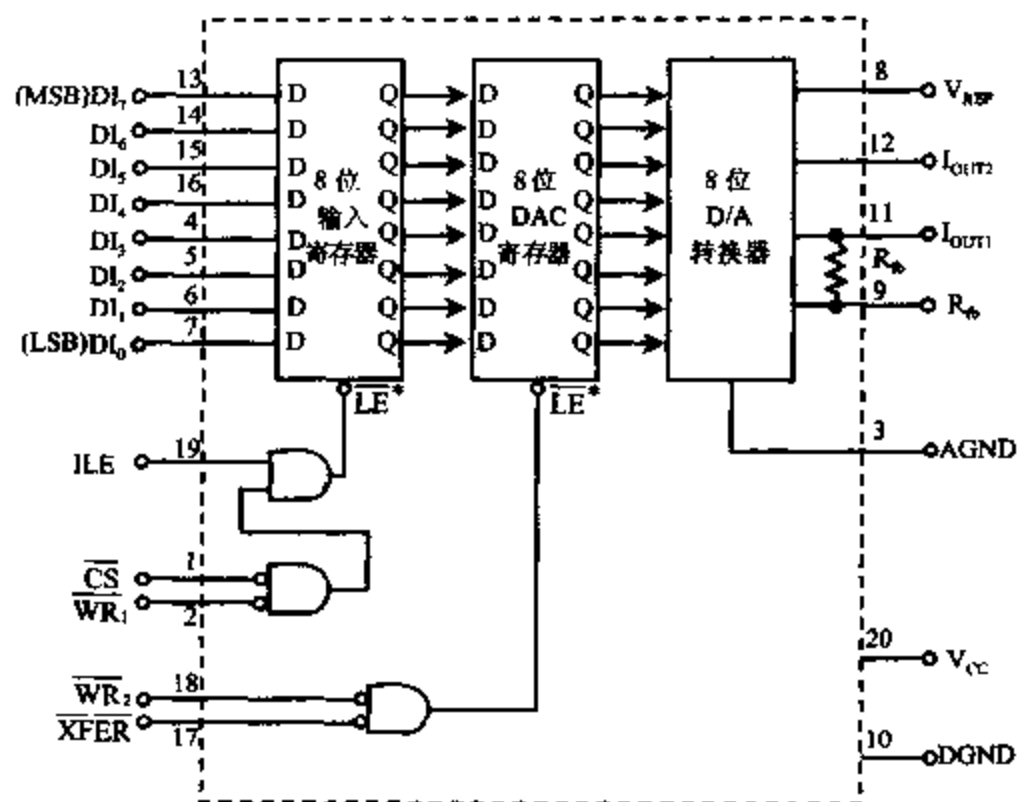


图 11-13 DAC0832 的内部结构图

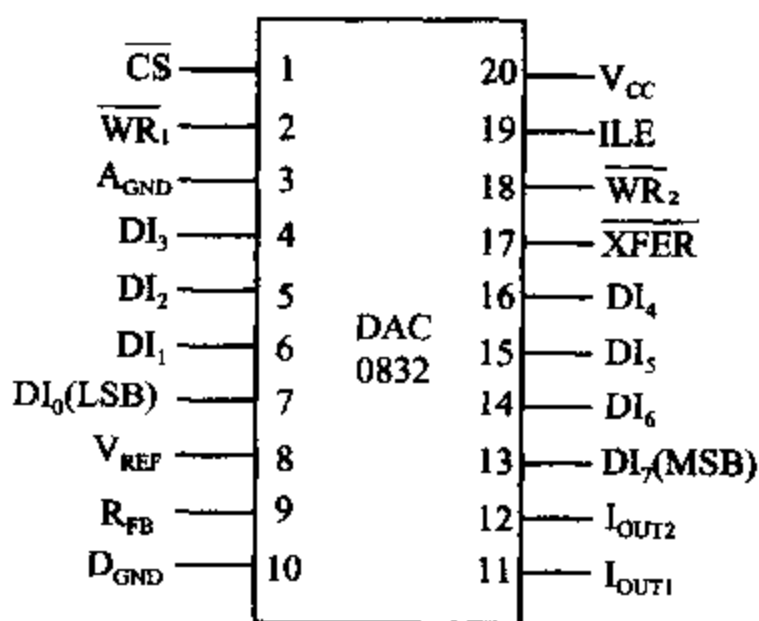


图 11-14 DAC0832 的引脚图

在 DAC0832 内部有一个 8 位输入寄存器和一个 8 位 DAC 寄存器,它们可以分别选通。这样,就可以把从 CPU 送来的数据先打入输入寄存器,在需要进行 D/A 转换时,再选通 DAC 寄存器,实现 D/A 转换,这种工作方式称为双缓冲工作方式。

各引脚的功能分述如下:

V_{REF} 参考电压输入端。根据需要接一定大小的电压,由于它是转换的基准,要求数值正确,稳定性好,常用稳压电路产生,或用专门的参考电压源提供。

V_{CC} 工作电压输入端。

A_{GND} 为模拟地, D_{GND} 为数字地。在模拟电路中,所有的模拟地要连在一起,数字地也连在一起,然后将模拟地和数字地连到一个公共接地点,以提高系统的抗干扰能力。

$DI_7 \sim DI_0$ 数据输入。可直接连到数据总线,也可以经 8255A 等并行 I/O 接口与数据

总线相连。

I_{OUT1} 和 I_{OUT2} 互补的电流输出端。为了输出模拟电压,输出端需加 I/V 转换电路。

R_{FB} 片内反馈电阻引脚。与运放配合构成 I/V 转换器。

ILE 输入锁存使能信号输入端,高电平有效。

$\overline{CS}$ 片选信号输入端。

$\overline{WR}_1$ 、 $\overline{WR}_2$ 两个写命令输入,均为低电平有效。

$\overline{XFER}$ 传输控制信号输入端,低电平有效。

当 ILE 为高电平, $\overline{CS}$ 和 $\overline{WR}_1$ 同时为低电平时,在片内,输入寄存器的锁存使能端 $\overline{LE}$ 为 1,这时,8 位数字量可以通过 DI 引脚输入寄存器;当 $\overline{CS}$ 或 $\overline{WR}_1$ 由低变高时, $\overline{LE}$ 变成低电平,数据被锁存在输入寄存器的输出端。

对于 DAC 寄存器来说,当 $\overline{XFER}$ 和 $\overline{WR}_2$ 同时为低电平时,DAC 寄存器的锁存使能端 $\overline{LE}$ 为高电平,DAC 寄存器中的内容与输入寄存器的输出数据一致;当 $\overline{WR}_2$ 或 $\overline{XFER}$ 由低变高时, $\overline{LE}$ 变成低电平,输入寄存器送来的数据被锁存在 DAC 寄存器的输出端,即可加到 D/A 转换器去进行转换。

(3) 三种工作方式

改变图 11-13 中几个控制信号的时序和电平,就可使 DAC0832 处于三种不同的工作方式。下面介绍这三种工作方式。

① 直通方式

当 ILE 接高电平, $\overline{CS}$ 、 $\overline{WR}_1$ 、 $\overline{WR}_2$ 和 $\overline{XFER}$ 都接数字地时,DAC 处于直通方式,8 位数字量一旦到达 $DI_7 \sim DI_0$ 输入端,就立即加到 8 位 D/A 转换器,被转换成模拟量。有些场合可能要用到这种工作方式。例如,在构成波形发生器时,是把要产生的基本波形数据存在 ROM 中,然后连续取出来送到 DAC 去转换成电压信号,而不需要用任何外部控制信号,就可以用这种直通方式。

② 单缓冲方式

只要把两个寄存器中的任何一个接成直通方式,而用另一个锁存数据,DAC 就可处于单缓冲工作方式。一般的做法是将 $\overline{WR}_2$ 和 $\overline{XFER}$ 都接地,使 DAC 寄存器处于直通方式,另外把 ILE 接高电平, $\overline{CS}$ 接端口地址译码信号, $\overline{WR}_1$ 接 CPU 系统总线的 $\overline{IOW}$ 信号,这样便可通过执行一条 OUT 指令,选中该端口,使 $\overline{CS}$ 和 $\overline{WR}_1$ 有效,启动 D/A 转换。

③ 双缓冲方式

主要在以下两种情况下需要用双缓冲式的 D/A 转换:

第一种情况,需要在程序的控制下,把要转换的数据先打入输入寄存器,然后再在某个时刻启动 D/A 转换。这样可以做到对某数据转换的同时,能进行下一个数据的输入,因此转换速度较高。这时,可将 ILE 接高电平, $\overline{WR}_1$ 和 $\overline{WR}_2$ 接 CPU 的 $\overline{IOW}$, $\overline{CS}$ 和 $\overline{XFER}$ 分别接两个不同的 I/O 地址译码信号。执行 OUT 指令时, $\overline{WR}_1$ 和 $\overline{WR}_2$ 均变低电平。这样,可先执行一条 OUT 指令,选中 $\overline{CS}$ 端口,把数据写入输入寄存器;再执行第二条 OUT 指令,选中 $\overline{XFER}$ 端口,把输入寄存器内容写入 DAC 寄存器,实现 D/A 转换。

图 11-15 是 DAC0832 工作于双缓冲方式下,与 8 位数据总线的微处理器相连的逻辑图。

其中, $\overline{CS}$ 的口地址为 320H, $\overline{XFER}$ 的口地址为 321H,当 CPU 执行第一条 OUT 指令,

选中 $\overline{CS}$ 端口时,选通输入寄存器,将累加器中的数打入输入寄存器。再执行第二条 OUT 指令,选中 $\overline{XFER}$ 端口,把输入寄存器的内容写入 DAC 寄存器,并启动 D/A 转换。执行第二条 OUT 指令时,AL 中的数据为多少是无关紧要的,主要目的是使 $\overline{XFER}$ 有效。

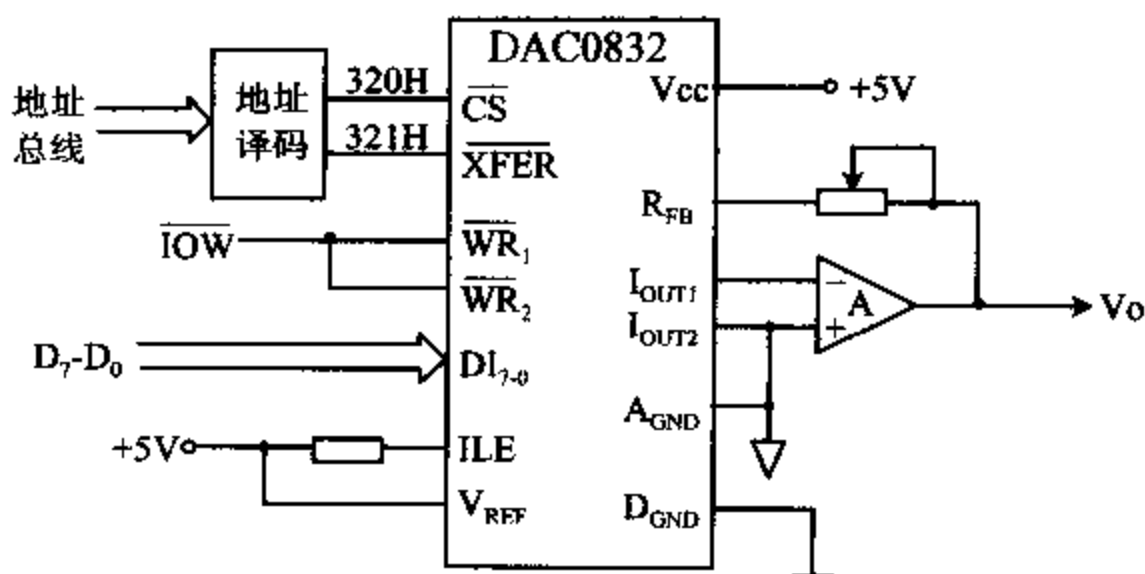


图 11-15 DAC0832 与 8 位数据总线微机的连接图

把一个数据经两次锁存,通过 DAC0832 输出的典型程序段如下:

```
MOV    DX, 320H           ;指向输入寄存器
MOV    AL, DATA          ;DATA 为被转换的数据
OUT    DX, AL             ;数据打入输入寄存器
INC    DX                 ;指向 DAC 寄存器
OUT    DX, AL             ;选通 DAC 寄存器,启动 D/A 转换
```

第二种情况,在需要同步进行 D/A 转换的多路 DAC 系统中,采用双缓冲方式,可以在不同的时刻把要转换的数据分别打入各 DAC 的输入寄存器,然后由一个转换命令同时启动多个 DAC 的转换。图 11-16 是一个用 3 片 DAC0832 构成的 3 路 DAC 系统。

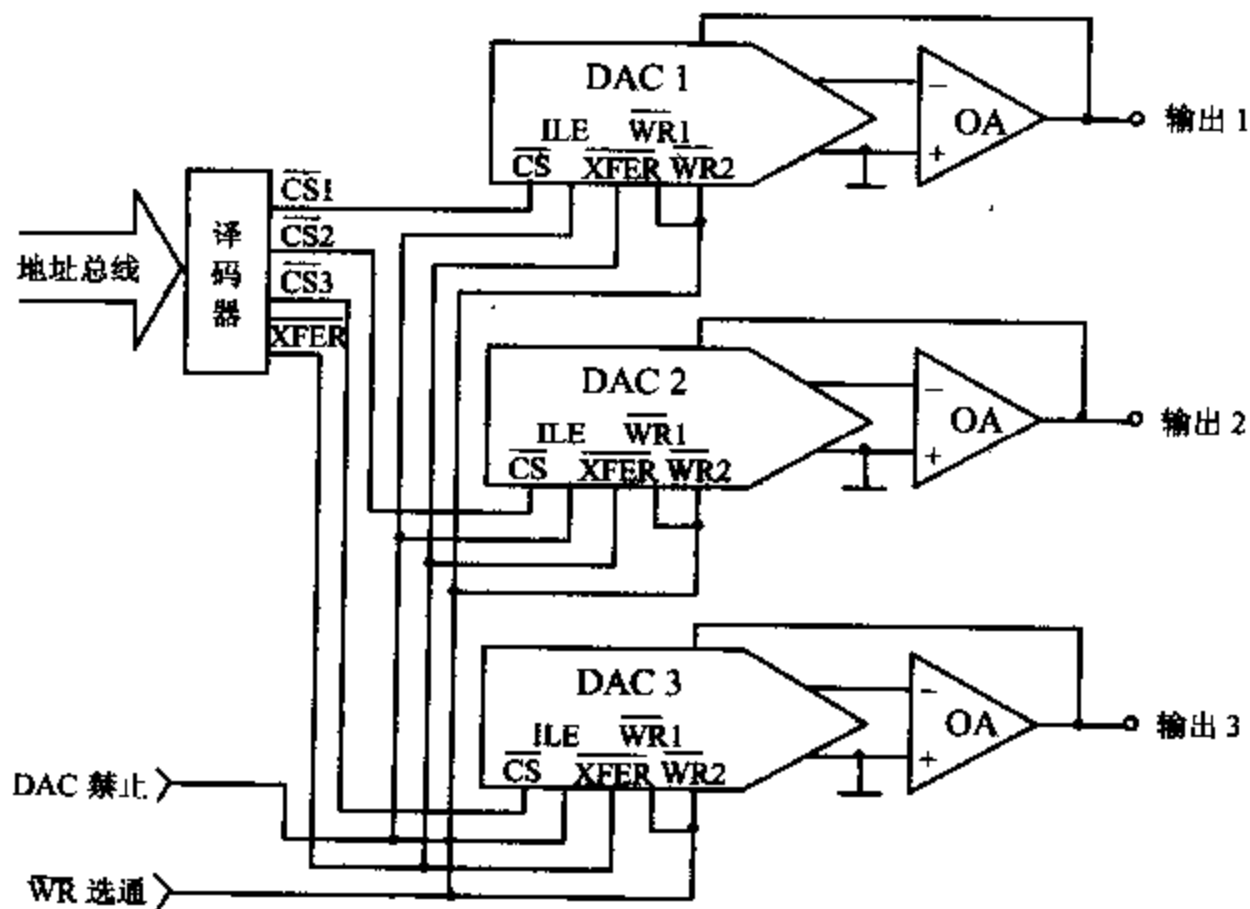


图 11-16 用 DAC0832 构成的 3 路 DAC 系统

图中, $\overline{WR}_1$ 和 $\overline{WR}_2$ 接 CPU 的写信号 $\overline{WR}$, 3 个 DAC 的 $\overline{CS}$ 引脚各由一个片选信号控制, 3 个 $\overline{XFER}$ 信号连在一起, 接到第 4 个选片信号上。ILE 可以根据需要来控制, 一般接高电平, 保持选通状态。它也可以由微处理器形成的一个禁止信号来控制, 该信号为低电平时, 禁止将数据写入 DAC 寄存器。这样, 可在禁止信号为高电平时, 先用 3 条输出指令选择 3 个端口, 分别将数据写入各个 DAC 的输入寄存器; 当数据都就绪后, 再执行一次写操作, 使 $\overline{XFER}$ 变低, 同时选通 3 个 D/A 的 DAC 寄存器, 实现同步转换。

3. DAC1210

DAC1210 是 12 位高分辨率电流输出型 D/A 转换器, 它是一种具有 24 引脚的双列直插式器件, 也是 NSC 公司的产品, 输入信号电平与 TTL 电平兼容。它的主要指标为: 电流建立时间 $t_s = 1\mu s$, 工作电压 $+5V \sim +15V$, 参考电压范围为 $\pm 25V$ 。它的工作原理与 8 位的 DAC0832 没有多大的区别。图 11-17 是 DAC1210 的逻辑图。

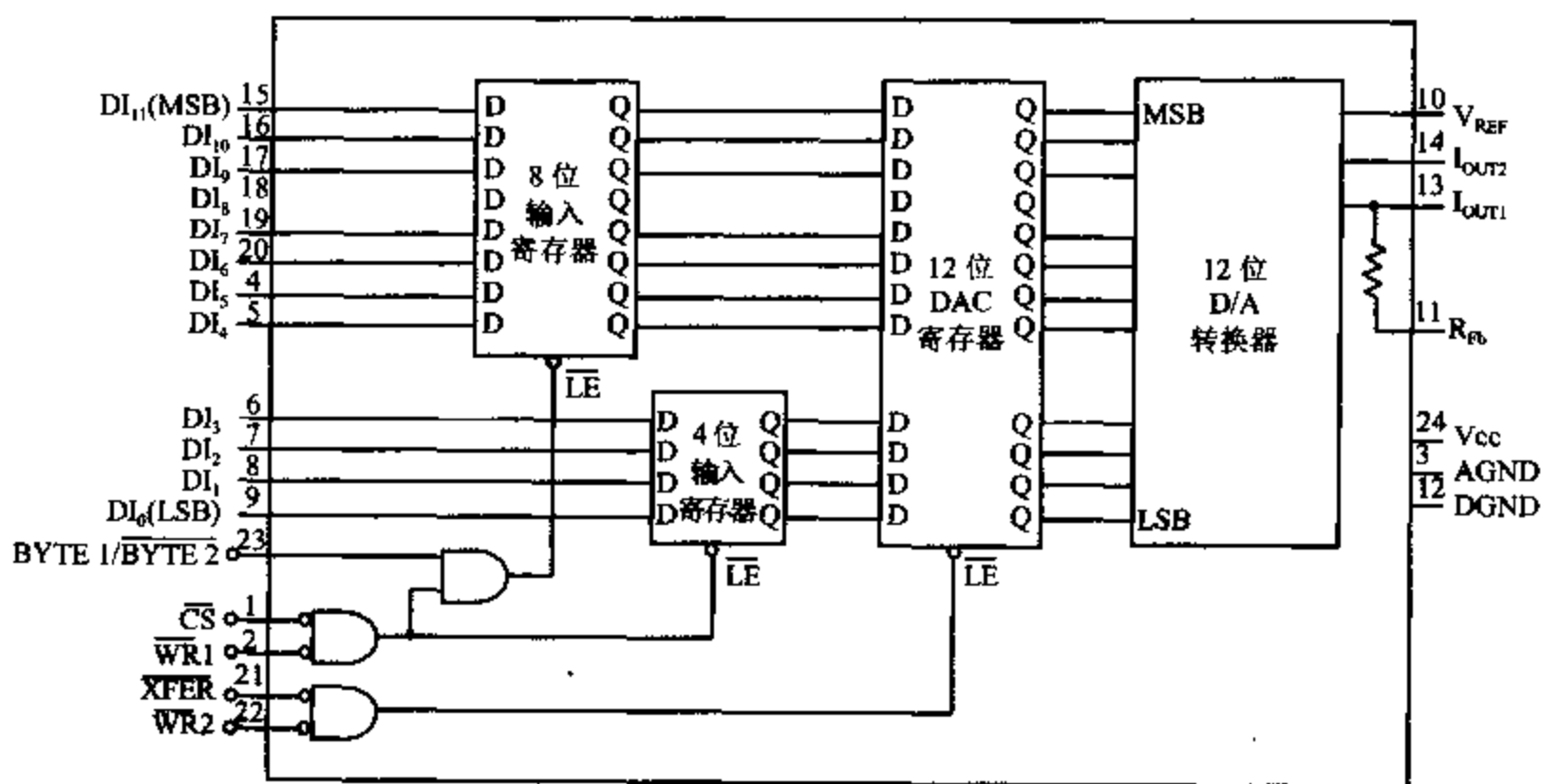


图 11-17 DAC1210 的逻辑图

DAC1210 的输入寄存器由 8 位和 4 位两个寄存器构成, 它们都在 $\overline{CS}$ 、 $\overline{WR}_1$ 为低电平时才允许输入数据, 而 8 位寄存器还要求 $\overline{BYTE1}/\overline{BYTE2}$ ($B_1/\overline{B}_2$) 端为高电平才能输入。它有 $DI_{11} \sim DI_4$ 和 $DI_3 \sim DI_0$ 共 12 根数据输入线, 可直接与 16 位总线的 CPU 相连, 也可以与 8 位数据总线的 CPU 接口。

图 11-18 是将 DAC1210 接到具有 8 位数据总线的微处理器的方案。 $DI_{11} \sim DI_4$ 与数据总线相连, 而 $DI_3 \sim DI_0$ 并接到数据总线的高 4 位上, $\overline{WR}_1$ 和 $\overline{WR}_2$ 与系统总线的 $\overline{IOW}$ 相连, 12 位数据应分两次写入输入寄存器, 写入方法如下:

先执行 OUT 指令, 使 $\overline{CS}$ 和 $\overline{WR}$ 产生负脉冲, $B_1/\overline{B}_2$ 端为高, 写入高 8 位数据 $DI_{11} \sim DI_4$ 。这时, 因 $\overline{CS}$ 和 $\overline{WR}$ 的低电平, 使 12 位数据中的高 4 位也写进了 4 位输入寄存器。再执行第二条 OUT 指令, 使 $\overline{CS}$ 、 $\overline{WR}$ 变为负脉冲, 并使 $B_1/\overline{B}_2$ 变低, 写入低 4 位数据, 这时高 8 位输入寄存器被禁止, 4 位输入寄存器的内容被更新, 写入了所需的低 4 位值。第三步再对另一个端口执行 OUT 指令, 使 $\overline{XFER}$ 和 $\overline{WR}_2$ 有效, 将已存在两个输入寄存器里的 12 位数据一起写入 12 位 DAC 寄存器, 并启动 D/A 转换。经 $1\mu s$ 后, 在输出端便得到转换结果。

由上可知, 控制 DAC1210 的转换共要用到 3 个 I/O 端口, 假设这三个端口的地址为

220~222H, 它们由 I/O 端口地址译码电路形成。A₀ 经反相后连到 B₁/B₂ 端, 用于选择偶地址或奇地址, A₉~A₁ 经译码电路形成端口地址, 接 CS 引脚的口地址为 220H~221H, 偶地址(220H)时选通 8 位输入寄存器, 奇地址(221H)时选通 4 位输入寄存器; 接 XFER 引脚的口地址为 222~223H(图中用的是 222H), 选择其中的任一个地址都可启动 D/A 转换。若待转换的数字量在 BX 寄存器的低 12 位, 则完成一次 D/A 转换的程序如下:

```

START:  MOV    DX, 220H      ;指向 220H 端口
        MOV    CL, 4        ;移位次数
        SHL    BX, CL       ;BX 中数左移 4 次后向左对齐
        MOV    AL, BH       ;取高 8 位
        OUT    DX, AL       ;写入 8 位输入寄存器
        INC    DX           ;口地址=221H
        MOV    AL, BL       ;取低 4 位
        OUT    DX, AL       ;写入 4 位输入寄存器
        INC    DX           ;口地址=222H
        OUT    DX, AL       ;启动 D/A 转换, AL 中可为任意值
        ;

```

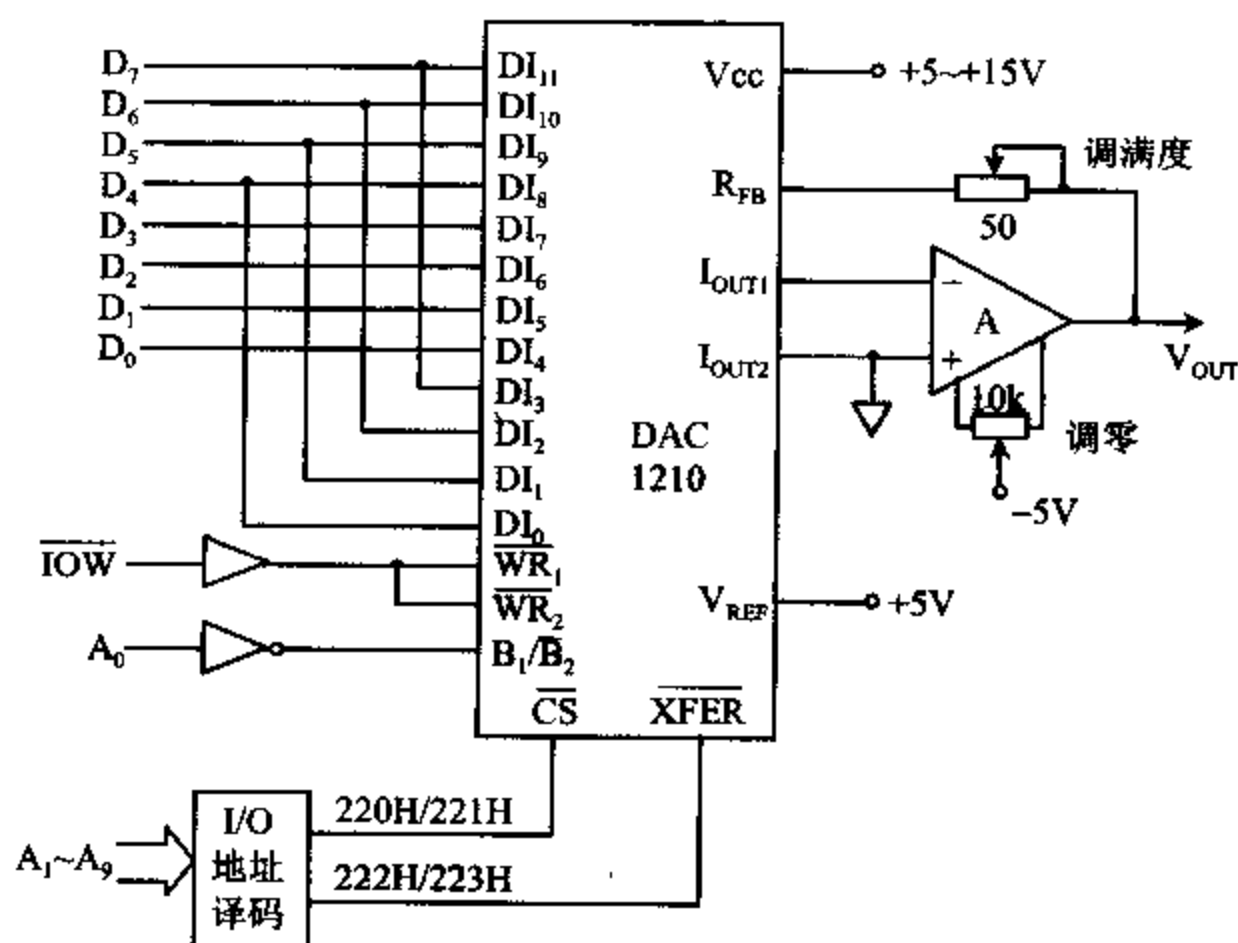


图 11-18 DAC1210 与 8 位数据总线微机连接方案

11-3 A/D 转换

一、模/数转换器原理

实现 A/D 转换的基本方法有十几种, 常用的有计数法、逐次逼近法、双斜积分法和并行

转换法。由于逐次逼近式 A/D 转换具有速度快,分辨率高等优点,而且采用这种方法的 ADC 芯片成本较低,因此在计算机数据采集系统中获得了广泛的应用。为此,我们仅介绍逐次逼近式 A/D 转换器的原理和它们的使用。

这类 A/D 转换器的转换原理是建立在逐次逼近的基础上的,即把输入电压 V_i 和一组从参考电压分层得到的量化电压进行比较,比较从最大的量化电压开始,由粗到细逐次进行,由每次比较的结果来确定相应的位是 1 还是 0。不断比较,不断逼近,直到两者的差别小于某一误差范围时即完成了一次转换。

这种逐次比较的过程与天平称量物体的过程很相似。若我们要用天平称量一个实际重量为 27.4 克的重物,天平具有 32 克、16 克、8 克、4 克、2 克和 1 克等 6 种砝码。称量时,先从最重的砝码试起,称量过程可用表 11-3 来说明。经过 6 步操作后,天平基本平衡,由于最小的砝码是 1 克,没有更小的砝码可用了,所以称量已告结束。结果为:

$$\begin{aligned} M_x &= 0 \times 32 + 1 \times 16 + 1 \times 8 + 0 \times 4 + 1 \times 2 + 1 \times 1 \\ &= 27 \text{ 克} \end{aligned}$$

表 11-3 一个 27.4 克重物的称量过程

次序	加砝码	天平指示	操作	记录
1	32 克	超重	去码	$X_1=0$
2	16 克	欠重	留码	$X_2=1$
3	8 克	欠重	留码	$X_3=1$
4	4 克	超重	去码	$X_4=0$
5	2 克	欠重	留码	$X_5=1$
6	1 克	平衡	留码	$X_6=1$

它与实际重量之间的误差为 0.4 克。由于砝码是以二进制加权分布的,因此也可以用二进制码 $d_1 d_2 d_3 d_4 d_5 d_6 = 011011$ 来表示该物体的重量。

如果再增加 0.5 克、0.25 克两种砝码,将使称量结果更精确。这时,相当于 $n=8$,即用 8 位二进制 01101101 来表示称量结果,也就是 27.25 克。

逐次逼近 A/D 转换器就像一架电子自动平衡天平。以一个量程为 +5V 的 4 位逐次逼近式 ADC 为例,用它来转换一个 $V_i=3V$ 的电压量,由于 $n=4$,它有 4 个以二进制码表示的电子砝码,它们与电压量的对应关系如表 11-4 所示。

表 11-4 4 个电子砝码与电压的对应关系

代码	相应的电压
1000	$5V \times 2^{-1} = 2.5V$
0100	$5V \times 2^{-2} = 1.25V$
0010	$5V \times 2^{-3} = 0.625V$
0001	$5V \times 2^{-4} = 0.3125V$

逐次逼近式 A/D 转换器的原理如图 11-19 所示,它由逐次逼近寄存器 SAR、D/A 转换器、比较器 A、缓冲器等组成。SAR 中包含一个移位寄存器、一个数据寄存器及决定去/留码的逻辑电路等几部分,它们在时钟脉冲 CLK 的作用下有序地进行操作。D/A 转换器用来形成电子砝码,送到比较器 A 的“—”输入端。比较器相当于天平的杠杆和指针,它对从

“+”端输入的模拟电压 V_i 和从“-”端输入的电子砵码进行比较,如 V_i 大于所加的砵码,输出为 1,SAR 中的去/留码逻辑决定保留这个砵码,否则就去除这个砵码。

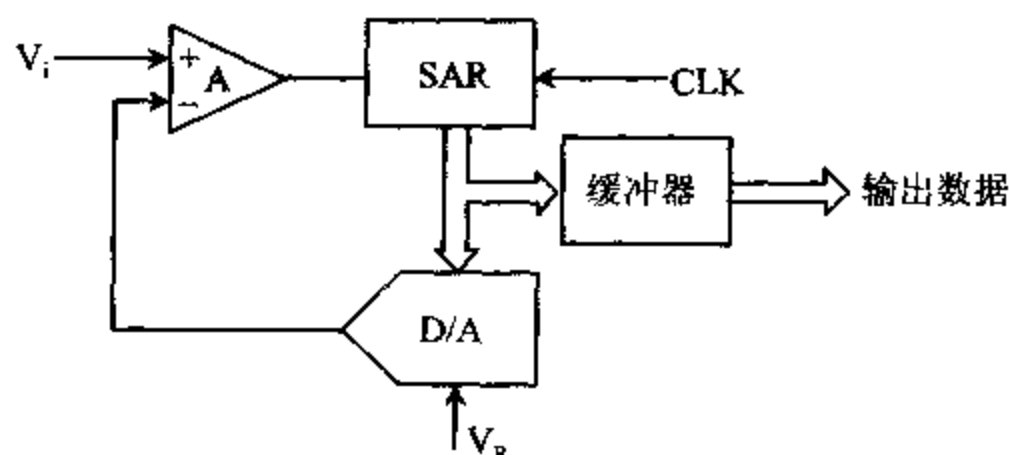


图 11-19 逐次逼近式 A/D 转换器的原理

对于 $V_i = 3V, n = 4$ 的情况,转换过程如下:

在时钟驱动下,SAR 中的移位寄存器的 MSB 位加码,其编码为 1000。D/A 转换器将它转换成 2.5V 电压,送到比较器 A 的“-”输入端,与 V_i 进行比较,由于 $2.5V < 3V$,所以去留逻辑保留最高位的 1,即这次比较结果为 1。

移位寄存器对第二位加码,由于上次比较后最高位保留了 1,因此送到 DAC 的代码为 1100,DAC 的输出电压就是 $2.5V + 1.25V = 3.75V$,它与 3V 进行比较。由于 $3.75V > 3V$,所以要去掉这位,本次结果为 0。

同理可对第三、第四位进行加码,整个比较过程如表 11-5 所示。

表 11-5 4 位逐次逼近式 A/D 转换过程
($V_R = +5V, V_i = 3.0V$)

次序	试探码	D/A 输出	去留码	本次结果
1	1000	$2.5V < V_i$	留	1000
2	1100	$3.75V > V_i$	去	1000
3	1010	$3.125V > V_i$	去	1000
4	1001	$2.8125V < V_i$	留	1001

经过 4 次比较后,比较过程结束,最后在 SAR 的数据寄存器中的结果是 1001,它就是 $V_i = 3V$ 所对应的数字量。它通过缓冲器输出,表示的实际电压为 2.8125V。这个数字量与输入电压之间的误差为 $2.8125V - 3V = -0.1875V$,由于该 A/D 转换器的量化单位 (1LSB) 为 0.3125V,这时的量化误差已小于 1LSB。

若要提高精度,可再增加几位,相当于再用更小的电子砵码进行比较。如将上述的 A/D 转换器增加到 8 位,相当于再增加 4 个电子砵码: 0.15625V、0.078125V、0.0390625V 和 0.01953125V。仿照上述步骤,3V 输入电压可转换成二进制码 1001 1001,它表示的实际电压为 2.98828125V,它和 3V 之间的误差为 0.01171875V,小于量化单位 0.0195321V。可见,增加位数后转换精度明显提高了。

逐次逼近式 A/D 转换器每进行一次比较,即决定数字码中的一位码的去留操作,需要 8 个时钟脉冲。这样,一个 8 位转换器完成一次转换需要 $8 \times 8 = 64$ 个时钟脉冲,再加上准备与结束阶段需要几个时钟脉冲,这就是转换器的转换时间 t_c 。大致认为经 64 个时钟脉冲

后,8次比较完成,通过缓冲器可输出数字量结果。

大部分 A/D 转换器的时钟是由外部提供的,也有一些片子可用外接的 RC 网络来设定时钟频率。很容易根据时钟频率来估计出转换器的转换时间。例如,ADC0804 和 ADC0809 均是 8 位逐次逼近式 A/D 转换器,典型的工作时钟频率为 640kHz,每个时钟脉冲的周期为 $1/(640 \times 10^3)$ 秒。于是,完成一次转换的时间大约为:

$$t_c = 64 \times \frac{1}{640 \times 10^3} \text{秒} = 0.0001 \text{秒} = 100 \mu\text{s}$$

如工作频率 $f = 500\text{kHz}$,则 $t_c = 128 \mu\text{s}$ 。

与 DAC 那样,ADC 也有若干性能指标,如分辨率、精度、转换时间、孔径时间、输入电压范围、输出数据格式、参考电压范围等,由于 A/D 转换器和 D/A 转换器具有互逆的关系,在理解了 D/A 转换器的性能指标的基础上,不难掌握 A/D 转换器的性能指标的含义,从而能根据需求选择合适的器件。

二、典型的模/数转换器

1. ADC0809

为便于用户构成多通道数据采集系统,一些厂家将多路模拟开关和 8 位 A/D 转换器集成在一个芯片内,构成多通道 ADC,其中以 NSC 公司的 8 通道 8 位 A/D 转换器 ADC0809 应用最为广泛。下面介绍它的基本原理和使用方法。

(1) 引脚

ADC0809 的引脚排列如图 11-20 所示,从功能上看,它可看成是由 ADC0804 和一个 8 通道模拟多路开关组合而成的。

各引脚的功能如下:

IN₇~IN₀ 8 通道模拟量输入端。

D₇~D₀ 结果数据输出端。其中 D₇ 为最高有效位 MSB, D₀ 为最低有效位 LSB。

START 启动转换命令输入端。在该引脚上加高电平,即开始转换。

EOC 转换结束指示脚。平时它为高电平,在转换开始后及转换过程中为低电平,转换一结束,它又变回高电平。

OE 输出使能端。此脚上加高电平,即打开输出缓冲器三态门,读出数据。

C、B 和 A 通道号选择输入端。其中 A 是 LSB 位,这三个引脚上所加电平的编码为 000~111 时,分别对应于选通通道 IN₀~IN₇。例如,当 C、B 和 A 为 100 时,选中通道 IN₄, 011 时选中通道 IN₃。(这三个引脚信号也被记作 ADD C、ADD B 和 ADD A)

ALE 通道号锁存控制端。当它为高电平时,将 C、B 和 A 三个输入引脚上的通道号选择码锁存,也就是使相应通道的模拟开关处于闭合状态。实际使用时,常把 ALE 和 START 连在一起,在 START 端加高电平启动信号的同时,将通道号锁存起来。

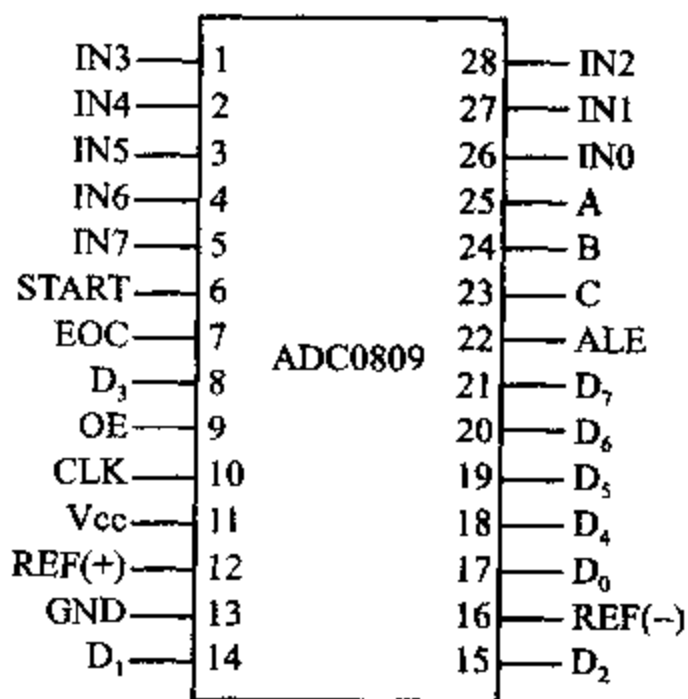


图 11-20 ADC0809 引脚图

CLK ADC0809 需要外接时钟,可从此脚接入。当 $V_{CC}=+5V$ 时,允许的最高时钟频率是 1280kHz,这时可达到 $t_c=50\mu s$ 的最快的转换速率。ADC0809 典型的时钟频率为 640kHz,转换时间是 $100\mu s$ 。

REF(+),REF(-) 两个参考电压输入脚。通常将 REF(-) 接模拟地,参考电压从 REF(+) 引入。当 $REF(+)=+5V$ 时,输入范围为 $0\sim+5V$ 。

(2) 工作过程

图 11-21 是 ADC0809 的定时图。对指定的通道采集一个数据的过程如下:

- ① 选择当前转换的通道,即将通道号编码送到 C、B 和 A 引脚上。
- ② 在 START 和 ALE 脚上加一个正脉冲,将通道选择码锁存并启动 A/D 转换。可以通过执行 OUT 指令产生负脉冲,经反相后形成正脉冲,也可由定时电路或可编程定时器提供启动脉冲。
- ③ 转换开始后,EOC 变低,经过 64 个时钟周期后,转换结束,EOC 变高。
- ④ 转换结束后,可通过执行 IN 指令,设法在 OE 脚上形成一个高电平脉冲,打开输出缓冲器的三态门,让转换后的数字量出现在数据总线上,并被读入累加器中。

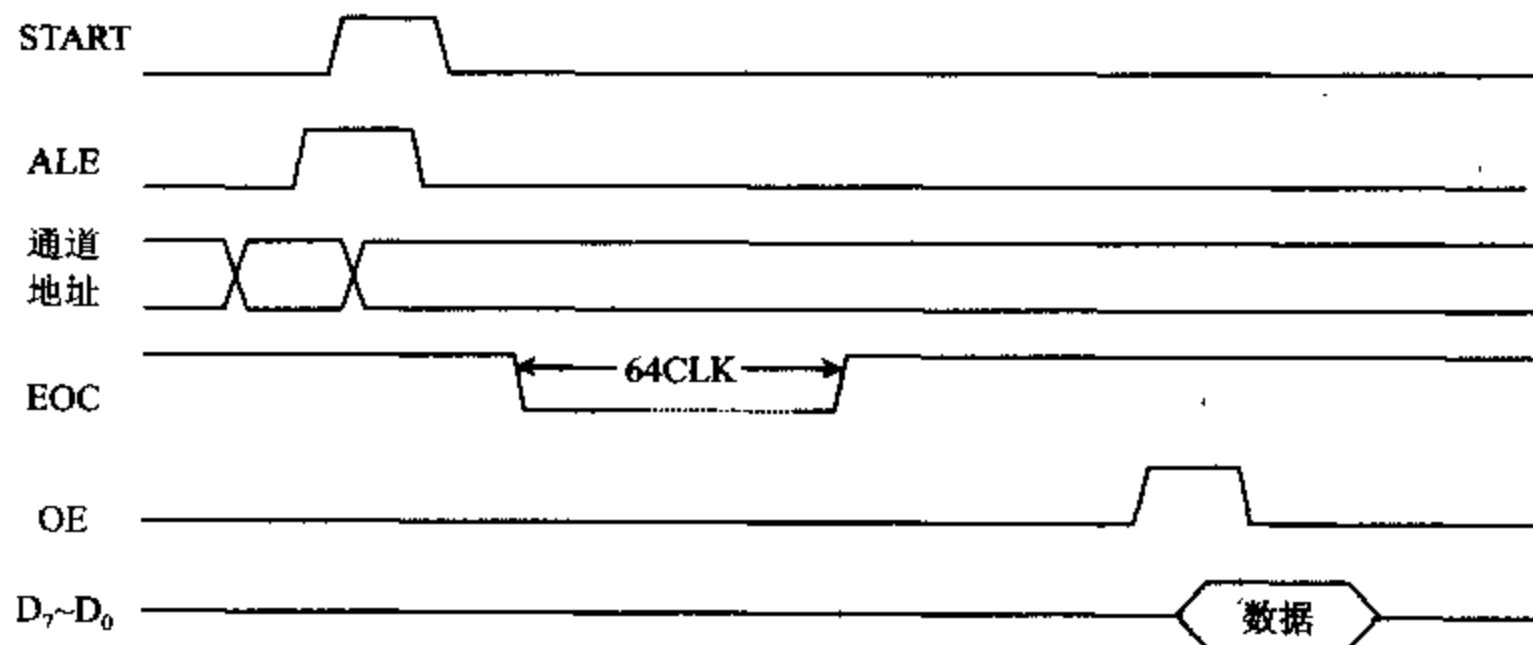


图 11-21 ADC0809 的定时图

用 ADC0809 来设计实用的数据采集系统时,除了要考虑采样率的控制和转换结束的检测方法外,还要设计合适的通道选择方案。例如,可用软件延时、定时中断或周期脉冲控制采样率,以及用延时程序、查询 EOC 电平或用 EOC 的正跳变请求中断来判断某个通道转换的结束。而向 ADC0809 提供通道号的方法也有好几种。例如,可先从数据总线送出通道号,用一个锁存器将它们锁存在 C、B 和 A 接脚上后,再启动转换。也可以在 I/O 地址译码时,不让 $A_2\sim A_0$ 参加译码,而将它们连到 C、B 和 A 端,当执行 OUT 指令启动各通道的转换时,同时将包含在端口地址中的通道号送给 ADC0809。

假设 8 个通道均接有模拟输入信号,可以编写一个循环程序,从通道 0 开始,依次启动各通道转换并读取数据,通常将这样的操作称为进行一遍扫描。多通道 ADC 工作时,必须等一个通道转换结束后,才能启动另一个通道转换,因此扫描一遍,也就是每个通道都采集一个数据,至少需要 8 倍的转换时间,这就限制了多通道 ADC 的最高采样率。在同样的时钟频率下,8 通道均使用时,ADC0809 的最高采样率便是单通道时的 $1/8$ 。

(3) 多通道数据采集方案

① 用定时中断控制采样率,用地址信号选择通道的方案

我们可以选用 8253 芯片来产生定时脉冲,控制采样率。假设加到 8253 的 CLK_0 的时钟脉冲的频率为 1MHz,编程使通道 0 工作于方式 2,由于采样率 $f_s=200Hz$,当选用时间常数为 $1MHz/200Hz=5000$ 使 8253 工作时,则可从 OUT_0 端输出 200Hz 的负脉冲序列,即每隔 5ms 会从 8253 的 OUT_0 引脚输出一个正跳变脉冲,该脉冲加到 PC 机上为用户保留的 IRQ_2 中断请求输入端,即加到系统板上 8259A 的 IR_2 引脚上,在 8259A 的控制下定时向 CPU 发中断请求,在每次中断时进行采样。在每次中断出现后,在中断服务程序中用 OUT 指令启动转换,然后查询 EOC 引脚的状态,当 EOC 为高时,表示转换结束,这时可用 IN 指令读入结果。轮流启动各通道的转换并读取数据,存入数据缓冲区,就完成一次扫描。这样可获得精确的采样间隔。

图 11-22 用 ADC0809 设计的多路数据采集电路

406

平,若为低表示已开始转换,再查 EOC 是否变高,若高说明转换已结束,就可用 IN 指令读取结果。PC/XT 机中 8259A 的口地址为 20H 和 21H,设数据采集卡上 8253 的通道 0 和控制寄存器的口地址分别为 318H 和 31BH。完成上述功能的程序如下:

```

DATA    SEGMENT                                ;数据段
DBUF    DB  8 * 1024  DUP(?)                    ;数据区(8×1024 字节)
DATA    ENDS
...
;-----
;数据采集子程序
;-----

CODE    SEGMENT                                ;代码段
        ASSUME  CS, CODE, DS, DATA
AD_8    PROC    FAR
        MOV     AX, DATA
        MOV     DS, AX                          ;DS 指向数据区段址
        CLI                                           ;禁止中断
        CLD                                           ;清方向标志
        ;设置 0AH 号中断矢量的段地址和偏移量,使 ES:DI=0000;(4 * 0AH)
        MOV     AX, 0
        MOV     ES, AX                          ;ES 指向中断矢量表段址 0000
        MOV     DI, 4 * 0AH                    ;DI=中断 IR2 的偏移地址
        MOV     AX, OFFSET ADINT               ;AX=中断服务子程序偏移地址
        STOSW                                     ;放入中断矢量表
        MOV     AX, SEG ADINT                  ;取中断矢量段地址
        STOSW                                     ;放入中断矢量表中
        ;对 8253 进行初始化编程,使通道 0 的控制字为:方式 2,先读写低字节,后高字
        ;节,BCD 计数;定时时间常数为 5000。
        MOV     DX, 31BH                      ;DX 指向 8253 控制寄存器
        MOV     AL, 00110101B                 ;通道 0 控制字
        OUT     DX, AL                        ;输出控制字
        MOV     DX, 318H                      ;DX 指向 8253 通道 0
        MOV     AX, 5000H                     ;时间常数
        OUT     DX, AL                        ;先送低 8 位
        MOV     AL, AH
        OUT     DX, AL                        ;后送高 8 位
        ;对 8259A 设置屏蔽字,仅允许 8259A 的 IR2 和键盘中断,其余禁止
        MOV     AL, 11111001B                 ;屏蔽字
        OUT     21H, AL                       ;向屏蔽寄存器输出屏蔽字
        ;设置数据缓冲区始址到 SI 中,计数初值到 BX 中,等待中断,每通道采完 1024 个数后结束中断
        MOV     SI,  OFFSET DBUF              ;SI 指向数据缓冲区始址
        MOV     BX, 1024                      ;BX 中存数据计数器初值
        STI                                     ;开中断,等待中断

```

```

AGAIN:  CMP    BX, 0           ;每中断一次 BX-1, BX 减 1 后为 0?
        JNZ     AGAIN         ;BX≠0, 未采完, 循环等待中断
        MOV     AL, 20H        ;采完, 送中断结束命令 EOI→AL
        OUT     20H, AL        ;输出中断结束命令到 8259A
        MOV     AH, 4CH        ;退出中断
        INT     21H
        RET                     ;从子程序返回
AD-8    ENDP                  ;AD-8 过程结束
;-----
;中断服务程序, 对每个通道均采集一个数据, 存进 DBUF
;-----
ADINT    PROC    NEAR
        MOV     CX, 0008H      ;设置通道计数器初值
        MOV     DX, 300H      ;DX 指向 ADC 通道 0
NEXT:    OUT     DX, AL        ;启动一次转换
        PUSH    DX            ;保存通道号
        MOV     DX, 308H      ;DX 指向状态口 308H
POLL:    IN      AL, DX        ;读入 EOC 状态
        TEST    AL, 80H       ;EOC(D7)=0? 即开始转换了?
        JNZ     POLL          ;非 0, 循环等待
NO-END   IN      AL, DX        ;EOC=0, 已开始转换
        TEST    AL, 80H       ;再查 EOC 是否为 1
        JZ      NO-END        ;EOC=0, 等待转换结束
        POP     DX            ;EOC=1, 恢复通道地址
        IN      AL, DX        ;读取结果
        MOV     [SI], AL      ;存储到缓冲区中
        INC     DX            ;DX 指向下一个通道
        INC     SI            ;地址指针指向下一个缓存单元
        LOOP    NEXT          ;通道计数器减 1, 结果非 0 则循环
        DEC     BX            ;为 0, 缓冲数据计数器减 1
        STI                     ;开放中断
        IRET                    ;自中断返回
ADINT    ENDP
CODE     ENDS
END

```

如果将上述电路中的 EOC 引脚悬空, 也可以在启动每个通道的转换后, 调用一个延时程序来等待转换结束, 然后读取数据。如前所述, 延时量应大于所用时钟频率下 ADC 的转换时间, 在本例中, ADC 的工作频率为 500kHz, 转换时间为 120 μ s, 所以有 150 μ s 的延时已足够了。

从原理上讲, 也可以用表示转换结束的 EOC 输出去请求 CPU 中断, 由中断服务程序读取数据, 这样能更充分地利用 CPU 的时间。但实际工作中很少有人这样做, 因为这会形成两个中断源, 而 PC 机上能供用户使用的中断很有限。采样率是数据采集系统中的最主要矛

盾, 所以总采用中断的办法来控制它, 而转换结束的检测就比较灵活, 软件延时方法能节省一个状态口, 因此也经常使用。

② 用 8255A 控制 ADC0809 的方法

如果系统中有 8255A, 用它来设计数据采集系统, 能较方便地将 ADC 接口到 CPU 总线, 并采用查询法检测转换结束标志。图 11-23 是采用 8255A 控制 ADC0809 的一种方案。

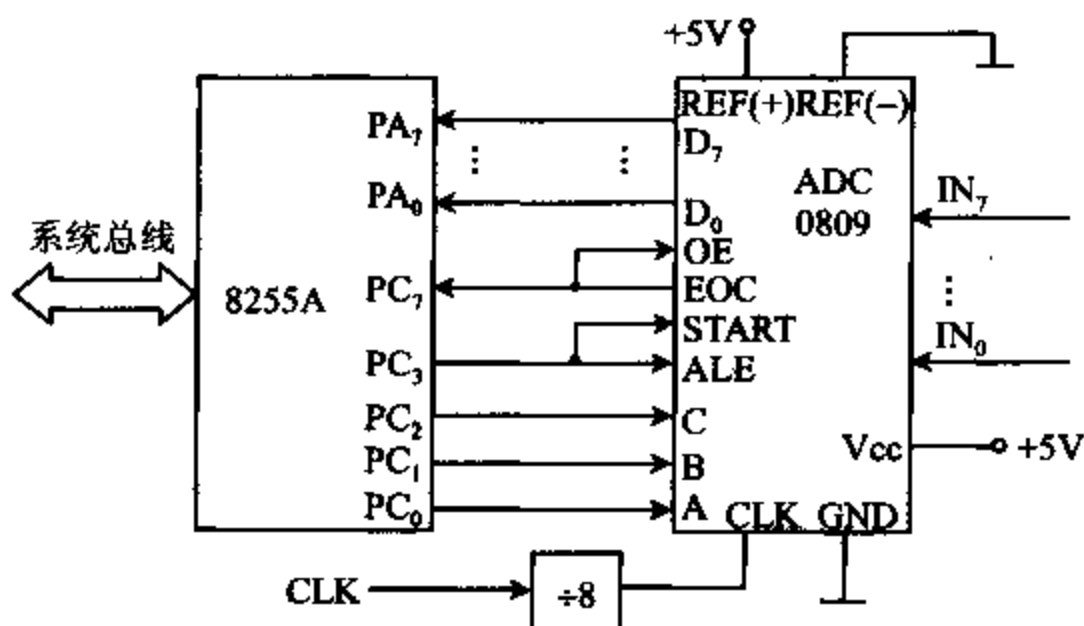


图 11-23 用 8255A 控制 ADC0809 的电路图

图中, ADC 的输出接在 8255A 的 A 口, 将 A 口编程为方式 0, 输入方式。C 口的高 4 位编程为输入, 低 4 位为输出。ADC 的 START 和 ALE 与 PC₃ 相连, 由 CPU 控制 PC₃ 发启动信号和通道号锁存信号, PC₃~PC₀ 输出 3 位通道号地址信号。EOC 输出信号和 PC₇ 相连, CPU 通过查询 PC₇ 的状态, 控制数据的输入过程。在启动脉冲结束后, 先要查到 EOC 为低电平, 表示转换已开始。然后继续查询, 当发现 EOC 变高, 便说明转换已结束。由于 EOC 还和输出使能端 OE 相连, 转换结束时 OE 也变高, 使 ADC 的输出缓冲器打开, 数据出现在 A 口上, 可由 IN 指令读入 CPU。

假设系统分配给 8255A 的端口地址为 320H~323H。又设, 已完成对 8255A 的初始化编程, 并使 ES 和 DS 有相同的段基地址。若要求 ADC0809 将 8 路模拟量转换成 8 个数字量后, 存放到内存中段基地址为 ES, 偏移量从 DATA-BUF 开始的存储单元中, 则用 ADC0809 完成一次 8 路模拟量的采集子程序 AD-SUB 如下:

AD-SUB	PROC	NEAR	
	MOV	CX, 8	;CX 作数据计数器
	CLD		;清方向标志
	MOV	BL, 00H	;模拟通道号存在 BL 中
	LEA	DI, DATA-BUF	;缓冲区偏移地址
NEXT-IN:	MOV	DX, 322H	;C 口地址
	MOV	AL, BL	
	OUT	DX, AL	;输出通道号
	MOV	DX, 323H	;指向控制口
	MOV	AL, 00000111B	;PC3 置 1
	OUT	DX, AL	;送出开始启动信号

	NOP		; 延时
	NOP		
	NOP		
	MOV	AL, 00000110B	; PC3 复位
	OUT	DX, AL	; 送出结束启动信号
	MOV	DX, 322H	; DX 指向 C 口
NO-CONV;	IN	AL, DX	; 读入 C 口内容
	TEST	AL, 80H	; 查 PC7, 即 EOC 信号
	JNZ	NO-CONV	; PC7=1, 还未开始转换, 等待
NO-EOC;	IN	AL, DX	; PC7=0, 已启动转换
	TEST	AL, 80H	; 再查 PC7
	JZ	NO-EOC	; PC7=0, 转换未结束, 等待
	MOV	DX, 320H	; PC7=1, 转换结束, DX 指向 A 口
	IN	AL, DX	; 读入数据
	STOS	DATA-BUF	; 存入 ES 段的数据缓冲区
	INC	BL	; 指向下个通道
	LOOP	NEXT-IN	; 尚未完成 8 路转换则循环
	RET		; 已完成, 返回
AD-SUB	ENDP		

本例中没有指定采样率, 用于实际系统上时, 可以根据要求的采样速率, 参考上面几个例子中介绍的方法, 用定时中断来控制采样率。这时, 子程序 AD-SUB 的内容必须放入中断服务程序。

2. 12 位 A/D 转换器 AD574A

AD574A 是一种带有三态缓冲器的 A/D 转换器, 可以直接与 8 位或 16 位微机总线接口, 芯片内有高精度的参考电压源和时钟电路, 所以不需要外接时钟和参考电压等电路就可以正常工作。此外, 芯片内还含有逐次逼近式寄存器 SAR、比较器、控制逻辑、DAC 转换电路及三态输出缓冲器等。

(1) AD574A 的引脚

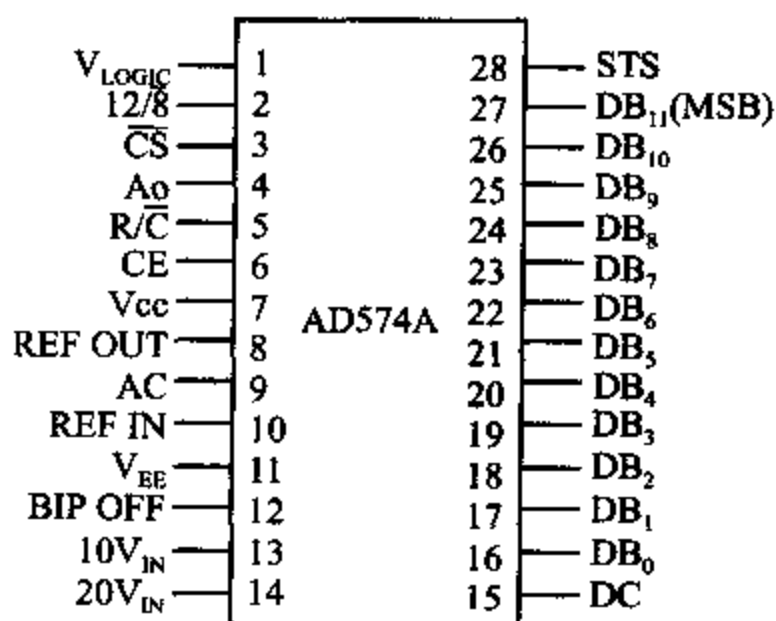


图 11-24 AD574A 的引脚图

AD574 的引脚排列如图 11-24 所示, 下面介绍各引脚的功能。

① 电源和地

- V_{CC} +12V 或 +15V 电源, 输入。
- V_{EE} -12V 或 -15V 电源, 输入。
- V_{LOGIC} 逻辑电源, 接 +5V。
- REF OUT 输出 10V 基准电压。
- REF IN 参考电压输入引脚。
- AC 模拟地 (Analog Common)。
- DC 数字地 (Digital Common)。

② 模拟量输入

- $10V_{IN}$ 量程为 0 ~ +10V 的单极性

输入端(10V Span Input)。

- $20V_{IN}$ 量程为 $0 \sim +20V$ 的单极性输入端(20V Span Input)。
- BIP OFF 双极性偏置输入端(Bipolar Offset), 量程为 $-5V \sim +5V$ 。

③ 数据输出引脚

- $DB_{11} \sim DB_0$ 共 12 条输出引线, 其中 DB_{11} 为最高有效位, DB_0 为最低有效位。

④ 控制和状态信号

- CE 芯片使能引脚, 高电平有效。
- $\overline{CS}$ 片选信号, 低电平有效。
- $R/\overline{C}$ 读/转换控制信号(Read/Convert), 高电平时为读, 低电平时为转换信号。

以上信号中, 只有当 CE 和 $\overline{CS}$ 同时有效时, AD574A 才工作(启动转换或读出转换结果), 这时, 如果 $R/\overline{C}$ 为低电平, 则启动 AD574 进行 A/D 转换; 如果 $R/\overline{C}$ 为高电平时, 则可从 $DB_{11} \sim DB_0$ 读出数字量。

- $12/\overline{8}$ 数据模式选择端(Data Mode Select)。当它为高电平时, 从 $DB_{11} \sim DB_0$ 输出 12 位数据, 低电平时, 12 位数据要分两次输出。

- A_0 字节地址短周期信号(Byte Address Short Cycle), 用于选择转换数据的长度。转换开始后, A_0 为低电平, 使 AD574A 初始化为 12 位转换, 高电平则仅产生 8 位短周期转换。在读出操作时, $12/\overline{8}$ 为低电平时, A_0 用于选择读出三态输出缓冲器中的高 8 位($A_0 = 0$)还是低 4 位($A_0 = 1$)数据; 若 $12/\overline{8}$ 为高电平, 则 A_0 不起作用。

- STS 状态输出信号, 它用于指示转换的状态。当它为高电平时表示正在转换, 低电平时表示转换已结束。

各控制信号的真值表如表 11-6 所示。

表 11-6 控制信号真值表

CE	$\overline{CS}$	$R/\overline{C}$	$12/\overline{8}$	A_0	操 作
0	×	×	×	×	无作用
×	1	×	×	×	无作用
1	0	0	×	0	启动 12 位转换
1	0	0	×	1	启动 8 位转换
1	0	1	V_{LOGIC}	×	并行输出 12 位数据
1	0	1	DC	0	输出高 8 位数据
1	0	1	AC	1	输出低 4 位并附加 4 个 0

(2) 单极性和双极性输入

AD574 工作于单极性和双极性输入的连线图分别如图 11-25(a)和(b)所示。

对于单极性输入方式, 当输入范围在 $0 \sim +10V$ 时, 从 $10V_{IN}$ 引脚输入, 当信号电压范围在 $0 \sim +20V$ 时, 则从 $20V_{IN}$ 引脚输入。图中 $100k$ 的电位器 R_1 用于零调整, 当模拟量输入为 $0V$ 时, 12 位输出数字量应为 0, 若不是全 0, 则需调整调零电位器。 100Ω 的电位器 R_2 用于满量程调整, 当模拟量输入为最大值($10V$ 或 $20V$)时, 12 位输出数字量为全 1, 若不是全 1, 则调整电位器 R_2 。对于双极性输入, 同样, R_1 和 R_2 分别用于零调整和满量程调整, 但 R_1

和 R_2 的阻均为 100Ω , 满量程输入电压范围为 $\pm 5V$ 或 $\pm 10V$ 。

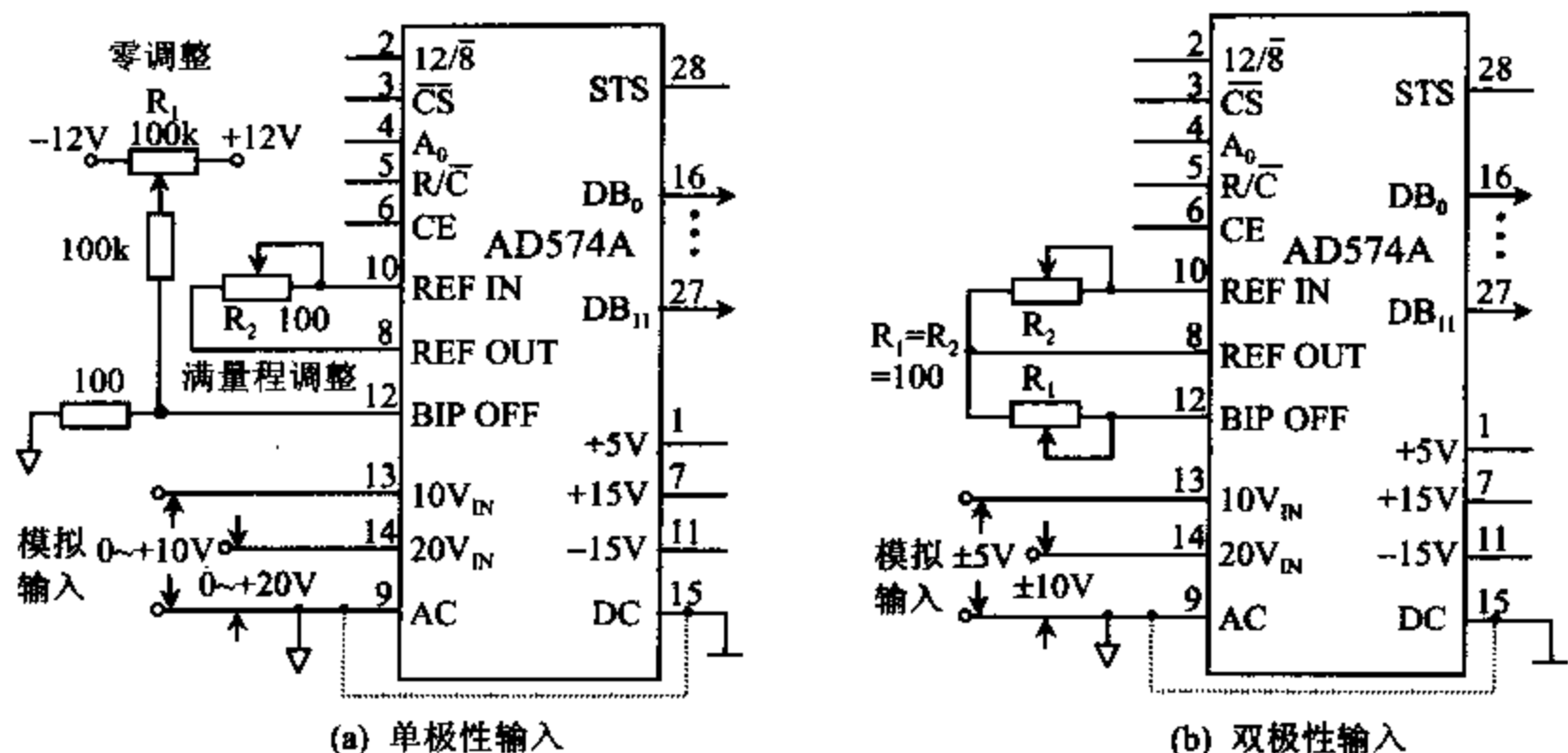


图 11-25 AD574A 单极性和双极性连线图

(3) 工作时序

AD574A 很容易与 8 位或 16 位微机及其它数字系统接口。工作时序如图 11-26 所示。

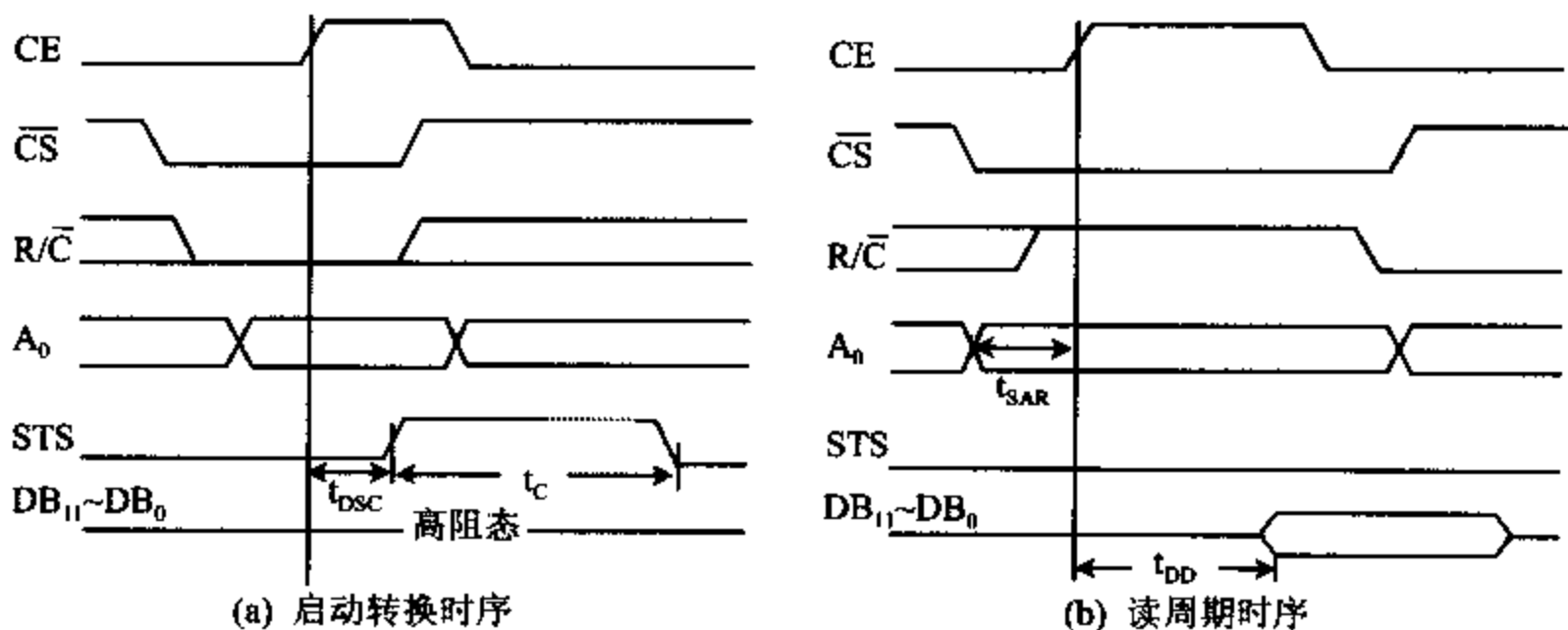


图 11-26 启动 AD574A 工作的时序

启动转换的时序如图 11-26(a)所示。启动 AD574A 开始转换时, 必须使 CE 和 $\overline{CS}$ 同时有效, 而且要求 $R/\overline{C}$ 为低电平, 如果这时 $R/\overline{C}$ 为高电平, 会立即执行读操作, 可能导致总线冲突。CE 和 $\overline{CS}$ 中哪个后变为有效, 就用哪个作启动信号, 通常建议使用 CE 作启动信号。启动后最多经 $400ns(t_{DSC})$ 后, 状态信号 STS 变高, 指示转换开始, 再经 t_c 时间后转换结束, STS 信号变低。 t_c 称为转换时间, 对于 8 位转换, t_c 最大为 $24\mu s$, 对于 12 位转换, t_c 最大为 $35\mu s$ 。

读周期的时序如图 11-26(b)所示。它要求 CE 和 $\overline{CS}$ 同时有效, 而且 $R/\overline{C}$ 为高电平时开始进行读操作, $R/\overline{C}$ 必须在 CE 和 $\overline{CS}$ 同时有效前至少提前 $150ns(t_{SAR})$ 就变高。图中用 CE 启动读操作, 如用 $\overline{CS}$ 启动读操作, 存取时间将扩展 $100ns$ 。读操作开始后最多经 $200ns$

(t_{10}), 转换后的结果就出现在 12 位数据引出线 $DB_{11} \sim DB_0$ 上, 并保留一定时间, 供 CPU 读取。

(4) AD574A 应用举例

若 AD574A 工作于 12 位转换方式, 输入电压范围为 $0 \sim +10V$, 采用单极性输入, 连线如图 11-27 所示。图中用 8255A 作接口芯片。模拟信号经放大后从 AD574A 的 $10V_{IN}$ 引脚输入, $12/\bar{8}$ 与 $+5V$ 相连, 调零和满量程调整电路与前面介绍的单极性方式相同。AC 和 DC 分别与模拟地和数字地相连, A_0 接地, AD574A 的 12 位输出引脚 $DB_{11} \sim DB_0$ 分别与 8255A 端口 A 的低 4 位、端口 B 的 8 位输入输出引线相连。

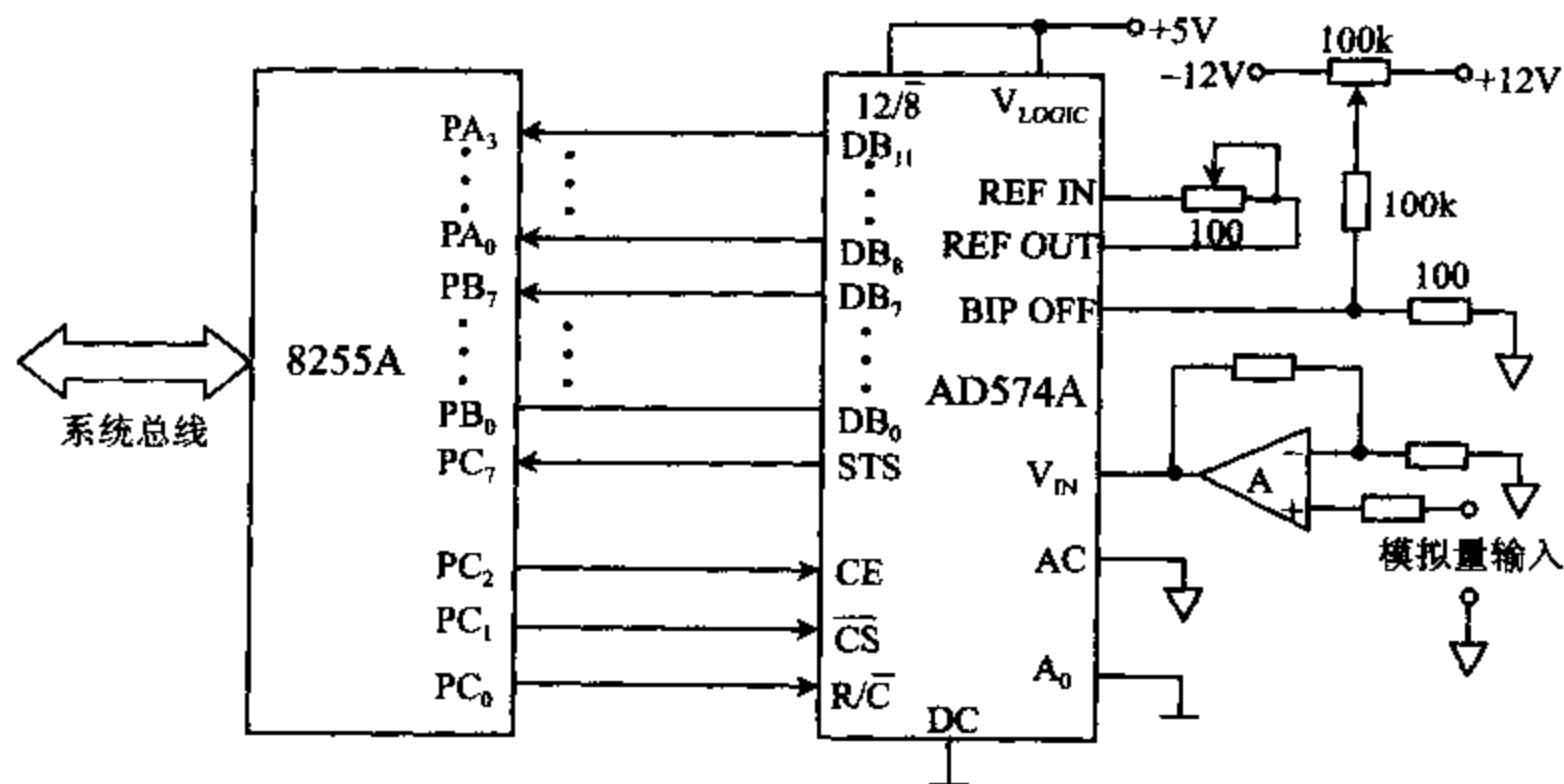


图 11-27 AD574A 的接口电路

对 8255A 编程时, A 口和 B 口都工作于方式 0, 输入, 它们用来读取转换后的 12 位数字量。C 口上半部分为输入, 用于读取状态信息; 下半部分为输出, 用来输出控制信号, 启动 AD574A 转换或发出读取结果数据的命令。设 8255A 的口地址为 $F0H \sim F3H$, 则启动 AD574A 转换和读取转换结果的程序段如下:

;8255A 的端口地址

POTR-A EQU 0F0H ;A 口地址

PORT-B EQU 0F1H ;B 口地址

PORT-C EQU 0F2H ;C 口地址

PORT-CTL EQU 0F3H ;控制口地址

;

;8255A 控制字: A 口和 B 口工作于方式 0, A 口、B 口和 C 口的上半部分为输入, C 口的下半部分为输出。

MOV AL, 10011010B ;方式字

OUT PORT-CTL, AL ;输出方式字

;启动 A/D 转换

MOV AL, 00H

OUT PORT-C, AL ;使 $\overline{CS}$ 、 $\overline{CE}$ 、 $\overline{R/C}$ 均为低

NOP		;延时
NOP		
MOV	AL, 04H	
OUT	PORT-C, AL	;使 CE=1,启动 A/D 转换
NOP		;延时
NOP		
MOV	AL, 03H	
OUT	PORT-C, AL	;使 CE=0, $\overline{CS}=R/\overline{C}=1$, 结束启动状态
READ_STS; IN	AL, PORT-C	;读 STS 状态
TEST	AL, 80H	;转换完(STS=0)了吗?
JNZ	READ_STS	;否,则循环等待
;转换完成,启动读操作		
MOV	AL, 01H	
OUT	POTR-C, AL	;使 $\overline{CS}=0$, CE=0, $R/\overline{C}=1$
NOP		
MOV	AL, 05H	;使 CE=1, $R/\overline{C}=1$, $\overline{CS}=0$
OUT	PORT-C, AL	;允许读出
;读取数据,存入 BX 中		
IN	AL, PORT-A	;读入高 4 位数据
AND	AL, 0FH	
MOV	BH, AL	;存入 BH
IN	AL, PORT-B	;读入低 8 位
MOV	BL, AL	;存入 BL
;结束读操作		
MOV	AL, 03H	;使 CE=0, $\overline{CS}=1$
OUT	PORT-C, AL	;结束读操作

习 题

1. 包含 A/D 和 D/A 的实时控制系统主要由哪几部分组成? 什么情况下要用多路开关? 什么时候要用采样保持器?
2. 什么叫采样、采样率、量化、量化单位? 12 位 D/A 转换器的分辨率是多少?
3. 某一 8 位 D/A 转换器的端口地址为 220H, 已知延时 20ms 的子程序为 DELAY_20MS, 参考电压为 +5V, 输出信号(电压值)送到示波器显示, 试编程产生如下波形:
 - (1) 下限为 0V, 上限为 +5V 的三角波。
 - (2) 下限为 1.2V, 上限为 4V 的梯形波。
4. 利用 DAC0832 产生锯齿波, 试画出硬件连线图, 并编写有关的程序。
5. (1) 画出 DAC1210 与 8 位数据总线的微处理器的硬件连接图, 若待转换的 12 位数字是存在 BUFF 开始的单元中, 试编写完成一次 D/A 转换的程序。
 - (2) 如果 DAC1210 与 8086 相连, 其余条件同上, 画出该硬件连线并编写 D/A 转换

程序。

6. 某系统利用 ADC0804 设计采样电路, 对一个 $0 \sim +5\text{V}$ 的模拟信号, 以 300Hz 的速率进行采样, 采得的数据存到内存缓冲器中, 存满 32K 字节后停止采样, 系统中用自时钟方式形成 500kHz 的工作脉冲, 用软件延时方法控制采样率, 要求:

(1) 设计采样电路。

(2) 编写数据采集程序。

7. 利用 8255A 和 ADC0809 等芯片设计 PC 机上的 A/D 转换卡, 设 8255A 的口地址为 $3\text{C}0\text{H} \sim 3\text{C}3\text{H}$, 要求对 8 个通道各采集 1 个数据, 存放到数据段中以 D_BUF 为始址的缓冲器中, 试完成以下工作:

(1) 画出硬件连线图

(2) 编写完成上述功能的程序。

8. 试利用 ADC0809、8253 和 8259A 等芯片设计 8 通道 A/D 转换电路。系统中用 8253 作定时器, 采用中断方式控制采样率, 采样率为 500Hz 。设 8253 的通道 0 输入时钟脉冲为 2MHz , 输出端 OUT_0 接 8259A 的 IR_2 , 8253 的口地址为 $300\text{H} \sim 303\text{H}$, 8259A 的口地址为 304H 和 305H , ADC0809 的 8 个输入通道的口地址为 $308\text{H} \sim 30\text{FH}$, 查询 EOC 信号和状态口地址为 306H , ADC0809 的输入时钟频率为 640kHz 。A/D 转换的结果依次存入数据段中以 BUFFER 为始址的内存中, 从通道 0 开始先存入各通道的第一个数据, 再存放第二个数据, 采集 10 秒钟后停止工作。要求:

(1) 画出硬件连线图

(2) 编写 8253、8259A 的初始化程序及采集 8 路模拟信号的中断服务程序。

9. 利用 8255A 和 AD574A 设计数据采集系统, 输入模拟电压为 $0 \sim +10\text{V}$, 若每秒采集 100 个数据, 转换后的数据字存放在 W_BUF 开始的缓冲器中, 低字节在前, 高字节在后, 采满 16K 字节的数据后停止工作, 要求:

(1) 画出硬件连线图。

(2) 编写启动 AD574A 工作和读取转换结果的子程序。

第十二章 8237A DMA 控制器及 PC/XT 机的系统板

利用 DMA 方式传送数据时,数据的传送过程完全由硬件控制,这种硬件电路称为 DMA 控制器,它具有以下基本功能:

- (1) 能向 CPU 提出 DMA 请求,请求信号加到 CPU 的 HOLD 引脚上。
- (2) CPU 响应 DMA 请求后,DMA 控制器从 CPU 那儿获得对总线的控制权。在整个 DMA 操作期间,由 DMA 控制器管理系统总线,控制数据传递,CPU 则暂停工作。
- (3) 能提供读/写存储器或 I/O 设备的各种控制命令。
- (4) 确定数据传输的起始地址和数据的长度,每传送一个数据,能自动修改地址,使地址增 1 或减 1,数据长度减 1。
- (5) 数据传送完毕,能发出结束 DMA 传送的信号。

CPU 在每一个非锁定时钟周期结束后,都要检测 HOLD 线上是否有 DAM 请求信号,若有,可转入 DMA 工作周期。

8237A 就是一种高性能的可编程 DMA 控制器,它内部有 4 个独立的通道,每个通道都具有 64K 地址和字节的计数能力,并具有 4 种不同的传送方式:单字节传送、数据块传送、请求传送和级联传送方式,通过级联,可以扩大通道数。对每个通道的 DMA 请求可以允许或禁止。4 个通道的 DAM 请求有不同的优先级,优先级可以是固定的,也可以是循环的。任一通道完成数据传送后,会产生过程结束信号 $\overline{EOP}$ (End of Process),同时结束 DMA 传送,还可以从外界输入 $\overline{EOP}$ 信号,中止正在执行的 DMA 传送。

8237A DMA 控制器可以处于两种不同的工作状态。在 DMA 控制器未取得总线控制权时,必须由 CPU 对 DMA 控制器进行编程,以确定通道的选择、数据传送的方式和类型、内存单元起始地址、地址是递增还是递减及要传送的总字节数等,CPU 也可以读取 DMA 控制器的状态。这时,CPU 处于主控状态,而 DMA 控制器就和一般的 I/O 芯片一样,是系统总线的从设备,DMA 控制器的这种工作方式称为从态方式。当 DMA 控制器取得总线控制权后,系统就完全在它的控制下,使 I/O 设备和存储器之间或者存储器与存储器之间进行直接的数据传送,DMA 控制器的这种工作方式称为主态方式。8237A 芯片的内部结构和外部连接与这两种工作状态密切相关。

12-1 8237A 的组成和工作原理

一、8237A 的内部结构

8237A 的内部结构如图 12-1 所示,它主要由 5 个部分组成,下面分别进行介绍。

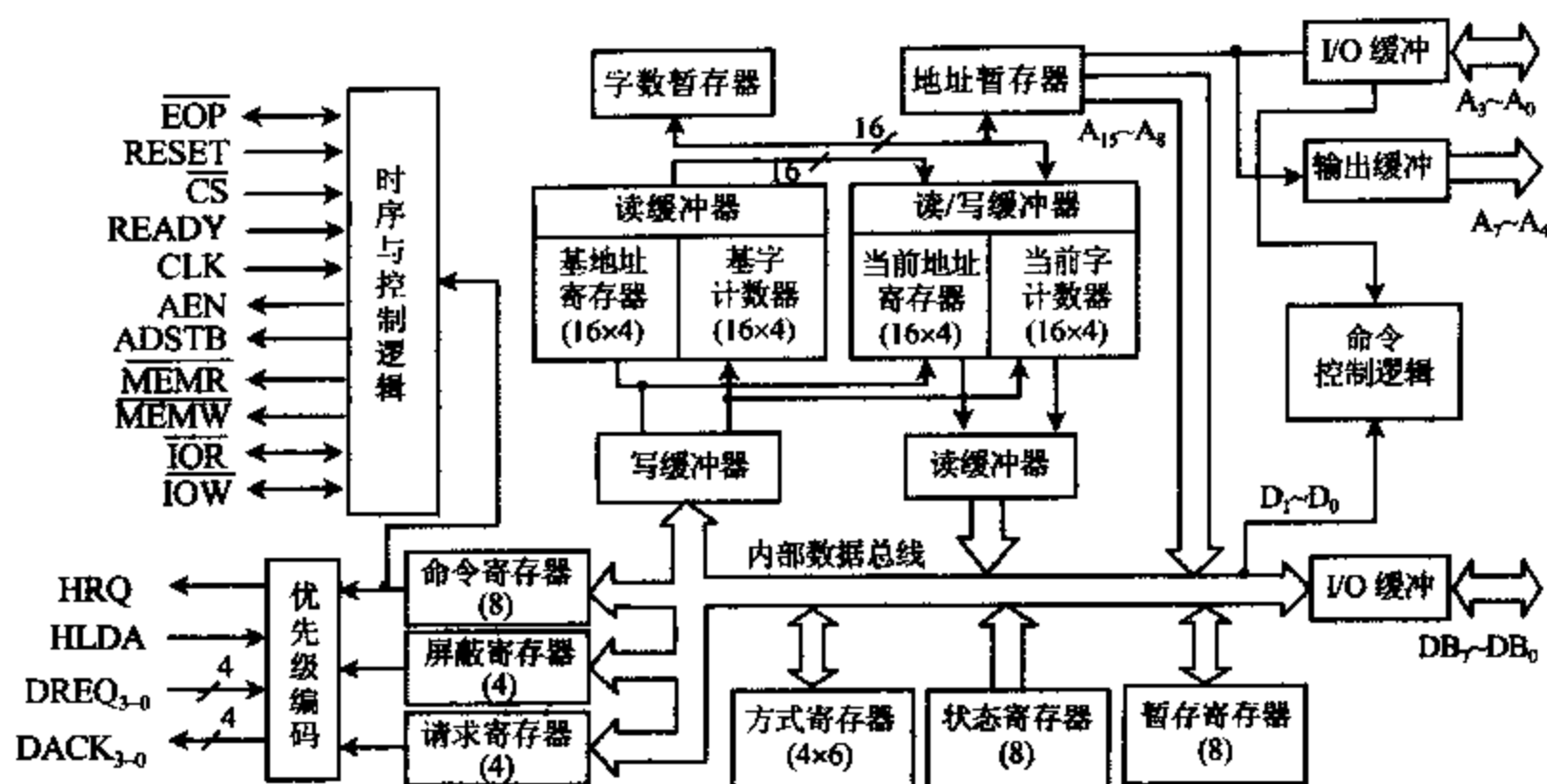


图 12-1 8237A 的内部结构图

1. 时序与控制逻辑

8237A 处于从态时,该部分电路接收系统送来的时钟、复位、片选和读/写控制等信号,完成相应的控制操作;主态时则向系统发出相应的控制信号。

2. 优先级编码电路

该部分电路根据 CPU 对 8237A 初始化时送来的命令,对同时提出 DMA 请求的多个通道进行排队判优,以决定哪一个通道的优先级最高。对优先级的管理有两种方式:固定优先级和循环优先级。无论采用哪种优先级管理,一旦某个优先级高的设备在服务时,其它通道的请求均被禁止,直到该通道的服务结束时为止。

3. 数据和地址缓冲器组

8237A 的 $A_7 \sim A_4$ 、 $A_3 \sim A_0$ 为地址线; $DB_7 \sim DB_0$ 在从态时传输数据信息,主态时传送地址信息。这些数据引线、地址引线都与三态缓冲器相连,因而可以接管或释放总线。

4. 命令控制逻辑

该部分电路在从态时,接收 CPU 送来的寄存器选择信号($A_3 \sim A_0$),选择 8237A 内部相应的寄存器;主态时,对方式字的最低两位($D_1 D_0$)进行译码,以确定 DMA 的操作类型。 $A_3 \sim A_3$ 与 $\overline{IOR}$ 、 $\overline{IOW}$ 配合可组成各种操作命令。

5. 内部寄存器组

8237A 内部的其余部分主要为寄存器。每个通道都有一个 16 位的基地址寄存器、基址计数器、当前地址寄存器和当前字计数器,都有一个 6 位的工作方式寄存器。8237A 有 4 个 DMA 通道,因此上述这几种寄存器在片内各有 4 个。片内还各有一个命令寄存器、屏蔽寄存器、请求寄存器、状态寄存器和暂存寄存器。上述这些寄存器均是可编程寄存器。另外还有字数暂存器和地址暂存器等不可编程的寄存器。

二、8237A 的引脚功能

8237A 是一种 40 引脚的双列直插式器件,其引脚排列如图 12-2 所示,下面分别介绍各引脚的功能。

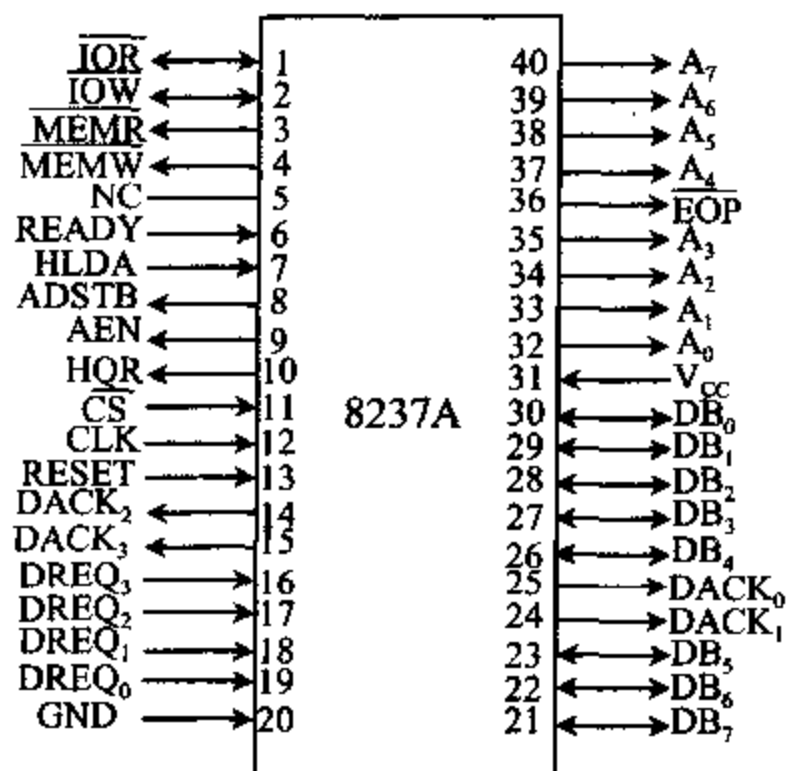


图 12-2 8237A 的引脚

1. CLK 时钟信号,输入

它用来控制 8237A 的内部操作和数据传送速率。8237A 的时钟频率为 3MHz, 8237A-5 的时钟频率可达到 5MHz。8237A-5 DMA 控制器是 8237A 的改进型产品,工作速度较高,但工作原理和使用方法与 8237A 完全一样。

2. CS 片选信号,输入,低电平有效

在从态工作模式下,CS 有效时选中 8237A,这时 DMA 控制器作为一个 I/O 设备,可以通过数据总线与 CPU 通信。

3. READY 准备好,输入,高电平有效

当参与 DMA 传送的设备中有慢速 I/O 设备或存储器时,可能要求延长读/写操作周期,这时可使 READY 变成低电平,使 8237A 可在 DMA 周期中插入等待周期 T_w 。当存储器或外设准备就绪时,READY 端变成高电平。

4. A₃~A₀ 最低 4 位地址线,三态,双向

在从态时,它们是输入信号,用来寻址 DMA 控制器的内部寄存器,使 CPU 对各种不同的寄存器进行读写操作,即对 8237A 进行编程。在主态时,输出的是要访问内存的最低 4 位地址。

5. A₇~A₄ 4 位地址线,三态,输出

这 4 位地址线始终工作于输出状态或浮空状态。在主态时输出 4 位地址信息 A₇~A₄。

6. DB₇~DB₀ 8 位数据线,三态,输入/输出

它们被连到系统数据总线上。从态时,CPU 可用 I/O 读命令从数据总线上读取 8237A 的地址寄存器、状态寄存器、暂存寄存器和字计数器的内容,CPU 还可以通过这些数据线用 I/O 写命令对各个寄存器进行编程。在主态时,高 8 位地址信号 A₁₅~A₈ 经 8 位的 I/O 缓冲器从 DB₇~DB₀ 引脚输出,并由 ADSTB 信号将 DB₇~DB₀ 输出的信号锁存到外部的高 8 位地址锁存器中,它们与 A₇~A₀ 输出的低 8 位地址线一起构成 16 位地址。当 8237A 工作于存储器到存储器的传送方式时,先把从源存储器中读出来的数据,经这些引线送到 8237A 的暂存寄存器中,再经这些引线将暂存器中的数据写到目的存储单元中。

7. AEN 地址允许信号,输出,高电平有效

AEN 信号使地址锁存器中锁存的高 8 位地址送到地址总线上,与芯片直接输出的低 8 位地址一起,构成 16 位内存偏移地址。AEN 信号也使与 CPU 相连的地址锁存器无效,这

样就保证了地址总线上的信号来自 DMA 控制器,而不是来自 CPU。

8. ADSTB 地址选通信号,输出,高电平有效

当它有效时,选通外部的地址锁存器,将 $DB_7 \sim DB_0$ 上输出的高 8 位地址送到外部的地址锁存器中。

9. $\overline{IOR}$ I/O 读信号,双向,三态,低电平有效

从态时,它作为输入控制信号,送入 8237A,当它有效时,CPU 读取 8237A 内部寄存器的值。主态时,它作为输出控制信号,与 $\overline{MEMW}$ 相配合,控制数据由外设传送到存储器中。

10. $\overline{IOW}$ I/O 写信号,双向,三态,低电平有效

在从态时,它是输入控制信号,当它有效时,CPU 向 DMA 控制器的内部寄存器中写入信息,对 8237A 进行初始化编程。在主态时,作输出控制信号,与 $\overline{MEMR}$ 相配合,把数据从存储器传送到外设。

11. $\overline{MEMR}$ 存储器读,三态,输出,低电平有效

主态时,它既可与 $\overline{IOW}$ 配合把数据从存储器读出送外设,也可用于控制内存间数据传送,使数据从源地址单元中读出。从态时该信号无效。

12. $\overline{MEMW}$ 存储器写,三态,输出,低电平有效

主态时,它可与 $\overline{IOR}$ 配合把数据从外设写入存储器,也可用于内存间数据传送的场合,控制把数据写入目的单元。同样,从态时该信号无效。

13. $DREQ_3 \sim DREQ_0$ 通道 3~0 的 DMA 请求信号,输入

当外设请求 DMA 服务时,就向 8237A 的 $DREQ$ 引脚送出一个有效的电平信号,有效电平的极性由编程确定。在固定优先级情况下, $DREQ_0$ 的优先级最高, $DREQ_3$ 的优先级最低,但优先权可通过编程改变。

14. HRQ 保持请求信号,输出,高电平有效

这个信号送到 CPU 的 $HOLD$ 端,是向 CPU 申请获得总线控制权的 DMA 请求信号。8237A 任一个未被屏蔽的通道有 DMA 请求($DREQ$ 有效)时,都可使 8237A 的 HQR (Hold Request)端输出有效的高电平。

15. $HLDA$ 保持响应信号,输入,高电平有效

与 CPU 的 $HLDA$ (Hold Acknowledge)端相连。当 CPU 收到 HRQ 信号后,至少必须经过一个时钟周期后,使 $HLDA$ 变高,表示 CPU 已把总线的控制权交给 8237A 了,8237A 收到 $HLDA$ 信号后,就开始进行 DMA 传送。

16. $DACK_3 \sim DACK_0$ 通道 3~0 的 DMA 响应信号,输出

其有效电平的极性由编程确定。当 8237A 收到 CPU 的 DMA 响应信号 $HLDA$,开始 DMA 传送后,相应通道的 $DACK$ (DMA Acknowledge)有效,将该信号输出到外部,通知外部电路现已进入 DMA 周期。

17. $\overline{EOP}$ 传输过程结束信号,双向,低电平有效

在 DMA 传送时,当 DMA 控制器的任一通道中的字计数器减为 0,再由 0 减为 $FFFFH$ 而终止计数时,会在 $\overline{EOP}$ 引脚上输出一个有效的低电平信号,作为 DMA 传输过程结束信号。8237A 也允许从外部输入一个低电平信号到 $\overline{EOP}$ 引脚上来终止 DMA 传送。不论是内部计数结束引起终止 DMA 过程,还是外部终止 DMA 过程,都会使请求寄存器的相应位复位。如对 8237A 的方式寄存器编程,将通道设置成自动预置状态的话,那么该通道完成一次

DMA 传送, 出现 $\overline{EOP}$ 信号后, 又能自动恢复有关寄存器的初值, 继续执行另一次 DMA 传送。

三、8237A 的内部寄存器

8237A 的内部可编程寄存器主要有 10 种, 列于表 12-1 中, 其内容可由 CPU 读出或者按要求写入。下面先分别介绍这些寄存器的功能, 再给出它们的端口地址, 另外还要介绍 3 条软件命令。

表 12-1 8237A 的内部寄存器

名 称	位数	数 量
当前地址寄存器	16	4 (每通道一个)
当前字计数寄存器	16	4 (每通道一个)
基地址寄存器	16	4 (每通道一个)
基字计数寄存器	16	4 (每通道一个)
工作方式寄存器	6	4 (每通道一个)
命令寄存器	8	1 (4 个通道公用一个)
状态寄存器	8	1 (4 个通道公用一个)
请求寄存器	4	1 (每通道 1 位)
屏蔽寄存器	4	1 (每通道 1 位)
暂存寄存器	8	1 (每通道 1 位)

1. 当前地址寄存器

每个通道都有一个 16 位的当前地址寄存器, 用于存放 DMA 传送的存储器地址值。每传送一个数据, 地址值自动增 1 或减 1, 以指向下一个存储单元。在编程状态下, CPU 可用输出指令对该寄存器写入初值, 也可由输入指令读出该寄存器中的值, 但每次只能读/写 8 位数据, 故对该寄存器的读/写操作要分两次进行。若将工作方式寄存器编程为自动预置操作, 则当 DMA 传送结束, 产生 $\overline{EOP}$ 信号后, 会自动将基地址的值重新装入该寄存器中。

2. 当前字计数寄存器

每个通道都有一个 16 位的当前字计数寄存器, 它的初值比实际传送的字节数少 1, 该值是在编程状态下由 CPU 写入的。在进行 DMA 传送时, 每传送一个字节, 字计数器的内容自动减 1, 当它的值减为 0, 再由 0 减为 FFFFH 时, 将产生终止计数信号 TC(Terminal Count)。若选择自动预置操作方式, 则在 $\overline{EOP}$ 信号有效时, 会自动将基字计数寄存器的内容重新装入该寄存器。

3. 基地址寄存器

每个通道都有一个 16 位的基地址寄存器, 它用来存放对应通道当前地址寄存器的初值, 该值是在 CPU 对 DMA 控制器进行编程时, 与当前地址寄存器的值一起被写入的, 即两个寄存器有相同的写入端口地址, 编程时写入相同的内容。但基地址寄存器的内容不能被 CPU 读出, 也不能被修改。设置该寄存器的目的主要在于, 当执行自动预置操作时, 使当前地址寄存器能恢复到初始值。

4. 基址计数寄存器

每个通道都有一个 16 位的基址计数寄存器,用于存放对应通道当前字计数器的初值,该值也是在 CPU 对 8237A 进行编程时与当前字计数器一起被写入的,且两者具有相同的写入端口,写入相同的内容。该寄存器的内容也不能被 CPU 读出,它主要用于自动预置操作时使当前字计数器恢复初值。

5. 命令寄存器

命令寄存器是一个 8 位寄存器,用来控制 8237A 的操作。编程状态时,由 CPU 对它进行编程,设置 8237A 的操作方式,复位时将其清除。命令寄存器的格式如图 12-3 所示。

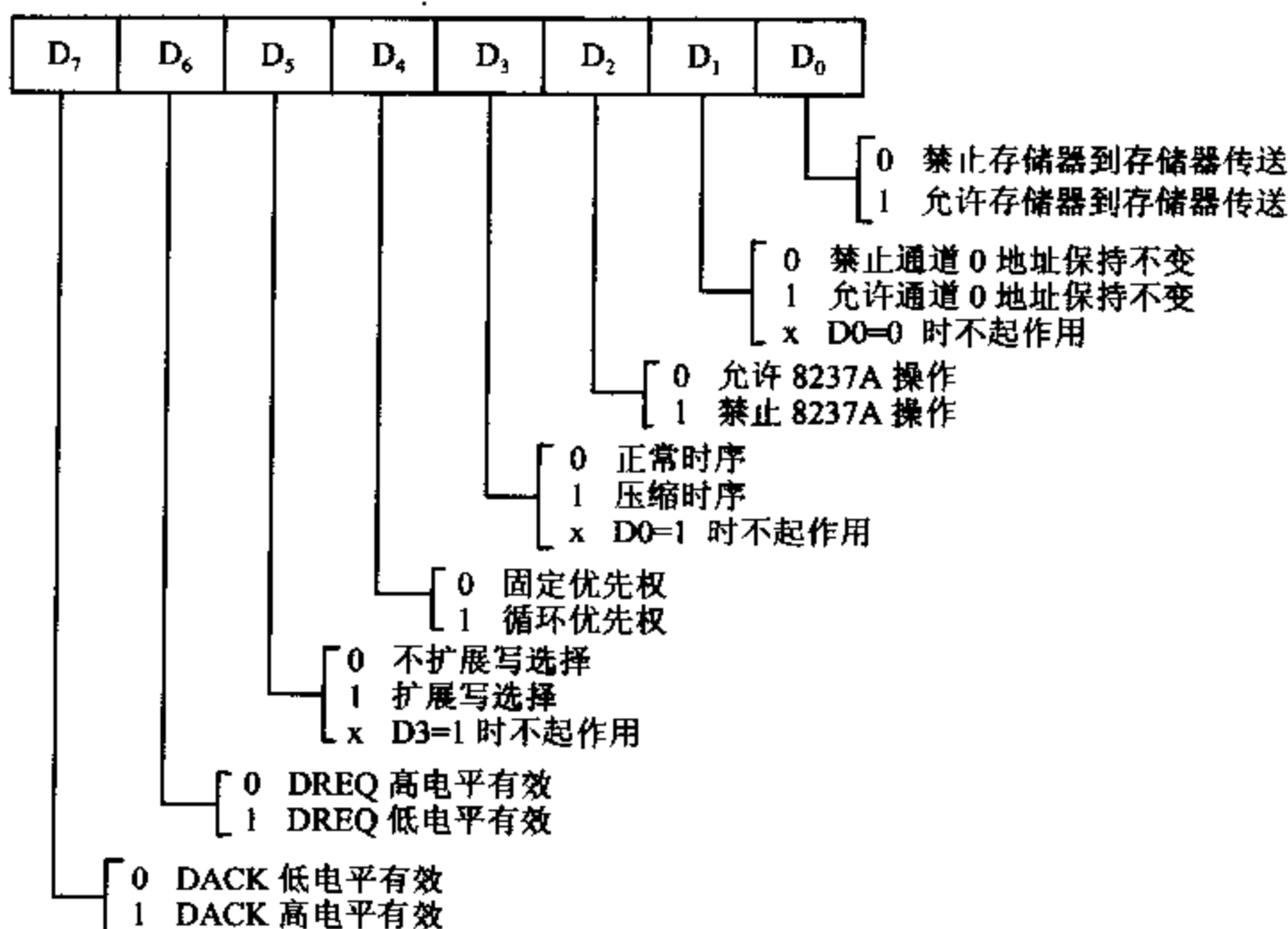


图 12-3 命令寄存器格式

D₀ 位用于决定是否进行存储器到存储器传送操作。若 D₀=1,则允许这种操作,并规定先用通道 0 从源地址存储单元读入数据字节,再放到暂存器中,然后由通道 1 把数据字节写到目的地址存储单元,接着将两通道的地址分别加 1 或减 1,通道 1 的字计数器减 1,当字计数器减为 0 时,产生终止计数信号 TC,并输出 $\overline{\text{EOP}}$ 信号,终止 DMA 服务。

D₁ 位用于执行存储器到存储器传送操作时,决定是否允许通道 0 的地址保持不变。当 D₁=1 时,可以使通道 0 在整个传送过程中保持同一地址,这样可把这个地址单元中的数(也即同一个数据)写到一组存储单元中去,例如可使一批存储单元均清 0。D₁=0 时禁止这种操作。当然,当 D₀=0 时,不允许在存储器间直接进行数据传送,这种方法也就无效。

D₂ 位用来表示允许还是禁止 8237A 工作,当它为 0 时允许 8237A 工作,否则禁止工作。

D₄ 位控制优先权。D₄ 位为 0 时,为固定优先权,这时规定通道 0 的优先级最高,通道 1 次之,通道 3 最低。D₄=1 时为循环优先权,它使刚服务过的通道 i(i 表示通道号)的

优先权变成最低,而让通道 $i+1$ 的优先权变为最高,当 $i+1=4$ 时使通道号回 0。例如,若某次传输前优先权从高到低的次序为 2-3-0-1,那么在通道 2 进行一次传输后,优先级次序变成 3-0-1-2,通道 3 完成传输后优先级次序成为 0-1-2-3,等等。随着 DMA 操作的不断进行,优先权也不断循环变化,这样可防止某一通道长时间占用总线。但要注意,任一个通道进入 DMA 服务后,其它通道均不能去打断它,这一点和 8259A 的管理方式不同。

D_6 位决定 DREQ 的电平是高电平有效还是低电平有效,0 为高电平有效,1 则低电平有效。

D_7 位决定 DACK 的电平是高电平有效还是低电平有效,1 为高电平有效,0 时低电平有效。

D_3 位和 D_6 位是有关时序的操作,在后面讨论时序时再作介绍。

6. 工作方式寄存器

每个通道都有一个 6 位的工作方式寄存器,用于选择 DMA 的传送方式和类型等,其格式如图 12-4 所示。

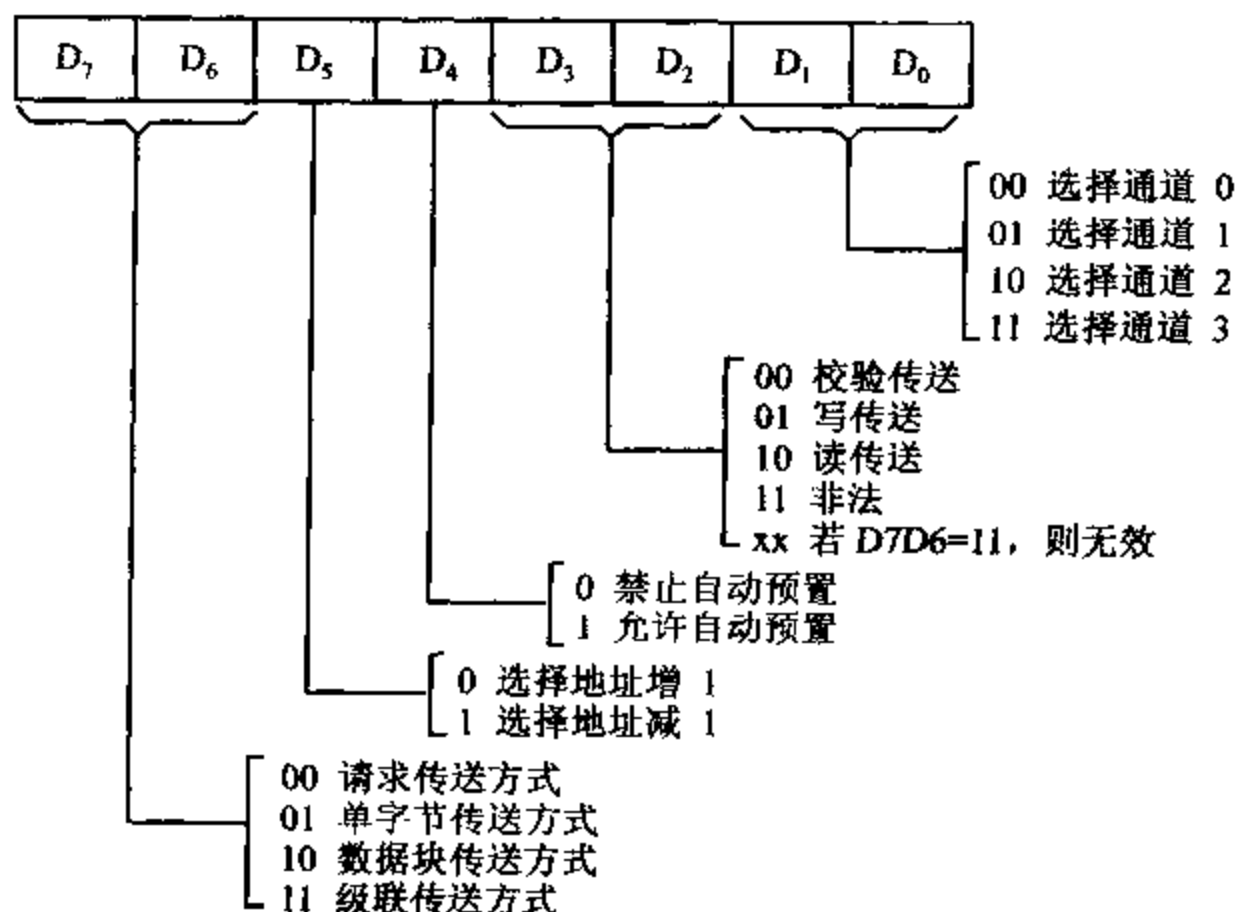


图 12-4 工作方式寄存器格式

$D_1 D_0$ 位用于选择通道, $D_7 \sim D_6$ 这 6 位是工作方式控制位。由于 4 个通道的方式寄存器使用相同的 I/O 端口地址, 向方式寄存器写入方式字时, 用最低两位 $D_1 D_0$ 来选择到底向哪个通道写入工作方式字。

$D_3 D_2$ 位决定所选通道的 DMA 操作的传送类型。8237A 共有 3 种 DMA 传送类型: 读传送、写传送和校验传送。读传送将数据从存储器传送到 I/O 设备中去, 8237A 发出 $\overline{MEMR}$ 和 $\overline{IOW}$ 信号; 写传送是把外部设备的数据写到存储器中, 8237A 发出 $\overline{IOR}$ 和 $\overline{MEMW}$ 信号; 校验传送是一种伪传送, 8237A 也会产生地址信息和 $\overline{EOP}$ 信号, 但不会发出对存储器和 I/O 设备的读写控制信号, 这种功能一般是在对器件进行测试时才使用。

D_4 位定义对所选通道是否进行自动预置操作。若 $D_4=1$, 选择自动预置。这时, 每当产生有效的 $\overline{EOP}$ (不管是由内部 TC 或外界产生的) 信号后, 该通道将自动把基地址寄存器和基址计数器的内容分别重新置入当前地址寄存器和当前字计数器中, 达到重新初始化的目的。这样就不需要 CPU 的干预, 又能自动执行下一次 DMA 操作。 $D_4=0$ 时, 禁止自动预置。注意, 如果某通道被置为具有自动预置功能, 则该通道的对应屏蔽位必须为 0, 表示清除屏蔽。

D_5 位是方向控制位。若 $D_5=0$, 表示数据传送的顺序由低地址向高地址方向进行, 每传送一个字节, 地址增 1。若 $D_5=1$ 时, 操作由低地址向高地址方向进行, 每传送一个字节, 地址减 1。

D_7D_6 位用来定义所选通道的操作方式, 8237A 进行 DMA 传送时, 有 4 种传送方式, 它们是:

(1) 单字节传送方式

在这种方式下, 每进行一次 DMA 操作, 只传送一个字节的数据。传送后字计数器减 1, 地址寄存器加 1 或减 1 (由 D_5 位决定), 保持请求信号 HRQ 无效, 并释放系统总线。当字计数器由 0 减为 FFFFH 时, 产生终止信号 TC。

为了保证请求信号 DREQ 得到确认, 在 DMA 响应信号 DACK 变为有效之前, DREQ 必须一直保持有效。但是, 如果执行一次 DMA 传送后, DREQ 没有失效而是继续保持有效的话, 8237A 的保持请求信号 HRQ 输出仍要进入无效状态, 并将总线让给 CPU 控制。由于 DREQ 仍处于有效状态, HRQ 很快会再次变为有效, 当 8237A 收到 CPU 发来的新的 HLDA 信号后, 又开始下一个字节的传送。这样, 在 8080A/8085A 及 8086/8088 系统中, 能保证在两次 DMA 传送之间, CPU 可执行一次完整的机器周期。

(2) 数据块传输方式

在这种传送方式下, 当 DREQ 有效, 芯片进入 DMA 服务以后, 可以连续传输数据, 一直到一批数据传送完毕, 字计数器由 0 减为 FFFFH 产生 TC 信号或外部送来有效的 $\overline{EOP}$ 信号时, 8237A 才释放总线, 结束 DMA 传输。

(3) 请求传送方式

这种传送方式与数据块传送方式相类似, 也可以连续传送数据, 直到字计数器由 0 减为 FFFFH 产生 TC 或外界送来有效的 $\overline{EOP}$ 信号时才停止传送。但与数据块传送方式不同之处在于, 每传送一个字节后, 8237A 都要对 DREQ 端进行测试, 一旦检测到 DREQ 信号无效, 则马上停止传送。在这种情况下, 8237A 会把地址和字计数器的中间值保存在相应通道的现行地址和字计数器中, 但测试 DREQ 状态的过程仍在不断进行。只要外设准备好了新的数据, 使 DREQ 再次变为有效后, 又可使 DMA 传输继续进行下去。

(4) 级联传送方式

级联传送方式将多个 8237A 连在一起, 以便扩充系统的 DMA 通道。级联传送方式的连线如图 12-5 所示。

从图中可以看到 8237A 从片的 HRQ 和主片的 DREQ 端相连, 从片的 HLDA 和主片的 DACK 相连。而主片的 HRQ 和 HLDA 分别与微处理器的 HOLD 和 HLDA 相连。1 块主片最多允许与 4 块从片相连, 若用 5 片 8237A 芯片构成两级 DMA 系统, 可得到 16 个 DMA 通道。编程时主片应置为级联传送方式, 从片不用设成级联方式, 而是设成其它三种

工作方式之一。

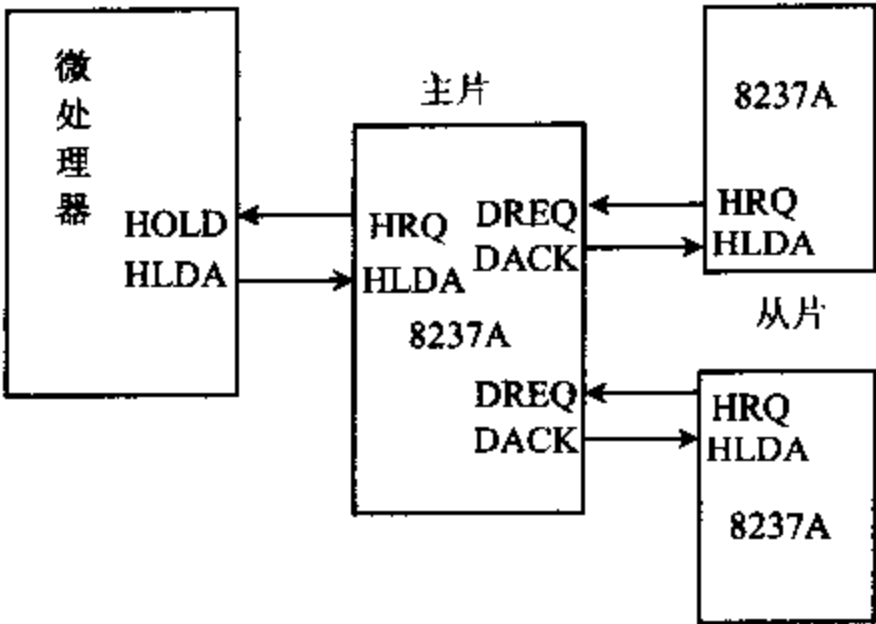


图 12-5 8237A 级联方式连线图

7. 请求寄存器

在 8237A 内部有一个 4 位的请求寄存器,每位对应一个通道。相应请求位置 1 时,对应的通道可产生 DMA 请求,清 0 时不产生请求。它们可用硬件方法由外部送到 DREQ 线上的请求信号使相应位置 1,产生 DMA 请求。当 8237A 工作于数据块传送时,也可用软件方法使请求位置 1 或清 0,而且每个通道的请求位可以分别进行设置。可对请求寄存器写入通道请求字来设置这 4 个请求位,请求字格式如图 12-6 所示。

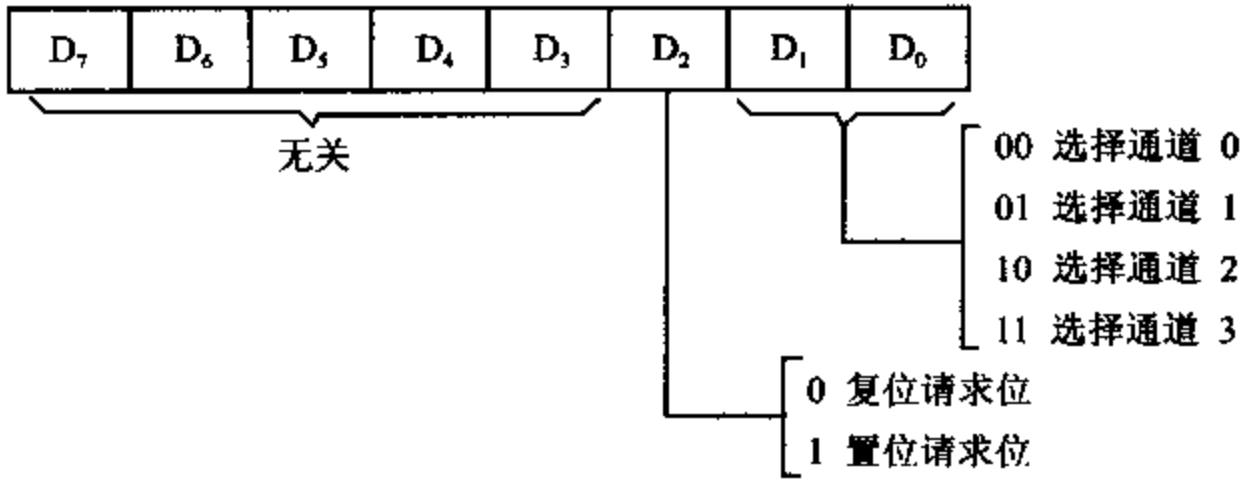


图 12-6 通道请求字格式

由图可见,请求字中的 D_1D_0 位用来选择通道号, D_2 位用来设置相应通道的请求位,即指定该通道是否设置 DMA 请求。软件请求位是不能被屏蔽的,其优先权同样受优先权逻辑的控制,TC 或外部的 $\overline{EOP}$ 信号能将相应的请求位清 0,RESET 信号则使整个请求寄存器清 0。

8. 屏蔽寄存器

8237A 内部有一个 4 位的屏蔽寄存器,每位对应一个通道。相应屏蔽位置 1 时,禁止对应通道的 DREQ 请求进入请求寄存器,屏蔽位复位时,允许 DREQ 请求。

如果某通道初始化时处于禁止自动预置方式,则该通道产生 $\overline{EOP}$ 信号时,它所对应的屏蔽位就置位,禁止该通道产生 DMA 请求。RESET 信号可使整个屏蔽寄存器置位,这时

禁止所有通道产生 DMA 请求，直到用一条清除屏蔽寄存器的软件命令使之复位后才允许接收 DMA 请求。

对 8237A 允许写入两种屏蔽字，使各屏蔽位置位或复位。两种屏蔽字需写入不同的端口地址中。

(1) 通道屏蔽字

与请求寄存器的编程那样，可用指令对屏蔽寄存器写入通道屏蔽字来对单个屏蔽位进行操作，使之置位或复位。通道屏蔽字的格式与通道请求字的格式相类似，如图 12-7 所示。

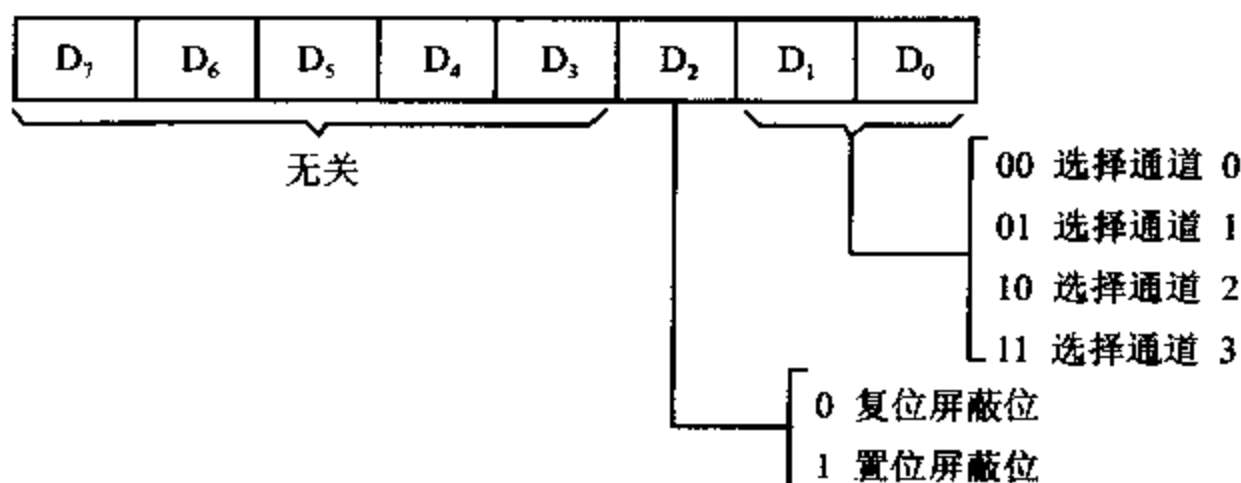


图 12-7 通道屏蔽字格式

(2) 主屏蔽字

另外，8237A 还允许使用主屏蔽命令来设置通道的屏蔽触发器。主屏蔽字格式如图 12-8 所示， $D_3 \sim D_0$ 位对应通道 3~0 的屏蔽位，各个屏蔽位中，0 表示清除屏蔽位，1 表示置位屏蔽位，这样利用一条主屏蔽字命令就可一次完成对 4 个通道的屏蔽位的设置。

在需要同时清除 4 个通道的屏蔽位时，还可用软件命令实现。

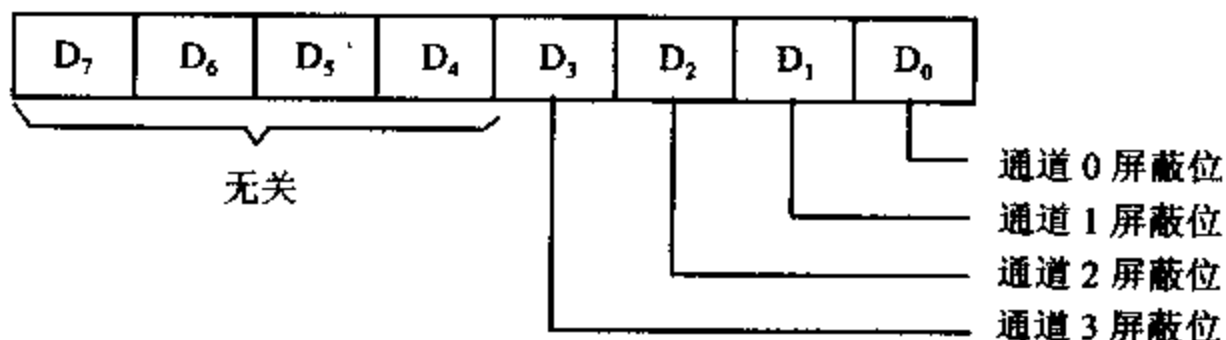


图 12-8 主屏蔽字格式

9. 状态寄存器

8237A 内部的 8 位状态寄存器用来存放状态信息，可供 CPU 读出。状态信息的低 4 位用来表示哪些通道已达到计数终点 TC，哪些尚未达到。只要通道计数达到终点或外界送来有效的 $\overline{EOP}$ 信号，相应位就被置成 1，否则被清 0。高 4 位表示哪些通道的 DMA 请求还未被处理，有请求存在的那些位被置 1，无请求的被清 0。状态寄存器的格式如图 12-9 所示，这些状态位在复位或被读出后均被清除。

10. 暂存寄存器

在存储器到存储器传送时，暂存寄存器用来保存所传送的数据。当传送完成时，暂存寄存器中始终保存着最后一个传送的数据字节，除非用 RESET 信号将其清除，在编程状态

下,这个数据字节可由 CPU 读出。

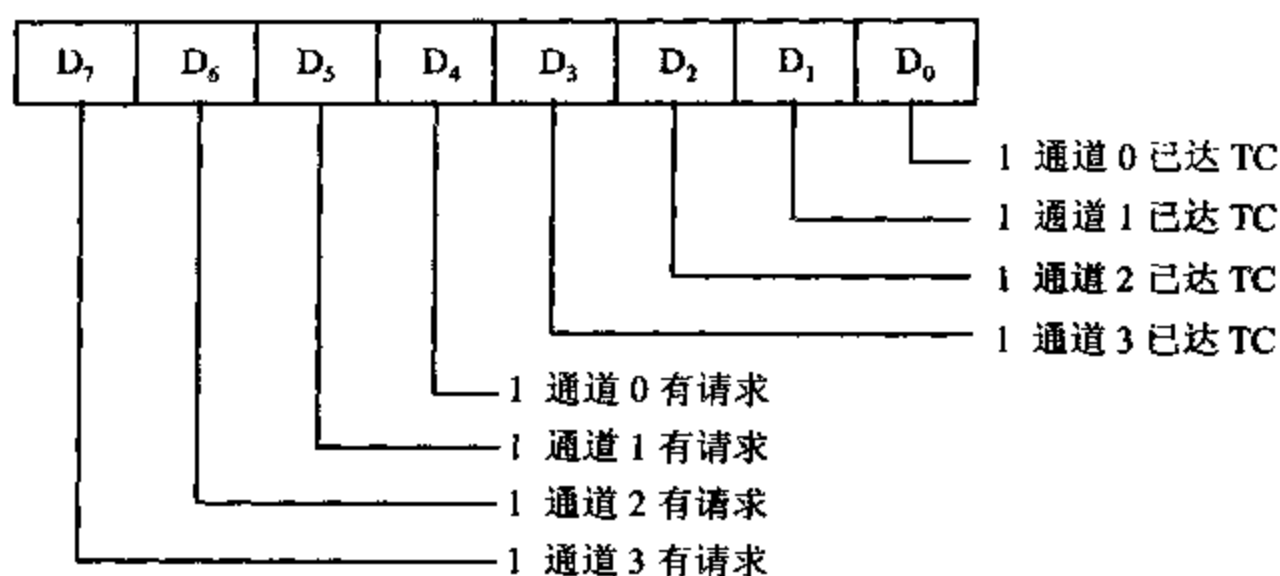


图 12-9 状态寄存器格式

11. 软件命令

在编程状态下,8237A 可执行 3 个附加的特殊软件命令,这 3 个软件命令不关心数据的具体格式,只要对特定的端口地址进行一次写操作,命令就生效。3 条软件命令是:

(1) 清除先/后触发器

由于 8237A 的数据线 $DB_7 \sim DB_0$ 的宽度只有 8 位,一次只能传送一个字节,而各通道的地址寄存器和字计数器的长度都是 16 位,CPU 读写这些寄存器时必须分两步进行。为此,在 8237A 内部设有一个先/后触发器(First/Last Flip-Flop),用于控制读/写的次序。当触发器清 0 时,读/写低 8 位数据,随后先/后触发器自动置成 1,读写高 8 位数据。接着,该触发器又自动清为 0,如此循环不断进行操作。

为了按正确的顺序访问寄存器中的高 8 位字节和低 8 位字节,CPU 应使用清除先/后触发器指令,将先/后触发器清成 0,对该寄存器执行一次写操作即可使其清 0。在复位和 $\overline{EOP}$ 信号有效后,该触发器复位为 0。

(2) 主清命令

主清命令也称为复位命令,其功能与 RESET 信号相同,它可以使命令寄存器、状态寄存器、请求寄存器、暂存寄存器和内部先/后触发器均清 0,而把屏蔽寄存器置 1。8237A 被复位后,进入空闲状态。

(3) 清除屏蔽寄存器

该命令能清除 4 个通道的全部屏蔽位,允许各通道接受 DMA 请求。

12. 各寄存器对应的端口地址

对 8237A 的内部寄存器进行读写操作时, $\overline{CS}$ 端必须为低电平,才能选中 8237A 芯片,该信号由高位地址经译码后产生。8237A 的 $A_3 \sim A_0$ 线用于选择 8237A 内部的不同寄存器,这些寄存器占有 16 个 I/O 端口地址。常把 8237A 的 $A_3 \sim A_0$ 与系统地址总线的低位地址线 $A_3 \sim A_0$ 相连,以产生寄存器选择信号,而系统高位地址线经译码后形成 I/O 端口选择信号。

例如,在 PC/XT 机中,高位地址线 $A_9 \sim A_4 = 000000$ 时,经 I/O 译码电路,选中 8237A 芯片,使该芯片的 $\overline{CS}$ 有效。地址总线的 $A_3 \sim A_0$ 与 8237A 的相应引脚相连,用于片内寻址,

这 4 位为 0000 时,选中的端口地址为 000H,它作为 DMA 的基地址,我们用 DMA 来表示。在执行 IN AL, DMA 指令时,读信号 $\overline{IOR}$ 有效,选中当前地址寄存器,并将该寄存器的内容读出来送到 AL 寄存器中。执行 OUT DMA, AL 指令时,写信号 $\overline{IOW}$ 有效,将 AL 寄存器中的内容写入通道 0 的基地址与当前地址寄存器中。当 $A_3A_2A_1A_0=1000$ 时,端口地址为 DMA+08H,执行 IN 指令时, $\overline{IOR}$ 有效,选中状态寄存器,执行 OUT 指令时, $\overline{IOW}$ 有效,选中命令寄存器。

8237A 各寄存器与读写端口配合后形成的 I/O 端口地址的分配如表 12-2 所示。其中基地址 DMA=000H。

表 12-2 8237A 内部寄存器口地址分配表

I/O 口地址 十六进制	寄 存 器	
	读 ($\overline{IOR}$ 有效)	写 ($\overline{IOW}$ 有效)
00	通道 0 当前地址寄存器	通道 0 基地址与当前地址寄存器
01	通道 0 当前字计数寄存器	通道 0 基字计数与当前字计数寄存器
02	通道 1 当前地址寄存器	通道 1 基地址与当前地址寄存器
03	通道 1 当前字计数寄存器	通道 1 基字计数与当前字计数寄存器
04	通道 2 当前地址寄存器	通道 2 基地址与当前地址寄存器
05	通道 2 当前字计数寄存器	通道 2 基字计数与当前字计数寄存器
06	通道 3 当前地址寄存器	通道 3 基地址与当前地址寄存器
07	通道 3 当前字计数寄存器	通道 3 基字计数与当前字计数寄存器
08	状态寄存器	命令寄存器
09	—	请求寄存器
0A	—	屏蔽寄存器(通道屏蔽字)
0B	—	工作方式寄存器
0C	—	清除先/后触发器
0D	暂存寄存器	主清命令寄存器
0E	—	屏蔽寄存器(清除屏蔽)
0F	—	屏蔽寄存器(主屏蔽字)

由表 12-2 可知,每个通道的基地址和当前地址寄存器合用一个端口,故在进行写入操作时,它们被装入相同的初始值,但当前地址寄存器的值可由 CPU 读出,而基地址的值不能被读出。另外,有些端口(如 09H 和 0AH 等)只能进行写入操作,不允许读出(表中用“—”表示),使用时要注意。

12-2 8237A 的时序

一、外设和内存间的 DMA 数据传送时序

8237A 主要用于外设和内存之间进行高速的数据传输,其时序如图 12-10 所示。

8237A 有两个主要的工作周期,即空闲周期(Idle Cycle)和有效周期(Active Cycle)。每个周期由若干个状态构成。8237A 设有 7 个独立的操作状态:SI、SO、S1、S2、S3、S4 和 SW,每个状态包含一个时钟周期。

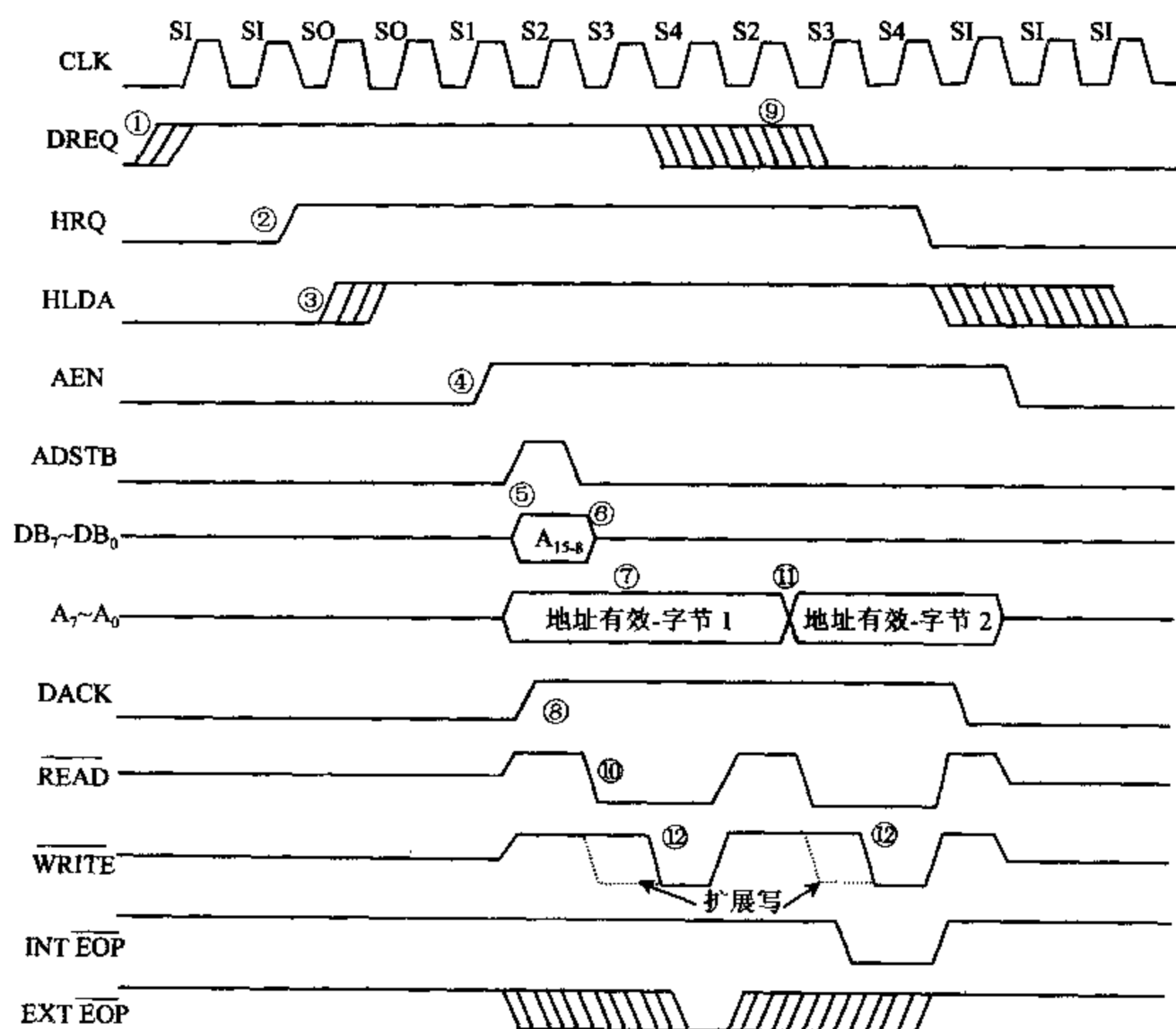


图 12-10 外设和内存间 DMA 数据传送时序

在 7 个状态中,SI 是非操作状态,当 8237A 未接到 DMA 请求时便进入 SI 状态,在此状态下,8237A 可由 CPU 编程,预置操作方式。状态 SO 是 DMA 服务的第一个状态,这时 8237A 已向 CPU 的 HOLD 引脚发出一个 DMA 请求信号,但还没收到回答信号,当 8237A 收到 CPU 的应答信号 HLDA 后,就意味着 DMA 传送已开始。S1、S2、S3 和 S4 是 DMA 服务的工作状态,必要时还可以由慢速设备使用 8237A 的 READY 线,在 S2 和 S4 或 S3 和 S4 之间插入等待状态 SW。

二、空闲周期、有效周期和扩展写周期

1. 空闲周期

系统复位后或无 DMA 请求时处于空闲周期,这时 DMA 处于从态方式。在空闲周期的

每一个时钟周期,8237A 都对 DREQ 线进行采样,以确定是否有 DMA 请求,若无请求则一直处于 SI 状态。同时还对 $\overline{CS}$ 端进行采样,若 $\overline{CS}$ 为低电平,而 DREQ 也为低(无效)则进入程序状态。这时,CPU 可对 8237A 进行编程,把数据写入内部寄存器,或者从内部寄存器中读出内容进行检查, $\overline{IOR}$ 和 $\overline{IOW}$ 信号控制读/写操作,地址信号 $A_3 \sim A_0$ 用于选择内部寄存器的口地址。

2. 有效周期

当 8237A 在 SI 状态采样到外部有效的 DMA 请求信号 DREQ 后①,就向 CPU 发 DMA 请求信号 HRQ②,并进入有效周期 SO 状态,等待 CPU 发出允许 DMA 操作的回答信号 HLDA。此时,8237A 仍可接受 CPU 的访问,SO 是由从态转至主态的过渡阶段。若在 SO 周期的上升沿采样到 HLDA 信号(高电平)③,则表示 CPU 已交出系统总线的控制权,下一个周期便进入 DMA 传送状态周期 S1,DMA 处于主态工作方式。

一个完整的 DMA 传送周期由 S1、S2、S3 和 S4 共 4 个状态组成。在 S1 状态周期,地址允许信号 AEN 有效④,把要访问的存储单元的高 8 位地址 $A_{15} \sim A_8$ 送到数据总线 $DB_7 \sim DB_0$ 上⑤,并发地址选通信号 ADSTB,其下降沿(S2 周期内)把高 8 位地址锁存到外部的地址锁存器中⑥。低 8 位地址 $A_7 \sim A_0$ 由 8237A 直接送到地址总线上⑦,在整个 DMA 传送中都要保持住。

S2 状态周期用来修改存储单元的低 16 位地址。此时,8237A 从 $DB_7 \sim DB_0$ 线上输出这 16 位地址的高 8 位 $A_{15} \sim A_8$,从 $A_7 \sim A_0$ 线上输出低 8 位。另外,8237A 向外设输出 DMA 响应信号 DACK⑧,并使读或写信号有效,这样外设与内存间可在读写信号控制下交换数据。通常 DREQ 信号必须保持到 DACK 有效之后才能失效,失效允许有一个时间范围,图中用多条斜线表示⑨。

若对命令寄存器的 D_3 位编程,设定为正常时序后,工作时序中将会出现 S3 状态⑩,用来延长读脉冲,即延长取数时间。如用压缩时序工作,就没有 S3 状态,直接由 S2 状态进入 S4 状态。

在 S4 状态,8237A 对传输模式进行测试,如果不是数据块传输方式,也不是请求传送方式,则在测试后可立即回到 S1 或 S2 状态。在数据块传送方式下,S4 后应接着传送下一个字节,在大部分情况下,地址高 8 位不变,每传送 256 个数据字节才变一次,仅低 8 位地址增 1 或减 1。所以,在大部分情况下锁存高 8 位地址的 S1 状态就用不着了,可直接由 S4 周期进入 S2 周期,从输出低 8 位地址起执行新的读写命令⑪,一直到数据传送完毕,8237A 又进入 SI 周期,等待新的请求。

3. 扩展写周期

从上面的讨论中可以看出,8237A 用正常时序工作时,一般要用到 3 个时钟周期 S2、S3 和 S4。在系统特性许可的范围内,为了加快传送速度,8237A 可采用压缩时序,将传送时间压缩到两个时钟周期 S2 和 S4 内,压缩时序只能出现在连续传送数据的 DMA 操作中。无论是正常时序还是压缩时序,当高 8 位地址要修正时,S1 状态仍必须出现。

如果外设的速度比较慢,必须用正常时序工作。若正常时序仍不能满足要求,以至于还是不能在指定时间内完成存取操作,那么就要在硬件上通过 READY 信号使 8237A 插入等待状态 SW。有些设备是利用 8237A 送出的 $\overline{IOW}$ 或 $\overline{MEMW}$ 信号的下降沿产生 READY 响应的,而这两个信号都是在传送过程的最后才送出的,为使 READY 信号提前到来,将写脉

冲拉宽,并使它们提前到来,这就要用到扩展写信号方法⑫。扩展写功能是通过命令寄存器的 D_5 位的设置来实现的,当 D_5 位置 1 时,写信号被扩展到 2 个时钟周期。

在 S4 状态开始前,8237A 要检测 READY 输入信号,若其为低则插入等待状态 SW(图 12-10 中未画出 SW 状态),直到 READY 变为高电平,才进入 S4。在 S4 结束时,8237A 完成数据传输。对于慢速的存储器和 I/O 设备,进行 DMA 传送时,可插入等待周期。

12-3 8237A 的编程和应用举例

一、PC/XT 机中的 DMA 控制逻辑

1. 各通道功能

在 PC/XT 机中,用一片 8237A-5 构成 DMA 控制电路形成 4 个 DMA 通道,提供数据宽度为 8 位的 DMA 传输。使用固定优先级,所以通道 0 的优先级最高,通道 3 最低。这 4 个 DMA 通道的功能分配如下:

通道 0 用于动态 RAM 的刷新

通道 1 为用户保留

通道 2 用于软盘 DMA 传送

通道 3 用作硬盘 DMA 传送

虽然,8237A 既能提供外设和存储器之间的 DMA 传输,也能进行存储器和存储器之间的 DMA 传输,但在 PC/XT 机的 BIOS 初始化系统时,将 8237A 的存储器和存储器间传送方式禁止掉了,因此只用它实现外设和内存间的高速数据交换。PC/XT 机的 DMA 控制逻辑包括 DMA 控制电路和应答控制电路两个部分,下面主要介绍 DMA 控制电路。

2. DMA 控制电路

PC/XT 机中的 DMA 控制电路如图 12-11 所示。它由 8237A-5 DMA 控制器、地址驱动器、地址锁存器和页面寄存器等器件组成。

在 DMA 服务期间,直接从 8237A-5 的 $A_7 \sim A_4$ 和 $A_3 \sim A_0$ 输出低 8 位地址,在整个 DMA 传输周期中这些地址信号都是稳定的,它们被送到地址驱动器 U12(74LS244)的输入端。

仅在 S1、S2 状态,从数据线 $DB_7 \sim DB_0$ 输出高 8 位地址 $A_{15} \sim A_8$,因此要用三态地址锁存器由 ADSTB 选通信号将其锁存。

PC 机的存储器有 20 根地址线,能对 1MB 的空间进行寻址,而 8237A-5 只能提供 16 位地址,即最多只能形成 64KB 的地址信息。为了实现对全部内存空间的寻址,在 PC/XT 中设置了一个页面寄存器 74LS670(U10),用来产生存储器的高 4 位地址 $A_{19} \sim A_{16}$,8237A-5 则管理低 16 位地址 $A_{15} \sim A_0$ 。通过对页面寄存器的编程,便可以在 1M 内存范围内寻址。但是,在 DMA 传输过程中,页面寄存器的值是固定的,即总是指向内存中某个 64KB 的地址范围。

页面寄存器内部有 4 个可以由程序读写的寄存器,每个寄存器对应一个 DMA 通道,长

度均为 4 位。对于每一个 DMA 通道来说,页面寄存器的 4 位输入接到系统数据总线的 D_3

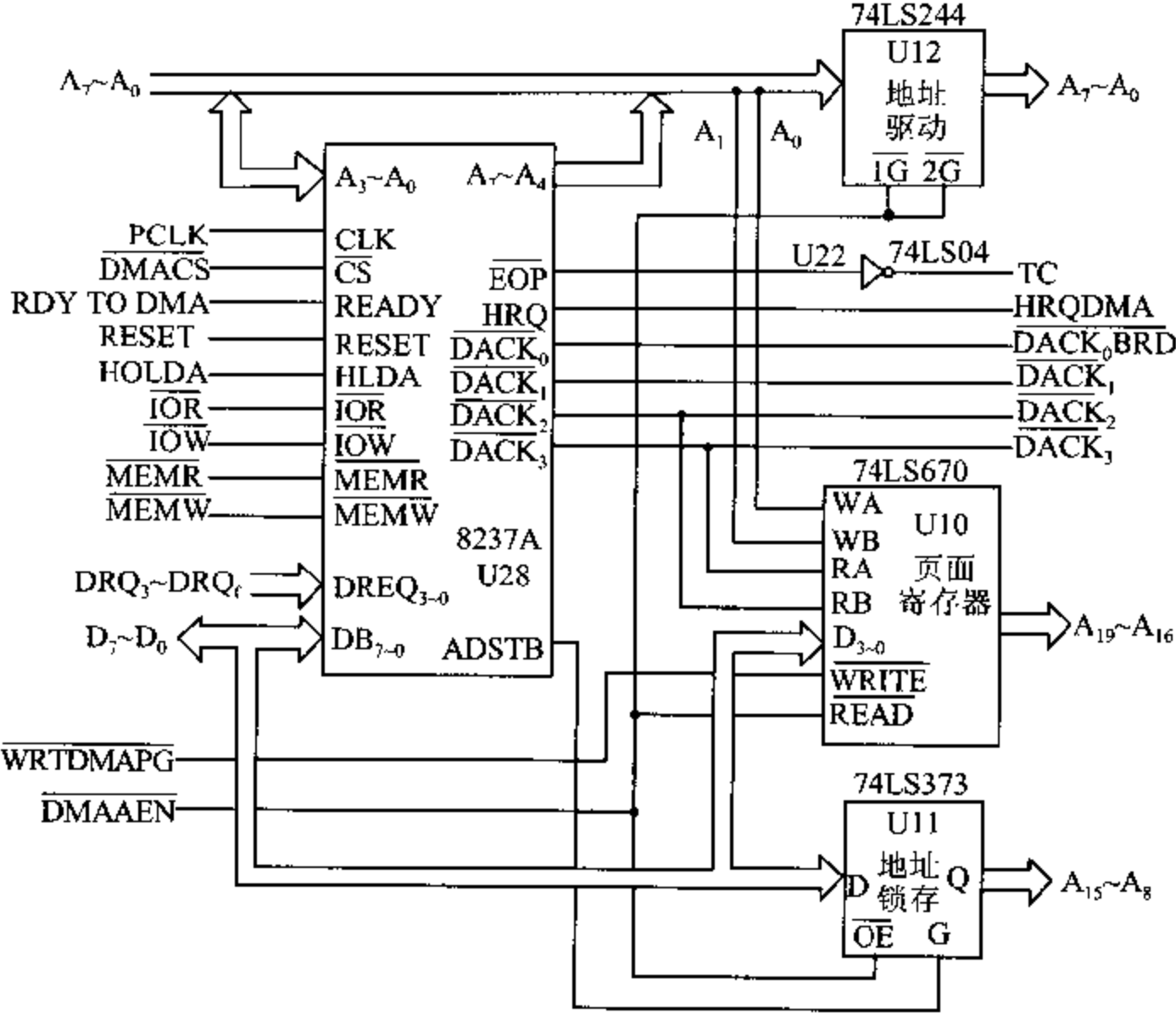


图 12-11 PC/XT 机的 DMA 控制逻辑

$\sim D_0$ ，4 位输出接系统地址总线的 $A_{19} \sim A_{16}$ 。当控制信号 $\overline{\text{WRITE}}$ 为低电平时,可从数据总线的低 4 位 $D_3 \sim D_0$ 将最高 4 位地址信息写入该通道的页面寄存器。寄存器号由 WA 、 WB 译码产生, WA 、 WB 分别与地址总线的最低两位 A_1 、 A_0 相连,其编码如表 12-3 所示。

表 12-3 74LS670 内部寄存器写入功能

$\overline{\text{WRITE}}$	WB	WA	功 能	对应通道
0	0	0	写入 0 号寄存器	未用
0	0	1	写入 1 号寄存器	通道 2
0	1	0	写入 2 号寄存器	通道 3
0	1	1	写入 3 号寄存器	通道 1

当控制端 $\overline{\text{READ}}$ 为低电平时,将某一个内部寄存器的地址信息从输出端读出,读出时寄存器号由 RB 、 RA 编码产生,读出功能编码如表 12-4 所示。

表 12-4 74LS670 内部寄存器读出功能

$\overline{\text{READ}}$	RB	RA	功 能	对应通道
0	0	0	读出 0 号寄存器	未用
0	0	1	读出 1 号寄存器	通道 2
0	1	0	读出 2 号寄存器	通道 3
0	1	1	读出 3 号寄存器	通道 1

系统中将 $\overline{\text{WRITE}}$ 信号和页寄存器的片选信号 $\overline{\text{WRTDMAPG}}$ 相连,该片选信号由 I/O 端口地址译码电路产生, $A_9 \sim A_5 = 00100$ 时可选中页寄存器芯片,因此在编程状态下,当 CPU 对 $80\text{H} \sim 9\text{FH}$ 口地址执行输出指令时, $\overline{\text{WRTDMAPG}}$ 为低电平,这样就可将数据线 $D_3 \sim D_0$ 上的内容写入页寄存器中。在 PC/XT 的 ROM BIOS 中,页面寄存器写入地址与通道号的对应关系为:

83H 为通道 1

81H 为通道 2

82H 为通道 3

通道 0 未用

在进行 DMA 传输,并输出页面地址时, $\overline{\text{DMAAEN}}$ 必为低电平,它与 U11 的 $\overline{\text{OE}}$, U12 的 $\overline{\text{1G}}$ 、 $\overline{\text{2G}}$ 相连,可输出低 16 位地址信息 $A_{15} \sim A_0$;同时,它还与页面寄存器的 READ 端相连,可将 74LS670 内部寄存器中的页面地址信息送到地址总线的 $A_{19} \sim A_{16}$ 上。在电路中 RA 与 DACK_3 相连, RB 与 DACK_2 相连。当通道 2 进行 DMA 传送时, DACK_2 即 RB 变为低电平,由表 12-4 知,可选中 1 号寄存器。当通道 3 进行 DMA 传送时, DACK_3 即 RA 变为低电平,由表 12-4 知,可选中 2 号寄存器,当通道 1 进行 DMA 传送时, DACK_2 和 DACK_3 必为无效(高电平),这时选中 3 号寄存器。通道 0 对应 0 号寄存器,在 PC/XT 中未用 0 号寄存器,这是因为通道 0 用来对动态 RMA 进行刷新,在这种情况下,不必使用页面寄存器。

3. DMA 应答控制电路

在 DMA 控制逻辑中,对所有想利用 PC/XT 机上的 8237A 进行 DMA 传输的外设或它们的接口电路,必须设有相应的 DMA 应答电路,实现与 8237A 进行通信联络,包括发出 DMA 请求,接收 8237A 发回的应答信号 DACK,以及能接收表示 DMA 传输结束的 $\overline{\text{EOP}}$ 信号而发出中断请求等功能。如果外设的速度较慢,还要有能向 8237A 发出插入等待状态的 READY 信号的逻辑电路。在外设希望 DMA 服务时,通过 I/O 扩展槽中的 PC 总线 $\text{DRQ}_3 \sim \text{DRQ}_1$,把请求信号送到 8237A-5 相应的 DREQ 端;在进入 DMA 服务时,8237A-5 向请求服务的设备输出回答信号 DACK,允许外部设备与内存间进行数据传输。

二、8237A 的一般编程方法

8237A 应用于不同场合时,对它进行编程的方法也不尽相同,下面以在外设与内存间进行 DMA 数据传送为例来说明它的编程方法。

1. 编程步骤

利用 8237A 实现外设与内存间的数据传送时,可按以下几步对它进行初始化编程:

- (1) 输出主清命令,使 8237A 复位。
- (2) 写入基地址和现行地址寄存器,确定起始地址。
- (3) 写入基字和现行字计数器,确定要传送的字节数。
- (4) 写入方式寄存器,指定工作方式。
- (5) 写入屏蔽寄存器。
- (6) 写入命令寄存器。

完成上述步骤后,8237A 便处于待命状态。若外设通过 PC 总线的 $DRQ_1 \sim DRQ_3$, 将有效的 DMA 请求信号送到 8237A 的某个通道的 DREQ 引脚上,就可启动该通道的 DMA 传送过程。如果系统工作于数据块传送方式,也可用软件方法启动指定通道,这就需要用到第 7 步。不过,PC/XT 不支持块传送方式。

(7) 写入请求寄存器。

2. 编程举例

在某一个系统中,用一片 8237A 设计了 DMA 传输电路,8237A 的基地址为 00H。要求利用它的通道 0,从外设(如磁盘)输入一个 1K 字节的数据块,传送到内存中 6000H 开始的区域中,每传送一个字节,地址增 1,采用数据块连续传送方式,禁止自动预置,外设的 DMA 请求信号 DREQ 和响应信号 DACK 均为高电平有效。则初始化 8237A 的程序如下:

```
DMA    EQU    00H                ;8237A 的基地址为 00H
;输出主清命令
        OUT    DMA+0DH, AL        ;发总清命令
;将基地址 6000H 写入通道 0 基地址和当前地址寄存器,分两次进行
        MOV    AX, 6000H          ;基地址和当前地址寄存器
        OUT    DMA+00H, AL        ;先写入低 8 位地址
        MOV    AL, AH
        OUT    DMA+00H, AL        ;后写入高 8 位地址
;把要传送的总字节数 1K=400H 减 1 后,送到基址计数器和当前字计数器
        MOV    AX, 0400H          ;总字节数
        DEC    AX                 ;总字节数减 1
        OUT    DMA+01H, AL        ;先写入字节数的低 8 位
        MOV    AL, AH
        OUT    DMA+01H, AL        ;后写入字节数的高 8 位
;写入方式字:数据块传送,地址增量,禁止自动预置,写传送,选择通道 0
        MOV    AL, 10000100B      ;方式字
        OUT    DMA+0BH, AL        ;写入方式字
;写入屏蔽字:通道 0 屏蔽位清 0
        MOV    AL, 00H            ;屏蔽字
        OUT    DMA+0AH, AL        ;写入 8237A
;写入命令字:DACK 和 DREQ 为高电平,固定优先级,非存储器间传送
        MOV    AL, 10000000B      ;命令字
        OUT    DMA+08H, AL        ;写入 8237A
;写入请求字:通道 0 产生请求
        MOV    AL, 04H            ;请求字
        OUT    DMA+09H, AL        ;将请求字写入 8237A,用软件方法启动 8237A 工作
```

三、PC/XT 机上的 DMA 控制器的使用

为了掌握对 8237A 的编程方法,下面再举一个在 PC/XT 机上对 8237A 进行初始化和测试的程序的例子,来说明非数据传送的 DMA 程序的编写方法。在 PC/XT 上,8237A 的

基地址为 00H。在开机后必须对 8237A 进行测试,具体方法为:先对地址为 DMA+0~DMA+7 的 8 个可读写的寄存器都写入 FFFFH,然后将它们的值读出来,看读出的值与写入的值是否相等。然后把写入值改为 0000H 后,进行同样的测试。在测试过程中,如发现读出的值与写入的值不一致,表示测试没有通过。下面是实施这种测试的程序段的主要内容:

```

DMA    EQU    00H                ;DMA 基地址
;先送命令字,禁止 8237A 工作
        MOV    AL, 04            ;命令字:禁止 DMA 控制器工作
        OUT    DMA+08H, AL       ;输出命令字到 8237A
        OUT    DMA+0DH, AL       ;发总清命令
;第一遍,将通道 0~3 的基地址和当前地址寄存器均置为 FFFFH,第二遍均置为 0000H
        MOV    DX, DMA           ;通道 0 的地址寄存器端口
        MOV    AL, 0FFH         ;AL=FFH
C8:     MOV    CX, 0008H         ;循环次数为 8
WRITE:  MOV    BH, AL            ;放进 BX 以便比较
        MOV    BL, AL           ;第一遍,AL=FFH,第二遍为 00H
        OUT    DX, AL           ;写入低 8 位
        OUT    DX, AL           ;写入高 8 位
        INC    DX               ;建立下个寄存器口地址
        LOOP   WRITE            ;写 4 个通道,8 个端口
;对通道 0 写入方式字:单字节,地址增量,允许自动预置,读传送
        MOV    AL, 58H          ;通道 0 方式字
        OUT    DMA+0BH, AL      ;写进通道 0
;设置命令字:DACK 低电平有效,DREQ 高电平有效,正常时序滞后写,固定优先权,
;允许 DMA 工作,禁止存储器到存储器操作(各通道相同)。
        MOV    AL, 00H          ;8237A 命令字
        OUT    DMA+08H, AL      ;输出到 8237A
;对通道 1~3 置方式字:单字节,地址增量,禁止自动预置,校验传输
        MOV    AL, 41H          ;通道 1 方式字
        OUT    DMA+0BH, AL      ;对通道 1 写入方式字
        MOV    AL, 42H          ;通道 2 方式字
        OUT    DMA+0BH, AL      ;对通道 2 写入方式字
        MOV    AL, 43H          ;通道 3 方式字
        OUT    DMA+0BH, AL      ;对通道 3 写入方式字
;设置屏蔽字,使 4 个通道的屏蔽位均清 0,都去除屏蔽
        MOV    AL, 00H          ;屏蔽字
        OUT    DMA+0FH, AL      ;输出到 8237A
;对通道 0~3 的地址值和计数值进行测试,看读出的值是否与写入的值(在 BX 中)相等
        MOV    DX, DMA          ;指向通道 0 地址寄存器
        MOV    CX, 0008         ;循环次数
READ:   IN     AL, DX            ;读低字节
        MOV    AH, AL           ;存入 AH

```

IN	AL, DX	;读高字节进 AL
CMP	AX, BX	;读出的值与写入的值相等吗?
JNE	STOP	;不等,则转 STOP
INC	DX	;相等,则指向下一个寄存器口地址
LOOP	READ	;测下一个寄存器
MOV	DX, DMA	;测完,DX 指向 8237A 的基地址
INC	AL	;AL←AL+1
JZ	C8	;若 AL=00H,写入此值,再测一遍
:		;若 AL=01H,结束测试
:		;继续对 8237A 进行初始化
STOP:	HLT	;如出错则停机

在以上程序中,初始化程序部分将通道 0 置为读出传输,而通道 1~3 为校验传输,这是一种虚拟传输,不修改地址,也并不真正传输数据,所以地址寄存器的值不变。在校验程序中,仅对通道 1~3 的地址寄存器的值进行测试,看其是否与写入的初值相等。

12-4 PC/XT 机的系统板

前面各章节我们已对构成微型机系统的各主要部件做了较详细的讨论,现在再通过简要介绍 PC/XT 系统的基本结构,主要分析系统板的电路图,来说明如何将各种基本部件连接起来,组成一台完整的 PC/XT 微型计算机系统。

PC/XT 系统的最基本配置包括主机箱、键盘和显示器等部件,还可配上打印机、绘图仪等其它外设,构成微型计算机系统。

主机箱是 PC 机的核心,它内部的主体是一块 8.5 英寸×12 英寸的系统板,上面装有 PC/XT 机的主要部件,此外机箱内还包含电源、扬声器和一个 10M 的硬盘驱动器及一个 360KB 的 5 英寸软盘驱动器。键盘和机箱内的扬声器可通过接插件直接与系统板相连,而 I/O 设备都不能直接与系统板相连,必须通过各自的适配器(Adapter)或接口电路板才能与系统板相连。适配器做成标准的接插件板(卡),插在系统板的扩展槽里,每块适配卡的接插件引脚都是 62 芯,引脚上的信号都符合 PC 总线的标准,这些板通过 PC/XT 的系统总线与系统板上的部件相连并进行通信。

PC/XT 机的系统板电路原理如图 12-12 所示,图中的大部分电路大家已比较熟悉,由图可以看到各种电路的关联以及如何将它们组合在一起,构成微型计算机系统的。下面分几部分对该图作简要的说明。

一、CPU 子系统

CPU 子系统由 8088 CPU、8284A 时钟产生器、8288 总线控制器、74LS373(或 8282)地址锁存器、74LS244、74LS245 数据缓冲器或 8286 数据收发器等组成,必要时可插入 8087 协处理器。8087 直接与 8088 的地址总线、数据总线和控制总线相连,这样 8087 可跟踪和译码由 8088 取出的每条指令。通常从存储器中取出的指令都由 8088 执行,当取到换码指

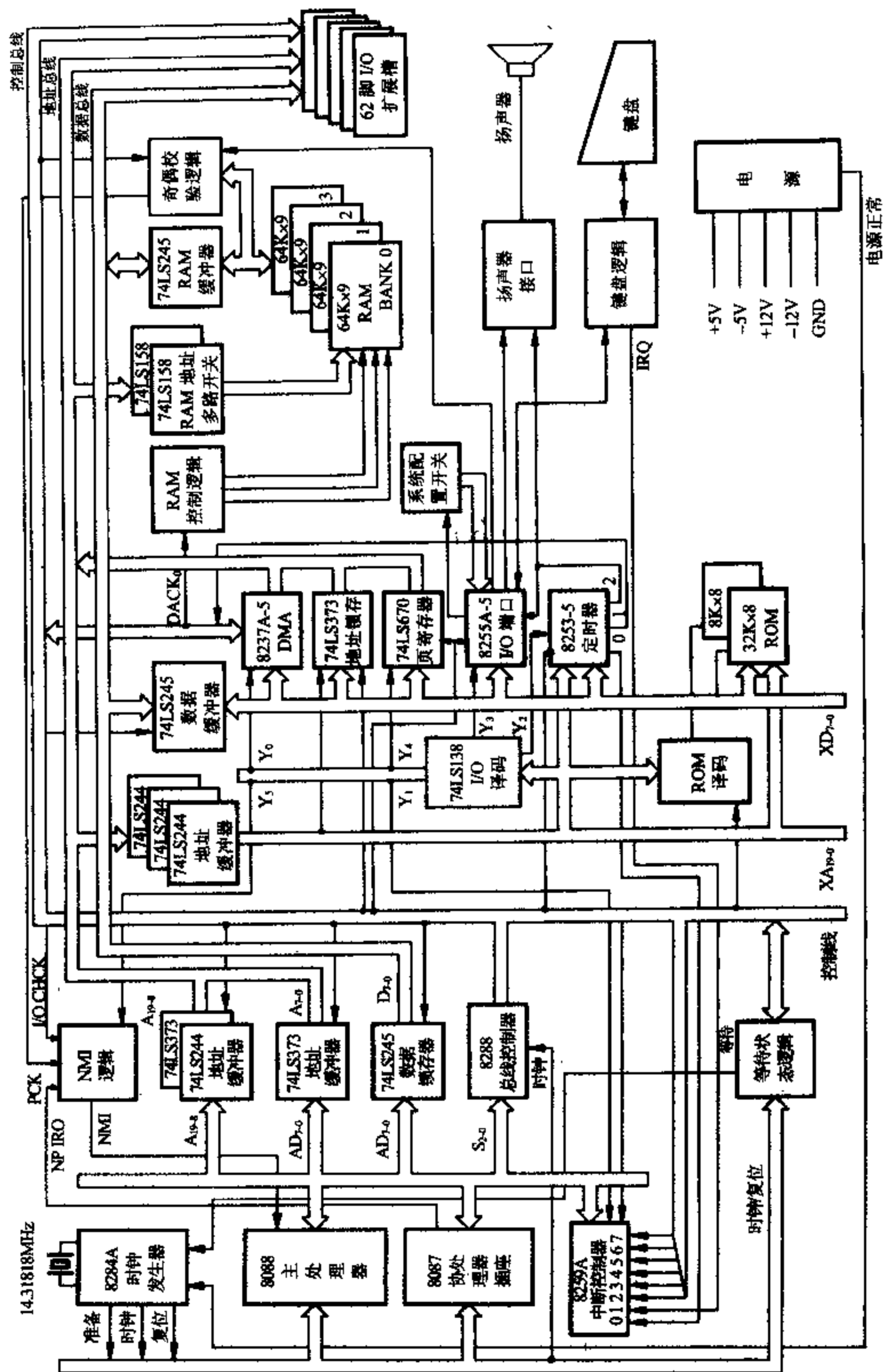


图 12-12 PC/XT 机系统板电路原理图

令时,就由 8087 执行。

8284A 时钟产生器将晶体振荡器产生的频率为 14.31818MHz 的信号进行分频后,向 8088 CPU 及微型计算机系统提供所需要的时钟脉冲信号、准备好信号和系统复位信号。其中送到 8088 CPU 的时钟信号的频率为 4.77MHz,这是 CPU 的工作脉冲信号。对慢速的存储器和 I/O 设备,则要利用等待状态逻辑电路产生等待信号,插入 T_w 状态,以延长数据的存取时间。

在 PC/XT 系统中,8088 CPU 工作于最大模式,系统的控制信号都通过总线控制器 8288 送到系统的控制总线上。地址信号分三部分送上地址总线, $A_{19} \sim A_{16}$ 经地址锁存器 74LS373 锁存后送到地址总线上, $A_{15} \sim A_8$ 在总线周期内呈现的都是地址信号,所以可经过缓冲器 74LS244 送到地址总线上,地址数据线 $AD_7 \sim AD_0$ 上的信号要经过分离后才能送出。在总线周期内,先出现的是低 8 位地址信号 $A_7 \sim A_0$,它经地址锁存器 74LS373 锁存后送到地址总线上;后出现的是 8 位数据信号 $D_7 \sim D_0$,它经过双向数据缓冲器 74LS245 驱动后送出。这 3 组控制总线、地址总线 and 数据总线信号被直接送到 I/O 扩展槽上,组成 62 芯的 PC 系统总线信号,在那儿还要经进一步驱动后才能使用。这样,通过 PC 总线,8088 就能和扩展槽上的 I/O 端口及 RAM、ROM 进行通信。

3 组系统总线信号中的控制总线还可直接控制系统板上的其它电路工作,而 20 位地址总线则需经过 3 片三态缓冲器 74LS244 进一步驱动成为 20 根外部地址总线 $XA_{19} \sim XA_0$,8 根数据总线需经 1 片双向三态缓冲器 74LS245 驱动为外部数据总线 $XD_7 \sim XD_0$ 之后,再送到系统板的其它接口电路和存储器上去。为简单起见,前而介绍接口电路时,把信号前的“X”省去了,统称为地址总线 $A_{19} \sim A_0$ 和数据总线 $D_7 \sim D_0$ 。

二、接口部件子系统

接口部件子系统由一片 I/O 译码电路 74LS138 和一组可编程接口芯片组成,74LS138 译码器根据系统要求统一进行编址,端口地址分配表见第六章的表 6-3。译码器输出端 $Y_0 \sim Y_5$ 依次分别连接以下接口芯片:8237A-5 DMA 控制器、8259A 中断控制器、8253-5 定时器/计数器、8255A-5 通用 I/O 接口芯片、74LS670 DMA 页寄存器及 NMI 不可屏蔽中断控制电路。这些接口芯片的主要功能分述如下:

1. 8237A-5 DMA 控制器

它提供 4 个 DMA 通道,其中通道 0 用作动态 RAM 刷新,通道 1 为用户保留,通道 2 用作软盘驱动器数据传送,通道 3 用作硬盘驱动器数据传送。

2. DMA 页寄存器

在 PC/XT 机中,用 8237A-5 控制 DMA 传输。系统有 20 根地址线,8237A-5 只能管理低 16 位地址线 $A_{15} \sim A_0$,其中 $A_7 \sim A_0$ 直接由 8237A-5 输出, $A_{15} \sim A_8$ 自 8237A-5 的数据线输出,经 74LS373 锁存后形成,而高 4 位地址 $A_{19} \sim A_{16}$ 则由页面寄存器 74LS670 产生,详见图 12-11。

3. 8259A-5 中断控制器

8259A-5 中断控制器可提供 8 级中断,其中 0 级中断输入端 IR_0 (图中用“0”表示)与 8253-5 计数器 0 的输出端相连,接收对电子钟进行计时的中断请求,每秒钟向 CPU 提出

18.2 次中断请求。1 级中断输入端与键盘接口电路相连,接收键盘的中断请求,每当机器收到一个键盘的扫描码时,键盘接口电路就向 8259A 的 IR_1 引脚上送出一个有效的中断请求信号。8259A-5 的 2~7 级中断输入端连接到 I/O 扩展槽上,其中 2 级为用户保留,3 级为串行口 2(COM2)、4 级为串行口 1(COM1)、5 级为硬盘、6 级为软盘、7 级为并行打印机所用。

4. 8253-5 定时器/计数器

8253-5 定时器/计数器内部有 3 个计数器通道,各计数通道功能如下:

通道 0 经编程用作定时器,每秒钟产生 18.2 次输出信号,送到 8259A 中断控制器的 IR_0 输入端,为系统提供稳定的时间基准。8088 对时间基准进行计数,就可用来计时。

通道 1 用作动态 RAM 刷新定时,大约每隔 $15\mu s$ 产生一次输出信号,请求执行动态 RAM 刷新操作。在 PC/XT 机中,用 8237A-5 DMA 控制器的通道 0 对动态 RAM 进行刷新。8253-5 通道 1 的输出信号送往 8237A-5 的通道 0,定时产生刷新请求信号。通道 2 向扬声器接口电路输出方波,为扬声器发声提供基本的音调。用程序设定通道 2 输出波形的频率和延续时间,就能控制扬声器的音调和发声时间的长短。

5. 8255A-5 通用接口芯片

8255A-5 通用接口芯片具有 3 个 8 位的端口 A、B 和 C,各端口功能如下:

系统刚加电时,处于自检方式,端口 A 用来输出当前检测部件的标志。在平时端口 A 用来读取键盘的扫描码。

端口 B 工作于输出方式,用来输出一些控制信号。如:允许 RAM 进行奇偶校验信号 $\overline{ENB RAM PCK}$ (低电平有效)、允许 I/O 通道校验信号 $\overline{ENABLE I/O CK}$ (低电平有效)、清除键盘数据信号和控制扬声器发声的信号。

端口 C 工作于输入方式,其中低 4 位用来读取系统配置 DIP 开关的设置情况,高 4 位用来读取系统板和 I/O 扩展板上的 RAM 的奇偶校验情况及读取扬声器的状态。若系统板上的 RAM 奇偶校验错,而 $\overline{ENB RAM PCK}$ 为有效的低电平时,则奇偶校验状态信号 PCK 变高,它通过 NMI 逻辑向 CPU 发出不可屏蔽中断请求。若 I/O 通道上的存储器出现奇偶校验错,而 $\overline{ENABLE I/O CK}$ 为有效的低电平时,则 I/O 通道奇偶校验状态信号 I/O CHCK 变高,它也可通过 NMI 逻辑向 CPU 发出不可屏蔽中断。

三、存储器子系统

在 PC/XT 机中,8088 CPU 管理 20 位存储器地址信号,系统具有 1M 存储空间,每个存储单元存放 1 个字节数据,所以存储容量为 1M 字节,即 1024KB,地址范围为 00000H~FFFFFH。1M 字节的存储空间分为三个区域:RAM 区、I/O 缓冲 RAM 区和 ROM 区。存储空间的地址分配如图 12-13 所示。

各区域的使用情况说明如下:

1. RAM 区

从 00000H~9FFFFH 范围内的 640K 内存区域为 RAM 区,也称为常规内存区,用来运行操作系统和应用程序,或用作数据区。其中 00000H~003FFH 的 1KB 内存区域为 BIOS 数据区,存放中断矢量表。

1983 年 PC/XT 机刚诞生时,系统板上最多可安装 256K RAM,扩展板上安装 384K RAM。从图 15-1 已经看到,系统板的左下方有 4 排 RAM 存储器,每排包含 9 块动态 RAM 芯片。若 RAM 芯片均由 $64K \times 1$ 的 2164 组成时,9 块芯片可构成 $64K \times 9$ 的存储器,其中 8 位为数据位,1 位为奇偶校验位,4 排 RAM 芯片构成 256KB 存储器(和 1 位奇偶校验位)。若 RAM 芯片由 $16K \times 1$ 的 2118 组成时,则 4 排 RAM 芯片可构成 64KB 的存储器(和 1 位奇偶校验位)。

1985 年后在改进的 PC/XT 及其兼容机上,640K 常规内存均安装在系统板上,仍由 4 排存储器芯片组成,每排为 9 片。其中有 2 排仍采用 $64K \times 1$ 的芯片,组成 128KB RAM,另外 2 排采用 $256K \times 1$ 的芯片,组成 512KB RAM。4 排存储芯片总共组成 640KB RAM(含奇偶校验位)。

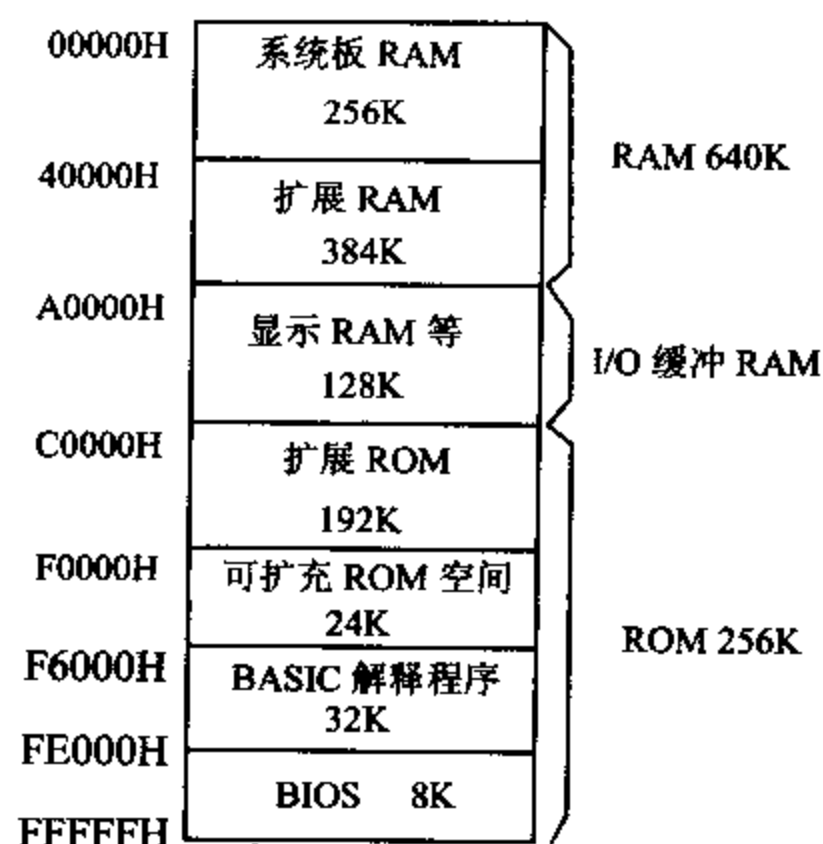


图 12-13 PC/XT 机的内存分配图

2. I/O 缓冲 RAM 区

从地址 A0000H 开始的 128K 存储空间称为 I/O 缓冲 RAM 区域,用作字符/图形显示缓冲区,保存显示屏幕上要显示的信息。对于不同的显示适配卡,使用的显示 RAM 区域各不相同。

IBM 单色显示器适配卡 MDA(Monochrome Display Adapter)只能提供字符显示功能,每屏最多显示 $80 \times 25 = 2000$ 个字符,每个字符占用两个字节,其中一个字节存放该字符的 ASCII 码,另一个字节规定字符的属性,如亮度、背景、前景和闪烁等。单显使用的显示缓冲器容量为 4K,地址范围为 B0000H~B0FFFH。

IBM 彩色显示适配卡 CGA(Color Graphics Adapter)能提供彩色图形显示功能。当它工作于 640×200 的高分辨率单色显示方式时,需占用 $640 \times 200 / 8 = 16000$ 字节内存单元(每单元存放 8 个像素),即约需 16KB 的显示内存,显示缓冲器的地址范围为 B8000H~BBFFFH。若用 CGA 卡显示彩色图形时,其分辨率还将降低。

对于高档 PC 机上使用的增强型图形适配卡 EGA(Enhance Graphics Adapter)、视频

图形显示阵列卡 VGA(Video Graphics Array)和 Super VGA 卡等高分辨率图形适配卡,则要用到全部 128K 显示 RAM 区。各种显示 RAM 芯片分别被安装在各显示适配卡上。

3. ROM 区

存储空间的最后 256K 是系统 ROM 区,这个区域安排的都是只读存储器 ROM。地址范围为 C0000H~EFFFFH 的 192KB 区域称为扩展 ROM 区,用来安装系统中的控制 ROM 和扩展 ROM。其中 C0000H~C7FFFH 为高分辨率显示适配器卡的控制 ROM,它作为图形显示卡上的 BIOS,用来管理这些高档显示器的工作。可见,显示适配卡工作时,既要用到显示 RAM,又要用到扩展 ROM。各种显示卡占用的存储空间如表 12-5 所示。

表 12-5 各种显示卡使用的存储器地址分配表

显示卡种类	显示 RAM 空间(H)	板上显示 RAM 总容量	扩展 ROM 空间(H)
MDA	B0000~B0FFF(4K)	4K	无
CGA	B8000~BC000(16K)	16K	无
EGA	A0000~BFFFF(128K)	256K	C0000~C3FFF(16K)
VGA	A0000~BFFFF(128K)	256K	C0000~C5FFF(24K)
Super VGA	A0000~BFFFF(128K)	512K~1024K	C0000~C7FFF(32K)

由上表可见,对于高档显示器,所需的显示 RAM 容量可能比 128K 大得多,如 Super VGA 卡上需用 512K~1024K 显示缓冲器,而系统仅提供 128K 显示 RAM 空间。实际使用时,可将显示缓冲器分成 8 个页面,每次只显示其中一个页面(128K)的内容,在显示一个页面的同时,也能对别的页面实施处理。显示完一个页面后快速切换到别一个页面,看起来仿佛各个页面同时在显示一样,这样就解决了显示 RAM 空间小和实际用显示缓冲器容量大的矛盾。

扩展 ROM 中地址为 C8000H~CBFFFH 的 16K ROM 空间为硬盘驱动程序所用,它被安装在硬盘驱动器适配卡上。其余的扩展 ROM 区地址为 CC000H~EFFFFH,系统中尚未使用,用户可用来安装固化的 ROM 程序。上述 ROM 都安装在 I/O 扩展卡上。

系统中最后 64K 存储器区域是基本系统 ROM 区,它们均可被安装在系统板上。地址范围为 F0000H~F5FFFH 的区域为 24K 的可扩充 ROM 空间。通常系统板上都安装 40KB 的基本 ROM,从 F6000H~FDFFFH 地址范围内存放 32KB 的 BASIC 解释程序,从 FE000H~FFFFFFH 地址范围内存放 8KB 的基本输入输出系统程序(ROM BIOS)。40KB 基本 ROM 由两块 ROM 芯片组成,一片采用 2764 EPROM,容量为 8KB,内装 BASIC 解释程序的前 8KB。另一片采用 27256 EPROM 芯片,其容量为 32KB,内含 BASIC 解释程序的后 24KB 和 8KB 的 BIOS。这两块 ROM 芯片的管脚是兼容的,都是 28 个引脚。当初 BASIC 解释程序是为无磁盘的用户所设置的,现在已没有用处,仅仅是为了兼容性才保留这个存储区域。

习 题

- 1. 一般 DMA 控制器应具有哪些基本功能?

2. 什么是 8237A DMA 控制器的主态工作方式？什么是从态工作方式？在这两种工作方式下，各控制信号的功能是什么？

3. 8237A DMA 控制器的当前地址寄存器、当前字节寄存器、基地址寄存器和基字节寄存器各保存什么值？

4. 8237A 具有几个 DMA 通道？每个通道有哪几种传送方式？各用于什么场合？什么叫自动预置方式？

5. 8237A 可执行哪几条软件命令？

6. 根据图 12-12、表 12-3 和表 12-4，说明如何产生 DMA 传送的 20 位存储器地址。

7. 若 8237A 的端口基地址为 000H，要求通道 0 和通道 1 工作在单字节读传输，地址减 1 变化，无自动预置功能。通道 2 和通道 3 工作在数据块传输方式，地址加 1 变化，有自动预置功能。8237A 的 DACK 为高电平有效，DREQ 为低电平有效，用固定优先级方式启动 8237A 工作，试编写 8237A 的初始化程序。

8. 设 8237A 的基地址为 DMA，要求利用通道 2，将软盘上 20 个扇区的数据传送到内存中地址为 2000：1000H 开始的单元中，软盘上每扇区的字节数为 1024。试编写有关的程序段。

第十三章 32 位微机基本工作原理概述

前面我们对以 8086 为 CPU 的计算机的工作原理、编程方法和 I/O 接口技术做了详细的描述,为掌握微型计算机的基本工作原理和应用技术奠定了坚实的基础。但目前广泛应用的都是 32 位高档奔腾(Pentium)微型计算机,那么这些 32 位计算机的基本结构、组成和工作原理以及接口电路又是怎样的呢?它们与以 8086 为 CPU 的微型计算机之间又有什么联系呢?本章将概要性的介绍这些内容。

自从 PC 计算机于上世纪 70 年代诞生以来,短短几十年已有了飞速发展。目前以 Intel Pentium 4 为处理器的微型计算机的功能十分强大,但它们也都是从 8086 一步步发展而来的,它们的指令系统和计算机的基本 I/O 端口都与前者兼容。因此,有了 16 位机的基础再来学习 32 位机就比较容易入手了。整个 Intel 的微处理器分为以下两大类:

一类是 80X86 处理器(8086、80286、80386、80486)及 Pentium 处理器。

另一类是 IA-32 结构处理器。它又可分为两种:一种是 P6 系列处理器,包括 Pentium Pro、Pentium II 及 Pentium III 微处理器;另一种是基于 Net-Burst 结构的处理器,包括 Pentium 4(简称 P4)以及 Intel Xeon Processor 处理器。

从 80386 开始,微处理器的数据总线达到 32 位,我们把它们称为 32 位机。比起 8086 CPU,各类具有 32 位数据总线的微处理器的工作原理要复杂得多。可以说,微处理器从 8 位到 16 位主要是加宽了数据总线和地址总线,增加了简单的内存分段功能;而 80386 与 8086 相比,就不仅仅是外部数据总线由 16 位变为 32 位,地址总线由 20 位变为 32 位,增加了数据宽度和扩大了直接寻址范围的问题,而是从体系结构到工作模式以及对内存的管理等方面都有了根本性的改变。今天广泛使用的 P4 处理器,与 80386 相比,在性能上又有了极大的提高,但它的工作模式,内存管理的思想等方面与 80386 是类似的,只是扩充了许多功能。当然,比起 P4 处理器来说,80386 毕竟要简单得多,所以本章仍从 80386 入手来介绍 32 位机,必要时会将它与其它 32 位处理器作比较。

本章首先对从 80386 开始的各类 32 位微处理器的结构和功能作一个简单概述,并列举了它们的 4 种工作模式;接着讨论 32 位机的寄存器组成和功能,在此基础上较系统地讲述了保护模式下的存储器管理技术;然后介绍 80386 的寻址方式、部分整数指令的格式和功能,并给出汇编语言程序设计的几个实例;最后通过对几种典型主板结构特别是最新的 P4 计算机的主板结构的描述,使大家对 32 位微型计算机的接口技术有一个概要性的了解。

13-1 32 位微处理器的结构与工作模式

一、32 位微处理器结构简介

1. 80386 CPU

80386 是 IA-32 结构系列中的第一个 32 位微处理器。它的数据总线是 32 位,内部寄存器和操作也是 32 位;外部地址总线 32 位,能直接寻址 4GB 物理地址空间($2^{32}=4\text{GB}$),并引入了新的分段分页概念;加上 80387 协处理器后可以处理浮点数。

80386 由 6 大部分组成,内部结构框图如图 13-1 所示。各部分功能如下:

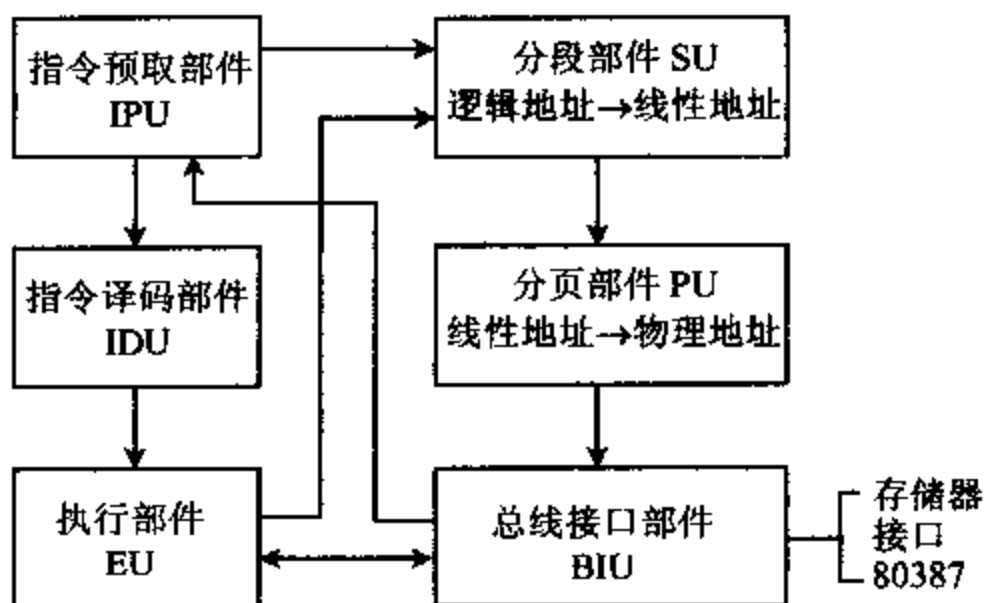


图 13-1 80386 的内部结构框图

(1) 总线接口部件(Bus Interface Unit, BIU)

它是 386 与外界之间联系的高速接口,产生和接受访问存储器和 I/O 端口所需的地址、数据及命令信号,也实现 80386 和 80387 之间的协调控制。

(2) 指令预取部件(Instruction Prefetch Unit, IPU)

它将存放在存储器中的指令经 BIU 取到 16 字节长的预取指令队列中,并向指令译码部件输送指令。CPU 在执行当前指令时,指令译码部件将对下一条指令进行译码,一旦预取指令队列空,又会从存储器中取出指令,将指令队列填满。

(3) 指令译码部件(Instruction Decode Unit, IDU)

从 IPU 中取出指令进行译码分析,然后将其放入 IDU 中的译码指令队列中,供执行部件使用。该队列能容纳 3 条已译码的指令,一旦队列有空,就会从预取指令队列中取出下一条指令进行译码分析。

(4) 执行部件(Execution Unit, EU)

执行部件 EU 包含算术逻辑运算单元 ALU,8 个 32 位的通用寄存器,一个 64 位的多位移位加法器,执行数据处理和运算操作。此外还包含 ALU 控制和保护测试部件,ALU 控制部件用以实现有效地址的计算,并提供乘除法加速等功能,保护测试部件用来检测所执行

的指令是否符合存储器分段分页规则。

(5) 分段部件(Segmentation Unit, SU)

按指令要求,分段部件 SU 将指令中的逻辑地址转换成线性地址。8086 中,每段地址的容量固定不变,都是 64K。而在 80386 中,每段地址容量可变,从 1 字节~4GB,使用灵活方便,但管理起来比较复杂。

(6) 分页部件(Paging Unit, PU)

分页部件 PU 将分段部件 SU 产生的线性地址转换成物理地址,每页容量为 4KB。当系统中不使用分页功能时,线性地址就是物理地址。有了物理地址后,总线接口部件就可以实现访问存储器和输入输出操作。

2. 80486 CPU

80486 也是 32 位微处理器,它基本沿用了 80386 的体系结构。与 80386 相比,芯片内部增加了增强型 80387 协处理器和高速缓存 Cache。前者也称为浮点部件(Floating Point Unit, FPU)。片内 FPU 的显著特点是引线短,其内部数据总线被加宽至 64 位,处理速度比 80387 提高了 3~5 倍。片内 Cache 为频繁访问的数据和指令提供快速的局部存储。

高速缓存 Cache 的设计思想是:对典型程序运行情况分析表明,程序产生的地址往往集中在存储器逻辑地址空间的很小范围内,即总是对某一部分地址进行频繁访问,而其它地址则用得很少。据此,可在主存和 CPU 之间设置一个速度很高、容量较小的 SRAM,把主存中经常被 CPU 访问的那一部分内容复制到 Cache 中,使 CPU 在一段时间内对 Cache 快速读取数据。主存可采用价格低的 DRAM 构成,它们容量大但速度较慢。这样既降低了成本,又提高了速度。80386 的 Cache 设在 CPU 芯片之外,速度要慢一些。

此外,80486 还采用精简指令集计算机(Reduce Instruction Set Computer, RISC)技术,使每个时钟可执行 1.2 条指令。

3. Pentium 微处理器

Pentium 也是 32 位微处理器,其芯片内部 ALU 和通用寄存器仍是 32 位,但外部数据总线是 64 位,与 80486 相比,在结构上有了很大改进,主要体现在:

(1) 超标量流水线结构

超标量流水线(Superscaler Pipeline)设计是 Pentium 的核心技术。一个处理器中有多个执行单元时,Intel 称之为超标量结构,这些执行单元称为流水线。Pentium 处理器内部有称之为“U”和“V”的两条流水线,每条流水线都有自己的 ALU、地址生成逻辑及 Cache 接口电路,在每个时钟周期内可执行两条整数指令。每条流水线工作时,分为指令预取、指令译码、地址生成、指令执行和回写 5 个步骤。当一条指令执行完预取步骤后,流水线就可以对另一条指令操作,大大地提高了指令的执行速度。

(2) 重新设计的浮点部件

Pentium 的浮点运算分为 8 级流水线,使每个时钟周期能完成 1~2 个浮点操作, Pentium 的浮点单元 FPU 对一些常用指令如 ADD、MUL 和 LOAD 等运算采用了新的算法,使运算速度至少提高了 3 倍,在许多应用程序中利用指令调度和流水线执行,可使速度提高 5 倍以上。

(3) 独立的指令 Cache 和数据 Cache

它使 Pentium 中的数据和指令的存取分开进行,减少了冲突,提高了性能。

(4) 指令固化

将常用指令如 MOV、INC、PUSH、JMP 等改用硬件实现,从而提高指令的执行速度。

(5) 分支预测

Pentium 内部设置了一个分支目标缓冲器(Branch Target Buffer,BTB),这是一个小的 Cache,用来动态预测程序的分支。当执行某条指令导致程序分支时,BTB 记忆下条指令和分支目标的地址,并用这些信息预测该条指令再次产生分支时的路径,预先从该处预取指令,保证流水线的指令预取步骤不会空置。

4. Pentium Pro 处理器

它也被称为高能奔腾,与 Pentium 芯片相比,增加了如下功能:

(1) 一个封装内安装了两个芯片

CPU 内核与 256KB 的二级 Cache 封装在一个芯片内,构成多芯片模块,使二级 Cache 经全速总线与 CPU 内核相连,从而提高了程序的执行速度。

(2) 乱序执行和分支预测技术

高能奔腾处理器并不完全按顺序执行指令,而是采用指令动态执行技术,将多条指令译码后放在指令池(Instruction Pool)中,准备运行。当一条正在执行的指令因等待操作未能执行完时,处理器就从指令池中找出其它的指令来执行。这样,指令可不完全按顺序执行,但执行结果仍按原有顺序产生。这种做法称为“乱序(out of order)执行”。利用数据流分析的方法,可找出最佳指令执行顺序。由于指令池较大,多条指令执行分支会产生冲突,处理器的分支预测技术可对程序的不同分支进行预测,并根据预测结果调整指令的执行顺序,从而使指令的执行效率更高。

(3) 超流水线和超标量技术

Pentium Pro 具有 3 路超标量结构,并行执行指令的能力比 Pentium 强。它又具有 14 级超流水线(Hyper Pipelined)结构,将任意一条指令的全部执行过程分成一连串的命令步(级),进一步提高了处理器的并行处理能力。

5. Pentium II 处理器

Pentium II 处理器简称 P II 或“奔腾 II”,它把多媒体扩展(Multi Media Extension, MMX)技术融合到高能奔腾处理器中,使它既保留了 Pentium Pro 原有的强大处理能力,又增强了 PC 机在三维图形、图像和多媒体方面的可视化计算功能和交互功能。P II 处理器采用了如下几种先进技术,使它在整数运算、浮点运算和多媒体信息处理等方面有具有较强的功能。

(1) MMX 技术

采用单指令多数据(Single Instruction Multiple Data, SIMD)技术,其核心是一条指令能并行执行多个数据的相同操作。

引入了新的数据类型。内部含有 8 个 64 位的寄存器 mm0~mm7,64 位可存放 8 个字节、4 个字或 2 个双字的数据,能同时并行执行 8 个字节、4 个字和 2 个双字的整数运算。视频会议、活动图像、二维或三维图形等几乎所有具有重复性和顺序性的整数计算的运用程序都可用 MMX 技术,但它只能处理定点数,不能完成浮点操作。

采用饱和运算这一新的运算方式。它是相对环绕运算而言的,在环绕运算方式下,上溢(8 位数 > 255)或下溢(8 位数 < -255)时结果数值被截断,仅保留结果的低位部分,显然误

差很大。在饱和运算方式下,如结果上溢或下溢,就将其数值限定在该数类型所允许的上限或下限上。这种运算方式在彩色图像运算中,使得计算出来的色彩在溢出情况下仍可保持“全黑”或“全白”,而不会产生环绕运算方式下的彩色反转现象。

(2) 动态执行技术

采用下列 3 种处理技巧相结合的动态执行技术,帮助处理器有效地处理多重数据。

采用多分支预测算法,处理器读指令时,同时查看以前的指令,从而对数据流向做出分析判断;使用数据流分析方法,查看译好码的指令,决定指令的最佳执行顺序;当处理器要同时处理多条指令时,利用推测执行技术,最大限度地提高并行执行指令的能力。

(3) 双独立总线结构

原先的处理器采用一条数据总线同时与主存、二级 Cache 和 PCI 总线相连,任何时候只能访问一个设备。采用双独立总线结构后,将这两条总线的一条连到 Cache,另一条连到主存,处理器可同时使用这两条总线,使 Pentium II 处理器的吞吐量是单一总线的两倍,而且,二级 Cache 的速度也是单一总线的两倍。二级 Cache 是指指令 Cache 和数据 Cache 统一于一体的 Cache。

6. Pentium III 处理器

设置了 8 个新的单精度浮点寄存器 xmm0~xmm7,增加了 70 条数据流单指令多数据扩展(Streaming SIMD Extensions,SSE)指令,能同时处理 4 个单精度浮点数,每秒达到 20 亿次的浮点运算速度,使 PIII 处理器在三维图像处理、语音识别、实时视频压缩及视频编解码等方面大有用武之地。但 SSE 指令还不能处理双精度浮点数和压缩整型数操作。

7. Pentium 4 微处理器

P4 处理器是一种基于新的 Net Burst™微结构的新一代 IA-32 结构处理器。ALU 运行在 2 倍的处理器频率上,有很高的时钟速度,主要具有如下特点:

(1) 采用超流水线技术(Hyper Pipelined Technology)

具有 20 条流水线,指令流水线深度达到 20 级,突破台式 PC 机和服务器的时钟速率。

(2) 先进的动态执行技术

(3) 增强型预测分支能力

(4) 流 SIMD 扩展 2(Streaming SIMD Extension 2,SSE2)技术

- 在不增加寄存器的情况下,增加了 2 个压缩浮点数操作。

- 可将 128 位的数据分别当成 16 字节、8 字、4 双字、2 个 4 字型的整型数来处理,在图像处理时可以并行操作,大大提高了处理速度。

- 增加各种数据混合(Shuffle)也就是寄存器数据交叉操作,以及数据高速缓存操作。

- 数据的类型可以是:4 个单精度浮点数(SSE),2 个双精度浮点数(SSE2),16 字节,8 字,4 个双字,2 个 4 字。

这些技术非常适合应用于 3D 图形渲染、语音识别、视频编码解码和数据加密(Encryption)等场合。

二、32 位微处理器的工作模式

80386 有 3 种工作模式,它们分别是实模式、保护模式和虚拟 8086(V86)模式,3 种工作

模式可以互相转换。从 Intel 80386 SL 处理器开始,在这 3 种模式之外又增加了一种系统管理模式(System Management Mode,SMM)。下面先介绍这 4 种工作模式,然后讨论 4 种模式之间的关系。

1. 实模式(Real-Address Mode)

当 32 位机工作于实模式时,有如下特点:

(1) 在实模式下,80386 只相当于一个快速的 8086,8086 的程序代码可以不加修改地在 80386 上运行,只是运算速度更快了。以 80386 为 CPU 的 PC 机上的 MS DOS 操作系统就是在实模式下运行的。

(2) 只有 1MB 的内存寻址能力,32 位地址线中只有低 20 位地址有效。

(3) 只支持单任务工作方式,不支持多任务方式。

(4) 80386 设置了 4 个优先级或特权级:0~3 级,其中 0 级为最高级。在实模式下,只能在优先级 0 下工作。

2. 保护模式(Protected Mode)

在这种模式下,80386 才能充分发挥它的高性能特点。保护模式下的内存管理等显得特别重要,也很复杂,后面将专门讨论。这里先介绍保护模式的特点,接着讨论与保护模式有关的一些重要的基本概念。

(1) 保护模式的特点

- 在保护模式下,80386 采用全新的分段和分页内存管理技术,不仅允许直接寻址 4GB 的内存空间,而且允许使用虚拟存储器,将磁盘等存储设备有效地映射到内存,使逻辑地址空间大大超过实际的物理地址空间,让用户感到主存的容量特别大,可达到 64TB(64MM)之多。

- 支持多任务工作方式。

- 可使用 0~3 级(优先级)保护功能,实现程序与程序之间,用户与操作系统之间的保护与隔离,为多任务操作系统提供优化支持。在 386 PC 机上运行的 WINDOWS、OS/2、UNIX 和 XENIX 操作系统都工作在保护模式下。

(2) 多任务

所谓多任务是指一台计算机可以同时干几件事。例如一台 386 PC 机,可以进行文字处理或图像处理,而后台却在进行科学计算或打印表格等。我们把这些事情定义成不同的任务(Task),字处理是 386 要完成的一个任务,科学计算则是另一个任务。386 可以支持多任务,但不支持并发的多任务,即在同一时刻不能有两个任务在同一微处理器中执行。

8086 只支持单个任务。这在硬件设计上,在程序和数据的安全性方面基本没做什么工作,特别是操作系统的安全性没有保障。如 DOS 操作系统下的系统功能调用(INT 21H)或用户设计的程序很可能破坏整个操作系统,这在单任务下是可以忍受的,最多在操作系统瘫痪之后,关机后再开机,重新装入操作系统,但在多任务下,是绝对不允许的。

(3) 优先级

为了满足多任务的需求,80386/80486 引入了优先级(Privilege Level)的概念。在计算机中,每个存放程序 and 数据的存储器段都被赋予不同的优先级。80386 共设置了 4 个优先级,用 0~3 级来表示。0 级为最高级,3 级为最低级。优先级实际上是某一任务使用微处理器资源的权利。0 级任务可以使用整个处理器的资源,一般操作系统(Operating System,

OS)的核心部分被赋予0级权利,如有关存储器管理、保护和访问控制等不太会被改变的程
序被赋予0级特权。1级赋予操作系统中可能改变的大部分程序,如外设驱动程序、系统服
务程序等。2级用来保护一些子系统,如数据库管理系统、办公自动化系统等。一般用户的
应用程序等只拥有3级权利,也称为用户级。图13-2是优先级的划分示意图。

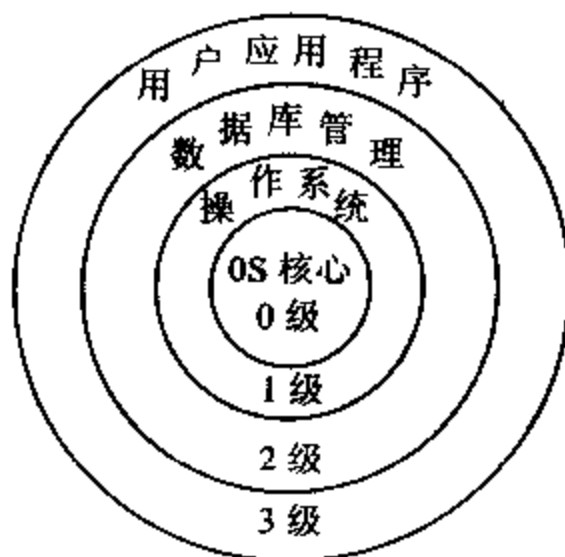


图 13-2 80386 的优先级划分

优先级也称为保护环,它的引入较好地解决了多任务环境下各任务间的相互干扰和冲突问题。如前所述,操作系统的核心程序优先级最高,它可以访问其它所有存储段的程序和数据,但别的级别的程序不能随便访问它。这样有力地保护了操作系统核心部分的安全。任务和优先级这两个概念总是密切相关的,任务可由不同优先级的代码执行。

(4) 门

有了保护机制,优先级较低的程序就不能调用优先级高的程序了,否则会产生“异常”。但这样一来,也会禁止用户从操作系统得到必要的服务。为了解决这个问题,80386 专门设置了一些合法的入口点,来允许较低级的程序调用较高级的程序。对这种入口点的访问必须通过一种特殊的机制——门(Gate)来实现。门指向某个优先级高的程序代码所规定的入口点,所有优先级低的程序要调用优先级高的程序,只能通过门重定位,进入门所规定的高级程序入口点,避免低级程序随意调用高级程序。门分为调用门、中断门、自陷门和任务门,功能比较复杂。

(5) 中断和异常

在保护模式下,由处理器外部事件产生的硬件中断称为中断,它分为不可屏蔽中断和可屏蔽中断两类。软件中断调用不归类为中断,而归类于异常。异常(Exception)是由微处理器执行某一条指令期间检测到的一种错误或无法解决的问题而产生的。32 位机定义了多种异常,每种异常用一个矢量号来标识,中断号也由矢量号来标识。根据异常或中断矢量号,从中断描述符表(IDT)中为给定的异常或中断选择相应的处理程序。

3. 虚拟 86 模式(Virtual 86 Mode)

用 8086 编制的程序,虽然能在实模式下正常执行,但在实模式下 80386 不支持多任务,因而许多用 8086 编写的程序都不能在 386 的多任务环境下正常执行。为了解决这一问题,80386 增加了虚拟 86 模式,也称为 V86 模式,使在多任务环境下,大量用 8086 编写的程序也能正常执行。

在 V86 模式下,支持保护机制,也支持内存的分页管理,并可进行任务切换,又与 8086

兼容。内存寻址空间仍为 1MB,段地址的计算方法与 8086 一样。

4. 系统管理模式(System Management Mode,SMM)

系统管理模式也是一种存储器管理模式,它从 Intel 80386 SL 处理器开始产生,成为标准的 IA-32 结构特点。该模式可以使系统设计人员实现与特定的平台有关的高级管理功能,例如电源管理和系统安全方面的功能。当外部系统管理中断(SMI#)引脚被触发,或者从先进可编程中断控制器(Advance Programable Interrupt control,APIC)接收到一个系统管理中断(SMI#)时,处理器便进入系统管理模式。处理器首先保存当前运行的程序和任务的状态,然后切换到一段独立的地址空间,去执行系统管理模式指定的代码。当从 SMM 模式返回时,处理器将回到响应系统管理中断之前的工作状态。

5. 四种模式的关系

四种工作模式之间的转换关系如图 13-3 所示。

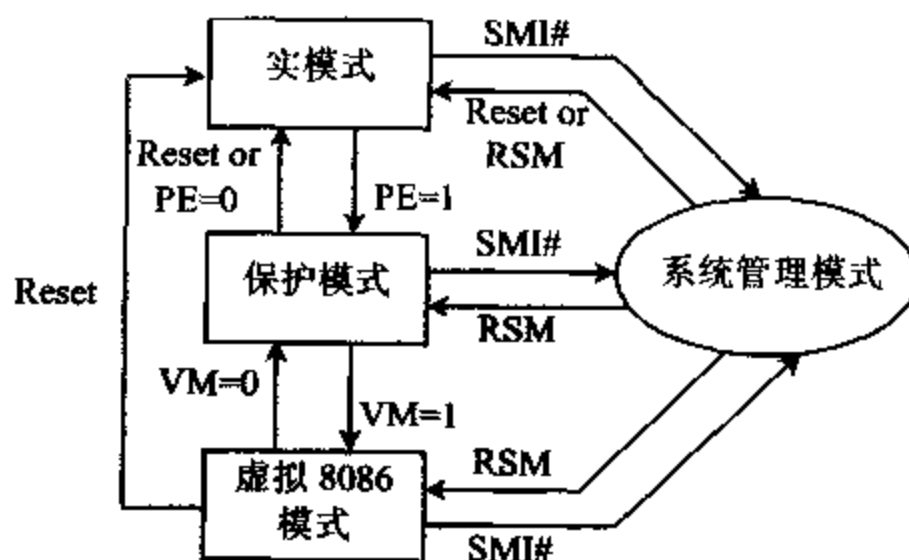


图 13-3 4 种模式之间的转换关系

80386 刚加电或系统复位后,便进入实模式方式,它为 386 进行保护模式所需要的数据结构做好各种配置和准备。修改控制寄存器 CR0 中的保护模式允许位 PE,使 PE 位=1 可使 CPU 从实模式转到保护模式;当 PE 从 1 改为 0,则从保护模式回到实模式。执行中断返回指令 IRETD 或进行任务切换时,可以从保护模式进入 V86 模式,此时 EFLAGS 寄存器中的 VM(Virtual 8086 Mode)位被置为 1。通过中断,可以使 CPU 从 V86 模式返回到保护模式,此时 VM 位被清 0。

CPU 处于实模式、保护模式或者 V86 模式中的任何一种时,只要接收到系统管理中断(SMI#)信号,它就会切换到系统管理模式。当执行到从管理模式返回指令 RSM(Return from system management mode)时,CPU 切换到进入系统管理模式前的工作模式。

13-2 寄存器

寄存器是微处理器中很重要的部件。32 位微型计算机的寄存器可以分成三大类:一类是用户级寄存器,也就是设计应用程序时必须要用到的寄存器,这对要进行汇编语言程序设计的读者来说是必须掌握的。第二类是系统级寄存器,它包括控制寄存器和支持存储器管理的段表寄存器。控制寄存器主要是供操作系统使用的,操作系统设计人员要熟悉这些寄

寄存器。存储器管理寄存器在应用程序设计时不能被直接使用,但系统运行期间可能被系统软件访问(间接使用),因此也被称为程序不可见寄存器。第三类是程序调试寄存器。对于 PII、PIII 及 P4 处理器还有新增的寄存器。如 PII 处理器增加了 mm0~mm7 寄存器,从 PIII 处理器开始又增加了 xmm0~xmm7 单精度浮点寄存器,用 SIMD 技术进行汇编语言程序设计时,会用到这些寄存器。本节主要介绍用户级寄存器和支持存储器管理的段表寄存器。

一、用户级寄存器

用户级寄存器也称为应用程序设计寄存器,在研究任何指令和编写汇编语言程序前,都必须先弄清这些寄存器的名称、功能和用法。

图 13-4 是 8086-Pentium 处理器内部的用户级寄存器。图中画有阴影的是 8086 的 16 位寄存器,这是大家所熟悉的。80386-Pentium 中的 32 位寄存器的名称在原有 16 位寄存器前加上“E”。这类寄存器又可以分成通用寄存器、指令和标志寄存器以及段寄存器 3 类,下面分别进行介绍。



图 13-4 用户级寄存器

1. 通用寄存器

8 个 32 位通用寄存器如图 13-4(a)所示,它们是 EAX、EBX、ECX、EDX、ESP、EBP、ESI

虚拟 8086 模式或 V86 模式。该位置 1 时,处理器将在虚拟 8086 模式下工作;若 VM=0,则工作于保护模式。

- AC(Alignment Check)

对界检查位。仅用于 80486 CPU,进行字或双字操作时,用于检查地址是否处于字或双字边界上。AC=1,地址不处于字或双字边界上,否则在边界上。

以下 3 位是 Pentium-Pro 处理器新增加的标志位。

- VIF(Virtual Interrupt Flag) 虚拟中断标志。

- VIP(Virtual Interrupt Pending) 虚拟中断挂起标志。

- ID(Indetification) 标识标志。用来指示 Pentium/Pentium Pro 对 CPUID 指令的支持状态。

3. 段寄存器

在 8086 CPU 中,段寄存器用来存放段基地址的值,每个段的长度固定为 64KB。在 80386 中,除了原有的段寄存器 CS、DS、SS 和 ES 外,又增加了两个新的段寄存器 FS 和 GS,没有另外给它们命名,如图 13-4(c)所示。在 80386 工作于实模式和 V86 模式时,段寄存器中存放 16 位段基地址,与 8086 CPU 兼容。在保护模式下,段的信息太多。例如,不仅要存放段基地址,还要在 1 字节~4GB 的内存空间中寻址;对应段可能已装入内存,也可能要从磁盘上调入内存;段有各种类型;还要考虑优先级及各种保护功能等。因此,16 位段寄存器根本无法完全描述段的所有信息。于是,6 个段寄存器中存放的不再是段基地址,而是存放一种称为段选择子(Segment Selector)的指示器。段选择子也称为段选择符,通过它可以找到段的全部信息。

段的全部信息存放在段描述符(Segment Descriptor)中,每个段描述符的长度为 8 字节,用它来存放段的基地址、段的长度范围以及段的各种属性,段描述符的格式将在 13-3 节讨论。所有的描述符都放在两张表中,一张为全局描述符表(Global Descriptor Table, GDT),另一张为局部描述符表(Local Descriptor Table, LDT)。如果有多个任务都要用到某个段描述符,则将此段描述符放入 GDT 中。GDT 中存放着由操作系统管理的段描述符,它不能被应用程序直接修改;LDT 中包含的仅仅是单个任务所使用的描述符。

段选择子的格式如图 13-6 所示,它由 3 部分组成。

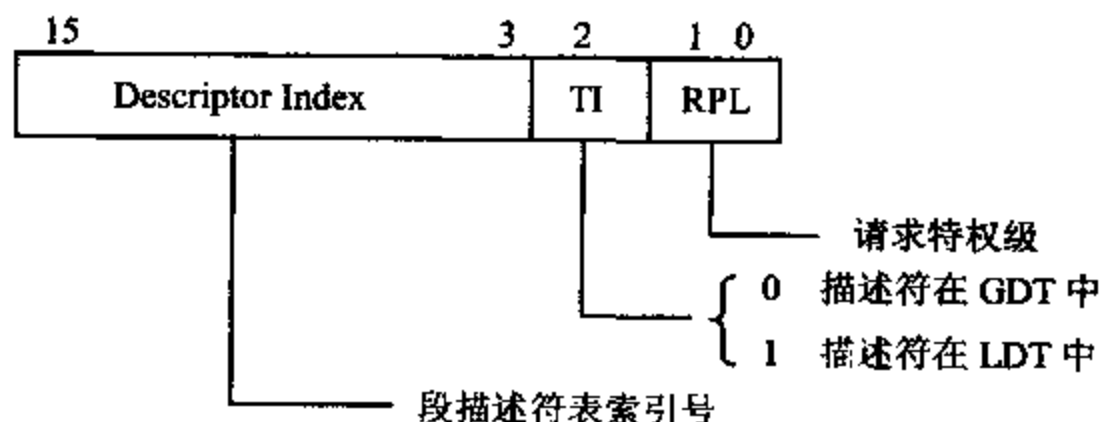


图 13-6 段选择子格式

- RPL(Request Priviledge Level) 请求优先级。用它给段选择子赋予一个特权属性,可以是 0~3 级。

- TI(Table Indicator) TI 位用来说明段选择子指向的段描述符在 GDT 中还是在

LDT 中。

TI=0, 段描述符在 GDT 中

TI=1, 段描述符在 LDT 中

• **Descriptor Index** 段描述符索引。用来表示段描述符在描述符表中的序号, 共 13 位, 允许寻找 $2^{13} = 8192$ 个描述符, 根据索引号可以在相应的 GDT 或 LDT 表中找到相应的描述符。

二、系统级寄存器

这类寄存器包括控制寄存器和系统段表寄存器。

1. 控制寄存器

控制寄存器 $CR_0 \sim CR_4$ 决定处理器的操作模式和现行执行任务的特征状态, 主要供操作系统使用。控制寄存器如图 13-7 所示。其中 CR_4 是 Pentium 以上处理器新增加的控制寄存器, 由于其功能比较复杂, 在此不作具体介绍了。下面主要介绍 $CR_0 \sim CR_3$ 的含义。

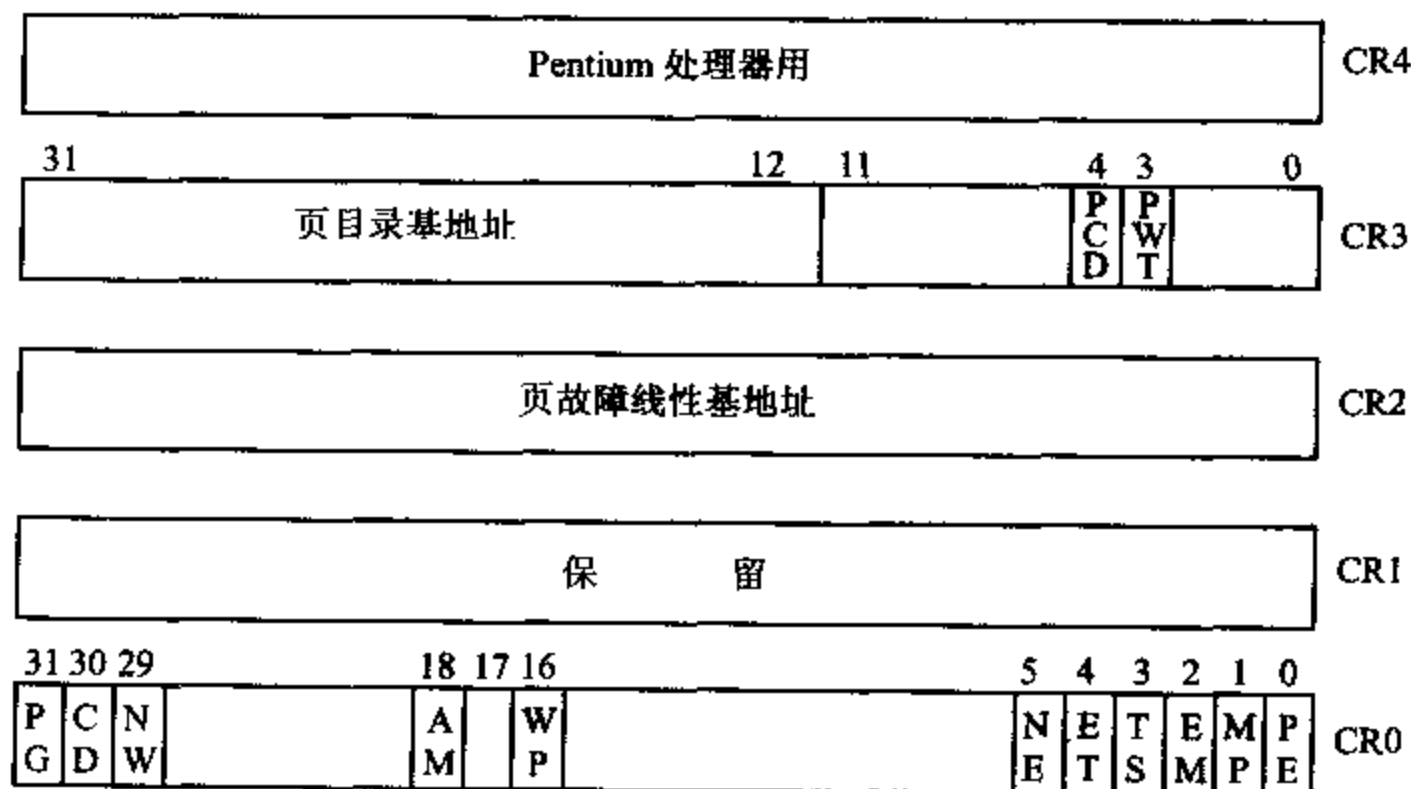


图 13-7 控制寄存器

(1) CR_0

包含控制操作模式的控制标志位, 并给出处理器的状态位, 其中控制标志位可以通过指令来设置。Pentium Pro 处理器的 CR_0 定义了 11 个控制位, 意义如下:

• **PG(Paging Enable)**, 允许分页位

PG=1, 允许分页操作

PG=0, 禁止分页操作

• **PE(Protection Enable)** 保护模式允许位

PE=1, 启动系统进入保护模式

PE=0, 系统以实模式方式工作

PE 和 PG 组合起来, 可以提供 3 种工作环境, 如表 13-1 所示。

表 13-1 PE 和 PG 组合提供的工作环境

PG	PE	工作环境
0	0	实地址方式,与 8086 兼容
0	1	不分页保护方式,有分段功能,但无分页功能
1	0	未定义
1	1	分页保护方式,具有分页分段功能

有关保护模式下存储器分段和分页的概念将在下节介绍。

• **MP(Monitor Coprocessor)和 TS(Task Switched)** 协处理器监控位和任务切换位。80386 是一个多任务系统,在任务切换时,系统硬件总是使 TS 置 1,如此时 MP 置 1,则 CPU 在执行 WAIT 指令时,会产生一个“协处理器无效”信号,即产生异常 7。

• **EM(Emulation)模拟协处理器控制位**

EM=1 时,指示处理器内部或外部没有 80x87 协处理器,这样执行协处理器指令时会产生异常,用软件可以模拟协处理器。

EM=0 时,协处理器存在,从 80486 开始,EM 肯定为 0,因协处理器在处理器芯片内部。

• **ET(Extension Type)处理器扩展类型控制位** 在 80386/80486 中,ET=1,支持 80387 协处理器工作;ET=0,则用 80287 协处理器。P6 系列和 Pentium 4 处理器保留此位。

• **NE(Numeric Error)数字错误位** 用来控制是由中断向量 16 还是由外部中断来处理未屏蔽的浮点异常。

NE=1,当执行 80x87 FPU 指令发生故障时,用异常 16 处理

NE=0,则用外部中断处理

• **WP(Write Protect)写保护位** 用来保护管理程序写访问用户级的只读页面。

WP=1,强制任意特权级写入只读页面时会发生“故障”

WP=0,允许只读页面由特权级 0~2 写入

• **AM(Alignment Mask)对界屏蔽位** 用来控制 EFLAGS 中的对界检查位(AC)是否允许进行对界检查。

AM=1,允许 AC 位进行对界检查

AM=0,禁止 AC 位进行对界检查

• **NM(Not Write-Through)非通写位** 用来选择片内数据 Cache 的操作模式。

NW=1,禁止通写,写命中,不修改主存

NW=0,允许通写

所谓“通写”是要求 Cache 写命中时,Cache 与存储器同时完成写修改,即处理器的写操作贯通 Cache,直达存储器。

• **CD(Cache Disable)Cache 禁止位** 用来控制是否允许向片内 Cache 填充新数据。当 Cache 未命中时:

CD=0,允许填充 Cache

CD=1,禁止填充 Cache

(2) CR₁ 保留

(3) CR₂

页故障线性基地址寄存器(Page Fault Linear Address),用来存放发生故障中断(异常 14)之前所访问的最后一个页面的线性地址。发生页故障时报告出错信息。当发生页异常时,处理器把引起异常的线性地址保存到 CR₂ 中。操作系统中的页异常处理程序可以检查 CR₂ 的内容,查出线性地址空间中的哪一页引起本次异常。当控制寄存器 CR₀ 中的 PG=1,即允许分页时,CR₂ 才是有效的。

有关线性地址、页异常等内容将在下节描述。

(4) CR₃

页目录基地址寄存器 CR₃ 用来存放页目录表的物理基地址。当 Pentium Pro 处理器使用分页寄存器管理机制时,页目录是按页(4KB 为一页)对齐的,也就是说必须从低 12 位地址为 0 的那些地方开始分页。例如,可以从 0000 0000H、0000 1000H、0002 8000H 等地址处分页,这些地址称为基地址。CR₃ 的高 20 位用来放页目录的基地址的高 20 位,低 20 位应该为 0,才能构成基地址。但系统中还将位 3、位 4 定义成其它用途。也就是说 CR₃ 的低 12 位一般为 0,而位 3、位 4 允许为 1,这两位的意义如下:

- PWT(Page Write-Through) 页面通写位,用于指示页面是通写还是回写:

PWT=1,外部 Cache 对页面进行通写(Write-Through)

PWT=0,外部 Cache 对页面进行回写(Write-Back)

- PCD(Page Cache Disable) 页面 Cache 禁止位,用于指示页面 Cache 的工作情况:

PCD=0,允许片内页 Cache

PCD=1,禁止片内页 Cache

2. 系统段表寄存器

(1) 各种描述符表

在 32 位处理器中,为了对存储器实现分段管理,系统把段的有关信息,即段的线性基地址、段的长度和属性,全部放在 8 字节“描述符”数据结构中,并把系统中所有的描述符编成表,放在存储区域中称为描述符表的地方,供硬件查找和识别。这些表分别称为:

全局描述符表(Global Descriptor Table,GDT)

中断描述符表(Interrupt Descriptor Table,IDT)

局部描述符表(Local Descriptor Table,LDT)

全局描述符表 GDT 和中断描述符表 IDT 是面向系统中所有任务的,是全局性的表;局部描述符表是面向某一任务的,每个任务可以都有一个独立的 LDT。对于分段机制来说,这些表都是非常重要的。

在分段机制中,除了用到上述 3 种表之外,32 位处理器为支持多任务操作,要实现从一个任务切换到另一任务的操作,在硬件上还为每个任务设置了一种称为任务状态段(Task State Segment,TSS)的系统段,它保存了当前正在处理器上执行的任務的各种重要信息。对于任务状态段 TSS,也要由 TSS 描述符来说明该特殊段的起始地址、限长和属性信息。有关 TSS 的功能将在下节作进一步讨论。

(2) 系统段表寄存器组成

由于上述 3 种表和一个特殊的 TSS 段都位于内存中,为了寻找和定义这些描述符表和 TSS 系统段,确定它们的线性基地址及表的长度等信息,系统中设置了全局描述符表寄存器 GDTR、中断描述符表寄存器 IDTR 和局部描述符表寄存器 LDTR,它们分别用来寻址对应的描述符表;另外用任务状态段寄存器(Task State Segment Register, TR)来寻址任务状态段 TSS。GDTR 和 IDTR 用来管理全局描述符表和中断描述符表这两张系统表,所以这两个寄存器也称为系统表寄存器。TSS 为系统段, LDT 也作为系统段来寻址, LDTR 和 TR 这两个寄存器用来管理这两个系统段,因此它们也称为系统段寄存器。4 个寄存器的组成如图 13-8 所示。

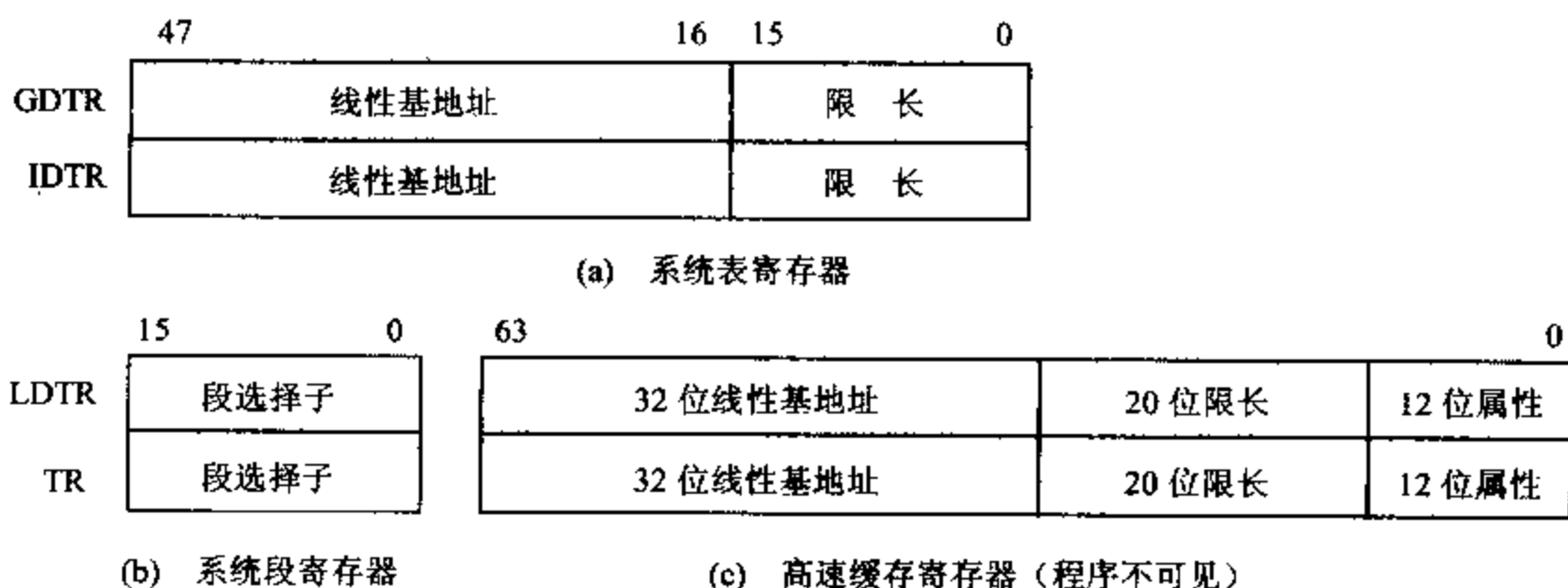


图 13-8 段表寄存器

(3) GDTR 和 IDTR 寄存器功能

系统表寄存器 GDTR 和 IDTR 如图 13-8(a)所示。全局描述符表寄存器 GDTR 指向 GDT 表, 48 位的 GDTR 定义 GDT 表的线性基地址和限长, 基地址说明 GDT 表中字节为 0 的表项的线性基地址值, 它指向表头; 限长说明 GDT 表的长度, 实际指向表的最后一个偏移地址。如限长=0FFFH, 则说明 GDT 表的字节编号为 000—0FFFH, 表的长度=0FFFH+1=1000H。

GDT 表中含有可供系统中所有任务使用的段描述符, 如操作系统用的数据段、堆栈段和任务状态段, 还包含各个局部描述符表 LDT 的描述符, 最多可存放 $2^{13}=8192$ 个描述符, 但 0 号为空表, 不能用, 如访问 GDT 表中的 0 号描述符, 将产生异常。

中断描述符表寄存器 IDTR 指向 IDT 表, 也由基地址和限长两部分组成。IDT 表中存放中断或异常的描述符, 每个中断或异常处理程序有一个对应的描述符, 最多可以有 256 项。

GDTR 和 IDTR 与对应的 GDT 表和 IDT 表的关系如图 13-9 所示。

GDTR 的值由下述指令装入:

LGDT mem48 ; mem48 为 48 位的内存操作数, 包含基地址和限长

IDTR 的值由下述指令装入:

LIDT mem48 ; mem48 为 48 位的内存操作数, 包含基地址和限长

由此可知, 只要对 GDTR 和 IDTR 两个寄存器装入一定的数值, 给出这两张表的线性

基地址和段限长,就完全可以在内存中对 GDT 表和 IDT 表进行定位了。

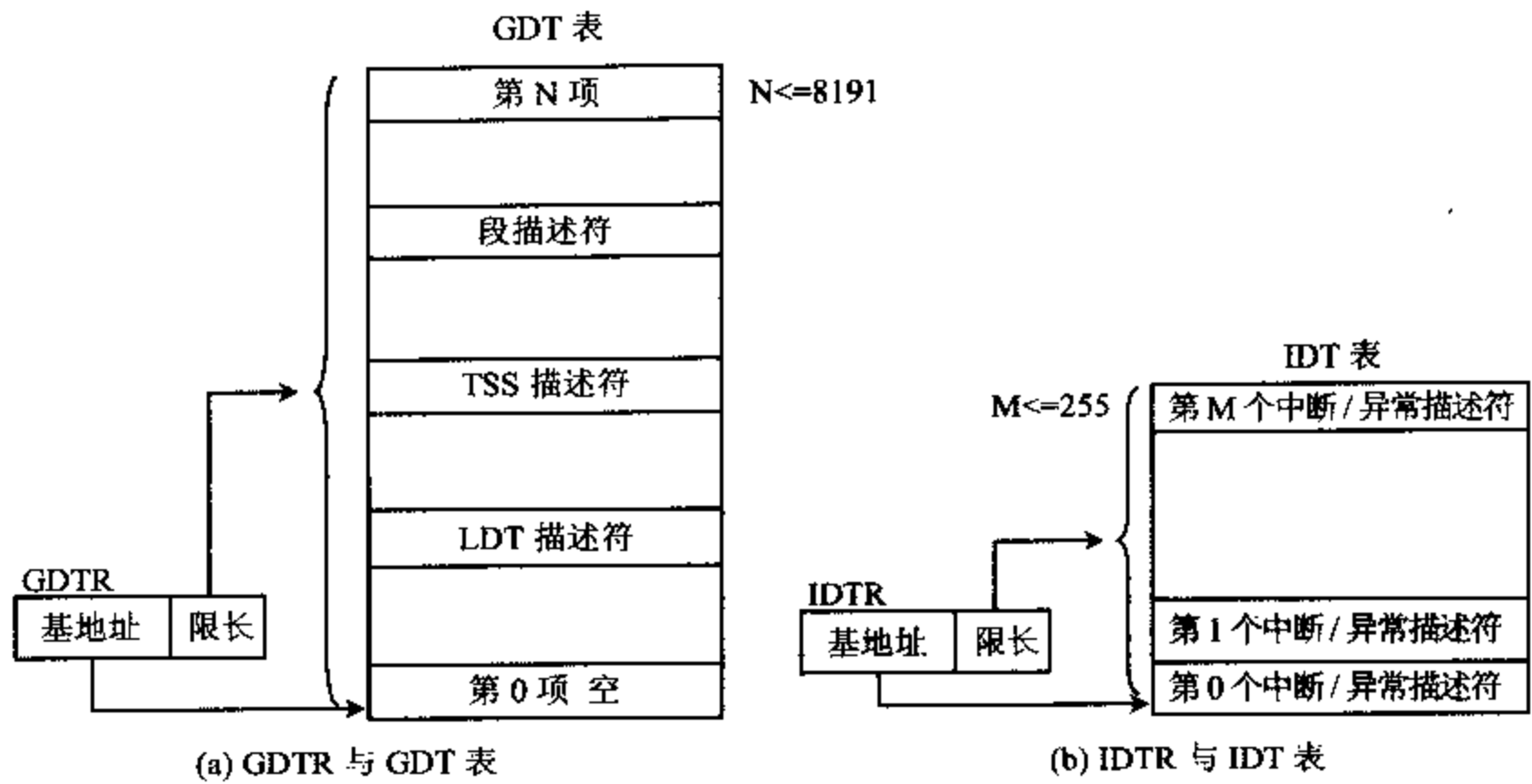


图 13-9 GDTR、IDTR 寄存器与 GDT 表、IDT 表的关系

(4) LDTR 寄存器的功能

LDTR 寄存器由 16 位的选择子和 64 位的高速缓冲寄存器组成,如图 13-8(b)、(c)所示。要定位 LDT 表,当然要通过 LDTR 寄存器来实现,但局部描述符表 LDT 的寻址方式与上述两种表不一样。

LDT 表是面向某个任务的,该任务所在的存储区作为对应任务的一个系统段,其位置由 16 位选择子指出。为了访问局部描述符表 LDT,需要在 LDTR 中装入当前任务的 LDT 描述符,再由描述符的值来确定 LDT 表的表头基地址、段限长及属性信息。LDTR 的值由下列指令装入:

LLDT reg16/mem16 ;操作数的内容应为 16 位的 LDT 段选择子

由上可知,LDTR 寄存器中只要包含 16 位段选择子,就可以从 GDT 表中找到相应的 64 位 LDT 描述符了,那么为什么还要有一个 64 位的高速缓冲寄存器呢?原因是,如果在每次访问存储器时都通过 LDTR 寄存器的 16 位选择子,就必须先访问 GDT 表中的 LDT 描述符,再读出描述符内容,并译码得到描述符的各种信息,这样效率太低。为此在系统中设有一个与 LDTR 相关联的 64 位寄存器,当使用 LLDT 指令加载 64 位 LDT 系统段描述符信息(包含 32 位线性基地址,20 位限长及各种属性)时,同时把描述符信息加载到高速缓冲寄存器中,如图 13-10 所示。这样,以后只要从高速缓存中找到 LDT 描述符信息,而不用每次去访问 GDT 表了,除非重新装载新的 LDT 描述符。这个高速缓冲寄存器对程序员来说是不可见的。

(5) TR 寄存器的功能

能在任务或进程之间快速切换是多任务/多用户操作系统的一个重要属性,每一个任务都有一个与之相关联的任务状态段(Task State Segment, TSS),TSS 保存着当前任务的所有环境,如寄存器、内存地址空间、I/O 地址空间等信息。TSS 位于内存中,它到底在何处

呢？回答是由任务状态段寄存器 TR 和 TSS 的描述符来定位。

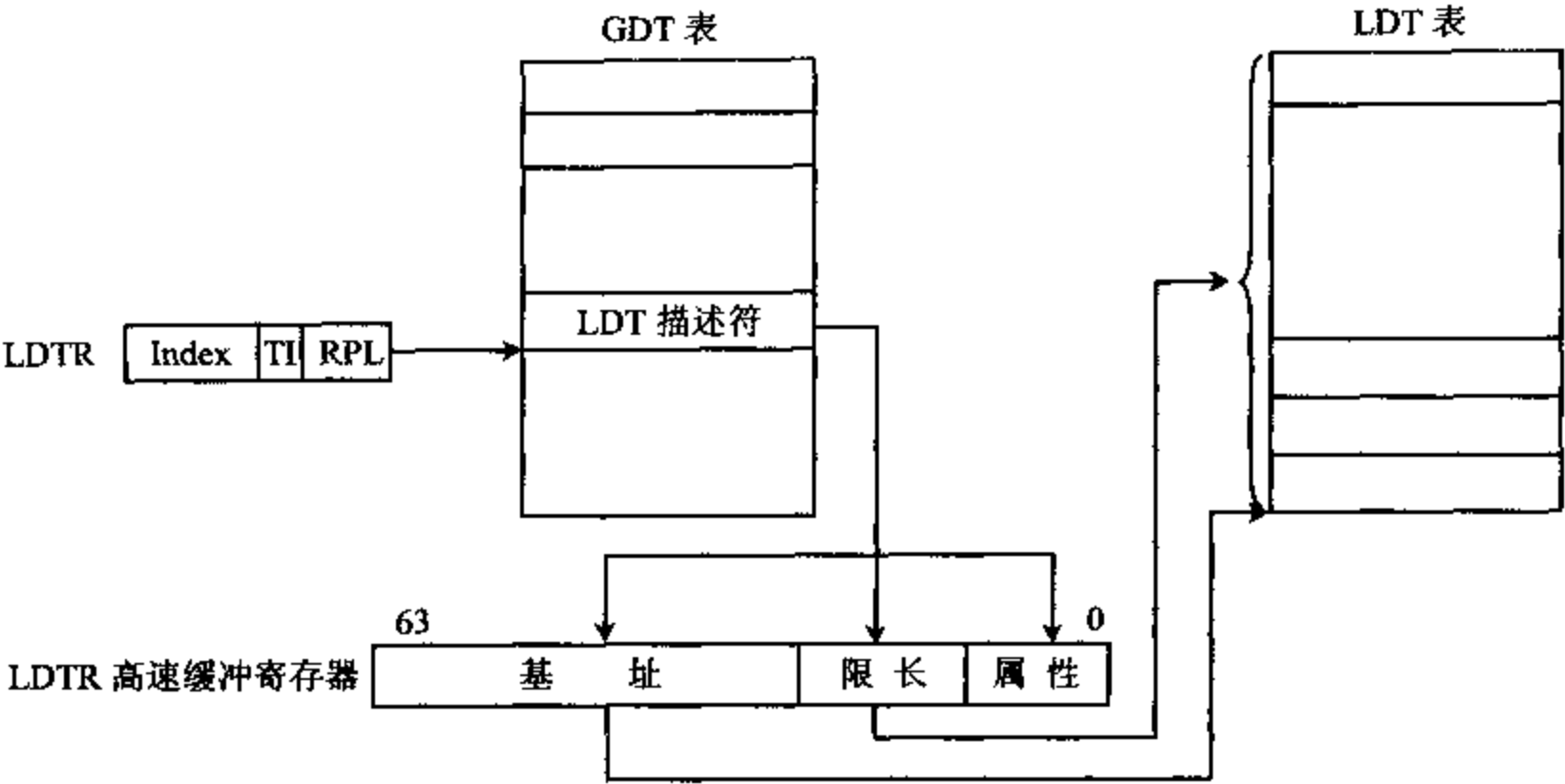


图 13-10 LDTR 与 GDT 表及 LDT 表的关系

TR 的用法与 LDTR 有类似之处,它也由 16 位的选择子和 64 位的高速缓冲寄存器组成,并由下述指令装入 16 位选择子的值:

LTR reg16/mem16 ;操作数为 16 位寄存器或内存变量

TR 的功能和工作过程如图 13-11 所示。先由 TR 的 16 位选择子在 GDT 表中找到当前任务的 TSS 描述符,根据描述符可读出 TSS 段的基地址、限长及属性,在读出描述符信息的同时,也把它们加载到了 64 位缓冲寄存器中,避免 TR 的 16 位选择子访问存储器时,每次都要从 GDT 中访问 TSS 描述符,使任务执行的效率更高。

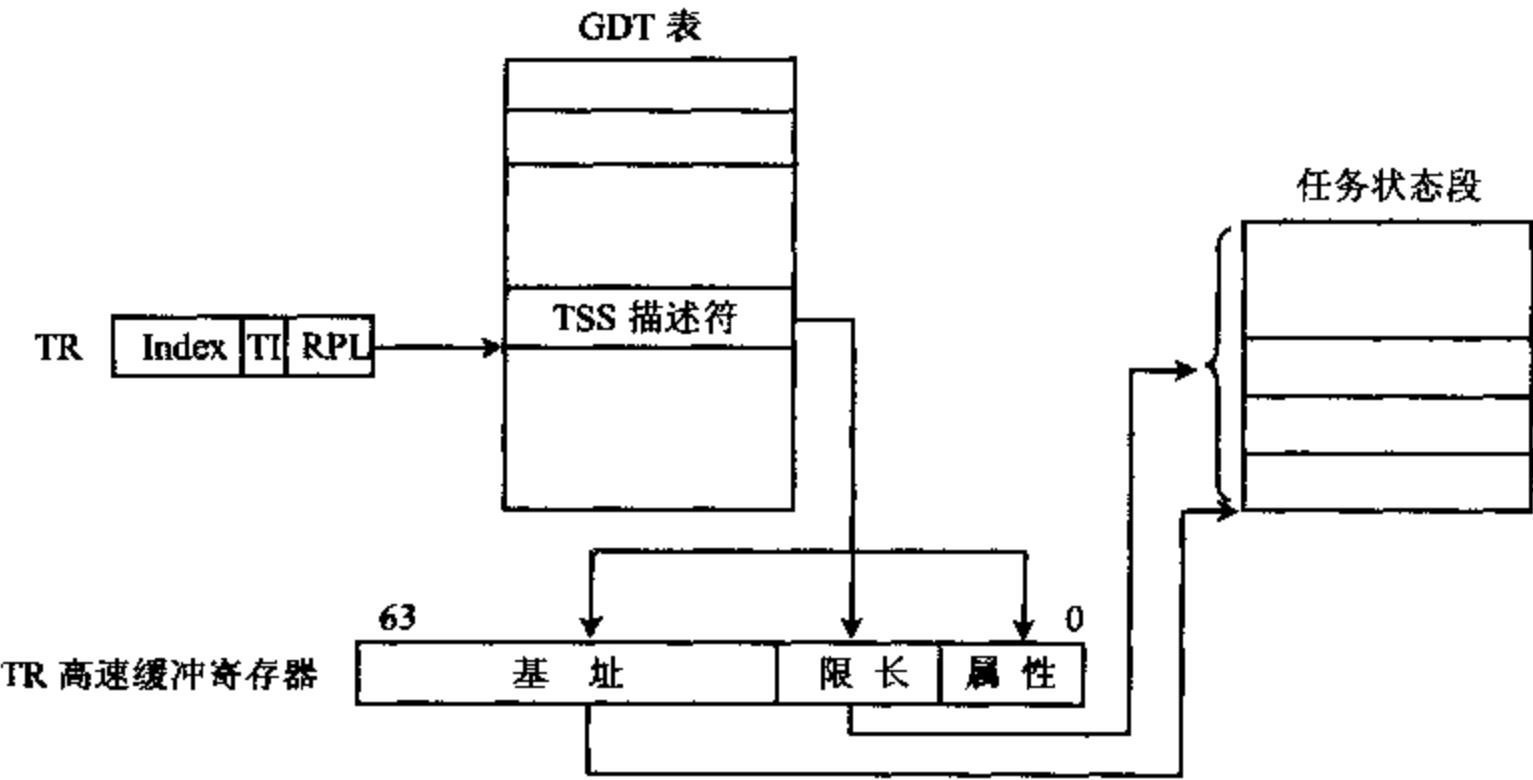


图 13-11 TR 的工作过程

3. 段寄存器和描述符表的关系

通过对段表地址寄存器的介绍,可以看到在内存中存有全局描述符表 GDT 和局部描

述符表 LDT, 分别用来存放全局描述符和局部描述符, 由 GDTR 和 LDTR 来寻址, 也就是确定它们在内存中的位置。通过段寄存器中的段选择子可以找到描述符表中的某一个描述符。下面通过图 13-12 进一步说明段寄存器和描述符的关系, 并讨论 LDT 表为什么要通过 GDT 表来定位, 具体怎样定位等问题。

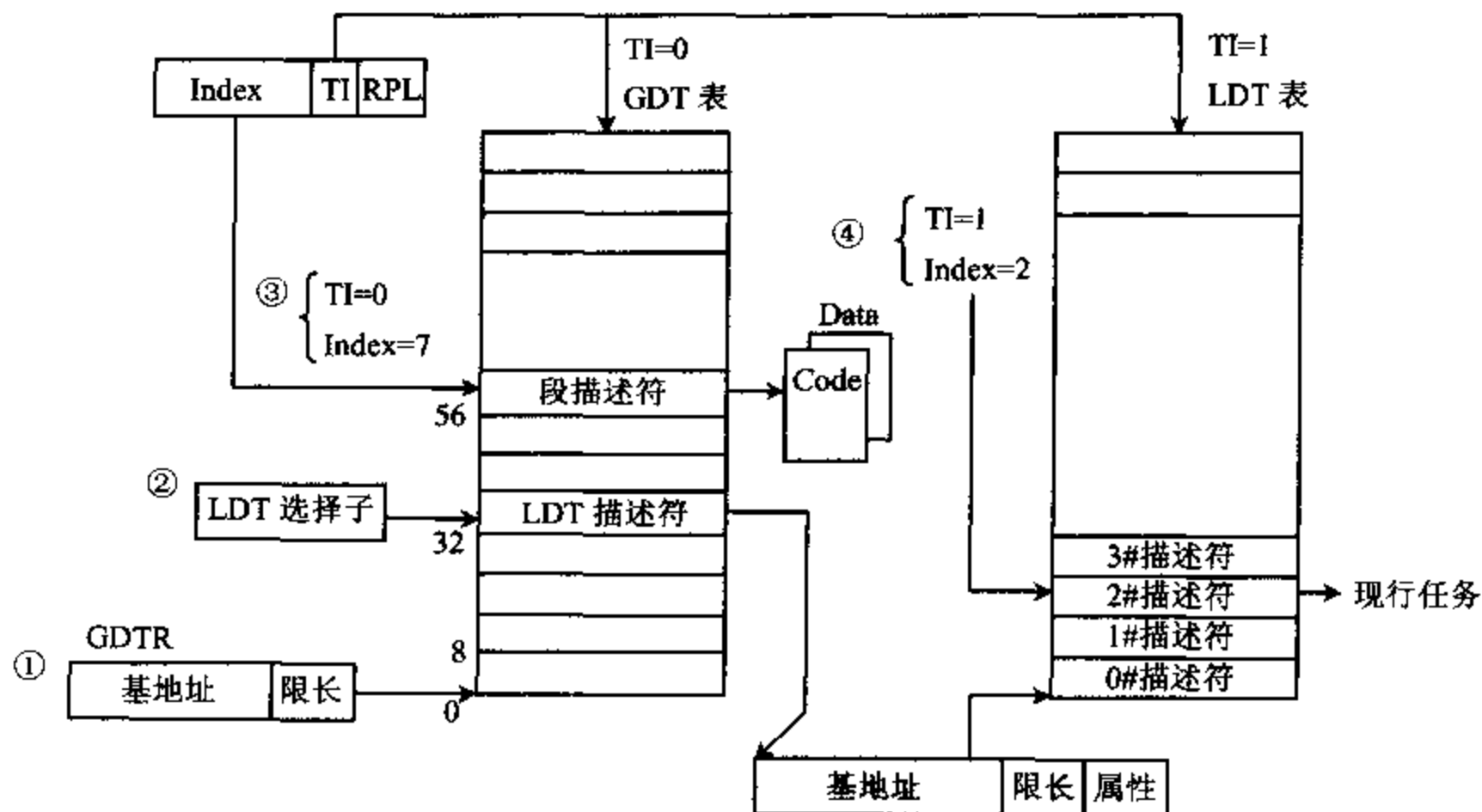


图 13-12 段寄存器与描述符表的关系

由图 13-12 可以看到: ① GDTR 寄存器给出 GDT 表的基地址和限长, 定位 GDT 表。② 任务切换时, 任务状态段 TSS 中有一个 16 位的 LDT 段选择子, 它的 TI=0, 用来在 GDT 表中定位某一任务的 LDT 表。设其 Index=4, 则选中 GDT 表中的 4 号段描述符, 因每个描述符占 8 字节, 所以 4 号描述符相对于 GDT 的基地址的偏移量为 32, 再从该单元开始取出 LDT 描述符, 得到基地址、限长和属性信息, 由基地址和限长的值就可以定位 LDT 表了。③ 如某一个段寄存器 (如 CS、DS) 装入的段选择子, 它的 TI=0, Index=7, 则指向 GDT 表中的 7 号描述符, 由该描述符指向一个代码段或数据段。④ 如某一个段寄存器的 TI=1, Index=2, 则访问 LDT 表中的 2 号描述符, 由此描述符指向现行任务段。

三、程序调试寄存器

80386 以前的处理器, 只提供两个调试接口: INT 01H 和 INT 03H, 分别支持单步中断与断点中断。80386 仍支持这两个中断, 但在调试方面有了重大改进, 主要是引入了 8 个调试寄存器 DR₀~DR₇, 不但可支持指令断点, 还可支持数据断点。

8 个调试寄存器如图 13-13 所示。

DR₀~DR₃ 这 4 个断点调试寄存器用来存放 4 个断点的线性地址。有了它们, 只需将断点指令的线性地址写入相应寄存器, 即能构造指令断点, 无须在指令断点处写入一条“INT 03H”指令来产生断点。80386 可支持 4 个断点。

DR₆ 为调试状态寄存器。当产生调试异常(01H 号中断)时,微处理器会在 DR₆ 中给出异常类型,如单步异常、Debug 异常等。

DR₇ 为调试控制寄存器,用来规定每一个断点寄存器的使能选择、断点类型(指令断点、数据断点)选择及所有断点寄存器的保护。

DR₄、DR₅ 保留。

断点 0 的线性地址	DR0
断点 1 的线性地址	DR1
断点 2 的线性地址	DR2
断点 3 的线性地址	DR3
保 留	DR4
保 留	DR5
调试状态寄存器	DR6
调试控制寄存器	DR7

图 13-13 调试寄存器

13-3 保护模式下的内存管理

在实模式下,80386 的段内存管理方法与 8086 的相同,每个段的大小是固定的,都是 64KB,只要把段寄存器中的内容左移 4 位就可得到段基地址,再加上偏移量就能得到存储单元的地址,这个地址就是物理地址。在 80386 中,偏移量可以是 32 位的操作数,但一般情况下,仍旧只能访问 1MB 内存地址空间,为了使处理器能寻址 4GB 的地址空间,须工作在保护模式下。

在保护模式下,采用了全新的段内存管理技术来进行内存管理。在 80386 中,不是简单的由段寄存器中的值直接获得对应的段基地址,而是采用可变长的分段技术,每一个段的大小从 1 字节~2³² 字节(4GB),随数据结构和代码模块的大小而定,使用起来灵活方便。而且在分段时,还对每一段赋予属性和保护信息,进行 4 级保护,实现程序与程序之间、用户与操作系统之间的隔离和保护,能有效地防止在多任务环境下各模块对存储器的越权访问,为多任务操作系统提供优化支持。此外,还采用分页管理技术,将 4KB 的内存空间作为一页来处理,这正好相当于磁盘系统中的一个簇,便于将磁盘等存储设备的存储空间有效地映射到内存,与分段技术相结合,使虚拟存储空间(实际放在磁盘上)大大超过物理地址,可达到 64TB 之多。还有,在多任务系统中,有了分页功能后,只要把每个活动任务当前所需的少量页面放在存储器中,其余放在磁盘上,可以明显地提高存取数据的效率。采用分页技术的另一个好处是,用户不必关心物理地址是否连续,可将不连续的内存区域连在一起,在线性地址空间上视为连续,这样能有效利用内存碎片。

下面介绍保护模式下的内存管理技术,先介绍分段技术,后讨论分页技术。为简单起见,主要以 80386 为例来进行讨论。

一、段内存管理技术

为了掌握保护模式下的段内存管理技术,首先要对 32 位系统中的 3 种存储器地址空间——逻辑地址、线性地址和物理地址的概念有基本的了解,清楚彼此间的关系,同时要熟悉段描述符的格式、功能和用法。

1. 逻辑地址、线性地址和物理地址

(1) 逻辑地址(Logic Address)

对段内存空间进行寻址的地址称为逻辑地址,也叫做虚拟地址(Virtual Address),这是应用程序设计人员进行编程时要用到的地址。逻辑地址由一个 16 位的段选择子(Segment Selector)和 32 位偏移量两部分组成。段选择子存放在段寄存器中;偏移量也称为偏移地址或有效地址(Effect Address, EA)。逻辑地址可以用“段选择子:偏移地址”这种形式来表示。

前面已经讲到,在保护模式下,段寄存器中存放的不是段基地址,而是“段选择子”,通过它可以找到段的全部信息,段选择子的格式如图 13-6 所示。段的全部信息由“段描述符”来说明,它们可以放在全局描述表 GDT 或局部描述符表 LDT 中。

进行汇编语言程序设计,涉及到存储器操作数时,需要了解偏移地址的计算方法。

在 8086 CPU 中,涉及到存储器操作数时,偏移地址为:

$$\text{偏移地址} = \text{基址} + \text{变址} + \text{位移量}$$

只有 BX、BP 可作基址寄存器用,SI、DI 可作变址寄存器用,而且只有这 4 个寄存器可以放在方括号[]中进行寻址,构成偏移地址。

在 80386 CPU 中,提供了更加灵活的寻址方式,偏移地址根据下式计算:

$$\text{偏移地址} = \text{基址} + \text{变址} * \text{比例因子} + \text{位移量}$$

存放基址的寄存器可以是 32 位通用寄存器,它们是: EAX、EBX、ECX、EDX、ESP、EBP、ESI 和 EDI;变址寄存器可以是除 ESP 以外的任一个 32 位通用寄存器;比例因子可以是 1、2、4 和 8;位移量为 8 位或 32 位的立即数。

(2) 物理地址(Physical Address)

物理地址是指内存芯片阵列中每个阵列所对应的唯一的地址,32 位地址线可直接寻址 $2^{32} = 4\text{GB}$ 内存单元。

(3) 线性地址(Linear Address)

是沟通逻辑地址与物理地址的桥梁,32 位微处理器芯片内的分段部件将逻辑地址空间转换成 32 位的线性地址。在保护模式下,每个段选择器中的 Index 指向 GDT 或 LDT 中的一个描述符,从段描述符表中可以读出相应的线性基地址值,再将它与 32 位偏移地址相加,就可以形成线性地址。

例如,对于下述含有存储器操作数的指令:

```
MOV AX, [EAX+ECX*2+100H]
```

段选择子默认在 DS 寄存器中,由 DS 中的段选择子可找到对应的段描述符,从而获得段线性基地址;有效地址 $EA = EAX + ECX * 2 + 100H$ 。将基地址与有效地址相加就可得到线性地址。

(4) 地址转换框图

在 80386 中,分段部件首先将逻辑地址(段选择子:偏移地址)转换成线性地址。如果分页功能被禁止,则线性地址就是物理地址;如果允许分页,则分页部件再将线性地址转换成物理地址。地址转换框图如图 13-14 所示。转换过程如下。

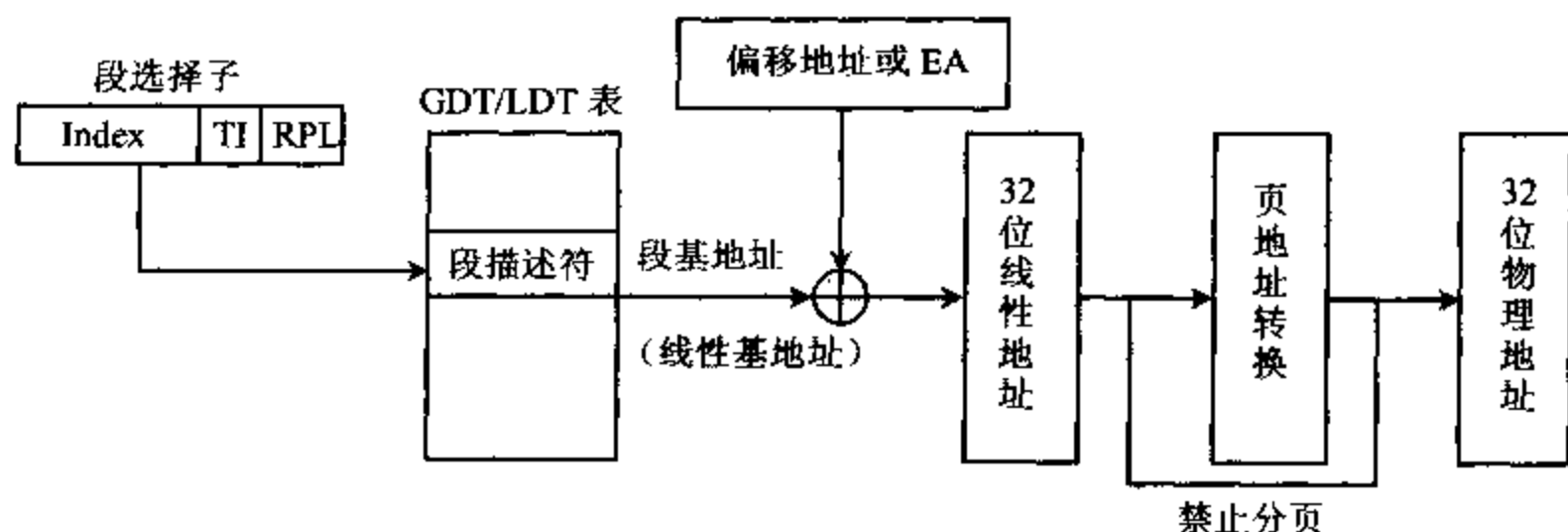


图 13-14 逻辑地址、线性地址和物理地址转换框图

从段寄存器中取出段选择子的值,段寄存器可以是 CS、DS、SS、ES、FS、GS 中的一个。若段选择子中的 $TI=0$,则到全局描述符表 GDT 中找相应段的段描述符,否则到 LDT 表中找。根据描述符中的索引值可以从 GDT 或 LDT 表中找到某一个段描述符。如 $Index=5$,则可找到 5 号段描述符。由段描述符可以得到该段的 32 位段基地址或称为线性基地址,还可得到段限长。偏移地址的计算方法前面已介绍过了,它可以由立即数或一、两个寄存器的值构成,分段部件把各地址分量送到一个加法器中形成偏移地址,再经一个加法器与段描述符中给出的段基地址相加,得到线性地址。另外,还通过一个 32 位的减法器与段的界限进行比较,检查是否越界,如越界则要进行异常处理。

得到 32 位线性地址后,如分页部件处于允许状态,即控制寄存器 CR_0 的允许分页位 $PG=1$,则通过页转换机制将线性地址转换成物理地址;如 $PG=0$,则禁止分页,线性地址就是物理地址。

在 80386 中,地址流水线具体体现在有效地址的形成、逻辑地址到线性地址的转换和线性地址到物理地址的转换这 3 个动作的重叠执行上。

为了加深对段选择子的理解,下面再举两个有关段选择子设置的具体例子。

例 13-1 有一个段描述符,放在全局描述表的第 9 个项中,该描述符的请求特权级为 2,求该描述符的选择子。

由于请求特权级 $PRL=2$,所以选择子的 D_1D_0 位 = 10

段描述符放在全局描述符表 GDT 中,因此 TI 位即 D_2 位 = 0

索引号为 9,所以 $D_{15}-D_3 = 0000000001001$

因此该段选择子的值为:0000 0000 0100 1010,各位含义如图 13-15 所示。

例 13-2 某一个段描述符的选择子为 0000 1001 0000 0100,请解释该选择子的含义。选择子各位的含义如图 13-16 所示。

$RPL=00$,表示选择子的请求优先级为 0

$TI=1$,表示段描述符在 LDT 中

描述符索引号 $Index=00010010000B=0120H=288$,表明选择子所访问的描述符

在 LDT 中的第 288 项。

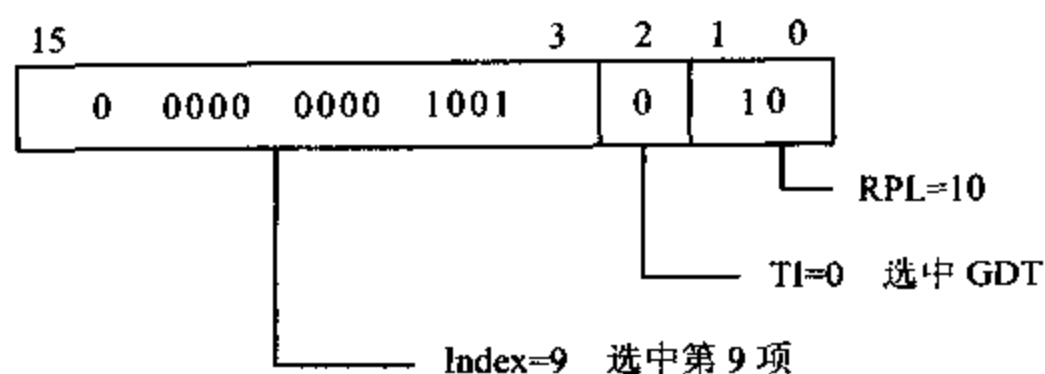


图 13-15 例 13-1 中段选择子的值

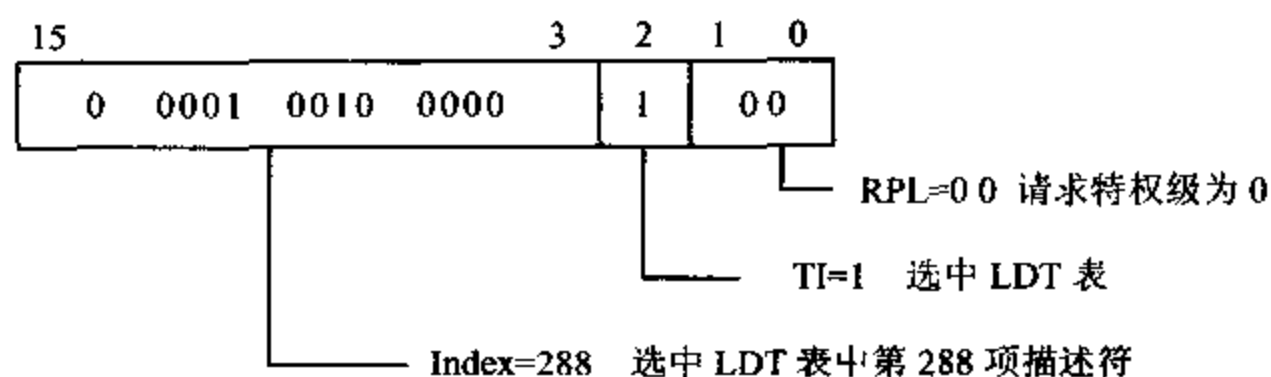


图 13-16 例 13-2 中段选择子的意义

2. 段描述符

为了实现分段管理,80386 把有关段的信息都放在段描述符数据结构中,并把系统中所有的描述符编成表,放在内存中,以便硬件查找。由段选择子可以找到相应的段描述符项。

段描述符分为内存段描述符、系统段描述符和门描述符 3 类,由描述符中属性部分的描述符类型(descriptor type)标志位 S 来决定。当 S=1 时,为内存段描述符,对应的段为代码段、数据段或堆栈段;当 S=0 时,为系统段描述符和门描述符。下面分别进行介绍。

(1) 内存段描述符

每个段描述符都由 8 字节组成,它分为 3 部分:段基地址(Base Address)、段限长(Limit)和段属性(Attribute),格式如图 13-17 所示。

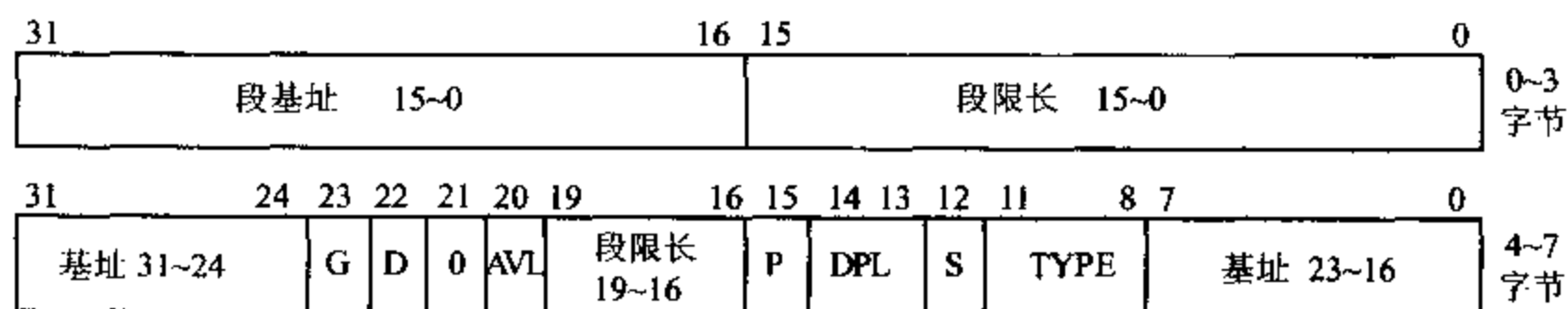


图 13-17 段描述符的格式

对于内存段描述符,S 位=1,各部分功能如下:

①段基地址

段基地址是 32 位的线性基地址 $A_{31} \sim A_0$,指出一个段的起始位置,它可以是 32 位线性地址中的任一个地址,允许段从 4GB 线性地址空间中的任何地方开始分段。在段描述符表中,32 位线性基地址分 3 个地方存放,主要是为了与 80286 兼容。

②段限长

段限长用 20 位表示,分 2 个地方存放,也是为了与 80286 兼容。限长决定了段的可寻址范围。描述符中的限长值包含段的最后偏移地址,这样,段的长度=段限长+1。由于段的大小可以在 1 字节~4GB(2^{32})范围内变化,段的限长值应该是 32 位,从 0~($2^{32}-1$)。例如,段的基地址如果是 0000 0000H,段限长为 FFFF FFFFH,则说明段的可寻址范围为 0000 0000H~FFFF FFFFH,可寻址整个 4GB 的线性地址空间。

但在段描述符中,段限长没有用 32 位表示,而只用 20 位来决定限长。为了达到能表示限长高达 2^{32} 的目的,在 80386 的段描述符中,采用 20 位限长和段属性中的 G 位(第 2 个双字中的位 23)来共同计算段限长,具体方法如下。

如 $G=0$,则表示段限长的高 12 位均为 0,低 20 位由段描述符中的 20 位限长值来表示,这时段的可寻址范围最大为 $2^{20}=1\text{MB}$ 。例如,设段基地址=0000 0000H,描述符中的段限长=FFFFFH, $G=0$,则该段可寻址范围为 0000 0000H~000F FFFFH,其限长就是 FFFFFFH,即 1MB。

如 $G=1$,则

$$\begin{aligned}\text{段限长} &= \text{描述符中的 20 位限长左移 12 位} + \text{FFFFH} \\ &= \text{描述符中的 20 位限长} \times 1000\text{H} + \text{FFFFH}\end{aligned}$$

例如,我们仍设段基地址=0000 0000H,描述符的段限长=FFFFFH,由于 $G=1$,则该段限长值应该 = FFFFFFH * 1000H + FFFFH = FFFFFFFFH,所以段寻址范围为 0000 0000H~FFFF FFFFH,可以寻址整个 4GB 的地址空间。

由此可知:当 $G=0$ 时,表明限长以字节为单位计算;当 $G=1$ 时,限长以页为单位计算,1 页=4KB。

③段内存属性

描述符中其它字段表示内存属性,各位的意义如下:

G(Granularity)粒度属性 上面已经讲过,即

当 $G=0$ 时,表示限长以字节为单位计算

当 $G=1$ 时,表示限长以页(4KB)为单位计算

D(Default Operation Size/Stack Point Size/and or Upper Bound) 它有 3 个功能:

- 对于指令代码段的描述符,D 位说明此段指令执行的代码是 32 位还是 16 位,即

$D=0$,表示是 32 位指令

$D=1$,表示是 16 位指令

- 对于数据段的段描述符,D 位用于决定内存段的上界,即

$D=0$,表示上界为 4GB 的段,即可以访问 32 位的段

$D=1$,表示上界只有 64KB,只能访问 16 位的段,用来与 80286 兼容

- 对于堆栈段的段描述符,用来说明堆栈指针用 32 位还是 16 位寄存器,即

$D=1$,用 32 位 ESP 寄存器作堆栈指针

$D=0$,用 16 位 SP 寄存器作堆栈指针,用来与 80286 兼容

P(Segment Present)段存在位

$P=1$,表示该段已装入主存

$P=0$,表示该段目前不在主存中,要从磁盘上调进来,假如此时访问该段,就会产生“段不存在异常”

DPL(Descriptor Privilege Level)段描述符优先级 共 2 位,它指出对应段的保护级别。

S(Descriptor type)描述符类型标志

S=1,该描述符为内存段描述符

S=0,该描述符为系统段描述符或门描述符

AVL(Available for use by System Software) AVL 标志 提供给系统软件用。80386 对 AVL 不作任何解释,可将其清 0。

位 21 保留,总是将其清 0

Type 类型域 4 位的 Type 字段(位 11-8)定义了内存段描述符的类型,其含义如表 13-2 所示。

表 13-2 内存段描述符中 Type 字段的含义

Type Field					Descriptor Type	Descriptor
Decimal	11 EXP	10 E	9 W	8 A		
0	0	0	0	0	Data	只读
1	0	0	0	1	Data	只读,已访问过
2	0	0	1	0	Data	读/写
3	0	0	1	1	Data	读/写,已访问过
4	0	1	0	0	Data	只读,向下扩展限长
5	0	1	0	1	Data	只读,向下扩展限长,已访问过
6	0	1	1	0	Data	读/写,向下扩展限长
7	0	1	1	1	Data	读/写,向下扩展限长,已访问过
		C	R	A		
8	1	0	0	0	Code	只执行
9	1	0	0	1	Code	只执行,已访问过
10	1	0	1	0	Code	可执行/读
11	1	0	1	1	Code	可执行/读,已访问过
12	1	1	0	0	Code	只执行,相容
13	1	1	0	1	Code	只执行,相容,已访问过
14	1	1	1	0	Code	可执行/读,相容
15	1	1	1	1	Code	可执行/读,相容,已访问过

从表 13-2 可以看出:

- 第 11 位(EXP)为可执行位,用它来区分是代码段还是数据段,若 $D_{11}=1$,且描述符类型位 $S=1$,则对应的段一般为代码段,可执行
 $D_{11}=0$,且描述符类型位 $S=1$,则对应的段为数据段,不可执行

- 第 8 位是访问位(A),若

A=1,表示已访问过

A=0,表示没有访问过

操作系统利用 A 位,对给定段进行使用率统计。

当型号为 0~7(即 EXP=0),也就是对应于数据段(含堆栈段)时:

- 第 10 位(E)为扩展方向位,若
E=1,动态向下扩展(Expand-Down)
E=0,向上扩展(Expand-Up)

- 第 9 位为写位,若
W=0,不可写
W=1,可写,堆栈段的 W 位必须为 1
当类型为 8~15(即 EXP=1),也就是对应于代码段时:

- 第 10 位(C)为相容(Conforming)位,若
C=1,本代码能被调用并执行
C=0,本代码不能被当前任务所调用
- 第 9 位(R)为读位,若
R=0,此段为代码段,不可读
R=1,此段为代码段,可读

注意,代码段是决不允许写入的,而堆栈段必须能写入。有关类型(Type)与保护机制的关系,说明如下:

在保护模式下,与内存打交道的指令,处理器都要对其进行类型检查,当某一个涉及到内存段描述符的段选择子要装到段寄存器时,会进行类型检查以确定该选择子能否装入段寄存器。例如,只有段描述符类型为可执行的段选择子,才能装入 CS 寄存器。

例 13-3

```
MOV AX, 段选择子      ;AX←选择子
MOV CS, AX             ;CS←选择子
```

通过上述两条指令,试图将段选择子装入 CS 寄存器,但只有当该选择子指向的段描述符中 Type=8~15 时,也就是可执行/只执行类型才允许装入,否则会产生异常。

同样,只有 Type=2,3,6,7,也就是可读可写的描述符的选择子,才能装入 SS 寄存器。而 Type=8,9,12,13,即不可读的段描述符的选择子,不能装入数据段寄存器 DS。

下面举例说明内存段描述符的设置方法。

例 13-4 如内存段描述符的 8 个字节为 00CF 9A00 0000 FFFFH,试说明该描述符的含义。

该内存段描述符可以用图 13-18 来表示,图中将用 16 进制数表示的 8 字节内存段描述符改成用二进制数来表示。

由图 13-18 可知,段基址位 31~24=00H,位 23~16=00H,位 15~0=0000H,所以段基址位 31~0=0000 0000H;段限长位 19~16=0FH,位 15~0=FFFFH,所以段限长位 19~0=FFFFH。由于 G=1,所以限长应以 4KB 为单位。

$$\begin{aligned}\text{实际限长} &= \text{FFFFH} * 1000\text{H} + \text{FFFH} \\ &= \text{FFFF FFFFH}\end{aligned}$$

属性位 D=1,表示此段为 32 位代码;

DPL=00,表示代码段的优先级为 0 级,属于最高级;

P=1,说明该段已装入主存。

S=1,该段为系统段

TYPE=1010,说明该段是可执行、可读的代码段。

31																16	15																0
段基址 15~0																段限长 15~0																	
0000 0000 0000 0000																1111 1111 1111 1111																	

31																34	23	22	21	20	19																16	15	14	13	12	11																8	7																0																				
基址 31~24																G	D	0	AVL	段限长 19~16																P	DPL																S	TYPE																基址 23~16																									
0000 0000																1100																1111																1001																1010																0000 0000															

图 13-18 例 13-4 内存段描述符的图示

例 13-5 有一个 32 位的堆栈段,段基址为 8ABE F000H,该段限长为(32K-1)字节,段的优先级为 0,试求出堆栈段的描述符。

可以按照以下步骤来求解:

①将基地址 8ABE F000H 分 3 段填入相应字段;

②因为段限长 = 32K - 1 = 7FFFH,用 20 位字段可以表示限长,所以取 G=0,不用左移;

③因为是堆栈段,类型需置成可读/写,向下扩展限长,未访问过,且是数据段而非代码段,所以 TYPE=6=0110;

④其余位 DPL、P、S 与上例相同。

综合以上结果,就可以得到如图 13-19 所示的堆栈段描述符。

31																16		15		0															
段基址 15~0																段限长 15~0																0~3			
F0 00																7F FF																			

31																24		23		22		21		20		19		16		15		14		13		12		11		8		7		0																				
基址 31~24																G		D		0		AVL		段限长 19~16																P		DPL		S		TYPE		基址 23~16																4~7
8A 40																96 BE																																																

图 13-19 例 13-5 的堆栈段描述符表示

(2) 系统段描述符和门描述符

除了内存段描述符外,32 位处理器还有两类描述符,它们分别是系统段描述符和门描述符,这两类描述符表中的属性位 S=0。系统段描述符用于描述有关 80386 操作系统的表 and 任务等信息。系统中,任务状态段(Task State Segment, TSS)为系统段,局部描述符表 LDT 也作为一种系统段来看待。

下面先叙述任务状态段 TSS 和门的基本概念,再介绍各种描述符的格式和含义。

①任务状态段 TSS

TSS 是为了在发生任务切换时保存环境而设计的一种数据结构。80386 中, 每一个任务都有自己的 TSS, 用于保存任务的环境。TSS 属于系统段, 它在内存中由低位和高位两部分组成。低位部分由微处理器定义, 它对应一个任务的各种信息, 存放各种寄存器的值, 占用 104(0~67H) 个字节。高位部分可以由操作系统定义, 存放与此任务有关的数据, 如调度优先级、文件描述符表、外设 I/O 寄存器的值以及任务的页目录、页表等。为创建一个新任务, 操作系统必须为其建立一个 TSS, 并将其初始化, 然后由微处理器自动维护低位部分, 而操作系统维护高位部分。

TSS 低位部分的格式如图 13-20 所示。低位部分的地址为 00H~67H, 从位 68H 开始为高位部分。

以上为高位部分			68H
I/O 位图偏址	0	T	64H
0	LDT		60H
0	GS		5CH
0	FS		58H
0	DS		54H
0	SS		50H
0	CS		4CH
0	ES		48H
EDI			44H
ESI			40H
EBP			3CH
ESP			38H
EBX			34H
EDX			30H
ECX			2CH
EAX			28H
EFLAGS			24H
EIP			20H
CR3			1CH
0	SS2		18H
ESP2			14H
0	SS1		10H
ESP1			0CH
0	SS0		08H
ESP0			04H
0	LINK		00H

图 13-20 TSS 低位部分的格式

②门(Gate)

门实际上是一种转换机构。在保护模式下, 当程序的控制由一个代码段(源代码段)转到另一个目标代码段时, 往往通过门来实现。门是在目标代码段的入口处设置的, 用于控制对该目标代码段访问的权限。门的类型有 4 种: 调用门(Call Gate)、任务门(Task Gate)、中

断门(Interrupt Gate)和陷阱门(Trap Gate)。调用门用来改变任务或程序的特权,通过调用门,可以使优先级别低的代码转到优先级高的或同级代码去。任务门用来切换任务,中断门和陷阱门用于指定系统的中断和异常处理程序。

③系统段描述符和门描述符中 TYPE 字段的含义

系统段描述符、门描述符与内存段描述符的大部分字段是相同的,但当 $S=0$,选中系统段描述符和门描述符时,TYPE 字段的含义与内存段的是不一样的。表 13-3 列出了系统段描述符和门描述符中 TYPE 字段的含义。

表 13-3 系统段描述符和门描述符中 Type 字段的含义

Type Field					Descriptor	中文含义
Decimal	11	10	9	8		
0	0	0	0	0	Reserved	保留
1	0	0	0	1	16-bit TSS(Available)	16 位有效任务状态段
2	0	0	1	0	LDT	LDT 描述符
3	0	0	1	1	16-bit TSS(Busy)	16 位忙碌任务状态段
4	0	1	0	0	16-bit Call Gate	16 位调用门
5	0	1	0	1	Task Gate	任务门
6	0	1	1	0	16-bit Interrupt Gate	16 位中断门
7	0	1	1	1	16-bit Trap Gate	16 位陷阱门
8	1	0	0	0	Reserved	保留
9	1	0	0	1	32-bit TSS(Available)	32 位有效任务状态段
10	1	0	1	0	Reserved	保留
11	1	0	1	1	32-bit TSS(Busy)	32 位忙碌任务状态段
12	1	1	0	0	32-bit Call Gate	32 位调用门
13	1	1	0	1	Reserved	保留
14	1	1	1	0	32-bit Interrupt Gate	32 位中断门
15	1	1	1	1	32-bit Trap Gate	32 位陷阱门

从表 13-3 可以看到:

- 当 $Type=1, 3, 9$ 和 11 时,为任务状态段 TSS 的描述符。TSS 中包含一个任务的全部信息以及与嵌套任务连接的信息(TSS 表中的 LINK 字段),包括关于某一个任务状态段的位置、长度和优先级的信息。TSS 处于忙碌状态时,意味着本任务为当前任务;如不忙,则指出该任务是否有效。类型字段也指出对应段是 16 位(80286)的任务还是 32 位(80386)的任务状态段。之所以要包括 16 位的段描述符,是为了与 80286 应用程序兼容。

- $Type=2$ 时,对应一个局部描述符表 LDT。对一个特定的任务来说,只有一个 LDT。在多任务系统中,会有多个 LDT 描述符,它们分别对应于各个任务的 LDT。

- $Type=4, 5, 6, 7, 12, 14, 15$ 时,均为门描述符。

- 其余情况,即 $Type=0, 8, 10, 13$ 为保留。

④任务状态段 TSS 的描述符

TSS 位于内存中,微处理器通过任务状态寄存器 TR 和 TSS 描述符来定位 TSS 段。TSS 描述符的格式如图 13-21 所示。

基址 15~0					限长 15~0					0
基址 31~24	G	0 0	AVL	限长 19~16	P	DPL	0	TYPE	基址 23~16	+ 4

图 13-21 TSS 描述符的格式

由图 13-21 可知,TSS 描述符定义了任务状态段在内存中的线性基地址、段限长和类型。根据表 13-3 可知,类型字段 Type=1、3、9、11 分别用来定义 16 位有效的 TSS、16 位忙碌的 TSS、32 位有效的 TSS 和 32 位忙碌的 TSS。对当前正在执行的任务,微处理器会自动将其类型置成“忙碌”状态。如果企图切换到一个忙碌的 TSS,则会产生一般性保护异常。之所以要包括 16 位的段描述符,是为了与 80286 兼容。另外要指出的是,TSS 描述符只能放在 GDT 中,不能放在 LDT 中,否则会引起非法的 TSS 异常。TSS 的限长必须大于等于 67H,这是因为 TSS 的低位部分的偏移量为 0~67H。如果限长小于 67H,当切换到此 TSS 对应的任务时,会产生非法 TSS 异常(异常号为 0AH)。

LDT 描述符与 TSS 描述符的格式是类似的,只是类型字段 type=0010。但门描述符与系统段描述符的格式有些差别,下面进行讨论。

⑤门描述符

门描述符的格式如图 13-22 所示。

段选择子 15~0					偏移量 15~0					0
偏移量 31~16	P	DPL	0	TYPE	双字计数器					+ 4

图 13-22 门描述符格式

从图中可以看到,门描述符的属性位存放在字节 4 和字节 5 中,其意义如下:

P 存在位 P=1,表示门有效;P=0,表示门无效,使用无效门将引起异常。

DPL 描述符优先级 定义访问该门的任务应具备的优先级。

TYPE 域 用来定义门的类型 它可以定义 16/32 位的调用门、任务门、中断门和陷阱门,如表 13-3 所示。

对于调用门描述符,它的段选择子和偏移量指向另一个存储段,也就是要调用的目标代码的起始地址。双字计数器仅用于调用门,它用来表示要传递参数的个数。

任务段描述符中,只有段选择子起作用,它指向 GDT 中的一个 TSS 描述符;偏移量则不起作用。

中断门和陷阱门的描述符有一点区别,进入中断门时,标志寄存器 EFLAGS 中 IF 标志自动清 0,关闭中断,而进入陷阱门时不会关闭中断。这两种门描述符中的选择子和偏移量构成中断处理子程序或陷阱处理程序的入口地址。

二、分页内存管理技术

分页内存管理是 80386 对 8086 微处理器的主要功能扩充。采用分页管理,便于实现虚拟存储器管理,可以方便地以页为单位把内存空间映射到磁盘空间,分页还能明显提高存取数据的效率,有效利用内存碎片。

80386 采用固定大小的页,以 4KB 作为一个页。4GB(2^{32})内存空间可以被分成 2^{20} 个页面,能被 4KB 整除的那些地址处可以开始分页,如从 00000000H,00001000H,00003000H 处都可以开始分页。

分段技术将逻辑地址转换成了线性地址。当控制寄存器 CR0 的 PE 字段设为 0 而禁止分页时,线性地址就是物理地址;在 PE=1 允许分页时,需通过分页部件把将线性地址转换成物理地址。如何进行分页?怎样将线性地址转换成物理地址?下面将讨论这些问题。

1. 页目录与页表

80386 采用了两层表来实现分页管理。第一层表称为页目录(Page Director),第二层表称为页表(Page Table)。页目录和页表结构如图 13-23 所示。

页目录表中包含 $2^{10}=1024$ 个页目录项(Director Entry),每项为 4 字节(32 位);页表中也包含 1024 个 32 位的页表项(Page Table Entry),每个页表项对应一个 4KB(即一页)的连续物理内存空间。每个页目录项对应的页表最多可以有 1024 个,故整个页目录最多可映射的物理地址空间的大小为:

$$1024 \text{ 页目录项} \times 1024 \text{ 页表项/页目录项} \times 4\text{KB/页表项} = 4\text{GB}$$

分页机制将 32 位线性地址分成 3 部分:

- 线性地址的高 10 位(位 31~22)作为页目录的索引号,指向 $2^{10}=1024$ 个页目录项中的某一项。页目录项中每项长度为 32 位,其中高 20 位指向页表基地址,低 12 位为其属性。
- 线性地址的中间 10 位(位 21~12)作为页表的索引,指向 1024 个页表项中的某一项,页表项中每项长度也是 32 位,其中高 20 位对应物理地址的高 20 位,这 20 位物理地址也称为页帧(Page Frame),低 12 位为其属性。
- 线性地址的低 12 位(位 11~0)作为页面的偏移地址,也就是物理地址的低 12 位。

为什么要采用看起来比较麻烦的两级页表结构来进行页管理呢?这是因为 80386 中页表项共有 $2^{20}=1024 \times 1024$ 个,每项占 4 字节,如果所有的页表项都存储在一张表中,则该表将占用 $2^{20} \times 4 \text{ 字节} = 4\text{MB}$ 字节的连续物理存储空间。为避免页表占有如此巨大的物理存储空间资源,所以页表采用了两级表的结构,每级 10 位,2 张表总共只需占用 $(1024 \text{ 项} \times 4 \text{ 字节/项}) \times 2 = 1024 \times 8 = 8\text{KB}$ 内存空间。

图 13-23 所示的两层页表的工作过程大致如下:

①32 位的控制寄存器 CR3 指向页目录的基地址,用来存放页目录表在存储器中的起始物理地址。由于页目录表占用 4K 字节,CR3 的低 12 位一般都设为 0,保证页目录表始址的选取总是从能被 4K 整除的地方开始。例如,设 CR3=0000 8000H,则页目录表的基地址为 0000 8000H,表占用的地址范围为 0000 8000H~0000 8FFFH。但 CR3 寄存器的位 4 和位 3 可以分别定义为 PCD 和 PWT 字段。当 CR3 的低 12 位都为 0 时,表示允许片内页缓存与回写。

②10 位页目录索引号指向 1024 个页目录项中的某一项。

③每个页目录项占 4 字节(32)位,其中高 20 位为下一层表即页表的基地址,低 12 位为其属性。

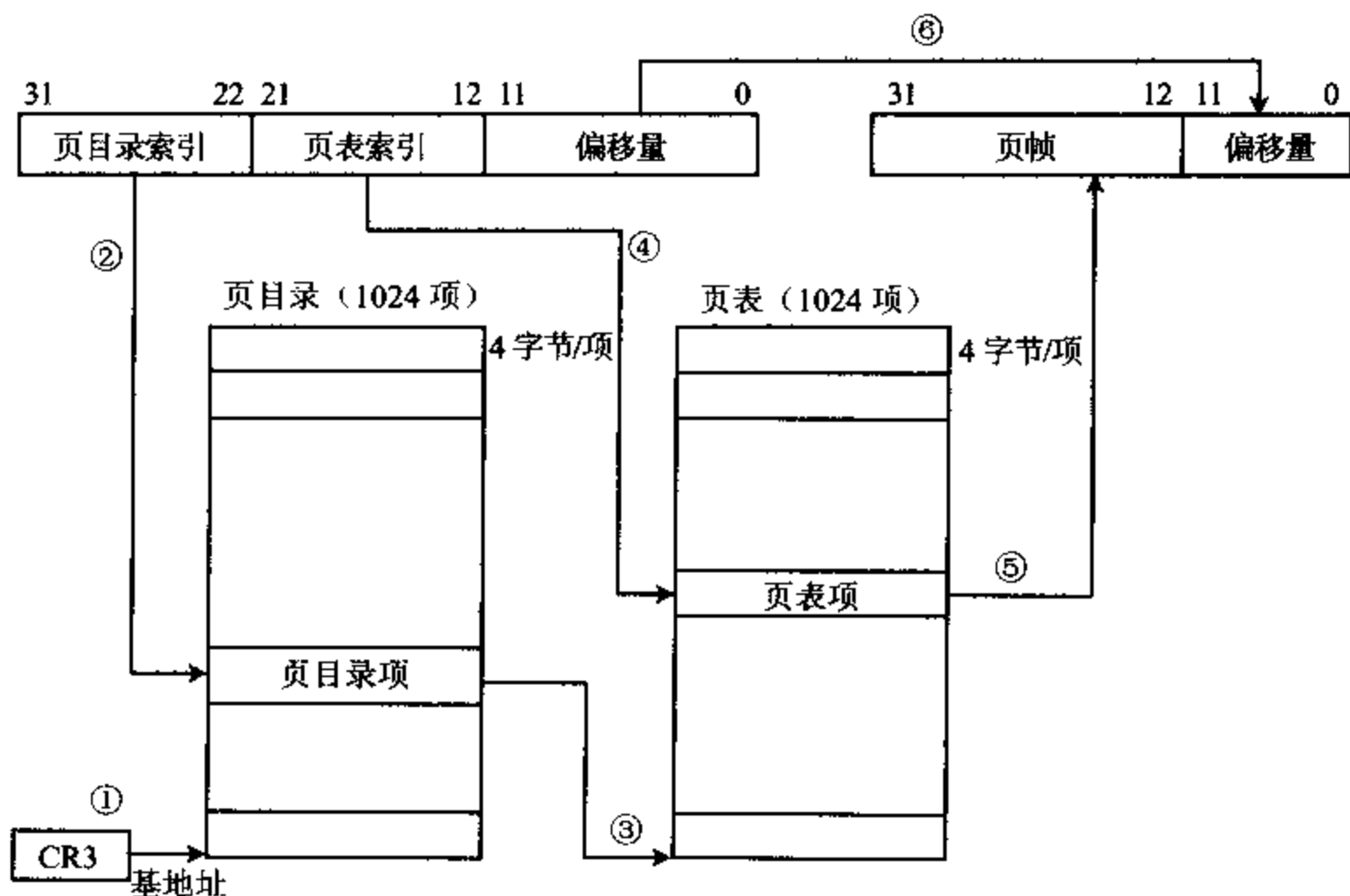


图 13-23 页目录和页表结构

④10 位页表索引号指向 1024 个页表项中的某一项。

⑤每个页表项也占 32 位。根据索引号从页表项中取出某一项,32 位中的高 20 位作为物理地址的高 20 位,即页帧。

⑥线性地址的低 12 位直接指向物理地址的低 12 位。

这样,利用两层表,加上 CR3 寄存器,就可以从 32 位线性地址求得 32 位物理地址了。

2. 页目录项和页表项格式

前面讲到,每个页目录项和页表项都由 4 字节(32 位)组成,我们把这些项称为描述符。它们的格式基本相同,高 20 位为页帧地址,低 12 位为属性。80486 和 Pentium 的页表项格式如图 13-24 所示,80386 的页表项中,位 4 和位 3 均被清 0。页目录项格式也与此类似。

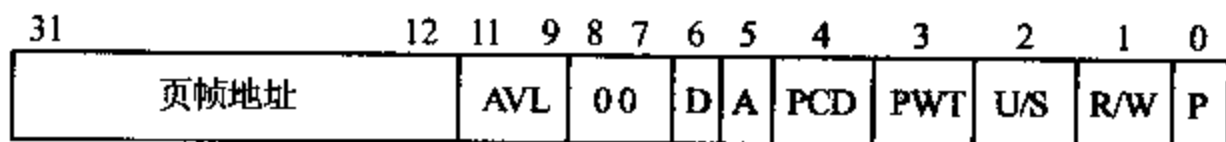


图 13-24 80486 和 Pentium 的页表项格式

(1) 页帧地址

页帧地址实际上就是一个页的物理地址的起始基地址,也就是物理地址的高 20 位,从位 31~12。对于页目录项,页帧地址给出页表的起始地址的高 20 位。

(2) 属性位

属性位由位 12~0 给出,它们包括以下这些属性:

• P(Present)存在位

$P=1$,表示页表或页目前在主存中。

$P=0$,对应的页不在主存中,要从磁盘中调出来。若此时访问该页,则会产生页异常(异常 14),并把引起异常的线性地址保存到 CR2 寄存器中。产生页异常时,操作系统会把此页表或页从磁盘中取出,装到主存中,并重新启动刚才引起异常的指令。对于应用程序来说,完全觉察不到所发生的一切,仿佛允许它访问 64TB 的地址空间一样。

• R/W 读写位

$R/W=1$,表示此目录项/页表项是可读/写、可执行的。

$R/W=0$,则表示该项可读、可执行,但不能进行写操作。

对于优先级为 0、1、2 的程序,此项被忽略。

• U/S (User/Supervisor) 用户/超级用户位

$U/S=1$,程序可在任意优先级上执行,包括在用户级即 3 级上执行。

$U/S=0$,程序只能在 0、1、2 级上执行。

• A(Access)存取位或访问位

当此页目录或页表被存取时,386 将此位置 1,表示该项已被访问过。

• D(Dirty)脏位

是已写标志位,用来报告对某一页的写操作的情况。当某一页从磁盘调进内存时,操作系统使 $D=0$,如以后对此页进行过写操作,则使 D 位置 1,并保持 1。如选中某页调往磁盘时, D 仍为 0,说明此页一直没有被写过。如果该页是从磁盘调进来的,则当前内存中的页内容与磁盘中的一致,无需再写入磁盘,节省了这一操作过程,在页目录项中 D 位无意义,因为此类项不会有写操作。可见, A 、 D 位可以用来跟踪页的使用情况。

• AVL 域

这两位一般供操作系统记录页的使用情况,如记录页面使用次数等,处理器不会对其进行修改。

• PWT(Page Write Transparent)页透明写位 用于控制写策略。

$PWT=1$,表示当前页采用通写政策。

$PWT=0$,允许回写。

• PCD(Page Cache Disable)页 Cache 禁止位 用于控制超高速缓存。

$PCD=1$,允许在片内超高速缓存器中进行超高速缓存操作。

$PCD=0$,禁止超高速缓存操作。

3. 将线性地址转换成物理地址的实例

前面已介绍了通过两级页表转换,将线性地址转换成物理地址的大致工作过程,下面举一个实例来进一步具体说明线性地址是如何转换成物理地址的。

例 13-6 设系统的线性地址为 1234 5678H,CR3=0000 8000H,求对应的物理地址,并说明转换过程。

可以将 32 位线性地址分成 3 部分:最高 10 位为 00 0100 1000B=048H,将其作为页目录索引;中间 10 位为 11 0100 0101B=345H,作为页表索引;后 12 位为 678H,不用转换,直

接作为 12 位物理地址。转换过程示意图如图 13-25 所示。

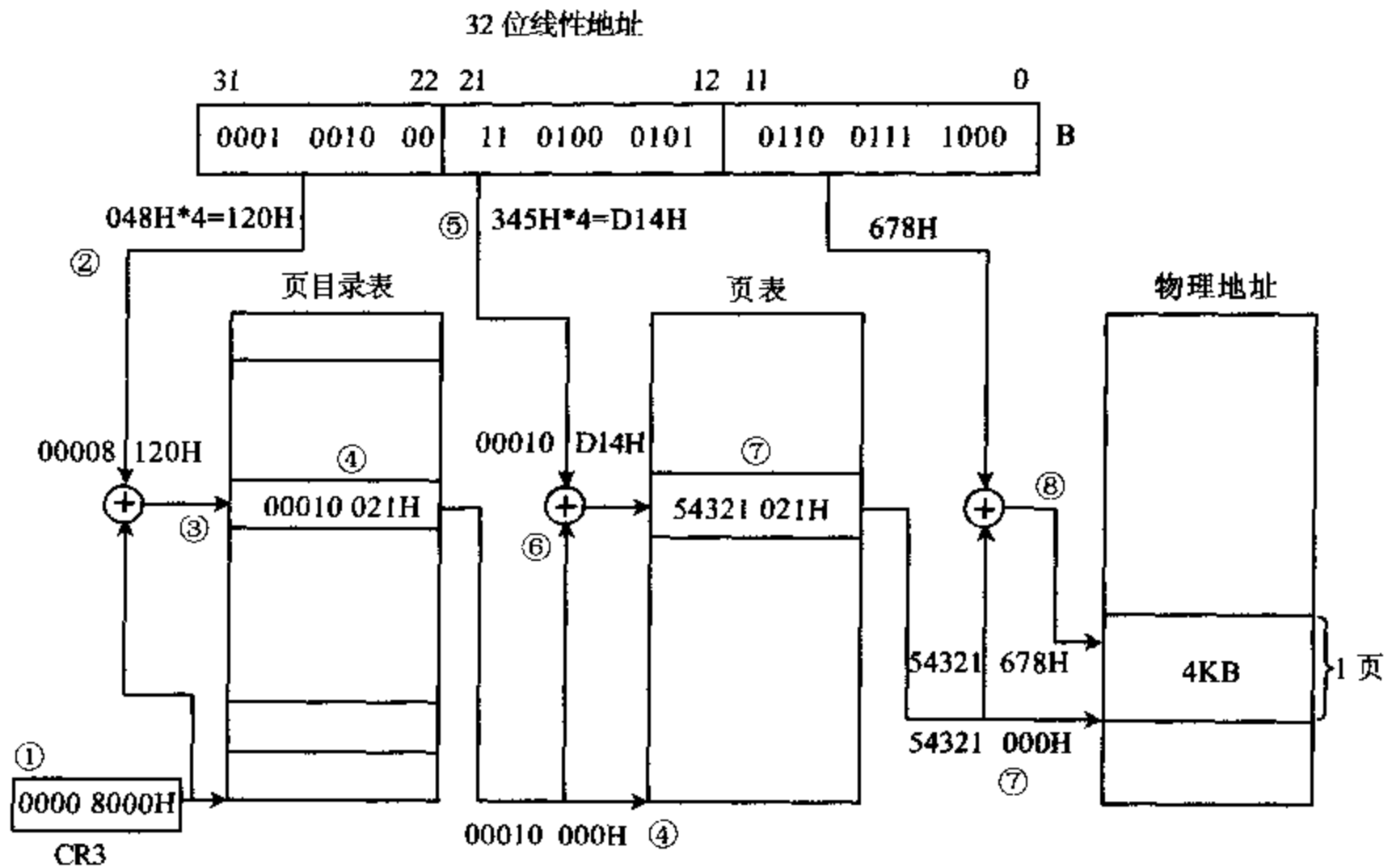


图 13-25 线性地址转换成物理地址的实例

转换分以下几步进行：

①先查询 CR3,得知 $CR3=0000\ 8000H$,将其作为页目录表的物理基地址。

②取线性地址的高 10 位 $00\ 0100\ 1000B=048H$,作为页目录索引号。由于每个目录项占 4 个字节,所以要将索引号乘以 4 即左移 2 位,才能得到页目录项的首字节地址,也就是页目录项的偏移地址。本例中偏移地址为 $048H \times 4=120H$ 。

③求页目录项始址的物理地址。其值 = CR3 中的基地址 + 页目录项的偏移地址 = $00008000H+120H=00008120H$ 。

④查页目录项的内容。假设查得该目录中的 4 字节内容($08120H$)= $00010021H$,其中高 20 位为 $00010H$,它将作为下一级表即页表的基地址的高 20 位。而低 12 位为属性,它等于 $021H$,根据图 13-24 所示的页表项属性位内容可知:

$P=1$,页表存在

$A=1$,该目录被访问过

$U/S=0$,只能在 0、1、2 级上访问

$R/W=0$,由于 $U/S=0$,该项被忽略

$D=0$,对页目录项无意义。

⑤页表索引序号由线性地址的中间 10 位给出,其值为 $345H$ 。同样因为页表项每项占 4 字节,所以页表项的偏移地址为 $345H \times 4=0D14H$ 。

⑥页表项的物理地址为其基地址 + 偏移量 = $00010000H+0D14H=00010D14H$ 。

⑦从指定页表项中查得其内容,即($010D14H$)= $54321021H$ 。其中高 20 位 $54321H$ 即

为物理地址的页帧值,即该页的基地址为 54321000H。后 12 位为属性,它与页目录的属性相同。

⑧物理地址=页帧+线性地址=54321000H+678H=54321678H。

至此,通过一个实例,我们从线性地址求得了物理地址。

在保护模式下,对于 Pentium Pro 及以上的处理器的物理地址扩展后可以访问 36 位即 64GB 的物理地址空间。每页大小可以是 4KB、2MB 或 4MB。

13-4 32 位机新增指令和程序设计简介

8086 CPU 只能处理 8 位和 16 位操作数,而 80386~Pentium 除了保持与 8086 兼容,可以直接处理 8 位、16 位操作数外,还增加了许多新的指令,并能直接处理 32 位操作数。80486 DX 和 Pentium 芯片内还包含一个数字协处理器,可以用它完成复杂的浮点运算。对于 8086 和 80386,协处理器作为独立的部件,置于 CPU 芯片之外。

80386 和 80387 的指令相当丰富,有很强的功能和很大的灵活性,按用途可分为三大类:整数指令、操作系统指令和浮点指令。本章主要介绍部分整数指令,重点介绍在 8086 指令基础上功能方面有较强扩充的那些新指令,介绍它们的寻址方式、指令特点、指令功能,并给出几个程序设计实例,使大家能初步掌握 32 位微处理器的汇编语言编程设计方法。至于 P4 处理器的 SIMD 指令的编程方法,比起 80386 来说又要复杂得多,也非常有用,特别是在 3D 图像处理等方面,有许多独到之处,限于篇幅,在此不作介绍了。

一、80386 的寻址方式

80386 的寻址方式与 8086 类似,也可以分成三大类:立即数寻址、寄存器寻址和存储器寻址方式,但操作数可以是 8 位、16 位和 32 位数。下面以 MOV 指令的源操作数为例来概要介绍这几种寻址方式,主要介绍 32 位存储器寻址方式。

1. 立即数寻址方式

在这种寻址方式下,操作数以立即数的形式出现在指令中。

例 13-7

```
MOV    EBX, 12345678H           ;EBX←12345678H
MOV    DWORD PTR[MEM], 0100FF20H ;双字存储单元←0100FF20H
```

2. 寄存器寻址方式

操作数在寄存器中称为寄存器寻址方式。寄存器间、寄存器与存储器间可以互相传送数据。

例 13-8

```
MOV    EAX, EBX                 ;EAX←EBX
MOV    M_DWORD, EDX             ;存储单元←EDX
```

要注意的是对于 MOV 指令,目标操作数与源操作数的长度必须一致,段寄存器之间不能传送数据,CS 不能作目的操作数,因此下面 3 条指令是非法的:

MOV	AL, EBX	;两操作数长度不一致
MOV	DS, ES	;段寄存器间不能传送数据
MOV	CS, BX	;CS不能当目的操作数

3. 存储器寻址方式

与 8086 一样,在这种寻址方式下,指令的操作数都放在存储器中,需用不同的方法求得操作数的物理地址,来获得操作数。按照 80386 的存储器组织方法,应用程序设计人员可使用的地址称为逻辑地址。在保护模式下,有了逻辑地址,计算机会自动计算出线性地址,并把它转换成物理地址,再从中取出存储器操作数,或者将别的操作数送至该存储单元。

逻辑地址由段选择子和偏移地址两部分组成,段选择子存放在段寄存器中,为方便起见,这里用“段寄存器:偏移量”来表示逻辑地址,可以用显式、隐式或默认方式来指定段寄存器。由段选择子可以从 GDT 或 LDT 表中找到相应的段描述符,从而获得段基地址等信息。段基地址加上偏移地址等于物理地址。偏移地址的计算公式为:

偏移地址 = 基址 + 变址 * 比例因子 + 位移量

基址寄存器: EAX、EBX、ECX、EDX、EBP、ESP、EDI、ESI

变址寄存器: EAX、EBX、ECX、EDX、EBP、EDI、ESI

比例因子: 1、2、4、8

位移量: 8 位/32 位立即数

在指令中,对构成偏移地址的基址和变址寄存器要加上方括号[],例如,[EAX+40H],[EBP+EAX*4+100H]等都表示存储器操作数。当这些操作数中的方括号内出现 ESP 或 EBP 时,则段寄存器使用 SS,其余情况默认用 DS 作段寄存器。如果要用 ES、FS 或 GS 作段寄存器,必须采用段超越前缀来表示,如 FS:[EBX+EAX],GS:[ECX+EAX*8+80H]。

按照偏移量的计算方法,4 个分量可以有不同的组合,又构成了不同的寻址方式,从而使 32 位处理器可以在 4GB 范围内寻址。下面仅举几个例子来说明。

例 13-9

MOV	EDX, [EAX + ESI]	;源操作数的地址为 DS : [EAX + ESI]
		;采用基址变址寻址方式

例 13-10

MOV	EAX, [ESI+EBP + 54]	;源操作数的地址为 SS : [ESI + EBP + 54]
		;采用带位移量的基址变址寻址方式

例 13-11

MOV	BX, [EDX+4 * ESI+60H]	;源操作数为 DS : [EDX + 4 * ESI + 60H]
		;采用带位移量的基址比例因子寻址方式

处理器可以工作在实模式下,在这种模式下,存储器操作数也是由段寄存器和偏移量两部分组成,段寄存器中存放的是段基地址,物理地址=段基址*10H+偏移量。

偏移地址的表示方法与保护模式下的公式是一样的,即也可以用基址、变址、比例因子和位移量 4 部分来表示,而且基址和变址可以用 32 位寄存器来表示,只是计算物理地址的方法与保护模式下的不一样。例如,对于下述指令:

MOV EAX, [EBX+ESI*8+25H]

在实模式下,物理地址等于 DS*10H+(EBX+ESI*8+25H)。

在保护模式下,段寄存器也是 DS,但它存放的不再是段基地址,而是段选择子,这时不能再按实模式下的计算公式去求物理地址,而是要采用 13-3 节所讲的分段和分页技术去求得物理地址。我们主要考虑处理器工作于保护模式下情形。

二、80386 的指令系统

80386 的许多指令与 8086 兼容,只是操作由 8 位、16 位变成 32 位,有关这方面内容就不介绍了,下面介绍一些新增加的指令。

1. 数据传送指令

(1) 通用数据传送指令

当源操作数和目的操作数的长度不一样时,是不能用 MOV 指令来传送数据的,但如果要传送数据,可以使用零扩展指令 MOVZX 和符号扩展指令 MOVSX 来实现。

例 13-12

MOVZX AX, BH

设该指令执行前,AX=1234H,BH=9EH,指令执行过程中,将 BH 中的内容 9EH 零扩展为 009EH,再传送至 AX。

所以指令执行后,AX=009EH,BH=9EH

例 13-13

MOVSX AX, BH

设该指令执行前,AX=1234H,BH=9EH=10011110B,指令执行过程中将 9EH 用符号扩展为 FF9EH,即把 BH 寄存器的符号位“1”扩展到目的操作数 AX 的高半部分 AH 中。

所以指令执行后,AX=FF9EH,BH=9EH

(2) 堆栈操作指令

对于堆栈操作指令 PUSH 和 POP,也有扩展功能。

PUSH 指令的操作数除了可以是 16 位或 32 位的寄存器/存储器外,还可以是 8/16/32 位立即数,但 POP 指令的操作数不能用立即数。

例 13-14

PUSH 1234H ;将立即数 1234H 压入堆栈

PUSH EAX ;将 32 位寄存器 EAX 的内容压入堆栈

但是下面的指令是错误的。

POP 5678H ;POP 指令不能用立即数作操作数。

此外还增加了 PUSHAD 和 POPAD 指令,用一条指令就可以将所有通用寄存器的内容压入堆栈。这样在编程序时可以节省许多指令。

例 13-15

PUSHA ;将 AX,CX,DX,BX,SP,BP,SI,DI 顺序压入堆栈

PUSHAD ;将 EAX,ECX,EDX,EBX,ESP,EBP,ESI,EDI 顺序压入堆栈

用 POPA 和 POPD 指令可以进行相反的弹出操作。

(3) 地址目标传送指令

这类指令用来传送 6 字节地址指针,地址指针存放在 6 个连续的存储单元中,目的地址

为“段寄存器:双字通用寄存器”。这类指令中,源操作数必须是存储器操作数;目的操作数为双字通用寄存器,段寄存器隐含在操作码中。

例 13-16

LDS EBX, MEM	;DS : EBX←MEM 单元开始的内容
LES EDI, MEM	;ES : EDI←MEM 单元开始的内容
LSS ESP, MEM	;SS : ESP←MEM 单元开始的内容
LFS EDX, MEM	;FS : EDX←MEM 单元开始的内容
LGS ESI, MEM	;GS : ESI←MEM 单元开始的内容

这些指令适用于 32 位微机的多任务操作系统中,使一条指令完成多种功能。

2. 算术运算指令

算术运算指令包括加、减、乘、除 4 种运算,还有一些调整指令和数据类型转换指令。下面主要介绍乘法扩展指令和数据类型转换指令。

(1) 乘法指令

8086 中,无符号数乘法指令 MUL 和整数乘法指令中都只能有一个源操作数,而且该操作数不能是立即数,另一个操作数默认放在累加器中。例如指令 IMUL BL,表示将 AL 与 BL 相乘后,乘积送入 AX 中。32 位乘法指令允许有 2 个或 3 个操作数,而且操作数可以是立即数。两种扩充指令如下:

• 立即数乘法指令

这类指令允许用一个立即数与寄存器或存储器操作数相乘,结果放在指定的寄存器中,立即数乘法指令只能用于带符号数相乘的场合。

例 13-17

IMUL EBX, 10	;EBX←EBX * 10
IMUL BX, CX, 123	;BX←CX * 123
IMUL EAX, EBX, 40H	;EAX←EBX * 40H
IMUL ECX, [EBX+EDI], 50	;ECX←50 * 双字存储单元内容

• 寄存器与存储器相乘指令

允许任一个 8 位、16 位或 32 位的寄存器操作数和另一个相同长度的寄存器或存储器操作数相乘,结果放在寄存器中。

例 13-18

IMUL AX, BX	;AX←AX * BX
IMUL EAX, EDX	;EAX←EAX * EDX
IMUL ECX, MEM_DW	;ECX←内存单元内容 * ECX

上述两类 IMUL 扩充指令中,因被乘数、乘数和积的长度一致,相乘后就可能发生溢出,溢出时,OF 标志置 1。这两类扩展指令增加了灵活性,但因容易产生溢出而限制了它们的应用。

(2) 数据类型转换指令

在 8086 中有两条指令可以完成数据类型转换:一条是 CBW 指令,将字节转换成字;另一条是 CWD 指令,将字转换成双字。80386 中除了有上述两条指令外,又增加了以下两条指令:

CWDE	;将 AX 中的字扩展成 EAX 中的双字
------	-----------------------

CDQ

;将 EAX 中的双字扩展成 EDX, EAX 中的 4 字

这些指令的操作数都在累加器中,通常安排在除法指令前面,用于产生双倍字长的被除数,将累加器中的符号位进行扩展。

例 13-19 对于 CWDE 指令

如指令执行前 AX=1234H, 则指令执行后, EAX=00001234H

如指令执行前 AX=8000H, 则指令执行后, EAX=FFFF8000H

3. 移位指令

80386 增加了两条多字节左移指令 SHLD 和多字节右移指令 SHRD, 也称为双精度移位指令。这类指令的格式为:

SHLD OP1, OP2, imm8/CL

SHRD OP1, OP2, imm8/CL

它们有 3 个操作数,第一操作数 OP1 可以是 16 位/32 位的寄存器或存储器,第 2 操作数 OP2 为 16 位/32 位的寄存器,第 3 操作数为 8 位立即数或 CL 寄存器,后者用来表示移位次数。

例 13-20

SHLD EAX, EBX, 4

其功能是将 EAX 左移 4 位,将 EBX 的高 4 位移入 EAX 的低 4 位。(也可把 EAX 和 EBX 看成一个 64 位的双精度操作数,各位均左移 4 位)

设指令执行前: EAX=12345678H, EBX=87654321H

指令执行后: EAX=23456788H, EBX=76543211H

例 13-21

SHRD ECX, EDX, CL ;设 CL=4

指令功能:将 ECX 的内容右移 4 位,高 4 位由 EDX 的低 4 位补充。

设指令执行前: ECX=12345678H, EDX=87654321H

指令执行后: ECX=11234567H, EDX=88765432H

4. 字符串操作指令

(1) 基本字符串操作指令

80386 的字符串操作指令与 8086 的基本一样,也包含字符串传送 MOVS、字符串比较 CMPS、字符串扫描 SCAS、字符串装入 LODS 和字符串存储 STOS 共 5 大类指令。每类指令除了可进行字节、字操作外,还可按双字操作。下面以字符串传送指令为例进行说明。

字符串传送指令可以有 4 种形式:MOVS、MOVSB、MOVSW 和 MOVSD。它们均用 ESI 作源变址寄存器,EDI 为目的变址寄存器,ECX 中存放要传送的字节、字或双字数。

例如,双字字符串传送指令 MOVSD 的功能为:将以 DS:[ESI]为始址的双字传送到以 ES:[EDI]开始的地址单元中去,每传送一个双字,ESI 和 EDI 分别增 4 或减 4,若 DF 标志为 0,则加 4,如 DF=1,则减 4。但 CX 在每次传送后,仍减 1,表示传送了一个双字。下面再用具体的例子来进一步说明 MOVSD 的操作过程。

例 13-22

MOVSD

设指令执行前: ESI=00112233H, EDI=44556677H

DS:[ESI]中的内容=12345678H(源地址内容)
ES:[EDI]中的内容=87654321H(目的地址内容)
DF=0,ECX=5

指令执行后: ESI=00112237H(加4),EDI=4455667BH(加4)
DS:[ESI]中的内容=12345678H(不变)
ES:[EDI]中的内容=12345678H(由源地址单元送过来)
ECX=4(减1)

使用字符串操作指令时,也可以在指令前加重复前缀 REP、REPE、REPZ、REPNE、REPNZ,以便重复执行串操作。

(2) 输入输出字符串操作指令

80386 还增加了两条输入串操作指令 INS 和输出串操作指令 OUTS。INS 指令允许从一个输入端口读入一串数据,传送到以 ES:[EDI]为始址的一连串存储单元中,OUTS 指令则可以从以 DS:[ESI]开始的连续存储单元向输出口写入一串数据。端口号必须放在 DX 寄存器中。

INS 指令可以有 INSB、INSW 和 INSD 三种形式,分别表示字节串、字串或双字串输入操作。每输入一个数据,EDI 增或减 1、2、4。DF=0 时,为增量操作,DF=1 时为减量操作。OUTS 指令的形式为 OUTSB、OUTSW 和 OUTSD,传送过程中,ESI 根据 DF=0 或 1 作增量或减量操作。使用输入输出字符串操作指令时,也可以在指令前加 REP 等重复前缀,以便重复执行串操作。

下面举例说明

例 13-23

INSD

设指令执行前 EDI=00000052, DX=004CH(端口号)
DF=0,DX 端口内容=FFFFD832H(源)

指令执行后 EDI=00000056(加4)
ES:[EDI]= FFFFD832H,表示将源数据传到了目的地

5. 转移指令

转移指令包含无条件转移、条件转移、调用和返回指令。下面介绍 80386 中无条件转移和条件转移指令的一些特点。

(1) 无条件转移指令 JMP

指令格式:JMP 目标地址

指令功能:表示跳转到指令中指定的偏移量处执行。目标地址可以是一个标号,包含 32 位偏移量;也可以是一个内存操作数,32 位指针位于内存中。

例 13-24

JMP label

设 label=00001234H

则指令执行后,EIP=00001234H

例 13-25

JMP [EBX+EDI*4]

设指令执行前,DS:[EBX+EDI*4]指向的 32 字节内存单元内容为 2000FAB0H。

则指令执行后,EIP=2000FAB0H。

(2) 条件转移指令 Jcc

这类指令有 JO、JNO、JB、JC 等,共有 16 种,形式与 8086 的条件转移指令相同,但 8086 相对地址用一字节表示,转移的范围仅为(-128~+127),而 80386 相对转移地址可用 4 字节表示,范围大多了。

例 13-26

JO LAB_N

如指令执行前,EIP=12340000H,近标号 LAB_N 在 JO 指令后偏移 200H。

指令执行后,如 OF=1,则 EIP=12340200H;如 OF=0,则执行下条指令。

6. 条件设置指令

指令格式:SETcc 8 位寄存器或存储器

指令中条件 cc 可以是 O、NO、C、NC 等,共有 16 种,与 Jcc 指令中的条件一样。如条件成立,则使 8 位寄存器或存储器操作数置 1,否则清 0。

例 13-27

SETZ BL ;若 ZF=1,则使 BL 置 1;否则 BL 清 0

SETNBE MEM8 ;若 CF=0 和 ZF=0(不低于等于,即高于)

;则使 MEM8 单元置 1;否则清 0

这类指令可用来帮助高级语言评估布尔代数式,简化编译过程。

7. 位处理指令

32 位系统常用来处理大块数据,但也常常要对某些位组成的阵列进行操作;另外计算机处理图像和语音数据时,也常常要用到位处理功能。为了实现位处理功能,80386 设置了位处理指令,包含位测试和位扫描指令,有些位测试指令的功能很强,也很复杂。

(1) 位测试指令

这类指令可以有如下几种形式:

BT OP1, OP2 ;位测试

BTR OP1, OP2 ;位测试,并复位

BTS OP1, OP2 ;位测试,并置位

BTC OP1, OP2 ;位测试,并求反

这类指令均有 2 个操作数,第一个操作数 OP1 可以是 16 /32 位的寄存器或存储器,第二个操作数 OP2 为 8 位立即数或 16/32 位寄存器。当 OP2 为寄存器时,其长度应与 OP1 相同。下面举例说明其功能。

例 13-28

BT EAX, 5 ;EFLAGS 的 CF 位←EAX 的位 5 的值

JC BIT5_S ;若 CF=1,则转移;否则执行下条指令

例 13-29

BTC EBX, 7 ; EFLAGS 的 CF 位←EBX 的位 7 的值

;然后再对 EBX 的位 7 求反。

设指令执行前,EBX=12345678H

指令执行后,CF=0 (因 EBX 的位 7=0)

EBX=123456F8H (EBX 位 7 取反)

例 13-30

BTR CX, AX

设指令执行前: CX=FFFFH, AX=0006H

指令功能: 将 CX 的位 6 送到 EFLAGS 的 CF 中, 再将 CX 的位 6 清 0。

所以指令执行后: CF=1, CX=FFBFH

上面的例子都是比较简单的, 它们的第二操作数 OP2 的值较小, 这时 OP1 是被测的数, OP2 指出被测位的序号。指令执行时先将被测位的值送 EFLAGS 的 CF 中去, 然后将该位或清 0 或置 1 或求反。但当 OP2 大于 8 时, 要对其求模才能得到被测位数, 而且这类指令可访问的位串的范围是很大的, 当 OP2 为 16 位操作数时, 可访问 $(-32K \sim (32K-1))$ 范围内的位串; 当 OP2 为 32 位操作数时, 可访问 $(-2G \sim (2G-1))$ 内的位串。当 OP1 为存储器操作数时待测数的地址由下式给出:

待测数的地址 = $OP1 + K * OP2 / (OP1 \text{ 的位数})$

当 OP1 的位数为 16 时, $K=2$; 当 OP1 的位数为 32 时, $K=4$ 。

例 13-31

BT EAX, 54H

指令执行前, EAX=8000ABCDH

待测位的序号 $i = OP2 \% (OP1 \text{ 的位数})$, 由于 OP1 的位数为 32 位, 所以 $i = 54H \% 32 = 20$, 也就是说要测试位串的第 20 位。接着将 EAX 的位 20 送到 EFLAGS 的 CF 中。

因为 EAX 的位 20 等于 0, 所以 CF=0。对于 BT 指令, EAX 的内容不变。

例 13-32

BT DWORD PTR ARRAY, EAX

• 指令执行前, EAX=04A6B48H, 它大于 8, 所以要对它求模。

OP1 为 32 位操作数, 所以待测位的序号为:

$i = EAX \% 32 = 04A6B48H \% 32 = 8$, 即要测试第 8 位。

• 再求待测数的地址

设 OP1 即 ARRAY=1000H, 由于 OP1 的位数为 32 位, 所以 $K=4$

待测数的地址 = $OP1 + K * OP2 / (OP1 \text{ 的位数})$

$= 1000H + 4 * 04A6B48H / 32$

$= 1000H + 4 * 2535H = 1000H + 94D4H = 0A4D4H$

表示从 1000H 处偏移了 $2535 * 4$ 个 byte (4 个字节表示双字)。

• 再从 0A4D4H 双字单元中取出一个数, 将其第 8 位送到 CF 中。

(2) 位扫描指令

有两条位扫描指令: 前向扫描指令 BSF 和逆向扫描指令 BSR。

• BSF OP1, OP2 ; 前向扫描指令

OP1 为 16/32 位的寄存器, OP2 可以是 16/32 位的寄存器/存储器, 两者长度必须一致。其功能为: 对 OP2 指定的字或双字从低位向高位(向左)扫描, 遇到第一个“1”时停止扫描, 并将其索引号送入 OP1 中。如源操作数 OP2=0, 则 ZF 置 1, 否则 ZF 清 0。

例 13-33

BSF EAX, EBX

设指令执行前:EBX=12003180H

指令功能:从低位向高位扫描,扫到位 7 时不等于 0,就将索引号 7 送入 EAX

指令执行后:EAX=0000 0007H。

• BSR OP1,OP2 ;逆向扫描指令

对 OP2 自高位向低位(向右)扫描,遇到第一个内容为 1 的位则停止扫描,并将该位的索引号送入 OP1 中。

8. 部分操作系统类指令

这些指令通常只用在操作系统代码中,不会出现在应用程序中,也就是说,一般情况下它们只能在 0 级特权级上运行。

(1) 加载和存储指令

前面已介绍过 4 条指令:LGDT、LIDT、LLDT 和 LTR,它们分别用来加载 GDT 表寄存器、IDT 表寄存器、LDT 表寄存器和任务寄存器 TR。与这些加载指令相对应的还有 4 条存储指令:SGDT、SIDT、SLDT 和 STR,分别用来将 GDT 表寄存器、IDT 表寄存器、LDT 表寄存器和任务寄存器 TR 存储到存储器中。4 条存储指令可以在 0~3 级上运行。

例 13-34

SGDT MEMORY ;将 GDTR 的内容存储到以 MEMORY 开始的 6 个字节单元中

SLDT [EAX] ;将 LDTR 内容存储到 EAX 指出的字单元中

(2) 设置和存储控制寄存器指令

MOV CRn, EAX ;将 EAX 的值赋予 CR0、CR2 或 CR3 中的一个

MOV EBX, CRn ;将 CR0、CR2 或 CR3 的值存到 EBX 中

(3) 设置和存储调试寄存器指令

MOV DRn, EAX ;将 EAX 的值赋予 DR0~DR3、DR6、DR7 中的一个

MOV EBX, DRn ;将调试寄存器的值存到 EBX 中

三、程序设计实例

掌握了 80386 的寻址方式和指令系统后,就可以利用 386 的 32 位指令来进行汇编语言程序设计了。不过,由于目前很多 PC 机上没有汇编语言程序的开发环境,而绝大多数人熟悉 C 语言的编程方法,因此可以采用在 C 语言中调用汇编语言程序段(函数)的方法来进行编程,或者说将汇编语言嵌入到 C 语言中去。利用嵌入式汇编程序,无须额外的汇编及连接步骤,就可以将汇编语言指令直接嵌入到 C 或 C++ 源程序中。下面以当前流行的 VC 6.0 为编程环境,举例说明在 C 程序中嵌入汇编语言程序的基本步骤。这些例子稍加修改便可以运行在其他的 C 语言编译环境中。

基本的编程和调用步骤如下:

(1) 与所有的 C 程序那样,在程序的开头必须用 #include 语句包含进需要用到的头文件。例如,用 #include<stdio.h> 语句把 C 语言标准 I/O 库函数包含进程序;其次是对必要的变量和数据进行定义。

(2) 用关键字 `_asm` 和 `{ }` 将要调用的汇编语言程序段(函数)嵌入到 C 语言程序中,也可以认为是用它们将汇编指令与 C 程序分隔开来,汇编语言程序段(函数)放在 `{ }` 中。

(3) 为便于 C 编译程序把嵌入的汇编语言程序当成函数调用,必须将这段汇编程序定义成函数,并在函数名之前说明函数返回参数的类型,同时在函数名后面的括号内说明输入参数及它们的类型,例如 `int`(整型)、`float`(浮点数)、`void`(无数据类型)等。

(4) 可以用 `_stdcall`、`_cdeclspec(naked)` 等定义函数的调用方式。默认的回值是放在 `EAX` 寄存器中的。

(5) 编写 C 语言主程序 `main()`。主程序常常用来显示提示信息、调用汇编语言程序和显示程序运行结果等。用 C 语言编程来完成显示提示信息和程序运行结果是很方便的,如果要用汇编语言来实现这种功能,则必须在 DOS 环境下,利用 DOS 功能调用,即 `INT 21H` 来实现。现在 DOS 已用得很少了,我们何不采用目前广泛使用的 VC 来完成这些功能呢。还要注意的,VC6.0 只能在 Windows 下运行,不允许使用 `INT 21H` 中断功能调用。

为使读者初步掌握 386 汇编语言程序设计以及它们与 C 语言混合编程的方法,下面用 3 个编程的实例加以说明。

例 13-35 冒泡法排序程序

所谓排序就是将内存中的一个无序数列按“从大到小”或从“小到大的”次序排列的一种操作。排序可以有多种方法,其中“冒泡法排序”是一种常用的方法。可以将内存中 n 个无序数据看作重量各不相同的气泡,数的大小代表气泡的重量,垂直排列在内存中。根据重气泡在下,轻气泡在上的原则从下到上进行扫描,如违反原则,就交换位置,经 $(n-1)$ 比较后,最轻的气泡冒到了最上面。再进行第二次扫描,经 $(n-2)$ 次扫描后,第二轻的气泡又冒到了上面。经 $(n-1)$ 轮扫描后,次序就按轻者在上的原则全部都排好了。

用此方法可对内存中的无序数据排序。设有一个 $n=6$ 的无序数列(可以是 32 位数): 42, 75, 21, 13, 36, 24 存于内存中,如图 13-26 (a)所示,进行第一次扫描时,将序号为 0 和 1 的数据 42、75 比较,违反了上小下大的原则,所以要交换位置,得到(b);再对 1 号和 2 号数据 21、36 比较,无须交换,得到(c);如此重复,直到经 $(n-1)=5$ 次比较后,最小的数 13 冒到了最上面。接着进行第二次扫描,这次只要进行 $(n-2)=4$ 次两两比较就能使第二小的数 21 冒到上面。经 $(n-1)$ 次扫描后,就将 6 个数据按“上小下大”的次序完全排好了。

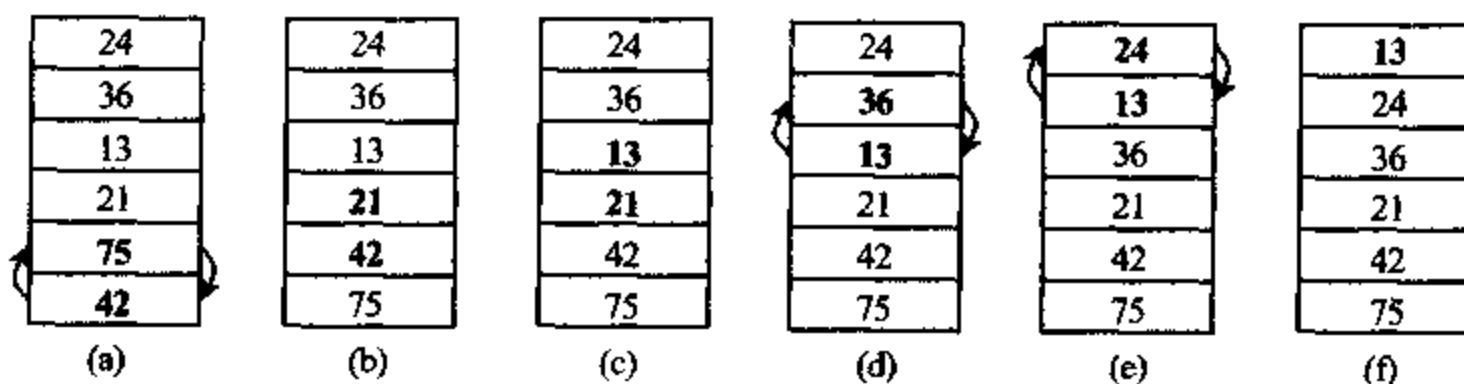


图 13-26 冒泡法排序示意图

下面给出实现冒泡法排序的程序。为便于阅读,C 语言程序一般都用小写字母书写,嵌入的汇编语言程序用大写或小写表示都可以,这里采用大写字母表示,与前面介绍的指令书写格式一致。

```
#include <stdio.h>
```

```
// 包含头文件
```



```

int nums[] = {24,36, 13, 21,75 ,42};    // 内存中待排序的 6 个无序数的数组 nums。
// -----用汇编语言编写的冒泡法排序函数 -----
void __stdcall bubble(int * nums, int count) //定义 bubble 函数
{
    __asm {
        MOV     ESI, [nums]           ;ESI 中存储了数组 nums 的起始地址
        MOV     ECX, [count]          ;需排序数的个数,作为参数传递给函数 bubble
        DEC     ECX                    ;对于 n 个数的排序,只需 n-1 次扫描
    outloop:
        CMP     ECX, 1                 ;检查排序是否结束
        JL      done                   ;ECX<1,转,表明排序已经完成。
        MOV     EDX, 0                  ;EDX←序号初值 0,每比较一次,EDX 加 1
        MOV     EAX, [ESI]              ;EAX←从数组中取一个数
    innerloop:
        CMP     EDX, ECX
        JGE     bottom                 ;完成一轮扫描,转
        INC     EDX                     ;未完成,指向下一对数据
        MOV     EBX, [ESI+EDX*4]        ;EBX 中放入上边的数
        MOV     EAX, [ESI+EDX*4-4]      ;EAX 中放入下边的数
        CMP     EBX, EAX                ;比较两个数的大小
        JBE     innerloop               ;上面数较小或两者相等则无须交换,转
    swap:
        MOV     [ESI+EDX*4-4], EBX      ;上面数较大则交换这两个数
        MOV     [ESI+EDX*4], EAX
        JMP     innerloop               ;进行下两个数据的比较,
    bottom:
        DEC     ECX                     ;扫描次数减 1
        JMP     outloop                 ;进行下一次扫描
    done:
        ;已经完成
    }
}
//-----汇编语言程序段结束,以下为主程序-----
int main()
{
    int i = 0;                          //打印排序前数组
    printf("bubble sort\n");
    printf("before:\n");
    for(i = 0; i < 6; i++) {
        printf("%d ", nums[i]);
    }
    bubble(&nums[0], 6);                 //调用 bubble 函数,实现排序算法
    printf("\nafter:\n");                 //打印排好序的数组
    for(i = 0; i < 6; i++) {

```



```

        printf("%d ", nums[i]);
    }
    return 0;
}

```

程序在 VC6.0 中编译通过。程序运行后,数据按从大到小的顺序存储在数组中,显示结果为:

```

bubble sort
before:
24 36 13 21 75 42
after:
13 21 24 36 42 75

```

例 13-36 字符串查找程序

在源字符串中,查找是否含有目标字符串,如有,则将目标字符串在源字符串中首次出现的位置序号送给 EAX 寄存器,否则,将-1 赋予 EAX。

程序中,设源字符串为:This is a string,目标串为:str,各寄存器存放如下参数:

DS:ESI 源串指针

ES:EDI 目标串指针

ECX 源串长度。即确定要搜索的范围,可以只搜索源字符串中前几个字符。这里 ECX=16,也就是搜索全部的源字符串。

EBX 目标串长度。将要搜索的字符串的长度送入 EBX 中,这里是 3,也就是在源字符串中,搜索由“stroke”字符串中前 3 个字符组成的字符串“str”。

完整的程序如下:

```

#include <stdio.h> //头文件
#include <string.h>
char astr[] = {16,"This is a string"}; //源字符串(含长度)
char bstr[] = {3,"str"}; //目标字符串(含长度)
int __stdcall strstrsearch(char * astr, char * bstr) //定义 strstrsearch 函数
{
    __asm { //在 C++ 语言中插入汇编语言程序
        CLD //递增方向
        MOV EBX, [astr] //EBX←源串始址
        MOVZX ECX, BYTE PTR [EBX] //ECX←源字符串的长度

        INC EBX
        MOV EDI, EBX //EDI←取一个源字符串
        MOV EBX, [bstr] //EBX←目标串长度
        INC EBX
        MOV ESI, EBX //ESI←目标串首地址
        MOVZX EAX, BYTE PTR [ESI] //读入目标串中的第一个字符's'到 al 中
        INC ESI
    }
    first_ch;
}

```

```

        REPNE    SCASB                ;在源串中搜索是否有's'
        JECXZ    not_found           ;没找到或比完了,转
        PUSH     ECX                  ;找到了,参数入栈
        PUSH     EDI
        MOV      EBX, [bstr]
        MOVZX    ECX, BYTE PTR [EBX]
        DEC      ECX                  ;第一个字符已经比较过了
        PUSH     ESI
        REPE     CMPSB                ;比较下一个字符
        POP      ESI
        POP      EDI
        POP      ECX
        JNE      first_ch             ;某次比较不相等,转
found:
        MOV      EBX, [astr]          ;找到了,计算目标串首次出现的位置并存入 EAX
        MOVZX    EAX, [EBX]
        SUB      EAX, ECX
        JMP      quit
not_found:
        MOV      EAX, -1              ;没有找到则 EAX 中返回-1
quit:
    }
}
int main()                            //主函数
{
    printf("string search\n");
    printf("source string: %s\n", &astr[1]);
    printf("search string: %s\n", &bstr[1]);
    printf("pos is %d\n", strstr(&astr[0], &bstr[0])); //调用 strstr 函数
    return 0;
}

```

在主函数 main 中调用 strstr 函数,以显示搜索的结果,strstr 函数中实现了字符串查找算法。程序在 VC6.0 中编译通过。程序运行后显示:

```

string search
source string: This is a string
search string: str
pos is 11

```

程序运行结果表明,在“This is a string”字符串中的第 11 个字符开始找到了字符串“str”。

例 13-37 阶乘运算程序

在数学上,阶乘是个很重要的函数,其运算公式为:

$$n! = n * (n-1) * (n-2) * \dots * 1$$

可以采用两种方法实现阶乘运算：一种是按上述公式简单的将各数进行连乘；另一种是使用递归调用的方法计算阶乘。

所谓递归调用，就是将 $n!$ 的值用 $(n-1)!$ 来表示，即：

$$n = n * (n-1)!$$

以 $5!$ 为例，函数调用过程为：

$$\begin{aligned}\text{fun}(5) &= 5 * \text{fun}(4) \\ &= 5 * 4 * \text{fun}(3) \\ &= 5 * 4 * 3 * \text{fun}(2) \\ &= 5 * 4 * 3 * 2 * \text{fun}(1) \\ &= 5 * 4 * 3 * 2 * 1 * \text{fun}(0) \\ &= 5 * 4 * 3 * 2 * 1 * 1 = 120\end{aligned}$$

32 位寄存器能表示的最大阶乘为 $13!$ 。下面给出用两种方法求 5 的阶乘的程序。

```
#include <stdio.h>
#include <string.h>
int __stdcall fact1(int num)           //定义函数 fact1,用连乘法求阶乘
{
    __asm {                           //在 C++ 语言中插入用连乘法编写的汇编语言程序
        MOV EAX, [num]                ;EAX (需要求阶乘的数
        MOV ECX, EAX
        DEC ECX
top:
        IMUL EAX, ECX                 ;采用递减连乘计算阶乘。
        LOOP top
    }
}
__declspec(naked) int fact2(int num)   //定义函数 fact2,用递归法求阶乘
{
    __asm {
        MOV ECX, [ESP+4]              //在 C++ 语言中插入用递归法编写的汇编语言程序
        MOV EAX, 1
        PUSH ECX                      ;传递参数入栈
        CALL __fact                   ;调用递归函数
        RET
    }
__fact:
    MOV ECX, [ESP+4]                  ;取入口参数
    DEC ECX
    CMP ECX, 1
    JBE end
    PUSH ECX                          ;若比 1 大,继续入栈,递归调用自己
    CALL __fact
end:
    MOV ECX, [ESP+4]
```

```

        IMUL  EAX, ECX           ;不断从堆栈中弹出数据相乘
        RET  4                  ;返回,并且从栈中弹出 4byte 的入口参数
    }
}
int main()                      //主函数
{
    int i = 5;
    printf("\n %d! =", i);
    printf("%d\n", fact1(i));    //调用 fact1,用连乘法求 5!
    printf("%d! =", i);
    printf("%d (recursive)\n", fact2(i)); //调用 fact2,用递归法求 5!
                                        //这里采用 __declspec( naked ) 等定义函数的(naked),
                                        //会把数 5 作为入口参数 push 到堆栈中

    return 0;
}

```

程序在 VC6.0 中编译通过。程序执行结果为：

5! =120

5! =120 (recursive)

程序运行过程说明如下：

(1) 在主函数 main 中先调用 fact1,采用连乘法求 5!,运算结果作为 fact1 的返回参数,可以显示出来。

(2) 再调用 fact2 函数,这里采用“__declspec (naked)”定义函数。首先执行一条“PUSH”指令,将 5 压入堆栈,并将“return 0”所在的地址压入堆栈,再从 __asm {……} 的第一条指令开始执行。

(3) 进入 __asm {……} 程序后,从堆栈[ESP+4]处取出一数“5”,将其作为递归调用的参数压入堆栈,执行第一条 CALL __fact 指令,EIP 入栈,开始执行递归调用过程,如图 13-27(a)所示。

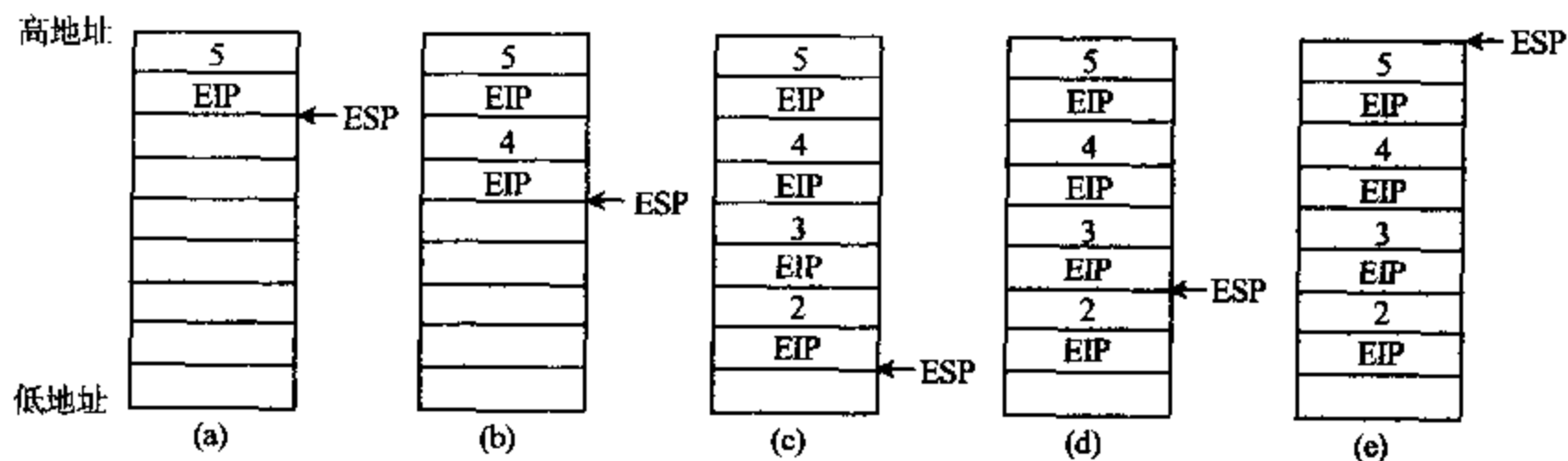


图 13-27 计算阶乘的示意图

(4) 递归调用 __fact,从中取出参数 5,将其减 1,若比 1 大,继续压入堆栈,直至将 2 压入堆栈为止,如图 13-27 (b)和(c)所示。

(5) 转“end;”开始的语句执行,不断从栈中弹出数据作乘法运算,如图 13-27(d),直至所有参数都相乘完毕,如图 13-27(e)。再返回主程序,结束操作。

13-5 32 位机接口技术

32 位微型计算机的基本硬件配置是由主机箱、键盘、显示器、鼠标等组成。主机箱内有一块主板,上面安装着 PC 机的主要部件及接口电路,通过接口电路或相应的接插口与外设相连。本节主要介绍主板的组成和相关的接口技术,先介绍一般 32 位 PC 机的主板由哪些主要部件组成,接着讨论具有北桥、南桥和 PCI 总线的 Pentium II 主板的结构及相应接口芯片的功能,最后介绍用于 P4 计算机的最新集成型主板的组成原理,重点讲述主板上的核心接口部件 Intel 845G 的功能。

一、主板的组成

一般 32 位微型计算机的主板上主要包含以下部件:

1. CPU 芯片

微型计算机性能的好坏主要取决于主板和安装在主板上的 CPU。386 或 486 CPU 芯片可以插在主板的插座(socket)上,Pentium 处理器则插在主板的插槽(Slot)上。一块主板只能插规定种类的 CPU,如 Pentium 主板只能插 Pentium 处理器。

2. 内存条插槽

在 PC/XT 中,存储器芯片一部分直接插在主板的双列直插式插座上,另一部分存储器芯片作为内存条,插在主板的插槽上。后来的 PC 机所采用的内存基本是单列存储模块(Single In-line Memory Modular, SIMM)内存条和双列存储模块(Double In-line Memory Modular, DIMM)内存条。前者为单面内存条,后者为双面内存条。内存条由芯片组控制,内存条的类型有:

FPM(Fast Page Mode Memory)

EDO DRAM(Extended Data Out Dynamic Random Access Memory)

BEDO DRAM(Burst EDO DRAM)

RDRAM (Rambus DRAM)

SDRAM (Synchronous DRAM)

DDR-SDRAM (Double Data Rate Synchronous DRAM)。

目前,PC 机上广泛使用的内存条为 SDRAM 和 DDR-SDRAM。其中,SDRAM (同步动态随机存取存储器)曾经非常流行,目前大多已被 DDR-SDRAM (双数据率同步动态 RAM)所取代,但仍然可以在有的系统上见到。对于基本的计算任务,用户不会注意到 SDRAM 和速度更快的 DDR-SDRAM 之间有多大的性能差别。当然,DDR-SDRAM 是今天存储器的主流产品,除非在需要获得最大性能的场所可以考虑使用 RDRAM,一般情况下,DDR-SDRAM 已经是最佳选择了。

RDRAM (Rambus 动态 RAM) 是一种可以获得更高性能但也要花更高代价的 DDR-SDRAM 替代产品。根据其数量和速度的不同,RDRAM 的价格可以是等量的 DDR-SDRAM 的 2 到 4 倍,只有在有限数量的计算机上安装了这种存储器。对于一般应用,分别

安装了这两类不同存储器的计算机,在性能上难于分辨出高低来。

3. 外部 Cache

Cache 一般分为两级,在 80486 和 Pentium 中,一级 Cache 容量较小,内置于 CPU 中。二级 Cache 安装在主板上,容量可达到 256KB 或 512KB,从而可以提高 CPU 的访问速度。Pentium Pro 已将二级 Cache 连同 CPU 封装在一起,容量也达到了 256KB 或 512KB。

4. 总线插槽

主板上有多条总线插槽或扩展槽,通常有 ISA、EISA、PCI 等总线的扩展槽,用得最多的是 PCI 总线扩展槽。

5. 配置芯片和器件

主板上包含各种接口芯片和器件,它们协调配合以使 CPU 和整个计算机系统能正常工作。主要芯片和部件有以下几种:

(1) 芯片组芯片

我们已经知道,在 PC/XT 机的主板上有一组接口芯片,如 8253 计数器/定时器、8259A 中断控制器、8237A DMA 控制器等,用来实现各种接口功能。32 位微型计算机同样需要这一类接口芯片及支持电路来控制 PC 机工作。但从 80286 起,就不再采用独立的接口芯片,在主板上也不能直接看到这些器件了,而是采用大规模集成电路技术,将众多的接口芯片及支持电路,按不同功能分别集成到一块或几块芯片中,构成芯片组芯片(ChipSet)。不同的主板上有不同的芯片组芯片,如 386 机用的 82357 芯片中,就集成了 7 个 32 位的 DMA 通道,5 个定时器/计数器通道,2 个 8 通道的中断控制器等等。尽管这些接口芯片都集成到芯片组芯片中去了,但许多芯片的端口地址仍与 PC/XT 机的兼容。例如端口 40H,仍是计数器/定时器通道 0 的端口地址,为系统提供频率 $f=18.2$ 次/秒的时间基准。

(2) 系统 BIOS 芯片

PC 系列的 BIOS(Basic Input Output System)芯片中的程序用来完成启动、上电自检、基本 I/O 设备(键盘、显示器、打印机等)的驱动,引导 DOS 启动等功能,其程序固化在 ROM 芯片中,称为 ROM BIOS。早期 PC 机上的 BIOS 芯片由 27C512 等 EPROM 芯片承担,80486 以上的 PC 机则采用闪烁存储器 Flash ROM 作为 BIOS 芯片,后者具有电可擦除功能,而且擦除速度快。

(3) CMOS RAM 芯片

CMOS RAM 芯片用作 PC 机的实时时钟电路和低功耗静态存储器,用于提供系统的日期、时间,并保存硬件配置参数。386 和 486 机中采用一块高集成度的多功能 CMOS RAM 芯片 82C206,内含 2 片 8237 DMA 控制器,2 片 8259 可编程中断控制器和 1 片 8254 定时器/计数器等。

(4) 其它器件

主板上的其它器件还包括:晶体振荡器、键盘插座、电源插座、扬声器插座等。随着各种新的芯片组芯片和新的总线技术等不断涌现,主板也在不断更新。

二、Pentium II 主板

随着 PCI 总线和处理器技术的发展,出现了基于 PCI 总线的并具有所谓“北桥”和“南

桥”芯片的主板,比较典型的是采用 Pentium II CPU 的 440BX AGP 主板,我们以这类主板为例简单介绍 Pentium II 主板的基本结构和功能。

1. Pentium II 主板结构

Pentium II 主板的结构如图 13-28 所示。从图 13-28 中可见,主板的部件是 CPU 和两块接口芯片。CPU 采用 Pentium II 处理器,主频为 233MHz~400MHz。另外两块接口芯片由芯片组芯片组成:一块是 82443BX 芯片,也称为北桥(North Bridge)或主桥(Host);另一块为 82371 AB PCI ISA IDE(P II X4E),它是一块多功能 I/O 芯片,也称为南桥(South Bridge)。系统中还采用了多种新的接口技术,主要有 USB 接口、IDE 接口和 AGP 接口等。

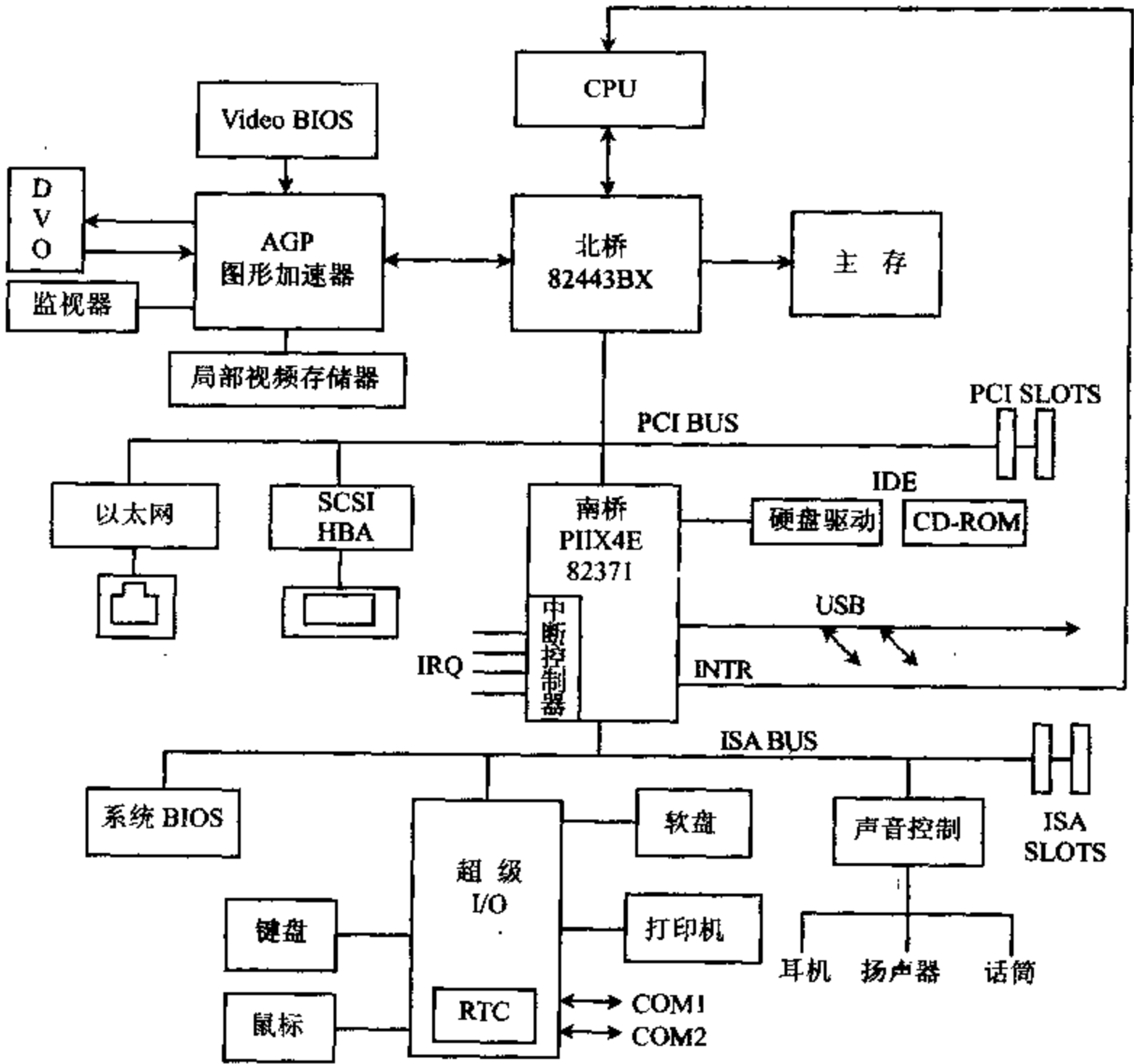


图 13-28 具有 PCI 总线的 PII 主板结构

北桥是在主处理器和 PCI 总线之间提供桥接的设备,它提供双向翻译的功能,为主处理器提供数据缓冲和二级 Cache,负责管理主存储器,还通过 AGP 接口管理图形显示设备。

南桥是连接 PCI 总线和 ISA 总线的桥接设备。它还通过 IDE 接口连接硬盘,通过 USB 接口连接串行设备。Super I/O 接口与键盘、鼠标、软驱和打印机等外设相连。此外,

PⅡ X4E还提供了全部的即插即用兼容功能,只要将扩展卡插入扩展槽中,微机系统就能自动进行扩展卡的配置工作,保证系统资源合理分配。

下面分别对南桥和北桥芯片组芯片的功能再做些具体的介绍。

2. 南桥芯片组芯片 82371(PⅡ X4 E)

PⅡ X4E 芯片集成了许多公用的输入输出功能,内部包含许多逻辑部件,外部具有 324 个引脚,主要提供以下逻辑功能:

(1) PCI 总线接口

PCI(Peripheral Component Interconnect)总线是外围部件互连总线的缩写,用于连接 PCI 卡。PCI 1.0 版本于 1992 年问世,1999 年发布 2.2 版本。与标准的 ISA 总线相比主要有如下特点:

- 传送速度快。ISA 总线传送速度为 8MB/s,而 32 位 PCI 总线在读写传送中支持 132MB/s 的峰值传送速率,对于 64 位 66MHz 的 PCI 总线,传送速率可达到 526MB/s。

- 支持突发传送工作方式。突发传送(Burst Transfer)是一种包含一个地址,后面跟着两个或两个以上数据的数据传送方式。如要传送的数据为: $d_1, d_2, d_3 \cdots d_n$,它们在内存中连续存放,访问该数据时,只有传递第一个数据 d_1 时需用 2 个时钟周期,传送后面的数据时,无须再给出地址,使处理器以接近自身总线的速度全速访问适配器。

- 传送的数据位数多。ISA 总线的数据位数为 8/16 位,PCI 总线为 32/64 位。

- 良好的兼容性。PCI 总线部件和接插件相对于处理器是独立的,PCI 总线支持各种不同结构的计算机,因此具有相对长的生命周期。

- 支持即插即用功能。PCI 设备中有存放设备具体信息的寄存器,这些信息使 BIOS 和操作系统软件能自动配置 PCI 总线部件和插件,使用很方便。

- 提供了 5 V 和 3.3 V 两种环境;相对低的成本等,说明 PCI 总线确实有较好的应用和发展前景。

(2) ISA 总线接口

能直接驱动 5 个 ISA 插槽。关于 ISA 总线,我们已在第六章中做了详细介绍。

(3) IDE 接口(Integrated Drive Electronics)

IDE 也称为集成驱动器电子电路接口,IDE 接口卡只能驱动两个硬盘驱动器,当接入一个 CD-ROM 驱动器后,只能再接一个硬盘。

(4) 具有兼容性的模块

芯片具有与 80X86~Pentium Pro 兼容的功能,主要包括:

- 芯片内含两个 8237C DMA 控制器,通过级联可提供 7 个 DMA 通道。0~3 通道以字节为单位计数传送,5~7 通道则以字为单位计数传送。它还能产生 ISA 的刷新时钟。

- 计数器/定时器模块。功能上与 82C54 等价,也包含 3 个计数通道,分别用来提供系统定时、刷新及扬声器发出特定音调的功能。其输入时钟频率仍为 14.31818MHz。

- 提供一个与 ISA 兼容的中断控制器。内部将 2 片 82C59 中断控制器级联,可以提供 15 个中断,还可提供串行中断,提供 I/O APIC 支持逻辑,对外部 I/O APIC 的使用给予全面的支持。

(5) USB 接口(Universal Serial Bus)

这是目前正在广泛使用的一种通用串行总线接口。PⅡ X4E 芯片内配置增强的通用串

行总线 USB 控制,给通用的主控制器接口 UHIC 提供强有力的支持。早期的 PC 机只提供很少的 RS-232 串行口。USB 接口实际上是一个万能插口,不但可以取代老的 PC 机上的所有串行、并行端口,用户几乎还可将所有外设的插头插入标准 USB 插口,例如显示器、键盘、鼠标、MODEM、打印机、扫描仪和视频摄像机等。此外,用户还能将一些 USB 外设串接,使 PC 的一大串设备共用 PC 上的端口。有些 USB 产品可不用加电源独立工作。

(6) 实时时钟 RTC(Real Time Clock)逻辑部件

内部的 RTC 逻辑部件带有 3V 锂电池电源和 256 字节 RAM,用来保存并跟踪每天的时间,断电时系统数据保存在 RAM 中,电池能工作 7 年。

(7) SCSI 接口

小型计算机系统接口(Small Computer System Interface,SCSI)定义了一种输入输出总线和逻辑接口,逻辑接口用来支持计算机和外设互连的总线。SCSI 接口是一种智能型接口,传输速度快,可驱动的外设数量多,可靠性高,互连性好,可以用来连接硬盘、CD-ROM 驱动器和扫描仪等外部设备。

(8) 系统电源管理

采用动态电源管理结构(DPMA),延长了电源的使用寿命;支持高级配置和电源接口(Advanced Configuration and Power Interface,ACPI),使台式机也能像笔记本 PC 机那样,通过操作系统软件直接进行电源管理,实现节能保护和控制。

(9) 通用 I/O 接口和 X-BUS 支持逻辑

为自定义系统设计提供了各种各样的通用串行总线。P II X4E 芯片配置不同,其输入输出的数量也是不一样的。P II X4E 芯片提供的两个可编程芯片选择逻辑,允许设计人员将许多设备都连在 X-BUS 上,根本不需要外部的译码逻辑。

(10) 其它

还包含系统复位逻辑及测试逻辑等。

3. 北桥芯片 82443BX

首次使用四端口加速技术,把 CPU、主存、AGP 端口和 PCI 总线这四者相互连接起来,并控制它们间的数据传送,从多方面提高了系统的性能。82443BX 芯片具有 492 个引脚,主要功能包括:

(1) 处理器总线接口

支持单/双 Pentium II 处理器,支持 64 位数据、36 位地址的处理,总线频率高达 100MHz。

(2) 集成了主存控制器

主存为 100/66MHz 的 SDRAM。3 个双面安装的 DIMM,最大容量为 512MB。使用 DIMMS,可扩展到 1GB。支持 EDO 和 SDRAM 两种主存,但两者不能混用。

(3) PCI 总线接口

支持除南桥芯片以外的 6 个 PCI 总线主设备。支持处理器总线、AGP、PCI 设备到主存的并发数据传输。

(4) 集成了 AGP 接口

AGP 为图形加速端口(Accelerated Graphics Port),主要用来支持三维图形显示设备。在进行三维图形显示时要用到许多技术,纹理贴图是其中一项很重要的技术,它可将图形图

像或背景图案像贴墙纸一样贴在三维图案的表面。所用的纹理数据越多、越复杂,则显示的物体越逼真。但需要将大量的纹理数据存放在显卡上的显存中,这必然导致成本的提高,解决矛盾的办法是将大容量的纹理数据存储在主存中。然而主存与图形卡之间是用 PCI 总线连接的,传输 3D 图像数据时仍嫌其传输速率太低,无法满足显示速度的要求。AGP 接口就是为解决图形纹理数据的高速传输的瓶颈而问世的,它采用新型接口技术标准,通过一种高速连接结构实现从主存中快速存取数据。

(5) 具有强大的省电功能和 ACPI 电源管理能力。

三、集成型主板

自从 Pentium 处理器和 PCI 总线诞生以来,推出了一系列芯片组芯片和不同类型的主板。芯片组基本上保持由北桥和南桥芯片组成的结构,北桥集成了存储器控制器和 PCI 控制器,南桥集成了 ISA 桥和 IDE 控制器。

1999 年开始,控制芯片组采用了整合技术,把对 I/O 卡和其它功能卡(例如显卡、声卡、MODEM、硬盘接口等)的支持功能集成到一起,形成了集成型芯片组,进一步加强了芯片组的功能。Intel 810 和 Intel 820 是最早的集成型芯片组。该芯片组采用了与以往芯片组完全不同的结构设计,取代北桥的是图形和存储器控制中心芯片 GMCH(Graphics and Memory Control Hub),取代南桥的则是输入/输出控制中心 ICH(I/O Control Hub),并将 BIOS 改为 FWH(FirmWare Hub)。两块芯片间不再用 PCI 总线互连,而是通过一条带宽为 PCI 总线两倍的专用 HUB 总线连接。它用分流的方式改善 PCI 的瓶颈问题,因此能充分发挥新的 Intel CPU 的处理性能,使系统性能更加优越。多种设备可以方便地连接到 I/O 控制器上,它们能直接与 CPU 交换数据,从而提高了系统的整体工作效率。两块芯片的尺寸也明显小于原先的北桥、南桥芯片。

Intel 845G 是专为 Intel P4 处理器设计的芯片组,有 478 个引脚,其功能更加强大,体积更小。图 13-29 是采用 Intel 845G 芯片组平台的方框图,图中虚线部分是 845G 芯片组的两个部件,即用作主桥的 Intel 82845G(GMCH)和集成了 I/O 子系统的 Intel 82801(ICH4)。CPU 和 SDRAM 内存分别连接在 GMCH 上,此外它还支持 VGA 显示器以及若干显示器加速图形端口(AGP)。GMCH 提供的专用 Hub 总线接口将它与 ICH4 相连。而 ICH4 是一种多功能 I/O 控制器,为现代计算机提供各种 I/O 接口。除 PCI 总线接口外,还有 IDE 硬盘接口(4 个)、USB 通用串行接口(6 个)、LAN 局域网接口以及连接各种外设(例如键盘、鼠标、软驱、打印机、扬声器、红外接口等)的超级 I/O 接口。此外,还有系统管理总线 SMBus、进行调试用的端口、可供计算机设计者自己裁剪的 GPIO 端口以及能连接语音编解码器的 AC'97 2.3 接口。FWH Flash BIOS 相当于原来主板上的 BIOS 芯片,也通过 ICH4 接入系统。

下面简单介绍 GMCH 和 ICH4 芯片的功能。

1. Intel 82845G Graphics and Memory Controller Hub (GMCH)

图 13-29 中的图形和存储器控制 Hub 芯片(Intel 82845G Graphics and Memory Controller Hub,GMCH)提供处理器接口、SDRAM 系统内存接口、将 GMCH 与 ICH4 连接起来的专用 Hub 接口,并具有几个 AGP 端口,还有能对图形设备提供支持的集成图形设备

(IGD)控制器,IGD有 3D、2D 和视频能力,也具有支持 DVO(数字视频输出端口)设备的多个 DVO 端口。GMCH 的主要功能如下:

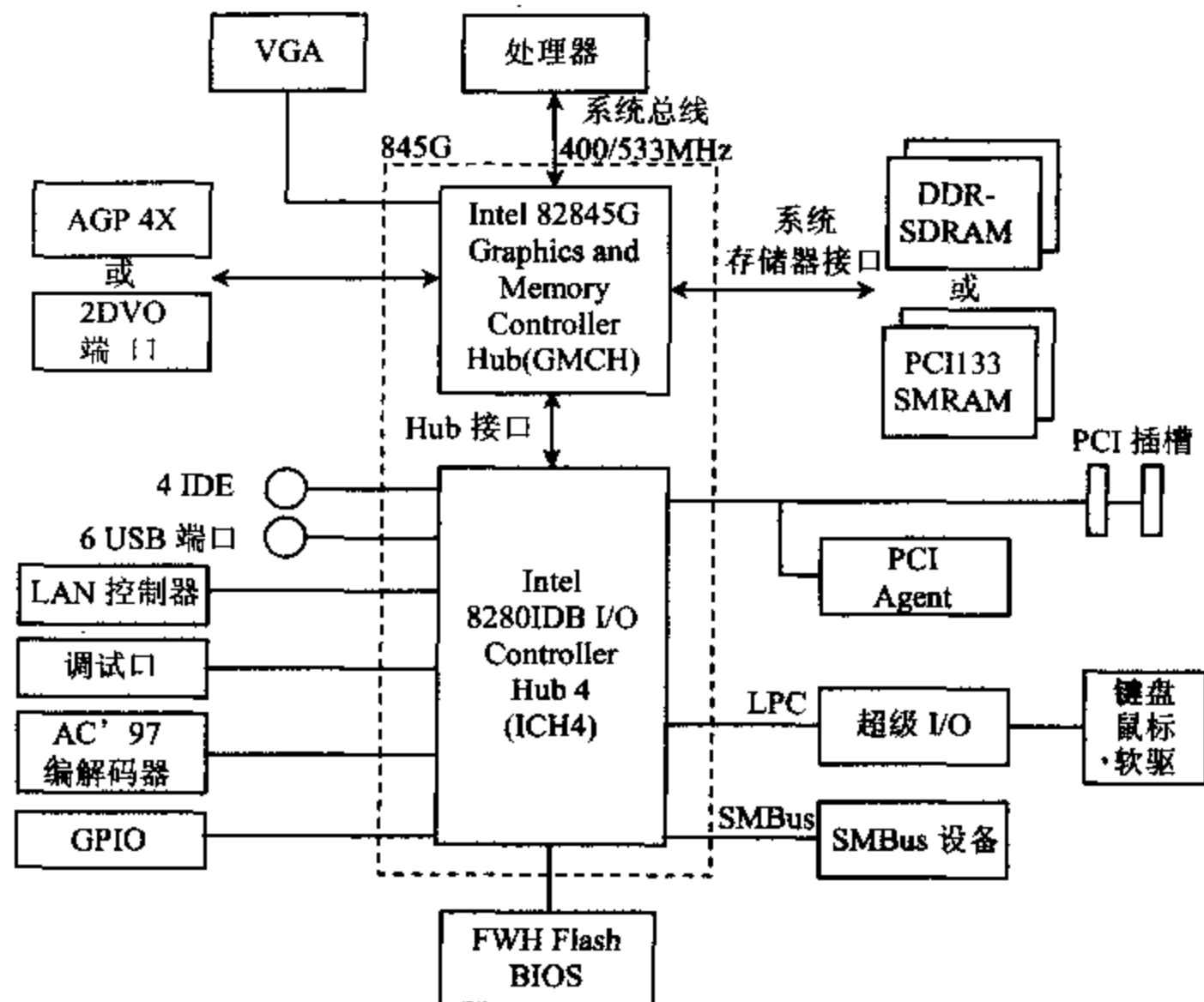


图 13-29 Intel 845G 芯片组系统方框图

• 主接口(Host Interface)

845G 芯片组主要是为采用 Pentium 4 处理器的台式 PC 机设计的,其 GMCH 支持的 P4 处理器,采用 mPGA 478 引脚封装,0.13 μ m 的半导体工艺,前端总线频率为 400/533MHz,主电源 1.15V~1.75V,可以带 256KB L2 Cache 或 512KB L2 Cache,支持 32 位地址总线,可寻址 4GB 存储空间,并支持可扩展的总线协议。

• 系统存储器接口

845G 支持单通道 SDR-SDRAM(单数据率同步 DRAM)或 DDR-SDRAM(双数据率同步 DRAM)这两种存储器配置,用 64 位宽的数据总线,系统存储器总容量可达 2GB。它对每种配置都支持两个 DIMM 插槽,内存控制器接口可以通过位于芯片内部的一系列控制寄存器来进行配置,内存接口支持大小为 64MB、128MB、256MB、512MB 的 SDRAM 技术。

• Hub 接口

Hub 接口将 GMCH 连接到 ICH4 上,GMCH 与 ICH4 之间的大多数通信都通过此接口完成。Hub 接口工作在 66MHz 的频率下,传输速率为 266MB/s,工作电压为 1.5V。

• 显示器接口

GMCH 集成了 256 位的 2D/3D 显卡,图形核心中整合了 350MHz 的高速数模转换器 RAMDAC,能直接驱动逐行扫描模拟显示器。

• 多路复用的 AGP 接口

GMCH 具有与 DVO 端口多路复用的 AGP 接口,支持单个 AGP 或 PCI-66 组件或连接器,对单个 PCI-66 设备的支持符合 2.0 版本的加速图形端口接口标准。由于 GMCH 内部已经有集成的图形设备(IGD),所以系统并非一定需要外部的图形加速设备。当系统检测到已安装了一个外部 AGP 设备(如显卡)时,BIOS 将禁用 GMCH 的 IGD。

- 两个多路复用的 DVO 端口

如上所述,这两个 Intel DVO 端口是与 AGP 接口多路复用的,每个端口能驱动的元素频率高达 165MHz。当系统中存在 AGP 连接器时,GMCH 就能通过 AGP 数字显示卡(ADD)来使用这些数字显示通道。

- 图形功能

GMCH 集成的图形加速设备能够处理 3D、2D 和视频图像。它有一套扩展的指令集,主要用于 3D 操作、BLT(块传送)和 STRBLT(扩展 BLT)操作、运动补偿、覆盖和显示控制。它的视频引擎支持视频会议和其它视频应用,通过 AGP 接口支持外部图形加速卡,但是不能和外部 AGP 图形设备共用。

GMCH 通过内存端口提供高带宽数据的存取,能够以 1.0GB/s(SDR PC133),1.6GB(DDR 200)或 2.2GB/s(DDR266)的速率存取内存中的图像数据。它使用 Intel 的直接内存执行模式获取位于内存中的纹理数据,还利用一个 Cache 控制器来避免对刚使用过的纹理数据进行频繁的内存读取操作。此外,还能为数据块传输提供 2D 硬件加速等。

令人印象深刻的数字视频技术支持。GMCH 最高可以达到 330MP/s 的 DVO 像素生成率。它支持 DVO 回放中的硬件动态补偿技术,5×2 重叠过滤功能使得 DVO 在缩放的时候生成更加平滑的画面,让用户得到更好的视频享受。由于它还提供两个与 AGP 多路复用、能驱动 AGP 数字显卡的 DVO 端口,因此能全面支持 TMDS、LVDS、TV-out、S-Video 等外接视频输出接口,可直接连接到平板显示器、电视机或数字显示器等设备上。GMCH 符合 1.0 版本的数字视频接口(DVI)标准,当接入 DVI 兼容的外设和连接器时,就能提供一个高速的数字显示接口。

2. Intel 82801DB I/O Controller Hub 4 (ICH4)

ICH4 是一种高集成度的多功能 I/O 控制器 Hub,提供 PCI 总线接口,集成许多当今 PC 平台所需要的功能。82801DB ICH4 的功能主要有:

- Hub 结构。这种 Hub 接口结构,将 I/O 桥设置在 Hub 接口上而不是设置在 PCI 中,能够确保集成到 ICH4 中的 I/O 功能和 PCI 设备都能获得足够的带宽以达到最佳性能。

- PCI 接口。ICH4 集成了一个 PCI 仲裁器,能够支持 6 个外部 PCI 总线主控制器和 4 个 ICH4 内部的 PCI 请求。利用附加芯片还可以设计一个 ISA 插槽,提供对 ISA 设备的支持。

- IDE 接口。这是能支持 4 个 IDE 设备(如硬盘)的快速 IDE 接口,每个 IDE 设备可以拥有独立的时钟。该系统包含 2 个可在电气上隔离的独立 IDE 信号通道,并且可以被设置成标准的主从通道,因此能支持 4 个设备。

- 低引脚数(Low Pin Count,LPC)接口。这是一种按照 LPC 接口规范设计的接口,它使用 PCI 总线的 33MHz 时钟,因此速度要比传统的 8MHz ISA 总线高得多,同时它是基于 1 字节(8 位)数据传输设计的,所用的引脚数小,又带来少占空间、省电和减少热量等优点。如前所述,键盘、鼠标、打印机等 I/O 设备均通过 LPC 接口连接到 ICH4 上。

• USB 总线控制器。ICH4 有一个能支持高速 USB 2.0 传输的增强型主控接口 (EHCI) 控制器, 数据传输速率可以高达 480Mb/s, 是全速 USB 的 40 倍。另有 3 个能支持全速和低速 USB 的通用型主控接口 (UHCI) 控制器。ICH4 共能支持 6 个 USB 2.0 接口, 它们均能工作在高速、全速和低速状态。

• LAN 控制器。ICH4 集成进了局域网 (LAN) 控制器, 包含 32 位的 PCI 控制器, 提供增强的分散-聚集总线 (Scatter-gather Bus), 使 LAN 控制器通过 PCI 总线完成高速数据传递。LAN 控制器可工作于全双工或半双工模式。

• AC'97 2.3 规范编解码控制器。它提供一个数字接口, 能与音频编解码器 (AC)、MODEM 编解码器 (MC)、音频/MODEM 编解码器 (AMC) 或 AC 和 MC 的组合相连接。该规范还定义了称为 AC-link 的系统逻辑与音频或 Modem 编解码器间的接口。系统设计者可以根据这个控制器提供的功能, 构造软 MODEM, 连接麦克风, 设计出价格低廉、性能优良的 6 声道立体声音响系统。

• 兼容模块。为了与以前的 CPU 兼容, ICH4 中设置了一个包含 DMA 控制器、定时器/计数器和中断控制器的兼容模块。

DMA 控制器把 2 个 82C37 DMA 控制器的逻辑结合在一起, 共有 7 个独立的可编程通道。通道 0~3 控制 8 位按字节计数的传输, 通道 5~7 控制 16 位按字计数的传输, 通道 4 是为一般的总线主控设备请求保留的。

支持两种 DMA (LPC 和 PC/PCI) 82C54 计数器/定时器, ICH4 提供一个与 ISA 兼容的可编程中断控制器 PIC。将两个 8259A 中断控制器级联起来, 提供 15 个中断, 还支持一个串行中断。

定时器/计数器包含 3 个计数通道, 每一个计数通道均与 82C54 可编程定时器/计数器的功能一样, 提供系统的定时器功能以及生成扬声器的音调。它们的时钟源频率也是 14.31818MHz。

• 先进的可编程中断控制器。除了标准的 ISA 兼容的 PIC 外, ICH4 还包含进了先进的可编程中断控制器 (APIC)。

• 系统管理总线 SMBus 2.0。包含 SMBUS 主接口, 允许处理器与 SMBUS 从设备通信。该接口与大部分的 I²C 设备兼容, 支持 8 个命令协议。

• 支持 Firmware Hub (FWH BIOS) 接口。

• 包含一个 Motorola MC146818A 兼容的实时时钟 (RTC), 带有 256 字节的 RAM, 由电池供电, 用于存储时间和系统数据。掉电时, 数据也会被保存。3V 锂电池的寿命为 7 年。此外, 它还能够预置 30 天之内的唤醒事件, 到了日期可以进行警示。

• 提供各种供用户裁剪的通用 I/O, I/O 数目随 ICH4 配置而变。

• 增强的电源管理功能。

附录 A 8086/8088 指令系统一览表

助记符	汇编语言格式	功 能	操作数	时 钟 周期数	字节数	标志位 ODITSZAPC
MOV	MOV dst,src	$\text{dst} \leftarrow \text{src}$	mem,ac	10	3	-----
			ac,mem	10	3	
			reg,reg	2	2	
			reg,mem	8+EA	2-4	
			mem,reg	9+EA	2-4	
			reg,data	4	2-3	
			mem,data	10+EA	3-6	
			segreg,reg	2	2	
			segreg,mem	8+EA	2-4	
			reg,segreg	2	2	
			mem,segreg	9+EA	2-4	
PUSH	PUSH src	$\text{SP} \leftarrow \text{SP} - 2$	reg	11	1	-----
		$\text{SP} + 1, \text{SP} \leftarrow \text{src}$	segreg	10	1	
			mem	16+EA	2-4	
POP	POP dst	$\text{dst} \leftarrow \text{SP} + 1, \text{SP}$	reg	8	1	-----
		$\text{SP} \leftarrow \text{SP} + 2$	segreg	8	1	
			mem	17+EA	2-4	
XCHG	XCHG opr1,opr2	$\text{opr1} \leftrightarrow \text{opr2}$	reg,ac	3	1	-----
			reg,mem	17+EA	2-4	
			reg,reg	4	2	
IN	IN ac,port	$\text{ac} \leftarrow \text{port}$		10	2	-----
	IN ac,DX	$\text{ac} \leftarrow (\text{DX})$		8	1	
OUT	OUT port,ac	$\text{port} \leftarrow \text{ac}$		10	2	-----
	OUT port,DX	$(\text{DX}) \leftarrow \text{ac}$		8	1	
XLAT	XLAT			11	1	-----
LEA	LEA reg,src	$\text{reg} \leftarrow \text{src}$	reg,mem	2+EA	2-4	-----
LDS	LDS reg,src	$\text{reg} \leftarrow \text{src}$	reg,mem	16+EA	2-4	-----
		$\text{DS} \leftarrow \text{src} + 2$				
LES	LES reg,src	$\text{reg} \leftarrow \text{src}$	reg,mem	16+EA	2-4	-----
		$\text{ES} \leftarrow \text{src} + 2$				
LAHF	LAHF	$\text{AH} \leftarrow \text{PSW 低字节}$		4	1	-----
SAHF	SAHF	$\text{PSW 低字节} \leftarrow \text{AH}$		4	1	rrrrr
PUSHF	PUSHF	$\text{SP} \leftarrow \text{SP} - 2$		10	1	-----
		$(\text{SP} + 1, \text{SP}) \leftarrow \text{PSW}$				
POPF	POPF	$\text{PSW} \leftarrow (\text{SP} + 1, \text{SP})$		8	1	rrrrrrrrr
		$\text{SP} \leftarrow \text{SP} + 2$				
ADD	ADD dst,src	$\text{dst} \leftarrow \text{src} + \text{dst}$	reg,reg	3	2	x---xxxxx
			reg,mem	9+EA	2-4	
			mem,reg	16+EA	2-4	
			reg,data	4	3-4	
			mem,data	17+EA	3-6	
			ac,data	4	2-3	

续表

助记符	汇编语言格式	功 能	操作数	时 钟 周期数	字节数	标志位 (O)D(T)S(Z)A(P)C
ADC	ADC, dst, src	$dst \leftarrow src + dst + CF$	reg, reg	3	2	x---xxxxx
			reg, mem	9+EA	2-4	
			mem, reg	16+EA	2-4	
			reg, data	4	3-4	
			mem, data	17+EA	3-6	
			ac, data	4	2-3	
INC	INC opr	$opr \leftarrow opr + 1$	reg	2-3	1-2	x---xxxx-
			mem	15+EA	2-4	
SUB	SUB dst, src	$dst \leftarrow dst - src$	reg, reg	3	2	x---xxxxx
			reg, mem	9+EA	2-4	
			mem, reg	16+EA	2-4	
			ac, data	4	2-3	
			reg, data	4	3-4	
			mem, data	17+EA	3-6	
SBB	SBB dst, src	$dst \leftarrow dst - src - CF$	reg, reg	3	3	x---xxxxx
			reg, mem	9+EA	2-4	
			mem, reg	16+EA	2-4	
			ac, data	4	2-3	
			reg, data	4	3-4	
			mem, data	17+EA	3-6	
DEC	DEC opr	$opr \leftarrow opr - 1$	reg	2-3	1-2	x---xxxx-
			mem	15+EA	2-4	
NEG	NEG opr	$opr \leftarrow -opr$	reg	3	2	x---xxxxx
			mem	16+EA	2-4	
CMP	CMP opr1, opr2	$opr1 - opr2$	reg, reg	3	2	x---xxxxx
			reg, mem	9+EA	2-4	
			mem, reg	9+EA	2-4	
			reg, data	4	3-4	
			mem, data	10+EA	3-6	
			ac, data	4	2-3	
MUL	MUL src	$AX \leftarrow AL * src$	8 位 reg	70-77	2	x---uuuuu
		$(DX, AX) \leftarrow AX * src$	8 位 mem	$(76-83) + EA$	2-4	
			16 位 reg	118-133	2	
			16 位 mem	$(124-139) + EA$	2-4	
IMUL	IMUL src	$AX \leftarrow AL * src$	8 位 reg	80-98	2	x---uuuuu
		$(DX, AX) \leftarrow AX * src$	8 位 mem	$(86-104) + EA$	2-4	
			16 位 reg	128-154	2	
			16 位 mem	$(134-160) + EA$	2-4	
DIV	DIV src	$AL \leftarrow AX / src$ 的商	8 位 reg	80-90	2	u---uuuuu
		$AH \leftarrow AX / src$ 的余数	8 位 mem	$(86-96) + EA$	2-4	
		$AX \leftarrow (DX, AX) / src$ 的商	16 位 reg	144-162	2	
		$DX \leftarrow (DX, AX) / src$ 的余数	16 位 mem	$(150-168) + EA$	2-4	
IDIV	DIV src	$AL \leftarrow AX / src$ 的商	8 位 reg	101-112	2	u---uuuuu
		$AH \leftarrow AX / src$ 的余数	8 位 mem	$(107-118) + EA$	2-4	
		$AX \leftarrow (DX, AX) / src$ 的商	16 位 reg	165-184	2	
		$DX \leftarrow (DX, AX) / src$ 的余数	16 位 mem	$(171-190) + EA$	2-4	

续表

助记符	汇编语言格式	功 能	操作数	时 钟 周期数	字节数	标志位 ODITSZAPC
DAA	DAA	AL←把 AL 中的和调整到 压缩的 BCD 格式		4	1	0---xxxxx
DAS	DAS	AL←把 AL 中的差调整到 压缩的 BCD 格式		4	1	0---xxxxx
AAA	AAA	AL←把 AL 中的和调整到 非压缩的 BCD 格式		4	1	0---uuxux
AAS	AAS	AH←AH+调整产生的进位值 AL←把 AL 中的差调整到 非压缩的 BCD 格式		4	1	0---uuxux
AAM	AAM	AH←AH-调整产生的借位值 AX←把 AH 中的积调整到 非压缩的 BCD 格式		83	2	0---xxuxu
AAD	AAD	AL←10 * AH + AL AH←0		60	2	0---xxuxu
AND	AND dst,src	实现除法的非压缩的 BCD 调整 dst←dst ∧ src	reg, reg reg, mem mem, reg reg, data mem, data ac, data	3 9+EA 16+EA 4 17+EA 4	2 2-4 2-4 3-4 3-6 2-3	0---xxux0
OR	OR dst,src	dst←dst ∨ src	reg, reg reg, mem mem, reg ac, data reg, data mem, data	3 9+EA 16+EA 4 4 17+EA	2 2-4 2-4 2-3 3-4 3-6	0---xxux0
NOT	NOT opr	opr← $\overline{\text{opr}}$	reg mem	3 16+EA	2 2-4	-----
XOR	XOR dst,src	dst←dst ⊕ src	reg, reg reg, mem mem, reg ac, data reg, data mem, data	3 9+EA 16+EA 4 4 17+EA	2 2-4 2-4 2-3 3-4 3-6	0---xxux0
TEST	TEST opr1,opr2	opr1 ∧ opr2	reg, reg reg, mem ac, data reg, data mem, data	3 9+EA 4 5 11+EA	2 2-4 2-3 3-4 3-6	0---xxux0
SHL	SHL opr,1	逻辑左移	reg mem	2 15+EA	2 2-4	x---xxuxx
	SHL opr,CL		reg mem	8+4/位 20+EA+4/位	2 2-4	

续表

助记符	汇编语言格式	功 能	操作数	时 钟 周期数	字节数	标志位 ODITSZAPC
SAL	SAL opr, l	算术左移	reg	2	2	x---xxuxxx
			mem	15+EA	2-4	
	SAL opr, CL		reg	8+4/位	2	
			mem	20+EA+4/位	2-4	
SHR	SHR opr, l	逻辑右移	reg	2	2	x---xxuxxx
			mem	15+EA	2-4	
	SHL opr, CL		reg	8+4/位	2	
			mem	20+EA+4/位	2-4	
SAR	SAR opr, l	算术右移	reg	2	2	x---xxuxxx
			mem	15+EA	2-4	
	SAR opr, CL		reg	8+4/位	2	
			mem	20+EA+4/位	2-4	
ROL	ROL opr, l	循环左移	reg	2	2	x-----x
			mem	15+EA	2-4	
	ROL opr, CL		reg	8+4/位	2	
			mem	20+EA+4/位	2-4	
ROR	ROR opr, l	循环右移	reg	2	2	x-----x
			mem	15+EA	2-4	
	ROR opr, CL		reg	8+4/位	2	
			mem	20+EA+4/位	2-4	
RCL	RCL opr l	带进位循环左移	reg	2	2	x-----x
			mem	15+EA	2-4	
	RCL opr, CL		reg	8+4/位	2	
			mem	20+EA+4/位	2-4	
RCR	RCR opr, l	带进位循环右移	reg	2	2	x-----x
			mem	15+EA	2-4	
	RCR opr, CL		reg	8+4/位	2	
			mem	20+EA+4/位	2-4	
MOVS	MOVSB	(DI) $\leftarrow$ (SI)		不重复:18	1	-----
	MOVSW	SI $\leftarrow$ SI $\pm$ 1 或 2 DI $\leftarrow$ DI $\pm$ 1 或 2		重复: 9+17/rep		
STOS	STOSB	(DI) $\leftarrow$ AC		不重复:11	1	-----
	STOSW	DI $\leftarrow$ DI $\pm$ 1 或 2		重复: 9+10/rep		
LODS	LODSB	AC $\leftarrow$ (SI)		不重复:12		
	LODSW	SI $\leftarrow$ SI $\pm$ 1 或 2		重复: 9+13/rep		
REP	REP 串指令	当 CX=0,退出重复,否则 CX $\leftarrow$ CX-1,执行其后的串指令		2	1	-----
CMPS	CMPSB	(SI)-(DI)		不重复:22	1	x---xxxxxx
	CMPSW	SI $\leftarrow$ SI $\pm$ 1 或 2 DI $\leftarrow$ SI $\pm$ 1 或 2		重复: 9+22/rep		
SCAS	SCASB	AC $\leftarrow$ (DI)		不重复:15	1	x-----x
	SCASW	DI $\leftarrow$ DI $\pm$ 1 或 2		重复: 9+15/rep		

续表

助记符	汇编语言格式	功 能	操 作 数	时 钟 周期数	字节数	标志位 ODITSZAPC
REPE	REPE/REPZ	当 CX=0 或 ZF=0 退出重复;否则,		2	1	-----
或 REPZ	串指令	CX←CX-1,执行其后的串指令				
REPNE	REPNE/REPNZ	当 CX=0 或 ZF=1 退出重复;否则,		2	1	-----
或 REPNZ	串指令	CX←CX-1,执行其后的串指令				
JMP	JMP short opr	无条件转移		15	2	-----
	JMP near ptr opr			15	3	
	JMP far ptr opr			15	5	
	JMP word ptr opr		reg	11	2	
			mem	18+EA	2-4	
	JMP dword ptr opr			24+EA	2-4	
JZ	JZ/JE opr	ZF=1 则转移		16/4	2	-----
或 JE						
JNZ	JNZ/JNE opr	ZF=0 则转移		16/4	2	-----
或 JNE						
JS	JS opr	SF=1 则转移		16/4	2	-----
JNS	JNS opr	SF=0 则转移		16/4	2	-----
JO	JO opr	OF=1 则转移		16/4	2	-----
JNO	JNO opr	OF=0 则转移		16/4	2	-----
JP	JP/JPE opr	PF=1 则转移		16/4	2	-----
或 JPE						
JNP	JNP/JPO opr	PF=0 则转移		16/4	2	-----
或 JPO						
JC	JC/JB/JNAE opr	CF=0 则转移		16/4	2	-----
或 JB						
或 JNAE						
JNC	JNC/JNE/JAE opr	CF=0 则转移		16/4	2	-----
或 JNB						
或 JAE						
JBE	JBE/JNA opr	CF V ZF=1 则转移		16/4	2	-----
或 JNA						
JNBE	JNBE/JA opr	CF V ZF=0 则转移		16/4	2	-----
或 JA						
JL	JL/JNGE opr	SF V OF=1 则转移		16/4	2	-----
或 JNGE						
JNL	JNL/JGE opr	SF V OF=0 则转移		16/4	2	-----
或 JGE						
JLE	JLE/JNG opr	(SF V OF) V ZF=1 则转移		16/4	2	-----
或 JNG						
JNLE	JNLE/JG opr	(SF V OF) V ZF=0 则转移		16/4	2	-----
或 JG						
JCXZ	JCXZ opr	CX=0 则转移		18/6	2	-----
LOOP	LOOP opr	CX≠0 则循环		17/5	2	-----
LOOPZ	LOOPZ/LOOPE opr	ZF=1 且 CX≠0 则循环		18/6	2	-----
或 LOOPE						

续表

助记符	汇编语言格式	功 能	操作数	时 钟 周期数	字节数	标志位 ODITSZAPC
LOOPNZ 或 LOOPNE	LOOPNZ/LOOPNE opr	ZF=0 且 CX≠0 则循环		19/5	2	-----
CALL	CALL dst	段内直接: SP←SP-2 (SP+1, SP)←IP IP←IP+D16		19	3	-----
		段内间接: SP←SP-2 (SP+1, SP)←IP IP←EA	reg mem	16 21+EA	2 2-4	
		段间直接: SP←SP-2 (SP+1, SP)←CS SP←SP-2 (SP+1, SP)←IP IP←转向偏移地址 CS←转向段地址		28	5	
		段间间接: SP←SP-2 (SP+1, SP)←CS SP←SP-2 (SP+1, SP)←IP IP←EA CS←EA+2		37+EA	2-4	
RET	RET	段内: IP←(SP+1, SP) SP←SP+2		16	1	-----
		段间: IP←(SP+1, SP) SP←SP+2 CS←(SP+1, SP) SP←SP+2		24	1	
RET	RET exp	段内: IP←(SP+1, SP) SP←SP+2 SP←SP+D16		20	3	
		段间: IP←(SP+1, SP) SP←SP+2 CS←(SP+1, SP) SP←SP+2 SP←SP+D16		23	3	
INT	INT type	SP←SP-2	type=3	52	1	--00-----
	INT (当 type=3 时)	(SP+1, SP)←PSW SP←SP-2 (SP+1, SP)←CS SP←SP-2 (SP+1, SP)←IP IP←type*4 CS←type*4+2	type≠3	51	2	
INTO	INTO(type=4)	若 OF=1, 则		53(OF=1)	1	--00-----

续表

助记符	汇编语言格式	功 能	操作数	时 钟 周期数	字节数	标志位 ODITSZAPC
		SP←SP-2 (SP+1,SP)←PSW SP←SP-2 (SP+1,SP)←CS SP←SP-2 (SP+1,SP)←IP IP←10H CS←12H		4(OF=0)		
IRET	IRET	IP←(SP+1,SP) SP←SP+2 CS←(SP+1,SP) SP←SP+2 PSW←(SP+1,SP) SP←SP+2		24	2	rrrrrrrrr
CBW	CBW	AL 的符号扩展到 AH		2	1	-----
CWD	CWD	AX 的符号扩展到 DX		5	1	-----
CLC	CLC	进位位置 0		2	1	-----0
CMC	CMC	进位位求反		2	1	-----x
STC	STC	进位位置 1		2	1	-----1
CLD	CLD	方向标志置 0		2	1	-0-----
STD	STD	方向标志置 1		2	1	-1-----
CLI	CLI	中断标志置 0		2	1	--0-----
STI	STI	中断标志置 1		2	1	--1-----
NOP	NOP	无操作		3	1	-----
HLT	HLT	停机		2	1	-----
WAIT	WAIT	等待		3 或更多	1	-----
ESC	ESC	换码		8+EA	2-4	-----
LOCK	LOCK	封锁		2	1	-----
	segreg;	段前缀		2	1	-----

符号说明如下:

0—置 0;1—置 1;x—根据结果设置;—不影响;u—无定义;r—恢复原先保存的值。

opr—操作数;src—源操作数;dst—目的操作数;reg—寄存器;segreg—段寄存器;mem—存储器;data—立即数。

附录 B 通用汇编程序伪指令

伪指令	功 能
. 286	选择 80286 指令系统
. 286P	选择 80286 保护模式指令系统
. 386	选择 80386 指令系统
. 386P	选择 80386 保护模式指令系统
. 486	选择 80486 指令系统
. 486P	选择 80486 保护模式指令系统
. 586	选择 Pentium 指令系统
. 586P	选择 Pentium 保护模式指令系统
. 287	选择 80287 数字协处理器
. 387	选择 80387 数字协处理器
. EXIT	用来使程序设计模型退回到 DOS
. MODEL	选择编程模型
. STARTUP	在编程模型中指示程序的开始
ALIGN2	按字或双字分界的段中数据的开始
ASSUME	规定段所属的段寄存器
BYTE	指示字节长度的操作数, 如 BYTE PTR
WORD	起字操作数的作用, 如 WORD PTR
DWORD	定义双字长度的操作数, 如 DWORD PTR
DB	定义字节(8 位)
DW	定义字(16 位)
DD	定义双字(32 位)
DQ	定义四字(64 位)
DT	定义 10 个字节(80 位)
DUP	产生重复的字符或数字
END	指示程序结束
MACRO	定义宏的名字、参数和开始 宏名 MACRO
ENDM	指示宏序列结束 ENDM
PROC	指示过程开始 过程名 PROC
ENDP	指示过程结束 过程名 ENDP
STRUC	定义结构的开始 结构名 STRUC
ENDS	指示结构结束 结构名 ENDS

续表

伪指令	功 能
EQU	标号等于数据
=	赋值
FAR	定义远指针
NEAR	定义近指针
ORG	设置段内的起始地址
PTR	指示存储器指针
SEGMENT	定义段 段名 SEGMENT
STACK	指示这个段是堆栈段
USES	MASM 6. X 版本指示自动保存过程使用的寄存器
USE16	指导汇编程序 80386~Pentium 以上微处理器使用 16 位的指令模式和数据长度
USE32	指导汇编程序 80386~Pentium 以上微处理器使用 32 位的指令模式和数据长度
PUBLIC	说明在本模块中定义的外部符号
EXTRN	可使指定的段都在 64K 的物理段内

附录 C ASCII 码编码表

	列	0	1	2	3	4	5	6	7
行	低 高	000	001	010	011	100	101	110	111
0	0000	NUL	DLE	SP	0	@	P	,	p
1	0001	SOH	DC1	!	1	A	Q	a	q
2	0010	STX	DC2	"	2	B	R	b	r
3	0011	ETX	DC3	#	3	C	S	c	s
4	0100	EOT	DC4	\$	4	D	T	d	t
5	0101	ENQ	NAK	%	5	E	U	e	u
6	0110	ACK	SYN	&	6	F	V	f	v
7	0111	BEL	ETB	'	7	G	W	g	w
8	1000	BS	CAN	(	8	H	X	h	x
9	1001	HT	EM	)	9	I	Y	i	y
A	1010	LF	SUB	*	:	J	Z	j	z
B	1011	VT	ESC	+	;	K	[	k	{
C	1100	FF	FS	,	<	L	\	l	
D	1101	CR	GS	-	=	M	]	m	}
E	1110	SO	RS	.	>	N	Ω	n	~
F	1111	SI	US	/	?	O	—	o	DEL

控制符号的定义

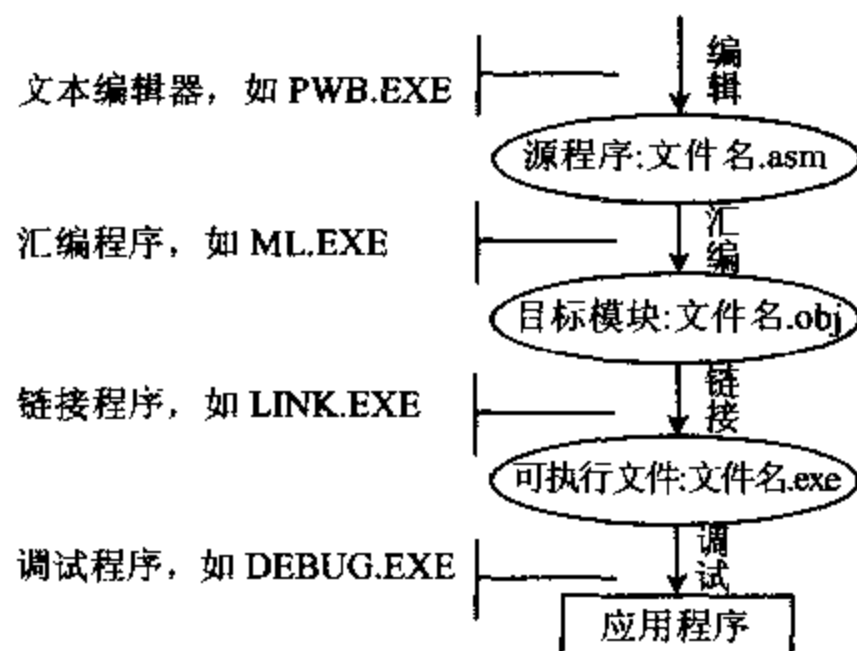
NUL	Null'	空白	DLE	Data line escape	转义
SOH	Start of heading	序始	DC1	Device control 1	机控 1
STX	Start of text	文始	DC2	Device control 2	机控 2
ETX	End of text	文终	DC3	Device control 3	机控 3
EOT	End of tape	送毕	DC4	Device control 4	机控 4
ENQ	Enquiry	询问	NAK	Negative acknowledge	未应答
ACK	Acknowledge	应答	SYN	Synchronize	同步
BEL	Bell	响铃	ETB	End of transmitted block	组终
BS	Backspace	退格	CAN	Cancel	作废
HT	Horizontal tab	横表	EM	End of medium	载终
LF	Line feed	换行	SUB	Substitute	取代
VT	Vertical tab	纵表	ESC	Escape	换码
FF	Form feed	换页	FS	File separator	文件隔离符
CR	Carriage return	回车	GS	Group separator	组隔离符
SO	Shift out	移出	RS	Record separator	记录隔离符
SI	Shift in	移入	US	Union separator	单元隔离符
SP	Space	空格	DEL	Delete	删除

附录 D 中断向量地址表

一、8088 中断向量			四、提供给用户的中断		
0-3	0	除以零	6C-6F	1B	Ctrl-Break 控制的软中断
4-7	1	单步(用于 DEBUG)	70-73	1C	定时器控制的软中断
8-B	2	非屏蔽中断	五、数据表指针		
C-F	3	断点指令(用于 DRBUG)	74-77	1D	显示器参量表
10-13	4	溢出	78-7B	1E	软盘参量表
14-17	5	打印屏幕	7C-7F	1F	图形表
18-1F	6,7	保留	六、DOS 中断		
二、8259 中断向量			80-83	20	程序结束
20-23	8	定时器	84-87	21	系统功能调用
24-27	9	键盘	88-8B	22	结束退出
28-2B	A	彩色/图形	8C-8F	23	Ctrl-Break 退出
2C-2F	B	异步通讯(secondary)	90-93	24	严重错误处理
30-33	C	异步通讯(primary)	94-97	25	绝对磁盘读功能
34-37	D	硬磁盘	98-9B	26	绝对磁盘写功能
38-3B	E	软磁盘	9C-9F	27	驻留退出
3C-3F	F	并行打印机	A0-BB	28-2E	DOS 保留
三、BIOS 中断			BC-BF	2F	打印机
40-43	10	屏幕显示	C0-FF	30-3F	DOS 保留
44-47	11	设备检验	七、BASIC 中断		
48-4B	12	测定存储器容量	100-17F	40-5F	保留
4C-4F	13	磁盘 I/O	180-19F	60-67	用户软中断
50-53	14	串行通讯口 I/O	1A0-1FF	68-7F	保留
54-57	15	盒式磁带 I/O	200-217	80-85	由 BASIC 保留
58-5B	16	键盘输入	218-3C3	86-F0	BASIC 中断
5C-5F	17	打印机输出	3C4-3FF	F1-FF	保留
60-63	18	BASIC 入口代码			
64-67	19	引导装入程序			
68-6B	1A	日时钟			

附录 E · 汇编程序的开发过程

几乎所有的汇编语言源程序的开发过程都需要编辑、汇编、链接和调试等步骤。下图给出了汇编语言程序开发过程。



附图 1 汇编语言程序开发过程

一、源程序的编辑

源程序文件要以 ASM 为扩展名。用文本编辑程序:EDIT、Turbo C2.0、Windows 记事本和 MASM 集成开发环境 PWB 等都可以。例如:MASM 程序员工作平台 PWB 中的编辑环境:

PWB T.ASM

二、源程序的汇编

汇编是将源程序翻译成由机器代码组成的浮动目标文件(.OBJ)和列表文件(.LST)的过程。汇编程序主要有以下功能:检查源程序中语法错误,给出出错信息;产生目标文件(.OBJ);展开宏指令。MASM 6.x 提供的汇编程序是 ML.EXE,一般格式如下:

ML /参数选项 文件列表 Link 连接参数选项

例如:ML /c/Cp T.ASM

汇编命令中的/c 选项表示只汇编,不自动连接;/Cp 选项表示标识符区分大小写。

如果源程序中没有语法错误,MASM 将自动生成一个目标模块文件(T.obj),否则 MASM 将给出相应的错误信息。这时应根据错误信息,重新编辑修改源程序后,再进行汇编。

汇编过程及屏幕显示如下:

C:\MASM611\BIN>ML/c T.asm ;T.asm 是源文件名

Microsoft <R> Macro Assembler Version 6.11

Copyright <C> Microsoft Corp 1981~1993. All rights reserved.

Assembling: T.asm

三、链接

汇编程序产生的二进制目标文件(.OBJ)仍然不可执行,必须经过链接,将它转换成 EXE 文件才可执行,链接程序为 LINK. EXE,它能把一个或多个目标文件和库文件合成一个可执行程序(. EXE,. COM 文件):

```
LINK T. obj
```

链接过程及屏幕显示如下:

```
Microsoft <R> Segmented Executable Linker Version 5. 31. 009 Jul 13 1992
```

```
Copyright <C> Microsoft Crop 1984~1992. All rights reserved.
```

```
Run File [T. exe]:
```

```
List File [nul. map]:
```

```
Libraries [ . lib]:
```

```
Definitions File [nul. def]:
```

如果链接多个目标文件,将多个目标文件名一次输入,中间用加号“+”连接。LINK 命令要求输入的 LIB 文件,是程序中需要用到的库文件,若无特殊需要,对[. lib]提问直接回车。

链接命令产生两个文件,一个为可执行的 EXE 文件,对此提问用户回答回车。另一个为 MAP 文件,它是连接映象文件,给出了每个段在存储器中的分配情况,一般不需要此文件,需要时打入文件名。

源程序中若没有堆栈段,链接结果给出无堆栈段的警告错误,但不影响程序执行。

如果没有发现错误,LINK 将生成一个可执行文件(T. exe);否则将提示相应的错误信息。这时需要根据错误信息重新修改源程序后再汇编、链接,直到生成可执行文件。

四、汇编和链接依次自动连续进行

ML 汇编程序可以自动调用 LINK 连接程序,实现汇编和连接的依次进行,常用格式如下(不用小写字母 c 这个参数):

```
ML T. asm
```

汇编程序 ML. EXE 可带其他参数(参数大小写敏感),该命令除产生模块文件 T. obj 和可执行文件 T. exe 外,还将生成列表文件 T. lst 和映象文件 T. map。

例如: ML /Fl /Fo /Fe /Fm /Sg T. asm

1) /Fl 文件名:创建一个汇编列表文件(. lst),列表文件是一种文本文件,含有源程序和目标代码,并给出行号及符号表,表中给出段名,段的大小及属性,用户定义的符号名、类型及属性等。

2) /Fo 文件名:生成指定的. OBJ 文件,不用缺省名。

3) /Fe 文件名:生成指定的. EXE 文件,不用缺省名。

4) /Fm 文件名:创建一个连接映象文件(. MAP),映象文件包括各个段的起始地址、结束地址、长度、名称等。

5) /Sg 在列表文件中列出由汇编程序产生的指令,例如. EXIT 产生 MOV AH, 4CH, INT 21H 语句。

自动链接过程及屏幕显示如下:

```
C:\MASM611\BIN>ML T. asm
```

```
Microsoft <R> Macro Assembler Version 6. 11
```

```
Copyright <C> Microsoft Crop 1981~1993. All rights reserved.
```

```
Assembling: T. asm
```

Microsoft <R> Segmented Executable Linker Version 5.31.009 Jul 13 1992
 Copyright <C> Microsoft Crop 1984~1992. All rights reserved.
 Object Modules [.obj]: T.obj
 Run File [T.exe]: "T.exe"
 List File [nul.map]: NUL
 Libraries [.lib]:
 Definitions File [nul.def]:

五、运行

经汇编、链接生成的可执行程序在操作系统下只要输入文件名就可以运行:

T 或者 T.exe

六、调试

汇编语言源程序在汇编及连接中能够检查出语法错误,有些逻辑错误,结构错误,只有在调试运行中,才能发现。调试工具 DEBUG 是为汇编语言设计的,它给出了一些调试命令,也可通过单步、断点、跟踪等方法有效地进行调试。

1. DEBUG 的调用

在 DOS 提示符下,可键入命令:

C: >DEBUG [D:] [Path] [Filename, ext] [Parm1] [Parm2]

D:驱动器名

PATH:路径名

FILENAME, EXT:文件名,扩展名

PARM1, PARM2:命令参数

在 DEBUG 命令后用户键入文件名,DEBUG 将指定的文件装入内存中,若用户未键入文件名,则 DEBUG 对当前存储器中的内容进行操作,或用 N 和 L 命令可将需要的文件装入存储器。

DEBUG 程序调入后,出现提示符“-”,此时系统已在 DEBUG 管理下,可以键入 DEBUG 的各种命令进行调试。命令输入大写,小写均可,命令后带有参数时,两个 16 进制参数间要有分界符。Ctrl-Break 键可中止命令,返回 DEBUG 提示符。当一个命令产生多行输出时,可用 Ctrl-Lock 键暂停上滚,按任一键继续显示下面内容。

2. DEBUG 的主要命令

(1) 显示存储单元命令 D(Dump)

格式有两种

-D 地址 从指定地址起显示 80 个字节的内容

-D 范围 显示所指定范围内的内容,指定范围为起始地址和终止地址

-D 显示从上一个 D 命令的最后一个单元后面的 80 个字节的内容,或者在 D 未使用过的情况下,显示 DS:0100 为起始地址的内容。

例 1 -D 220 240

```
1CE2:0220 C7 06 04 02 38 01 C7 06 - 06 02 00 02 C7 06 08 02    G...8.G....G...
1CE2:0230 02 02 BB 04 02 E8 02 00 - CD 20 50 51 56 57 8B 37    ...h..MPQVM 7
1CE2:0240 8B
```

其中,0220~0240 为显示的地址,中间为十六进制数表示的字节内容,右边为用 ASCII 字符表示的字节内容,“.”表示不可显示的 ASCII 码。

(2) 修改存储单元内容的命令 E(Enter)

格式有两种

—E 地址 [内容表] 用指定的内容表去修改指定地址开始的存储器中内容

例 2 —E DS:120 F3 A5 ‘XYZ’96

表示用 F3,A5,‘X’,‘Y’,‘Z’,96 共 6 个字节代替了 DS:120 至 DS:125 中的内容

—E 地址 逐个单元修改内容

例 3 —E CS:100 屏幕显示为:

18E4:0100 89. -

用户可键入‘78’,修改此内容,再键入空格键显示下一个内容,不修改时可用空格键跳过,逐个修改存储器内容,直到回车键结束。上例显示为:

18E4:0100 89.78 18.56 37. - 36.35- (.后为用户键入内容)

(3) 填入命令格式 F(Fill)

—F 范围(内容表) 将内容表中内容填入所指定的范围。

例 4 —F 18E4:0200 L10 00

将 0200H 单元开始的 16 个字节全部填入 0。若内容表中字节数超过指定范围,则忽略超过项,若内容表中字节数小于指定范围,则反复填入,直到填满所有单元为止。

(4) 检查和修改寄存器命令 R(Register)

格式有三种:

—R 显示 CPU 内所有寄存器内容和标志位状态

例 5 —R

AX=0000 BX=0000 CX=0106 DX=0000 SP=FFFE BP=0000 SI=0000 DI=0000
DS=1CE2 FS=1CE2 SS=1CE2 CS=1CE2 IP=0100 NV UP DI PL NZ NA PO NC
1CE2:0100 C70604023801 MOV WORD PTR[0204], 0138

—R 寄存器名 显示和修改某一个寄存器内容

例 6 —R AX 屏幕显示:

AX 1200 ;AX 当前内容为 1200,若不修改则按回车键,否则键入要修改的内容。
: 1000 ;将 AX 内容修改为 1000

—RF 显示和修改标志位内容(除标志位 T 以外)

例 7 —RF 屏幕显示:

OV DN EI NG ZR AC PE CY— PO NZ DI NV

若不修改则按回车键,否则键入要修改的内容,键入次序可任意。

(5) 运行命令 G(GO)

格式为:

—G=地址 1 地址 2 地址 3...

G 命令开始运行被调试的程序,运行中遇到断点时,则停止运行,并显示当前寄存器,标志位的内容和下一条将要执行的指令,按下回车键从断点处继续向下运行。

地址 1 为程序运行的起始地址,若不指定,则从当前的 CS:IP 开始运行。地址 2,地址 3 为设定的断点地址,断点最多可设 10 个。要注意的是,当执行 G 命令时,断点地址里的指令码被 INT 3(CCH)所代替,

产生中断并显示各寄存器内容后,再恢复被 CCH 取代的指令码,但若程序执行不到某个中断点,原设断点的地址中指令码被 CCH 代替后不能恢复。

用 G 命令不带参数时,程序运行到结束为止。程序执行完后屏幕显示:

Program Terminated Normally,程序要重新装入后才能再次运行。

(6) 跟踪命令 T(Trace)

格式有两种:

—T=地址 单条指令追踪,执行指定地址的一条指令,并显示 CPU 所有寄存器内容,标志位的状态,及下条指令的地址和内容。若命令中没有指定地址,则从当前 CS:IP 开始执行。

—T=地址 N,N 为多条指令追踪,从指定地址开始执行指令,共执行指定的 N 条后停止,每执行一条就显示 CPU 中所有寄存器内容,标志位的状态及下条指令的地址和内容。

(7) 汇编命令 A(Assemble)

格式为:

—A 地址

从指定地址开始,输入汇编语言的语句,DEBUG 能将其汇编成机器码,并存放在指定地址开始的存储区中。若没有指定地址,则接着上一个汇编命令的最后一个单元开始存放,若未用过 A 命令,则从 CS:0100 单元开始存放。另外输入数必须是十六进制数,如要输入十进制数,后面要加“D”说明。

(8) 反汇编命令 U(Unassemble)

有两种格式:

—U 地址 从指定地址开始,反汇编 32 个字节,若没有指定地址,则接着上一个 U 命令继续向下反汇编,若没有用过 U 命令,则从 CS:100 开始反汇编。

—U 范围 对指定范围的存储单元反汇编,可指定起始地址,结束地址,也可指定起始地址及长度。

例 8 —U 1CE2:0110 011A 或 —U 1CE2:0110 L0B

屏幕显示:

1CE2:0110 BB 0402	MOV BX,0204
1CE2:0115 E8 0200	CALL 0118
1CE2:0116 CD20	INT 20
1CE2:0118 50	PUSH AX
1CE2:0119 56	PUSH SI
1CE2:011A 8B37	MOV SI,[BX]

(9) 命名命令 N(Name)

格式为:—N [D:] [Path] [Filename. ext]

N 命令把两个文件标识符格式化在 CS:5CH 和 CS:6CH 的两个文件控制块中,使文件能用 L 命令装入或用 W 命令存盘。

例 9 装入文件 EXAMPLE 到存储器

—N EXAMPLE

—L

—

在 DEBUG 中可用 —N, —L 命令装入另一个文件到存储器中调试。

(10) 装入命令 L(Load)

有两种格式:

—L 地址 装入已在 CS:5CH 中格式化了的文件控制块所指定的文件到指定的内存地址中,若未指定地址,则装入到 CS:0100 开始的存储区中。

—L 地址 驱动器 扇区 1 扇区 2 将磁盘上指定扇区范围的内容装入到存储器指定的地址开始的区域中。

(11) 写命令 W(Write)

有两种格式：

—W 地址 把存储器中指定地址的数据写入由 CS:5CH 处的文件控制块所指定的文件中，若未指定地址，则从 CS:0100 地址开始，要写入文件的字节数预先置入 BX 和 CX 中。

—W 地址 驱动器 扇区 1 扇区 2

把存储器中指定地址开始的数据写入到磁盘的指定扇区中。

例 10 A>DEBUG

—L 0100 0 0 1 ;读 A 盘 0 扇区到内存 0100,读 1 个扇区

—R CX ;改写 512 个字节

;200

--W 0100 1 0 1 ;将内存 0100 中的内容写到 B 盘 0 扇区,写 1 个扇区

(12) 输入命令 I(Input)

—I 端口地址 I 命令从指定端口输入一个字节数据并显示

例 11 —I 34

5F ;从端口 34 输入一个字节为 5FH

(13) 输出命令 O(Output)

—O 端口地址 O 命令向指定端口输出一个指定值。

例 12 —O 34 6E ;从端口 34 输出值 6EH

(14) 退出命令 Q(Quit)

—Q 此命令退出 DEBUG 返回 DOS,但无存盘功能。

附录 F DOS 功能调用

AH 功能	调用参数	返回参数
00 程序终止 (同 INT 20H)	CS=程序段前缀	
01 键盘输入并回显		AL=输入字符
02 显示输出	DL=输出字符	
03 异步通讯输入		AL=输入数据
04 异步通讯输出	DL=输出数据	
05 打印机输出	DL=输出字符	
06 直接控制台 I/O	DL=FF(输入) DL=字符(输出)	AL=输入字符
07 键盘输入(无回显)		AL=输入字符
08 键盘输入(无回显) 检测 Ctrl-Break		AL=输入字符
09 显示字符串	DS:DX=串地址 '\$'结束字符串	
0A 键盘输入到缓冲区	DS:DX=缓冲区首地址 (DS:DX)=缓冲区最大字符数	(DS:DX+1)=实际输入的字符数
0B 检验键盘状态		AL=00 有输入 AL=FF 无输入
0C 清除输入缓冲 请求指定的输入功能	AL=输入功能号 (1,6,7,8,A)	
0D 磁盘复位		清除文件缓冲区
0E 指定当前缺省的磁盘 驱动器	DL=驱动器号 0=A,1=B,...	AL=驱动器数
0F 打开文件	DS:DX=FCB 首地址	AL=00 文件找到 AL=FF 文件未找到
10 关闭文件	DS:DX=FCB 首地址	AL=00 目录修改成功 AL=FF 目录中未找到文件
11 查找第一个目录项	DS:DX=FCB 首地址	AL=00 找到 AL=FF 未找到
12 查找下一个目录项	DS:DX=FCB 首地址 (文件名中带 * 或?)	AL=00 找到 AL=FF 未找到
13 删除文件	DS:DX=FCB 首地址	AL=00 删除成功 AL=FF 未找到

AH 功能	调用参数	返回参数
14 顺序读	DS:DX=FCB 首地址	AL=00 读成功 =01 文件结束,记录中无数据 =02 DTA 空间不够 =03 文件结束,记录不完整
15 顺序写	DS:DX=FCB 首地址	AL=00 写成功 =01 盘满 =02 DTA 空间不够
16 建文件	DS:DX=FCB 首地址	AL=00 建立成功 =FF 无磁盘空间
17 文件改名	DS:DX=FCB 首地址 (DS:DX+1)=旧文件名 (DS:DX+17)=新文件名	AL=00 成功 =FF 未成功
19 取当前缺省 磁盘驱动器		AL=缺省的驱动器号 0=A,1=B,2=C,...
1A 置 DTA 地址	DS:DX=DTA 地址	
1B 取缺省驱动器 FAT 信息		AL=每簇的扇区数 DS:BX=FTA 标识字节 CX=物理扇区的大小 DX=缺省驱动器的簇数
1C 取任一驱动器 FAT 信息	DL=驱动器号	同上
21 随机读	DS:DX=FCB 首地址	AL=00 读成功 =01 文件结束 =02 缓冲区溢出 =03 缓冲区不满
22 随机写	DS:DX=FCB 首地址	AL=00 写成功 =01 盘满 =02 缓冲区溢出
23 测定文件大小	DS:DX=FCB 首地址	AL=00 成功 文件长度填入 FCB AL=FF 未找到
24 设置随机记录号	DS:DX=FCB 首地址	
25 设置中断向量	DS:DX=中断向量 AL=中断类型号	
26 建立程序段前缀	DX=新的程序段的段前缀	
27 随机分块读	DS:DX=FCB 首地址 CX=记录数	AL=00 读成功 =01 文件结束 =02 缓冲区大小,传输结束 =03 缓冲区不满 CX=读取的记录数

AH 功能	调用参数	返回参数
28 随机分块写	DS:DX=FCB 首地址 CX=记录数	AL=00 写成功 =01 盘满 =02 缓冲区溢出
29 分析文件名	ES:DI=FCB 首地址 DS:SI=ASCII Z 串 AL=控制分析标志	AL=00 标准文件 =01 多义文件 =FF 非法盘符
2A 取日期		CX=年 DH:DL=月:日(二进制)
2B 设置日期	CX;DH;DL=年:月:日	AL=00 成功 =FF 无效
2C 取时间		CH:CL=时:分 DH:DL=秒:1/100 秒
2D 设置时间	CH:CL=时:分 DH:DL=秒:1/100 秒	AL=00 成功 =FF 无效
2E 置磁盘自动 读写标志	AL=00 关闭标志 AL=01 打开标志	
2F 取磁盘缓冲区的首址		ES:BX=缓冲区首址
30 取 DOS 版本号		AH=发行号,AL=版本号
31 结束并驻留	AL=返回码 DX=驻留区大小	
33 Ctrl-Break 检测	AL=00 取状态 AL=01 置状态(DL) DL=00 关闭检测 =01 打开检测	DL=00 关闭 Ctrl-Break 检测 =01 打开 Ctrl-Break 检测
35 取中断向量	AL=中断类型	ES:BX=中断向量
36 取空闲磁盘空间	DL=驱动器号 0=缺省,1=A,2=B,...	成功:AX=每簇扇区数 BX=有效簇数 CX=每扇区字节数 DX=总簇数 失败:AX=FFFF
38 置/取国家信息	DS:DX=信息区首地址	BX=国家码(国际电话前缀码) AX=错误码
39 建立子目录(MKDIR)	DS:DX=ASCII-Z 串地址	AX=错误码
3A 删除子目录(BMDIR)	DS:DX=ASCII-Z 串地址	AX=错误码
3B 改变当前目录 (CHDIR)	DS:DX=ASCII-Z 串地址	AX=错误码
3C 建立文件	DS:DX=ASCII-Z 串地址 CX=文件属性	成功:AX=文件代号 失败:AX=错误码

AH 功能	调用参数	返回参数
3D 打开文件	DS:DX=ASCII-Z 串地址 AL=0 读 =1 写 =2 读/写	成功:AX=文件代码 失败:AX=错误码
3E 关闭文件	BX=文件号	失败:AX=错误码
3F 读文件或设备	DS:DX=数据缓冲区地址 BX=文件代号 CX=读取的字节数	读成功: AX=实际读入的字节数 AX=0 已到文件尾 读出错:AX=错误码
40 写文件或设备	DS:DX=数据缓冲区地址 BX=文件代号 CX=写入的字节数	写成功: AX=实际写入的字节数 写出错:AX=错误码
41 删除文件	DS:DX=ASCII-Z 串地址	成功:AX=00 出错:AX=错误码(2,5)
42 移动文件指针	BX=文件代号 CX:DX=位移量 AL=移动方式(0,1,2)	成功:DX;AX=新指针位置 出错:AX=错误码
43 置/取文件属性	DS:DX=ASCII-Z 串地址 AL=0 取文件属性 AL=1 置文件属性 CX=文件属性	成功:CX=文件属性 失败:AX=错误码
44 设备文件 I/O 控制	BX=文件代号 AL=0 取状态 =1 置状态 DX =2 读数据 =3 写数据 =6 取输入状态 =7 取输出状态	DX=设备信息
45 复制文件代号	BX=文件代号 1	成功:AX=文件代号 2 失败:AX=错误码
46 人工复制文件代号	BX=文件代号 1 CX=文件代号 2	失败:AX=错误码
47 取当前目录路径名	DL=驱动器号 DS:SI=ASCII-Z 串地址	DS:SI=ASCII-Z 串 失败:AX=错误码
48 分配内存空间	BX=申请内存容量	成功:AX=分配内存首址 失败:BX=最大可用空间
49 释放内存空间	ES=内存起始段地址	失败:AX=错误码
4A 调整已分配的存储块	ES=原内存起始地址 BX=再申请的容量	失败:BX=最大可用空间 AX=错误码

续表

AH 功能	调用参数	返回参数
4B 装配/执行程序	DS:DX=ASCII-Z 串地址 ES:BX=参数区首地址 AL=0 装入执行 AL=3 装入不执行	失败:AX=错误码
4C 带返回码结束	AL=返回码	
4D 取返回代码		AX=返回代码
4E 查找第一个匹配文件	DS:DX=ASCII-Z 串地址 CX=属性	AX=出错代码(02,18)
4F 查找下一个匹配文件	DS:DX=ASCII-Z 串地址 (文件名中带? 或*)	AX=出错代码(18)
54 取盘自动读写标志		AL=当前标志值
56 文件改名	DS:DX=ASCII-Z 串(旧) ES:DI=ASCII-Z 串(新)	AX=出错码(03,15,17)
57 置/取文件日期和时间	BX=文件代号 AL=0 读取 AL=1 设置(DX,CX)	DX,CX=日期和时间 失败:AX=错误码
58 取/置分配策略码	AL=0 取码 =1 置码(BX) BX=策略码	成功:AX=策略码 失败:AX=错误码
59 取扩充错误码		AX=扩充错误码 BH=错误类型 BL=建议的操作 CH=错误场所
5A 建立临时文件	CX=文件属性 DS:DX=ASCII-Z 串地址	成功:AX=文件代号 失败:AX=错误码
5B 建立新文件	CX=文件属性 DS:DX=ASCII-Z 串地址	成功:AX=文件代号 失败:AX=错误码
5C 控制文件存取	AL=00 封锁 =01 开启 BX=文件代号 CX:DX=文件位移 SI:DI=文件长度	失败:AX=错误码
62 取程序段前缀地址		BX=PSP 地址

参 考 文 献

- [1] Intel. The 8086 Family User's Manual, Oct. 1979
- [2] Intel. Peripheral Design Handbook, Aug. 1981
- [3] National Semiconductor. Data Conversion/Acquisition Data Book, 1980
- [4] Analog Device. Data Converter Reference Manual Volume, 1992
- [5] Sybex Inc. The Complete PC Upgrade & Maintenance Guide, 1991
- [6] Intel 845G/845GL Chipset Datasheet, May 2002
- [7] IA-32 Intel Architecture Software Developer's Manual, Volume 1: Basic Architecture, 2002
- [8] Barry B. Brey. The Intel Microprocessors 8086/8088, 80186/80188, 80286, 80386, 80486, Pentium, Pentium Pro and Pentium II Processors Architecture, Programming, and Interfacing Fifth Edition, 2000
- [9] 艾德才。Pentium 系列微型计算机原理与接口技术, 高等教育出版社, 2001
- [10] Douglas V. Hall 著, 赵振西等译。微处理器和接口技术, 中国科学技术大学出版社, 1994
- [11] 戴梅萼。微型计算机技术及应用, 第 3 版, 清华大学出版社, 2003
- [12] 沈美明、温冬婵。IBM PC 汇编语言程序设计, 第 2 版, 清华大学出版社, 2002
- [13] 孙德文。微型计算机技术, 高等教育出版社, 2002
- [14] 周明德。保护方式下的 80386 及其编程, 清华大学出版社, 1993
- [15] 吕晓庆。80386/80486 系统编程实践, 浙江大学出版社, 1993

[G e n e r a l I n f o r m a t i o n]

书名 = 中国科学院指定考研参考书 微型计算机原理与接口技术 (第三版)

作者 = 周荷琴

页数 = 5 2 1

S S 号 = 1 1 3 7 2 0 6 3

出版日期 = 2 0 0 4 年 1 2 月第 3 版

前言
目录

目录第一章

绪论

1 - 1 微型计算机的发展概况

1 - 2 微型计算机系统

一、微型计算机

二、存储器

三、I / O接口

四、总线

五、微型计算机的性能指标

1 - 3 计算机数据格式

一、数制

二、计算机数据格式

习题

第二章 8086系统结构

2 - 1 8086 CPU结构

一、8086 CPU的内部结构

二、寄存器结构

2 - 2 8086 CPU的引脚及其功能

一、8086 / 8088 CPU在最小模式中引脚定义

二、8086 / 8088 CPU在最大模式中引脚定义

三、8088与8086 CPU的不同之处

2 - 3 8086存储器组织

一、存储器地址的分段

二、8086存储器的分体结构

三、堆栈的概念

2 - 4 8086系统配置

一、最小模式系统

二、最大模式系统

2 - 5 8086 CPU时序

一、系统的复位和启动

二、最小模式下的总线操作

三、最大模式下的总线操作

四、最小模式下的总线保持

习题

第三章 8086的寻址方式和指令系统

3 - 1 8086的寻址方式

一、立即寻址方式

二、寄存器寻址方式

三、直接寻址方式

四、寄存器间接寻址方式

五、寄存器相对寻址方式

六、基址变址寻址方式

七、相对基址变址寻址方式

八、其它

3 - 2 指令的机器码表示方法

一、机器语言指令的编码目的和特点

二、机器语言指令代码的编制

3 - 3 8086的指令系统

一、数据传送指令

二、算术运算指令

三、逻辑运算和移位指令

四、字符串处理指令

五、控制转移指令

六、处理器控制指令

七、指令的执行时间和软件延时

习题

第四章 汇编语言程序设计

4 - 1 汇编语言程序格式

- 一、指令性语句
- 二、伪指令语句
- 三、数据项

4 - 2 M A S M中的表达式

- 一、算术运算符
- 二、逻辑运算符
- 三、关系运算符
- 四、数值返回运算符
- 五、修改属性运算符
- 六、其它运算符
- 七、优先级

4 - 3 伪指令语句

- 一、数据定义语句
- 二、表达式赋值语句
- 三、段定义语句
- 四、过程定义语句
- 五、程序开始和结束语句
- 六、结构定义语句
- 七、外部伪指令及对准伪指令
- 八、高档微机增加的伪指令

4 - 4 D O S系统功能调用和B I O S中断调用

- 一、常用的软件中断
- 二、D O S系统功能调用
- 三、B I O S中断调用

4 - 5 程序设计方法

- 一、顺序结构
- 二、分支结构
- 三、循环程序结构
- 四、子程序结构
- 五、综合举例

4 - 6 宏汇编和条件汇编

- 一、宏汇编
- 二、条件汇编

习题

第五章 存储器

5 - 1 存储器分类

- 一、按用途分类
- 二、按存储器性质分类

5 - 2 随机存取存储器R A M

- 一、静态随机存取存储器 (S R A M)
- 二、动态随机存取存储器 (D R A M)
- 三、存储器的工作时序
- 四、高速缓冲存储器

5 - 3 只读存储器

- 一、掩膜型R O M
- 二、可编程R O M (P R O M)
- 三、可编程可擦除R O M (E P R O M)
- 四、电可擦除可编程R O M (E E P R O M)

5 - 4 C P U与存储器的连接

- 一、存储器的地址选择
- 二、存储器的数据线及控制线的连接

5 - 5 存储器空间的分配和使用

- 一、I B M P C / X T机中存储器空间分配
- 二、I B M P C / A T机中存储器空间分配
- 三、P C机中存储器的使用

	习题
第六章	I / O接口和总线
6 - 1	I / O接口
一、	I / O接口的功能
二、	简单的输入输出接口芯片
三、	I / O端口及其寻址方式
四、	C P U与外设间的数据传送方式
五、	P C机的 I / O地址分配
6 - 2	总线
一、	总线的概念
二、	I B M P C总线
三、	A T总线或 I S A总线
	习题
第七章	微型计算机中断系统
7 - 1	概述
一、	中断概念
二、	中断分类
7 - 2	中断处理过程
一、	C P U响应中断过程
二、	中断向量表
三、	中断服务子程序
四、	中断响应时序
7 - 3	中断优先级和中断嵌套
一、	中断优先级
二、	中断嵌套
7 - 4	可编程中断控制器 8 2 5 9 A
一、	功能和引脚
二、	内部结构
三、	8 2 5 9 A的中断管理方式
四、	8 2 5 9 A的编程方法
五、	8 2 5 9 A的中断级联
	习题
第八章	可编程计数器／定时器 8 2 5 3 及其应用
8 - 1	8 2 5 3 的工作原理
一、	8 2 5 3 的内部结构和引脚信号
二、	初始化编程步骤和门控信号的功能
三、	8 2 5 3 的工作方式
8 - 2	8 2 5 3 的应用举例
一、	8 2 5 3 定时功能的应用例子
二、	8 2 5 3 计数功能的应用例子
三、	8 2 5 3 在 P C / X T 机中的应用
	习题
第九章	可编程外围接口芯片 8 2 5 5 A 及其应用
9 - 1	8 2 5 5 A 的工作原理
一、	8 2 5 5 A 的结构和功能
二、	8 2 5 5 A 的控制字
三、	8 2 5 5 A 的工作方式和 C 口状态字
9 - 2	8 2 5 5 A 的应用举例
一、	基本输入输出应用举例
二、	键盘接口
三、	8 2 5 5 A 在 P C / X T 机中的应用
四、	P C / X T 机中的扬声器接口电路
五、	并行打印机接口
	习题
第十章	串行通信和可编程接口芯片 8 2 5 1 A
1 0 - 1	串行通信的基本概念
一、	数据传送的方向

二、串行传送的两种基本工作方式

三、串行传送速率

四、串行接口芯片U A R T和U S A R T

五、调制解调器

1 0 - 2 可编程串行通信接口芯片8 2 5 1 A

一、8 2 5 1 A的内部结构和外部引脚

二、8 2 5 1 A的编程

三、8 2 5 1 A初始化编程举例

1 0 - 3 E I A R S - 2 3 2 C串行口和8 2 5 1 A应用举例

一、E I A R S - 2 3 2 C串行口

二、8 2 5 1 A应用举例

1 0 - 4 串行同步数据通信协议

一、二进制同步通信协议B I S Y N C

二、高级数据链路控制协议H D L C

习题

第十一章 模数（A / D）和数模（D / A）转换

1 1 - 1 概述

一、一个实时控制系统

二、多路模拟开关

三、采样、量化和编码

四、采样保持器

1 1 - 2 D / A转换器

一、数／模转换器原理

二、数／模转换器的主要性能指标

三、几种数／模转换器

1 1 - 3 A / D转换

一、模／数转换器原理

二、典型的模／数转换器

习题

第十二章 8 2 3 7 A D M A控制器及P C / X T机的系统板

1 2 - 1 8 2 3 7 A的组成和工作原理

一、8 2 3 7 A的内部结构

二、8 2 3 7 A的引脚功能

三、8 2 3 7 A的内部寄存器

1 2 - 2 8 2 3 7 A的时序

一、外设和内存间的D M A数据传送时序

二、空闲周期、有效周期和扩展写周期

1 2 - 3 8 2 3 7 A的编程和应用举例

一、P C / X T机中的D M A控制逻辑

二、8 2 3 7 A的一般编程方法

三、P C / X T机上的D M A控制器的使用

1 2 - 4 P C / X T机的系统板

一、C P U子系统

二、接口部件子系统

三、存储器子系统

习题

第十三章 3 2位微机基本工作原理概述

1 3 - 1 3 2位微处理器的结构与工作模式

一、3 2位微处理器结构简介

二、3 2位微处理器的工作模式

1 3 - 2 寄存器

一、用户级寄存器

二、系统级寄存器

三、程序调试寄存器

1 3 - 3 保护模式下的内存管理

一、段内存管理技术

二、分页内存管理技术

	1 3 - 4	3 2 位机新增指令和程序设计简介
	一、	8 0 3 8 6 的寻址方式
	二、	8 0 3 8 6 的指令系统
	三、	程序设计实例
	1 3 - 5	3 2 位机接口技术
	一、	主板的组成
	二、	P e n t i u m II 主板
	三、	集成型主板
附录 A		8 0 8 6 / 8 0 8 8 指令系统一览表
附录 B		通用汇编程序伪指令
附录 C		A S C I I 码编码表
附录 D		中断向量地址表
附录 E		汇编程序的开发过程
	一、	源程序的编辑
	二、	源程序的汇编
	三、	链接
	四、	汇编和链接依次自动连续进行
	五、	运行
	六、	调试
附录 F		D O S 功能调用
参考文献		